
SECUENCIAS · ALGORITMOS · SISTEMAS

Bioinformática **Práctica**

*De la secuencia al sistema: fundamentos, algoritmos
y laboratorio computacional*



Juvenal Yosa, PhD

con la ayuda de Claude (Anthropic) como copiloto

Bioinformática Práctica

De la secuencia al sistema: fundamentos, algoritmos y laboratorio computacional

Autor: Juvenal Yosa, PhD · juvenal.yosa@gmail.com

Copiloto: Claude (Anthropic), asistente de inteligencia artificial empleado en la revisión y corrección de estilo, la programación de figuras y la verificación bibliográfica, sobre la escritura inicial del autor y bajo su dirección y revisión.

Primera edición: 2026.

Licencia: © 2026 Juvenal Yosa. El texto, las figuras y el libro en su conjunto se distribuyen bajo la licencia Creative Commons Atribución-NoComercial-SinDerivadas 4.0 Internacional (CC BY-NC-ND 4.0, creativecommons.org/licenses/by-nc-nd/4.0). Se permite copiar y redistribuir el libro completo y sin cambios, citando al autor, sin fines comerciales. No se permite distribuir versiones modificadas ni hacer uso comercial sin autorización escrita del autor. Los fragmentos de código del libro y de sus laboratorios se distribuyen bajo la licencia MIT.



Prólogo

En 1977, secuenciar unos pocos miles de bases del bacteriófago ϕX_{174} fue una hazaña que llevó años. Hoy, un solo instrumento produce en un día más secuencias de las que existían en todas las bases de datos del mundo a comienzos de este siglo. La biología se ha convertido, casi sin pedir permiso, en una ciencia de la información. Quien quiera hacer preguntas sobre genes, células, poblaciones o ecosistemas necesita hoy dos alfabetos: el de las moléculas y el de los algoritmos.

Estas páginas son, en buena medida, las memorias de cinco años enseñando bioinformática. En ese tiempo aprendí que las preguntas de los estudiantes se repiten con una regularidad casi estadística: por qué un alineamiento óptimo no es necesariamente el verdadero, qué significa en realidad un valor E , por qué un árbol filogenético puede estar bien calculado y aun así contar una historia equivocada. Aprendí también qué explicaciones encendían la luz y cuáles solo producían apuntes. Aquí he querido reunir las primeras y dejar atrás las segundas: los ejemplos que funcionaron en clase, los errores que vi repetirse una y otra vez, y el orden en que las ideas resultan más naturales de aprender.

El libro nació así como acompañante de un curso de bioinformática práctica. Cada capítulo corresponde a un módulo del curso, y cada sección a una lección con un laboratorio ejecutable en Google Colab. Pero el libro está pensado para leerse por sí mismo: allí donde el notebook muestra *cómo* hacer algo, el texto se detiene a explicar *por qué* funciona, de dónde viene cada ecuación y qué supuestos la sostienen.

El recorrido tiene una lógica deliberada. Cada tema comienza con una situación cotidiana que ofrece un punto de apoyo intuitivo; sigue con las definiciones básicas y, paso a paso, sube hasta la teoría formal que se encuentra en la literatura especializada. Las ecuaciones no aparecen como adornos: cada una va acompañada de una tabla que explica todos sus símbolos, de un ejemplo numérico resuelto y de un fragmento de código que la pone a trabajar. Las figuras se generaron a partir de cálculos reales

y reproducibles, y las referencias, que remiten siempre a las fuentes originales, fueron verificadas una a una.

El libro está organizado en cinco partes. La primera prepara el laboratorio digital y el vocabulario molecular. La segunda estudia la comparación de secuencias, desde el alineamiento hasta la filogenética. La tercera sigue a las lecturas de secuenciación desde el instrumento hasta las variantes. La cuarta aborda la genómica de poblaciones, la transcriptómica, la epigenómica y la metagenómica. La quinta sube de escala hacia la estructura de las proteínas, las redes, el aprendizaje automático y los flujos de trabajo reproducibles.

Nada de esto sustituye la práctica. La bioinformática se aprende como se aprende un instrumento: leyendo la partitura, sí, pero sobre todo tocando. Por eso le invito a leer con el notebook abierto, a cambiar los parámetros, a romper el código y a repararlo. Si al terminar un capítulo usted puede explicar a un colega por qué un algoritmo funciona, y no solo cómo se ejecuta, el libro habrá cumplido su propósito.

Una nota sobre cómo se hizo. Escribí este libro con la ayuda de Claude, un asistente de inteligencia artificial de Anthropic, que actuó como copiloto: sobre mi escritura inicial, me ayudó en la revisión y corrección de estilo, a programar las figuras y a verificar cada referencia contra sus registros originales. Las decisiones sobre qué enseñar, en qué orden y con qué profundidad, así como la revisión final de cada página, son mías, igual que los errores que el lector pueda encontrar.

Juvenal Yosa, PhD

2026

Cómo leer este libro

Cada capítulo abre con una lista de objetivos y cierra con un resumen, una serie de ejercicios graduados y las referencias citadas. Entre medias, el texto se apoya en un pequeño repertorio de recuadros con funciones precisas. Conocerlos de antemano le ayudará a decidir qué leer con calma y qué puede saltarse en una primera lectura.

Recuadros de intuición

Abren cada tema con una situación cotidiana que sirve de punto de apoyo. No sustituyen la teoría: preparan el terreno para ella.

Definición. (Definiciones)

Enuncian con precisión los objetos matemáticos o biológicos con los que trabajaremos. Están numeradas por capítulo.

Teorema. (Resultados)

Recogen las ecuaciones y los resultados centrales de cada método, normalmente seguidos de una tabla de símbolos.

DONDE

x

Cada ecuación importante va acompañada de una tabla como esta, que explica el significado de todos sus símbolos.

Ejemplo. (Ejemplos resueltos)

Aplican la teoría a números concretos, paso a paso, para que usted pueda reproducirlos con lápiz o con Python.

ejemplo.py

```
# Los fragmentos de código son breves, completos y ejecutables.
gc = sum(base in "GC" for base in "ATGCGC") / 6
```

📖 Notas históricas

Cuentan quién, cuándo y por qué se desarrolló una idea. La ciencia es una actividad humana.

⚠ Advertencias

Señalan errores frecuentes, supuestos que se olvidan y trampas de interpretación.

🔬 Para profundizar

Material avanzado que puede omitirse en una primera lectura sin perder el hilo.

 **Colab** • enlace al laboratorio de cada lección

 repositorio del curso

Convenciones. Las explicaciones están en español; el código, los nombres de funciones y muchos términos técnicos se mantienen en inglés, como aparecen en la literatura y en la documentación de las herramientas; su primera aparición se marca en *cursiva*. Los decimales se escriben con coma. Las secuencias de ADN se colorean con el código de la tabla 0.1, el mismo que se usa en los notebooks del curso.

Tabla 0.1. Colores de nucleótidos usados en todo el libro.

A	adenina	C	citosa	G	guanina	T	timina	(U	uracilo)
---	---------	---	--------	---	---------	---	--------	----	----------

Ejercicios. Están marcados según su dificultad: ●●● básica, ●●● intermedia y ●●● avanzada. Los de nivel avanzado suelen requerir programación o una demostración.

Requisitos. Se asume una familiaridad básica con cálculo, probabilidad y programación en Python. El capítulo 0 repasa las herramientas computacionales y el capítulo 1, la biología molecular necesaria.

Índice general

Prólogo	iii
----------------	------------

Cómo leer este libro	v
-----------------------------	----------

I Fundamentos

0 Preparación del laboratorio digital	3
--	----------

0.1 Colab como laboratorio: Python, numpy, pandas y la visualización científica	4
0.2 Biopython y herramientas de línea de comandos	17
0.3 Reproducibilidad: semillas, versiones, hashes y Git	27
Ejercicios	39
Referencias del capítulo	40

1 Biología molecular para bioinformáticos	43
--	-----------

1.1 El dogma central como sistema de información	44
1.2 Genes, genomas y marcos abiertos de lectura	58
1.3 Composición de secuencias: contenido GC, GC skew y el origen de replicación	68
Ejercicios	79
Referencias del capítulo	80

2 Secuencias, formatos y bases de datos	83
--	-----------

2.1 El idioma de los archivos	84
2.2 Bases de datos biológicas desde Python	107
Ejercicios	120
Referencias del capítulo	121

II Comparar secuencias

3	Alineamiento de secuencias	127
3.1	Dot plots: mirar antes de calcular	128
3.2	Programación dinámica: Needleman-Wunsch y Smith-Waterman	137
3.3	Matrices de sustitución y penalización de gaps	153
3.4	BLAST y la estadística de Karlin-Altschul	164
	Ejercicios	176
	Referencias del capítulo	177
4	Alineamiento múltiple, motivos y perfiles	183
4.1	Alineamiento múltiple: el método progresivo	184
4.2	Motivos, matrices de peso posicional y logos	195
4.3	Modelos ocultos de Márkov y perfiles (HMMER)	204
	Ejercicios	219
	Referencias del capítulo	220
5	Filogenética y evolución molecular	223
5.1	Distancias evolutivas y modelos de sustitución	224
5.2	UPGMA y Neighbor-Joining	239
5.3	Máxima verosimilitud y bootstrap	247
	Ejercicios	260
	Referencias del capítulo	261
 III Del secuenciador al genoma		
6	Secuenciación de nueva generación (NGS)	265
6.1	Tecnologías de secuenciación	266
6.2	Calidad Phred, control de calidad y <i>trimming</i>	276
6.3	Cobertura y la teoría de Lander-Waterman	289
	Ejercicios	299
	Referencias del capítulo	301
7	Mapeo de lecturas	305
7.1	Transformada de Burrows-Wheeler y FM-index	306
7.2	BWA, minimap2 y samtools	320
7.3	Visualización de alineamientos y cobertura	333
	Ejercicios	341
	Referencias del capítulo	343
8	Ensamblaje y anotación de genomas	347
8.1	k -mers y espectros de k -mers	348

8.2	Grafos de De Bruijn	359
8.3	Ensamblaje con SPAdes y evaluación con QUAST	368
8.4	Anotación genómica	378
	Ejercicios	389
	Referencias del capítulo	390
9	Detección de variantes	393
9.1	Verosimilitud de genotipos	394
9.2	Pipeline con bcftools	409
9.3	Anotación funcional de variantes	419
	Ejercicios	429
	Referencias del capítulo	431
IV	Genomas en acción	
10	Genómica de poblaciones y GWAS	437
10.1	Hardy-Weinberg y deriva génica	438
10.2	Desequilibrio de ligamiento y PCA	454
10.3	GWAS: Manhattan, QQ plots y corrección múltiple	463
	Ejercicios	475
	Referencias del capítulo	476
11	Transcriptómica (RNA-seq)	479
11.1	Cuantificación de transcritos	480
11.2	Normalización y binomial negativa	493
11.3	Expresión diferencial y FDR	502
11.4	Enriquecimiento funcional	512
	Ejercicios	520
	Referencias del capítulo	521
12	Transcriptómica de célula única	525
12.1	Control de calidad y normalización	526
12.2	PCA, t-SNE y UMAP	540
12.3	Clustering y genes marcadores	549
12.4	Trayectorias y pseudotiempo	559
	Ejercicios	569
	Referencias del capítulo	570
13	Epigenómica y regulación	575
13.1	ChIP-seq y ATAC-seq	576
13.2	Metilación del ADN	591

13.3	Descubrimiento de motivos regulatorios	601
	Ejercicios	611
	Referencias del capítulo	612
14	Metagenómica y microbioma	615
14.1	El gen 16S rRNA y las variantes de secuencia de amplicón	616
14.2	Diversidad alfa y beta	628
14.3	Metagenómica shotgun	642
	Ejercicios	655
	Referencias del capítulo	656
V	Estructura, sistemas e inteligencia	
15	Bioinformática estructural	661
15.1	Estructura de proteínas y el Protein Data Bank	662
15.2	Predicción de estructura: de la homología a AlphaFold y ESMFold	680
15.3	Acoplamiento molecular (docking)	693
	Ejercicios	703
	Referencias del capítulo	705
16	Biología de sistemas y redes	709
16.1	Redes de interacción proteína-proteína	710
16.2	Redes de regulación génica y modelos dinámicos	725
	Ejercicios	744
	Referencias del capítulo	745
17	Machine learning e inteligencia artificial en bioinformática	749
17.1	Clasificación de secuencias con ML clásico	750
17.2	Deep learning para ADN	766
17.3	Modelos de lenguaje de proteínas (ESM)	779
	Ejercicios	792
	Referencias del capítulo	794
18	Flujos reproducibles y proyecto final	797
18.1	Snakemake y Nextflow	798
18.2	Proyecto integrador	828
	Ejercicios	841
	Referencias del capítulo	842

PARTE

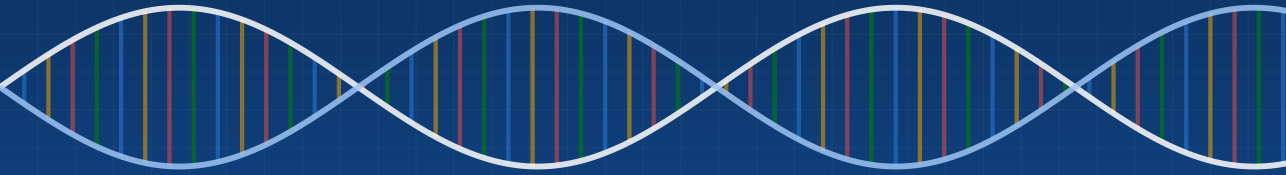


Fundamentos

El laboratorio, la molécula y el archivo

Antes de comparar genomas hay que saber qué es un genoma, cómo se guarda en un archivo y cómo se interroga con un computador. Esta parte prepara el laboratorio digital y el vocabulario molecular que usaremos durante todo el libro.

Preparación del laboratorio digital



Un artículo sobre ciencia computacional en una publicación científica no es la obra académica en sí misma; es solo la publicidad de esa obra. La verdadera obra es el entorno completo de desarrollo de software y el conjunto completo de instrucciones que generaron las figuras.

— Buckheit y Donoho (1995), resumiendo una idea de Jon Claerbout

● Al terminar este capítulo usted podrá

- Representar secuencias biológicas como arreglos en memoria, estimar el costo de un cálculo con notación asintótica y explicar por qué la vectorización con numpy es órdenes de magnitud más rápida que un bucle.
- Reconocer los límites de la aritmética de punto flotante y usar el espacio logarítmico para trabajar con probabilidades diminutas.
- Organizar datos en tablas ordenadas (*tidy*) con pandas y construir figuras que respeten los principios de la percepción visual.
- Leer, traducir y verificar un genoma real con Biopython y combinar Python con herramientas de línea de comandos como BLAST+.

- Explicar cómo funcionan un generador de números pseudoaleatorios, una función *hash* y un repositorio de Git, y usar los tres para que un análisis sea reproducible.

Antes de iniciar un experimento, un biólogo molecular aprende a calibrar una micropipeta, preparar un tampón y registrar el protocolo. El bioinformático también necesita conocer sus instrumentos: la computadora, los archivos de secuencias, las bibliotecas de programación y los programas de línea de comandos.

Este capítulo prepara ese laboratorio. No es un tutorial de programación: queremos entender *qué ocurre dentro de la máquina* cuando analizamos datos biológicos, porque de ello dependen la velocidad, la exactitud y la credibilidad de todo lo que haremos en el resto del libro. Avanzaremos en tres pasos. Primero convertiremos el cuaderno de Colab en un instrumento de medida: veremos cómo se representan las secuencias en memoria, cuánto cuesta un cálculo, qué precisión tienen los números de la máquina y cómo se organiza y se muestra un conjunto de datos (sección 0.1). Después incorporaremos las herramientas de la disciplina: Biopython, las bases de datos del NCBI y la línea de comandos, con un genoma real, el de SARS-CoV-2, como material de trabajo (sección 0.2). Por último, estudiaremos la maquinaria de la reproducibilidad: los números aleatorios que no lo son, las huellas digitales de los archivos, el control de versiones y los principios que la comunidad ha acordado para que un análisis pueda verificarse (sección 0.3).

0.1 Colab como laboratorio: Python, numpy, pandas y la visualización científica

 Colab · 0.1 · Colab como laboratorio

 [abrir notebook](#)

Un cuaderno que se ejecuta solo

En el laboratorio húmedo, el cuaderno contiene tres cosas mezcladas: el protocolo (lo que pensábamos hacer), las notas (lo que hicimos de verdad) y los resultados pegados con cinta adhesiva (la foto del gel, la curva de crecimiento). Si seis meses después alguien pregunta cómo se obtuvo una banda, la respuesta está en el cuaderno, junto a la banda. Ahora imagine un cuaderno en el que el protocolo es el experimento: cada vez que alguien lo lee de principio a fin, el

experimento vuelve a ocurrir y la foto del gel se regenera ante sus ojos. Eso es un *notebook* computacional.

0.1.1 La computación como instrumento de laboratorio

Un *notebook* de Jupyter, y su variante alojada en la nube, Google Colab, es un documento formado por *celdas*. Las celdas de texto contienen la narración, con ecuaciones y enlaces; las celdas de código contienen instrucciones que se envían a un proceso independiente, el *núcleo* (*kernel*), que las ejecuta y devuelve el resultado para que se muestre debajo de la celda. Kluyver et al. (2016) describen este diseño como un formato de publicación para flujos de trabajo computacionales reproducibles: el mismo archivo documenta, ejecuta y muestra.

La separación entre documento y núcleo tiene una consecuencia que conviene comprender desde el principio. El núcleo guarda en memoria un *estado*: todas las variables, funciones y módulos que se han definido hasta el momento. Ese estado depende del *orden en que se ejecutaron* las celdas, no del orden en que aparecen en la página. Si ejecuta la celda 5, luego corrige la celda 2 y la vuelve a ejecutar, el núcleo contiene una mezcla que no corresponde a ninguna lectura lineal del documento (figura 0.1). Los números entre corchetes a la izquierda de cada celda, [1], [2], ..., registran precisamente ese orden de ejecución.

Colab añade una segunda capa: el núcleo corre en una máquina virtual prestada que se destruye tras un rato de inactividad, llevándose consigo todo lo no guardado y las bibliotecas instaladas. Esa fragilidad obliga a escribir cuadernos que reconstruyan su entorno desde cero, que es justo lo que exige la reproducibilidad (sección 0.3).

El estado oculto

Antes de compartir un cuaderno o de confiar en sus resultados, use *Entorno de ejecución* → *Reiniciar y ejecutar todo*. Si el cuaderno falla o produce otros números, el resultado anterior dependía de un estado que no está escrito en ninguna parte.

0.1.2 Una secuencia es un arreglo de símbolos

La información genética de una molécula de ADN depende solo del orden de sus bases. Para la computadora, por lo tanto, una secuencia es una cadena finita de símbolos.

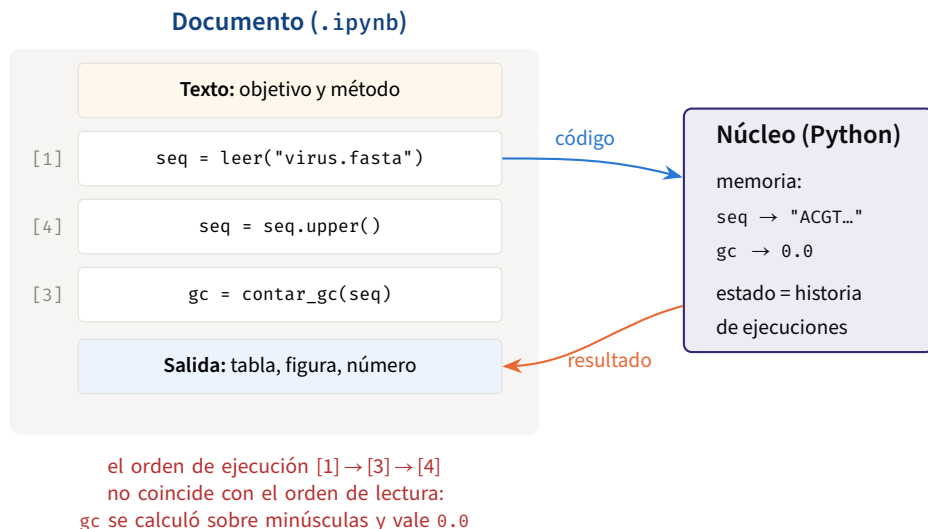


Figura 0.1. Anatomía de un *notebook*. El documento (izquierda) alterna celdas de texto, de código y de salida; el núcleo (derecha) ejecuta el código y conserva en memoria todas las variables. Los contadores [1], [3], [4] revelan un *estado oculto*: la celda que calcula `gc` se ejecutó antes que la que convierte la secuencia a mayúsculas, así que contó cero bases G o C en un archivo escrito en minúsculas. La memoria contiene ya la secuencia corregida, pero `gc` conserva el valor erróneo, distinto del que obtendría un lector que ejecutara el cuaderno de arriba abajo.

Definición 0.1. (Alfabeto, secuencia y codificación)

Un *alfabeto* Σ es un conjunto finito de símbolos; para el ADN, $\Sigma = \{A, C, G, T\}$ y para las proteínas, los 20 aminoácidos. Una *secuencia* de longitud n es una función $x : \{0, 1, \dots, n-1\} \rightarrow \Sigma$, que escribimos $x = x_0x_1 \dots x_{n-1}$. Una *codificación* es una función inyectiva $c : \Sigma \rightarrow \{0, 1\}^b$ que asigna a cada símbolo una palabra de b bits.

Hemos numerado las posiciones desde cero, como Python, y no desde uno, como los biólogos. Esta diferencia es fuente de incontables errores, y conviene fijar la regla de conversión. En Python, el corte `x[i:j]` contiene las posiciones $i, i+1, \dots, j-1$: es un intervalo *semiabierto* $[i, j)$ cuya longitud es simplemente $j-i$. Un gen anotado en las posiciones biológicas $a..b$ (ambas incluidas, numeradas desde 1) se extrae así:

$$\text{posiciones biológicas } [a, b] \longleftrightarrow x[a-1:b], \quad \text{longitud} = b - a + 1. \quad (0.1)$$

DONDE

a, b	Primera y última posición del segmento en la numeración biológica (base 1, extremos incluidos).
$x[a-1:b]$	Corte de Python equivalente: empieza en el índice $a-1$ y se detiene <i>antes</i> del índice b .

¿Cuánta memoria ocupa una secuencia? Si cada símbolo se codifica con b bits, una secuencia de n símbolos ocupa

$$M(n) = \frac{nb}{8} \text{ bytes}, \quad b \geq \lceil \log_2 |\Sigma| \rceil. \quad (0.2)$$

DONDE

$M(n)$	Memoria necesaria, en bytes (1 byte = 8 bits).
b	Bits por símbolo de la codificación elegida.
$ \Sigma $	Tamaño del alfabeto; $\lceil \log_2 \Sigma \rceil$ es el mínimo número de bits que distingue todos sus símbolos.

Con cuatro bases bastan $\log_2 4 = 2$ bits, pero el texto ASCII usa 8. Para un genoma humano de $3,1 \times 10^9$ bases, la ecuación (0.2) da 775 MB empaquetado a 2 bits (formato `.2bit`), 3,1 GB como texto (`str` o `np.uint8`), 24,8 GB como lista de Python (un puntero de 8 bytes por base) y 99,2 GB como codificación *one-hot* en `float64`, habitual en aprendizaje automático: la misma información, con un factor de 128 de diferencia.

0.1.3 Cuánto cuesta un cálculo: complejidad asintótica

Antes de ejecutar un análisis sobre un genoma completo conviene saber si tardará un segundo o un año. La herramienta para responder sin medir es la notación asintótica, que describe cómo crece el costo de un algoritmo cuando crece el tamaño n de la entrada, sin preocuparse por las constantes que dependen de la máquina.

Definición 0.2. (Notación O grande)

Sean f y g funciones positivas de n . Decimos que $f(n) = O(g(n))$ si existen constantes $c > 0$ y n_0 tales que

$$f(n) \leq c g(n) \quad \text{para todo } n \geq n_0. \quad (0.3)$$

Si además $f(n) \geq c' g(n)$ para alguna $c' > 0$, escribimos $f(n) = \Theta(g(n))$: f y g crecen al mismo ritmo.

DONDE

$f(n)$	Costo real del algoritmo (operaciones o segundos) para una entrada de tamaño n .
$g(n)$	Función de referencia sencilla: n , $n \log n$, n^2 , $2^n \dots$
c, c', n_0	Constantes que no dependen de n ; absorben la velocidad de la máquina y los detalles de implementación.

La intuición es sencilla: $O(g)$ ignora las constantes y se queda con la *forma* del crecimiento. Contar el contenido GC recorre la secuencia una vez: es $\Theta(n)$. Comparar todas las posiciones de una secuencia con todas las de otra, como en el alineamiento del capítulo 3, es $\Theta(n^2)$. Ordenar n lecturas es $\Theta(n \log n)$. A 10^9 operaciones por segundo, un algoritmo $\Theta(n^2)$ tarda menos de un segundo sobre el genoma de SARS-CoV-2 (3×10^4 bases), unas siete horas sobre el de *E. coli* (5×10^6) y casi tres siglos sobre el humano (3×10^9), mientras que uno lineal tarda, respectivamente, microsegundos, milisegundos y segundos.

0.1.4 Vectorización: la misma operación sobre millones de datos

Python es un lenguaje *interpretado*: en un bucle `for`, el intérprete vuelve a averiguar en cada vuelta qué tipo de objeto tiene delante, dónde está su método de comparación y cómo sumar el resultado. Cada iteración cuesta decenas de nanosegundos, aunque la operación útil (comparar un byte) cueste menos de uno. Llenar una placa de 96 pozos con una micropipeta monocal, pozo por pozo, se parece mucho a eso.

La biblioteca NumPy resuelve el problema con una estructura de datos, el arreglo `ndarray`, y un estilo de programación, la *vectorización* (Harris et al., 2020). Un arreglo es un bloque contiguo de memoria descrito por cuatro atributos: un puntero al primer byte, un tipo de dato (`dtype`) común a todos los elementos, una forma

(shape) y unos pasos (strides) que indican cuántos bytes hay que avanzar para moverse una posición en cada eje. La dirección del elemento (i, j) de una matriz es

$$\text{dir}(i, j) = \text{dir}_0 + i s_0 + j s_1. \quad (0.4)$$

DONDE

dir_0	Dirección de memoria del primer elemento.
s_0, s_1	Pasos (strides) en bytes: cuánto avanzar para cambiar de fila y de columna.
i, j	Índices de fila y columna (desde 0).

Como todos los elementos tienen el mismo tipo y están uno tras otro, una operación como `a == 71` (¿es esta base una G, cuyo código ASCII es 71?) puede ejecutarse en un bucle escrito en C, compilado, que el procesador además puede paralelizar con instrucciones vectoriales (SIMD). La ecuación (0.4) explica también un truco muy útil: transponer una matriz o tomar una de cada tres bases (`a[:,3]`, las primeras posiciones de los codones) no copia datos, solo cambia los pasos. Esas *vistas* son gratuitas.

```
gc_vectorizado.py

import numpy as np

def gc_loop(seq: str) -> float:
    n = 0
    for b in seq:
        if b == "G" or b == "C":
            n += 1
    return n / len(seq)

def gc_numpy(seq: str) -> float:
    a = np.frombuffer(seq.encode("ascii"), dtype=np.uint8)
    is_gc = (a == ord("G")) | (a == ord("C")) # máscara booleana
    return np.count_nonzero(is_gc) / a.size
```

La figura 0.2 compara ambas versiones. Extrapolada al genoma humano, la diferencia es de unos 160 segundos frente a menos de uno. Observe que ambas versiones siguen siendo $\Theta(n)$: la vectorización cambia la constante, no la complejidad.

La vectorización se completa con la *difusión* (broadcasting): cuando se combinan arreglos de formas distintas, NumPy repite virtualmente las dimensiones de tamaño 1 sin copiar datos. La expresión `x[:, None] == y[None, :]`, que compara cada base

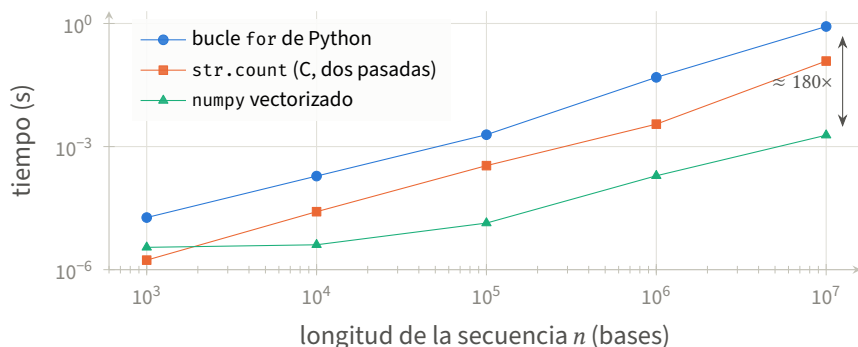


Figura 0.2. Tiempo para calcular el contenido GC de secuencias aleatorias de 10^3 a 10^7 bases (mejor de tres repeticiones, medido con `figuras/cap00/generar.py`). En escala doble logarítmica, un algoritmo $\Theta(n)$ es una recta de pendiente 1: los tres métodos son lineales, como predice el análisis asintótico. Lo que cambia es la constante c de la ecuación (0.3): el bucle interpretado gasta unos 50 ns por base y la versión vectorizada unos 0,3 ns.

de una secuencia con cada base de otra y construye la matriz de un *dot plot* en una línea, es el ejemplo que usaremos en el capítulo 3.

0.1.5 Números en la máquina: enteros y punto flotante

Los enteros de la máquina tienen un tamaño fijo: un `np.uint8` solo guarda valores de 0 a 255, y $255 + 1$ da 0 sin aviso, como un cuentakilómetros que da la vuelta. Los `int` de Python crecen según haga falta, pero los arreglos de NumPy no.

Los números reales plantean un problema más sutil. Una computadora no puede almacenar π , ni siquiera 0,1, con infinitos decimales. El estándar IEEE 754, que siguen prácticamente todos los procesadores, representa un número de doble precisión (`float64`) con 64 bits: uno de signo, once de exponente y 52 de fracción (Goldberg, 1991):

$$x = (-1)^s \times \left(1 + \sum_{k=1}^{52} f_k 2^{-k} \right) \times 2^{e-1023}. \quad (0.5)$$

DONDE

s	Bit de signo (0 positivo, 1 negativo).
f_k	Bits de la fracción o mantisa; el 1 inicial está implícito (“bit escondido”).
e	Exponente almacenado, un entero de 11 bits; se le resta el sesgo 1023.

La consecuencia fundamental de la ecuación (0.5) es que los números representables no están distribuidos uniformemente: entre dos potencias de 2 consecutivas hay siempre el mismo número de ellos (2^{52}), así que su separación *absoluta* crece con la magnitud mientras su separación *relativa* se mantiene casi constante: entre 1 y 2 los números vecinos distan 2^{-52} , entre 2^{52} y 2^{53} distan exactamente 1, y $2^{53} + 1$ ya no es representable. El número que mide esa separación relativa es el *épsilon de la máquina*, $\varepsilon = 2^{-52} \approx 2,22 \times 10^{-16}$, y de él se deriva el modelo estándar del error de redondeo:

$$\text{fl}(x \circ y) = (x \circ y)(1 + \delta), \quad |\delta| \leq u = \frac{\varepsilon}{2}. \quad (0.6)$$

DONDE

$\text{fl}(\cdot)$	Resultado que devuelve la máquina: el número representable más cercano al valor exacto.
$\circ$	Cualquiera de las operaciones $+$, $-$, $\times$, $\div$.
δ, u	Error relativo cometido y su cota, la unidad de redondeo ($\approx 1,1 \times 10^{-16}$).

La ecuación (0.6) parece inocente, porque un error de 10^{-16} rara vez importa. Hay, sin embargo, tres situaciones en las que muerde. La primera es la comparación exacta: `0.1 + 0.2 == 0.3` es `False`, porque ni 0,1 ni 0,2 tienen representación binaria finita y sus errores de redondeo no se cancelan; por eso se compara con una tolerancia (`np.isclose`). La segunda es la *cancelación catastrófica*: al restar dos números casi iguales, los dígitos correctos se anulan y queda el error, de modo que `1e16 + 1 - 1e16` devuelve 0. La tercera, la más importante en bioinformática, es el *subdesbordamiento* (*underflow*) al multiplicar muchas probabilidades.

Probabilidades diminutas y el espacio logarítmico

La probabilidad de observar una secuencia concreta de 1000 bases bajo un modelo uniforme es $0,25^{1000} \approx 10^{-602}$. El número positivo más pequeño que admite un `float64` es del orden de 5×10^{-324} , así que el producto se redondea a cero exacto,

y con él cualquier razón de verosimilitudes que dependa de él. Los modelos de Márkov, los HMM y la inferencia filogenética del resto del libro multiplican miles de probabilidades; sin un remedio, serían inutilizables.

El remedio es trabajar con logaritmos: $\log \prod_i p_i = \sum_i \log p_i$, y la suma de 1000 términos de $\ln 0,25$ es simplemente $-1386,29$, un número perfectamente ordinario. La dificultad aparece cuando hay que *sumar* probabilidades (por ejemplo, para marginalizar sobre estados ocultos) que solo conocemos por sus logaritmos. La solución es un pequeño teorema que se usa en todas partes.

Teorema 0.1. (Log-suma-exp)

Sean $a_1, \dots, a_k$ números reales (logaritmos de probabilidades) y $m = \max_i a_i$. Entonces

$$\log \sum_{i=1}^k e^{a_i} = m + \log \sum_{i=1}^k e^{a_i - m}, \quad (0.7)$$

y el lado derecho puede evaluarse sin desbordamiento ni subdesbordamiento, porque todos los exponentes son ≤ 0 y al menos uno vale exactamente 0.

DONDE

a_i	Logaritmo de la i -ésima probabilidad, $a_i = \log p_i$.
m	El mayor de los a_i ; se “factoriza” fuera de la suma.

La demostración es una línea: $\sum_i e^{a_i} = e^m \sum_i e^{a_i - m}$, y se toma el logaritmo. El truco está en que la suma de la derecha vale entre 1 y k , un número que la máquina maneja sin problemas.

Por ejemplo, con $\log p_1 = -1000$ y $\log p_2 = -1001$, la evaluación directa (`np.log` de la suma de `np.exp(-1000)` y `np.exp(-1001)`) devuelve $-\infty$, mientras que la ecuación (0.7) da $-1000 + \log(1 + e^{-1}) = -999,68674$; NumPy y SciPy la ofrecen como `np.logaddexp` y `scipy.special.logsumexp`.

0.1.6 Un primer modelo: el contenido GC y la distribución binomial

Con las herramientas anteriores podemos formular nuestra primera medida bioinformática. Los pares G–C se unen con tres puentes de hidrógeno y los pares A–T con dos, de modo que la proporción de G y C influye en la estabilidad térmica del ADN, en el diseño de cebadores y en la eficiencia de la secuenciación. Además,

varía mucho entre especies y dentro de un mismo genoma. El *contenido GC* de una secuencia x es

$$GC(x) = \frac{n_G + n_C}{n_A + n_C + n_G + n_T}. \quad (0.8)$$

DONDE

$n_A, \dots, n_T$ Número de apariciones de cada base en x ; las ambigüedades como N no cuentan en el denominador.

Al calcular el GC en ventanas deslizantes a lo largo de un genoma, los valores fluctúan. ¿Cuánto de esa fluctuación es biología y cuánto es puro azar de muestreo? Para responder necesitamos un *modelo nulo*. Supongamos que cada base es G o C con probabilidad p , independientemente de las demás. El número de bases GC en una ventana de W posiciones sigue entonces una distribución binomial, y la proporción observada tiene media y varianza

$$K \sim \text{Bin}(W, p), \quad \mathbb{E}\left[\frac{K}{W}\right] = p, \quad \text{Var}\left[\frac{K}{W}\right] = \frac{p(1-p)}{W}. \quad (0.9)$$

DONDE

K Número de bases G o C en la ventana (variable aleatoria).
 W Tamaño de la ventana, en bases.
 p Probabilidad de que una base sea G o C; se estima con el GC del genoma completo.

La desviación estándar $\sqrt{p(1-p)/W}$ decrece como $1/\sqrt{W}$: ventanas cuatro veces más grandes reducen el ruido a la mitad, a costa de perder resolución espacial. Por el teorema central del límite, alrededor del 95 % de las ventanas deberían caer dentro de $p \pm 2\sqrt{p(1-p)/W}$ si el modelo nulo fuera cierto.

Ejemplo 0.1. (El GC de SARS-CoV-2 frente a su modelo nulo)

El genoma de referencia de SARS-CoV-2 (NC_045512.2) tiene 29 903 bases, de las cuales 11 355 son G o C: $p = 0,3797$. Con ventanas de $W = 500$ bases,

$$\sigma = \sqrt{\frac{0,3797 \times 0,6203}{500}} = 0,0217,$$

de modo que el modelo binomial predice que casi todas las ventanas tendrán un GC entre 0,336 y 0,423. En el genoma real (figura 0.3), el GC por ventana

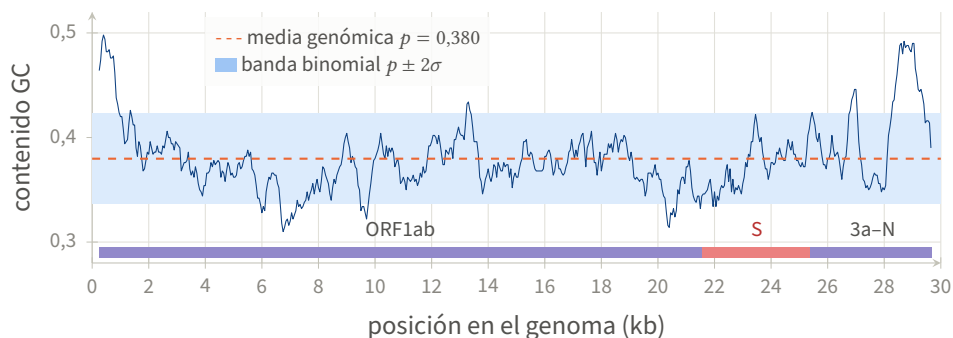


Figura 0.3. Contenido GC en ventanas de 500 bases (paso de 50) a lo largo del genoma de referencia de SARS-CoV-2. La banda azul es la región $p \pm 2\sigma$ que predice el modelo binomial (ecuación (0.9)); si el genoma fuera homogéneo, solo un 5 % de las ventanas escaparía de ella. Escapa casi el triple. Los extremos, sobre todo hacia el final del genoma (genes estructurales y regiones no traducidas), revelan heterogeneidad biológica real.

oscila entre 0,310 y 0,498, y el 14,4 % de las ventanas cae fuera de la banda de $\pm 2\sigma$, casi el triple de lo esperado. El exceso de variación (*sobredispersión*) nos dice que el genoma no es una sucesión de monedas idénticas: su composición cambia de una región a otra.

Calcular el GC de cada ventana por separado costaría $\Theta(nW)$. Un truco clásico lo reduce a $\Theta(n)$: se calcula una vez la suma acumulada $S_i = \sum_{k < i} g_k$ del indicador g_k (1 si la base k es G o C) y el GC de la ventana $[i, i + W)$ es simplemente $(S_{i+W} - S_i) / W$.

gc_ventanas.py

```
import numpy as np
from Bio import SeqIO

rec = SeqIO.read("NC_045512.2.gb", "genbank")
a = np.frombuffer(bytes(rec.seq), dtype=np.uint8)
g = ((a == ord("G")) | (a == ord("C"))).astype(float)
S = np.concatenate([[0.0], np.cumsum(g)])           # sumas acumuladas
W, step = 500, 50
starts = np.arange(0, a.size - W + 1, step)
gc = (S[starts + W] - S[starts]) / W                 # Theta(n), no Theta(nW)
p = g.mean(); sd = np.sqrt(p * (1 - p) / W)
print(f"p={p:.4f} fuera de 2 sd: {np.mean(abs(gc - p) > 2*sd):.1%}")
```

0.1.7 Datos tabulares: pandas y el formato ordenado

Casi todo resultado bioinformático termina en una tabla: genes con sus coordenadas, muestras con sus metadatos, resultados de BLAST con sus puntuaciones. La biblioteca pandas introdujo en Python el `DataFrame`, una tabla bidimensional cuyas columnas pueden tener tipos distintos y cuyas filas y columnas llevan etiquetas, con operaciones de agrupación, unión y remodelado inspiradas en las bases de datos relacionales y en R (McKinney, 2010).

Tener la herramienta no basta; hay que decidir la *forma* de la tabla. Una misma información puede escribirse de muchas maneras, y algunas facilitan el análisis mientras que otras lo entorpecen. Wickham (2014) formalizó la forma más conveniente con el nombre de datos *ordenados* (*tidy data*).

Definición 0.3. (Datos ordenados)

Un conjunto de datos está ordenado si (1) cada variable forma una columna, (2) cada observación forma una fila y (3) cada tipo de unidad observacional forma una tabla.

Las hojas de cálculo del laboratorio casi nunca cumplen la definición: suelen usar la forma “ancha” de la figura 0.4, con la variable *muestra* escondida en los encabezados. Para agrupar, filtrar o graficar conviene la forma “larga”, una medición por fila; en pandas se pasa de una a otra con `melt` y `pivot`.

Una advertencia: al abrir una lista de genes, Excel convierte por defecto símbolos como *SEPT2* o *MARCH1* en fechas, y Ziemann et al. (2016) hallaron que alrededor de una quinta parte de los artículos de genómica con listas de genes en Excel contenían estas conversiones erróneas. Lea y escriba las tablas con código y trate los archivos originales como de solo lectura.

0.1.8 Principios de visualización científica

Una figura es un instrumento de medida para el ojo. Como cualquier instrumento, tiene una resolución, y esa resolución no es la misma para todas las magnitudes visuales. Cleveland y McGill (1984) lo midieron experimentalmente: pidieron a observadores que estimaran proporciones codificadas de distintas maneras y ordenaron las codificaciones según la exactitud de los juicios. De más a menos precisas: posición sobre una escala común; posición en escalas no alineadas; longitud, dirección y ángulo; área; volumen y curvatura; y, por último, sombreado y saturación del color.

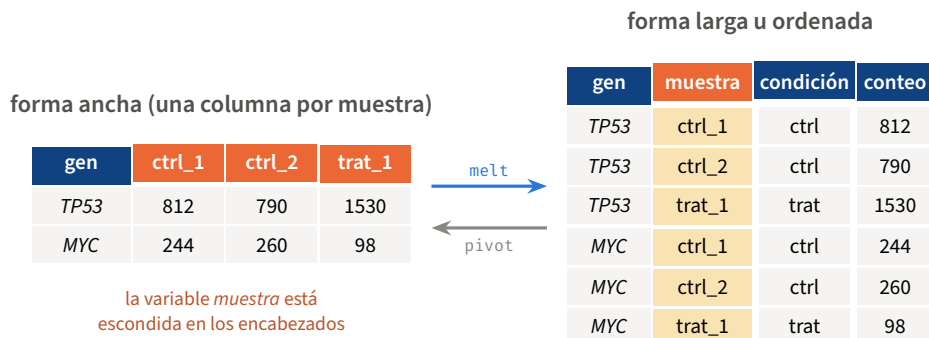


Figura 0.4. La misma tabla de conteos de expresión en dos formas. En la forma ancha (izquierda) los nombres de las muestras son encabezados; en la forma ordenada (derecha) son valores de una columna, cada fila es una observación y aparece una variable nueva, la condición, derivada de la muestra. Todas las operaciones de agrupación y de gráficas de pandas, seaborn o plotly esperan la forma ordenada.

La jerarquía justifica reglas prácticas que usaremos en todo el libro: preferir barras o puntos a gráficas de pastel (que codifican con ángulos y áreas), reservar el color para distinguir categorías o para magnitudes secundarias, y comparar lo que importa comparar sobre un eje común. Tufte (2001) añadió un principio de economía: la *razón datos-tinta*,

$$R = \frac{\text{tinta dedicada a los datos}}{\text{tinta total de la figura}}, \quad (0.10)$$

debe ser tan alta como sea razonable. Rejillas gruesas, fondos sombreados, efectos tridimensionales y leyendas redundantes consumen tinta sin aportar información; Tufte los llamó *chartjunk*. En la práctica, esto se traduce en rotular directamente las series en lugar de usar leyendas, en rejillas tenues y en títulos que enuncian la conclusión.

El color como dato

Cuando el color codifica una magnitud continua (una matriz de expresión, un mapa de contactos), el mapa de color debe ser *perceptualmente uniforme*: pasos iguales en el dato deben producir pasos iguales en la sensación visual. La luminosidad L^* del espacio CIELAB es una buena medida de esa sensación. El mapa arcoíris jet, durante años el predeterminado de muchos programas, viola esta condición: su luminosidad sube y baja de forma irregular, de modo que crea bordes donde el dato

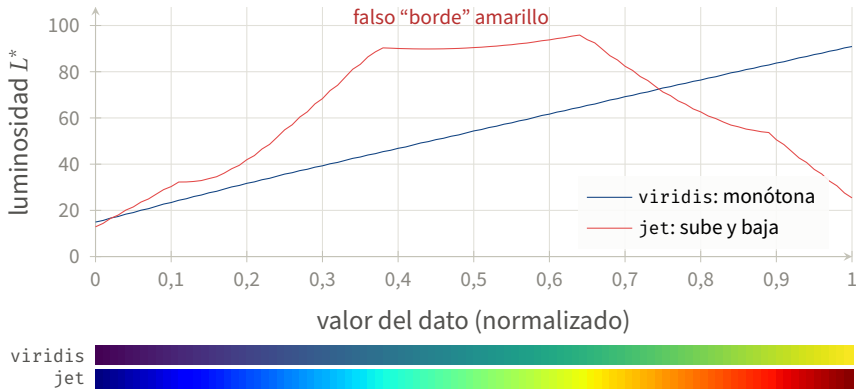


Figura 0.5. Luminosidad perceptual L^* (CIE LAB) a lo largo de dos mapas de color, calculada a partir de sus valores RGB. En viridis (azul) crece de forma monótona y casi lineal: el ojo lee un orden correcto incluso en escala de grises. En jet (rojo) sube hasta el cian y el amarillo y vuelve a caer: dos datos distintos pueden verse igual de claros, y la franja amarilla central aparece como un borde que no existe en los datos.

es suave y oculta cambios donde el dato varía (figura 0.5). Además, es ilegible para las personas con deficiencias en la visión del color, que confunden sus rojos y verdes. Crameri et al. (2020) documentaron lo extendido de este mal uso y recomiendan mapas derivados científicamente, como viridis o cividis.

Todas las figuras de este libro, y las del curso en Colab, se construyen con estos criterios. En Python la herramienta básica es matplotlib (Hunter, 2007), sobre la que se apoyan bibliotecas de más alto nivel; para figuras interactivas usaremos plotly en los cuadernos.

0.2 Biopython y herramientas de línea de comandos

 Colab · 0.2 · Biopython y herramientas

 abrir notebook

Instrumentos calibrados por otros

Nadie fabrica su propia micropipeta antes de cada experimento. Se compra un instrumento calibrado, probado por miles de usuarios, y se dedica el esfuerzo a la pregunta biológica. En la sección anterior escribimos a mano

funciones para contar bases; fue útil para entender qué ocurre por dentro. A partir de aquí usaremos dos tipos de instrumentos ya calibrados: las bibliotecas de Python, que son como la gaveta de pipetas, puntas y gradillas de la mesa de trabajo, y los programas de línea de comandos, que son los equipos grandes del laboratorio (el termociclador, el secuenciador), especializados, muy rápidos y controlados mediante instrucciones escritas.

Biopython es la biblioteca de referencia para biología molecular en Python: un conjunto de herramientas libres, desarrollado por una comunidad internacional, que va de la manipulación de secuencias y la lectura de decenas de formatos al acceso programático a bases de datos (Cock et al., 2009). Aquí la usaremos para ir del ADN a la proteína, descargar y verificar un genoma real y conectar Python con programas externos.

0.2.1 El objeto Seq: complemento y complemento reverso

Un objeto Seq se comporta como una cadena de texto, pero “conoce” la biología: sabe complementar, transcribir y traducir. La operación más frecuente, y la que más errores provoca, es el complemento reverso. Las dos hebras de la doble hélice son *antiparalelas*: una corre en sentido $5' \rightarrow 3'$ y la otra en sentido $3' \rightarrow 5'$. Como por convención toda secuencia se escribe de $5'$ a $3'$, leer la hebra opuesta exige complementar cada base e invertir el orden.

Definición 0.4. (Complemento reverso)

Sea $\kappa : \Sigma \rightarrow \Sigma$ el complemento de Watson y Crick, $\kappa(A) = T$, $\kappa(T) = A$, $\kappa(C) = G$, $\kappa(G) = C$. El complemento reverso de $x = x_0 \cdots x_{n-1}$ es la secuencia $\bar{x}$ de la misma longitud con

$$\bar{x}_i = \kappa(x_{n-1-i}), \quad i = 0, \dots, n-1. \quad (0.11)$$

DONDE

κ	Función de complemento base a base.
$\bar{x}_i$	Base en la posición i del complemento reverso; se toma de la posición simétrica $n-1-i$ de x .

De la definición se sigue que el complemento reverso es una *involución*: aplicado dos veces devuelve la secuencia original, $\bar{\bar{x}} = x$, porque tanto κ como la inversión del orden lo son. Una secuencia que coincide con su complemento reverso, $\bar{x} = x$, se llama *palindrómica*; muchos sitios de restricción lo son, como GAATTC (EcoRI), porque las enzimas que los cortan actúan como dímeros simétricos.

0.2.2 El código genético como función

La traducción es una función que lee la secuencia de tres en tres. ¿Por qué tres? Porque con palabras de k letras sobre un alfabeto de cuatro se pueden escribir 4^k palabras distintas, y hacen falta al menos 21 (veinte aminoácidos y una señal de parada): $4^1 = 4$ y $4^2 = 16$ no alcanzan, mientras que $4^3 = 64$ sobran. El código genético estándar es, por lo tanto, una función sobreyectiva

$$\tau : \{A, C, G, U\}^3 \longrightarrow \{20 \text{ aminoácidos}\} \cup \{*\}, \quad |\text{dominio}| = 64, \quad |\text{imagen}| = 21. \quad (0.12)$$

DONDE

τ	Tabla de traducción; Biopython la guarda en <code>Bio.Data.CodonTable</code> (la estándar es la tabla 1 del NCBI).
$\{\dots\}^3$	Conjunto de todos los codones, palabras de tres nucleótidos.
$*$	Señal de parada (UAA, UAG, UGA).

Como hay más codones que significados, el código es *degenerado*: varios codones sinónimos codifican el mismo aminoácido. La figura 0.6 muestra que la degeneración no es aleatoria. Los sinónimos suelen diferir solo en la tercera base, de modo que muchas mutaciones en esa posición son silenciosas, y los aminoácidos químicamente parecidos ocupan casillas vecinas, lo que amortigua el efecto de los errores de lectura. Leucina, serina y arginina tienen seis codones cada una; metionina y triptófano, uno solo. AUG, el codón de la metionina, es además la señal de inicio más común.

0.2.3 Seis marcos de lectura y la estadística de los ORF

El ADN no tiene espacios entre codones. Una secuencia puede leerse empezando en la primera, la segunda o la tercera base, y lo mismo en su complemento reverso: hay seis *marcos de lectura*. En general solo uno de ellos, en cada región, corresponde a la proteína real. Un *marco abierto de lectura* (ORF, *open reading frame*) es un tramo de codones sin paradas; los genes que codifican proteínas son ORF largos, y buscar ORF largos fue durante décadas la primera estrategia de anotación de genomas.

		2.ª base				
		U	C	A	G	
1.ª base	U	UUU PheF	UCU SerS	UAU TyrY	UGU CysC	U
		UUC PheF	UCC SerS	UAC TyrY	UGC CysC	C
		UUA LeuL	UCA SerS	UAA Stop*	UGA Stop*	A
		UUG LeuL	UCG SerS	UAG Stop*	UGG TrpW	G
	C	CUU LeuL	CCU ProP	CAU HisH	CGU ArgR	U
		CUC LeuL	CCC ProP	CAC HisH	CGC ArgR	C
		CUA LeuL	CCA ProP	CAA GlnQ	CGA ArgR	A
		CUG LeuL	CCG ProP	CAG GlnQ	CGG ArgR	G
	A	AUU IleI	ACU ThrT	AAU AsnN	AGU SerS	U
		AUC IleI	ACC ThrT	AAC AsnN	AGC SerS	C
		AUA IleI	ACA ThrT	AAA LysK	AGA ArgR	A
		AUG MetM	ACG ThrT	AAG LysK	AGG ArgR	G
	G	GUU ValV	GCU AlaA	GAU AspD	GGU GlyG	U
		GUC ValV	GCC AlaA	GAC AspD	GGC GlyG	C
		GUA ValV	GCA AlaA	GAA GluE	GGA GlyG	A
		GUG ValV	GCG AlaA	GAG GluE	GGG GlyG	G

Figura 0.6. El código genético estándar (tabla 1 del NCBI), generado a partir de Bio.Data.CodonTable. Cada bloque corresponde a una primera base (filas) y una segunda base (columnas); dentro del bloque, la tercera base recorre U, C, A, G. Colores: hidrofóbicos (amarillo), polares sin carga (verde azulado), con carga positiva (azul), con carga negativa (rojo), especiales G, P y C (violeta) y paradas (gris). Los bloques casi monocromáticos de la columna U (hidrofóbicos) y la vecindad de D y E muestran que el código agrupa los aminoácidos por química.

¿Qué significa “largo”? Una vez más, necesitamos un modelo nulo. Si las bases fueran independientes con frecuencias p_A, p_C, p_G, p_T , la probabilidad de que un codón cualquiera sea de parada sería

$$\pi = p_T p_A p_A + p_T p_A p_G + p_T p_G p_A = p_T p_A (p_A + 2p_G), \tag{o.13}$$

sumando las probabilidades de TAA, TAG y TGA. El número L de codones consecutivos sin parada sigue entonces una distribución geométrica: cada codón es un “intento” que termina el ORF con probabilidad π . Así,

$$P(L \geq k) = (1 - \pi)^k, \quad E[L] = \frac{1 - \pi}{\pi}. \tag{o.14}$$

DONDE

$p_A, \dots, p_T$	Frecuencias de las bases en el genoma (modelo de bases independientes).
π	Probabilidad de que un codón al azar sea de parada.
L	Número de codones de un tramo abierto antes de encontrar una parada.
k	Longitud umbral, en codones.

Ejemplo 0.2. (¿Cuántos ORF largos produce el azar?)

Con bases equiprobables, $\pi = 3/64$ y un ORF aleatorio tiene en promedio $E[L] = 61/3 \approx 20$ codones; la probabilidad de superar 100 codones es $(61/64)^{100} = 0,0082$. El genoma de SARS-CoV-2, sin embargo, es rico en A y T ($p_A = 0,299$, $p_T = 0,321$, $p_G = 0,196$), lo que hace las paradas más frecuentes: la ecuación (0.13) da $\pi = 0,0664$, es decir, unas $9968 \times 0,0664 \approx 662$ paradas esperadas por marco, y un ORF aleatorio medio de solo 14 codones. La probabilidad de que un tramo aleatorio supere 100 codones cae a $(1 - 0,0664)^{100} = 0,0010$, así que en todo un marco esperaríamos $662 \times 0,0010 \approx 0,7$ ORF así de largos por puro azar. Los más de 7000 codones de ORF1ab son inexplicables por azar.

La figura 0.7 confirma la predicción: los genes destacan como tramos largos sin paradas. Pero la hebra complementaria contiene algunos ORF de más de 100 codones que no codifican nada, porque el genoma real no está hecho de bases independientes.

0.2.4 Archivos y registros: SeqRecord y SeqIO

Una secuencia aislada rara vez basta; necesitamos su identificador, su descripción y sus anotaciones. Biopython agrupa todo ello en un objeto SeqRecord, y el módulo SeqIO traduce entre registros y archivos en decenas de formatos. Los dos formatos que usaremos desde el primer día son FASTA, mínimo (una línea de encabezado que empieza con > seguida de la secuencia), y GenBank, rico en metadatos. Estudiaremos los formatos con detalle en el capítulo 2; aquí basta con saber que el encabezado de un GenBank se convierte en los atributos .id, .description y .annotations; su tabla de características, en una lista de objetos SeqFeature (con tipo, localización y calificadores), y su sección ORIGIN, en el atributo .seq.

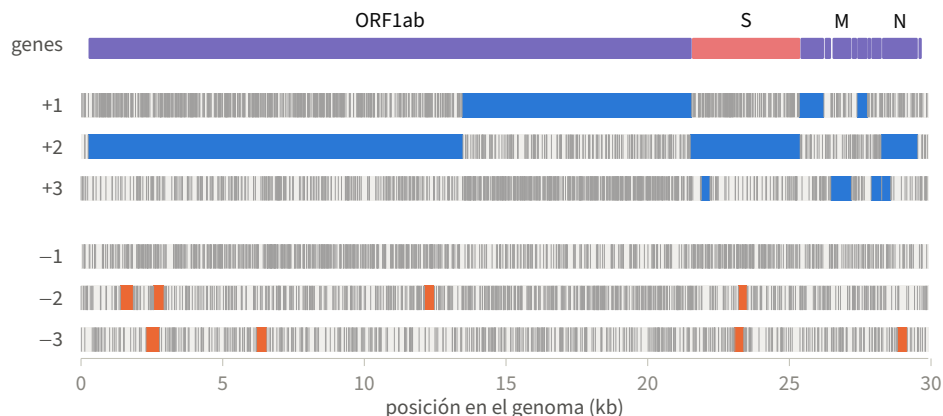


Figura 0.7. Los seis marcos de lectura del genoma de SARS-CoV-2. Cada raya gris es un codón de parada; los bloques de color son ORF de al menos 100 codones (azul, hebra directa; naranja, hebra complementaria). Arriba, los genes anotados en el GenBank. El marco +2 tiene solo 255 paradas, frente a las ~660 que predice la ecuación (0.13), porque contiene los más de 13 000 nucleótidos abiertos de ORF1a. En la hebra complementaria aparecen algunos ORF largos que no codifican nada, más de los que predice el modelo independiente: la longitud es una pista, no una prueba.

SeqIO ofrece dos funciones de lectura con una diferencia importante. `SeqIO.read` espera un archivo con *exactamente* un registro y falla si hay más, lo que protege contra errores silenciosos. `SeqIO.parse` devuelve un *iterador*: un objeto que produce los registros uno por uno, a medida que se le piden, sin cargar el archivo completo en memoria. Si un archivo contiene N registros de tamaño medio ℓ , leerlo todo en una lista necesita memoria $\Theta(N\ell)$, mientras que recorrerlo con el iterador necesita solo $\Theta(\ell)$, la del registro actual. Con un archivo FASTQ de 50 millones de lecturas, la diferencia es la que hay entre un análisis que funciona en Colab y otro que agota la memoria de la máquina.

0.2.5 Bases de datos programables: Entrez y el genoma de SARS-CoV-2

El Centro Nacional de Información Biotecnológica de Estados Unidos (NCBI) mantiene decenas de bases de datos enlazadas, desde GenBank y RefSeq hasta PubMed, y las expone mediante una interfaz programática, las E-utilities, que el sistema Entrez unifica (Sayers et al., 2022). El módulo `Bio.Entrez` traduce esas llamadas a funciones de Python: `esearch` busca identificadores que satisfacen una

consulta, `efetch` descarga los registros y `elink` navega entre bases de datos. El NCBI exige identificarse con un correo y limita las solicitudes por segundo.

```
descargar_genoma.py

from Bio import Entrez, SeqIO

Entrez.email = "su.correo@universidad.edu"      # obligatorio
acc = "NC_045512.2"
with Entrez.efetch(db="nucleotide", id=acc, rettype="gbwithparts",
                  retmode="text") as handle:
    rec = SeqIO.read(handle, "genbank")
    cds = [f for f in rec.features if f.type == "CDS"]
    print(rec.description, len(rec), "nt", len(cds), "CDS")
```

El registro que descargamos es la secuencia de referencia del aislado Wuhan-Hu-1. Wu et al. (2020) lo obtuvieron por secuenciación metagenómica de ARN a partir del lavado broncoalveolar de un paciente hospitalizado en Wuhan en diciembre de 2019; el genoma completo mide 29 903 nucleótidos, y su análisis filogenético mostró que el nuevo virus estaba estrechamente emparentado (89,1 % de similitud nucleotídica) con un grupo de coronavirus similares al del SARS hallados en murciélagos. Es un excelente material de prácticas: cualquier resultado puede contrastarse con una literatura enorme.

Ejemplo 0.3. (Verificar una anotación y descubrir un salto de marco)

El GenBank anota 12 regiones codificantes (CDS) en 11 genes. Una prueba de consistencia natural es traducir cada CDS y comparar el resultado con la proteína que el propio archivo registra en el calificador `/translation`. Si extraemos cada región ingenuamente, desde su inicio hasta su final (con `seq[start:end]` y `translate`), 11 de las 12 traducciones coinciden. La excepción es la poliproteína ORF1ab, de 7096 aminoácidos: la traducción ingenua se detiene mucho antes. Si en cambio usamos `feature.extract(seq)`, que respeta la localización compuesta `join(266..13468,13468..21555)`, la traducción coincide residuo a residuo. La base 13 468 aparece en los dos intervalos: se lee dos veces. Es la huella de un *desplazamiento ribosómico del marco* en -1 : una estructura de ARN en forma de pseudonudo hace que, en una fracción de las traducciones, el ribosoma retroceda un nucleótido y continúe en otro marco. Así, un mismo ARN produce la poliproteína corta pp1a (sin salto, termina

en la parada del marco original) y la larga ppiab (con salto). La verificación computacional no solo detectó una “anomalía”: redescubrió un mecanismo biológico.

0.2.6 La línea de comandos y la filosofía Unix

Buena parte de las herramientas bioinformáticas más usadas (BLAST, BWA, samtools, SPAdes, IQ-TREE) son programas de línea de comandos escritos en C o C++. No tienen interfaz gráfica: se invocan escribiendo su nombre y sus opciones en una terminal, leen archivos o flujos de texto y escriben archivos o flujos de texto. Esta forma de trabajar se remonta al sistema operativo Unix. En su descripción original, Ritchie y Thompson (1974) presentaron, entre otras ideas, la *tubería (pipe)*: un canal que conecta la salida de un programa con la entrada del siguiente, de modo que programas pequeños, cada uno especializado en una tarea, se combinan para resolver problemas grandes.

Todo proceso de Unix tiene tres flujos estándar: la entrada (stdin), la salida (stdout) y el canal de errores (stderr), además de un *código de salida*, un entero que vale o si el programa terminó bien. Si vemos cada programa como una función que transforma texto en texto, una tubería es una composición de funciones:

$$p_1 | p_2 | \dots | p_k \equiv f_k \circ \dots \circ f_2 \circ f_1. \tag{0.15}$$

DONDE	
p_i	El i -ésimo programa de la tubería, con sus opciones.
$ $	Operador de tubería del intérprete de comandos: conecta stdout de la izquierda con stdin de la derecha.
f_i	Transformación de texto que realiza p_i ; la composición se lee de derecha a izquierda.

Los programas de una tubería se ejecutan *simultáneamente*: el segundo empieza a procesar las primeras líneas mientras el primero todavía está leyendo el archivo. Igual que el iterador de SeqIO.parse, una tubería procesa archivos mucho más grandes que la memoria disponible. La figura 0.8 muestra una tubería típica, que cuenta los tipos de anotación de un archivo GFF.

0.2.7 BLAST+ desde Python

BLAST es el ejemplo arquetípico de herramienta de línea de comandos: el conjunto BLAST+ reorganizó el código original en C++ y lo dividió en programas

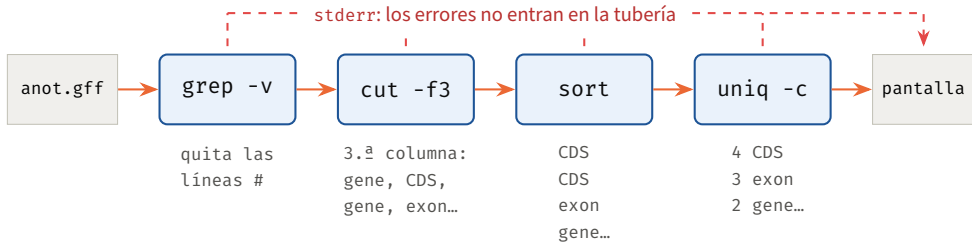


Figura 0.8. La tubería `grep -v '^#' anot.gff | cut -f3 | sort | uniq -c`. Cada programa lee el flujo de texto que le entrega el anterior (flechas naranjas, `stdout`→`stdin`), lo transforma y lo pasa al siguiente; debajo de cada uno, un fragmento de lo que produce. Los cuatro procesos corren a la vez y ninguno necesita cargar el archivo completo.

especializados, `makeblastdb` para indexar una base de datos y `blastn`, `blastp` y compañía para buscar en ella (Camacho et al., 2009). Estudiaremos su algoritmo y su estadística en el capítulo 3. Aquí nos interesa como instrumento: cómo se instala, cómo se invoca desde Python y cómo se lee lo que produce.

En Colab, BLAST+ se instala desde los repositorios del sistema (`apt-get install ncbi-blast+`); en un equipo propio conviene usar un gestor de entornos como `conda`, que veremos en la sección 0.3. Para llamarlo desde Python se usa el módulo estándar `subprocess`, pasando los argumentos como una lista y pidiendo `check=True` para que un código de salida distinto de cero se convierta en una excepción. El formato tabular `-outfmt 6` produce una fila por alineamiento, lista para `pandas`.

```

blast_subprocess.py

import subprocess
import pandas as pd

subprocess.run(["makeblastdb", "-in", "sarscov2.fasta",
               "-dbtype", "nucl", "-out", "sarscov2_db"], check=True)
cols = ("qseqid sseqid pident length mismatch gapopen "
        "qstart qend sstart send evalue bitscore").split()
subprocess.run(["blastn", "-query", "consulta.fasta",
               "-db", "sarscov2_db", "-out", "hits.tsv",
               "-outfmt", "6 " + " ".join(cols)], check=True)
hits = pd.read_csv("hits.tsv", sep="\t", names=cols)
print(hits.sort_values("bitscore", ascending=False).head())

```

¿Cuándo encuentra BLAST una secuencia mutada?

El cuaderno de la lección propone un experimento: tomar un fragmento de 300 nucleótidos del genoma, introducirle mutaciones al azar y comprobar si `blastn` lo encuentra. La respuesta depende de la estrategia de BLAST, que solo empieza a alinear donde encuentra una *semilla*: una palabra de w nucleótidos idéntica en la consulta y en la base de datos ($w = 28$ en el modo predeterminado de `blastn`, llamado megablast, y $w = 11$ con la opción `-task blastn`). Si cada posición conserva su base con probabilidad q (la identidad) de forma independiente, una palabra concreta sobrevive intacta con probabilidad q^w , y el número esperado de semillas en un fragmento de longitud ℓ es

$$E[\text{semillas}] = (\ell - w + 1) q^w. \quad (0.16)$$

DONDE

ℓ	Longitud del fragmento de consulta.
w	Longitud de palabra de la semilla (<code>-word_size</code>).
q	Identidad por posición entre la consulta y su origen, $q = 1 - \text{tasa de mutación}$.

Ejemplo 0.4. (Quince mutaciones en trescientas bases)

Con 15 mutaciones en 300 nucleótidos la identidad es $q = 0,95$. Con palabras de 11 bases, cada una sobrevive con probabilidad $0,95^{11} = 0,569$; hay $300 - 11 + 1 = 290$ palabras, así que esperamos unas $290 \times 0,569 \approx 165$ semillas. Con el modo predeterminado ($w = 28$) la supervivencia cae a $0,95^{28} = 0,238$, pero sobre 273 palabras aún quedan unas 65 semillas: BLAST encontrará el fragmento en ambos casos. El panorama cambia con $q = 0,80$ (60 mutaciones): con $w = 11$ esperamos todavía $290 \times 0,80^{11} \approx 25$ semillas, mientras que con $w = 28$ solo $273 \times 0,80^{28} \approx 0,5$. Es muy probable que megablast no encuentre nada. Para homólogos aún más lejanos, la búsqueda de ADN pierde sensibilidad incluso con palabras cortas y conviene comparar proteínas.

El ejemplo resume el compromiso: el tamaño de palabra regula el equilibrio entre velocidad y sensibilidad, y es uno de los parámetros que conviene registrar siempre junto con los resultados. Que las mutaciones se distribuyan al azar es, a su vez, un supuesto: en genes reales las mutaciones se concentran en las terceras posiciones de los codones, y eso favorece a las semillas “discontinuas” que ignoran esas posiciones, una idea que retomaremos en el capítulo 3.

0.3 Reproducibilidad: semillas, versiones, hashes y Git

 Colab · 0.3 · Reproducibilidad

 abrir notebook

La receta de la abuela

“Agregue un poco de harina, una pizca de sal y hornee hasta que esté listo”. La receta funciona para quien la escribió, que sabe cuánto es “un poco” y reconoce el punto exacto del horno, pero nadie más obtiene el mismo pastel. Un análisis bioinformático sin semillas aleatorias registradas, sin versiones de los programas y sin una huella de los datos de entrada es esa receta: funcionó una vez, en una computadora, un día concreto, y nadie, ni siquiera su autor seis meses después, puede garantizar que volverá a obtener lo mismo.

0.3.1 ¿Qué significa reproducir un análisis?

La palabra “reproducibilidad” se usa con sentidos distintos, y conviene separarlos. Hablaremos de *repetibilidad* cuando el mismo investigador, con los mismos datos, el mismo código y el mismo entorno, obtiene el mismo resultado; de *reproducibilidad computacional* cuando lo obtiene otra persona a partir de los datos y el código publicados; y de *replicabilidad* cuando un estudio independiente, con datos nuevos, llega a la misma conclusión. La reproducibilidad computacional es el requisito mínimo: sin ella, una discrepancia con otro estudio no puede atribuirse ni a la biología ni a un error.

El problema no es hipotético. En una encuesta de *Nature* a 1576 investigadores, más del 70 % declaró haber intentado sin éxito reproducir un experimento de otro grupo, y más de la mitad, uno propio (Baker, 2016). En bioinformática la evidencia es más directa. Ioannidis et al. (2009) encargaron a dos equipos independientes reproducir una tabla o figura de 18 análisis de expresión con micromatrices publicados en *Nature Genetics*: lograron reproducir dos en principio y seis de forma parcial o con discrepancias; los diez restantes no pudieron reproducirse. La causa principal fue la falta de disponibilidad de los datos, y las discrepancias se debieron sobre todo a descripciones incompletas del procesamiento.

Un análisis computacional puede dejar de ser reproducible por cuatro vías: porque depende de *números aleatorios* no registrados, porque sus *datos* cambiaron sin que

nadie lo notara, porque su *código* se modificó después de producir los resultados, o porque su *entorno* (versiones de bibliotecas, del sistema operativo) ya no es el mismo. El resto del capítulo presenta un instrumento para cada vía.

0.3.2 El azar determinista: generadores pseudoaleatorios

Simulaciones, remuestreos *bootstrap*, inicializaciones de algoritmos de agrupamiento, divisiones de datos para entrenar un modelo: la bioinformática consume cantidades enormes de números aleatorios. Pero una computadora es una máquina determinista y no puede lanzar una moneda. Lo que hace es calcular una sucesión que *parece* aleatoria.

Definición 0.5. (Generador pseudoaleatorio)

Un generador de números pseudoaleatorios es una tupla $(\mathcal{S}, f, g, s_0)$ formada por un conjunto finito de estados $\mathcal{S}$, una función de transición $f : \mathcal{S} \rightarrow \mathcal{S}$, una función de salida $g : \mathcal{S} \rightarrow [0, 1)$ y un estado inicial s_0 , la *semilla*. El generador produce $u_n = g(s_n)$ con $s_{n+1} = f(s_n)$. Como $\mathcal{S}$ es finito, la sucesión es tarde o temprano periódica; el menor P tal que $s_{n+P} = s_n$ para todo n grande es su *período*.

La definición contiene la clave de la reproducibilidad: toda la sucesión queda determinada por la semilla. Es como una baraja mezclada de antemano: la semilla dice en qué carta empezar.

El generador más antiguo y más estudiado es el *congruencial lineal* (LCG), cuyo estado es un único entero:

$$x_{n+1} = (a x_n + c) \bmod m, \quad u_n = \frac{x_n}{m}. \tag{0.17}$$

DONDE

x_n	Estado del generador, un entero entre 0 y $m - 1$; x_0 es la semilla.
a, c, m	Multiplicador, incremento y módulo: constantes que definen el generador.
u_n	Número pseudoaleatorio en $[0, 1)$ que se entrega al usuario.

El período de un LCG no puede superar m , y alcanzar ese máximo exige elegir bien las constantes. El resultado clásico, que Knuth (1997) demuestra con detalle, es el siguiente.

Teorema 0.2. (*Período completo de un LCG (Hull-Dobell)*)

Si $c \neq 0$, el generador de la ecuación (0.17) tiene período m para toda semilla s y solo si: (i) c y m son primos entre sí; (ii) $a - 1$ es divisible por todos los factores primos de m ; y (iii) $a - 1$ es divisible por 4 si m lo es.

Ejemplo 0.5. (Un generador de juguete con período completo)

Tome $m = 16$, $a = 5$, $c = 3$. Las condiciones se cumplen: $\text{mcd}(3, 16) = 1$; el único primo de 16 es 2, que divide a $a - 1 = 4$; y como $4 \mid 16$, también $4 \mid (a - 1)$. Con semilla $x_0 = 7$, la sucesión es

$$\begin{aligned} 7 \rightarrow 6 \rightarrow 1 \rightarrow 8 \rightarrow 11 \rightarrow 10 \rightarrow 5 \rightarrow 12 \rightarrow 15 \\ \rightarrow 14 \rightarrow 9 \rightarrow 0 \rightarrow 3 \rightarrow 2 \rightarrow 13 \rightarrow 4 \rightarrow 7 \rightarrow \dots \end{aligned}$$

que recorre los 16 estados antes de repetirse. Con $a = 3$, en cambio, la condición (ii) se cumple pero la (iii) no, y el período se acorta. Observe también que los bits bajos son pésimos: $x_n \bmod 2$ alterna 1, 0, 1, 0..., un defecto general de los LCG con módulo potencia de 2.

0.3.3 Cuando el azar tiene estructura: RANDU

Período largo no significa buena calidad. El caso más famoso es RANDU, el generador de la biblioteca científica de IBM en los años sesenta, con $a = 65539$, $c = 0$ y $m = 2^{31}$, usado durante años en simulaciones publicadas. Su multiplicador tiene una forma especial, $a = 2^{16} + 3$, y eso basta para arruinarlo. Elevando al cuadrado,

$$a^2 = 2^{32} + 6 \cdot 2^{16} + 9 \equiv 6 \cdot 2^{16} + 9 = 6(2^{16} + 3) - 9 = 6a - 9 \pmod{2^{31}}, \quad (0.18)$$

porque 2^{32} es múltiplo de 2^{31} . Multiplicando por x_n y usando $x_{n+1} \equiv ax_n$ y $x_{n+2} \equiv a^2x_n$, obtenemos una relación lineal entre tres valores consecutivos:

$$x_{n+2} \equiv 6x_{n+1} - 9x_n \pmod{2^{31}} \implies 9u_n - 6u_{n+1} + u_{n+2} = k \in \mathbb{Z}. \quad (0.19)$$

DONDE

u_n	Salida normalizada $x_n/2^{31} \in [0, 1)$.
k	Entero; como $9u_n - 6u_{n+1} + u_{n+2}$ está entre -6 y 10 , solo puede tomar 15 valores $(-5, \dots, 9)$.

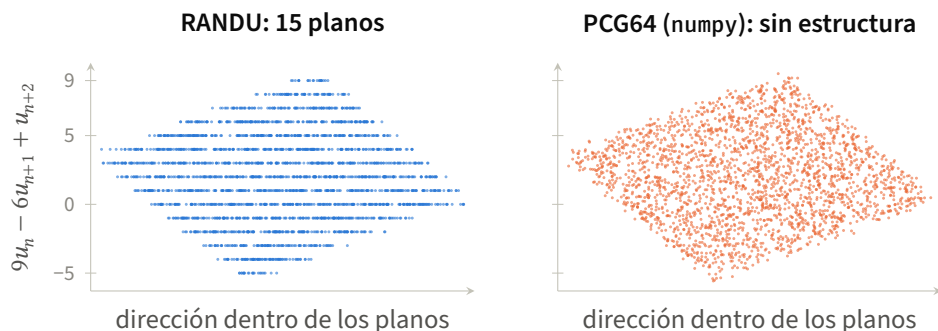


Figura 0.9. 2500 tripletes consecutivos (u_n, u_{n+1}, u_{n+2}) proyectados sobre dos direcciones: el eje vertical es la combinación $9u_n - 6u_{n+1} + u_{n+2}$ de la ecuación (0.19) y el horizontal, una dirección perpendicular a ella. Los tripletes de RANDU (izquierda) solo toman 15 valores enteros en el eje vertical: son 15 planos vistos de canto. Los de PCG64, el generador predeterminado de NumPy (derecha), llenan el espacio de forma homogénea.

La ecuación (0.19) dice que todos los tripletes (u_n, u_{n+1}, u_{n+2}) caen sobre 15 planos paralelos, con vector normal $(9, -6, 1)$ (figura 0.9). Una simulación tridimensional con RANDU explora solo 15 láminas del cubo y deja vacío todo lo demás. Marsaglia (1968) demostró que el defecto es inherente a todos los generadores congruenciales multiplicativos, aunque en grado muy distinto: los puntos de d valores consecutivos caen siempre sobre una familia de hiperplanos paralelos, cuyo número no puede superar $(d! m)^{1/d}$. Para $m = 2^{31}$ y $d = 3$ esa cota es de unos 2344 planos; RANDU se queda con 15.

0.3.4 Generadores modernos: Mersenne Twister y PCG

Los generadores actuales se diseñan con pruebas teóricas de uniformidad en muchas dimensiones y se someten a baterías de pruebas estadísticas. El Mersenne Twister (MT19937) de Matsumoto y Nishimura (1998), el generador del módulo random de Python y durante años el de NumPy, tiene un período de $2^{19937} - 1$ y está equidistribuido en 623 dimensiones: una garantía que RANDU no cumple ni en tres. Pero es lineal sobre el cuerpo de dos elementos: unas cuantas salidas consecutivas bastan para reconstruir su estado, así que sirve para simular, nunca para criptografía.

Desde la versión 1.17, NumPy usa por defecto PCG64, un generador de la familia de congruenciales permutados propuesta por Melissa O'Neill: un LCG de 128 bits cuyo estado no se entrega directamente, sino a través de una permutación que mezcla

sus bits (se combinan sus dos mitades con un o exclusivo y el resultado se rota un número de posiciones que dictan sus bits altos), lo que elimina las regularidades de los bits bajos que vimos en el ejemplo 0.5. Tiene período 2^{128} , un estado pequeño y puede “saltar” hacia adelante en la sucesión de forma eficiente, lo que permite crear flujos independientes para cálculos en paralelo (NumPy Developers, 2024).

La forma moderna de usarlo es crear un generador propio con una semilla explícita y pasarlo a las funciones que lo necesitan, en lugar de fijar el estado global con `np.random.seed`, que cualquier biblioteca importada puede alterar sin aviso.

semillas.py

```
import numpy as np

rng = np.random.default_rng(20240517)           # semilla explícita
reads = rng.choice(list("ACGT"), size=(3, 12))
print("".join(r) for r in reads)                # idéntico en cada ejecución

# flujos independientes para 4 procesos en paralelo
children = np.random.SeedSequence(20240517).spawn(4)
streams = [np.random.default_rng(s) for s in children]
```

La misma semilla solo reproduce la misma sucesión con el mismo generador y la misma versión del código; por eso se registra junto con las versiones del software.

0.3.5 Monte Carlo y su error

Una simulación Monte Carlo estima una cantidad (una probabilidad, una media) promediando muchas repeticiones aleatorias. Si $X_1, \dots, X_N$ son repeticiones independientes de una variable con media μ y desviación estándar σ , el estimador es la media muestral $\hat{\mu}_N$, y su precisión obedece a un resultado fundamental.

Teorema 0.3. (Error estándar de Monte Carlo)

El estimador $\hat{\mu}_N = \frac{1}{N} \sum_{i=1}^N X_i$ es insesgado, $E[\hat{\mu}_N] = \mu$, y su error estándar es

$$EE(\hat{\mu}_N) = \sqrt{\text{Var}(\hat{\mu}_N)} = \frac{\sigma}{\sqrt{N}}. \quad (0.20)$$

Por el teorema central del límite, $\hat{\mu}_N \pm 2\sigma/\sqrt{N}$ contiene a μ con probabilidad cercana al 95 %.

DONDE

X_i	Resultado de la i -ésima repetición (por ejemplo, 1 si una lectura simulada no tiene errores y 0 si los tiene).
μ, σ	Media y desviación estándar verdaderas de X_i para una variable de Bernoulli, $\sigma = \sqrt{p(1-p)}$.
N	Número de repeticiones de la simulación.

La demostración es directa: por independencia, $\text{Var}(\sum X_i) = N\sigma^2$, y dividir entre N divide la varianza entre N^2 . La consecuencia práctica es la ley de la raíz cuadrada: *para ganar un decimal de precisión hay que multiplicar el trabajo por cien*. Invirtiendo la ecuación (0.20), el número de repeticiones necesario para un error estándar ϵ es $N = \sigma^2/\epsilon^2$.

Ejemplo 0.6. (Lecturas sin errores)

Un secuenciador comete un error en cada base con probabilidad $e = 0,01$, de forma independiente. ¿Qué fracción de las lecturas de 150 bases no tiene ningún error? La respuesta exacta es $p = (1 - e)^{150} = 0,99^{150} = 0,2215$; se parece a $e^{-1,5} = 0,2231$, la aproximación de Poisson. Si no conociéramos la fórmula, podríamos simular $N = 10\,000$ lecturas y contar las perfectas. El error estándar sería $\sqrt{0,2215 \times 0,7785 / 10\,000} = 0,0042$, y las cuatro simulaciones de la figura 0.10 terminan en 0,2195, 0,2240, 0,2204 y 0,2223 tras 10^5 lecturas: todas dentro de la banda teórica. Para obtener la tercera cifra decimal con seguridad ($\epsilon = 0,0005$) harían falta unas $0,17/0,0005^2 \approx 690\,000$ lecturas simuladas.

0.3.6 Huellas digitales: funciones *hash*

Los datos también cambian sin avisar. Un archivo se trunca durante una descarga, una base de datos publica una nueva versión con el mismo nombre de archivo, un colaborador “limpia” una tabla y la guarda encima de la original. Para detectar cualquier cambio, por pequeño que sea, se usa la huella digital del archivo, el resultado de una función *hash* criptográfica.

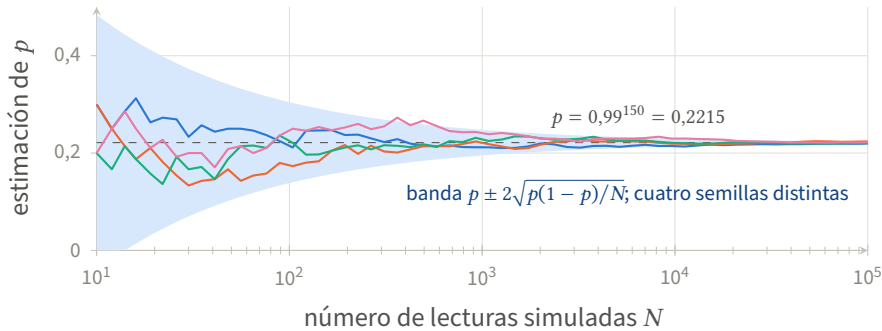


Figura 0.10. Estimación Monte Carlo de la fracción de lecturas de 150 bases sin errores de secuenciación ($e = 0,01$), con cuatro semillas distintas (líneas de color). Al principio las estimaciones difieren mucho; a medida que crece N convergen hacia el valor exacto (línea discontinua) y se mantienen dentro de la banda de ± 2 errores estándar, cuyo ancho se reduce como $1/\sqrt{N}$: en escala logarítmica, cada década de N estrecha la banda en un factor $\sqrt{10} \approx 3,2$.

Definición 0.6. (Función *hash* criptográfica)

Una función $h : \{0, 1\}^* \rightarrow \{0, 1\}^n$ que asigna a cualquier mensaje, de cualquier longitud, un resumen de n bits, y que satisface tres propiedades: (1) *resistencia a la preimagen*: dado un resumen y , es inviable encontrar un mensaje x con $h(x) = y$; (2) *resistencia a la segunda preimagen*: dado x , es inviable encontrar $x' \neq x$ con $h(x') = h(x)$; (3) *resistencia a colisiones*: es inviable encontrar cualquier par $x \neq x'$ con $h(x) = h(x')$.

SHA-256, definida en el estándar federal estadounidense de *hash* seguro (National Institute of Standards and Technology, 2015), produce resúmenes de $n = 256$ bits, que se escriben como 64 dígitos hexadecimales. Procesa el mensaje en bloques de 512 bits, cada uno mediante 64 rondas de sumas modulares, rotaciones y funciones lógicas, diseñadas para que cada bit de la salida dependa de todos los bits de la entrada. El resultado es el *efecto avalancha*: cambiar un solo bit de la entrada cambia, en promedio, la mitad de los bits de la salida, y el resumen modificado no guarda ninguna relación visible con el original.

Ejemplo 0.7. (Una base de diferencia)

Las secuencias **GATTACA** y **GATTACC** difieren en una sola base. Sus resúmenes SHA-256 son

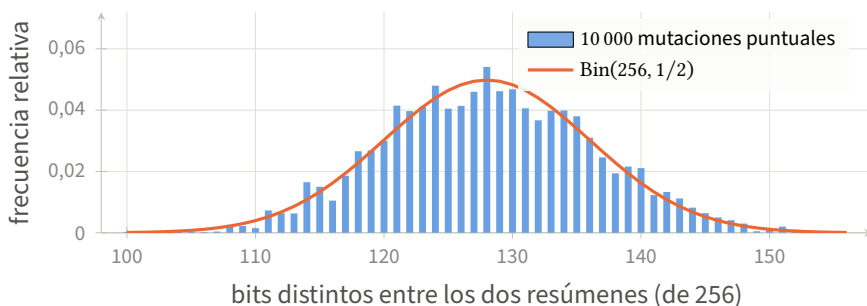


Figura 0.11. Efecto avalancha. Se tomó una secuencia aleatoria de 1000 bases y se le aplicaron 10 000 mutaciones puntuales independientes; para cada una se contaron los bits que difieren entre el SHA-256 de la secuencia original y el de la mutada (media observada 128,2; desviación estándar 8,2). La distribución coincide con la de 256 monedas justas (naranja): un cambio mínimo en los datos produce una huella completamente distinta, y nada en la huella delata dónde estuvo el cambio.

```
d74f6c423e80cbf69d76149048e458a10c96f927c896ea9ff4f44616b643eb22
60bcfb9d965bb9c40959b91781d457c8801eca61d70fee26377d612d205eecb7
```

y difieren en 129 de sus 256 bits. Si la salida fuera una sucesión de bits aleatorios e independientes, el número de bits distintos seguiría una binomial $\text{Bin}(256, \frac{1}{2})$, con media 128 y desviación estándar 8. La figura 0.11 muestra que eso es exactamente lo que ocurre.

¿Pueden dos archivos distintos tener la misma huella?

Sí, necesariamente: hay infinitos mensajes y solo 2^{256} resúmenes. La pregunta relevante es con qué probabilidad ocurrirá *por accidente*. Es el problema del cumpleaños: si k archivos reciben resúmenes que se comportan como uniformes e independientes sobre $M = 2^n$ valores, la probabilidad de que no haya ninguna coincidencia es

$$\mathbb{P}(\text{sin colisión}) = \prod_{i=1}^{k-1} \left(1 - \frac{i}{M}\right) \approx \exp\left(-\frac{k(k-1)}{2M}\right), \quad (0.21)$$

usando $1 - x \approx e^{-x}$ para x pequeño y $\sum_{i=1}^{k-1} i = k(k-1)/2$.

DONDE

k	Número de archivos a los que se calcula la huella.
$M = 2^n$	Número de resúmenes posibles; $n = 256$ para SHA-256.

La probabilidad de colisión se vuelve apreciable solo cuando k se acerca a $\sqrt{M} = 2^{n/2}$. Para SHA-256 eso significa $2^{128} \approx 3 \times 10^{38}$ archivos. Con un billón (10^{12}) de archivos, la probabilidad de alguna colisión accidental es del orden de $10^{24}/2^{257} \approx 4 \times 10^{-54}$. Para efectos prácticos, dos archivos con el mismo SHA-256 son el mismo archivo.

Verificar la integridad de los datos

```
# calcular y guardar las huellas de los datos crudos
sha256sum datos/*.fastq.gz > datos/SHA256SUMS
# meses después, u otra persona: ¿son los mismos archivos?
sha256sum -c datos/SHA256SUMS
# lecturas_R1.fastq.gz: OK
# lecturas_R2.fastq.gz: FAILED      <- el archivo cambió
```

0.3.7 Git: el historial como grafo

La tercera vía de pérdida de reproducibilidad es el código. Carpetas llenas de `analisis_final.py`, `analisis_final_v2.py` y `analisis_final_DEFINITIVO.py` son la versión computacional del cuaderno de laboratorio con páginas arrancadas. Un sistema de control de versiones registra cada estado del proyecto, quién lo produjo, cuándo y por qué, y permite volver a cualquiera de ellos. Git, el sistema dominante, tiene un modelo interno sorprendentemente sencillo, y entenderlo evita la mayoría de los sustos.

Definición 0.7. (Historial de Git)

Un *commit* es una instantánea completa de los archivos del proyecto, acompañada de metadatos (autor, fecha, mensaje) y de la referencia a uno o más commits *padres*. Cada commit se identifica por el *hash* de todo su contenido, incluidos los identificadores de sus padres. El historial es un *grafo acíclico dirigido* (DAG) $G = (V, E)$: los vértices V son commits y hay una arista $(c \rightarrow p) \in E$ si p es padre de c .

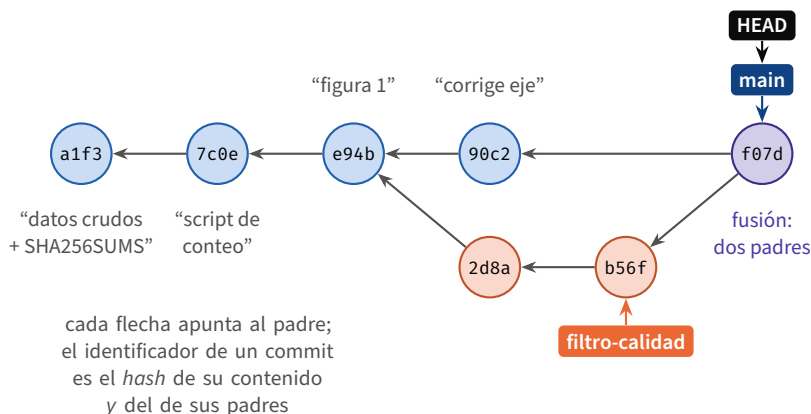


Figura 0.12. El historial de un proyecto como grafo acíclico dirigido. Cada nodo es un commit, identificado por los primeros caracteres de su *hash*; las flechas apuntan a los padres. La rama *filtro-calidad* (naranja) se separó en e94b para probar un nuevo filtro sin tocar la línea principal; al terminar, se fusionó con main en el commit f07d, que tiene dos padres. Cualquier resultado puede asociarse al identificador exacto del commit que lo produjo.

Que el identificador sea un *hash* del contenido tiene dos consecuencias profundas. La primera es la integridad: modificar un archivo antiguo cambia el *hash* de ese commit, que forma parte del contenido de su hijo, cuyo *hash* cambia también, y así hasta el último. Es imposible alterar el pasado en silencio: el identificador del commit más reciente certifica todo el historial, igual que una cadena de huellas digitales. La segunda es que el grafo es necesariamente acíclico: para que un commit fuera su propio antepasado, su *hash* tendría que aparecer dentro de su propio contenido, lo que exigiría resolver una ecuación que la resistencia a la preimagen hace inviable.

Sobre ese grafo, los conceptos de Git se vuelven simples. Una *rama* no es una copia del proyecto: es un puntero móvil, un nombre que apunta a un commit y avanza con cada commit nuevo. HEAD es el puntero que indica en qué rama estamos. Una *fusión* (*merge*) crea un commit con dos padres, que reúne dos líneas de trabajo (figura 0.12).

Perez-Riverol et al. (2016) resumen en diez reglas cómo aprovechar Git y la plataforma GitHub en la investigación, entre ellas usar ramas y bifurcaciones para desarrollar funciones nuevas sin romper lo que funciona, etiquetar las versiones con números semánticos, automatizar las pruebas, hacer el código citable y usar la plataforma para que la investigación sea más reproducible. Una regla práctica:

incluya el identificador del commit en cada figura o tabla generada, y guarde los datos grandes fuera de Git, referenciados por su huella.

0.3.8 Entornos, versiones y contenedores

La cuarta vía es el entorno. El mismo código puede producir resultados distintos con otra versión de NumPy, de Biopython o de BLAST, y un análisis que hoy funciona puede fallar mañana porque Colab actualizó una biblioteca. La defensa tiene varias capas, de menor a mayor aislamiento, y un análisis es tan reproducible como la capa más débil que haya quedado sin registrar.

La capa mínima es *registrar*: guardar junto con los resultados la lista exacta de versiones (`pip freeze`, `Bio.__version__`, `blastn -version`), por ejemplo en la primera celda de cada cuaderno. La siguiente es *fixar*: declarar esas versiones en un archivo (`requirements.txt` o `environment.yml`) a partir del cual cualquiera pueda recrear el entorno. Con versionado semántico (MAYOR.MENOR.PARCHE), fijar la versión mayor (`numpy>=1.26,<2`) evita rupturas, y fijar la exacta (`numpy==1.26.4`) garantiza además los mismos resultados numéricos.

La capa más robusta es el *contenedor*, una imagen que empaqueta el programa con todas las bibliotecas del sistema de las que depende y que se ejecuta igual en cualquier computadora con el motor adecuado. Grüning et al. (2018) describen cómo combinar estas capas en la práctica: gestores de paquetes como conda y su canal Bioconda para instalar software bioinformático con versiones exactas, contenedores generados automáticamente a partir de esos paquetes, y sistemas de gestión de flujos de trabajo que registran cada paso.

0.3.9 Reglas y principios: de Sandve a FAIR

Las herramientas de esta sección responden a un conjunto de principios que la comunidad ha ido formulando. Sandve et al. (2013) los condensaron en diez reglas simples para la investigación computacional reproducible, que la tabla 0.1 relaciona con los instrumentos de este capítulo. En paralelo, Wilson et al. (2014) reunieron las *mejores prácticas* de la computación científica (escribir programas para personas, automatizar lo repetitivo, hacer cambios pequeños y versionarlos, planear para los errores), y Wilson et al. (2017) propusieron un conjunto “suficientemente bueno” al alcance de cualquier grupo: guardar los datos crudos sin modificar, dar a cada proyecto una estructura predecible, versionar el código y documentar cómo se ejecuta.

Los principios FAIR amplían la mirada de los análisis a los *datos*. Wilkinson et al. (2016) propusieron que los datos científicos sean *localizables* (*Findable*: con

Tabla 0.1. Las diez reglas de Sandve et al. (2013) para la investigación computacional reproducible (traducción libre) y el instrumento del capítulo que ayuda a cumplirlas.

Regla	Instrumento
1 Registre cómo se produjo cada resultado.	cuaderno, commit
2 Evite los pasos manuales de manipulación de datos.	código, pandas
3 Archive las versiones exactas de los programas externos.	pip freeze, conda
4 Ponga bajo control de versiones todos sus <i>scripts</i> .	Git
5 Registre los resultados intermedios, en formatos estándar si es posible.	CSV, FASTA
6 En los análisis con aleatoriedad, anote las semillas.	default_rng
7 Guarde siempre los datos crudos detrás de cada gráfica.	.dat, .csv
8 Genere salidas jerárquicas, que permitan inspeccionar niveles de detalle.	tablas y figuras
9 Conecte las afirmaciones del texto con los resultados que las sustentan.	cuaderno
10 Dé acceso público a los <i>scripts</i> , las ejecuciones y los resultados.	GitHub

identificadores persistentes y metadatos ricos, registrados en recursos donde se puedan buscar), *accesibles* (*Accessible*: recuperables mediante protocolos abiertos y estandarizados, con metadatos que persisten aunque los datos dejen de estar disponibles), *interoperables* (*Interoperable*: descritos con vocabularios y formatos compartidos) y *reutilizables* (*Reusable*: con licencia clara, procedencia documentada y conformes a los estándares de su comunidad). Una idea central del artículo es que los datos deben ser legibles no solo por personas sino también por máquinas. Un número de acceso como NC_045512.2, con su versión explícita, es un buen ejemplo: el sufijo de versión .2 garantiza que cualquier cambio futuro recibirá un identificador nuevo.

Semilla, huella, commit y entorno:
cuatro registros baratos que convierten una receta en un protocolo.

☰ En resumen

- Un *notebook* separa el documento del núcleo que lo ejecuta; el estado del núcleo depende del orden de ejecución, y solo “reiniciar y ejecutar todo” garantiza que el documento y sus resultados coinciden.
- Una secuencia es un arreglo de símbolos; su costo en memoria depende de la codificación ($nb/8$ bytes) y el costo de un algoritmo se resume con la notación O . La vectorización con NumPy no cambia la complejidad, pero reduce la constante en dos órdenes de magnitud.
- El punto flotante tiene error relativo $\approx 10^{-16}$ y se desborda por abajo al multiplicar probabilidades: se trabaja con logaritmos y log-suma-exp.
- Los datos ordenados y las figuras diseñadas según la percepción forman parte del método, no de la estética.
- Biopython representa secuencias (Seq), registros (SeqRecord) y archivos (SeqIO); Entrez descarga datos del NCBI; las herramientas de línea de comandos se combinan con tuberías y se llaman desde Python con subprocess.
- Un generador pseudoaleatorio es una máquina determinista cuya salida fija la semilla; RANDU muestra que un mal generador introduce estructura (15 planos), y los modernos (Mersenne Twister, PCG64) se diseñan para evitarla. El error Monte Carlo decrece como $\sigma/\sqrt{N}$.
- SHA-256 da a cada archivo una huella de 256 bits con efecto avalancha y colisiones accidentales prácticamente imposibles; Git encadena huellas en un grafo acíclico dirigido de commits.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 0.1** ●●● Un gen está anotado en las posiciones 21 563..25 384 de un genoma (numeración de GenBank). Escriba el corte de Python que lo extrae y calcule su longitud en nucleótidos y en codones. ¿Qué error se comete si se escribe `seq[21563:25384]`?
- 0.2** ●●● Calcule cuánta memoria ocupa el genoma de *Escherichia coli* ($4,6 \times 10^6$ pb) en cada una de las representaciones descritas tras la ecuación (0.2). ¿Cuál elegiría para calcular un *dot plot* contra otro genoma del mismo tamaño?

- 0.3** ●●● Implemente `logsumexp(a)` a partir de la ecuación (0.7) y compárela con `scipy.special.logsumexp` para $a = [-1000, -1001, -1002]$ y para $a = [1000, 1001]$. ¿Por qué falla la evaluación directa en cada caso?
- 0.4** ●●● Repita el ejemplo 0.1 con ventanas de 100, 500 y 2000 bases. ¿Cómo cambia la fracción de ventanas fuera de la banda binomial? Interprete el resultado en términos de ruido de muestreo y de heterogeneidad biológica.
- 0.5** ●●● Demuestre la aproximación de la ecuación (0.21) y úsela para calcular cuántos archivos harían falta para tener un 50 % de probabilidad de colisión con MD5 (128 bits) y con SHA-256. Compare con el número estimado de archivos almacenados en el mundo (del orden de 10^{15}).
- 0.6** ●●● Escriba una función que reciba una semilla, simule N lecturas de 150 bases con una tasa de error por base e y devuelva la estimación de la fracción de lecturas sin errores con su error estándar. Determine empíricamente el N necesario para que el intervalo de ± 2 errores estándar tenga un ancho menor que 0,001 y compárelo con la predicción de la ecuación (0.20).
- 0.7** ●●● Tome un análisis propio (o el cuaderno de la sección 0.2) y evalúelo frente a las diez reglas de la tabla 0.1. Para cada regla incumplida, proponga el cambio concreto (un archivo, una línea de código, un comando) que la haría cumplir.

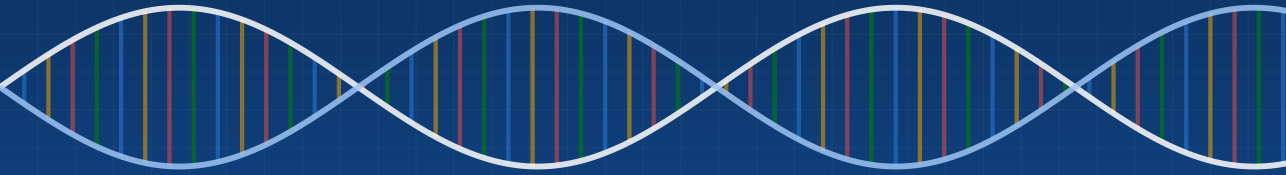
Referencias del capítulo

- Baker, M. (2016). 1,500 scientists lift the lid on reproducibility. *Nature*, 533(7604), 452-454. <https://doi.org/10.1038/533452a>
- Buckheit, J. B., & Donoho, D. L. (1995). WaveLab and Reproducible Research. En *Wavelets and Statistics* (pp. 55-81). Springer New York. https://doi.org/10.1007/978-1-4612-2544-7_5
- Camacho, C., Coulouris, G., Avagyan, V., Ma, N., Papadopoulos, J., Bealer, K., & Madden, T. L. (2009). BLAST+: architecture and applications. *BMC Bioinformatics*, 10(1), Artículo 421. <https://doi.org/10.1186/1471-2105-10-421>
- Cleveland, W. S., & McGill, R. (1984). Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods. *Journal of the American Statistical Association*, 79(387), 531-554. <https://doi.org/10.1080/01621459.1984.10478080>

- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Cramer, F., Shephard, G. E., & Heron, P. J. (2020). The misuse of colour in science communication. *Nature Communications*, 11(1), Artículo 5444. <https://doi.org/10.1038/s41467-020-19160-7>
- Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1), 5-48. <https://doi.org/10.1145/103162.103163>
- Grüning, B., Chilton, J., Köster, J., Dale, R., Soranzo, N., van den Beek, M., Goecks, J., Backofen, R., Nekrutenko, A., & Taylor, J. (2018). Practical Computational Reproducibility in the Life Sciences. *Cell Systems*, 6(6), 631-635. <https://doi.org/10.1016/j.cels.2018.03.014>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
- Ioannidis, J. P. A., Allison, D. B., Ball, C. A., Coulibaly, I., Cui, X., Culhane, A. C., Falchi, M., Furlanello, C., Game, L., Jurman, G., Mangion, J., Mehta, T., Nitzberg, M., Page, G. P., Petretto, E., & van Noort, V. (2009). Repeatability of published microarray gene expression analyses. *Nature Genetics*, 41(2), 149-155. <https://doi.org/10.1038/ng.295>
- Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & Jupyter Development Team. (2016). Jupyter Notebooks – a publishing format for reproducible computational workflows. En *Positioning and Power in Academic Publishing: Players, Agents and Agendas* (pp. 87-90). IOS Press. <https://doi.org/10.3233/978-1-61499-649-1-87>
- Knuth, D. E. (1997). *The Art of Computer Programming* (3.^a ed., Vol. 2). Addison-Wesley.
- Marsaglia, G. (1968). Random numbers fall mainly in the planes. *Proceedings of the National Academy of Sciences*, 61(1), 25-28. <https://doi.org/10.1073/pnas.61.1.25>
- Matsumoto, M., & Nishimura, T. (1998). Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1), 3-30. <https://doi.org/10.1145/272991.272995>
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- National Institute of Standards and Technology. (2015). *Secure Hash Standard* (Federal Information Processing Standards Publication N.º 180-4). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.FIPS.180-4>

- NumPy Developers. (2024). *Permuted Congruential Generator (64-bit, PCG64)*. NumPy documentation. https://numpy.org/doc/stable/reference/random/bit_generators/pcg64.html
- Perez-Riverol, Y., Gatto, L., Wang, R., Sachsenberg, T., Uszkoreit, J., Leprevost, F. d. V., Fufezan, C., Ternent, T., Eglen, S. J., Katz, D. S., Pollard, T. J., Kononov, A., Flight, R. M., Blin, K., & Vizcaino, J. A. (2016). Ten Simple Rules for Taking Advantage of Git and GitHub. *PLOS Computational Biology*, 12(7), Artículo e1004947. <https://doi.org/10.1371/journal.pcbi.1004947>
- Ritchie, D. M., & Thompson, K. (1974). The UNIX time-sharing system. *Communications of the ACM*, 17(7), 365-375. <https://doi.org/10.1145/361011.361061>
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten Simple Rules for Reproducible Computational Research. *PLoS Computational Biology*, 9(10), Artículo e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>
- Sayers, E. W., Bolton, E. E., Brister, J. R., Canese, K., Chan, J., Comeau, D. C., Connor, R., Funk, K., Kelly, C., Kim, S., Madej, T., Marchler-Bauer, A., Lanczycki, C., Lathrop, S., Lu, Z., Thibaud-Nissen, F., Murphy, T., Phan, L., Skripchenko, Y., ... Sherry, S. T. (2022). Database resources of the National Center for Biotechnology Information. *Nucleic Acids Research*, 50(D1), D20-D26. <https://doi.org/10.1093/nar/gkab1112>
- Tufte, E. R. (2001). *The Visual Display of Quantitative Information* (2.^a ed.). Graphics Press.
- Wickham, H. (2014). Tidy Data. *Journal of Statistical Software*, 59(10), 1-23. <https://doi.org/10.18637/jss.v059.i10>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, Artículo 160018. <https://doi.org/10.1038/sdata.2016.18>
- Wilson, G., Aruliah, D. A., Brown, C. T., Chue Hong, N. P., Davis, M., Guy, R. T., Haddock, S. H. D., Huff, K. D., Mitchell, I. M., Plumbley, M. D., Waugh, B., White, E. P., & Wilson, P. (2014). Best Practices for Scientific Computing. *PLoS Biology*, 12(1), Artículo e1001745. <https://doi.org/10.1371/journal.pbio.1001745>
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), Artículo e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>
- Wu, F., Zhao, S., Yu, B., Chen, Y.-M., Wang, W., Song, Z.-G., Hu, Y., Tao, Z.-W., Tian, J.-H., Pei, Y.-Y., Yuan, M.-L., Zhang, Y.-L., Dai, F.-H., Liu, Y., Wang, Q.-M., Zheng, J.-J., Xu, L., Holmes, E. C., & Zhang, Y.-Z. (2020). A new coronavirus associated with human respiratory disease in China. *Nature*, 579(7798), 265-269. <https://doi.org/10.1038/s41586-020-2008-3>
- Ziemann, M., Eren, Y., & El-Osta, A. (2016). Gene name errors are widespread in the scientific literature. *Genome Biology*, 17(1), Artículo 177. <https://doi.org/10.1186/s13059-016-1044-7>

Biología molecular para bioinformáticos



No ha escapado a nuestra atención que el apareamiento específico que hemos postulado sugiere de inmediato un posible mecanismo de copia del material genético.

— Watson y Crick (1953)

● Al terminar este capítulo usted podrá

- Describir el dogma central como un sistema que almacena, copia y ejecuta información, y distinguir las transferencias generales, especiales y prohibidas de Crick (1970).
- Medir la información de una secuencia con la entropía de Shannon, relacionarla con la compresión y explicar por qué el ADN real está cerca de 2 bits por base.
- Cuantificar la robustez del código genético frente a mutaciones puntuales y compararlo con códigos alternativos generados al azar.
- Implementar un buscador de marcos abiertos de lectura (ORFs) en los seis marcos, construir su modelo nulo geométrico y evaluarlo contra la anotación real de *Mycoplasma genitalium*.

- Calcular el contenido GC, el sesgo GC (*GC skew*) y su versión acumulada, y usarlos para localizar el origen de replicación de *Escherichia coli* a partir de la secuencia sola.

Un bioinformático no necesita saber pipetear, pero sí entender qué representa cada letra que procesa. Una secuencia de ADN no es una cadena arbitraria: procede de miles de millones de años de copias imperfectas, filtradas por la selección natural y condicionadas por la química de las bases. Una región pobre en codones de parada, un exceso de guaninas en una mitad del cromosoma o un dinucleótido infrecuente son pistas estadísticas de procesos moleculares. En este capítulo iremos de las moléculas a esas pistas y volveremos de las pistas a las moléculas.

Seguiremos tres pasos. Primero examinaremos la célula como sistema de información: cómo almacena, copia y traduce un mensaje, cuánta información aporta cada base y cómo el código genético tolera ciertos errores (sección 1.1). Después buscaremos genes en el pequeño genoma de una bacteria y separaremos la señal biológica del ruido mediante un modelo nulo explícito (sección 1.2). Por último, contaremos bases a lo largo de un cromosoma completo. La replicación deja allí una señal que permite localizar el origen de *E. coli* con un error de unos pocos cientos de bases entre 4,6 millones (sección 1.3).

1.1 El dogma central como sistema de información

 Colab · 1.1 · El dogma central

 abrir notebook

La bóveda y la fotocopia

Piense en el archivo de planos de una gran fábrica. Los originales se guardan en una bóveda; nunca salen de allí y solo se copian, con muchísimo cuidado, cuando la fábrica abre una sucursal. Cuando un taller necesita construir una pieza, no se lleva el original: pide una fotocopia de la página que le interesa, la lleva al taller y allí una máquina lee la fotocopia y ensambla la pieza. La fotocopia se tira a las pocas horas; si tenía una mancha, se estropean unas cuantas piezas, pero el original sigue intacto. Esa división del trabajo (un archivo permanente, copias de trabajo desechables y máquinas que ejecutan) es exactamente la que usa la célula.

1.1.1 La doble hélice: un medio de almacenamiento

El ADN es un polímero de cuatro nucleótidos, que se distinguen por su base nitrogenada: adenina (A), citosina (C), guanina (G) y timina (T). Cada nucleótido se une al siguiente mediante un enlace fosfodiéster entre el carbono 3' de un azúcar y el carbono 5' del siguiente, de modo que cada hebra tiene una dirección química: un extremo 5' y un extremo 3'. Por convención, las secuencias se escriben siempre de 5' a 3'.

Watson y Crick (1953) propusieron que dos hebras se enrollan en una doble hélice *antiparalela* (una corre 5'→3' y la otra 3'→5') unidas por puentes de hidrógeno entre pares de bases complementarias: A con T y G con C. La frase que abre este capítulo resume la consecuencia: si cada hebra determina por completo a la otra, cualquiera de las dos sirve de molde para reconstruir la molécula entera. El ADN no es solo un almacén; es un almacén *autocopiabile*.

Para representarla computacionalmente bastan una cadena de cuatro letras y una regla determinista que permite reconstruir la hebra complementaria.

Definición 1.1. (Secuencia y complemento reverso)

Una secuencia de ADN es una palabra $x = x_1x_2 \cdots x_n$ sobre el alfabeto $\mathcal{A} = \{A, C, G, T\}$, leída de 5' a 3'. Su *complemento reverso* es

$$\bar{x} = \kappa(x_n)\kappa(x_{n-1}) \cdots \kappa(x_1), \quad \kappa : A \leftrightarrow T, C \leftrightarrow G. \quad (1.1)$$

La hebra complementaria de x , leída también de 5' a 3', es exactamente $\bar{x}$.

DONDE

x_i	Base en la posición i de la hebra, contando desde el extremo 5'.
n	Longitud de la secuencia, en nucleótidos (nt) o pares de bases (pb).
κ	Función de complementariedad de Watson-Crick, que es su propia inversa: $\kappa(\kappa(b)) = b$.
$\bar{x}$	Complemento reverso: la otra hebra, escrita en la dirección convencional.

Usaremos dos propiedades de la ecuación (1.1) a lo largo del capítulo. La primera es $\bar{\bar{x}} = x$: aplicar dos veces el complemento reverso devuelve la secuencia original. La segunda es que la complementariedad *intercambia* los conteos: cada G de una

hebra corresponde a una **C** de la otra. Por esa simetría, el sesgo GC de la sección 1.3 cambia de signo al leer la hebra opuesta.

1.1.2 Tres procesos, dos alfabetos: el dogma central

La información guardada en el ADN fluye por tres procesos principales:

- **Replicación** (ADN→ADN): la ADN polimerasa copia cada hebra por complementariedad antes de la división celular.
- **Transcripción** (ADN→ARN): la ARN polimerasa lee una hebra *molde* de 3' a 5' y sintetiza un ARN de 5' a 3'. El ARN resultante es idéntico a la otra hebra, la *codificante*, salvo que lleva uracilo (**U**) en lugar de timina.
- **Traducción** (ARN→proteína): el ribosoma lee el ARN mensajero (ARNm) de tres en tres nucleótidos (*codones*) y, con la ayuda de los ARN de transferencia, une el aminoácido que corresponde a cada codón.

Francis Crick bautizó este esquema como *dogma central* en 1958 y lo reformuló con precisión en 1970. En su versión madura, el dogma no afirma que la información fluya siempre de ADN a ARN y de ARN a proteína; afirma algo más sutil: que la información *secuencial*, residuo a residuo, no puede transferirse desde una proteína a otra proteína ni a un ácido nucleico (Crick, 1970). Crick clasificó las nueve transferencias posibles entre los tres tipos de polímero en tres grupos (figura 1.1): *generales*, que ocurren en todas las células; *especiales*, que ocurren solo en circunstancias particulares, como en ciertos virus; y *desconocidas*, que el dogma postula que no ocurren.

La palabra *dogma* ha causado confusión: Crick la empleó para una hipótesis general todavía sin prueba directa, no para una creencia incuestionable. Por eso, cuando Temin y Mizutani (1970) y, de forma independiente, David Baltimore descubrieron en 1970 la transcriptasa reversa, que sintetiza ADN a partir de ARN, no refutaron el dogma: Crick (1970) ya contemplaba ARN→ADN como transferencia especial. Lo que el dogma prohíbe es la salida de información secuencial desde las proteínas, y eso sigue sin observarse.

¿Para qué sirve la copia intermedia? Permite exigir distinta fidelidad al archivo y al mensaje de trabajo. Tras la corrección de pruebas y la reparación de apareamientos erróneos, la replicación del ADN deja del orden de un error por cada 10^9 nucleótidos copiados (Alberts et al., 2022): un error allí puede heredarse. El ARN se sintetiza con aproximadamente un error por cada 10^4 nucleótidos, se produce en muchas copias y se degrada en minutos. Un ARNm defectuoso afecta a unas cuantas proteínas; un

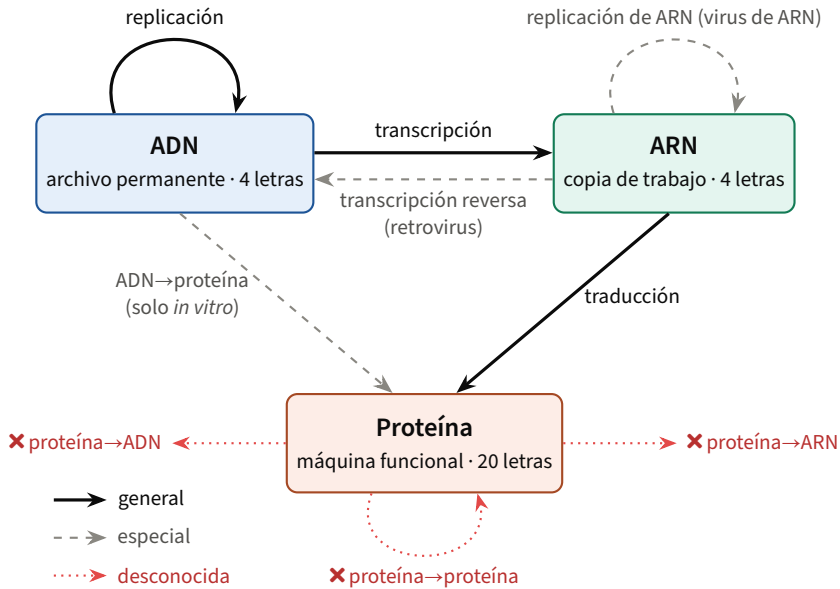


Figura 1.1. El dogma central en la formulación de Crick (1970). Las flechas continuas son las transferencias generales de información secuencial, presentes en toda célula; las discontinuas, las transferencias especiales (la transcripción reversa y la replicación de ARN ocurren en virus; la traducción directa de ADN solo se ha logrado en sistemas acelulares). Las flechas punteadas rojas son las transferencias que el dogma declara inexistentes: una vez que la información llega a una proteína, no puede volver a salir como secuencia. Observe la asimetría de alfabetos: dos cajas usan 4 letras y una usa 20.

cambio en el ADN puede pasar a los descendientes. En términos de ingeniería, la célula invierte la mayor fidelidad en el soporte permanente.

1.1.3 Transcripción y traducción, paso a paso

Tomemos un ejemplo real: los primeros 36 nucleótidos del gen S del coronavirus SARS-CoV-2, que codifica la proteína de la espiga (*Spike*). La figura 1.2 muestra el flujo completo. La hebra codificante y la hebra molde se aparean base a base; la ARN polimerasa lee la molde y produce un ARNm que reproduce la codificante con **U** en lugar de **T**. El ribosoma lee después el ARNm en codones consecutivos, sin solapamiento y sin separadores, y cada codón aporta un aminoácido.

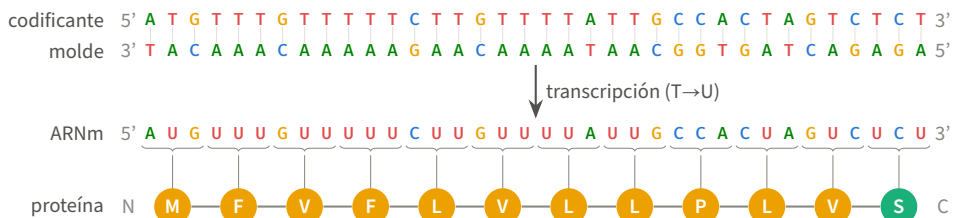


Figura 1.2. Transcripción y traducción de los primeros 36 nucleótidos del gen S de SARS-CoV-2 (NC_045512.2). La hebra molde, que la ARN polimerasa lee de 3' a 5', es la complementaria de la codificante; el ARNm reproduce la codificante con **U** en lugar de **T**. Cada llave agrupa un codón, y el círculo inferior es el aminoácido correspondiente, coloreado por clase fisicoquímica (amarillo: hidrofóbico; azul: polar o especial; rojo: básico; verde: ácido). El tramo MFVFLVLLPLVS, casi todo hidrofóbico, es el péptido señal que dirige la proteína a la membrana. Figura generada con `figuras/cap01/generar.py`.

Ambos procesos se programan en pocas líneas. Escribirlas ayuda a fijar dos convenciones que causan errores frecuentes en los análisis: qué hebra se lee y en qué dirección.

```
dogma.py

BASES = "TCAG"
AA = ("FFLLSSSSYY**CC*WLLLLPPPHHHQRRRR"
      "IIIMTTTNNKKSSRRVVVVAAADDEEGGG") # orden clásico TCAG
CODIGO = {a + b + c: AA[16 * i + 4 * j + k]
          for i, a in enumerate(BASES) for j, b in enumerate(BASES)
          for k, c in enumerate(BASES)}
COMP = str.maketrans("ACGT", "TGCA")

def complemento_reverso(x: str) -> str:
    return x.translate(COMP)[::-1] # ecuación (1.1)

def transcribir(codificante: str) -> str:
    return codificante.replace("T", "U")

def traducir(adn: str, hasta_parada: bool = True) -> str:
    prot = []
    for i in range(0, len(adn) - 2, 3):
        aa = CODIGO[adn[i:i + 3]]
        if aa == "*" and hasta_parada:
            break
    prot.append(aa)
```



```
return "".join(prot)
```

Aplicado al gen *S* completo (3822 nt, incluida la parada), traducir produce $3822/3 - 1 = 1273$ aminoácidos que coinciden letra a letra con la traducción depositada en el GenBank. Traducir por error la hebra molde, o el complemento sin invertir, da una secuencia sin relación alguna, casi siempre interrumpida por paradas a las pocas decenas de residuos.

1.1.4 ¿Cuánta información cabe en una base?

Si el ADN es un medio de almacenamiento, podemos preguntarnos por su capacidad, igual que con un disco duro. La herramienta para responder la creó Shannon (1948) para las líneas telefónicas, y se aplica sin cambios a las secuencias biológicas.

Empecemos por un juego. Alguien elige una base al azar y usted debe adivinarla con preguntas de sí o no. La estrategia óptima es partir el conjunto por la mitad: “¿es una purina (A o G)?”, y después “¿es A?”. Siempre bastan dos preguntas, porque $\log_2 4 = 2$. Esa es la capacidad máxima de una posición: *2 bits por base*. Un genoma humano haploide de unos $3,1 \times 10^9$ pares de bases cabe, por lo tanto, en $3,1 \times 10^9 \times 2/8 \approx 775$ megabytes, algo menos que un CD.

Pero la capacidad es solo un techo. Si las bases no fueran equiprobables (por ejemplo, si el 70 % fueran A), usted podría empezar preguntando “¿es A?” y acertar a la primera la mayoría de las veces. El número *medio* de preguntas bajaría, y con él la información que realmente aporta cada base. Shannon definió esa cantidad media como la entropía.

Definición 1.2. (Entropía de Shannon)

Sea X una variable aleatoria que toma valores en un alfabeto finito $\mathcal{A}$ con probabilidades $p_a = P(X = a)$. Su entropía, en bits, es

$$H(X) = - \sum_{a \in \mathcal{A}} p_a \log_2 p_a. \quad (1.2)$$

con el convenio $0 \log_2 0 = 0$. La cantidad $-\log_2 p_a$ se llama *sorpresa* (o autoinformación) del símbolo a , y H es la sorpresa promedio.

DONDE

$H(X)$	Entropía: información media, en bits, que aporta observar un símbolo.
$\mathcal{A}$	Alfabeto: {A, C, G, T} para ADN, los 20 aminoácidos para proteínas.
p_a	Probabilidad (en la práctica, frecuencia observada) del símbolo a .
$-\log_2 p_a$	Sorpresas de a : un símbolo raro sorprende mucho; uno casi seguro, casi nada.

La definición no es arbitraria. Shannon demostró que H es, salvo una constante multiplicativa, la *única* función que es continua en las probabilidades, crece con el número de alternativas equiprobables y se descompone de forma aditiva cuando una elección se hace en dos etapas, como en nuestro juego de preguntas (Shannon, 1948). Sus cotas fundamentales se derivan en pocas líneas.

Teorema 1.1. (*Cotas de la entropía*)

Para cualquier distribución sobre un alfabeto de K símbolos,

$$0 \leq H(X) \leq \log_2 K, \quad (1.3)$$

con $H = 0$ si y solo si un símbolo tiene probabilidad 1, y $H = \log_2 K$ si y solo si todos son equiprobables.

La cota inferior es inmediata: cada término $-p_a \log_2 p_a$ es no negativo porque $0 \leq p_a \leq 1$. Para la superior usamos la desigualdad $\ln y \leq y - 1$, válida para todo $y > 0$ con igualdad solo en $y = 1$. Tomando $y = 1/(K p_a)$,

$$H(X) - \log_2 K = \sum_a p_a \log_2 \frac{1}{K p_a} \leq \frac{1}{\ln 2} \sum_a p_a \left(\frac{1}{K p_a} - 1 \right) = \frac{1}{\ln 2} \left(\sum_a \frac{1}{K} - 1 \right) = 0,$$

y la igualdad exige $K p_a = 1$ para todo a . Para el ADN, $K = 4$ y el techo es 2 bits; para las proteínas, $K = 20$ y el techo es $\log_2 20 \approx 4,32$ bits.

Ejemplo 1.1. (La entropía de SARS-CoV-2)

El genoma de SARS-CoV-2 tiene 29 903 nucleótidos con composición $p_A = 0,2994$, $p_C = 0,1837$, $p_G = 0,1961$ y $p_T = 0,3208$: es rico en **A** y **T**. Aplicando la ecuación (1.2),

$$\begin{aligned} H &= -(0,2994 \log_2 0,2994 + 0,1837 \log_2 0,1837 \\ &\quad + 0,1961 \log_2 0,1961 + 0,3208 \log_2 0,3208) \\ &= 0,5210 + 0,4491 + 0,4608 + 0,5262 = 1,957 \text{ bits por base.} \end{aligned}$$

El sesgo de composición le resta apenas 0,043 bits por base al máximo teórico. Para comparar, una secuencia en la que una base aparece el 70 % de las veces y las otras tres el 10 % tendría $H = 1,357$ bits, y la repetición **AAAAAAAA** tendría $H = 0$.

La entropía calculada base por base ignora el orden. En la repetición **CAGCAGCAG**, asociada a la enfermedad de Huntington cuando se expande, las tres letras son equiprobables y dan $H = \log_2 3 \approx 1,585$ bits por base. Sin embargo, una vez conocida una letra, la siguiente es predecible. Podemos recoger esa estructura calculando la entropía de *palabras* de longitud k (k -mers) y dividiéndola entre k :

$$h_k = \frac{H_k}{k}, \quad H_k = - \sum_{w \in \mathcal{A}^k} p_w \log_2 p_w, \quad h = \lim_{k \rightarrow \infty} h_k. \quad (1.4)$$

donde w recorre las 4^k palabras posibles, p_w es su frecuencia entre todas las ventanas de longitud k , H_k es la *entropía de bloque* y su límite h es la *tasa de entropía*.

Como las dependencias entre vecinos solo pueden reducir la incertidumbre, h_k es no creciente en k y converge a la tasa de entropía h , que mide la información genuinamente nueva que aporta cada base. En la práctica, k no puede crecer mucho: con 4^k palabras posibles y una secuencia finita, la mayoría de las palabras largas no aparecen nunca y la estimación se sesga hacia abajo. Para un genoma viral de 30 kb, $k \leq 3$ es razonable; para uno bacteriano de 5 Mb, $k \leq 8$.

La figura 1.3b aplica estas ideas a tres secuencias de la misma longitud. La secuencia aleatoria tiene 2 bits por base con cualquier k . El genoma de SARS-CoV-2 queda muy cerca ($h_1 = 1,957$, $h_3 = 1,933$): su composición está sesgada y hay algo de dependencia entre vecinos, pero a simple vista el genoma es casi indistinguible del azar. La repetición, en cambio, se desploma de 1,585 a 0,528 bits cuando se tienen en cuenta los vecinos.

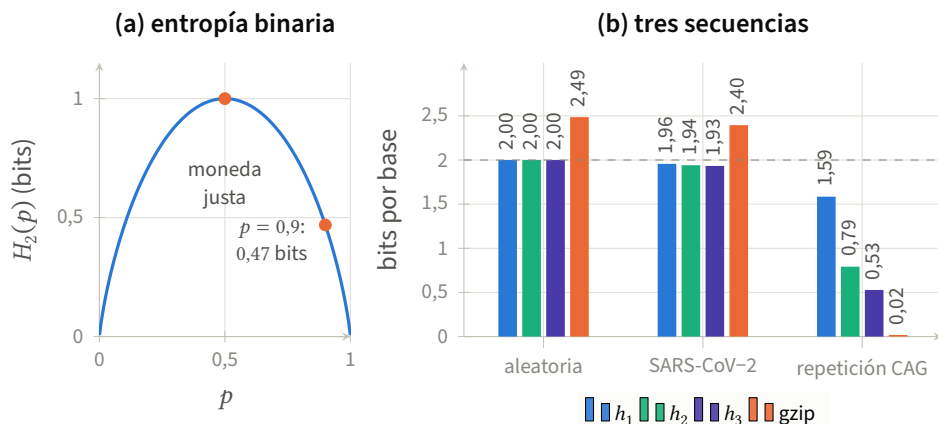


Figura 1.3. Medir la información de una secuencia. (a) Entropía de un alfabeto de dos símbolos en función de la probabilidad p de uno de ellos: es máxima (1 bit) cuando ambos son equiprobables y cae rápidamente cuando uno domina. (b) Entropía por base con palabras de $k = 1, 2, 3$ letras (ecuación (1.4)) y bits por base tras comprimir con **gzip** (nivel 9) para tres secuencias de 29 903 nt: una aleatoria, el genoma de SARS-CoV-2 y una repetición CAG. La línea discontinua marca el techo de 2 bits. El genoma real se comporta casi como una secuencia aleatoria; la repetición tiene 1,59 bits con $k = 1$ pero solo 0,53 con $k = 3$. El compresor genérico ni siquiera alcanza los 2 bits en las dos primeras secuencias.

⚠ Entropía no es significado

Que un genoma tenga casi 2 bits por base no quiere decir que “contenga mucha información biológica”, ni que una región de baja entropía sea inútil. La entropía de Shannon mide la imprevisibilidad *estadística* de las letras, no lo que hacen. Un texto en español y el mismo texto con las letras barajadas tienen idéntica h_1 . En bioinformática, la baja entropía es sobre todo una *alarma*: las regiones de baja complejidad (colas poli-A, microsatélites) producen alineamientos espurios y se enmascaran antes de buscar homólogos, como vimos con BLAST en el capítulo 3.

1.1.5 Compresión: la entropía medida a la fuerza

La entropía tiene una interpretación operativa que la vuelve muy concreta: es el límite de la compresión sin pérdida. El *teorema de codificación de la fuente* de Shannon (1948) establece que ningún código binario decodificable sin ambigüedad

puede usar, en promedio, menos de H bits por símbolo, $\bar{\ell} = \sum_a p_a \ell_a \geq H$ (donde ℓ_a es la longitud de la palabra de código del símbolo a), y que codificando bloques cada vez más largos se puede llegar tan cerca de H como se quiera.

Para el ADN, el código trivial $A \mapsto 00$, $C \mapsto 01$, $G \mapsto 10$, $T \mapsto 11$ usa 2 bits por base y es óptimo si las bases son equiprobables. Un archivo FASTA, en cambio, gasta un byte (8 bits) por base: cuatro veces más de lo necesario. La figura 1.3b muestra que **gzip**, un compresor genérico que busca repeticiones pero no “sabe” que solo hay cuatro letras, se queda en 2,4–2,5 bits por base con la secuencia aleatoria y con el genoma viral (peor que el código trivial), mientras reduce la repetición CAG a 0,02 bits por base. Por eso existen formatos especializados como CRAM.

1.1.6 El código genético: un diccionario de 64 entradas

Tres letras de un alfabeto de cuatro producen $4^3 = 64$ codones, que deben codificar 20 aminoácidos y una señal de parada. En términos de información, un codón podría transportar $\log_2 64 = 6$ bits, pero el mensaje que transporta (uno de 21 significados) necesita como máximo $\log_2 21 \approx 4,39$ bits. El código es, por lo tanto, *redundante*: la mayoría de los aminoácidos tienen varios codones sinónimos. Esa redundancia, lejos de ser un desperdicio, resultará ser la clave de su robustez.

🔍 Descifrar el código en cinco años

En 1961, Marshall Nirenberg y Heinrich Matthaei añadieron a un extracto de *E. coli* sin células un ARN sintético formado solo por uracilos (poli-U) y obtuvieron un polipéptido formado solo por fenilalanina: UUU codificaba Phe (Nirenberg & Matthaei, 1961). Ese mismo año, Crick et al. (1961) demostraron con mutantes de desplazamiento de marco en el bacteriófago T4 que el código se lee en tripletes, sin solapamiento y desde un punto de inicio fijo. Nirenberg y Leder (1964) idearon después un ensayo de unión de trinucleótidos al ribosoma que permitió asignar codones uno por uno, y el grupo de Har Gobind Khorana confirmó las asignaciones con ARN sintéticos de secuencia repetitiva. Hacia 1966 la tabla estaba completa. Nirenberg, Khorana y Robert Holley compartieron el Nobel de 1968.

La figura 1.4 presenta el código estándar como una rueda que se lee desde el centro: primera base en el anillo interno, segunda en el intermedio y tercera en el externo. La disposición revela de inmediato un patrón que la tabla rectangular oculta: los codones sinónimos se agrupan en sectores contiguos que casi siempre comparten las dos primeras bases y difieren solo en la tercera. Crick (1966) explicó el patrón con la hipótesis del *bamboleo* (*wobble*): el anticodón del ARN de transferencia

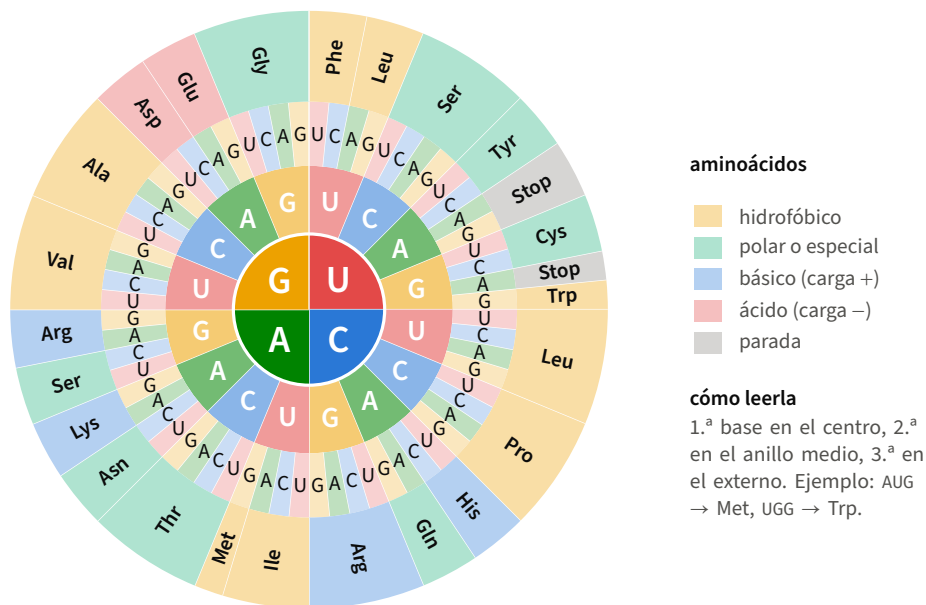


Figura 1.4. El código genético estándar como rueda. Cada sector externo agrupa los codones que comparten las dos primeras bases y codifican el mismo aminoácido. Tres regularidades saltan a la vista: la tercera base casi nunca cambia el aminoácido dentro de un bloque de cuatro; los codones con **U** en la segunda posición (cuadrante superior de cada primera base) codifican casi siempre aminoácidos hidrofóbicos (amarillo); y los aminoácidos cargados (azul y rojo) se concentran en los codones con **A** o **G** en la segunda posición. Los codones de parada UAA, UAG y UGA aparecen en gris.

se aparea de forma estricta con las dos primeras bases del codón pero de forma relajada con la tercera, de modo que un mismo ARNt puede leer varios codones.

El código es casi universal, pero no del todo. Las mitocondrias de muchos organismos y algunas bacterias usan variantes: en los micoplasmas, por ejemplo, UGA no es de parada sino que codifica triptófano (Yamao et al., 1985). El NCBI numera estas variantes como “tablas de traducción” (la 1 es la estándar, la 11 la de bacterias y la 4 la de micoplasmas y mitocondrias de hongos), y en la sección 1.2 veremos que ignorarlas puede arruinar la anotación de un genoma completo.

1.1.7 Robustez ante errores: ¿es óptimo el código?

Cuando la ADN polimerasa se equivoca, cambia una base por otra. En una región codificante, la consecuencia depende del codón y de la posición:

- una mutación **sinónima** (o silenciosa) cambia el codón pero no el aminoácido;
- una mutación **de sentido erróneo** (*missense*) cambia el aminoácido;
- una mutación **sin sentido** (*nonsense*) crea un codón de parada y trunca la proteína.

Contemos todas las posibilidades. Hay 61 codones con sentido y cada uno tiene $3 \times 3 = 9$ vecinos que difieren en una sola base, de modo que existen $61 \times 9 = 549$ mutaciones puntuales. Un recuento directo sobre la tabla estándar (CODIGO en `dogma.py`) da 134 sinónimas (24,4 %), 23 sin sentido y 392 de sentido erróneo. Pero el promedio esconde una enorme diferencia entre posiciones.

Ejemplo 1.2. (Mutaciones en el gen de Spike)

Tomemos los 1273 codones con sentido del gen *S* de SARS-CoV-2 y apliquemos, a cada uno, las nueve mutaciones puntuales posibles: 11 457 mutaciones en total. Clasificándolas por posición del codón se obtiene:

Posición	Sinónimas	Sentido erróneo	Sin sentido
1. ^a	2,4 %	92,1 %	5,4 %
2. ^a	0,0 %	96,3 %	3,7 %
3. ^a	64,1 %	31,7 %	4,2 %

En la segunda posición *ninguna* mutación es silenciosa; en la tercera, casi dos de cada tres lo son. La primera posición tiene unas pocas sinónimas gracias a leucina y arginina, que tienen seis codones repartidos entre dos bloques (CUN/UUR y CGN/AGR). Esta asimetría es la base del cociente d_N/d_S entre tasas de sustitución no sinónima y sinónima, que usaremos para detectar selección natural en genómica evolutiva.

La pregunta profunda es otra: cuando una mutación *sí* cambia el aminoácido, ¿lo cambia por uno parecido? Si el código se hubiera fijado al azar, un error cambiaría con la misma facilidad un aminoácido hidrofóbico, que suele ir enterrado en el interior de la proteína, por uno cargado, que prefiere el agua; y eso suele destruir el plegamiento. Haig y Hurst (1991) convirtieron esta intuición en una prueba estadística. La idea es definir un *costo* que mida el daño medio que causan las mutaciones puntuales y comparar el código real con una población de códigos alternativos.

Definición 1.3. (Costo de error de un código)

Sea g un código genético (una función de los 61 codones con sentido en los 20 aminoácidos) y $h(\cdot)$ una propiedad fisicoquímica de los aminoácidos. Sea M el conjunto de pares (c, c') de codones con sentido que difieren en una sola base. El costo de g es

$$\Phi(g) = \frac{1}{|M|} \sum_{(c, c') \in M} \left(h(g(c)) - h(g(c')) \right)^2. \quad (1.5)$$

DONDE

$g(c)$	Aminoácido que el código g asigna al codón c .
M	Pares de codones con sentido separados por una mutación puntual; en el código estándar, $ M = 526$ (las 549 mutaciones menos las 23 que crean una parada).
$h(a)$	Propiedad del aminoácido a ; aquí, la hidropatía de Kyte y Doolittle (1982), positiva para los hidrofóbicos y negativa para los hidrofílicos.
$\Phi(g)$	Cambio cuadrático medio de la propiedad por mutación: cuanto menor, más robusto es el código. Las sinónimas contribuyen con 0.

¿Con qué códigos alternativos comparamos? El procedimiento de Haig y Hurst (1991) conserva la estructura de bloques del código real (qué codones van juntos y dónde están las paradas) y *baraja* los 20 aminoácidos entre los 20 bloques. Hay $20! \approx 2,4 \times 10^{18}$ códigos así, demasiados para enumerarlos, pero basta con muestrear al azar unos cuantos miles para estimar la distribución de Φ . Con este diseño, cualquier diferencia entre el código real y los barajados se debe a *qué aminoácido* se asignó a cada bloque, no a la redundancia en sí.

La figura 1.5b muestra el resultado con 100 000 códigos barajados. El código estándar tiene $\Phi = 9,39$, frente a una media de 13,30 para los barajados, y solo 824 de los 100 000 (el 0,82 %) igualan o mejoran su robustez. Haig y Hurst (1991), usando el “requerimiento polar” de los aminoácidos como propiedad, obtuvieron un resultado del mismo tipo. Freeland y Hurst (1998) refinaron el análisis teniendo en cuenta dos hechos biológicos: las transiciones (purina↔purina, pirimidina↔pirimidina) son mucho más frecuentes que las transversiones, y los errores de traducción son más frecuentes en la primera y tercera posiciones que en la segunda. Con esa ponderación, solo uno de cada millón de códigos alternativos resultó mejor que el natural, de ahí el título de su artículo.

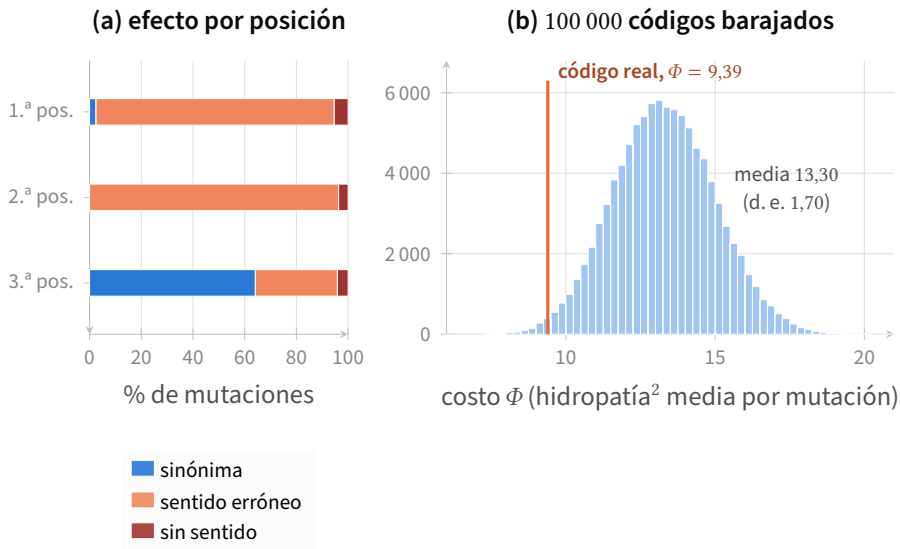


Figura 1.5. El código genético amortigua las mutaciones. (a) Efecto de las 11 457 mutaciones puntuales posibles en los codones del gen S de SARS-CoV-2, según la posición del codón: la tercera posición absorbe la mayoría de los cambios sin alterar la proteína. (b) Distribución del costo Φ (ecuación (1.5)), medido con la hidropatía de Kyte-Doolittle, para 100 000 códigos que conservan los bloques de codones del código real pero barajan los aminoácidos. El código real (línea naranja) queda en la cola izquierda: solo el 0,82 % de los códigos barajados es tan robusto o más. Con medidas más refinadas, que ponderan los tipos de error, esa fracción cae a una en un millón (Freeland & Hurst, 1998).

⚠ Lo que el experimento demuestra, y lo que no

Nuestra cifra (0,82 %) y la de Freeland y Hurst (una en un millón) no son contradictorias: dependen de la propiedad elegida y de cómo se ponderan los errores, y la versión con hidropatía sin ponderar es una aproximación didáctica. Además, que el código sea robusto no demuestra que la selección natural lo haya *optimizado*. Crick (1968) sostuvo que el código podría ser un “accidente congelado”: una vez que muchas proteínas dependen de él, cualquier cambio de asignación sería letal, y por eso ya no puede evolucionar. Las hipótesis estereoquímica, de coevolución con las rutas biosintéticas de los aminoácidos y adaptativa siguen en discusión; Koonin y Novozhilov (2009) ofrecen una revisión equilibrada.

El código genético guarda la información con redundancia, y la reparte de forma que los errores más probables sean los menos dañinos.

1.2 Genes, genomas y marcos abiertos de lectura

 Colab · 1.2 · Buscador de ORFs

 abrir notebook

Un recetario sin espacios

Imagine un libro de cocina de 580 000 letras escrito sin espacios, sin puntos y sin títulos: solo **A**, **C**, **G** y **T**, una detrás de otra, y además impreso dos veces, una al derecho y otra al revés y cambiando cada letra por su pareja. En alguna parte hay unas 500 recetas, pero nadie las subrayó. Usted sabe, eso sí, que toda receta empieza con la palabra **ATG** y termina con **TAA**, **TAG** o **TGA**, y que las palabras tienen siempre tres letras. ¿Cómo encontraría las recetas? ¿Y cómo sabría que un tramo que “parece” receta no es una casualidad del alfabeto?

1.2.1 Anatomía de un gen: procariotas y eucariotas

Un gen codificante reúne una región que especifica una proteína y señales que la maquinaria celular reconoce. En las bacterias, esa organización es compacta (figura 1.6, arriba):

- Un **promotor**, con dos cajas conservadas a unas 10 y 35 bases antes del inicio de la transcripción, donde se une la ARN polimerasa.
- Un **sitio de unión al ribosoma** (RBS), con la secuencia de **Shine-Dalgarno**, parecida a **AGGAGG**, unas 8 bases antes del codón de inicio. Shine y Dalgarno (1974) observaron que el extremo 3' del ARN ribosómico 16S de *E. coli* es complementario de esa secuencia: el ribosoma “encuentra” el inicio apareándose con ella.
- Una **región codificante** (CDS) continua, desde un codón de inicio (**ATG** casi siempre; **GTG** o **TTG** en una minoría de genes bacterianos) hasta el primer codón de parada en el mismo marco.

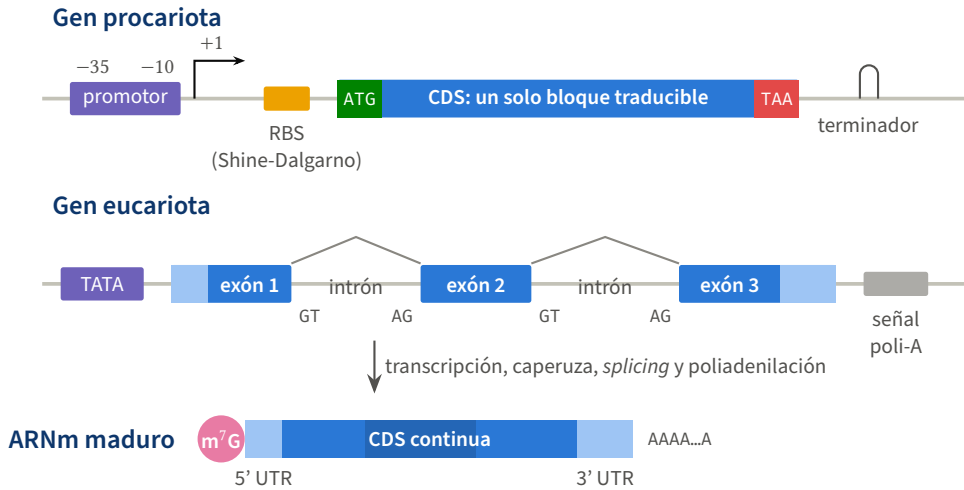


Figura 1.6. Anatomía de un gen. **Arriba:** en una bacteria, la región codificante (azul) es un bloque continuo entre un codón de inicio (verde) y el primer codón de parada del mismo marco (rojo); el ribosoma se posiciona gracias a la secuencia de Shine-Dalgarno. **Abajo:** en un eucariota la información codificante está repartida en exones separados por intrones, que se eliminan por *splicing* (bordes GT...AG); solo el ARNm maduro contiene una CDS continua, flanqueada por regiones no traducidas (UTR, azul claro), la caperuza 5' y la cola poli-A. Un buscador de ORFs como el de esta sección funciona sobre el genoma bacteriano, pero no sobre el ADN genómico eucariota.

- Un **terminador**, a menudo una horquilla de ARN rica en GC seguida de uracilos.

Además, los genes bacterianos suelen organizarse en *operones*: varios CDS consecutivos transcritos en un solo ARNm. En los eucariotas la situación es muy distinta. Berget et al. (1977), estudiando ARNm del adenovirus, descubrieron que la secuencia del mensajero maduro no es colineal con el ADN: faltan segmentos internos que se eliminan durante el procesamiento. Gilbert (1978) propuso llamar *intrones* a los segmentos eliminados y *exones* a los conservados, y sugirió que esta organización en piezas facilita la recombinación de dominios proteicos durante la evolución. El *corte y empalme* (*splicing*) reconoce señales en los bordes del intrón, casi siempre **G** **T** en el extremo 5' y **A** **G** en el 3', y el ARNm maduro recibe además una caperuza de 7-metilguanosina en su extremo 5' y una cola de poliadenina en el 3'.

1.2.2 Marcos de lectura

Como los codones se leen de tres en tres sin separadores, una misma secuencia puede cortarse en codones de tres maneras en cada hebra, según la letra en que empecemos. Con las dos hebras, hay seis *marcos de lectura* (figura 1.7).

Definición 1.4. (Marco abierto de lectura (ORF))

Sea s una de las dos hebras de un genoma (la directa x o su complemento reverso $\bar{x}$) y $f \in \{0, 1, 2\}$ un desplazamiento. El marco (s, f) es la sucesión de codones $c_k = s_{f+3k+1} s_{f+3k+2} s_{f+3k+3}$, $k = 0, 1, \dots$. Dados un conjunto de codones de inicio $\mathcal{I}$ y de parada $\mathcal{T}$, un *marco abierto de lectura* es un tramo de codones $c_a, \dots, c_b$ de un mismo marco tal que

$$c_a \in \mathcal{I}, \quad c_b \in \mathcal{T}, \quad c_k \notin \mathcal{T} \text{ para } a \leq k < b, \quad (1.6)$$

y c_a es el primer codón de inicio posterior a la parada anterior del marco. Su longitud es $\ell = b - a$ codones (sin contar la parada).

DONDE

s, f	Hebra y desplazamiento que definen uno de los seis marcos: $+1, +2, +3$ en x y $-1, -2, -3$ en $\bar{x}$.
c_k	k -ésimo codón del marco.
$\mathcal{I}, \mathcal{T}$	Codones de inicio ($\{\text{ATG}\}$, o $\{\text{ATG, GTG, TTG}\}$ en bacterias) y de parada (dependen de la tabla de traducción).
ℓ	Longitud del ORF en codones, igual al número de aminoácidos de la proteína que codificaría.

Elegir el *primer* inicio tras la parada anterior da el ORF más largo posible en ese tramo; el inicio biológico real puede estar más adentro, y predecirlo con precisión es un problema aparte que los programas modernos resuelven combinando la señal de Shine-Dalgarno con el uso de codones (Hyatt et al., 2010). El final, en cambio, es inequívoco: el primer codón de parada del marco. Por eso, cuando comparemos predicciones con la anotación, usaremos el extremo 3' como criterio de acierto.

1.2.3 Un buscador de ORFs

La ecuación (1.6) se traduce directamente en un algoritmo de una sola pasada por marco: recorrer los codones de izquierda a derecha con una variable de estado

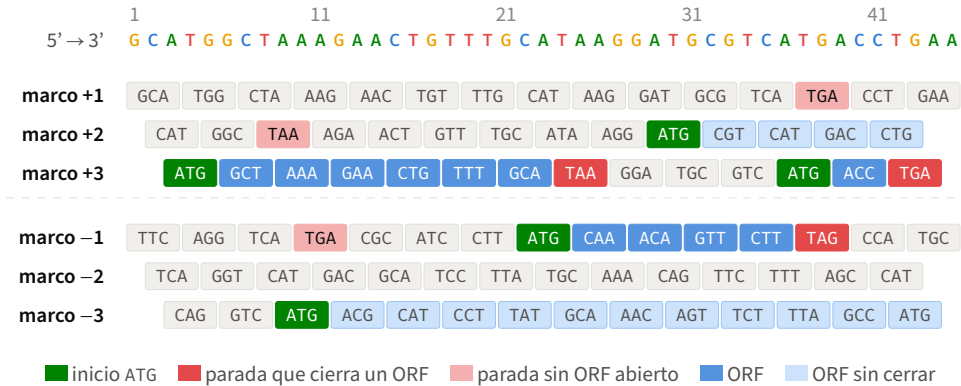


Figura 1.7. Una secuencia de 45 nt leída en sus seis marcos. Los tres marcos superiores cortan la hebra directa empezando en la primera, segunda o tercera base; los tres inferiores hacen lo mismo con el complemento reverse, de modo que todas las filas se leen de 5' a 3'. Un ORF se abre con un ATG y se cierra con la primera parada *del mismo marco*: en el marco +3, el ATG inicial abre un ORF que la parada TAA cierra seis codones después, aunque en el marco +1 haya paradas intercaladas. En el marco -3 el ORF llega al final de la secuencia sin cerrarse. Figura generada con `figuras/cap01/generar.py`.

que indica si hay un ORF abierto. Al ver un inicio con el ORF cerrado, se abre; al ver una parada con el ORF abierto, se cierra y se registra. Las coordenadas de los ORFs de la hebra inversa se convierten a coordenadas de la hebra directa al final.

`buscar_orfs.py`

```
PARADAS_11 = {"TAA", "TAG", "TGA"} # tabla estándar/bacteriana
PARADAS_4 = {"TAA", "TAG"} # Mycoplasma: TGA = Trp

def buscar_orfs(x, min_cod=100, inicios=("ATG",), paradas=PARADAS_11):
    """ORFs en los 6 marcos; coordenadas 0-based [ini, fin) en x."""
    L, orfs = len(x), []
    for hebra, s in ((+1, x), (-1, complemento_reverso(x))):
        for f in range(3):
            abierto = None
            for k in range(f, len(s) - 2, 3):
                c = s[k:k + 3]
                if abierto is None and c in inicios:
                    abierto = k # se abre
                elif abierto is not None and c in paradas:
                    n = (k - abierto) // 3 # codones
```

```

        if n >= min_cod:
            a, b = abierto, k + 3
            if hebra == -1:                # a hebra +
                a, b = L - b, L - a
                orfs.append((a, b, hebra, f + 1, n))
            abierto = None                 # se cierra
    return orfs

```

El algoritmo recorre cada base una vez por marco, así que su costo es $O(L)$ en tiempo. En Python puro tarda unos segundos para un genoma bacteriano; la versión vectorizada del notebook convierte cada codón en un entero entre 0 y 63 y localiza, para cada parada, el primer inicio posterior a la parada anterior con una búsqueda binaria (`np.searchsorted`), lo que la hace unas cien veces más rápida con resultados idénticos.

1.2.4 El modelo nulo: ¿cuán largo puede ser un ORF por azar?

Aquí está el corazón estadístico de la lección. Si escribimos letras al azar también aparecen ORFs, pero cortos: tarde o temprano tropezamos con una parada. Para decidir si un ORF largo es un gen necesitamos saber qué longitudes produce el azar.

Supongamos que las bases son independientes, con frecuencias $\pi_A, \pi_C, \pi_G, \pi_T$. La probabilidad de que un codón cualquiera sea de parada es

$$p = \sum_{c \in \mathcal{T}} \pi_{c_1} \pi_{c_2} \pi_{c_3}, \quad (1.7)$$

que con bases equiprobables vale $p = 3/64 \approx 0,0469$. Recorriendo un marco codón a codón, cada codón es un ensayo de Bernoulli con probabilidad de “éxito” (parada) p , independiente de los demás. La longitud de un tramo entre dos paradas consecutivas es el número de fracasos antes del primer éxito.

Teorema 1.2. (Longitud de un ORF aleatorio)

Bajo el modelo de bases independientes, la longitud L (en codones) de un tramo sin paradas sigue una distribución geométrica:

$$\mathbb{P}(L = \ell) = (1 - p)^\ell p, \quad \mathbb{P}(L \geq \ell) = (1 - p)^\ell, \quad \mathbb{E}[L] = \frac{1 - p}{p}. \quad (1.8)$$

En escala logarítmica, la función de supervivencia $\mathbb{P}(L \geq \ell)$ es una recta de pendiente $\ln(1 - p)$.

DONDE

π_b	Frecuencia de la base b en el genoma.
p	Probabilidad de que un codón al azar sea de parada (ecuación (1.7)).
L	Número de codones consecutivos sin parada en un marco.
$P(L \geq \ell)$	Supervivencia: probabilidad de que un tramo alcance al menos ℓ codones.

La supervivencia se deduce sin cálculo: $L \geq \ell$ ocurre exactamente cuando los primeros ℓ codones no son de parada, y como son independientes, la probabilidad es el producto $(1 - p)^\ell$. La media se obtiene de la serie $E[L] = \sum_{\ell \geq 1} P(L \geq \ell) = \sum_{\ell \geq 1} (1 - p)^\ell = (1 - p)/p$. Con bases equiprobables, $E[L] = 61/3 \approx 20,3$ codones, y el 1 % de los tramos supera $\ell_{1\%} = \ln 0,01 / \ln(61/64) \approx 96$ codones. Un tramo de 300 codones sin paradas tiene probabilidad $(61/64)^{300} \approx 5,6 \times 10^{-7}$.

 La composición manda

Los codones de parada son ricos en **A** y **T**. Con 20 % de GC, $p = 0,096$ y el 1 % de los tramos al azar supera solo 46 codones; con 72 % de GC, $p = 0,017$ y ese percentil sube a 271. En genomas ricos en GC, como los de *Streptomyces*, el azar fabrica ORFs muy largos: nunca use un umbral “universal” sin recalcular p con la composición de su genoma.

El teorema también nos dice cómo elegir un umbral. Si el genoma tiene L_g bases, cada uno de los seis marcos contiene unos $L_g/3$ codones y, por lo tanto, unas $pL_g/3$ paradas; el número total de tramos entre paradas es aproximadamente $N \approx 2pL_g$. El número esperado de tramos aleatorios de longitud al menos ℓ es $N(1 - p)^\ell$, y exigir que sea menor que un número tolerable α de falsos positivos da

$$N(1 - p)^{\ell^*} \leq \alpha \iff \ell^* \geq \frac{\ln(\alpha/N)}{\ln(1 - p)}. \quad (1.9)$$

DONDE

L_g	Longitud del genoma en pares de bases.
N	Número esperado de tramos entre paradas en los seis marcos, $\approx 2pL_g$.
α	Número de ORFs espurios que estamos dispuestos a tolerar en todo el genoma.
ℓ^*	Longitud mínima que garantiza, en promedio, no más de α falsos positivos.

Esta es una corrección por comparaciones múltiples disfrazada: cuanto mayor es el genoma, más “intentos” tiene el azar y más largo debe ser el umbral. El umbral es conservador por una segunda razón: exigir además un codón de inicio elimina muchos tramos.

1.2.5 Un genoma mínimo real: *Mycoplasma genitalium*

Mycoplasma genitalium es una bacteria parásita del tracto urogenital humano, sin pared celular, con uno de los genomas más pequeños conocidos entre los organismos capaces de crecer en cultivo. Fraser et al. (1995) lo secuenciaron completo, el segundo genoma bacteriano de la historia, y describieron unos 580 kb con alrededor de 470 regiones codificantes predichas; su título, “el complemento génico mínimo”, anunciaba la pregunta que ha guiado desde entonces la investigación sobre este organismo: ¿cuántos genes necesita una célula para vivir? Glass et al. (2006) respondieron en parte con mutagénesis por transposones, concluyendo que 382 de sus 482 genes codificantes de proteínas son esenciales. Hutchison et al. (2016) llevaron la pregunta a la síntesis: diseñaron y construyeron JCVI-syn3.0, una célula de *Mycoplasma mycoides* con un genoma sintético de 531 kb y 473 genes, de los cuales 149 tenían una función biológica desconocida.

La versión actual del genoma de referencia (NC_000908.2) tiene 580 076 pb, un contenido GC de solo 31,7% y 504 CDS anotadas, más 20 pseudogenes. La mediana de longitud de sus proteínas es de 287 aminoácidos, y solo 35 tienen menos de 100.

El codón que no para

En los micoplasmas, UGA no es un codón de parada sino que codifica triptófano. Yamao et al. (1985) lo demostraron en *Mycoplasma capricolum* y la variante se comparte en el linaje, hasta el punto de que el NCBI le asigna una tabla de traducción propia (la 4). Se cree que la presión hacia genomas pobres en GC favoreció el cambio: UGG,

el codón habitual del triptófano, es rico en G, y UGA fue “capturado” para el mismo aminoácido.

Esta diferencia cambia el resultado de la anotación. Al traducir las 504 CDS de *M. genitalium* con la tabla 4, las 504 proteínas coinciden con la referencia. Con la tabla bacteriana estándar (11), solo 145 (el 28,8 %) salen completas: las otras 359 contienen al menos un TGA interno y se truncan allí. Un anotador automático que usara la tabla equivocada predeciría cientos de genes fragmentados.

Ejemplo 1.3. (El umbral de longitud para *M. genitalium*)

Con la composición real del genoma ($\pi_A = 0,3457$, $\pi_C = 0,1578$, $\pi_G = 0,1591$, $\pi_T = 0,3374$) y las dos paradas de la tabla 4, la ecuación (1.7) da

$$p = \pi_T \pi_A \pi_A + \pi_T \pi_A \pi_G = 0,3374 \times 0,3457 \times (0,3457 + 0,1591) = 0,0589.$$

La longitud media de un tramo al azar es $E[L] = 0,9411/0,0589 \approx 16,0$ codones. El número de tramos en los seis marcos es $N \approx 2 \times 0,0589 \times 580\,076 \approx 68\,300$, y la ecuación (1.9) con $\alpha = 1$ da

$$\ell^* \geq \frac{\ln(1/68\,300)}{\ln 0,9411} = \frac{-11,13}{-0,0607} \approx 183 \text{ codones.}$$

Por debajo de 100 codones, en cambio, el azar produce unos $N(1-p)^{100} \approx 158$ tramos: demasiados para confiar en la longitud sola. Observe que la mayoría de los genes reales del genoma (mediana de 287 aa) están por encima del umbral estricto, pero no todos.

1.2.6 Señal contra ruido

Para comprobar si los ORFs largos superan lo esperable por azar, construimos un *control negativo*: barajamos las letras del genoma. La secuencia resultante conserva exactamente la composición y, por tanto, la misma p , pero pierde el orden biológico. La figura 1.8 compara las funciones de supervivencia del genoma real, del barajado y de la predicción geométrica.

Para longitudes cortas, las tres curvas coinciden: $P(L \geq 50)$ vale 0,049 en el genoma real, 0,050 en el barajado y 0,048 en la teoría. La mayoría de los tramos del genoma real son, estadísticamente, ruido: están en los marcos que no se traducen. Pero a 100 codones la frecuencia real es de $1,55 \times 10^{-2}$ frente a $2,4 \times 10^{-3}$ en el

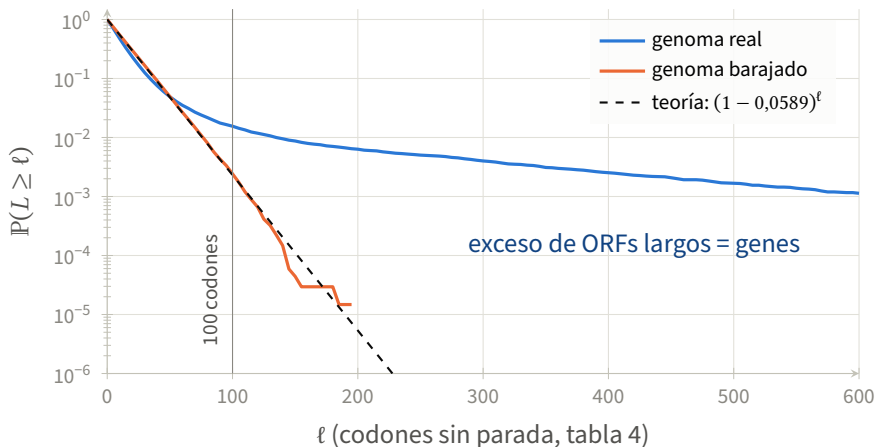


Figura 1.8. Longitud de los tramos sin codones de parada en los seis marcos de *M. genitalium* (tabla 4). El genoma barajado (naranja), que conserva la composición pero no el orden, sigue casi exactamente la recta geométrica del teorema 1.2 (discontinua) y no produce ningún tramo de más de 200 codones. El genoma real (azul) coincide con el azar para tramos cortos, pero a partir de unos 60 codones se separa y mantiene una cola pesada que llega a cientos de codones: ahí viven los genes. A 100 codones, la cola real es 6,4 veces más frecuente que la barajada; a 200, la barajada ya es cero.

barajado, y a 200 codones el barajado no produce ningún tramo mientras que el real conserva un 0,63 %. El genoma barajado es, además, la mejor validación del modelo: sus puntos caen sobre la recta teórica, lo que confirma que la dependencia entre bases vecinas apenas afecta a la distribución de las paradas.

El barajado permite también estimar directamente la proporción de falsos descubrimientos que cometería un umbral $\ell_{\min}$:

$$\widehat{\text{FDR}}(\ell_{\min}) \approx \frac{N_{\text{barajado}}(\ell_{\min})}{N_{\text{real}}(\ell_{\min})}, \quad (1.10)$$

DONDE

$N_{\text{real}}(\ell_{\min})$	Número de ORFs (con inicio ATG, GTG o TTG) de al menos $\ell_{\min}$ codones en el genoma real.
$N_{\text{barajado}}(\ell_{\min})$	Lo mismo en el genoma barajado, que estima cuántos de los anteriores esperaríamos por azar.

Con $\ell_{\min} = 40$, el genoma barajado produce 2861 ORFs frente a 2909 en el real ($\widehat{\text{FDR}} \approx 0,98$): casi todo es ruido. Con 60 codones la proporción baja a 0,55; con 100, a 0,083 (67 frente a 809); con 150, a 0,004. No existe un umbral perfecto, solo un compromiso entre falsos positivos y genes cortos perdidos.

1.2.7 Sensibilidad, precisión y ORFs sombra

La anotación del GenBank nos da una “respuesta correcta” contra la que evaluar el buscador. Decimos que un ORF predicho es un *verdadero positivo* (VP) si termina en el mismo codón de parada y la misma hebra que una CDS anotada; si no, es un *falso positivo* (FP). Las CDS anotadas que ningún ORF alcanza son *falsos negativos* (FN).

$$\text{sensibilidad} = \frac{\text{VP}}{\text{VP} + \text{FN}}, \quad \text{precisión} = \frac{\text{VP}}{\text{VP} + \text{FP}}. \quad (1.11)$$

La sensibilidad responde “¿qué fracción de los genes reales encontré?”; la precisión, “¿qué fracción de mis predicciones son genes?”.

Con un umbral de 100 codones y los tres inicios bacterianos, el buscador predice 809 ORFs, de los cuales 473 coinciden con CDS anotadas: sensibilidad de 93,8 % pero precisión de solo 58,5 %. ¿De dónde salen los 336 falsos positivos? Al examinarlos, 289 (el 86 %) se solapan en más de la mitad de su longitud con un gen real situado en *otro marco o en la otra hebra*, y 29 caen en pseudogenes. Son *ORFs sombra*: una región codificante está depurada de paradas en su marco, y como efecto colateral de su composición y su uso de codones, los marcos solapados también tienen menos paradas de lo que el azar predice.

Como dos genes bacterianos rara vez se solapan mucho, un filtro sencillo elimina la mayoría de las sombras: se ordenan los ORFs de más largo a más corto y se descarta cualquier ORF que se solape más de un 50 % con otro más largo ya aceptado. Con este filtro, a 100 codones quedan 502 predicciones, la sensibilidad apenas baja a 92,5 % y la precisión sube a 92,8 %. La figura 1.9 muestra el compromiso para todos los umbrales.

Los 38 genes que el buscador filtrado no encuentra a 100 codones tienen algo en común: 31 de ellos codifican proteínas de menos de 100 aminoácidos, por debajo del umbral. Los demás empiezan con un codón de inicio poco habitual o se solapan con un gen vecino más largo. Detectar genes pequeños sigue siendo un problema abierto; muchas proteínas de menos de 50 aminoácidos se han descubierto apenas en la última década.

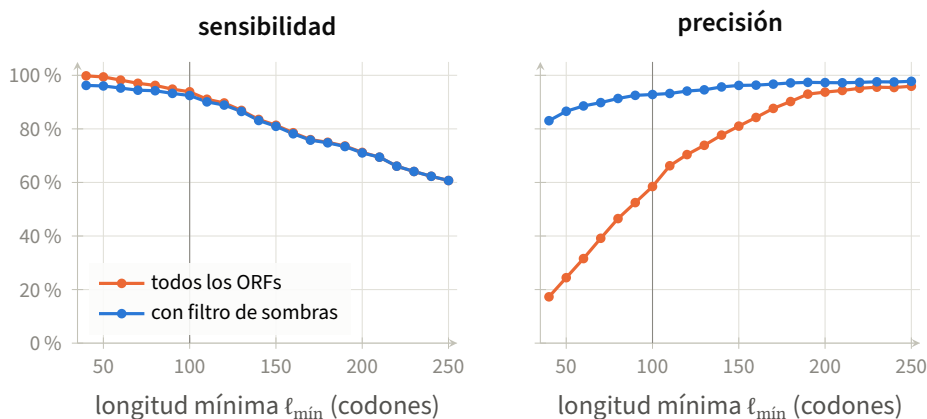


Figura 1.9. Evaluación del buscador de ORFs contra las 504 CDS anotadas de *M. genitalium* (acierto: mismo codón de parada y misma hebra). Al subir el umbral, la sensibilidad cae porque se pierden los genes cortos, y la precisión sube porque desaparece el ruido. El filtro de sombras (azul) dispara la precisión a cualquier umbral casi sin coste en sensibilidad: con 100 codones (línea gris) pasa del 58 % al 93 % manteniendo el 92 % de los genes.

Más allá de la longitud: contenido codificante

Las regiones codificantes tienen una *periodicidad de tres*: la composición de la primera, segunda y tercera posición del codón es distinta, porque la tercera tolera más cambios. Fickett (1982) convirtió esa asimetría en un estadístico (TESTCODE) que reconoce regiones codificantes sin buscar ORFs. Los buscadores modernos combinan varias señales en modelos probabilísticos: Prodigal (Hyatt et al., 2010) modela el uso de codones, la señal de Shine-Dalgarno y el contexto del inicio, y supera el 95 % de sensibilidad en genomas bacterianos. Todos parten, sin embargo, del esqueleto que hemos construido: enumerar ORFs y compararlos con un modelo nulo.

1.3 Composición de secuencias: contenido GC, GC skew y el origen de replicación

Un saldo bancario que se lee en el ADN

Imagine que lleva la cuenta de un cajero que, a lo largo del día, recibe billetes de dos colores: suma uno por cada billete verde y resta uno por cada billete rojo. Si los billetes llegan al azar, el saldo oscila sin rumbo alrededor de cero. Pero si durante la mañana llega un pequeño exceso de verdes y durante la tarde un pequeño exceso de rojos, el saldo subirá toda la mañana y bajará toda la tarde, y su máximo marcará, con bastante precisión, la hora del almuerzo, aunque ningún billete individual lo revele. En esta sección haremos exactamente eso con las guaninas y las citosinas de un cromosoma bacteriano, y la “hora del almuerzo” será el punto donde la célula empieza a copiar su ADN.

1.3.1 Contenido GC

La medida de composición más sencilla es la proporción de bases que son **G** o **C**.

Definición 1.5. (Contenido GC)

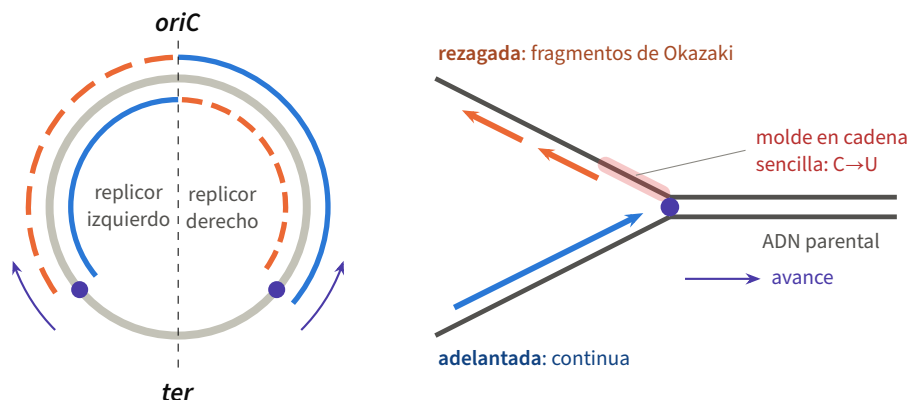
Para una secuencia (o una ventana) con n_A , n_C , n_G y n_T bases de cada tipo,

$$GC = \frac{n_G + n_C}{n_A + n_C + n_G + n_T}. \quad (1.12)$$

Cada **G** de una hebra se aparea con una **C** de la otra. Por ello, ambas tienen el mismo contenido GC: la medida corresponde a la *molécula*, no a una hebra concreta. Entre organismos varía mucho: 31,7 % en *M. genitalium*, 38,0 % en SARS-CoV-2, 50,8 % en *E. coli* y más de 70 % en algunas actinobacterias. Influye en la estabilidad térmica del ADN (un par G–C tiene tres puentes de hidrógeno; uno A–T, dos) y en la probabilidad de encontrar paradas por azar, como vimos en la sección 1.2. Dentro de un genoma bacteriano, las variaciones locales de GC suelen señalar *islas genómicas* adquiridas por transferencia horizontal.

1.3.2 Un cromosoma circular que se copia en dos direcciones

El cromosoma de *Escherichia coli* K-12, secuenciado por Blattner et al. (1997), es una sola molécula circular de unos 4,64 Mb. Su replicación sigue un plan muy ordenado. Comienza siempre en el mismo lugar, el *origen de replicación* (*oriC*), donde la proteína DnaA abre la doble hélice. Desde allí parten dos *horquillas de replicación* en direcciones opuestas, que se encuentran en el lado contrario del círculo, en la



las dos horquillas avanzan en sentidos opuestos

Figura 1.10. **Izquierda:** el cromosoma bacteriano se replica desde *oriC* con dos horquillas (violeta) que avanzan en sentidos opuestos hasta *ter*. En cada replicor se dibujan la hebra hija continua (azul) y la discontinua (naranja); observe que sus papeles se *intercambian* al pasar de un replicor al otro. **Derecha:** detalle de una horquilla. La hebra adelantada crece de forma continua hacia la horquilla; la rezagada se sintetiza en fragmentos de Okazaki que crecen en sentido contrario, de modo que su molde queda temporalmente como cadena sencilla (sombreado rojo), donde la desaminación de la citosina es mucho más rápida.

región de terminación (*ter*), donde la proteína Tus, unida a los sitios *Ter*, las detiene. El cromosoma queda dividido en dos mitades, los *replicores*, cada una copiada por una sola horquilla.

Dentro de cada horquilla, las dos hebras hijas no se sintetizan igual. Las ADN polimerasas solo alargan cadenas en dirección $5' \rightarrow 3'$, y las dos hebras molde son antiparalelas. Una de las hebras hijas, la *adelantada* (*leading*), puede crecer de corrido en el mismo sentido en que avanza la horquilla. La otra, la *rezagada* (*lagging*), debe sintetizarse hacia atrás, en fragmentos cortos que luego se unen: los fragmentos que Okazaki et al. (1968) detectaron marcando con radiactividad el ADN recién sintetizado. Mientras la horquilla avanza y el siguiente fragmento aún no ha empezado, el molde de la hebra rezagada queda expuesto como *cadena sencilla* (figura 1.10).

Ahora viene la observación clave. Fijemos *una* hebra del cromosoma, por ejemplo la que aparece escrita en el archivo FASTA. En un replicor, esa hebra es el molde de la hija adelantada; en el otro, el de la rezagada. El cambio de papel ocurre exactamente en *oriC* y en *ter*. Si la replicación trata de forma distinta a los dos moldes, la hebra

del archivo debería mostrar un sesgo de composición que cambia de signo en esos dos puntos.

1.3.3 La cicatriz de la replicación

La citosina tiene un punto débil químico: puede perder su grupo amino y convertirse en uracilo, que en la siguiente replicación se aparea con adenina. El resultado neto es una mutación $C \rightarrow T$. Esta *desaminación* es una de las fuentes más importantes de mutaciones espontáneas, y es mucho más rápida en ADN de cadena sencilla que en la doble hélice, donde la citosina está protegida por su pareja: Frederico et al. (1990) midieron una constante de velocidad más de cien veces mayor en cadena sencilla. Juntemos las piezas: el molde de la hebra rezagada pasa más tiempo como cadena sencilla, así que acumula más $C \rightarrow T$; esa hebra pierde C y, por complementariedad, la hebra opuesta pierde G . Lobry (1996) mostró que en varias bacterias la hebra adelantada es, en efecto, más rica en G que en C , y que el sesgo cambia de signo en el origen y el término de la replicación.

Un modelo mínimo permite ver cuánto sesgo produce una asimetría pequeña. Consideremos una hebra dentro de un replicor y fijémonos solo en las G y las C . Sea c la tasa de mutación $C \rightarrow T$ en esa hebra y a la tasa de $G \rightarrow A$ (que no es otra cosa que $C \rightarrow T$ en la hebra complementaria). Supongamos que las G y las C se reponen a partir de A y T con una misma tasa u , y que la hebra tiene aproximadamente tantas A como T .

Teorema 1.3. (Sesgo de equilibrio)

En el modelo anterior, las frecuencias de equilibrio satisfacen $u\pi_A = a\pi_G$ y $u\pi_T = c\pi_C$, de modo que, si $\pi_A \approx \pi_T$,

$$\frac{\pi_G}{\pi_C} = \frac{c}{a}, \quad S^* = \frac{\pi_G - \pi_C}{\pi_G + \pi_C} = \frac{c - a}{c + a}. \quad (1.13)$$

DONDE

c, a	Tasas de las mutaciones $C \rightarrow T$ y $G \rightarrow A$ leídas en la hebra considerada.
u	Tasa (simétrica) de las mutaciones que crean G o C a partir de A o T .
π_b	Frecuencia de equilibrio de la base b en esa hebra.
S^*	Sesgo GC de equilibrio: positivo si la hebra pierde más C que G .

La derivación es un balance de flujos: en equilibrio, las **G** que se pierden por unidad de tiempo ($a\pi_G$) deben igualar a las que se crean ($u\pi_A$), y lo mismo con las **C**. Dividiendo ambas igualdades se obtiene la ecuación (1.13). El resultado es revelador: el sesgo no depende de la magnitud absoluta de las tasas, sino solo de su *cociente*. Una asimetría minúscula, sostenida durante millones de generaciones, basta para dejar una huella medible en todo el genoma. Frank y Lobry (1999) revisan los mecanismos propuestos, que no se reducen a la desaminación: también contribuyen diferencias en la fidelidad de las polimerasas de cada hebra y efectos selectivos ligados a la orientación de los genes.

1.3.4 El sesgo GC y su versión acumulada

Para medir la huella necesitamos un número que, a diferencia del contenido GC, distinga entre las dos hebras.

Definición 1.6. (Sesgo GC y sesgo acumulado)

El *sesgo GC* (*GC skew*) de una ventana de la hebra leída es

$$S = \frac{n_G - n_C}{n_G + n_C} \in [-1, 1]. \quad (1.14)$$

El *sesgo acumulado* en la posición j es el saldo

$$C(j) = \sum_{i=1}^j \left(\mathbb{1}[x_i = G] - \mathbb{1}[x_i = C] \right). \quad (1.15)$$

DONDE

n_G, n_C	Número de guaninas y citosinas en la ventana, en la hebra que estamos leyendo.
S	Sesgo GC: +1 si solo hay G , -1 si solo hay C , 0 si están equilibradas.
$C(j)$	Saldo acumulado: suma +1 por cada G y -1 por cada C desde el inicio hasta la posición j .

Por la complementariedad, leer la otra hebra invierte el signo de S en cada ventana: el sesgo es una propiedad de la *hebra*. El sesgo acumulado, introducido como herramienta sistemática por Grigoriev (1998), tiene una ventaja decisiva sobre las ventanas: no obliga a elegir un tamaño de ventana y suma la señal a lo largo de todo el replicor, de modo que un sesgo diminuto pero sostenido produce una

pendiente clara. Si en un replicor el sesgo medio es s y el contenido GC es g , cada base aporta en promedio

$$E[C(j+1) - C(j)] = \frac{g}{2}(1+s) - \frac{g}{2}(1-s) = gs, \quad (1.16)$$

así que C sube con pendiente gs en el replicor de sesgo positivo y baja con pendiente $g|s|$ en el otro. El resultado es una curva en forma de V (o de V invertida) con el *mínimo en oriC* y el *máximo en ter*.

Ejemplo 1.4. (Sesgo a mano)

Tomemos la ventana **GGCAGCTGGA**. Las **G** están en las posiciones 1, 2, 5, 8 y 9, así que $n_G = 5$; las **C**, en las posiciones 3 y 6, así que $n_C = 2$. Por la ecuación (1.14),

$$S = \frac{5-2}{5+2} = \frac{3}{7} \approx 0,43.$$

El saldo acumulado, base a base, es 1, 2, 1, 1, 2, 1, 1, 2, 3, 3: sube con cada **G**, baja con cada **C** y se queda quieto con **A** y **T**. Su valor final, $C(10) = 3$, es exactamente $n_G - n_C$. En la hebra complementaria, **TCCAGCTGCC**, se obtiene $n_G = 2$, $n_C = 5$ y $S = -3/7$.

gc_skew.py

```
import numpy as np
```

```
def sesgo_ventanas(x: str, w: int = 10_000) -> np.ndarray:
    """Sesgo (G-C)/(G+C) en ventanas contiguas de w bases."""
    a = np.frombuffer(x.encode("ascii"), dtype=np.uint8)
    n = len(a) // w
    a = a[: n * w].reshape(n, w)
    g = (a == ord("G")).sum(axis=1)
    c = (a == ord("C")).sum(axis=1)
    return (g - c) / (g + c)

def sesgo_acumulado(x: str) -> np.ndarray:
    """Saldo C(j): +1 por cada G, -1 por cada C (ecuación 1.15)."""
    a = np.frombuffer(x.encode("ascii"), dtype=np.uint8)
    paso = (a == ord("G")).astype(np.int32) - (a == ord("C"))
    return np.cumsum(paso)

C = sesgo_acumulado(ecoli) # ecol: str con 4 641 652 pb
ori_pred = int(np.argmax(C)) + 1 # coordenada 1-based
ter_pred = int(np.argmin(C)) + 1
```

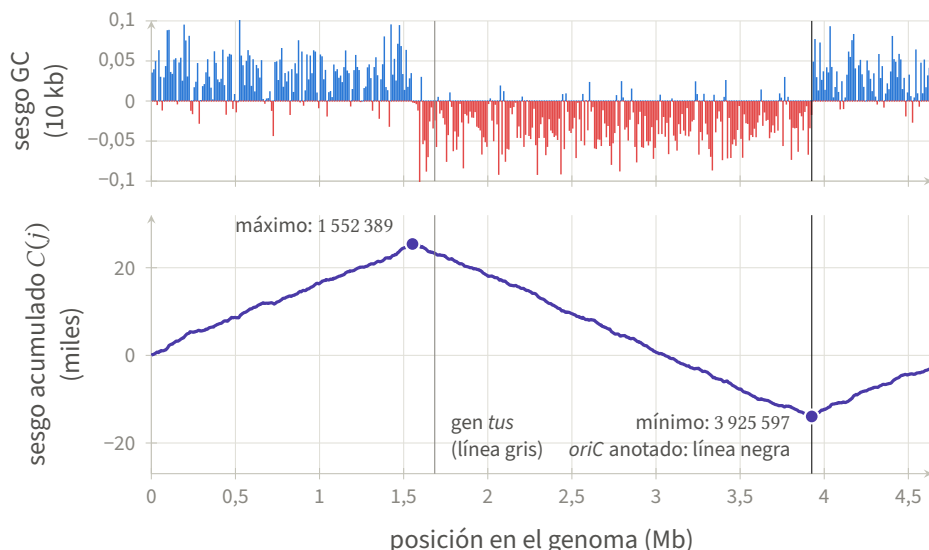


Figura 1.11. El sesgo GC de *E. coli* K-12 (NC_000913.3). **Arriba:** sesgo en 464 ventanas de 10 kb; es positivo en el replicor que va de *oriC* a *ter* en el sentido de la numeración (pasando por la posición 0) y negativo en el otro, aunque cada ventana individual es ruidosa. **Abajo:** el sesgo acumulado de la ecuación (1.15), que integra la señal y dibuja una V invertida limpia. Su mínimo (3 925 597) cae a 262 pb del centro del *oriC* anotado (línea negra); su máximo (1 552 389) está a 132 kb del gen *tus*, en la región de terminación.

1.3.5 Encontrar *oriC* en *E. coli*

Apliquemos el método al genoma de referencia de *E. coli* K-12 MG1655 (NC_000913.3, 4 641 652 pb). Según su propia anotación, el origen de replicación (*feature rep_origin*) ocupa las posiciones 3 925 744–3 925 975. No hay una anotación de *ter* como tal, pero sí del gen *tus* (1 684 259–1 685 188), cuyo producto se une a los sitios de terminación: es un buen marcador de esa región.

La figura 1.11 muestra el resultado. El sesgo por ventana es pequeño (en promedio +0,032 en un replicor y −0,033 en el otro) y ruidoso: 224 de las 464 ventanas son positivas, casi la mitad, pero no están mezcladas al azar sino agrupadas en dos bloques. El sesgo acumulado convierte ese patrón en una curva inequívoca. Su mínimo global está en la posición 3 925 597: a solo 262 pb del centro del *oriC* anotado, un error del 0,006 % del genoma. Su máximo está en 1 552 389, a 132 kb del gen *tus*; los dos replicores miden 2,37 y 2,27 Mb, casi iguales, como corresponde a dos

horquillas que avanzan a velocidades parecidas. Con un cálculo de una línea sobre la secuencia, sin pipetas ni geles, hemos localizado el origen de replicación.

Ejemplo 1.5. (¿Cuadra la amplitud de la V?)

La ecuación (1.16) permite predecir cuánto desciende el saldo entre el máximo y el mínimo. El arco que va de *ter* ($\approx 1\,552\,389$) a *oriC* ($\approx 3\,925\,597$) mide $2\,373\,208$ pb, el contenido GC es $g = 0,5079$ y el sesgo medio de ese replicor es $s = -0,0327$. El descenso esperado es

$$|\Delta C| \approx g |s| \Delta j = 0,5079 \times 0,0327 \times 2\,373\,208 \approx 39\,400.$$

El observado es $C_{\text{máx}} - C_{\text{mín}} = 25\,418 - (-13\,943) = 39\,361$. La coincidencia, mejor que el 0,2 %, confirma que la V es la suma de un sesgo pequeño y casi constante a lo largo de todo el replicor, y no el efecto de unas pocas regiones extremas.

1.3.6 El genoma como un reloj

Como el cromosoma es circular, la forma natural de representarlo es como la esfera de un reloj, con la posición o arriba y avanzando en el sentido de las agujas (figura 1.12). Este tipo de mapa, con anillos concéntricos para distintas pistas, es el formato estándar para presentar genomas bacterianos.

1.3.7 Señal contra ruido: el tamaño de la ventana

¿Por qué el sesgo por ventana es tan ruidoso, y cuánto hay que agrandar la ventana para ver la señal? Supongamos que una ventana contiene m bases que son G o C y que no hay ningún sesgo real: cada una es G con probabilidad $1/2$, independientemente de las demás. Entonces el número de guaninas es $K \sim \text{Bin}(m, 1/2)$, y el sesgo es $S = (K - (m - K))/m = (2K - m)/m$.

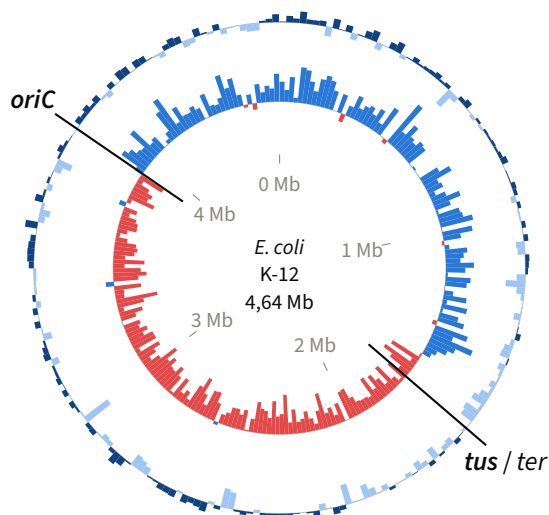


Figura 1.12. El cromosoma de *E. coli* K-12 como un reloj, en ventanas de 20 kb. **Anillo interno:** sesgo GC (azul, $G > C$, hacia fuera; rojo, $C > G$, hacia dentro). **Anillo externo:** desviación del contenido GC de cada ventana respecto a la media del genoma (azul oscuro hacia fuera, más GC; azul claro hacia dentro, menos GC). El sesgo divide el círculo en dos mitades de signo opuesto cuyas fronteras coinciden con *oriC* y con la región de *tus*. El contenido GC no muestra ese patrón: es una propiedad de la molécula, no de las hebras. Sus caídas bruscas (ventanas con hasta un 41 % de GC) señalan regiones de composición anómala, como las islas genómicas.

Teorema 1.4. (Ruido del sesgo en una ventana)

Sin sesgo real, $E[S] = 0$ y

$$DE(S) = \frac{2}{m} DE(K) = \frac{2}{m} \sqrt{m \cdot \frac{1}{2} \cdot \frac{1}{2}} = \frac{1}{\sqrt{m}}, \quad m \approx w \cdot g. \quad (1.17)$$

Si además existe un sesgo real que varía entre ventanas con desviación típica σ_s , las varianzas se suman: $DE_{obs}^2 \approx 1/m + \sigma_s^2$.

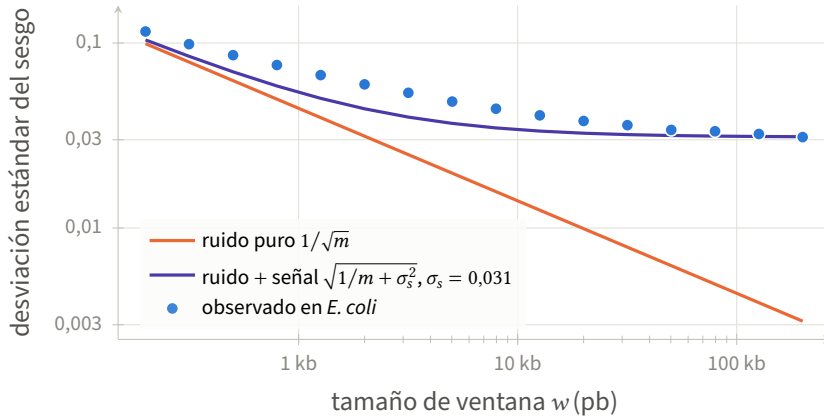


Figura 1.13. Dispersión del sesgo GC entre ventanas de *E. coli* en función del tamaño de ventana (escala log-log). Para ventanas pequeñas, la dispersión observada (puntos) está cerca del ruido binomial puro (naranja), que decrece como $1/\sqrt{m}$: casi toda la variación es azar de muestreo. Para ventanas grandes, el ruido sigue bajando pero la dispersión observada se estabiliza en torno a 0,03: es la señal de los dos replicores, que no depende del tamaño de ventana. El modelo que suma ambas varianzas (violeta) describe los extremos; los puntos intermedios quedan algo por encima porque existe además variación local real (genes de orientación particular, operones ribosomales, islas genómicas).

DONDE

m	Número de bases G o C en la ventana.
w, g	Tamaño de la ventana (pb) y contenido GC del genoma.
K	Número de guaninas entre esas m bases.
$DE(S)$	Desviación estándar del sesgo debida solo al azar del muestreo.
σ_s	Variabilidad real del sesgo entre ventanas (señal biológica).

Con ventanas de 1 kb en *E. coli*, $m \approx 508$ y el ruido es $\pm 0,044$, mayor que la señal de $\pm 0,03$: la señal queda enterrada. Con ventanas de 100 kb el ruido baja a $\pm 0,0044$ y la señal destaca con claridad. La figura 1.13 compara la dispersión observada con la predicción del teorema 1.4.

La lección es general y reaparecerá a lo largo del libro: todo promedio en ventana es un compromiso entre resolución y ruido, y el tamaño de ventana adecuado es aquel en el que el ruido de muestreo, $1/\sqrt{m}$, cae por debajo de la magnitud de la señal que se quiere ver. En *E. coli*, eso exige $m \gtrsim 1/0,03^2 \approx 1100$ bases G o C, es

decir, ventanas de al menos unos 2–3 kb, y en la práctica bastante más. El sesgo en la otra pareja de bases, $(n_A - n_T)/(n_A + n_T)$, también existe pero es unas tres veces más débil: en ventanas de 100 kb su magnitud media es 0,011 frente a 0,031 del sesgo GC.

La replicación organiza todo el genoma

El sesgo de composición es solo una de las huellas de la replicación. Rocha (2004) revisa las demás: los genes, sobre todo los esenciales y los muy expresados, tienden a estar en la hebra adelantada, probablemente para evitar choques frontales entre la ARN polimerasa y la horquilla; y en bacterias de crecimiento rápido los genes cercanos a *oriC* están en más copias que los cercanos a *ter*. Como parte del sesgo GC se debe a esa orientación de los genes, separar la contribución mutacional de la selectiva no es trivial (Frank & Lobry, 1999).

En resumen

- El dogma central describe un sistema de información: el ADN guarda, el ARN es la copia de trabajo y la proteína ejecuta. La información secuencial no sale de las proteínas; la transcripción reversa y la replicación de ARN son transferencias especiales (Crick, 1970).
- La entropía de Shannon $H = -\sum p \log_2 p$ mide la sorpresa media por símbolo: como máximo 2 bits por base en ADN. Los genomas reales están cerca del máximo; las repeticiones, muy por debajo. La entropía es el límite de la compresión sin pérdida.
- El código genético es redundante y robusto: la tercera posición del codón absorbe la mayoría de las mutaciones, y las sustituciones de aminoácido que produce una mutación puntual tienden a conservar la hidropatía mejor que en casi cualquier código alternativo.
- Un ORF va de un codón de inicio a la primera parada del mismo marco; toda secuencia tiene seis marcos. Por azar, la longitud de un tramo sin paradas es geométrica, $P(L \geq \ell) = (1 - p)^\ell$, con p determinada por la composición y el código genético del organismo.
- En *M. genitalium* (tabla 4, TGA=Trp), los genes forman una cola de ORFs largos muy por encima del modelo nulo. Un umbral de 100 codones con filtro de sombras recupera el 92 % de los genes con 93 % de precisión; los genes cortos siguen siendo difíciles.

- La replicación bidireccional desde *oriC* y la desaminación de la citosina en el molde de la hebra rezagada dejan un sesgo $S = (G - C)/(G + C)$ que cambia de signo en *oriC* y *ter*. El mínimo del sesgo acumulado localiza el *oriC* de *E. coli* a 262 pb de su posición anotada; el ruido en una ventana, $1/\sqrt{m}$, dicta el tamaño de ventana.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 1.1 ●●● Escriba a mano el complemento reverso de **ATGGCCTTAGC** y compruebe con Biopython que `Seq(x).reverse_complement()` da lo mismo. ¿Cuál es el complemento reverso del complemento reverso?
- 1.2 ●●● Calcule la entropía de Shannon de **ATATATAT** con $k = 1$ y con $k = 2$. ¿Por qué $h_2 < h_1$? ¿Qué valor tomaría la tasa de entropía h para una repetición infinita de AT?
- 1.3 ●●● Usando la ecuación (1.7), calcule p , la longitud media al azar y el percentil 99 de un ORF para un genoma con 20 %, 50 % y 72 % de GC (tabla 11, $\pi_G = \pi_C = GC/2$). Compare con los valores de la sección 1.2.
- 1.4 ●●● Repita el cálculo de robustez del código del cuaderno 1.1 usando la carga de los aminoácidos (+1 para K, R; -1 para D, E; o para el resto) en lugar de la hidropatía. ¿Sigue siendo el código real excepcional? ¿Por qué cree que la respuesta cambia?
- 1.5 ●●● Modifique `buscar_orfs` para que, además de las coordenadas, devuelva la proteína traducida con la tabla que se le indique. Aplíquelo al genoma de SARS-CoV-2 (NC_045512.2) con $\ell_{\min} = 50$ y compare sus cinco ORFs más largos con los genes anotados. ¿Qué ocurre con *ORF1ab*, que requiere un desplazamiento ribosomal de marco?
- 1.6 ●●● Demuestre que, bajo el modelo de bases independientes, la longitud esperada de un ORF que empieza en ATG y se detiene en la primera parada es la misma que la de un tramo entre paradas. Después, verifique con una simulación que el número esperado de ORFs de al menos ℓ codones en una secuencia aleatoria de longitud L_g es aproximadamente $2L_g p q (1-p)^\ell / (p+q)$,

donde q es la probabilidad de un codón de inicio. (Pista: una parada solo cierra un ORF si antes hubo un inicio.)

- 1.7** ●●● Descargue el genoma de *Bacillus subtilis* 168 (NC_000964.3) y localice su origen de replicación con el sesgo acumulado. ¿Dónde cae el mínimo? Rote la secuencia para centrar la V y estime la longitud de cada replicor. Compare la amplitud observada con la predicha por la ecuación (1.16).

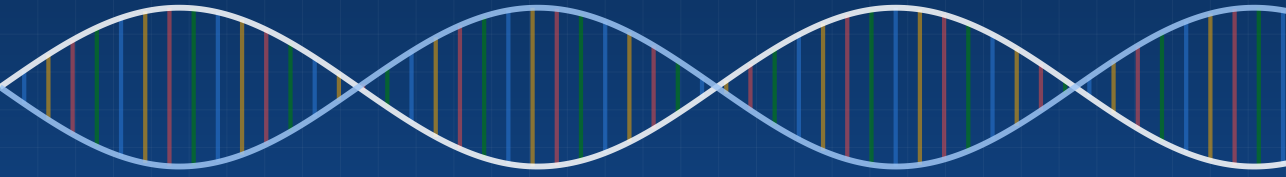
Referencias del capítulo

- Alberts, B., Heald, R., Johnson, A., Morgan, D., Raff, M., Roberts, K., & Walter, P. (2022). *Molecular Biology of the Cell* (7.^a ed.). W. W. Norton & Company.
- Berget, S. M., Moore, C., & Sharp, P. A. (1977). Spliced segments at the 5' terminus of adenovirus 2 late mRNA. *Proceedings of the National Academy of Sciences*, 74(8), 3171-3175. <https://doi.org/10.1073/pnas.74.8.3171>
- Blattner, F. R., Plunkett, G., Bloch, C. A., Perna, N. T., Burland, V., Riley, M., Collado-Vides, J., Glasner, J. D., Rode, C. K., Mayhew, G. F., et al. (1997). The Complete Genome Sequence of *Escherichia coli* K-12. *Science*, 277(5331), 1453-1462. <https://doi.org/10.1126/science.277.5331.1453>
- Crick, F. H. C. (1966). Codon-anticodon pairing: The wobble hypothesis. *Journal of Molecular Biology*, 19(2), 548-555. [https://doi.org/10.1016/S0022-2836\(66\)80022-0](https://doi.org/10.1016/S0022-2836(66)80022-0)
- Crick, F. H. C. (1968). The origin of the genetic code. *Journal of Molecular Biology*, 38(3), 367-379. [https://doi.org/10.1016/0022-2836\(68\)90392-6](https://doi.org/10.1016/0022-2836(68)90392-6)
- Crick, F. H. C., Barnett, L., Brenner, S., & Watts-Tobin, R. J. (1961). General Nature of the Genetic Code for Proteins. *Nature*, 192(4809), 1227-1232. <https://doi.org/10.1038/1921227a0>
- Crick, F. (1970). Central Dogma of Molecular Biology. *Nature*, 227(5258), 561-563. <https://doi.org/10.1038/227561a0>
- Fickett, J. W. (1982). Recognition of protein coding regions in DNA sequences. *Nucleic Acids Research*, 10(17), 5303-5318. <https://doi.org/10.1093/nar/10.17.5303>
- Frank, A. C., & Lobry, J. R. (1999). Asymmetric substitution patterns: a review of possible underlying mutational or selective mechanisms. *Gene*, 238(1), 65-77. [https://doi.org/10.1016/S0378-1119\(99\)00297-8](https://doi.org/10.1016/S0378-1119(99)00297-8)
- Fraser, C. M., Gocayne, J. D., White, O., Adams, M. D., Clayton, R. A., Fleischmann, R. D., Bult, C. J., Kerlavage, A. R., Sutton, G., Kelley, J. M., et al. (1995). The Minimal Gene Complement of *Mycoplasma genitalium*. *Science*, 270(5235), 397-404. <https://doi.org/10.1126/science.270.5235.397>

- Frederico, L. A., Kunkel, T. A., & Shaw, B. R. (1990). A sensitive genetic assay for the detection of cytosine deamination: determination of rate constants and the activation energy. *Biochemistry*, 29(10), 2532-2537. <https://doi.org/10.1021/bi00462a015>
- Freeland, S. J., & Hurst, L. D. (1998). The Genetic Code Is One in a Million. *Journal of Molecular Evolution*, 47(3), 238-248. <https://doi.org/10.1007/PL00006381>
- Gilbert, W. (1978). Why genes in pieces? *Nature*, 271(5645), 501. <https://doi.org/10.1038/271501a0>
- Glass, J. I., Assad-Garcia, N., Alperovich, N., Yooseph, S., Lewis, M. R., Maruf, M., Hutchison, C. A., Smith, H. O., & Venter, J. C. (2006). Essential genes of a minimal bacterium. *Proceedings of the National Academy of Sciences*, 103(2), 425-430. <https://doi.org/10.1073/pnas.0510013103>
- Grigoriev, A. (1998). Analyzing genomes with cumulative skew diagrams. *Nucleic Acids Research*, 26(10), 2286-2290. <https://doi.org/10.1093/nar/26.10.2286>
- Haig, D., & Hurst, L. D. (1991). A quantitative measure of error minimization in the genetic code. *Journal of Molecular Evolution*, 33(5), 412-417. <https://doi.org/10.1007/BF02103132>
- Hutchison, C. A., Chuang, R.-Y., Noskov, V. N., Assad-Garcia, N., Deerinck, T. J., Ellisman, M. H., Gill, J., Kannan, K., Karas, B. J., Ma, L., et al. (2016). Design and synthesis of a minimal bacterial genome. *Science*, 351(6280), Artículo aad6253. <https://doi.org/10.1126/science.aad6253>
- Hyatt, D., Chen, G.-L., LoCascio, P. F., Land, M. L., Larimer, F. W., & Hauser, L. J. (2010). Prodigal: prokaryotic gene recognition and translation initiation site identification. *BMC Bioinformatics*, 11, Artículo 119. <https://doi.org/10.1186/1471-2105-11-119>
- Koonin, E. V., & Novozhilov, A. S. (2009). Origin and evolution of the genetic code: The universal enigma. *IUBMB Life*, 61(2), 99-111. <https://doi.org/10.1002/iub.146>
- Kyte, J., & Doolittle, R. F. (1982). A simple method for displaying the hydropathic character of a protein. *Journal of Molecular Biology*, 157(1), 105-132. [https://doi.org/10.1016/0022-2836\(82\)90515-0](https://doi.org/10.1016/0022-2836(82)90515-0)
- Lobry, J. R. (1996). Asymmetric substitution patterns in the two DNA strands of bacteria. *Molecular Biology and Evolution*, 13(5), 660-665. <https://doi.org/10.1093/oxfordjournals.molbev.a025626>
- Nirenberg, M., & Leder, P. (1964). RNA Codewords and Protein Synthesis. *Science*, 145(3639), 1399-1407. <https://doi.org/10.1126/science.145.3639.1399>
- Nirenberg, M. W., & Matthaei, J. H. (1961). The dependence of cell-free protein synthesis in *E. coli* upon naturally occurring or synthetic polyribonucleotides. *Proceedings of the National Academy of Sciences*, 47(10), 1588-1602. <https://doi.org/10.1073/pnas.47.10.1588>
- Okazaki, R., Okazaki, T., Sakabe, K., Sugimoto, K., & Sugino, A. (1968). Mechanism of DNA chain growth. I. Possible discontinuity and unusual secondary structure of newly synthesized chains. *Proceedings of the National Academy of Sciences*, 59(2), 598-605. <https://doi.org/10.1073/pnas.59.2.598>

- Rocha, E. P. C. (2004). The replication-related organization of bacterial genomes. *Microbiology*, 150(6), 1609-1627. <https://doi.org/10.1099/mic.0.26974-0>
- Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3), 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Shine, J., & Dalgarno, L. (1974). The 3'-Terminal Sequence of *Escherichia coli* 16S Ribosomal RNA: Complementarity to Nonsense Triplets and Ribosome Binding Sites. *Proceedings of the National Academy of Sciences*, 71(4), 1342-1346. <https://doi.org/10.1073/pnas.71.4.1342>
- Temin, H. M., & Mizutani, S. (1970). Viral RNA-dependent DNA Polymerase: RNA-dependent DNA Polymerase in Virions of Rous Sarcoma Virus. *Nature*, 226(5252), 1211-1213. <https://doi.org/10.1038/2261211a0>
- Watson, J. D., & Crick, F. H. C. (1953). Molecular Structure of Nucleic Acids: A Structure for Deoxyribose Nucleic Acid. *Nature*, 171(4356), 737-738. <https://doi.org/10.1038/171737a0>
- Yamano, F., Muto, A., Kawauchi, Y., Iwami, M., Iwagami, S., Azumi, Y., & Osawa, S. (1985). UGA is read as tryptophan in *Mycoplasma capricolum*. *Proceedings of the National Academy of Sciences*, 82(8), 2306-2309. <https://doi.org/10.1073/pnas.82.8.2306>

Secuencias, formatos y bases de datos



La buena gestión de los datos no es un fin en sí misma, sino el conducto clave que lleva al descubrimiento y la innovación, y a la posterior integración y reutilización de datos y conocimiento por parte de la comunidad.

— Wilkinson et al. (2016)

● Al terminar este capítulo usted podrá

- Reconocer la pregunta que responde cada formato (FASTA, FASTQ, GenBank, GFF3/GTF, BED, SAM/BAM/CRAM y VCF) y ubicarlo en el flujo de un análisis genómico.
- Convertir entre calidades Phred, probabilidades de error y caracteres ASCII, y calcular los errores esperados de una lectura.
- Traducir sin errores entre coordenadas 1-based cerradas y 0-based semiabiertas, en ambas hebras.
- Interpretar una CIGAR, descomponer una FLAG bit a bit y explicar cómo BGZF y los índices permiten el acceso aleatorio a archivos de cientos de gigabytes.

- Consultar NCBI, Ensembl, UniProt y PDB desde Python mediante APIs REST, respetando límites de tasa, registrando versiones e identificadores estables y aplicando los principios FAIR.

Una secuencia de ADN aislada es apenas una cadena de letras. Para convertirla en un *dato científico* necesitamos saber de qué organismo procede, con qué confianza se leyó cada base, dónde se encuentra en el genoma, qué genes ocupa y cómo difiere de la referencia. Esa información se distribuye entre varios formatos de archivo y bases de datos públicas. Conocerlos evita perder horas, o incluso semanas, por errores ajenos a la biología: una coordenada desplazada, una calidad mal decodificada o un identificador sin versión.

El capítulo se divide en dos partes. Primero estudiaremos los formatos como *contratos* entre programas: tienen una gramática precisa y recurren a logaritmos para expresar calidades, intervalos para fijar coordenadas, bits para codificar banderas e índices para acceder a archivos grandes (sección 2.1). Después seguiremos al gen humano *TP53*, el “guardián del genoma”, por NCBI, Ensembl, UniProt y PDB. Así practicaremos consultas reproducibles y trazables que respeten los límites de los servidores (sección 2.2).

2.1 El idioma de los archivos

 Colab · 2.1 · Formatos de archivo

 [abrir notebook](#)

Una cocina profesional

En la cocina de un buen restaurante, la receta está escrita en un cuaderno, los ingredientes llegan en cajas etiquetadas con fecha de vencimiento y grado de frescura, el plan de la noche está en una pizarra y los comentarios de los comensales se anotan en un libro aparte. Nadie mezclaría todo en una sola hoja: cada soporte responde a una pregunta distinta y lo leen personas distintas. Pero todos comparten un vocabulario (el nombre del plato, el número de mesa) que permite cruzarlos. Los formatos bioinformáticos funcionan igual: uno guarda la receta (la secuencia de referencia), otro los ingredientes con su frescura (las lecturas y su calidad), otro el plan (las anotaciones), otro dónde

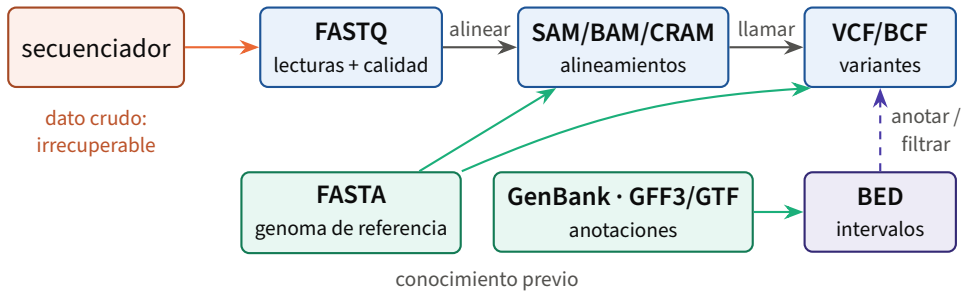


Figura 2.1. La vida de los datos en un análisis de resecuenciación. En azul, los archivos que el propio análisis produce, de izquierda a derecha cada vez más resumidos: de cientos de gigabytes de lecturas a unos pocos megabytes de variantes. En verde, el conocimiento previo (la referencia y sus anotaciones), que se descarga de las bases de datos de la sección 2.2. BED es el formato “comodín” para describir regiones con las que filtrar o anotar. Todo el sistema depende de que los nombres de cromosoma y las coordenadas signifiquen lo mismo en cada archivo.

terminó cada ingrediente (los alineamientos) y otro las quejas (las variantes). El vocabulario común son los nombres de cromosomas y las coordenadas.

2.1.1 Por qué hay tantos formatos

La figura 2.1 sigue los datos de un análisis típico de resecuenciación. El FASTQ contiene las lecturas procedentes del secuenciador; los demás archivos incorporan cálculos posteriores (alineamiento, comparación o anotación) o conocimiento previo (el genoma de referencia y sus genes). Esto sugiere una regla práctica: los archivos intermedios pueden regenerarse, pero las lecturas crudas deben conservarse. Para ello existen repositorios públicos como el *Sequence Read Archive* (Leinonen et al., 2011).

Muchos de estos formatos son *texto delimitado*: una línea por registro, campos separados por tabuladores y encabezados reconocibles por un carácter especial. Los archivos ocupan más espacio, pero se pueden inspeccionar con `head`, filtrar con `grep` y procesar con `awk`, sin herramientas especializadas, como señala Buffalo (2015). Para datos de mayor tamaño existen versiones binarias comprimidas e indexadas, como BAM para SAM y BCF para VCF. HTSlib implementa su lectura y escritura (Bonfield et al., 2021).

2.1.2 FASTA: la secuencia desnuda

El formato FASTA nació como el formato de entrada del programa de búsqueda de similitud del mismo nombre (Lipman & Pearson, 1985; Pearson & Lipman, 1988), y sobrevivió al programa porque es casi imposible de simplificar más:

```
genoma.fasta
```

```
>NC_045512.2 Severe acute respiratory syndrome coronavirus 2
ATTAAAGGTTTATACCTTCCCAGGTAACAAACCAACCAACTTCGATCTCTTGTAGATCT
GTTCTCTAAACGAACCTTTAAATCTGTGTGGCTGCTCACTCGGCTGCATGCTTAGTGCACT
```

Definición 2.1. (Registro FASTA)

Un archivo FASTA es una sucesión de registros. Cada registro comienza con una línea de encabezado que empieza por >; la primera palabra tras el > (hasta el primer espacio) es el *identificador* y el resto es una descripción libre. Las líneas siguientes, hasta el próximo > o el fin del archivo, se concatenan para formar la secuencia, escrita con el alfabeto IUPAC de nucleótidos o de aminoácidos.

Dos detalles de esta definición causan más errores de los que parece. El primero es que muchas herramientas cortan el identificador en el primer espacio: si el encabezado es >chr1 Homo sapiens, el cromosoma se llamará chr1 en el BAM, y un archivo de anotaciones que diga 1 (la convención de Ensembl) no casará con él. El segundo es que el FASTA no tiene un ancho de línea obligatorio, pero los índices (.faí) que permiten extraer una región sin leer el archivo completo sí exigen que todas las líneas de un registro tengan la misma longitud. Con líneas de w caracteres, la base x (o-based) de un registro cuya secuencia empieza en el byte o se encuentra en el byte

$$b(x) = o + \left\lfloor \frac{x}{w} \right\rfloor (w + 1) + (x \bmod w), \quad (2.1)$$

DONDE

$b(x)$	Posición en bytes, dentro del archivo, del carácter que codifica la base x .
o	Desplazamiento (en bytes) donde empieza la primera línea de secuencia del registro; se guarda en el <code>.fa.i</code> .
w	Número de bases por línea; el <code>+1</code> cuenta el salto de línea <code>\n</code> .
x	Posición de la base dentro de la secuencia, contando desde o .

Es decir: saltamos $\lfloor x/w \rfloor$ líneas completas, cada una de $w + 1$ bytes, y avanzamos $x \bmod w$ caracteres dentro de la última. Si una sola línea tiene otra longitud, la fórmula apunta al lugar equivocado; por eso `samtools faidx` se niega a indexar archivos irregulares. El genoma de SARS-CoV-2 (29 903 nt) escrito con $w = 60$ ocupa $\lceil 29\,903/60 \rceil = 499$ líneas de secuencia.

2.1.3 FASTQ: la secuencia con su nivel de confianza

Un secuenciador no solo dice *qué* base leyó, sino *cuán seguro está*. Esa segunda información es la que nos permite, más adelante, distinguir una mutación real de un error de la máquina. El formato FASTQ, documentado formalmente por Cock et al. (2010) mucho después de volverse ubicuo, guarda cada lectura en exactamente cuatro líneas (figura 2.2): el nombre precedido de `@`, la secuencia, un separador `+` y una cadena de calidades con *un carácter por base*.

La escala Phred

La calidad no se guarda como probabilidad, sino en una escala logarítmica introducida por el programa `phred`, que asignaba bases y calidades a las trazas de los secuenciadores capilares del Proyecto Genoma Humano (Ewing & Green, 1998; Ewing et al., 1998).

Definición 2.2. (Calidad Phred)

Si p es la probabilidad de que la base asignada sea incorrecta, su calidad Phred es

$$Q = -10 \log_{10} p \quad \Longleftrightarrow \quad p = 10^{-Q/10}. \quad (2.2)$$

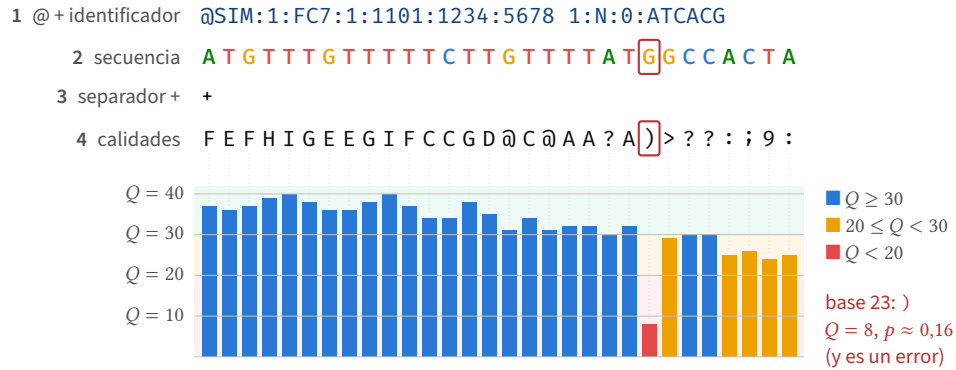


Figura 2.2. Anatomía de un registro FASTQ: las primeras 30 bases del gen S de SARS-CoV-2 leídas por un secuenciador simulado. Cada carácter de la línea 4 codifica la calidad de la base que tiene encima; las barras muestran el valor Phred decodificado ($Q = \text{código ASCII} - 33$). La calidad cae lentamente a lo largo de la lectura, como en un Illumina real. La base 23 tiene $Q = 8$ (carácter `)`: el secuenciador “avisa” que se equivoca con probabilidad $10^{-0,8} \approx 0,16$, y en efecto leyó G donde la referencia tiene T. Generada con `figuras/cap02/generar.py`.

DONDE	
p	Probabilidad estimada de error de la base (entre 0 y 1).
Q	Calidad Phred; se redondea a un entero. Más alta significa más confiable.
-10	Factor de escala: 10 unidades de Q equivalen a un factor 10 en p (como los decibelios).

¿Por qué un logaritmo? Porque lo que importa de una probabilidad de error es su *orden de magnitud*. Pasar de $p = 10^{-2}$ a $p = 10^{-3}$ es una mejora tan importante como pasar de 10^{-3} a 10^{-4} , y la escala Phred convierte ambas en el mismo salto de 10 puntos. Así, $Q = 20$ significa un error en 100 bases, $Q = 30$ uno en mil y $Q = 40$ uno en diez mil. Hay además una razón estadística más profunda: $-\log p$ es la *sorpresa* (en unidades de información) de observar un error, de modo que las calidades de bases independientes se combinan sumando logaritmos, como las puntuaciones de alineamiento del capítulo 3.

Ewing y Green (1998) no se limitaron a definir la escala: la *calibraron*. Compararon millones de bases asignadas por `phred` con secuencias de referencia de alta calidad y verificaron que, entre todas las bases con calidad Q , la fracción de errores observada era, en efecto, cercana a $10^{-Q/10}$. Esta calibración es la que da sentido a todo lo

que sigue: una calidad Phred no es un “puntaje” arbitrario, sino una predicción probabilística verificable.

Solexa: la otra escala.. Los primeros secuenciadores Solexa (luego Illumina) usaron una escala distinta, basada en la razón de probabilidades (*odds*) en lugar de la probabilidad (Cock et al., 2010):

$$Q_{\text{Sol}} = -10 \log_{10} \frac{p}{1-p}. \quad (2.3)$$

donde Q_{Sol} puede ser negativa (hasta -5 en la práctica). Despejando p en ambas definiciones y eliminándola se obtiene la conversión exacta

$$Q_{\text{Phred}} = 10 \log_{10} (10^{Q_{\text{Sol}}/10} + 1). \quad (2.4)$$

DONDE

Q_{Sol} Calidad Solexa; $p/(1-p)$ es la razón de probabilidades de error frente a acierto (*odds*).

Q_{Phred} Calidad Phred equivalente (ecuación (2.2)); siempre ≥ 0 .

Para calidades altas, $p \ll 1$ y $p/(1-p) \approx p$, de modo que ambas escalas coinciden: $Q_{\text{Phred}} = 20$ corresponde a $Q_{\text{Sol}} = 10 \log_{10} 99 = 19,96$. Solo divergen cuando la base es muy dudosa (figura 2.3): $Q_{\text{Phred}} = 3$ equivale a $Q_{\text{Sol}} \approx 0$, y el mínimo Solexa, -5 , corresponde a $p = 10^{0,5}/(1 + 10^{0,5}) \approx 0,76$, una base peor que una elección al azar entre cuatro.

De número a carácter

Para que la línea de calidades tenga la misma longitud que la secuencia, cada Q se escribe como un único carácter ASCII. Los primeros 32 códigos ASCII son invisibles (tabuladores, saltos de línea), así que se suma un desplazamiento: el estándar actual, heredado del Instituto Sanger, es *Phred+33*, de modo que $Q = 0$ se escribe ! (código 33), $Q = 30$ se escribe ? (63) y $Q = 40$ se escribe I (73). La figura 2.4 muestra la tabla completa para el rango usual.

⚠ Adivinar la codificación

El FASTQ no declara su codificación. Los archivos de Illumina anteriores a 2011 (versiones 1.3 a 1.7 de su programa) usan *Phred+64*, y todavía circulan en repositorios. Si se lee un archivo *Phred+64* como *Phred+33*, todas las calidades

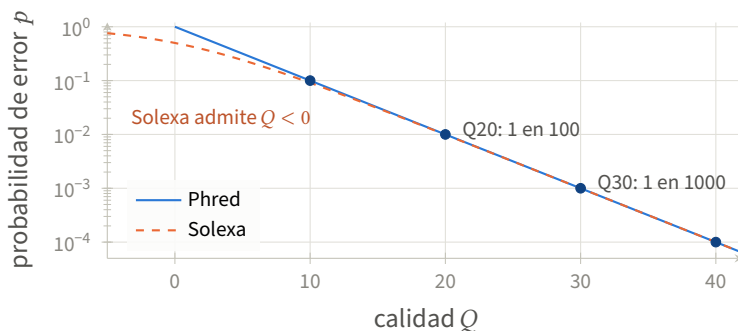


Figura 2.3. La ecuación (2.2) es una recta en escala logarítmica; cada 10 puntos de Q la probabilidad de error se divide entre 10. La escala Solexa (ecuación (2.3)) se superpone a Phred para calidades altas ($Q \geq 15$) y solo diverge cuando p se acerca a 1, donde admite valores negativos. Por eso confundir las escalas apenas afecta a las bases buenas, pero sí a las dudosas, precisamente las que un filtro de calidad debe descartar.

aparecen infladas en 31 puntos: una base con $p = 0,1$ parece tener $p \approx 10^{-4}$, y ningún filtro de calidad la eliminará. Antes de analizar datos antiguos, compruebe el rango de caracteres presentes (figura 2.4); herramientas como FastQC lo hacen automáticamente.

Errores esperados de una lectura

¿Cuántos errores tiene una lectura? No lo sabemos, pero podemos calcular cuántos *esperamos*. Sea X_i la variable indicadora de que la base i es errónea; entonces $E[X_i] = p_i$ y, por la linealidad de la esperanza (que no exige independencia),

$$EE = E\left[\sum_{i=1}^{\ell} X_i\right] = \sum_{i=1}^{\ell} p_i = \sum_{i=1}^{\ell} 10^{-Q_i/10}. \quad (2.5)$$

Si además suponemos que los errores ocurren de forma independiente, la probabilidad de que la lectura esté completamente libre de errores es

$$P(\text{sin errores}) = \prod_{i=1}^{\ell} (1 - p_i) \approx e^{-EE}, \quad (2.6)$$

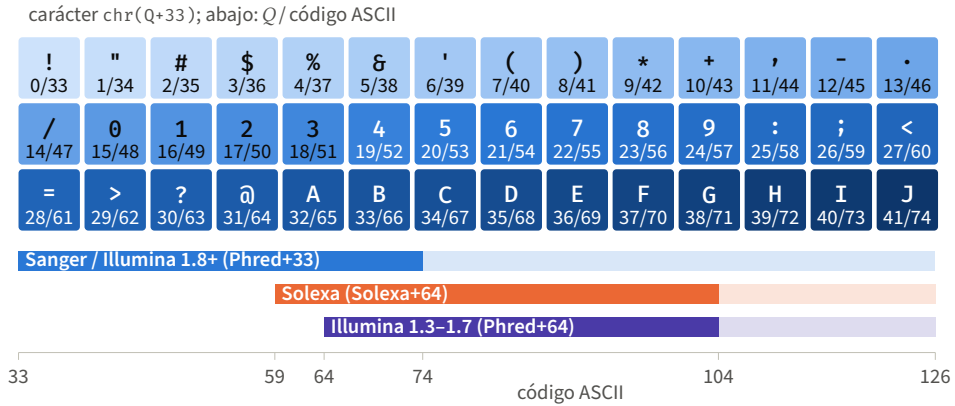


Figura 2.4. Arriba: la codificación Phred+33, carácter por carácter, coloreada desde calidades bajas (claro) hasta altas (oscuro). Abajo: los tres “dialectos” históricos de FASTQ (Cock et al., 2010). La banda clara es el rango permitido por cada codificación; la banda oscura, el rango que aparece en la práctica. Los rangos típicos se solapan entre los códigos 64 y 74 (@–J): un archivo cuyas calidades caen todas ahí es ambiguo, pero basta un solo carácter por debajo de ; (código 59) para concluir que es Phred+33.

DONDE

ℓ	Longitud de la lectura (número de bases).
X_i	Indicadora del evento “la base i es incorrecta” (vale 0 o 1).
p_i, Q_i	Probabilidad de error y calidad Phred de la base i .
EE	Errores esperados (<i>expected errors</i>) de la lectura.

donde la aproximación usa $\ln(1 - p) \approx -p$ para p pequeño: el número de errores es aproximadamente una variable de Poisson de media EE. Esta cantidad resume una lectura mucho mejor que la calidad *media*, porque la media de Q es la media de logaritmos y esconde las pocas bases malas que dominan la suma de probabilidades.

Ejemplo 2.1. (Errores esperados, a mano y en la figura 2.2)

Una lectura corta tiene calidades $Q = (40, 30, 20, 20, 10)$. Sus probabilidades de error son $(10^{-4}, 10^{-3}, 10^{-2}, 10^{-2}, 10^{-1})$ y

$$EE = 0,0001 + 0,001 + 0,01 + 0,01 + 0,1 = 0,1211.$$

Casi todo el valor procede de la única base con $Q = 10$. Su calidad media es 24, que “parece” buena, y sin embargo la probabilidad de que la lectura sea perfecta es solo $\prod(1 - p_i) = 0,8811$ (la aproximación $e^{-0,1211} = 0,8859$ es buena).

En la lectura de 30 bases de la figura 2.2 la calidad media es 32,5, pero $EE = 0,183$ y $P(\text{sin errores}) = 0,821$: la base 23 ($Q = 8$, $p = 0,158$) aporta por sí sola el 87 % de los errores esperados.

La figura 2.5 aplica estas ideas a una corrida simulada de 4000 lecturas de 150 nt, generada con el mismo modelo del notebook: una calidad media que cae de 36,9 en el primer ciclo a 24,1 en el último, más un 6 % de *clusters* defectuosos cuya calidad se desploma en la cola. Los errores se introdujeron exactamente con la probabilidad que dicta su calidad, así que conocemos la verdad: la tasa real de error resultó ser de 0,20 % (0,304 errores por lectura) y la media de EE, 0,296. El panel derecho muestra que los errores esperados predicen los observados, lo que solo ocurre si las calidades están bien calibradas. En datos reales no siempre es así, y por eso existen métodos de recalibración de calidades que estudiaremos junto con la llamada de variantes.

```
fastq_ee.py

import math
from Bio import SeqIO                                     # Biopython >= 1.83

def expected_errors(quals):
    return sum(10 ** (-q / 10) for q in quals)

kept = []
for rec in SeqIO.parse("lecturas.fastq", "fastq"):      # Phred+33
    q = rec.letter_annotations["phred_quality"]
    ee = expected_errors(q)
    if ee <= 1.0:                                       # filtro por EE
        kept.append(rec)
    p0 = math.prod(1 - 10 ** (-x / 10) for x in q)
print(len(kept), "lecturas con EE <= 1")
SeqIO.write(kept, "filtradas.fastq", "fastq")
```

Biopython (Cock et al., 2009) decodifica las calidades según el nombre del formato: "fastq" (o "fastq-sanger") para Phred+33, "fastq-illumina" para Phred+64 y "fastq-solexa" para Solexa+64, y convierte entre ellos con la ecuación (2.4) al escribir.

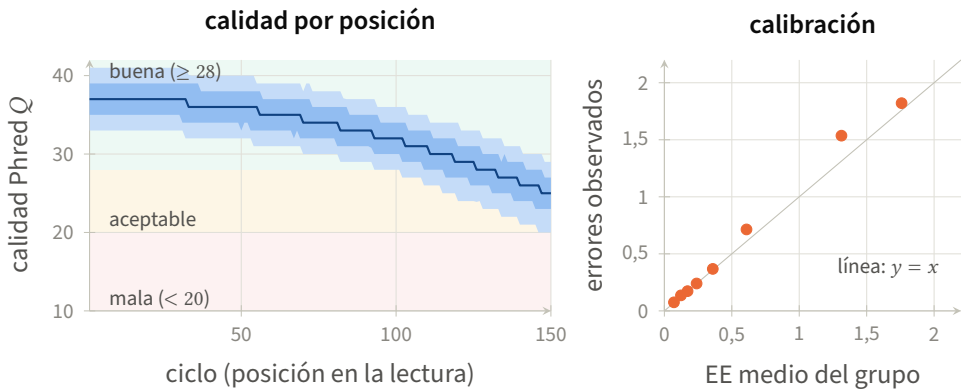


Figura 2.5. Izquierda: calidad por ciclo de 4000 lecturas simuladas, al estilo de FastQC. La línea es la mediana; la banda oscura contiene el 50 % central de las lecturas y la clara el 80 %. La caída hacia el final refleja el desfase progresivo de las moléculas dentro de cada *cluster*; el ensanchamiento de la banda inferior se debe a los *clusters* defectuosos. Esta forma es la que justifica recortar las colas antes de alinear. Derecha: las lecturas se agrupan por su EE (ecuación (2.5)) y se compara el EE medio de cada grupo con el número medio de errores realmente introducidos. Los puntos caen sobre la diagonal: con calidades calibradas, EE es un predictor insesgado del número de errores.

2.1.4 GenBank y el sistema INSDC

Un FASTA dice *qué* es la secuencia; un registro GenBank dice además *qué hay en ella*. El formato, con su encabezado (LOCUS, DEFINITION, ACCESSION, VERSION, SOURCE, referencias bibliográficas), su tabla de características (FEATURES: genes, CDS, UTR, péptidos maduros, con sus coordenadas y calificadores) y la secuencia al final (ORIGIN), es el formato nativo de GenBank, la base de datos pública de secuencias del NCBI (Sayers, Cavanaugh et al., 2025). GenBank intercambia sus datos diariamente con el *European Nucleotide Archive* (ENA) y con el *DNA Data Bank of Japan* (DDBJ); las tres forman la *International Nucleotide Sequence Database Collaboration* (INSDC), de modo que una secuencia depositada en cualquiera de ellas aparece en las tres con el mismo número de acceso (Arita et al., 2021).

fragmento de NC_045512.2.gb

```
gene          21563..25384
              /gene="S"
CDS           21563..25384
```

```

/gene="S"
/codon_start=1
/product="surface glycoprotein"
/protein_id="YP_009724390.1"

```

Las coordenadas de GenBank son *1-based e inclusivas*: el gen S ocupa las bases 21563 a 25384, ambas incluidas, es decir, $25384 - 21563 + 1 = 3822$ nucleótidos (1273 codones más el de terminación). Las localizaciones admiten una pequeña gramática: `complement(100..200)` para la hebra negativa, `join(1..50,80..120)` para exones separados, `<1..300` para un extremo incompleto. Biopython la traduce a objetos `SimpleLocation` y `CompoundLocation`, que, como veremos enseguida, usan internamente coordenadas *o-based*.

2.1.5 Coordenadas: el error de uno

Aquí está la fuente de errores más famosa de la bioinformática. Existen dos convenciones para numerar posiciones en una secuencia, y ambas son correctas a su manera (figura 2.6).

Definición 2.3. (Sistemas de coordenadas)

En el sistema *1-based cerrado*, las posiciones numeran *bases*: la primera base es la 1 y un intervalo $[s_1, e_1]$ incluye ambos extremos. Lo usan GenBank, GFF3/GTF, SAM (columna POS), VCF y los navegadores genómicos. En el sistema *o-based semiabierto*, las posiciones numeran los *bordes entre bases*, como las marcas de una regla: el primer borde es el 0 y un intervalo $[s_0, e_0)$ va del borde s_0 al borde e_0 , incluyendo la base que sigue a s_0 pero no la que sigue a e_0 . Lo usan BED, el formato binario BAM, Python (`seq[s:e]`) y la mayoría de las bibliotecas.

La conversión entre ambos sistemas se deduce de la figura. La base número s_1 (1-based) está a la derecha del borde $s_1 - 1$, y la última base del intervalo, e_1 , termina en el borde e_1 :

$$s_0 = s_1 - 1, \quad e_0 = e_1, \quad \text{longitud} = e_0 - s_0 = e_1 - s_1 + 1. \quad (2.7)$$

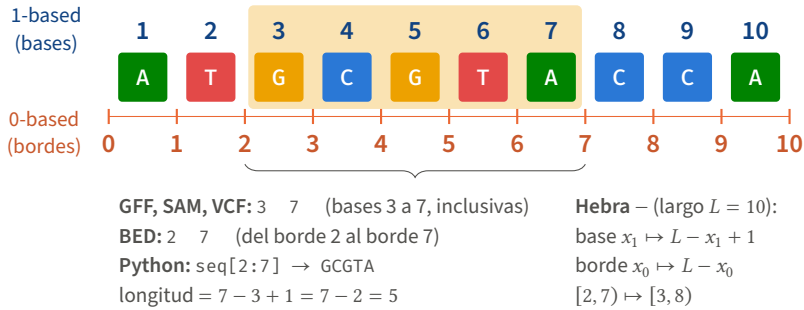


Figura 2.6. Las dos convenciones de coordenadas sobre la misma secuencia. Arriba, en azul, se numeran las bases (1-based); abajo, en naranja, los bordes entre bases, como en una regla (0-based). El intervalo resaltado, GCGTA, se escribe 3 7 en GFF y 2 7 en BED: el inicio cambia en una unidad, el final no. A la derecha, la regla para pasar a la hebra complementaria: en el sistema de bordes basta reflejar ($x_0 \mapsto L - x_0$) e intercambiar los extremos.

DONDE

s_1, e_1	Primera y última base del intervalo en el sistema 1-based cerrado (GFF, VCF, SAM).
s_0, e_0	Bordes inicial y final del intervalo en el sistema 0-based semiabierto (BED, Python).

¿Por qué los programadores prefieren el sistema semiabierto? Porque su álgebra es más limpia. La longitud es una resta directa, sin el +1 que tantas veces se olvida; dos intervalos contiguos comparten el borde ($[0, 5)$ y $[5, 10)$ ni se solapan ni dejan hueco); un intervalo vacío se representa naturalmente como $[s, s)$, lo que permite describir el *punto* donde ocurre una inserción; y la condición de solapamiento es simétrica:

$$[a, b) \cap [c, d) \neq \emptyset \iff \max(a, c) < \min(b, d), \quad (2.8)$$

$$|[a, b) \cap [c, d)| = \max(0, \min(b, d) - \max(a, c)). \quad (2.9)$$

DONDE

$[a, b), [c, d)$	Dos intervalos semiabiertos en el mismo cromosoma.
$ \cdot $	Longitud (número de bases) de la intersección.

Esta es la operación elemental de herramientas como BEDTools (Quinlan & Hall, 2010), que responden preguntas como “¿qué variantes caen en exones?” o “¿qué

picos de ChIP-seq solapan promotores?”. Con un índice ordenado por inicio, la búsqueda de todos los intervalos que solapan una consulta cuesta $O(\log n + k)$ en lugar de $O(n)$, donde k es el número de resultados.

Finalmente, la hebra negativa. Si una secuencia tiene longitud L , el borde x_0 de la hebra positiva corresponde al borde $L - x_0$ de su reverso complementario, de modo que $[s_0, e_0) \mapsto [L - e_0, L - s_0)$. En el sistema 1-based, la base x_1 corresponde a la base $L - x_1 + 1$. Los formatos de anotación *no* aplican esa transformación: GFF, BED y VCF expresan siempre las coordenadas sobre la hebra positiva y guardan la hebra en una columna aparte (+/-). Para obtener la secuencia de un gen en la hebra negativa se extrae el intervalo de la hebra positiva y luego se toma su reverso complementario.

Ejemplo 2.2. (El gen S en tres sistemas)

GenBank dice que el gen S de SARS-CoV-2 ocupa 21563..25384. En BED se escribe NC_045512.2 21562 25384, y en Python la secuencia es `ref[21562:25384]`, cuyas tres primeras letras son **ATG**. Si por descuido escribimos `ref[21563:25384]`, obtenemos una secuencia que empieza en **TGT**: el marco de lectura se desplaza y la traducción es basura. La longitud es $25384 - 21562 = 3822$ en ambos sistemas.

En la otra dirección, el servicio esummary del NCBI informa para *TP53* humano (hebra negativa del cromosoma 17) `chrstart = 7687489` y `chrstop = 7668420`, en coordenadas 0-based y en el sentido de lectura del gen. Sumando 1 a cada extremo y ordenándolos se obtiene 7 668 421–7 687 490, que es exactamente lo que reporta Ensembl, en 1-based, para el transcrito canónico (sección 2.2).

⚠ Tres síntomas del error de uno

1. *Traducciones sin sentido*: una proteína llena de codones de parada suele indicar un marco desplazado en una base.
2. *Alelos de referencia que no coinciden*: en un VCF correcto, la columna REF debe ser igual a la base de la referencia en POS. Si casi ninguna coincide, alguien convirtió mal las coordenadas. Es la prueba más barata y eficaz que existe.

3. *Intervalos de longitud cero o negativa*: un BED con `start = end` tras una conversión suele venir de restar 1 al final en lugar de al inicio.

2.1.6 GFF3, GTF y BED: anotaciones e intervalos

El GenBank es riquísimo, pero su estructura depende de sangrías y es incómodo de procesar. Para intercambiar anotaciones se usa el *General Feature Format*, cuya versión 3 (GFF3) es un archivo tabular de nueve columnas: `seqid`, `source`, `type`, `start`, `end`, `score`, `strand`, `phase` y `attributes`. La columna `type` no es texto libre: debe ser un término de la *Sequence Ontology* (`gene`, `mRNA`, `exon`, `CDS`, `five_prime_UTR`...), un vocabulario controlado creado precisamente para que las anotaciones de distintos proyectos significaran lo mismo (Eilbeck et al., 2005). Los atributos `ID` y `Parent` enlazan las filas y forman un árbol: un `gen` es padre de sus transcritos, cada transcrito es padre de sus exones y `CDS`.

`sars.gff3`

```
##gff-version 3
NC_045512.2 RefSeq gene 21563 25384 . + . ID=gene-S;Name=S
NC_045512.2 RefSeq CDS 21563 25384 . + 0 ID=cds-S;Parent=gene-S
```

El GTF (*Gene Transfer Format*, a veces llamado GFF2.5) usa las mismas nueve columnas pero con atributos obligatorios `gene_id` y `transcript_id` y una sintaxis distinta (`gene_id "TP53"`;). Ensembl y GENCODE lo distribuyen porque es la entrada estándar de los programas de cuantificación de RNA-seq.

La columna `phase` merece una explicación, porque es la menos intuitiva. Un `CDS` partido en exones no tiene por qué empezar cada fragmento al inicio de un codón. La fase de un fragmento es el número de bases que hay que saltar desde su inicio para llegar al primer codón completo. Si antes de ese fragmento ya se recorrieron d bases codificantes,

$$\phi = (3 - d \bmod 3) \bmod 3 \in \{0, 1, 2\}. \quad (2.10)$$

DONDE

ϕ	Fase del fragmento de CDS (columna 8 del GFF3).
d	Número de bases codificantes contenidas en los fragmentos anteriores, en el sentido de la traducción.

Por ejemplo, si el primer exón codificante tiene 100 bases, $d = 100$, $100 \bmod 3 = 1$ y el segundo fragmento tiene fase $\phi = 2$: sus dos primeras bases completan el codón que quedó abierto.

El formato BED, creado para el navegador genómico de la Universidad de California en Santa Cruz (Kent et al., 2002), es aún más sencillo: tres columnas obligatorias (chrom, chromStart, chromEnd) en coordenadas o-based semiabiertas, y hasta nueve opcionales (nombre, puntuación, hebra, colores, bloques). Es el formato de las *regiones*: exones de un panel de captura, picos de ChIP-seq, regiones de baja complejidad que se deben excluir. Convertir de GFF3 a BED consiste, sencillamente, en aplicar la ecuación (2.7): restar 1 al inicio y dejar el final.

2.1.7 SAM, BAM y CRAM: dónde cayó cada lectura

Tras la secuenciación, un alineador (capítulo 3) busca en qué lugar de la referencia encaja cada lectura. El resultado se escribe en el formato *Sequence Alignment/Map* (SAM), propuesto por Li et al. (2009) en el contexto del Proyecto 1000 Genomas para poner fin a la proliferación de formatos propios de cada alineador. Un SAM tiene un encabezado (líneas que empiezan con @: @SQ para cada secuencia de referencia con su longitud, @RG para los grupos de lecturas, @PG para los programas que lo procesaron) y una fila por alineamiento con once campos obligatorios, resumidos en la tabla 2.1.

La columna MAPQ reutiliza la escala Phred para otra probabilidad, la de que la *posición* sea incorrecta: $\text{MAPQ} = -10 \log_{10} P(\text{posición errónea})$. Un MAPQ de 60 corresponde a una probabilidad de error de 10^{-6} ; un MAPQ de 0 significa que la lectura encaja igual de bien en al menos otro lugar del genoma (una repetición), y no debe usarse para llamar variantes. Cada alineador estima esta probabilidad a su manera, por lo que los valores de MAPQ no son comparables entre programas.

La CIGAR: instrucciones para colocar la lectura

La CIGAR (*Compact Idiosyncratic Gapped Alignment Report*) codifica el alineamiento como una sucesión de pares (n_k, o_k) : una longitud y una operación. La clave para leerla es saber qué operaciones *consumen* bases de la lectura, de la referencia o de ambas (tabla 2.2).

Tabla 2.1. Los once campos obligatorios de una línea de alineamiento SAM.

#	Campo	Tipo	Ejemplo	Significado
1	QNAME	texto	r2	nombre de la lectura (el mismo para las dos del par)
2	FLAG	entero	99	doce banderas codificadas en bits (figura 2.8)
3	RNAME	texto	NC_045512.2	secuencia de referencia
4	POS	entero	21608	primera base alineada, <i>1-based</i>
5	MAPQ	entero	60	confianza en la posición, en escala Phred
6	CIGAR	texto	3S9M2I8M	cómo encaja la lectura (figura 2.7)
7-9	RNEXT, PNEXT, TLEN		= 21790 332	posición de la pareja y longitud del fragmento
10	SEQ	texto	TGTTT...	bases, en la orientación de la referencia
11	QUAL	texto	FEFHI...	calidades Phred+33

Teorema 2.1. (Longitudes implícitas en una CIGAR)

Sea una CIGAR $C = (n_1, o_1) \cdots (n_K, o_K)$. La longitud de la lectura almacenada en SEQ, el tramo de referencia cubierto y la última posición alineada son

$$\ell_q = \sum_{k: o_k \in \{M, I, S, =, X\}} n_k, \quad (2.11)$$

$$\ell_r = \sum_{k: o_k \in \{M, D, N, =, X\}} n_k, \quad (2.12)$$

$$\text{END} = \text{POS} + \ell_r - 1. \quad (2.13)$$

Tabla 2.2. Operaciones de la CIGAR y qué secuencia consume cada una.

Op.	Nombre	Lect.	Ref.	Uso típico
M	alineada (coincida o no)	sí	sí	base frente a base
= / X	coincidencia / sustitución	sí	sí	versión explícita de M
I	inserción	sí	no	bases que la referencia no tiene
D	delección	no	sí	bases de la referencia ausentes en la lectura
N	salto	no	sí	intrón en RNA-seq
S	recorte suave	sí	no	extremo no alineado, presente en SEQ
H	recorte duro	no	no	extremo no alineado, ausente de SEQ

DONDE

n_k, o_k	Longitud y operación del k -ésimo elemento de la CIGAR.
ℓ_q	Número de bases de la lectura guardadas en SEQ; debe coincidir con la longitud de SEQ.
ℓ_r	Número de bases de la referencia que abarca el alineamiento.
POS, END	Primera y última posición de la referencia cubiertas (1-based, inclusivas).

La demostración es inmediata: al recorrer la CIGAR, un puntero avanza sobre la lectura y otro sobre la referencia, cada uno según la tabla. Lo importante es la consecuencia práctica: los recortes suaves *no* desplazan POS, que siempre apunta a la primera base *alineada*. En o-based semiabierto, el alineamiento ocupa $[\text{POS} - 1, \text{POS} - 1 + \ell_r]$, que es lo que devuelve `pysam` como `reference_start` y `reference_end`.

Ejemplo 2.3. (Leer una CIGAR)

Una lectura tiene $\text{POS} = 1000$ y $\text{CIGAR} = 3\text{S}40\text{M}1\text{I}20\text{M}5\text{D}30\text{M}2\text{S}$. Según las ecuaciones (2.11) y (2.12),

$$\ell_q = 3 + 40 + 1 + 20 + 30 + 2 = 96, \quad \ell_r = 40 + 20 + 5 + 30 = 95,$$

de modo que SEQ debe tener 96 caracteres y el alineamiento termina en $\text{END} = 1000 + 95 - 1 = 1094$. Las bases 1 a 3 de la lectura (recorte suave) no están alineadas; la base 4 de la lectura está frente a la posición 1000 de la

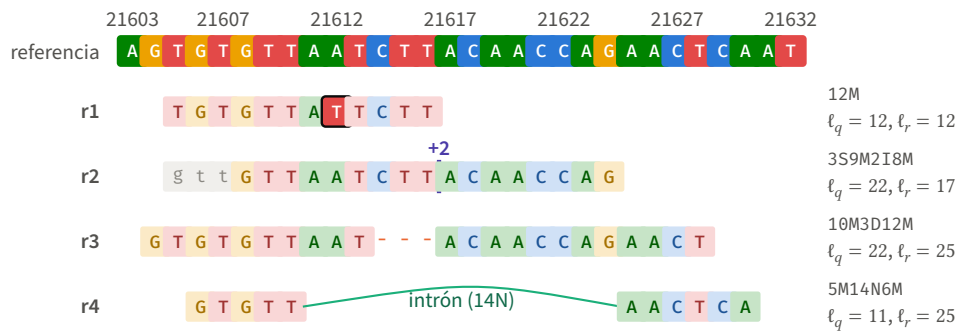


Figura 2.7. Cuatro lecturas alineadas contra el inicio del gen S de SARS-CoV-2 (posiciones 1-based arriba). **r1**: coincidencia completa con una sustitución (recuadro), que la CIGAR M no distingue. **r2**: tres bases recortadas (en minúscula, a la izquierda de POS), luego una inserción de dos bases, marcada entre dos posiciones porque no ocupa lugar en la referencia; la lectura mide 22 bases pero cubre solo 17 de la referencia. **r3**: una delección de tres bases, que sí ocupa referencia. **r4**: una lectura de RNA-seq que salta un intrón de 14 bases (N). A la derecha, ℓ_q y ℓ_r según las ecuaciones (2.11) y (2.12).

referencia; la inserción aparece entre las posiciones 1039 y 1040; la delección elimina las posiciones 1060 a 1064.

La FLAG: doce preguntas en un número

La columna FLAG guarda doce respuestas de sí o no en un único entero. Cada respuesta es un bit $b_k \in \{0, 1\}$ y el número es

$$F = \sum_{k=0}^{11} b_k 2^k, \quad b_k = \left\lfloor F/2^k \right\rfloor \text{ mód } 2. \tag{2.14}$$

DONDE	
F	Valor de la columna FLAG (entero entre 0 y 4095).
b_k	Bit k : 1 si la condición k se cumple (p. ej., b_4 : la lectura está en la hebra reversa).
2^k	Valor decimal del bit k : 1, 2, 4, ..., 2048.

Como la representación binaria de un entero es única, la descomposición también lo es: cada FLAG corresponde exactamente a una combinación de condiciones. En

	suplement. 2048	duplicado 1024	falla QC 512	secundaria 256	R2 128	R1 64	pareja – 32	hebra – 16	pareja no al. 8	no alineada 4	par correcto 2	pareada 1	
99	0	0	0	0	0	1	1	0	0	0	1	1	1 + 2 + 32 + 64 R1 de un par típico
147	0	0	0	0	1	0	0	1	0	0	1	1	1 + 2 + 16 + 128 R2 de ese par
83	0	0	0	0	0	1	0	1	0	0	1	1	1 + 2 + 16 + 64 R1 en hebra –
163	0	0	0	0	1	0	1	0	0	0	1	1	1 + 2 + 32 + 128 R2 en hebra +
4	0	0	0	0	0	0	0	0	0	1	0	0	4 no alineada
1171	0	1	0	0	1	0	0	1	0	0	1	1	1 + 2 + 16 + 128 + 1024 R2 duplicado
2064	1	0	0	0	0	0	0	1	0	0	0	0	16 + 2048 suplementaria, hebra –

Figura 2.8. Siete valores de FLAG frecuentes, descompuestos en sus doce bits (el de menor valor a la derecha, como en la escritura binaria). 99 y 147 son la pareja canónica de un par bien alineado: R1 en la hebra directa con su pareja en la reversa, y viceversa. 83/163 es la configuración espejo. 4 es una lectura sin alinear. 1171 es la R2 anterior marcada como duplicado de PCR. 2064 es un alineamiento suplementario: un trozo de una lectura quimérica que cae en otro lugar del genoma, señal típica de reordenamientos estructurales.

código, la pregunta “¿está encendido el bit de valor 16?” se formula con el Y binario: `flag & 16 != 0`. La figura 2.8 muestra las banderas más frecuentes.

Ejemplo 2.4. (Construir y descomponer una FLAG)

¿Qué FLAG tiene la segunda lectura de un par, bien emparejada, alineada en la hebra reversa y marcada como duplicado? Encendemos los bits de valor 1 (pareada), 2 (par correcto), 16 (hebra reversa), 128 (R2) y 1024 (duplicado): $F = 1 + 2 + 16 + 128 + 1024 = 1171$.

A la inversa, $F = 83$: la mayor potencia de 2 que cabe es 64 (resta 19), luego 16 (resta 3), luego 2 (resta 1) y 1. Así, $83 = 64 + 16 + 2 + 1$: primera lectura del par, en la hebra reversa, par correcto. Su pareja típica es $163 = 128 + 32 + 2 + 1$.

BAM, BGZF e índices: comprimir sin perder el acceso

Un SAM de un genoma humano a 30× pesa cientos de gigabytes. El formato BAM, su versión binaria, resuelve dos problemas a la vez. El primero es el tamaño: los campos numéricos se guardan como enteros binarios, las bases se empaquetan en 4 bits y el resultado se comprime. El segundo, más sutil, es el *acceso aleatorio*. Un archivo gzip ordinario es un único flujo comprimido: para leer el último megabyte hay que descomprimir todo lo anterior. BAM usa en cambio BGZF (*Blocked GNU Zip Format*): una concatenación de bloques gzip independientes, cada uno con a lo sumo 64 KiB de datos sin comprimir (Li et al., 2009). Cualquier programa que entienda gzip puede leerlo, pero uno que conozca la estructura puede saltar directamente a un bloque.

Para ello, cada posición del archivo se describe con un *desplazamiento virtual* de 64 bits:

$$v = c \cdot 2^{16} + u, \qquad 0 \leq u < 2^{16}, \qquad (2.15)$$

DONDE	
v	Desplazamiento virtual, el número que se guarda en el índice.
c	Desplazamiento en bytes, dentro del archivo <i>comprimido</i> , del inicio del bloque BGZF (48 bits).
u	Desplazamiento dentro del bloque, una vez <i>descomprimido</i> (16 bits, pues el bloque mide a lo sumo 64 KiB).

Leer en v significa: saltar al byte c , descomprimir un solo bloque y avanzar u bytes. Como los desplazamientos virtuales conservan el orden del archivo, se pueden comparar como enteros ordinarios.

Falta saber *qué* v corresponde a una región del genoma. Si el BAM está ordenado por coordenada, el índice `.bai` lo responde con un esquema de *casillas* (*bins*) heredado del navegador de UCSC (Kent et al., 2002): el cromosoma se divide en una jerarquía de seis niveles, y cada alineamiento se asigna a la casilla más pequeña que lo contiene por completo (figura 2.9).

Teorema 2.2. (Esquema de casillas del índice BAI)

En el nivel $l \in \{0, \dots, 5\}$ hay 8^l casillas, cada una de 2^{29-3l} pares de bases, numeradas consecutivamente a partir de

$$\text{off}(l) = \frac{8^l - 1}{7} \in \{0, 1, 9, 73, 585, 4681\}. \quad (2.16)$$

Un intervalo $[s, e)$ se asigna a la casilla $\text{off}(l^*) + \lfloor s/2^{29-3l^*} \rfloor$, donde l^* es el nivel más profundo en el que $\lfloor s/2^{29-3l} \rfloor = \lfloor (e-1)/2^{29-3l} \rfloor$. En total hay $\sum_{l=0}^5 8^l = 37\,449$ casillas.

DONDE

l	Nivel de la jerarquía: 0 es una casilla de 512 Mb; 5 son casillas de 16 kb.
2^{29-3l}	Tamaño de cada casilla del nivel l ($2^{29} \approx 5,4 \times 10^8$ pb en el nivel 0).
$\text{off}(l)$	Número de la primera casilla del nivel l (suma de las casillas de los niveles anteriores).
$[s, e)$	Intervalo o-based semiabierto que ocupa el alineamiento.

Para cada casilla, el índice guarda los intervalos de desplazamientos virtuales (*chunks*) donde se encuentran sus alineamientos. Para consultar una región se calculan las casillas que la solapan (a lo sumo una por nivel más las vecinas; unas pocas decenas en total), se leen solo los bloques correspondientes y se descartan los alineamientos que no solapan de verdad. Un índice lineal adicional, con una entrada por ventana de 16 kb, permite saltarse los *chunks* de casillas grandes que terminan antes de la región. El formato tabix generaliza exactamente este mecanismo a cualquier archivo de texto tabular ordenado y comprimido con BGZF, como VCF, BED o GFF (Li, 2011).

Ejemplo 2.5. (¿En qué casilla cae TP53?)

El transcrito canónico de *TP53* ocupa [7 668 420, 7 687 490) del cromosoma 17. En el nivel 5 sus extremos caen en casillas distintas ($\lfloor s/2^{14} \rfloor = 468$, $\lfloor (e-1)/2^{14} \rfloor = 469$); en el nivel 4 ambos dan 58, así que su casilla es $\text{off}(4) + 58 = 585 + 58 = 643$. Una lectura de 150 nt en [7 674 219, 7 674 369) cabe en la casilla de nivel 5 número $4681 + 468 = 5149$.

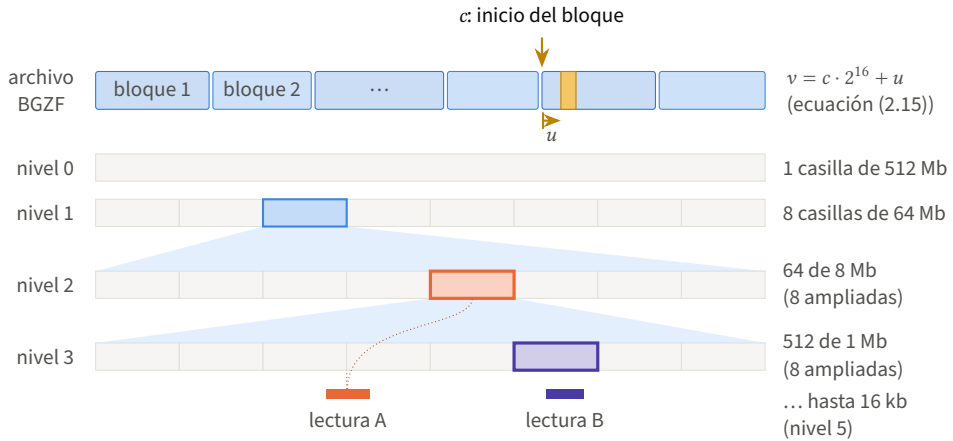


Figura 2.9. Arriba: un archivo BGZF es una cadena de bloques gzip independientes. Un desplazamiento virtual combina el byte c donde empieza un bloque comprimido con la posición u dentro del bloque descomprimido. Abajo: los primeros niveles de la jerarquía de casillas; cada casilla se divide en ocho del nivel siguiente. Los niveles 2 y 3 se muestran ampliados: solo aparecen las ocho hijas de la casilla sombreada del nivel anterior. La lectura B cabe dentro de una casilla de 1 Mb y se asigna a ella; la lectura A cruza la frontera entre dos casillas de nivel 3 y sube a la casilla de 8 Mb que las contiene. Consultar una región implica leer unas pocas casillas por nivel, nunca el archivo completo.

La última vuelta de tuerca es CRAM (Hsi-Yang Fritz et al., 2011). Su idea es que la inmensa mayoría de las bases de una lectura coinciden con la referencia, así que basta guardar la posición y las *diferencias*: una lectura perfecta de 150 nt se reduce a unos pocos números. Combinado con modelos estadísticos para las calidades y los nombres, CRAM ocupa típicamente entre un 40 y un 60 % menos que BAM; a cambio, descomprimirlo exige disponer *exactamente* de la misma referencia con la que se creó (de ahí la importancia de versionar los ensamblajes, sección 2.2). HTSlib lee y escribe SAM, BAM y CRAM de forma transparente (Bonfield et al., 2021), y **samtools** expone esas capacidades en la línea de órdenes (Danecek et al., 2021).

2.1.8 VCF: las diferencias con la referencia

Si el BAM contiene todas las lecturas, el *Variant Call Format* (Danecek et al., 2011) contiene solo sus conclusiones: las posiciones donde una o más muestras difieren de la referencia. Un VCF tiene metadatos (##) que declaran cada campo, una línea de encabezado (#CHROM) y una fila por variante con ocho columnas fijas

(CHROM, POS, ID, REF, ALT, QUAL, FILTER, INFO), seguidas de una columna FORMAT y una columna por muestra.

```
variantes.vcf

##fileformat=VCFv4.2
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotipo">
##FORMAT=<ID=PL,Number=G,Type=Integer,Description="Verosimilitudes">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT S1 S2
chr17 101 . C T 50 PASS AC=3 GT:PL 0/1:50,0,90 1/1:.
chr17 102 . CAG C 38 PASS AC=1 GT:PL 0/1:. 0/0:.
```

POS es 1-based. El genotipo GT de un diploide lista dos alelos (0 = REF, 1 = primer ALT, 2 = segundo ALT...): 0/1 es heterocigoto y 1/1 homocigoto alternativo; con | en lugar de / el genotipo está *fasado*, es decir, se sabe qué alelo está en cada cromosoma homólogo.

Verosimilitudes de genotipo

El campo PL resume la evidencia que llevó al genotipo, y es otra aplicación de la escala Phred. Supongamos que en una posición cubierta por n lecturas observamos a con el alelo alternativo y $n - a$ con el de referencia, y que cada base se lee mal con probabilidad ϵ (su calidad Phred). Bajo cada genotipo diploide G , cada lectura muestrea uno de los dos cromosomas al azar, de modo que

$$P(D \mid G) = \prod_{j=1}^n \left[\frac{1}{2} P(d_j \mid A_1) + \frac{1}{2} P(d_j \mid A_2) \right], \quad PL_G = -10 \log_{10} \frac{P(D \mid G)}{\max_{G'} P(D \mid G')},$$

(2.17)

DONDE	
$D = (d_1, \dots, d_n)$	Bases observadas en las n lecturas que cubren la posición.
$G = A_1/A_2$	Genotipo diploide candidato (o/o, o/1 o 1/1).
$P(d_j \mid A)$	Probabilidad de leer d_j si el alelo verdadero es A : $1 - \epsilon$ si coinciden, ϵ si no.
PL_G	Verosimilitud del genotipo en escala Phred, normalizada para que el más verosímil valga 0.

Ejemplo 2.6. (Del recuento de lecturas al campo PL)

Diez lecturas cubren la posición 101: seis dicen **C** (REF) y cuatro **T** (ALT), con $\varepsilon = 0,01$ ($Q = 20$). Entonces

$$P(D \mid 0/0) = 0,99^6 0,01^4 = 9,41 \times 10^{-9},$$

$$P(D \mid 0/1) = 0,5^{10} = 9,77 \times 10^{-4},$$

$$P(D \mid 1/1) = 0,01^6 0,99^4 = 9,61 \times 10^{-13}.$$

El heterocigoto es el más verosímil ($PL = 0$); para los otros, $-\log_{10}(9,41 \times 10^{-9}/9,77 \times 10^{-4}) = 50$ y $-\log_{10}(9,61 \times 10^{-13}/9,77 \times 10^{-4}) = 90$. El VCF guarda $PL=50, 0, 90$: el homocigoto de referencia es 10^5 veces menos verosímil que el heterocigoto.

A partir de los genotipos de N individuos diploides se estima la frecuencia del alelo alternativo, $\widehat{AF} = \sum_i g_i / (2N)$, donde $g_i \in \{0, 1, 2\}$ es el número de copias del alelo alternativo del individuo i ; los campos AC y AN de INFO guardan precisamente el numerador y el denominador. Volveremos a esta cantidad al estudiar genética de poblaciones.

Una variante, muchas escrituras

Las inserciones y deleciones se escriben con una *base ancla*: la base anterior al evento, para que ni REF ni ALT queden vacíos. Pero en regiones repetitivas la misma variante admite varias escrituras. Si la referencia es **GCAGAGT** (posiciones 101 a 107) y la muestra perdió una copia de **AG**, tanto POS=102 REF=CAG ALT=C como POS=104 REF=GAG ALT=G producen la misma secuencia **GCAGT**. Dos VCF que describen la misma variante de maneras distintas no “casan” al compararlos. Tan et al. (2015) demostraron que toda variante tiene una única representación *normalizada*: la más parsimoniosa (sin bases redundantes a ambos lados) y alineada a la izquierda, en este caso la de POS=102. Normalizar (`bcftools norm`) antes de comparar o anotar variantes es una práctica obligatoria.

2.2 Bases de datos biológicas desde Python

Un sistema de bibliotecas

Una ciudad con buenas bibliotecas no guarda cada libro en un solo edificio. Hay un archivo nacional que conserva, sin tocarlos, todos los manuscritos depositados; hay bibliotecas especializadas cuyos bibliotecarios escogen la mejor edición de cada obra y le añaden notas; y hay catálogos que no guardan libros, sino fichas que dicen dónde está cada uno y qué otros libros lo citan. Todo el sistema funciona porque cada libro tiene un número de catálogo que nunca cambia, y porque las nuevas ediciones reciben un número de edición en lugar de borrar la anterior. Las bases de datos biológicas están organizadas exactamente así.

2.2.1 Primarias, curadas e integradoras

Los laboratorios producen secuencias, estructuras y mediciones cada día. Para compartirlas, la comunidad utiliza repositorios públicos y recursos especializados. Conviene distinguir tres niveles (figura 2.10):

- **Archivos primarios:** guardan lo que los autores depositaron, tal cual, con su número de acceso. Son GenBank, ENA y DDBJ para secuencias (Arita et al., 2021), el SRA para lecturas crudas (Leinonen et al., 2011) y el *Protein Data Bank* para estructuras. Nadie “corrige” un registro ajeno: si hay diez depósitos de la misma proteína, hay diez registros.
- **Bases curadas:** expertos (o procesos automáticos con reglas explícitas) eligen una secuencia representativa por entidad biológica y la anotan con evidencia. Es el caso de RefSeq, del NCBI (O’Leary et al., 2016), y de UniProtKB/Swiss-Prot (The UniProt Consortium, 2025).
- **Recursos integradores:** combinan ensamblajes, anotaciones, variantes y comparaciones entre especies, y enlazan con todo lo demás. Ensembl (Dyer et al., 2025) y el conjunto de bases del NCBI (Gene, dbSNP, ClinVar, PubMed...) (Sayers, Beck et al., 2025) son los ejemplos más usados.

2.2.2 Identificadores estables y versiones

La regla más útil de esta sección es que *un nombre de gen no es un identificador*. Según el contexto, “*TP53*” puede referirse a un locus del cromosoma 17, a uno de sus transcritos, a una isoforma proteica, al ortólogo de otra especie o a la proteína

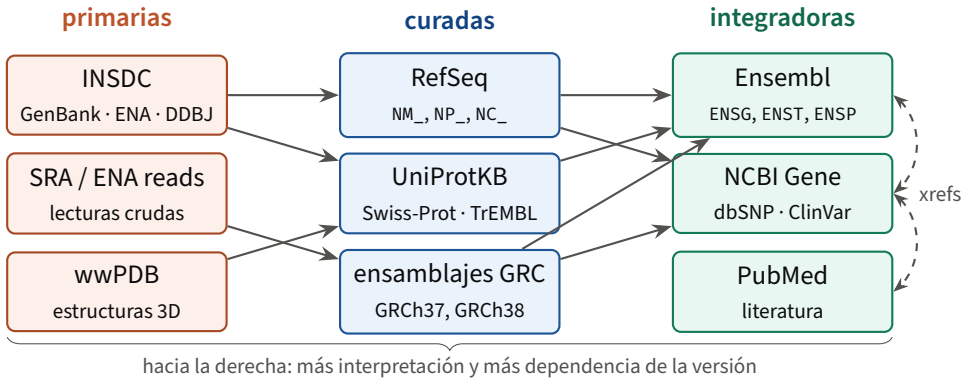


Figura 2.10. El ecosistema de bases de datos organizado por nivel de interpretación. Las flechas indican de dónde toma cada recurso sus datos; las líneas discontinuas, referencias cruzadas (*xrefs*) mantenidas entre recursos. Los archivos primarios nunca reescriben lo depositado; las bases curadas eligen y anotan una secuencia por entidad; los recursos integradores la sitúan en un genoma y la conectan con variantes, literatura y otras especies. Un mismo gen tiene un identificador distinto en cada caja.

p53 en general. Los identificadores de las bases de datos aportan dos propiedades adicionales:

Definición 2.4. (Identificador estable versionado)

Un identificador estable es una cadena que designa siempre la misma entidad, que nunca se reasigna a otra y que se sigue resolviendo aunque la entidad quede obsoleta. Cuando el contenido de la entidad cambia, el identificador conserva su parte estable y se incrementa un número de versión: `acceso.version`.

Así, NM_000546 es el ARNm de referencia de *TP53* en RefSeq, y NM_000546.6 es su sexta versión; cualquier cambio en la secuencia, por pequeño que sea, genera la .7. Ensembl sigue la misma lógica (ENST00000269305.9). UniProt separa dos contadores: la entrada Po4637 va por la versión 315 de su *anotación*, pero por la versión 4 de su *secuencia*, que no cambia desde el 24 de noviembre de 2009. En un artículo o en un cuaderno de laboratorio hay que registrar el identificador *con* su versión, y la versión de la base de datos consultada.

Los identificadores tienen además una sintaxis reconocible: ENSG, ENST y ENSP seguidos de once dígitos en Ensembl; NM_, NP_ o NC_ en RefSeq; seis o diez caracteres

alfanuméricos en UniProt; cuatro caracteres en el PDB. Esa regularidad permite validarlos con expresiones regulares antes de enviarlos a un servidor. Los de *TP53* se reúnen en la figura 2.15.

El ensamblaje también tiene versión

Las coordenadas genómicas solo tienen sentido respecto de un ensamblaje. Entre GRCh37 (2009) y GRCh38 (2013) se corrigieron errores, se cerraron huecos y se añadieron secuencias, de modo que casi todas las coordenadas cambiaron. *TP53* está en 17:7565097-7590856 en GRCh37 y en 17:7661779-7687546 en GRCh38, unos 96 682 pb más adelante; su identificador de gen es el mismo, pero su versión pasó de 11 a 21, y la del transcrito canónico, de 4 a 9. Mezclar un VCF en GRCh37 con una anotación en GRCh38 no produce ningún error de ejecución: produce resultados silenciosamente equivocados. La traslación de coordenadas entre ensamblajes (*liftover*) es posible, pero no es trivial, porque algunas regiones no tienen equivalente.

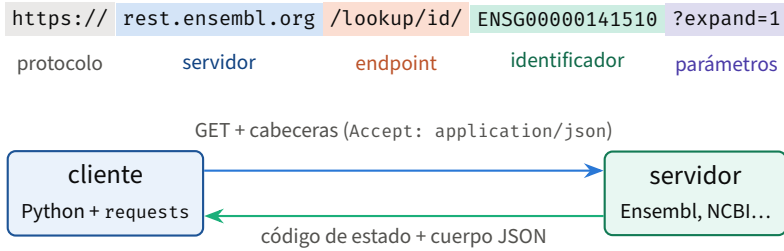
De un libro de proteínas a millones de secuencias

El primer “banco de datos” de secuencias fue un libro: el *Atlas of Protein Sequence and Structure* que Margaret Dayhoff publicó desde 1965 y que, en el capítulo 3, dio origen a las matrices PAM. GenBank se fundó en 1982 en Los Álamos y pasó al NCBI a finales de esa década; el *Protein Data Bank* se creó en 1971 con apenas siete estructuras (Berman et al., 2000). En 2003 los centros que lo gestionaban en Estados Unidos, Europa y Japón formalizaron el *worldwide PDB*, un único archivo global mantenido en colaboración (Berman et al., 2003), siguiendo el modelo que la INSDC ya aplicaba a las secuencias.

2.2.3 APIs REST: preguntarle a un servidor

Casi todas las bases de datos modernas ofrecen, además de su página web, una *API REST*: una forma de consultar sus datos mediante direcciones web que devuelven texto estructurado (JSON, XML, FASTA) en lugar de páginas para humanos. Una petición tiene cuatro partes (figura 2.11): el protocolo y el servidor, el *endpoint* (qué tipo de pregunta hacemos), el recurso (sobre qué entidad) y los parámetros (cómo queremos la respuesta). El servidor responde con un *código de estado* de tres dígitos y un cuerpo.

La respuesta JSON se convierte directamente en diccionarios y listas de Python. En el caso de Ensembl, un gen contiene una lista *Transcript*; cada transcrito, una lista *Exon* y, si codifica proteína, un diccionario *Translation*.



200 OK: leer el cuerpo. **400/404** pregunta mal formada o recurso inexistente: no reintentar.
429 demasiadas peticiones: esperar (Retry-After) y reintentar. **5xx** fallo del servidor: reintentar con espera creciente.

Figura 2.11. Arriba: anatomía de una URL de la API REST de Ensembl (Yates et al., 2015), que pide el gen *TP53* con sus transcritos y exones (`expand=1`). Abajo: el ciclo petición-respuesta y la interpretación de los códigos de estado más frecuentes. La distinción importante es entre errores del cliente (4xx: repetir la petición no servirá, salvo el 429) y errores transitorios (429 y 5xx: vale la pena esperar y reintentar).

Límites de tasa

Los servidores públicos atienden a millones de usuarios y protegen su capacidad con *límites de tasa*. Las E-utilities del NCBI admiten 3 peticiones por segundo sin registro y 10 con una clave de API gratuita; Ensembl admite en promedio unas 15 por segundo. Quien los excede recibe respuestas 429 y, si insiste, puede ver bloqueada su dirección IP (y la de todo su laboratorio, que comparte la misma).

Un límite de r peticiones por segundo impone una cota inferior al tiempo de una descarga de N peticiones:

$$T \geq \frac{N}{r}. \quad (2.18)$$

DONDE

T	Tiempo total de la descarga, en segundos.
N	Número de peticiones necesarias.
r	Tasa máxima admitida por el servidor (peticiones por segundo).

La consecuencia práctica es que el único modo de acelerar una descarga grande es reducir N : pedir muchos registros por petición. Las E-utilities lo permiten a través de su *History server*: `esearch` con `usehistory=y` guarda la lista de resultados en el servidor y devuelve dos claves (`WebEnv` y `query_key`), con las que `efetch` descarga los registros en lotes de hasta varios cientos.

Ejemplo 2.7. (Descargar 5000 proteínas)

Queremos las secuencias FASTA de 5000 proteínas del NCBI. Pidiéndolas de una en una ($N = 5000$) sin clave de API ($r = 3$), la ecuación (2.18) da $T \geq 1667$ s, casi media hora; con clave ($r = 10$), 500 s. Si en cambio usamos el *History server* y lotes de 500 registros, bastan una petición `esearch` y $[5000/500] = 10$ peticiones `efetch`: $N = 11$ y $T \geq 3,7$ s. El cuello de botella deja de ser el límite de tasa y pasa a ser el volumen de datos transferido.

Reintentos con espera exponencial

Los errores transitorios (429, 5xx, cortes de red) se tratan reintentando, pero no inmediatamente: si mil clientes reintentan a la vez tras una caída, la provocan de nuevo. La estrategia estándar es la *espera exponencial con variación aleatoria*:

$$t_k = \min(t_{\text{máx}}, t_0 2^k) + U_k, \quad U_k \sim \text{Uniforme}(0, t_0), \quad (2.19)$$

DONDE

t_k	Tiempo de espera antes del reintento k ($k = 0, 1, 2, \dots$).
t_0	Espera inicial (típicamente 1 s o el valor de la cabecera <code>Retry-After</code>).
$t_{\text{máx}}$	Espera máxima, para no esperar indefinidamente.
U_k	Variación aleatoria (<i>jitter</i>) que desincroniza a los clientes.

Con $t_0 = 1$ s y cinco reintentos, la espera determinista total es $1 + 2 + 4 + 8 + 16 = 31$ s. Si cada intento falla de forma independiente con probabilidad $q = 0,2$, la probabilidad de que fallen los seis (el original y los cinco reintentos) es $q^6 = 6,4 \times 10^{-5}$: la espera exponencial convierte un servicio poco fiable en uno casi perfecto a cambio de latencia ocasional.

Un cliente robusto combina estas ideas con una *caché* (no preguntar dos veces lo mismo) y un *registro de procedencia* (qué se pidió, a quién, cuándo, y si la respuesta vino del servidor o de la caché). El notebook de esta lección construye la función `fetch()` que usaremos en el resto del curso; su núcleo es el siguiente:

```
cliente.py

import time, random, requests

def get_json(url, params=None, t0=1.0, tmax=30, tries=6):
```



```
for k in range(tries):
    r = requests.get(url, params=params, timeout=30,
                    headers={"Accept": "application/json"})
    if r.status_code == 200:
        return r.json()
    if r.status_code not in (429, 500, 502, 503, 504):
        r.raise_for_status() # error del cliente
    wait = float(r.headers.get("Retry-After", 0))
    wait = max(wait, min(tmax, t0 * 2 ** k))
    time.sleep(wait + random.uniform(0, t0))
raise RuntimeError(f"sin respuesta tras {tries} intentos")
```

2.2.4 NCBI y las E-utilities

El NCBI agrupa más de treinta bases de datos (Nucleotide, Protein, Gene, PubMed, SRA, dbSNP, ClinVar, Taxonomy...) bajo un sistema de búsqueda común llamado Entrez (Sayers, Beck et al., 2025). Sus *Entrez Programming Utilities* son una familia de servicios que se combinan como piezas:

Utilidad	Pregunta que responde	Devuelve
esearch	¿qué registros cumplen esta búsqueda?	identificadores (UID) y recuento
esummary	¿qué dice el resumen de estos registros?	metadatos
elink	¿qué registros de otra base están ligados a estos?	UID relacionados
efetch	dame el registro completo	FASTA, GenBank, XML...

El flujo típico es esearch → esummary/elink → efetch: primero se encuentra, luego se mira, al final se descarga. Para *TP53*, esearch en la base Gene con la consulta TP53[sym] AND human[orgn] devuelve el UID 7157; esummary devuelve su localización (ejemplo 2.2); elink con linkname=gene_protein_refseq lleva a sus proteínas RefSeq, y efetch descarga NP_000537.3 en FASTA, una proteína de 393 aminoácidos. Biopython envuelve estas llamadas en el módulo Bio.Entrez, que además respeta automáticamente el límite de tasa (Cock et al., 2009).

```
entrez_tp53.py

from Bio import Entrez, SeqIO

Entrez.email = "usted@universidad.edu" # obligatorio
# Entrez.api_key = "..." # opcional: 10/s
```

```

h = Entrez.esearch(db="gene", term="TP53[sym] AND human[orgn]")
gene_id = Entrez.read(h)["IdList"][0] # '7157'
h = Entrez.elink(dbfrom="gene", db="protein", id=gene_id,
                 linkname="gene_protein_refseq")
links = Entrez.read(h)[0]["LinkSetDb"][0]["Link"]
h = Entrez.efetch(db="protein", id="NP_000537.3",
                 rettype="fasta", retmode="text")
p53 = SeqIO.read(h, "fasta")
print(gene_id, len(links), p53.id, len(p53))

```

Con el parámetro `rettype=count`, `esearch` devuelve solo el número de registros que cumplen una búsqueda, lo que permite medir, por ejemplo, cuántos artículos por año indexa PubMed sobre un tema; el notebook de la lección usa esa consulta para estimar el tiempo de duplicación de la literatura sobre CRISPR (menos de dos años entre 2013 y 2020).

2.2.5 Ensembl: el gen en su genoma

Ensembl anota automáticamente los genomas de cientos de especies de vertebrados y, en su versión actual, integra además los resultados del proyecto GENCODE para el ser humano y el ratón (Dyer et al., 2025). Su API REST (Yates et al., 2015) expone casi todo su contenido; el endpoint `lookup/id` con `expand=1` devuelve el gen con todos sus transcritos y exones.

Para *TP53* en GRCh38, Ensembl informa un gen de 25 768 pb en la hebra negativa del cromosoma 17, con 40 transcritos anotados: distintos sitios de inicio, combinaciones de exones por *splicing* alternativo y transcritos que no codifican proteína. Entre ellos, uno se marca como *canónico*: ENST00000269305.9, que corresponde al ARNm de referencia NM_000546.6 de RefSeq. La figura 2.12 lo dibuja a partir de la respuesta de la API.

⚠ «La proteína de *TP53*» no existe

Con 40 transcritos anotados, la frase “la secuencia de *TP53*” es ambigua. Hay que especificar el transcrito (ENST00000269305.9 o NM_000546.6) y el ensamblaje. Las herramientas de anotación de variantes pueden describir la misma mutación con posiciones proteicas distintas según el transcrito elegido; por eso las recomendaciones clínicas exigen el transcrito de referencia versionado.

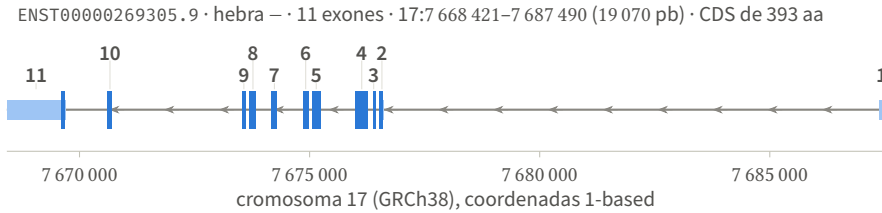


Figura 2.12. Modelo del transcrito canónico de *TP53* dibujado directamente a partir de la respuesta de la API de Ensembl. Los bloques gruesos son región codificante (CDS), los delgados son regiones no traducidas (UTR) y la línea con flechas son intrones; el gen se transcribe de derecha a izquierda porque está en la hebra negativa, así que el exón 1 es el de la derecha. El exón 1 es completamente no codificante: la proteína empieza en el exón 2. Los 11 exones suman 2512 nt, apenas un 13 % de los 19 070 pb del transcrito; el resto son intrones.

2.2.6 UniProt: la proteína y su conocimiento

UniProt es el recurso central para las proteínas (The UniProt Consortium, 2025). Su base de conocimiento, UniProtKB, tiene dos secciones: Swiss-Prot, revisada manualmente por curadores que resumen la literatura con evidencia explícita, y TrEMBL, anotada automáticamente y órdenes de magnitud mayor. La entrada revisada P04637 (p53 humana, 393 aminoácidos) describe sus dominios, sitios de modificación, interacciones, isoformas, estructuras y más de mil variantes naturales, casi todas mutaciones somáticas halladas en tumores. La API REST las devuelve en JSON desde <https://rest.uniprot.org/uniprotkb/P04637.json>, como una lista features con su tipo y localización.

La figura 2.13 sugiere que las variantes no están repartidas al azar. Podemos cuantificarlo con una prueba binomial, un buen ejemplo de cómo los datos de una base de datos se convierten en una afirmación estadística.

Ejemplo 2.8. (¿Se concentran las variantes en el dominio de unión al ADN?)

La entrada registra $N = 1363$ variantes naturales en 332 posiciones distintas. El dominio de unión al ADN abarca 191 de los 393 residuos, una fracción $\pi_0 = 191/393 = 0,486$. Si las variantes cayeran al azar a lo largo de la proteína, el número dentro del dominio, K , seguiría una distribución $\text{Bin}(N, \pi_0)$, con

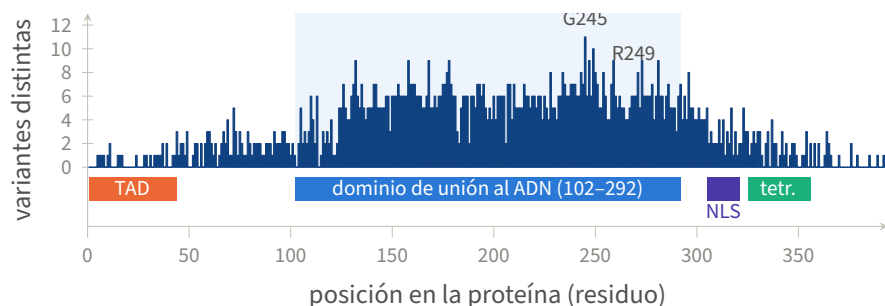


Figura 2.13. Número de variantes de un aminoácido distintas registradas en UniProtKB (P04637) en cada posición de p53. Abajo, las regiones funcionales anotadas en la misma entrada: dominio de transactivación (TAD, 1–44), dominio de unión al ADN (102–292), señal de localización nuclear bipartita (NLS) y dominio de oligomerización o tetramerización (325–356). Las variantes se concentran de forma abrumadora en el dominio de unión al ADN (fondo azul), donde están las posiciones con más variantes distintas, como G245 y R249.

media $N\pi_0 = 662,4$ y desviación típica $\sqrt{N\pi_0(1 - \pi_0)} = 18,5$. Se observan $K = 1052$ (el 77 %), lo que da

$$z = \frac{1052 - 662,4}{18,5} = 21,1,$$

y una probabilidad exacta $P(K \geq 1052) \approx 8 \times 10^{-104}$. La concentración es incuestionable. Pero cuidado con la interpretación: el resultado mezcla biología (las mutaciones que destruyen la unión al ADN son las seleccionadas en los tumores) con un *sesgo de verificación*, porque durante años muchos estudios secuenciaron solo los exones 5 a 8, que codifican ese dominio. Los datos de una base de datos reflejan también cómo se recolectaron.

2.2.7 PDB: la estructura

El *Protein Data Bank* archiva las estructuras tridimensionales de macromoléculas determinadas experimentalmente (Berman et al., 2000). Cada entrada tiene un identificador de cuatro caracteres, las coordenadas atómicas (en los formatos PDB clásico o mmCIF) y los metadatos del experimento: método, resolución, cadenas, ligandos. El portal estadounidense, RCSB PDB, ofrece además APIs de búsqueda y de datos (esta última en GraphQL), y desde 2022 muestra, junto a las estructu-

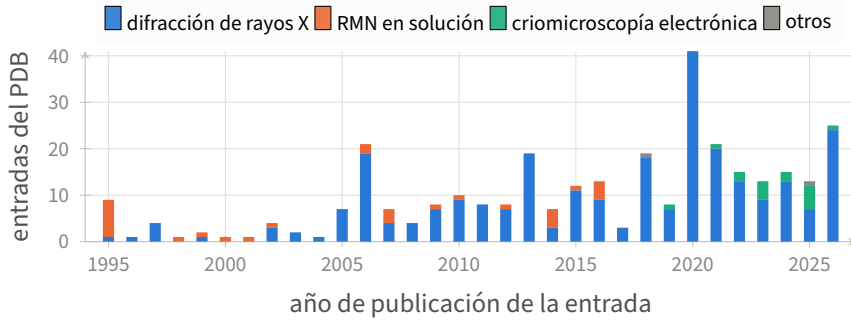


Figura 2.14. Entradas del PDB que contienen p53 humana (acceso UniProt P04637), por año de publicación y método experimental, obtenidas de la API de RCSB PDB en septiembre de 2026 (el último año está incompleto). Las primeras estructuras de mediados de los noventa, como 1TSR, resolvieron el dominio de unión al ADN; la mayoría de las posteriores son variantes de ese dominio con fármacos candidatos a “rescatar” la p53 mutada. La criomicroscopía aparece solo en los últimos años, típicamente con complejos grandes y resoluciones más modestas.

ras experimentales, modelos computacionales predichos por inteligencia artificial, claramente separados (Burley et al., 2025).

Una búsqueda de todas las entradas cuyas cadenas remiten al acceso UniProt P04637 devuelve 323 estructuras de p53 humana. La figura 2.14 resume su historia: 275 por difracción de rayos X, 30 por resonancia magnética nuclear y 16 por criomicroscopía electrónica. La resolución mediana es de 1,8 Å para rayos X y de 3,83 Å para criomicroscopía; la mejor alcanza 1,01 Å.

La entrada 1TSR (publicada en el PDB en enero de 1996, 2,2 Å) contiene tres copias del dominio de unión al ADN de p53 (cadenas A, B y C) unidas a una doble hélice de ADN (cadenas E y F). Su publicación (Cho et al., 1994) fue un hito, porque mostró por qué los “puntos calientes” de mutación en cáncer son los que son: residuos como R248 y R273 contactan directamente el ADN, y otros como R175, G245 y R249 sostienen la superficie de contacto. Una sola sustitución en esos lugares basta para que p53 suelte el ADN o pierda su forma. Acabamos de encadenar tres bases de datos (la variante en UniProt, la posición en el genoma de Ensembl y la estructura en el PDB) para explicar un fenotipo.

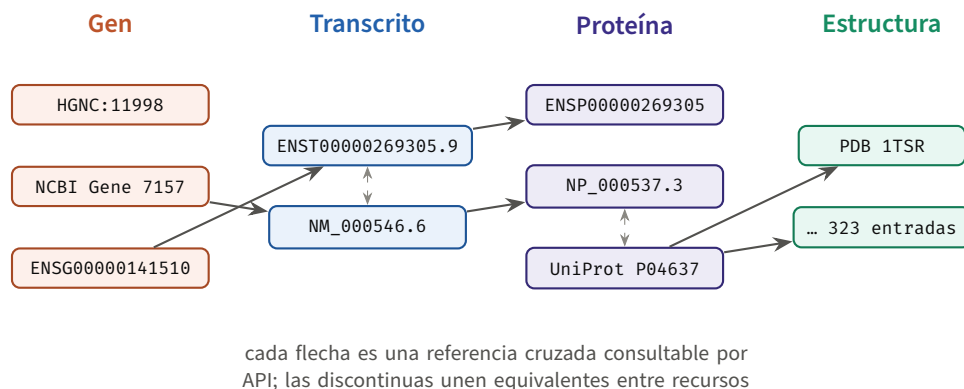


Figura 2.15. El “pasaporte” de *TP53*, construido con los identificadores obtenidos en esta sección. El nivel de detalle crece de izquierda a derecha, como en el dogma central: un gen, uno de sus transcritos, la proteína que codifica y una de sus estructuras. Cada identificador de transcrito y proteína lleva su versión; los de gen y estructura designan entidades cuyo contenido puede actualizarse sin cambiar de nombre, por lo que conviene registrar también la versión de la base de datos consultada.

2.2.8 El pasaporte de un gen y los principios FAIR

Los recursos que hemos visitado no son islas: cada uno mantiene referencias cruzadas hacia los demás. El endpoint `xrefs/id` de Ensembl, por ejemplo, devuelve para ENSG00000141510 su identificador HGNC, su UID de NCBI Gene y su número OMIM, y para la proteína ENSP00000269305 su acceso de UniProt, su proteína RefSeq y la lista de estructuras del PDB. Con esas referencias se construye el “pasaporte” de la figura 2.15.

Esta capacidad de pasar de un recurso a otro sin intervención humana no es casual. Wilkinson et al. (2016) la formularon como cuatro principios que deberían cumplir todos los datos científicos, resumidos en el acrónimo FAIR:

- **F** (*Findable*, localizables): los datos tienen identificadores globales y persistentes, y metadatos ricos indexados en un recurso consultable.
- **A** (*Accessible*, accesibles): se recuperan por su identificador mediante un protocolo abierto y estandarizado (como HTTP), y los metadatos siguen disponibles aunque los datos dejen de estarlo.
- **I** (*Interoperable*, interoperables): usan lenguajes y vocabularios formales y compartidos (como la *Sequence Ontology*) e incluyen referencias a otros datos.

- **R** (*Reusable*, reutilizables): tienen una licencia clara, una procedencia detallada y cumplen los estándares de su comunidad.

Los ejemplos del capítulo muestran esos principios en práctica: identificadores versionados (F), APIs REST (A), formatos y ontologías comunes (I). Para facilitar la reutilización (R), debemos registrar *qué* consultamos, *dónde*, *cuándo* y en *qué versión*.

Identificador + versión + fecha + versión de la base.
Sin esos cuatro datos, un análisis no es reproducible.

En resumen

- FASTA guarda secuencias; FASTQ añade una calidad Phred por base, $Q = -10 \log_{10} p$, codificada como $\text{chr}(Q+33)$. Los errores esperados, $EE = \sum 10^{-Q_i/10}$, resumen una lectura mejor que su calidad media.
- GenBank, GFF3, GTF y SAM/VCF usan coordenadas 1-based cerradas; BED, BAM y Python, 0-based semiabiertas. La conversión es $s_0 = s_1 - 1$, $e_0 = e_1$, y el REF de un VCF es la prueba de fuego.
- En SAM, la CIGAR determina la longitud de la lectura y el tramo de referencia según qué operaciones consumen cada secuencia; la FLAG codifica doce condiciones como suma de potencias de 2; MAPQ es la calidad Phred de la posición.
- BAM y CRAM comprimen en bloques BGZF; un índice jerárquico de 37 449 casillas y desplazamientos virtuales $v = c \cdot 2^{16} + u$ permite leer una región sin descomprimir todo el archivo. VCF describe variantes con genotipos y verosimilitudes (PL) en escala Phred, y debe normalizarse.
- Las bases de datos se organizan en primarias (INSDC, SRA, PDB), curadas (RefSeq, Swiss-Prot) e integradoras (Ensembl, NCBI Gene). Se consultan mediante APIs REST, respetando límites de tasa ($T \geq N/r$), reintentando con espera exponencial y guardando caché y procedencia.
- Un gen se identifica con identificadores estables *versionados* y en un ensamblaje concreto; las referencias cruzadas entre recursos materializan los principios FAIR.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 2.1** ●●● (a) ¿Qué calidad Phred corresponde a una probabilidad de error de 0,0005? (b) ¿Con qué carácter se escribe en Phred+33? (c) Decodifique la cadena 5?I+ a calidades y probabilidades de error.
- 2.2** ●●● Descomponga las FLAG 83, 2064 y 3844. ¿Qué filtro -F de `samtools view` excluye a la vez lecturas no alineadas, secundarias, duplicadas y suplementarias?
- 2.3** ●●● Para la CIGAR 3S40M1I20M5D30M2S con POS=1000: (a) ¿qué posición de la referencia queda frente a la base 50 de la lectura? (b) ¿Qué intervalo o-based semiabierto devuelve `reference_start`, `reference_end` en `pysam`? Escriba una función que, dada una CIGAR y POS, devuelva el par de listas de posiciones alineadas.
- 2.4** ●●● Usando la ecuación (2.6), calcule la calidad Phred uniforme mínima que debe tener una lectura de 150 nt para que la probabilidad de estar libre de errores sea al menos 0,9. Compare la aproximación de Poisson con el cálculo exacto.
- 2.5** ●●● Implemente la función `reg2bin(beg, end)` del teorema 2.2 y verifique los resultados del ejemplo 2.5. ¿Cuántas casillas hay que examinar como máximo para una región de 100 kb?
- 2.6** ●●● Escriba un cliente que obtenga para *BRCA1* humano su UID de NCBI Gene (e`search`), su identificador de Ensembl (e`ndpoint lookup/symbol`), su transcrito canónico y su acceso de UniProt (x`refs`), respetando el límite de tasa de cada servicio y registrando la fecha y la versión de cada respuesta.
- 2.7** ●●● Repita el ejemplo 2.6 para 20 lecturas con 3 alternativas y $\varepsilon = 0,01$, y luego con $\varepsilon = 0,05$. ¿Cómo cambia la decisión entre o/o y o/1? Explique por qué una calidad mal calibrada (figura 2.5) produce falsos heterocigotos.

Referencias del capítulo

- Arita, M., Karsch-Mizrachi, I., & Cochrane, G. (2021). The international nucleotide sequence database collaboration. *Nucleic Acids Research*, 49(D1), D121-D124. <https://doi.org/10.1093/nar/gkaa967>
- Berman, H., Henrick, K., & Nakamura, H. (2003). Announcing the worldwide Protein Data Bank. *Nature Structural & Molecular Biology*, 10(12), 980. <https://doi.org/10.1038/nsb1203-980>
- Berman, H. M., Westbrook, J., Feng, Z., Gilliland, G., Bhat, T. N., Weissig, H., Shindyalov, I. N., & Bourne, P. E. (2000). The Protein Data Bank. *Nucleic Acids Research*, 28(1), 235-242. <https://doi.org/10.1093/nar/28.1.235>
- Bonfield, J. K., Marshall, J., Danecek, P., Li, H., Ohan, V., Whitwham, A., Keane, T., & Davies, R. M. (2021). HTSlib: C library for reading/writing high-throughput sequencing data. *GigaScience*, 10(2), giaboo7. <https://doi.org/10.1093/gigascience/giab007>
- Buffalo, V. (2015). *Bioinformatics data skills: Reproducible and robust research with open source tools*. O'Reilly Media.
- Burley, S. K., Bhatt, R., Bhikadiya, C., Bi, C., Biester, A., Biswas, P., Bittrich, S., Blaumann, S., Brown, R., Chao, H., et al. (2025). Updated resources for exploring experimentally-determined PDB structures and Computed Structure Models at the RCSB Protein Data Bank. *Nucleic Acids Research*, 53(D1), D564-D574. <https://doi.org/10.1093/nar/gkae1091>
- Cho, Y., Gorina, S., Jeffrey, P. D., & Pavletich, N. P. (1994). Crystal structure of a p53 tumor suppressor-DNA complex: Understanding tumorigenic mutations. *Science*, 265(5170), 346-355. <https://doi.org/10.1126/science.8023157>
- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Cock, P. J. A., Fields, C. J., Goto, N., Heuer, M. L., & Rice, P. M. (2010). The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, 38(6), 1767-1771. <https://doi.org/10.1093/nar/gkp1137>
- Danecek, P., Auton, A., Abecasis, G., Albers, C. A., Banks, E., DePristo, M. A., Handsaker, R. E., Lunter, G., Marth, G. T., Sherry, S. T., McVean, G., Durbin, R., & 1000 Genomes Project Analysis Group. (2011). The variant call format and VCFtools. *Bioinformatics*, 27(15), 2156-2158. <https://doi.org/10.1093/bioinformatics/btr330>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), giaboo8. <https://doi.org/10.1093/gigascience/giab008>
- Dyer, S. C., Austine-Orimoloye, O., Azov, A. G., Barba, M., Barnes, I., Barrera-Enriquez, V. P., Becker, A., Bennett, R., Beracochea, M., Berry, A., et al. (2025). Ensembl 2025. *Nucleic Acids Research*, 53(D1), D948-D957. <https://doi.org/10.1093/nar/gkae1071>

- Eilbeck, K., Lewis, S. E., Mungall, C. J., Yandell, M., Stein, L., Durbin, R., & Ashburner, M. (2005). The Sequence Ontology: A tool for the unification of genome annotations. *Genome Biology*, 6(5), R44. <https://doi.org/10.1186/gb-2005-6-5-r44>
- Ewing, B., & Green, P. (1998). Base-calling of automated sequencer traces using *Phred*. II. Error probabilities. *Genome Research*, 8(3), 186-194. <https://doi.org/10.1101/gr.8.3.186>
- Ewing, B., Hillier, L., Wendl, M. C., & Green, P. (1998). Base-calling of automated sequencer traces using *Phred*. I. Accuracy assessment. *Genome Research*, 8(3), 175-185. <https://doi.org/10.1101/gr.8.3.175>
- Hsi-Yang Fritz, M., Leinonen, R., Cochrane, G., & Birney, E. (2011). Efficient storage of high throughput DNA sequencing data using reference-based compression. *Genome Research*, 21(5), 734-740. <https://doi.org/10.1101/gr.114819.110>
- Kent, W. J., Sugnet, C. W., Furey, T. S., Roskin, K. M., Pringle, T. H., Zahler, A. M., & Haussler, D. (2002). The human genome browser at UCSC. *Genome Research*, 12(6), 996-1006. <https://doi.org/10.1101/gr.229102>
- Leinonen, R., Sugawara, H., & Shumway, M. (2011). The Sequence Read Archive. *Nucleic Acids Research*, 39(Database), D19-D21. <https://doi.org/10.1093/nar/gkq1019>
- Li, H. (2011). Tabix: Fast retrieval of sequence features from generic TAB-delimited files. *Bioinformatics*, 27(5), 718-719. <https://doi.org/10.1093/bioinformatics/btq671>
- Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., & 1000 Genome Project Data Processing Subgroup. (2009). The Sequence Alignment/Map format and SAMtools. *Bioinformatics*, 25(16), 2078-2079. <https://doi.org/10.1093/bioinformatics/btp352>
- Lipman, D. J., & Pearson, W. R. (1985). Rapid and sensitive protein similarity searches. *Science*, 227(4693), 1435-1441. <https://doi.org/10.1126/science.2983426>
- O'Leary, N. A., Wright, M. W., Brister, J. R., Ciuffo, S., Haddad, D., McVeigh, R., Rajput, B., Robbertse, B., Smith-White, B., Ako-Adjei, D., et al. (2016). Reference sequence (RefSeq) database at NCBI: Current status, taxonomic expansion, and functional annotation. *Nucleic Acids Research*, 44(D1), D733-D745. <https://doi.org/10.1093/nar/gkv1189>
- Pearson, W. R., & Lipman, D. J. (1988). Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences*, 85(8), 2444-2448. <https://doi.org/10.1073/pnas.85.8.2444>
- Quinlan, A. R., & Hall, I. M. (2010). BEDTools: A flexible suite of utilities for comparing genomic features. *Bioinformatics*, 26(6), 841-842. <https://doi.org/10.1093/bioinformatics/btq033>
- Sayers, E. W., Beck, J., Bolton, E. E., Brister, J. R., Chan, J., Connor, R., Feldgarden, M., Fine, A. M., Funk, K., Hoffman, J., Kannan, S., Kelly, C., Klimke, W., Kim, S., Lathrop, S., Marchler-Bauer, A., Murphy, T. D., et al. (2025). Database resources of the National Center for Biotechnology Information in 2025. *Nucleic Acids Research*, 53(D1), D20-D29. <https://doi.org/10.1093/nar/gkae979>

- Sayers, E. W., Cavanaugh, M., Frisse, L., Pruitt, K. D., Schneider, V. A., Underwood, B. A., Yankie, L., & Karsch-Mizrachi, I. (2025). GenBank 2025 update. *Nucleic Acids Research*, 53(D1), D56-D61. <https://doi.org/10.1093/nar/gkae1114>
- Tan, A., Abecasis, G. R., & Kang, H. M. (2015). Unified representation of genetic variants. *Bioinformatics*, 31(13), 2202-2204. <https://doi.org/10.1093/bioinformatics/btv112>
- The UniProt Consortium. (2025). UniProt: The Universal Protein Knowledgebase in 2025. *Nucleic Acids Research*, 53(D1), D609-D617. <https://doi.org/10.1093/nar/gkae1010>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Yates, A., Beal, K., Keenan, S., McLaren, W., Pignatelli, M., Ritchie, G. R. S., Ruffier, M., Taylor, K., Vullo, A., & Flicek, P. (2015). The Ensembl REST API: Ensembl data for any language. *Bioinformatics*, 31(1), 143-145. <https://doi.org/10.1093/bioinformatics/btu613>

PARTE

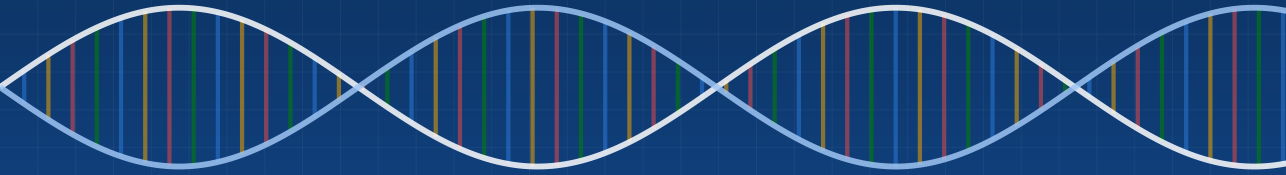


Comparar secuencias

Alineamiento, perfiles y árboles

La comparación es la operación fundamental de la bioinformática. Aquí convertimos la intuición del parecido en algoritmos exactos, en modelos probabilísticos y, finalmente, en la historia evolutiva que conecta a las especies.

Alineamiento de secuencias



Nada tiene sentido en biología si no es a la luz de la evolución.

— Dobzhansky (1973)

● Al terminar este capítulo usted podrá

- Leer un *dot plot* e identificar en él identidad, repeticiones, inversiones e inserciones, y elegir la ventana y el umbral a partir de la probabilidad de ruido.
- Formular el alineamiento global, local, semiglobal y de solapamiento como problemas de optimización sobre un grafo, y resolverlos por programación dinámica (Needleman-Wunsch, Smith-Waterman, Gotoh), incluida la versión en espacio lineal de Hirschberg y el alineamiento con bandas.
- Derivar las matrices PAM y BLOSUM como razones de verosimilitud logarítmicas, calcular su entropía relativa y elegir la matriz adecuada según la divergencia de las secuencias.
- Explicar la heurística de BLAST (vecindad de palabras, umbral T , dos *hits*, *X-drop*) e interpretar un resultado: puntuación en bits, valor E , longitud efectiva y los supuestos de la estadística de Karlin-Altschul.

La comparación atraviesa la historia de la biología. Linneo comparó flores; Darwin, picos de pinzones; nosotros, secuencias. Si dos secuencias se parecen más de

lo esperable por azar, podemos plantear que comparten un ancestro. Esa hipótesis de *homología* sustenta muchas tareas de este libro: anotar genes, predecir funciones, reconstruir árboles, mapear lecturas y llamar variantes. La herramienta para examinarla es el *alineamiento*, que construiremos desde sus fundamentos.

Desde el inicio conviene distinguir dos conceptos. La *similitud* se mide: puede expresarse como porcentaje de identidad o como puntuación. La *homología* es una hipótesis histórica sobre el origen común: dos secuencias son homólogas o no lo son; no son “un 40 % homólogas”. El alineamiento cuantifica la similitud y la estadística ayuda a valorar si respalda la hipótesis de homología. Doolittle (1981) formuló la cuestión en el título de su artículo: ante dos secuencias de aminoácidos parecidas, ¿azar o ascendencia común?

Avanzaremos en cuatro pasos. Primero miraremos las secuencias con los ojos, mediante una matriz de puntos (sección 3.1). Luego convertiremos esa intuición visual en un algoritmo exacto, la programación dinámica, y estudiaremos sus variantes y su coste (sección 3.2). Después nos preguntaremos cuánto debe valer cada coincidencia, cada sustitución y cada hueco, lo que nos llevará a las matrices de sustitución y a la teoría de la información (sección 3.3). Por último, escalaremos el problema a bases de datos con millones de secuencias y aprenderemos a distinguir un parecido significativo de uno casual con BLAST y la estadística de Karlin-Altschul (sección 3.4). Dos libros acompañan bien todo el capítulo: el de Durbin et al. (1998), con el enfoque probabilístico, y el de Gusfield (1997), con el algorítmico.

3.1 Dot plots: mirar antes de calcular

 Colab · 3.1 · Dot plots

 abrir notebook

Dos manuscritos sobre la mesa

Imagine que recibe dos copias manuscritas de un mismo poema, copiadas por escribas distintos con siglos de diferencia. Para compararlas, podría escribir una copia a lo largo del borde superior de una hoja cuadrículada y la otra a lo largo del borde izquierdo, y marcar con un punto cada casilla donde las dos letras coinciden. Allí donde los escribas copiaron fielmente, aparecerá una línea diagonal limpia; donde uno de ellos saltó un verso, la diagonal se desplazará; donde repitió una estrofa, aparecerá una segunda diagonal

paralela. Sin hacer una sola cuenta, la hoja le habrá contado la historia de las copias.

La hoja cuadriculada es un *dot plot* (matriz de puntos), representación propuesta por Gibbs y McIntyre (1970) cuando empezaban a acumularse las secuencias de proteínas. Permite ver simultáneamente las coincidencias y reconocer diagonales, interrupciones y patrones repetidos. Un alineamiento, como veremos en la sección 3.2, selecciona *un* camino por la matriz; el *dot plot* deja visibles también otros patrones, incluidas inversiones y translocaciones que un alineamiento lineal no puede representar.

3.1.1 Definición formal

Sean $x = x_1 x_2 \cdots x_n$ e $y = y_1 y_2 \cdots y_m$ dos secuencias sobre un alfabeto $\mathcal{A}$ (por ejemplo, $\mathcal{A} = \{A, C, G, T\}$).

Definición 3.1. (Dot plot con ventana)

Dada una ventana de longitud impar w y un umbral $t \leq w$, el *dot plot* de x contra y es la matriz binaria $D \in \{0, 1\}^{n \times m}$ con

$$D_{ij} = \mathbb{1} \left[\sum_{k=-h}^h \mathbb{1}[x_{i+k} = y_{j+k}] \geq t \right], \quad h = \frac{w-1}{2}. \quad (3.1)$$

DONDE

D_{ij}	Celda de la fila i (posición en x) y la columna j (posición en y); vale 1 si se dibuja un punto.
$\mathbb{1}[\cdot]$	Función indicadora: 1 si la condición es verdadera, o si no.
w, h	Longitud de la ventana deslizante y su semiancho.
t	Número mínimo de coincidencias dentro de la ventana para marcar la celda.

Observe que la ventana se desliza *a lo largo de la diagonal*: compara el segmento $x_{i-h..i+h}$ con el segmento $y_{j-h..j+h}$, que avanzan juntos. Esa elección no es casual. Una diagonal del *dot plot* representa dos segmentos alineados sin huecos, de modo que la ventana pregunta, en esencia: “¿se parecen estos dos fragmentos de longitud w lo suficiente como para sospechar que están alineados?”.

3.1.2 Ruido, señal y la elección de la ventana

Con $w = 1$ y $t = 1$ dibujamos un punto por cada par de letras idénticas. En ADN la señal queda oculta: si las cuatro bases son equiprobables, dos posiciones elegidas al azar coinciden con probabilidad $1/4$. Una ventana permite exigir varias coincidencias antes de marcar un punto. Bajo el supuesto de bases independientes y equiprobables, su número sigue una distribución $\text{Bin}(w, 1/4)$ y la probabilidad de un punto espurio es

$$p_{\text{ruido}} = \sum_{k=t}^w \binom{w}{k} \left(\frac{1}{4}\right)^k \left(\frac{3}{4}\right)^{w-k}. \quad (3.2)$$

Para $w = 11$, $t = 7$, esta probabilidad es de apenas 0,0076: menos de un punto espurio por cada cien celdas. Esta es la idea de filtrado que Maizel y Lenk (1981) popularizaron y que, como veremos, reaparece en el corazón de BLAST.

El ruido, sin embargo, es solo la mitad de la historia. Lo que queremos es que las diagonales *verdaderas* sobrevivan al filtro. Supongamos que dos segmentos homólogos conservan una fracción π de identidad, de modo que cada posición coincide con probabilidad π independientemente de las demás. La probabilidad de que la ventana marque un punto sobre la diagonal verdadera, que llamaremos *sensibilidad*, es entonces la misma suma binomial con π en lugar de $1/4$:

$$p_{\text{señal}}(\pi) = \sum_{k=t}^w \binom{w}{k} \pi^k (1 - \pi)^{w-k}, \quad \mathbb{E}[\text{puntos espurios}] = nm p_{\text{ruido}}. \quad (3.3)$$

DONDE

π	Fracción de identidad esperada entre las regiones homólogas.
$p_{\text{señal}}$	Probabilidad de que una celda de la diagonal verdadera reciba un punto.
nm	Número de celdas de la matriz; multiplica el ruido por celda.

El umbral plantea un compromiso: al aumentar t reducimos el ruido, pero también perdemos sensibilidad. Podemos aumentar w a la vez para separar mejor las dos distribuciones. La cota de Chernoff cuantifica cómo cae el ruido: si $\tau = t/w$ es la fracción de coincidencias exigida y $\tau > 1/4$,

$$p_{\text{ruido}} \lesssim \exp\left[-wD\left(\tau \parallel \frac{1}{4}\right)\right], \quad D(\tau \parallel p) = \tau \ln \frac{\tau}{p} + (1 - \tau) \ln \frac{1 - \tau}{1 - p}. \quad (3.4)$$

DONDE

τ	Fracción de coincidencias que exige el umbral, t/w .
$D(\tau\ p)$	Divergencia de Kullback-Leibler (entropía relativa) entre dos monedas de probabilidades τ y p , en nats.

La ecuación (3.4) dice que, con τ fijo, el ruido decae *exponencialmente* con la longitud de la ventana, con una tasa igual a la entropía relativa. Es una cota superior (para $w = 21$, $t = 15$ da $4,7 \times 10^{-5}$ frente al valor exacto de $1,0 \times 10^{-5}$), pero captura lo esencial, y anticipa un tema que volverá con fuerza en las secciones 3.3 y 3.4: la entropía relativa mide cuánta evidencia aporta, por posición, una región similar frente al azar.

Ejemplo 3.1. (Diseñar el filtro para dos genomas virales)

Queremos comparar dos genomas de 10 kb cuyas regiones homólogas tienen alrededor de $\pi = 0,85$ de identidad. La matriz tiene $nm = 10^8$ celdas. Con $w = 11$, $t = 7$ la ecuación (3.3) da una sensibilidad excelente, $p_{\text{señal}} = 0,984$, pero el número esperado de puntos espurios es $10^8 \times 0,0076 \approx 760\,000$: la figura sería un cielo estrellado. Con $w = 21$, $t = 15$ el ruido baja a $1,0 \times 10^{-5}$, es decir, unos 1000 puntos dispersos en 10^8 celdas (invisibles a la escala de la figura), mientras que la sensibilidad sigue siendo 0,971. Si las regiones homólogas fueran más divergentes, con $\pi = 0,70$, esa misma ventana solo marcaría el 55 % de las celdas verdaderas: la diagonal aparecería punteada, pero seguiría siendo visible porque el ojo integra a lo largo de ella. Esta es la lógica que *dotter* automatiza al permitir ajustar el umbral de forma interactiva mientras se mira la figura (Sonnhammer & Durbin, 1995).

3.1.3 Puntuaciones en lugar de identidades

Para proteínas, contar identidades desperdicia información: una leucina frente a una isoleucina es casi tan buena señal como dos leucinas. La generalización natural reemplaza la indicadora de la ecuación (3.1) por una puntuación de sustitución $s(a, b)$, como las matrices que construiremos en la sección 3.3:

$$D_{ij} = \mathbb{1} \left[\sum_{k=-h}^h s(x_{i+k}, y_{j+k}) \geq T \right]. \quad (3.5)$$

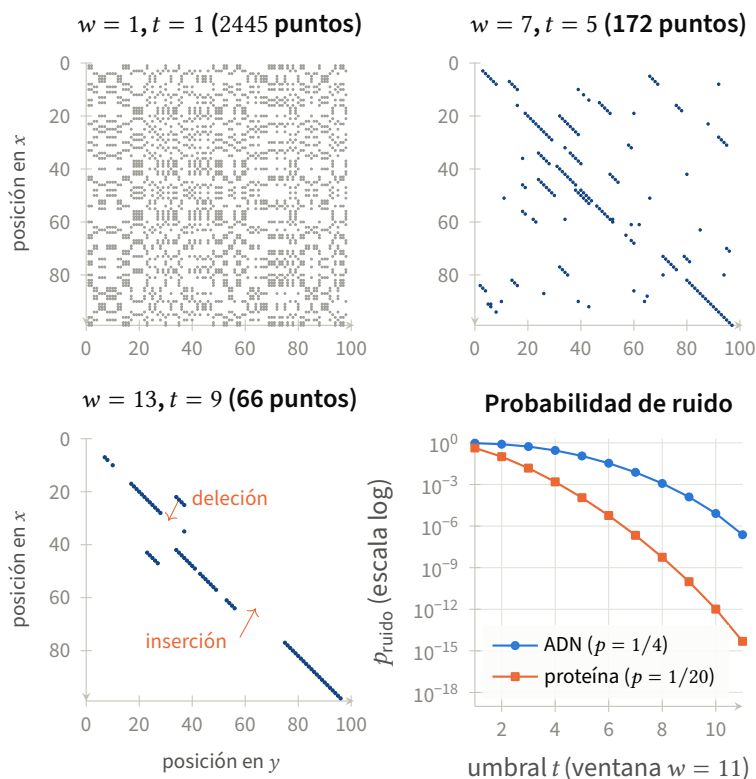


Figura 3.1. El mismo par de secuencias de ADN simuladas (100 pb; y es una copia de x con 15 % de sustituciones, una delección de 8 pb y una inserción de 6 pb) visto con tres filtros distintos. Con $w = t = 1$ (arriba a la izquierda) una cuarta parte de las celdas tiene un punto y la diagonal verdadera se ahoga en el ruido. Con $w = 7, t = 5$ la diagonal emerge, y con $w = 13, t = 9$ el ruido casi desaparece y se leen los dos *indels* como saltos laterales de la diagonal (flechas). Abajo a la derecha, la ecuación (3.2) para $w = 11$: con 20 letras el ruido cae mucho más rápido que con 4, por eso los *dot plots* de proteínas toleran ventanas más cortas. Datos de `figuras/cap03/generar.py`.

DONDE

$s(a, b)$	Puntuación de sustitución (p. ej., BLOSUM62); positiva para pares frecuentes entre homólogos.
T	Umbral de puntuación de la ventana, en las unidades de la matriz.

Ya no hay una fórmula binomial para el ruido, pero podemos calibrar T por simulación: generamos muchas ventanas aleatorias con las frecuencias de aminoácidos

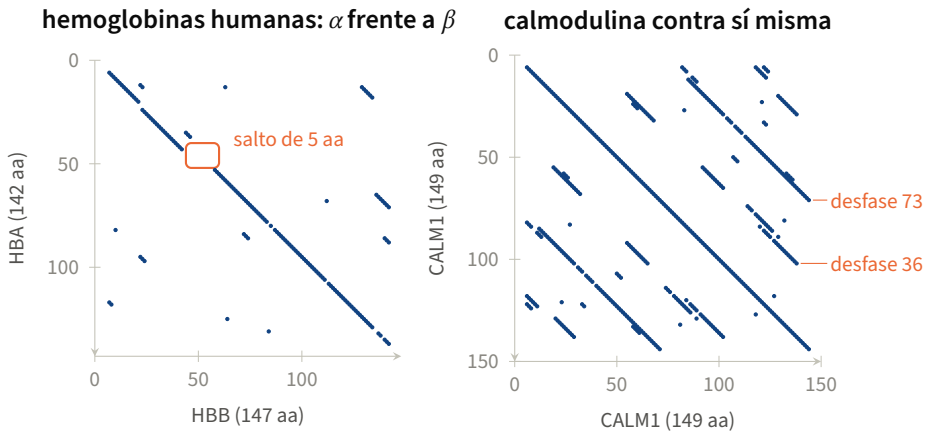


Figura 3.2. Izquierda: cadenas α (HBA, UniProt P69905) y β (HBB, P68871) de la hemoglobina humana, con ventana $w = 11$ y BLOSUM62, umbral $T = 15$. La diagonal casi continua refleja la homología de extremo a extremo (43 % de identidad); su desplazamiento hacia la izquierda a partir de la posición ~ 50 de HBB (recuadro) corresponde a los cinco residuos que HBA no tiene en esa región, y que un alineamiento global codificará como un hueco. Derecha: calmodulina humana (P0DP23) comparada consigo misma. Además de la diagonal principal, aparecen diagonales paralelas desplazadas unos 36–40 residuos (manos EF vecinas) y, con más fuerza, 73 residuos: los dos lóbulos de la proteína, cada uno con dos manos EF, son el producto de una duplicación interna. Datos calculados en `figuras/cap03/generar.py`.

de fondo y tomamos como umbral un cuantil alto de su puntuación. Para $w = 11$ y BLOSUM62, el cuantil 0,999 resulta ser $T = 15$. La figura 3.2 aplica ese filtro a dos casos reales.

Ejemplo 3.2. (Leer la historia evolutiva en un dot plot)

En la figura 3.2, la diagonal de las hemoglobinas no sigue exactamente la recta $x = y$. Si contamos los desfases de los 148 puntos, la mayoría está en la diagonal “posición en HBA = posición en HBB -5 ”, y unos pocos en desfases ± 1 cerca del extremo amino. La lectura es inmediata: los dos genes descenden de una duplicación antigua y, desde entonces, una de las copias perdió (o la otra ganó) unos cinco codones en la región de la hélice D. Un alineamiento global con BLOSUM62 y penalizaciones afines (Biopython, apertura 11, extensión 1) confirma la lectura: 65 identidades en 149 columnas, con el hueco principal

justamente allí. La calmodulina cuenta otra historia: la proteína es un tándem de cuatro motivos similares, y los desfases de 36–40 y 73 residuos registran dos rondas de duplicación interna. Ningún alineamiento global puede mostrar esto de un vistazo, porque un alineamiento lineal no puede usar la misma región dos veces.

3.1.4 Cómo se lee un dot plot

La figura 3.3 muestra un ejemplo construido a propósito: la secuencia y es una copia de x en la que un segmento central fue invertido y complementado. Aparecen los motivos que conviene aprender a reconocer:

- **Diagonal principal:** regiones idénticas u homólogas en el mismo orden.
- **Diagonales paralelas desplazadas:** repeticiones, pues un mismo segmento de x se parece a dos lugares de y .
- **Salto lateral de la diagonal:** inserciones o deleciones (*indels*); la diagonal continúa, pero desplazada una fila o una columna.
- **Antidiagonales:** inversiones; se detectan comparando x con el *reverso complementario* de y (puntos naranjas).
- **Bloques cuadrados:** regiones de baja complejidad (p. ej., **ATATATAT**), que coinciden consigo mismas en cualquier desfase.

Una regla práctica útil al leer desfases: si un punto está en la celda (i, j) , su *diagonal* es $d = j - i$. Todas las celdas de un mismo segmento alineado sin huecos comparten d ; un *indel* de longitud g cambia d en $\pm g$. Esta observación, trivial en apariencia, es la base de las heurísticas de BLAST (sección 3.4), que agrupan semillas por diagonal, y del alineamiento con bandas (sección 3.2), que solo explora diagonales cercanas a una dada.

3.1.5 Algoritmos: de la matriz completa a las semillas

La implementación directa de la ecuación (3.1) cuesta $O(nmw)$ operaciones, porque cada celda suma w términos. Un truco elemental baja el coste a $O(nm)$: a lo largo de cada diagonal, la suma de la ventana en la celda $(i+1, j+1)$ se obtiene de la de (i, j) sumando el término que entra y restando el que sale,

$$\Sigma_{i+1, j+1} = \Sigma_{i, j} + 1[x_{i+h+1} = y_{j+h+1}] - 1[x_{i-h} = y_{j-h}]. \quad (3.6)$$

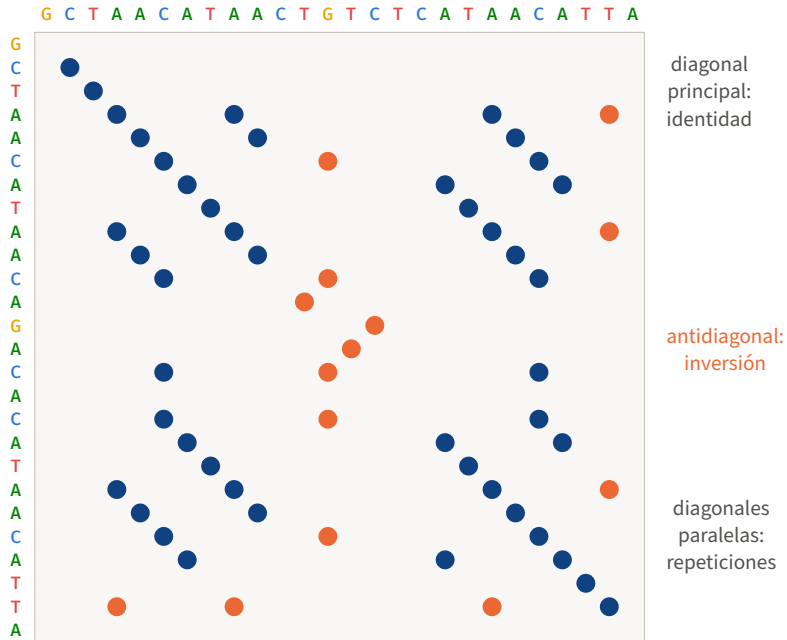


Figura 3.3. Anatomía de un *dot plot* ($w = 3, t = 3$). Las filas corresponden a x y las columnas a y . En azul, coincidencias directas: la diagonal principal y dos diagonales paralelas producidas por un segmento repetido. En naranja, coincidencias con el reverso complementario, que revelan el segmento invertido como una antidiagonal. Figura generada con `figuras/cap03/generar.py`.

Aun así, la memoria y el tiempo cuadráticos son prohibitivos para cromosomas. Las herramientas modernas invierten la lógica: en lugar de recorrer la matriz, indexan los k -mers de una secuencia en una tabla *hash* y, para cada k -mer de la otra, consultan en tiempo constante dónde aparece. El coste pasa a ser $O(n + m + \text{coincidencias})$, y la figura se dibuja solo con los puntos que existen. Así funcionan, con muchos refinamientos, *dotter* y los modos de comparación de genomas de *minimap2* (Li, 2018; Sonnhammer & Durbin, 1995).

Ejemplo 3.3. (Un dot plot en diez líneas)

El siguiente fragmento calcula la matriz de la ecuación (3.1) de forma vectorizada: en lugar de recorrer ventanas, se suman copias desplazadas (a lo largo de la diagonal) de la matriz de identidad. Los desplazamientos se hacen

con rebanadas, no con `np.roll`, que “daría la vuelta” a los bordes y crearía puntos falsos.

```
dotplot.py

import numpy as np

def dotplot(x: str, y: str, w: int = 11, t: int = 7) -> np.ndarray:
    """Matriz binaria D (n x m) con ventana w y umbral t."""
    X = np.frombuffer(x.encode(), dtype=np.uint8)
    Y = np.frombuffer(y.encode(), dtype=np.uint8)
    eq = (X[:, None] == Y[None, :]).astype(np.int16)
    n, m, h = len(x), len(y), w // 2
    acc = np.zeros_like(eq)
    for k in range(-h, h + 1):
        # sumar sobre la diagonal
        i0, i1 = max(0, -k), min(n, n - k)
        j0, j1 = max(0, -k), min(m, m - k)
        acc[i0:i1, j0:j1] += eq[i0 + k:i1 + k, j0 + k:j1 + k]
    return acc >= t
```

Con dos genomas de 30 kb la matriz tiene 9×10^8 celdas; por eso las herramientas reales calculan solo semillas de k -mers exactos, como en el siguiente código, que devuelve las coordenadas de todas las coincidencias de longitud k sin construir la matriz:

```
kmer_dotplot.py

from collections import defaultdict

def kmer_hits(x: str, y: str, k: int = 12):
    """Pares (i, j) tales que x[i:i+k] == y[j:j+k]."""
    indice = defaultdict(list)
    for j in range(len(y) - k + 1):
        indice[y[j:j + k]].append(j)
    return [(i, j) for i in range(len(x) - k + 1)
            for j in indice.get(x[i:i + k], ())]
```

⚠ Lo que un dot plot no hace

Un *dot plot* no produce un alineamiento ni una puntuación. Es una herramienta de *exploración*: sugiere hipótesis (“aquí hay una duplicación en

tándem”) que luego deben cuantificarse. Tampoco escala en su forma ingenua: su coste en memoria es $O(nm)$. Y su aspecto depende por completo de w y t : una diagonal “ausente” puede deberse a un filtro demasiado estricto y no a la falta de homología, como muestra la figura 3.1.

3.2 Programación dinámica: Needleman-Wunsch y Smith-Waterman

 Colab · 3.2 · Programación dinámica

 abrir notebook

3.2.1 ¿Qué es un alineamiento?

Un *alineamiento* de x e y es una forma de escribir una secuencia encima de la otra, insertando huecos (*gaps*, representados con “-”) para que las posiciones que creemos que descienden de un mismo ancestro queden en la misma columna. Cada columna es de uno de tres tipos:

Columna	Ejemplo	Interpretación evolutiva
Coincidencia (<i>match</i>)	$\begin{pmatrix} A \\ A \end{pmatrix}$	la posición se conservó
Sustitución (<i>mismatch</i>)	$\begin{pmatrix} A \\ G \end{pmatrix}$	hubo una mutación puntual
Hueco (<i>gap</i>)	$\begin{pmatrix} A \\ - \end{pmatrix}, \begin{pmatrix} - \\ A \end{pmatrix}$	inserción en una rama o delección en la otra

Un alineamiento es, por lo tanto, una *hipótesis evolutiva* escrita en forma de tabla. Y como toda hipótesis, puede ser mejor o peor. Para compararlas asignamos a cada columna una puntuación: $s(a, b)$ para un par de letras y $-d$ por cada posición con hueco. La puntuación del alineamiento es la suma de sus columnas, y el *alineamiento óptimo* es el de puntuación máxima.

Definición 3.2. (Alineamiento global y su puntuación)

Un alineamiento global de x (longitud n) e y (longitud m) es una sucesión de columnas $\binom{u_1}{v_1} \cdots \binom{u_L}{v_L}$, con $u_k, v_k \in \mathcal{A} \cup \{-\}$, tal que ninguna columna es $\binom{-}{-}$ y, al borrar los huecos, la fila superior es x y la inferior es y . Su puntuación con penalización lineal es

$$S = \sum_{k: u_k, v_k \neq -} s(u_k, v_k) - d \cdot \#\{k : u_k = - \text{ o } v_k = -\}. \quad (3.7)$$

DONDE

L	Longitud del alineamiento (número de columnas), $\max(n, m) \leq L \leq n + m$.
u_k, v_k	Símbolos de la columna k en la fila de x y en la de y .
s, d	Puntuación de sustitución y costo lineal de cada posición con hueco.

El problema de contar caminos

¿Por qué no probar todos los alineamientos y quedarnos con el mejor? Porque son astronómicamente muchos. El número de alineamientos globales de dos secuencias de longitud n crece aproximadamente como

$$\binom{2n}{n} \approx \frac{2^{2n}}{\sqrt{\pi n}},$$

de modo que para dos proteínas modestas de 300 residuos hay unos 10^{179} alineamientos, muchísimos más que átomos en el universo observable (unos 10^{80}). Necesitamos una idea más astuta que la fuerza bruta.

3.2.2 El grafo de edición y el conteo exacto de alineamientos

La cifra del recuadro anterior merece una derivación, porque el razonamiento que la produce es, casi palabra por palabra, el mismo que resolverá el problema de optimización. Dibujemos una rejilla de $(n+1) \times (m+1)$ nodos, (i, j) con $0 \leq i \leq n$, $0 \leq j \leq m$, y tres tipos de aristas: diagonales $(i-1, j-1) \rightarrow (i, j)$, que alinean x_i con y_j ; verticales $(i-1, j) \rightarrow (i, j)$, que ponen x_i frente a un hueco; y horizontales $(i, j-1) \rightarrow (i, j)$, que ponen y_j frente a un hueco. Este es el *grafo de edición* (figura 3.4).

Teorema 3.1. (Alineamientos y caminos)

Existe una biyección entre los alineamientos globales de x e y y los caminos dirigidos de $(0, 0)$ a (n, m) en el grafo de edición. Si cada arista lleva como peso la puntuación de su columna, la puntuación del alineamiento es el peso del camino, y el alineamiento óptimo es el camino de peso máximo.

Contemos, pues, caminos. Sea $N(i, j)$ el número de caminos de $(0, 0)$ a (i, j) . Todo camino llega a (i, j) por exactamente una de sus tres aristas de entrada, así que

$$N(i, j) = N(i-1, j-1) + N(i-1, j) + N(i, j-1), \quad N(i, 0) = N(0, j) = 1, \quad (3.8)$$

cuya solución cerrada es el *número de Delannoy*

$$N(n, m) = \sum_{k=0}^{\min(n, m)} \binom{n}{k} \binom{m}{k} 2^k. \quad (3.9)$$

DONDE

$N(i, j)$	Número de caminos (alineamientos) entre los prefijos $x_{1..i}$ e $y_{1..j}$.
k	Número de columnas diagonales (pares de letras alineados).
$\binom{n}{k} \binom{m}{k}$	Formas de elegir qué k letras de x y cuáles de y quedan emparejadas.
2^k	Formas de intercalar los huecos: entre dos pares consecutivos, los huecos de x pueden ir antes o después de los de y .

¿De dónde sale, entonces, el binomial del recuadro? Del hecho de que las columnas $\binom{A}{(-)} \binom{(-)}{C}$ y $\binom{(-)}{C} \binom{A}{(-)}$ describen la misma hipótesis biológica: ni A ni C tienen homólogo. Si identificamos esos alineamientos, que solo difieren en el orden de huecos adyacentes, lo único que importa es qué pares se emparejan, y su número es $\sum_k \binom{n}{k} \binom{m}{k} = \binom{n+m}{n}$ (identidad de Vandermonde). Para $n = m = 3$ hay $\binom{6}{3} = 20$ alineamientos distintos, pero 63 caminos en el grafo; para $n = m = 300$, $10^{179,1}$ y $10^{228,2}$, respectivamente (figura 3.4, derecha). Ambos crecen exponencialmente: el binomial como 4^n , el de Delannoy como $(3 + 2\sqrt{2})^n \approx 5,83^n$. La conclusión es la misma con cualquier convención: la enumeración es imposible.

3.2.3 El principio de optimalidad

La idea astuta proviene de la investigación de operaciones. Bellman (1954) observó que muchos problemas de decisión secuencial satisfacen un *principio de optimalidad*: la mejor solución global contiene, como piezas, las mejores soluciones

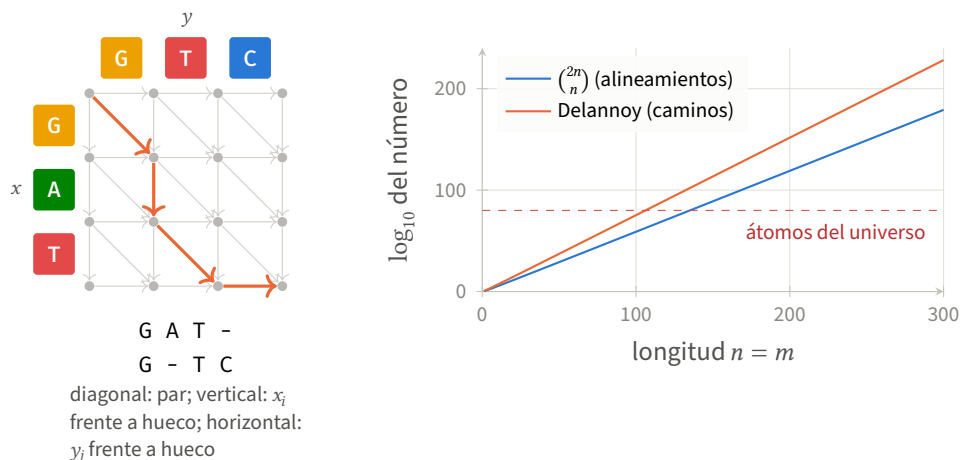


Figura 3.4. Izquierda: grafo de edición para $x = \text{GAT}$ e $y = \text{GTC}$. Cada camino de la esquina superior izquierda a la inferior derecha es un alineamiento; el camino naranja (diagonal, vertical, diagonal, horizontal) corresponde al alineamiento mostrado. Derecha: el número de alineamientos crece exponencialmente con la longitud; a partir de $n \approx 140$ supera el número de átomos del universo observable. La programación dinámica evita la enumeración porque la recurrencia de la ecuación (3.8), con $\max$ en lugar de suma, calcula el mejor camino con el mismo esfuerzo con que se cuentan todos.

de subproblemas más pequeños. En el alineamiento, esto se traduce en una observación casi evidente una vez que se enuncia: la última columna de un alineamiento óptimo solo puede ser de tres tipos, y lo que queda a su izquierda debe ser, a su vez, un alineamiento óptimo de los prefijos correspondientes. Si no lo fuera, podríamos sustituirlo por uno mejor y mejorar el total, lo que contradice la optimalidad. Eddy (2004a) ofrece una introducción breve y muy clara a esta forma de pensar.

Teorema 3.2. (Subestructura óptima)

Sea $\mathcal{A}^*$ un alineamiento óptimo de $x_{1..i}$ e $y_{1..j}$ cuya última columna es c . Entonces lo que queda al quitar c es un alineamiento óptimo del par de prefijos que corresponde: $(x_{1..i-1}, y_{1..j-1})$ si $c = \begin{pmatrix} x_i \\ y_j \end{pmatrix}$, $(x_{1..i-1}, y_{1..j})$ si $c = \begin{pmatrix} x_i \\ - \end{pmatrix}$ y $(x_{1..i}, y_{1..j-1})$ si $c = \begin{pmatrix} - \\ y_j \end{pmatrix}$.

La demostración es del tipo “cortar y pegar”, y vale la pena escribirla porque todas las recurrencias del capítulo se justifican igual. Supongamos que $c = \begin{pmatrix} x_i \\ y_j \end{pmatrix}$ (los

otros casos son idénticos) y que el resto, $\mathcal{B}$, no fuera óptimo para $(x_{1..i-1}, y_{1..j-1})$: existiría $\mathcal{B}'$ con $S(\mathcal{B}') > S(\mathcal{B})$. Como la puntuación es *aditiva por columnas*, $S(\mathcal{B}'c) = S(\mathcal{B}') + s(x_i, y_j) > S(\mathcal{B}) + s(x_i, y_j) = S(\mathcal{A}^*)$, y $\mathcal{B}'c$ es un alineamiento válido de $x_{1..i}$ e $y_{1..j}$, lo que contradice la optimalidad de $\mathcal{A}^*$. $\square$

Fíjese en qué se apoyó el argumento: únicamente en que la puntuación de una columna no depende de las demás. Cuando esa independencia se rompe, como ocurre con los huecos afines, habrá que enriquecer el estado de la recurrencia para recuperarla; eso es exactamente lo que hará Gotoh más adelante.

3.2.4 Needleman-Wunsch: alineamiento global

Sea $F(i, j)$ la puntuación del mejor alineamiento entre los prefijos $x_{1..i}$ e $y_{1..j}$. Needleman y Wunsch (1970) mostraron que F se puede llenar como una tabla, fila por fila, con la recurrencia

Teorema 3.3. (Recurrencia de Needleman-Wunsch)

Con penalización lineal $d > 0$ por posición de hueco,

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) & (\text{alinear } x_i \text{ con } y_j) \\ F(i-1, j) - d & (x_i \text{ frente a un hueco}) \\ F(i, j-1) - d & (y_j \text{ frente a un hueco}) \end{cases} \quad (3.10)$$

con condiciones de frontera $F(0, 0) = 0$, $F(i, 0) = -id$ y $F(0, j) = -jd$. La puntuación óptima es $F(n, m)$.

DONDE

$F(i, j)$	Mejor puntuación posible alineando los primeros i símbolos de x con los primeros j de y .
$s(a, b)$	Puntuación de sustitución de la letra a por la b (positiva si se parecen).
d	Costo de cada posición alineada contra un hueco.
n, m	Longitudes de x e y .

La corrección de la recurrencia es consecuencia directa del teorema 3.2: el óptimo de (i, j) termina en uno de tres tipos de columna y, en cada caso, el resto es óptimo para el prefijo correspondiente, cuyo valor ya está en la tabla. Una inducción sobre

	ϵ	G	T	T	A	G	C	A
ϵ	0	-2	-4	-6	-8	-10	-12	-14
G	-2	1	-1	-3	-5	-7	-9	-11
A	-4	-1	0	-2	-2	-4	-6	-8
T	-6	-3	0	1	-1	-3	-5	-7
T	-8	-5	-2	1	0	-2	-4	-6
A	-10	-7	-4	-1	2	0	-2	-3
C	-12	-9	-6	-3	0	1	1	-1
A	-14	-11	-8	-5	-2	-1	0	2

Figura 3.5. Matriz de programación dinámica para $x = \text{GATTACA}$ (filas) e $y = \text{GTTAGCA}$ (columnas), con $s = +1$ para coincidencias, -1 para sustituciones y $d = 2$. Las celdas resaltadas y las flechas naranjas marcan el *traceback* desde la esquina inferior derecha. Los valores negativos aparecen en rojo.

$i + j$ completa el argumento. Compare la ecuación (3.10) con la ecuación (3.8): es la misma recurrencia con “máx” en lugar de “+” y con pesos en las aristas. Contar caminos y encontrar el mejor cuestan lo mismo.

La tabla se llena en $O(nm)$ operaciones, ya que cada celda consulta tres vecinas. Para recuperar el alineamiento, y no solo su puntuación, se recorre la tabla en sentido inverso (*traceback*) desde (n, m) , preguntando en cada celda cuál de los tres casos produjo el máximo. Una diagonal equivale a una columna de pares; un paso vertical, a una letra de x frente a un hueco; uno horizontal, a una letra de y frente a un hueco. Cuando dos o tres casos empatan, hay varios alineamientos co-óptimos; su número se puede contar con una segunda tabla, sumando los conteos de las celdas que alcanzan el máximo, y a menudo es sorprendentemente grande. La elección entre ellos (por ejemplo, “preferir la diagonal”) es una convención del programa, no una propiedad biológica.

Ejemplo 3.4. (Leyendo la matriz de la figura 3.5)

La puntuación óptima es $F(7, 7) = 2$. Siguiendo las flechas desde la esquina se obtiene

G	A	T	T	A	-	C	A
G	-	T	T	A	G	C	A

Seis coincidencias (+6) y dos huecos (−4) dan $6 - 4 = 2$, lo que confirma el valor de la esquina. Observe el primer paso vertical tras la G: el algoritmo “decidió” que era mejor pagar un hueco que forzar una A contra una T, porque el hueco permite que el bloque TTA quede perfectamente alineado.

needleman_wunsch.py

```
def needleman_wunsch(x, y, match=1, mismatch=-1, gap=2):
    n, m = len(x), len(y)
    F = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1): F[i][0] = -i * gap      # frontera
    for j in range(1, m + 1): F[0][j] = -j * gap      # solo huecos
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            s = match if x[i - 1] == y[j - 1] else mismatch
            F[i][j] = max(F[i - 1][j - 1] + s,          # diagonal
                          F[i - 1][j] - gap,            # hueco en y
                          F[i][j - 1] - gap)            # hueco en x
    return F[n][m], F
```

3.2.5 Distancia de edición: el problema dual

Los informáticos llegaron al mismo algoritmo por otro camino. La *distancia de edición* o de Levenshtein, $d_L(x, y)$, es el número mínimo de operaciones elementales (sustituir, insertar o borrar un símbolo) que transforman x en y . Wagner y Fischer (1974) la calcularon con la recurrencia

$$D(i, j) = \min \begin{cases} D(i - 1, j - 1) + 1[x_i \neq y_j] \\ D(i - 1, j) + 1 \\ D(i, j - 1) + 1 \end{cases} \quad D(i, 0) = i, \quad D(0, j) = j, \quad (3.11)$$

que es Needleman-Wunsch con $s(a, a) = 0$, $s(a, b) = -1$ y $d = 1$, cambiando el signo: $D(i, j) = -F(i, j)$. Maximizar similitud y minimizar distancia son el mismo problema cuando las puntuaciones se eligen de forma compatible. Sellers (1974) y Waterman et al. (1976) estudiaron cuándo esta distancia es una métrica genuina

(simétrica, que se anula solo entre secuencias iguales y que cumple la desigualdad triangular) y generalizaron el cálculo a costos arbitrarios, incluidos costos de hueco que dependen de su longitud.

Por ejemplo, la tabla de la ecuación (3.11) para GATTACA y GCATGCA da $D(7, 7) = 3$, con un camino óptimo

G	-	A	T	T	A	C	A
G	C	A	T	-	G	C	A

que usa una inserción (C), una delección (T) y una sustitución (A→G). Ese camino nunca se aleja más de una diagonal de la principal, lo que anticipa la idea de bandas: si sabemos que $d_L \leq k$, el camino óptimo no puede salir de la franja $|i - j| \leq k$, y basta con llenar $O(kn)$ celdas (Ukkonen, 1985).

3.2.6 Variantes: global, semiglobal y de solapamiento

La ecuación (3.10) castiga por igual los huecos del interior y los de los extremos. Eso es razonable si x e y son homólogos de extremo a extremo, pero no en dos situaciones muy frecuentes. Al mapear una lectura corta contra un gen, queremos que la lectura se alinee completa, pero sin pagar por las partes del gen que quedan fuera: es el alineamiento *semiglobal* o de ajuste (*fitting*). Al ensamblar, queremos detectar que el final de una lectura coincide con el principio de otra: es el alineamiento de *solapamiento* (*overlap*). Ambos se obtienen con la misma recurrencia cambiando únicamente dos cosas: dónde empieza gratis el camino y dónde se lee el resultado (tabla 3.1).

Un ejemplo muestra la diferencia. Con $x = \text{ACGGTTAGC}$, $y = \text{TAGCTTGA}$, $+1/-1$ y $d = 2$, el alineamiento global obtiene -2 : no parece haber relación. El de solapamiento no cobra los huecos de los extremos y encuentra que el sufijo TAGC de x es el prefijo de y , con puntuación $+4$:

A	C	G	G	T	T	A	G	C	-	-	-	-
-	-	-	-	-	T	A	G	C	T	T	G	A

Así funcionan, en esencia, los ensambladores de tipo *overlap-layout-consensus*; en Biopython basta con fijar `end_gap_score=0` en `PairwiseAligner`.

3.2.7 Smith-Waterman: alineamiento local

El alineamiento global asume que las dos secuencias son homólogos de extremo a extremo. Muchas veces no es así: dos proteínas pueden compartir un único dominio

Tabla 3.1. Las variantes de alineamiento difieren solo en las fronteras y en la celda donde se lee el óptimo; la recurrencia interior es siempre la ecuación (3.10) (o la ecuación (3.12) en el caso local).

Variante	Frontera	Óptimo en	Uso típico
Global (NW)	$F(i, 0) = -id$, $F(0, j) = -jd$	$F(n, m)$	proteínas completas homólogos
Semiglobal	$F(i, 0) = -id$, $F(0, j) = 0$	$\max_j F(n, j)$	lectura x dentro de un gen y
Solapamiento	$F(i, 0) = 0$, $F(0, j) = 0$	$\max_j F(n, j)$ o $\max_i F(i, m)$	ensamblaje de lecturas
Local (SW)	$H(i, 0) = H(0, j) = 0$ y $\max\{0, \dots\}$	$\max_{i,j} H(i, j)$	dominios, búsqueda en bases de datos

en medio de regiones sin relación, o una lectura corta de secuenciación puede provenir de un pequeño trozo de un cromosoma. En esos casos queremos el *par de subcadenas* más parecido. Smith y Waterman (1981) lo consiguieron con un cambio mínimo pero profundo: permitir que un alineamiento “vuelva a empezar” en cualquier celda.

Teorema 3.4. (Recurrencia de Smith-Waterman)

$$H(i, j) = \max \begin{cases} 0 & (\text{empezar de nuevo}) \\ H(i-1, j-1) + s(x_i, y_j) \\ H(i-1, j) - d \\ H(i, j-1) - d \end{cases} \quad (3.12)$$

con $H(i, 0) = H(0, j) = 0$. La puntuación local óptima es $\max_{i,j} H(i, j)$ y el traceback termina al encontrar un 0.

DONDE

$H(i, j)$	Mejor puntuación de un alineamiento local que <i>termina</i> exactamente en x_i e y_j (o o si todos son negativos).
0	Opción de empezar un alineamiento nuevo en (i, j) , sin arrastrar un prefijo perjudicial.

El cero cumple un papel decisivo. Un prefijo que acumula puntuación negativa nunca ayuda a una extensión posterior, así que la recurrencia lo “olvida”. Para que esto funcione, la puntuación esperada de un par aleatorio debe ser negativa, $\sum_{a,b} p_a p_b s(a, b) < 0$; de lo contrario, los alineamientos locales crecerían indefinidamente y se volverían, en la práctica, globales. Esta condición reaparecerá como supuesto central de la estadística de BLAST.

La afirmación anterior tiene un contenido matemático preciso. Arratia y Waterman (1994) demostraron que, para dos secuencias aleatorias de longitud n , la puntuación local óptima presenta una *transición de fase* en el espacio de parámetros (s, d) : en una región crece como $\log n$ (régimen local, el útil) y en la complementaria crece linealmente con n (régimen global). Vingron y Waterman (1994) discutieron las consecuencias prácticas: las combinaciones de matriz y penalizaciones de hueco deben elegirse dentro del régimen logarítmico, porque solo allí la puntuación máxima de dos secuencias no relacionadas es pequeña y la de dos homólogas destaca. Para alineamientos sin huecos, el régimen logarítmico coincide exactamente con la condición $E[s] < 0$.

Ejemplo 3.5. (Recorrer un alineamiento local)

En la figura 3.6, el máximo $H(6, 7) = 13$ corresponde al alineamiento

G	T	T	-	A	C
G	T	T	G	A	C

Cinco coincidencias ($5 \times 3 = 15$) y un hueco (-2) suman 13. El *traceback* se detiene en la celda $(1, 1)$, cuyo valor es 0: la T inicial de x y la G inicial de y no aportan nada y el algoritmo simplemente las deja fuera. Si en lugar del máximo global buscáramos el *segundo* mejor alineamiento local, no bastaría con tomar la segunda celda más alta, porque casi siempre pertenece al mismo camino. Waterman y Eggert (1987) resolvieron este problema de los alineamientos

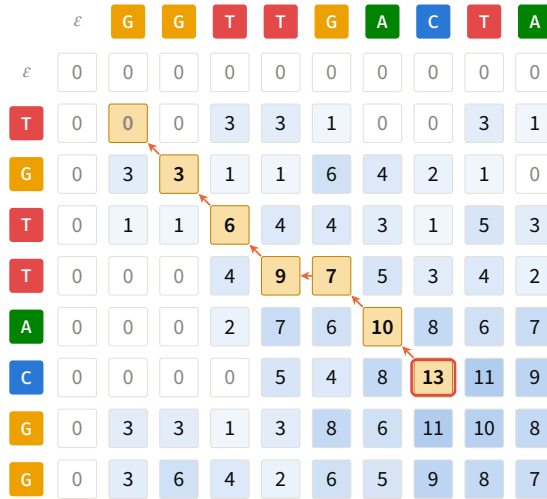


Figura 3.6. Matriz de Smith-Waterman para $x = \text{TGGTACGG}$ (filas) e $y = \text{GGTGACTA}$ (columnas), con $+3/-3$ y $d = 2$. A diferencia de la figura 3.5, no hay valores negativos: el cero de la recurrencia los trunca (celdas blancas). La intensidad del azul crece con la puntuación. El máximo global, $H = 13$, está en la celda (6,7) (recuadro rojo); el *traceback* (flechas naranjas) avanza en diagonal y con un paso horizontal hasta tocar un cero, y produce GTT-AC frente a GTTGAC. Las letras de x e y fuera de ese recorrido no participan en el alineamiento.

subóptimos no solapados recalculando la matriz tras “prohibir” las parejas ya usadas.

smith_waterman.py

```
def smith_waterman(x, y, match=3, mismatch=-3, gap=2):
    n, m = len(x), len(y)
    H = [[0] * (m + 1) for _ in range(n + 1)]
    best, bi, bj = 0, 0, 0
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            s = match if x[i - 1] == y[j - 1] else mismatch
            H[i][j] = max(0, H[i - 1][j - 1] + s,
                          H[i - 1][j] - gap, H[i][j - 1] - gap)
            if H[i][j] > best:
                best, bi, bj = H[i][j], i, j
    ax, ay, i, j = "", "", bi, bj
    while H[i][j] > 0:
```

traceback hasta un 0

```

s = match if x[i - 1] == y[j - 1] else mismatch
if H[i][j] == H[i - 1][j - 1] + s:
    ax, ay, i, j = x[i - 1] + ax, y[j - 1] + ay, i - 1, j - 1
elif H[i][j] == H[i - 1][j] - gap:
    ax, ay, i = x[i - 1] + ax, "-" + ay, i - 1
else:
    ax, ay, j = "-" + ax, y[j - 1] + ay, j - 1
return best, ax, ay

print(smith_waterman("TGTTACGG", "GGTTGACTA"))
# (13, 'GTT-AC', 'GTTGAC')

```

🔗 Sellers, Waterman y la métrica evolutiva

Entre Needleman-Wunsch (1970) y Smith-Waterman (1981) media una década de formalización matemática. Sellers (1974) demostró que, bajo ciertas condiciones, la distancia de edición entre secuencias es una métrica genuina, lo que dio al alineamiento un fundamento geométrico, y Waterman et al. (1976) extendieron la teoría a pesos de hueco arbitrarios. Temple Smith y Michael Waterman, un físico y un matemático, trabajaban en Los Álamos cuando publicaron su algoritmo en un artículo de apenas tres páginas, hoy uno de los más citados de la biología computacional.

3.2.8 Huecos afines y el algoritmo de Gotoh

Con una penalización lineal, un hueco de 10 posiciones cuesta lo mismo que diez huecos de 1 posición dispersos. Biológicamente esto es absurdo: un único evento de inserción o deleción suele afectar a varias bases contiguas, y abrir un hueco es mucho más raro que alargarlo. La solución estándar es la *penalización afin*:

$$\gamma(g) = -(d_o + (g - 1)d_e), \quad d_o > d_e > 0, \quad (3.13)$$

DONDE

g	Longitud del hueco (número de posiciones consecutivas).
d_o	Costo de <i>abrir</i> el hueco (primer símbolo).
d_e	Costo de <i>extender</i> el hueco en cada posición adicional.

Para una función de hueco arbitraria $\gamma(g)$, Waterman et al. (1976) ya habían dado un algoritmo exacto, pero cada celda debe consultar todas las longitudes posibles del último hueco, lo que lleva el costo a $O(nm(n + m))$. Una función afin rompe la recurrencia simple, porque el costo de una celda depende de si el hueco ya estaba

abierto, pero su estructura permite recuperar la eficiencia. Gotoh (1982) resolvió el problema usando tres matrices acopladas, una por cada “estado” en el que puede terminar un alineamiento:

$$M(i, j) = s(x_i, y_j) + \max\{M, I_x, I_y\}(i-1, j-1), \quad (3.14)$$

$$I_x(i, j) = \max\{M(i-1, j) - d_o, I_x(i-1, j) - d_e\}, \quad (3.15)$$

$$I_y(i, j) = \max\{M(i, j-1) - d_o, I_y(i, j-1) - d_e\}. \quad (3.16)$$

DONDE

$M(i, j)$	Mejor puntuación de los prefijos $x_{1..i}, y_{1..j}$ si la última columna es el par $\begin{pmatrix} x_i \\ y_j \end{pmatrix}$.
$I_x(i, j)$	Ídem, si la última columna es $\begin{pmatrix} x_i \\ - \end{pmatrix}$ (hueco en y).
$I_y(i, j)$	Ídem, si la última columna es $\begin{pmatrix} - \\ y_j \end{pmatrix}$ (hueco en x).

Las fronteras son $M(0, 0) = 0$, $I_x(i, 0) = -d_o - (i-1)d_e$, $I_y(0, j) = -d_o - (j-1)d_e$, y $-\infty$ en las demás celdas de borde; el óptimo global es $\max\{M, I_x, I_y\}(n, m)$. La justificación es otra vez el teorema 3.2, pero aplicado a un problema con *estado*: sabiendo en cuál de las tres matrices termina el prefijo, el costo de la columna siguiente vuelve a ser independiente del resto del pasado. Tal como está escrita, la recurrencia no permite pasar directamente de I_x a I_y (un hueco en y seguido de uno en x), porque con $d_o > 0$ razonable ese patrón nunca es mejor que una sustitución; algunas implementaciones lo permiten explícitamente. El coste sigue siendo $O(nm)$ en tiempo.

Esta arquitectura de tres estados no es un truco aislado: es exactamente un *modelo oculto de Márkov de pares*, como mostraron Durbin et al. (1998), y la retomaremos en el capítulo 4 al estudiar perfiles y HMMER. En el HMM de pares (figura 3.7), el estado M emite un par de letras con probabilidad p_{ab} , los estados X e Y emiten una letra frente a un hueco con probabilidad q_a , y las transiciones valen δ (abrir un hueco), ε (extenderlo) y $1 - 2\delta$ o $1 - \varepsilon$ (volver a M). Tomando logaritmos de las probabilidades del camino de Viterbi y dividiendo por el modelo nulo se recuperan las ecuaciones de Gotoh con

$$s(a, b) = \log \frac{p_{ab}}{q_a q_b} + \log \frac{1 - 2\delta}{(1 - \eta)^2}, \quad (3.17)$$

$$d_o = -\log \frac{\delta(1 - \varepsilon)}{(1 - \eta)(1 - 2\delta)}, \quad d_e = -\log \frac{\varepsilon}{1 - \eta},$$

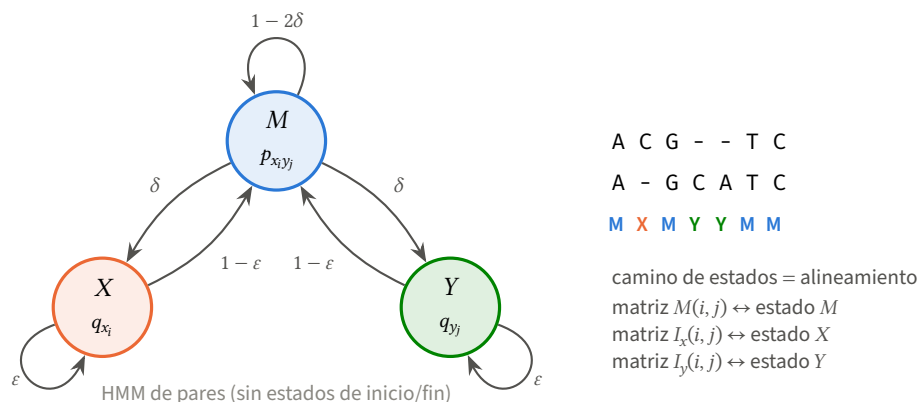


Figura 3.7. Las tres matrices de Gotoh vistas como un modelo oculto de Márkov de pares. El estado M emite un par alineado; X e Y emiten una letra de una sola secuencia frente a un hueco. Abrir un hueco ($M \rightarrow X$ o $M \rightarrow Y$, probabilidad δ) es más improbable que prolongarlo (ε), y el logaritmo de esas probabilidades da las penalizaciones d_o y d_e (ecuación (3.17)). A la derecha, un alineamiento y su camino de estados: cada columna es una visita a un estado, y cada matriz de la ecuación (3.14) guarda la mejor puntuación de los caminos que terminan en su estado.

donde η es la probabilidad de terminar en el modelo nulo (Durbin et al., 1998). La lectura es profunda: la penalización afín no es un parche heurístico, sino el logaritmo de una distribución geométrica de longitudes de hueco, con media $1/(1 - \varepsilon)$.

Ejemplo 3.6. (Lineal frente a afín)

Alineemos $x = \text{AAGTCGATCCTA}$ (12 nt) con $y = \text{AAGTCCTA}$ (8 nt), con +2 por coincidencia y -1 por sustitución. Con penalización lineal $d = 2$ el óptimo vale 8 (ocho coincidencias, 16, menos cuatro posiciones de hueco, 8), pero hay cinco alineamientos co-óptimos, y el que Biopython devuelve primero reparte los huecos en dos bloques:

AAGTCGATCCTA	AAGTCGATCCTA
AAGTC---C-TA	AAG----TCCTA
lineal, $d = 2$: $S = 8$	afín, $d_o = 5$, $d_e = 1$: $S = 8$

Con penalización afín $d_o = 5$, $d_e = 1$, el hueco único de 4 posiciones cuesta $5 + 3 = 8$ y la puntuación vuelve a ser $16 - 8 = 8$; en cambio, la solución de

dos bloques pagaría $(5 + 2) + 5 = 12$ y bajaría a 4. El modelo afín elimina, pues, las soluciones fragmentadas, que son biológicamente implausibles: un solo evento de delección de cuatro bases explica los datos mejor que dos eventos independientes.

⚠ Dos convenciones para el mismo hueco

La ecuación (3.13) cobra d_o por la *primera* posición y d_e por cada una de las siguientes; es la convención de Biopython (atributos `open_gap_score` y `extend_gap_score` de su alineador) y de EMBOSS. BLAST usa otra: un costo de *existencia* a más b por *cada* posición, $\gamma(g) = -(a + b g)$. Así, los parámetros por defecto de `blastp`, `-gapopen 11 -gapextend 1`, cobran 12 por un hueco de longitud 1 y equivalen a $d_o = 12$, $d_e = 1$ en nuestra notación. Traducir mal esta convención al comparar programas cambia los resultados sin dar ningún error.

3.2.9 Complejidad y memoria: el truco de Hirschberg

Needleman-Wunsch y Smith-Waterman cuestan $O(nm)$ en tiempo y en memoria. El tiempo es difícil de mejorar en general, pero la memoria es otra cosa. Para calcular solo la *puntuación* basta con guardar dos filas de la tabla, porque la ecuación (3.10) solo mira la fila anterior. El problema es el *traceback*, que necesita la tabla completa. Para alinear dos cromosomas bacterianos de 5 Mb, una matriz $n \times m$ necesitaría $2,5 \times 10^{13}$ celdas. Hirschberg (1975) mostró que el alineamiento óptimo puede recuperarse en espacio $O(n + m)$ con una idea de *divide y vencerás*.

Sea $F(i, j)$ la tabla hacia adelante y $B(i, j)$ la puntuación óptima de alinear los *sufijos* $x_{i+1..n}$ e $y_{j+1..m}$, que se obtiene aplicando la misma recurrencia a las secuencias invertidas. Todo camino de $(0, 0)$ a (n, m) cruza la fila central $i^* = \lfloor n/2 \rfloor$ en alguna columna, y su puntuación se descompone en la parte anterior y la posterior. Por lo tanto,

$$F(n, m) = \max_{0 \leq j \leq m} [F(i^*, j) + B(i^*, j)], \quad j^* = \arg \max_j [F(i^*, j) + B(i^*, j)]. \quad (3.18)$$

DONDE

$B(i, j)$	Mejor puntuación de alinear los sufijos $x_{i+1..n}$ e $y_{j+1..m}$.
i^*	Fila central de la tabla.
j^*	Columna por la que el camino óptimo cruza la fila central.

El camino óptimo pasa, por lo tanto, por (i^*, j^*) . Ambas filas se calculan en memoria lineal. Una vez conocido ese punto, el problema se parte en dos subproblemas independientes, $(x_{1..i^*}, y_{1..j^*})$ y $(x_{i^*+1..n}, y_{j^*+1..m})$, que se resuelven recursivamente. El tiempo total es $nm + nm/2 + nm/4 + \dots \leq 2nm$: apenas se duplica, pero la memoria se reduce en órdenes de magnitud. Myers y Miller (1988) extendieron la técnica a huecos afines, y desde entonces es la base de todos los alineadores exactos que trabajan con secuencias largas.

Ejemplo 3.7. (Encontrar el punto medio)

Volvamos a $x = \text{GATTACA}$ e $y = \text{GTTAGCA}$ con los parámetros de la figura 3.5. La fila central es $i^* = 3$ (tras GAT). Hacia adelante, la fila 3 vale $F(3, \cdot) = (-6, -3, 0, 1, -1, -3, -5, -7)$; hacia atrás, alineando TACA con los sufijos de y , $B(3, \cdot) = (-2, 0, 2, 0, -1, -2, -5, -8)$. La suma es $(-8, -3, 2, 1, -2, -5, -10, -15)$, cuyo máximo, 2, coincide con $F(7, 7)$ y se alcanza en $j^* = 2$. El camino óptimo pasa entonces por la celda $(3, 2)$, es decir, GAT se alinea con GT y TACA con TAGCA, exactamente como en el ejemplo 3.4. Cada mitad se resuelve de nuevo del mismo modo, sin guardar nunca más de dos filas.

3.2.10 Alineamiento con bandas y aceleración

Cuando sabemos de antemano que el camino óptimo está cerca de una diagonal, porque las secuencias son muy parecidas o porque una semilla nos lo indica, podemos restringir el cálculo a una *banda* de diagonales $d = j - i \in [d_0 - k, d_0 + k]$ y tratar el resto de las celdas como $-\infty$. El costo baja de $O(nm)$ a $O(kn)$. Chao et al. (1992) describieron cómo alinear dentro de una banda diagonal especificada en tiempo proporcional a su área y con memoria lineal, y Ukkonen (1985) demostró que para la distancia de edición la banda puede crecer adaptativamente: si la distancia es D , basta con una banda de ancho $O(D)$, y el algoritmo cuesta $O(Dn)$. Esta es la idea que usan la extensión con huecos de BLAST y los alineadores de lecturas modernos, como [minimap2](#) (Li, 2018). La otra gran vía de aceleración es el hardware: la implementación “rayada” de Smith-Waterman de Farrar (2007) procesa 8 o 16 celdas a la

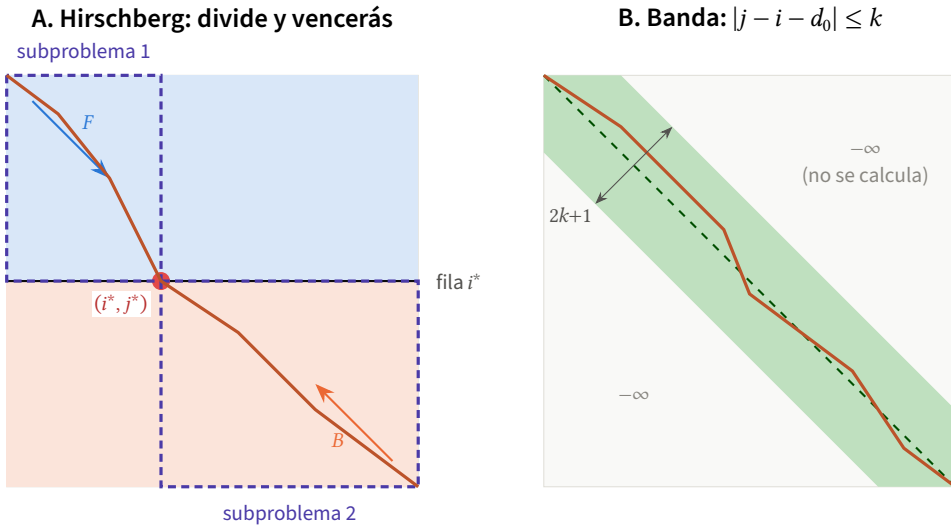


Figura 3.8. Dos estrategias para reducir el costo de la programación dinámica. **A:** Hirschberg calcula la mitad superior hacia adelante (F , azul) y la inferior hacia atrás (B , naranja) guardando solo una fila; la suma $F + B$ sobre la fila central localiza la celda (i^*, j^*) por la que pasa el camino óptimo, y el problema se divide en dos rectángulos (líneas violeta) cuya área total es la mitad de la original. **B:** con una banda de ancho $2k + 1$ alrededor de la diagonal esperada solo se calculan $O(kn)$ celdas (verde); el resultado es exacto siempre que el camino óptimo no salga de la banda, lo que ocurre si ningún *indel* acumulado supera k posiciones.

vez con instrucciones vectoriales SIMD, y, según el propio título del artículo, resulta unas seis veces más rápida que las implementaciones SIMD anteriores.

3.3 Matrices de sustitución y penalización de gaps

 Colab · 3.3 · Matrices de sustitución

 abrir notebook

Las pistas raras pesan más

Dos testigos describen por separado a la persona que vieron salir de un edificio. Si ambos dicen “tenía el pelo castaño”, la coincidencia apenas significa algo: medio mundo tiene el pelo castaño. Si ambos mencionan “un tatuaje

de un pez en la muñeca”, la coincidencia es casi una prueba. El valor de una coincidencia depende de lo *improbable* que sea por azar. Con los aminoácidos sucede lo mismo: que dos proteínas tengan un triptófano en la misma posición dice mucho más que dos leucinas, porque el triptófano es raro y, cuando está, rara vez se sustituye.

Hasta ahora hemos usado puntuaciones arbitrarias: +1 para coincidencias, −1 para sustituciones. En ADN esa simplificación es tolerable; en proteínas, no. Cambiar una leucina por una isoleucina (ambas hidrofóbicas, casi del mismo tamaño) apenas altera una proteína, mientras que cambiar un triptófano por una glicina puede destruir su plegamiento. Necesitamos puntuaciones que reflejen la química y la evolución.

3.3.1 Puntuaciones como razones de verosimilitud

La pregunta correcta es estadística. Frente a un par alineado (a, b) , comparamos dos hipótesis:

- $\mathcal{H}_1$ (*homología*): a y b descienden de un ancestro común y aparecen juntos con probabilidad q_{ab} , estimada a partir de alineamientos confiables.
- $\mathcal{H}_0$ (*azar*): a y b se eligieron de forma independiente, con probabilidad $p_a p_b$ según sus frecuencias de fondo.

La puntuación natural es el logaritmo de la razón de verosimilitudes:

$$s(a, b) = \frac{1}{\lambda} \log \frac{q_{ab}}{p_a p_b}. \quad (3.19)$$

DONDE

q_{ab}	Frecuencia con que a y b aparecen alineados en secuencias homólogas (frecuencias <i>objetivo</i>).
p_a, p_b	Frecuencias de fondo de cada aminoácido en las proteínas.
λ	Factor de escala; con $\lambda = \ln 2/2$ las puntuaciones quedan en medios bits.

Esta forma tiene una consecuencia elegante: como el logaritmo convierte productos en sumas, sumar las puntuaciones de las columnas equivale a multiplicar

probabilidades, siempre que las columnas se consideren independientes. Para un alineamiento sin huecos de longitud L ,

$$S = \sum_{k=1}^L s(u_k, v_k) = \frac{1}{\lambda} \log \frac{\prod_k q_{u_k v_k}}{\prod_k p_{u_k} p_{v_k}} = \frac{1}{\lambda} \log \frac{\mathbb{P}(\text{alineamiento} \mid \mathcal{H}_1)}{\mathbb{P}(\text{alineamiento} \mid \mathcal{H}_0)}. \quad (3.20)$$

La puntuación de un alineamiento completo es, por lo tanto, el logaritmo de la razón de verosimilitudes de la hipótesis de homología frente a la de azar, y por el lema de Neyman-Pearson es el estadístico más potente para distinguirlas. Altschul (1991) demostró además que *cualquier* matriz de puntuación con esperanza negativa y al menos un valor positivo es implícitamente una matriz de este tipo, para algún conjunto de frecuencias q_{ab} “objetivo”: las puntuaciones determinan las q_{ab} a través de $q_{ab} = p_a p_b e^{\lambda s(a,b)}$, con el único $\lambda > 0$ que hace que sumen 1. Elegir una matriz es, lo sepamos o no, elegir un modelo de qué parejas de aminoácidos esperamos ver entre homólogos.

3.3.2 PAM: extrapolar la evolución

Margaret Dayhoff y sus colaboradores construyeron las primeras matrices a partir de 71 familias de proteínas muy similares, con al menos 85 % de identidad (Dayhoff et al., 1978). Trabajar con secuencias tan cercanas tenía una ventaja decisiva: cada diferencia observada podía atribuirse, con poca ambigüedad, a una única mutación aceptada por la selección natural, sin sustituciones múltiples en el mismo sitio. La construcción procede en tres pasos.

1. Contar.. Sobre los árboles filogenéticos de cada familia se cuentan las sustituciones A_{ab} entre cada par de aminoácidos (simétricas: $A_{ab} = A_{ba}$).

2. Mutabilidad relativa.. No todos los aminoácidos cambian con la misma facilidad. La *mutabilidad relativa* m_a es el número de cambios en que participa a dividido por su abundancia y por la exposición total a la mutación. Dayhoff encontró que la asparagina y la serina son muy mutables, y el triptófano y la cisteína muy poco: sustituirlos suele ser letal para la proteína.

3. Normalizar a 1 PAM.. La matriz de transición de un paso es

$$P_{ab}^{(1)} = \frac{c m_a A_{ab}}{\sum_{b' \neq a} A_{ab'}} \quad (a \neq b), \quad P_{aa}^{(1)} = 1 - c m_a, \quad \sum_a p_a (1 - P_{aa}^{(1)}) = 0,01. \quad (3.21)$$

DONDE

A_{ab}	Número de sustituciones aceptadas $a \leftrightarrow b$ contadas en los árboles.
m_a	Mutabilidad relativa del aminoácido a .
c	Constante de escala elegida para que la tercera condición se cumpla.
$P_{ab}^{(1)}$	Probabilidad de que a sea reemplazado por b en un intervalo de 1 PAM.

La tercera condición define la unidad: 1 PAM (*point accepted mutation*) es la divergencia en la que, en promedio, ha cambiado un 1 % de los aminoácidos. Suponiendo que la evolución es un proceso de Márkov homogéneo, en el que el futuro de cada posición depende solo de su aminoácido actual, la matriz para distancias mayores se obtiene elevando a potencias:

$$P^{(k)} = \left(P^{(1)}\right)^k, \quad s_k(a, b) = \frac{1}{\lambda} \log \frac{p_a P_{ab}^{(k)}}{p_a p_b} = \frac{1}{\lambda} \log \frac{P_{ab}^{(k)}}{p_b}. \quad (3.22)$$

Dos propiedades del modelo hacen que la matriz resultante sea simétrica. Las frecuencias p son *estacionarias*, $pP^{(1)} = p$, de modo que la composición no deriva con el tiempo; y el proceso es *reversible*, $p_a P_{ab} = p_b P_{ba}$, de modo que $q_{ab} = p_a P_{ab}^{(k)}$ es simétrica aunque $P^{(k)}$ no lo sea. La PAM250, muy usada en su época, corresponde a una distancia en la que sobreviven solo alrededor del 20 % de las identidades. El punto débil del método es precisamente la extrapolación: los errores de estimación de $P^{(1)}$ se amplifican al elevarla a la potencia 250. Las matrices posteriores de Gonnet et al. (1992), que compararon todas contra todas las secuencias de la base de datos de su época, y de Jones et al. (1992), que actualizaron los conteos de Dayhoff con muchas más familias, mantuvieron el marco de Márkov pero con estimaciones mucho más ricas.

3.3.3 Entropía relativa: cuánta información aporta cada columna

¿Cómo comparar matrices construidas para distancias distintas? La cantidad adecuada es la puntuación *esperada* por columna cuando las secuencias sí son homólogas, que resulta ser una entropía relativa:

$$H = \sum_{a,b} q_{ab} \log_2 \frac{q_{ab}}{p_a p_b} = \lambda \log_2(e) \sum_{a,b} q_{ab} s(a, b) \quad (\text{bits por columna}). \quad (3.23)$$

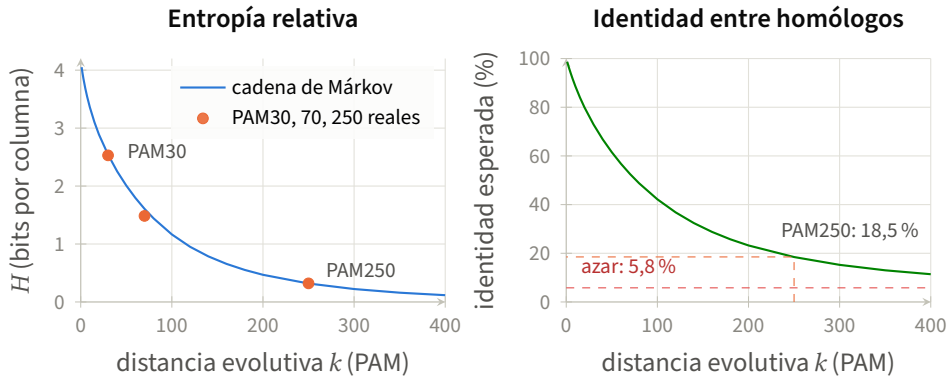


Figura 3.9. La familia PAM como cadena de Márkov. Tomamos la PAM250 de Biopython, recuperamos sus frecuencias objetivo con las frecuencias de fondo que usa BLAST y calculamos su raíz 250-ésima como “PAM1”; las potencias $P^{(k)}$ dan las curvas. **Izquierda:** la entropía relativa (ecuación (3.23)) cae de unos 4 bits por columna a menos de medio bit; los puntos naranjas son las matrices publicadas, que se desvían de la curva porque se estimaron y redondearon por separado. **Derecha:** la identidad esperada entre dos homólogos decrece hacia el nivel de azar (5,8 %, línea roja) mucho más despacio que la entropía; a 250 PAM todavía comparten cerca del 19 % de sus residuos, aunque cada columna aporte ya muy poca información. Datos de `figuras/cap03/generar.py`.

DONDE

H	Entropía relativa (divergencia de Kullback-Leibler) de las frecuencias objetivo respecto al modelo nulo.
$\log_2(e)$	Factor que convierte nats en bits.

H es siempre positiva y mide cuánta evidencia a favor de la homología aporta, en promedio, cada columna alineada. Su contraparte es la esperanza bajo el azar, $\sum_{a,b} p_a p_b s(a, b) < 0$. Una matriz con H grande distingue muy bien secuencias cercanas: pocas columnas bastan para acumular una puntuación significativa. A medida que k crece, $P^{(k)}$ se acerca a la distribución estacionaria, $q_{ab} \rightarrow p_a p_b$, y $H \rightarrow 0$: las secuencias muy divergentes contienen muy poca información por posición, y hacen falta alineamientos largos para reconocerlas (figura 3.9). Altschul (1991) cuantificó esta relación: para alcanzar una puntuación significativa en un espacio de búsqueda de tamaño mn se necesitan, aproximadamente, $L_{\min} \approx \log_2(mn)/H$ columnas.

Ejemplo 3.8. (Reconstruir la familia PAM)

Si conocemos una matriz PAM, podemos invertir el proceso. Resolviendo $\sum p_a p_b e^{\lambda s(a,b)} = 1$ para la PAM250 obtenemos sus frecuencias objetivo q_{ab} , y de ellas la matriz de transición $P_{ab}^{(250)} = q_{ab}/p_a$. Como el proceso es reversible, $D^{1/2} P^{(250)} D^{-1/2}$ (con $D = \text{diag}(p)$) es simétrica, sus autovalores son reales y, en este caso, todos positivos, así que su raíz 250-ésima está bien definida. El resultado, nuestra “PAM1”, cambia el 1,1 % de los residuos por paso (y no exactamente el 1 %, porque los enteros publicados están redondeados) y conserva el triptófano con probabilidad 0,998 y la asparagina con 0,963, como la mutabilidad de Dayhoff predice. Sus potencias dan: PAM30, $H = 2,54$ bits y 73 % de identidad; PAM120, 0,95 bits y 37 %; PAM250, 0,32 bits y 18,5 %. Un detalle revelador: la raíz exacta tiene algunos elementos ligeramente negativos (el mínimo es $-0,005$), que hay que recortar a cero. Es la manifestación numérica de la advertencia anterior: extrapolar con potencias amplifica cualquier imprecisión de la matriz de partida.

```
pam_markov.py

import numpy as np
from scipy.linalg import expm, logm

def familia_pam(q, ks):
    """q: frecuencias objetivo (20x20, simétrica) de una PAM250."""
    p = q.sum(1)
    P250 = q / p[:, None]          # P(b | a) a 250 PAM
    Q = logm(P250).real / 250      # generador: "1 PAM"
    for k in ks:
        Pk = np.clip(expm(Q * k), 0, None)
        Pk /= Pk.sum(1, keepdims=True)
        S = np.log2(Pk / p[None, :]) # puntuación en bits
        H = (p[:, None] * Pk * S).sum()
        yield k, H, (p * np.diag(Pk)).sum()
```

3.3.4 BLOSUM: observar directamente

Henikoff y Henikoff (1992) tomaron el camino opuesto. En lugar de extrapolar desde secuencias cercanas, usaron la base de datos BLOCKS: unos 2000 segmentos conservados, sin huecos, de más de 500 familias de proteínas. Para construir la BLOSUML agruparon las secuencias de cada bloque con más de L % de identidad, con

el fin de que las familias muy representadas no dominaran los conteos, y estimaron q_{ab} directamente de los pares observados. Así, la BLOSUM62 se basa en secuencias con a lo sumo 62 % de identidad, y la BLOSUM45 en otras aún más divergentes. El procedimiento completo es sorprendentemente corto:

$$q_{ab} = \frac{f_{ab}}{\sum_{c \leq d} f_{cd}}, \quad p_a = q_{aa} + \frac{1}{2} \sum_{b \neq a} q_{ab}, \quad (3.24)$$

$$e_{ab} = \begin{cases} p_a^2 & a = b \\ 2 p_a p_b & a \neq b \end{cases}, \quad s(a, b) = \text{round} \left(2 \log_2 \frac{q_{ab}}{e_{ab}} \right). \quad (3.25)$$

DONDE

f_{ab}	Número (ponderado) de pares $\{a, b\}$ observados en una misma columna de un bloque, sin importar el orden.
p_a	Frecuencia de a en los pares, que hace de frecuencia de fondo.
e_{ab}	Frecuencia esperada del par no ordenado $\{a, b\}$ bajo el azar; el 2 cuenta los dos órdenes.
$2 \log_2$	Escala de medios bits, redondeada al entero más cercano.

Ejemplo 3.9. (BLOSUM en miniatura)

Consideremos un bloque de seis secuencias y cuatro columnas: ASLT, ASLT, ATLS, SSIT, ATLT, SAIT. Cada columna aporta $\binom{6}{2} = 15$ pares, 60 en total. Por ejemplo, la primera columna (A, A, A, S, A, S) aporta 6 pares AA, 8 pares AS y 1 par SS. Sumando las cuatro columnas y aplicando las ecuaciones (3.24) y (3.25) se obtiene la tabla 3.2, con $p_A = 0,208$, $p_I = 0,083$, $p_L = 0,167$, $p_S = 0,250$ y $p_T = 0,292$.

Tabla 3.2. Construcción de una BLOSUM en miniatura a partir de un bloque de seis secuencias sin agrupar.

Par	AA	AS	AT	II	IL	LL	SS	ST	TT
f_{ab}	6	11	2	1	8	6	4	11	11
q_{ab}	0,100	0,183	0,033	0,017	0,133	0,100	0,067	0,183	0,183
e_{ab}	0,043	0,104	0,122	0,007	0,028	0,028	0,063	0,146	0,085
$2 \log_2(q/e)$	2,41	1,63	-3,73	2,53	4,53	3,70	0,19	0,66	2,22
$s(a, b)$	2	2	-4	3	5	4	0	1	2

El par IL recibe la puntuación más alta (5): aparece mucho más de lo que sus frecuencias predicen. El par SS, en cambio, puntúa o porque la serina es tan abundante en el bloque que verla emparejada consigo misma no sorprende. Ahora apliquemos el agrupamiento de la BLOSUM62: ASLT, ASLT, ATLT y ATLS quedan en un grupo (cada una tiene al menos 62 % de identidad con alguna otra del grupo) y SSIT y SAIT en otro. Los pares *dentro* de un grupo ya no se cuentan, y cada par entre grupos pesa $1/(4 \times 2)$. Solo sobrevive un par efectivo por columna, cuatro en total, repartidos como AS 1,25, IL 1,00, TT 0,75, ST 0,50, SS 0,25 y AT 0,25. Los pares idénticos AA y LL, que provenían de secuencias casi iguales, desaparecen: el agrupamiento elimina la redundancia y deja la información de las sustituciones entre secuencias divergentes, que es justo la que interesa para detectar homólogos lejanos.

Número alto en PAM $\Rightarrow$ secuencias más divergentes.
 Número alto en BLOSUM $\Rightarrow$ secuencias más cercanas.

Un error afortunado

Una década y media después de su publicación, Styczynski et al. (2008) descubrieron que el programa original que generó las BLOSUM contenía un error en el agrupamiento de secuencias. Paradójicamente, la matriz “incorrecta” encuentra homólogos remotos *mejor* que la versión corregida, y por eso la BLOSUM62 original sigue siendo la matriz por defecto de BLAST. Eddy (2004b) explica, en dos páginas, cómo reconstruirla desde los datos.

¿Medios bits o no? Depende del fondo

La BLOSUM62 se publica “en medios bits”, es decir, con $\lambda = \ln 2/2 \approx 0,347$, y con sus propias frecuencias de fondo su entropía relativa es de unos 0,70 bits. Pero BLAST calcula la estadística con otras frecuencias de fondo, las de Robinson y Robinson; con ellas, resolver $\sum p_a p_b e^{\lambda s(a,b)} = 1$ da $\lambda = 0,3176$ y $H = 0,58$ bits, que son los valores que usaremos en la sección 3.4. No hay contradicción: λ y H no son propiedades de la matriz sola, sino de la matriz

	C	S	T	A	G	P	D	E	Q	N	H	R	K	M	I	L	V	W	Y	F	
C	9	-1	-1	0	-3	-3	-3	-4	-3	-3	-3	-3	-3	-1	-1	-1	-1	-2	-2	-2	
S	-1	4	1	1	0	-1	0	0	0	1	-1	-1	0	-1	-2	-2	-2	-3	-2	-2	
T	-1	1	5	0	-2	-1	-1	-1	-1	0	-2	-1	-1	-1	-1	-1	0	-2	-2	-2	
A	0	1	0	4	0	-1	-2	-1	-1	-2	-2	-1	-1	-1	-1	-1	0	-3	-2	-2	pequeños
G	-3	0	-2	0	6	-2	-1	-2	-2	0	-2	-2	-2	-3	-4	-4	-3	-2	-3	-3	
P	-3	-1	-1	-1	-2	7	-1	-1	-1	-2	-2	-2	-1	-2	-3	-3	-2	-4	-3	-4	
D	-3	0	-1	-2	-1	-1	6	2	0	1	-1	-2	-1	-3	-3	-4	-3	-4	-3	-3	
E	-4	0	-1	-1	-2	-1	2	5	2	0	0	0	1	-2	-3	-3	-2	-3	-2	-3	ácidos/amidas
Q	-3	0	-1	-1	-2	-1	0	2	5	0	0	1	1	0	-3	-2	-2	-2	-1	-3	
N	-3	1	0	-2	0	-2	1	0	0	6	1	0	0	-2	-3	-3	-3	-4	-2	-3	
H	-3	-1	-2	-2	-2	-2	-1	0	0	1	8	0	-1	-2	-3	-3	-3	-2	2	-1	
R	-3	-1	-1	-1	-2	-2	-2	0	1	0	0	5	2	-1	-3	-2	-3	-3	-2	-3	básicos
K	-3	0	-1	-1	-2	-1	-1	1	1	0	-1	2	5	-1	-3	-2	-2	-3	-2	-3	
M	-1	-1	-1	-1	-3	-2	-3	-2	0	-2	-2	-1	-1	5	1	2	1	-1	-1	0	
I	-1	-2	-1	-1	-4	-3	-3	-3	-3	-3	-3	-3	-3	1	4	2	3	-3	-1	0	
L	-1	-2	-1	-1	-4	-3	-4	-3	-2	-3	-3	-2	-2	2	2	4	1	-2	-1	0	alifáticos
V	-1	-2	0	0	-3	-2	-3	-2	-2	-3	-3	-3	-2	1	3	1	4	-3	-1	-1	
W	-2	-3	-2	-3	-2	-4	-4	-3	-2	-4	-2	-3	-3	-1	-3	-2	-3	11	2	1	
Y	-2	-2	-2	-2	-3	-3	-3	-2	-1	-2	2	-2	-2	-1	-1	-1	-1	2	7	3	aromáticos
F	-2	-2	-2	-2	-3	-4	-3	-3	-3	-3	-1	-3	-3	0	0	0	-1	1	3	6	

Figura 3.10. La matriz BLOSUM62, en medios bits, con los aminoácidos ordenados por propiedades fisicoquímicas. Los bloques recuadrados en la diagonal muestran que las sustituciones *dentro* de un grupo (p. ej., $I \leftrightarrow L \leftrightarrow V$) reciben puntuaciones positivas o cercanas a cero, mientras que las sustituciones entre grupos son penalizadas. El valor de W-W (+11) refleja que el triptófano es raro y muy conservado. Valores tomados de Biopython (Cock et al., 2009).

junto con el modelo nulo. Por la misma razón, el redondeo a enteros hace que ninguna matriz publicada esté exactamente en la escala nominal.

3.3.5 Elegir la matriz según la divergencia

No existe una matriz universalmente óptima. Cada una está calibrada para un nivel de divergencia: una matriz “profunda” (BLOSUM45, PAM250) detecta homólogos lejanos pero puntúa en exceso los parecidos casuales en alineamientos cortos, mientras que una matriz “superficial” (BLOSUM80, PAM30) hace lo contrario. El argumento de Altschul (1991) hace esta intuición precisa. Si las secuencias están

realmente a distancia k , la puntuación esperada por columna usando la matriz calibrada para la distancia j es

$$\bar{s}(j | k) = \sum_{a,b} q_{ab}^{(k)} s_j(a, b) = H(k) - D(q^{(k)} \| q^{(j)}) \leq H(k). \quad (3.26)$$

DONDE

$q_{ab}^{(k)}$	Frecuencias objetivo verdaderas a distancia k , $p_a p_b^{(k)}$.
s_j	Matriz calibrada para la distancia j , en bits.
$D(\cdot \ \cdot)$	Divergencia de Kullback-Leibler entre las frecuencias verdaderas y las supuestas.

La igualdad se da solo si $j = k$. Usar una matriz mal calibrada siempre cuesta información, y el costo es exactamente la divergencia entre lo que la matriz espera y lo que realmente ocurre. La figura 3.11 muestra el resultado para tres matrices de la familia: cada una domina en su propio rango, y la PAM30 aplicada a secuencias lejanas da puntuaciones esperadas *negativas*, es decir, no puede detectarlas por mucho que se alarguen. Altschul (1993) propuso, en consecuencia, usar un pequeño conjunto de matrices (por ejemplo, PAM40, PAM120 y PAM250) y quedarse con la mejor, corrigiendo la estadística por las comparaciones múltiples.

La relación $L_{\min} \approx \log_2(mn)/H$ traduce todo esto a longitudes. Una búsqueda de una consulta de 300 residuos contra 10^8 residuos necesita unos $\log_2(3 \times 10^{10}) \approx 35$ bits para destacar. Con BLOSUM62 ($H \approx 0,58$ bits con el fondo de BLAST) hacen falta unas 60 columnas de homología típica; con PAM30 ($H \approx 2,5$ bits) bastan unas 14. Por eso, para péptidos cortos se recomienda PAM30 o BLOSUM80, mientras que BLOSUM62 es el punto de partida de `blastp` para proteínas completas (Pearson, 2013). Henikoff y Henikoff (1993) compararon sistemáticamente las matrices en búsquedas reales y mostraron que las BLOSUM, y en particular la BLOSUM62, rendían mejor que las PAM de contenido de información comparable, lo que explica su adopción como estándar. Por debajo de cierta identidad, sin embargo, ninguna matriz basta: Rost (1999) caracterizó la “zona crepuscular”, en torno al 20–35 % de identidad, en la que el porcentaje de identidad por sí solo deja de separar homólogos de no homólogos, y en la que los métodos de perfiles del capítulo 4 se vuelven imprescindibles.

3.3.6 Penalizaciones de hueco

Para los huecos no existe una teoría tan limpia como la de las matrices. Una penalización afin equivale, como vimos con el HMM de pares, a suponer que las

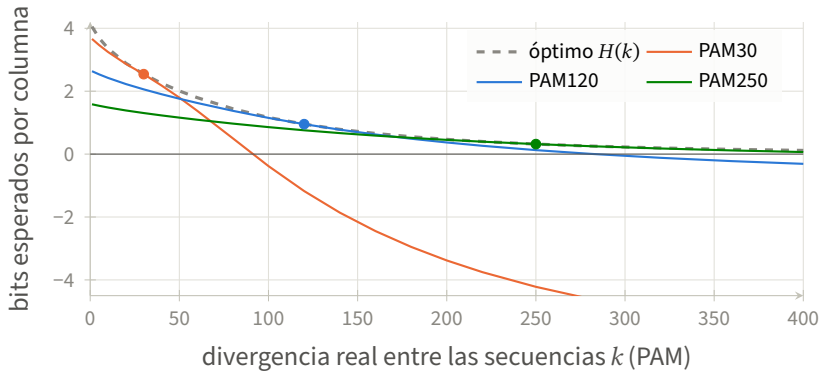


Figura 3.11. Puntuación esperada por columna (ecuación (3.26)) de tres matrices de la familia PAM reconstruida, en función de la divergencia real entre las secuencias. Cada curva toca la envolvente óptima $H(k)$ (línea discontinua gris) exactamente en su distancia de diseño (puntos) y queda por debajo en las demás. A 120 PAM, la PAM120 aporta 0,95 bits por columna, la PAM250 0,76 y la PAM30 -1,18: una matriz demasiado “superficial” no solo pierde sensibilidad, sino que convierte a los homólogos lejanos en puntuaciones negativas.

longitudes de los *indels* siguen una distribución geométrica; los datos empíricos muestran colas más pesadas, pero la aproximación afín es suficientemente buena y computacionalmente barata. En la práctica, los parámetros se eligen empíricamente para cada matriz y deben quedar dentro del régimen logarítmico que discutimos en la sección 3.2 (Arratia & Waterman, 1994; Vingron & Waterman, 1994). Con BLOSUM62, BLAST usa por defecto existencia 11 y extensión 1 (en nuestra notación, $d_o = 12$, $d_e = 1$).

⚠ Parámetros que no se pueden mezclar

Las penalizaciones de hueco no son independientes de la matriz: ambas deben estar en la misma escala y, para búsquedas en bases de datos, calibradas conjuntamente, porque de ellas dependen los parámetros λ y K de la sección siguiente. Cambiar d_o sin recalibrar invalida los valores E . Por eso BLAST solo acepta ciertas combinaciones de matriz y huecos, para las que tiene λ y K precalculados.

3.4 BLAST y la estadística de Karlin-Altschul

Buscar un rostro en un estadio

Para encontrar a un amigo entre cincuenta mil personas no examinamos cada cara con detenimiento. Buscamos primero algo breve y llamativo, una camiseta roja o un sombrero, y solo nos acercamos a mirar con atención cuando lo vemos. Perderemos a nuestro amigo si ese día no llevaba la camiseta, pero recorreremos el estadio en minutos en lugar de días. BLAST hace exactamente este trato: renuncia a la garantía de encontrar el alineamiento óptimo a cambio de una velocidad de varios órdenes de magnitud, y luego usa una estadística rigurosa para decirnos cuánto debemos creer en lo que encontró.

3.4.1 El problema de escala

Smith-Waterman es exacto, pero comparar una proteína de 300 residuos contra UniProt (unos 10^{11} residuos) requeriría 3×10^{13} celdas por consulta. A finales de los ochenta, FASTA (Pearson & Lipman, 1988) mostró que se podía ganar velocidad buscando primero palabras cortas idénticas. BLAST (Altschul et al., 1990) llevó esa idea más lejos con una estrategia de tres pasos, conocida como *seed-and-extend*:

1. **Siembra.** Se divide la consulta en palabras de longitud w (3 para proteínas, 11 o más para ADN) y, para cada una, se genera la *vecindad* de palabras que puntúan al menos T contra ella con la matriz de sustitución.
2. **Búsqueda.** Un índice precalculado de la base de datos localiza en tiempo casi constante todas las apariciones de esas palabras (*hits*).
3. **Extensión.** Cada *hit* se extiende en ambas direcciones y se detiene cuando la puntuación cae más de X unidades por debajo del máximo alcanzado (*X-drop*). Los segmentos resultantes son los *high-scoring segment pairs* (HSP).

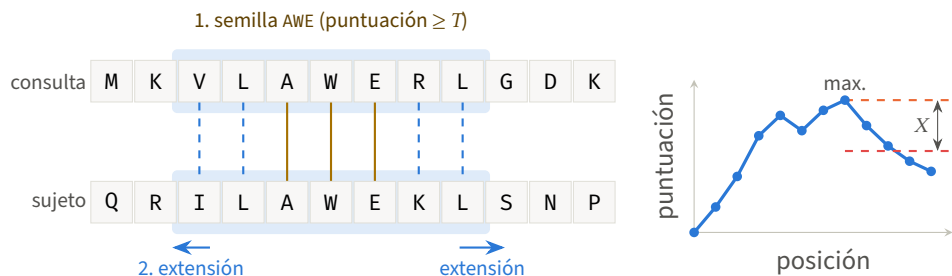


Figura 3.12. La estrategia *seed-and-extend*. **Izquierda:** una palabra de la consulta (AWE) encuentra una semilla en el sujeto; la extensión sin huecos añade pares vecinos mientras la puntuación acumulada lo justifique. **Derecha:** perfil de puntuación durante la extensión; se abandona cuando la puntuación cae X unidades por debajo del máximo, y el HSP se recorta en ese máximo.

3.4.2 La vecindad de palabras y el umbral T

El paso de siembra merece una mirada cuantitativa, porque en él se decide casi todo el equilibrio entre velocidad y sensibilidad. Para una palabra $u = u_1u_2u_3$ de la consulta, su vecindad es

$$\mathcal{N}_T(u) = \left\{ v \in \mathcal{A}^3 : \sum_{k=1}^3 s(u_k, v_k) \geq T \right\}. \quad (3.27)$$

DONDE	
$\mathcal{N}_T(u)$	Conjunto de palabras de la base de datos que cuentan como semilla para u .
$\mathcal{A}^3$	Las $20^3 = 8000$ palabras posibles de tres aminoácidos.
T	Umbral de vecindad; más bajo significa más semillas y más sensibilidad.

Con BLOSUM62, la palabra PQG (el ejemplo del artículo original) puntúa 18 contra sí misma, y con $T = 13$ su vecindad tiene nueve palabras: PQG (18), PEG (15), PRG y PKG (14), y PHG, PNG, PDG, PMG, PSG (13). Todas conservan la prolina y la glicina, raras y muy puntuadas, y varían solo la posición central, donde la glutamina admite sustituciones conservadoras. La figura 3.13 muestra cómo crece la vecindad al bajar T . El costo es directo: con $T = 11$, una palabra típica tiene unos 19 vecinos, y la probabilidad de que una posición cualquiera de la base de datos sea una semilla para ella es $2,1 \times 10^{-3}$; contra 10^8 residuos, eso son unos 200 000 *hits* por posición

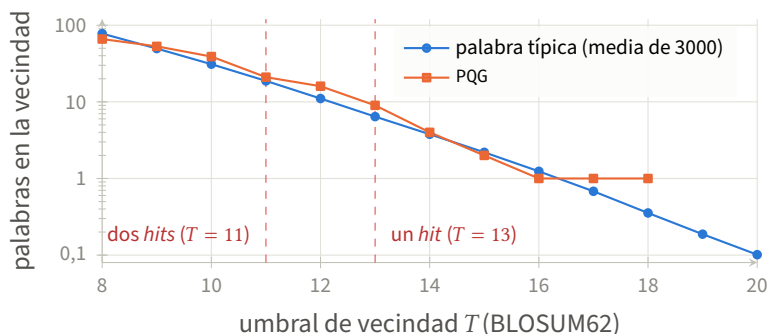


Figura 3.13. Número de palabras de tres aminoácidos que puntúan al menos T contra una palabra de la consulta, con BLOSUM62. La curva azul promedia 3000 palabras aleatorias con las frecuencias de fondo; la naranja corresponde a PQG. El crecimiento es aproximadamente exponencial al bajar T : cada unidad de umbral multiplica por 1,6–1,8 el número de semillas y, con él, el trabajo de extensión. La heurística de dos *hits* de 1997 permitió bajar T de 13 a 11 sin que el tiempo se disparara.

de la consulta, cada uno de los cuales habría que extender. Con $T = 13$ son unos 60 000. Un punto de T vale casi un factor dos en tiempo.

Para ADN la estrategia es distinta: con solo cuatro letras, las palabras son largas y exactas ($w = 11$ en *blastn*, 28 en *megablast*), y la ganancia de sensibilidad viene de otro lado. Ma et al. (2002) mostraron con PatternHunter que las *semillas espaciadas*, que exigen coincidencia solo en ciertas posiciones de la palabra (p. ej., 111010010100110111), detectan más homologías que las semillas contiguas del mismo peso, porque sus aciertos en posiciones vecinas están menos correlacionados. Para secuencias casi idénticas, Zhang et al. (2000) propusieron un algoritmo voraz de extensión, base de *megablast*, muchísimo más rápido que la programación dinámica.

3.4.3 Dos *hits*, *X-drop* y extensión con huecos

La versión de 1997 (Altschul et al., 1997) añadió dos mejoras decisivas. La primera es la heurística de “dos *hits*”. Los autores observaron que la mayor parte del tiempo se iba en extender semillas que no conducían a nada, y que un HSP de interés suele contener varias semillas. En lugar de extender cada semilla, BLAST exige dos semillas no solapadas en la *misma diagonal* y a menos de A posiciones una de otra (por defecto $A = 40$); solo entonces extiende sin huecos (figura 3.14). Como

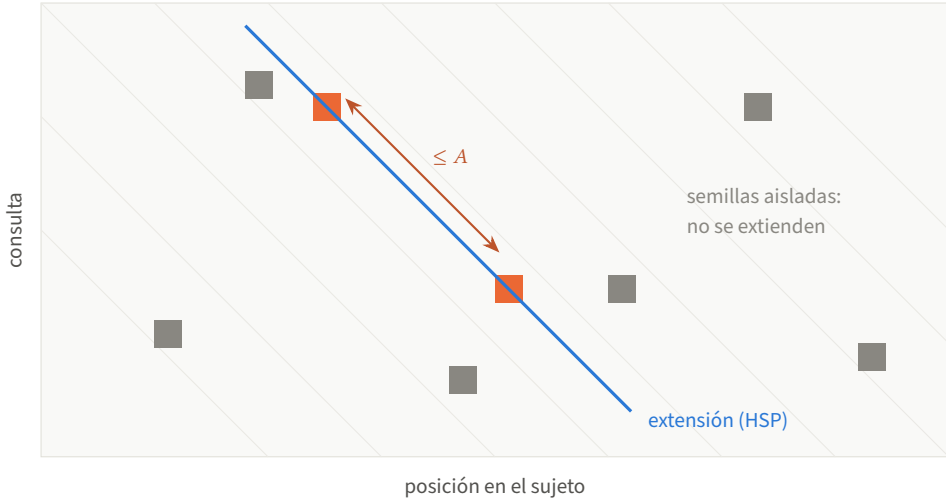


Figura 3.14. Heurística de dos *hits*. Cada cuadrado es una semilla (una palabra de la vecindad hallada en el sujeto). Las semillas aisladas (gris) se descartan sin extender; solo cuando dos semillas caen en la misma diagonal, a una distancia menor que A , BLAST las extiende (azul). Como los HSP verdaderos contienen varias semillas y el ruido rara vez produce dos en la misma diagonal, el filtro ahorra la mayor parte del trabajo con una pérdida mínima de sensibilidad.

cada semilla aislada rara vez tiene compañera, esto permite bajar T para ganar sensibilidad y, aun así, reducir el trabajo total.

La extensión sin huecos se controla con la regla *X-drop*. Si S_k es la puntuación acumulada tras extender k posiciones, la extensión se detiene en el primer k tal que

$$\max_{k' \leq k} S_{k'} - S_k > X, \quad (3.28)$$

y el HSP se recorta en el máximo. La lógica es probabilística: como la deriva de la puntuación es negativa entre secuencias no relacionadas, una caída de X unidades hace muy improbable que la extensión vuelva a superar el máximo. Si el HSP sin huecos supera un segundo umbral, BLAST lanza una extensión *con* huecos mediante programación dinámica restringida a las celdas cuya puntuación no cae más de X_g por debajo del máximo, una banda adaptativa emparentada con la de la sección 3.2. La implementación moderna, BLAST+ (Camacho et al., 2009), reorganizó el código en C++ y es la que usaremos en el laboratorio.

3.4.4 ¿Cuándo un parecido es significativo?

Toda búsqueda produce un mejor resultado, incluso si la consulta es una secuencia aleatoria. La pregunta crucial es: *¿qué puntuación esperaríamos por puro azar?* La respuesta, obtenida por Karlin y Altschul (1990), es uno de los resultados más bellos de la bioinformática.

Consideremos la puntuación máxima S de un alineamiento local *sin huecos* entre dos secuencias aleatorias de longitudes m y n . La suma de puntuaciones a lo largo de una diagonal es un paseo aleatorio con deriva negativa (porque $E[s] < 0$). Los segmentos de alta puntuación son *excursiones* excepcionales de ese paseo, y la teoría de valores extremos predice que su máximo sigue una distribución de Gumbel.

Teorema 3.5. (Karlin-Altschul)

Si $\sum_{a,b} p_a p_b s(a,b) < 0$ y existe al menos un par con $s(a,b) > 0$, el número esperado de HSP distintos con puntuación $\geq S$ es, asintóticamente,

$$E = K m n e^{-\lambda S}, \quad (3.29)$$

donde $\lambda > 0$ es la única raíz positiva de

$$\sum_{a,b} p_a p_b e^{\lambda s(a,b)} = 1. \quad (3.30)$$

El número de HSP con puntuación $\geq S$ sigue una distribución de Poisson de media E , de modo que $\mathbb{P}(\text{al menos uno}) = 1 - e^{-E}$.

DONDE

E	Valor E (<i>expect value</i>): cuántos resultados tan buenos esperaríamos por azar en esta búsqueda.
m, n	Longitud de la consulta y tamaño total de la base de datos, en residuos.
S	Puntuación bruta del alineamiento, en las unidades de la matriz.
λ	Escala natural de la matriz; convierte puntuaciones en unidades de información.
K	Constante que corrige por la dependencia entre HSP solapados (típicamente $\approx 0,1$).

La ecuación (3.30) merece una pausa. Si comparamos con la ecuación (3.19), vemos que $e^{\lambda s(a,b)} = q_{ab}/(p_a p_b)$, de modo que la condición dice simplemente que las frecuencias objetivo q_{ab} suman 1. El mismo λ que define la matriz como razón de verosimilitudes gobierna la estadística de sus máximos. Teoría de la información y teoría de valores extremos se encuentran en una sola ecuación. Que la raíz positiva exista y sea única se ve estudiando $f(\lambda) = \sum p_a p_b e^{\lambda s(a,b)}$: es convexa, vale 1 en $\lambda = 0$, empieza decreciendo porque $f'(0) = E[s] < 0$ y crece sin límite porque hay puntuaciones positivas; por lo tanto cruza el 1 exactamente una vez más.

Ejemplo 3.10. (λ para ADN, con lápiz y papel)

Con bases equiprobables y puntuaciones +1 (coincidencia, probabilidad 1/4) y -1 (sustitución, probabilidad 3/4), la ecuación (3.30) es $\frac{1}{4}e^\lambda + \frac{3}{4}e^{-\lambda} = 1$. Con $z = e^\lambda$ queda $z^2 - 4z + 3 = 0$, cuyas raíces son $z = 1$ (la trivial, $\lambda = 0$) y $z = 3$, así que $\lambda = \ln 3 \approx 1,099$. Las frecuencias objetivo implícitas son $q_{\text{ident}} = 4 \times \frac{1}{16} \times e^\lambda = 0,75$: el esquema +1/-1 “espera” homólogos con 75 % de identidad. Con +1/-3 la ecuación se vuelve $z^4 - 4z^3 + 3 = 0$, que, eliminada la raíz $z = 1$, deja $z^3 - 3z^2 - 3z - 3 = 0$, con $z \approx 3,847$ y $\lambda \approx 1,374$; ahora la identidad objetivo es 98,8 %. Es la razón por la que **megablast**, pensado para secuencias casi idénticas, usa por defecto recompensa y penalización 1/-2 (identidad objetivo del 95 %), mientras que para homologías entre especies conviene un esquema más tolerante, como +2/-3 (89 %) o +5/-4 (65 %). States et al. (1991) desarrollaron esta idea de matrices de nucleótidos específicas para cada aplicación.

3.4.5 La distribución de Gumbel, comprobada por simulación

El teorema de Karlin-Altschul se puede reescribir como una afirmación sobre la *distribución* de la puntuación máxima $S_{\text{máx}}$ de una comparación. Como el número de HSP por encima de x es de Poisson con media $Kmn e^{-\lambda x}$, la probabilidad de que no haya ninguno es

$$P(S_{\text{máx}} < x) \approx \exp(-Kmn e^{-\lambda x}) = \exp(-e^{-\lambda(x-\mu)}), \quad \mu = \frac{\ln(Kmn)}{\lambda}. \quad (3.31)$$

DONDE

μ	Moda (parámetro de localización) de la distribución de Gumbel.
$1/\lambda$	Parámetro de escala; la desviación estándar es $\pi/(\lambda\sqrt{6})$.

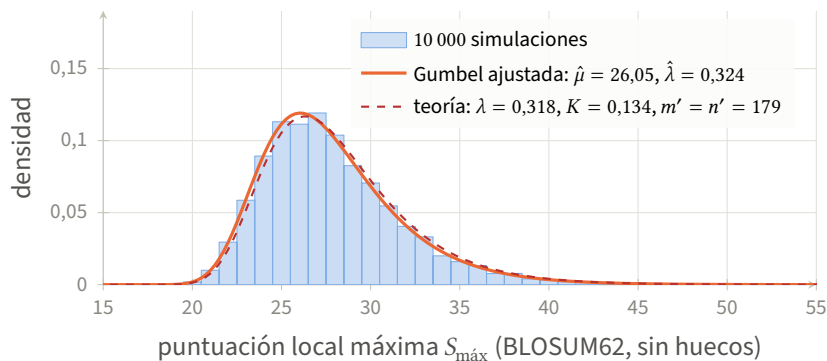


Figura 3.15. Histograma de la puntuación local máxima sin huecos entre 10 000 pares de secuencias aleatorias de 200 aminoácidos, con BLOSUM62 y las frecuencias de fondo de BLAST. La curva naranja es la Gumbel ajustada por máxima verosimilitud; la discontinua, la predicción de Karlin-Altschul con λ calculado por la ecuación (3.30), el valor de K que publica BLAST y la corrección de longitud efectiva (ecuación (3.32)). La asimetría hacia la derecha es la firma de la distribución de valores extremos: los máximos grandes son raros, pero mucho menos raros que en una normal. Datos de `figuras/cap03/generar.py`.

Es la distribución de valores extremos de Gumbel. Dembo et al. (1994) dieron la demostración rigurosa del límite. La predicción es muy concreta: la moda crece como el *logaritmo* del espacio de búsqueda y la anchura es $1/\lambda$, independiente de las longitudes. Comprobémoslo. Generamos 10 000 pares de secuencias aleatorias de 200 aminoácidos con las frecuencias de fondo de BLAST, calculamos para cada par la puntuación local máxima sin huecos con BLOSUM62 y ajustamos una Gumbel por máxima verosimilitud, el método que Mott (1992) propuso para este fin (figura 3.15).

El ajuste da $\hat{\lambda} = 1/\hat{\beta} = 0,324$, a un 2 % del valor teórico $\lambda = 0,3176$, y una moda $\hat{\mu} = 26,05$. La teoría, con el $K = 0,134$ que BLAST+ imprime para BLOSUM62 sin huecos, predice $\mu = \ln(0,134 \times 200^2)/0,3176 = 27,04$: un punto de más. La discrepancia no es un error de la teoría, sino de su aplicación a secuencias cortas. Un HSP que empiece cerca del final de una secuencia no tiene espacio para alcanzar su puntuación típica, de modo que las secuencias se comportan como si fueran más cortas. Altschul y Gish (1996) cuantificaron el efecto: un HSP significativo mide en promedio $\ell \approx \ln(Kmn)/H$ columnas, donde H es la entropía relativa en nats, y las longitudes *efectivas* son

$$m' = m - \ell, \quad n' = n - N\ell, \quad \ell = \frac{\ln(Kmn)}{H}. \quad (3.32)$$

DONDE

ℓ	Longitud esperada de un HSP con la puntuación típica, en residuos.
N	Número de secuencias en la base de datos (cada una pierde ℓ posiciones de arranque).
H	Entropía relativa de la matriz, en nats (0,401 para BLOSUM62 sin huecos, según BLAST+).

Con $H = 0,401$ nats, $\ell = 21,4$ y $m' = n' = 178,6$, la moda corregida es 26,32, ya muy cerca del ajuste. La cola, que es lo que importa para la significancia, concuerda aún mejor: la fracción simulada de pares con $S_{\text{máx}} \geq 40$ es 0,0125 y la teoría con longitudes efectivas predice 0,0129. Estimar K con la moda ajustada y las longitudes efectivas da 0,145, a un 8 % del valor de BLAST.

```
gumbel_simulacion.py

import numpy as np
from scipy import stats

def max_local_sin_huecos(S):
    """S: puntuaciones (n x m) de un par; Kadane por diagonal."""
    H, best = np.zeros(S.shape[1]), 0.0
    for fila in S:
        H = np.maximum(0, np.concatenate(([0], H[:-1]))) + fila
        best = max(best, H.max())
    return best

rng = np.random.default_rng(1)
# B62: BLOSUM62 como arreglo 20x20; p: frecuencias de fondo
maximos = [max_local_sin_huecos(B62[np.ix_(rng.choice(20, 200, p=p),
                                             rng.choice(20, 200, p=p))])]
    for _ in range(10_000)]
mu, beta = stats.gumbel_r.fit(maximos)
print(f"lambda estimado = {1 / beta:.3f}")           # ~0,32
```

3.4.6 Puntuación en bits

Como λ y K dependen de la matriz, las puntuaciones brutas no son comparables entre búsquedas. BLAST las normaliza a *bits*:

$$S' = \frac{\lambda S - \ln K}{\ln 2}, \quad \text{de modo que} \quad E = mn 2^{-S'}. \quad (3.33)$$

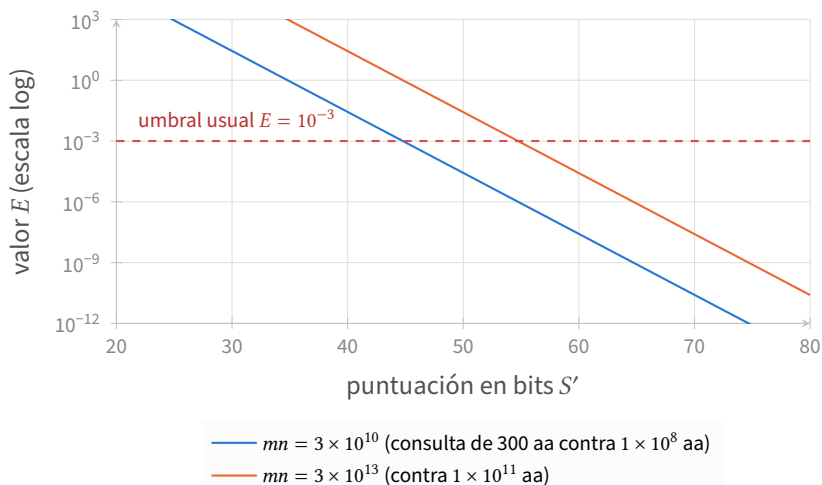


Figura 3.16. El valor E decrece exponencialmente con la puntuación en bits (ecuación (3.33)). La misma puntuación que es muy significativa contra una base de datos pequeña (azul) puede no serlo contra una mil veces mayor (naranja): el tamaño del espacio de búsqueda forma parte de la evidencia.

Esta forma es muy práctica: cada bit adicional de puntuación reduce a la mitad el número esperado de falsos positivos, y para saber si un resultado es significativo basta comparar S' con $\log_2(mn)$, que es el número de bits necesarios para “destacar” en un espacio de búsqueda de tamaño mn .

Ejemplo 3.11. (Interpretar un resultado real)

Una búsqueda **blastp** de una proteína de 350 aminoácidos contra una base de datos de 2×10^{11} residuos devuelve un resultado con puntuación de 85 bits. El valor esperado es

$$E = 350 \times 2 \times 10^{11} \times 2^{-85} \approx 7 \times 10^{13} \times 2,6 \times 10^{-26} \approx 1,8 \times 10^{-12}.$$

Esperaríamos encontrar por azar un resultado así una vez en cada medio billón de búsquedas: la homología es muy probable. En cambio, un resultado de 40 bits en la misma búsqueda tendría $E \approx 64$: decenas de resultados tan buenos aparecerían por azar, así que no debe interpretarse como evidencia de homología.

Ejemplo 3.12. (El peso de la longitud efectiva)

Para alineamientos con huecos no hay fórmula para λ y K , y BLAST usa valores estimados por simulación; para BLOSUM62 con existencia 11 y extensión 1 imprime $\lambda = 0,267$, $K = 0,041$ y $H = 0,140$ nats. Supongamos una consulta de $m = 350$ residuos contra una base de datos del tamaño aproximado de Swiss-Prot, con $n = 2,06 \times 10^8$ residuos en $N = 570\,000$ secuencias. La ecuación (3.32) da $\ell = \ln(0,041 \times 350 \times 2,06 \times 10^8)/0,140 \approx 156$, de modo que $m' \approx 194$ y $n' \approx 2,06 \times 10^8 - 570\,000 \times 156 \approx 1,17 \times 10^8$. El espacio de búsqueda efectivo, $m'n' \approx 2,3 \times 10^{10}$, es 3,2 veces menor que el nominal, $mn = 7,2 \times 10^{10}$. Un alineamiento con puntuación bruta $S = 120$ vale $S' = (0,267 \times 120 - \ln 0,041)/\ln 2 \approx 50,8$ bits, y su valor E es $1,1 \times 10^{-5}$ con la corrección, frente a $3,6 \times 10^{-5}$ sin ella. La corrección importa más cuanto más corta es la consulta y más secuencias tiene la base de datos.

blast_local.py

```
import subprocess
from Bio import Blast                                # Biopython >= 1.83

subprocess.run(["blastp", "-query", "consulta.fasta",
               "-db", "swissprot",
               "-evalue", "1e-3", "-matrix", "BLOSUM62",
               "-gapopen", "11", "-gapextend", "1",
               "-outfmt", "5", "-out", "resultado.xml"], check=True)
with open("resultado.xml", "rb") as fh:
    for hit in Blast.read(fh):                        # un Hit por secuencia sujeto
        hsp = hit[0]                                  # mejor HSP
        print(f"{hit.target.description[:40]:40s} "
              f"{hsp.annotations['bit score']:6.1f} bits "
              f"E={hsp.annotations['evalue']:.1e}")
```

3.4.7 Los supuestos, y cuándo fallan

La elegancia de la ecuación (3.29) no debe ocultar sus supuestos. La teoría es exacta solo para alineamientos *sin huecos* entre secuencias *largas* de composición *aleatoria e independiente*. Para alineamientos con huecos no existe una fórmula cerrada: λ y K se estiman por simulación para cada combinación de matriz y penalizaciones, y las secuencias cortas requieren la corrección de longitud efectiva (Altschul et al., 1997; Karlin et al., 1990). La evidencia empírica y teórica indica que,

dentro del régimen logarítmico, las puntuaciones con huecos también siguen una Gumbel, y Altschul et al. (2001) revisaron los métodos para estimar sus parámetros con rapidez. Eddy (2008) propuso un camino alternativo: un modelo probabilístico de alineamiento local en el que λ puede fijarse de antemano, lo que simplifica mucho la estimación de la significancia y es la base de la estadística de HMMER. Por su parte, FASTA estima la distribución directamente de las puntuaciones de cada búsqueda, aprovechando que casi todas las secuencias de la base de datos no están relacionadas con la consulta (Pearson, 1998). Cuando un HSP no basta, BLAST combina varios HSP compatibles de un mismo par con la estadística de sumas de Karlin y Altschul (1993).

Las regiones de baja complejidad y los sesgos de composición también incumplen el supuesto de independencia. Un tramo como **AAAAAGAAAAA**, o una región rica en prolina, puede parecerse a muchas secuencias no relacionadas y producir valores E engañosos. BLAST enmascara estas regiones antes de buscar semillas, con SEG para proteínas (Wootton & Federhen, 1993) y DUST para ADN (Morgulis et al., 2006), y ajusta la estadística a la composición de cada par de secuencias (Altschul et al., 2005; Schäffer et al., 2001).

3.4.8 PSI-BLAST: aprender de los resultados

Una sola matriz de sustitución trata igual todas las posiciones de una proteína, pero la evolución no lo hace: un sitio catalítico no tolera cambios y un bucle superficial admite casi cualquiera. PSI-BLAST (Altschul et al., 1997) aprovecha los propios resultados para aprender esas diferencias. Tras una primera búsqueda con BLOSUM62, reúne los resultados con E menor que un umbral de inclusión (en BLAST+, 0,002 por defecto), construye con ellos un alineamiento múltiple y lo resume en una *matriz de puntuación específica de posición* (PSSM): una columna de 20 puntuaciones por cada posición de la consulta, calculadas como en la ecuación (3.19) pero con frecuencias objetivo propias de esa posición. Busca de nuevo con la PSSM y repite hasta que no aparecen secuencias nuevas. Cada iteración detecta homólogos más lejanos, porque la PSSM “sabe” qué posiciones importan. El riesgo simétrico es la *contaminación*: si una secuencia no homóloga entra en una iteración, la PSSM aprende su señal y arrastra consigo a toda su familia. Los refinamientos de Schäffer et al. (2001), en particular la estadística basada en composición, redujeron mucho este problema. Las PSSM son el primer paso hacia los perfiles y los modelos ocultos de Márkov de perfiles que estudiaremos en el capítulo 4.

PSI-BLAST con BLAST+

```
psiblast -query consulta.fasta -db swissprot -num_iterations 3 \
        -inclusion_ethresh 0.002 -evalue 1e-3 \
        -out_ascii_pssm consulta.pssm -outfmt 6 -out resultados.tsv
```

⚠ Tres errores frecuentes al leer un valor E

1. *Tratarlo como una probabilidad.* E es un número esperado y puede ser mayor que 1. Solo cuando $E < 0,01$ se cumple que $E \approx p$.
2. *Compararlo entre bases de datos.* Crece linealmente con n : la misma búsqueda contra una base de datos diez veces mayor produce un E diez veces mayor.
3. *Confundir significancia con homología global.* Un E minúsculo indica que *algún segmento* es homólogo; puede tratarse de un único dominio compartido entre proteínas por lo demás distintas.

☰ En resumen

- Un *dot plot* muestra todas las similitudes entre dos secuencias; la ventana y el umbral se eligen equilibrando el ruido (ecuación (3.2)), que cae exponencialmente con w , y la sensibilidad sobre las regiones homólogas.
- Un alineamiento es un camino en el grafo de edición. Su número crece exponencialmente, pero el principio de optimalidad reduce la búsqueda del mejor a llenar una tabla en $O(nm)$. Needleman-Wunsch alinea globalmente; Smith-Waterman, localmente, gracias al cero de la recurrencia; las variantes semiglobal y de solapamiento solo cambian las fronteras.
- Los huecos afines modelan *indels* de varias bases y requieren tres matrices (Gotoh), equivalentes a un HMM de pares. Hirschberg reduce la memoria a $O(n + m)$ y las bandas reducen el tiempo a $O(kn)$.
- Las matrices PAM y BLOSUM son razones de verosimilitud logarítmicas: PAM extrapola con una cadena de Márkov desde secuencias cercanas; BLOSUM observa directamente bloques divergentes. Su entropía relativa H mide la información por columna, y cada matriz es óptima a su propia distancia evolutiva.

- BLAST acelera la búsqueda con vecindades de palabras, dos *hits* y extensiones *X-drop*; su valor $E = Kmn e^{-\lambda S}$, corregido por longitud efectiva, cuantifica cuántos resultados así esperaríamos por azar, y la puntuación en bits lo hace comparable entre búsquedas.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 3.1** ●●● Dibuje a mano el *dot plot* ($w = 1$) de ACGTACGT contra sí misma. ¿Cuántas diagonales aparecen y qué representa cada una?
- 3.2** ●●● Usando la ecuación (3.2), calcule la probabilidad de un punto espurio con $w = 15$, $t = 10$ para ADN y para proteínas (suponga 20 aminoácidos equiprobables). ¿Por qué los *dot plots* de proteínas pueden usar ventanas más cortas?
- 3.3** ●●● Enumere a mano los 13 caminos del grafo de edición para dos secuencias de longitud 2 y agrúpelos en los $\binom{4}{2} = 6$ alineamientos biológicamente distintos. Compruebe con la ecuación (3.9) que para longitudes 2 y 3 hay 25 caminos.
- 3.4** ●●● Complete a mano la matriz de Needleman-Wunsch para GAATTC contra GATTA con $+2/-1$ y $d = 2$. Encuentre *todos* los alineamientos óptimos. ¿Por qué puede haber más de uno?
- 3.5** ●●● Modifique la función `needleman_wunsch` para implementar el alineamiento de solapamiento de la tabla 3.1 y compruebe el ejemplo del texto, ACGGTTAGC contra TAGCTTGA.
- 3.6** ●●● Demuestre que, si $E[s] > 0$, la puntuación de Smith-Waterman entre dos secuencias aleatorias crece linealmente con su longitud. ¿Qué implica esto para el uso de matrices de puntuación de identidad pura (+1/0)?
- 3.7** ●●● Implemente Hirschberg y verifique, para GATTACA y GTTAGCA, que el punto medio es la celda (3, 2) del ejemplo 3.7. Mida la memoria máxima frente a la versión con tabla completa para secuencias de 5000 bases.

- 3.8** ●●● Repita el ejemplo 3.9 añadiendo una séptima secuencia, GSIT. ¿En qué grupo cae con el umbral del 62 %? ¿Qué puntuaciones cambian?
- 3.9** ●●● Para ADN con $s = +1$ (coincidencia) y $s = -2$ (sustitución), y bases equiprobables, demuestre que $\lambda = \ln((3 + \sqrt{21})/2)$. Compruebe éste y los demás valores del ejemplo 3.10 numéricamente con `scipy.optimize.brentq`.
- 3.10** ●●● Calcule, con BLOSUM62 y $w = 3$, el tamaño de la vecindad de WWW y de AAA para $T = 11$. Explique la diferencia en términos de la ecuación (3.19) y de las frecuencias de fondo.
- 3.11** ●●● Extienda la simulación de la figura 3.15 a alineamientos *con huecos* (use PairwiseAligner en modo local con BLOSUM62, apertura -12 y extensión -1 en la convención de Biopython) y 2000 pares de 200 residuos. Compare el λ estimado con el valor 0,267 que publica BLAST. ¿Por qué la distribución sigue siendo de Gumbel aunque la teoría no lo garantice?

Referencias del capítulo

- Altschul, S. F., Bundschuh, R., Olsen, R., & Hwa, T. (2001). The estimation of statistical parameters for local alignment score distributions. *Nucleic Acids Research*, 29(2), 351-361. <https://doi.org/10.1093/nar/29.2.351>
- Altschul, S. F. (1991). Amino acid substitution matrices from an information theoretic perspective. *Journal of Molecular Biology*, 219(3), 555-565. [https://doi.org/10.1016/0022-2836\(91\)90193-A](https://doi.org/10.1016/0022-2836(91)90193-A)
- Altschul, S. F. (1993). A protein alignment scoring system sensitive at all evolutionary distances. *Journal of Molecular Evolution*, 36(3), 290-300. <https://doi.org/10.1007/BF00160485>
- Altschul, S. F., & Gish, W. (1996). Local alignment statistics. En *Methods in Enzymology* (pp. 460-480, Vol. 266). Elsevier. [https://doi.org/10.1016/S0076-6879\(96\)66029-7](https://doi.org/10.1016/S0076-6879(96)66029-7)
- Altschul, S. F., Gish, W., Miller, W., Myers, E. W., & Lipman, D. J. (1990). Basic local alignment search tool. *Journal of Molecular Biology*, 215(3), 403-410. [https://doi.org/10.1016/S0022-2836\(05\)80360-2](https://doi.org/10.1016/S0022-2836(05)80360-2)
- Altschul, S. F., Madden, T. L., Schäffer, A. A., Zhang, J., Zhang, Z., Miller, W., & Lipman, D. J. (1997). Gapped BLAST and PSI-BLAST: A new generation of protein database search programs. *Nucleic Acids Research*, 25(17), 3389-3402. <https://doi.org/10.1093/nar/25.17.3389>

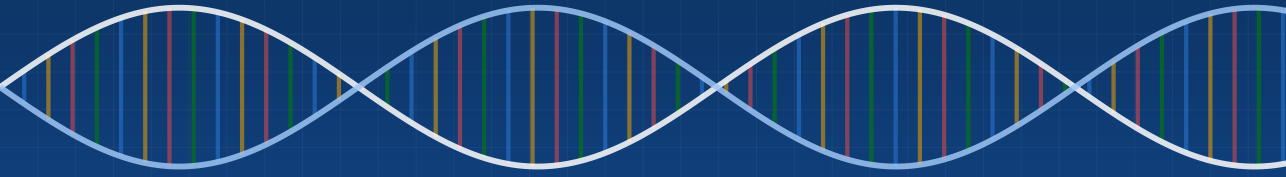
- Altschul, S. F., Wootton, J. C., Gertz, E. M., Agarwala, R., Morgulis, A., Schäffer, A. A., & Yu, Y.-K. (2005). Protein database searches using compositionally adjusted substitution matrices. *The FEBS Journal*, 272(20), 5101-5109. <https://doi.org/10.1111/j.1742-4658.2005.04945.x>
- Arratia, R., & Waterman, M. S. (1994). A phase transition for the score in matching random sequences allowing deletions. *The Annals of Applied Probability*, 4(1). <https://doi.org/10.1214/aoap/1177005208>
- Bellman, R. (1954). The theory of dynamic programming. *Bulletin of the American Mathematical Society*, 60(6), 503-515. <https://doi.org/10.1090/S0002-9904-1954-09848-8>
- Camacho, C., Coulouris, G., Avagyan, V., Ma, N., Papadopoulos, J., Bealer, K., & Madden, T. L. (2009). BLAST+: Architecture and applications. *BMC Bioinformatics*, 10, 421. <https://doi.org/10.1186/1471-2105-10-421>
- Chao, K.-M., Pearson, W. R., & Miller, W. (1992). Aligning two sequences within a specified diagonal band. *Bioinformatics*, 8(5), 481-487. <https://doi.org/10.1093/bioinformatics/8.5.481>
- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Dayhoff, M. O., Schwartz, R. M., & Orcutt, B. C. (1978). A model of evolutionary change in proteins. En M. O. Dayhoff (Ed.), *Atlas of protein sequence and structure* (pp. 345-352, Vol. 5). National Biomedical Research Foundation.
- Dembo, A., Karlin, S., & Zeitouni, O. (1994). Limit distribution of maximal non-aligned two-sequence segmental score. *The Annals of Probability*, 22(4). <https://doi.org/10.1214/aop/1176988493>
- Dobzhansky, T. (1973). Nothing in biology makes sense except in the light of evolution. *The American Biology Teacher*, 35(3), 125-129. <https://doi.org/10.2307/4444260>
- Doolittle, R. F. (1981). Similar amino acid sequences: Chance or common ancestry? *Science*, 214(4517), 149-159. <https://doi.org/10.1126/science.7280687>
- Durbin, R., Eddy, S. R., Krogh, A., & Mitchison, G. (1998). *Biological sequence analysis: Probabilistic models of proteins and nucleic acids*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511790492>
- Eddy, S. R. (2004a). What is dynamic programming? *Nature Biotechnology*, 22(7), 909-910. <https://doi.org/10.1038/nbt0704-909>
- Eddy, S. R. (2004b). Where did the BLOSUM62 alignment score matrix come from? *Nature Biotechnology*, 22(8), 1035-1036. <https://doi.org/10.1038/nbt0804-1035>
- Eddy, S. R. (2008). A probabilistic model of local sequence alignment that simplifies statistical significance estimation. *PLoS Computational Biology*, 4(5), e1000069. <https://doi.org/10.1371/journal.pcbi.1000069>

- Farrar, M. (2007). Striped Smith–Waterman speeds database searches six times over other SIMD implementations. *Bioinformatics*, 23(2), 156–161. <https://doi.org/10.1093/bioinformatics/btl582>
- Gibbs, A. J., & McIntyre, G. A. (1970). The diagram, a method for comparing sequences: Its use with amino acid and nucleotide sequences. *European Journal of Biochemistry*, 16(1), 1–11. <https://doi.org/10.1111/j.1432-1033.1970.tb01046.x>
- Gonnet, G. H., Cohen, M. A., & Benner, S. A. (1992). Exhaustive matching of the entire protein sequence database. *Science*, 256(5062), 1443–1445. <https://doi.org/10.1126/science.1604319>
- Gotoh, O. (1982). An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3), 705–708. [https://doi.org/10.1016/0022-2836\(82\)90398-9](https://doi.org/10.1016/0022-2836(82)90398-9)
- Gusfield, D. (1997). *Algorithms on strings, trees and sequences: Computer science and computational biology*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511574931>
- Henikoff, S., & Henikoff, J. G. (1992). Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22), 10915–10919. <https://doi.org/10.1073/pnas.89.22.10915>
- Henikoff, S., & Henikoff, J. G. (1993). Performance evaluation of amino acid substitution matrices. *Proteins: Structure, Function, and Bioinformatics*, 17(1), 49–61. <https://doi.org/10.1002/prot.340170108>
- Hirschberg, D. S. (1975). A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6), 341–343. <https://doi.org/10.1145/360825.360861>
- Jones, D. T., Taylor, W. R., & Thornton, J. M. (1992). The rapid generation of mutation data matrices from protein sequences. *Bioinformatics*, 8(3), 275–282. <https://doi.org/10.1093/bioinformatics/8.3.275>
- Karlin, S., & Altschul, S. F. (1990). Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes. *Proceedings of the National Academy of Sciences*, 87(6), 2264–2268. <https://doi.org/10.1073/pnas.87.6.2264>
- Karlin, S., & Altschul, S. F. (1993). Applications and statistics for multiple high-scoring segments in molecular sequences. *Proceedings of the National Academy of Sciences*, 90(12), 5873–5877. <https://doi.org/10.1073/pnas.90.12.5873>
- Karlin, S., Dembo, A., & Kawabata, T. (1990). Statistical composition of high-scoring segments from molecular sequences. *The Annals of Statistics*, 18(2), 571–581. <https://doi.org/10.1214/aos/1176347616>
- Li, H. (2018). Minimap2: Pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18), 3094–3100. <https://doi.org/10.1093/bioinformatics/bty191>
- Ma, B., Tromp, J., & Li, M. (2002). PatternHunter: Faster and more sensitive homology search. *Bioinformatics*, 18(3), 440–445. <https://doi.org/10.1093/bioinformatics/18.3.440>

- Maizel, J. V., & Lenk, R. P. (1981). Enhanced graphic matrix analysis of nucleic acid and protein sequences. *Proceedings of the National Academy of Sciences*, 78(12), 7665-7669. <https://doi.org/10.1073/pnas.78.12.7665>
- Morgulis, A., Gertz, E. M., Schäffer, A. A., & Agarwala, R. (2006). A fast and symmetric DUST implementation to mask low-complexity DNA sequences. *Journal of Computational Biology*, 13(5), 1028-1040. <https://doi.org/10.1089/cmb.2006.13.1028>
- Mott, R. (1992). Maximum-likelihood estimation of the statistical distribution of Smith-Waterman local sequence similarity scores. *Bulletin of Mathematical Biology*, 54(1), 59-75. <https://doi.org/10.1007/BF02458620>
- Myers, E. W., & Miller, W. (1988). Optimal alignments in linear space. *Bioinformatics*, 4(1), 11-17. <https://doi.org/10.1093/bioinformatics/4.1.11>
- Needleman, S. B., & Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3), 443-453. [https://doi.org/10.1016/0022-2836\(70\)90057-4](https://doi.org/10.1016/0022-2836(70)90057-4)
- Pearson, W. R. (1998). Empirical statistical estimates for sequence similarity searches. *Journal of Molecular Biology*, 276(1), 71-84. <https://doi.org/10.1006/jmbi.1997.1525>
- Pearson, W. R. (2013). An introduction to sequence similarity ("homology") searching. *Current Protocols in Bioinformatics*, 42(1), 3.1.1-3.1.8. <https://doi.org/10.1002/0471250953.bi0301s42>
- Pearson, W. R., & Lipman, D. J. (1988). Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences*, 85(8), 2444-2448. <https://doi.org/10.1073/pnas.85.8.2444>
- Rost, B. (1999). Twilight zone of protein sequence alignments. *Protein Engineering, Design and Selection*, 12(2), 85-94. <https://doi.org/10.1093/protein/12.2.85>
- Schäffer, A. A., Aravind, L., Madden, T. L., Shavirin, S., Spouge, J. L., Wolf, Y. I., Koonin, E. V., & Altschul, S. F. (2001). Improving the accuracy of PSI-BLAST protein database searches with composition-based statistics and other refinements. *Nucleic Acids Research*, 29(14), 2994-3005. <https://doi.org/10.1093/nar/29.14.2994>
- Sellers, P. H. (1974). On the theory and computation of evolutionary distances. *SIAM Journal on Applied Mathematics*, 26(4), 787-793. <https://doi.org/10.1137/0126070>
- Smith, T. F., & Waterman, M. S. (1981). Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1), 195-197. [https://doi.org/10.1016/0022-2836\(81\)90087-5](https://doi.org/10.1016/0022-2836(81)90087-5)
- Sonnhammer, E. L. L., & Durbin, R. (1995). A dot-matrix program with dynamic threshold control suited for genomic DNA and protein sequence analysis. *Gene*, 167(1-2), GC1-GC10. [https://doi.org/10.1016/0378-1119\(95\)00714-8](https://doi.org/10.1016/0378-1119(95)00714-8)
- States, D. J., Gish, W., & Altschul, S. F. (1991). Improved sensitivity of nucleic acid database searches using application-specific scoring matrices. *Methods*, 3(1), 66-70. [https://doi.org/10.1016/S1046-2023\(05\)80165-3](https://doi.org/10.1016/S1046-2023(05)80165-3)

- Styczynski, M. P., Jensen, K. L., Rigoutsos, I., & Stephanopoulos, G. (2008). BLOSUM62 miscalculations improve search performance. *Nature Biotechnology*, 26(3), 274-275. <https://doi.org/10.1038/nbt0308-274>
- Ukkonen, E. (1985). Algorithms for approximate string matching. *Information and Control*, 64(1-3), 100-118. [https://doi.org/10.1016/S0019-9958\(85\)80046-2](https://doi.org/10.1016/S0019-9958(85)80046-2)
- Vingron, M., & Waterman, M. S. (1994). Sequence alignment and penalty choice. *Journal of Molecular Biology*, 235(1), 1-12. [https://doi.org/10.1016/S0022-2836\(05\)80006-3](https://doi.org/10.1016/S0022-2836(05)80006-3)
- Wagner, R. A., & Fischer, M. J. (1974). The string-to-string correction problem. *Journal of the ACM*, 21(1), 168-173. <https://doi.org/10.1145/321796.321811>
- Waterman, M. S., Smith, T. F., & Beyer, W. A. (1976). Some biological sequence metrics. *Advances in Mathematics*, 20(3), 367-387. [https://doi.org/10.1016/0001-8708\(76\)90202-4](https://doi.org/10.1016/0001-8708(76)90202-4)
- Waterman, M. S., & Eggert, M. (1987). A new algorithm for best subsequence alignments with application to tRNA-rRNA comparisons. *Journal of Molecular Biology*, 197(4), 723-728. [https://doi.org/10.1016/0022-2836\(87\)90478-5](https://doi.org/10.1016/0022-2836(87)90478-5)
- Wootton, J. C., & Federhen, S. (1993). Statistics of local complexity in amino acid sequences and sequence databases. *Computers & Chemistry*, 17(2), 149-163. [https://doi.org/10.1016/0097-8485\(93\)85006-X](https://doi.org/10.1016/0097-8485(93)85006-X)
- Zhang, Z., Schwartz, S., Wagner, L., & Miller, W. (2000). A greedy algorithm for aligning DNA sequences. *Journal of Computational Biology*, 7(1-2), 203-214. <https://doi.org/10.1089/10665270050081478>

Alineamiento múltiple, motivos y perfiles



Los HMM de perfil convierten un alineamiento múltiple de secuencias en un sistema de puntuación específico de cada posición, adecuado para buscar en bases de datos secuencias remotamente homólogas.

— Eddy (1998)

● Al terminar este capítulo usted podrá

- Definir un alineamiento múltiple de secuencias (MSA), puntuarlo con la suma de pares y explicar por qué su solución exacta es computacionalmente intratable.
- Reproducir paso a paso el alineamiento progresivo: distancias, árbol guía y alineamiento de perfiles; reconocer sus limitaciones y cómo las corrigen MUSCLE, MAFFT, T-Coffee y Clustal Omega.
- Construir una matriz de peso posicional (PWM) a partir de sitios de unión, con pseudoconteos, y usarla para buscar sitios en una secuencia.
- Calcular la entropía y el contenido de información de cada posición de un motivo e interpretar un *sequence logo*.
- Formular un modelo oculto de Márkov y resolver sus tres problemas clásicos con los algoritmos *forward*, de Viterbi y de Baum-Welch.

- Describir la arquitectura de un HMM de perfil (estados de coincidencia, inserción y delección) y explicar cómo HMMER y Pfam lo usan para anotar dominios de proteínas.

En el capítulo 3 comparamos *dos* secuencias. Ahora consideraremos familias de proteínas con cientos o miles de miembros repartidos por el árbol de la vida. Nos interesa saber qué comparten *todos* ellos. Si una posición se conserva en la hemoglobina humana, en la del ratón y en la mioglobina de un cachalote tras cientos de millones de años de evolución, puede cumplir una función importante: sostener el grupo hemo o estabilizar una hélice, por ejemplo. Con solo dos secuencias cuesta distinguir esa conservación de una coincidencia debida a su cercanía evolutiva; muchas secuencias aportan más evidencia.

Partiremos de secuencias emparentadas y terminaremos con un *modelo estadístico* de su familia. Primero construiremos un alineamiento múltiple. La programación dinámica de la sección 3.2 se vuelve demasiado costosa y tendremos que recurrir a métodos como el alineamiento progresivo (sección 4.1). Después resumiremos sitios alineados sin huecos mediante una matriz de peso posicional y mediremos la información de cada posición (sección 4.2). Por último, incorporaremos inserciones y delecciones con modelos ocultos de Márkov de perfil, utilizados por HMMER y Pfam (sección 4.3). Allí retomaremos el HMM de pares asociado al algoritmo de Gotoh (ecuación (3.14)).

4.1 Alineamiento múltiple: el método progresivo

 Colab · 4.1 · MSA: alineamiento progresivo

 abrir notebook

La receta de la abuela

Cinco primos conservan, cada uno, una copia manuscrita de la receta de pan de la abuela. Con los años, cada copia ha acumulado cambios: alguien sustituyó la manteca por aceite, otro añadió una pizca de anís, otro olvidó copiar una línea entera. Si quisiera reconstruir la receta original, no empezaría comparando las copias más distintas. Empezaría por las de las dos hermanas que viven juntas, casi idénticas, y fijaría entre ambas una versión común; luego la compararía con la del primo que vive en la misma ciudad; y solo al final

se enfrentaría a la copia del pariente lejano. Cada decisión que tome en los primeros pasos (“esta línea se perdió aquí”) condicionará todas las siguientes. Ese procedimiento, con sus virtudes y su punto débil, es exactamente el alineamiento progresivo.

4.1.1 De dos secuencias a muchas

Un alineamiento múltiple extiende la idea del alineamiento por pares: se escriben k secuencias una debajo de otra, insertando huecos, de modo que los residuos que descienden de un mismo residuo ancestral queden en la misma columna.

Definición 4.1. (Alineamiento múltiple de secuencias)

Sean $x^{(1)}, \dots, x^{(k)}$ secuencias sobre un alfabeto $\mathcal{A}$. Un *alineamiento múltiple* es una matriz M de k filas y L columnas con entradas en $\mathcal{A} \cup \{-\}$ tal que (i) al eliminar los huecos de la fila p se recupera exactamente $x^{(p)}$ y (ii) ninguna columna está formada solo por huecos. Cada columna es una hipótesis de homología posicional.

Los MSA sirven para inferir árboles filogenéticos, identificar residuos catalíticos, predecir estructuras, construir los perfiles y HMM de la sección 4.3 y detectar sitios bajo selección. Cada aplicación depende de la calidad del alineamiento. Como ocurría con dos secuencias, evaluar esa calidad exige decidir primero cómo puntuarlo.

4.1.2 La puntuación de suma de pares

La función objetivo más usada es la *suma de pares* (SP, *sum of pairs*): puntuar cada columna como la suma de las puntuaciones de todos los pares de residuos que contiene, con la misma matriz de sustitución que usamos para dos secuencias.

$$S_{\text{SP}}(\mathbf{M}) = \sum_{c=1}^L \sum_{p < r} w_p w_r \sigma(M_{pc}, M_{rc}), \quad \sigma(a, b) = \begin{cases} s(a, b) & a, b \in \mathcal{A} \\ -d & \text{uno de los dos es } - \\ 0 & a = b = - \end{cases} \quad (4.1)$$

DONDE	
M, M_{pc}	Alineamiento múltiple y su símbolo en la fila p , columna c .
L	Número de columnas del alineamiento.
$p < r$	Recorre los $\binom{k}{2}$ pares de filas distintas.
w_p	Peso de la secuencia p (todos iguales a 1 en la versión sin ponderar).
$s(a, b)$	Puntuación de sustitución (p. ej., BLOSUM62, sección 3.3).
d	Penalización por cada par residuo–hueco; en la práctica se usan huecos afines por par.

Sin ponderación, la SP cuenta todos los pares por igual. Si el conjunto contiene diez variantes casi idénticas de una proteína de mamífero y una sola de levadura, dominarán las comparaciones entre mamíferos y el alineamiento puede perjudicar a la secuencia divergente. Los pesos w_p reducen ese sesgo. El método de Henikoff y Henikoff (1994) asigna más peso a las secuencias que aportan residuos poco frecuentes en cada columna:

$$w_p \propto \sum_{c=1}^L \frac{1}{r_c n_c(M_{pc})}, \tag{4.2}$$

DONDE	
r_c	Número de símbolos distintos presentes en la columna c .
$n_c(a)$	Número de veces que aparece el símbolo a en la columna c .

Una secuencia que comparte cada residuo con otras nueve recibe, en esa columna, una décima parte del crédito que recibe una que tiene un residuo único. Los pesos se normalizan después para que sumen uno.

Ejemplo 4.1. (Suma de pares de cuatro columnas de globinas)

Las cuatro primeras columnas del alineamiento de cinco globinas que construiremos en esta sección (figura 4.3) son

MYG_HUMAN	M	G	-	L
MYG_PHYMC	M	V	-	L
HBA_HUMAN	M	V	-	L
HBB_HUMAN	M	V	H	L
HBB1_MOUSE	M	V	H	L

Con BLOSUM62, $d = 8$ y pesos unitarios, hay $\binom{5}{2} = 10$ pares por columna:

- Columna 1: diez pares M-M, $10 \times 5 = 50$.
- Columna 2: cuatro pares G-V (-3) y seis V-V (+4): $-12 + 24 = 12$.
- Columna 3: tres pares hueco-hueco (0), seis pares hueco-H (-8) y un par H-H (+8): $-48 + 8 = -40$.
- Columna 4: diez pares L-L, $10 \times 4 = 40$.

El total es $50 + 12 - 40 + 40 = 62$. Los pesos de Henikoff de las cinco filas (tratando el hueco como un símbolo más) resultan 0,267, 0,173, 0,173, 0,194 y 0,194: la mioglobina humana recibe más peso porque su G es única en la columna 2.

4.1.3 Programación dinámica en k dimensiones

La recurrencia de Needleman-Wunsch (ecuación (3.10)) se generaliza sin dificultad conceptual. En lugar de una tabla bidimensional, llenamos un hipercubo con una celda por cada tupla de prefijos $(i_1, \dots, i_k)$. La última columna de un alineamiento óptimo de esos prefijos contiene, en cada fila, o bien el último residuo del prefijo, o bien un hueco; hay por tanto $2^k - 1$ posibilidades (se excluye la columna de solo huecos):

$$F(\mathbf{i}) = \max_{\substack{\delta \in \{0,1\}^k \\ \delta \neq \mathbf{0}}} \left\{ F(\mathbf{i} - \delta) + \text{SP}(\text{columna definida por } \delta) \right\}, \quad (4.3)$$

DONDE

$\mathbf{i} = (i_1, \dots, i_k)$	Vector de longitudes de los prefijos considerados.
δ	Vector binario: $\delta_p = 1$ si la fila p aporta su residuo $x_p^{(p)}$ a la columna; $\delta_p = 0$ si aporta un hueco.
$F(\mathbf{i})$	Mejor puntuación SP de un alineamiento de los k prefijos.

Si todas las secuencias miden n , el hipercubo tiene $(n + 1)^k$ celdas, cada una consulta $2^k - 1$ vecinas y cada columna candidata requiere $\binom{k}{2}$ sumas. El coste total es

$$T(n, k) = O(k^2 2^k n^k) \text{ en tiempo,} \quad O(n^k) \text{ en memoria.} \quad (4.4)$$

Para las cinco globinas de este capítulo ($n \approx 150$) eso son unas $151^5 \approx 7,9 \times 10^{10}$ celdas, más de 300 gigabytes de memoria con enteros de 4 bytes, y $2,4 \times 10^{12}$

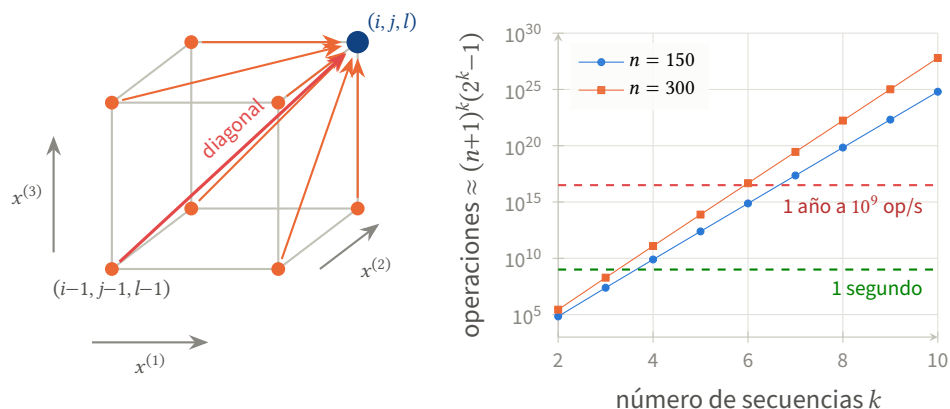


Figura 4.1. El costo del alineamiento múltiple exacto. **Izquierda:** con tres secuencias, cada celda (i, j, l) del cubo de programación dinámica depende de $2^3 - 1 = 7$ vecinas (puntos naranjas): la diagonal principal (rojo) coloca un residuo de cada secuencia en la columna; las otras seis combinan residuos y huecos. **Derecha:** el número de operaciones crece exponencialmente con k . Con proteínas de 150 residuos, la línea de un año de cómputo se cruza ya con siete secuencias; ninguna mejora de hardware alcanzará a una curva así.

transiciones. Con diez secuencias, el número de celdas ($\approx 6 \times 10^{21}$) supera la memoria de todos los ordenadores del planeta juntos (figura 4.1).

¿Se trata solo de que no hemos encontrado un algoritmo más astuto? Probablemente no.

Teorema 4.1. (Intratabilidad del MSA)

Encontrar el alineamiento múltiple de puntuación SP óptima es un problema NP-difícil; su versión de decisión es NP-completa aun con matrices de puntuación sencillas (Wang & Jiang, 1994).

En consecuencia, salvo que $P = NP$, no existe un algoritmo que resuelva el MSA exacto en tiempo polinómico en k . Wang y Jiang (1994) demostraron además la NP-completitud del *alineamiento sobre un árbol*, una variante que puntúa el MSA a lo largo de una filogenia en lugar de sumar todos los pares.

Podar el hipercubo: la cota de Carrillo-Lipman

No todo está perdido para valores pequeños de k . Carrillo y Lipman (1988) observaron que la proyección del MSA óptimo sobre cada par de secuencias es un alineamiento por pares cuya puntuación no puede quedar muy por debajo del óptimo de ese par: si lo hiciera, la suma total sería inferior a la de un MSA heurístico ya conocido. Esta desigualdad define una región alrededor de la diagonal del hipercubo fuera de la cual el camino óptimo no puede pasar, y permite alinear exactamente una decena de proteínas cortas. Más allá, incluso la región podada crece de forma exponencial.

4.1.4 La heurística progresiva

La salida práctica se debe a Feng y Doolittle (1987), que propusieron construir el MSA *de manera progresiva*, siguiendo el orden de parentesco de las secuencias. El procedimiento, que con variaciones usan casi todos los programas actuales, tiene tres etapas:

1. **Distancias.** Se alinean todos los pares de secuencias (o se estima su parecido con un método rápido) y se convierte cada similitud en una distancia d_{ij} , por ejemplo $d_{ij} = 1 - \text{identidad}$.
2. **Árbol guía.** Con la matriz de distancias se construye un árbol mediante un método de agrupamiento (UPGMA o *neighbor-joining*, que estudiaremos a fondo en el capítulo de filogenia).
3. **Alineamiento progresivo.** Se recorren los nodos internos del árbol desde las hojas hacia la raíz; en cada nodo se alinean los dos subalineamientos hijos mediante programación dinámica, y el resultado se congela y se pasa al nodo padre.

UPGMA, el método de agrupamiento más simple, une en cada paso los dos grupos A y B más cercanos y actualiza las distancias como promedio ponderado por tamaño:

$$d(A \cup B, C) = \frac{|A| d(A, C) + |B| d(B, C)}{|A| + |B|}, \quad h(A \cup B) = \frac{1}{2} d(A, B). \quad (4.5)$$

DONDE

A, B, C	Grupos (conjuntos de secuencias) en el paso actual.
$ A $	Número de secuencias del grupo A .
$h(\cdot)$	Altura del nodo nuevo en el dendrograma.

El árbol guía no pretende ser una filogenia correcta; solo es un *orden de trabajo* razonable: alinear primero lo fácil (lo más parecido) y dejar para el final lo difícil.

Alinear perfiles. El paso tres exige alinear no dos secuencias, sino dos *alineamientos*. Para ello cada subalineamiento se resume en un *perfil*: para cada columna u , el vector de frecuencias $f_u(a)$ de cada símbolo. La programación dinámica es idéntica a la de Needleman-Wunsch o Gotoh; solo cambia la puntuación de “alinear la columna u del perfil P con la columna v del perfil Q ”, que se toma como la SP media entre ambas:

$$S(P_u, Q_v) = \sum_{a \in \mathcal{A}} \sum_{b \in \mathcal{A}} f_u^P(a) f_v^Q(b) s(a, b). \quad (4.6)$$

DONDE

P_u, Q_v	Columna u del perfil P y columna v del perfil Q .
$f_u^P(a)$	Fracción (posiblemente ponderada) de secuencias de P con el símbolo a en la columna u .

Alinear una secuencia con otra es el caso particular en que ambos perfiles tienen una sola fila y f es una función indicadora. Como una columna de perfil es una distribución y no una letra, la ecuación (4.6) ya anticipa la idea central de la sección 4.3: puntuar *posiciones* con distribuciones propias.

Ejemplo 4.2. (El árbol guía de cinco globinas)

Las distancias de la figura 4.2 muestran dos parejas muy cercanas: las mioglobinas de humano y cachalote ($d = 0,156$) y las β -globinas de humano y ratón ($d = 0,197$). UPGMA procede así:

1. Une las mioglobinas, a altura $0,156/2 = 0,078$.
2. Une las β -globinas, a altura $0,197/2 \approx 0,099$.
3. Actualiza la distancia de HBA_HUMAN al grupo β con la ecuación (4.5): $\frac{1}{2}(0,536 + 0,543) = 0,539$. Es la menor, así que la cadena α se une a las β a altura 0,270.
4. Quedan dos grupos (mioglobinas y hemoglobinas); su distancia es la media de los seis pares cruzados, $4,305/6 \approx 0,718$, y la raíz queda a altura 0,359.

El orden de alineamiento reproduce la biología: las cadenas α y β de la hemoglobina proceden de una duplicación génica más reciente que la que separó a

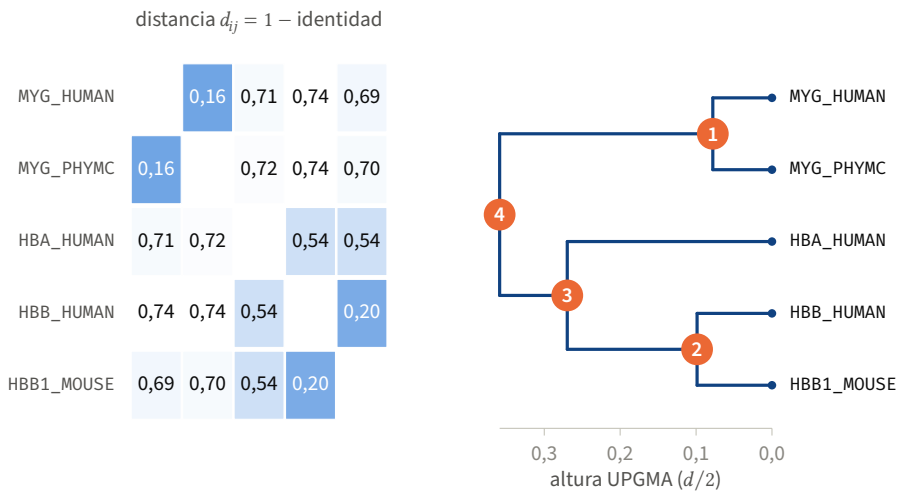


Figura 4.2. Las dos primeras etapas del alineamiento progresivo aplicadas a cinco globinas reales de UniProt. **Izquierda:** distancias $d_{ij} = 1 - \text{identidad}$ calculadas a partir de alineamientos globales por pares (BLOSUM62, huecos $-10/-1$); cuanto más oscuro, más parecidas. **Derecha:** árbol guía UPGMA; los círculos naranjas indican el *orden* en que se alinean los perfiles. Primero se unen las dos mioglobinas (1) y las dos cadenas β (2); luego la cadena α se incorpora a las β (3) y, por último, se alinean entre sí los dos grandes perfiles (4). Calculado con `figuras/cap04/generar.py`.

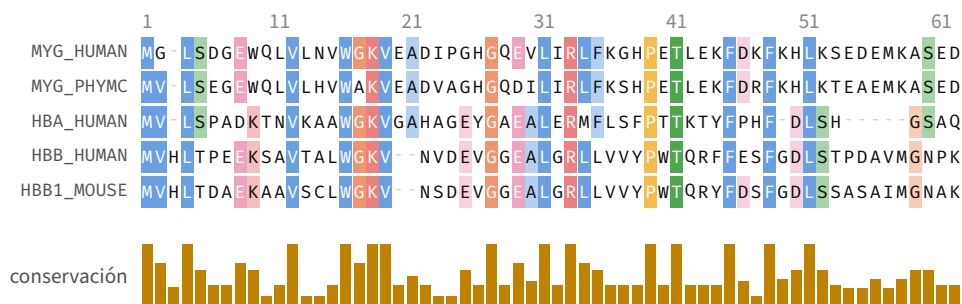


Figura 4.3. Primeras 62 columnas del alineamiento progresivo de las cinco globinas, calculado con perfiles y huecos afines. Los residuos se colorean por clase fisicoquímica (azul, hidrofóbicos; rojo, básicos; magenta, ácidos; verde, polares; naranja, glicina; amarillo, prolina; aqua, histidina y tirosina) cuando al menos el 60 % (claro) u 80 % (intenso) de la columna comparte el residuo. Las barras inferiores miden la conservación, $1 - H_c / \log_2 5$, penalizada por los huecos. Destacan el triptófano invariante de la columna 16 y el hueco de cinco posiciones de la cadena α (columnas 54–58), una delección real de la hemoglobina α respecto a la β .

la mioglobina. El MSA resultante, de 155 columnas, tiene una puntuación SP de 2473 (BLOSUM62, huecos afines $-10/-1$ por par).

De Feng-Doolittle a la familia Clustal

El artículo de Feng y Doolittle (1987) tenía un objetivo filogenético: obtener alineamientos coherentes para construir árboles. Un año después, Higgins y Sharp (1988) publicaron CLUSTAL, un paquete diseñado para ejecutarse en los microordenadores personales de la época, que combinaba alineamientos rápidos por pares, un dendrograma y alineamiento progresivo de perfiles. Su sucesor, CLUSTAL W (Thompson et al., 1994), añadió tres ideas que siguen vigentes: ponderar las secuencias para compensar la sobrerrepresentación, penalizar los huecos de forma distinta según la posición (más baratos donde ya hay huecos o en regiones hidrofílicas) y elegir la matriz de sustitución según la divergencia de las secuencias que se alinean en cada nodo.

4.1.5 “Una vez un hueco, siempre un hueco”

El alineamiento progresivo es *codicioso*: cada nodo toma la mejor decisión local y la congela. Feng y Doolittle (1987) lo convirtieron en regla explícita: una vez que un hueco se introduce en un subalineamiento, no se elimina en pasos posteriores. La regla es sensata (los huecos entre secuencias cercanas son los más fiables) pero tiene un costo evidente: un error cometido al alinear las dos primeras secuencias se propaga a todo el alineamiento final, y ninguna información aportada después por las demás secuencias puede corregirlo.

Errores que se heredan

Si en el paso 1 de la figura 4.2 la programación dinámica hubiera colocado un hueco de las mioglobinas un par de posiciones a la izquierda de su lugar correcto (un empate de puntuación basta), ese hueco seguiría allí al final, aunque las tres hemoglobinas “votaran” por otra ubicación. Conviene revisar los huecos y comparar varios programas.

La historia de los programas posteriores es, en buena medida, la historia de los intentos de mitigar este problema sin perder la velocidad del método progresivo (figura 4.4).

Refinamiento iterativo. Una vez obtenido un MSA, se divide el árbol guía quitando una rama, lo que parte las secuencias en dos grupos; se extrae el subalineamiento de cada grupo, se eliminan las columnas que hayan quedado solo con huecos y se vuelven a alinear los dos perfiles. Si la SP mejora, se acepta el cambio. MUSCLE (Edgar, 2004) aplica este refinamiento restringido por el árbol tras dos etapas progresivas: la primera usa distancias rápidas por conteo de k -meros; la segunda recalcula el árbol a partir del primer MSA. Su función de puntuación perfil-perfil, la *log-expectation*, sustituye a la SP media de la ecuación (4.6).

Consistencia. T-Coffee (Notredame et al., 2000) ataca el problema desde otro ángulo: antes de alinear nada, construye una *biblioteca* con todos los alineamientos por pares, globales y locales. El peso de alinear el residuo $x_i^{(1)}$ con $x_j^{(2)}$ aumenta si existe una tercera secuencia $x^{(3)}$ en la que ambos residuos se alinean con un mismo residuo $x_l^{(3)}$ (*extensión* de la biblioteca). Así, cuando el método progresivo alinea dos secuencias, “consulta” implícitamente a todas las demás, y la decisión codiciosa deja de serlo tanto.

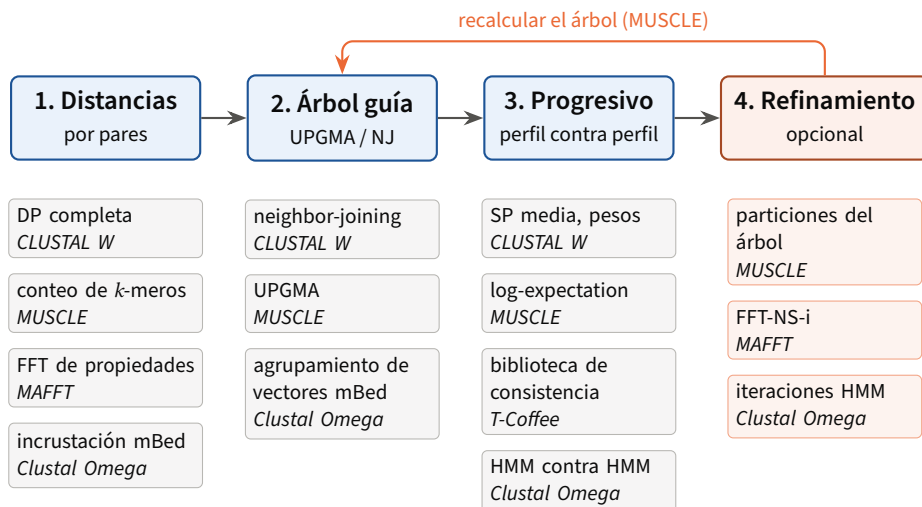


Figura 4.4. Casi todos los programas modernos de MSA comparten el esqueleto progresivo de Feng y Doolittle (1987) y se distinguen en cómo implementan cada etapa. Debajo de cada caja se indican, en cursiva, los programas que popularizaron cada variante. Las etapas 1 y 2 determinan la *velocidad*; las etapas 3 y 4, la *exactitud*. La flecha superior representa la segunda iteración de MUSCLE, que reconstruye el árbol a partir del primer alineamiento.

Velocidad. MAFFT (Katoh et al., 2002) convierte cada proteína en una señal numérica de volumen y polaridad de sus residuos y usa la transformada rápida de Fourier para localizar en segundos las regiones homólogas, que luego se alinean con programación dinámica restringida; ofrece un modo progresivo (FFT-NS-2) y uno con refinamiento iterativo (FFT-NS-i). Clustal Omega (Sievers et al., 2011) escala a conjuntos de decenas de miles de secuencias o más construyendo el árbol guía a partir de distancias a un subconjunto de secuencias de referencia (mBed) y alineando los perfiles como HMM contra HMM.

¿Cuál es el mejor? La pregunta solo tiene respuesta empírica. Los programas se comparan con *benchmarks* de alineamientos de referencia, casi siempre derivados de superposiciones de estructuras tridimensionales, como BALiBASE o PREFAB; la exactitud se mide como la fracción de pares de residuos (o de columnas) de la referencia que el programa reproduce. La revisión de Edgar y Batzoglou (2006) resume las lecciones de esas comparaciones: la consistencia y el refinamiento mejoran la exactitud sobre todo en familias divergentes (por debajo de un 25–30 % de identidad), donde ningún método es fiable, mientras que con secuencias cercanas todos los programas dan resultados muy parecidos.

msa_globinas.py

```

import subprocess
from Bio import AlignIO

# Clustal Omega: FASTA de entrada, formato Clustal de salida
subprocess.run(["clustalo", "-i", "globinas.fasta",
                "-o", "globinas.aln", "--outfmt=clu",
                "--force"], check=True)
aln = AlignIO.read("globinas.aln", "clustal")
print(aln.get_alignment_length(), "columnas")
for fila in aln:
    print(f"{fila.id:12s} {str(fila.seq[:60])}")

filas = [str(f.seq) for f in aln]
print("SP =", suma_de_pares(filas))          # ver ejercicio 4.2
print("pesos =", pesos_henikoff(filas))

```

Las funciones `suma_de_pares` y `pesos_henikoff` implementan la ecuación (4.1) con penalización lineal y la ecuación (4.2). Un ejercicio muy instructivo es comparar los MSA de Clustal Omega, MAFFT (`mafft --auto`) y MUSCLE sobre las mismas globinas y visualizar las columnas en las que discrepan.

4.2 Motivos, matrices de peso posicional y logos



Colab · 4.2 · Motivos, PWM, entropía y logos



abrir notebook

Reconocer una firma

Nadie firma dos veces exactamente igual. Aun así, un empleado de banco experimentado reconoce la firma de un cliente: sabe que la inicial siempre tiene el mismo bucle, que el trazo final a veces sube y a veces no, y que el punto sobre la *i* es opcional. En su cabeza no hay una firma “correcta”, sino una idea de qué rasgos son obligatorios, cuáles son frecuentes y cuáles dan igual. Una proteína que se une al ADN hace algo muy parecido: tolera variación en algunas posiciones de su sitio de unión y es muy exigente en otras. Esta sección enseña a escribir esa “idea de la firma” como una tabla de números.

4.2.1 Motivos y secuencias consenso

Un *motivo* es un patrón corto y recurrente de ADN o proteína asociado a una función. Puede representar el sitio de unión de un factor de transcripción, una señal de corte y empalme, el sitio de unión al ribosoma o un segmento catalítico. Los motivos de ADN que estudiaremos aquí suelen tener entre 6 y 20 posiciones y se representan como *sitios alineados* de igual longitud, sin huecos; los dominios completos de la sección 4.3 requieren modelos más flexibles.

Tomemos un ejemplo clásico. La proteína de unión a la caja TATA (TBP) reconoce, unas 30 bases antes del inicio de la transcripción de muchos genes eucariotas, una secuencia rica en **A** y **T**. La base de datos JASPAR (Castro-Mondragon et al., 2022) guarda su perfil MA0108.2, construido a partir de 389 sitios alineados de 15 bases. La forma más antigua de resumir esos sitios es la *secuencia consenso*, que toma la base mayoritaria de cada columna:

GTATAAAAGCGGGG

El consenso es fácil de leer, pero oculta las frecuencias. Representa igual una columna con un 96 % de **T** (posición 4) y otra donde la **G** alcanza apenas un 39 % (posición 1). Por eso, un sitio con una discrepancia frente al consenso puede ajustarse mejor al motivo que otro con tres. Necesitamos conservar las frecuencias en una representación *cuantitativa*.

4.2.2 De sitios a frecuencias: conteos y pseudoconteos

Sea un conjunto de N sitios alineados de longitud W . La *matriz de conteos* (o de frecuencias de posición, PFM) registra cuántas veces aparece cada base en cada columna:

$$n_{b,i} = \sum_{s=1}^N \mathbb{1}[x_i^{(s)} = b], \quad b \in \{A, C, G, T\}, \quad i = 1, \dots, W. \quad (4.7)$$

DONDE

$x_i^{(s)}$ Base en la posición i del sitio s .

$n_{b,i}$ Número de sitios con la base b en la posición i ; $\sum_b n_{b,i} = N$.

La estimación de máxima verosimilitud de la probabilidad de la base b en la posición i es $n_{b,i}/N$. Con pocas observaciones, esta estimación es peligrosa: si una base no aparece en ninguno de los sitios conocidos, su probabilidad estimada es

exactamente cero, y cualquier sitio futuro que la contenga será declarado *imposible*. La solución estándar es sumar *pseudoconteos*:

$$f_{b,i} = \frac{n_{b,i} + \alpha_b}{N + \sum_{b'} \alpha_{b'}}. \quad (4.8)$$

DONDE

$f_{b,i}$	Probabilidad estimada de la base b en la posición i .
α_b	Pseudoconteo de la base b (p. ej., $\alpha_b = 0,5$, o $\alpha_b = \beta q_b$ proporcional al fondo).

Desde el punto de vista bayesiano, la ecuación (4.8) es la media posterior de una distribución multinomial con una distribución previa de Dirichlet de parámetros α_b : los pseudoconteos son “observaciones imaginarias” que expresan lo que creíamos antes de ver los datos. Cuando N es grande pesan poco; cuando N es pequeño, dominan. Para proteínas, en las que el alfabeto tiene 20 letras y los datos son siempre escasos, conviene usar mezclas de distribuciones de Dirichlet que codifican preferencias fisicoquímicas (Sjölander et al., 1996): si una columna contiene solo I y L, la previa “sabe” que una V es mucho más plausible que una D.

4.2.3 La matriz de peso posicional

La pregunta que queremos responder ante una ventana $y = y_1 \cdots y_W$ de una secuencia es la misma del capítulo 3: ¿es más verosímil bajo el modelo del motivo o bajo el modelo de fondo? Si suponemos que las posiciones son independientes, la razón de verosimilitudes se factoriza y su logaritmo es una suma:

$$W_{b,i} = \log_2 \frac{f_{b,i}}{q_b}, \quad S(y) = \log_2 \frac{\mathbb{P}(y \mid \text{motivo})}{\mathbb{P}(y \mid \text{fondo})} = \sum_{i=1}^W W_{y_i,i}. \quad (4.9)$$

DONDE

$W_{b,i}$	Entrada de la matriz de peso posicional (PWM o PSSM), en bits.
q_b	Frecuencia de fondo de la base b (p. ej., 0,25, o la composición del genoma).
y	Ventana candidata de longitud W .
$S(y)$	Puntuación de la ventana: log-razón de verosimilitudes motivo/fondo.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	-0,7	-2,6	1,9	-4,8	1,9	1,5	1,9	1,2	0,7	-0,8	-0,2	-0,2	-0,2	-0,5	-0,3
C	0,6	-1,1	-7,6	-3,2	-7,6	-7,6	-4,8	-5,3	-1,1	0,5	0,6	0,4	0,3	0,1	0,1
G	0,6	-2,4	-5,3	-5,3	-4,2	-7,6	-2,3	-1,1	0,7	0,6	0,4	0,4	0,4	0,5	0,5
T	-1,6	1,7	-1,5	1,9	-1,7	0,3	-3,9	0,3	-1,5	-1,0	-1,6	-0,9	-0,7	-0,4	-0,5
R_i (bits)	0,23	0,97	1,49	1,69	1,49	1,08	1,51	0,61	0,28	0,16	0,17	0,08	0,06	0,06	0,05

Figura 4.5. Matriz de peso posicional $W_{b,i} = \log_2(f_{b,i}/0,25)$ del perfil de TBP (JASPAR MA0108.2, 389 sitios), con pseudoconteos $\alpha_b = 0,5$. Azul: bases favorecidas; rojo: desfavorecidas. El núcleo **TATAAA** (posiciones 2–7) concentra los pesos extremos; en la posición 3, la **C** vale $-7,6$ bits: verla en un sitio multiplica su verosimilitud por $2^{-7,6} \approx 1/190$. La fila inferior da el contenido de información R_i de cada columna (ecuación (4.11)); las posiciones 10–15 apenas aportan señal.

Esta es exactamente la lógica de las matrices de sustitución (ecuación (3.19)), aplicada ahora a una sola secuencia contra un modelo *posicional*. Una entrada positiva indica que la base es más frecuente en los sitios que en el fondo; una negativa, lo contrario; y la suma de las entradas a lo largo de la ventana decide.

Un perceptrón para los ribosomas

Las matrices de peso nacieron antes que su interpretación probabilística. Stormo et al. (1982) buscaban distinguir los verdaderos sitios de inicio de la traducción en *Escherichia coli* de los codones AUG que no se usan, y entrenaron para ello un *perceptrón*: una tabla de pesos, uno por base y posición, que suma sus valores sobre la ventana y compara el total con un umbral. Los pesos aprendidos resultaron ser un buen clasificador y, a la vez, una descripción legible del sitio. Dos décadas después, Stormo (2000) mostró que, bajo el supuesto de aditividad, la puntuación de una PWM es proporcional a la energía libre de unión de la proteína con el sitio, lo que da a la matriz un significado físico además del estadístico.

Ejemplo 4.3. (La columna 3 del perfil de TBP)

En la posición 3 del perfil MA0108.2 los conteos son $n_A = 352$, $n_C = 0$, $n_G = 2$, $n_T = 35$ ($N = 389$). Con $\alpha_b = 0,5$ el denominador de la ecuación (4.8) es $389 + 2 = 391$ y

$$\begin{aligned} f_{A,3} &= \frac{352,5}{391} = 0,9015, & f_{C,3} &= \frac{0,5}{391} = 0,0013, \\ f_{G,3} &= \frac{2,5}{391} = 0,0064, & f_{T,3} &= \frac{35,5}{391} = 0,0908. \end{aligned}$$

Con fondo uniforme, los pesos son $W_{A,3} = \log_2(0,9015/0,25) = 1,85$, $W_{C,3} = -7,61$, $W_{G,3} = -5,29$ y $W_{T,3} = -1,46$ bits. Sin pseudoconteos, $W_{C,3}$ sería $-\infty$ y un único error de secuenciación en esa posición anularía cualquier sitio. Sumando el máximo de cada columna se obtiene la puntuación del consenso, 16,21 bits; la peor secuencia posible puntúa $-49,13$.

4.2.4 Entropía, información y logos

¿Cuánta “señal” hay en cada columna? Claude Shannon definió la incertidumbre de una variable aleatoria discreta como su *entropía* (Shannon, 1948). Aplicada a la columna i de un motivo,

$$H_i = - \sum_b f_{b,i} \log_2 f_{b,i} \quad (\text{bits}). \quad (4.10)$$

DONDE

H_i Entropía de la columna i : número medio de preguntas de sí o no necesarias para adivinar la base.

Una columna uniforme ($f_{b,i} = 1/4$) tiene la entropía máxima, $\log_2 4 = 2$ bits; una columna invariante, entropía cero. El *contenido de información* de la columna es la reducción de incertidumbre que produce conocer que estamos en un sitio:

$$R_i = \log_2 4 - (H_i + e_N). \quad (4.11)$$

DONDE

R_i Contenido de información de la posición i , entre 0 y 2 bits para ADN.

e_N Corrección por muestra pequeña: con pocos sitios, H_i se subestima sistemáticamente; $e_N \rightarrow 0$ cuando $N \rightarrow \infty$.

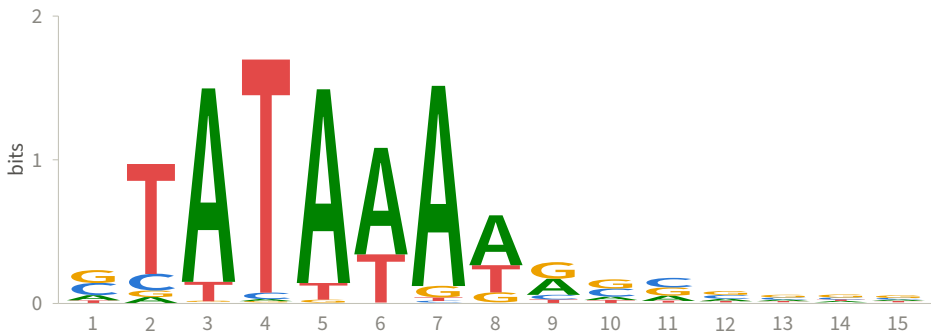


Figura 4.6. *Sequence logo* del perfil de TBP, dibujado en TikZ directamente a partir de las frecuencias de la figura 4.5: la altura de cada letra es $f_{b,i}R_i$ (ecuación (4.12)). Las posiciones 3–7 alternan **A** y **T** con más de 1 bit cada una; la posición 6 muestra que TBP tolera tanto **A** como **T**; la cola **G/C** de las posiciones 9–15 es casi ruido (menos de 0,3 bits). El contenido total del motivo es $\sum_i R_i \approx 9,9$ bits.

Schneider y Stephens (1990) usaron R_i para inventar el *sequence logo*: en cada posición se apilan las cuatro letras, ordenadas de menor a mayor frecuencia, con altura

$$h_{b,i} = f_{b,i} R_i, \quad (4.12)$$

de modo que la altura total de la pila es R_i y la proporción de cada letra dentro de la pila es su frecuencia. De un vistazo, el logo muestra *qué* posiciones importan (pilas altas) y *qué* base se prefiere en ellas (la letra de arriba). WebLogo (Crooks et al., 2004) popularizó la herramienta en la web y es hoy el estándar de facto.

Con un fondo no uniforme (un genoma rico en **A/T**, por ejemplo), la medida adecuada es la *entropía relativa* o divergencia de Kullback-Leibler entre la columna y el fondo:

$$D_i = D_{\text{KL}}(f_{\cdot,i} \| q) = \sum_b f_{b,i} \log_2 \frac{f_{b,i}}{q_b} \geq 0, \quad (4.13)$$

que se reduce a R_i (sin corrección) cuando $q_b = 1/4$. Esta cantidad tiene una interpretación que conecta las dos mitades de la sección.

Teorema 4.2. (*Puntuación esperada de una PWM*)

Si los sitios se generan con las frecuencias $f_{b,i}$ del propio modelo, la puntuación esperada de la ecuación (4.9) es $\mathbb{E}_f[S] = \sum_i D_{\text{KL}}(f_{\cdot,i} \| q)$. Si las ventanas proceden del fondo, $\mathbb{E}_q[S] = -\sum_i D_{\text{KL}}(q \| f_{\cdot,i}) \leq 0$.

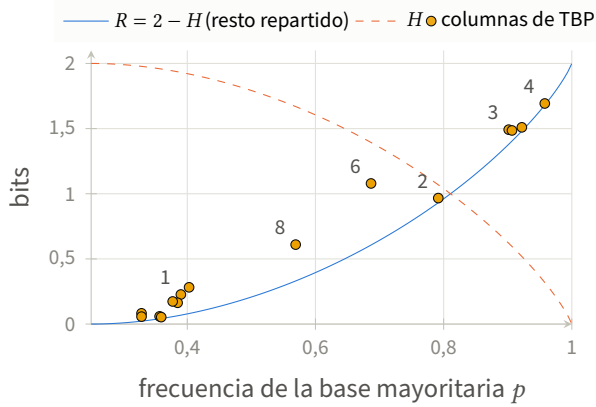


Figura 4.7. Entropía H (naranja) y contenido de información $R = 2 - H$ (azul) de una columna de ADN en la que la base mayoritaria tiene frecuencia p y las otras tres se reparten el resto por igual. Los puntos son las 15 columnas reales de TBP (se numeran algunas; las posiciones 9–15 se agrupan cerca de $p \approx 0,4$ y las 5 y 7 junto a la 3). Caen sobre la curva o por encima de ella, porque en ellas el resto no se reparte por igual (p. ej., la posición 6 mezcla **A** y **T**), y un reparto desigual *incrementa* la información. La relación no es lineal: pasar del 90 % al 99 % de conservación añade más información que pasar del 40 % al 60 %.

La demostración es una línea: por linealidad de la esperanza, $E_f[S] = \sum_i \sum_b f_{b,i} W_{b,i}$, que es la ecuación (4.13) sumada sobre las posiciones. El teorema dice que el contenido de información de un motivo es la *separación media*, en bits, entre sitios y fondo. Para TBP, $\sum_i D_i = 9,93$ bits, y una simulación de 20 000 sitios da una media de 9,95; las ventanas aleatorias promedian $-18,09$ bits, en perfecto acuerdo con $-\sum_i D_{\text{KL}}(q \| f_{\cdot,i}) = -18,09$ (figura 4.8). Un motivo con R bits aparece por azar, aproximadamente, una vez cada 2^R posiciones: el de TBP, una vez cada mil bases.

4.2.5 Buscar sitios en una secuencia

Con la PWM en la mano, buscar sitios es sencillo: se desliza una ventana de longitud W por la secuencia (y por su reverso complementario), se calcula $S(y)$ en cada posición y se reportan las que superan un umbral τ . El coste es $O(nW)$ para una secuencia de longitud n . La dificultad no es computacional, sino estadística: elegir τ .

La figura 4.8 muestra el dilema. Un umbral de 6 bits deja pasar solo un 0,19 % de las ventanas aleatorias, lo que parece poco; pero el genoma humano tiene unos $3,1 \times 10^9$ pares de bases y dos hebras, así que esperaríamos $2 \times 3,1 \times 10^9 \times 0,0019 \approx 1,2 \times 10^7$

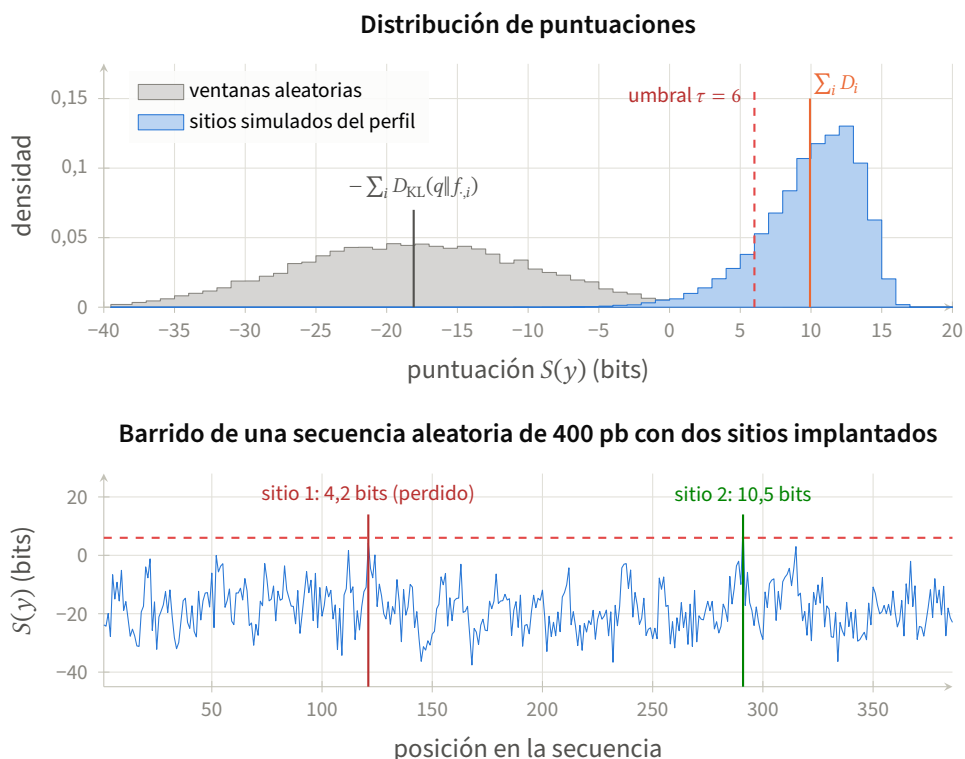


Figura 4.8. Arriba: distribución de $S(y)$ para 20 000 sitios generados a partir de las frecuencias de TBP (azul) y 20 000 ventanas de fondo uniforme (gris). Sus medias coinciden con las predicciones del teorema 4.2. Con $\tau = 6$ bits se recupera el 87 % de los sitios y solo pasa el 0,19 % de las ventanas aleatorias. Abajo: barrido de una secuencia aleatoria en la que se implantaron dos sitios muestreados del perfil. El sitio 2 (GTATAAGTAGCGAGG) destaca claramente; el sitio 1 (CCATAATAACTAGCG), también muestreado del modelo, puntúa solo 4,2 bits y cae bajo el umbral: un falso negativo tan legítimo como inevitable.

“sitios” de TBP, la inmensa mayoría sin ninguna función. Esta abundancia de aciertos espurios no es un defecto de la PWM: refleja que la especificidad de un factor de transcripción *in vivo* depende también de la accesibilidad de la cromatina, de otros factores cercanos y de la posición respecto a los genes, información que la matriz no contiene.

```

motivo_tbp.py

from Bio import motifs
from Bio.Seq import Seq

with open("MA0108.2.jaspar") as fh:           # descargado de JASPAR
    m = motifs.read(fh, "jaspar")
    pwm = m.counts.normalize(pseudocounts=0.5) # f con pseudoconteos
    pssm = pwm.log_odds()                     # W = log2(f / q)
    print(m.name, m.consensus)                # TBP GTATAAAAGGCGGGG
    print(f"máx={pssm.max:.2f} mín={pssm.min:.2f} " #
          f"media en sitios={pssm.mean():.2f}")    # 16.21 -49.13 9.93

prom = Seq("GGCCGCTATAAAAGGCGCGGCATGCGCCGCCATG")
for pos, score in pssm.search(prom, threshold=6.0):
    hebra = "+" if pos >= 0 else "-"           # pos < 0: hebra reversa
    print(pos, hebra, f"{score:.2f}")          # 5 + 15.55

```

El módulo `Bio.motifs` de Biopython (Cock et al., 2009) implementa las ecuaciones (4.8) y (4.9); su método `mean()` devuelve precisamente la puntuación esperada del teorema 4.2. La secuencia `prom` es un fragmento sintético con una caja TATA en la posición 5 (contando desde cero).

⚠ El supuesto de independencia

La ecuación (4.9) supone que cada posición contribuye de forma independiente a la unión. Es una aproximación sorprendentemente buena para muchos factores, pero falla cuando dos bases vecinas interactúan (por ejemplo, por la flexibilidad del ADN) o cuando el factor se une como dímero con espaciadores variables. En esos casos se usan modelos con dependencias entre posiciones, o los modelos de Márkov de la sección siguiente.

Descubrir motivos sin conocerlos

Hasta ahora partimos de sitios ya alineados. El problema inverso, encontrar un motivo desconocido compartido por un conjunto de secuencias (por ejemplo, promotores de genes coexpresados), es mucho más difícil, porque hay que estimar a la vez la PWM y la posición del sitio en cada secuencia. Lawrence et al. (1993) lo resolvieron con un muestreador de Gibbs: se elige al azar una posición en cada secuencia, se construye la PWM con todas menos una, se remuestrea la posición de la excluida en proporción a $2^{S(y)}$ y se repite. El método converge hacia los motivos de alto contenido de información. Las alternativas deterministas se basan en el algoritmo EM (Dempster et al., 1977), el mismo principio que usaremos para entrenar HMM.

4.3 Modelos ocultos de Márkov y perfiles (HMMER)

 Colab · 4.3 · HMM y perfiles con HMMER

 abrir notebook

Un partido por la radio

Imagine que escucha un partido de fútbol por una radio extranjera, en un idioma que no entiende. No ve el campo, pero oye el rugido del público: murmullo, silbidos, gritos. Cuando el equipo local ataca, los gritos son frecuentes; cuando defiende, predominan los silbidos. Además, la posesión no cambia a cada segundo: quien ataca suele seguir atacando un rato. Con esas dos regularidades (qué sonidos produce cada situación y cuánto duran las situaciones) usted puede reconstruir, con bastante acierto, la historia del partido que no vio. Un modelo oculto de Márkov formaliza exactamente ese razonamiento: estados que no se observan, que persisten y cambian con ciertas probabilidades, y que emiten señales que sí se observan.

4.3.1 Estados, transiciones y emisiones

En bioinformática los “sonidos” son nucleótidos o aminoácidos, y los estados ocultos son categorías biológicas: región codificante o intergénica, isla CpG o ADN ordinario, hélice o lámina, posición 17 de un dominio o inserción. Eddy (2004) ofrece una introducción breve y muy clara; el tutorial de Rabiner (1989), escrito para el reconocimiento del habla, sigue siendo la referencia clásica.

Definición 4.2. (Modelo oculto de Márkov)

Un HMM con K estados sobre un alfabeto $\mathcal{A}$ es una terna $\theta = (\pi, \mathbf{A}, \mathbf{E})$ que genera un camino de estados $\pi_1 \pi_2 \cdots \pi_L$ y una secuencia $x_1 \cdots x_L$: el primer estado se elige con probabilidad π_k ; cada estado siguiente depende solo del anterior, con probabilidad a_{kl} ; y cada estado π_i emite el símbolo x_i con probabilidad $e_{\pi_i}(x_i)$. Se observa x ; el camino permanece oculto.

La probabilidad conjunta de una secuencia y un camino es el producto de las decisiones que los generaron:

$$P(x, \pi \mid \theta) = \pi_{\pi_1} e_{\pi_1}(x_1) \prod_{i=2}^L a_{\pi_{i-1}\pi_i} e_{\pi_i}(x_i). \quad (4.14)$$

DONDE

$\pi = \pi_1 \cdots \pi_L$	Camino de estados ocultos (no confundir con π_k , la probabilidad inicial).
π_k	Probabilidad de empezar en el estado k .
a_{kl}	Probabilidad de transición del estado k al l ; $\sum_l a_{kl} = 1$.
$e_k(b)$	Probabilidad de que el estado k emita el símbolo b ; $\sum_b e_k(b) = 1$.
L	Longitud de la secuencia observada.

Usaremos a lo largo de la sección un modelo de juguete con dos estados para ADN: H, regiones ricas en G+C (como las islas CpG), y L, regiones pobres en G+C (figura 4.9). El modelo plantea los tres problemas canónicos que Rabiner (1989) ordenó:

1. **Evaluación:** ¿cuál es la probabilidad $P(x \mid \theta)$ de la secuencia, sumando sobre todos los caminos? (algoritmo *forward*).
2. **Decodificación:** ¿cuál es el camino de estados más probable? (algoritmo de Viterbi).
3. **Aprendizaje:** ¿qué parámetros θ hacen más probables las secuencias observadas? (algoritmo de Baum-Welch).

4.3.2 El algoritmo forward: sumar sobre todos los caminos

Hay K^L caminos posibles; para $K = 2$ y $L = 300$, unos 10^{90} . Por suerte, la estructura de Márkov permite reutilizar cálculos, igual que en la programación

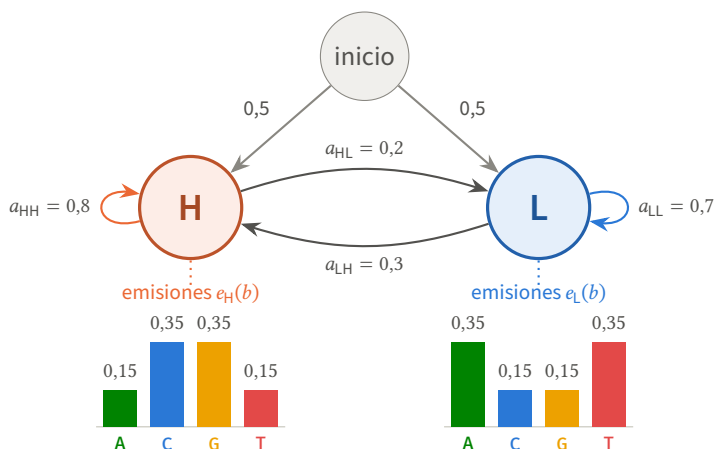


Figura 4.9. El modelo de juguete de esta sección. Dos estados ocultos, H (rico en G+C) y L (pobre), con transiciones que favorecen la permanencia en el mismo estado, y distribuciones de emisión opuestas (barras). Ninguna base individual revela el estado: una G es $0,35/0,15 \approx 2,3$ veces más probable en H que en L, pero puede proceder de ambos. La evidencia sobre el estado se acumula a lo largo de las posiciones consecutivas, porque los estados persisten.

dinámica del capítulo 3, pero *sumando* en lugar de maximizar. Definimos la variable *forward* como la probabilidad de haber emitido los primeros i símbolos y estar en el estado k :

$$f_k(1) = \pi_k e_k(x_1), \quad (4.15)$$

$$f_l(i+1) = e_l(x_{i+1}) \sum_{k=1}^K f_k(i) a_{kl}, \quad i = 1, \dots, L-1, \quad (4.16)$$

$$P(x | \theta) = \sum_{k=1}^K f_k(L). \quad (4.17)$$

DONDE

$f_k(i)$ $P(x_1 \dots x_i, \pi_i = k)$: probabilidad del prefijo terminando en el estado k .

K Número de estados.

El costo es $O(LK^2)$: en cada posición, cada estado suma las contribuciones de todos los estados anteriores. El algoritmo simétrico, *backward*, recorre la secuencia desde el final y calcula la probabilidad del sufijo dado el estado:

$$b_k(L) = 1, \quad b_k(i) = \sum_{l=1}^K a_{kl} e_l(x_{i+1}) b_l(i+1), \quad i = L-1, \dots, 1. \quad (4.18)$$

DONDE

$b_k(i)$ $P(x_{i+1} \dots x_L \mid \pi_i = k)$: probabilidad del sufijo dado que en i estamos en k .

Combinando ambos se obtiene la *probabilidad posterior* de cada estado en cada posición, la cantidad más útil en la práctica:

$$P(\pi_i = k \mid x, \theta) = \frac{f_k(i) b_k(i)}{P(x \mid \theta)}. \quad (4.19)$$

Ejemplo 4.4. (Forward a mano)

Con el modelo de la figura 4.9 y $x = \text{GCGATTAT}$:

$$f_H(1) = 0,5 \times 0,35 = 0,175, \quad f_L(1) = 0,5 \times 0,15 = 0,075.$$

Para la segunda base (C), según la ecuación (4.16),

$$f_H(2) = 0,35 (0,175 \times 0,8 + 0,075 \times 0,3) = 0,35 \times 0,1625 = 0,0569,$$

$$f_L(2) = 0,15 (0,175 \times 0,2 + 0,075 \times 0,7) = 0,15 \times 0,0875 = 0,0131.$$

Continuando hasta $i = 9$ y sumando (ecuación (4.17)) se obtiene $P(x \mid \theta) = 5,10 \times 10^{-6}$, es decir, $\log_2 P(x) = -17,58$ bits. Las probabilidades posteriores del estado H (ecuación (4.19)) son 0,80, 0,87, 0,86, 0,75 en las cuatro primeras posiciones (GCGC) y caen a 0,36, 0,22, 0,18, 0,18, 0,24 en el tramo ATTAT.

⚠ Desbordamiento por abajo

Las probabilidades forward decrecen exponencialmente con L . Para una secuencia de 5000 bases como la que usaremos para entrenar el modelo, $P(x) \approx 2^{-9712}$, un número que ningún formato de coma flotante puede representar (el menor double positivo es del orden de 2^{-1074}). Hay dos solu-

ciones: trabajar en escala logarítmica, sustituyendo las sumas por $\log \sum \exp$ (p. ej., `numpy.logaddexp`), o *reescalar* $f_k(i)$ en cada posición dividiendo por $c_i = \sum_k f_k(i)$ y acumulando $\log P(x) = \sum_i \log c_i$. HMMER usa una variante de la segunda, el *sparse rescaling* (Eddy, 2011).

4.3.3 El algoritmo de Viterbi: el mejor camino

Si en lugar de sumar sobre caminos queremos el *más probable*, basta con cambiar la suma de la ecuación (4.16) por un máximo y recordar de dónde vino cada máximo. El algoritmo fue propuesto por Viterbi (1967) para decodificar códigos convolucionales en telecomunicaciones.

Teorema 4.3. (Algoritmo de Viterbi)

Sea $v_k(i)$ la probabilidad del camino más probable que emite $x_1 \cdots x_i$ y termina en el estado k . Entonces

$$v_k(1) = \pi_k e_k(x_1), \quad (4.20)$$

$$v_l(i+1) = e_l(x_{i+1}) \max_k [v_k(i) a_{kl}], \quad (4.21)$$

y $P(x, \pi^*) = \max_k v_k(L)$, donde π^* es el camino óptimo. El costo es $O(LK^2)$ en tiempo y $O(LK)$ en memoria.

DONDE

$v_k(i)$	Probabilidad del mejor camino parcial que termina en k en la posición i .
π^*	Camino de Viterbi: $\arg \max_{\pi} P(x, \pi)$.
$\text{ptr}_i(l)$	Puntero: el estado k que maximizó la ecuación (4.21) para llegar a l en i .

El camino se recupera, como en Needleman-Wunsch, con un *traceback*: $\pi_L^* = \arg \max_k v_k(L)$ y, hacia atrás, $\pi_{i-1}^* = \text{ptr}_i(\pi_i^*)$. En la práctica se trabaja con logaritmos, de modo que los productos se vuelven sumas y la ecuación (4.21) adopta la forma exacta de una recurrencia de programación dinámica: $V_l(i+1) = \log e_l(x_{i+1}) + \max_k [V_k(i) + \log a_{kl}]$.

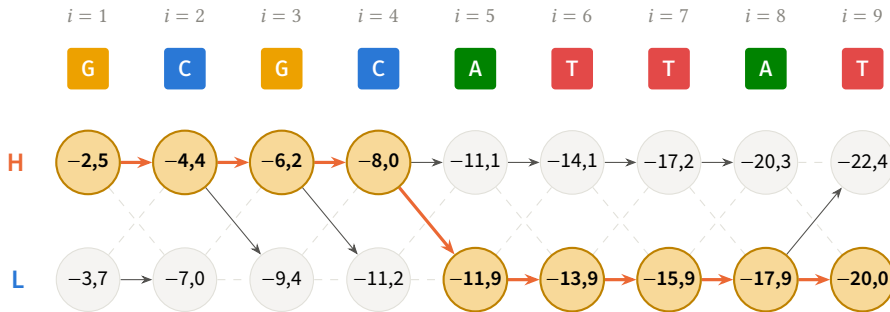


Figura 4.10. Enrejado (*trellis*) de Viterbi para $x = \text{GCGCATTAT}$ con el modelo de la figura 4.9. Cada nodo muestra $\log_2 v_k(i)$; las flechas grises indican el predecesor elegido por el máximo de la ecuación (4.21) y las discontinuas, las alternativas descartadas. El camino óptimo (naranja, nodos amarillos) es HHHHHLLLL: la región GCGC se asigna a H y el tramo ATTAT a L. Obsérvese que en $i = 5$ el mejor predecesor de L ya es H: el cambio de estado “se paga” una sola vez, con $\log_2 0,2$.

Ejemplo 4.5. (Viterbi frente a posterior)

En la figura 4.10, $\log_2 v_L(9) = -19,98$ supera a $\log_2 v_H(9) = -22,42$, así que el camino termina en L. Su probabilidad conjunta es $2^{-19,98}$, y como $P(x) = 2^{-17,58}$, la probabilidad posterior del camino completo es

$$P(\pi^* | x) = \frac{2^{-19,98}}{2^{-17,58}} = 2^{-2,40} \approx 0,19.$$

El camino más probable acapara menos de una quinta parte de la probabilidad: el 81 % restante se reparte entre los otros 511 caminos. Por eso, cuando interesa la etiqueta de cada posición más que un camino global coherente, se prefiere la *decodificación posterior*, que asigna a cada i el estado con mayor ecuación (4.19).

forward_viterbi.py

```
import numpy as np
```

```
def forward_viterbi(x, A, E, pi, alfabeto="ACGT"):
    """Log-verosimilitud (forward) y camino de Viterbi, en log2."""
    o = [alfabeto.index(c) for c in x]
    lA, lE = np.log2(A), np.log2(E)
```

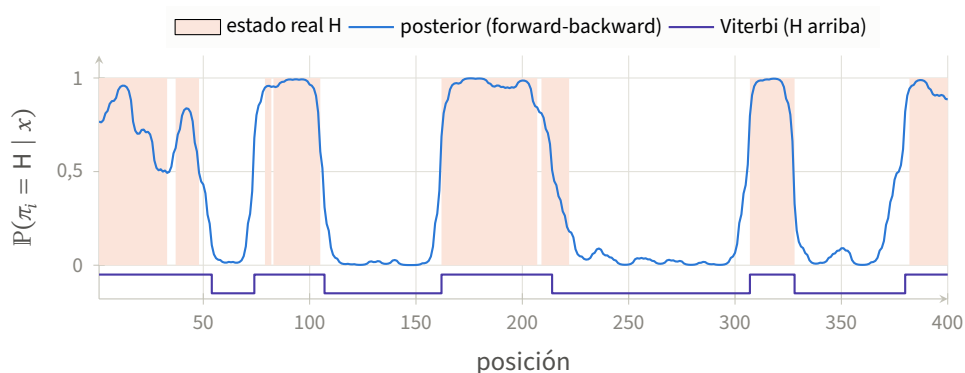


Figura 4.11. Decodificación de 400 bases simuladas con el modelo de la figura 4.9, pero con transiciones más persistentes ($a_{HH} = a_{LL} = 0,98$). Fondo naranja: tramos generados realmente por H. Curva azul: probabilidad posterior de H; su umbral en 0,5 acierta el estado en el 94 % de las posiciones. Escalón violeta (abajo): camino de Viterbi, con un 92,5 % de acierto. Ambos métodos detectan los bloques largos; los tramos cortos y los bordes son inciertos, y solo la posterior lo *dice*.

```
f = np.log2(pi) + lE[:, o[0]] # inicio forward (log)
v, ptr = f.copy(), []
for s in o[1:]:
    f = lE[:, s] + np.logaddexp2.reduce(f[:, None] + lA, axis=0)
    cand = v[:, None] + lA # K x K
    ptr.append(cand.argmax(axis=0))
    v = lE[:, s] + cand.max(axis=0)
camino = [int(v.argmax())]
for p in reversed(ptr):
    camino.append(int(p[camino[-1]])) # traceback
return np.logaddexp2.reduce(f), v.max(), camino[::-1]

A = np.array([[0.8, 0.2], [0.3, 0.7]])
E = np.array([[.15, .35, .35, .15], [.35, .15, .15, .35]])
lp, lv, c = forward_viterbi("GCGCATTAT", A, E, np.array([.5, .5]))
print(f"{lp:.2f} {lv:.2f}", ".join("HL"[k] for k in c))
# -17.58 -19.98 HHHHLLLLL
```

4.3.4 Baum-Welch: aprender los parámetros

Hasta ahora supusimos conocidos A y E. Si dispusiéramos de secuencias con los estados anotados, estimarlos sería contar: la fracción de veces que H va seguido de L, la fracción de G emitidas por H, etc. Pero, por definición, los estados están ocultos.

El algoritmo de Baum et al. (1970) resuelve el problema sustituyendo los conteos que no podemos observar por sus *valores esperados* bajo los parámetros actuales, y repitiendo.

Para cada posición i definimos la probabilidad posterior de una transición concreta y de un estado concreto:

$$\xi_i(k, l) = \mathbb{P}(\pi_i = k, \pi_{i+1} = l \mid x, \theta) = \frac{f_k(i) a_{kl} e_l(x_{i+1}) b_l(i+1)}{\mathbb{P}(x \mid \theta)}, \quad (4.22)$$

$$\gamma_i(k) = \mathbb{P}(\pi_i = k \mid x, \theta) = \frac{f_k(i) b_k(i)}{\mathbb{P}(x \mid \theta)}. \quad (4.23)$$

DONDE

$\xi_i(k, l)$	Probabilidad posterior de que la transición $k \rightarrow l$ ocurra entre las posiciones i e $i + 1$.
$\gamma_i(k)$	Probabilidad posterior de estar en k en la posición i (ecuación (4.19)).

Los conteos esperados se obtienen sumando sobre las posiciones (y sobre las secuencias de entrenamiento, si hay varias), y los nuevos parámetros son las frecuencias relativas de esos conteos:

$$\hat{a}_{kl} = \frac{\sum_{i=1}^{L-1} \xi_i(k, l)}{\sum_{l'} \sum_{i=1}^{L-1} \xi_i(k, l')}, \quad \hat{e}_k(b) = \frac{\sum_{i: x_i=b} \gamma_i(k)}{\sum_{i=1}^L \gamma_i(k)}, \quad \hat{\pi}_k = \gamma_1(k). \quad (4.24)$$

DONDE

$\hat{a}_{kl}, \hat{e}_k(b), \hat{\pi}_k$	Parámetros reestimados para la siguiente iteración.
$\sum_{i: x_i=b}$	Suma solo sobre las posiciones en las que se observó el símbolo b .

Se calculan forward y backward con los parámetros actuales (paso E), se reestiman los parámetros con la ecuación (4.24) (paso M) y se itera hasta que la verosimilitud deja de aumentar. Baum-Welch es un caso particular del algoritmo EM, formulado en general años después por Dempster et al. (1977), y hereda su propiedad fundamental.

Teorema 4.4. (Monotonía de Baum-Welch)

Cada iteración de la ecuación (4.24) cumple $\mathbb{P}(x \mid \hat{\theta}) \geq \mathbb{P}(x \mid \theta)$, con igualdad solo en un punto estacionario de la verosimilitud (Baum et al., 1970).

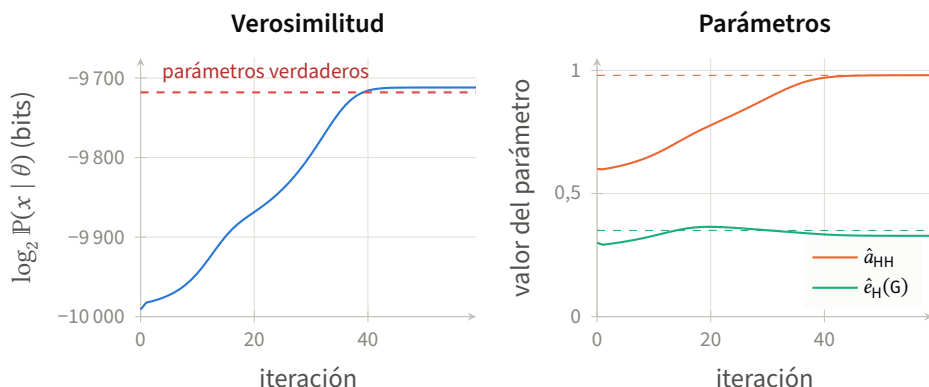


Figura 4.12. Baum-Welch aplicado a 5000 bases simuladas con $a_{HH} = a_{LL} = 0,98$, partiendo de parámetros deliberadamente erróneos ($a_{HH} = 0,6$; emisiones casi uniformes). **Izquierda:** la log-verosimilitud aumenta en cada iteración, como garantiza el teorema 4.4, desde -9991 hasta -9712 bits; tras 47 iteraciones queda a menos de $0,1$ bits de su valor final. Termina *por encima* de la de los parámetros verdaderos (línea roja, -9718): el modelo se ajusta a las fluctuaciones de la muestra. **Derecha:** los parámetros convergen a $\hat{a}_{HH} = 0,981$ y $\hat{e}_H(G) = 0,328$, cerca de los valores verdaderos (discontinuas).

El teorema garantiza la convergencia a un máximo *local*, no al global. La verosimilitud de un HMM tiene en general muchos máximos locales, así que en la práctica se ejecuta el algoritmo desde varios puntos de partida, o se inicializa con una estimación razonable (por ejemplo, a partir de un MSA, como veremos con los perfiles).

4.3.5 HMM de pares: el alineamiento como modelo probabilístico

Estamos ahora en condiciones de cerrar un cabo que dejamos suelto en el capítulo 3. Las tres matrices de Gotoh (ecuación (3.14)) corresponden a tres estados de un HMM que, en lugar de emitir una secuencia, emite *dos* a la vez: un estado M emite un par alineado (x_i, y_j) con probabilidad $p_{x_i y_j}$; un estado X emite x_i frente a un hueco; un estado Y emite y_j frente a un hueco. La transición $M \rightarrow X$ tiene probabilidad δ (abrir un hueco) y $X \rightarrow X$, probabilidad ε (extenderlo). Durbin et al. (1998) muestran que el algoritmo de Viterbi de este HMM de pares, escrito en log-razones frente a un modelo de fondo, es el algoritmo de Gotoh, con

$$s(a, b) = \log \frac{p_{ab}}{q_a q_b} + \text{cte.}, \quad d_o \approx -\log \delta, \quad d_e \approx -\log \varepsilon.$$

4.3.6 HMM de perfil: un modelo para cada familia

Un perfil (ecuación (4.6)) asigna una distribución propia a cada columna de un MSA; una PWM hace lo mismo para motivos sin huecos. Lo que les falta a ambos es una forma principiaada de modelar inserciones y deleciones que dependan de la posición: en una familia de proteínas, los bucles superficiales aceptan inserciones con facilidad, mientras que el núcleo hidrofóbico casi nunca las tolera. Krogh et al. (1994) resolvieron el problema con una arquitectura de HMM que hoy llamamos *HMM de perfil*, y Eddy (1998) la convirtió en una herramienta de uso general.

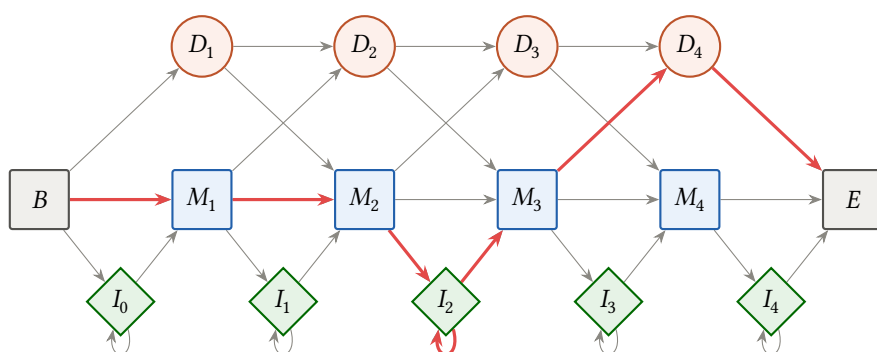
Definición 4.3. (HMM de perfil)

Un HMM de perfil de longitud m tiene, para cada posición consenso $j = 1, \dots, m$, tres estados: un estado de *coincidencia* M_j , que emite un residuo con distribución $e_{M_j}(\cdot)$ específica de la posición; un estado de *inserción* I_j , que emite residuos extra entre las posiciones j y $j + 1$ con una distribución cercana al fondo; y un estado *silencioso* de *delección* D_j , que salta la posición j sin emitir. Estados inicial (B) y final (E) completan el modelo.

La idea clave es que las penalizaciones de hueco, que en Needleman-Wunsch eran constantes, se convierten en probabilidades de transición *distintas en cada posición*: $a_{M_j D_{j+1}}$ puede ser minúscula en una hélice del núcleo y grande en un bucle. Y como el modelo es un HMM, todo lo que sabemos (forward, Viterbi, Baum-Welch) se aplica directamente.

Construcción a partir de un MSA. Dado un MSA de la familia, se decide primero qué columnas son *consenso*: una regla sencilla es declarar de coincidencia toda columna en la que al menos la mitad de las secuencias tienen un residuo, y de inserción las demás. Después, cada secuencia del MSA define un camino por el modelo y basta con contar transiciones y emisiones, añadir pseudoconteos (o una previa de mezcla de Dirichlet, Sjölander et al., 1996) y normalizar:

$$e_{M_j}(a) = \frac{c_j(a) + \alpha_a}{\sum_{a'} (c_j(a') + \alpha_{a'})}, \quad a_{kl} = \frac{C_{kl} + \beta}{\sum_{l'} (C_{kl'} + \beta)}. \quad (4.25)$$



■ coincidencia (emite con e_{M_j}) ◆ inserción (emite $\approx$ fondo) ● delección (silencioso)

camino de CAACT-: $B M_1 M_2 I_2 M_3 D_4 E$

Figura 4.13. Arquitectura de un HMM de perfil de longitud 4, en la variante *Plan7* de HMMER, que no permite transiciones directas entre estados de inserción y delección. Los estados M_j (cuadrados) y D_j (círculos) forman la columna vertebral; los I_j (rombos) cuelgan entre posiciones y tienen un bucle que permite inserciones de cualquier longitud. En rojo, el camino que recorre la secuencia CAACT del ejemplo 4.6: dos residuos insertados tras la posición 2 y una delección de la posición 4. Toda secuencia alineada al MSA corresponde a un camino, y viceversa.

DONDE

$c_j(a)$	Número ponderado de secuencias con el residuo a en la columna consenso j .
C_{kl}	Número ponderado de veces que los caminos del MSA usan la transición $k \rightarrow l$.
α_a, β	Pseudoconteos de emisión y de transición.

Los conteos se ponderan (por ejemplo, con los pesos de la ecuación (4.2)) para que las subfamilias sobrerrepresentadas no dominen el modelo.

Ejemplo 4.6. (Un HMM de perfil de juguete)

Consideremos el MSA de cinco secuencias de ADN y seis columnas:

columna	1	2	3	4	5	6
s1	C	A	-	-	T	G
s2	C	A	A	-	T	G
s3	C	G	-	-	T	G
s4	C	A	A	C	T	-
s5	T	A	-	-	T	G
estado	M_1	M_2	I_2	I_2	M_3	M_4

Las columnas 3 y 4 tienen residuos en solo 2 y 1 de las 5 secuencias, así que son de inserción; las demás definen $M_1, \dots, M_4$. Con un pseudoconteo de 1 por base, las emisiones de M_1 (conteos C:4, T:1) son $e_{M_1} = (\frac{1}{9}, \frac{5}{9}, \frac{1}{9}, \frac{2}{9}) = (0,111; 0,556; 0,111; 0,222)$ para (A,C,G,T); la columna 5, invariante, da $e_{M_3}(T) = \frac{5+1}{5+4} = 0,667$; y M_4 , con solo cuatro residuos, $e_{M_4}(G) = \frac{4+1}{4+4} = 0,625$. Los caminos son $B M_1 M_2 M_3 M_4 E$ para s1, s3 y s5; $B M_1 M_2 I_2 M_3 M_4 E$ para s2; y $B M_1 M_2 I_2 I_2 M_3 D_4 E$ para s4 (figura 4.13). Desde M_2 hay tres transiciones a M_3 y dos a I_2 ; con $\beta = 1$ sobre las tres salidas posibles (M_3, I_2, D_3), $a_{M_2 M_3} = \frac{4}{8} = 0,5$, $a_{M_2 I_2} = \frac{3}{8} = 0,375$ y $a_{M_2 D_3} = \frac{1}{8} = 0,125$.

Alinear una secuencia al perfil. Para puntuar una secuencia x contra el modelo se usa Viterbi (o forward) en log-razones frente al fondo q . Las recurrencias tienen la misma forma que las de Gotoh, pero con parámetros que dependen de j :

$$V_j^M(i) = \log \frac{e_{M_j}(x_i)}{q_{x_i}} + \max \begin{cases} V_{j-1}^M(i-1) + \log a_{M_{j-1}M_j} \\ V_{j-1}^I(i-1) + \log a_{I_{j-1}M_j} \\ V_{j-1}^D(i-1) + \log a_{D_{j-1}M_j} \end{cases} \quad (4.26)$$

$$V_j^I(i) = \log \frac{e_{I_j}(x_i)}{q_{x_i}} + \max \begin{cases} V_j^M(i-1) + \log a_{M_jI_j} \\ V_j^I(i-1) + \log a_{I_jI_j} \end{cases} \quad (4.27)$$

$$V_j^D(i) = \max \begin{cases} V_{j-1}^M(i) + \log a_{M_{j-1}D_j} \\ V_{j-1}^D(i) + \log a_{D_{j-1}D_j} \end{cases} \quad (4.28)$$

DONDE

$V_j^M(i)$	Mejor log-razón de un camino que ha emitido $x_1 \dots x_i$ y termina en M_j (ídem para I_j, D_j).
q_{x_i}	Frecuencia de fondo del residuo x_i ; normalmente $e_{I_j} \approx q$, por lo que el primer término de la ecuación (4.27) es casi cero.
i, j	Posición en la secuencia y posición en el modelo, respectivamente.

Obsérvese que D_j no consume residuos (su índice i no cambia) y que, siguiendo la arquitectura Plan7, no hay transiciones $I \leftrightarrow D$. El costo es $O(mL)$, el mismo que alinear dos secuencias de longitudes m y L . Sustituyendo cada $\max$ por una suma (en espacio de probabilidades) se obtiene el algoritmo forward del perfil, que da la probabilidad de la secuencia bajo el modelo sumando sobre *todos* sus alineamientos posibles.

4.3.7 HMMER y Pfam

HMMER implementa este programa completo. Sus versiones iniciales puntuaban con Viterbi y eran unas cien veces más lentas que BLAST, lo que limitaba su uso. HMMER3 (Eddy, 2011) cambió la situación con dos ideas: puntuar con el algoritmo forward completo, que es más sensible porque integra la evidencia de todos los alineamientos, y colocar delante una cascada de filtros heurísticos que descartan pronto la inmensa mayoría de las secuencias no homólogas (figura 4.14). El primero, el filtro MSV (*multiple segment Viterbi*), calcula la mejor suma de varios segmentos sin huecos con instrucciones vectoriales SIMD, la misma técnica que acelera Smith-

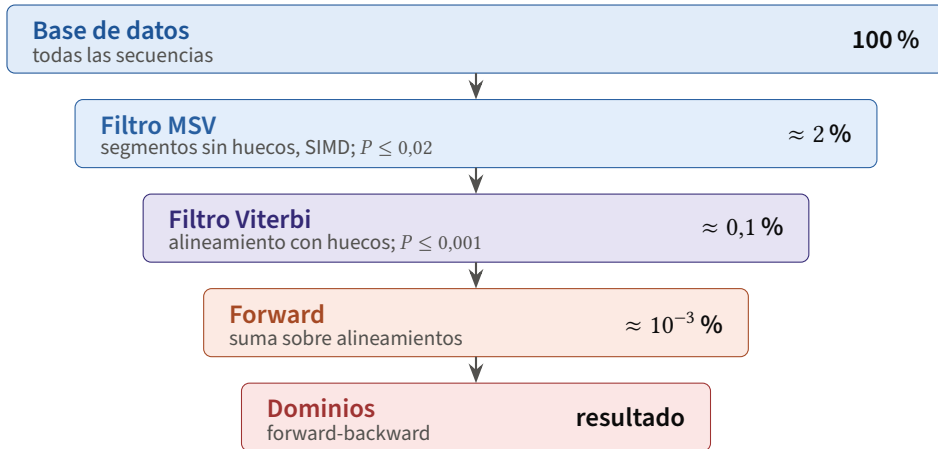


Figura 4.14. La cascada de aceleración de HMMER3 (Eddy, 2011). Cada etapa es más lenta y más sensible que la anterior, pero solo procesa las secuencias que la anterior dejó pasar. Los umbrales se expresan como valores P : el filtro MSV deja pasar, por defecto, el 2 % de las secuencias no homólogas, y el de Viterbi, el 0,1 %. La puntuación final, sobre la que se calculan los valores E , es siempre la del algoritmo forward completo; los filtros solo deciden a quién merece la pena aplicarlo.

Waterman. Según el propio artículo, HMMER3 es entre 100 y 1000 veces más rápido que HMMER2 y aproximadamente tan rápido como BLAST para proteínas.

HMMER expresa sus resultados igual que BLAST en el capítulo 3: una puntuación en bits (log-razón en base 2 entre el modelo y el fondo) y un valor E . Para asignar valores E sin simulaciones costosas, Eddy (2011) conjeturó, y comprobó empíricamente con miles de modelos, que las puntuaciones Viterbi en bits de secuencias no homólogas siguen una distribución de Gumbel de pendiente fija $\lambda = \ln 2$, y que la cola alta de las puntuaciones forward sigue una exponencial de la misma pendiente. Así, solo hay que estimar un parámetro de posición por modelo, lo que se hace una vez al construirlo.

Construir un HMM de perfil y buscar homólogos

```
# 1. HMM a partir de un MSA (Stockholm o FASTA alineado)
hmmbuild globinas.hmm globinas.sto
# 2. Buscar el perfil en una base de datos de proteínas
hmmsearch --tblout aciertos.tbl -E 1e-5 globinas.hmm uniprot_sprot.fasta
# 3. Anotar dominios de una proteína contra todo Pfam
```

```
hmmcompress Pfam-A.hmm # indexa la base (una vez)
hmmsearch --domtblout dominios.tbl Pfam-A.hmm consulta.fasta
```

```
leer_hmmer.py
```

```
from Bio import SearchIO

for q in SearchIO.parse("dominios.tbl", "hmmsearch3-domtbl"):
    for hit in q:
        for hsp in hit:
            print(f"{hit.id:15s} {hsp.bitscore:6.1f} bits "
                  f"E={hsp.evalue:.1e} "
                  f"{hsp.query_start + 1}-{hsp.query_end}")
```

La aplicación más influyente de los HMM de perfil es la anotación de dominios. Pfam (Mistry et al., 2021) es una colección de familias de proteínas, cada una definida por un *alineamiento semilla* curado a mano, un HMM de perfil construido con HMMER a partir de él y un *umbral de recolección* (*gathering threshold*) en bits, elegido por los curadores para que no incluya falsos positivos conocidos. El alineamiento completo de la familia se obtiene buscando el HMM en UniProt. La versión de 2021 añadió más de 350 familias nuevas, revisó las que cubren el proteoma del SARS-CoV-2 y reintrodujo Pfam-B, un suplemento de 136 730 agrupaciones automáticas de secuencias aún no cubiertas por ninguna familia.

De los ribosomas a las familias de proteínas

Los HMM llegaron a la biología desde el reconocimiento del habla, donde el tutorial de Rabiner (1989) los había popularizado. El grupo de David Haussler, en Santa Cruz, los adaptó a las proteínas: Krogh et al. (1994) modelaron con la arquitectura de la figura 4.13 las globinas, las cinasas y las proteínas EF-hand, y mostraron que un modelo entrenado solo con secuencias no alineadas podía producir un MSA y detectar miembros lejanos de la familia. El libro de Durbin et al. (1998), escrito por cuatro protagonistas de esta historia, sigue siendo la mejor introducción a todo el tema.

PWM: una distribución por posición, sin huecos.
HMM de perfil: una distribución por posición, con huecos que también dependen de la posición.

En resumen

- Un MSA se puntúa típicamente con la suma de pares, idealmente ponderada para compensar secuencias redundantes. Su optimización exacta cuesta $O(k^2 2^k n^k)$ y es NP-difícil.
- El alineamiento progresivo (distancias, árbol guía, alineamiento de perfiles) es rápido pero codicioso: “una vez un hueco, siempre un hueco”. El refinamiento iterativo (MUSCLE), la consistencia (T-Coffee), la FFT (MAFFT) y los árboles mBed con alineamiento HMM–HMM (Clustal Omega) mitigan sus errores o lo escalan.
- Una PWM resume sitios alineados como log-razones $\log_2(f_{b,i}/q_b)$; los pseudo-conteos evitan probabilidades nulas. Su puntuación esperada en sitios verdaderos es la entropía relativa total del motivo.
- El contenido de información $R_i = 2 - H_i$ mide la conservación de cada columna en bits; el *sequence logo* lo muestra apilando letras de altura $f_{b,i}R_i$.
- Un HMM se evalúa con forward, se decodifica con Viterbi o por probabilidades posteriores y se entrena con Baum-Welch (EM), todos en $O(LK^2)$.
- Un HMM de perfil añade a cada posición estados de coincidencia, inserción y delección, con parámetros propios. HMMER3 lo busca en bases de datos con filtros acelerados y puntuación forward; Pfam lo usa para definir y anotar familias de dominios.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 4.1** ●●● Calcule a mano la puntuación SP (BLOSUM62, $d = 8$) de la columna K, K, R, -, E. ¿Cuánto cambia si se asigna peso 0,5 a cada una de las dos primeras secuencias y peso 1 a las demás?
- 4.2** ●●● Escriba la función `suma_de_pares(filas, d)` que implementa la ecuación (4.1) con penalización lineal y compruebe que las cuatro primeras columnas del ejemplo 4.1 suman 62.
- 4.3** ●●● Una columna de un motivo tiene frecuencias (0,5; 0,5; 0; 0) y otra (0,7; 0,1; 0,1; 0,1). ¿Cuál tiene más contenido de información? Dibuje a mano sus pilas en un *logo*.

- 4.4 ●●● Con la matriz de distancias de la figura 4.2, construya el árbol por *neighbor-joining* en lugar de UPGMA. ¿Cambia el orden en que se alinean los perfiles? Alinee las globinas con Clustal Omega y MAFFT y cuente las columnas en que difieren.
- 4.5 ●●● Descargue de JASPAR la matriz de conteos de otro factor (por ejemplo, CTCF, MA0139) y calcule su contenido total de información. ¿Cada cuántas bases esperaría un sitio por azar? Compare con el número de sitios de CTCF que se conocen en el genoma humano.
- 4.6 ●●● Verifique numéricamente, con el código de `forward_viterbi.py`, que $\sum_k f_k(i) b_k(i) = P(x)$ para *todas* las posiciones i . ¿Por qué debe cumplirse esta identidad?
- 4.7 ●●● Implemente Baum-Welch con reescalado para el modelo de dos estados y reproduzca la figura 4.12. Repita desde 20 puntos de partida aleatorios: ¿llegan todos al mismo máximo? ¿Qué ocurre si inicializa con $e_H = e_L$ exactamente?
- 4.8 ●●● Construya con `hmmbuild` un HMM de perfil a partir del MSA de globinas y búsquelo con `hmmsearch` en Swiss-Prot. Compare el número de aciertos con $E < 10^{-5}$ con el que obtiene `blastp` usando como consulta solo la hemoglobina β humana. Explique la diferencia en términos de la ecuación (4.26).

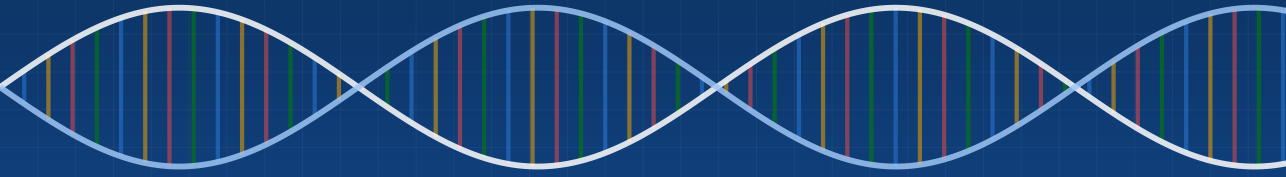
Referencias del capítulo

- Baum, L. E., Petrie, T., Soules, G., & Weiss, N. (1970). A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. *The Annals of Mathematical Statistics*, 41(1), 164-171. <https://doi.org/10.1214/aoms/1177697196>
- Carrillo, H., & Lipman, D. (1988). The multiple sequence alignment problem in biology. *SIAM Journal on Applied Mathematics*, 48(5), 1073-1082. <https://doi.org/10.1137/0148063>
- Castro-Mondragon, J. A., Riudavets-Puig, R., Rauluseviciute, I., Berhanu Lemma, R., Turchi, L., Blanc-Mathieu, R., Lucas, J., Boddie, P., Khan, A., Manosalva Pérez, N., Fornes, O., Leung, T. Y., Aguirre, A., Hammal, F., Schmelter, D., Baranasic, D., Ballester, B., Sandelin, A., Lenhard, B., ... Mathelier, A. (2022). JASPAR 2022: The 9th release of the open-access database of transcription factor binding profiles. *Nucleic Acids Research*, 50(D1), D165-D173. <https://doi.org/10.1093/nar/gkab1113>

- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Crooks, G. E., Hon, G., Chandonia, J.-M., & Brenner, S. E. (2004). WebLogo: A sequence logo generator. *Genome Research*, 14(6), 1188-1190. <https://doi.org/10.1101/gr.849004>
- Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1), 1-22. <https://doi.org/10.1111/j.2517-6161.1977.tb01600.x>
- Durbin, R., Eddy, S. R., Krogh, A., & Mitchison, G. (1998). *Biological sequence analysis: Probabilistic models of proteins and nucleic acids*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511790492>
- Eddy, S. R. (1998). Profile hidden Markov models. *Bioinformatics*, 14(9), 755-763. <https://doi.org/10.1093/bioinformatics/14.9.755>
- Eddy, S. R. (2004). What is a hidden Markov model? *Nature Biotechnology*, 22(10), 1315-1316. <https://doi.org/10.1038/nbt1004-1315>
- Eddy, S. R. (2011). Accelerated profile HMM searches. *PLoS Computational Biology*, 7(10), Artículo e1002195. <https://doi.org/10.1371/journal.pcbi.1002195>
- Edgar, R. C. (2004). MUSCLE: Multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Research*, 32(5), 1792-1797. <https://doi.org/10.1093/nar/gkh340>
- Edgar, R. C., & Batzoglou, S. (2006). Multiple sequence alignment. *Current Opinion in Structural Biology*, 16(3), 368-373. <https://doi.org/10.1016/j.sbi.2006.04.004>
- Feng, D.-F., & Doolittle, R. F. (1987). Progressive sequence alignment as a prerequisite to correct phylogenetic trees. *Journal of Molecular Evolution*, 25(4), 351-360. <https://doi.org/10.1007/BF02603120>
- Henikoff, S., & Henikoff, J. G. (1994). Position-based sequence weights. *Journal of Molecular Biology*, 243(4), 574-578. [https://doi.org/10.1016/0022-2836\(94\)90032-9](https://doi.org/10.1016/0022-2836(94)90032-9)
- Higgins, D. G., & Sharp, P. M. (1988). CLUSTAL: A package for performing multiple sequence alignment on a microcomputer. *Gene*, 73(1), 237-244. [https://doi.org/10.1016/0378-1119\(88\)90330-7](https://doi.org/10.1016/0378-1119(88)90330-7)
- Katoh, K., Misawa, K., Kuma, K.-i., & Miyata, T. (2002). MAFFT: A novel method for rapid multiple sequence alignment based on fast Fourier transform. *Nucleic Acids Research*, 30(14), 3059-3066. <https://doi.org/10.1093/nar/gkf436>
- Krogh, A., Brown, M., Mian, I. S., Sjölander, K., & Haussler, D. (1994). Hidden Markov models in computational biology: Applications to protein modeling. *Journal of Molecular Biology*, 235(5), 1501-1531. <https://doi.org/10.1006/jmbi.1994.1104>

- Lawrence, C. E., Altschul, S. F., Boguski, M. S., Liu, J. S., Neuwald, A. F., & Wootton, J. C. (1993). Detecting subtle sequence signals: A Gibbs sampling strategy for multiple alignment. *Science*, 262(5131), 208-214. <https://doi.org/10.1126/science.8211139>
- Mistry, J., Chuguransky, S., Williams, L., Qureshi, M., Salazar, G. A., Sonnhammer, E. L. L., Tosatto, S. C. E., Paladin, L., Raj, S., Richardson, L. J., Finn, R. D., & Bateman, A. (2021). Pfam: The protein families database in 2021. *Nucleic Acids Research*, 49(D1), D412-D419. <https://doi.org/10.1093/nar/gkaa913>
- Notredame, C., Higgins, D. G., & Heringa, J. (2000). T-Coffee: A novel method for fast and accurate multiple sequence alignment. *Journal of Molecular Biology*, 302(1), 205-217. <https://doi.org/10.1006/jmbi.2000.4042>
- Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257-286. <https://doi.org/10.1109/5.18626>
- Schneider, T. D., & Stephens, R. M. (1990). Sequence logos: A new way to display consensus sequences. *Nucleic Acids Research*, 18(20), 6097-6100. <https://doi.org/10.1093/nar/18.20.6097>
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Sievers, F., Wilm, A., Dineen, D., Gibson, T. J., Karplus, K., Li, W., Lopez, R., McWilliam, H., Remmert, M., Söding, J., Thompson, J. D., & Higgins, D. G. (2011). Fast, scalable generation of high-quality protein multiple sequence alignments using Clustal Omega. *Molecular Systems Biology*, 7(1), Artículo 539. <https://doi.org/10.1038/msb.2011.75>
- Sjölander, K., Karplus, K., Brown, M., Hughey, R., Krogh, A., Mian, I. S., & Haussler, D. (1996). Dirichlet mixtures: A method for improved detection of weak but significant protein sequence homology. *Bioinformatics*, 12(4), 327-345. <https://doi.org/10.1093/bioinformatics/12.4.327>
- Stormo, G. D. (2000). DNA binding sites: Representation and discovery. *Bioinformatics*, 16(1), 16-23. <https://doi.org/10.1093/bioinformatics/16.1.16>
- Stormo, G. D., Schneider, T. D., Gold, L., & Ehrenfeucht, A. (1982). Use of the «Perceptron» algorithm to distinguish translational initiation sites in *E. coli*. *Nucleic Acids Research*, 10(9), 2997-3011. <https://doi.org/10.1093/nar/10.9.2997>
- Thompson, J. D., Higgins, D. G., & Gibson, T. J. (1994). CLUSTAL W: Improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position-specific gap penalties and weight matrix choice. *Nucleic Acids Research*, 22(22), 4673-4680. <https://doi.org/10.1093/nar/22.22.4673>
- Viterbi, A. J. (1967). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2), 260-269. <https://doi.org/10.1109/TIT.1967.1054010>
- Wang, L., & Jiang, T. (1994). On the complexity of multiple sequence alignment. *Journal of Computational Biology*, 1(4), 337-348. <https://doi.org/10.1089/cmb.1994.1.337>

Filogenética y evolución molecular



Las afinidades de todos los seres de la misma clase se han representado a veces por un gran árbol. Creo que este símil expresa en gran parte la verdad.

— Darwin (1859/2009)

● Al terminar este capítulo usted podrá

- Describir un árbol filogenético con precisión (raíz, nodos, ramas, clados, biparticiones) y calcular cuántas topologías posibles existen para n taxones.
- Formular la evolución de una secuencia como una cadena de Márkov de tiempo continuo, derivar la distancia de Jukes-Cantor y explicar qué añaden K80, F81, HKY85, GTR y la heterogeneidad gamma.
- Reconstruir árboles a partir de matrices de distancias con UPGMA y Neighbor-Joining, y reconocer cuándo cada método es consistente mediante las condiciones de ultrametricidad y de cuatro puntos.
- Calcular la verosimilitud de un árbol con el algoritmo de poda de Felsenstein, seleccionar modelos con AIC y BIC, interpretar el soporte bootstrap y UFBoot, y explicar la atracción de ramas largas.

En el capítulo 3 alineamos secuencias y evaluamos si su parecido supera lo esperable por azar. Una similitud significativa puede indicar ascendencia común. Ahora preguntaremos en qué orden se separaron los linajes y cuánto cambiaron desde entonces. Representaremos esas hipótesis mediante *árboles filogenéticos* y estudiaremos cómo reconstruirlos a partir de las secuencias.

El capítulo sigue ese orden. Primero construiremos el lenguaje probabilístico de la evolución molecular: cadenas de Márkov, matrices de tasas y los modelos de sustitución, desde el sencillo Jukes-Cantor hasta el modelo general reversible con tasas gamma (sección 5.1). Después usaremos esas distancias para reconstruir árboles con dos algoritmos clásicos y rapidísimos, UPGMA y Neighbor-Joining, y entenderemos con precisión matemática cuándo funcionan (sección 5.2). Por último, pasaremos al método que domina la práctica actual, la máxima verosimilitud, junto con la selección de modelos y el bootstrap, y lo contrastaremos con la parsimonia para ver por qué un buen modelo puede evitar errores sistemáticos (sección 5.3).

5.1 Distancias evolutivas y modelos de sustitución



Colab · 5.1 · Distancias y modelos de sustitución



abrir notebook

La pared repintada

Imagine dos paredes que hace años se pintaron del mismo color. Desde entonces, cada cierto tiempo, alguien elige al azar uno de cuatro colores y repinta una de ellas. Hoy usted solo ve el color final de cada pared. Si los colores difieren, sabe que hubo al menos un repintado; pero si coinciden, no puede descartar que hayan pasado por rojo, azul y verde para volver, por casualidad, al mismo tono. Tras muchísimos repintados, las paredes coinciden una de cada cuatro veces, sin importar si pasaron diez o mil años. Contar diferencias visibles subestima la historia, y la subestima cada vez más cuanto más larga es. Para recuperar el número real de repintados hace falta un *modelo* de cómo se repinta.

Cada posición de una secuencia de ADN se comporta como esa pared: acumula sustituciones que se superponen, se revierten y se ocultan unas a otras. Esta sección construye los modelos que permiten «ver a través» de esas superposiciones. Antes, sin embargo, necesitamos un vocabulario preciso para hablar de árboles.

5.1.1 Árboles: el vocabulario mínimo

Definición 5.1. (Árbol filogenético)

Un *árbol filogenético* es un grafo conexo y sin ciclos cuyas *hojas* (nodos de grado 1) representan los taxones o secuencias observadas y cuyos *nodos internos* representan ancestros hipotéticos. Cada *rama* (arista) tiene una longitud $b \geq 0$, medida en sustituciones esperadas por sitio. El árbol es *binario* si todo nodo interno tiene grado 3; es *enraizado* si uno de sus nodos, la *raíz*, se distingue como el ancestro común de todos los demás y define así la dirección del tiempo.

La figura 5.1 resume los términos. Dos más serán imprescindibles. Un *clado* es un ancestro junto con *todos* sus descendientes, y solo tiene sentido en árboles enraizados. En un árbol no enraizado, el concepto equivalente es la *bipartición* (*split*): al cortar una rama, el conjunto de hojas se divide en dos, por ejemplo $\{A, B\} | \{C, D, E\}$. Un árbol no enraizado queda completamente determinado por el conjunto de sus biparticiones, un hecho que usaremos para comparar árboles y para calcular el soporte bootstrap. Finalmente, la *topología* es la forma del árbol (quién está emparentado con quién) sin atender a las longitudes de las ramas.

¿Cuántos árboles hay?

Antes de reconstruir un árbol conviene saber entre cuántos candidatos debemos elegir. Consideremos árboles binarios no enraizados. Con tres hojas hay uno solo: una estrella de tres puntas. Un árbol binario no enraizado con k hojas tiene $2k - 3$ ramas; la $(k + 1)$ -ésima hoja puede insertarse en el punto medio de cualquiera de ellas, y cada elección produce una topología distinta. Multiplicando las opciones desde $k = 3$ hasta $k = n - 1$ se obtiene el resultado clásico, que se encuentra, por ejemplo, en Felsenstein (2004).

Teorema 5.1. (Número de topologías)

El número de topologías binarias distintas con $n \geq 3$ hojas etiquetadas es

$$U(n) = \prod_{k=3}^{n-1} (2k - 3) = (2n - 5)!!, \quad R(n) = (2n - 3)!! = U(n + 1), \quad (5.1)$$

para árboles no enraizados y enraizados, respectivamente.

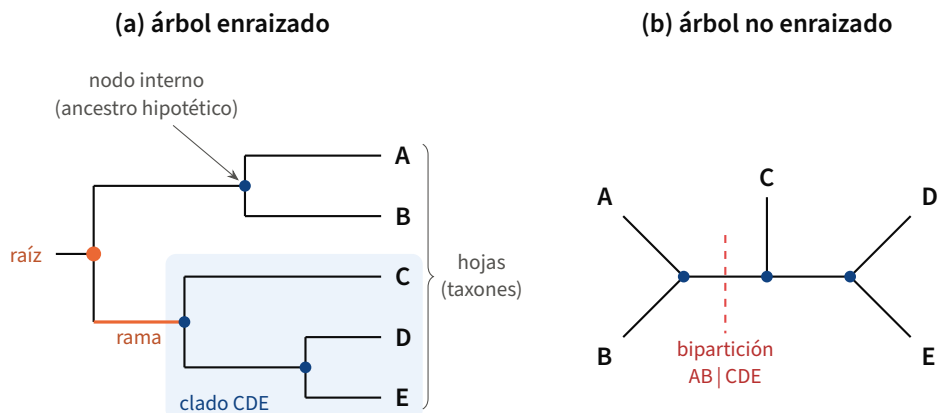


Figura 5.1. Anatomía de un árbol filogenético con cinco taxones. (a) En el árbol enraizado, la raíz (naranja) fija la dirección del tiempo: cada nodo interno (azul) es un ancestro hipotético, y el recuadro sombreado marca el clado formado por C, D, E y su ancestro común. (b) El mismo árbol sin raíz: las relaciones de parentesco se expresan como biparticiones; la línea roja corta la rama interna que separa $\{A, B\}$ de $\{C, D, E\}$. Los métodos de este capítulo producen árboles no enraizados; la raíz se añade después, por ejemplo con un grupo externo.

DONDE

$U(n)$, $R(n)$	Número de árboles binarios no enraizados y enraizados con n hojas.
$m!!$	Doble factorial de un impar: $m!! = m \cdot (m - 2) \cdots 3 \cdot 1$.
$2k - 3$	Número de ramas de un árbol no enraizado con k hojas: lugares donde insertar la siguiente.

La igualdad $R(n) = U(n + 1)$ tiene una lectura sencilla: enraizar un árbol equivale a añadirle una hoja ficticia (la raíz) en alguna de sus $2n - 3$ ramas. La tabla 5.1 muestra lo rápido que crece el problema. Con diez taxones hay unos dos millones de árboles no enraizados; con cincuenta, alrededor de 10^{74} , una cifra que se acerca al número de átomos del universo observable. Ningún computador puede evaluarlos todos, y por eso los métodos prácticos son, o bien algoritmos constructivos que edifican un único árbol paso a paso (sección 5.2), o bien búsquedas heurísticas que exploran una pequeña fracción del espacio (sección 5.3).

Tabla 5.1. Número de topologías binarias para n taxones (ecuación (5.1)). Valores calculados de forma exacta en `figuras/cap05/generar.py`.

n	no enraizados $(2n - 5)!!$	enraizados $(2n - 3)!!$
3	1	3
4	3	15
5	15	105
6	105	945
7	945	10 395
8	10 395	135 135
10	$2,03 \times 10^6$	$3,45 \times 10^7$
15	$7,91 \times 10^{12}$	$2,13 \times 10^{14}$
20	$2,22 \times 10^{20}$	$8,20 \times 10^{21}$
50	$2,84 \times 10^{74}$	$2,75 \times 10^{76}$

5.1.2 El reloj molecular y la distancia p

El reloj de las proteínas

A comienzos de los años sesenta, al comparar las secuencias de hemoglobinas y citocromos de distintos animales, Emile Zuckerkandl y Linus Pauling notaron que el número de diferencias entre dos proteínas homólogas crecía de forma aproximadamente proporcional al tiempo transcurrido desde la separación de las especies, estimado a partir de los fósiles. Propusieron entonces que las proteínas funcionan como un *reloj evolutivo molecular* (Zuckerkandl & Pauling, 1965). La idea transformó la biología: si las moléculas cambian a ritmo aproximadamente constante, sus diferencias no solo revelan parentescos, sino también fechas.

La medida más directa de divergencia entre dos secuencias alineadas es la proporción de sitios en que difieren.

Definición 5.2. (Distancia p)

Sean x e y dos secuencias alineadas con L sitios comparables (sin huecos). La *distancia p* es

$$p = \frac{1}{L} \sum_{h=1}^L 1[x_h \neq y_h]. \quad (5.2)$$

DONDE		
p		Proporción observada de sitios diferentes (distancia p o distancia de Hamming normalizada).
L		Número de sitios alineados comparables.
x_h, y_h		Nucleótidos de las dos secuencias en el sitio h .

La distancia p es un buen estimador del número de sustituciones solo cuando la divergencia es pequeña. A medida que el tiempo avanza, un sitio puede cambiar dos veces ($A \rightarrow G \rightarrow C$), revertir a su estado original ($A \rightarrow G \rightarrow A$) o cambiar en paralelo en ambos linajes hacia la misma base. Todos estos *impactos múltiples* (*multiple hits*) quedan ocultos. El resultado es la *saturación*: p crece cada vez más despacio y, para ADN con bases equiprobables, se estanca en $3/4$, la probabilidad de que dos bases elegidas al azar difieran. Para corregir la saturación necesitamos un modelo explícito del proceso de sustitución.

5.1.3 La evolución como cadena de Márkov de tiempo continuo

Modelaremos cada sitio de la secuencia como un proceso aleatorio $X(t)$ que toma valores en $\{A, C, G, T\}$ y salta de un estado a otro a lo largo del tiempo. Los modelos estándar hacen cuatro supuestos:

- 1. **Independencia:** los sitios evolucionan de forma independiente (y, salvo el modelo gamma, idéntica).
- 2. **Propiedad de Márkov:** la probabilidad de un cambio futuro depende solo del estado actual, no de la historia.
- 3. **Homogeneidad:** las tasas de cambio no varían a lo largo del tiempo ni entre ramas.
- 4. **Estacionariedad y reversibilidad:** la composición de bases está en equilibrio y el proceso «se ve igual» hacia adelante y hacia atrás en el tiempo.

Definición 5.3. (Matriz de tasas instantáneas)

Una cadena de Márkov de tiempo continuo sobre s estados se describe mediante una matriz $Q = (q_{ij})$ de tamaño $s \times s$ tal que, para un intervalo de tiempo Δt muy pequeño,

$$\mathbb{P}(X(t + \Delta t) = j | X(t) = i) = q_{ij} \Delta t + o(\Delta t), \quad i \neq j; \quad q_{ii} = - \sum_{j \neq i} q_{ij}. \quad (5.3)$$

DONDE

Q	Matriz de tasas instantáneas de sustitución (4×4 para ADN, 20×20 para proteínas).
q_{ij}	Tasa a la que el estado i se sustituye por j ($q_{ij} \geq 0$ si $i \neq j$).
q_{ii}	Menos la tasa total de salida del estado i ; hace que cada fila sume cero.
$o(\Delta t)$	Términos que se anulan más rápido que Δt (p. ej., dos cambios en el mismo intervalo).

La matriz Q describe lo que ocurre en un instante; lo que observamos, sin embargo, son los resultados después de un tiempo finito. Sea $P(t)$ la matriz de probabilidades de transición, con $P_{ij}(t) = \mathbb{P}(X(t) = j | X(0) = i)$. Para llegar de i a j en el tiempo $t + \Delta t$, el proceso debe estar en algún estado k en el tiempo t y luego pasar de k a j en el breve intervalo final:

$$P_{ij}(t + \Delta t) = \sum_k P_{ik}(t) (\delta_{kj} + q_{kj} \Delta t) + o(\Delta t) \implies \frac{P(t + \Delta t) - P(t)}{\Delta t} \rightarrow P(t)Q.$$

Esta es la *ecuación de Kolmogorov hacia adelante*, $P'(t) = P(t)Q$ con $P(0) = I$. Su solución es la exponencial de una matriz:

Teorema 5.2. (Probabilidades de transición)

Para una cadena de Márkov homogénea con matriz de tasas Q ,

$$P(t) = e^{Qt} = \sum_{k=0}^{\infty} \frac{(Qt)^k}{k!} = U \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_s t}) U^{-1}, \quad (5.4)$$

y se cumple la propiedad de Chapman-Kolmogorov $P(t + u) = P(t)P(u)$.

DONDE

$P(t)$	Matriz de probabilidades de transición: $P_{ij}(t)$ es la probabilidad de ver j tras un tiempo t partiendo de i .
e^{Qt}	Exponencial matricial, definida por la serie de potencias.
U, λ_k	Matriz de vectores propios de Q y sus valores propios ($\lambda_1 = 0$, los demás negativos).
I	Matriz identidad: con $t = 0$ nada ha cambiado.

La descomposición espectral de la ecuación (5.4) es la forma en que los programas calculan $P(t)$: se diagonaliza Q una sola vez y, para cada rama, basta con exponenciar s números. El valor propio nulo corresponde a la distribución estacionaria π , que satisface $\pi Q = 0$ y, por tanto, $\pi P(t) = \pi$ para todo t : si la composición de bases está en equilibrio, la evolución no la altera.

Dos condiciones adicionales hacen que los modelos sean manejables. La primera es la *reversibilidad temporal*, o condición de balance detallado:

$$\pi_i q_{ij} = \pi_j q_{ji} \quad \text{para todo } i, j. \quad (5.5)$$

DONDE

π_i	Frecuencia de equilibrio del nucleótido i ($\sum_i \pi_i = 1$).
$\pi_i q_{ij}$	Flujo de sustituciones $i \rightarrow j$ en equilibrio; la reversibilidad exige que iguale al flujo $j \rightarrow i$.

Con reversibilidad, la probabilidad de observar x en un extremo de una rama e y en el otro no depende de cuál extremo sea el «antiguo». Por eso, la distancia entre dos secuencias actuales separadas por dos ramas de longitudes b_1 y b_2 que se unen en su ancestro común se comporta como una sola rama de longitud $b_1 + b_2$, y por eso los métodos de inferencia pueden trabajar con árboles no enraizados. La segunda condición es una *normalización*: escalamos Q de modo que la tasa media de sustitución en equilibrio sea uno,

$$-\sum_i \pi_i q_{ii} = 1. \quad (5.6)$$

DONDE

$-\sum_i \pi_i q_{ii}$	Número esperado de sustituciones por sitio y por unidad de tiempo, promediado sobre la composición de equilibrio.
------------------------	---

Con esta convención, el tiempo t se mide directamente en *sustituciones esperadas por sitio*, y la longitud de una rama coincide con la distancia evolutiva d que queremos estimar. Tiempo y tasa quedan confundidos en un único parámetro: sin un reloj externo (fósiles, fechas de muestreo) no es posible separarlos.

5.1.4 Jukes-Cantor: la derivación completa

El modelo más sencillo, propuesto por Jukes y Cantor (1969), supone que las cuatro bases son equiprobables ($\pi_i = 1/4$) y que cualquier base se sustituye por cualquier otra con la misma tasa α :

$$Q_{JC} = \begin{pmatrix} -3\alpha & \alpha & \alpha & \alpha \\ \alpha & -3\alpha & \alpha & \alpha \\ \alpha & \alpha & -3\alpha & \alpha \\ \alpha & \alpha & \alpha & -3\alpha \end{pmatrix} \quad (\text{orden A, C, G, T}). \quad (5.7)$$

DONDE

α	Tasa de sustitución hacia cada una de las otras tres bases.
3α	Tasa total de salida de cualquier base; con la ecuación (5.6), $3\alpha = 1$.

Por simetría, todas las probabilidades de permanecer son iguales, $P_{ii}(t) \equiv a(t)$, y todas las de cambio también, $P_{ij}(t) \equiv (1 - a(t))/3$. La ecuación de Kolmogorov para un elemento diagonal da

$$a'(t) = \underbrace{-3\alpha a(t)}_{\text{salir de } i} + \underbrace{3 \cdot \alpha \frac{1 - a(t)}{3}}_{\text{volver a } i} = \alpha - 4\alpha a(t), \quad a(0) = 1.$$

Es una ecuación lineal de primer orden, cuya solución es $a(t) = \frac{1}{4} + \frac{3}{4}e^{-4\alpha t}$. Como el número esperado de sustituciones es $d = 3\alpha t$, podemos escribir $4\alpha t = 4d/3$ y obtenemos el resultado central de esta sección.

Teorema 5.3. (Distancia de Jukes-Cantor)

Bajo el modelo JC69, dos secuencias separadas por d sustituciones esperadas por sitio difieren en una proporción esperada de sitios

$$p(d) = \frac{3}{4} \left(1 - e^{-4d/3}\right). \quad (5.8)$$

Invertiendo esta relación con el valor observado $\hat{p}$ se obtiene el estimador de la distancia y , por el método delta, su varianza aproximada:

$$\hat{d}_{JC} = -\frac{3}{4} \ln\left(1 - \frac{4}{3} \hat{p}\right), \quad \text{Var}(\hat{d}_{JC}) \approx \frac{\hat{p}(1 - \hat{p})}{L \left(1 - \frac{4}{3} \hat{p}\right)^2}. \quad (5.9)$$

DONDE

d	Distancia evolutiva: sustituciones esperadas por sitio entre las dos secuencias (suma de las ramas que las separan).
$p(d), \hat{p}$	Proporción de diferencias esperada según el modelo y observada en los datos.
$\hat{d}_{JC}$	Estimador de máxima verosimilitud de d bajo JC69.
L	Longitud del alineamiento; la varianza decrece como $1/L$.

La ecuación (5.8) captura la saturación con exactitud: para d pequeño, $p \approx d$ (casi no hay impactos múltiples), pero cuando $d \rightarrow \infty$, $p \rightarrow 3/4$. La ecuación (5.9) tiene dos consecuencias prácticas. Primero, $\hat{d}_{JC}$ no está definida si $\hat{p} \geq 3/4$: los datos son compatibles con cualquier distancia grande. Segundo, el denominador de la varianza se acerca a cero cuando $\hat{p}$ se acerca a $3/4$, de modo que las distancias grandes son muy imprecisas. La figura 5.2 muestra ambas cosas: la curva teórica, los puntos simulados y lo mucho que se aplana la relación a partir de $d \approx 1$.

5.1.5 Más allá de Jukes-Cantor: K80, F81, HKY85 y GTR

JC69 es elegante, pero dos de sus supuestos son claramente falsos en datos reales. El primero es que todas las sustituciones son igual de probables. Las bases se dividen químicamente en *purinas* (**A**, **G**, de dos anillos) y *pirimidinas* (**C**, **T**, de un anillo). Las *transiciones* intercambian bases dentro de un mismo grupo (**A**↔**G**, **C**↔**T**); las *transversiones*, entre grupos. Aunque hay el doble de tipos de transversión que de transición, en la mayoría de los genomas las transiciones son bastante más

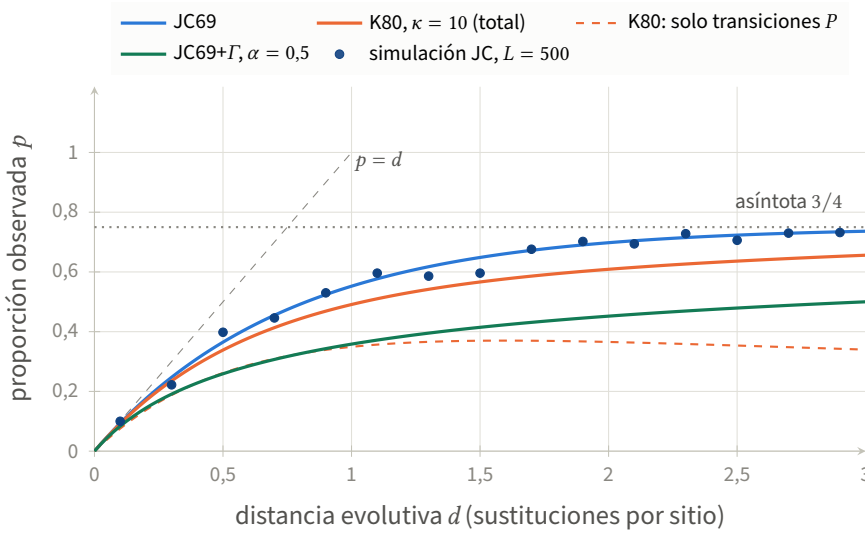


Figura 5.2. Saturación de la distancia observada. La proporción de sitios diferentes crece casi como d al principio, pero se curva y se estanca en $3/4$ (JC69, azul). Los puntos son pares de secuencias de 500 pb simulados bajo JC69: su dispersión alrededor de la curva recuerda que $\hat{p}$ es una estimación ruidosa. Con un sesgo fuerte hacia transiciones (K80 con $\kappa = 10$, naranja), las transiciones saturan mucho antes (línea discontinua, $P \rightarrow 1/4$) y el total avanza más lentamente. Con heterogeneidad de tasas (aqua), los sitios rápidos se saturan pronto y los lentos casi no cambian, de modo que la curva se aplana más despacio y engaña más: un mismo $\hat{p}$ corresponde a un d mucho mayor.

frecuentes, porque los errores de replicación y la desaminación de bases favorecen cambios que conservan el tamaño del anillo. Kimura (1980) incorporó esta asimetría con dos tasas: α para transiciones y β para transversiones (figura 5.3). Su parámetro clave es el cociente $\kappa = \alpha/\beta$, y la distancia correspondiente es

$$\hat{d}_{K80} = -\frac{1}{2} \ln(1 - 2P - Q) - \frac{1}{4} \ln(1 - 2Q). \quad (5.10)$$

DONDE

P, Q	Proporciones observadas de sitios que difieren por una transición y por una transversión ($p = P + Q$).
$\kappa = \alpha/\beta$	Cociente de tasas transición/transversión (en mamíferos suele estar entre 2 y 10).

(Aquí Q es la proporción de transversiones, no la matriz de tasas.) El segundo supuesto discutible es la composición equilibrada. Muchos genomas son ricos en **A+T** o en **G+C**. En el modelo F81 de Felsenstein (1981), la tasa hacia una base es proporcional a su frecuencia de equilibrio, $q_{ij} = \mu \pi_j$, y la distancia generaliza la de Jukes-Cantor sustituyendo el 3/4 por $B = 1 - \sum_i \pi_i^2$, la probabilidad de que dos bases elegidas al azar según π difieran:

$$\hat{d}_{F81} = -B \ln\left(1 - \frac{\hat{p}}{B}\right), \quad B = 1 - \sum_i \pi_i^2. \quad (5.11)$$

DONDE

μ	Tasa global de sustitución (fijada por la normalización).
B	Nivel de saturación del modelo: la p esperada entre secuencias no relacionadas.

Hasegawa et al. (1985) combinaron ambas ideas en el modelo HKY85, que tiene frecuencias desiguales y un cociente κ . Finalmente, el *modelo general reversible* (GTR), cuya forma general analizaron Rodríguez et al. (1990), permite una tasa distinta para cada uno de los seis pares de bases. Todos ellos pueden escribirse con una sola fórmula,

$$q_{ij} = \mu r_{ij} \pi_j \quad (i \neq j), \quad r_{ij} = r_{ji}, \quad (5.12)$$

DONDE

r_{ij}	Parámetro de intercambiabilidad simétrico entre las bases i y j (uno de ellos se fija en 1).
π_j	Frecuencia de equilibrio de la base de llegada.
μ	Factor de escala que impone la ecuación (5.6).

y la reversibilidad (ecuación (5.5)) se cumple automáticamente, porque $\pi_i q_{ij} = \mu r_{ij} \pi_i \pi_j$ es simétrico en i, j . Los modelos más sencillos son casos particulares que restringen los r_{ij} y los π_j (tabla 5.2). Esta estructura *anidada* será importante al compararlos mediante pruebas de razón de verosimilitudes en la sección 5.3.

5.1.6 Heterogeneidad de tasas entre sitios: el modelo Γ

El supuesto más dañino de todos los modelos anteriores es que todos los sitios evolucionan a la misma velocidad. En un gen codificante, las terceras posiciones de los codones, a menudo sinónimas, cambian mucho más deprisa que las segundas;

Tabla 5.2. Modelos reversibles de sustitución de nucleótidos como casos particulares de GTR (ecuación (5.12)). Los parámetros libres no incluyen las longitudes de las ramas. Cualquiera de ellos puede combinarse con heterogeneidad gamma (+ Γ , un parámetro más) o con una proporción de sitios invariables (+I).

Modelo	Frecuencias π	Intercambiabilidades r_{ij}	Referencia	Libres
JC69	iguales	todas iguales	Jukes y Cantor (1969)	0
K80	iguales	transición κ , transversión 1	Kimura (1980)	1
F81	libres	todas iguales	Felsenstein (1981)	3
HKY85	libres	transición κ , transversión 1	Hasegawa et al. (1985)	4
GTR	libres	seis valores (uno fijo)	Rodríguez et al. (1990)	8

en un ARN ribosómico, los bucles varían mientras los tallos apareados se conservan. Si ignoramos esta variación, los sitios rápidos saturan pronto, los lentos no aportan diferencias, y la distancia resultante se subestima sistemáticamente.

Yang (1994) popularizó la solución estándar: suponer que la tasa relativa r de cada sitio es una variable aleatoria con distribución gamma de media 1,

$$g(r; \alpha) = \frac{\alpha^\alpha}{\Gamma(\alpha)} r^{\alpha-1} e^{-\alpha r}, \quad \mathbb{E}[r] = 1, \quad \text{Var}(r) = \frac{1}{\alpha}, \quad (5.13)$$

DONDE

r	Tasa relativa de un sitio: multiplica todas las longitudes de rama para ese sitio.
α	Parámetro de forma; α pequeño significa tasas muy desiguales, $\alpha \rightarrow \infty$ recupera tasas iguales.
$\Gamma(\alpha)$	Función gamma de Euler, que normaliza la densidad.

La probabilidad de un sitio se calcula entonces promediando sobre r . Como la integral no tiene forma cerrada para árboles, Yang propuso aproximarla con K categorías de igual probabilidad, cada una representada por la tasa media de su intervalo. Con $K = 4$ y $\alpha = 0,5$, las cuatro tasas son 0,033, 0,252, 0,820 y 2,894: una cuarta parte de los sitios es casi invariable y otra cuarta parte evoluciona casi tres veces más rápido que el promedio. Para distancias entre pares la integral sí es explícita y da la distancia JC+ Γ ,

$$\hat{d}_{\text{JC}+\Gamma} = \frac{3}{4} \alpha \left[\left(1 - \frac{4}{3} \hat{p} \right)^{-1/\alpha} - 1 \right], \quad (5.14)$$

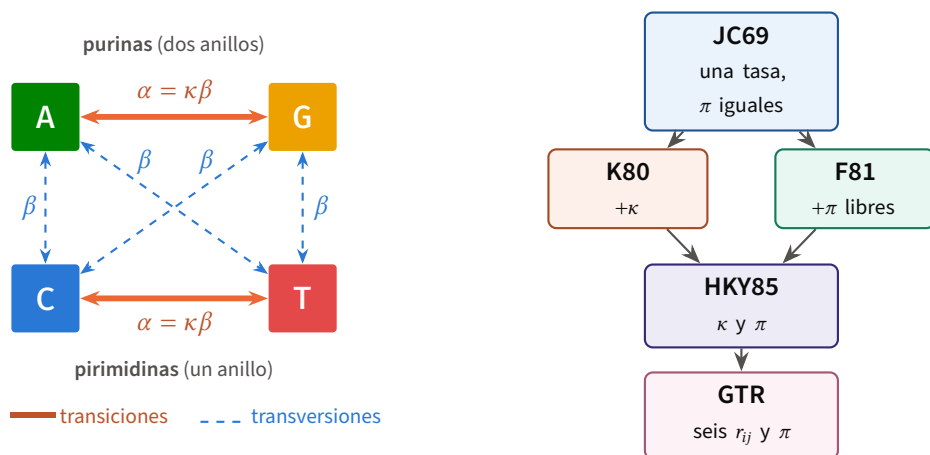


Figura 5.3. Izquierda: las seis clases de sustitución entre nucleótidos. Las transiciones (naranja, trazo grueso) conservan el tipo de anillo y ocurren a una tasa α típicamente varias veces mayor que la de las transversiones (azul, β). JC69 hace iguales todas las flechas; K80 distingue los dos colores; GTR permite un valor distinto para cada una de las seis. Derecha: jerarquía de modelos anidados; cada flecha añade parámetros y cada modelo inferior contiene al superior como caso particular.

DONDE

$\hat{d}_{JC+\Gamma}$ Distancia de Jukes-Cantor corregida por heterogeneidad gamma; tiende a $\hat{d}_{JC}$ cuando $\alpha \rightarrow \infty$.

que siempre es mayor que la de JC para el mismo $\hat{p}$. La figura 5.4 ilustra cómo cambia la forma de la distribución con α . En la práctica, además de $+\Gamma$, suele añadirse una proporción de sitios invariables ($+I$), y los programas modernos ofrecen modelos de tasas libres (*FreeRate*) que no imponen la forma gamma.

Ejemplo 5.1. (Tres distancias para el mismo par de secuencias)

Considere el siguiente par de fragmentos alineados de 40 pb (las marcas indican i para transición y v para transversión):

```

ATGGCTAAGCTTACGGATCCAGTACTGAAACGTTGCAAGT
|||i|||i|||v|||v|||i|||i||v||
ATGACTAACTTACTGATCCTGTACTGGAACGTCGCCAGT
  
```

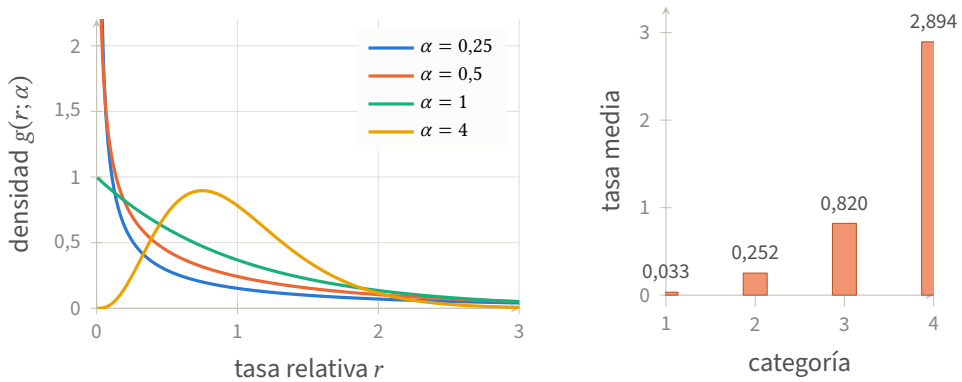


Figura 5.4. Izquierda: densidad gamma de media 1 para varios valores del parámetro de forma. Con $\alpha < 1$ la distribución tiene forma de L: muchos sitios casi invariables y unos pocos muy rápidos, la situación habitual en genes codificantes. Con $\alpha = 4$ las tasas se concentran alrededor de 1. Derecha: las cuatro categorías discretas de igual probabilidad para $\alpha = 0,5$ (Yang, 1994): la categoría más rápida evoluciona unas 90 veces más deprisa que la más lenta.

Hay 7 diferencias: 4 transiciones y 3 transversiones. Por lo tanto, $\hat{p} = 7/40 = 0,175$, $P = 0,100$ y $Q = 0,075$.

- **JC69** (ecuación (5.9)): $\hat{d} = -\frac{3}{4} \ln(1 - \frac{4}{3} \cdot 0,175) = -\frac{3}{4} \ln(0,7667) = 0,199$, con error estándar $\sqrt{\text{Var}} \approx 0,078$.
- **K80** (ecuación (5.10)): $\hat{d} = -\frac{1}{2} \ln(1 - 0,2 - 0,075) - \frac{1}{4} \ln(1 - 0,15) = 0,161 + 0,041 = 0,201$.
- **JC+ Γ** (ecuación (5.14)): con $\alpha = 1$, $\hat{d} = 0,228$; con $\alpha = 0,5$, $\hat{d} = 0,263$.

Tres lecciones se desprenden de estas cifras. Primera: todas las correcciones superan a $\hat{p}$, porque añaden los cambios ocultos. Segunda: K80 apenas difiere de JC aquí, porque la proporción de transiciones observada (4 de 7) no está muy sesgada; la diferencia crece con κ y con la divergencia. Tercera: la corrección gamma es la que más pesa (un 32 % más que JC con $\alpha = 0,5$), y el error estándar ($\pm 0,078$ para JC) es casi tan grande como las diferencias entre modelos: con 40 sitios, la incertidumbre de muestreo domina. Todas las cifras se reproducen con `figuras/cap05/generar.py`.

distancias.py

```
import numpy as np
from scipy.linalg import expm

TS = {("A", "G"), ("G", "A"), ("C", "T"), ("T", "C")}

def distancias(x: str, y: str) -> dict:
    """Distancias p, JC69 y K80 entre dos secuencias alineadas."""
    pares = [(a, b) for a, b in zip(x, y) if "-" not in (a, b)]
    L = len(pares)
    P = sum((a, b) in TS for a, b in pares) / L      # transiciones
    Q = sum(a != b and (a, b) not in TS for a, b in pares) / L
    p = P + Q
    d_jc = -0.75 * np.log(1 - 4 * p / 3)
    d_k80 = -0.5 * np.log(1 - 2 * P - Q) - 0.25 * np.log(1 - 2 * Q)
    return dict(p=p, jc=d_jc, k80=d_k80)

def matriz_P_k80(d: float, kappa: float) -> np.ndarray:
    """P(t) = exp(Qt) para K80 normalizado a d sustituciones."""
    Q = np.array([[0, 1, kappa, 1], [1, 0, 1, kappa],
                  [kappa, 1, 0, 1], [1, kappa, 1, 0]], float)
    np.fill_diagonal(Q, -Q.sum(1))
    Q /= -np.mean(np.diag(Q))      # tasa media = 1
    return expm(Q * d)            # orden A, C, G, T

print(distancias("ATGGCTAAGCTTACGG", "ATGACTAACTTACTG"))
print(matriz_P_k80(0.3, kappa=4).round(4)[0])  # fila de A
```

La segunda función muestra la ecuación (5.4) en acción. Con $d = 0,3$ y $\kappa = 4$, la fila de **A** de $P(t)$ es (0,758; 0,045; 0,151; 0,045): la probabilidad de terminar en **G** (transición) es más del triple que la de terminar en **C** o en **T**.

Distancias que mienten

1. **Saturación:** si $\hat{p}$ se acerca al nivel de saturación ($3/4$ en JC, B en F81), la distancia corregida explota o no existe. Para genes muy divergentes conviene usar proteínas en lugar de ADN, o eliminar las terceras posiciones de los codones.

2. *Modelo inadecuado*: ignorar la heterogeneidad de tasas subestima sobre todo las distancias grandes y deforma el árbol entero, no solo las ramas largas.
3. *Huecos*: la ecuación (5.2) supone sitios comparables; los programas eliminan los huecos par a par o en todo el alineamiento, y la elección cambia el resultado.

p mide lo que vemos; d mide lo que ocurrió.
El modelo de sustitución es el puente entre ambos.

5.2 UPGMA y Neighbor-Joining

 Colab · 5.2 · UPGMA y Neighbor-Joining

 abrir notebook

Reconstruir las carreteras desde la tabla de kilómetros

En la guantera de un coche viejo aparece una tabla con las distancias por carretera entre cinco pueblos, pero no hay mapa. ¿Es posible dibujar la red de carreteras? Si las carreteras forman un árbol, sin circuitos, y las distancias se midieron recorriéndolas, la respuesta es sí, y además de forma exacta: cada distancia es la suma de los tramos que la componen, y esa redundancia basta para deducir dónde están los cruces y cuánto mide cada tramo. Pero si alguien anotó las distancias en línea recta, o se equivocó al copiarlas, la tabla deja de ser coherente con ningún árbol, y hay que buscar el que mejor se le ajuste.

Los métodos de distancias siguen exactamente ese programa. Primero reducen el alineamiento a una matriz D de distancias evolutivas entre pares, calculadas con los modelos de la sección 5.1; luego buscan un árbol cuyas distancias de camino reproduzcan D . Son rápidos, intuitivos y todavía se usan mucho, tanto como árboles finales en análisis exploratorios como árboles iniciales para métodos más costosos. Para entender cuándo funcionan necesitamos dos definiciones.

5.2.1 Árboles aditivos y ultramétricos

Definición 5.4. (Distancias aditivas y ultramétricas)

Una matriz de distancias D sobre n taxones es *aditiva* si existe un árbol con longitudes de rama no negativas tal que, para todo par i, j , D_{ij} es la suma de las longitudes de las ramas del camino entre i y j . Es *ultramétrica* si, además, el árbol puede enraizarse de modo que todas las hojas estén a la misma distancia de la raíz.

Un árbol ultramétrico es un árbol con *reloj molecular*: todos los linajes han acumulado el mismo número de sustituciones desde la raíz, de modo que la profundidad de cada nodo es proporcional a su edad. Un árbol aditivo no exige nada de eso: las ramas pueden tener longitudes arbitrarias, como ocurre cuando unos linajes evolucionan más deprisa que otros (figura 5.5). Ambas propiedades pueden comprobarse directamente sobre la matriz, sin conocer el árbol.

Teorema 5.4. (Condiciones de tres y cuatro puntos)

(i) D es ultramétrica si y solo si, para toda terna i, j, k ,

$$D_{ij} \leq \max\{D_{ik}, D_{jk}\}, \quad (5.15)$$

es decir, los dos valores mayores de cada terna son iguales. (ii) D es aditiva si y solo si, para todo cuarteto i, j, k, l , de las tres sumas

$$D_{ij} + D_{kl}, \quad D_{ik} + D_{jl}, \quad D_{il} + D_{jk} \quad (5.16)$$

las dos mayores son iguales (Buneman, 1974).

DONDE

i, j, k, l	Cuatro taxones cualesquiera, todos distintos.
D_{ij}	Distancia entre los taxones i y j .

La condición de cuatro puntos tiene una explicación gráfica muy clara (figura 5.6). En un árbol, cuatro hojas siempre se agrupan en dos pares separados por un camino interno de longitud $e \geq 0$. Si los pares son $\{i, j\}$ y $\{k, l\}$, la suma $D_{ij} + D_{kl}$ no recorre ese camino, mientras que las otras dos sumas lo recorren dos veces cada una. Así, las dos sumas «cruzadas» son iguales y superan a la primera en exactamente $2e$.

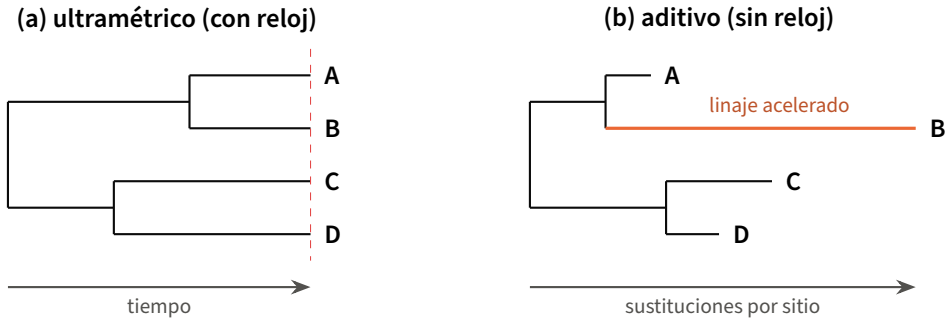


Figura 5.5. (a) En un árbol ultramétrico todas las hojas quedan alineadas (línea roja): la distancia raíz-hoja es la misma para todos los taxones, como exige un reloj molecular estricto, y la altura de cada nodo es proporcional a su edad. (b) En un árbol aditivo las ramas tienen longitudes arbitrarias; el linaje de B (naranja) ha acumulado muchas más sustituciones que su hermano A. Las distancias entre hojas siguen siendo sumas de ramas, pero ya no reflejan directamente los tiempos. UPGMA supone el caso (a); Neighbor-Joining solo necesita el caso (b).

Esto no solo prueba la condición: dice también cómo leer la topología del cuarteto (el par con la suma *menor* son vecinos) y cuánto mide su rama interna.

5.2.2 UPGMA: agrupar por promedios

El método de agrupamiento más antiguo que todavía se enseña es UPGMA (*unweighted pair group method with arithmetic mean*), introducido a finales de los años cincuenta en el contexto de la taxonomía numérica (Felsenstein, 2004). Su lógica es la de cualquier agrupamiento jerárquico aglomerativo:

1. Comience con cada taxón como un grupo de tamaño 1, situado a altura 0.
2. Encuentre el par de grupos (A, B) con la menor distancia D_{AB} .
3. Únalos en un nuevo nodo situado a altura $h = D_{AB}/2$ (cada rama mide h menos la altura del grupo hijo).
4. Calcule la distancia del nuevo grupo a cualquier otro grupo K como el promedio ponderado por tamaño:

$$D_{(AB),K} = \frac{|A| D_{AK} + |B| D_{BK}}{|A| + |B|}. \quad (5.17)$$

5. Repita desde el paso 2 hasta que quede un solo grupo, que será la raíz.

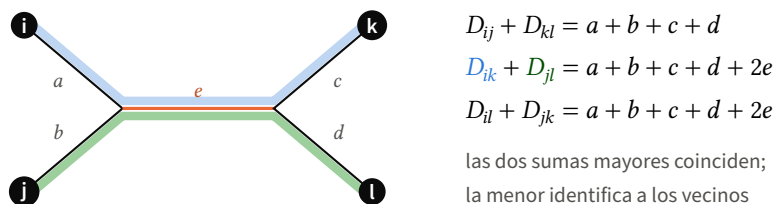


Figura 5.6. La condición de cuatro puntos. En un cuarteto con topología $ij|kl$, los caminos $i \rightarrow k$ (azul) y $j \rightarrow l$ (verde) atraviesan ambos la rama interna e (naranja), mientras que los caminos $i \rightarrow j$ y $k \rightarrow l$ no lo hacen. Por eso las dos sumas «cruzadas» son iguales y exceden a la tercera en $2e$. En la matriz del ejemplo 5.2, para el cuarteto A, B, C, D, las sumas son 1,1, 1,3 y 1,3: A y B son vecinos y la rama interna mide 0,1.

DONDE

A, B, K	Grupos (clados) formados hasta el momento.
$ A $	Número de taxones del grupo A.
$D_{(AB),K}$	Distancia media entre los taxones de $A \cup B$ y los de K.

UPGMA produce siempre un árbol enraizado y ultramétrico, y su coste es $O(n^2)$ en memoria y $O(n^3)$ en tiempo en la versión directa. Su gran virtud es también su gran defecto: si D es ultramétrica, UPGMA recupera exactamente el árbol verdadero, pero si no lo es, porque los linajes evolucionan a ritmos distintos, puede unir taxones que no son vecinos. La razón es simple. UPGMA une el par más cercano, y en un árbol sin reloj el par más cercano no tiene por qué ser un par de hermanos: un taxón con una rama muy larga puede estar más lejos de su hermano que de un primo con ramas cortas. El ejemplo 5.2 y la figura 5.8 muestran exactamente esa situación.

5.2.3 Neighbor-Joining: corregir por la divergencia neta

Saitou y Nei (1987) propusieron una solución ingeniosa. En lugar de unir el par con la menor distancia, Neighbor-Joining (NJ) une el par con la menor distancia *corregida por lo lejos que está cada uno de todos los demás*. Un taxón con una rama larga está lejos de todo el mundo; restando su divergencia total se elimina ese efecto.

Teorema 5.5. (Criterio y actualización de Neighbor-Joining)

Con n nodos activos, defina la divergencia neta $r_i = \sum_k D_{ik}$ y la matriz

$$Q_{ij} = (n - 2) D_{ij} - r_i - r_j. \quad (5.18)$$

Se unen los nodos i, j que minimizan Q_{ij} en un nuevo nodo u , con ramas

$$b_{iu} = \frac{D_{ij}}{2} + \frac{r_i - r_j}{2(n - 2)}, \quad b_{ju} = D_{ij} - b_{iu}, \quad (5.19)$$

y las distancias del nuevo nodo a los demás se actualizan como

$$D_{uk} = \frac{1}{2} (D_{ik} + D_{jk} - D_{ij}). \quad (5.20)$$

El proceso se repite hasta que quedan tres nodos, que se unen en una estrella. Si D es aditiva, NJ recupera el árbol verdadero con sus longitudes de rama exactas.

DONDE

n	Número de nodos activos (hojas o nodos ya formados) en el paso actual.
r_i	Divergencia neta de i : suma de sus distancias a todos los nodos activos.
Q_{ij}	Criterio de selección; su mínimo identifica un par de vecinos.
b_{iu}	Longitud de la rama que une i con el nuevo nodo u .
D_{uk}	Distancia del nuevo nodo a otro nodo k : lo que queda del camino $i \rightarrow k$ al descontar la rama b_{iu} , promediado con el de j .

¿De dónde sale la ecuación (5.18)? Saitou y Nei la derivaron desde el principio de *evolución mínima*: partiendo de un árbol en estrella, unir i y j produce un árbol cuya longitud total S_{ij} puede calcularse a partir de D , y el par elegido es el que minimiza esa longitud. Studier y Keppler (1988) mostraron que minimizar S_{ij} equivale a minimizar la expresión más simple de la ecuación (5.18) (ambas difieren en un factor positivo y una constante) y que el algoritmo completo corre en tiempo $O(n^3)$. La fórmula de las ramas (ecuación (5.19)) también se entiende: la rama de i es la mitad de D_{ij} más una corrección proporcional a cuánto más lejos de todos está i que j . La ecuación (5.20), por su parte, es la condición de aditividad despejada: si u está en el camino entre i y k , entonces $D_{ik} = b_{iu} + D_{uk}$.

Lo que NJ optimiza realmente

Durante dos décadas se discutió qué criterio optimiza NJ, si es que optimiza alguno. Gascuel y Steel (2006) resumieron la respuesta: NJ es un algoritmo voraz (*greedy*) para el criterio de *evolución mínima balanceada*, que estima la longitud total del árbol con una ponderación particular de las distancias. Esto explica tanto su robustez (tolera bastante bien el ruido en D) como sus límites: al ser voraz, una decisión temprana errónea no se corrige después. Programas como FastME aplican búsquedas locales sobre ese mismo criterio para mejorar el árbol de NJ.

Ejemplo 5.2. (Neighbor-Joining paso a paso)

La tabla 5.3 contiene las distancias de camino de un árbol *aditivo pero no ultramétrico* con cinco taxones, en el que B tiene una rama larga (figura 5.8, izquierda). Puede comprobarse que satisface la condición de cuatro puntos: para A, B, C, D, las sumas son $D_{AB} + D_{CD} = 1,1$, $D_{AC} + D_{BD} = 1,3$ y $D_{AD} + D_{BC} = 1,3$.

Paso 1 ($n = 5$). Las divergencias netas son $r = (2,0; 2,9; 2,2; 2,6; 2,3)$. Por ejemplo, $Q_{AB} = 3 \cdot 0,5 - 2,0 - 2,9 = -3,4$ y $Q_{AC} = 3 \cdot 0,4 - 2,0 - 2,2 = -3,0$. El mínimo, $-3,4$, se alcanza en dos pares, A-B y D-E, ambos vecinos genuinos; tomamos D-E (resaltado en la tabla). Observe que A-C, el par más cercano (0,4), *no* gana. Las ramas son $b_D = 0,25 + (2,6 - 2,3)/6 = 0,30$ y $b_E = 0,20$. El nuevo nodo u_1 queda a distancias $D_{u_1A} = \frac{1}{2}(0,6 + 0,5 - 0,5) = 0,3$, $D_{u_1B} = 0,6$ y $D_{u_1C} = 0,3$.

Paso 2 ($n = 4$). Con los nodos A, B, C, u_1 resulta $r = (1,2; 1,8; 1,4; 1,2)$ y $Q_{AB} = 2 \cdot 0,5 - 1,2 - 1,8 = -2,0$, que empata con Q_{Cu_1} (en un cuarteto, los dos pares vecinos siempre empatan). Unimos A y B: $b_A = 0,25 + (1,2 - 1,8)/4 = 0,10$ y $b_B = 0,40$.

Paso 3 ($n = 3$). Quedan C, u_1 , u_2 con $D_{Cu_1} = 0,3$, $D_{Cu_2} = 0,3$ y $D_{u_1u_2} = 0,2$; la estrella final da ramas 0,2, 0,1 y 0,1.

El árbol obtenido coincide exactamente, topología y longitudes, con el verdadero (figura 5.7).

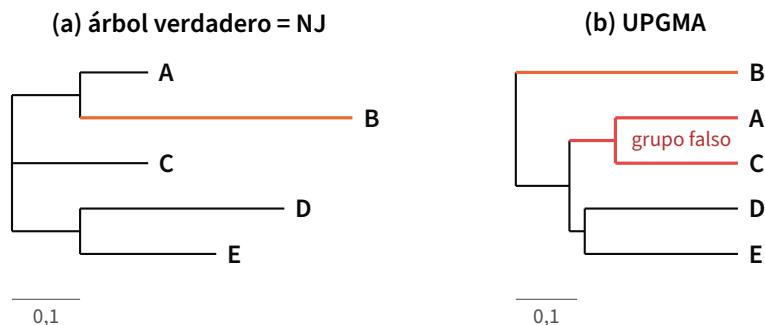


Figura 5.8. Cuando falla el reloj. (a) Árbol generador del ejemplo 5.2, dibujado desde su nodo central: B pertenece al linaje de A, pero su rama (naranja) es cuatro veces más larga. NJ lo recupera exactamente. (b) UPGMA sobre la misma matriz. Como $D_{AC} = 0,4$ es la menor distancia, UPGMA une primero A con C (rojo), un grupo que no existe en el árbol verdadero, y relega a B a la posición más externa, porque su rama larga lo aleja de todos. La hipótesis de ultrametricidad fuerza además a que todas las hojas terminen alineadas.

```
nombres = ["A", "B", "C", "D", "E"]
tri = [[0],
        [0.5, 0],
        [0.4, 0.7, 0],
        [0.6, 0.9, 0.6, 0],
        [0.5, 0.8, 0.5, 0.5, 0]] # triangular inferior
dm = DistanceMatrix(nombres, tri)
cons = DistanceTreeConstructor()
for metodo in (cons.upgma, cons.nj):
    arbol = metodo(dm)
    print(metodo.__name__)
    for clado in arbol.find_clades():
        if clado.branch_length:
            print(f" {clado.name:8s} {clado.branch_length:.3f}")
Phylo.draw_ascii(arbol)
```

⚠ Tres advertencias sobre los métodos de distancias

1. *Ramas negativas*: con datos reales, D nunca es exactamente aditiva, y la ecuación (5.19) puede dar longitudes negativas. Se suelen truncar a cero, pero son un síntoma de ruido o de un modelo de distancia inadecuado.

2. *La raíz no es un resultado*: el árbol de NJ es no enraizado. Dibujarlo con la raíz en el punto donde lo deja el programa sugiere relaciones ancestrales que el método no ha inferido.
3. *Información perdida*: al resumir el alineamiento en $\binom{n}{2}$ números, se descarta qué sitios concretos difieren. Los métodos basados en caracteres (sección 5.3) usan toda esa información, y por eso son estadísticamente más eficientes.

UPGMA supone un reloj; NJ solo supone aditividad.
 Cuando el reloj falla, el par más cercano no es necesariamente
 un par de hermanos.

5.3 Máxima verosimilitud y bootstrap



Colab · 5.3 · Máxima verosimilitud y bootstrap



abrir notebook

El detective que mide la sorpresa

Un buen detective no se pregunta primero «¿cuál es la historia más probable?», porque no tiene forma de asignar probabilidades a historias que nunca ha visto. Se pregunta algo más modesto y más poderoso: «si esta historia fuera cierta, ¿qué tan probable sería encontrar exactamente las huellas, las colillas y las coartadas que tengo delante?». La historia bajo la cual las pruebas resultan *menos sorprendentes* es su mejor candidata. Y si varias historias explican las pruebas casi igual de bien, un detective honesto lo dice: su confianza depende de cuánto cambiaría la conclusión con un poco menos de evidencia.

Esa es, en esencia, la filosofía de la máxima verosimilitud (ML, *maximum likelihood*), llevada a la filogenética por Felsenstein (1981). Las «historias» son árboles con longitudes de rama y un modelo de sustitución; las «pruebas» son las columnas del alineamiento. A diferencia de los métodos de distancias, ML usa todos los sitios individualmente y dispone de un marco estadístico completo para estimar

parámetros, comparar modelos y medir la incertidumbre. Los libros de Felsenstein (2004) y Yang (2014) desarrollan esta teoría con gran detalle.

5.3.1 La verosimilitud de un árbol

Sea X un alineamiento de n secuencias y N sitios, y sea x_h la columna h . Dado un árbol T con longitudes de rama $\mathbf{b}$ y un modelo con parámetros θ (por ejemplo, κ , π y α), la independencia entre sitios permite escribir:

Definición 5.5. (Verosimilitud filogenética)

$$L(T, \mathbf{b}, \theta) = \mathbb{P}(X \mid T, \mathbf{b}, \theta) = \prod_{h=1}^N \mathbb{P}(x_h \mid T, \mathbf{b}, \theta), \quad (5.21)$$

$$\ell = \ln L = \sum_{h=1}^N \ln \mathbb{P}(x_h \mid T, \mathbf{b}, \theta).$$

El estimador de máxima verosimilitud es la terna $(\hat{T}, \hat{\mathbf{b}}, \hat{\theta})$ que maximiza ℓ .

DONDE

X, x_h	Alineamiento completo y su columna h (el estado de cada hoja en ese sitio).
N	Número de sitios del alineamiento.
$T, \mathbf{b}$	Topología y vector de longitudes de rama ($2n - 3$ ramas en un árbol no enraizado).
θ	Parámetros del modelo de sustitución.
ℓ	Logaritmo de la verosimilitud, que se usa porque L es un número diminuto.

Observe que la verosimilitud *no* es la probabilidad de que el árbol sea correcto: es la probabilidad de los datos suponiendo que lo es. El trabajo consiste en calcular $\mathbb{P}(x_h \mid \cdot)$ para una columna. Tomemos el árbol de cuatro hojas de la figura 5.9, enraizado (arbitrariamente) en el nodo interno 5. Las hojas muestran los estados observados; los nodos internos 5 y 6 son ancestros cuyos estados desconocemos, así que sumamos sobre todas sus posibilidades:

$$\mathbb{P}(x_h) = \sum_{x_5} \sum_{x_6} \pi_{x_5} P_{x_5 x_1}(b_1) P_{x_5 x_2}(b_2) P_{x_5 x_6}(b_5) P_{x_6 x_3}(b_3) P_{x_6 x_4}(b_4). \quad (5.22)$$

DONDE

$x_1, \dots, x_4$	Estados observados en las hojas para el sitio h .
x_5, x_6	Estados desconocidos de los nodos internos; se suman los 16 casos.
π_{x_5}	Probabilidad del estado en la raíz (frecuencia de equilibrio).
$P_{xy}(b)$	Probabilidad de transición de x a y a lo largo de una rama de longitud b (ecuación (5.4)).

Cada término es la probabilidad de una historia completa del sitio; sumarlos todos da la probabilidad de lo observado. El problema es que, con n hojas, hay $n - 2$ nodos internos en un árbol no enraizado y 4^{n-2} historias por sitio: para 50 secuencias, unas 10^{29} . Necesitamos, otra vez, una idea más astuta que la fuerza bruta.

5.3.2 El algoritmo de poda de Felsenstein

La idea es la misma que en la programación dinámica del capítulo 3: reordenar las sumas para no repetir cálculos. En la ecuación (5.22), los factores que dependen de x_6 pueden agruparse en una suma interior, que se calcula una sola vez para cada valor de x_6 . Generalizando este reordenamiento a cualquier árbol, Felsenstein (1981) obtuvo un recorrido desde las hojas hacia la raíz, denominado *poda* (*pruning*) porque cada subárbol ya procesado se «corta» y se sustituye por un vector de cuatro números.

Teorema 5.6. (Algoritmo de poda)

Para cada nodo k y estado x , sea $L_k(x)$ la probabilidad de los estados observados en las hojas que descienden de k , condicionada a que k esté en el estado x . Entonces

$$L_k(x) = \begin{cases} 1 [x = x_k] & \text{si } k \text{ es una hoja,} \\ \prod_{c \in \text{hijos}(k)} \sum_y P_{xy}(b_c) L_c(y) & \text{si } k \text{ es interno,} \end{cases} \quad (5.23)$$

y la probabilidad del sitio es $\mathbb{P}(x_h) = \sum_x \pi_x L_{\text{raíz}}(x)$. El coste por sitio es $O(ns^2)$ en lugar de $O(s^{n-2})$.

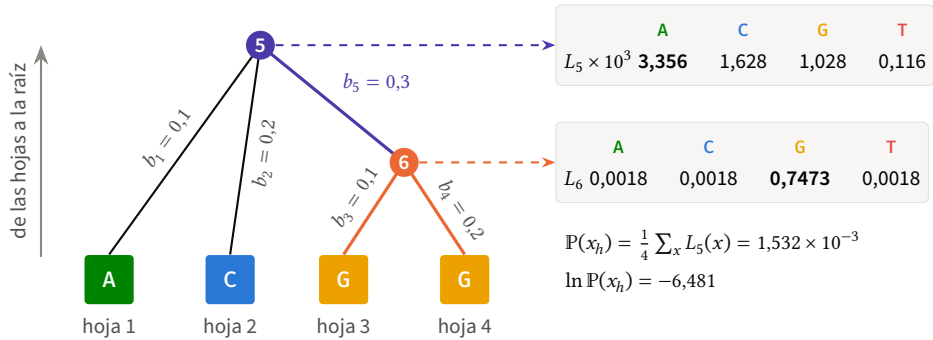


Figura 5.9. El algoritmo de poda para un sitio con patrón **ACGG** bajo JC69. El vector L_6 (naranja) resume el subárbol de las hojas 3 y 4: como ambas muestran **G**, el estado **G** en el nodo 6 es unas 400 veces más verosímil que cualquier otro. El vector L_5 (violeta) combina las hojas 1 y 2 con el subárbol ya podado, y su suma ponderada por $\pi = 1/4$ da la probabilidad del sitio. Todos los valores se calculan en `figuras/cap05/generar.py` y coinciden con la suma directa de las 16 historias de la ecuación (5.22).

DONDE	
$L_k(x)$	Verosimilitud parcial (o condicional) del subárbol que cuelga del nodo k .
b_c	Longitud de la rama que une k con su hijo c .
s	Número de estados: 4 para ADN, 20 para proteínas, 61 para codones.
$1[\cdot]$	Indicador; para una base ambigua (p. ej., $R = A$ o G) o un hueco se usa 1 en todos los estados compatibles.

Dos observaciones completan el cuadro. Primero, bajo un modelo reversible la verosimilitud no depende de dónde se coloque la raíz: es el llamado *principio de la polea* de Felsenstein, que justifica trabajar con árboles no enraizados. Segundo, los sitios con el mismo patrón de estados tienen la misma probabilidad, así que basta con calcularla una vez por patrón distinto y multiplicarla por su frecuencia. En el alineamiento simulado de 600 sitios que usaremos más adelante solo hay 118 patrones distintos.

Ejemplo 5.3. (Verosimilitud de un sitio, paso a paso)

Usemos JC69 con las ramas de la figura 5.9. Para JC, $P_{xx}(b) = \frac{1}{4} + \frac{3}{4}e^{-4b/3}$ y $P_{xy}(b) = \frac{1}{4} - \frac{1}{4}e^{-4b/3}$, de modo que la probabilidad de permanecer y la de cambiar a una base concreta valen 0,9064 y 0,0312 para $b = 0,1$; 0,8245 y 0,0585 para $b = 0,2$; y 0,7527 y 0,0824 para $b = 0,3$.

Nodo 6. Sus hijos son hojas con **G**, de modo que $L_6(x) = P_{xG}(0,1)P_{xG}(0,2)$. Para $x = \mathbf{G}$: $0,9064 \times 0,8245 = 0,7473$; para cualquier otra base: $0,0312 \times 0,0585 = 0,0018$.

Nodo 5. Para $x = \mathbf{A}$, el factor de la rama hacia 6 es $\sum_y P_{Ay}(0,3)L_6(y) = 0,7527 \cdot 0,0018 + 0,0824 \cdot (0,0018 + 0,7473 + 0,0018) = 0,0633$. Multiplicando por $P_{AA}(0,1) = 0,9064$ (hoja 1) y por $P_{AC}(0,2) = 0,0585$ (hoja 2) se obtiene $L_5(\mathbf{A}) = 3,356 \times 10^{-3}$. Análogamente, $L_5 = (3,356; 1,628; 1,028; 0,116) \times 10^{-3}$.

Raíz. $P(x_h) = \frac{1}{4} \sum_x L_5(x) = 1,532 \times 10^{-3}$, y $\ln P(x_h) = -6,481$. El estado más verosímil del nodo 5 es **A**, apoyado por la hoja 1, que cuelga de la rama más corta: la verosimilitud «pondera» la evidencia de cada hoja según la longitud de su rama, algo que ningún conteo de diferencias hace.

poda.py

```
import numpy as np
```

```
def P_jc(t: float) -> np.ndarray:
    """Matriz de transición de Jukes-Cantor para una rama t."""
    e = np.exp(-4 * t / 3)
    return np.full((4, 4), 0.25 - 0.25 * e) + e * np.eye(4)
```

```
def parcial(nodo, sitio: dict) -> np.ndarray:
    """Vector L_k(x) de verosimilitudes parciales (poda)."""
    nombre, hijos = nodo
    if not hijos:
        # hoja: indicador
        v = np.zeros(4)
        v["ACGT".index(sitio[nombre])] = 1.0
        return v
    v = np.ones(4)
    for hijo, t in hijos:
        # producto sobre hijos
        v *= P_jc(t) @ parcial(hijo, sitio)
    return v
```

```
hoja = lambda n: (n, [])
```

```

arbol = ("5", [(hoja("1"), 0.1), (hoja("2"), 0.2),
               ((("6", [(hoja("3"), 0.1), (hoja("4"), 0.2)]), 0.3)])
sitio = {"1": "A", "2": "C", "3": "G", "4": "G"}
L = 0.25 * parcial(arbol, sitio).sum() # pi = 1/4
print(f"L = {L:.4e}   lnL = {np.log(L):.4f}") # 1.5317e-03 -6.4814

```

5.3.3 Optimizar ramas, parámetros y topologías

Calcular ℓ para un árbol dado es la parte fácil. Maximizarla exige resolver tres problemas anidados. Las **longitudes de rama** se optimizan de una en una: al variar solo b_c , la verosimilitud es una función suave de una variable, y unas pocas iteraciones de Newton-Raphson bastan. Los **parámetros del modelo** (κ , α , r_{ij}) se optimizan de forma similar, alternando con las ramas. La **topología**, en cambio, es un objeto discreto que vive en el espacio astronómico de la tabla 5.1, y ahí solo cabe la búsqueda heurística: partir de un árbol razonable (a menudo, uno de NJ o de parsimonia) y mejorarlo con reordenamientos locales mientras la verosimilitud suba.

Los reordenamientos más comunes se muestran en la figura 5.10. El **intercambio de vecinos más cercanos** (NNI, *nearest neighbor interchange*) toma una rama interna, que separa cuatro subárboles, y prueba las dos formas alternativas de conectarlos; un árbol con n hojas tiene $n - 3$ ramas internas y, por lo tanto, $2(n - 3)$ vecinos NNI. La **poda y reinjerto** (SPR, *subtree pruning and regrafting*) corta un subárbol y lo vuelve a unir en cualquier otra rama, lo que permite saltos mucho mayores; la **bisección y reconexión** (TBR) generaliza SPR probando además todas las raíces posibles del subárbol cortado.

Los programas modernos combinan estas piezas con astucia. PhyML (Guindon & Gascuel, 2003) mostró que optimizar a la vez la topología (mediante NNI) y las longitudes de rama permitía analizar cientos de secuencias en minutos. RAXML (Stamatakis, 2014) se apoya en movimientos SPR con radio limitado y en implementaciones muy optimizadas para escalar a miles de taxones. IQ-TREE (Nguyen et al., 2015) introdujo una búsqueda estocástica que perturba aleatoriamente el mejor árbol encontrado y vuelve a optimizarlo con NNI, lo que ayuda a escapar de óptimos locales; su versión 2 (Minh et al., 2020) añadió modelos nuevos y métodos eficientes para datos genómicos. Ninguno garantiza encontrar el árbol de máxima verosimilitud global, y por eso se recomienda realizar varias búsquedas independientes y comparar sus resultados.

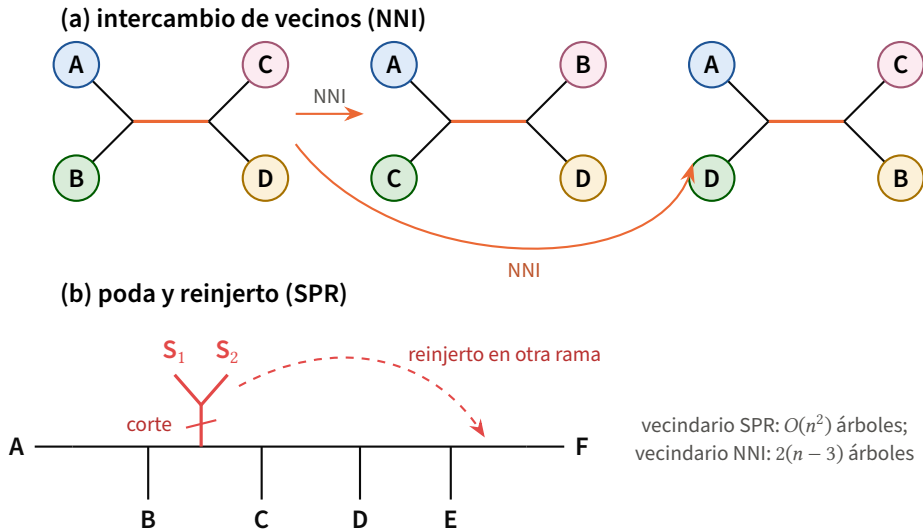


Figura 5.10. Movimientos de búsqueda en el espacio de topologías. (a) Una rama interna (naranja) separa cuatro subárboles; NNI prueba las dos formas alternativas de agruparlos ($AC | BD$ y $AD | BC$). Es un movimiento pequeño y barato, pero puede quedar atrapado en óptimos locales. (b) SPR corta un subárbol (rojo) y lo reinjerta en otra rama del árbol; su vecindario es mucho mayor y permite escapar de óptimos locales, a cambio de más evaluaciones de la verosimilitud.

5.3.4 Selección de modelos: razón de verosimilitudes, AIC y BIC

¿Qué modelo usar? Un modelo con más parámetros siempre alcanza una verosimilitud igual o mayor, porque contiene a los más sencillos como casos particulares; la pregunta es si la mejora compensa la complejidad añadida. Para modelos anidados, como JC69 dentro de K80, la *prueba de razón de verosimilitudes* compara $2\Delta\ell = 2(\ell_1 - \ell_0)$ con una distribución χ^2 con tantos grados de libertad como parámetros adicionales. Para comparar muchos modelos, anidados o no, se usan criterios de información:

$$AIC = 2k - 2\hat{\ell}, \quad BIC = k \ln N - 2\hat{\ell}, \quad (5.24)$$

Tabla 5.4. Selección de modelos para un alineamiento de $N = 600$ sitios simulado bajo HKY85+ Γ ($\kappa = 4$, $\alpha = 0,5$) en el árbol de la figura 5.13. Todos los modelos se ajustaron sobre la misma topología (la de NJ); k incluye las 9 ramas. Δ es la diferencia con el mejor valor de cada criterio. Calculado con el motor de verosimilitud de `figuras/cap05/generar.py`.

Modelo	k	$\hat{\ell}$	AIC	Δ AIC	BIC	Δ BIC
JC	9	-2095,09	4208,2	135,0	4247,8	113,0
K8o	10	-2055,63	4131,3	58,1	4175,2	40,5
F81	12	-2084,13	4192,3	119,1	4245,0	110,3
HKY	13	-2042,54	4111,1	37,9	4168,2	33,5
HKY+ Γ	14	-2022,59	4073,2	0,0	4134,7	0,0
GTR+ Γ	18	-2020,78	4077,6	4,4	4156,7	22,0

DONDE	
$\hat{\ell}$	Logaritmo de la verosimilitud maximizada del modelo.
k	Número de parámetros libres, <i>incluidas</i> las $2n - 3$ longitudes de rama.
N	Número de sitios del alineamiento (tamaño de la muestra).

y se elige el modelo con el valor más bajo. El AIC de Akaike (1974) estima la capacidad predictiva del modelo sobre datos nuevos; el BIC de Schwarz (1978) aproxima la probabilidad posterior del modelo y, como su penalización crece con $\ln N$, favorece modelos más sencillos cuando hay muchos sitios. ModelFinder (Kalyaanamoorthy et al., 2017), integrado en IQ-TREE, automatiza este proceso sobre cientos de combinaciones de matrices, frecuencias y modelos de heterogeneidad (incluidos los de tasas libres) y, por defecto, elige según el BIC.

Ejemplo 5.4. (Elegir el modelo con datos simulados)

La tabla 5.4 permite seguir el razonamiento completo. Pasar de JC69 a K8o añade un parámetro (κ) y aumenta $\hat{\ell}$ en 39,5 unidades: $2\Delta\ell = 78,9$, con $p \approx 6 \times 10^{-19}$ para una χ^2_1 . La relación transición/transversión es real. Pasar de HKY85 a HKY85+ Γ añade α y gana 19,95 unidades ($2\Delta\ell = 39,9$, $p \approx 3 \times 10^{-10}$): la heterogeneidad de tasas también es real. En cambio, pasar de HKY85+ Γ a GTR+ Γ añade cuatro parámetros y apenas gana 1,81 unidades ($2\Delta\ell = 3,6$,

$p = 0,46$ para una χ^2_4): las seis tasas libres no aportan nada que κ no explique ya. Los dos criterios de información coinciden y eligen HKY85+ Γ , precisamente el modelo con que se simularon los datos; el BIC penaliza a GTR+ Γ con más dureza ($\Delta\text{BIC} = 22,0$) que el AIC ($\Delta\text{AIC} = 4,4$), porque $\ln 600 \approx 6,4 > 2$. Las estimaciones, $\hat{\kappa} = 3,71$ y $\hat{\alpha} = 0,57$, están cerca de los valores verdaderos (4 y 0,5).

5.3.5 Un contraste: parsimonia y atracción de ramas largas

Antes de la verosimilitud, el criterio dominante para datos de caracteres era la *máxima parsimonia*: preferir el árbol que explica los datos con el menor número de cambios. Fitch (1971) dio un algoritmo eficiente para contar ese mínimo en un árbol dado. En cada nodo interno se calcula un conjunto de estados posibles: si los conjuntos de los dos hijos se intersecan, el nodo recibe la intersección; si no, recibe su unión y se contabiliza un cambio. La parsimonia no necesita modelo, es rápida y funciona bien cuando las ramas son cortas. Pero precisamente por no tener modelo, no puede corregir los impactos múltiples.

Felsenstein (1978) demostró que existen árboles para los que la parsimonia es *estadísticamente inconsistente*: converge al árbol equivocado a medida que se añaden datos. El caso típico es el de la figura 5.11: dos ramas largas no hermanas (1 y 3) y ramas cortas en el resto. En las ramas largas se acumulan tantos cambios que, por puro azar, 1 y 3 coinciden a menudo en el mismo estado derivado. La parsimonia interpreta esas coincidencias como un único cambio compartido y agrupa a 1 con 3. Este fenómeno, la *atracción de ramas largas*, sigue siendo uno de los errores sistemáticos más importantes de la filogenética. La verosimilitud con un modelo adecuado es consistente en esa misma región, porque «sabe» que en ramas largas son esperables las convergencias; pero un modelo inadecuado (por ejemplo, ignorar la heterogeneidad de tasas) puede reintroducir el problema.

5.3.6 ¿Cuánta confianza? El bootstrap

Un árbol de máxima verosimilitud es una estimación puntual; necesitamos saber qué partes de él son sólidas. La herramienta más usada es el *bootstrap no paramétrico* de Efron (1979), cuya idea es simular la variabilidad de muestreo usando los propios datos como sustituto de la población. Felsenstein (1985) lo llevó a la filogenética tratando las columnas del alineamiento como la muestra:

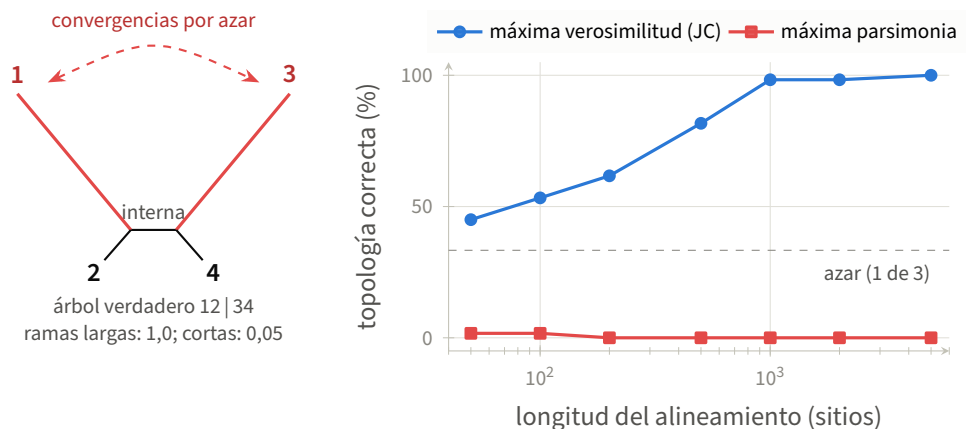


Figura 5.11. La atracción de ramas largas en la «zona de Felsenstein». **Izquierda:** árbol generador con dos ramas largas no hermanas (rojo). **Derecha:** proporción de 60 réplicas simuladas bajo JC69 en que cada método recupera la topología correcta. Con más datos, la máxima verosimilitud converge al árbol verdadero (100 % con 5000 sitios), mientras que la parsimonia converge con seguridad creciente al árbol *equivocado* 13 | 24, y lo hace por debajo incluso del 33 % que daría elegir al azar.

1. Construya un alineamiento réplica eligiendo N columnas *con reemplazo* del original (algunas aparecerán repetidas y otras faltarán; figura 5.12).
2. Infiera un árbol a partir de la réplica con el mismo método que usó para los datos originales.
3. Repita B veces (típicamente entre 100 y 1000) y registre, para cada bipartición del árbol original, la fracción de réplicas en que aparece.

Definición 5.6. (Soporte bootstrap)

El soporte bootstrap de una bipartición S es

$$\text{BS}(S) = \frac{1}{B} \sum_{b=1}^B \mathbf{1}[S \in \hat{T}_b^*] \times 100 \%. \quad (5.25)$$

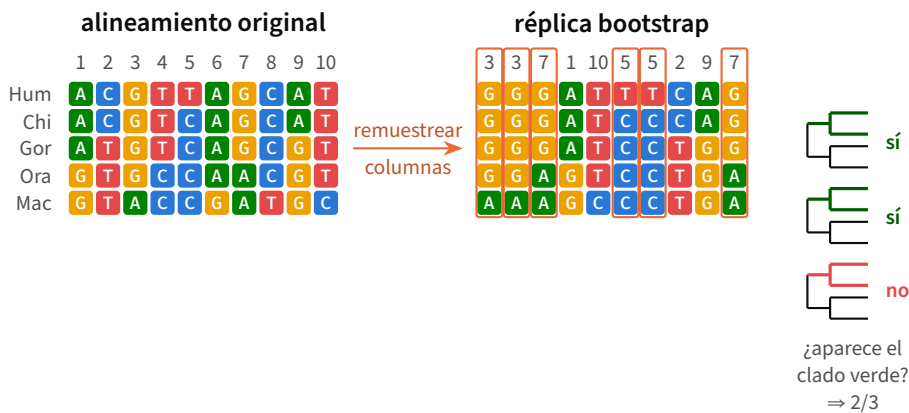


Figura 5.12. El bootstrap no paramétrico. Cada réplica se construye eligiendo columnas al azar, con reemplazo, del alineamiento original (los números indican qué columna se copió; las recuadradas en naranja aparecen más de una vez, y las columnas 4, 6 y 8 no aparecen). A partir de cada réplica se infiere un árbol, y el soporte de un clado es la fracción de réplicas en que ese clado reaparece (derecha: presente en dos de tres árboles). La variación entre réplicas imita la que habría entre alineamientos independientes del mismo tamaño.

DONDE

B	Número de réplicas bootstrap.
$\hat{T}_b^*$	Árbol estimado a partir de la réplica b .
$S \in \hat{T}_b^*$	La bipartición S está presente en ese árbol.

La figura 5.13 muestra el resultado para el alineamiento simulado de seis primates de la tabla 5.4. Los soportes deben leerse con cuidado. Un valor del 95 % no significa que el clado tenga un 95 % de probabilidad de ser verdadero: el bootstrap mide cuán *estable* es la inferencia frente a la variación de muestreo, y en la práctica suele ser conservador para clados verdaderos (Felsenstein, 2004). Además, el bootstrap estándar exige repetir la búsqueda completa B veces, lo que es prohibitivo para alineamientos grandes. El *bootstrap ultrarrápido* (UFBoot) de IQ-TREE reutiliza los árboles visitados durante la búsqueda y evalúa las réplicas por remuestreo de las verosimilitudes de cada sitio (RELL), con un coste muy inferior (Hoang et al., 2018). Sus valores no son equivalentes a los del bootstrap estándar: los autores recomiendan considerar fiables los clados con UFBoot $\geq 95\%$, mientras que para el bootstrap clásico suele usarse un umbral más bajo, del orden del 70 %.

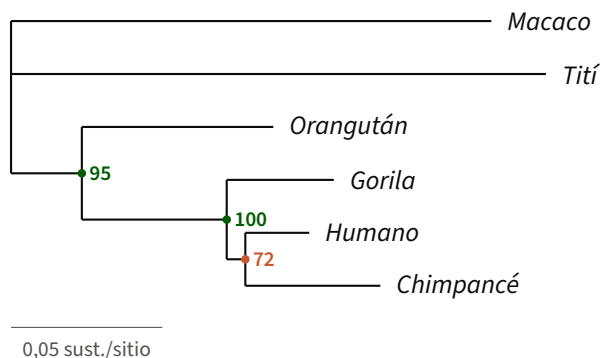


Figura 5.13. Árbol de seis primates inferido a partir de 600 sitios simulados bajo HKY85+ Γ sobre un árbol conocido. La topología es la de NJ (distancias JC) y coincide con la verdadera; las longitudes de rama se reoptimizaron por máxima verosimilitud con el modelo elegido (HKY85+ Γ). Los números son soportes bootstrap sobre 1000 réplicas (verde: ≥ 95 ; naranja: 70–94). El clado humano-chimpancé recibe solo un 72 %: la rama que lo define mide 0,008 sustituciones por sitio en el árbol verdadero, lo que equivale a unos 5 cambios esperados en todo el alineamiento, muy poca evidencia. El árbol se dibuja sin raíz explícita, con Macaco y Tití en la base.

Ejemplo 5.5. (Leer el soporte de la figura 5.13)

El clado humano-chimpancé-gorila aparece en el 100 % de las réplicas y el que añade al orangután, en el 95 %: sus ramas definitorias miden 0,03 y 0,06 sustituciones por sitio en el árbol generador, es decir, unas 18 y 36 sustituciones esperadas en 600 sitios. En cambio, el clado humano-chimpancé (72 %) se apoya en una rama de 0,008: en un 28 % de las réplicas, el azar del remuestreo reúne a gorila con uno de los dos. Sin embargo, la topología estimada es correcta. La lección es general: un soporte moderado no indica que el clado sea falso, sino que *estos datos* contienen poca información sobre él.

En la práctica, selección de modelo, búsqueda de ML y bootstrap se ejecutan con una sola orden: en IQ-TREE 2, -m MFP activa ModelFinder, -B pide réplicas UFBoot y -alrt añade la prueba SH-aLRT; en RAXML 8, -f a combina bootstrap rápido y búsqueda de ML.

Inferencia de máxima verosimilitud en la terminal

```
# IQ-TREE 2: selección de modelo + árbol ML + UFBoot + SH-aLRT
iqtree2 -s primates.fasta -m MFP -B 1000 -alrt 1000 -T AUTO
# salida: primates.fasta.iqtree (informe), .treefile (árbol)

# RAXML 8: 100 réplicas bootstrap rápidas + búsqueda ML (GTR+G)
raxmlHPC -f a -s primates.phy -n prim -m GTRGAMMA \
-p 12345 -x 12345 -N 100
```

⚠ Lo que el bootstrap no puede detectar

El bootstrap cuantifica el error *aleatorio*, el que desaparece con más datos. No detecta el error *sistemático*. En la zona de Felsenstein, la parsimonia agrupa las ramas largas con soporte cercano al 100 % (figura 5.11), y un modelo inadecuado puede hacer lo mismo con la verosimilitud.

La verosimilitud pregunta qué tan probable es lo observado bajo cada árbol.
El bootstrap pregunta cuánto cambiaría la respuesta con otra muestra de sitios.

☰ En resumen

- Un árbol filogenético no enraizado con n hojas es una de $(2n - 5)!!$ topologías posibles; para 50 taxones hay unas 10^{74} , por lo que la inferencia exige algoritmos constructivos o búsquedas heurísticas.
- La distancia p se satura por los impactos múltiples. Los modelos de sustitución, cadenas de Márkov con $P(t) = e^{Qt}$, corrigen ese sesgo: JC69 da $d = -\frac{3}{4} \ln(1 - \frac{4}{3}p)$; K80, F81, HKY85 y GTR añaden sesgo transición/transversión y composición desigual; + Γ modela la variación de tasas entre sitios.
- UPGMA supone un reloj molecular (distancias ultramétricas) y falla cuando los linajes evolucionan a ritmos distintos. Neighbor-Joining solo requiere aditividad (condición de cuatro puntos) y recupera exactamente el árbol cuando las distancias son aditivas.

- La máxima verosimilitud evalúa cada árbol por la probabilidad de todo el alineamiento; el algoritmo de poda de Felsenstein la calcula en tiempo lineal en el número de taxones, y la búsqueda de topologías usa NNI, SPR o TBR.
- Los modelos se comparan con razón de verosimilitudes, AIC y BIC. La parsimonia puede ser inconsistente (atracción de ramas largas); ML con un modelo adecuado no lo es. El bootstrap y UFBoot miden la estabilidad de cada clado, no los errores sistemáticos.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 5.1** ●●● Dibuje las 15 topologías no enraizadas posibles para cinco taxones. Compruebe que su número coincide con la ecuación (5.1) y explique por qué cada una puede enraizarse de 7 formas.
- 5.2** ●●● Dos secuencias de 300 pb difieren en 90 sitios. Calcule $\hat{p}$, $\hat{d}_{JC}$ y su error estándar. ¿Qué ocurre con la distancia si difieren en 230 sitios?
- 5.3** ●●● Compruebe que la matriz de distancias de la tabla 5.3 *no* es ultramétrica encontrando una terna que viole la ecuación (5.15).
- 5.4** ●●● A partir de la ecuación de Kolmogorov $P'(t) = P(t)Q$, derive $P_{ij}(t)$ para el modelo K80 y demuestre la ecuación (5.10). (Pista: considere por separado la probabilidad de terminar con una transición y con una transversión.)
- 5.5** ●●● Verifique numéricamente con `scipy.linalg.expm` que, para una matriz GTR construida con la ecuación (5.12), se cumplen $\pi P(t) = \pi$ y $\pi_i P_{ij}(t) = \pi_j P_{ji}(t)$ para cualquier t .
- 5.6** ●●● Amplíe el programa `poda.py` para calcular el logaritmo de la verosimilitud de un alineamiento completo sumando sobre sitios, y optimice la longitud de una rama con `scipy.optimize.minimize_scalar`.
- 5.7** ●●● Implemente el algoritmo de Fitch y úselo, junto con su función de verosimilitud del ejercicio anterior, para reproducir la figura 5.11. ¿A partir de qué cociente entre ramas largas y cortas empieza a fallar la parsimonia con 1000 sitios?

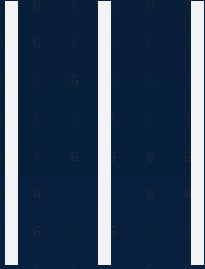
- 5.8 ●●● Descargue 10–15 secuencias del gen mitocondrial *COI* de especies de un mismo orden, alinéelas y ejecute IQ-TREE con `-m MFP -B 1000`. Compare el modelo elegido por el BIC con el elegido por el AIC y discuta qué clados reciben UFBoot < 95 % y por qué.

Referencias del capítulo

- Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716-723. <https://doi.org/10.1109/TAC.1974.1100705>
- Buneman, P. (1974). A note on the metric properties of trees. *Journal of Combinatorial Theory, Series B*, 17(1), 48-50. [https://doi.org/10.1016/0095-8956\(74\)90047-1](https://doi.org/10.1016/0095-8956(74)90047-1)
- Darwin, C. (2009). *The Origin of Species*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511694295> (Fecha inicial de publicación 1859)
- Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1-26. <https://doi.org/10.1214/aos/1176344552>
- Felsenstein, J. (1978). Cases in which parsimony or compatibility methods will be positively misleading. *Systematic Zoology*, 27(4), 401-410. <https://doi.org/10.2307/2412923>
- Felsenstein, J. (1981). Evolutionary trees from DNA sequences: A maximum likelihood approach. *Journal of Molecular Evolution*, 17(6), 368-376. <https://doi.org/10.1007/BF01734359>
- Felsenstein, J. (1985). Confidence limits on phylogenies: An approach using the bootstrap. *Evolution*, 39(4), 783-791. <https://doi.org/10.1111/j.1558-5646.1985.tb00420.x>
- Felsenstein, J. (2004). *Inferring Phylogenies*. Sinauer Associates.
- Fitch, W. M. (1971). Toward defining the course of evolution: Minimum change for a specific tree topology. *Systematic Zoology*, 20(4), 406-416. <https://doi.org/10.2307/2412116>
- Gascuel, O., & Steel, M. (2006). Neighbor-joining revealed. *Molecular Biology and Evolution*, 23(11), 1997-2000. <https://doi.org/10.1093/molbev/msl072>
- Guindon, S., & Gascuel, O. (2003). A simple, fast, and accurate algorithm to estimate large phylogenies by maximum likelihood. *Systematic Biology*, 52(5), 696-704. <https://doi.org/10.1080/10635150390235520>
- Hasegawa, M., Kishino, H., & Yano, T.-a. (1985). Dating of the human-ape splitting by a molecular clock of mitochondrial DNA. *Journal of Molecular Evolution*, 22(2), 160-174. <https://doi.org/10.1007/BF02101694>
- Hoang, D. T., Chernomor, O., von Haeseler, A., Minh, B. Q., & Vinh, L. S. (2018). UFBoot2: Improving the ultrafast bootstrap approximation. *Molecular Biology and Evolution*, 35(2), 518-522. <https://doi.org/10.1093/molbev/msx281>

- Jukes, T. H., & Cantor, C. R. (1969). Evolution of protein molecules. En H. N. Munro (Ed.), *Mammalian Protein Metabolism* (pp. 21-132). Academic Press. <https://doi.org/10.1016/B978-1-4832-3211-9.50009-7>
- Kalyaanamoorthy, S., Minh, B. Q., Wong, T. K. F., von Haeseler, A., & Jermini, L. S. (2017). ModelFinder: Fast model selection for accurate phylogenetic estimates. *Nature Methods*, 14(6), 587-589. <https://doi.org/10.1038/nmeth.4285>
- Kimura, M. (1980). A simple method for estimating evolutionary rates of base substitutions through comparative studies of nucleotide sequences. *Journal of Molecular Evolution*, 16(2), 111-120. <https://doi.org/10.1007/BF01731581>
- Minh, B. Q., Schmidt, H. A., Chernomor, O., Schrempf, D., Woodhams, M. D., von Haeseler, A., & Lanfear, R. (2020). IQ-TREE 2: New models and efficient methods for phylogenetic inference in the genomic era. *Molecular Biology and Evolution*, 37(5), 1530-1534. <https://doi.org/10.1093/molbev/msaa015>
- Nguyen, L.-T., Schmidt, H. A., von Haeseler, A., & Minh, B. Q. (2015). IQ-TREE: A fast and effective stochastic algorithm for estimating maximum-likelihood phylogenies. *Molecular Biology and Evolution*, 32(1), 268-274. <https://doi.org/10.1093/molbev/msu300>
- Rodríguez, F., Oliver, J. L., Marín, A., & Medina, J. R. (1990). The general stochastic model of nucleotide substitution. *Journal of Theoretical Biology*, 142(4), 485-501. [https://doi.org/10.1016/S0022-5193\(05\)80104-3](https://doi.org/10.1016/S0022-5193(05)80104-3)
- Saitou, N., & Nei, M. (1987). The neighbor-joining method: A new method for reconstructing phylogenetic trees. *Molecular Biology and Evolution*, 4(4), 406-425. <https://doi.org/10.1093/oxfordjournals.molbev.a040454>
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461-464. <https://doi.org/10.1214/aos/1176344136>
- Stamatakis, A. (2014). RAxML version 8: A tool for phylogenetic analysis and post-analysis of large phylogenies. *Bioinformatics*, 30(9), 1312-1313. <https://doi.org/10.1093/bioinformatics/btu033>
- Studier, J. A., & Keppler, K. J. (1988). A note on the neighbor-joining algorithm of Saitou and Nei. *Molecular Biology and Evolution*, 5(6), 729-731. <https://doi.org/10.1093/oxfordjournals.molbev.a040527>
- Yang, Z. (1994). Maximum likelihood phylogenetic estimation from DNA sequences with variable rates over sites: Approximate methods. *Journal of Molecular Evolution*, 39(3), 306-314. <https://doi.org/10.1007/BF00160154>
- Yang, Z. (2014). *Molecular Evolution: A Statistical Approach*. Oxford University Press. <https://doi.org/10.1093/acprof:oso/9780199602605.001.0001>
- Zuckerkandl, E., & Pauling, L. (1965). Evolutionary divergence and convergence in proteins. En V. Bryson & H. J. Vogel (Eds.), *Evolving Genes and Proteins* (pp. 97-166). Academic Press. <https://doi.org/10.1016/B978-1-4832-2734-4.50017-6>

PARTE

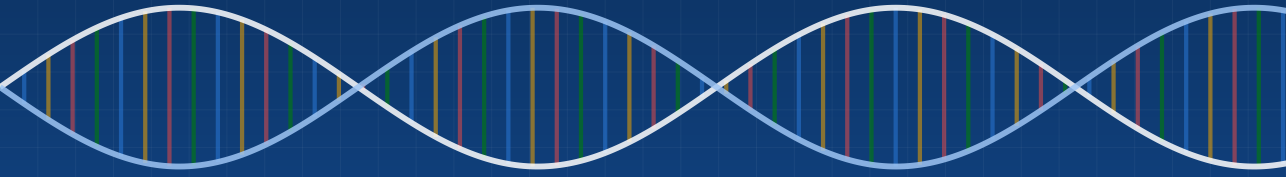


Del secuenciador al genoma

Lecturas, mapeo, ensamblaje y variantes

Los secuenciadores producen millones de fragmentos imperfectos. Esta parte sigue su recorrido completo: la calidad de cada base, su ubicación en una referencia, la reconstrucción de genomas desconocidos y la detección de las diferencias que nos hacen únicos.

Secuenciación de nueva generación (NGS)



Predecimos que, en la perspectiva larga de la historia, el impacto de la secuenciación de ADN estará a la par con el del microscopio.

— Shendure et al. (2017)

● Al terminar este capítulo usted podrá

- Explicar la química y la física de las principales plataformas de secuenciación (Sanger, 454, Illumina, PacBio y Oxford Nanopore) y relacionar cada mecanismo con su perfil de errores característico.
- Traducir una puntuación de calidad Phred en una probabilidad de error, decodificar la línea de calidades de un archivo FASTQ y calcular el número esperado de errores de una lectura.
- Leer un informe de control de calidad (FastQC/MultiQC) y decidir, con criterio, cómo recortar adaptadores y extremos de baja calidad.
- Derivar la teoría de Lander-Waterman (cobertura de Poisson, fracción no cubierta, número de *contigs*) y usarla, junto con modelos sobredispersos, para diseñar la profundidad de un experimento.

En 2001, secuenciar un genoma humano costaba cerca de cien millones de dólares; dos décadas después, unos pocos cientos (National Human Genome Research Institute, 2022). Esa caída abrió aplicaciones muy distintas: contar transcritos, localizar proteínas unidas al ADN, detectar patógenos y estudiar poblaciones. El secuenciador, sin embargo, no entrega un genoma completo. Produce millones de fragmentos cortos, o miles de fragmentos largos, cada uno con incertidumbre. El mapeo, el llamado de variantes, el ensamblaje y la cuantificación de expresión parten de esas lecturas. Comprender sus errores es necesario para interpretar los resultados.

El capítulo recorre el camino que va de la molécula al archivo. Primero veremos cómo funcionan las plataformas y por qué cada una se equivoca a su manera (sección 6.1). Luego aprenderemos el lenguaje con que la máquina expresa su confianza, la calidad Phred, y cómo usarlo para inspeccionar y limpiar las lecturas (sección 6.2). Por último, preguntaremos cuántas lecturas hacen falta para “ver” un genoma completo, una pregunta que Lander y Waterman (1988) respondieron con una elegante teoría probabilística que sigue guiando el diseño de cada experimento (sección 6.3).

6.1 Tecnologías de secuenciación

 Colab · 6.1 · Tecnologías de secuenciación

 abrir notebook

Un estadio que se fotografía a sí mismo

Piense en esas coreografías de las tribunas en las que cada espectador levanta una cartulina de color a una señal. Suponga ahora que en cada asiento hay una persona con un mazo de cuatro colores y una lista secreta, y que desde el techo del estadio una cámara toma una foto después de cada señal. Tras cien señales tendremos cien fotografías; si seguimos un mismo asiento a través de todas ellas, leeremos su lista secreta, color por color. Nadie tuvo que leer las listas una por una: bastó sincronizar a miles de personas y fotografiarlas a la vez. Esta es, en esencia, la idea de la secuenciación masiva en paralelo. Y ya se adivinan sus defectos: si alguien se retrasa una señal, sus colores se desfasan; si la foto está borrosa, los asientos vecinos se confunden; y cuanto más dura el espectáculo, más espectadores se distraen.

6.1.1 Leer una molécula: el método de Sanger

Todas las tecnologías de secuenciación resuelven el mismo problema: convertir una sucesión de bases químicamente casi idénticas en una señal física medible (una banda en un gel, un destello de luz, una variación de corriente) que pueda asignarse a una posición. La primera solución práctica, y la que dominó durante casi treinta años, fue la de Sanger et al. (1977). Su idea fue usar a la propia ADN polimerasa como “lectora”. La enzima copia una hebra molde a partir de un cebador, pero en la mezcla se añade, junto a los desoxinucleótidos normales (dNTP), una pequeña proporción de *didesoxinucleótidos* (ddNTP), que carecen del grupo hidroxilo 3' necesario para unir la siguiente base. Cada vez que la polimerasa incorpora un ddNTP, la síntesis se detiene. El resultado es una colección de fragmentos que terminan en cada posición donde aparece la base correspondiente; separándolos por tamaño con resolución de una base, se lee la secuencia.

Definición 6.1. (Probabilidad de terminación)

Sea r la probabilidad de que, frente a una posición del molde que admite la base b , la polimerasa incorpore el ddNTP en lugar del dNTP. Si las incorporaciones son independientes, la probabilidad de que la síntesis termine exactamente en la k -ésima aparición de b es geométrica:

$$\mathbb{P}(K = k) = (1 - r)^{k-1} r, \quad k = 1, 2, \dots \quad (6.1)$$

DONDE

r	Probabilidad de terminación por aparición; depende del cociente de concentraciones ddNTP/dNTP y de la preferencia de la enzima.
K	Número de apariciones de la base b que la polimerasa “atraviesa” hasta terminar (incluida la última).
k	Valor concreto de K .

La ecuación (6.1) explica de inmediato la limitación más conocida del método. Si una base aparece, en promedio, una vez cada cuatro posiciones, la señal de los fragmentos que terminan cerca de la posición ℓ decae aproximadamente como $(1 - r)^{\ell/4}$. Con $r = 0,005$, a las 400 bases queda el 61 % de la intensidad inicial y a las 800 bases solo el 37 %; con $r = 0,01$, la señal a 800 bases cae al 13 %. Aumentar r produce más fragmentos cortos (más señal al principio); disminuirlo alarga la lectura pero debilita todas las bandas. A esto se suma que la electroforesis separa

cada vez peor un fragmento de n bases de otro de $n + 1$ a medida que n crece. Por ambas razones, las lecturas de Sanger se quedaron en el rango de 400 a 800 pares de bases en el uso rutinario, con máximos cercanos a 1000 en capilares optimizados (Bentley et al., 2008), largas para los estándares de la época pero imposibles de paralelizar a gran escala: cada capilar leía un único fragmento.

El genoma que se leyó a mano

Frederick Sanger recibió dos premios Nobel de Química: uno por secuenciar la insulina y otro, en 1980, por el método de terminación de cadena. En su artículo de 1977 lo aplicó al bacteriófago ϕX_{174} y señaló que era más rápido y exacto que sus métodos anteriores (Sanger et al., 1977). Automatizado con fluorocromos y electroforesis capilar, el método de Sanger produjo la primera secuencia del genoma humano. Para dar a cada base una medida objetiva de confianza nació el programa [phred](#) (Ewing et al., 1998), cuya escala de calidad estudiaremos en la sección 6.2. Shendure et al. (2017) ofrecen una crónica de estos cuarenta años.

6.1.2 Paralelizar: pirosecuenciación y la plataforma 454

La ruptura conceptual llegó al abandonar la electroforesis. En lugar de separar fragmentos, se puede *observar la síntesis mientras ocurre*. Ronaghi et al. (1998) describieron la *pirosecuenciación*: cada vez que la polimerasa incorpora un nucleótido, libera una molécula de pirofosfato (PP_i); una cascada enzimática convierte ese pirofosfato en ATP (ATP sulfurilasa) y el ATP en un destello de luz (luciferasa). Si se inyecta en la cámara de reacción un solo tipo de nucleótido a la vez, un destello indica que esa base es la siguiente del molde.

Margulies et al. (2005) llevaron esta química a una escala sin precedentes. Cada fragmento de ADN se amplificaba clonalmente sobre una microesfera dentro de una gota de emulsión agua-aceite (PCR en emulsión), y las esferas se depositaban en una placa de fibra óptica con cientos de miles de pocillos de picolitros. En un único experimento de cuatro horas el sistema leía 25 millones de bases con una exactitud del 99 % o superior, y con él se ensambló *de novo* el genoma de *Mycoplasma genitalium*. Había nacido la “nueva generación”.

La pirosecuenciación tiene un talón de Aquiles que se deduce de su propio principio. Si el molde contiene un homopolímero, por ejemplo [AAAAAA](#), la polimerasa incorpora las seis bases en un único flujo y la luz emitida es, idealmente, proporcional a seis. Distinguir seis de siete a partir de una intensidad luminosa es mucho más difícil que distinguir una de dos, porque el ruido de la medición crece con la señal. Por eso el error dominante de 454 (y de otras químicas sin terminadores, como Ion

Torrent) no son las sustituciones sino las *inserciones y deleciones en homopolímeros* (Metzker, 2010).

6.1.3 Illumina: síntesis con terminadores reversibles

La tecnología que terminó por dominar el mercado combina dos ideas: amplificar cada molécula *en su sitio*, sobre una superficie de vidrio, y leerla una base por ciclo con nucleótidos que se detienen *de forma reversible* (Bentley et al., 2008).

Preparación de la biblioteca. El ADN se fragmenta (por ultrasonido o enzimáticamente) hasta tamaños de unos cientos de pares de bases, y a cada fragmento se le ligan *adaptadores*: secuencias sintéticas conocidas que contienen los sitios de unión a la celda de flujo, los sitios de los cebadores de secuenciación y, a menudo, *índices* (*barcodes*) que identifican la muestra y permiten mezclar decenas de muestras en una misma corrida (*multiplexing*). La parte biológica del fragmento, entre los dos adaptadores, se llama *inserto*.

Amplificación en puente. La celda de flujo está tapizada de dos tipos de oligonucleótidos complementarios a los adaptadores. Un fragmento monocatenario se hibrida a uno de ellos, la polimerasa lo copia, el original se lava, y la copia anclada se dobla para hibridarse con un oligonucleótido vecino del otro tipo, formando un “puente”. Repetido decenas de veces, este ciclo produce un *clúster*: un grupo compacto de unas mil copias idénticas de la molécula original, lo bastante brillante para ser fotografiado (figura 6.1). La química de anclaje permite densidades de unos diez millones de colonias por centímetro cuadrado (Fedurco et al., 2006).

El ciclo de secuenciación. Cada clúster se secuencia por síntesis (SBS) con cuatro nucleótidos modificados de dos maneras: llevan un fluorocromo distinto por base y un *grupo bloqueador* en el extremo 3' que impide que la polimerasa añada una segunda base. Un ciclo consta de cuatro pasos (figura 6.2): (1) se inyectan los cuatro nucleótidos y la polimerasa incorpora exactamente uno en cada hebra; (2) se lava el exceso y se excita la celda con láser; una cámara registra el color de cada clúster; (3) se escinden químicamente el fluorocromo y el bloqueador, restaurando un 3'-OH libre; (4) se repite. Como los cuatro nucleótidos compiten a la vez, los homopolímeros ya no son un problema: el error típico de Illumina es la *sustitución*, no el *indel*.

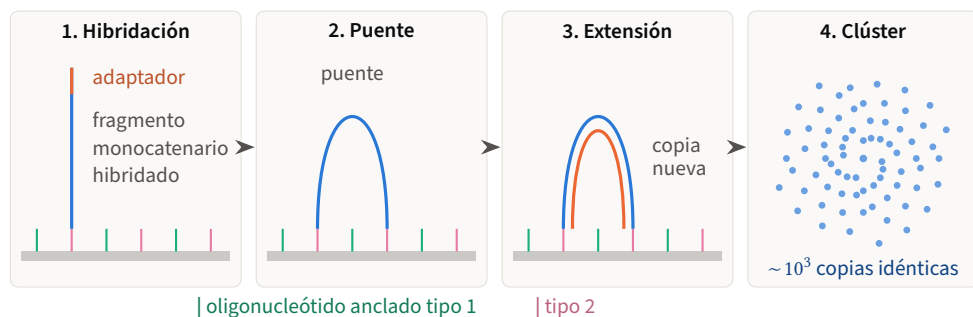


Figura 6.1. Amplificación en puente sobre la celda de flujo. (1) Un fragmento de la biblioteca, flanqueado por adaptadores, se hibrida con un oligonucleótido anclado. (2) La copia sintetizada se dobla y se hibrida con un oligonucleótido vecino del otro tipo. (3) La polimerasa la extiende y forma un puente de doble cadena, que se desnaturaliza para empezar otra ronda. (4) Tras unas treinta rondas, cada molécula original ha dado lugar a un clúster de alrededor de mil copias, visto aquí desde arriba. La amplificación convierte una molécula invisible en un punto luminoso, pero también introduce sesgos (sección 6.3).

6.1.4 Por qué la calidad cae a lo largo de la lectura: fase y desfase

Si todas las hebras de un clúster avanzaran en perfecta sincronía, la calidad sería la misma en el ciclo 1 que en el 150. No es así, y la razón es instructiva. En cada ciclo, una pequeña fracción de las hebras no incorpora ninguna base (por ejemplo, porque la escisión del bloqueador falló o la polimerasa no actuó): quedan *retrasadas* una posición (*phasing*). Otra fracción incorpora dos bases, porque algún nucleótido carecía de bloqueador: quedan *adelantadas* (*pre-phasing*). Las hebras desfasadas siguen emitiendo luz, pero del color equivocado, y contaminan la señal del clúster.

Teorema 6.1. (Decaimiento de la fracción en fase)

Sea $\varepsilon = p + q$ la probabilidad, por ciclo y por hebra, de un evento de desfase (retraso con probabilidad p , adelanto con probabilidad q), independiente entre ciclos. La fracción de hebras que siguen exactamente en fase tras n ciclos es

$$\phi(n) = (1 - \varepsilon)^n \approx e^{-\varepsilon n}. \quad (6.2)$$

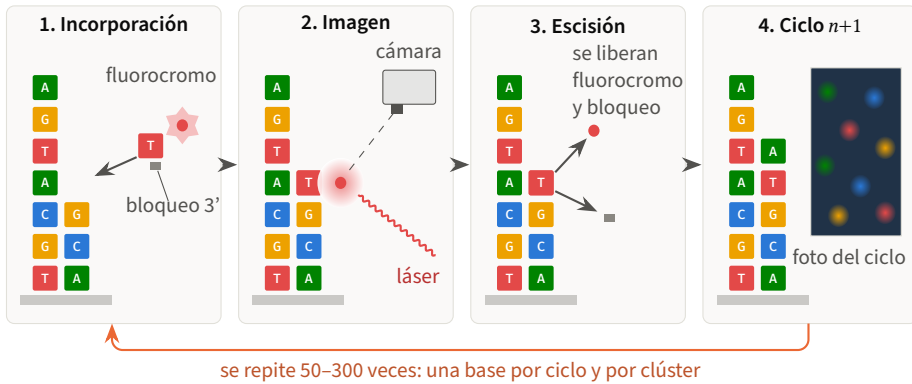


Figura 6.2. Un ciclo de secuenciación por síntesis con terminadores reversibles. (1) La polimerasa incorpora un único nucleótido marcado (aquí T, complementario de la A del molde); el grupo bloqueador impide añadir otro. (2) Un láser excita el fluorocromo y la cámara registra el color de cada clúster de la celda. (3) Se escinden el fluorocromo y el bloqueo, dejando un 3'-OH libre. (4) Comienza el ciclo siguiente; la secuencia de un clúster es la sucesión de colores que emite, foto tras foto. Una lectura de 150 bases requiere 150 ciclos y otras tantas imágenes de toda la celda.

DONDE

p	Probabilidad por ciclo de no incorporar ninguna base (<i>phasing</i>).
q	Probabilidad por ciclo de incorporar dos bases (<i>pre-phasing</i>).
ε	Tasa total de desfase por ciclo, $p + q$.
n	Número de ciclos completados (posición en la lectura).
$\phi(n)$	Fracción de hebras del clúster que nunca se han desfasado hasta el ciclo n y, por tanto, emiten la base correcta. Es una cota inferior de la fracción en fase: una hebra que se retrasa y luego se adelanta vuelve a estar en fase, un evento de segundo orden ($\propto pq$) que la ecuación (6.2) ignora.

La demostración es inmediata: una hebra está en fase tras n ciclos si y solo si no sufrió ningún evento en ninguno de ellos, y la probabilidad de ese suceso es el producto de n factores $(1 - \varepsilon)$. La aproximación exponencial usa $\ln(1 - \varepsilon) \approx -\varepsilon$ para ε pequeño. Aunque ε sea diminuto, su efecto se *acumula*. Con $\varepsilon = 0,002$, en el ciclo 150 solo el 74 % de las hebras emite el color correcto, y en el ciclo 300 apenas el 55 % (figura 6.3). El resto forma un fondo de colores de las posiciones vecinas que, ciclo a ciclo, hace más difícil decidir cuál es la base dominante. El programa de

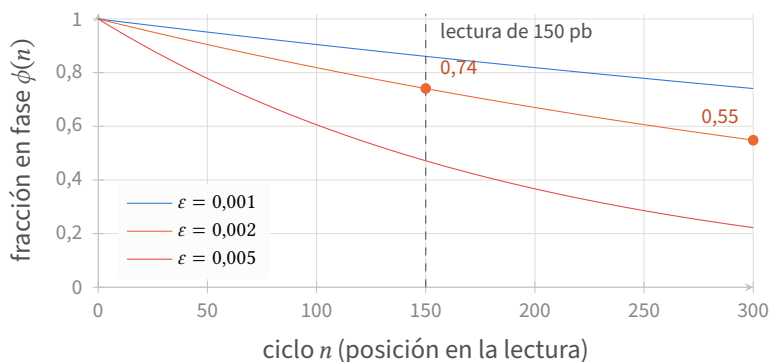


Figura 6.3. Fracción de hebras de un clúster que permanecen en fase según la ecuación (6.2), para tres tasas de desfase por ciclo. Tasas del orden de una milésima, que parecen despreciables, hacen que tras 300 ciclos casi la mitad de la señal provenga de hebras desfasadas. Este mecanismo, junto con el desgaste de los reactivos y el fotoblanqueo, fija el límite práctico de longitud de las lecturas cortas y explica la caída de calidad hacia el extremo 3'.

llamado de bases (*basecaller*) corrige parte de este efecto modelando el desfase, pero no todo, y por eso *la calidad de las lecturas de Illumina disminuye hacia el extremo 3'*. Este patrón es el que veremos en todos los informes de control de calidad de la sección 6.2.

Lecturas pareadas y química de dos colores. Tras leer un extremo del inserto, la hebra puede regenerarse para leer el extremo opuesto: se obtienen *lecturas pareadas* (*paired-end*), dos lecturas cuya distancia y orientación relativas son conocidas aproximadamente. Esa restricción geométrica será valiosísima para el mapeo y el ensamblaje. Bentley et al. (2008) secuenciaron así el genoma de un varón yoruba con pares de lecturas de 35 bases y una profundidad media superior a 30×. Los instrumentos más recientes usan una química de *dos colores*, en la que una de las cuatro bases (G) se reconoce por la *ausencia* de fluorescencia; la consecuencia práctica es que un clúster que se apaga produce falsas colas de GGGGG, un artefacto que herramientas como *fastp* recortan de forma específica (Chen et al., 2018).

6.1.5 Tercera generación: molécula única y lecturas largas

Las plataformas de lecturas cortas leen clústeres, no moléculas; necesitan amplificar, y el desfase limita su longitud a unos cientos de bases. Las llamadas tecnologías

de *tercera generación* observan una sola molécula en tiempo real y producen lecturas de decenas o cientos de kilobases.

PacBio SMRT. Eid et al. (2009) fijaron una única polimerasa en el fondo de un *guía de onda de modo cero* (*zero-mode waveguide*), un pozo de dimensiones inferiores a la longitud de onda de la luz que limita la observación a un volumen de zeptolitros. Los fluorocromos están unidos al fosfato terminal del nucleótido, de modo que se liberan con el pirofosfato al incorporarse la base: la hebra sintetizada queda sin modificar y la polimerasa puede seguir durante miles de bases. Cada incorporación se ve como un pulso de luz del color correspondiente. Los errores de esta lectura continua (*continuous long read*) son frecuentes, del orden del 10 al 15 %, dominados por inserciones y deleciones, pero tienen una propiedad preciosa: son *aproximadamente aleatorios*, sin sesgo sistemático fuerte. Eid et al. (2009) ya mostraron que el consenso de lecturas a 15× alcanzaba una exactitud mediana del 99,3 %.

Esa aleatoriedad es la base de las lecturas HiFi. Si el inserto se circulariza con adaptadores en horquilla, la polimerasa da muchas vueltas a la misma molécula y cada vuelta (*pass*) es una lectura independiente del mismo molde. La secuencia consenso circular (CCS) combina las pasadas.

Ejemplo 6.1. (Cuántas pasadas hacen una lectura HiFi)

Modelemos cada pasada como un voto independiente que acierta la base con probabilidad $1 - e$ y supongamos, de forma pesimista, que basta que la mayoría de las pasadas se equivoque para que el consenso falle. Con k pasadas (impar),

$$e_{\text{cons}}(k) = \sum_{j=\lceil k/2 \rceil}^k \binom{k}{j} e^j (1 - e)^{k-j}. \quad (6.3)$$

Con $e = 0,10$ por pasada: una pasada da $Q10$; tres pasadas, $e_{\text{cons}} = 0,028$ ($Q15,5$); cinco, $0,0086$ ($Q20,7$); nueve, $8,9 \times 10^{-4}$ ($Q30,5$), y quince, $3,4 \times 10^{-5}$ ($Q44,7$). La cota es pesimista porque, en la realidad, varias pasadas erróneas raramente coinciden en el *mismo* error. El resultado ilustra un principio general: los errores aleatorios se corrigen con redundancia; los sistemáticos, no. Wenger et al. (2019) optimizaron este procedimiento y obtuvieron lecturas HiFi con un 99,8 % de exactitud y una longitud media de 13,5 kb, capaces de ensamblar un genoma humano con *contigs* N50 superiores a 15 Mb.

Oxford Nanopore. La idea más radical prescinde por completo de la óptica y de la polimerasa como sensor. Una proteína motora hace pasar una hebra de ADN, base a base, a través de un nanoporo proteico insertado en una membrana con un voltaje aplicado; la corriente iónica que atraviesa el poro depende de las *varias* bases que lo ocupan en cada instante (un k -mer), y la señal resultante (el *squiggle*) se traduce a bases con redes neuronales (Jain et al., 2016; Rang et al., 2018). No hay límite químico intrínseco a la longitud de lectura: Jain et al. (2018) secuenciaron un genoma humano con lecturas ultralargas de N50 superior a 100 kb y una lectura máxima de 882 kb, que permitieron ensamblar completo el complejo principal de histocompatibilidad. El precio ha sido una tasa de error alta, entre el 5 y el 15 % en las químicas R9 según la revisión de Rang et al. (2018), con una fracción *sistemática* importante en homopolímeros (la corriente cambia poco mientras el poro “ve” siempre la misma base). Las químicas posteriores han reducido mucho ese problema: con los poros R10.4 ya es posible obtener genomas bacterianos casi terminados sin corrección con lecturas cortas (Sereika et al., 2022).

6.1.6 Comparación y perfiles de error

La tabla 6.1 y la figura 6.4 resumen el panorama. La lección fundamental no es qué plataforma es “mejor”, sino que *cada mecanismo físico deja una huella característica en los errores*, y los algoritmos posteriores deben conocerla: un alineador pensado para sustituciones aisladas sufrirá con lecturas cuyos errores son *indels*, y un ensamblador que confía en k -mers exactos fracasará con lecturas al 10 % de error. Las revisiones de Metzker (2010) y Goodwin et al. (2016) describen estas diferencias con detalle.

Aleatorio no es lo mismo que sistemático

Un error aleatorio se diluye al aumentar la profundidad: con 30 lecturas independientes, el consenso será correcto aunque cada lectura falle una vez de cada diez. Un error *sistemático*, en cambio, se repite en todas las lecturas que cubren el mismo contexto (un homopolímero largo, una región muy rica en GC) y ninguna profundidad lo corrige. Antes de confiar en una variante observada en muchas lecturas, pregúntese si todas ellas pudieron equivocarse *de la misma manera*.

Tabla 6.1. Características aproximadas de las principales plataformas. Las cifras son órdenes de magnitud típicos, que cambian con cada química y versión de instrumento; véanse las fuentes citadas en el texto.

Plataforma	Señal	Longitud	Error	Error dominante
Sanger	fluorescencia, electroforesis	400–800 pb	bajo	caída de señal al final
454 / Ion Torrent	luz / pH por flujo	100–700 pb	~1 %	<i>indels</i> en homopolímeros
Illumina	fluorescencia por ciclo	50–300 pb	~0,1 %	sustituciones; peor en 3'
PacBio CLR	pulso de luz en ZMW	10–50 kb	10–15 %	<i>indels</i> aleatorios
PacBio HiFi	consenso circular	10–25 kb	≤0,5 %	residual en homopolímeros
Nanopore	corriente iónica	1–>100 kb	1–15 %	<i>indels</i> sistemáticos en homopolímeros

6.1.7 El costo de un genoma

La figura 6.5 muestra la serie histórica que mantiene el Instituto Nacional de Investigación del Genoma Humano de EE. UU. (National Human Genome Research Institute, 2022). Hasta 2007 el costo cayó al ritmo de la ley de Moore, que describe la mejora de los microprocesadores (una reducción a la mitad cada dos años). A partir de 2008, con la llegada de las plataformas masivamente paralelas, la curva se desplomó: si la secuenciación hubiera seguido a los procesadores, un genoma costaría hoy unos 70 000 dólares; cuesta del orden de 500. Esa divergencia tiene una consecuencia que se repite a lo largo del libro: hoy es mucho más barato *generar* datos que *analizarlos*, y el cuello de botella de la genómica se ha desplazado del laboratorio a la computadora.

Cada plataforma convierte bases en una señal física distinta, y su perfil de errores es la huella de esa física: sustituciones en Illumina, *indels* en homopolímeros en 454 y Nanopore, *indels* aleatorios corregibles por consenso en PacBio.

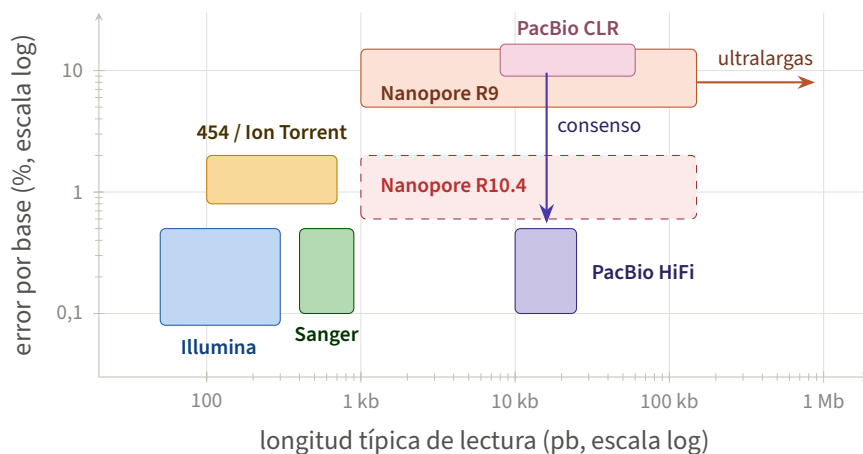


Figura 6.4. Mapa aproximado de las plataformas según longitud de lectura y tasa de error por base (ambos ejes logarítmicos). Los rectángulos indican órdenes de magnitud típicos, no especificaciones exactas (Goodwin et al., 2016; Jain et al., 2018; Rang et al., 2018; Wenger et al., 2019). La flecha violeta muestra la estrategia HiFi: sacrificar longitud y rendimiento para convertir errores aleatorios en exactitud por consenso. Durante una década existió un compromiso entre longitud y exactitud; las químicas recientes de lecturas largas lo han ido borrando.

6.2 Calidad Phred, control de calidad y *trimming*

Colab · 6.2 · Calidad Phred y control de calidad

abrir notebook

El pronóstico del tiempo de cada base

Un buen servicio meteorológico no dice “mañana lloverá”: dice “mañana hay un 70 % de probabilidad de lluvia”. Y lo que lo hace bueno no es acertar siempre, sino estar *calibrado*: de todos los días en que anunció un 70 %, llovió aproximadamente en siete de cada diez. Un secuenciador moderno hace lo mismo con cada base que entrega. Junto a la letra **A** escribe, en clave, algo como “me equivoco en una de cada mil veces que digo esto”. Aprender a leer esa clave, y a desconfiar cuando el pronóstico empeora, es el primer paso de cualquier análisis.

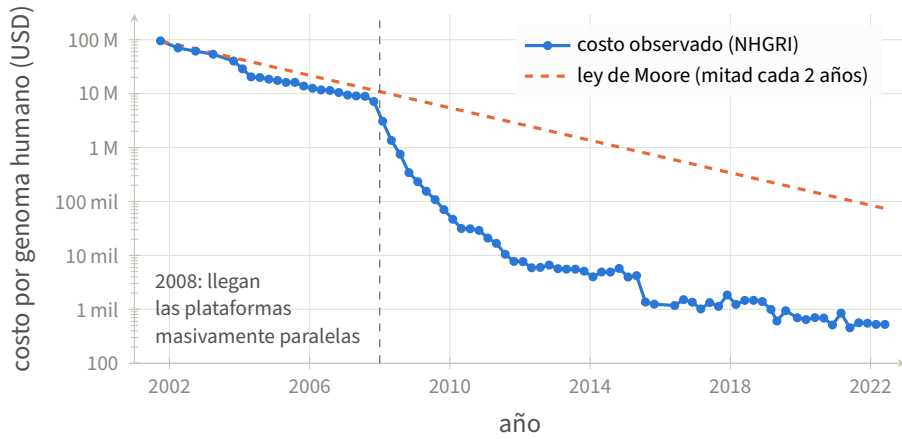


Figura 6.5. Costo de secuenciar un genoma humano, 2001–2022, según los datos del NHGRI (National Human Genome Research Institute, 2022), en escala logarítmica. La línea naranja muestra lo que habría costado si hubiera seguido la ley de Moore desde 2001. La brecha de dos órdenes de magnitud que se abre a partir de 2008 corresponde a la transición de la electroforesis capilar a la secuenciación masiva en paralelo. Datos: `figuras/cap06/nhgri_costos.csv`.

6.2.1 La escala Phred

Cuando el programa `phred` asignaba bases a partir de las curvas de fluorescencia de los secuenciadores capilares, calculaba para cada pico varios parámetros de la traza (espaciado entre picos, cociente entre la señal llamada y la no llamada, etc.) y, mediante una tabla calibrada con lecturas de secuencia conocida, los convertía en una probabilidad de error (Ewing & Green, 1998; Ewing et al., 1998). Para expresar esa probabilidad de forma compacta, los autores propusieron una escala logarítmica que hoy es universal.

Definición 6.2. (Calidad Phred)

Si p es la probabilidad de que la base asignada sea incorrecta, su calidad Phred es

$$Q = -10 \log_{10} p \quad \Leftrightarrow \quad p = 10^{-Q/10}. \quad (6.4)$$

DONDE	
p	Probabilidad de error de la base, estimada por el <i>basecaller</i> .
Q	Calidad Phred, en “decibelios” de probabilidad; normalmente se redondea a un entero.

La escala tiene dos virtudes. Primero, es *intuitiva en órdenes de magnitud*: cada 10 unidades de Q dividen el error por diez. $Q10$ es un error de cada 10 bases (90 % de exactitud), $Q20$ uno de cada 100, $Q30$ uno de cada mil y $Q40$ uno de cada diez mil (99,99 %). Segundo, convierte productos en sumas: la probabilidad de que dos bases independientes sean ambas incorrectas, $p_1 p_2$, corresponde a una calidad $Q_1 + Q_2$. La cifra “ $Q30$ ” que aparece en las especificaciones de los instrumentos es, por lo tanto, una promesa concreta: como mucho un error por cada mil bases.

6.2.2 Del registro a la probabilidad: el formato FASTQ

Las lecturas se almacenan en archivos FASTQ, que Cock et al. (2010) documentaron formalmente junto con sus variantes históricas. Cada lectura ocupa cuatro líneas: un identificador que empieza con @, la secuencia, una línea separadora que empieza con +, y una cadena de calidades *de la misma longitud que la secuencia*, en la que cada carácter codifica el Q de la base correspondiente.

Definición 6.3. (Codificación ASCII de las calidades)

En la codificación estándar (llamada *Sanger* o Phred+33), el carácter κ de la línea de calidades representa

$$Q = \text{ord}(\kappa) - 33, \tag{6.5}$$

de modo que ! (ASCII 33) es $Q0$, + (43) es $Q10$, 5 (53) es $Q20$, ? (63) es $Q30$ e I (73) es $Q40$.

DONDE	
κ	Carácter imprimible de la línea de calidades.
$\text{ord}(\kappa)$	Código ASCII del carácter (por ejemplo, $\text{ord}(\text{I}) = 73$).
33	Desplazamiento (<i>offset</i>); se elige 33 porque es el primer carácter ASCII imprimible después del espacio.

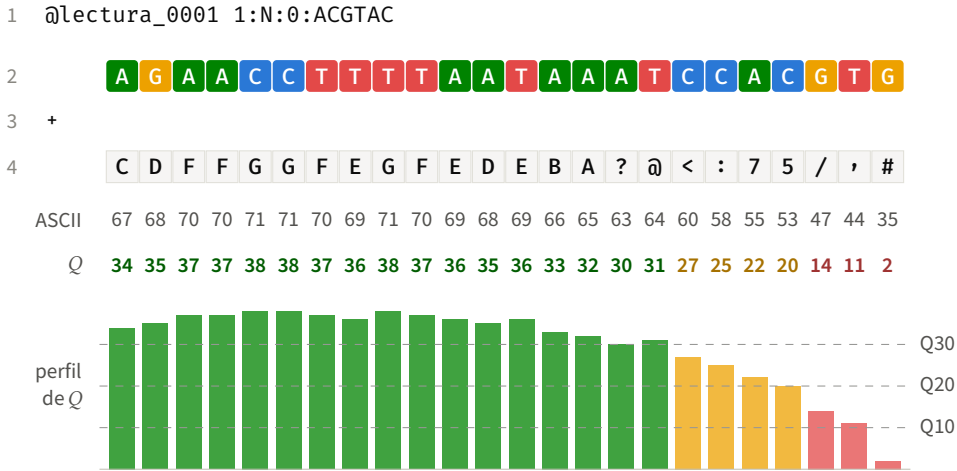


Figura 6.6. Anatomía de un registro FASTQ. Las líneas 1 a 4 son el identificador, la secuencia, el separador y la cadena de calidades. Debajo de cada carácter de calidad se muestra su código ASCII y la calidad $Q = \text{ord} - 33$ (ecuación (6.5)); las barras reproducen el perfil de calidad con las referencias Q_{10} , Q_{20} y Q_{30} . Obsérvese la caída hacia el extremo 3' y el último carácter, # (Q_2), que algunos *basecallers* usan como marca de “base no fiable”. Lectura simulada con `figuras/cap06/generar.py`.

Por desgracia, no siempre fue así. Los primeros instrumentos de Solexa/Illumina usaron un desplazamiento de 64 y, además, una escala distinta, basada en el cociente de probabilidades (*odds*),

$$Q_{\text{Solexa}} = -10 \log_{10} \frac{p}{1-p}, \quad Q_{\text{Phred}} = 10 \log_{10} (10^{Q_{\text{Solexa}}/10} + 1), \quad (6.6)$$

que admite valores negativos y coincide con la escala Phred solo para calidades altas (Cock et al., 2010). Versiones posteriores pasaron a Phred+64 y, finalmente, a la codificación Sanger que hoy es el estándar. La consecuencia práctica es que un archivo antiguo leído con el desplazamiento equivocado tendrá todas sus calidades desplazadas en 31 unidades: un error silencioso que convierte bases malas en excelentes, o al revés.

DONDE

Q_{Solexa}	Calidad en la escala de <i>odds</i> logarítmicos de Solexa.
Q_{Phred}	Calidad equivalente en la escala Phred de la ecuación (6.4).

6.2.3 Errores esperados en una lectura

Si conocemos la probabilidad de error de cada base, podemos resumir la calidad de toda una lectura. Considerando las bases independientes (una aproximación razonable), el número de errores de una lectura de longitud L es una suma de variables de Bernoulli.

Teorema 6.2. (Errores esperados y probabilidad de lectura perfecta)

Para una lectura con probabilidades de error $p_1, \dots, p_L$ independientes, el número de errores X cumple

$$\mathbb{E}[X] = \sum_{i=1}^L p_i = \sum_{i=1}^L 10^{-Q_i/10}, \quad \mathbb{P}(X = 0) = \prod_{i=1}^L (1 - p_i). \quad (6.7)$$

DONDE

X	Número (aleatorio) de bases incorrectas en la lectura.
p_i, Q_i	Probabilidad de error y calidad Phred de la base i .
L	Longitud de la lectura.

La esperanza de una suma es la suma de las esperanzas, sin necesidad siquiera de independencia; la segunda fórmula, en cambio, sí la requiere. Un detalle importante es que el *promedio de las calidades* no es un buen resumen: como p es exponencial en Q , una sola base de $Q2$ aporta más errores esperados que treinta bases de $Q30$. Por eso algunos filtros (por ejemplo, en el análisis de amplicones) descartan lecturas según su $\mathbb{E}[X]$, no según su Q medio.

Ejemplo 6.2. (La lectura de la figura 6.6)

Las 24 calidades de la figura suman $\mathbb{E}[X] = 0,78$ errores esperados. Sin embargo, 0,63 de ellos proceden de la última base ($Q2$, $p = 10^{-0,2} \approx 0,63$); las 20 primeras, todas con $Q \geq 20$, aportan en conjunto menos de 0,02. Recortar solo las cuatro últimas bases elimina más del 95 % de los errores esperados.

A mayor escala, comparemos dos lecturas de 150 bases con calidad uniforme. A $Q30$, $\mathbb{E}[X] = 150 \times 10^{-3} = 0,15$ y $\mathbb{P}(X = 0) = 0,999^{150} \approx 0,86$: casi nueve de cada diez lecturas no tienen ningún error. A $Q20$, $\mathbb{E}[X] = 1,5$ y $\mathbb{P}(X = 0) = 0,99^{150} \approx 0,22$: casi cuatro de cada cinco lecturas llevan al menos

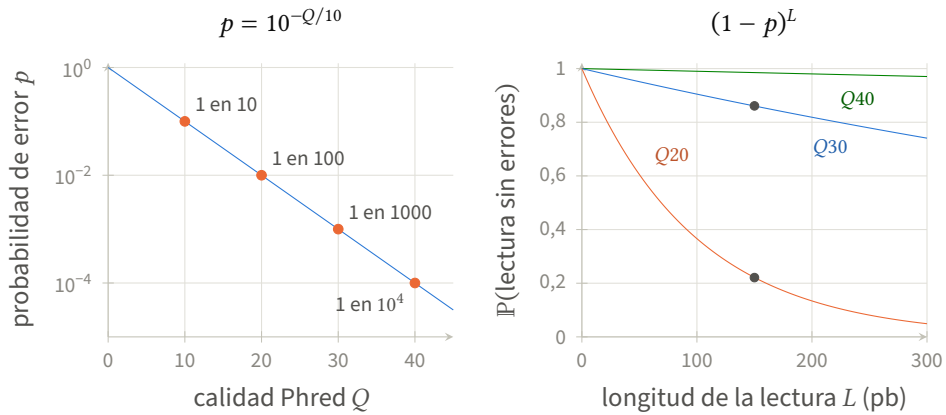


Figura 6.7. **Izquierda:** la escala Phred es logarítmica; cada 10 unidades de Q reducen el error en un factor 10. **Derecha:** probabilidad de que una lectura no contenga ningún error (ecuación (6.7)) si todas sus bases tienen la misma calidad. Los puntos grises marcan las lecturas de 150 pb del ejemplo 6.2: 0,86 a $Q30$ frente a 0,22 a $Q20$.

un error. La diferencia entre $Q20$ y $Q30$, que parece modesta en la escala logarítmica, cambia por completo la naturaleza de los datos (figura 6.7).

⚠ Las calidades son estimaciones, no verdades

Un Q es tan bueno como la calibración del *basecaller* que lo produjo. Las calidades pueden estar sistemáticamente infladas o desinfladas según el instrumento, la química y el contexto de secuencia; por eso los flujos de llamado de variantes suelen *recalibrarlas* empíricamente usando posiciones conocidas. Además, muchos instrumentos actuales agrupan las calidades en unos pocos niveles para comprimir los archivos, de modo que un Q individual tiene una resolución limitada.

6.2.4 Control de calidad: FastQC y MultiQC

Antes de cualquier análisis conviene mirar los datos en conjunto. **FastQC** (Andrews, 2010) es la herramienta de referencia: recorre un archivo FASTQ y produce un informe con una serie de módulos, cada uno marcado como aprobado, dudoso o fallido. Los más útiles son:

- **Calidad por posición** (*per base sequence quality*): diagramas de caja de Q en cada posición de la lectura (figura 6.8). Revela la caída hacia 3' que predice la ecuación (6.2), o problemas puntuales en ciclos concretos (una burbuja en la celda, un fallo de reactivo).
- **Contenido de bases por posición**: en una biblioteca aleatoria, las proporciones de A, C, G y T deberían ser constantes a lo largo de la lectura. Las oscilaciones en las primeras posiciones son habituales en RNA-seq por los sesgos de los cebadores aleatorios, y no justifican por sí solas un recorte.
- **Contenido de GC por lectura**: debería parecerse a una distribución unimodal centrada en el GC del organismo; una segunda moda sugiere contaminación.
- **Niveles de duplicación y secuencias sobrerrepresentadas**: detectan bibliotecas de baja complejidad, dímeros de adaptadores y contaminantes abundantes.
- **Contenido de adaptadores**: fracción acumulada de lecturas en las que aparece una secuencia de adaptador conocida a partir de cada posición.

Un proyecto real tiene decenas o cientos de muestras, y abrir cientos de informes es inviable. **MultiQC** (Ewels et al., 2016) resuelve este problema: busca en un directorio los resultados de FastQC y de muchas otras herramientas (recortadores, alineadores, cuantificadores) y los agrega en un único informe interactivo, donde las muestras atípicas y los efectos de lote saltan a la vista.

6.2.5 Duplicados y complejidad de la biblioteca

Un duplicado es un par de lecturas que proceden de la *misma molécula original*: copias creadas por la PCR de la preparación de la biblioteca, o duplicados “ópticos” en los que un mismo clúster se registró dos veces. Los duplicados no aportan información independiente y, si no se identifican, inflan artificialmente la confianza en lo que sea que esas copias contengan, incluido un error de la PCR temprana. Pero no todos los duplicados son artefactos: si la biblioteca tiene pocas moléculas distintas, incluso un muestreo perfecto las repetirá.

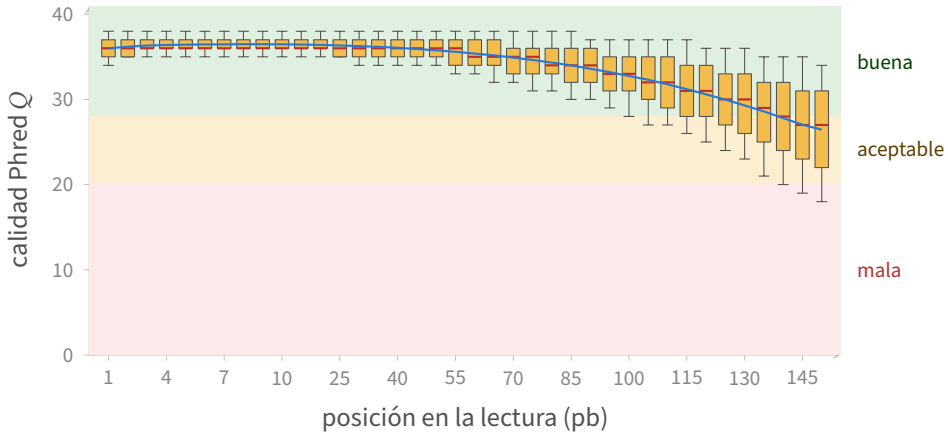


Figura 6.8. Calidad por posición de 20 000 lecturas simuladas de 150 pb, al estilo del módulo *per base sequence quality* de FastQC. Como en FastQC, las primeras nueve posiciones se muestran individualmente y el resto en bloques de cinco. Cada caja abarca del cuartil 1 al 3, con la mediana en rojo y bigotes entre los percentiles 10 y 90; la línea azul es la media. El patrón es el típico de Illumina: calidad alta y estable al principio, una ligera caída en las primeras bases y un deterioro creciente, con mayor dispersión, hacia el extremo 3'. En las últimas posiciones el percentil 10 cae por debajo de Q_{20} : candidatas a recorte.

Teorema 6.3. (Moléculas distintas observadas)

Si una biblioteca contiene M moléculas distintas igualmente representadas y se secuencian N lecturas elegidas al azar, el número esperado de moléculas distintas observadas es

$$E[D] = M \left[1 - \left(1 - \frac{1}{M} \right)^N \right] \approx M \left(1 - e^{-N/M} \right), \quad (6.8)$$

y la tasa de duplicación esperada es $1 - E[D]/N$.

DONDE

M	Complejidad de la biblioteca: número de moléculas únicas disponibles.
N	Número de lecturas secuenciadas.
D	Número de moléculas distintas observadas al menos una vez.

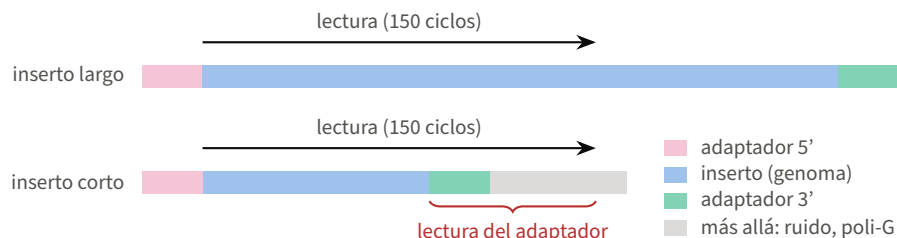


Figura 6.9. Cuando el inserto (azul) es más largo que el número de ciclos, la lectura termina dentro del genoma. Cuando es más corto, la lectura lo atraviesa y continúa en el adaptador 3' (aqua) y, más allá, en una región sin molde que produce ruido o, en la química de dos colores, colas de GGG. Las bases desde el inicio del adaptador deben recortarse. En lecturas pareadas, los dos miembros del par se solapan por completo en este caso, lo que permite localizar el adaptador sin conocer su secuencia.

La demostración usa un argumento que reaparecerá, casi idéntico, en la sección 6.3: una molécula concreta no es elegida por ninguna de las N lecturas con probabilidad $(1 - 1/M)^N \approx e^{-N/M}$, y la esperanza de D es M veces la probabilidad de lo contrario. Con $M = 10^7$ moléculas y $N = 2 \times 10^7$ lecturas, $E[D] \approx 8,65 \times 10^6$ y la tasa de duplicación es del 57 %: la mitad del presupuesto de secuenciación se gasta en releer lo mismo. Con una biblioteca diez veces más compleja ($M = 10^8$), la misma cantidad de lecturas produce solo un 9 % de duplicados. La lección práctica es que *la complejidad de la biblioteca, no el número de lecturas, limita la información*; secuenciar más una biblioteca pobre solo produce más duplicados.

6.2.6 Recorte de adaptadores

Si el inserto es más corto que la longitud de lectura, la polimerasa lo atraviesa por completo y sigue leyendo el adaptador del otro extremo (figura 6.9). Esas bases no pertenecen al organismo: si no se eliminan, impedirán el alineamiento o, peor, producirán falsas discrepancias en el extremo 3'. El fenómeno es inevitable en bibliotecas de fragmentos cortos, como las de ARN pequeños o las de ADN antiguo y degradado.

Detectar el adaptador parece trivial, porque su secuencia es conocida (por ejemplo, el prefijo **AGATCGGAAGAGC**, común a los adaptadores TruSeq de Illumina). La dificultad está en el extremo: cuando el adaptador empieza a solo k bases del final de la lectura, apenas se ven sus primeras k letras, y una coincidencia tan corta puede ser casual.

Definición 6.4. (Coincidencia casual con el adaptador)

Si las bases de la lectura son independientes y equiprobables, la probabilidad de que sus últimas k bases coincidan por azar con las primeras k del adaptador es

$$P(\text{coincidencia casual de longitud } k) = 4^{-k}. \quad (6.9)$$

DONDE

k Longitud del solapamiento entre el extremo 3' de la lectura y el comienzo del adaptador.

Con $k = 3$, uno de cada 64 extremos (1,6 %) coincide por azar; con $k = 10$, menos de uno por millón. Las herramientas, por lo tanto, deben aceptar un compromiso: exigir un solapamiento mínimo largo deja adaptadores residuales cortos; aceptar solapamientos cortos recorta de más unas pocas bases genuinas en una fracción de las lecturas, un daño normalmente menor. [cutadapt](#) (Martin, 2011) busca el adaptador mediante un alineamiento semiglobal que tolera una fracción de errores, y recorta desde el punto de coincidencia. [Trimmomatic](#) (Bolger et al., 2014) añade un modo “palíndromo” para lecturas pareadas, que aprovecha que ambos miembros del par, cuando el inserto es corto, son complementarios inversos entre sí. [fastp](#) (Chen et al., 2018) usa la misma idea de solapamiento para detectar adaptadores automáticamente y, en un único recorrido del archivo, realiza también el control de calidad, el recorte por calidad y el de colas de poli-G.

6.2.7 Recorte por calidad

Una vez eliminados los adaptadores, queda decidir qué hacer con los extremos de baja calidad. Hay dos estrategias clásicas.

Ventana deslizante. [Trimmomatic](#) recorre la lectura desde el extremo 5' con una ventana de w bases y la corta cuando la calidad media de la ventana cae por debajo de un umbral t (opción `SLIDINGWINDOW:w:t`) (Bolger et al., 2014). Formalmente, la lectura se trunca en

$$x^* = \min \left\{ x : \frac{1}{w} \sum_{i=x}^{x+w-1} Q_i < t \right\}, \quad (6.10)$$

conservando las bases $1, \dots, x^* - 1$. Promediar sobre una ventana evita que una única base mala, aislada en medio de una lectura excelente, provoque un corte prematuro.

DONDE

w, t	Anchura de la ventana y umbral de calidad media (p. ej., $w = 4$, $t = 20$).
x^*	Primera posición en la que empieza una ventana con calidad media inferior al umbral.

Suma acumulada desde 3'. La alternativa, heredada del alineador BWA y usada por la opción `-q` de `cutadapt`, recorre la lectura desde el extremo 3' acumulando $t - Q_i$ y corta donde esa suma alcanza su máximo:

$$x^* = \arg \max_x \sum_{i=x}^L (t - Q_i). \quad (6.11)$$

Es exactamente el problema del segmento de suma máxima que vimos en el capítulo 3 con Smith-Waterman, restringido a sufijos: las bases con $Q < t$ suman a favor del recorte y las buenas en contra, de modo que se elimina el sufijo cuya “mala calidad neta” es máxima. Si varios x empatan en el máximo se elige el más cercano al extremo 3', que conserva más bases. Las implementaciones reales añaden además una regla de parada: el recorrido desde 3' se detiene en cuanto la suma acumulada se vuelve negativa, y BWA respeta una longitud mínima de lectura.

Ejemplo 6.3. (Comparar los dos criterios)

La figura 6.10 aplica ambos criterios a la misma lectura con $t = 20$. Las medias de ventana de cuatro bases descienden lentamente (37,75, 37,25, ..., 22,25) hasta que la ventana que empieza en la posición 21, con calidades (22, 19, 21, 17), da $19,75 < 20$: `Trimmomatic` corta allí y conserva 20 bases. El criterio de BWA acumula desde el final: $20 - 6 = 14$, $+12 = 26$, $+10 = 36$, $+8 = 44$, $+2 = 46$, $+5 = 51$, $+3 = 54$ (posición 24), $-1 = 53$, $+1 = 54$, $-2 = 52$... El máximo, 54, se alcanza por primera vez en la posición 24, así que se conservan las 23 primeras bases. La diferencia (tres bases) es pequeña aquí, pero ilustra por qué dos herramientas con el “mismo” umbral no producen los mismos archivos.

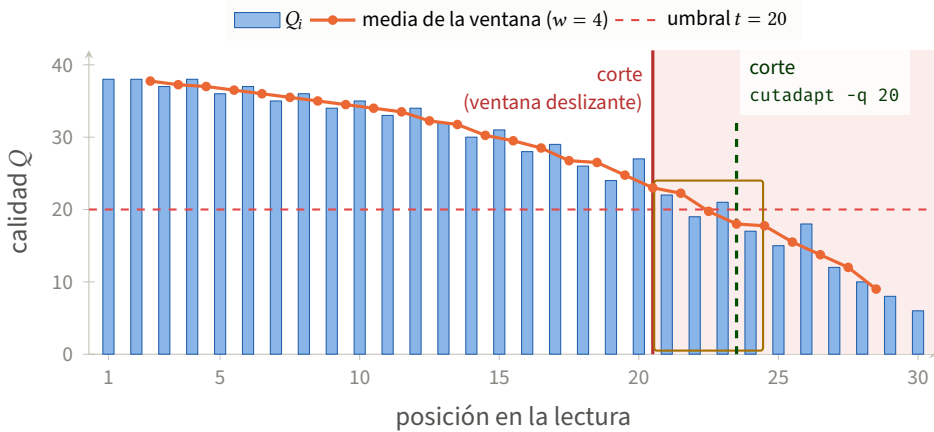


Figura 6.10. Recorte por calidad de una lectura de 30 bases. Las barras son las calidades individuales; la curva naranja, la media de cada ventana de cuatro bases, dibujada en su centro. La primera ventana con media inferior a 20 empieza en la posición 21 (recuadro amarillo, media 19,75), así que la ventana deslizando conserva 20 bases (ecuación (6.10)). El criterio de suma acumulada (ecuación (6.11)) es menos conservador y conserva 23: las bases 21 a 23 ($Q = 22, 19, 21$) tienen, en conjunto, una calidad neta por encima del umbral. Ninguno de los dos es “el correcto”; son reglas distintas para el mismo compromiso.

```
recorte.py

from Bio import SeqIO

def errores_esperados(q):
    return sum(10 ** (-qi / 10) for qi in q)

def ventana_deslizante(q, w=4, t=20):
    """Devuelve cuántas bases conservar (criterio de Trimmomatic)."""
    for x in range(len(q) - w + 1):
        if sum(q[x:x + w]) / w < t:
            return x
    return len(q)

with open("recortadas.fastq", "w") as out:
    for rec in SeqIO.parse("muestra_R1.fastq", "fastq"):
        q = rec.letter_annotations["phred_quality"] # ya sin +33
        corte = ventana_deslizante(q)
        rec = rec[:corte] # corta seq y Q
```

```
if len(rec) >= 50 and errores_esperados(q[:corte]) < 1:
    SeqIO.write(rec, out, "fastq")
```

En la práctica no escribiremos nuestro propio recortador: usaremos herramientas probadas, rápidas y que manejan correctamente las lecturas pareadas (si un miembro del par se descarta, el otro debe ir a un archivo aparte, o los archivos quedarán desincronizados). Un flujo típico de control de calidad es el siguiente:

Control de calidad y recorte de un par de archivos

```
# 1. Diagnóstico inicial
fastqc -t 4 -o qc_crudo/ muestra_R1.fastq.gz muestra_R2.fastq.gz

# 2. Adaptadores + calidad + longitud mínima en un solo paso
fastp -i muestra_R1.fastq.gz -I muestra_R2.fastq.gz \
-o limpia_R1.fastq.gz -O limpia_R2.fastq.gz \
--detect_adapter_for_pe --trim_poly_g \
--cut_right --cut_right_window_size 4 \
--cut_right_mean_quality 20 --length_required 50 \
-h fastp.html -j fastp.json -w 8

# 3. Diagnóstico después del recorte y resumen de todo el proyecto
fastqc -t 4 -o qc_limpio/ limpia_R1.fastq.gz limpia_R2.fastq.gz
multiqc qc_crudo/ qc_limpio/ fastp.json -o informe_qc/
```

No recorte por reflejo

El recorte agresivo tiene costos: acorta lecturas (lo que empeora el mapeo en regiones repetidas), descarta pares enteros y puede introducir sesgos si la calidad depende de la secuencia. Muchos alineadores modernos realizan alineamientos locales que ignoran (*soft-clip*) los extremos que no alinean, y los llamadores de variantes ya ponderan cada base por su calidad. Recortar adaptadores es casi siempre necesario; recortar por calidad debe hacerse con umbrales moderados (Q15–Q20) y comprobando su efecto con un segundo informe de FastQC, no como un ritual. Schirmer et al. (2015) mostraron, para amplicones de MiSeq, que la preparación de la biblioteca y los cebadores influyen en los patrones de error tanto como el propio instrumento: ningún recorte arregla un problema del laboratorio.

$Q = -10 \log_{10} p$. Resuma la calidad de una lectura con sus errores esperados $\sum p_i$, no con el promedio de Q ; recorte adaptadores siempre y calidad con moderación.

6.3 Cobertura y la teoría de Lander-Waterman

 Colab · 6.3 · Cobertura y Lander-Waterman

 abrir notebook

Gotas sobre la acera

Empieza a llover sobre una acera seca. Las primeras gotas dejan manchas aisladas; al cabo de un rato las manchas se tocan y forman islas mojadas; mucho después, la acera está completamente oscura. Pero incluso cuando ya ha caído agua más que suficiente para cubrir la acera dos o tres veces, si mira con atención encontrará todavía algún rincón seco: las gotas caen al azar y el azar no reparte con justicia. La secuenciación de un genoma por fragmentos aleatorios se parece mucho a esa lluvia. ¿Cuántas “gotas” (lecturas) necesitamos para que no queden rincones secos? ¿Cuántas islas habrá mientras tanto? La respuesta es una de las aplicaciones más bellas de la distribución de Poisson.

6.3.1 Cobertura y profundidad

Supongamos que secuenciamos N lecturas de longitud L de un genoma de tamaño G . La cantidad total de bases leídas es NL , y la definimos en relación con el tamaño del genoma.

Definición 6.5. (Cobertura media)

La *cobertura* (o *profundidad media*) de un experimento es

$$c = \frac{NL}{G}, \quad (6.12)$$

y se expresa como “ $c\times$ ” (por ejemplo, $30\times$). La *profundidad* D_x en una posición x es el número de lecturas que la contienen.

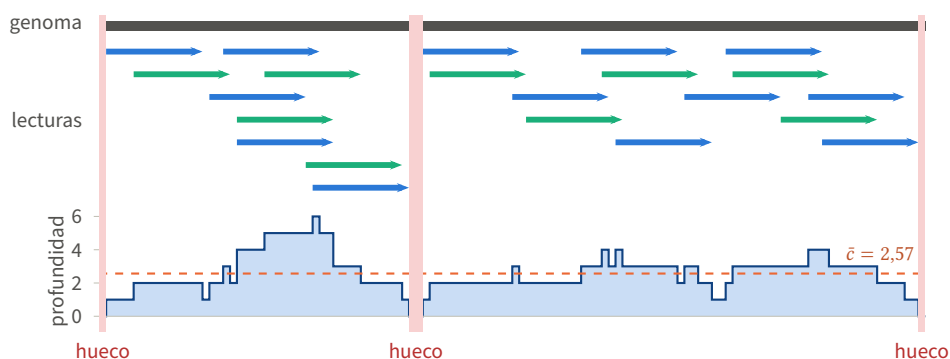


Figura 6.11. Veintidós lecturas de 14 posiciones colocadas al azar sobre un genoma de 120 posiciones ($c = 22 \cdot 14/120 \approx 2,57$). **Arriba:** las lecturas, apiladas para que no se solapen visualmente. **Abajo:** la profundidad en cada posición. Pese a que en promedio cada base se leyó más de dos veces y media, la profundidad varía entre 0 y más del doble de la media, y aparecen huecos (rojo) que ninguna lectura cubre; dos de ellos están en los extremos, donde un genoma lineal es más difícil de cubrir. La teoría predice $Ge^{-c} \approx 9$ posiciones descubiertas; en esta realización hay 4. Generado con `figuras/cap06/generar.py`.

DONDE

N	Número de lecturas (o de pares de lecturas, si se cuenta el inserto).
L	Longitud de cada lectura, en pares de bases.
G	Tamaño del genoma (haploide), en pares de bases.
c	Cobertura media: cuántas veces, en promedio, se lee cada base.
D_x	Profundidad (aleatoria) en la posición x .

Por construcción, la profundidad media sobre todas las posiciones es c ; la pregunta interesante es cómo se *distribuye* alrededor de ese valor. La figura 6.11 muestra una realización: aunque la cobertura media es de $2,6\times$, hay posiciones leídas seis veces y pequeños huecos sin ninguna lectura.

6.3.2 La distribución de Poisson de la profundidad

Lander y Waterman (1988) formularon el modelo que usamos hasta hoy. Aunque lo propusieron para el mapeo físico con clones, sus supuestos se trasladan sin cambios a las lecturas: (i) las lecturas comienzan en posiciones independientes y uniformemente distribuidas a lo largo del genoma; (ii) todas tienen la misma

longitud L ; (iii) el genoma es mucho mayor que las lecturas, $G \gg L$. Waterman desarrolla el modelo con detalle en su libro de texto (Waterman, 1995).

Teorema 6.4. (Profundidad de Poisson)

Bajo los supuestos (i)–(iii), la profundidad en una posición fija sigue, aproximadamente, una distribución de Poisson de media c :

$$\mathbb{P}(D_x = k) = \frac{e^{-c} c^k}{k!}, \quad k = 0, 1, 2, \dots \quad (6.13)$$

En particular, la fracción esperada del genoma no cubierta es $\mathbb{P}(D_x = 0) = e^{-c}$, y la fracción cubierta, $1 - e^{-c}$.

Derivación paso a paso. Fijemos una posición x . Una lectura cubre x si y solo si empieza en alguna de las L posiciones $x - L + 1, \dots, x$. Como hay (aproximadamente) G posiciones de inicio posibles y todas son equiprobables, la probabilidad de que una lectura dada cubra x es

$$\pi = \frac{L}{G}.$$

Las N lecturas son independientes, de modo que el número de ellas que cubren x es binomial:

$$\mathbb{P}(D_x = k) = \binom{N}{k} \pi^k (1 - \pi)^{N-k}, \quad \mathbb{E}[D_x] = N\pi = \frac{NL}{G} = c.$$

En un experimento real N es enorme (millones) y π minúscula ($L/G \sim 10^{-7}$ en un genoma humano), mientras que su producto c es moderado. Es el régimen clásico en que la binomial converge a la Poisson. Explícitamente, con $\pi = c/N$,

$$\begin{aligned} \binom{N}{k} \left(\frac{c}{N}\right)^k \left(1 - \frac{c}{N}\right)^{N-k} &= \underbrace{\frac{N(N-1)\cdots(N-k+1)}{N^k}}_{\rightarrow 1} \frac{c^k}{k!} \underbrace{\left(1 - \frac{c}{N}\right)^N}_{\rightarrow e^{-c}} \underbrace{\left(1 - \frac{c}{N}\right)^{-k}}_{\rightarrow 1} \\ &\xrightarrow{N \rightarrow \infty} \frac{e^{-c} c^k}{k!}. \end{aligned}$$

Con $k = 0$ obtenemos la fracción no cubierta, e^{-c} . $\square$

DONDE

π	Probabilidad de que una lectura concreta cubra la posición x .
k	Valor de la profundidad cuya probabilidad se calcula.
e^{-c}	Fracción esperada del genoma sin ninguna lectura.

La fórmula es implacable. Con $c = 1$, el 36,8 % del genoma queda sin leer, aunque “hayamos leído el genoma una vez”. Con $c = 3$ falta el 5 %; con $c = 5$, el 0,67 %; con $c = 10$, el 0,0045 %. Como la fracción descubierta decrece exponencialmente, cada unidad adicional de cobertura divide lo que falta por $e \approx 2,72$, pero nunca lo anula. Observe también el paralelismo formal con la ecuación (6.8): en ambos casos se pregunta por los objetos (posiciones o moléculas) que un muestreo aleatorio no ha tocado, y en ambos la respuesta es $e^{-(\text{número medio de toques})}$.

6.3.3 Islas, contigs y huecos

Para el ensamblaje, más que las bases no cubiertas importa cuántos fragmentos contiguos (*islas* o *contigs*) podremos reconstruir. Dos lecturas solo pueden unirse si se solapan lo suficiente para detectarlo; llamemos T al solapamiento mínimo detectable y $\sigma = 1 - T/L$ a la fracción de la lectura que *no* necesita solaparse.

Teorema 6.5. (*Lander-Waterman: número y tamaño de las islas*)

Bajo los supuestos anteriores y con solapamiento mínimo detectable $T = (1 - \sigma)L$:

$$E[\text{número de islas}] = Ne^{-c\sigma}, \quad (6.14)$$

$$E[\text{lecturas por isla}] = e^{c\sigma}, \quad (6.15)$$

$$E[\text{longitud de una isla}] = L \left[\frac{e^{c\sigma} - 1}{c} + 1 - \sigma \right]. \quad (6.16)$$

DONDE

T	Solapamiento mínimo (en pb) necesario para reconocer que dos lecturas son vecinas.
σ	Fracción $1 - T/L$; con $T = 0$ vale 1.
$Ne^{-c\sigma}$	Número esperado de islas; en un genoma lineal, los huecos son una menos que las islas.

Derivación. Cada isla tiene exactamente una lectura “final”, la situada más a la derecha. Una lectura que empieza en la posición s es final si ninguna otra lectura empieza en el intervalo $(s, s + L - T]$, de longitud $L\sigma$: si alguna empezara allí, se solaparía con ella en al menos T bases y la isla continuaría. Para cada una de las

otras $N - 1$ lecturas, la probabilidad de no empezar en ese intervalo es $1 - L\sigma/G$, luego

$$P(\text{una lectura es final}) = \left(1 - \frac{L\sigma}{G}\right)^{N-1} \approx e^{-NL\sigma/G} = e^{-c\sigma}.$$

Contar islas es contar lecturas finales, y por linealidad de la esperanza se obtiene la ecuación (6.14). Si cada lectura, recorriendo de izquierda a derecha, “cierra” la isla con probabilidad $e^{-c\sigma}$, el número de lecturas por isla es geométrico con esa probabilidad de éxito, de media $e^{c\sigma}$ (ecuación (6.15)). La longitud de la isla (ecuación (6.16)) se obtiene sumando la longitud de la última lectura a las distancias entre inicios consecutivos dentro de la isla, que son exponenciales truncadas en $L\sigma$; el cálculo detallado está en Lander y Waterman (1988). $\square$

La ecuación (6.14) esconde un comportamiento no monótono. Midamos el número de islas en unidades de G/L (el número de lecturas que “cabrían” en el genoma sin solaparse): como $N = cG/L$,

$$\frac{E[\text{islas}]}{G/L} = ce^{-c\sigma}, \quad \frac{d}{dc}(ce^{-c\sigma}) = e^{-c\sigma}(1 - c\sigma) = 0 \implies c^* = \frac{1}{\sigma}. \quad (6.17)$$

Al principio, cada lectura nueva suele caer en territorio virgen y *crea* una isla; más tarde, las lecturas nuevas *unen* islas existentes. El número de islas alcanza su máximo, $(G/L)/(e\sigma)$, cuando la cobertura vale $1/\sigma$, y después decae exponencialmente (figura 6.12). Un ensamblaje con cobertura baja no es simplemente “un poco peor”: está en la zona de máxima fragmentación.

Ejemplo 6.4. (Planificar el ensamblaje de una bacteria)

Queremos ensamblar un genoma bacteriano de $G = 5 \times 10^6$ pb con lecturas de $L = 150$ pb, y nuestro ensamblador necesita un solapamiento de al menos $T = 30$ pb, de modo que $\sigma = 0,8$. Aplicando las ecuaciones (6.13), (6.14) y (6.16):

c	N (lecturas)	bases sin cubrir, Ge^{-c}	islas, $Ne^{-c\sigma}$	longitud media de isla
1	33 333	1 839 397	14 978	214 pb
5	166 667	33 690	3053	1638 pb
10	333 333	227	112	44,7 kb

Con $5\times$ tendríamos más de tres mil fragmentos; con $10\times$, alrededor de un centenar. Con $30\times$ la teoría predice $Ne^{-24} \approx 4 \times 10^{-5}$ islas adicionales, es decir, esencialmente un único *contig*. ¿Por qué entonces los ensamblajes reales de lecturas cortas a $30\times$ o $50\times$ terminan en decenas o cientos de *contigs*?

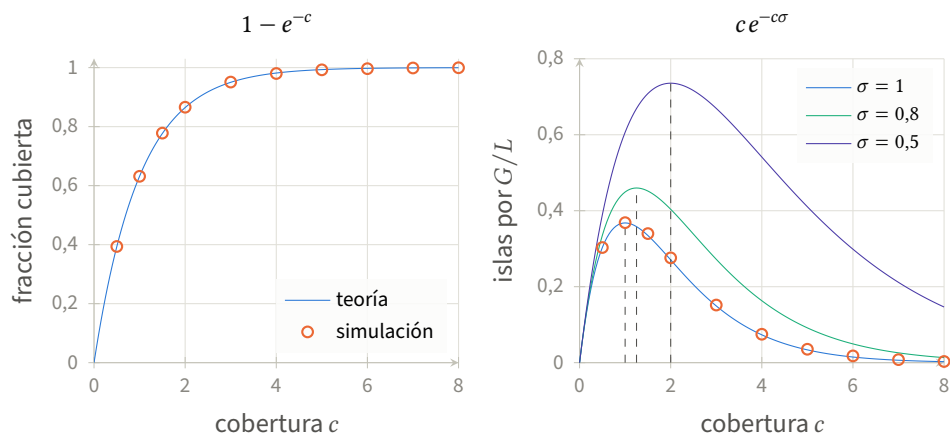


Figura 6.12. Las dos curvas de Lander-Waterman. **Izquierda:** fracción del genoma cubierta por al menos una lectura. **Derecha:** número esperado de islas (*contigs*), en unidades de G/L , para tres valores de $\sigma = 1 - T/L$; las líneas discontinuas marcan los máximos en $c^* = 1/\sigma$ (ecuación (6.17)). Los círculos naranjas son el promedio de cinco simulaciones con $G = 10^6$ y $L = 500$ (con $\sigma = 1$) y coinciden con la teoría. Exigir más solapamiento (menor σ) retrasa la fusión de islas: para la misma cobertura hay más *contigs*.

Porque el modelo ignora dos cosas: las *repeticiones* y el *sesgo de cobertura*. Las abordamos a continuación.

Lo que Lander-Waterman no ve: las repeticiones

El modelo supone que dos lecturas que se solapan *siempre* pueden unirse sin ambigüedad. Si el genoma contiene una repetición más larga que las lecturas (un elemento transponible, un operón de ARN ribosómico), las lecturas que caen dentro de ella son compatibles con varias posiciones y el ensamblador no puede decidir cómo continuar: la isla se rompe *independientemente de la cobertura*. Ninguna profundidad de lecturas cortas resuelve una repetición de 6 kb; una sola lectura de 20 kb que la atravesase de lado a lado, sí. Esta es la razón de fondo por la que las lecturas largas han transformado el ensamblaje: las lecturas ultralargas de Nanopore permitieron cerrar huecos de la referencia humana GRCh38 (Jain et al., 2018), y las HiFi producen *contigs* de decenas de megabases (Wenger et al., 2019). Lo retomaremos al estudiar el ensamblaje *de novo*.

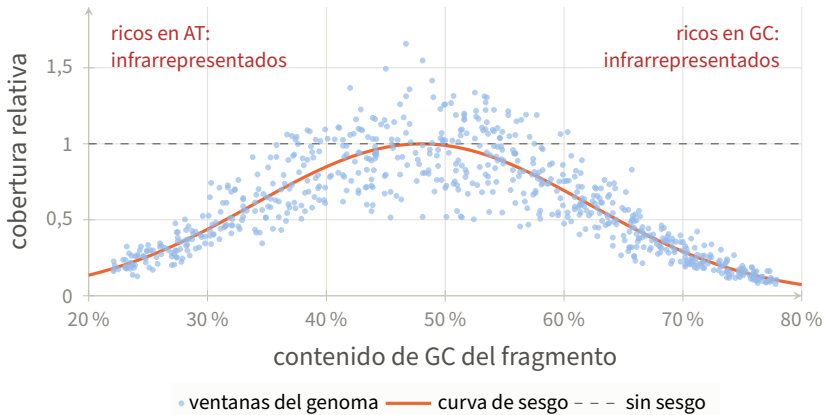


Figura 6.13. Esquema del sesgo de GC (datos simulados). Cada punto es una ventana del genoma; su cobertura relativa depende del contenido de GC según una curva unimodal, como la descrita por Benjamini y Speed (2012), más ruido de muestreo. Si el sesgo no se corrige, las regiones de GC extremo parecen tener menos copias de ADN de las que realmente tienen, un error grave al estimar variaciones en el número de copias. La forma exacta de la curva cambia entre bibliotecas, por lo que debe estimarse en cada muestra.

6.3.4 Más allá de Poisson: sesgos y sobredispersión

La distribución de Poisson tiene varianza igual a su media. Los datos reales casi nunca la cumplen: la varianza observada de la profundidad es mayor, a veces mucho mayor. Se dice entonces que la profundidad está *sobredispersa*. La causa principal es que el supuesto (i), inicio uniforme de las lecturas, es falso: la probabilidad de que un fragmento acabe secuenciado depende de su secuencia.

El sesgo mejor estudiado es el del **contenido de GC**. Benjamini y Speed (2012) analizaron datos de Illumina y encontraron que la relación entre el número de fragmentos y su contenido de GC tiene forma *unimodal*: tanto los fragmentos muy ricos en GC como los muy ricos en AT están infrarrepresentados. Mostraron además que lo que importa es el GC del *fragmento completo*, no solo el de la lectura, lo que apunta a la PCR de la biblioteca como causa principal, y que la forma de la curva varía de una muestra a otra (figura 6.13). Ross et al. (2013) compararon varias plataformas y concluyeron que PacBio presenta la cobertura menos sesgada, seguida de Illumina, pero que todas las tecnologías muestran sesgos en regiones extremas de GC y en homopolímeros largos; en un genoma humano secuenciado a 120× con Illumina, el 0,20 % del genoma autosómico quedó cubierto a menos del 10 % de la media, entre ellas muchas regiones promotoras ricas en GC.

Un modelo para la sobredispersión. Si la tasa de muestreo varía de una posición a otra (por GC, mapeabilidad u otros factores), podemos modelarla como una variable aleatoria. La elección clásica es una tasa con distribución gamma.

Teorema 6.6. (Mezcla Poisson-gamma)

Si, en cada posición, $D \mid \lambda \sim \text{Poisson}(\lambda)$ y la tasa local sigue $\lambda \sim \text{Gamma}(r, c/r)$ (forma r , escala c/r), la profundidad marginal es binomial negativa:

$$\mathbb{P}(D = k) = \frac{\Gamma(k+r)}{k! \Gamma(r)} \left(\frac{r}{r+c} \right)^r \left(\frac{c}{r+c} \right)^k, \quad \mathbb{E}[D] = c, \quad \text{Var}(D) = c + \frac{c^2}{r}. \quad (6.18)$$

DONDE

λ	Tasa de muestreo local (profundidad esperada en esa posición).
r	Parámetro de forma; mide la homogeneidad: $r \rightarrow \infty$ recupera la Poisson.
$\Gamma(\cdot)$	Función gamma, extensión continua del factorial: $\Gamma(n) = (n-1)!$.

La derivación es un cálculo directo: se integra la Poisson contra la densidad gamma,

$$\mathbb{P}(D = k) = \int_0^\infty \frac{e^{-\lambda} \lambda^k}{k!} \frac{\lambda^{r-1} e^{-\lambda r/c}}{\Gamma(r) (c/r)^r} d\lambda = \frac{1}{k! \Gamma(r) (c/r)^r} \int_0^\infty \lambda^{k+r-1} e^{-\lambda(1+r/c)} d\lambda,$$

y la última integral vale $\Gamma(k+r)/(1+r/c)^{k+r}$; reordenando se llega a la ecuación (6.18). La varianza se obtiene por la ley de la varianza total: $\text{Var}(D) = \mathbb{E}[\text{Var}(D \mid \lambda)] + \text{Var}(\mathbb{E}[D \mid \lambda]) = \mathbb{E}[\lambda] + \text{Var}(\lambda) = c + c^2/r$. El término c^2/r es la sobredispersión: crece con el cuadrado de la cobertura, así que *secuenciar más no la elimina*.

6.3.5 ¿Cuánta profundidad necesito?

Con estos modelos podemos responder la pregunta de diseño más frecuente. Suponga que, para llamar con confianza una variante en una posición, necesitamos al menos k lecturas, y que queremos que eso ocurra en una fracción $1 - \alpha$ del genoma. Buscamos el menor c tal que

$$\mathbb{P}(D \geq k \mid c) = 1 - F(k-1 \mid c) \geq 1 - \alpha, \quad (6.19)$$

donde F es la función de distribución acumulada del modelo elegido (Poisson o binomial negativa). No hay fórmula cerrada, pero la búsqueda numérica es trivial.

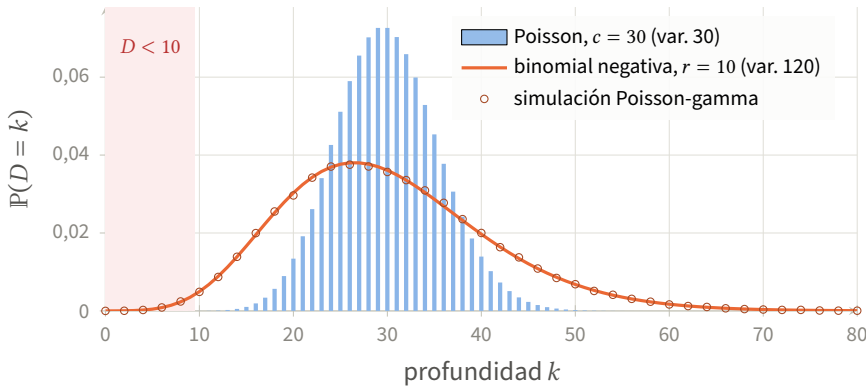


Figura 6.14. Distribución de la profundidad para una cobertura media de $30\times$. Las barras azules son el modelo ideal de Poisson; la curva naranja, una binomial negativa con la misma media y cuatro veces su varianza ($r = 10$), y los círculos, una simulación de 200 000 posiciones con tasas gamma (ecuación (6.18)). Bajo Poisson, la fracción de posiciones con menos de 10 lecturas (zona roja) es 7×10^{-6} ; con sobredispersión es del 0,9 %, más de mil veces mayor. En un genoma humano eso son decenas de millones de bases donde no se podrán llamar variantes con confianza.

DONDE

k	Profundidad mínima necesaria en una posición (p. ej., 10 lecturas).
α	Fracción tolerada del genoma por debajo de ese mínimo (p. ej., 0,01).
$F(\cdot c)$	Función de distribución acumulada de la profundidad con cobertura media c .

Ejemplo 6.5. (Diseño de la profundidad de un genoma humano)

Queremos que el 99 % del genoma tenga al menos 10 lecturas. Resolviendo la ecuación (6.19) numéricamente:

- con el modelo de Poisson basta $c \geq 18,8\times$;
- con una binomial negativa de $r = 10$ hace falta $c \geq 29,4\times$;
- con un sesgo mayor ($r = 5$), $c \geq 44,1\times$.

Si exigimos 20 lecturas, los valores suben a $31,9\times$, $53,9\times$ y $83,3\times$. El modelo ideal subestima la profundidad necesaria en un factor de 1,5 a 2. No es casual que la recomendación habitual para la resecuenciación de genomas humanos

con lecturas cortas se sitúe en torno a 30× (Bentley et al., 2008; Sims et al., 2014): es aproximadamente lo que pide un modelo realista para tener unas diez lecturas en casi todo el genoma. Sims et al. (2014) revisan cómo cambian estas consideraciones para otros diseños, como exomas, RNA-seq o ChIP-seq, donde la “cobertura” relevante no es uniforme por naturaleza.

profundidad.py

```
import numpy as np
from scipy import stats
from scipy.optimize import brentq

def lander_waterman(G, L, c, T=0):
    s = 1 - T / L
    N = c * G / L
    return {"lecturas": N,
            "sin_cubrir_pb": G * np.exp(-c),
            "islas": N * np.exp(-c * s),
            "long_isla": L * ((np.exp(c * s) - 1) / c + 1 - s)}

def cobertura_necesaria(k, alfa=0.01, r=None):
    """Menor c con  $P(D \geq k) \geq 1 - \text{alfa}$  (Poisson o NB)."""
    def f(c):
        if r is None:
            return stats.poisson.sf(k - 1, c) - (1 - alfa)
        return stats.nbinom.sf(k - 1, r, r / (r + c)) - (1 - alfa)
    return brentq(f, k / 10, 50 * k)

print(lander_waterman(5e6, 150, c=5, T=30))    # ~3053 islas
print(round(cobertura_necesaria(10), 2))        # 18.78 (Poisson)
print(round(cobertura_necesaria(10, r=10), 2))  # 29.38 (NB)
```

⚠ La cobertura “bruta” engaña

La cobertura que importa es la *efectiva*: la que queda después de descartar duplicados, lecturas de baja calidad, lecturas que no mapean o que mapean en varios sitios, y bases recortadas. Un experimento anunciado como 30× puede quedarse en 22× útiles tras estas pérdidas. Planifique con la cobertura efectiva y verifique siempre la distribución observada de la profundidad, no solo su media.

Con cobertura c , falta una fracción e^{-c} del genoma y hay unas $Ne^{-c\sigma}$ islas. Los datos reales son sobredispersos: diseñe con un modelo binomial negativo y con la cobertura efectiva, no con la bruta.

En resumen

- El método de Sanger lee fragmentos terminados por ddNTP; su longitud está limitada por la estadística geométrica de la terminación y por la resolución de la electroforesis. La pirosecuenciación (454) paralelizó la lectura, pero comete *indels* en homopolímeros.
- Illumina amplifica cada molécula en un clúster (amplificación en puente) y la lee con terminadores reversibles, una base por ciclo. El desfase acumulado, $\phi(n) = (1 - \varepsilon)^n$, explica la caída de calidad hacia 3'; su error típico es la sustitución.
- PacBio y Oxford Nanopore leen moléculas únicas de decenas a cientos de kilobases. Los errores aleatorios de PacBio se corrigen por consenso circular (HiFi); los de Nanopore tienen un componente sistemático en homopolímeros que las químicas recientes han reducido.
- La calidad Phred es $Q = -10 \log_{10} p_i$, codificada en FASTQ como $\text{ord}(\kappa) - 33$. El número esperado de errores de una lectura es $\sum_i p_i$.
- FastQC y MultiQC diagnostican calidad, contenido, duplicación y adaptadores. La tasa de duplicados depende de la complejidad de la biblioteca: $E[D] \approx M(1 - e^{-N/M})$. Los adaptadores se recortan siempre; la calidad, con ventanas deslizantes o sumas acumuladas y umbrales moderados.
- Con cobertura $c = NL/G$, la profundidad es aproximadamente Poisson(c): falta una fracción e^{-c} del genoma y se esperan $Ne^{-c\sigma}$ islas, con un máximo en $c = 1/\sigma$. El sesgo de GC produce sobredispersión (binomial negativa), que eleva la profundidad necesaria hacia 30× para genomas humanos.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 6.1 ●●● Decodifique a mano la línea de calidades `II?5+#` en `Phred+33`: obtenga Q , p y el número esperado de errores de esas seis bases. ¿Qué calidades obtendría si, por error, leyera el archivo como `Phred+64`?
- 6.2 ●●● Un genoma de levadura mide 12 Mb. ¿Cuántas lecturas de 150 pb necesita para una cobertura de 40×? Si cada lectura pareada aporta dos lecturas, ¿cuántos pares?
- 6.3 ●●● Explique, a partir del mecanismo físico, por qué 454 y Nanopore cometen *indels* en homopolímeros mientras que Illumina comete sobre todo sustituciones.
- 6.4 ●●● Con la ecuación (6.2), estime la tasa de desfase ε que haría que en el ciclo 250 solo la mitad de las hebras siguieran en fase. ¿Cómo cambiaría la longitud máxima útil si ε se redujera a la mitad?
- 6.5 ●●● Usando la ecuación (6.3), calcule cuántas pasadas se necesitan para alcanzar Q_{30} si el error por pasada es del 15 %. Discuta por qué el modelo es pesimista.
- 6.6 ●●● Una biblioteca de ChIP-seq muestra un 60 % de duplicados con 3×10^7 lecturas. Estime su complejidad M con la ecuación (6.8) y prediga la tasa de duplicados si se secuencian 6×10^7 lecturas. ¿Vale la pena?
- 6.7 ●●● Implemente el criterio de la ecuación (6.11) y compárelo con la ventana deslizante sobre un archivo FASTQ real: ¿qué fracción de bases elimina cada uno con $t = 20$? Visualice el resultado con FastQC antes y después.
- 6.8 ●●● Simule el experimento de Lander-Waterman con $G = 10^6$, $L = 100$ y $T = 20$, para $c \in \{0, 5, 1, \dots, 8\}$, y compare el número de islas y su longitud media con las ecuaciones (6.14) y (6.16). ¿Dónde se desvía la simulación de la teoría y por qué?
- 6.9 ●●● Demuestre que, bajo la mezcla Poisson-gamma, la fracción del genoma no cubierta es $(1 + c/r)^{-r}$ y compárela con e^{-c} para $c = 10$ y $r = 5$. Interprete: ¿qué nos dice sobre la eficacia de “secuenciar más” para cerrar huecos cuando hay sesgo?
- 6.10 ●●● Descargue un archivo BAM público de un genoma bacteriano, calcule la profundidad por posición con `samtools depth` y ajuste una Poisson y una

binomial negativa. Estime r y relacione las posiciones de baja cobertura con su contenido de GC en ventanas de 500 pb.

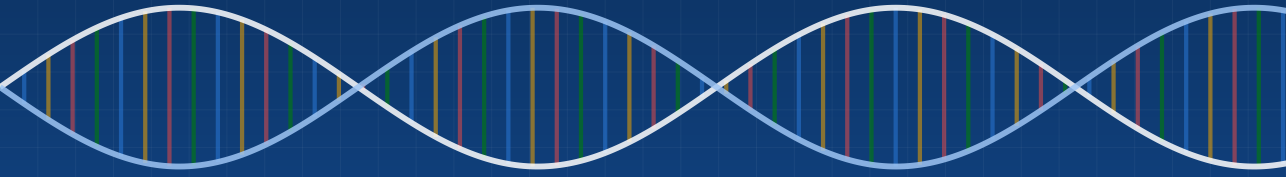
Referencias del capítulo

- Andrews, S. (2010). *FastQC: A quality control tool for high throughput sequence data*. <https://www.bioinformatics.babraham.ac.uk/projects/fastqc/>
- Benjamini, Y., & Speed, T. P. (2012). Summarizing and correcting the GC content bias in high-throughput sequencing. *Nucleic Acids Research*, 40(10), e72. <https://doi.org/10.1093/nar/gks001>
- Bentley, D. R., Balasubramanian, S., Swerdlow, H. P., Smith, G. P., Milton, J., et al. (2008). Accurate whole human genome sequencing using reversible terminator chemistry. *Nature*, 456(7218), 53-59. <https://doi.org/10.1038/nature07517>
- Bolger, A. M., Lohse, M., & Usadel, B. (2014). Trimmomatic: a flexible trimmer for Illumina sequence data. *Bioinformatics*, 30(15), 2114-2120. <https://doi.org/10.1093/bioinformatics/btu170>
- Chen, S., Zhou, Y., Chen, Y., & Gu, J. (2018). fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics*, 34(17), i884-i890. <https://doi.org/10.1093/bioinformatics/bty560>
- Cock, P. J. A., Fields, C. J., Goto, N., Heuer, M. L., & Rice, P. M. (2010). The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, 38(6), 1767-1771. <https://doi.org/10.1093/nar/gkp1137>
- Eid, J., Fehr, A., Gray, J., Luong, K., Lyle, J., et al. (2009). Real-time DNA sequencing from single polymerase molecules. *Science*, 323(5910), 133-138. <https://doi.org/10.1126/science.1162986>
- Ewels, P., Magnusson, M., Lundin, S., & Käller, M. (2016). MultiQC: summarize analysis results for multiple tools and samples in a single report. *Bioinformatics*, 32(19), 3047-3048. <https://doi.org/10.1093/bioinformatics/btw354>
- Ewing, B., & Green, P. (1998). Base-calling of automated sequencer traces using Phred. II. Error probabilities. *Genome Research*, 8(3), 186-194. <https://doi.org/10.1101/gr.8.3.186>
- Ewing, B., Hillier, L., Wendl, M. C., & Green, P. (1998). Base-calling of automated sequencer traces using Phred. I. Accuracy assessment. *Genome Research*, 8(3), 175-185. <https://doi.org/10.1101/gr.8.3.175>
- Fedurco, M., Romieu, A., Williams, S., Lawrence, I., & Turcatti, G. (2006). BTA, a novel reagent for DNA attachment on glass and efficient generation of solid-phase amplified DNA colonies. *Nucleic Acids Research*, 34(3), e22. <https://doi.org/10.1093/nar/gnj023>
- Goodwin, S., McPherson, J. D., & McCombie, W. R. (2016). Coming of age: ten years of next-generation sequencing technologies. *Nature Reviews Genetics*, 17(6), 333-351. <https://doi.org/10.1038/nrg.2016.49>

- Jain, M., Koren, S., Miga, K. H., Quick, J., Rand, A. C., et al. (2018). Nanopore sequencing and assembly of a human genome with ultra-long reads. *Nature Biotechnology*, 36(4), 338-345. <https://doi.org/10.1038/nbt.4060>
- Jain, M., Olsen, H. E., Paten, B., & Akeson, M. (2016). The Oxford Nanopore MinION: delivery of nanopore sequencing to the genomics community. *Genome Biology*, 17(1), Artículo 239. <https://doi.org/10.1186/s13059-016-1103-0>
- Lander, E. S., & Waterman, M. S. (1988). Genomic mapping by fingerprinting random clones: A mathematical analysis. *Genomics*, 2(3), 231-239. [https://doi.org/10.1016/0888-7543\(88\)90007-9](https://doi.org/10.1016/0888-7543(88)90007-9)
- Margulies, M., Egholm, M., Altman, W. E., Attiya, S., Bader, J. S., et al. (2005). Genome sequencing in microfabricated high-density picolitre reactors. *Nature*, 437(7057), 376-380. <https://doi.org/10.1038/nature03959>
- Martin, M. (2011). Cutadapt removes adapter sequences from high-throughput sequencing reads. *EMBnet.journal*, 17(1), 10-12. <https://doi.org/10.14806/ej.17.1.200>
- Metzker, M. L. (2010). Sequencing technologies — the next generation. *Nature Reviews Genetics*, 11(1), 31-46. <https://doi.org/10.1038/nrg2626>
- National Human Genome Research Institute. (2022). *DNA sequencing costs: Data*. Consultado el 29 de septiembre de 2026, desde <https://www.genome.gov/about-genomics/fact-sheets/DNA-Sequencing-Costs-Data>
- Rang, F. J., Kloosterman, W. P., & de Ridder, J. (2018). From squiggle to basepair: computational approaches for improving nanopore sequencing read accuracy. *Genome Biology*, 19(1), Artículo 90. <https://doi.org/10.1186/s13059-018-1462-9>
- Ronaghi, M., Uhlén, M., & Nyrén, P. (1998). A sequencing method based on real-time pyrophosphate. *Science*, 281(5375), 363-365. <https://doi.org/10.1126/science.281.5375.363>
- Ross, M. G., Russ, C., Costello, M., Hollinger, A., Lennon, N. J., Hegarty, R., Nusbaum, C., & Jaffe, D. B. (2013). Characterizing and measuring bias in sequence data. *Genome Biology*, 14(5), Artículo R51. <https://doi.org/10.1186/gb-2013-14-5-r51>
- Sanger, F., Nicklen, S., & Coulson, A. R. (1977). DNA sequencing with chain-terminating inhibitors. *Proceedings of the National Academy of Sciences*, 74(12), 5463-5467. <https://doi.org/10.1073/pnas.74.12.5463>
- Schirmer, M., Ijaz, U. Z., D'Amore, R., Hall, N., Sloan, W. T., & Quince, C. (2015). Insight into biases and sequencing errors for amplicon sequencing with the Illumina MiSeq platform. *Nucleic Acids Research*, 43(6), e37. <https://doi.org/10.1093/nar/gku1341>
- Sereika, M., Kirkegaard, R. H., Karst, S. M., Michaelsen, T. Y., Sørensen, E. A., Wollenberg, R. D., & Albertsen, M. (2022). Oxford Nanopore R10.4 long-read sequencing enables the generation of near-finished bacterial genomes from pure cultures and metagenomes without short-read or reference polishing. *Nature Methods*, 19(7), 823-826. <https://doi.org/10.1038/s41592-022-01539-7>

- Shendure, J., Balasubramanian, S., Church, G. M., Gilbert, W., Rogers, J., Schloss, J. A., & Waterston, R. H. (2017). DNA sequencing at 40: past, present and future. *Nature*, 550(7676), 345-353. <https://doi.org/10.1038/nature24286>
- Sims, D., Sudbery, I., Iltott, N. E., Heger, A., & Ponting, C. P. (2014). Sequencing depth and coverage: key considerations in genomic analyses. *Nature Reviews Genetics*, 15(2), 121-132. <https://doi.org/10.1038/nrg3642>
- Waterman, M. S. (1995). *Introduction to computational biology: Maps, sequences and genomes*. Chapman & Hall.
- Wenger, A. M., Peluso, P., Rowell, W. J., Chang, P.-C., Hall, R. J., et al. (2019). Accurate circular consensus long-read sequencing improves variant detection and assembly of a human genome. *Nature Biotechnology*, 37(10), 1155-1162. <https://doi.org/10.1038/s41587-019-0217-9>

Mapeo de lecturas



Ignorar sin más las repeticiones no es una opción: eso crea sus propios problemas y puede hacer que pasen inadvertidos fenómenos biológicos importantes.

— Treangen y Salzberg (2012)

● Al terminar este capítulo usted podrá

- Explicar por qué el alineamiento por programación dinámica no escala al mapeo de cientos de millones de lecturas, y comparar las dos familias de índices que lo resuelven: tablas de *hash* de semillas e índices de sufijos.
- Construir a mano el arreglo de sufijos, la transformada de Burrows-Wheeler y el índice FM de una secuencia corta, y ejecutar paso a paso la búsqueda hacia atrás con el *LF-mapping*.
- Describir el paradigma semilla-encadenamiento-extensión de BWA-MEM y minimap2, incluidos los minimizadores, la función de encadenamiento y la calidad de mapeo (MAPQ), y leer sin ayuda un registro SAM.
- Construir un flujo reproducible con `samtools` (ordenar, marcar duplicados, indexar), inspeccionar alineamientos en IGV y usar perfiles de cobertura para detectar deleciones y duplicaciones.

Al terminar el capítulo 6 obtuvimos un FASTQ limpio con millones de lecturas de 150 bases y sus calidades Phred. La teoría de Lander-Waterman nos ayudó a estimar cuántas hacen falta para cubrir un genoma. Ahora necesitamos averiguar de qué región procede cada lectura. El *mapeo* la sitúa sobre una secuencia de referencia y sirve de base para el llamado de variantes, la cuantificación de RNA-seq, el análisis de ChIP-seq y los estudios de metilación. Un error de mapeo puede propagarse a cualquiera de esas etapas.

El problema parece una aplicación directa del capítulo 3: basta con alinear cada lectura contra el genoma con Smith-Waterman. La dificultad es de escala. Un genoma humano a 30× son unos 620 millones de lecturas de 150 pares de bases, y la referencia mide $3,1 \times 10^9$ bases. Li y Homer (2010) resumieron la respuesta de la comunidad en una frase que hoy parece obvia pero que en 2008 no lo era: con los algoritmos adecuados, el alineamiento de lecturas cortas “ha dejado de ser el cuello de botella” del análisis. Este capítulo cuenta cómo se logró.

Empezaremos por la idea algorítmica más bella del área: una permutación de la secuencia, la transformada de Burrows-Wheeler, que permite buscar cualquier patrón en un genoma humano ocupando menos memoria que el propio genoma (sección 7.1). Después veremos cómo las herramientas reales (BWA, Bowtie, minimap2) combinan índices, semillas y programación dinámica, cómo expresan su confianza con la calidad de mapeo y cómo escriben el resultado en el formato SAM/BAM que manipularemos con `samtools` (sección 7.2). Terminaremos mirando los alineamientos con los ojos, en IGV, y convirtiéndolos en perfiles de cobertura capaces de revelar deleciones y duplicaciones (sección 7.3).

7.1 Transformada de Burrows-Wheeler y FM-index

 Colab · 7.1 · Transformada de Burrows-Wheeler y FM-index  [abrir notebook](#)

Un diccionario de todos los finales

Imagine un diccionario muy extraño: en lugar de palabras, contiene *todos los finales* de un libro. Cada entrada empieza en una letra distinta del texto y continúa hasta el punto final, y todas están ordenadas alfabéticamente. Si quiere saber dónde aparece la frase “había una vez”, no necesita leer el libro: abre el diccionario por la mitad, decide si su frase va antes o después, vuelve

a partir por la mitad, y en una veintena de saltos llega al bloque de entradas que empiezan con esa frase. Cada entrada, además, recuerda en qué página empezaba. El problema es que el diccionario pesa muchísimo más que el libro. La transformada de Burrows-Wheeler es el truco que permite quedarse con una sola columna de ese diccionario, tan delgada como el propio texto, sin perder la capacidad de buscar.

7.1.1 El problema del mapeo

Formalicemos lo que pedimos a un mapeador. Sea $R = r_1 r_2 \dots r_G$ la secuencia de referencia (un genoma, con sus cromosomas concatenados) y $q = q_1 \dots q_m$ una lectura, con $m \ll G$.

Definición 7.1. (Mapeo de una lectura)

Dada una función de puntuación de alineamiento local $S(q, R_{u..v})$ (por ejemplo, la de Smith-Waterman con huecos afines del capítulo 3), mapear q consiste en encontrar el intervalo de la referencia

$$(\hat{u}, \hat{v}) = \arg \max_{1 \leq u \leq v \leq G} \max \{S(q, R_{u..v}), S(\bar{q}, R_{u..v})\}, \quad (7.1)$$

y en cuantificar la probabilidad de que $(\hat{u}, \hat{v})$ sea el origen verdadero de la lectura.

DONDE

R, G	Secuencia de referencia y su longitud en bases.
q, m	Lectura y su longitud.
$\bar{q}$	Reverso complementario de la lectura: el fragmento pudo secuenciarse desde cualquiera de las dos hebras.
$R_{u..v}$	Subcadena de la referencia entre las posiciones u y v .
$S(\cdot, \cdot)$	Puntuación del mejor alineamiento local entre la lectura y la subcadena.

La segunda mitad de la definición es tan importante como la primera, y a menudo se olvida: un mapeador no solo responde *dónde*, sino *con qué seguridad*. Volveremos a ello con la calidad de mapeo en la sección 7.2.

Resolver la ecuación (7.1) por fuerza bruta es imposible. Con Smith-Waterman, cada lectura exige llenar una matriz de $m \times G = 150 \times 3,1 \times 10^9 \approx 4,7 \times 10^{11}$ celdas; multiplicado por $6,2 \times 10^8$ lecturas, son unos $2,9 \times 10^{20}$ celdas. Aun a mil millones de celdas por segundo, el cálculo tardaría unos nueve mil años. Necesitamos *filtrar*: descartar en un instante casi todo el genoma y reservar la programación dinámica para unas pocas regiones candidatas.

7.1.2 Semillas y tablas de *hash*

La primera generación de mapeadores tomó prestada la idea de BLAST: si la lectura procede de la posición u , casi seguro comparte con la referencia algún k -mero exacto (una *semilla*). Basta con un índice que, para cada k -mero, liste las posiciones donde aparece. ¿Qué k elegir? Si el genoma fuera una secuencia aleatoria de bases equiprobables, el número esperado de apariciones *casuales* de un k -mero concreto sería

$$E[N_k] = \frac{G - k + 1}{4^k} \approx \frac{G}{4^k}. \tag{7.2}$$

DONDE	
N_k	Número de posiciones de la referencia donde aparece, por azar, un k -mero dado.
$G - k + 1$	Número de k -meros (posiciones de inicio) en la referencia.
4^k	Número de k -meros distintos posibles sobre {A, C, G, T}.

Para el genoma humano, un 12-mero aparece por azar unas 185 veces, un 16-mero 0,72 veces y un 20-mero apenas 0,0028 veces (figura 7.1). Las semillas deben tener, pues, unas 20 bases para ser *informativas*. Pero una tabla directa indexada por k -mero necesita 4^k entradas: con enteros de 32 bits, 64 MiB para $k = 12$ y 16 GiB para $k = 16$, antes de guardar una sola posición. Por eso los mapeadores basados en *hash*, como MAQ (Li et al., 2008), indexaban semillas cortas, a menudo espaciadas, y a veces indexaban las *lecturas* en lugar del genoma. Funcionaban, pero eran lentos cuando había que escalar a cientos de individuos (Li & Durbin, 2009).

La alternativa, que acabó por imponerse para lecturas cortas, fue indexar *todos los sufijos* de la referencia. Un índice de sufijos no depende de un k fijo: permite preguntar por un patrón de cualquier longitud y, como veremos, extenderlo carácter a carácter hasta que deje de aparecer.

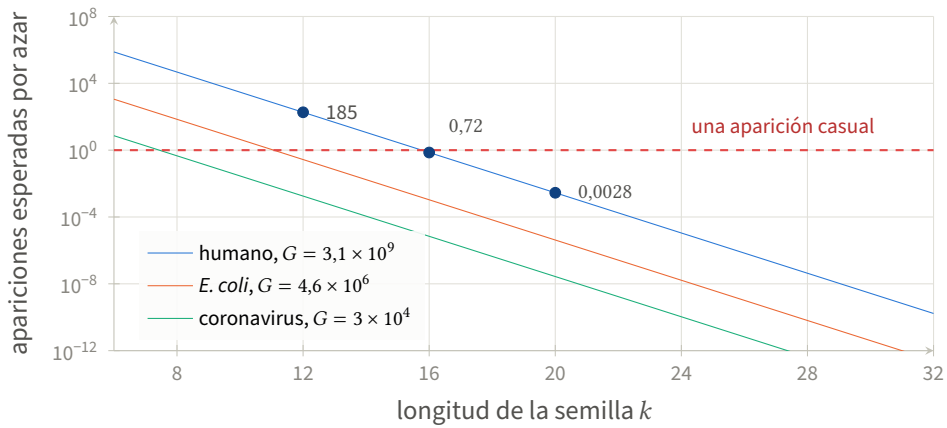


Figura 7.1. Número esperado de apariciones casuales de un k -mero en genomas aleatorios de distinto tamaño (ecuación (7.2)). En escala logarítmica cada base adicional divide la cifra por cuatro. En el genoma humano, una semilla necesita unas 20 bases para que una coincidencia exacta sea más probablemente verdadera que casual; en un virus basta con 8 o 9. Los genomas reales no son aleatorios: sus repeticiones hacen que muchos k -meros largos aparezcan miles de veces, como veremos en la sección 7.2.

7.1.3 El arreglo de sufijos

Añadimos al final de la referencia un símbolo centinela $\$$ que no pertenece al alfabeto y que, por convenio, es menor que cualquier base. Así, $T = R\$$ tiene longitud $n = G + 1$ y ningún sufijo es prefijo de otro.

Definición 7.2. (Arreglo de sufijos)

El arreglo de sufijos de T es la permutación $SA[0..n-1]$ de $\{0, \dots, n-1\}$ tal que

$$T[SA[0]..] < T[SA[1]..] < \dots < T[SA[n-1]..] \quad (7.3)$$

en orden lexicográfico, donde $T[i..]$ denota el sufijo que empieza en la posición i (numeramos desde cero).

DONDE

T, n	Texto indexado (referencia más centinela) y su longitud.
$T[i..]$	Sufijo de T que empieza en la posición i .
$SA[r]$	Posición en T del sufijo que ocupa el lugar r en el orden alfabético.

La propiedad clave es que todos los sufijos que empiezan por un patrón P quedan *contiguos* en el arreglo, porque comparten prefijo. Encontrar las apariciones de P se reduce a encontrar un *intervalo* $[sp, ep)$ de filas, lo que se hace con dos búsquedas binarias en $O(m \log n)$ comparaciones de caracteres. Manber y Myers (1993) propusieron esta estructura como alternativa compacta a los árboles de sufijos, que resuelven el mismo problema en tiempo $O(m)$ pero ocupan, en la práctica, más de diez bytes por base (Gusfield, 1997). Aun así, el arreglo de sufijos necesita $n \lceil \log_2 n \rceil$ bits: para el genoma humano, 32 bits por posición, unos 12 GB. En 2008, eso era más memoria de la que tenía un servidor típico.

7.1.4 La transformada de Burrows-Wheeler

Escribamos todas las rotaciones cíclicas de T , una debajo de otra, y ordenémoslas alfabéticamente. Obtenemos una matriz M de $n \times n$ caracteres (figura 7.2). Como T termina en el centinela único, ordenar las rotaciones equivale a ordenar los sufijos: la fila r de M empieza con el sufijo $T[SA[r]..]$.

Definición 7.3. (Transformada de Burrows-Wheeler)

La BWT de T es la última columna L de la matriz de rotaciones ordenadas. Equivalentemente, a partir del arreglo de sufijos,

$$L[r] = \begin{cases} T[SA[r] - 1] & \text{si } SA[r] > 0, \\ \$ & \text{si } SA[r] = 0. \end{cases} \quad (7.4)$$

Llamamos F a la primera columna de M : son los caracteres de T ordenados.

DONDE

M	Matriz cuyas filas son las n rotaciones de T en orden lexicográfico.
L	Última columna de M : la transformada de Burrows-Wheeler, BWT(T).
F	Primera columna de M ; se deduce de L ordenando sus caracteres.

Rotaciones de T		Matriz ordenada M	
i		$r(SA)$	
0	T A C A T A C A G \$	0 (9)	\$ T A C A T A C A G
1	A C A T A C A G \$ T	1 (5)	A C A G \$ T A C A T
2	C A T A C A G \$ T A	2 (1)	A C A T A C A G \$ T
3	A T A C A G \$ T A C	3 (7)	A G \$ T A C A T A C
4	T A C A G \$ T A C A	4 (3)	A T A C A G \$ T A C
5	A C A G \$ T A C A T	5 (6)	C A G \$ T A C A T A
6	C A G \$ T A C A T A	6 (2)	C A T A C A G \$ T A
7	A G \$ T A C A T A C	7 (8)	G \$ T A C A T A C A
8	G \$ T A C A T A C A	8 (4)	T A C A G \$ T A C A
9	\$ T A C A T A C A G	9 (0)	T A C A T A C A G \$
			<i>F</i> <i>L</i>

ordenar →

$BWT(T) = L = GTTCCAAAA\$$

Figura 7.2. Construcción de la BWT de $T = TACATACAG\$$. **Izquierda:** las diez rotaciones cíclicas de T , indexadas por su posición de inicio i . **Derecha:** las mismas rotaciones en orden alfabético ($\$$ precede a todas las bases). Junto a cada fila r aparece, entre paréntesis, $SA[r]$. La primera columna F (azul) contiene las letras de T ordenadas; la última, L (naranja), es la transformada. Observe cómo las cuatro A de L quedan juntas: los sufijos que empiezan por C, G o T tienden a estar precedidos por la misma letra, y la BWT agrupa esas repeticiones. Generada con `figuras/cap07/generar.py`.

En palabras: $L[r]$ es el carácter que *precede*, en el texto, al sufijo de la fila r . Para nuestro ejemplo de juguete, $T = TACATACAG\$$, el arreglo de sufijos es $(9, 5, 1, 7, 3, 6, 2, 8, 4, 0)$ y la transformada es $L = GTTCCAAAA\$$.

Un algoritmo de compresión que nadie esperaba

Michael Burrows y David Wheeler describieron la transformada en 1994, en un reporte técnico del Centro de Investigación de Sistemas de Digital Equipment Corporation, con un propósito muy distinto al nuestro: comprimir texto. Su observación fue que L tiende a formar rachas largas de un mismo carácter (en inglés, las letras que preceden a “he” suelen ser t), lo que la hace muy fácil de comprimir, y que, sorprendentemente, la permutación es *reversible*. El compresor `bzip2` se basa en ella. Adjero et al. (2008) reconstruyen esta historia y su conexión con los arreglos de sufijos. Pasaron seis años hasta que Paolo Ferragina y Giovanni Manzini, en un congreso de 2000, se dieron cuenta de que la BWT comprimida no solo guardaba el texto, sino que permitía *buscar* en él; la versión extendida de su trabajo es Ferragina

y Manzini (2005). Y pasaron otros nueve años hasta que Bowtie y BWA llevaron la idea a la genómica (Langmead et al., 2009; Li & Durbin, 2009).

7.1.5 El *LF-mapping*: por qué la BWT es reversible

¿Cómo recuperar T a partir de L sola? La clave es una propiedad que, una vez vista, parece evidente.

Teorema 7.1. (Orden de las apariciones)

La j -ésima aparición de un carácter c en la columna L y la j -ésima aparición de c en la columna F corresponden a la misma posición del texto T .

Demostración. Considere dos filas de M cuya última letra es c , digamos cX rotada como Xc y Yc , con $X < Y$ porque las filas están ordenadas. Si rotamos ambas un carácter a la derecha obtenemos cX y cY , que son también filas de M (con c en la columna F), y como comparten el primer carácter, su orden relativo lo decide el resto: $cX < cY$. El orden entre las c se conserva al pasar de L a F . $\square$

Esta propiedad permite definir, para cada fila r , la fila $LF(r)$ en la que aparece como primera letra el mismo carácter que $L[r]$. Necesitamos dos tablas:

$$C[c] = \#\{i : T[i] < c\}, \quad (7.5)$$

$$\text{Occ}(c, r) = \#\{i < r : L[i] = c\}, \quad (7.6)$$

y con ellas

$$LF(r) = C[L[r]] + \text{Occ}(L[r], r). \quad (7.7)$$

DONDE

$C[c]$	Número de caracteres de T estrictamente menores que c : la fila donde empieza el bloque de c en la columna F .
$\text{Occ}(c, r)$	Número de apariciones de c en las primeras r filas de L (filas 0 a $r - 1$): el rango de $L[r]$ entre sus iguales.
$LF(r)$	Fila de M cuyo primer carácter es el mismo carácter del texto que $L[r]$; equivale a retroceder una posición en T .

La ecuación (7.7) es la traducción directa del teorema: el bloque de c en F empieza en la fila $C[c]$, y dentro del bloque las c están en el mismo orden que en L , así que la c de rango j ocupa la fila $C[c] + j$. Como la fila $LF(r)$ empieza justo con el carácter

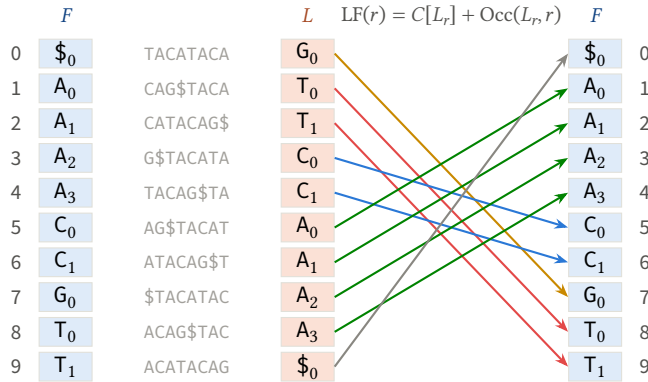


Figura 7.3. El LF-mapping para $T = \text{TACATACAG\$}$. Los subíndices numeran las apariciones de cada letra de arriba abajo. Cada flecha une la fila r de L con la fila $LF(r)$ de F (repetida a la derecha para mayor claridad): la j -ésima A de L va a la j -ésima A de F , y lo mismo para cada letra. Las flechas del mismo color nunca se cruzan; esa es la propiedad del teorema 7.1. En gris, el centro de cada fila, que el índice *no* almacena.

que precedía al sufijo de la fila r , aplicar LF equivale a *dar un paso hacia atrás* en el texto.

Ejemplo 7.1. (Reconstruir T desde su BWT)

Con $L = \text{GTTCCAAAA\$}$ y $F = \text{\$AAAACCGTT}$ tenemos $C[A] = 1$, $C[C] = 5$, $C[G] = 7$ y $C[T] = 8$. Empezamos en la fila 0, la que comienza con $\$$; su última letra, $L[0] = G$, es el último carácter de R . Aplicamos la ecuación (7.7) repetidamente:

$$0 \xrightarrow{G} 7 \xrightarrow{A} 3 \xrightarrow{C} 5 \xrightarrow{A} 1 \xrightarrow{T} 8 \xrightarrow{A} 4 \xrightarrow{C} 6 \xrightarrow{A} 2 \xrightarrow{T} 9,$$

donde sobre cada flecha escribimos $L[r]$ en la fila de partida. Por ejemplo, $LF(0) = C[G] + \text{Occ}(G, 0) = 7 + 0 = 7$ y $LF(7) = C[A] + \text{Occ}(A, 7) = 1 + 2 = 3$. Las letras recogidas, GACATACAT, leídas al revés dan TACATACAG: el texto original. En la fila 9 encontramos $L[9] = \$$ y paramos.

7.1.6 El índice FM y la búsqueda hacia atrás

El mismo mecanismo que reconstruye el texto sirve para *buscar* en él. La idea de Ferragina y Manzini (2005) es procesar el patrón $P = p_1 \dots p_m$ de derecha a izquierda, manteniendo el intervalo de filas de M que empiezan con el sufijo de P ya procesado.

Teorema 7.2. (Búsqueda hacia atrás)

Sea $[sp, ep)$ el intervalo de filas de M que empiezan con la cadena W . Entonces las filas que empiezan con cW forman el intervalo

$$sp' = C[c] + \text{Occ}(c, sp), \quad ep' = C[c] + \text{Occ}(c, ep). \quad (7.8)$$

Empezando con W vacío, $[sp, ep) = [0, n)$, y aplicando la regla para $c = p_m, p_{m-1}, \dots, p_1$, el patrón aparece en T exactamente $ep - sp$ veces; si en algún paso $sp' \geq ep'$, no aparece.

DONDE

W	Sufijo del patrón ya procesado (al principio, la cadena vacía).
$[sp, ep)$	Intervalo semiabierto de filas de M (o del arreglo de sufijos) que empiezan con W .
c	Siguiente carácter del patrón, leído de derecha a izquierda.
$[sp', ep')$	Intervalo de las filas que empiezan con cW .

Por qué funciona. Las filas que empiezan con cW son exactamente las rotaciones $cW\dots$; al rotarlas un carácter a la izquierda se convierten en filas $W\dots c$, es decir, filas del intervalo $[sp, ep)$ cuya última letra es c . Por el teorema 7.1, esas c de L se mapean, en el mismo orden, a un bloque contiguo del bloque de c en F . Las c de L situadas antes de sp son $\text{Occ}(c, sp)$; por eso el nuevo bloque empieza en $C[c] + \text{Occ}(c, sp)$ y termina justo antes de $C[c] + \text{Occ}(c, ep)$. $\square$

Lo extraordinario es el coste: cada carácter del patrón cuesta dos consultas a Occ , sin importar el tamaño del genoma. Buscar una lectura de 150 bases en el genoma humano cuesta 300 consultas, lo mismo que buscarla en el genoma de un virus.

Ejemplo 7.2. (Buscar ACA en TACATACAG\$)

Usamos la tabla 7.1. Empezamos con $[0, 10)$ y leemos el patrón al revés:

Tabla 7.1. Tabla $\text{Occ}(c, r)$ para $L = \text{GTTCCAAAA}\$$: cada fila r cuenta las apariciones de cada base en $L[0..r-1]$. La tabla C es $C[A] = 1, C[C] = 5, C[G] = 7, C[T] = 8$.

r	$L[r]$	$\text{Occ}(A, r)$	$\text{Occ}(C, r)$	$\text{Occ}(G, r)$	$\text{Occ}(T, r)$
0	G	0	0	0	0
1	T	0	0	1	0
2	T	0	0	1	1
3	C	0	0	1	2
4	C	0	1	1	2
5	A	0	2	1	2
6	A	1	2	1	2
7	A	2	2	1	2
8	A	3	2	1	2
9	\$	4	2	1	2
10	–	4	2	1	2

1. $c = A$: $sp' = 1 + \text{Occ}(A, 0) = 1 + 0 = 1$; $ep' = 1 + \text{Occ}(A, 10) = 1 + 4 = 5$. Intervalo $[1, 5)$: las cuatro filas que empiezan con A.
2. $c = C$: $sp' = 5 + \text{Occ}(C, 1) = 5$; $ep' = 5 + \text{Occ}(C, 5) = 5 + 2 = 7$. Intervalo $[5, 7)$: filas que empiezan con CA.
3. $c = A$: $sp' = 1 + \text{Occ}(A, 5) = 1$; $ep' = 1 + \text{Occ}(A, 7) = 1 + 2 = 3$. Intervalo $[1, 3)$.

Hay $3 - 1 = 2$ apariciones, en las posiciones $SA[1] = 5$ y $SA[2] = 1$ del texto: TACATACAG. Si buscamos GAC, tras procesar AC tenemos $[1, 3)$ y el paso con G da $sp' = 7 + \text{Occ}(G, 1) = 8$ y $ep' = 7 + \text{Occ}(G, 3) = 8$: el intervalo se vacía y sabemos, sin mirar el texto, que GAC no aparece.

El siguiente programa construye el índice de forma ingenua (ordenando sufijos, lo que cuesta $O(n^2 \log n)$ y solo sirve para ejemplos) y busca por el método del teorema 7.2. Los programas reales construyen el arreglo de sufijos en tiempo lineal o casi lineal, con algoritmos especializados que no requieren tener en memoria todos los sufijos (Mäkinen et al., 2015).

	Paso 0: inicio	Paso 1: A	Paso 2: CA	Paso 3: ACA
0	\$TACATACAG	\$TACATACAG	\$TACATACAG	\$TACATACAG
1	ACAG\$TACAT	ACAG\$TACAT	ACAG\$TACAT	ACAG\$TACAT
2	ACATACAG\$T	ACATACAG\$T	ACATACAG\$T	ACATACAG\$T
3	AG\$TACATAC	AG\$TACATAC	AG\$TACATAC	AG\$TACATAC
4	ATACAG\$TAC	ATACAG\$TAC	ATACAG\$TAC	ATACAG\$TAC
5	CAG\$TACATA	CAG\$TACATA	CAG\$TACATA	CAG\$TACATA
6	CATACAG\$TA	CATACAG\$TA	CATACAG\$TA	CATACAG\$TA
7	G\$TACATACA	G\$TACATACA	G\$TACATACA	G\$TACATACA
8	TACAG\$TACA	TACAG\$TACA	TACAG\$TACA	TACAG\$TACA
9	TACATACAG\$	TACATACAG\$	TACATACAG\$	TACATACAG\$
	[0, 10) 10 filas	[1, 5) 4 filas	[5, 7) 2 filas	[1, 3) 2 filas

Figura 7.4. Búsqueda hacia atrás de $P = ACA$ en la matriz ordenada de $T = TACATACAG\$$, leída como una secuencia de fotogramas. En cada paso se antepone un carácter del patrón (de derecha a izquierda) y el intervalo amarillo se contrae a las filas que empiezan con el sufijo procesado, resaltado en rojo. El índice nunca almacena estas filas completas: solo necesita L , C y Occ para calcular los extremos del intervalo con la ecuación (7.8). El tamaño final del intervalo, dos filas, es el número de apariciones.

indice_fm.py

```
def indice_fm(T):
    """T debe terminar en '$'. Devuelve SA, L, C y Occ."""
    sa = sorted(range(len(T)), key=lambda i: T[i:])
    L = "".join(T[i - 1] for i in sa)          # T[-1] es '$'
    alf = sorted(set(T))
    C, tot = {}, 0
    for c in alf:
        C[c], tot = tot, tot + T.count(c)
    occ = {c: [0] for c in alf}
    for ch in L:
        for c in alf:
            occ[c].append(occ[c][-1] + (ch == c))
    return sa, L, C, occ

def buscar(P, sa, C, occ):
    sp, ep = 0, len(sa)
    for c in reversed(P):
        if c not in C:
            return []
        sp, ep = C[c] + occ[c][sp], C[c] + occ[c][ep]
```

```

    if sp >= ep:
        return []
    return sorted(sa[sp:ep])

sa, L, C, occ = indice_fm("TACATACAG$")
print(L, buscar("ACA", sa, C, occ))  # GTTCCAAAA$ [1, 5]

```

7.1.7 Muestreo: cómo cabe el genoma humano en un portátil

El programa anterior guarda Occ completa: cuatro enteros por posición. Para el genoma humano, con enteros de 32 bits, eso son $4 \times 32 \times 3,1 \times 10^9$ bits, unos 50 GB, peor que el arreglo de sufijos. El índice FM resuelve el problema con dos muestreos (Ferragina & Manzini, 2005).

Puntos de control de Occ. Se guardan los valores de $\text{Occ}(c, r)$ solo cuando r es múltiplo de un intervalo d (los *checkpoints*). Para cualquier otra fila, se toma el punto de control anterior y se cuentan las apariciones de c en el trozo de L que falta, a lo sumo $d - 1$ caracteres. Con L codificada a 2 bits por base, ese recuento se hace con operaciones de bits sobre unas pocas palabras de máquina.

Muestreo del arreglo de sufijos. La búsqueda hacia atrás devuelve un intervalo de *filas*, pero necesitamos *posiciones* del genoma, es decir, valores de SA. Se guarda $\text{SA}[r]$ solo para las filas cuyo valor es múltiplo de s . Para una fila no muestreada, aplicamos LF (un paso atrás en el texto) hasta caer en una fila muestreada; si hicieron falta j pasos, $\text{SA}[r] = \text{SA}[\text{LF}^j(r)] + j$, con $j < s$.

La memoria total, en bits, es

$$\mathcal{M}(d, s) = \frac{2n}{\tilde{L}} + \underbrace{\frac{4bn}{d}}_{\text{Occ}} + \underbrace{\frac{bn}{s}}_{\text{SA}}, \quad (7.9)$$

mientras que el tiempo de localizar cada aparición crece como $O(s)$ pasos de LF, cada uno de coste $O(d)$.

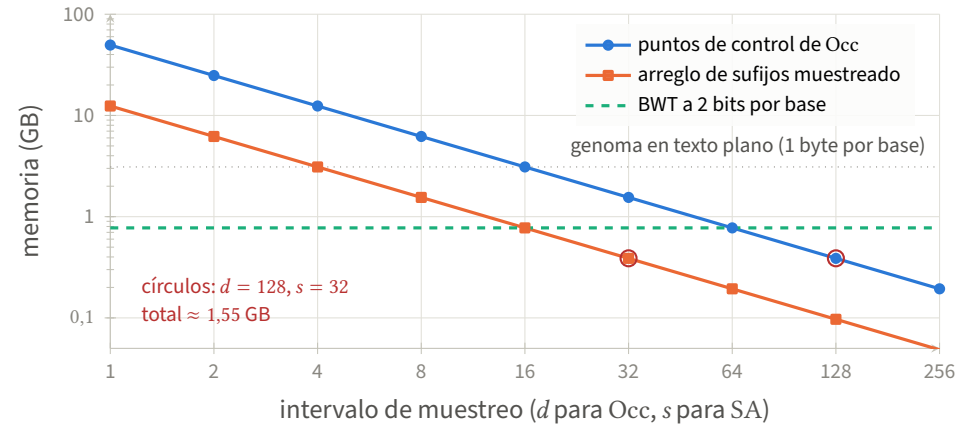


Figura 7.5. Memoria de cada componente del índice FM del genoma humano ($n = 3,1 \times 10^9$, enteros de 32 bits) en función del intervalo de muestreo (ecuación (7.9)). Sin muestreo, Occ ocuparía unos 50 GB y el arreglo de sufijos 12 GB. Con un punto de control cada 128 filas y una muestra del arreglo cada 32 posiciones (círculos), el índice completo de una hebra ocupa unos 1,55 GB, la mitad que el genoma escrito como texto. El precio es tiempo: cada posición recuperada exige, en promedio, varios pasos de *LF-mapping*.

DONDE	
$\mathcal{M}(d, s)$	Memoria del índice, en bits.
n	Longitud del texto indexado.
b	Bits por entero almacenado ($b = \lceil \log_2 n \rceil = 32$ para el genoma humano).
d	Distancia entre puntos de control de Occ.
s	Intervalo de muestreo del arreglo de sufijos.

Con $d = 128$ y $s = 32$, la ecuación (7.9) da 0,775 GB para L , 0,388 GB para los puntos de control y otros 0,388 GB para el arreglo muestreado: unos 1,55 GB (figura 7.5). Estos son, de hecho, los intervalos que describen Li y Durbin (2009) para BWA, que además indexa el genoma invertido para poder acotar la búsqueda aproximada; Bowtie, con técnicas similares, alineaba más de 25 millones de lecturas por hora de CPU usando unos 1,3 GB de memoria (Langmead et al., 2009).

Buscar con errores: *backtracking* acotado

La búsqueda hacia atrás encuentra coincidencias *exactas*, pero una lectura real contiene errores de secuenciación y variantes. Bowtie y la primera versión de BWA exploran variantes del patrón: en cada paso, además de la base leída, prueban las otras tres (una sustitución) o saltan un carácter (un *indel*), y retroceden cuando el intervalo se vacía. El espacio de búsqueda crece exponencialmente con el número de diferencias permitidas, así que ambas herramientas lo podan. Bowtie prioriza sustituir primero las bases de baja calidad Phred (Langmead et al., 2009). BWA precalcula un arreglo $D(i)$, cota inferior del número de diferencias que necesitará el prefijo $P[0..i]$, y abandona una rama en cuanto las diferencias acumuladas más $D(i)$ superan el máximo permitido (Li & Durbin, 2009). Un detalle elegante: como los sufijos repetidos del genoma comparten intervalo, una repetición con mil copias se explora *una sola vez*.

Detalles que el ejemplo oculta

- *Las dos hebras*. Una lectura puede proceder de cualquier hebra: el mapeador busca q y su reverso complementario $\bar{q}$, o indexa ambas hebras de la referencia.
- *Las N*. Las bases ambiguas de la referencia (huecos del ensamblaje) se sustituyen por bases aleatorias o se excluyen al construir el índice; en la lectura se tratan como diferencias.
- *Varios cromosomas*. Se concatenan en un único texto; una tabla auxiliar traduce la posición global a (cromosoma, posición) y descarta las coincidencias que cruzan una frontera.

La BWT reordena el genoma de modo que buscar un patrón cuesta dos consultas por carácter, independientemente del tamaño del genoma, y el índice completo cabe en menos memoria que el propio texto.

7.2 BWA, minimap2 y samtools

Huellas digitales de un texto copiado

Un profesor sospecha que un ensayo fue copiado de alguno de los millones de documentos de internet. No puede compararlo palabra por palabra con todos. En cambio, toma de cada documento unas pocas “huellas”: por ejemplo, en cada tramo de diez frases, la que tenga el código más pequeño según una regla fija. Dos textos que compartan un párrafo largo compartirán, por fuerza, algunas huellas, porque la regla elige la misma frase en ambos. Con las huellas coincidentes en la mano, el profesor comprueba si aparecen en el mismo orden y a distancias parecidas; si es así, ha encontrado la fuente, y solo entonces lee con atención el pasaje sospechoso. Semilla, encadenamiento y extensión: así trabajan los mapeadores modernos.

7.2.1 El paradigma semilla-encadenamiento-extensión

La búsqueda exacta con retroceso funciona bien para lecturas de 36 bases con dos o tres diferencias, las de los primeros secuenciadores Illumina. Con lecturas de 100 a 250 bases, y más aún con lecturas largas de nanoporos, la búsqueda con errores se vuelve inviable y, además, innecesaria: una lectura larga contiene, casi con seguridad, varios tramos exactos suficientemente largos para ser únicos. Los mapeadores actuales combinan tres etapas:

1. **Semillas:** coincidencias exactas entre la lectura y la referencia, halladas con un índice (FM o *hash*).
2. **Encadenamiento:** agrupar las semillas que son *colineales* (mismo orden y distancias compatibles en lectura y referencia), porque proceden probablemente de un mismo origen.
3. **Extensión:** alinear con programación dinámica, restringida a una banda, las bases entre semillas y los extremos de la cadena.

La programación dinámica, cara, solo se ejecuta en un puñado de regiones candidatas por lectura.

BWA-MEM. La familia BWA pasó por tres algoritmos: BWA-*backtrack*, para lecturas cortas (Li & Durbin, 2009); BWA-SW, que alineaba lecturas largas recorriendo el

índice con programación dinámica (Li & Durbin, 2010); y BWA-MEM (Li, 2013), hoy el estándar para lecturas Illumina. BWA-MEM siembra con *coincidencias exactas supermaximales* (SMEM): para cada posición de la lectura, la coincidencia exacta más larga que la cubre y que no está contenida en otra. El índice FM es ideal para esto, porque extender una coincidencia un carácter cuesta una actualización del intervalo. Si una SMEM es muy larga (más de 28 bases por defecto), BWA-MEM vuelve a sembrar con coincidencias más cortas que aparecen más veces, para no perder el origen verdadero cuando la SMEM cae en una repetición. Después encadena semillas cercanas y colineales, descarta cadenas contenidas en otras mucho mejores, y extiende con programación dinámica de huecos afines en banda. Durante la extensión aplica una variante del criterio *X-drop* de BLAST que vimos en el capítulo 3, y decide automáticamente entre alineamiento local y de extremo a extremo.

Bowtie 2. Langmead y Salzberg (2012) siguieron un camino paralelo: el índice FM encuentra semillas sin huecos y la extensión se hace con programación dinámica vectorizada con instrucciones SIMD del procesador. Bowtie 2 sigue siendo muy usado en ChIP-seq y ATAC-seq.

7.2.2 Minimizadores

Para lecturas largas y ruidosas, Li (2018) optó en minimap2 por un índice de *hash*, pero sin guardar todos los k -meros del genoma: solo sus *minimizadores*, una idea de Roberts et al. (2004).

Definición 7.4. $((w, k)$ -minimizador)

Sea h una función que ordena los k -meros (por ejemplo, un *hash* pseudo-aleatorio). Para cada ventana de w k -meros consecutivos de una secuencia, el minimizador de la ventana es el k -mero de valor h mínimo:

$$\mu_j = \arg \min_{j \leq i < j+w} h(s[i..i+k-1]). \quad (7.10)$$

El conjunto de minimizadores de la secuencia es $\{\mu_j\}$ para todas las ventanas j .

DONDE

k	Longitud de cada k -mero.
w	Número de k -meros consecutivos por ventana; la ventana abarca $w + k - 1$ bases.
h	Orden total sobre los k -meros; el orden alfabético es el más simple, un <i>hash</i> aleatorio el más eficaz.
μ_j	Posición del k -mero elegido en la ventana que empieza en j .

La definición garantiza dos propiedades. La primera es la de *cobertura*: toda ventana de $w + k - 1$ bases aporta al menos un minimizador, así que dos secuencias que comparten un tramo exacto de esa longitud comparten al menos un minimizador; es la misma garantía que buscaba el algoritmo de *winnowing* para detectar plagio en documentos (Schleimer et al., 2003). La segunda es la de *escasez*: ventanas vecinas comparten casi todos sus k -meros y suelen elegir el mismo, de modo que el número de minimizadores distintos es mucho menor que el de k -meros.

Teorema 7.3. (*Densidad de los minimizadores*)

Si los k -meros de una secuencia aleatoria reciben valores h independientes y sin empates, la fracción esperada de posiciones elegidas como minimizador es

$$\delta \approx \frac{2}{w + 1}. \quad (7.11)$$

DONDE

δ	Densidad: número de minimizadores dividido por el número de k -meros.
----------	---

Esbozo. Compare dos ventanas consecutivas, j y $j + 1$. Juntas abarcan $w + 1$ k -meros, y por simetría cada uno tiene la misma probabilidad, $1/(w + 1)$, de ser el menor de todos. Si el menor es el primero (que solo pertenece a la ventana j), la ventana $j + 1$ tiene que elegir otro k -mero: aparece un minimizador nuevo. Si el menor es el último (el que acaba de entrar), la ventana $j + 1$ lo elige y también es nuevo. Si está en cualquier posición intermedia, ambas ventanas eligen el mismo y no se añade nada. La probabilidad de estrenar minimizador al avanzar una posición es, por tanto, $2/(w + 1)$ (Schleimer et al., 2003). $\square$

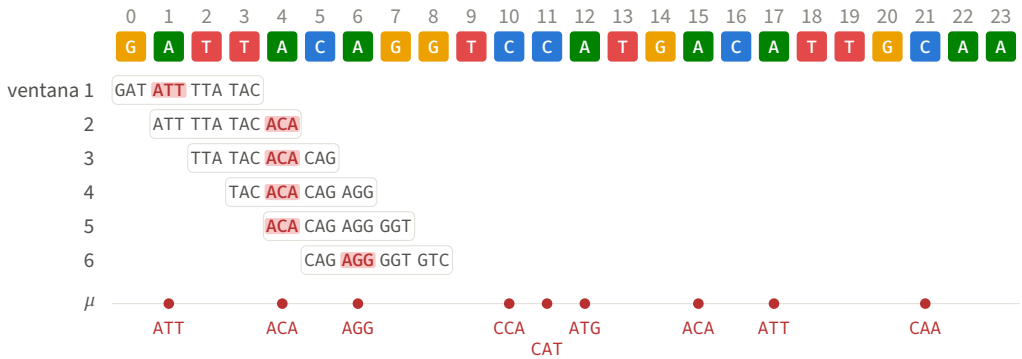


Figura 7.6. Minimizadores ($w = 4$, $k = 3$) de una secuencia de 24 bases, con orden alfabético para que puedan comprobarse a mano. Cada fila muestra una ventana de cuatro 3-meros consecutivos, cada uno escrito bajo su base inicial, con el menor resaltado en rojo. Las ventanas 2 a 5 eligen el mismo ACA: por eso se guardan muchos menos minimizadores que k -meros. La última fila marca los nueve minimizadores de toda la secuencia (de 22 k -meros; densidad 0,41, frente a $2/(w + 1) = 0,40$). Una lectura que contenga cualquier tramo de $w + k - 1 = 6$ bases de esta secuencia compartirá con ella al menos uno de estos minimizadores.

Con $w = 10$ se guarda solo un k -mero de cada 5,5: el índice se reduce a menos de una quinta parte. En una secuencia aleatoria de 200 000 bases con $k = 15$ y $w = 10$, nuestra simulación da una densidad de 0,188 con un *hash* aleatorio (la teoría predice 0,182) y de 0,199 con orden alfabético, que tiende a elegir k -meros ricos en A consecutivos y es menos eficiente.

minimizadores.py

```
import zlib

def minimizadores(s, k=15, w=10):
    """Posiciones de los (w,k)-minimizadores de s."""
    h = [zlib.crc32(s[i:i + k].encode())          # orden pseudoaleatorio
          for i in range(len(s) - k + 1)]
    elegidos = set()
    for j in range(len(h) - w + 1):
        v = h[j:j + w]
        elegidos.add(j + v.index(min(v)))
    return sorted(elegidos)
```

7.2.3 Encadenamiento

Cada minimizador de la lectura que también aparece en el índice produce un *ancla*: un par de posiciones (x, y) en referencia y lectura donde termina una coincidencia exacta de longitud w_i (aquí, $w_i = k$). Las anclas verdaderas se alinean a lo largo de una diagonal; las espurias, procedentes de repeticiones, quedan dispersas. Li (2018) define la puntuación $f(i)$ de la mejor cadena que termina en el ancla i mediante programación dinámica:

$$f(i) = \max \left\{ \max_{j < i} \{f(j) + \alpha(j, i) - \beta(j, i)\}, w_i \right\}, \quad (7.12)$$

con

$$\alpha(j, i) = \min \{ \min \{y_i - y_j, x_i - x_j\}, w_i \}, \quad \beta(j, i) = \gamma((y_i - y_j) - (x_i - x_j)), \quad (7.13)$$

$$\gamma(\ell) = \begin{cases} 0,01 \bar{w} |\ell| + 0,5 \log_2 |\ell| & \ell \neq 0, \\ 0 & \ell = 0. \end{cases} \quad (7.14)$$

DONDE

(x_i, y_i, w_i)	Ancla i : termina en la posición x_i de la referencia y y_i de la lectura, y cubre w_i bases idénticas.
$f(i)$	Puntuación máxima de una cadena de anclas que termina en i .
$\alpha(j, i)$	Bases coincidentes que el ancla i añade tras la j (no cuenta dos veces el solapamiento).
$\beta(j, i)$	Penalización por el desfase entre las distancias en lectura y referencia: un <i>indel</i> de longitud $ \ell $.
$\bar{w}$	Longitud media de las semillas.

β vale infinito si el ancla j no precede a la i en la lectura o si están demasiado lejos. La ecuación (7.12) es una recurrencia de tipo Smith-Waterman (compárela con la del capítulo 3) sobre *anclas* en lugar de bases: el término w_i permite empezar una cadena nueva, igual que el cero permitía empezar un alineamiento local. En la práctica se evalúan solo los 50 predecesores más cercanos, lo que reduce el coste de cuadrático a lineal en el número de anclas sin perder casi nunca la cadena óptima.

Tras encadenar, minimap2 rellena con programación dinámica los huecos entre anclas consecutivas y los extremos, con una penalización de huecos “afín en dos tramos” que permite *indels* largos. El mismo programa, con distintos parámetros preestablecidos (`-x sr` para lecturas cortas, `map-ont` y `map-hifi` para lecturas largas, `splice` para ARN, `asm5` para ensamblajes), cubre casi todos los casos de uso.

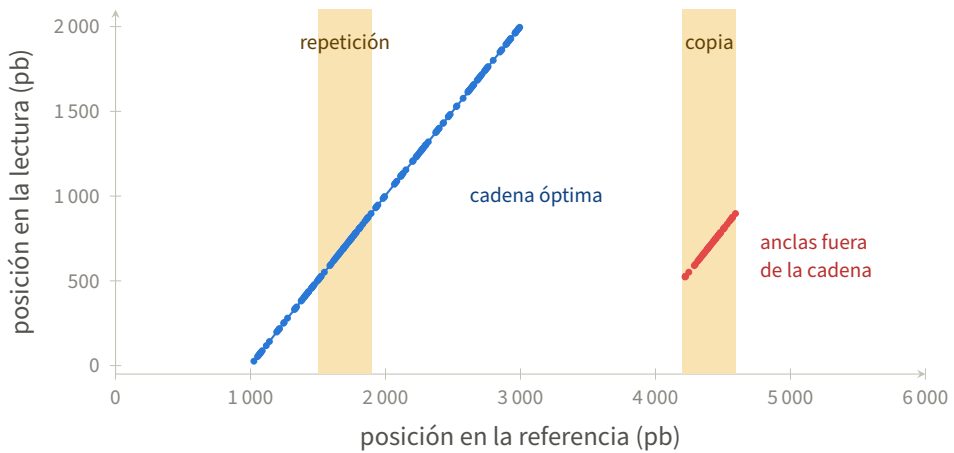


Figura 7.7. Encadenamiento de minimizadores ($k = 15$, $w = 10$) de una lectura simulada de 2000 pb, con un 4 % de errores, contra una referencia de 6000 pb que contiene una repetición de 400 pb (bandas amarillas). La ecuación (7.12) elige 187 de las 236 anclas (azul), alineadas sobre una diagonal casi perfecta: la lectura procede de las posiciones 1000 a 3000. Las anclas rojas del extremo derecho son minimizadores de la repetición que también aparecen en su copia; no pueden encadenarse con el resto sin pagar un *indel* de casi 2700 bases, y quedan descartadas. Si la lectura cayera *entera* dentro de la repetición, las dos cadenas empatarían: es el origen de la ambigüedad que mide la calidad de mapeo.

Índices y mapeo

```
# BWA-MEM: índice FM (una sola vez) y mapeo de lecturas pareadas
bwa index ref.fa
bwa mem -t 8 -R '@RG\tID:m1\tSM:m1' ref.fa r1.fq.gz r2.fq.gz \
> m1.sam
# minimap2: índice de minimizadores al vuelo (o guardado con -d)
minimap2 -ax sr ref.fa r1.fq.gz r2.fq.gz > m1_sr.sam
minimap2 -ax map-ont ref.fa nanoporo.fq.gz > m1_ont.sam
```

7.2.4 Calidad de mapeo

Supongamos que una lectura tiene dos ubicaciones candidatas. ¿Cuán seguros podemos estar de la mejor? Li et al. (2008) introdujeron en MAQ la *calidad de mapeo*, que expresa en escala Phred (la misma del capítulo 6) la probabilidad de que la posición asignada sea errónea.

Definición 7.5. (Calidad de mapeo)

Sean $u_1, \dots, u_K$ las ubicaciones candidatas de una lectura q y $p(q | u)$ la probabilidad de observar q si procede de u . Suponiendo un origen *a priori* uniforme, la probabilidad *a posteriori* de que la mejor ubicación $\hat{u}$ sea la verdadera es

$$\mathbb{P}(\hat{u} | q) = \frac{p(q | \hat{u})}{\sum_{k=1}^K p(q | u_k)}, \quad (7.15)$$

y la calidad de mapeo es

$$\text{MAPQ} = -10 \log_{10} (1 - \mathbb{P}(\hat{u} | q)). \quad (7.16)$$

DONDE

u_k	Posición candidata k (con su hebra).
$p(q u)$	Verosimilitud de la lectura dada la posición; si solo hay sustituciones, aproximadamente el producto de las probabilidades de error $\varepsilon_i = 10^{-Q_i/10}$ de las bases discordantes.
MAPQ	Calidad de mapeo en escala Phred: 20 significa 1 % de probabilidad de posición errónea, 30 significa 0,1 %.

Ejemplo 7.3. (Dos candidatas y un empate)

Una lectura coincide con la posición u_1 salvo por una base de calidad $Q = 35$, y con u_2 salvo por dos bases de calidades 25 y 30. Aproximando $p(q | u)$ por el producto de las probabilidades de error de las discordancias,

$$p(q | u_1) = 10^{-3,5} = 3,16 \times 10^{-4}, \quad p(q | u_2) = 10^{-2,5} \cdot 10^{-3} = 3,16 \times 10^{-6}.$$

La probabilidad de error es $3,16 \times 10^{-6} / (3,16 \times 10^{-4} + 3,16 \times 10^{-6}) = 0,0099$, y $\text{MAPQ} = -10 \log_{10} 0,0099 = 20,0$. Una sola base discordante de diferencia, pero de buena calidad, basta para preferir u_1 con un 99 % de confianza. Si en cambio u_1 y u_2 fueran idénticas, la ecuación (7.16) daría $-10 \log_{10} 0,5 = 3,01$; con tres copias idénticas, 1,76.

En la práctica, los programas no enumeran todas las candidatas: estiman el denominador a partir de las dos mejores y de cuántas subóptimas encontraron. BWA supone que la ubicación verdadera siempre está entre las halladas y comprueba por simulación que sus valores son razonables: de 1 569 108 lecturas simuladas con $\text{MAPQ} = 60$, solo 11 estaban mal colocadas (Li & Durbin, 2009). minimap2 usa una fórmula empírica basada en las puntuaciones de encadenamiento de la cadena primaria (f_1) y de la mejor secundaria (f_2):

$$\text{MAPQ} = 40 \left(1 - \frac{f_2}{f_1} \right) \min \left\{ 1, \frac{a}{10} \right\} \ln f_1, \quad (7.17)$$

donde a es el número de anclas de la cadena primaria (Li, 2018); el valor final se trunca en 60. Con $f_1 = 200$ y 40 anclas, una secundaria con $f_2 = 150$ da un MAPQ de 53, y una casi empatada con $f_2 = 195$ lo hunde hasta 5.

⚠ Lo que el MAPQ no es

- *No es una calidad del alineamiento.* Una lectura puede alinearse perfectamente, sin una sola discordancia, y tener $\text{MAPQ} = 0$ porque encaja igual de bien en dos copias de una repetición.
- *No es comparable entre programas.* BWA, Bowtie 2 y minimap2 calculan y truncan el MAPQ de maneras distintas. Un filtro “ $\text{MAPQ} \geq 20$ ” significa cosas diferentes en cada caso.

- *Depende de la referencia.* Si falta en la referencia la copia verdadera de una repetición, las lecturas se mapean con confianza al lugar equivocado.

7.2.5 Repeticiones y multimap

Casi la mitad del genoma humano está formada por secuencias repetidas: elementos transponibles, satélites, duplicaciones segmentarias (Treangen & Salzberg, 2012). Una lectura de 150 bases que cae en el interior de una copia reciente de un elemento Alu o de una duplicación segmentaria no tiene un origen identificable: se *multimapea*. Los formatos distinguen tres tipos de registros: el alineamiento *primario*, uno por lectura, elegido al azar entre los empatados; los *secundarios*, que señalan otras ubicaciones plausibles; y los *suplementarios*, que no son alternativas sino *piezas* de una lectura quimérica, cuyo extremo izquierdo se alinea en un lugar y el derecho en otro, como ocurre en una translocación.

Cuánto de este problema es inevitable depende también de la referencia. GRCh38 dejó, pese a su calidad (Schneider et al., 2017), huecos en centrómeros y brazos cortos acrocéntricos. El genoma T2T-CHM13 completó el 8 % restante: 3055 millones de bases sin huecos, con casi 200 millones de bases nuevas, en buena parte satélites y duplicaciones segmentarias recientes (Nurk et al., 2022). Las lecturas que antes no tenían dónde caer, o que caían en una copia paráloga equivocada, ahora tienen un sitio.

7.2.6 Lecturas pareadas y tamaño de inserto

Cuando se secuencian ambos extremos de un fragmento, las dos lecturas no son independientes: deben mapearse en hebras opuestas, orientadas una hacia la otra, a una distancia compatible con el tamaño de los fragmentos de la biblioteca. Esa información resuelve muchas ambigüedades: si una lectura cae en una repetición pero su pareja cae en secuencia única, la pareja “ancla” a la primera. BWA-MEM estima la distribución del tamaño de inserto a partir de los pares mapeados sin ambigüedad y puntúa cada combinación (i, j) de ubicaciones de las dos lecturas con (Li, 2013)

$$S_{ij} = S_i + S_j - \min \{ -a \log_4 P(d_{ij}), U \}, \quad (7.18)$$

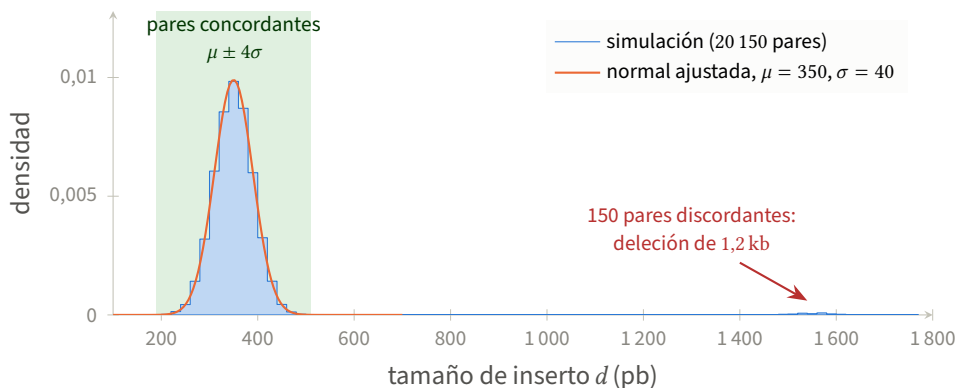


Figura 7.8. Tamaño de inserto de 20 000 pares simulados con $\mu = 350$ y $\sigma = 40$ pb, más 150 pares que atraviesan una deleción de 1200 pb en la muestra. Como la referencia conserva el segmento que la muestra perdió, esos pares parecen 1200 pb más largos. Bajo el modelo normal, la probabilidad de un inserto mayor que $\mu + 4\sigma = 510$ pb es 3×10^{-5} : esperaríamos menos de un par así por azar, y observamos 150 agrupados en la misma región. Los detectores de variantes estructurales combinan esta señal con la cobertura (sección 7.3) y con las lecturas partidas.

DONDE	
S_i, S_j	Puntuaciones de Smith-Waterman de cada lectura en su ubicación.
d_{ij}	Distancia entre las dos ubicaciones (infinita si la orientación es incorrecta).
$P(d)$	Probabilidad de observar un inserto mayor que d según una distribución normal ajustada.
a	Puntuación de una coincidencia; convierte probabilidades en unidades de puntuación.
U	Penalización máxima: si el par es muy improbable, es preferible tratar las lecturas como no emparejadas.

El logaritmo en base 4 aparece al interpretar la puntuación de Smith-Waterman como una razón de verosimilitudes, como vimos en el capítulo 3. Los pares que el modelo no puede explicar, por estar demasiado separados, demasiado juntos o mal orientados, son *discordantes*, y son precisamente la señal que buscan los detectores de variantes estructurales (figura 7.8).

Tabla 7.2. Los once campos obligatorios de un registro SAM, ilustrados con el primer extremo de un par ficticio de 20 bases. Las posiciones son base 1.

#	Campo	Ejemplo	Significado
1	QNAME	r017	Nombre de la lectura (igual en ambos extremos del par).
2	FLAG	99	Suma de bits (tabla 7.3): 1 + 2 + 32 + 64.
3	RNAME	chr7	Secuencia de referencia.
4	POS	1021	Posición de la primera base <i>alineada</i> (no recortada).
5	MAPQ	60	Calidad de mapeo (ecuación (7.16)); 255 si no está disponible.
6	CIGAR	3S12M1D5M	Operaciones del alineamiento: 3 recortadas, 12 alineadas, 1 delección, 5 alineadas.
7	RNEXT	=	Referencia de la pareja (=: la misma).
8	PNEXT	1180	Posición de la pareja.
9	TLEN	179	Longitud observada del fragmento, de 1021 a 1199.
10	SEQ	GTA...	Bases de la lectura, en la hebra de la referencia.
11	QUAL	FFF...	Calidades Phred+33 (capítulo 6).

7.2.7 El formato SAM/BAM

Todos los mapeadores escriben sus resultados en el formato SAM (*Sequence Alignment/Map*), creado para el Proyecto 1000 Genomas (Li et al., 2009) y mantenido hoy como especificación formal (SAM/BAM Format Specification Working Group, 2024). Un archivo SAM es texto tabulado: una cabecera con líneas que empiezan por @ (secuencias de referencia @SQ, grupos de lectura @RG, programas @PG) y una línea por alineamiento con once campos obligatorios (tabla 7.2), seguidos de etiquetas opcionales TAG:TIPO:VALOR, como NM:i:2 (distancia de edición) o AS:i: (puntuación del alineamiento).

La cadena CIGAR. Describe el alineamiento como una sucesión de longitudes y operaciones: M (alineada, coincida o no; = y X distinguen ambos casos), I (inserción respecto a la referencia), D (delección), N (salto en la referencia, típico de un intrón en RNA-seq), S (recorte suave: las bases están en SEQ pero no alineadas) y H (recorte duro: las bases ni siquiera se guardan). En el ejemplo, las operaciones que consumen la lectura (S, M, I) suman 3 + 12 + 5 = 20, su longitud, y las que consumen referencia (M, D, N) suman 12 + 1 + 5 = 18: el alineamiento cubre las posiciones 1021 a 1038. Si la pareja se alinea como 20M a partir de 1180, termina en 1199 y el fragmento mide 1199 – 1021 + 1 = 179 pb.

Tabla 7.3. Bits del campo FLAG de SAM (SAM/BAM Format Specification Working Group, 2024). Un filtro frecuente es -F 0x904 (excluir no mapeadas, secundarias y suplementarias), que deja un registro primario por lectura mapeada.

Bit	Dec.	Significado	Bit	Dec.	Significado
0x1	1	lectura pareada	0x40	64	primera del par
0x2	2	par concordante (<i>proper pair</i>)	0x80	128	segunda del par
0x4	4	lectura no mapeada	0x100	256	alineamiento secundario
0x8	8	pareja no mapeada	0x200	512	no pasa filtros de calidad
0x10	16	lectura en hebra reversa	0x400	1024	duplicado de PCR u óptico
0x20	32	pareja en hebra reversa	0x800	2048	alineamiento suplementario

BAM es la versión binaria y comprimida de SAM, en bloques BGZF independientes; eso permite *indexarla* (archivo `.bai` o `.csi`) y saltar directamente a una región sin leer el resto del archivo, siempre que esté ordenada por coordenada. CRAM va más allá y guarda solo las diferencias respecto a la referencia, con archivos aún menores.

7.2.8 samtools: del SAM crudo al BAM analizable

El SAM que sale del mapeador está en el orden de las lecturas del FASTQ. Para analizarlo hacen falta varios pasos, que `samtools` resuelve en un flujo de tuberías (Danecek et al., 2021; Li et al., 2009). El orden importa (figura 7.9):

1. `samtools fixmate -m` rellena, con los dos extremos de cada par a la vista, la información sobre la pareja y añade las etiquetas que necesitará el marcado de duplicados. Requiere los pares contiguos, como los entrega BWA-MEM.
2. `samtools sort` ordena por coordenada.
3. `samtools markdup` marca con el bit 0x400 los *duplicados*: pares cuyos extremos 5' coinciden en posición y orientación, que probablemente son copias por PCR de un mismo fragmento original o duplicados ópticos del secuenciador (recuerde la complejidad de la biblioteca del capítulo 6). `samblaster` lo hace en flujo, sin ordenar, y además extrae las lecturas discordantes y partidas (Faust & Hall, 2014).
4. `samtools index` crea el índice para el acceso aleatorio.

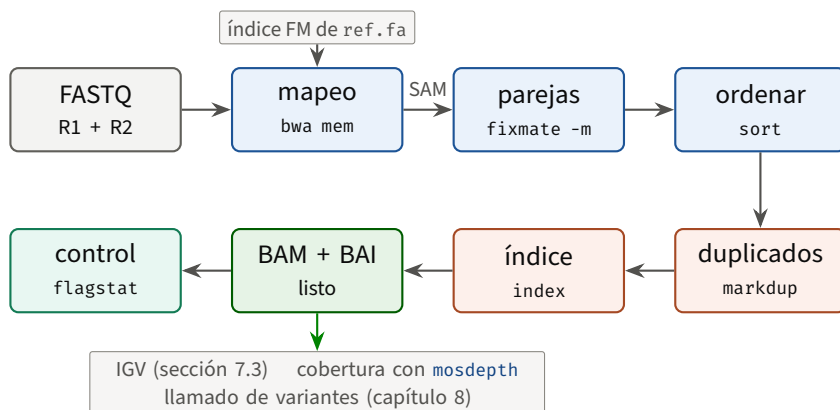


Figura 7.9. Flujo de trabajo típico de mapeo de lecturas Illumina pareadas. En azul, los pasos que se encadenan en una sola tubería sin archivos intermedios; en naranja, los que operan sobre el BAM ordenado. *fixmate* debe ir *antes* de ordenar, cuando los dos extremos de cada par aún están contiguos, y *markdup*, *después*. El BAM indexado es el punto de partida de la visualización, la cobertura y el llamado de variantes.

De SAM a BAM analizable

```

bwa mem -t 8 -R '@RG\tID:m1\tSM:m1' ref.fa r1.fq.gz r2.fq.gz \
| samtools fixmate -m - - \
| samtools sort -@ 4 -o m1.ordenado.bam -
samtools markdup -@ 4 m1.ordenado.bam m1.bam
samtools index m1.bam
samtools flagstat m1.bam # resumen de FLAG
samtools view -c -F 0x904 -q 20 m1.bam # primarias con MAPQ>=20
samtools view m1.bam chr7:55019017-55211628 | head # una región
  
```

⚠ Errores frecuentes en el flujo

- Olvidar el grupo de lectura (-R). Muchos programas posteriores lo exigen para distinguir muestras y carriles.
- Marcar duplicados en datos de amplicones o de RNA-seq sin pensarlo: allí los “duplicados” son la norma biológica o técnica, no un artefacto.

- Mezclar referencias: el BAM solo es válido junto con la misma versión exacta del FASTA (chr7 frente a 7, GRCh37 frente a GRCh38). Consulte las líneas @SQ de la cabecera con `samtools view -H`.

7.3 Visualización de alineamientos y cobertura

 Colab · 7.3 · Visualización de alineamientos y cobertura  abrir notebook

Una autopista vista desde un helicóptero

Desde un helicóptero se ve el tráfico de una autopista de dos carriles como una densidad de coches más o menos uniforme. Si un tramo tiene un carril cerrado por obras, la densidad cae a la mitad; si el tramo entero está cortado, no pasa ningún coche; si se abre un tercer carril, la densidad sube un cincuenta por ciento. Y hay un tramo en que la autopista corre paralela a otra idéntica: desde el aire es imposible saber por cuál va cada coche, y el contador automático, prudente, no cuenta ninguno. Un perfil de cobertura es esa vista aérea del genoma: las lecturas son los coches, el número de copias son los carriles, y las repeticiones son las autopistas gemelas.

7.3.1 Mirar antes de calcular: IGV

Así como el *dot plot* del capítulo 3 precedía al alineamiento, mirar los alineamientos precede a cualquier análisis serio. El *Integrative Genomics Viewer* (IGV) permite navegar un BAM indexado a cualquier escala, desde un cromosoma entero hasta una base (Robinson et al., 2011). Su diseño carga solo la región visible, aprovechando el índice del BAM, y muestra la información en *pistas* apiladas: la anotación de genes, la cobertura, las lecturas individuales (Thorvaldsdóttir et al., 2013). Las convenciones visuales codifican casi todos los campos del SAM:

- las bases que *coinciden* con la referencia no se dibujan; solo se colorean las discordantes, de modo que el ojo va directo a las diferencias;
- una línea negra horizontal indica una delección, y una marca violeta, una inserción;

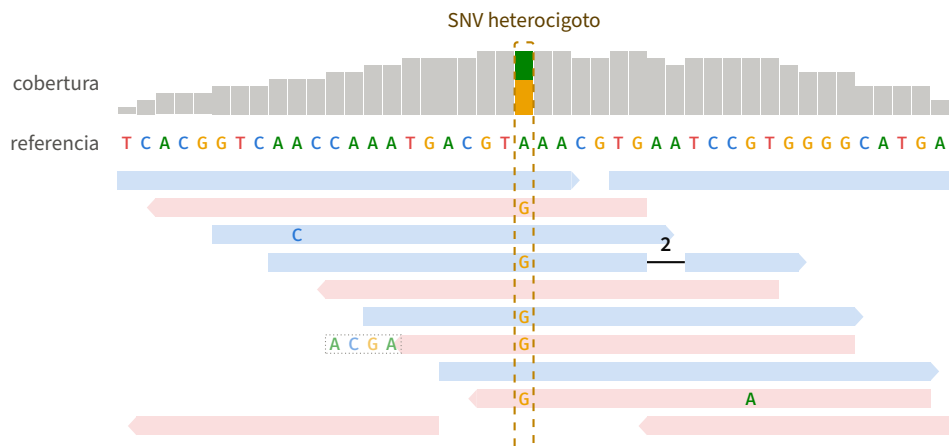


Figura 7.10. Apilamiento (*pileup*) de lecturas sobre 44 bases de referencia, dibujado con las convenciones de IGV. Las lecturas azules se alinean en la hebra directa y las rojas en la reversa (la punta indica el sentido). En la columna recuadrada, 5 de las 9 lecturas llevan una G donde la referencia tiene A: la pista de cobertura se colorea con esa proporción (0,56), la firma de un SNV heterocigoto. Las letras aisladas en otras columnas (una C y una A) son errores de secuenciación: aparecen en una sola lectura. La línea negra marca una delección de 2 pb en una lectura, y las letras tenues dentro del recuadro punteado son bases con recorte suave (4S), presentes en la lectura pero no alineadas.

- el color de fondo de cada lectura puede indicar su hebra, y las lecturas con $\text{MAPQ} = 0$ se dibujan transparentes;
- en la pista de cobertura, las posiciones donde una fracción notable de las lecturas difiere de la referencia se colorean con la proporción de cada base.

La figura 7.10 resume la lectura de un *pileup*. Distinguir una variante de un error es una cuestión de conteo. Si la tasa de error por base es $\varepsilon = 10^{-3}$ (calidad 30), la probabilidad de que 5 o más de 9 lecturas muestren la misma base errónea por azar es del orden de $\binom{9}{5}\varepsilon^5 \approx 1,3 \times 10^{-13}$; la de que una sola lectura muestre un error, $9\varepsilon \approx 0,009$. El siguiente capítulo convertirá este razonamiento en un modelo bayesiano de genotipos.

⚠ Artefactos que se ven en IGV

- *Sesgo de hebra*: una “variante” presente solo en lecturas de una hebra suele ser un artefacto químico o de alineamiento.
- *Variantes al final de las lecturas*: cerca de un *indel*, el alineador puede preferir varias sustituciones en el extremo de la lectura a abrir un hueco, lo que produce falsos SNV agrupados.
- *Pilas de lecturas con MAPQ bajo y muchas discordancias*: huella de una repetición o de una región que falta en la referencia; las lecturas de otro lugar “se refugian” allí.

7.3.2 La cobertura como señal

La definición de cobertura del capítulo 6 era un promedio; ahora la necesitamos posición a posición.

Definición 7.6. (Profundidad por posición)

Sea $\mathcal{A}$ el conjunto de alineamientos que superan los filtros (por ejemplo, primarios, no duplicados y con $\text{MAPQ} \geq 20$), y sea $B(a)$ el conjunto de bloques de referencia $[s, e)$ que el alineamiento a cubre, es decir, sus operaciones $\mathbb{M}$, $=$ o $\times$ de la cadena CIGAR. La profundidad en la posición x es

$$c(x) = \sum_{a \in \mathcal{A}} \sum_{[s, e) \in B(a)} \mathbb{1}[s \leq x < e]. \quad (7.19)$$

DONDE

$c(x)$	Número de lecturas filtradas que cubren la posición x con una base alineada.
$\mathcal{A}$	Alineamientos que se cuentan; los filtros cambian mucho el resultado en repeticiones.
$[s, e)$	Bloque semiabierto de posiciones de referencia cubierto sin interrupción; una delección o un intrón parten un alineamiento en varios bloques.

Calcular la ecuación (7.19) de forma directa cuesta $O(\sum_a |a|)$: unos 10^{11} incrementos para un genoma a 30×. Pedersen y Quinlan (2018) observaron en `mosdepth`

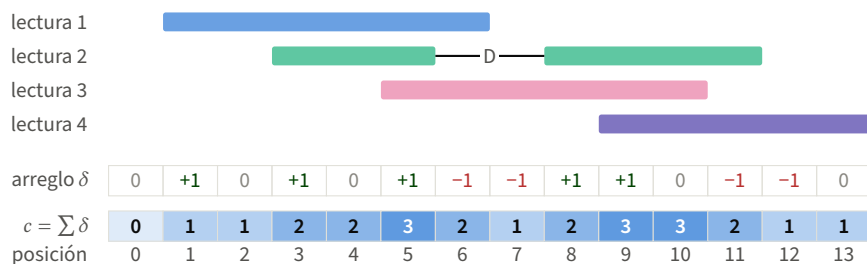


Figura 7.11. El algoritmo de `mosdepth` (ecuación (7.20)) sobre 14 posiciones. Cada bloque alineado suma 1 en su inicio y resta 1 en su final (posición siguiente a la última cubierta). La lectura 2 contiene una deleción (D) y aporta dos bloques. La suma acumulada del arreglo δ reproduce la profundidad de cada posición, coloreada de claro a oscuro. Con N bloques y un cromosoma de longitud G , el coste es $O(N + G)$, sin importar cuántas lecturas se apilen en cada base.

que basta con anotar dónde *empieza* y dónde *termina* cada bloque en un arreglo de diferencias δ del tamaño del cromosoma,

$$\delta(s) += 1, \quad \delta(e) -= 1 \quad \text{para cada bloque } [s, e), \quad c(x) = \sum_{y \leq x} \delta(y), \quad (7.20)$$

y reconstruir la profundidad con una suma acumulada. El coste pasa a ser proporcional al número de bloques más la longitud del cromosoma, y no a la profundidad (figura 7.11). El precio es memoria: un entero de 32 bits por base, cerca de 1 GB para el cromosoma 1 humano. `mosdepth` evita además contar dos veces la región donde los dos extremos de un par se solapan, que no aporta información independiente. Otras herramientas cubren usos distintos: `samtools depth`, `bedtools genomecov` (Quinlan & Hall, 2010) y `bamCoverage` de `deepTools`, que produce pistas normalizadas en formato `bigWig` para comparar muestras en un navegador (Ramírez et al., 2016).

`profundidad.py`

```
import numpy as np
import pysam

def profundidad(bam, crom, mapq=20):
    """Profundidad por base con el arreglo de diferencias."""
    with pysam.AlignmentFile(bam) as f:
        G = f.get_reference_length(crom)
        d = np.zeros(G + 1, dtype=np.int32)
```



```

for a in f.fetch(crom):
    if (a.is_unmapped or a.is_secondary or a.is_supplementary
        or a.is_duplicate or a.mapping_quality < mapq):
        continue
    for s, e in a.get_blocks():          # bloques M/=X
        d[s] += 1
        d[e] -= 1
return np.cumsum(d[:-1])

```

Cobertura con mosdepth

```

# profundidad media en ventanas de 500 pb, solo MAPQ >= 20
mosdepth -t 4 -n --by 500 -Q 20 m1 m1.bam
zcat m1.regions.bed.gz | head -3      # crom inicio fin media
cat m1.mosdepth.summary.txt          # media por cromosoma

```

7.3.3 Cobertura y número de copias

Si las lecturas se muestrean uniformemente, la profundidad esperada en una región es proporcional a su número de copias. Para un genoma diploide,

$$\mathbb{E}[c(x)] = \bar{c} \frac{\text{CN}(x)}{2} b(x), \quad (7.21)$$

y el estadístico natural para comparar una ventana W con el resto del genoma es el logaritmo del cociente

$$\ell_W = \log_2 \frac{\bar{c}_W}{\text{mediana}_V \bar{c}_V}. \quad (7.22)$$

DONDE

$\bar{c}$	Cobertura media del genoma (p. ej., 30×).
$\text{CN}(x)$	Número de copias de la región en la muestra: 2 normal, 1 delección heterocigota, o homocigota, 3 duplicación heterocigota.
$b(x)$	Sesgo local multiplicativo (contenido de GC, mapeabilidad), idealmente 1.
$\bar{c}_W$	Profundidad media en la ventana W .
ℓ_W	Razón logarítmica: 0 normal, -1 delección heterocigota, $\log_2 1,5 = 0,58$ duplicación, $-\infty$ delección homocigota.

La figura 7.12 muestra estos patrones en un perfil simulado. Obsérvese la trampa de la repetición: con todas las lecturas, su cobertura es normal (unos 35); con el

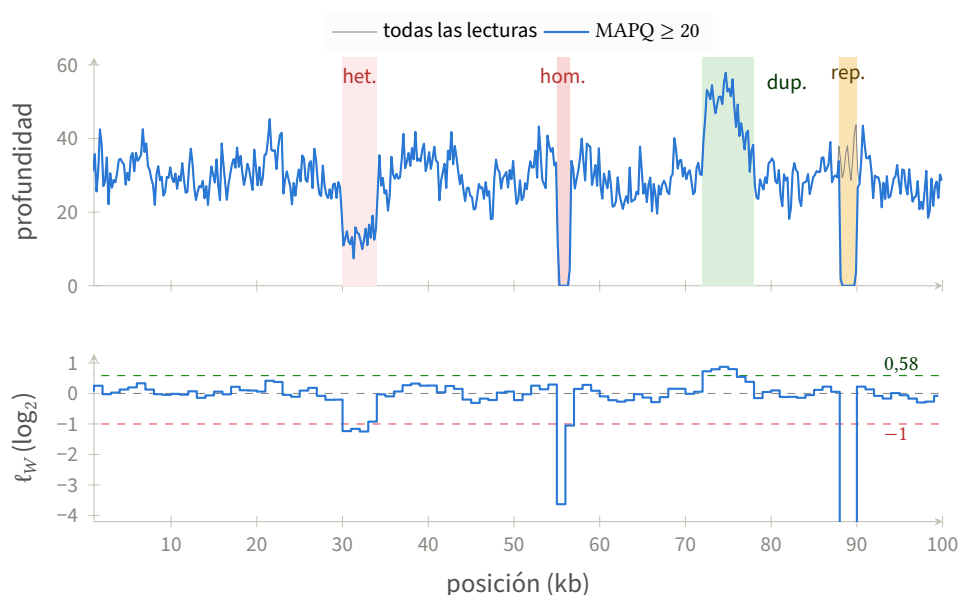


Figura 7.12. Perfil de cobertura simulado de 100 kb a 30× con lecturas de 150 pb. **Arriba:** profundidad media en bloques de 200 pb, con todas las lecturas (gris) y solo con $\text{MAPQ} \geq 20$ (azul). Se aprecian una delección heterocigota de 4 kb (la profundidad cae a la mitad), una homocigota de 1,5 kb (cae a cero), una duplicación de 6 kb (sube un 50 %) y una repetición de 2 kb cuya otra copia está en el genoma: las lecturas se mapean pero con $\text{MAPQ} = 0$, así que la cobertura filtrada se desploma aunque no falte ADN. El sesgo ondulante imita el efecto del contenido de GC. **Abajo:** razón logarítmica en ventanas de 1 kb (ecuación (7.22)); las ventanas de la delección heterocigota promedian $-1,14$ y las de la duplicación $0,69$, cerca de los valores teóricos (líneas discontinuas).

filtro de $\text{MAPQ} \geq 20$, cae a 0,6 y se confundiría con una delección homocigota. Un análisis de cobertura siempre debe acompañarse de una máscara de *mapeabilidad* que diga qué regiones del genoma pueden recibir lecturas únicas. CNVnator, uno de los métodos clásicos de profundidad de lectura, corrige el sesgo de GC, segmenta el perfil con un algoritmo de *mean-shift* y alcanza, con cobertura alta, una resolución de los puntos de corte de menos de 200 pb en el 90 % de los casos (Abyzov et al., 2011). Sus autores subrayan que la profundidad es complementaria a las lecturas discordantes y partidas: cada señal detecta variantes que las otras no ven.

7.3.4 ¿Qué delección es detectable?

Hagamos la cuenta que decide si una delección se ve o se pierde en el ruido. En una ventana de W bases con cobertura $\bar{c}$ y lecturas de longitud L , el número de lecturas que empiezan en la ventana es aproximadamente de Poisson con media $\lambda = \bar{c}W/L$, y su desviación típica es $\sqrt{\lambda}$. Una delección heterocigota reduce la media a $\lambda/2$; la diferencia, medida en desviaciones típicas de la hipótesis nula, es

$$z_{\text{het}} = \frac{\lambda - \lambda/2}{\sqrt{\lambda}} = \frac{\sqrt{\lambda}}{2}, \quad z_{\text{hom}} = \frac{\lambda}{\sqrt{\lambda}} = \sqrt{\lambda}. \quad (7.23)$$

Exigiendo $z \geq z^*$ y despejando W ,

$$W_{\min}^{\text{het}} = \frac{4z^{*2}L}{\bar{c}}, \quad W_{\min}^{\text{hom}} = \frac{z^{*2}L}{\bar{c}}. \quad (7.24)$$

DONDE

λ	Número esperado de lecturas que empiezan en la ventana: $\bar{c}W/L$.
z^*	Umbral de significación en desviaciones típicas; con millones de ventanas en el genoma, $z^* \approx 5$ controla los falsos positivos.
$W_{\min}$	Tamaño mínimo de una delección detectable por profundidad.

Ejemplo 7.4. (Diseño para detectar delecciones)

Con $\bar{c} = 30$, $L = 150$ y $z^* = 5$, la ecuación (7.24) da $W_{\min}^{\text{het}} = 4 \cdot 25 \cdot 150/30 = 500$ pb y $W_{\min}^{\text{hom}} = 125$ pb. A $10\times$, los valores se triplican: 1500 y 375 pb; a $60\times$, se reducen a la mitad: 250 y 62 pb. Una duplicación heterocigota, que cambia la media de λ a $1,5\lambda$, produce la misma diferencia absoluta que una delección heterocigota y requiere, por tanto, ventanas similares. Por debajo de unos

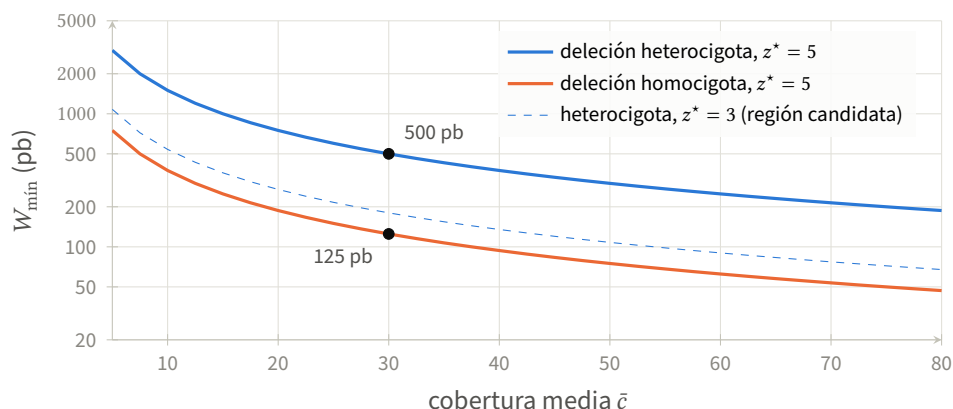


Figura 7.13. Tamaño mínimo de una deleción detectable solo por profundidad, según la ecuación (7.24), para lecturas de 150 pb. Duplicar la cobertura reduce a la mitad el tamaño detectable; pasar de una búsqueda en todo el genoma ($z^* = 5$) a la comprobación de una región candidata ($z^* = 3$) lo reduce casi a un tercio. El modelo es optimista: supone muestreo de Poisson perfecto, y el sesgo residual tras la corrección de GC aumenta la varianza.

cientos de bases, la profundidad no basta y hay que recurrir a lecturas partidas y a pares discordantes (figura 7.8).

El techo de la sobredispersión

Si el recuento de una ventana no es de Poisson sino binomial negativo, con varianza $\lambda + \lambda^2/r$ como en el capítulo 6, la ecuación (7.23) se convierte en

$$z_{\text{het}} = \frac{\lambda/2}{\sqrt{\lambda + \lambda^2/r}} \xrightarrow{\lambda \rightarrow \infty} \frac{\sqrt{r}}{2}.$$

La señal ya no crece indefinidamente con la ventana ni con la cobertura: tiene un techo fijado por la sobredispersión. Con $r = 50$, z_{het} nunca supera 3,5, por mucho que se secuencie. Esta es la razón matemática por la que los métodos de profundidad dedican tanto esfuerzo a corregir los sesgos de GC y de mapeabilidad: reducir la varianza sistemática rinde más que añadir lecturas.

La cobertura es una medida del número de copias, pero solo después de filtrar duplicados, controlar la calidad de mapeo y corregir los sesgos; una caída de cobertura puede ser una delección o una repetición.

☰ En resumen

- Mapear es encontrar el origen de cada lectura y cuantificar la confianza en él. La programación dinámica exhaustiva es inviable a escala genómica; los mapeadores filtran con índices y reservan el alineamiento para unas pocas regiones candidatas.
- El arreglo de sufijos agrupa las apariciones de un patrón en un intervalo. La BWT es la columna de caracteres precedentes de ese arreglo; el *LF-mapping* ($LF(r) = C[L_r] + \text{Occ}(L_r, r)$) la hace reversible y la búsqueda hacia atrás encuentra un patrón con dos consultas por carácter.
- Los puntos de control de Occ y el muestreo del arreglo de sufijos reducen el índice FM del genoma humano a 1–2 GB, a cambio de más pasos de *LF-mapping* por posición localizada.
- BWA-MEM siembra con SMEM sobre el índice FM; minimap2 con minimizadores, cuya densidad es $\approx 2/(w + 1)$. Ambos encadenan anclas colineales y extienden con programación dinámica en banda.
- El MAPQ es la probabilidad, en escala Phred, de que la posición asignada sea errónea; es bajo en repeticiones aunque el alineamiento sea perfecto, y no es comparable entre programas.
- SAM/BAM guarda cada alineamiento con su FLAG, su CIGAR y la información de la pareja; *samtools* ordena, marca duplicados e indexa. Los pares discordantes y las lecturas partidas señalan variantes estructurales.
- La profundidad por base se calcula en tiempo lineal con un arreglo de diferencias; su razón logarítmica revela delecciones $(-1, -\infty)$ y duplicaciones $(0, 58)$, con un tamaño mínimo detectable inversamente proporcional a la cobertura.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 7.1 ●●● Construya a mano el arreglo de sufijos y la BWT de GATTACA\$. Compruebe con `indice_fm` su resultado y verifique que las letras iguales de L y F conservan su orden relativo.
- 7.2 ●●● Usando la tabla 7.1, ejecute la búsqueda hacia atrás de TAC y de CAT en TACATACAG\$. Dé los intervalos intermedios y las posiciones finales.
- 7.3 ●●● Decodifique los FLAG 83, 147, 163 y 2064. ¿Cuáles pertenecen a pares concordantes? ¿Cuál es un alineamiento suplementario y qué sugiere sobre la lectura?
- 7.4 ●●● Con la ecuación (7.9), calcule la memoria del índice FM para el genoma de trigo ($1,6 \times 10^{10}$ bases) con $d = 128$ y $s = 32$. ¿Cuántos bits necesita cada entero? Discuta qué intervalo s elegiría para que el índice quepa en 32 GB.
- 7.5 ●●● Implemente la reconstrucción de T a partir de L con el *LF-mapping* y compruebe que funciona con una secuencia aleatoria de 10 000 bases.
- 7.6 ●●● Simule secuencias aleatorias y mida la densidad de los minimizadores con orden alfabético y con un *hash* aleatorio para $w \in \{5, 10, 20\}$ y $k = 15$. Compare con la ecuación (7.11). ¿Por qué el orden alfabético es peor en secuencias con muchas A?
- 7.7 ●●● Una lectura tiene tres ubicaciones: una con una discordancia de calidad 30, y dos con dos discordancias de calidades 30 y 30. Calcule el MAPQ con la ecuación (7.16). ¿Cómo cambia si la calidad de la única discordancia de la primera es 10?
- 7.8 ●●● Descargue un BAM público de una muestra humana (por ejemplo, del Proyecto 1000 Genomas), calcule con `mosdepth` la profundidad en ventanas de 1 kb de un cromosoma y busque tramos con $\ell_w < -0,7$. Contraste sus candidatos con las lecturas discordantes vistas en IGV.
- 7.9 ●●● Demuestre la aproximación $2/(w+1)$ de la ecuación (7.11) con detalle, suponiendo que los valores de *hash* son variables continuas independientes. ¿Qué ocurre con la densidad en una región de baja complejidad como ATATA TATAT?

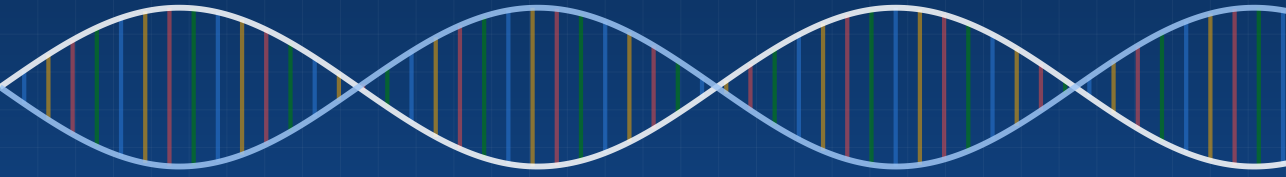
Referencias del capítulo

- Abyzov, A., Urban, A. E., Snyder, M., & Gerstein, M. (2011). CNVnator: An approach to discover, genotype, and characterize typical and atypical CNVs from family and population genome sequencing. *Genome Research*, 21(6), 974-984. <https://doi.org/10.1101/gr.114876.110>
- Adjeroh, D., Bell, T., & Mukherjee, A. (2008). *The Burrows-Wheeler Transform: Data Compression, Suffix Arrays, and Pattern Matching*. Springer. <https://doi.org/10.1007/978-0-387-78909-5>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), Artículo giab008. <https://doi.org/10.1093/gigascience/giab008>
- Faust, G. G., & Hall, I. M. (2014). SAMBLASTER: fast duplicate marking and structural variant read extraction. *Bioinformatics*, 30(17), 2503-2505. <https://doi.org/10.1093/bioinformatics/btu314>
- Ferragina, P., & Manzini, G. (2005). Indexing compressed text. *Journal of the ACM*, 52(4), 552-581. <https://doi.org/10.1145/1082036.1082039>
- Gusfield, D. (1997). *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511574931>
- Langmead, B., & Salzberg, S. L. (2012). Fast gapped-read alignment with Bowtie 2. *Nature Methods*, 9(4), 357-359. <https://doi.org/10.1038/nmeth.1923>
- Langmead, B., Trapnell, C., Pop, M., & Salzberg, S. L. (2009). Ultrafast and memory-efficient alignment of short DNA sequences to the human genome. *Genome Biology*, 10(3), Artículo R25. <https://doi.org/10.1186/gb-2009-10-3-r25>
- Li, H. (2013). *Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM*. arXiv: 1303.3997 [q-bio.GN]. <https://doi.org/10.48550/arXiv.1303.3997>
- Li, H. (2018). Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18), 3094-3100. <https://doi.org/10.1093/bioinformatics/bty191>
- Li, H., & Durbin, R. (2009). Fast and accurate short read alignment with Burrows-Wheeler transform. *Bioinformatics*, 25(14), 1754-1760. <https://doi.org/10.1093/bioinformatics/btp324>
- Li, H., & Durbin, R. (2010). Fast and accurate long-read alignment with Burrows-Wheeler transform. *Bioinformatics*, 26(5), 589-595. <https://doi.org/10.1093/bioinformatics/btp698>
- Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., & 1000 Genome Project Data Processing Subgroup. (2009). The Sequence Alignment/Map format and SAMtools. *Bioinformatics*, 25(16), 2078-2079. <https://doi.org/10.1093/bioinformatics/btp352>

- Li, H., & Homer, N. (2010). A survey of sequence alignment algorithms for next-generation sequencing. *Briefings in Bioinformatics*, 11(5), 473-483. <https://doi.org/10.1093/bib/bbq015>
- Li, H., Ruan, J., & Durbin, R. (2008). Mapping short DNA sequencing reads and calling variants using mapping quality scores. *Genome Research*, 18(11), 1851-1858. <https://doi.org/10.1101/gr.078212.108>
- Mäkinen, V., Belazzougui, D., Cunial, F., & Tomescu, A. I. (2015). *Genome-Scale Algorithm Design: Biological Sequence Analysis in the Era of High-Throughput Sequencing*. Cambridge University Press. <https://doi.org/10.1017/CBO9781139940023>
- Manber, U., & Myers, G. (1993). Suffix arrays: A new method for on-line string searches. *SIAM Journal on Computing*, 22(5), 935-948. <https://doi.org/10.1137/0222058>
- Nurk, S., Koren, S., Rhie, A., Rautiainen, M., et al. (2022). The complete sequence of a human genome. *Science*, 376(6588), 44-53. <https://doi.org/10.1126/science.abj6987>
- Pedersen, B. S., & Quinlan, A. R. (2018). Mosdepth: quick coverage calculation for genomes and exomes. *Bioinformatics*, 34(5), 867-868. <https://doi.org/10.1093/bioinformatics/btx699>
- Quinlan, A. R., & Hall, I. M. (2010). BEDTools: a flexible suite of utilities for comparing genomic features. *Bioinformatics*, 26(6), 841-842. <https://doi.org/10.1093/bioinformatics/btq033>
- Ramírez, F., Ryan, D. P., Grüning, B., Bhardwaj, V., Kilpert, F., Richter, A. S., Heyne, S., Dündar, F., & Manke, T. (2016). deepTools2: a next generation web server for deep-sequencing data analysis. *Nucleic Acids Research*, 44(W1), W160-W165. <https://doi.org/10.1093/nar/gkw257>
- Roberts, M., Hayes, W., Hunt, B. R., Mount, S. M., & Yorke, J. A. (2004). Reducing storage requirements for biological sequence comparison. *Bioinformatics*, 20(18), 3363-3369. <https://doi.org/10.1093/bioinformatics/bth408>
- Robinson, J. T., Thorvaldsdóttir, H., Winckler, W., Guttman, M., Lander, E. S., Getz, G., & Mesirov, J. P. (2011). Integrative genomics viewer. *Nature Biotechnology*, 29(1), 24-26. <https://doi.org/10.1038/nbt.1754>
- SAM/BAM Format Specification Working Group. (2024). *Sequence Alignment/Map Format Specification*. GA4GH / hts-specs. <https://samtools.github.io/hts-specs/SAMv1.pdf>
- Schleimer, S., Wilkerson, D. S., & Aiken, A. (2003). Winnowing: Local algorithms for document fingerprinting. *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*, 76-85. <https://doi.org/10.1145/872757.872770>
- Schneider, V. A., Graves-Lindsay, T., Howe, K., Bouk, N., Chen, H.-C., Kitts, P. A., Murphy, T. D., Pruitt, K. D., Thibaud-Nissen, F., Albracht, D., et al. (2017). Evaluation of GRCh38 and de novo haploid genome assemblies demonstrates the enduring quality of the reference assembly. *Genome Research*, 27(5), 849-864. <https://doi.org/10.1101/gr.213611>

- Thorvaldsdóttir, H., Robinson, J. T., & Mesirov, J. P. (2013). Integrative Genomics Viewer (IGV): high-performance genomics data visualization and exploration. *Briefings in Bioinformatics*, 14(2), 178-192. <https://doi.org/10.1093/bib/bbs017>
- Treangen, T. J., & Salzberg, S. L. (2012). Repetitive DNA and next-generation sequencing: computational challenges and solutions. *Nature Reviews Genetics*, 13(1), 36-46. <https://doi.org/10.1038/nrg3117>

Ensamblaje y anotación de genomas



Abandonamos el enfoque clásico de «solapamiento–disposición–consenso» en favor de un nuevo algoritmo, EULER, que por primera vez resuelve el «problema de las repeticiones», con veinte años de antigüedad, en el ensamblaje de fragmentos.

— Pevzner et al. (2001)

● Al terminar este capítulo usted podrá

- Contar k -mers, construir e interpretar un espectro de k -mers, y usarlo para estimar sin referencia el tamaño, la heterocigosidad y la tasa de error de un genoma.
- Formular el ensamblaje como un problema de grafos, distinguir el paradigma de solapamiento (camino hamiltoniano) del de De Bruijn (camino euleriano) y reconocer puntas, burbujas y repeticiones en un grafo real.
- Ejecutar un ensamblaje con SPAdes y evaluarlo críticamente con N50, NG50, QUASt, BUSCO y métricas basadas en k -mers, sin dejarse engañar por la contigüidad aparente.

- Explicar cómo se predicen genes en procariotas y eucariotas (modelos de Márkov y modelos ocultos de Márkov), por qué se enmascaran las repeticiones y cómo integran la evidencia herramientas como Prokka, Bakta, AUGUSTUS y BRAKER.

Imagine que debe reconstruir un periódico a partir de las tiras producidas al triturar cien ejemplares. No tiene la portada como guía. Las tiras son cortas, algunas letras están manchadas y ciertos fragmentos, como los anuncios, se repiten en distintos lugares. El *ensamblaje de genomas* plantea dificultades parecidas: la secuenciación entrega millones de lecturas de un genoma que aún no hemos visto completo (capítulo 6), y debemos deducir cómo encajan en la secuencia original.

Reconstruir la secuencia es solo el primer paso. Para aprovechar un genoma ensamblado hay que *anotarlo*: localizar los genes, inferir qué proteínas codifican, identificar los ARN ribosómicos y de transferencia y señalar las regiones repetitivas. El ensamblaje y la anotación convierten las lecturas de un FASTQ en un recurso para estudiar biología.

Seguiremos ese recorrido. Primero contaremos k -mers: el histograma de sus frecuencias permite estimar el tamaño del genoma, la heterocigosidad y la calidad de los datos antes de ensamblarlos (sección 8.1). Después construiremos un grafo a partir de ellos y veremos cómo el cambio de representación permite resolver el problema con un algoritmo lineal (sección 8.2). Examinaremos el funcionamiento de SPAdes y los criterios para evaluar su resultado (sección 8.3). Por último, buscaremos genes en la secuencia ensamblada (sección 8.4).

8.1 k -mers y espectros de k -mers

 Colab · 8.1 · k -mers y espectros de k -mers

 abrir notebook

Medir un libro sin leerlo

Suponga que le entregan cuarenta fotocopias de un libro, cada una cortada en frases sueltas, todo mezclado en una caja. No sabe cuántas páginas tiene el libro, pero puede contar cuántas veces aparece cada frase de, digamos, seis palabras. La mayoría aparecerá unas cuarenta veces, una por fotocopia; si cuenta el total de frases de la caja y lo divide entre cuarenta, obtendrá la longitud del libro sin haberlo leído. Además, las frases que aparecen una sola vez delatan

erratas de la fotocopidora; las que aparecen ochenta veces, estribillos que el autor repitió; y si el libro es en realidad *dos* ediciones ligeramente distintas mezcladas, habrá un grupo de frases que aparece veinte veces, las que difieren entre ediciones. Todo eso está escrito en un solo histograma.

8.1.1 Qué es un *k*-mer

La lectura de un secuenciador es demasiado larga y demasiado ruidosa para compararla entera con todas las demás, y demasiado corta para situarla sola en el genoma. Casi todos los algoritmos de ensamblaje, y muchos de mapeo, clasificación metagenómica y detección de variantes, trabajan con una unidad intermedia: las subcadenas de longitud fija.

Definición 8.1. (*k*-mer y *k*-mer canónico)

Un *k*-mer es una subcadena de longitud *k* de una secuencia. Una lectura *s* de longitud *L* contiene $L - k + 1$ *k*-mers, $s_{i..i+k-1}$ para $i = 1, \dots, L - k + 1$. Como no sabemos de qué hebra del ADN procede cada lectura, se identifica cada *k*-mer *x* con su reverso complementario $\bar{x}$ y se representa la pareja por su *k*-mer canónico

$$\kappa(x) = \min\{x, \bar{x}\}, \quad (8.1)$$

donde el mínimo es en orden lexicográfico ($A < C < G < T$).

DONDE

<i>k</i>	Longitud de las palabras que se cuentan (típicamente entre 21 y 127 para lecturas cortas).
<i>L</i>	Longitud de la lectura en pares de bases.
$\bar{x}$	Reverso complementario de <i>x</i> : se invierte el orden y se cambia A↔T, C↔G.
$\kappa(x)$	Representante canónico: la misma clave para las dos hebras.

Hay 4^k *k*-mers posibles. Con $k = 21$ son unos $4,4 \times 10^{12}$, más de mil veces el número de posiciones del genoma humano; un *k*-mer tomado al azar del espacio posible casi nunca estará en un genoma concreto. Esa escasez es precisamente lo que los hace útiles como “huellas”: si un *k*-mer largo aparece en una lectura y en un genoma, es muy probable que la lectura venga de ahí.

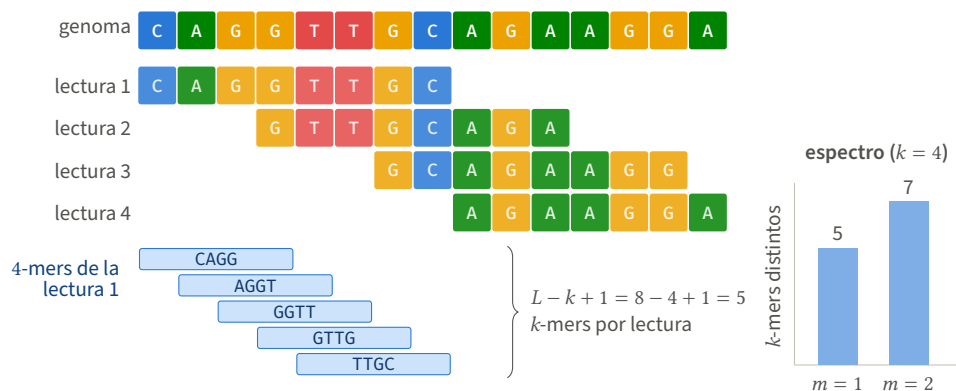


Figura 8.1. De las lecturas a los k -mers. Un genoma de 15 pb (arriba) se ha secuenciado con cuatro lecturas solapadas. Cada lectura se descompone en $L - k + 1$ palabras de longitud k ; aquí se muestran los cinco 4-mers de la primera lectura. Contando los 4-mers canónicos de las cuatro lecturas se obtienen 12 distintos: 5 aparecen una vez y 7 dos veces (derecha). Ese histograma, el *espectro de k -mers*, es el objeto central de esta sección. Observe que la información de *dónde* estaba cada k -mer se ha perdido: solo queda *cuántas* veces aparece.

Un detalle con consecuencias prácticas: casi todas las herramientas usan k impar. La razón es que un k -mer de longitud impar nunca es su propio reverso complementario, porque la base central tendría que ser igual a su complemento, lo que es imposible. Con k par existen palíndromos como GAAATTC (el sitio de *EcoRI*), para los que $x = \bar{x}$ y la hebra queda ambigua; con k impar, cada k -mer canónico corresponde exactamente a dos k -mers orientados distintos.

Por eficiencia, los k -mers no se guardan como texto sino como enteros: cada base ocupa 2 bits (A= 00, C= 01, G= 10, T= 11), de modo que un k -mer con $k \leq 32$ cabe en una palabra de 64 bits. El paso de un k -mer al siguiente dentro de la lectura se hace con un desplazamiento de bits y una máscara, en tiempo constante, y el orden lexicográfico de la ecuación (8.1) coincide con el orden numérico de los enteros.

8.1.2 Cobertura de k -mers

En el capítulo 6 definimos la cobertura de un experimento como $c = NL/G$, el número medio de lecturas que cubren una base. Para los k -mers la cuenta es ligeramente distinta, porque una lectura cubre un k -mer solo si lo contiene *entero*.

Teorema 8.1. (Cobertura de k -mers)

Con N lecturas de longitud L tomadas al azar de un genoma de tamaño G y una tasa de error por base e independiente entre posiciones, el número medio de veces que se observa sin errores un k -mer del genoma es

$$\lambda_k = \underbrace{\frac{N(L - k + 1)}{G}}_{c_k = c \frac{L - k + 1}{L}} \cdot (1 - e)^k. \quad (8.2)$$

DONDE

c	Cobertura por bases, NL/G (capítulo 6).
c_k	Cobertura de k -mers sin tener en cuenta los errores.
e	Probabilidad de que una base de la lectura sea errónea.
$(1 - e)^k$	Probabilidad de que las k bases de una aparición del k -mer sean todas correctas.
λ_k	Cobertura efectiva de k -mers: la posición del pico principal del espectro.

Derivación. Una lectura contiene el k -mer que empieza en la posición i del genoma si la lectura empieza en alguna de las $L - k + 1$ posiciones $i - (L - k), \dots, i$. Con G posiciones de inicio equiprobables, cada lectura lo contiene con probabilidad $(L - k + 1)/G$, y el número esperado de lecturas que lo contienen es $N(L - k + 1)/G = c(L - k + 1)/L$. De esas apariciones, solo cuentan como el k -mer correcto las que no tienen ningún error en sus k bases, lo que ocurre con probabilidad $(1 - e)^k$. Las demás se convierten en k -mers *distintos*, casi siempre únicos. $\square$

La ecuación (8.2) tiene dos factores que crecen en direcciones opuestas con k , y los dos castigan los k grandes: $(L - k + 1)/L$ disminuye linealmente y $(1 - e)^k$ lo hace geométricamente. Con lecturas de 150 pb, $e = 0,5\%$ y $k = 21$, una cobertura de $20\times$ se convierte en $c_k = 17,3$ y en una cobertura efectiva $\lambda_k = 17,3 \times 0,900 = 15,6$. Uno de cada diez k -mers leídos contiene al menos un error.

8.1.3 El espectro de k -mers

Definición 8.2. (Espectro de k -mers)

Sea $n(x)$ el número de veces que el k -mer canónico x aparece en el conjunto de lecturas. El *espectro de k -mers* es la función

$$h(m) = \#\{x : n(x) = m\}, \quad m = 1, 2, 3, \dots \quad (8.3)$$

es decir, el número de k -mers *distintos* que aparecen exactamente m veces. Su “masa” $\sum_m m h(m)$ es el número total de k -mers leídos.

DONDE

$n(x)$	Multiplicidad del k -mer x : cuántas veces se leyó.
m	Multiplicidad (eje horizontal del espectro).
$h(m)$	Número de k -mers distintos con multiplicidad m (eje vertical).

Si el genoma fuera haploide, sin repeticiones y sin errores de secuenciación, cada uno de sus $\approx G$ k -mers se observaría un número de veces aproximadamente de Poisson con media λ_k (por el mismo argumento de Lander & Waterman, 1988, visto en el capítulo 6), y el espectro sería una única campana centrada en λ_k con área G . Los genomas reales añaden tres ingredientes, cada uno con una firma inconfundible:

- **Errores de secuenciación.** Cada error crea hasta k k -mers que no existen en el genoma y que casi nunca se repiten. Forman un pico enorme en $m = 1-3$ que decae muy deprisa.
- **Heterocigosidad.** En un diploide, los k -mers que cubren un sitio heterocigoto existen en una sola de las dos copias del cromosoma y se leen la mitad de veces que los demás. Forman un segundo pico a la mitad de la cobertura.
- **Repeticiones.** Un k -mer presente en r copias del genoma se lee r veces más. Forman picos (o colas) en múltiplos de la cobertura.

8.1.4 Un modelo para el espectro de un diploide

Formalicemos la heterocigosidad, que es la parte más útil del modelo. Sea h la tasa de heterocigosidad: la probabilidad de que una posición del genoma difiera entre los dos haplotipos. Sea λ la cobertura efectiva de k -mers *por haplotipo*, de modo que un k -mer presente en los dos haplotipos se lee en promedio 2λ veces.

Teorema 8.2. (Espectro de un genoma diploide)

Un k -mer del genoma cubre al menos un sitio heterocigoto con probabilidad

$$\alpha = 1 - (1 - h)^k. \quad (8.4)$$

Si el genoma haploide tiene G posiciones, hay en promedio $G(1 - \alpha)$ k -mers homocigotos, con multiplicidad media 2λ , y $2G\alpha$ k -mers heterocigotos (uno por haplotipo), con multiplicidad media λ . Ignorando repeticiones, el espectro es la mezcla

$$h(m) \approx h_{\text{err}}(m) + 2G\alpha f(m; \lambda) + G(1 - \alpha) f(m; 2\lambda), \quad (8.5)$$

donde $f(\cdot; \mu)$ es una distribución de conteo de media μ (Poisson o, más realista, binomial negativa).

DONDE

h	Tasa de heterocigosidad: fracción de sitios que difieren entre los dos haplotipos.
α	Fracción de posiciones cuyo k -mer contiene al menos un sitio heterocigoto.
λ	Cobertura efectiva de k -mers de un haplotipo (el pico “1×”).
$h_{\text{err}}(m)$	Contribución de los k -mers erróneos, concentrada en m pequeños.
$f(m; \mu)$	Probabilidad de observar m copias cuando la media es μ .

La ecuación (8.4) explica una observación que sorprende a muchos: una heterocigosidad modesta produce un pico heterocigoto enorme. Con $h = 1\%$ y $k = 21$, $\alpha = 1 - 0.99^{21} = 0.19$: casi uno de cada cinco k -mers es heterocigoto, y como cada posición heterocigota aporta *dos* k -mers distintos (uno por alelo), el pico de λ contiene la mitad de k -mers que el de 2λ . El efecto crece con k , porque una palabra más larga tiene más oportunidades de tocar un sitio variable.

Estimación del tamaño del genoma. El modelo contiene un resultado de gran belleza práctica. Calculemos la masa total de los k -mers verdaderos (los que no son errores):

$$\underbrace{G(1 - \alpha) \cdot 2\lambda}_{\text{homocigotos}} + \underbrace{2G\alpha \cdot \lambda}_{\text{heterocigotos}} = 2G\lambda \implies \hat{G} = \frac{\sum_{m \geq v} m h(m)}{2\lambda}. \quad (8.6)$$

La heterocigosidad α se cancela. Basta sumar el espectro por encima del valle v que lo separa de los errores y dividir por la multiplicidad del pico homocigoto, 2λ . Las repeticiones tampoco estorban: un k -mer con r copias aporta r veces más masa, exactamente lo que le corresponde. Para un organismo haploide, el denominador es simplemente la posición del único pico, λ .

Estimación de la heterocigosidad. Si ajustamos la ecuación (8.5) y obtenemos los tamaños de los dos componentes, $N_{\text{het}} \approx 2G\alpha$ y $N_{\text{hom}} \approx G(1 - \alpha)$, despejamos

$$\hat{\alpha} = \frac{N_{\text{het}}}{N_{\text{het}} + 2N_{\text{hom}}}, \quad \hat{h} = 1 - (1 - \hat{\alpha})^{1/k}. \quad (8.7)$$

DONDE		
v		Multiplicidad del valle entre el pico de errores y el primer pico genómico.
$\hat{G}$		Tamaño estimado del genoma haploide.
$N_{\text{het}}, N_{\text{hom}}$		Número de k -mers distintos asignados a cada componente del ajuste.

Este es, en esencia, el modelo de GenomeScope (Vurture et al., 2017), que ajusta por mínimos cuadrados no lineales una mezcla de binomiales negativas en λ , 2λ , 3λ y 4λ (las dos últimas para capturar las repeticiones de dos copias) y devuelve el tamaño del genoma, la heterocigosidad, la fracción repetitiva y la tasa de error a partir de lecturas sin procesar. Sus autores lo validaron con 324 conjuntos de datos simulados y 16 reales. GenomeScope 2.0 generalizó el modelo a poliploides mediante un razonamiento combinatorio sobre cuántos haplotipos comparten cada k -mer, y lo acompañó de Smudgeplot, que estima la ploidía a partir de pares de k -mers heterocigotos (Ranallo-Benavidez et al., 2020).

Ejemplo 8.1. (Perfilar un genoma antes de ensamblarlo)

Tomamos el conjunto simulado de la figura 8.2. El conteo produce 34 666 580 21-mers leídos, de los cuales 4 535 463 son distintos y 3 297 024 aparecen una sola vez.

1. *Valle.* El mínimo local entre el pico de errores y el primer pico genómico está en $v = 4$ ($h(4) = 166$).

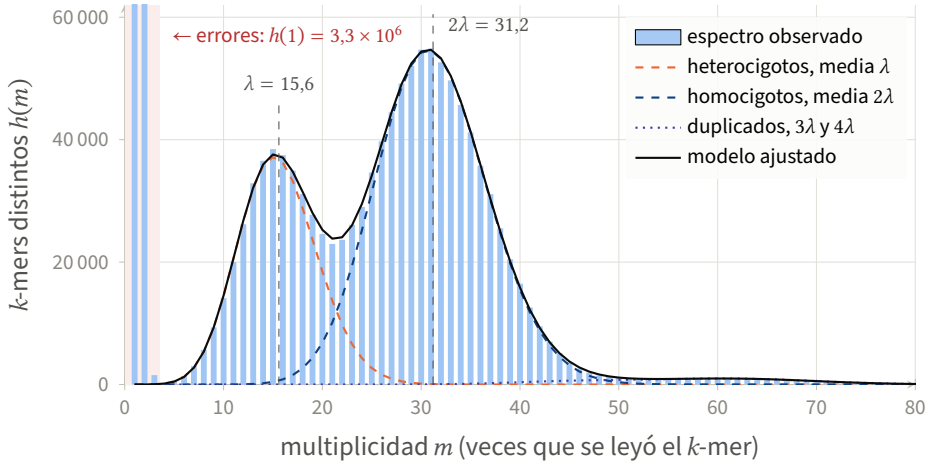


Figura 8.2. Espectro de 21-mers de un genoma diploide simulado de 1 Mb con heterocigosidad del 1 %, 15 duplicaciones segmentarias de 2 kb, lecturas de 150 pb a 20× por haplotipo y 0,5 % de errores. La zona roja ($m \leq 3$) contiene los k -mers erróneos: son el 74 % de los k -mers *distintos*, aunque solo el 10 % de los leídos. A la derecha del valle aparecen el pico heterocigoto (naranja), en λ , y el homocigoto (azul), en 2λ , más una pequeña cola de k -mers duplicados. El pico heterocigoto tiene alrededor de la mitad de k -mers que el homocigoto, como predice la ecuación (8.4) con $\alpha \approx 0,19$. El ajuste de la ecuación (8.5) recupera $\hat{G} = 1\,000\,409$ pb y $\hat{h} = 1,02$ %. Generado con `figuras/cap08/generar.py`.

2. **Cobertura.** El ajuste da $\lambda = 15,60$. La ecuación (8.2) predice $20 \times \frac{130}{150} \times 0,995^{21} = 17,33 \times 0,900 = 15,60$: la coincidencia confirma que el desplazamiento del pico respecto de c_k se debe a los errores.
3. **Tamaño.** La masa por encima del valle es $\sum_{m \geq 4} m h(m) = 31\,213\,016$, luego

$$\hat{G} = \frac{31\,213\,016}{2 \times 15,60} = 1\,000\,409 \text{ pb},$$

a menos de un 0,05 % del valor real (10^6 pb).

4. **Heterocigosidad.** El ajuste asigna $N_{\text{het}} = 367\,251$ y $N_{\text{hom}} = 765\,306$ k -mers distintos a los dos picos. Con la ecuación (8.7): $\hat{\alpha} = 367\,251 / (367\,251 + 2 \times 765\,306) = 0,1935$ y $\hat{h} = 1 - (1 - 0,1935)^{1/21} = 1,02$ %, frente al 1,00 % simulado.

Sin alinear una sola lectura contra nada, sabemos el tamaño del genoma, que es diploide, cuán heterocigoto es y que la cobertura es suficiente. Con esa información se elige el ensamblador y sus parámetros.

⚠ Confundir los picos duplica el genoma

Si la heterocigosidad es alta o la cobertura baja, el pico heterocigoto puede ser *más alto* que el homocigoto. Un análisis ingenuo que tome el pico más alto como “la cobertura” usará λ en lugar de 2λ en la ecuación (8.6) y estimará un genoma del *doble* de tamaño. Otros motivos de espectros engañosos son la contaminación (un tercer pico ajeno, a otra cobertura), la amplificación del genoma completo (picos anchos y deformados) y las muestras de varios individuos mezclados. Mire siempre el espectro completo, no solo los números que devuelve el programa.

8.1.5 Contar miles de millones de k -mers

Contar k -mers parece trivial, y en principio lo es: basta un diccionario. El problema es la escala. Un genoma humano a 30× con lecturas de 150 pb contiene unos 8×10^{10} 31-mers leídos, y el número de k -mers distintos, inflado por los errores, puede superar los 5×10^9 . Un diccionario de Python necesitaría cientos de gigabytes.

Marçais y Kingsford (2011) resolvieron el problema con Jellyfish, un contador basado en una tabla *hash* de direccionamiento abierto cuyos incrementos se hacen con instrucciones atómicas del procesador (*compare-and-swap*). Así, muchos hilos pueden insertar en la misma tabla sin bloquearla (*lock-free*), y el conteo escala casi linealmente con el número de núcleos. La versión original admitía k -mers de hasta 31 bases, la longitud que cabe en 62 bits. KMC 3 (Kokot et al., 2017) adoptó una estrategia complementaria: reparte los k -mers en particiones en disco y las cuenta después por lotes, de modo que la memoria necesaria es una fracción de la de una tabla completa, e incluye utilidades para operar con los conjuntos resultantes (intersecciones, diferencias, filtros por multiplicidad).

```
espectro.py
```

```
from collections import Counter
```

```
COMP = str.maketrans("ACGT", "TGCA")
```

```
def canonico(kmer: str) -> str:
    """El menor entre el  $k$ -mer y su reverso complementario."""
    return min(kmer, kmer.translate(COMP)[::-1])

def contar_kmers(lecturas, k=21):
    c = Counter()
    for s in lecturas:
        for i in range(len(s) - k + 1):
            km = s[i:i + k]
            if "N" not in km:
                c[canonico(km)] += 1
    return c

def espectro(conteos):
    """h[m] = número de  $k$ -mers distintos vistos  $m$  veces."""
    return Counter(conteos.values())

lect = ["CAGGTTGC", "GTTGCAGA", "GCAGAAGG", "AGAAGGA"]
print(sorted(espectro(contar_kmers(lect, k=4)).items()))
# [(1, 5), (2, 7)] <- el espectro de la figura 8.1
```

Para datos reales se usan las herramientas especializadas. El flujo típico es contar, construir el histograma y ajustarlo:

Perfil del genoma con Jellyfish (o KMC) y GenomeScope

```
# 1) contar 21-mers canónicos (-C) con 8 hilos
jellyfish count -C -m 21 -s 1G -t 8 -o k21.jf \
    <(zcat lect_R1.fq.gz) <(zcat lect_R2.fq.gz)
jellyfish histo -t 8 k21.jf > k21.histo

# alternativa con KMC: contar e histograma
kmc -k21 -t8 -ci1 -cs100000 @archivos.txt k21 tmp/
kmc_tools transform k21 histogram k21.histo -cx100000

# 2) ajustar el modelo (diploide: -p 2)
genomescope2 -i k21.histo -o perfil/ -k 21 -p 2
```

8.1.6 Elegir k

La elección de k es un compromiso entre dos exigencias opuestas. Por un lado, k debe ser lo bastante grande para que los k -mers sean *únicos* en el genoma: si un k -mer aparece en muchos lugares por puro azar, deja de identificar una posición. Por

otro, debe ser lo bastante pequeño para que los k -mers tengan cobertura suficiente y no se vean arruinados por los errores (ecuación (8.2)).

Cuantifiquemos la primera exigencia con un modelo nulo: un genoma aleatorio de G bases equiprobables. Un k -mer dado puede reaparecer en cualquiera de las $\approx 2G$ posiciones de las dos hebras, cada una con probabilidad 4^{-k} . El número de apariciones casuales es aproximadamente de Poisson con media $2G/4^k$, y la probabilidad de que haya al menos una es

$$p_{\text{azar}}(k) = 1 - \exp\left(-\frac{2G}{4^k}\right), \quad p_{\text{azar}} \ll 1 \iff k \gg \log_4(2G). \quad (8.8)$$

DONDE

$p_{\text{azar}}(k)$	Probabilidad de que un k -mer del genoma aparezca otra vez por azar (no por una repetición biológica).
$2G/4^k$	Número esperado de coincidencias casuales en las dos hebras.
$\log_4(2G)$	Longitud a partir de la cual los k -mers aleatorios empiezan a ser únicos.

Para una bacteria de 5 Mb, $\log_4(2G) \approx 11,6$; para el genoma humano, $\approx 16,3$. Con $k = 17$, un k -mer bacteriano tiene una probabilidad de coincidencia casual de 6×10^{-4} , pero uno humano, del 30 %. Con $k = 21$ ambas son despreciables (2×10^{-6} y $1,4 \times 10^{-3}$), y por eso $k = 21$ es un valor habitual para perfilar genomas. La figura 8.3 muestra el compromiso completo. Por supuesto, los genomas reales no son aleatorios: sus repeticiones biológicas (transposones, familias génicas, duplicaciones) son mucho más largas que cualquier k razonable, y ningún k las resuelve. Lo retomaremos en la sección 8.2.

Los k -mers como regla de medir la calidad

El espectro no solo sirve *antes* del ensamblaje. Si contamos por separado los k -mers de las lecturas y los del ensamblaje terminado, cada k -mer del ensamblaje que no aparece en las lecturas señala casi con certeza un error de consenso. Merqury (Rhie et al., 2020) convierte esa idea en una estimación de la exactitud por base sin necesidad de referencia, y usa los espectros de los padres para medir si un ensamblaje separa correctamente los dos haplotipos. Lo usaremos en la sección 8.3.

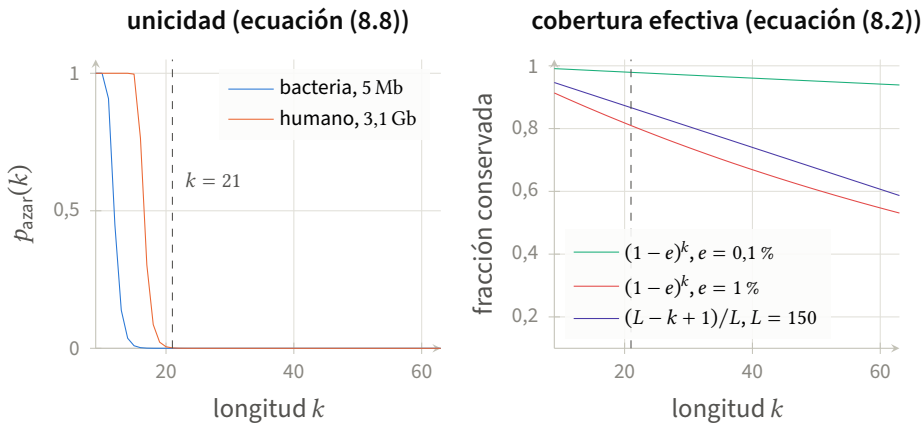


Figura 8.3. El compromiso en la elección de k . **Izquierda:** probabilidad de que un k -mer reaparezca por azar en un genoma aleatorio; la transición ocurre cerca de $\log_4(2G)$, unas cuatro bases más tarde en el genoma humano que en una bacteria. **Derecha:** los dos factores de la ecuación (8.2) que reducen la cobertura efectiva al crecer k . Con errores del 1 %, un 31-mer se conserva intacto solo el 73 % de las veces. Un k pequeño confunde posiciones; uno grande diluye la señal. Los ensambladores modernos resuelven el dilema usando varios valores de k a la vez (sección 8.3).

El espectro de k -mers es una radiografía gratuita del genoma: el pico en 2λ da la cobertura, la masa bajo la curva dividida por 2λ da el tamaño, y el pico en λ delata la heterociguidad.

8.2 Grafos de De Bruijn

Colab · 8.2 · Grafos de De Bruijn

abrir notebook

El cartero y el inspector

Dos funcionarios municipales reciben el plano de un barrio. El cartero debe recorrer *cada calle* exactamente una vez; el inspector debe visitar *cada casa* exactamente una vez. Parecen tareas hermanas, pero no lo son. El cartero tiene una regla sencilla para saber si su ruta existe: basta con que en cada

esquina entren tantas calles como salen, y cualquier estudiante puede construir el recorrido en un par de minutos. El inspector, en cambio, no dispone de ninguna regla general: para un barrio grande, ni el mejor ordenador del mundo garantiza encontrar su ruta en un tiempo razonable. Durante dos décadas, los ensambladores de genomas trabajaron como el inspector. El gran avance consistió en redibujar el plano para que el problema fuera el del cartero.

8.2.1 El ensamblaje como problema de grafos

Formulado de la manera más ingenua, el ensamblaje busca la *supercadena común más corta*: la cadena más breve que contiene todas las lecturas como subcadenas. Es un problema NP-difícil, y además está mal planteado desde el punto de vista biológico, porque la supercadena más corta colapsa todas las copias de una repetición en una sola. Las formulaciones útiles se apoyan en grafos.

Solapamiento–disposición–consenso (OLC). El paradigma clásico construye un *grafo de solapamiento*: cada lectura es un nodo, y se traza una arista $u \rightarrow v$ cuando un sufijo de u coincide con un prefijo de v en al menos T bases. Reconstruir el genoma equivale a encontrar un camino que visite cada nodo una vez, un *camino hamiltoniano*. El paradigma tiene tres fases: calcular todos los solapamientos (en principio, $O(N^2)$ comparaciones de lecturas), decidir la disposición de las lecturas y calcular la secuencia de consenso. Encontrar un camino hamiltoniano es NP-completo, así que los ensambladores OLC recurren a heurísticas. Myers (2005) dio al paradigma una forma más limpia, el *grafo de cadenas* (*string graph*), que elimina las aristas transitivas (si $u \rightarrow v \rightarrow w$ y $u \rightarrow w$, la arista $u \rightarrow w$ no añade información) y deja un grafo en el que muchas regiones del genoma aparecen como caminos simples.

De Bruijn. El enfoque alternativo cambia los papeles de nodos y aristas. En lugar de lecturas, los nodos son palabras cortas de longitud $k-1$, y cada k -mer leído es una arista entre su prefijo y su sufijo. Idury y Waterman (1995) propusieron representar las lecturas de esta forma, y Pevzner et al. (2001) mostraron que el ensamblaje se reduce entonces a una variante del problema de encontrar un *camino euleriano*, que recorre cada arista una vez y se resuelve en tiempo lineal.

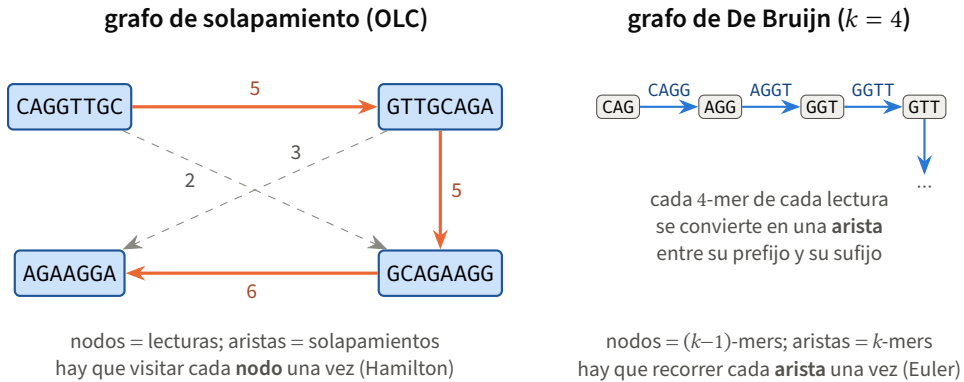


Figura 8.4. Las mismas cuatro lecturas de la figura 8.1 en los dos paradigmas. **Izquierda:** en el grafo de solapamiento cada lectura es un nodo y cada solapamiento sufijo–prefijo, una arista con su longitud; las aristas naranjas forman el camino hamiltoniano correcto, y las grises son solapamientos cortos espurios que un umbral $T \geq 4$ eliminaría. **Derecha:** en el grafo de De Bruijn las lecturas desaparecen como entidades; solo quedan sus k -mers, convertidos en aristas. El grafo completo se muestra en la figura 8.5.

Definición 8.3. (Grafo de De Bruijn de un conjunto de lecturas)

Dado un conjunto de lecturas y un entero k , el *grafo de De Bruijn* $G_k = (V, E)$ es el multigrafo dirigido en el que

- V es el conjunto de $(k-1)$ -mers distintos presentes en las lecturas;
- por cada k -mer $x = x_1 x_2 \dots x_k$ se añade una arista del nodo $x_1 \dots x_{k-1}$ (su prefijo) al nodo $x_2 \dots x_k$ (su sufijo).

Si un k -mer aparece varias veces en el genoma, se conserva una arista por aparición (o una arista con multiplicidad).

8.2.2 El teorema de Euler

La razón por la que el cambio de representación importa es un teorema de 1736. Leonhard Euler lo formuló al estudiar si era posible pasear por Königsberg cruzando cada uno de sus siete puentes exactamente una vez; la versión para grafos dirigidos es la que necesitamos.

Teorema 8.3. (Caminos eulerianos en grafos dirigidos)

Sea $G = (V, E)$ un grafo dirigido conexo (ignorando la orientación de las aristas) y sean $d^-(v)$ y $d^+(v)$ el número de aristas que entran en y salen de v . Entonces:

1. G tiene un ciclo euleriano si y solo si $d^-(v) = d^+(v)$ para todo v (el grafo está balanceado).
2. G tiene un camino euleriano de s a $t \neq s$ si y solo si

$$d^+(s) = d^-(s) + 1, \quad d^-(t) = d^+(t) + 1, \quad d^-(v) = d^+(v) \quad \forall v \notin \{s, t\}. \quad (8.9)$$

DONDE

$d^-(v), d^+(v)$	Grado de entrada y de salida del nodo v .
s, t	Nodos inicial y final del camino (en un genoma lineal, el primer y el último $(k - 1)$ -mer).

Por qué es cierto. La necesidad es casi evidente: cada vez que un recorrido entra en un nodo intermedio debe volver a salir, así que las aristas de entrada y salida se emparejan; solo el inicio tiene una salida “de más” y el final una entrada “de más”. La suficiencia es constructiva, y esa construcción es el algoritmo. Si añadimos una arista ficticia $t \rightarrow s$, el grafo queda balanceado y basta con encontrar un ciclo. Empiece en cualquier nodo y camine por aristas no usadas; como el grafo está balanceado, solo puede quedarse atascado en el nodo de partida, de modo que ha trazado un ciclo. Si quedan aristas sin usar, alguna sale de un nodo del ciclo (por conexidad); inicie allí un nuevo recorrido y empálmelo en el ciclo anterior. Repitiendo, se agotan las aristas. $\square$

Esta demostración es el *algoritmo de Hierholzer*, que se implementa con una pila y recorre cada arista una sola vez: tiempo $O(|E|)$. Para un genoma bacteriano con $|E| \approx 5 \times 10^6$ aristas, son milisegundos. Compárese con el camino hamiltoniano, para el que no se conoce ningún algoritmo polinómico. Compeau et al. (2011) explican con detalle por qué este cambio de perspectiva transformó el ensamblaje con lecturas cortas.

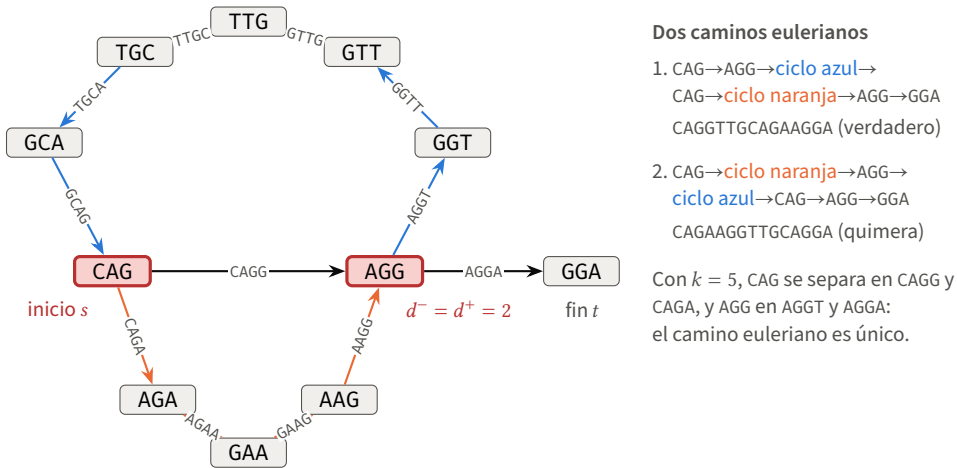


Figura 8.5. Grafo de De Bruijn con $k = 4$ del genoma CAGGTTGCAGAAGGA. Cada nodo es un 3-mer y cada arista, rotulada con su 4-mer, une el prefijo con el sufijo. Los nodos rojos, CAG y AGG, aparecen dos veces en el genoma: son una repetición de longitud $k - 1 = 3$ que crea dos ciclos (azul y naranja). Todo camino euleriano de CAG a GGA usa las 12 aristas, pero hay dos, que recorren los ciclos en órdenes distintos y deletrean dos secuencias igualmente compatibles con los datos. Solo una es el genoma. Los números se comprobaron con `figuras/cap08/generar.py`.

8.2.3 Un ejemplo completo

Tomemos el genoma de juguete $S = \text{CAGGTTGCAGAAGGA}$ (15 pb) y $k = 4$. Sus 12 4-mers son todos distintos, pero los 3-mers CAG y AGG aparecen dos veces cada uno: son una *repetición* de longitud $k - 1$. El grafo de De Bruijn (figura 8.5) tiene 11 nodos y 12 aristas. Los grados cumplen la ecuación (8.9): CAG tiene una salida de más (es s), GGA una entrada de más (es t), y AGG, con dos entradas y dos salidas, está balanceado.

La figura 8.5 muestra el fenómeno central del ensamblaje: el grafo admite *dos* caminos eulerianos,

$$\text{CAGGTTGCAGAAGGA} \quad \text{y} \quad \text{CAGAAGGTTGCAGGA},$$

que contienen exactamente los mismos 4-mers con la misma multiplicidad. Ningún algoritmo puede distinguirlos usando solo esos 4-mers; de hecho, la implementación de Hierholzer que damos más abajo devuelve la segunda, la incorrecta. Con $k = 5$ la repetición CAG/AGG queda dentro de nodos más largos y distintos, y el camino euleriano pasa a ser único. Esta es la forma precisa de la regla general: *si ningún $(k - 1)$ -mer se repite en el genoma, el grafo de De Bruijn es un camino simple y el*

ensamblaje es único. Cada repetición más larga que $k - 1$ introduce, en cambio, ramificaciones cuyo orden de recorrido no está determinado por los datos.

🔧 ¿Cuántos caminos eulerianos hay?

El número de ciclos eulerianos de un grafo dirigido balanceado se puede contar exactamente con el teorema BEST (por de Bruijn, van Aardenne-Ehrenfest, Smith y Tutte): es el número de árboles de expansión orientados hacia un nodo raíz, multiplicado por $\prod_v (d^+(v) - 1)!$. Como el número de árboles de expansión puede ser exponencial, un genoma con muchas repeticiones admite una cantidad astronómica de reconstrucciones compatibles con los k -mers. Por eso los ensambladores no intentan elegir una: se detienen en las ramificaciones y entregan los tramos no ambiguos.

```
de_bruijn.py

from collections import defaultdict

def de_bruijn(lecturas, k):
    """Nodos: (k-1)-mers; una arista u->v por cada k-mer."""
    g = defaultdict(list)
    for s in lecturas:
        for i in range(len(s) - k + 1):
            g[s[i:i + k - 1]].append(s[i + 1:i + k])
    return g

def camino_euleriano(g, inicio):
    """Algoritmo de Hierholzer,  $O(|E|)$ ."""
    g = {u: list(v) for u, v in g.items()} # copia consumible
    pila, camino = [inicio], []
    while pila:
        u = pila[-1]
        if g.get(u): # quedan aristas sin usar
            pila.append(g[u].pop())
        else: # callejón: fijar el nodo
            camino.append(pila.pop())
    camino.reverse()
    return camino[0] + "".join(v[-1] for v in camino[1:])

g = de_bruijn(["CAGGTTGCAGAAGGA"], k=4)
print(camino_euleriano(g, "CAG")) # CAGAAGGTTGCAGGA (quimera)
g = de_bruijn(["CAGGTTGCAGAAGGA"], k=5)
print(camino_euleriano(g, "CAGG")) # CAGGTTGCAGAAGGA (correcto)
```

8.2.4 Unitigs: lo que sí es seguro

Si no podemos elegir entre caminos, ¿qué podemos afirmar? Que los tramos *sin ramificaciones* son correctos. Un *unitig* es un camino máximo cuyos nodos internos tienen exactamente una entrada y una salida. En el ejemplo hay cuatro: **CAGG**, **AG**, **GTTGCAG**, **CAGAAGG** y **AGGA**. Cualquier camino euleriano es una concatenación de estos unitigs (con solapamiento de $k - 1$ bases), y los ensambladores de lecturas cortas los entregan como *contigs* en lugar de arriesgarse con una elección. El *grafo compactado*, en el que cada unitig se reduce a un único nodo, tiene muchísimos menos nodos que el original y es la estructura con la que trabajan en la práctica.

8.2.5 El grafo real: puntas, burbujas y marañas

El ejemplo anterior era limpio: una lectura perfecta de cada k -mer. Con datos reales, el grafo se llena de estructuras parásitas (figura 8.6).

- **Puntas** (*tips*). Un error cerca del extremo de una lectura crea un camino corto que se desprende del grafo y termina en un callejón sin salida, con cobertura muy baja.
- **Burbujas** (*bubbles*). Un error en el interior de una lectura crea un camino alternativo de k aristas que se separa y vuelve a unirse al camino principal. Un sitio heterocigoto produce exactamente la misma forma, pero con las dos ramas a cobertura parecida (cada una a λ , frente a 2λ del resto).
- **Marañas** (*tangles*). Una repetición más larga que $k - 1$ se colapsa en un solo camino por el que pasan varios recorridos del genoma; el tramo colapsado tiene una cobertura igual a la de una región única multiplicada por el número de copias.

¿Cuántas de estas estructuras hay? Cada error de secuenciación en el interior de una lectura genera hasta k k -mers inexistentes en el genoma. Con NL bases leídas y tasa de error e , el número de k -mers espurios es aproximadamente

$$\mathbb{E}[\#k\text{-mers erróneos}] \approx NLek. \quad (8.10)$$

DONDE

NL	Número total de bases leídas (= cG).
NLe	Número esperado de errores de secuenciación.
k	Cada error contamina hasta k k -mers consecutivos.

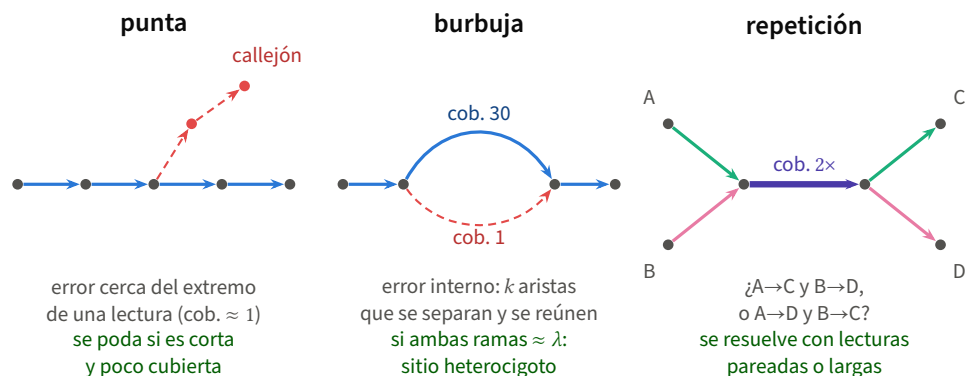


Figura 8.6. Las tres estructuras que complican un grafo de De Bruijn real (cada flecha representa un camino no ramificado). **Punta:** un error cerca del extremo de una lectura genera una rama corta sin salida. **Burbuja:** un error interior, o un sitio heterocigoto, crea dos caminos paralelos; la cobertura de cada rama distingue ambos casos, lo mismo que el espectro de la figura 8.2. **Repetición:** dos regiones distintas del genoma (A y B) comparten una secuencia más larga que k , que el grafo colapsa en un único tramo con el doble de cobertura; los datos del grafo no dicen si A continúa hacia C o hacia D.

Para un genoma humano a $30\times$ ($NL = 9,3 \times 10^{10}$) con $e = 0,1\%$ y $k = 31$, la ecuación (8.10) da unos $2,9 \times 10^9$ k -mers espurios: casi tantos como los 3×10^9 verdaderos. El grafo “en bruto” es el doble de grande de lo necesario. De ahí que el primer paso de todo ensamblador sea corregir las lecturas o descartar los k -mers por debajo del valle del espectro (sección 8.1). La ecuación (8.10) es una aproximación de primer orden: la forma exacta, que cuenta las ventanas con al menos un error y no los errores multiplicados por k , es $N(L - k + 1)[1 - (1 - e)^k]$; con ek pequeño y $k \ll L$ ambas coinciden. En el ejemplo humano, con lecturas de $L = 150$, la forma exacta da unos $2,3 \times 10^9$ en lugar de $2,9 \times 10^9$: el orden de magnitud, y la conclusión, no cambian.

Velvet y la limpieza del grafo. Zerbino y Birney (2008) publicaron Velvet, el primer ensamblador de De Bruijn de uso generalizado para las lecturas muy cortas (25–50 pb) de los primeros secuenciadores Solexa/Illumina. Su aportación principal fue un conjunto de operaciones de limpieza: fusionar los caminos no ramificados, podar las puntas cortas y de baja cobertura, eliminar las burbujas con un recorrido del grafo que compara caminos paralelos y conserva el mejor cubierto, y usar después la información de las lecturas pareadas para atravesar repeticiones. En simulaciones

de genomas procariotas, Velvet alcanzó *contigs* con N50 de hasta 50 kb usando lecturas pareadas; con datos reales de Solexa sin pares, unos 8 kb en una bacteria.

⚠ Podar demasiado borra biología

Las operaciones de limpieza se basan en la cobertura: lo poco cubierto se considera error. Pero hay biología genuina con poca cobertura: alelos de una población minoritaria en un metagenoma, plásmidos de baja copia, regiones ricas en GC infrarrepresentadas por la PCR (capítulo 6). Y la eliminación de burbujas funde los dos haplotipos de un diploide en un mosaico que no existe en ninguna célula. Si su pregunta biológica depende de variantes de baja frecuencia o de la fase de los haplotipos, un ensamblaje “limpio” puede haber borrado justamente la respuesta.

8.2.6 Repeticiones, pares y lecturas largas

El análisis del ejemplo de juguete se generaliza. Una repetición de longitud R solo puede resolverse si alguna pieza de información la *atraviesa* de lado a lado, anclada en secuencia única a ambos lados:

- un k -mer con $k > R + 1$, lo que exige lecturas más largas que R y buena cobertura con k grande (figura 8.3);
- un par de lecturas cuya distancia (el tamaño del inserto) sea mayor que R , de modo que una lectura caiga a cada lado;
- una lectura larga, de PacBio u Oxford Nanopore, que contenga la repetición completa.

Por eso las limitaciones del ensamblaje con lecturas cortas no se superan secuenciando *más*, sino secuenciando *más largo*. La teoría de Lander-Waterman (capítulo 6) predice un único *contig* para una bacteria a 30×; en la práctica se obtienen decenas, uno por cada repetición más larga que la lectura (o que el inserto, si se usan pares), como los operones de ARN ribosómico de unos 5 kb, que las bacterias suelen tener en varias copias. Nagarajan y Pop (2013) ofrecen una revisión muy clara de cómo interactúan cobertura, longitud de lectura y repeticiones.

🔗 De los puentes de Königsberg al genoma

La idea de convertir lecturas en palabras y palabras en aristas surgió en el contexto de la secuenciación por hibridación, una tecnología que nunca llegó a imponerse.

Idury y Waterman (1995) la trasladaron al ensamblaje de fragmentos, y Pevzner et al. (2001) la llevaron a su forma madura con EULER, que, según sus autores, en lugar de enmascarar las repeticiones las usaba como herramienta de ensamblaje. Durante unos años la idea pareció una curiosidad académica, porque las lecturas de Sanger (capítulo 6) eran largas y los ensambladores OLC funcionaban bien. Cuando llegaron las lecturas de 36 pb de Solexa, calcular N^2 solapamientos entre cientos de millones de lecturas se volvió imposible, y el grafo de De Bruijn pasó a ser la estructura dominante.

OLC: lecturas como nodos, camino hamiltoniano (NP-completo).
De Bruijn: k -mers como aristas, camino euleriano (lineal). Las repeticiones más largas que k hacen que el camino no sea único; los ensambladores entregan los tramos sin ambigüedad (unitigs).

8.3 Ensamblaje con SPAdes y evaluación con QUAST

 Colab · 8.3 · Ensamblaje con SPAdes y evaluación con QUAST  [abrir notebook](#)

El rompecabezas sin la foto de la caja

Cuando termina un rompecabezas de mil piezas, sabe si lo hizo bien porque compara el resultado con la foto de la caja. Ahora imagine que no hay caja. Puede contar cuántos bloques sueltos le quedaron y medir el más grande, pero eso no le dice si algún bloque está mal armado: dos piezas de cielo azul encajan igual de bien en muchos sitios, y forzarlas produce un bloque grande y falso. Para juzgar su trabajo necesita otras pistas: que no le sobren ni le falten piezas de los bordes, que ciertas piezas que sabe que existen (la firma del pintor, el sol) aparezcan una sola vez, que los colores de cada unión sean coherentes. Evaluar un ensamblaje es exactamente esto: combinar medidas de tamaño, de exactitud y de completitud, sabiendo que ninguna basta por sí sola.

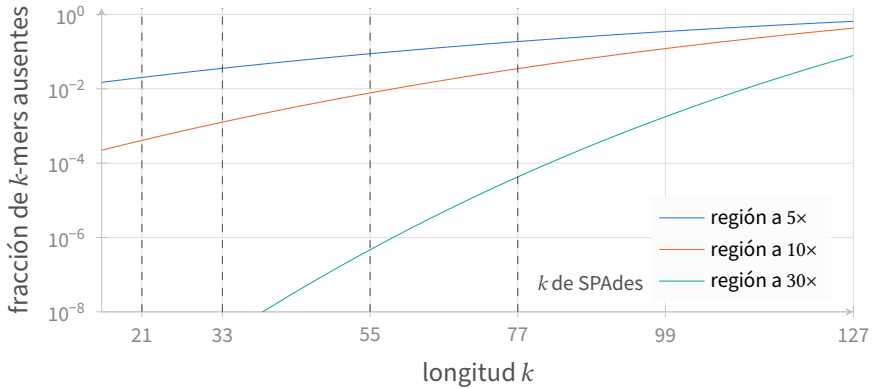


Figura 8.7. Fracción de k -mers del genoma que no aparecen en ninguna lectura (ecuación (8.11)) para lecturas de 150 pb con 0,5 % de error. En una región bien cubierta (30×), incluso $k = 77$ deja menos de un k -mer ausente cada 20 000; en una región a 5×, un k grande fragmenta el grafo sin remedio. Las líneas discontinuas marcan los valores de k que SPAdes usa por defecto con lecturas de 150 pb: los pequeños mantienen conectadas las regiones pobres y los grandes resuelven repeticiones en las ricas.

8.3.1 Un solo k no basta

La sección 8.2 dejó un dilema abierto. Un k grande resuelve más repeticiones, pero la ecuación (8.2) nos dice que reduce la cobertura efectiva, y con ella aumenta la probabilidad de que algún k -mer del genoma no aparezca en ninguna lectura. Cada k -mer ausente corta el camino del grafo y divide un *contig* en dos. Por el mismo argumento de Poisson del capítulo 6,

$$\mathbb{P}(\text{un } k\text{-mer del genoma no se observa}) = e^{-\lambda_k}, \quad \lambda_k = c \frac{L - k + 1}{L} (1 - e)^k. \quad (8.11)$$

DONDE

$e^{-\lambda_k}$	Fracción esperada de k -mers del genoma que faltan en el grafo.
λ_k	Cobertura efectiva de k -mers (ecuación (8.2)).

Con cobertura alta el efecto es invisible, pero la cobertura no es uniforme. Una región que por sesgo de GC (capítulo 6) recibe solo 10× con lecturas de 150 pb y 0,5 % de error pierde el 0,04 % de sus k -mers con $k = 21$, pero el 3,5 % con $k = 77$ y el 43 % con $k = 127$ (figura 8.7). En un genoma de 5 Mb con esa cobertura, son unos 2000, 175 000 y más de dos millones de k -mers ausentes, respectivamente.

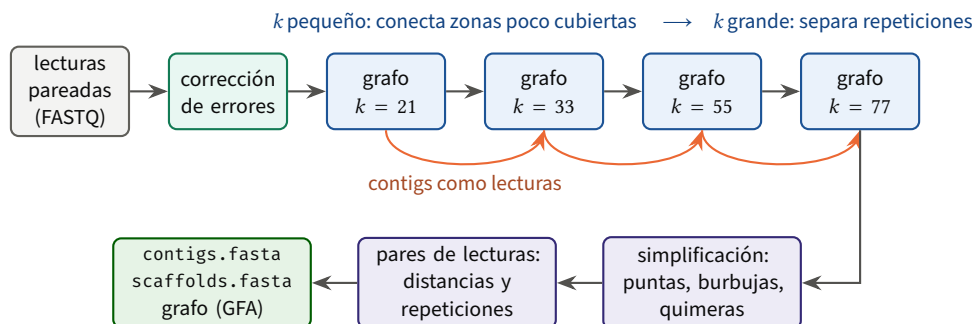


Figura 8.8. Esquema del ensamblaje multi-*k* de SPAdes. Tras corregir las lecturas, se construye el grafo de De Bruijn con un *k* pequeño; sus *contigs* se reinyectan como lecturas largas y sin errores en el grafo del siguiente *k* (flechas naranjas), de modo que las regiones pobremente cubiertas que el *k* grande perdería quedan representadas. El grafo final se simplifica y se recorre con la información de los pares. Además de las secuencias, SPAdes escribe el grafo de ensamblaje en formato GFA, que puede inspeccionarse con visores como Bandage.

8.3.2 SPAdes

Bankevich et al. (2012) diseñaron SPAdes para un caso extremo: el ensamblaje de genomas bacterianos amplificados a partir de *una sola célula*, donde la cobertura varía en órdenes de magnitud a lo largo del genoma y abundan los errores y las lecturas quiméricas. En ese escenario ningún *k* fijo funciona, y la respuesta de SPAdes fue construir el grafo con *varios* valores de *k* de forma iterativa (figura 8.8). Primero ensambla con un *k* pequeño, que conecta las regiones de baja cobertura; luego usa los *contigs* obtenidos como lecturas adicionales, largas y sin huecos, al construir el grafo con un *k* mayor, que separa repeticiones. El resultado combina lo mejor de ambos extremos. Después, SPAdes simplifica el grafo (puntas, burbujas, conexiones quiméricas) y usa las lecturas pareadas para estimar distancias entre aristas y atravesar repeticiones. Aunque nació para células individuales, sus autores mostraron que también mejoraba a Velvet y SOAPdenovo en datos convencionales, y hoy es uno de los ensambladores más usados para genomas bacterianos con lecturas cortas.

Ensamblar un aislado bacteriano con SPAdes

```
# lecturas pareadas ya recortadas (capítulo 6)
spades.py --isolate -1 cepa_R1.fq.gz -2 cepa_R2.fq.gz \
-k 21,33,55,77 -t 8 -m 32 -o spades_cep
```

```
ls spades_cepa/
# contigs.fasta scaffolds.fasta assembly_graph.gfa spades.log ...

# contigs muy cortos o poco cubiertos suelen ser basura: filtrar
seqkit seq -m 500 spades_cepa/contigs.fasta > contigs_500.fasta
```

⚠ Atención

El esquema de la figura 8.8 describe el flujo general, con la corrección de errores (BayesHammer) como primer paso. En las versiones recientes (SPAdes 4), el modo `--isolate`, pensado para aislados con cobertura alta y uniforme, *omite* por defecto esa corrección y el registro lo anuncia como `Mode: ONLY assembling (without read error correction)`; los *k*-mers erróneos se eliminan entonces al simplificar el grafo. Revise siempre `spades.log` para saber qué pasos se ejecutaron.

Los nombres de los *contigs* de SPAdes llevan la información más útil para una primera inspección, por ejemplo `>NODE_1_length_812345_cov_41.2`: longitud y cobertura de *k*-mers. Un *contig* con cobertura cercana al doble de la mediana es probablemente una repetición colapsada de dos copias; uno con cobertura diez veces mayor puede ser un plásmido multicopia; y uno con cobertura muy baja, un contaminante o un artefacto.

8.3.3 Lecturas largas: el regreso del solapamiento

Las lecturas largas cambiaron de nuevo el equilibrio. Con lecturas de decenas de kilobases hay pocas lecturas y muy largas, y calcular solapamientos vuelve a ser factible; además, una lectura larga atraviesa repeticiones que ningún *k* de lecturas cortas resolvería. La contrapartida es que las primeras lecturas largas tenían tasas de error del 10 al 15 %, incompatibles con *k*-mers exactos.

- **Canu** (Koren et al., 2017), sucesor del Celera Assembler, es un ensamblador OLC diseñado para lecturas ruidosas de PacBio y Nanopore. Detecta solapamientos con una estrategia adaptativa de *MinHash* ponderada con *tf-idf* (que da menos peso a los *k*-mers repetitivos), corrige las lecturas y construye un grafo disperso que evita colapsar repeticiones y haplotipos divergentes. Obtuvo *contigs* con NG50 superior a 21 Mb en datos de PacBio humanos y de *Drosophila melanogaster*.

- **Flye** (Kolmogorov et al., 2019) vuelve a la idea de los grafos de repeticiones: genera caminos arbitrarios (*disjointigs*) a través de un grafo de repeticiones desconocido y reconstruye a partir de ellos un grafo exacto. Frente a cinco ensambladores de referencia fue un orden de magnitud más rápido y casi duplicó la contigüidad (NGA50) del ensamblaje humano.
- **hifiasm** (Cheng et al., 2021) aprovecha las lecturas HiFi, largas y con exactitud superior al 99 %, para construir un *grafo de ensamblaje con fase*, que conserva la contigüidad de *todos* los haplotipos en lugar de colapsarlos en un consenso. Ensambló, entre otros, el genoma hexaploide de unos 30 Gb de la secuoya de California.

La culminación de esta línea fue el primer genoma humano completo de telómero a telómero. Nurk et al. (2022) combinaron lecturas HiFi y ultralargas de Nanopore de la línea celular CHM13, prácticamente homocigota, para producir una secuencia de 3055 millones de pares de bases sin huecos en todos los cromosomas salvo el Y. El ensamblaje T2T-CHM13 corrigió errores de las referencias previas y añadió casi 200 millones de pares de bases (el 8 % del genoma que faltaba), con 1956 predicciones génicas, 99 de ellas codificantes de proteínas: los centrómeros, las duplicaciones segmentarias recientes y los brazos cortos de los cinco cromosomas acrocéntricos, las regiones que ninguna lectura corta había podido atravesar.

8.3.4 Métricas de contigüidad: N50 y sus parientes

Un ensamblaje es una lista de secuencias de longitudes $\ell_1 \geq \ell_2 \geq \dots \geq \ell_n$ (ordenadas de mayor a menor), con longitud total $A = \sum_i \ell_i$. La media de las longitudes es una mala medida de calidad: la dominan los miles de *contigs* diminutos que casi todo ensamblaje produce. La comunidad adoptó en su lugar una medida ponderada por longitud.

Definición 8.4. (Nx , Lx y NGx)

Para $0 < x \leq 100$, el Nx es la longitud ℓ_j del *contig* en el que la suma acumulada alcanza el $x\%$ del total:

$$Nx = \ell_{j^*}, \quad j^* = \min \left\{ j : \sum_{i=1}^j \ell_i \geq \frac{x}{100} A \right\}, \quad Lx = j^*. \quad (8.12)$$

El NGx (y el LGx) se define igual, sustituyendo A por el tamaño del genoma G , conocido o estimado (ecuación (8.6)). Si el ensamblaje no alcanza el $x\%$ de G , el NGx no está definido.

DONDE

ℓ_i	Longitud del i -ésimo <i>contig</i> más largo.
A	Longitud total del ensamblaje.
j^*	Número mínimo de <i>contigs</i> que, tomados de mayor a menor, suman el $x\%$.
Lx	Número de <i>contigs</i> (el L_{50} es cuántos hacen falta para cubrir la mitad).

Una lectura equivalente e intuitiva: el N_{50} es la longitud tal que la mitad de las bases del ensamblaje están en *contigs* de esa longitud o mayores. Si se escoge una base del ensamblaje al azar, hay una probabilidad del 50 % de que caiga en un *contig* de al menos N_{50} bases. Esa interpretación sugiere una medida sin umbral arbitrario: la longitud esperada del *contig* que contiene una base elegida al azar,

$$auN = \sum_i \frac{\ell_i}{A} \ell_i = \frac{\sum_i \ell_i^2}{\sum_i \ell_i}, \quad (8.13)$$

que es además el área bajo la curva Nx (dividida por 100) cuando x recorre de 0 a 100 (figura 8.9).

DONDE

ℓ_i/A	Probabilidad de que una base elegida al azar pertenezca al <i>contig</i> i .
auN	Longitud media del <i>contig</i> “visto desde una base”; resume toda la curva Nx .

Ejemplo 8.2. (Calcular N50, NG50 y auN a mano)

Un ensamblaje tiene diez *contigs* de 320, 250, 180, 120, 90, 60, 40, 25, 10 y 5 kb; total $A = 1100$ kb. El genoma estimado con el espectro de k -mers mide $G = 1300$ kb.

- N_{50} : la mitad de A es 550 kb. Acumulando, 320 y luego $320 + 250 = 570 \geq 550$. Por tanto $N_{50} = 250$ kb y $L_{50} = 2$.
- NG_{50} : la mitad de G es 650 kb. Hace falta llegar a $320 + 250 + 180 = 750$: $NG_{50} = 180$ kb y $LG_{50} = 3$. El NG_{50} es menor porque penaliza los 200 kb del genoma que el ensamblaje no contiene.
- N_{90} : el 90 % de A son 990 kb, que se alcanzan en el sexto *contig*: $N_{90} = 60$ kb.
- auN : $\sum \ell_i^2 = 320^2 + 250^2 + \dots + 5^2 = 225\,750$, y $225\,750/1100 = 205,2$ kb.

Ahora supongamos que el ensamblador une por error los dos *contigs* mayores a través de una repetición. El nuevo *contig* de 570 kb supera por sí solo la mitad del total: el N_{50} salta a 570 kb y el L_{50} baja a 1. *Un error de ensamblaje ha más que duplicado el N_{50} .*

n50.py

```
def nx(longitudes, x=50, G=None):
    """Devuelve (Nx, Lx); con G, (NGx, LGx)."""
    ls = sorted(longitudes, reverse=True)
    meta = x / 100 * (G if G else sum(ls))
    acum = 0
    for i, l in enumerate(ls, start=1):
        acum += l
        if acum >= meta:
            return l, i
    return 0, None          # el ensamblaje no alcanza x% de G

def auN(longitudes):
    return sum(l * l for l in longitudes) / sum(longitudes)

contigs = [320, 250, 180, 120, 90, 60, 40, 25, 10, 5]    # kb
print(nx(contigs), nx(contigs, G=1300), round(auN(contigs), 1))
# (250, 2) (180, 3) 205.2
```

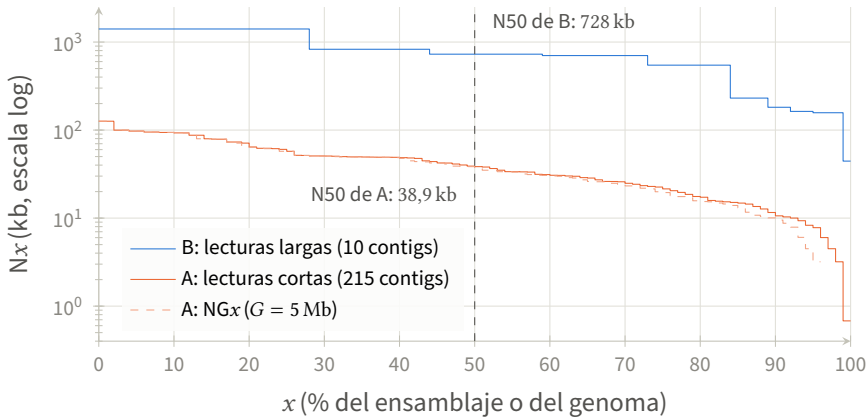


Figura 8.9. Curvas N_x de dos ensamblajes simulados del mismo genoma bacteriano de 5 Mb. Cada curva escalonada indica, para cada x , la longitud del *contig* en el que la suma acumulada alcanza el $x\%$; la ordenada en $x = 50$ es el N50. El ensamblaje de lecturas largas (azul) tiene un N50 casi veinte veces mayor que el de lecturas cortas (naranja). La curva NG_x discontinua cae a cero antes de $x = 100$ porque el ensamblaje A solo contiene el 97 % del genoma. Comparar curvas completas es más informativo que comparar un único punto. Generado con `figuras/cap08/generar.py`.

⚠ El N50 mide tamaño, no verdad

El ejemplo anterior muestra el defecto fundamental de todas las métricas de contigüidad: premian las uniones, correctas o no. Un ensamblador agresivo que atraviesa repeticiones sin evidencia obtiene mejores N50 que uno prudente. Además, el N50 depende del umbral de longitud mínima: eliminar los *contigs* de menos de 500 pb reduce A y puede *aumentar* el N50 sin cambiar nada del ensamblaje. Nunca compare N50 sin comparar también la exactitud, y prefiera el NG50, que usa un denominador fijo.

8.3.5 Andamiaje

Los *contigs* terminan donde el grafo es ambiguo, pero a menudo sabemos qué *contig* sigue a cuál aunque no conozcamos la secuencia intermedia. El *andamiaje* (*scaffolding*) ordena y orienta *contigs* usando información de largo alcance: pares de lecturas con insertos largos, lecturas largas, mapas ópticos o contactos de cromatina (Hi-C). El resultado, un *scaffold*, es una secuencia con tramos de N que representan huecos de tamaño estimado. Si una lectura de un par cae a una distancia a del final

del *contig* C_1 y su compañera a una distancia b del inicio de C_2 , y el tamaño medio del inserto es μ , el hueco estimado es

$$\hat{g} = \mu - a - b, \quad (8.14)$$

y promediando muchos pares se reduce la incertidumbre. Un $\hat{g}$ negativo indica que los *contigs* se solapan (a menudo por una repetición colapsada en sus extremos).

DONDE

μ	Tamaño medio del inserto de la biblioteca (distancia entre los extremos externos del par).
a, b	Distancias de cada lectura del par al extremo correspondiente de su <i>contig</i> .
$\hat{g}$	Longitud estimada del hueco que se rellenará con N.

8.3.6 Exactitud: QUAST

Cuando existe un genoma de referencia cercano, podemos medir la exactitud directamente alineando los *contigs* contra él. QUAST (Gurevich et al., 2013) se ha convertido en el estándar para ello. Con referencia, además de las métricas de contigüidad, calcula la fracción del genoma cubierta, la razón de duplicación, los desajustes e *indels* por cada 100 kb y, sobre todo, los *errores de ensamblaje* (*misassemblies*): puntos de un *contig* en los que el tramo izquierdo y el derecho se alinean en la referencia a más de 1 kb de distancia, en hebras opuestas o en cromosomas distintos (figura 8.10). Con esos puntos corta los *contigs* y recalcula la contigüidad: el NGA_{50} es el NG_{50} de los bloques alineados correctamente, una medida que ya no premia las uniones falsas. Sin referencia, QUAST sigue siendo útil para comparar varios ensamblajes de los mismos datos con las métricas de la sección anterior.

Evaluación con QUAST, BUSCO y Merqury

```
# contigüidad y exactitud frente a una referencia cercana
quast.py contigs_500.fasta otro_ensamblaje.fasta \
  -r referencia.fasta -g genes.gff -t 8 -o quast_cepa

# completitud génica con ortólogos de copia única
busco -i contigs_500.fasta -m genome -l bacteria_odb10 \
  -c 8 -o busco_cepa

# exactitud sin referencia a partir de los k-mers de las lecturas
```

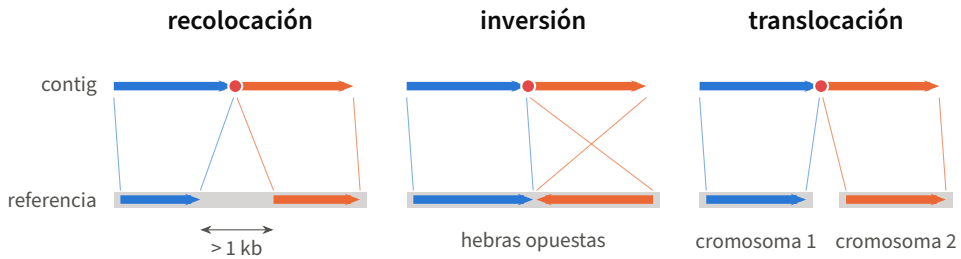



Figura 8.10. Los tres tipos de errores de ensamblaje que QUAST detecta al alinear un *contig* (arriba) contra la referencia (abajo, gris). El punto rojo es el punto de ruptura. **Recolocación:** los dos tramos se alinean en la misma hebra pero separados (o solapados) por más de 1 kb. **Inversión:** el tramo derecho se alinea en la hebra opuesta. **Translocación:** los tramos proceden de cromosomas (o replicones) distintos. Casi siempre el origen es una repetición atravesada por el camino equivocado del grafo (figura 8.6), aunque algunos “errores” son variación estructural real entre la muestra y la referencia.

```
meryl k=21 count output lect.meryl cepa_R1.fq.gz cepa_R2.fq.gz
mercury.sh lect.meryl contigs_500.fasta mercury_cepa
```

8.3.7 Completitud y exactitud sin referencia

Para la mayoría de los organismos no existe una referencia. Dos familias de métodos cubren ese hueco.

Genes que no pueden faltar. BUSCO (Simão et al., 2015) se basa en una observación evolutiva: en cada linaje hay genes que están presentes en casi todas las especies y casi siempre en una sola copia (*benchmarking universal single-copy orthologs*). Si buscamos esos genes en un ensamblaje, cada uno debería aparecer exactamente una vez. BUSCO clasifica cada ortólogo como completo y de copia única (S), completo y duplicado (D), fragmentado (F) o ausente (M). Un buen ensamblaje bacteriano suele superar el 95 % de completos; muchos fragmentados sugieren errores de consenso o *contigs* cortos; muchos duplicados, que los dos haplotipos de un diploide quedaron separados sin querer o que hay contaminación con un organismo emparentado. La versión 5 amplió los conjuntos de datos (sincronizados con OrthoDB v10), incorporó la selección automática del linaje por ubicación filogenética, lo que permite evaluar genomas de metagenomas de origen desconocido, y admite eucariotas, procariotas y virus (Manni et al., 2021).

***k*-mers como testigos.** Merqury (Rhie et al., 2020) compara los *k*-mers del ensamblaje con los de las lecturas. Supongamos que el ensamblaje tiene K_{asm} *k*-mers y que K_{solo} de ellos no aparecen en ninguna lectura (errores de consenso). Si los errores son independientes y afectan a una base con probabilidad ε , un *k*-mer es correcto con probabilidad $(1 - \varepsilon)^k$, así que

$$(1 - \varepsilon)^k \approx 1 - \frac{K_{solo}}{K_{asm}} \implies \hat{\varepsilon} = 1 - \left(1 - \frac{K_{solo}}{K_{asm}}\right)^{1/k}, \quad QV = -10 \log_{10} \hat{\varepsilon}. \quad (8.15)$$

DONDE	
K_{asm}	Número de <i>k</i> -mers del ensamblaje.
K_{solo}	<i>k</i> -mers del ensamblaje ausentes de las lecturas.
$\hat{\varepsilon}$	Tasa de error por base estimada del ensamblaje.
QV	Calidad de consenso en escala Phred (la misma escala del capítulo 6).

Con $k = 21$, un ensamblaje bacteriano de 5 millones de *k*-mers con 250 *k*-mers exclusivos tiene $\hat{\varepsilon} = 2,4 \times 10^{-6}$ y $QV = 56$: un error cada 420 000 bases. Con 5000 *k*-mers exclusivos, $QV = 43$. Merqury también mide la completitud como la fracción de *k*-mers “fiabes” de las lecturas (los que superan el valle del espectro) que aparecen en el ensamblaje, y dibuja espectros coloreados según cuántas veces aparece cada *k*-mer en el ensamblaje, lo que revela de un vistazo si se perdió un haplotipo o se duplicó.

Un ensamblaje se juzga en tres ejes: contigüidad (N50, NG50, auN), exactitud (errores de QUAST, NGA50, QV de Merqury) y completitud (fracción del genoma, BUSCO, *k*-mers). Mejorar uno a costa de otro es fácil; mejorar los tres a la vez es lo que distingue un buen ensamblaje.

8.4 Anotación genómica

Encontrar palabras en un texto sin espacios

Lea esta línea:

ELQUEMUCHOABARCAPOCOAPRIETAYXQWZKTRBLAMPENCIAHJXV

Sin que nadie le haya dicho dónde termina cada palabra, usted separa el refrán al principio y reconoce que el final es ruido. Lo consigue porque conoce el español: sabe qué combinaciones de letras son frecuentes (QUE, CHO), cuáles son casi imposibles (XQWZ) y qué forma tiene una palabra. Un programa de predicción génica hace lo mismo con el ADN. No sabe biología; sabe estadística: aprende cómo “suena” el ADN codificante de un organismo y busca en el genoma los tramos que suenan así, respetando una gramática (un gen empieza con un codón de inicio, no puede contener codones de parada en su marco y, en eucariotas, sus intrones empiezan y terminan con señales características).

8.4.1 Qué significa anotar

Anotar un genoma tiene dos niveles. La *anotación estructural* asigna coordenadas: dónde están los genes codificantes, sus exones y sus codones de inicio y parada; los genes de ARN ribosómico y de transferencia; las repeticiones y los elementos móviles. La *anotación funcional* asigna significado: nombre del producto génico, función enzimática, dominios, términos de ontología. El resultado se guarda en formatos como GFF3 o GenBank (capítulo 2). La figura 8.11 resume el flujo de trabajo, que difiere sobre todo en la complejidad de la predicción de genes: en procariotas los genes son casi siempre marcos de lectura continuos y densos (alrededor del 88 % del genoma de *Escherichia coli* codifica proteínas); en eucariotas están fragmentados en exones separados por intrones y dispersos en un mar de ADN no codificante y repetitivo.

8.4.2 Genes procariotas: marcos abiertos de lectura

Definición 8.5. (Marco abierto de lectura (ORF))

Un *marco abierto de lectura* (*open reading frame*, ORF) es un tramo de ADN que, leído en uno de los seis marcos posibles (tres por hebra), comienza en un codón de inicio (típicamente ATG, y en bacterias también GTG o TTG) y continúa sin codones de parada (TAA, TAG, TGA) hasta el primero que aparece en ese marco.

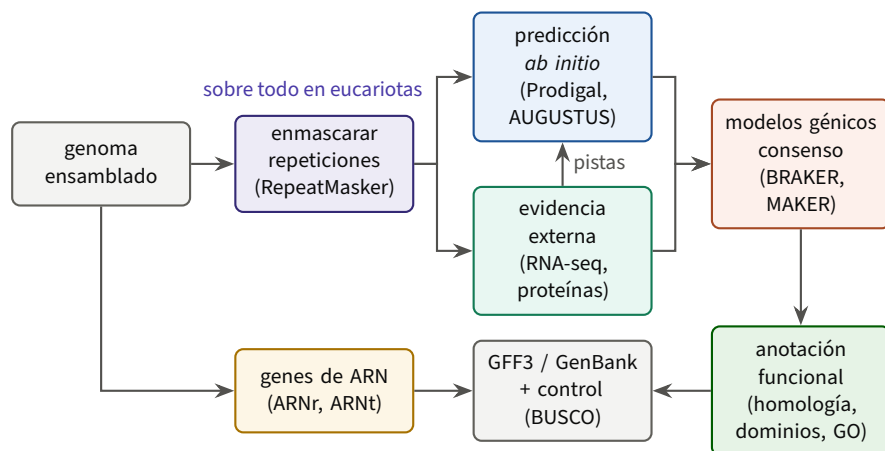


Figura 8.11. Flujo típico de anotación. Las repeticiones se enmascaran primero para que sus marcos de lectura (transposasas, transcriptasas inversas) no se confundan con genes del huésped. La predicción *ab initio* usa solo la secuencia y un modelo estadístico entrenado para la especie; la evidencia externa (transcritos de RNA-seq alineados, proteínas de especies cercanas) se incorpora como “pistas” que el predictor debe respetar o como modelos independientes que después se combinan. Los genes de ARN estructurales se buscan con modelos propios. Por último, a cada gen se le asigna una función por homología y el conjunto se valida, por ejemplo, con BUSCO en modo proteína. En procariotas, herramientas como Prokka y Bakta automatizan todo el flujo.

Todo gen codificante bacteriano es un ORF, pero no todo ORF es un gen: en cualquier secuencia aparecen ORFs por azar. ¿Cuán largos? Si los codones fueran independientes, cada uno sería de parada con probabilidad p , y la longitud de un ORF aleatorio (en codones) sería geométrica:

$$P(\text{ORF aleatorio} \geq n \text{ codones}) = (1 - p)^n, \quad p = \sum_{xyz \in \{\text{TAA, TAG, TGA}\}} f_x f_y f_z. \quad (8.16)$$

DONDE

p	Probabilidad de que un codón al azar sea de parada, dada la composición de bases.
f_x	Frecuencia de la base x en el genoma.
n	Longitud del ORF en codones.

En el genoma de *E. coli* K-12 (50,8 % de GC), $p = 0,0456$, aproximadamente un codón de parada cada 22. La probabilidad de que un ORF aleatorio supere los 100

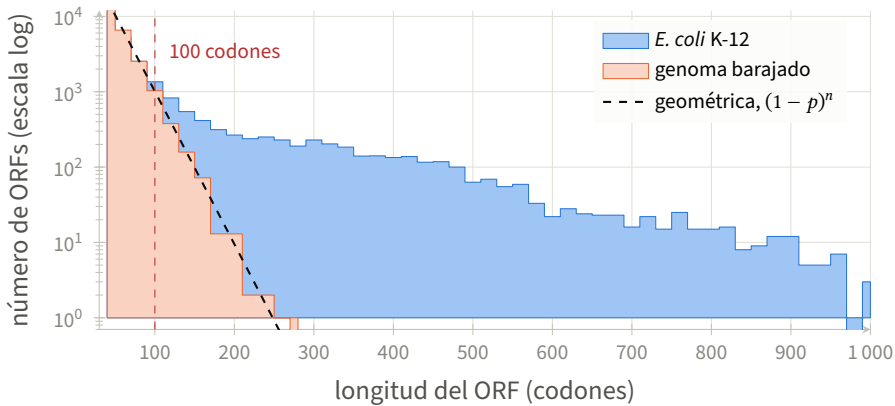


Figura 8.12. Distribución de longitudes de los ORFs (de ATG a codón de parada, seis marcos, ≥ 30 codones) en el genoma de *E. coli* K-12 MG1655 (azul) y en el mismo genoma barajado base a base (naranja). El genoma barajado sigue la ley geométrica de la ecuación (8.16) (línea discontinua): su ORF más largo tiene unos 250 codones (251 con el barajado usado aquí; la cifra exacta depende de la semilla). El genoma real muestra una cola de miles de ORFs largos, los genes. Por debajo de unos 100 codones las dos distribuciones se confunden: allí la longitud ya no basta y hace falta el contenido de la secuencia. Datos: NC_000913.3; generado con `figuras/cap08/generar.py`.

codones es $(1 - p)^{100} = 0,0094$, y la de que supere los 300, 8×10^{-7} . La figura 8.12 compara los ORFs reales del genoma de *E. coli* con los de una versión barajada con la misma composición: el genoma barajado no produce *ningún* ORF de 300 codones o más, mientras que el real tiene 2030. Los ORFs largos son genes casi con certeza.

8.4.3 El sabor del ADN codificante: modelos de Márkov

Para los ORFs cortos, la longitud no basta: hay que mirar su composición. El ADN codificante tiene una periodicidad de tres bases que el no codificante no tiene, porque cada posición del codón está sometida a restricciones distintas (la tercera es la más libre y refleja el sesgo de uso de codones de cada especie). La forma más simple de explotarlo es un modelo de codones independientes.

Definición 8.6. (Puntuación log-odds de codones)

Sea $P_{\text{cod}}(c)$ la frecuencia del codón c en genes conocidos y $P_{\text{nc}}(c) = f_{c_1}f_{c_2}f_{c_3}$ su probabilidad bajo un modelo nulo de bases independientes. La puntuación de un ORF $x = c_1c_2 \cdots c_n$ es

$$S(x) = \frac{1}{n} \sum_{i=1}^n \log_2 \frac{P_{\text{cod}}(c_i)}{P_{\text{nc}}(c_i)} \quad (\text{bits por codón}). \quad (8.17)$$

DONDE

$P_{\text{cod}}(c)$	Probabilidad del codón c en regiones codificantes de la especie.
$P_{\text{nc}}(c)$	Probabilidad del mismo trinucleótido bajo el modelo nulo.
$S(x)$	Evidencia media, en bits por codón, de que x es codificante.

Es la misma lógica de razón de verosimilitudes de las matrices de sustitución (capítulo 3). ¿De dónde salen las frecuencias P_{cod} de un genoma recién ensamblado, sin genes conocidos? De la figura 8.12: los ORFs de más de 300 codones son genes casi con certeza, así que podemos *autoentrenar* el modelo con ellos. En *E. coli*, los 2030 ORFs largos aportan casi un millón de codones. Los codones más favorecidos resultan ser CTG (+1,76 bits, el codón de leucina preferido por la bacteria), GAA (+1,32) y AAA (+1,09); los más desfavorecidos, los codones de arginina raros AGG (−3,79) y AGA (−2,83).

Ejemplo 8.3. (Puntuar un fragmento)

Con el modelo autoentrenado en *E. coli*, el fragmento ATG AAA CGC ATT AGC ACC ACC ATT recibe

$$S = \frac{1}{8} (0,83+1,09+0,48+0,98+0,03+0,65+0,65+0,98) = \frac{5,69}{8} = 0,71 \text{ bits/codón},$$

un valor claramente codificante. Un fragmento del mismo tamaño que contuviera AGG, AGA y CGA acumularía casi −9 bits solo por esos tres codones. Aplicado a los ORFs de 60 a 149 codones, la región de la figura 8.12 donde las dos distribuciones se confunden, el 27,8 % de los ORFs reales supera 0,1 bits/codón, frente al 0,22 % de los ORFs del genoma barajado (figura 8.13).

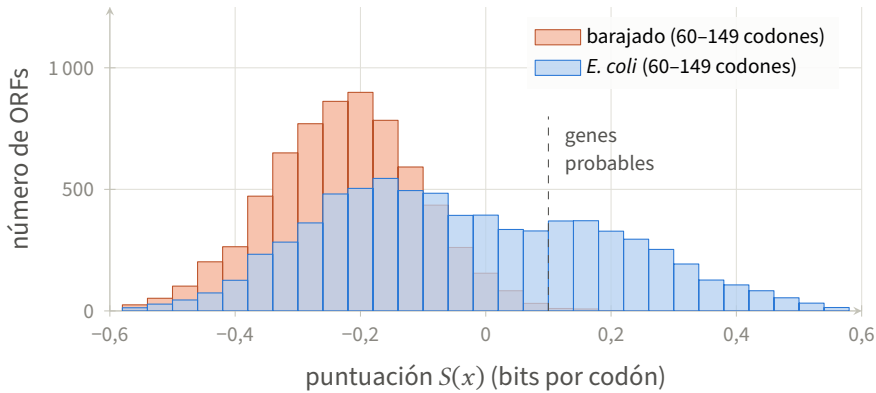


Figura 8.13. Distribución de la puntuación de la ecuación (8.17) para los ORFs de 60 a 149 codones de *E. coli* (azul) y del genoma barajado (naranja), con un modelo autoentrenado con los ORFs de más de 300 codones. Los ORFs barajados forman una única campana negativa; los reales son una mezcla de una campana parecida (ORFs espurios, a menudo en otro marco de un gen verdadero) y una cola positiva de genes cortos. Incluso este modelo de orden cero separa buena parte de los genes; los predictores reales usan cadenas de Márkov de orden superior dependientes de la fase del codón.

El modelo de codones independientes ignora el contexto. Los predictores reales usan *cadenas de Márkov de orden m dependientes de la fase*: la probabilidad de cada base depende de las m anteriores y de la posición que ocupa dentro del codón,

$$\log P_{\text{cod}}(x) = \sum_i \log P(x_i \mid x_{i-m} \cdots x_{i-1}, \phi(i)), \quad \phi(i) \in \{1, 2, 3\}. \quad (8.18)$$

DONDE

m	Orden de la cadena: cuántas bases previas se usan como contexto (típicamente 5).
$\phi(i)$	Posición de la base i dentro del codón (fase).

Con $m = 5$ hay $3 \times 4^5 \times 3 = 9216$ parámetros independientes, y los contextos raros se estiman mal. Glimmer resolvió ese problema con *modelos de Márkov interpolados*, que combinan órdenes de 0 a 8 con pesos que dependen de cuántas veces se observó cada contexto; su versión 3 redujo drásticamente los falsos positivos manteniendo una sensibilidad del 99 % y mejoró la identificación de los codones de inicio (Delcher et al., 2007). GeneMark.hmm (Lukashin & Borodovsky, 1998) insertó los modelos de Márkov de GeneMark en un modelo oculto de Márkov en el que los límites

de los genes son transiciones entre estados ocultos, y usó un patrón del sitio de unión al ribosoma para precisar los codones de inicio; sus autores observaron alta exactitud incluso con cadenas de orden cero, uno y dos. Prodigal (Hyatt et al., 2010), desarrollado a partir de la experiencia de curación manual de genomas en el Joint Genome Institute, usa programación dinámica sobre todos los pares inicio–parada posibles, con tres objetivos explícitos: mejor estructura génica, mejor sitio de inicio de la traducción y menos falsos positivos. Todos son autoentrenables: aprenden los parámetros del propio genoma que anotan.

8.4.4 Genes eucariotas: modelos ocultos de Márkov generalizados

En eucariotas, un gen es una frase con gramática: región intergénica, codón de inicio, exón, sitio donador (GT), intrón, sitio aceptor (AG), exón... y codón de parada. Un modelo oculto de Márkov (HMM) es la herramienta natural para describir esa gramática: cada *estado* oculto corresponde a un tipo de región, cada estado *emite* bases con su propio modelo (una cadena de Márkov de fase para los exones, otra para los intrones), y las *transiciones* entre estados codifican el orden permitido (figura 8.14). Como un intrón puede interrumpir un codón en cualquiera de sus tres posiciones, hay que distinguir tres estados de intrón según la fase, para que el exón siguiente continúe el marco correctamente.

La anotación de una secuencia $x_1 \dots x_T$ consiste en encontrar la sucesión de estados π^* más probable dada la secuencia, lo que se resuelve con el algoritmo de Viterbi. Llamando $V_j(t)$ a la probabilidad (en logaritmos) del mejor camino que termina en el estado j tras emitir $x_1 \dots x_t$,

$$V_j(t) = \log e_j(x_t) + \max_i [V_i(t-1) + \log a_{ij}], \quad \pi^* = \arg \max_{\pi} \mathbb{P}(x, \pi). \quad (8.19)$$

DONDE

a_{ij}	Probabilidad de transición del estado i al j (p. ej., de exón a sitio donador).
$e_j(x_t)$	Probabilidad de que el estado j emita la base x_t (dado su contexto).
$V_j(t)$	Log-probabilidad del mejor camino de estados que explica $x_1 \dots x_t$ y acaba en j .
π^*	Anotación más probable: la etiqueta (intergénica, exón, intrón...) de cada base.

Es la misma programación dinámica de Needleman-Wunsch (capítulo 3), ahora sobre estados en lugar de columnas de alineamiento, con coste $O(T|Q|^2)$ para $|Q|$

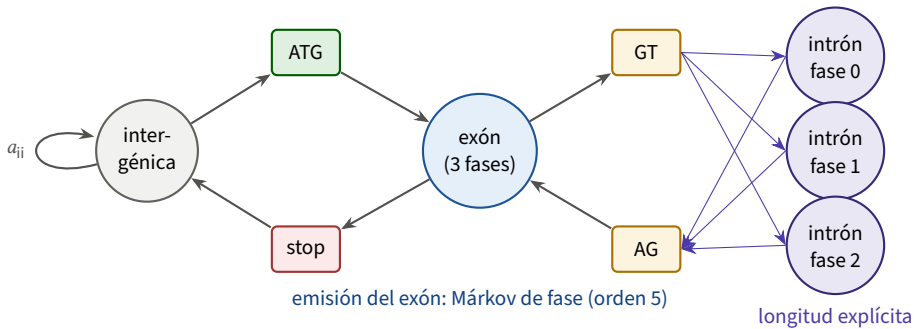


Figura 8.14. Esquema simplificado (una sola hebra) de un HMM generalizado para la estructura de un gen eucariota, en el espíritu de AUGUSTUS. Los estados circulares emiten tramos de secuencia con modelos de contenido propios (el exón, con cadenas de Márkov de fase); los rectangulares, señales cortas (inicio, parada, sitios de empalme donador GT y aceptor AG). Los tres estados de intrón recuerdan en qué posición del codón se interrumpió el exón. En un HMM “generalizado” cada estado de contenido emite un tramo entero cuya longitud sigue una distribución arbitraria, en lugar de la geométrica implícita en un bucle sobre sí mismo. Un modelo completo duplica todo en la hebra opuesta.

estados. Un HMM simple impone, sin embargo, longitudes geométricas a cada región (la probabilidad de seguir en un estado es constante), y las longitudes reales de exones e intrones no son geométricas. Los HMM *generalizados* permiten que cada estado emita un segmento entero con una distribución de longitud arbitraria, a cambio de un coste computacional mayor. AUGUSTUS (Stanke & Waack, 2003) introdujo una nueva forma de modelar la longitud de los intrones (explícita para los cortos, con una cola geométrica para los largos), un nuevo modelo del sitio donador y una estimación de parámetros dependiente del contenido de GC. Sus autores partían de un diagnóstico preciso: los predictores de entonces funcionaban bien en secuencias cortas con un solo gen, pero en secuencias largas con un número desconocido de genes predecían muchos exones falsos. En ese escenario, AUGUSTUS predijo correctamente muchos más genes humanos y de *Drosophila* que los programas con los que se comparó, y a la vez fue más específico. Hoy es uno de los predictores eucariotas más usados.

8.4.5 Enmascarar repeticiones

Antes de predecir genes en un eucariota hay que ocuparse de las repeticiones. Casi la mitad del genoma humano, y porcentajes aún mayores en muchas plantas,

está formada por elementos transponibles y sus restos. Muchos de ellos contienen ORFs genuinos (transposasas, transcriptasas inversas) que un predictor *ab initio* anotaría como genes del huésped, inflando el número de genes con miles de falsos positivos. RepeatMasker (Tarailo-Graovac & Chen, 2009) busca en el genoma las secuencias de una biblioteca de repeticiones conocidas (Dfam, Repbase) o construida *de novo* para la especie, y las marca. El marcado puede ser *duro* (sustituir por N) o *blando* (*soft-masking*: pasar a minúsculas); el blando es preferible, porque permite que el predictor evite iniciar genes en esas regiones sin impedir que un exón verdadero se extienda sobre un fragmento repetitivo.

8.4.6 Evidencia: RNA-seq y proteínas

Los predictores *ab initio* aciertan bien la estructura de genes típicos, pero fallan en los extremos (exones terminales, regiones no traducidas), en los genes atípicos y en el empalme alternativo. La evidencia experimental corrige esas debilidades. Las lecturas de RNA-seq alineadas con un mapeador que admite empalmes indican directamente dónde están los intrones: cada lectura dividida entre dos exones es una “pista” de intrón. Las proteínas de especies emparentadas, alineadas contra el genoma, marcan exones codificantes conservados. Yandell y Ence (2012) revisan cómo los flujos de anotación combinan estas fuentes y cómo medir la concordancia entre un modelo génico y la evidencia que lo apoya.

BRAKER automatiza la combinación: usa la evidencia para entrenar GeneMark sin supervisión, genera con él un conjunto de genes de entrenamiento para AUGUSTUS y predice los genes finales con AUGUSTUS respetando las pistas. La versión BRAKER2 (Brůna et al., 2021) permite hacerlo con una base de datos de proteínas de cualquier distancia evolutiva, incluso sin RNA-seq, mediante GeneMark-EP+.

Anotación eucariota con enmascarado y BRAKER

```
# 1) biblioteca de repeticiones de la especie y enmascarado blando
BuildDatabase -name especie genoma.fasta
RepeatModeler -database especie -threads 16
RepeatMasker -lib especie-families.fa -xsmall -pa 16 genoma.fasta

# 2) predicción con evidencia de proteínas y RNA-seq alineado
braker.pl --genome=genoma.fasta.masked --softmasking \
  --prot_seq=orthodb_proteinas.fa --bam=rnaseq.bam \
  --threads 16 --workingdir=braker_out

# 3) completitud del conjunto de proteínas predicho
```

```
busco -i braker_out/braker.aa -m proteins \
-l eukaryota_odb10 -o busco_prot
```

8.4.7 Anotación procariota automatizada: Prokka y Bakta

En bacterias y arqueas el flujo completo puede automatizarse. Prokka (Seemann, 2014) encadena herramientas especializadas: Prodigal para las regiones codificantes y otros programas para los genes de ARNr, ARNt, ARN no codificantes y péptidos señal; asigna funciones buscando las proteínas predichas en bases de datos jerárquicas, de las más fiables (proteínas curadas) a las más generales (familias de dominios). Anota un genoma bacteriano en borrador en unos diez minutos en un ordenador de escritorio y produce archivos que cumplen los estándares de los repositorios públicos.

Bakta (Schwengers et al., 2021) persigue los mismos objetivos con otras prioridades: ser independiente del taxón, detectar proteínas pequeñas que los predictores suelen pasar por alto y asignar referencias cruzadas precisas a bases de datos públicas. Para ello identifica muchas proteínas *sin alineamiento*, reconociendo secuencias idénticas a las ya catalogadas, lo que acelera el proceso. Exporta GFF3, archivos planos compatibles con INSDC y JSON, y en sus comparaciones superó a otras herramientas rápidas en anotaciones funcionales y referencias cruzadas con tiempos de ejecución comparables.

Anotación de un genoma bacteriano

```
prokka --outdir prokka_cepa --prefix cepa --kingdom Bacteria \
--cpus 8 contigs_500.fasta

bakta --db bakta_db/ --output bakta_cepa --prefix cepa \
--threads 8 contigs_500.fasta

awk '$3 == "CDS"' bakta_cepa/cepa.gff3 | wc -l # nº de CDS
```

⚠ Anotaciones que se copian a sí mismas

La anotación funcional por homología es transitiva: si una proteína se anotó mal en una base de datos, todas las que se anoten por parecido con ella heredarán el error, y la etiqueta equivocada se multiplica con cada genoma nuevo. Desconfíe de funciones muy específicas asignadas con identidades

bajas, prefiera bases de datos curadas y recuerde que “proteína hipotética” es una respuesta honesta. Tampoco compare el número de genes entre genomas anotados con flujos distintos: parte de la diferencia será metodológica, no biológica.

Los genes se encuentran porque “suenan” distinto: ORFs largos que el azar no produce y composición de codones característica, capturada por cadenas de Márkov de fase dentro de un HMM. En eucariotas, enmascarar repeticiones y añadir evidencia de RNA-seq y proteínas es tan importante como el modelo.

☰ En resumen

- Una lectura de longitud L contiene $L-k+1$ k -mers; se cuentan en forma canónica ($\min\{x, \bar{x}\}$) con k impar. La cobertura efectiva de k -mers es $\lambda_k = c \frac{L-k+1}{L} (1-e)^k$.
- El espectro de k -mers $h(m)$ muestra un pico de errores en m pequeños, un pico heterocigoto en λ y uno homocigoto en 2λ . El tamaño del genoma es $\hat{G} = \sum_{m \geq v} m h(m) / 2\lambda$, y la heterocigosidad se obtiene de $\alpha = 1 - (1-h)^k$ (GenomeScope).
- El ensamblaje OLC busca un camino hamiltoniano en el grafo de solapamiento; el de De Bruijn, un camino euleriano en un grafo cuyos nodos son $(k-1)$ -mers y cuyas aristas son k -mers, resoluble en tiempo lineal. Las repeticiones de longitud $\geq k-1$ hacen que el camino no sea único; los ensambladores entregan los unitigs.
- Los errores crean puntas y burbujas (unos NLe^k k -mers espurios), la heterocigosidad crea burbujas equilibradas y las repeticiones, marañas. Solo la información que atraviesa una repetición (pares, lecturas largas) la resuelve.
- SPAdes combina varios k para conectar regiones poco cubiertas y resolver repeticiones; Canu, Flye y hifiasm ensamblan lecturas largas y llevaron al primer genoma humano completo (T2T-CHM13).
- Un ensamblaje se evalúa en contigüidad (N50, NG50, auN), exactitud (QUAST, NGA50, QV de Merquy) y completitud (BUSCO, k -mers). El N50 premia las uniones erróneas.

- La anotación enmascara repeticiones, predice genes con modelos de Márkov y HMM autoentrenados (Glimmer, GeneMark, Prodigal, AUGUSTUS), integra evidencia (BRAKER) y asigna funciones por homología; Prokka y Bakta automatizan el proceso en procariotas.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 8.1** ●●● Escriba todos los 3-mers canónicos de **ACGTTGCA** y su espectro. ¿Hay algún 3-mer que sea su propio reverso complementario? ¿Y algún 4-mer? Relacione la respuesta con la paridad de k .
- 8.2** ●●● Un organismo haploide se secuenció con lecturas de 150 pb. El espectro de 25-mers tiene su pico en $m = 42$, el valle en $m = 6$ y $\sum_{m \geq 6} m h(m) = 1,68 \times 10^9$. Estime el tamaño del genoma y la cobertura por bases c (suponga errores despreciables).
- 8.3** ●●● Construya a mano el grafo de De Bruijn con $k = 3$ de **TTACGTACGC**, compruebe la condición de la ecuación (8.9) e identifique los unitigs. ¿Es único el camino euleriano?
- 8.4** ●●● Con la ecuación (8.4), calcule qué fracción de 21-mers y de 31-mers es heterocigota en un genoma con $h = 0,1\%$ y con $h = 2\%$. ¿En cuál de los cuatro casos el pico heterocigoto supera en altura al homocigoto? Use el resultado para explicar la advertencia de la sección 8.1 sobre confundir picos.
- 8.5** ●●● Modifique `camino_euleriano` para que devuelva *todos* los caminos eulerianos de un grafo pequeño. Aplíquelo al ejemplo de la figura 8.5 con $k = 4$ y $k = 5$ y a un genoma de 30 pb con una repetición de 6 pb: ¿a partir de qué k se vuelve único?
- 8.6** ●●● Descargue lecturas Illumina de un aislado de *E. coli*, ensámblelas con SPAdes usando solo $k = 21$, solo $k = 77$ y la combinación 21, 33, 55, 77. Compare los tres ensamblajes con QUAST frente a K-12 MG1655 (N50, NGA50, errores de ensamblaje, fracción del genoma) y explique las diferencias con la figura 8.7.

- 8.7    Implemente el cálculo de la curva Nx completa y del auN, y demuestre que el auN es igual al área bajo la curva Nx dividida por 100.
- 8.8    Demuestre la ecuación (8.6) incluyendo repeticiones: si una fracción ρ del genoma está en dos copias idénticas, ¿cómo cambia el espectro y por qué el estimador $\hat{G}$ sigue siendo insesgado? ¿Qué ocurre si las dos copias difieren en un 2 %?
- 8.9    Entrene un modelo de Márkov de fase de orden 2 (ecuación (8.18)) con los ORFs de más de 300 codones de *E. coli* y compárelo con el modelo de codones de la ecuación (8.17) como clasificador de los ORFs de 60 a 149 codones, usando como verdad la anotación de RefSeq. Dibuje las curvas de precisión-sensibilidad de ambos.
- 8.10    Programe el algoritmo de Viterbi para un HMM de dos estados (codificante con Márkov de fase de orden 0, no codificante) y úselo para segmentar un fragmento de 20 kb del genoma de *E. coli*. ¿Qué limitación del HMM simple aparece al comparar las longitudes predichas con las de los genes reales?

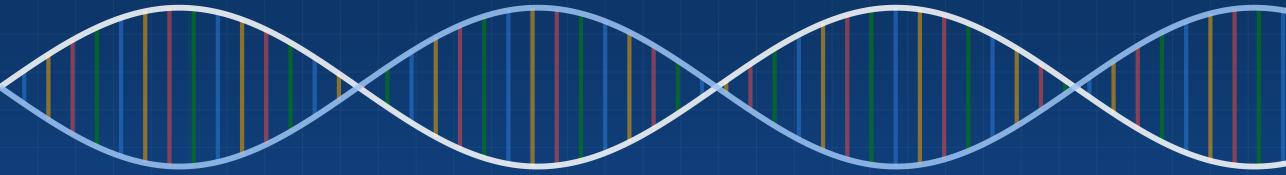
Referencias del capítulo

- Bankevich, A., Nurk, S., Antipov, D., Gurevich, A. A., Dvorkin, M., Kulikov, A. S., et al. (2012). SPAdes: A new genome assembly algorithm and its applications to single-cell sequencing. *Journal of Computational Biology*, 19(5), 455-477. <https://doi.org/10.1089/cmb.2012.0021>
- Brůna, T., Hoff, K. J., Lomsadze, A., Stanke, M., & Borodovsky, M. (2021). BRAKER2: automatic eukaryotic genome annotation with GeneMark-EP+ and AUGUSTUS supported by a protein database. *NAR Genomics and Bioinformatics*, 3(1), lqaa108. <https://doi.org/10.1093/nargab/lqaa108>
- Cheng, H., Concepcion, G. T., Feng, X., Zhang, H., & Li, H. (2021). Haplotype-resolved de novo assembly using phased assembly graphs with hifiasm. *Nature Methods*, 18(2), 170-175. <https://doi.org/10.1038/s41592-020-01056-5>
- Compeau, P. E. C., Pevzner, P. A., & Tesler, G. (2011). How to apply de Bruijn graphs to genome assembly. *Nature Biotechnology*, 29(11), 987-991. <https://doi.org/10.1038/nbt.2023>
- Delcher, A. L., Bratke, K. A., Powers, E. C., & Salzberg, S. L. (2007). Identifying bacterial genes and endosymbiont DNA with Glimmer. *Bioinformatics*, 23(6), 673-679. <https://doi.org/10.1093/bioinformatics/btm009>
- Gurevich, A., Saveliev, V., Vyahhi, N., & Tesler, G. (2013). QUAST: quality assessment tool for genome assemblies. *Bioinformatics*, 29(8), 1072-1075. <https://doi.org/10.1093/bioinformatics/btt086>

- Hyatt, D., Chen, G.-L., LoCascio, P. F., Land, M. L., Larimer, F. W., & Hauser, L. J. (2010). Prodigal: prokaryotic gene recognition and translation initiation site identification. *BMC Bioinformatics*, 11(1), 119. <https://doi.org/10.1186/1471-2105-11-119>
- Idury, R. M., & Waterman, M. S. (1995). A new algorithm for DNA sequence assembly. *Journal of Computational Biology*, 2(2), 291-306. <https://doi.org/10.1089/cmb.1995.2.291>
- Kokot, M., Długosz, M., & Deorowicz, S. (2017). KMC 3: counting and manipulating k-mer statistics. *Bioinformatics*, 33(17), 2759-2761. <https://doi.org/10.1093/bioinformatics/btx304>
- Kolmogorov, M., Yuan, J., Lin, Y., & Pevzner, P. A. (2019). Assembly of long, error-prone reads using repeat graphs. *Nature Biotechnology*, 37(5), 540-546. <https://doi.org/10.1038/s41587-019-0072-8>
- Koren, S., Walenz, B. P., Berlin, K., Miller, J. R., Bergman, N. H., & Phillippy, A. M. (2017). Canu: scalable and accurate long-read assembly via adaptive k-mer weighting and repeat separation. *Genome Research*, 27(5), 722-736. <https://doi.org/10.1101/gr.215087.116>
- Lander, E. S., & Waterman, M. S. (1988). Genomic mapping by fingerprinting random clones: A mathematical analysis. *Genomics*, 2(3), 231-239. [https://doi.org/10.1016/0888-7543\(88\)90007-9](https://doi.org/10.1016/0888-7543(88)90007-9)
- Lukashin, A. V., & Borodovsky, M. (1998). GeneMark.hmm: new solutions for gene finding. *Nucleic Acids Research*, 26(4), 1107-1115. <https://doi.org/10.1093/nar/26.4.1107>
- Manni, M., Berkeley, M. R., Seppey, M., Simão, F. A., & Zdobnov, E. M. (2021). BUSCO update: Novel and streamlined workflows along with broader and deeper phylogenetic coverage for scoring of eukaryotic, prokaryotic, and viral genomes. *Molecular Biology and Evolution*, 38(10), 4647-4654. <https://doi.org/10.1093/molbev/msab199>
- Marçais, G., & Kingsford, C. (2011). A fast, lock-free approach for efficient parallel counting of occurrences of k-mers. *Bioinformatics*, 27(6), 764-770. <https://doi.org/10.1093/bioinformatics/btr011>
- Myers, E. W. (2005). The fragment assembly string graph. *Bioinformatics*, 21(suppl_2), ii79-ii85. <https://doi.org/10.1093/bioinformatics/bti1114>
- Nagarajan, N., & Pop, M. (2013). Sequence assembly demystified. *Nature Reviews Genetics*, 14(3), 157-167. <https://doi.org/10.1038/nrg3367>
- Nurk, S., Koren, S., Rhie, A., Rautiainen, M., Bzikadze, A. V., Mikheenko, A., et al. (2022). The complete sequence of a human genome. *Science*, 376(6588), 44-53. <https://doi.org/10.1126/science.abj6987>
- Pevzner, P. A., Tang, H., & Waterman, M. S. (2001). An Eulerian path approach to DNA fragment assembly. *Proceedings of the National Academy of Sciences*, 98(17), 9748-9753. <https://doi.org/10.1073/pnas.171285098>
- Ranallo-Benavidez, T. R., Jaron, K. S., & Schatz, M. C. (2020). GenomeScope 2.0 and Smudgplot for reference-free profiling of polyploid genomes. *Nature Communications*, 11(1), 1432. <https://doi.org/10.1038/s41467-020-14998-3>

- Rhie, A., Walenz, B. P., Koren, S., & Phillippy, A. M. (2020). Merquy: reference-free quality, completeness, and phasing assessment for genome assemblies. *Genome Biology*, 21(1), 245. <https://doi.org/10.1186/s13059-020-02134-9>
- Schwengers, O., Jelonek, L., Dieckmann, M. A., Beyvers, S., Blom, J., & Goesmann, A. (2021). Bakta: rapid and standardized annotation of bacterial genomes via alignment-free sequence identification. *Microbial Genomics*, 7(11), 000685. <https://doi.org/10.1099/mgen.0.000685>
- Seemann, T. (2014). Prokka: rapid prokaryotic genome annotation. *Bioinformatics*, 30(14), 2068-2069. <https://doi.org/10.1093/bioinformatics/btu153>
- Simão, F. A., Waterhouse, R. M., Ioannidis, P., Kriventseva, E. V., & Zdobnov, E. M. (2015). BUSCO: assessing genome assembly and annotation completeness with single-copy orthologs. *Bioinformatics*, 31(19), 3210-3212. <https://doi.org/10.1093/bioinformatics/btv351>
- Stanke, M., & Waack, S. (2003). Gene prediction with a hidden Markov model and a new intron submodel. *Bioinformatics*, 19(suppl_2), ii215-ii225. <https://doi.org/10.1093/bioinformatics/btg1080>
- Tarailo-Graovac, M., & Chen, N. (2009). Using RepeatMasker to identify repetitive elements in genomic sequences. *Current Protocols in Bioinformatics*, 25(1). <https://doi.org/10.1002/0471250953.bi0410s25>
- Vurture, G. W., Sedlazeck, F. J., Nattestad, M., Underwood, C. J., Fang, H., Gurtowski, J., & Schatz, M. C. (2017). GenomeScope: fast reference-free genome profiling from short reads. *Bioinformatics*, 33(14), 2202-2204. <https://doi.org/10.1093/bioinformatics/btx153>
- Yandell, M., & Ence, D. (2012). A beginner's guide to eukaryotic genome annotation. *Nature Reviews Genetics*, 13(5), 329-342. <https://doi.org/10.1038/nrg3174>
- Zerbino, D. R., & Birney, E. (2008). Velvet: Algorithms for de novo short read assembly using de Bruijn graphs. *Genome Research*, 18(5), 821-829. <https://doi.org/10.1101/gr.074492.107>

Detección de variantes



A pesar de los rápidos avances en las tecnologías de secuenciación, identificar con exactitud las variantes genéticas presentes en un genoma individual a partir de miles de millones de lecturas cortas y con errores sigue siendo un desafío.

— Poplin et al. (2018)

● Al terminar este capítulo usted podrá

- Distinguir los tipos de variación genética (SNV, *indels*, variantes estructurales y de número de copias) y reconocer cuáles puede detectar un llamador de variantes basado en lecturas cortas.
- Derivar, a partir de las calidades Phred, la verosimilitud de cada genotipo diploide $P(D | G)$, combinarla con una distribución *a priori* basada en la heterocigosidad θ y calcular la posterior, el QUAL, el GQ y los PL de un archivo VCF.
- Construir y justificar paso a paso un flujo de llamado con `bcftools` (`mpileup`, `call`, `norm`, `filter`), normalizar *indels* y evaluar el resultado frente a Genome in a Bottle con precisión, sensibilidad y la razón ti/tv.

- Anotar variantes con VEP o SnpEff usando los términos de la Sequence Ontology, interpretar frecuencias de gnomAD, predictores como SIFT, PolyPhen-2 y CADD, y aplicar la lógica de las guías ACMG/AMP.

En los capítulos anteriores obtuvimos lecturas (capítulo 6) y las situamos en un genoma de referencia (capítulo 7). Ahora preguntamos: *¿en qué difiere de esa referencia el genoma que hemos secuenciado?* Un genoma humano típico presenta entre 4,1 y 5,0 millones de diferencias (Auton et al., 2015). Algunas pueden explicar una enfermedad rara, la respuesta de un tumor a un fármaco o la transmisión de un brote bacteriano. Sin embargo, los datos no señalan directamente esas variantes: muestran bases observadas con distintas calidades y posibles errores de mapeo. Detectarlas exige *inferir bajo incertidumbre* qué secuencia produjo las lecturas.

Las tres lecciones siguen el recorrido de los datos. En la sección 9.1 usaremos las calidades Phred para construir un modelo que infiera el genotipo a partir de las bases observadas y cuantifique la confianza en el resultado. En la sección 9.2 aplicaremos esa teoría con `bcftools`: generaremos un VCF, normalizaremos *indels*, filtraremos variantes y evaluaremos el resultado frente a un conjunto de verdad. Por último, estudiaremos su interpretación: posición respecto a los genes, efecto sobre la proteína, frecuencia poblacional y evidencia de posible patogenicidad (sección 9.3).

9.1 Verosimilitud de genotipos

 Colab · 9.1 · Verosimilitud de genotipos

 abrir notebook

Seis testigos de un mismo accidente

Un juez debe decidir de qué color era el coche que huyó de un accidente. Declaran seis testigos: tres dicen “azul” y tres dicen “verde”. Pero no todos merecen la misma confianza: dos de los que dicen “verde” estaban a pocos metros y a plena luz, mientras que el tercero lo vio de noche y de lejos. El juez sensato no cuenta votos: pondera cada testimonio por la fiabilidad del testigo y se pregunta qué hipótesis explica mejor el *conjunto* de declaraciones. Quizá la mejor explicación sea sorprendente: que había *dos* coches, uno azul y otro verde. Además, antes de oír a nadie el juez ya sabe algo: en su ciudad los coches azules son mucho más comunes que los verdes. Cada lectura que cubre

una posición del genoma es uno de esos testigos; su calidad Phred mide su fiabilidad; los dos coches son los dos cromosomas homólogos de un organismo diploide; y lo que el juez sabía de antemano es la distribución *a priori*.

9.1.1 Qué es una variante

Una *variante* es cualquier diferencia entre la secuencia de un individuo y una secuencia de referencia. La referencia no es un genoma “normal” ni “sano”: es simplemente un sistema de coordenadas acordado, de modo que decir que un individuo porta el alelo de referencia en una posición solo significa que coincide con esa convención. Conviene distinguir los tipos de variantes por su tamaño, porque cada escala exige señales y algoritmos distintos (figura 9.1).

- **SNV** (*single nucleotide variant*): sustitución de una base por otra. Cuando es frecuente en la población se la suele llamar SNP. Es, con mucho, la clase más abundante: el Proyecto 1000 Genomas catalogó 84,7 millones de SNP frente a 3,6 millones de *indels* cortos (Auton et al., 2015).
- **MNV** (*multi-nucleotide variant*): varias sustituciones contiguas en el mismo haplotipo, que deben interpretarse juntas porque pueden caer en el mismo codón.
- **Indel**: inserción o delección de una a unas decenas de bases. Son más difíciles de detectar que las SNV, porque el alineador debe abrir un hueco (capítulo 7) y porque en regiones repetitivas admiten varias representaciones equivalentes (sección 9.2).
- **Variantes estructurales** (SV): reordenamientos de más de unas 50 bases, como grandes delecciones, inserciones, inversiones y translocaciones. Se detectan con señales distintas de la simple columna de bases: pares de lecturas con distancia o orientación anómalas, lecturas partidas y cambios de profundidad (Alkan et al., 2011).
- **CNV** (*copy number variants*): subclase de SV en la que un segmento está presente en un número de copias distinto de dos; su principal firma es la profundidad de cobertura.

Aunque las SV son pocas en número, afectan a más bases que todas las SNV juntas: un genoma típico contiene entre 2100 y 2500 variantes estructurales que en conjunto abarcan unos 20 millones de bases (Auton et al., 2015). Este capítulo

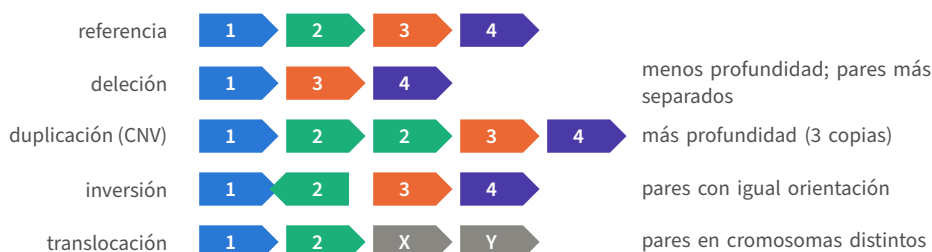
(a) Variantes pequeñas**(b) Variantes estructurales (escala de kb)**

Figura 9.1. Tipos de variación genética según su escala. **(a)** Las variantes pequeñas se leen directamente en la columna de bases alineadas: una sustitución (SNV), una inserción de dos bases y una deleción de dos bases; los recuadros indican el cambio. **(b)** Las variantes estructurales reordenan segmentos de miles de bases (numerados y orientados con flechas). No se ven en una columna, sino en la *geometría* de los alineamientos: profundidad, distancia y orientación de los pares, y lecturas partidas (Alkan et al., 2011). A la derecha, la firma principal de cada una.

se concentra en las variantes *pequeñas* (SNV e *indels*), que son las que tratan los llamadores basados en columnas de lecturas; las SV requieren herramientas específicas que merecen un tratamiento propio.

9.1.2 La columna de bases: el *pileup*

Una vez alineadas las lecturas, cada posición del genoma de referencia queda cubierta por una *columna* de bases, cada una con su calidad Phred. Esa columna, que en la jerga se llama *pileup* (apilamiento), es el dato bruto del llamado de variantes. El formato *pileup* nació con SAMtools y su sucesor, *mpileup*, sigue siendo el centro de *bcftools* (Danecek et al., 2021).

La figura 9.2 muestra un caso sencillo pero instructivo. En la columna 8, la mitad de las lecturas apoya la base de la referencia y la otra mitad una base alternativa. ¿Es el individuo heterocigoto A/G? ¿Podría ser homocigoto A/A y las tres G errores de secuenciación? ¿Y qué hacemos con la G de calidad 10, que tiene un 10 % de

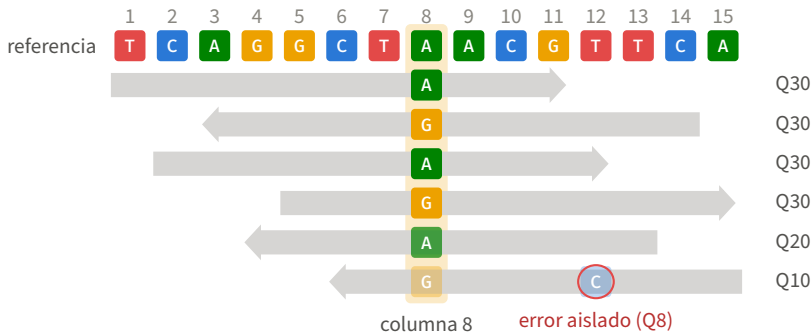


Figura 9.2. Un *pileup* de seis lecturas (barras grises; la punta indica la hebra) sobre 15 posiciones de la referencia. Las bases que coinciden con la referencia se omiten y solo se muestran las que importan. En la columna 8 (sombreada) tres lecturas muestran la A de la referencia y tres una G; la intensidad del color codifica la calidad Phred de cada base, anotada a la derecha. En la columna 12 aparece una C aislada de baja calidad: un error de secuenciación típico que un buen modelo debe ignorar. Esta columna es el dato del ejemplo 9.1.

probabilidad de ser un error? Contar votos (“tres contra tres”) no responde nada. Necesitamos un modelo.

9.1.3 El modelo de error de una base

Definición 9.1. (Genotipo)

En un organismo diploide, el *genotipo* en un sitio es el par no ordenado de alelos $G = \{a_1, a_2\}$ presentes en los dos cromosomas homólogos. Para un sitio bialélico con alelo de referencia R y alternativo A hay tres genotipos: RR (homocigoto de referencia, $0/0$ en VCF), RA (heterocigoto, $0/1$) y AA (homocigoto alternativo, $1/1$).

Empecemos por la unidad mínima: *una* base observada. Recuerde del capítulo 6 que una calidad Phred Q corresponde a una probabilidad de error $\varepsilon = 10^{-Q/10}$. Si la base verdadera de la molécula secuenciada es a , la máquina lee a con probabilidad $1 - \varepsilon$ y, en caso de error, lee cualquiera de las otras tres bases. El supuesto más simple es que las tres son igualmente probables:

$$P(b | a) = \begin{cases} 1 - \varepsilon & \text{si } b = a, \\ \varepsilon/3 & \text{si } b \neq a, \end{cases} \quad \varepsilon = 10^{-Q/10}. \quad (9.1)$$

DONDE	
b	Base observada en la lectura (A, C, G o T).
a	Base verdadera en el cromosoma del que proviene la lectura.
Q, ε	Calidad Phred de la base y su probabilidad de error.

Ahora bien, en un diploide la lectura pudo provenir de cualquiera de los dos cromosomas. Si la biblioteca muestrea ambos por igual, cada uno con probabilidad $1/2$, la ley de la probabilidad total da

$$P(b \mid G = \{a_1, a_2\}) = \frac{1}{2} P(b \mid a_1) + \frac{1}{2} P(b \mid a_2). \tag{9.2}$$

DONDE	
G	Genotipo diploide candidato.
a_1, a_2	Alelos de los dos cromosomas homólogos.
$\frac{1}{2}$	Probabilidad de que la lectura provenga de cada cromosoma (muestreo equilibrado).

Para un homocigoto $a_1 = a_2$ la ecuación se reduce a la ecuación (9.1). Para un heterocigoto $\{R, A\}$, una base que coincida con cualquiera de los dos alelos tiene probabilidad $\frac{1}{2}(1 - \varepsilon) + \frac{1}{2} \varepsilon/3 = \frac{1}{2} - \varepsilon/3$, que es casi exactamente $1/2$: en un heterocigoto, ver R o ver A es como lanzar una moneda, y la calidad casi no importa. En cambio, para un homocigoto, ver el alelo “equivocado” cuesta un factor $\varepsilon/3$, que para $Q30$ es $1/3000$. Toda la capacidad discriminatoria del modelo nace de esa asimetría.

9.1.4 La verosimilitud de un genotipo

Una columna contiene n lecturas con bases $D = (b_1, \dots, b_n)$ y calidades $(Q_1, \dots, Q_n)$. Si los errores de lecturas distintas son independientes, y cada lectura proviene de un cromosoma elegido al azar independientemente de las demás, la probabilidad de toda la columna es el producto de las probabilidades individuales.

Teorema 9.1. (Verosimilitud de un genotipo diploide)

Bajo los supuestos de errores independientes y muestreo equilibrado de los dos cromosomas,

$$L(G) = P(D | G) = \prod_{i=1}^n \left[\frac{1}{2} P(b_i | a_1) + \frac{1}{2} P(b_i | a_2) \right], \quad (9.3)$$

con $P(b_i | a)$ dada por la ecuación (9.1) y $\varepsilon_i = 10^{-Q_i/10}$.

DONDE

D	Datos: las n bases de la columna con sus calidades.
$L(G)$	Verosimilitud del genotipo G : probabilidad de observar D si G fuera el genotipo verdadero.
n	Profundidad de la columna (lecturas que la cubren tras los filtros de calidad).

Vale la pena subrayar lo que la ecuación (9.3) *no* es: no es la probabilidad del genotipo. Es la probabilidad de los datos bajo cada hipótesis. Compararla entre genotipos nos dice cuál explica mejor lo observado, pero para obtener la probabilidad de cada genotipo hará falta el teorema de Bayes y una distribución *a priori*.

La forma de L_i para cualquier ploidía. Li (2011b) escribe el mismo modelo de manera compacta para un sitio bialélico y una ploidía m cualquiera (útil para poliploides, para mezclas de ADN o para *pools*). Sea $g \in \{0, \dots, m\}$ el número de copias del alelo de *referencia* en el genotipo, y ordene las lecturas de modo que las l primeras muestren la referencia y las $n - l$ restantes el alelo alternativo. Entonces

$$L(g) = \frac{1}{m^n} \prod_{j=1}^l [(m - g) \varepsilon_j + g (1 - \varepsilon_j)] \prod_{j=l+1}^n [(m - g)(1 - \varepsilon_j) + g \varepsilon_j]. \quad (9.4)$$

DONDE

m	Ploidía (2 en diploides).
g	Número de copias del alelo de referencia en el genotipo ($0 \leq g \leq m$).
l	Número de lecturas que muestran el alelo de referencia.
ε_j	Probabilidad de error de la base de la lectura j .

Cada corchete es simplemente ecuación (9.2) generalizada: una lectura proviene de una de las m copias (de ahí el factor $1/m$), de las cuales g llevan la referencia. Hay una diferencia sutil con nuestra ecuación (9.1): en la ecuación (9.4) un error convierte una base en *el otro alelo* con probabilidad ε , no $\varepsilon/3$, porque el modelo es estrictamente bialélico. Las dos versiones dan resultados muy parecidos en la práctica; lo importante es no mezclarlas en un mismo cálculo.

Ejemplo 9.1. (Verosimilitudes de la columna 8)

Tomemos la columna de la figura 9.2: **A**(Q30), **A**(Q30), **A**(Q20), **G**(Q30), **G**(Q30), **G**(Q10), con $R = A$ y $A = G$, y usemos el modelo de cuatro bases (ecuación (9.1)). Las contribuciones de cada lectura son:

Lectura	ε	$P(b \mid RR)$	$P(b \mid RA)$	$P(b \mid AA)$
A, Q30 (×2)	0,001	0,999	0,49967	0,00033
A, Q20	0,01	0,990	0,49667	0,00333
G, Q30 (×2)	0,001	0,00033	0,49967	0,999
G, Q10	0,1	0,03333	0,46667	0,900

Multiplicando las seis filas (las marcadas con ×2 cuentan dos veces):

$$L(RR) = 3,66 \times 10^{-9}, \quad L(RA) = 1,44 \times 10^{-2}, \quad L(AA) = 3,33 \times 10^{-10}.$$

El heterocigoto explica los datos unos siete órdenes de magnitud mejor que cualquiera de los homocigotos. Observe el papel de la G de Q10: bajo RR contribuye con 0,033, un factor pequeño pero no catastrófico; si hubiera tenido Q30 habría contribuido con 0,00033. Las bases de baja calidad *pesan menos*, exactamente como los testigos poco fiables del juez.

9.1.5 De verosimilitudes a PL

Las verosimilitudes son números diminutos que se multiplican: con 30 lecturas, $L(RR)$ puede valer 10^{-90} . Por eso los programas trabajan con logaritmos y, siguiendo la tradición de Phred, los expresan en una escala de decibanes normalizada.

Definición 9.2. (Verosimilitudes en escala Phred (PL))

Para cada genotipo candidato G ,

$$PL(G) = -10 \log_{10} \frac{L(G)}{\max_{G'} L(G')}, \quad (9.5)$$

redondeado al entero más próximo. El genotipo más verosímil tiene $PL = 0$ y los demás, valores positivos.

DONDE

$PL(G)$	Cuántos “decibanes” peor explica los datos el genotipo G que el mejor genotipo.
$\max_{G'} L(G')$	Verosimilitud del genotipo que mejor explica los datos.

Para la columna del ejemplo 9.1, $\log_{10} L = (-8,437; -1,840; -9,478)$ y, por tanto, $PL = (66, 0, 76)$ en el orden RR, RA, AA que usa el formato VCF. Un PL de 66 significa que RR es $10^{6,6} \approx 4 \times 10^6$ veces menos verosímil que RA . En un VCF este vector aparece en el campo `FORMAT/PL` de cada muestra, y es la información mínima que hay que conservar si se desea recalculer genotipos más adelante.

9.1.6 La distribución *a priori* y la heterocigosidad θ

Antes de mirar ninguna lectura sabemos algo importante: en la inmensa mayoría de las posiciones del genoma un individuo es idéntico a la referencia. En humanos, dos cromosomas homólogos difieren en aproximadamente una de cada mil bases. Ignorar esta información, es decir, suponer que los tres genotipos son *a priori* igual de probables, llevaría a llamar variantes en cualquier lugar donde unas pocas lecturas con errores coincidan.

La forma estándar de introducir este conocimiento viene de la genética de poblaciones. Sea θ la *heterocigosidad* esperada por sitio, es decir, la probabilidad de que dos cromosomas tomados al azar difieran en una posición. Bajo el modelo neutral de sitios infinitos, el número esperado de sitios en los que el alelo derivado aparece exactamente i de k cromosomas muestreados es θ/i . Con $k = 2$ (los dos cromosomas de un diploide), el alelo alternativo aparece una vez (heterocigoto) con probabilidad θ y dos veces (homocigoto alternativo) con probabilidad $\theta/2$. Esto da la distribución

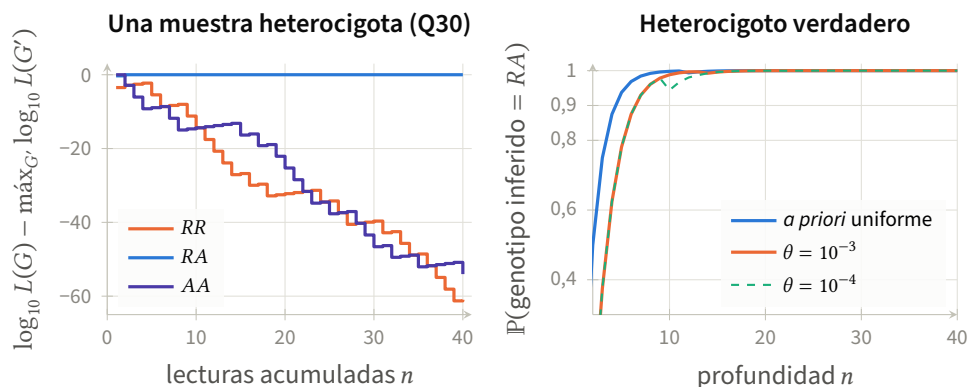


Figura 9.3. **Izquierda:** log-verosimilitud relativa de los tres genotipos a medida que se acumulan, una a una, lecturas simuladas de un heterocigoto verdadero con calidad Q30. Con pocas lecturas los genotipos están cerca (las diferencias son de pocas unidades); cada nueva lectura añade $\approx \log_{10}(1/2) - \log_{10}(\epsilon/3) \approx 3,2$ unidades de evidencia contra el homocigoto que contradice. Las curvas en escalón de *RR* y *AA* divergen casi linealmente con n . **Derecha:** probabilidad de que el genotipo de máxima posterior sea el correcto, promediada sobre el muestreo binomial de los alelos. Por debajo de unas 10 lecturas, el azar de ver pocas copias de un alelo hace fallar al llamador con frecuencia, y una distribución *a priori* escéptica (θ pequeño) empeora la sensibilidad a baja profundidad. Datos de `figuras/cap09/generar.py`.

a priori que usan, con variantes, los principales llamadores (Li, 2011b; Nielsen et al., 2011):

$$P(RA) = \theta, \quad P(AA) = \frac{\theta}{2}, \quad P(RR) = 1 - \frac{3\theta}{2}. \quad (9.6)$$

DONDE

θ	Heterocigosidad (o tasa de mutación escalada) por sitio; del orden de 10^{-3} en humanos.
$P(G)$	Probabilidad <i>a priori</i> del genotipo, antes de ver las lecturas.

De dónde sale θ/i

En una población de tamaño efectivo N_e con tasa de mutación μ por sitio y generación, se define $\theta = 4N_e\mu$. En la genealogía coalescente de k cromosomas, la longitud total esperada de las ramas que sostienen exactamente i hojas es $4N_e/i$ generaciones,

de modo que el número esperado de mutaciones que aparecerán en exactamente i copias es $\mu \cdot 4N_e/i = \theta/i$. Este es el *espectro de frecuencias* neutral. Para $k = 2$ solo hay $i \in \{1, 2\}$, con probabilidades θ y $\theta/2$, cuando $\theta \ll 1$. Li (2011b) usa este mismo espectro como distribución *a priori* de la frecuencia alélica en la llamada multimuestra.

9.1.7 La posterior, QUAL y GQ

Con verosimilitud y distribución *a priori*, el teorema de Bayes da la probabilidad de cada genotipo a la luz de los datos:

$$P(G | D) = \frac{P(D | G) P(G)}{\sum_{G' \in \{RR, RA, AA\}} P(D | G') P(G')}. \quad (9.7)$$

DONDE

$P(G D)$	Probabilidad posterior del genotipo G dados los datos.
$P(D G)$	Verosimilitud (ecuación (9.3)).
$P(G)$	Distribución <i>a priori</i> (ecuación (9.6)).

El genotipo que se informa es el de máxima posterior, $\hat{G} = \arg \max_G P(G | D)$. Dos preguntas distintas se responden con dos números distintos del VCF:

Definición 9.3. (QUAL y GQ)

$$\text{QUAL} = -10 \log_{10} P(G = RR | D), \quad \text{GQ} = -10 \log_{10} [1 - P(\hat{G} | D)]. \quad (9.8)$$

QUAL mide la confianza en que el sitio es *variable* (en alguna muestra); GQ mide la confianza en el *genotipo concreto* asignado a una muestra.

DONDE

QUAL	Calidad del sitio: probabilidad, en escala Phred, de que <i>no</i> haya variante.
GQ	Calidad del genotipo: probabilidad, en escala Phred, de que el genotipo informado sea incorrecto.
$\hat{G}$	Genotipo de máxima posterior.

Los programas difieren en los detalles. La especificación VCF define GQ como la calidad del genotipo *condicionada* a que el sitio sea variable (Danecek et al., 2011); GATK la aproxima como la diferencia entre los dos PL más pequeños, es decir, sin distribución *a priori* (DePristo et al., 2011). La ecuación (9.8) usa una tercera versión, incondicional (sobre todos los genotipos, sin condicionar a que el sitio sea variable), que es la que calcularemos a mano. Como veremos en el ejemplo siguiente, las convenciones pueden dar números muy diferentes, y por eso conviene saber cuál usa el programa antes de fijar umbrales.

Ejemplo 9.2. (Posterior, QUAL y GQ de la columna 8)

Con $\theta = 10^{-3}$, las probabilidades *a priori* son $P(RR) = 0,9985$, $P(RA) = 0,001$ y $P(AA) = 0,0005$. Multiplicando por las verosimilitudes del ejemplo 9.1:

$$P(D | RR)P(RR) = 3,66 \times 10^{-9} \times 0,9985 = 3,65 \times 10^{-9},$$

$$P(D | RA)P(RA) = 1,44 \times 10^{-2} \times 0,001 = 1,44 \times 10^{-5},$$

$$P(D | AA)P(AA) = 3,33 \times 10^{-10} \times 0,0005 = 1,66 \times 10^{-13}.$$

Normalizando, $P(RA | D) = 0,99975$, $P(RR | D) = 2,5 \times 10^{-4}$ y $P(AA | D) \approx 10^{-8}$. Por lo tanto

$$\text{QUAL} = -10 \log_{10}(2,5 \times 10^{-4}) \approx 36, \quad \text{GQ} = -10 \log_{10}(1 - 0,99975) \approx 36.$$

Aquí coinciden porque la alternativa más probable a RA es precisamente RR. En cambio, la versión “sin *a priori*” de GQ, la diferencia entre los dos PL más pequeños, daría $66 - 0 = 66$. La diferencia de 30 unidades es exactamente $-10 \log_{10}(0,001/0,9985) \approx 30$: la distribución *a priori* exige tres órdenes de magnitud adicionales de evidencia antes de creer en una variante.

La distribución *a priori* pesa mucho a baja profundidad. Con solo dos lecturas de Q30, una A y una G, los datos favorecen al heterocigoto (PL = (29, 0, 29)), pero con $\theta = 10^{-3}$ la posterior es $P(RR | D) = 0,571$: el genotipo informado sería RR. Una lectura alternativa aporta un factor $\approx (1/2)/(\epsilon/3) = 1500$ a favor del heterocigoto, insuficiente frente a la razón *a priori* $0,9985/0,001 \approx 1000$ multiplicada por el factor ≈ 2 de la lectura de referencia.

La figura 9.3 (derecha) cuantifica este fenómeno para un heterocigoto verdadero: además del error de secuenciación, está el *error de muestreo*. Con n lecturas, la probabilidad de no ver alguno de los dos alelos es $2 \cdot (1/2)^n$, que vale 0,125 con

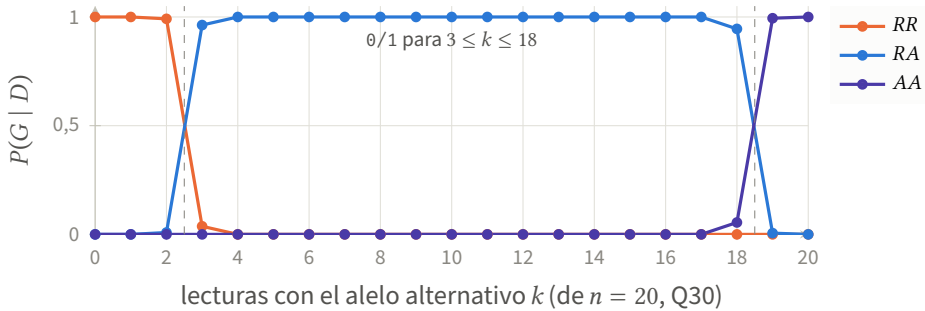


Figura 9.4. Probabilidad posterior de cada genotipo en función del número de lecturas alternativas k en una columna de 20 lecturas Q30, con $\theta = 10^{-3}$. Las fronteras de decisión son muy asimétricas: bastan 3 lecturas alternativas de 20 para declarar un heterocigoto ($P(RA | D) = 0,96$), pero hacen falta 19 para declarar un homocigoto alternativo, porque bajo RA ver 18 lecturas G de 20 es unas nueve veces más plausible que, bajo AA , ver dos errores de Q30. La distribución *a priori* desplaza la frontera izquierda hacia la derecha: con 2 lecturas alternativas la posterior de RR es todavía 0,99.

$n = 4$ y 0,002 con $n = 10$. Con $\theta = 10^{-3}$, la probabilidad de inferir correctamente el heterocigoto es 0,62 con 4 lecturas, 0,87 con 6, 0,988 con 10 y 0,9998 con 20. De ahí la regla práctica de que el genotipado individual fiable exige unas 15–20× de profundidad, y de ahí también que los estudios de baja cobertura necesiten la llamada multimuestra que veremos enseguida.

genotipos.py

```
import math
```

```
def log10_lik(bases, quals, a1, a2):
    """log10 P(D | G={a1,a2}) con el modelo de 4 bases."""
    s = 0.0
    for b, q in zip(bases, quals):
        e = 10 ** (-q / 10)
        p = lambda a: 1 - e if b == a else e / 3
        s += math.log10(0.5 * p(a1) + 0.5 * p(a2))
    return s

def llamar(bases, quals, ref, alt, theta=1e-3):
    G = {"0/0": (ref, ref), "0/1": (ref, alt), "1/1": (alt, alt)}
    prior = {"0/0": 1 - 1.5 * theta, "0/1": theta, "1/1": theta / 2}
    ll = {g: log10_lik(bases, quals, *a) for g, a in G.items()}
```

```

m = max(ll.values())
pl = {g: round(-10 * (v - m)) for g, v in ll.items()}
num = {g: 10 ** (ll[g] - m) * prior[g] for g in G}
z = sum(num.values())
post = {g: v / z for g, v in num.items()}
gt = max(post, key=post.get)
qual = -10 * math.log10(max(post["0/0"], 1e-30))
gq = -10 * math.log10(max(1 - post[gt], 1e-30))
return gt, pl, round(qual, 1), round(gq, 1)

print(llamar("AAAGGG", [30, 30, 20, 30, 30, 10], "A", "G"))
# ('0/1', {'0/0': 66, '0/1': 0, '1/1': 76}, 36.0, 36.0)

```

9.1.8 BAQ: cuando el error no es de la base sino del alineamiento

El modelo de la ecuación (9.3) trata cada base como si estuviera correctamente alineada. Cerca de un *indel* esto falla con frecuencia: si una lectura termina poco después de una delección, el alineador puede preferir colocar dos o tres desapareamientos en lugar de abrir un hueco (capítulo 7), y esas bases *mal alineadas*, con calidades Phred altísimas, aparecen en el *pileup* como SNV falsas agrupadas junto al *indel*.

Li (2011a) propuso corregirlo con la *calidad de alineamiento de la base* (BAQ). Para cada lectura se realinea localmente contra la referencia con un HMM de pares (el mismo tipo de modelo de tres estados que vimos en el algoritmo de Gotoh) y se calcula, por el algoritmo de avance-retroceso, la probabilidad posterior p_i de que la base i esté alineada con la posición a la que la asignó el alineador. La calidad efectiva de la base pasa a ser

$$Q_i^{\text{ef}} = \min(Q_i, \text{BAQ}_i), \quad \text{BAQ}_i = -10 \log_{10}(1 - p_i). \quad (9.9)$$

DONDE

Q_i	Calidad Phred original de la base i (error de lectura).
p_i	Probabilidad posterior, según el HMM de pares, de que la base esté bien alineada.
BAQ_i	Probabilidad de mal alineamiento en escala Phred.

Lejos de los *indels*, $p_i \approx 1$ y la calidad no cambia; en las zonas ambiguas, la BAQ cae y las bases dudosas pierden peso en la verosimilitud. Es un ejemplo elegante de cómo *propagar* una fuente de incertidumbre (el alineamiento) hacia la siguiente etapa en lugar de ignorarla. Un efecto similar se consigue con la calidad de mapeo

(MAPQ, capítulo 7): las lecturas con MAPQ bajo, que podrían pertenecer a otra copia de una repetición, se descartan o se ponderan a la baja.

9.1.9 Llamada conjunta de varias muestras

Cuando se secuencian muchos individuos de una población, analizarlos juntos es mucho más potente que analizarlos por separado. La idea es que la frecuencia del alelo alternativo en la población, ψ , es un parámetro compartido: si varias muestras muestran indicios débiles de la misma variante, la evidencia se acumula.

Bajo equilibrio de Hardy-Weinberg, la distribución *a priori* del genotipo de cada individuo depende de ψ : $P(RR) = (1 - \psi)^2$, $P(RA) = 2\psi(1 - \psi)$ y $P(AA) = \psi^2$. Li (2011b) estima ψ por máxima verosimilitud con el algoritmo EM. Si h indexa el número de alelos alternativos de un genotipo ($h \in \{0, 1, 2\}$) y N es el número de muestras, cada iteración actualiza

$$\psi^{(t+1)} = \frac{1}{2N} \sum_{s=1}^N \sum_{h=0}^2 h \frac{P(D_s | h) P(h | \psi^{(t)})}{\sum_{h'} P(D_s | h') P(h' | \psi^{(t)})}. \quad (9.10)$$

DONDE

ψ	Frecuencia del alelo alternativo en la población muestreada.
D_s	Lecturas de la muestra s en el sitio.
h	Número de copias del alelo alternativo en el genotipo (0, 1 o 2).
$P(h \psi)$	Probabilidad de Hardy-Weinberg del genotipo con h copias.
N	Número de muestras analizadas conjuntamente.

La ecuación (9.10) tiene una lectura muy intuitiva: el paso E calcula, para cada muestra, el número *esperado* de copias alternativas dados sus datos y la frecuencia actual; el paso M promedia esos números sobre los $2N$ cromosomas. Cada iteración no puede disminuir la verosimilitud conjunta, y el algoritmo converge rápidamente.

Ejemplo 9.3. (Ocho muestras de baja cobertura)

Ocho individuos se secuencian a 2–5× y en un sitio muestran (lecturas alternativas, profundidad) = (1, 3), (2, 4), (0, 3), (3, 5), (1, 2), (0, 4), (2, 3), (0, 2), todas con Q20. Usando el modelo bialélico de la ecuación (9.4) y partiendo de $\psi^{(0)} = 0,5$, el EM produce 0,354, 0,334, 0,331, ... y converge en 12 iteraciones a $\hat{\psi} = 0,331$. Considere la muestra M1, con una lectura alternativa de tres. Analizada sola, con la distribución *a priori* de la ecuación (9.6) y $\theta = 10^{-3}$,

su posterior es $P(RR) = 0,987$: se la declararía homocigota de referencia. Analizada con las demás, cuya evidencia colectiva establece que la variante es común, su posterior pasa a $P(RA) = 0,926$. La muestra no cambió; cambió lo que sabemos de la población.

La llamada conjunta no es gratuita: supone una población aproximadamente panmíctica, y un artefacto sistemático presente en todas las muestras quedará *reforzado* como si fuera una variante común; los filtros de la sección 9.2 existen precisamente para eso.

9.1.10 Más allá de la columna: HaplotypeCaller y DeepVariant

El modelo de la ecuación (9.3) mira cada posición por separado. Los llamadores de última generación han ido más allá en dos direcciones.

Reensamblaje local (GATK HaplotypeCaller). El Genome Analysis Toolkit nació como un marco de programación para procesar datos de secuenciación a escala (McKenna et al., 2010), y DePristo et al. (2011) lo convirtieron en un flujo completo de descubrimiento de variantes con recalibración de calidades de base (BQSR), realineamiento local y una recalibración de la calidad de las variantes (VQSR) basada en un modelo de mezcla gaussiana entrenado con sitios conocidos. Su sucesor, **HaplotypeCaller**, cambia el paradigma: detecta “regiones activas” con indicios de variación, reensambla localmente las lecturas de cada región con un grafo de De Bruijn para proponer *haplotipos* candidatos, calcula con un HMM de pares la verosimilitud $P(\text{lectura} \mid \text{haplotipo})$ y solo entonces obtiene las verosimilitudes de los genotipos, marginalizando sobre los haplotipos. Así, SNV e *indels* cercanos se evalúan juntos y los errores del alineador pesan mucho menos. Para cohortes enormes, Poplin et al. (2017) combinaron HaplotypeCaller con un *modelo de confianza de referencia*: las verosimilitudes de genotipos se calculan de forma independiente por muestra, también en los sitios donde la muestra parece idéntica a la referencia, y se guardan en un GVCF con bloques de referencia y un alelo genérico <NON_REF>; el genotipado conjunto de todas las muestras se hace después, sin volver a leer los BAM. Así llamaron más de 90 000 exomas del consorcio ExAC y obtuvieron una matriz completa de genotipos en todas las posiciones. En la práctica, esto evita el llamado “problema $N + 1$ ”: añadir una muestra no obliga a repetir todo el análisis.

Aprendizaje profundo (DeepVariant). Poplin et al. (2018) dieron un paso más radical: en lugar de escribir el modelo de error, lo *aprenden*. DeepVariant convierte el *pileup* alrededor de cada candidato en una imagen de varios canales (base, calidad, hebra, MAPQ, soporte del alelo) y una red neuronal convolucional devuelve directamente las probabilidades de los tres genotipos. Entrenado con genomas cuyo genotipo verdadero es conocido (sección 9.2), supera a los métodos estadísticos clásicos y se adapta a otras tecnologías de secuenciación con solo reentrenarlo.

De contar alelos a modelar la incertidumbre

Los primeros estudios con NGS llamaban variantes con reglas simples, del tipo “al menos tres lecturas alternativas y un 20 % de fracción alélica”. Nielsen et al. (2011) mostraron que esas reglas desperdiciaban información y no dan ninguna medida de incertidumbre, sobre todo a cobertura baja o media, y abogaron por métodos que calcularan verosimilitudes de genotipos y las propagaran a los análisis posteriores. Ese mismo año se publicaron los marcos estadísticos de SAMtools/BCFtools (Li, 2011b) y de GATK (DePristo et al., 2011).

9.2 Pipeline con bcftools

 Colab · 9.2 · Pipeline con bcftools

 abrir notebook

La cocina de un restaurante

En la cocina de un buen restaurante nadie cocina “un plato”: hay una estación que prepara los ingredientes, otra que cocina, otra que emplata y un jefe que prueba cada plato antes de que salga. Cada estación recibe algo en un formato conocido y entrega algo en otro formato conocido; si un plato sale mal, se sabe en qué estación buscar el problema. Y, sobre todo, el restaurante no mide su calidad por lo bien que *crea* que cocina, sino por lo que dicen los comensales. Un flujo de llamado de variantes es esa cocina: *mpileup* prepara las columnas, *call* cocina los genotipos, *norm* emplata las variantes de forma estándar, *filter* prueba el resultado, y un conjunto de verdad como Genome in a Bottle hace de comensal exigente.

cabecera (##)

```
##fileformat=VCFv4.2
##reference=GRCh38.fa
##INFO=<ID=DP,Number=1,Type=Integer,Description="Profundidad total">
##FORMAT=<ID=PL,Number=G,Type=Integer,Description="Verosimilitudes Phred">
##FILTER=<ID=BajaCal,Description="QUAL<30 || DP<10">
```

cuerpo

#CHROM	POS	ID	REF	ALT	QUAL	FILTER	INFO
chr1	10408	.	A	G	36	PASS	DP=6;AC=1;AN=2;MQ=60

FORMAT **muestra1**

```
GT:AD:DP:GQ:PL 0/1:3,3:6:36:66,0,76
```

GT 0/1: heterocigoto (0 = REF, 1 = primer ALT); AD 3,3: lecturas que apoyan REF y ALT;
 DP 6: profundidad; GQ 36: calidad del genotipo (ecuación (9.8));
 PL 66,0,76: PL(RR),PL(RA),PL(AA); QUAL 36: $-10 \log_{10} P(RR | D)$

Figura 9.5. Anatomía de un VCF. La cabecera (líneas ##) declara el significado de cada campo; el cuerpo tiene una línea por sitio. Las ocho primeras columnas describen el sitio; FORMAT indica el orden de los campos por muestra. El registro corresponde a la columna 8 de la figura 9.2: QUAL 36, PL (66, 0, 76) y GQ 36, exactamente los valores calculados en los ejemplos 9.1 y 9.2. POS usa coordenadas que empiezan en 1, y REF debe coincidir con el genoma de referencia.

9.2.1 El formato VCF

Antes de producir variantes necesitamos un lugar donde escribirlas. El *Variant Call Format* (VCF) fue diseñado para el Proyecto 1000 Genomas como un formato de texto tabulado, extensible y capaz de representar desde una SNV hasta variantes estructurales para miles de muestras (Danecek et al., 2011). Su versión binaria comprimida, BCF, es la que **bcftools** usa internamente para ganar velocidad.

Un VCF tiene dos partes (figura 9.5). La *cabecera*, con líneas que empiezan por ##, declara la versión del formato, la referencia, los contigs y, sobre todo, el significado de cada campo INFO, FORMAT y FILTER. Esta autodescripción es clave: un campo que no está declarado en la cabecera no debería aparecer en los registros. El *cuerpo* tiene una línea por sitio variable, con ocho columnas fijas (CHROM, POS, ID, REF, ALT, QUAL, FILTER, INFO), seguidas opcionalmente de una columna FORMAT y una columna por muestra.

Algunos detalles del formato causan una cantidad sorprendente de errores. POS es 1-based (a diferencia de BED, que es 0-based y semiabierto). En los *indels*, REF y ALT incluyen una base de anclaje a la izquierda: una delección de TC tras una G se escribe REF=GTC, ALT=G. El genotipo 0/1 indica que no se conoce la fase; 0|1 indica que el alelo de referencia está en el primer haplotipo. Y un sitio puede tener varios

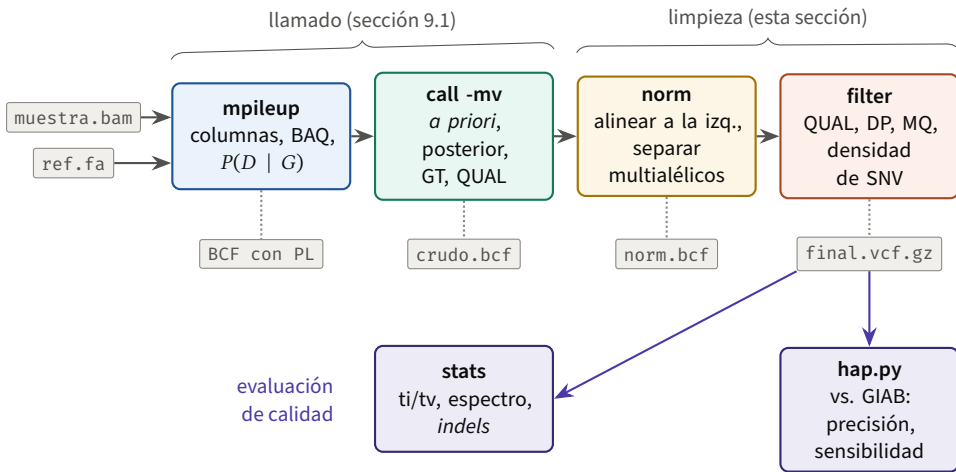


Figura 9.6. Flujo de llamado de variantes con `bcftools`. `mpileup` construye las columnas y calcula las verosimilitudes de genotipos (ajustando las calidades con BAQ); `call` combina verosimilitudes y distribución *a priori*, y con `-v` conserva solo los sitios variables; `norm` da a cada variante su representación canónica; `filter` marca o elimina las llamadas dudosas. La evaluación usa estadísticas internas (`bcftools stats`) y externas (`hap.py` frente a Genome in a Bottle). Los pasos pueden encadenarse con tuberías en formato BCF sin comprimir (`-Ou`) para evitar escribir archivos intermedios.

alelos alternativos (ALT=G, T), en cuyo caso el número de PL crece: con k alelos hay $\binom{k+1}{2}$ genotipos diploides.

9.2.2 El flujo de trabajo

`bcftools` es el compañero de SAMtools para manipular VCF/BCF y llamar variantes; Danecek et al. (2021) resumen doce años de su desarrollo. Un flujo mínimo para una o varias muestras tiene cinco pasos (figura 9.6).

llamado_bcftools.sh

```

REF=GRCh38.fa
# 1-2. Columnas + verosimilitudes, y llamado multialelico
bcftools mpileup -Ob -o pl.bcf -f $REF -q 20 -Q 20 \
  -a FORMAT/AD,FORMAT/DP muestra.bam
bcftools call -mv -Ob -o crudo.bcf pl.bcf
# 3. Normalización: alinear a la izquierda y separar multialelicos
bcftools norm -f $REF -m -any -Ob -o norm.bcf crudo.bcf
  
```

```
# 4. Filtro "suave": marca en FILTER en lugar de borrar
bcftools filter -s BajaCal -e 'QUAL<30 || INFO/DP<10' \
    -Oz -o final.vcf.gz norm.bcf
bcftools index -t final.vcf.gz
# 5. Estadísticas (incluye ti/tv) de los sitios que pasan
bcftools stats -f PASS final.vcf.gz > final.stats
grep ^TSTV final.stats
```

Detengámonos en las opciones, porque cada una traduce un supuesto del modelo:

- `mpileup -f` carga la referencia: sin ella no se sabe qué base es “referencia”. `-q 20` descarta lecturas con MAPQ menor que 20 y `-Q 20` ignora bases de calidad menor que 20; ambos umbrales eliminan testigos poco fiables antes de que entren en la ecuación (9.3). `-a FORMAT/AD, FORMAT/DP` añade al VCF el número de lecturas por alelo, imprescindible para filtrar y para detectar desequilibrios alélicos.
- BAQ (ecuación (9.9)) se aplica por omisión; `-B` lo desactiva. El modo predeterminado (completo o parcial, solo cerca de *indels*) ha cambiado entre versiones, así que conviene consultar `bcftools mpileup --help` de la versión instalada y registrarla en el cuaderno de laboratorio.
- `call -m` usa el llamador multialélico, que evalúa todos los alelos observados en la columna; `-v` escribe solo sitios variables. La distribución *a priori* se controla con `-P` (por omisión $1,1 \times 10^{-3}$, el papel de θ en la ecuación (9.6)): aumentarla hace el llamador más sensible y menos específico. Para una especie de alta diversidad, como muchos insectos o plantas silvestres, conviene ajustarla.
- Con varias muestras basta dar varios BAM (o una lista con `-b`) a `mpileup: call` hará entonces la llamada conjunta de la sección 9.1.

9.2.3 Normalización de *indels*

Considere la referencia **GGATC**C**ACACAGT**, que contiene el microsatélite (CA)₄ en las posiciones 5–12, y un individuo que perdió una de las cuatro unidades CA. Su haplotipo es **GGATC**C**ACACAGT**. ¿Dónde está la delección? La pregunta no tiene respuesta biológica: borrar cualquiera de las cuatro unidades CA, o incluso los pares AC desfasados, produce exactamente el mismo haplotipo. Pero en un VCF hay que escribir una posición concreta, y distintos llamadores (o el mismo llamador con lecturas distintas) pueden elegir distintas. Dos archivos que describen la misma

variante parecerán discrepar, y la anotación, la comparación con bases de datos y la evaluación frente a un conjunto de verdad se vendrán abajo.

Tan et al. (2015) formalizaron el problema y su solución.

Definición 9.4. (Variante normalizada)

Una variante (POS, REF, ALT) está *normalizada* si y solo si es *parsimoniosa* (se representa con los alelos más cortos posibles, sin bases compartidas superfluas) y está *alineada a la izquierda* (su posición no puede desplazarse más hacia la izquierda sin cambiar el haplotipo que representa).

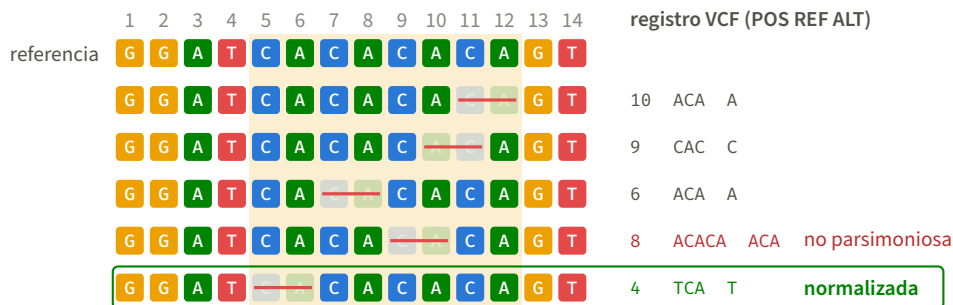
El algoritmo es sorprendentemente simple y se aplica a cualquier par de alelos, sean *indels* o sustituciones complejas:

1. Mientras los dos alelos terminen en la misma base, se elimina esa base final de ambos. Si alguno queda vacío, se antepone a ambos la base anterior de la referencia y se decrementa POS. Se repite hasta que ninguna de las dos condiciones se cumpla.
2. Mientras ambos alelos tengan longitud mayor que 1 y empiecen por la misma base, se elimina esa base inicial de ambos y se incrementa POS.

El paso 1 “empuja” la variante hacia la izquierda a través de la repetición; el paso 2 recorta el contexto sobrante a la izquierda, manteniendo una sola base de anclaje. Tan et al. (2015) demuestran que el resultado es único: todas las representaciones de un mismo haplotipo convergen a la misma forma normalizada.

Ejemplo 9.4. (Normalizar paso a paso)

Tomemos el registro POS=10, REF=ACA, ALT=A de la figura 9.7. Paso 1: ambos alelos terminan en A; se elimina: AC/vacío. ALT quedó vacío, así que se antepone la base 9 (C): POS=9, CAC/C. Terminan en C: CA/vacío; se antepone la base 8 (A): POS=8, ACA/A. El ciclo se repite desplazando la delección de dos en dos bases hasta POS=4, TCA/T: ahora terminan en A y T, distintas, y T no está vacío; el paso 1 se detiene. Paso 2: T tiene longitud 1, así que no hay nada que recortar. Resultado: 4 TCA T. El [generar.py](#) del capítulo comprueba que seis representaciones convergen al mismo registro: las cuatro primeras filas de la figura y otras dos equivalentes, 4 TCACA TCA (no parsimoniosa) y 11 CAG G.



Haplotipo resultante en todos los casos: GGATCACACAGT

Figura 9.7. Una misma deleción de dos bases en el microsatélite $(CA)_4$ (sombreado) admite muchas representaciones en VCF. Las bases tachadas son las que cada registro declara borradas; todas producen el haplotipo GGATCACACAGT. El registro de la cuarta fila, además, no es parsimonioso: arrastra bases compartidas por REF y ALT. El algoritmo de Tan et al. (2015), implementado en `bcftools norm` y `vt normalize`, lleva todas ellas a la forma única de la última fila: la deleción más a la izquierda posible, con una sola base de anclaje (POS=4, TCA→T). Otra representación válida, anclada a la derecha, es 11 CAG G, que también normaliza a la última fila.

normalizar.py

```
def normalizar(pos, ref, alt, genoma):
    """Tan et al. (2015). pos es 1-based; genoma, la referencia."""
    while True:
        cambio = False
        if ref and alt and ref[-1] == alt[-1]: # recorte der.
            ref, alt = ref[:-1], alt[:-1]
            cambio = True
        if not ref or not alt: # extender izq.
            pos -= 1
            b = genoma[pos - 1]
            ref, alt = b + ref, b + alt
            cambio = True
        if not cambio:
            break
        while len(ref) > 1 and len(alt) > 1 and ref[0] == alt[0]:
            ref, alt, pos = ref[1:], alt[1:], pos + 1 # recorte izq.
    return pos, ref, alt

print(normalizar(10, "ACA", "A", "GGATCACACACAGT")) # (4, 'TCA', 'T')
```

La opción `-m -any` de `bcftools norm` hace una segunda normalización, igualmente importante: separa los sitios multialélicos (ALT=G,T) en un registro por alelo alternativo, de modo que cada línea describa una sola variante. Sin este paso, un sitio con ALT=GA,G (una inserción y una SNV en el mismo lugar) no puede alinearse a la izquierda correctamente ni compararse con bases de datos que almacenan una variante por línea. Normalice siempre, con la misma versión de la referencia, antes de intersectar dos VCF, anotar o evaluar frente a un conjunto de verdad: una fracción notable de las “discrepancias” entre llamadores en *indels* desaparece al hacerlo (Tan et al., 2015).

9.2.4 Filtrado

Un llamado crudo contiene inevitablemente falsos positivos. Sus causas típicas son conocidas: lecturas mal mapeadas en regiones repetitivas o duplicaciones segmentarias ausentes de la referencia; errores de secuenciación sistemáticos dependientes del contexto; errores de PCR; y regiones de baja complejidad donde los *indels* son ambiguos. Li (2014) estudió estos artefactos con un ingenioso diseño: llamó variantes en un genoma humano *haploide* (una línea celular de mola hidatiforme), donde todo heterocigoto es por definición un error. Identificó dos fuentes principales: el realineamiento erróneo en regiones de baja complejidad y la incompletitud del genoma de referencia respecto a la muestra, cuyas secuencias ausentes se mapean sobre copias parecidas y producen falsos heterocigotos, a menudo con profundidad excesiva. Estimó que los genotipos crudos tienen un error cada 10–15 kb, y que un filtrado cuidadoso lo reduce a uno cada 100–200 kb sin pérdida apreciable de sensibilidad. Por eso, además de umbrales mínimos de QUAL y profundidad, es habitual imponer un umbral *máximo* de profundidad en genomas completos, del orden de la media más unas pocas desviaciones típicas de Poisson.

Los filtros más comunes para llamados de `bcftools` son:

- **QUAL** mínimo (evidencia de variación) y **GQ** mínimo por muestra.
- **DP** mínimo (poca evidencia) y máximo (copias colapsadas).
- **MQ** medio bajo: el sitio está en una región de mapeo ambiguo.
- **Sesgo de hebra**: una variante real debe aparecer en lecturas de ambas hebras; si todas las lecturas alternativas vienen de la misma, sospeche de un artefacto.
- **Desequilibrio alélico**: en un heterocigoto germinal, la fracción alternativa AD_{alt}/DP debe rondar 0,5; un 0,1 sugiere contaminación, un error sistemático o una variante somática en mosaico.

- **Proximidad a *indels*** (`bcftools filter -g, --SnpGap`, que marca los SNV a menos de n bases de un *indel*) y **racimos de *indels*** (`-G, --IndelGap`): los SNV apiñados alrededor de un *indel* suelen ser artefactos de alineamiento. La densidad de SNV en una ventana no tiene opción propia en `bcftools filter` y se evalúa aparte (por ejemplo, contando llamadas por ventana).

Los filtros “duros” son transparentes, pero fijan cada umbral por separado; VQSR, de GATK, aprende en cambio un modelo de mezcla gaussiana sobre las anotaciones de sitios de alta confianza, y requiere cohortes grandes o genomas completos (DePristo et al., 2011).

9.2.5 La razón t_i/t_v como termómetro

¿Cómo saber si un conjunto de variantes es bueno si no conocemos la verdad? Un indicador clásico aprovecha la química de las mutaciones. Las *transiciones* (t_i) cambian una purina por otra purina ($A \leftrightarrow G$) o una pirimidina por otra ($C \leftrightarrow T$); las *transversiones* (t_v), una purina por una pirimidina o viceversa. De las 12 sustituciones posibles, 4 son transiciones y 8 transversiones, de modo que si las variantes fueran errores aleatorios uniformes, la razón t_i/t_v sería $4/8 = 0,5$. Las mutaciones reales, en cambio, están muy sesgadas hacia las transiciones (por ejemplo, por la desaminación de citosinas metiladas en dinucleótidos CpG), y en humanos la razón observada en variantes de alta calidad ronda $2,0-2,1$ en genomas completos y valores más altos, cercanos a 3, en exomas, donde abundan los CpG. DePristo et al. (2011) usaron la t_i/t_v como una de sus métricas centrales de calidad.

Suponga que las variantes verdaderas tienen razón R_T y los falsos positivos, razón $R_F = 0,5$. Si una fracción α del conjunto son falsos positivos, la fracción observada de transiciones es una mezcla:

$$p_{\text{obs}} = (1 - \alpha) p_T + \alpha p_F, \quad p = \frac{R}{1 + R} \quad \Rightarrow \quad \alpha = \frac{p_T - p_{\text{obs}}}{p_T - p_F}. \quad (9.11)$$

DONDE

R_T, R_F	Razón t_i/t_v de las variantes verdaderas y de los errores.
p_T, p_F, p_{obs}	Fracción de transiciones entre las verdaderas, los errores y el conjunto observado.
α	Fracción de falsos positivos en el conjunto.

Con $R_T = 2,1$ y un conjunto observado con $R_{\text{obs}} = 1,8$: $p_T = 0,677$, $p_{\text{obs}} = 0,643$ y $p_F = 0,333$, así que $\alpha \approx 0,10$. Una caída de la t_i/t_v de $2,1$ a $1,8$, que parece modesta,

delata que uno de cada diez sitios es falso. Con $R_{\text{obs}} = 2,0$ la estimación baja a $\alpha \approx 0,03$. La estimación es aproximada (los errores reales no son uniformes y R_T depende de la región), pero como alarma temprana es muy útil: si al relajar un filtro la ti/tv cae, lo que se está añadiendo es sobre todo ruido.

9.2.6 Evaluar frente a la verdad: Genome in a Bottle

La única forma rigurosa de medir la exactitud de un llamador es compararlo con un genoma cuyo genotipo verdadero se conozca. El consorcio Genome in a Bottle (GIAB), coordinado por el NIST, construyó exactamente eso: Zook et al. (2014) integraron y arbitraron 14 conjuntos de datos de NA12878 (HG001), obtenidos con cinco tecnologías de secuenciación, siete alineadores y tres llamadores, para producir genotipos de alta confianza (SNP, *indels* y homocigotos de referencia), junto con la delimitación de las regiones donde no era posible un genotipo confiable. Zook et al. (2019) añadieron seis genomas del Personal Genome Project con consentimiento abierto (entre ellos, los tríos HG002–HG004 y HG005–HG007), con un flujo reproducible que combina lecturas cortas y enlazadas; respecto a las versiones anteriores, el nuevo recurso contiene un 17 % más de SNV, un 176 % más de *indels* y regiones de referencia un 12 % mayores.

Comparar dos VCF no es trivial: las mismas variantes pueden estar representadas de forma diferente (figura 9.7), incluso más allá de la normalización, cuando varias variantes cercanas se escriben como una sola sustitución compleja. El equipo de evaluación de la Global Alliance for Genomics and Health estableció buenas prácticas y herramientas, como [hap.py](https://hap.py.readthedocs.io/), que comparan *haplotipos* en lugar de registros (Krusche et al., 2019). Esos autores ilustran también por qué importa restringirse a las regiones de alta confianza: la concordancia de SNV entre dos métodos era del 99,7 % dentro de ellas y de solo el 76,5 % fuera. Las herramientas distinguen además varios niveles de coincidencia: que la variante esté (ignorando el genotipo), que el genotipo coincida, o que coincida la fase.

Definición 9.5. (Métricas de evaluación)

Sean TP las variantes del conjunto de verdad correctamente llamadas, FP las llamadas ausentes de la verdad y FN las variantes de la verdad no llamadas, todas restringidas a las regiones de alta confianza. Entonces

$$\text{precisión} = \frac{TP}{TP + FP}, \quad \text{sensibilidad} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 TP}{2 TP + FP + FN}. \quad (9.12)$$

DONDE

TP, FP, FN	Verdaderos positivos, falsos positivos y falsos negativos.
precisión	Fracción de llamadas que son reales (su complemento es la tasa de falsos descubrimientos).
sensibilidad	Fracción de variantes reales que se detectaron (<i>recall</i>).
F_1	Media armónica de precisión y sensibilidad.

Observe que no aparecen los verdaderos negativos: en un genoma hay miles de millones de posiciones sin variante, y cualquier métrica que los incluyera (como la especificidad) sería prácticamente 1 para cualquier llamador. Por la misma razón, en este contexto se prefieren las curvas de precisión-sensibilidad a las curvas ROC.

Ejemplo 9.5. (Leer un informe de [hap.py](#))

Un llamador aplicado a HG002 produce, en las regiones de alta confianza, TP = 3 290 000 SNV, FP = 9800 y FN = 21 500 (cifras redondas de orden realista). La precisión es $3\,290\,000 / 3\,299\,800 = 0,99703$, la sensibilidad $3\,290\,000 / 3\,311\,500 = 0,99351$ y $F_1 = 0,99527$. Todo parece excelente, pero en valores absolutos hay casi diez mil falsos positivos y más de veinte mil variantes perdidas. Por eso se informan las métricas por separado para SNV e *indels* y estratificadas por tipo de región.

9.2.7 Elegir el umbral de QUAL

Todo filtro establece un compromiso: subir el umbral elimina falsos positivos, pero también verdaderos. La figura 9.8 muestra ese compromiso en una simulación con 10 000 variantes verdaderas, de las cuales el llamador propone 9850 (las 150 restantes nunca se llaman, por ejemplo por falta de cobertura), más 1500 falsos

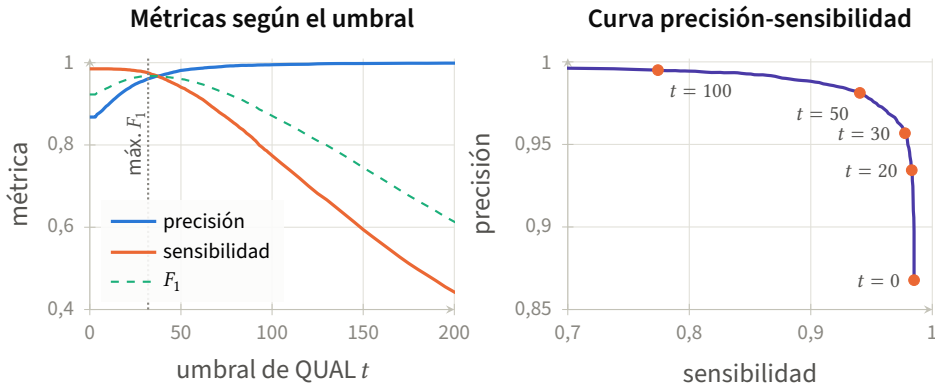


Figura 9.8. Efecto del umbral de QUAL en una simulación con 10 000 variantes verdaderas y 1500 falsos positivos. **Izquierda:** la precisión sube rápidamente con los primeros umbrales, porque los falsos positivos se concentran en QUAL bajo, mientras que la sensibilidad apenas baja hasta $t \approx 40$ y luego cae. **Derecha:** la misma información como curva precisión-sensibilidad; los puntos marcan algunos umbrales. La sensibilidad nunca supera 0,985: ningún filtro recupera las variantes que el llamador no propuso. El umbral óptimo depende del uso: un cribado clínico de primera línea prioriza la sensibilidad; un catálogo de referencia, la precisión. Datos de `figuras/cap09/generar.py`.

positivos con QUAL típicamente bajo. Sin filtro, la precisión es 0,868 y la sensibilidad 0,985. Con $\text{QUAL} \geq 20$, la precisión sube a 0,934 perdiendo apenas 19 verdaderos; con $\text{QUAL} \geq 30$ llega a 0,957 con sensibilidad 0,978. El máximo de F_1 (0,968) se alcanza hacia $\text{QUAL} = 32$. Más allá, cada unidad de precisión se paga cara: con $\text{QUAL} \geq 100$ la precisión es 0,995, pero la sensibilidad se desploma a 0,774.

9.3 Anotación funcional de variantes

 Colab · 9.3 · Anotación funcional de variantes

 abrir notebook

Una errata en un libro de recetas

Una errata en un libro de cocina puede ser inocua o desastrosa según dónde caiga. En el prólogo, nadie la nota. En el nombre de un plato, confunde pero no arruina la cena. En la cantidad de sal (“500 g” en lugar de “5 g”) arruina

el plato. Y si la errata borra una línea entera de instrucciones, todo lo que sigue queda desfasado y la receta se vuelve incomprensible. Además, el daño depende de si el plato es uno que se cocina todos los días o uno que casi nadie prepara, y de si otras ediciones del libro ya tenían esa misma errata sin que nadie se quejara. Anotar una variante es responder, con datos, a estas mismas preguntas: dónde cae, qué cambia, si se ha visto antes en personas sanas y qué sabemos de su efecto.

9.3.1 Consecuencias sobre la estructura del gen

El primer paso de la anotación es geométrico: cruzar la posición de cada variante con un modelo de genes (por ejemplo, los transcritos de Ensembl o RefSeq) y decidir qué parte de qué transcrito afecta. Para que distintos programas hablen el mismo idioma, las consecuencias se describen con términos de la *Sequence Ontology* (SO), una ontología de los elementos y alteraciones de las secuencias biológicas (Eilbeck et al., 2005). Términos como *missense_variant*, *splice_donor_variant* o *frameshift_variant* tienen una definición precisa y un lugar en una jerarquía, lo que permite, por ejemplo, preguntar por todas las variantes “que alteran la secuencia de la proteína” sin enumerarlas.

Un mismo cambio de base puede tener consecuencias distintas en distintos transcritos del mismo gen, y un gen humano tiene, en promedio, varios transcritos. Por eso los anotadores devuelven una lista de consecuencias por variante (una por transcrito afectado), y ofrecen opciones para elegir un transcrito representativo, por ejemplo el canónico o el de mayor impacto. La descripción textual de la variante sigue la nomenclatura HGVS (den Dunnen et al., 2016): NM_000546.6:c.743G>A indica una sustitución en la posición 743 de la secuencia codificante del transcrito, y p.(Arg248Gln) su efecto predicho sobre la proteína; el paréntesis señala que el efecto es una predicción, no una observación experimental.

9.3.2 Consecuencias sobre el codón

Dentro de la región codificante, la consecuencia depende del código genético. La figura 9.10 muestra los casos principales sobre una secuencia de cinco codones.

¿Con qué frecuencia cae una SNV en cada categoría? Podemos responderlo exhaustivamente: los 61 codones con sentido admiten $61 \times 9 = 549$ sustituciones de una base. Recorriéndolas con la tabla del código genético estándar, 134 (24,4 %) son sinónimas, 392 (71,4 %) de sentido erróneo y 23 (4,2 %) crean un codón de parada. La distribución por posición del codón es muy desigual (figura 9.11): en la segunda

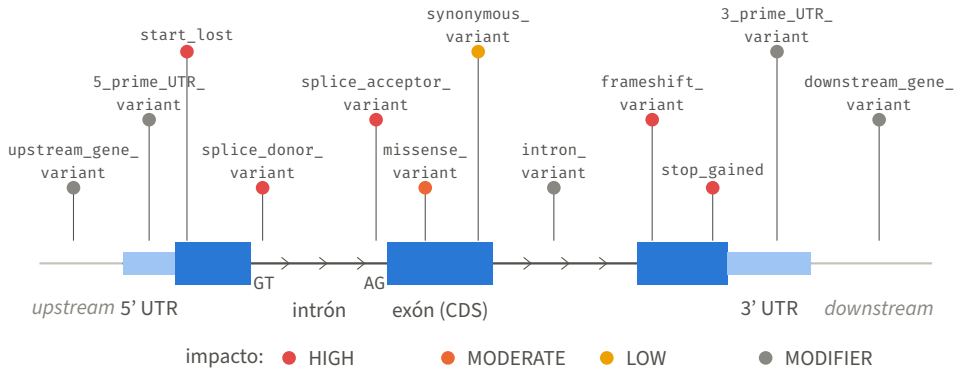


Figura 9.9. Consecuencias de una variante según dónde caiga en un gen de tres exones, con los términos de la Sequence Ontology que usan VEP y SnpEff. Las cajas gruesas son la región codificante (CDS) y las delgadas las regiones no traducidas (UTR); GT y AG marcan los sitios canónicos de corte y empalme del primer intrón. El color resume la categoría de impacto: HIGH (probable pérdida de función: sitios de *splicing* canónicos, *frameshift*, codón de parada prematuro, pérdida del inicio), MODERATE (cambio de aminoácido), LOW (sinónima) y MODIFIER (no codificante). La categoría es una heurística, no un veredicto: una variante intrónica profunda puede crear un sitio de *splicing* nuevo y ser patogénica.

posición *ninguna* sustitución es sinónima, mientras que en la tercera lo son dos de cada tres. Esta es la base del cociente d_N/d_S de la evolución molecular, y también de un hecho práctico: bajo neutralidad esperaríamos unas tres variantes de sentido erróneo por cada sinónima, así que un gen en el que la población muestra muchas menos está sometido a selección purificadora. Esa es precisamente la lógica de las métricas de restricción de gnomAD que veremos más adelante.

9.3.3 Herramientas de anotación: VEP, SnpEff y ANNOVAR

Tres herramientas dominan la anotación de variantes. **VEP** (*Variant Effect Predictor*), de Ensembl, determina el efecto de cada variante sobre genes, transcritos, secuencias proteicas y regiones reguladoras, y puede añadir frecuencias poblacionales, identificadores conocidos y predicciones de SIFT y PolyPhen (McLaren et al., 2016). **SnpEff** clasifica los efectos y les asigna las categorías de impacto de la figura 9.9; se distingue por su rapidez y por la facilidad para construir bases de datos de organismos no modelo a partir de un FASTA y un GTF (Cingolani et al., 2012). **ANNOVAR** organiza la anotación en tres tipos (basada en genes, en regiones y en

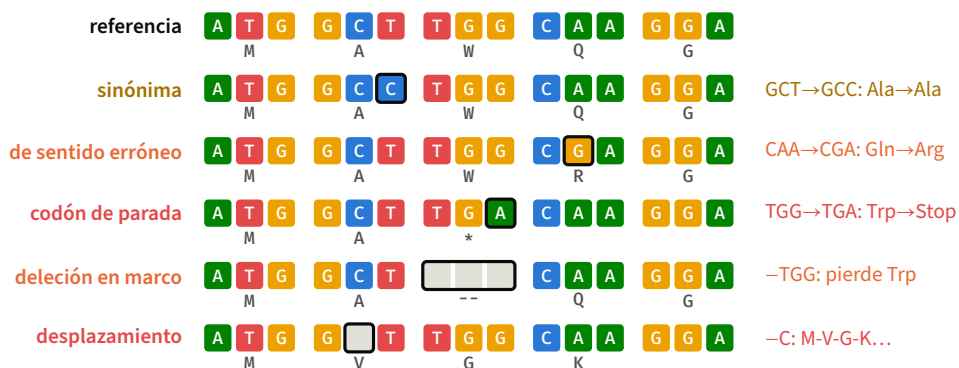


Figura 9.10. Efecto de cinco variantes sobre la secuencia codificante ATG GCT TGG CAA GGA (Met-Ala-Trp-Gln-Gly). Debajo de cada codón, el aminoácido resultante. Una sustitución en la tercera posición de GCT no cambia la alanina (synonymous_variant); CAA→CGA cambia glutamina por arginina (missense_variant); TGG→TGA crea un codón de parada prematuro (stop_gained). Una delección de tres bases elimina un aminoácido sin alterar el resto (inframe_deletion), mientras que la delección de una sola base desplaza el marco de lectura y cambia todos los aminoácidos siguientes (frameshift_variant).

filtros contra bases de datos) y fue de las primeras en permitir priorizar variantes candidatas en estudios de exoma (Wang et al., 2010). Las tres escriben su resultado dentro del propio VCF, en un campo INFO (CSQ en VEP, ANN en SnpEff) cuyos subcampos, separados por barras verticales, se describen en la cabecera. **bcftools** incluye además su propio anotador de consecuencias, **bcftools csq**, que tiene en cuenta la fase para evaluar juntas las variantes de un mismo codón (Danecek et al., 2021).

anotar.sh

```
# VEP con cache local (GRCh38) y salida VCF
vep -i final.vcf.gz --cache --offline --assembly GRCh38 \
    --everything --pick --vcf --compress_output bgzip \
    -o anotado.vcf.gz
# Alternativa: SnpEff (base de datos preconstruida)
java -Xmx8g -jar snpEff.jar GRCh38.105 final.vcf.gz \
    > anotado_snpeff.vcf
```

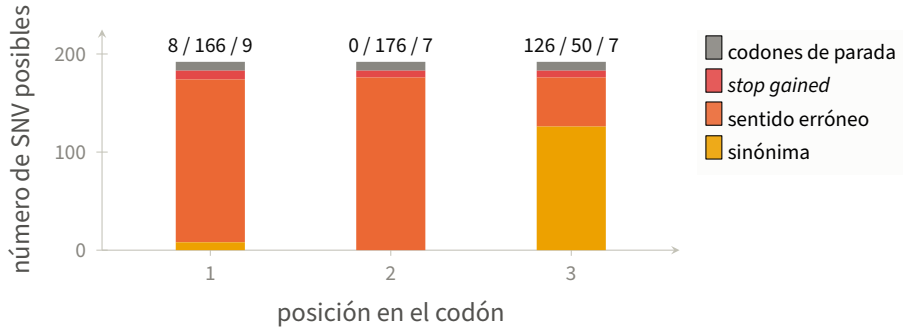


Figura 9.11. Las 576 sustituciones de una base posibles en los 64 codones del código genético estándar, clasificadas por posición y consecuencia (los números indican sinónimas / de sentido erróneo / *stop gained* sobre codones con sentido; en gris, los cambios que afectan a codones de parada, que se pierden o se mantienen). La tercera posición concentra casi todas las sinónimas, lo que refleja la degeneración del código; en la segunda, todo cambio altera el aminoácido. Calculado con la tabla de Biopython en `figuras/cap09/generar.py`.

9.3.4 Frecuencia poblacional: gnomAD

La pregunta más informativa sobre una variante suele ser la más simple: *¿se ha visto antes, y con qué frecuencia, en personas sin la enfermedad?* Una variante que porta el 5 % de la población no puede causar una enfermedad rara y grave de herencia dominante. La base de datos de referencia es gnomAD (*Genome Aggregation Database*); su versión 2 reunió 125 748 exomas y 15 708 genomas, 141 456 individuos en total (Karczewski et al., 2020). Para cada variante informa el número de alelos observados (AC), el de alelos genotipados (AN) y la frecuencia $AF = AC/AN$, desglosada por población.

Dos razonamientos cuantitativos son útiles aquí. El primero acota la frecuencia de una variante *no observada*. Si $AC = 0$ entre AN alelos, el límite superior al 95 % de la frecuencia verdadera cumple $(1 - f)^{AN} = 0,05$, de donde

$$f_{95} = 1 - 0,05^{1/AN} \approx \frac{3}{AN} \quad (9.13)$$

(la “regla del tres”). Con $AN = 250\,000$, $f_{95} \approx 1,2 \times 10^{-5}$: la ausencia en gnomAD es informativa, pero no demuestra que la variante sea inexistente en la población.

DONDE

AC, AN	Número de alelos alternativos observados y número total de alelos genotipados.
f_{95}	Frecuencia máxima compatible, al 95 %, con no haber observado la variante.

El segundo razonamiento pregunta cuál es la frecuencia *máxima* que podría tener una variante causal de una enfermedad dada. Para una enfermedad dominante de prevalencia π , penetrancia ρ , y en la que ninguna variante individual explica más de una fracción h de los casos, los portadores (con frecuencia $\approx 2f$) que enferman no pueden superar $h\pi$:

$$2f\rho \leq h\pi \implies f_{\text{máx}} = \frac{h\pi}{2\rho}. \quad (9.14)$$

DONDE

π	Prevalencia de la enfermedad en la población.
ρ	Penetrancia: probabilidad de enfermar siendo portador.
h	Máxima fracción de casos atribuible a una sola variante (heterogeneidad alélica).
$f_{\text{máx}}$	Frecuencia alélica máxima compatible con que la variante sea causal.

Ejemplo 9.6. (¿Demasiado común para ser causal?)

Una miocardiopatía hipotética de herencia dominante tiene prevalencia $\pi = 1/500$, penetrancia $\rho = 0,5$, y ninguna variante explica más del 2 % de los casos ($h = 0,02$). Entonces $f_{\text{máx}} = 0,02 \times 0,002 / (2 \times 0,5) = 4 \times 10^{-5}$. Una variante candidata observada en gnomAD con $AF = 10^{-3}$ es 25 veces más frecuente de lo que permitiría ser causal con esa arquitectura; salvo que alguno de los supuestos sea muy erróneo, se descarta. Este tipo de argumento es la base de los criterios de frecuencia de las guías ACMG que veremos al final de la sección.

gnomAD aporta además una medida de *restricción* por gen. Con un modelo mutacional que depende del contexto trinucleotídico y de la metilación, se predice cuántas variantes de pérdida de función (pLoF) *esperaríamos* ver en cada gen en ausencia de selección, y se compara con las *observadas*. El cociente observado/es-

perado, y en particular el límite superior de su intervalo de confianza (LOEUF), resume cuán intolerante es el gen a perder una copia: los genes con LOEUF bajo están fuertemente restringidos, y una variante pLoF en ellos merece más atención (Karczewski et al., 2020).

9.3.5 Predictores de efecto: SIFT, PolyPhen-2 y CADD

Para las variantes de sentido erróneo, que son la mayoría de las codificantes y las más difíciles de interpretar, existen predictores computacionales.

SIFT. Ng y Henikoff (2003) partieron de una idea evolutiva: si una posición de la proteína ha tolerado muchos aminoácidos distintos a lo largo de la evolución, un cambio en ella probablemente será tolerado. SIFT busca homólogos de la proteína, construye un alineamiento múltiple y estima, en cada posición i , la probabilidad p_{ia} de cada aminoácido a (con pseudoconteos que añaden aminoácidos plausibles no observados). La puntuación es la probabilidad normalizada por la del aminoácido más frecuente,

$$S_{ia} = \frac{p_{ia}}{\max_{a'} p_{ia'}}, \tag{9.15}$$

y se predice “deletérea” la sustitución si $S_{ia} < 0,05$.

DONDE	
p_{ia}	Probabilidad estimada del aminoácido a en la posición i del alineamiento de homólogos.
S_{ia}	Puntuación SIFT: 1 para el aminoácido más frecuente; cerca de 0 para los nunca vistos.

PolyPhen-2. Adzhubei et al. (2010) combinan conservación con rasgos estructurales (accesibilidad al solvente, cambios en volumen e hidrofobicidad, proximidad a sitios funcionales anotados) en un clasificador bayesiano ingenuo entrenado con variantes causales de enfermedad y variantes neutrales. Devuelve una probabilidad de que la variante sea dañina, que se resume en tres clases: benigna, posiblemente dañina y probablemente dañina.

CADD. Los dos anteriores solo evalúan cambios de aminoácido. Kircher et al. (2014) propusieron un marco general, aplicable a cualquier SNV o *indel* del genoma, con una idea ingeniosa para obtener datos de entrenamiento sin etiquetas clínicas. Compararon variantes *observadas*, fijadas en el linaje humano desde el ancestro

común con el chimpancé y, por tanto, filtradas por la selección natural, con variantes *simuladas* con el mismo modelo mutacional pero sin selección. Un clasificador (una máquina de vectores de soporte en la versión original; regresión logística en las posteriores) aprende a distinguirlas a partir de decenas de anotaciones: conservación, efecto sobre la proteína, marcas reguladoras, etc. Las variantes que “parecen simuladas” son las que la selección habría eliminado: probablemente deletéreas. La puntuación bruta se convierte en una escala Phred del *rango*:

$$C = -10 \log_{10} \frac{r}{N}, \quad (9.16)$$

DONDE

r	Posición de la variante en el ordenamiento de todas las SNV posibles del genoma, de más a menos deletérea.
N	Número total de SNV posibles en el genoma de referencia (unos $8,6 \times 10^9$).
C	Puntuación CADD escalada (<i>PHRED-like</i>).

de modo que $C \geq 10$ corresponde al 10 % más deletéreo de todas las sustituciones posibles, $C \geq 20$ al 1 % y $C \geq 30$ al 0,1 %. Rentzsch et al. (2019) describen las versiones posteriores, que incorporan más anotaciones, soporte para GRCh38 y un servicio web.

⚠ Los predictores no son diagnósticos

SIFT, PolyPhen-2 y CADD están correlacionados entre sí (usan información parecida, sobre todo conservación) y su exactitud en variantes individuales es limitada. Una puntuación “dañina” es evidencia de apoyo, nunca prueba de patogenicidad; y dos predictores que coinciden no son dos evidencias independientes. Las guías ACMG, precisamente por eso, les conceden el peso más bajo.

9.3.6 Del catálogo al diagnóstico: ClinVar y las guías ACMG/AMP

ClinVar es el archivo público de las interpretaciones clínicas de variantes: laboratorios de diagnóstico, grupos de expertos e investigadores depositan sus clasificaciones con la evidencia que las respalda, y la base de datos muestra las discrepancias entre remitentes y un nivel de revisión (las “estrellas”) que va desde una sola clasificación sin criterios hasta la revisión por un panel de expertos (Landrum et al., 2018).

Tabla 9.1. Reglas de combinación de evidencias de las guías ACMG/AMP (Richards et al., 2015). MF: muy fuerte; F: fuerte; M: moderada; A: de apoyo. Si se cumplen a la vez criterios patogénicos y benignos contradictorios, o no se alcanza ninguna regla, la variante es de significado incierto (VUS).

Clasificación	Combinaciones suficientes
Patogénica	1 MF + (≥ 1 F, o ≥ 2 M, o 1 M + 1 A, o ≥ 2 A); ≥ 2 F; 1 F + (≥ 3 M, o 2 M + ≥ 2 A, o 1 M + ≥ 4 A)
Probablemente patogénica	1 MF + 1 M; 1 F + 1-2 M; 1 F + ≥ 2 A; ≥ 3 M; 2 M + ≥ 2 A; 1 M + ≥ 4 A
Benigna	BA1 (independiente); ≥ 2 criterios benignos fuertes
Probablemente benigna	1 benigno fuerte + 1 de apoyo; ≥ 2 benignos de apoyo

Encontrar una variante en ClinVar como “patogénica” con varias estrellas es una evidencia fuerte; encontrarla con una sola estrella y clasificaciones contradictorias, no tanto.

Para clasificar una variante de manera reproducible, el American College of Medical Genetics and Genomics y la Association for Molecular Pathology publicaron unas guías que hoy son el estándar internacional (Richards et al., 2015). Proponen cinco categorías: patogénica, probablemente patogénica, de significado incierto (VUS), probablemente benigna y benigna. Cada tipo de evidencia recibe un código y una fuerza: *muy fuerte* (PVS1: variante nula, como un *frameshift* o un *stop gained*, en un gen donde la pérdida de función causa la enfermedad), *fuerte* (PS1-PS4; p. ej., PS2: aparición *de novo* confirmada), *moderada* (PM1-PM6; p. ej., PM2: ausente o extremadamente rara en controles poblacionales), y *de apoyo* (PP1-PP5; p. ej., PP3: múltiples predictores computacionales coinciden en un efecto dañino). Del lado benigno hay criterios independientes (BA1: frecuencia alélica mayor del 5 %), fuertes (BS1-BS4; p. ej., BS1: frecuencia mayor de lo esperado para la enfermedad, como en el ejemplo 9.6) y de apoyo (BP1-BP7). La tabla 9.1 resume las reglas de combinación.

Ejemplo 9.7. (Clasificar una variante)

En un niño con un trastorno del neurodesarrollo, la secuenciación del trío revela una variante *stop_gained* en heterocigosis, ausente en ambos padres (paternidad confirmada) y ausente en gnomAD. El gen tiene LOEUF muy bajo y la haploinsuficiencia es un mecanismo establecido de la enfermedad. Criterios: PVS1 (variante nula en un gen donde la pérdida de función es el



Figura 9.12. El embudo de priorización de variantes en un caso de enfermedad rara. Las dos primeras etapas usan las cifras por genoma del Proyecto 1000 Genomas (Auton et al., 2015); las siguientes son órdenes de magnitud ilustrativos, que dependen del diseño (individuo o trío, exoma o genoma) y de la población. Cada estrechamiento es una hipótesis explícita (“la variante causal es rara”, “es codificante”, “sigue un modelo dominante”) y debe revisarse si el análisis no encuentra candidatas.

mecanismo, muy fuerte), PS2 (*de novo* confirmada, fuerte) y PM2 (ausente en controles, moderada). La regla “1 MF + ≥ 1 F” basta para clasificarla como **patogénica**. Si, en cambio, fuese una variante de sentido erróneo heredada de un progenitor sano, con CADD 24 y ausente de gnomAD, solo tendríamos PM2 y PP3: ninguna regla se cumple y la variante quedaría como **VUS**, a la espera de más evidencia (segregación familiar, estudios funcionales, nuevos casos).

9.3.7 El embudo de priorización

En la práctica, todo lo anterior se encadena en un *embudo* que reduce millones de variantes a unas pocas candidatas (figura 9.12). Los números de las primeras etapas proceden del Proyecto 1000 Genomas: un genoma típico contiene entre 149 y 182 sitios con variantes que truncan proteínas y entre 10 000 y 12 000 sitios con variantes que alteran la secuencia peptídica (Auton et al., 2015). Cada filtro posterior, basado en frecuencia, modo de herencia, fenotipo e interpretación, debe aplicarse con conciencia de lo que puede perder: una variante causal en un gen aún no asociado a enfermedad, una variante no codificante o una variante estructural atravesarán intactas este embudo sin ser vistas.

En resumen

- Las variantes se clasifican por escala: SNV, MNV e *indels* se leen en la columna del *pileup*; las variantes estructurales y CNV, en la geometría de los alineamientos (profundidad, pares anómalos, lecturas partidas).
- Con errores independientes, $P(D | G) = \prod_i [\frac{1}{2}P(b_i | a_1) + \frac{1}{2}P(b_i | a_2)]$, con $P(b | a) = 1 - \varepsilon$ o $\varepsilon/3$ y $\varepsilon = 10^{-Q/10}$. Los PL expresan estas verosimilitudes en escala Phred relativa al mejor genotipo.
- La distribución *a priori* $P(RA) = \theta$, $P(AA) = \theta/2$ procede del espectro de frecuencias neutral. La posterior da el genotipo, $QUAL = -10 \log_{10} P(RR | D)$ y $GQ = -10 \log_{10} [1 - P(\hat{G} | D)]$. A baja profundidad la distribución *a priori* y el muestreo de alelos dominan: un genotipado individual fiable pide unas 15–20 lecturas.
- BAQ reduce la calidad de las bases posiblemente mal alineadas; la llamada conjunta estima la frecuencia alélica por EM y la usa como distribución *a priori* de Hardy-Weinberg. HaplotypeCaller reensambla haplotipos localmente y DeepVariant aprende el modelo de error con una red convolucional.
- El flujo `mpileup` $\rightarrow$ `call -mv` $\rightarrow$ `norm` $\rightarrow$ `filter` produce un VCF limpio. La normalización (parsimonia y alineamiento a la izquierda) da una representación única a cada *indel* y es imprescindible antes de comparar.
- La razón ti/tv ($\approx 2,0$ – $2,1$ en genomas humanos, $0,5$ para errores aleatorios) estima la fracción de falsos positivos. La evaluación rigurosa usa Genome in a Bottle, `hap.py` y las métricas de precisión, sensibilidad y F_1 , estratificadas por tipo de variante y región.
- La anotación asigna consecuencias con términos de la Sequence Ontology (VEP, SnpEff, ANNOVAR), consulta frecuencias en gnomAD (regla del tres, frecuencia máxima compatible con la enfermedad), predictores (SIFT, PolyPhen-2, CADD) y ClinVar, y combina la evidencia con las reglas ACMG/AMP.

Ejercicios

Dificultad: ● ● ● básica ● ● ● intermedia ● ● ● avanzada

- 9.1** ● ● ● Una columna tiene 8 lecturas de Q30: 7 C (referencia) y 1 T. Calcule $\log_{10} L$ para CC, CT y TT con la ecuación (9.3), los PL correspondientes y el genotipo de máxima posterior con $\theta = 10^{-3}$. ¿Cuántas lecturas T harían falta para que el genotipo inferido fuese CT?

- 9.2 ●●● Escriba en formato VCF (con base de anclaje) una inserción de AG inmediatamente después de la posición 100, donde la referencia tiene una T. Escriba después un sitio con dos alelos alternativos (G y T) e indique cuántos valores debe tener su campo PL y en qué orden.
- 9.3 ●●● Normalice a mano, con la referencia CTTTTAG (posiciones 1–8), la delección POS=5, REF=TT, ALT=T. Compruebe su resultado con la función `normalizar`.
- 9.4 ●●● Demuestre que, para un heterocigoto, $P(b \mid RA) = \frac{1}{2} - \varepsilon/3$ tanto si $b = R$ como si $b = A$, y que para $b \notin \{R, A\}$ vale $\varepsilon/3$. Use este resultado para explicar por qué, a partir de cierta profundidad, $\log_{10} L(RR) - \log_{10} L(RA)$ decrece linealmente con el número de lecturas alternativas.
- 9.5 ●●● Un conjunto de variantes de exoma tiene $t_i/t_v = 2,4$. Si las variantes verdaderas de exoma tienen $t_i/t_v = 3,0$, estime con la ecuación (9.11) la fracción de falsos positivos. Discuta dos razones por las que esta estimación podría estar sesgada.
- 9.6 ●●● Descargue un BAM de una región de HG002 (por ejemplo, un cromosoma pequeño) y la verdad de GIAB correspondiente. Ejecute el flujo de la figura 9.6 con y sin BAQ (-B) y compare precisión y sensibilidad para SNV e *indels* con `hap.py`. ¿Dónde se concentran las diferencias?
- 9.7 ●●● Implemente el algoritmo EM de la ecuación (9.10) y reproduzca el ejemplo 9.3. Luego simule 200 muestras a $4\times$ con una variante de frecuencia 0,05 y compare la sensibilidad de la llamada individual y la conjunta en función de la frecuencia. ¿Qué ocurre si la mitad de las muestras proceden de una población en la que la variante no existe?
- 9.8 ●●● Reproduzca la figura 9.11 y extiéndala: pondere cada una de las 576 sustituciones por una tasa de mutación en la que las transiciones son el doble de probables que cada transversión. ¿Cómo cambia la fracción esperada de variantes sinónimas? Relacione el resultado con el cálculo de d_N/d_S .

Referencias del capítulo

- Adzhubei, I. A., Schmidt, S., Peshkin, L., Ramensky, V. E., Gerasimova, A., Bork, P., et al. (2010). A method and server for predicting damaging missense mutations. *Nature Methods*, 7(4), 248-249. <https://doi.org/10.1038/nmeth0410-248>
- Alkan, C., Coe, B. P., & Eichler, E. E. (2011). Genome structural variation discovery and genotyping. *Nature Reviews Genetics*, 12(5), 363-376. <https://doi.org/10.1038/nrg2958>
- Auton, A., Abecasis, G. R., Altshuler, D. M., Durbin, R. M., et al. (2015). A global reference for human genetic variation. *Nature*, 526(7571), 68-74. <https://doi.org/10.1038/nature15393>
- Cingolani, P., Platts, A., Wang, L. L., Coon, M., Nguyen, T., Wang, L., et al. (2012). A program for annotating and predicting the effects of single nucleotide polymorphisms, SnpEff. *Fly*, 6(2), 80-92. <https://doi.org/10.4161/fly.19695>
- Danecek, P., Auton, A., Abecasis, G., Albers, C. A., Banks, E., DePristo, M. A., et al. (2011). The variant call format and VCFtools. *Bioinformatics*, 27(15), 2156-2158. <https://doi.org/10.1093/bioinformatics/btr330>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., et al. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), giab008. <https://doi.org/10.1093/gigascience/giab008>
- den Dunnen, J. T., Dalgleish, R., Maglott, D. R., Hart, R. K., Greenblatt, M. S., et al. (2016). HGVS Recommendations for the Description of Sequence Variants: 2016 Update. *Human Mutation*, 37(6), 564-569. <https://doi.org/10.1002/humu.22981>
- DePristo, M. A., Banks, E., Poplin, R., Garimella, K. V., Maguire, J. R., Hartl, C., et al. (2011). A framework for variation discovery and genotyping using next-generation DNA sequencing data. *Nature Genetics*, 43(5), 491-498. <https://doi.org/10.1038/ng.806>
- Eilbeck, K., Lewis, S. E., Mungall, C. J., Yandell, M., Stein, L., Durbin, R., et al. (2005). The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology*, 6(5), R44. <https://doi.org/10.1186/gb-2005-6-5-r44>
- Karczewski, K. J., Francioli, L. C., Tiao, G., Cummings, B. B., Alföldi, J., Wang, Q., et al. (2020). The mutational constraint spectrum quantified from variation in 141,456 humans. *Nature*, 581(7809), 434-443. <https://doi.org/10.1038/s41586-020-2308-7>
- Kircher, M., Witten, D. M., Jain, P., O’Roak, B. J., Cooper, G. M., & Shendure, J. (2014). A general framework for estimating the relative pathogenicity of human genetic variants. *Nature Genetics*, 46(3), 310-315. <https://doi.org/10.1038/ng.2892>
- Krusche, P., Trigg, L., Boutros, P. C., Mason, C. E., De La Vega, F. M., et al. (2019). Best practices for benchmarking germline small-variant calls in human genomes. *Nature Biotechnology*, 37(5), 555-560. <https://doi.org/10.1038/s41587-019-0054-x>
- Landrum, M. J., Lee, J. M., Benson, M., Brown, G. R., Chao, C., Chitipiralla, S., et al. (2018). ClinVar: improving access to variant interpretations and supporting evidence. *Nucleic Acids Research*, 46(D1), D1062-D1067. <https://doi.org/10.1093/nar/gkx1153>

- Li, H. (2011a). Improving SNP discovery by base alignment quality. *Bioinformatics*, 27(8), 1157-1158. <https://doi.org/10.1093/bioinformatics/btr076>
- Li, H. (2011b). A statistical framework for SNP calling, mutation discovery, association mapping and population genetical parameter estimation from sequencing data. *Bioinformatics*, 27(21), 2987-2993. <https://doi.org/10.1093/bioinformatics/btr509>
- Li, H. (2014). Toward better understanding of artifacts in variant calling from high-coverage samples. *Bioinformatics*, 30(20), 2843-2851. <https://doi.org/10.1093/bioinformatics/btu356>
- McKenna, A., Hanna, M., Banks, E., Sivachenko, A., Cibulskis, K., Kernytsky, A., et al. (2010). The Genome Analysis Toolkit: A MapReduce framework for analyzing next-generation DNA sequencing data. *Genome Research*, 20(9), 1297-1303. <https://doi.org/10.1101/gr.107524.110>
- McLaren, W., Gil, L., Hunt, S. E., Riat, H. S., Ritchie, G. R. S., Thormann, A., et al. (2016). The Ensembl Variant Effect Predictor. *Genome Biology*, 17(1), 122. <https://doi.org/10.1186/s13059-016-0974-4>
- Ng, P. C., & Henikoff, S. (2003). SIFT: predicting amino acid changes that affect protein function. *Nucleic Acids Research*, 31(13), 3812-3814. <https://doi.org/10.1093/nar/gkg509>
- Nielsen, R., Paul, J. S., Albrechtsen, A., & Song, Y. S. (2011). Genotype and SNP calling from next-generation sequencing data. *Nature Reviews Genetics*, 12(6), 443-451. <https://doi.org/10.1038/nrg2986>
- Poplin, R., Chang, P.-C., Alexander, D., Schwartz, S., Colthurst, T., Ku, A., et al. (2018). A universal SNP and small-indel variant caller using deep neural networks. *Nature Biotechnology*, 36(10), 983-987. <https://doi.org/10.1038/nbt.4235>
- Poplin, R., Ruano-Rubio, V., DePristo, M. A., Fennell, T. J., Carneiro, M. O., Van der Auwera, G. A., et al. (2017). *Scaling accurate genetic variant discovery to tens of thousands of samples*. bioRxiv. <https://doi.org/10.1101/201178>
- Rentzsch, P., Witten, D., Cooper, G. M., Shendure, J., & Kircher, M. (2019). CADD: predicting the deleteriousness of variants throughout the human genome. *Nucleic Acids Research*, 47(D1), D886-D894. <https://doi.org/10.1093/nar/gky1016>
- Richards, S., Aziz, N., Bale, S., Bick, D., Das, S., Gastier-Foster, J., et al. (2015). Standards and guidelines for the interpretation of sequence variants: a joint consensus recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology. *Genetics in Medicine*, 17(5), 405-424. <https://doi.org/10.1038/gim.2015.30>
- Tan, A., Abecasis, G. R., & Kang, H. M. (2015). Unified representation of genetic variants. *Bioinformatics*, 31(13), 2202-2204. <https://doi.org/10.1093/bioinformatics/btv112>
- Wang, K., Li, M., & Hakonarson, H. (2010). ANNOVAR: functional annotation of genetic variants from high-throughput sequencing data. *Nucleic Acids Research*, 38(16), e164. <https://doi.org/10.1093/nar/gkq603>

- Zook, J. M., Chapman, B., Wang, J., Mittelman, D., Hofmann, O., Hide, W., et al. (2014). Integrating human sequence data sets provides a resource of benchmark SNP and indel genotype calls. *Nature Biotechnology*, 32(3), 246-251. <https://doi.org/10.1038/nbt.2835>
- Zook, J. M., McDaniel, J., Olson, N. D., Wagner, J., Parikh, H., Heaton, H., et al. (2019). An open resource for accurately benchmarking small variant and reference calls. *Nature Biotechnology*, 37(5), 561-566. <https://doi.org/10.1038/s41587-019-0074-6>

PARTE

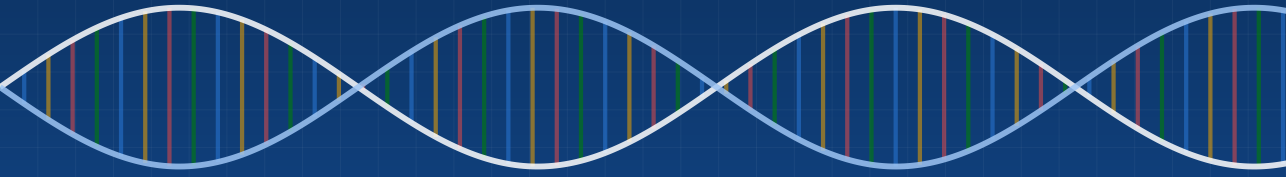
IV

Genomas en acción

Poblaciones, transcriptomas, epigenomas y comunidades

Un genoma no es un texto estático: varía entre individuos, se expresa de forma distinta en cada célula, se regula químicamente y convive con miles de genomas microbianos. Aquí aprendemos a medir y a modelar esa dinámica.

Genómica de poblaciones y GWAS



Me resisto a entrometerme en una discusión sobre asuntos de los que no tengo conocimiento experto, y habría esperado que el punto, muy sencillo, que deseo señalar fuera familiar para los biólogos.

— Hardy (1908)

● Al terminar este capítulo usted podrá

- Calcular frecuencias alélicas y genotípicas, derivar el equilibrio de Hardy-Weinberg y contrastarlo con la prueba χ^2 y con la prueba exacta, interpretando el coeficiente de endogamia F .
- Formular el modelo de Wright-Fisher como una cadena de Márkov binomial y deducir de él la pérdida de heterocigosidad, las probabilidades y tiempos de fijación, el tamaño efectivo y la aproximación por difusión de Kimura.
- Medir el desequilibrio de ligamiento con D , D' y r^2 , explicar su decaimiento por recombinación y reconocer bloques de haplotipos en un mapa de calor.
- Detectar estructura poblacional con el análisis de componentes principales de genotipos y cuantificarla con F_{ST} .

- Diseñar, ejecutar e interpretar un estudio de asociación del genoma completo (GWAS): modelo aditivo, control de la estratificación (λ_{GC} , componentes principales, modelos mixtos), corrección por comparaciones múltiples y lectura de gráficos Manhattan y QQ.

En el capítulo 9 detectamos variantes en un individuo y las registramos en un VCF. Ahora examinaremos esas variantes *en muchos individuos*. Cada posición polimórfica tiene una frecuencia que puede cambiar entre generaciones por deriva, selección, migración y mutación. La genética de poblaciones estudia esos cambios. Los datos genómicos añaden otro aspecto: las variantes próximas en un cromosoma tienden a heredarse juntas y, por ello, no son independientes.

Estas dos ideas, que las frecuencias derivan y que los alelos vecinos se heredan en bloque, son las que hacen posible (y a la vez delicado) el estudio de asociación del genoma completo, la herramienta con la que en los últimos veinte años se han encontrado decenas de miles de asociaciones entre variantes genéticas y rasgos humanos. El capítulo avanza en tres pasos. Primero construiremos el modelo nulo de la genética de poblaciones, el equilibrio de Hardy-Weinberg, y el modelo más sencillo de cambio, la deriva de Wright-Fisher (sección 10.1). Luego estudiaremos cómo se correlacionan los alelos a lo largo del cromosoma y entre poblaciones: desequilibrio de ligamiento y análisis de componentes principales (sección 10.2). Por último, reuniremos todo en el diseño y la interpretación de un GWAS (sección 10.3).

10.1 Hardy-Weinberg y deriva génica

 Colab · 10.1 · Hardy-Weinberg y deriva génica

 [abrir notebook](#)

Una bolsa de canicas que se rellena a ciegas

Imagine una bolsa con cien canicas, cincuenta azules y cincuenta naranjas. Cada generación se construye así: se saca una canica al azar, se anota su color, se devuelve a la bolsa, y se repite cien veces; la nueva bolsa contendrá exactamente los colores anotados. Nadie prefiere un color, y sin embargo la proporción no se queda quieta: una generación puede salir con 54 azules, la siguiente con 49, luego con 57... Tarde o temprano, por pura acumulación de pequeños accidentes, un color desaparece y la bolsa queda de un solo

color para siempre. Si la bolsa tuviera diez mil canicas, los accidentes serían proporcionalmente más pequeños y el proceso mucho más lento, pero el destino final sería el mismo. Esta bolsa es la población; las canicas son las copias de un gen; el muestreo es la reproducción. La sección entera trata de responder con precisión a dos preguntas: ¿qué ocurre en *una* extracción?, y ¿qué ocurre al repetirla indefinidamente?

10.1.1 Frecuencias alélicas y genotípicas

Consideremos un locus bialélico, con alelos A y a , en una muestra de n individuos diploides. Cada individuo tiene uno de tres genotipos, AA , Aa o aa , con recuentos n_{AA} , n_{Aa} y n_{aa} . En los archivos VCF del capítulo 9 estos genotipos aparecen como $0/0$, $0/1$ y $1/1$, y en los estudios de asociación se codifican como el número de copias del alelo alternativo, $g \in \{0, 1, 2\}$.

Definición 10.1. (Frecuencias alélicas y genotípicas)

Las frecuencias genotípicas son $P_{AA} = n_{AA}/n$, $P_{Aa} = n_{Aa}/n$ y $P_{aa} = n_{aa}/n$. Como cada individuo aporta dos copias del gen, la frecuencia del alelo A es

$$p = \frac{2n_{AA} + n_{Aa}}{2n} = P_{AA} + \frac{1}{2}P_{Aa}, \quad q = 1 - p. \quad (10.1)$$

La *heterocigosidad observada* es $H_{\text{obs}} = P_{Aa}$, y la *heterocigosidad esperada* (o diversidad génica) es $H = 2pq$, la probabilidad de que dos copias tomadas al azar de la población sean distintas.

DONDE

n	Número de individuos diploides muestreados ($2n$ copias del gen).
n_{AA} , n_{Aa} , n_{aa}	Número de individuos con cada genotipo.
p , q	Frecuencias de los alelos A y a ; en GWAS se suele reportar la del alelo menos frecuente (MAF, <i>minor allele frequency</i>).
H	Heterocigosidad esperada; mide la diversidad del locus y alcanza su máximo, $1/2$, cuando $p = q = 1/2$.

Observe la asimetría: las frecuencias genotípicas determinan las alélicas, pero no al revés. Una población con $p = 1/2$ podría estar formada solo por heterocigotos, o solo por homocigotos de ambos tipos a partes iguales. Hace falta un *modelo* de cómo

se forman los genotipos para pasar de unas a otras, y ese modelo es el principio de Hardy-Weinberg.

10.1.2 El principio de Hardy-Weinberg

Supongamos que la población es muy grande, que los apareamientos ocurren al azar con respecto al locus, y que no hay selección, mutación ni migración. Entonces cada individuo de la generación siguiente se forma uniendo dos gametos tomados independientemente de un mismo “fondo” de gametos en el que el alelo A tiene frecuencia p . La probabilidad de que ambos lleven A es $p \cdot p$; la de que ambos lleven a , $q \cdot q$; y la de un heterocigoto es $pq + qp$, porque el alelo A puede venir del padre o de la madre.

Teorema 10.1. (Equilibrio de Hardy-Weinberg)

Bajo apareamiento aleatorio en una población infinita, sin selección, mutación ni migración, las frecuencias genotípicas tras una sola generación son

$$P_{AA} = p^2, \quad P_{Aa} = 2pq, \quad P_{aa} = q^2, \quad (10.2)$$

y la frecuencia alélica no cambia: $p' = p^2 + \frac{1}{2} 2pq = p(p + q) = p$. Por lo tanto, las proporciones de la ecuación (10.2) se mantienen indefinidamente.

DONDE

$p^2, 2pq, q^2$	Términos del desarrollo de $(p + q)^2$: probabilidades de los tres genotipos al unir dos gametos independientes.
p'	Frecuencia del alelo A en la generación siguiente.

Dos consecuencias merecen subrayarse. La primera es que el equilibrio se alcanza en *una* generación, cualquiera que sea el punto de partida: una población formada solo por homocigotos AA y aa en proporción 1:1 produce, tras un apareamiento aleatorio, las proporciones $1/4 : 1/2 : 1/4$. La segunda, y la que motivó la carta de Hardy, es que la herencia mendeliana *conserva* la variación. Un alelo dominante no aumenta de frecuencia por ser dominante, y un recesivo no desaparece por quedar oculto en los heterocigotos. La figura 10.1 muestra las tres curvas y su representación geométrica en el triángulo de De Finetti.

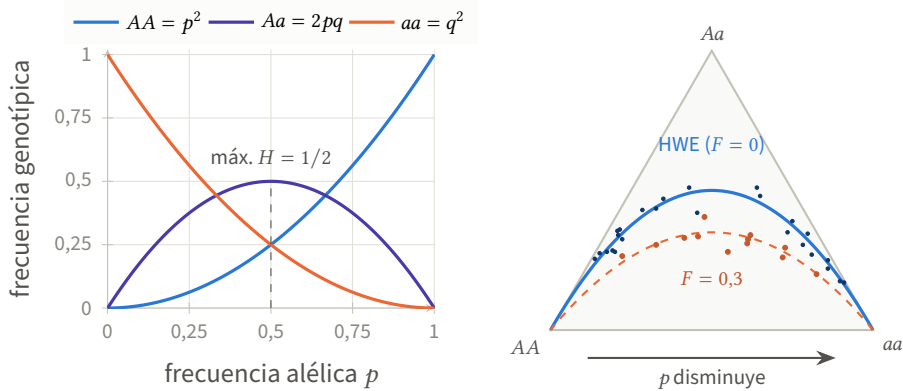


Figura 10.1. El equilibrio de Hardy-Weinberg desde dos ángulos. **Izquierda:** frecuencias de los tres genotipos en función de la frecuencia alélica. La heterocigosidad $2pq$ es máxima en $p = 1/2$ y, cuando un alelo es raro, casi todas sus copias viven en heterocigotos: con $q = 0,01$, el 99 % de los alelos a está en individuos Aa . **Derecha:** triángulo de De Finetti, en el que cada población es un punto cuyas distancias a los lados son sus tres frecuencias genotípicas. Las poblaciones en equilibrio caen sobre la parábola azul; los puntos oscuros son 25 muestras simuladas de 200 individuos de poblaciones en equilibrio, que se dispersan alrededor de ella solo por azar muestral. Los puntos naranjas son muestras de poblaciones endogámicas con $F = 0,3$ (ecuación (10.4)), que caen sistemáticamente por debajo: les faltan heterocigotos. Figura generada con `figuras/cap10/generar.py`.

Un matemático puro responde a los biólogos

En 1908, varios biólogos británicos discutían si un carácter dominante, como la braquidactilia, debía volverse inevitablemente más frecuente hasta alcanzar la proporción mendeliana 3:1. Godfrey H. Hardy, uno de los grandes matemáticos del siglo xx y orgulloso de la inutilidad práctica de su obra, escribió a la revista *Science* una carta de apenas dos páginas mostrando, con álgebra de bachillerato, que cualquier proporción $p^2 : 2pq : q^2$ es estable y que no hay tendencia alguna hacia 3:1 (Hardy, 1908). El epígrafe de este capítulo es la primera frase de esa carta. El médico alemán Wilhelm Weinberg llegó de forma independiente al mismo resultado ese mismo año, y por eso el principio lleva ambos nombres.

10.1.3 Contrastar el equilibrio: prueba χ^2 y prueba exacta

En la práctica, el principio se usa al revés: dados los recuentos observados, preguntamos si son compatibles con el equilibrio. La prueba clásica compara recuentos observados O_k y esperados E_k bajo la ecuación (10.2), con p estimada por la ecuación (10.1):

$$X^2 = \sum_{k \in \{AA, Aa, aa\}} \frac{(O_k - E_k)^2}{E_k}, \quad E_{AA} = n\hat{p}^2, \quad E_{Aa} = 2n\hat{p}\hat{q}, \quad E_{aa} = n\hat{q}^2. \quad (10.3)$$

DONDE

O_k, E_k	Recuento observado y esperado del genotipo k .
$\hat{p}, \hat{q}$	Frecuencias alélicas estimadas en la propia muestra.
X^2	Estadístico de Pearson; bajo el equilibrio sigue aproximadamente una χ^2 con un grado de libertad.

¿Por qué un solo grado de libertad, si hay tres categorías? Porque los tres recuentos suman n (se pierde uno) y porque p se estima de los mismos datos (se pierde otro): $3 - 1 - 1 = 1$.

Endogamia. La desviación más común del equilibrio es un déficit de heterocigotos. Ocurre cuando los individuos que se aparean están emparentados, o cuando la “población” es en realidad una mezcla de subpoblaciones con frecuencias distintas (el efecto Wahlund). Ambos casos se describen con un único parámetro, el *coeficiente de endogamia* F , la probabilidad de que las dos copias de un individuo sean idénticas por descendencia:

$$P_{AA} = p^2 + Fpq, \quad P_{Aa} = 2pq(1 - F), \quad P_{aa} = q^2 + Fpq, \quad \hat{F} = 1 - \frac{H_{\text{obs}}}{2\hat{p}\hat{q}}. \quad (10.4)$$

DONDE

F	Coefficiente de endogamia: 0 en equilibrio, 1 si todos los individuos son homocigotos. Valores negativos indican exceso de heterocigotos.
H_{obs}	Proporción observada de heterocigotos.

Ejemplo 10.1. (Dos muestras de mil personas)

En una muestra de $n = 1000$ personas se observan 298 AA, 489 Aa y 213 aa. Entonces $\hat{p} = (2 \cdot 298 + 489)/2000 = 0,5425$, y los esperados son 294,3, 496,4 y 209,3. El estadístico vale $X^2 = 0,221$, con $p = 0,64$: nada que objetar al equilibrio, y $\hat{F} = 0,015$.

En otra muestra se observan 350, 380 y 270. Ahora $\hat{p} = 0,54$ y los esperados son 291,6, 496,8 y 211,6. Faltan 117 heterocigotos: $X^2 = 55,3$, $p \approx 10^{-13}$ y $\hat{F} = 1 - 380/496,8 = 0,235$. Un valor así en humanos exogámicos no suele indicar endogamia real, sino un problema técnico: por ejemplo, un ensayo de genotipado que no detecta bien uno de los alelos y “convierte” heterocigotos en homocigotos.

La aproximación χ^2 falla cuando algún recuento esperado es pequeño, y eso ocurre precisamente con las variantes raras, cada vez más importantes en la era de la secuenciación. Wigginton et al. (2005) propusieron un algoritmo rápido para la *prueba exacta*. Condicionando en el número de copias del alelo raro, $n_a = 2n_{aa} + n_{Aa}$, la probabilidad de observar exactamente n_{Aa} heterocigotos bajo el equilibrio es

$$P(N_{Aa} = n_{Aa} \mid n, n_a) = \frac{2^{n_{Aa}} n!}{n_{AA}! n_{Aa}! n_{aa}!} \cdot \frac{n_A! n_a!}{(2n)!}, \quad (10.5)$$

y el valor p es la suma de las probabilidades de todas las configuraciones tan o menos probables que la observada. En lugar de evaluar factoriales enormes, el algoritmo parte de la configuración más probable y recorre las demás con la razón entre términos consecutivos,

$$\frac{P(n_{Aa} - 2)}{P(n_{Aa})} = \frac{n_{Aa}(n_{Aa} - 1)}{4(n_{aa} + 1)(n_{AA} + 1)}, \quad (10.6)$$

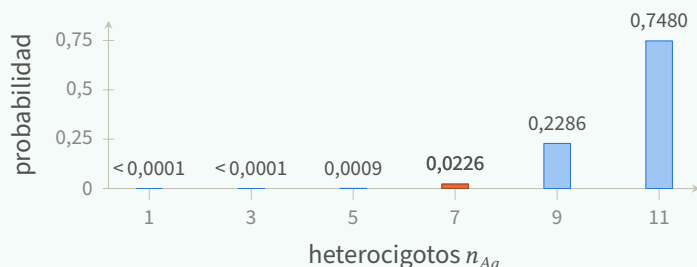
que se obtiene de la ecuación (10.5) al convertir dos heterocigotos en un homocigoto de cada tipo.

DONDE

n_A, n_a	Número de copias de cada alelo en la muestra ($n_A + n_a = 2n$); fijos en el condicionamiento.
N_{Aa}	Número aleatorio de heterocigotos; tiene la misma paridad que n_a .

Ejemplo 10.2. (Cuando la aproximación χ^2 engaña)

Entre 100 personas se observan 91 AA , 7 Aa y 2 aa ($\hat{q} = 0,055$). Los esperados son 89,3, 10,4 y 0,30: el recuento esperado de aa es diminuto, y los dos homocigotos observados pesan enormemente en $X^2 = 10,67$, que da $p = 0,0011$. La prueba exacta distribuye las 11 copias del alelo raro de todas las formas posibles:



El valor p exacto es la suma de las barras con probabilidad menor o igual que la observada (naranja): $0,0226 + 0,0009 + \dots = 0,0235$, casi veinte veces mayor que el de χ^2 . Con un umbral típico de control de calidad en GWAS ($p < 10^{-6}$), ninguna de las dos pruebas descartaría la variante; pero en un análisis de miles de variantes raras, usar χ^2 produciría un exceso sistemático de falsos rechazos.

hwe_exacta.py

```
import numpy as np
```

```
def hwe_exact(n_het, n_hom1, n_hom2):
    """Valor p exacto de HWE (Wigginton et al., 2005)."""
    n = n_het + n_hom1 + n_hom2
    n_rare = 2 * min(n_hom1, n_hom2) + n_het
    P = np.zeros(n_rare + 1)
    h = n_rare * (2 * n - n_rare) // (2 * n)      # moda aproximada
    h += (h % 2) != (n_rare % 2)                  # misma paridad
    P[h] = 1.0
    r = (n_rare - h) // 2; c = n - h - r
    for k in range(h, 1, -2):                      # hacia menos het.
        P[k - 2] = P[k] * k * (k - 1) / (4 * (r + 1) * (c + 1))
        r += 1; c += 1
    r = (n_rare - h) // 2; c = n - h - r
```

```

for k in range(h, n_rare - 1, 2):          # hacia más het.
    P[k + 2] = P[k] * 4 * r * c / ((k + 2) * (k + 1))
    r -= 1; c -= 1
P /= P.sum()
return P[P <= P[n_het] * (1 + 1e-9)].sum()

print(hwe_exact(7, 91, 2))                # 0.0235

```

⚠ El equilibrio como control de calidad

En los estudios de asociación, la prueba de Hardy-Weinberg se aplica sobre todo como *filtro técnico*: en los controles sanos, una desviación extrema suele delatar errores de genotipado. Pero hay que aplicarla con cautela en los casos: una variante fuertemente asociada con la enfermedad puede desviarse del equilibrio en los casos *precisamente porque* es causal. Por eso herramientas como PLINK permiten aplicar el filtro solo a los controles, con umbrales muy estrictos (del orden de 10^{-6} a 10^{-10}).

10.1.4 Deriva génica: el modelo de Wright-Fisher

El teorema de Hardy-Weinberg supone una población infinita. En una población finita, la *ley de los grandes números* no está garantizada: la generación siguiente es una *muestra* de la anterior, y las muestras fluctúan. Este cambio aleatorio de las frecuencias alélicas se llama *deriva génica*, y su modelo canónico se atribuye a los dos fundadores de la genética de poblaciones teórica, R. A. Fisher y Sewall Wright (Fisher, 1930; Wright, 1931).

Definición 10.2. (Modelo de Wright-Fisher)

Una población de N individuos diploides (es decir, $2N$ copias de un gen) tiene generaciones discretas que no se solapan. Cada una de las $2N$ copias de la generación $t + 1$ elige como madre, de forma independiente y uniforme, una de las $2N$ copias de la generación t . Si X_t es el número de copias del alelo A , entonces $X_{t+1} \mid X_t = i$ sigue una distribución binomial:

$$P(X_{t+1} = j \mid X_t = i) = \binom{2N}{j} \left(\frac{i}{2N}\right)^j \left(1 - \frac{i}{2N}\right)^{2N-j}, \quad i, j \in \{0, 1, \dots, 2N\}. \quad (10.7)$$

DONDE

N	Número de individuos diploides; la población contiene $2N$ copias del gen.
X_t	Número de copias del alelo A en la generación t ; $p_t = X_t/2N$ es su frecuencia.
i, j	Estados de la cadena: número de copias antes y después de una generación.

La ecuación (10.7) define una *cadena de Márkov* con $2N + 1$ estados: el futuro solo depende del presente, y no de cómo se llegó a él. La figura 10.2 dibuja el modelo como una genealogía. De la binomial se deducen de inmediato los dos primeros momentos:

$$\mathbb{E}[p_{t+1} | p_t] = p_t, \quad \text{Var}[p_{t+1} | p_t] = \frac{p_t(1 - p_t)}{2N}. \quad (10.8)$$

La primera igualdad dice que la deriva no tiene dirección: el valor esperado de la frecuencia futura es la frecuencia actual (en lenguaje probabilístico, p_t es una *martingala*). La segunda dice que el ruido de cada generación es inversamente proporcional al tamaño de la población. Y hay algo más: los estados 0 y $2N$ son *absorbentes*. Si un alelo se pierde, no puede reaparecer (sin mutación); si se fija, ya no puede perderse. Como toda la probabilidad termina, tarde o temprano, en uno de los dos extremos, la propiedad de martingala da la probabilidad de fijación sin cálculo alguno: si u es la probabilidad de fijarse, $\mathbb{E}[p_\infty] = u \cdot 1 + (1 - u) \cdot 0 = p_0$, luego

$$u(p_0) = p_0. \quad (10.9)$$

Una mutación neutral nueva, presente en una sola copia, se fija con probabilidad $1/2N$.

10.1.5 Pérdida de heterocigosidad

La figura 10.3 sugiere que la variabilidad genética se erosiona a un ritmo que depende de N . Podemos calcularlo exactamente siguiendo una cantidad bien elegida: la probabilidad F_t de que dos copias distintas tomadas al azar en la generación t sean *idénticas por descendencia*, es decir, copias de una misma copia ancestral.

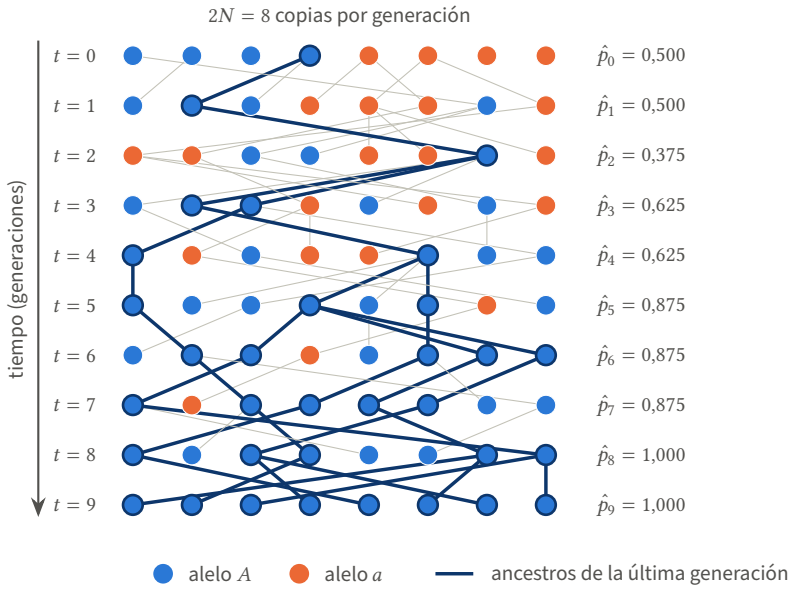


Figura 10.2. Diez generaciones de una población de Wright-Fisher con $2N = 8$ copias del gen. Cada copia (círculo) elige al azar una madre en la fila superior (líneas grises). La frecuencia $\hat{p}_t$ del alelo azul sube y baja sin ninguna causa, hasta fijarse en $t = 8$. Las líneas y los anillos oscuros siguen hacia atrás los linajes de las ocho copias de la última generación: se fusionan (*coalescen*) rápidamente y, ya en $t = 2$, todas descienden de una única copia ancestral. La fijación de un alelo y la coalescencia de todos los linajes en un solo ancestro son el mismo fenómeno visto en direcciones opuestas del tiempo (véase el recuadro sobre el coalescente al final de esta sección).

Teorema 10.2. (*Decaimiento de la heterocigosidad*)

En el modelo de Wright-Fisher neutral, la heterocigosidad esperada $H_t = \mathbb{E}[2p_t(1 - p_t)]$ satisface

$$H_t = H_0 \left(1 - \frac{1}{2N}\right)^t \approx H_0 e^{-t/2N}. \quad (10.10)$$

Derivación. Tomemos dos copias distintas de la generación $t+1$. Con probabilidad $1/2N$ eligieron la misma madre, y entonces son idénticas por descendencia. Con

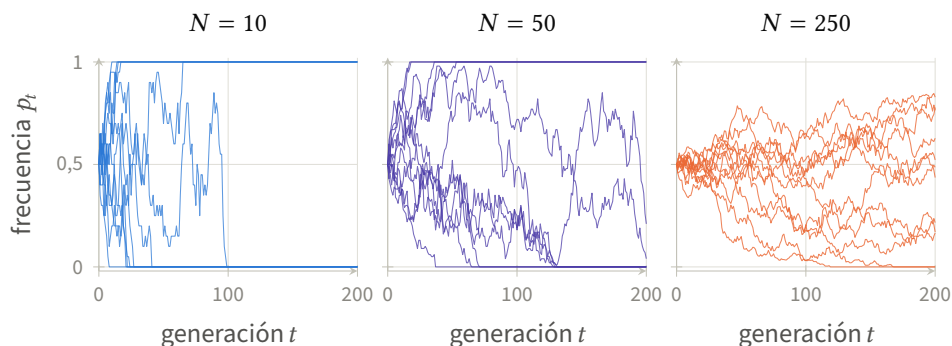


Figura 10.3. Doce réplicas independientes de la deriva de Wright-Fisher para tres tamaños de población, todas desde $p_0 = 0,5$. Con $N = 10$, cada trayectoria se absorbe en 0 o en 1 en pocas decenas de generaciones (en 5000 réplicas, todas lo habían hecho en $t = 200$). Con $N = 50$ lo había hecho el 79 %; con $N = 250$, apenas el 8 %, y las trayectorias se abren como un abanico lento. La mitad de las réplicas se fija en 1 y la otra mitad en 0, como predice la ecuación (10.9).

probabilidad $1 - 1/2N$ eligieron madres distintas, que a su vez son idénticas por descendencia con probabilidad F_t . Por lo tanto,

$$F_{t+1} = \frac{1}{2N} + \left(1 - \frac{1}{2N}\right)F_t \implies 1 - F_{t+1} = \left(1 - \frac{1}{2N}\right)(1 - F_t). \quad (10.11)$$

Como la heterocigosidad es proporcional a la probabilidad de *no* ser idénticas por descendencia, $H_t = H_0(1 - F_t)$ con $F_0 = 0$, e iterando se obtiene la ecuación (10.10). La aproximación exponencial usa $\ln(1 - x) \approx -x$ para x pequeño.

DONDE

H_t	Heterocigosidad esperada (promedio sobre réplicas) en la generación t .
F_t	Probabilidad de identidad por descendencia de dos copias de la generación t ; es el mismo F de la ecuación (10.4), ahora producido por la deriva.
$1/2N$	Probabilidad de que dos copias compartan madre en una generación: la “tasa de coalescencia”.

La figura 10.4 contrasta la ecuación (10.10) con simulaciones. La constante de tiempo es $2N$ generaciones: la heterocigosidad se reduce a la mitad en $2N \ln 2 \approx 1,39N$ generaciones. Para una población humana ancestral con un tamaño efectivo del orden de 10^4 , eso son unas 14 000 generaciones, más de 300 000 años; para una población de conservación de 50 individuos, apenas 69 generaciones.

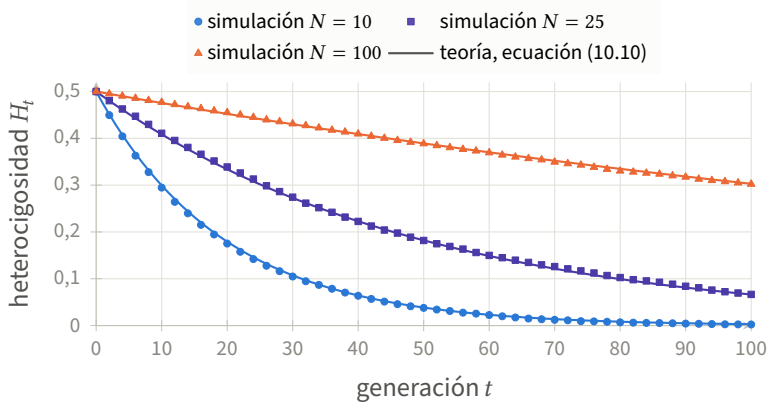


Figura 10.4. La heterocigosidad media de 4000 réplicas simuladas (símbolos) sigue con exactitud la curva teórica $H_0(1 - 1/2N)^t$ (líneas). Tras 50 generaciones, una población de 10 individuos conserva menos del 8 % de su diversidad inicial ($H_{50} = 0,038$ en la simulación, 0,0385 en la teoría), mientras que una de 100 conserva el 78 %. Cada réplica individual, sin embargo, es muy ruidosa (figura 10.3): la ecuación describe el promedio, no el destino de una población concreta.

Ejemplo 10.3. (Una población cautiva)

Un programa de conservación mantiene $N = 50$ ejemplares en cautiverio, y un locus microsatélite tiene inicialmente $H_0 = 0,5$. Tras 100 generaciones, $H_{100} = 0,5(1 - 1/100)^{100} = 0,5 \times 0,366 = 0,183$: se ha perdido el 63 % de la heterocigosidad. La vida media exacta es $\ln 0,5 / \ln(0,99) = 69,0$ generaciones, muy cerca de la aproximación $2N \ln 2 = 69,3$.

10.1.6 El tamaño efectivo de la población

Las poblaciones reales no son de Wright-Fisher: su tamaño fluctúa, los sexos no están equilibrados, unos individuos tienen muchos más descendientes que otros. Wright (1931) propuso una solución muy elegante: definir el *tamaño efectivo* N_e como el tamaño de la población de Wright-Fisher ideal que perdería heterocigosidad (o acumularía varianza de frecuencias) al mismo ritmo que la población real. Todas las fórmulas de la sección siguen valiendo al sustituir N por N_e .

Dos casos clásicos se deducen del razonamiento de la ecuación (10.11). Si el tamaño cambia de una generación a otra, la probabilidad de coalescencia acumulada es la suma de los $1/2N_t$, y el tamaño efectivo es la *media armónica*:

$$\frac{1}{N_e} = \frac{1}{T} \sum_{t=1}^T \frac{1}{N_t}, \quad N_e = \frac{4N_m N_f}{N_m + N_f}. \quad (10.12)$$

La segunda fórmula corresponde a una población con N_m machos y N_f hembras. La media armónica está dominada por los valores pequeños: una población que pasa por 1000, 10 y 1000 individuos tiene $N_e = 3/(0,001 + 0,1 + 0,001) \approx 29$. Un solo cuello de botella deja una huella duradera, y por eso el tamaño efectivo de la humanidad (unos 10^4) es órdenes de magnitud menor que su tamaño censal. De forma parecida, una granja con 10 sementales y 90 hembras tiene $N_e = 4 \cdot 10 \cdot 90/100 = 36$, no 100.

DONDE

N_e	Tamaño efectivo: el N que hay que poner en las fórmulas de Wright-Fisher para describir la población real.
N_t	Tamaño censal en la generación t ; T , número de generaciones consideradas.
N_m, N_f	Número de machos y hembras reproductores.

10.1.7 La aproximación por difusión de Kimura

La cadena de la ecuación (10.7) tiene $2N + 1$ estados, y calcular con ella distribuciones exactas a lo largo de muchas generaciones es engorroso. Cuando N es grande, los saltos de una generación son pequeños (del orden de $1/\sqrt{2N}$), y la frecuencia se parece a una partícula que se difunde en el intervalo $[0, 1]$. Motoo Kimura explotó sistemáticamente esta idea. Si $\phi(p, t)$ es la densidad de la frecuencia en el tiempo t , y $M(p)$ y $V(p)$ son la media y la varianza del cambio en una generación, entonces ϕ obedece la ecuación de Kolmogorov hacia adelante (de Fokker-Planck):

$$\frac{\partial \phi}{\partial t} = \frac{1}{2} \frac{\partial^2}{\partial p^2} [V(p) \phi] - \frac{\partial}{\partial p} [M(p) \phi], \quad V(p) = \frac{p(1-p)}{2N}. \quad (10.13)$$

DONDE

$\phi(p, t)$	Densidad de probabilidad de la frecuencia alélica p en el tiempo t (entre las poblaciones aún no absorbidas).
$V(p)$	Varianza del cambio de frecuencia en una generación, ecuación (10.8).
$M(p)$	Cambio esperado por generación: 0 en el caso neutral; $s p(1 - p)$ con selección genética de intensidad s .

Kimura (1955) resolvió el caso neutral ($M = 0$) expresando ϕ como una serie de polinomios de Gegenbauer. El término dominante decae como $e^{-t/2N}$, exactamente la tasa de la ecuación (10.10), y la solución revela tres fases (figura 10.5a): al principio, la densidad es una campana alrededor de p_0 ; luego se aplanan, hasta volverse casi uniforme en el interior; y finalmente la masa se oscurece, a ritmo constante, hacia los extremos absorbentes. La varianza entre réplicas crece como $\text{Var}(p_t) = p_0(1 - p_0)[1 - (1 - 1/2N)^t]$, que para $N = 50$, $t = 100$ vale 0,1585 (la simulación da 0,1581).

Selección y probabilidad de fijación. La difusión permite incorporar la selección cambiando solo el término de deriva $M(p)$. Si el alelo A confiere una ventaja aditiva s (cada copia multiplica la eficacia por $1 + s$), Kimura (1962) obtuvo la probabilidad de fijación resolviendo la ecuación *hacia atrás* correspondiente:

Teorema 10.3. (Probabilidad de fijación de Kimura)

Para un alelo con frecuencia inicial p y coeficiente de selección s en una población de tamaño efectivo N ,

$$u(p) = \frac{1 - e^{-4Nsp}}{1 - e^{-4Ns}}. \quad (10.14)$$

Para una mutación nueva ($p = 1/2N$) con s pequeño y $4Ns \gg 1$, $u \approx 2s$.

DONDE

$u(p)$	Probabilidad de que el alelo termine fijado.
s	Coefficiente de selección: ventaja relativa por copia del alelo ($s < 0$ si es perjudicial).
$4Ns$	Intensidad de la selección relativa a la deriva: la única combinación de N y s que importa.

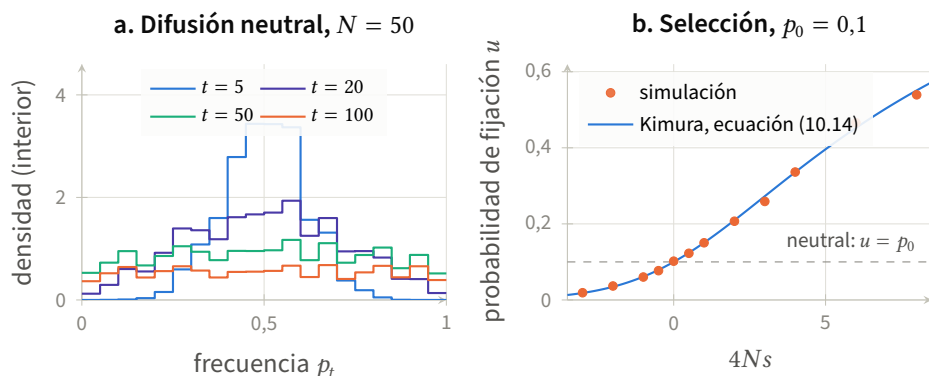


Figura 10.5. La aproximación por difusión frente a la simulación. **a:** densidad de p_t entre las réplicas aún polimórficas (40 000 réplicas, $N = 50$, $p_0 = 0,5$). La campana inicial ($t = 5$) se ensancha ($t = 20$), se vuelve casi plana ($t = 50$) y luego se hunde mientras la probabilidad escapa a los extremos: en $t = 100$, el 23 % de las réplicas ha perdido el alelo y otro 23 % lo ha fijado. **b:** probabilidad de fijación de un alelo que empieza con 10 de 100 copias, en función de $4Ns$ (3000 réplicas por punto). La curva de Kimura coincide con la simulación; una ventaja de solo $s = 0,01$ ($4Ns = 2$) duplica la probabilidad de fijación respecto al caso neutral.

La ecuación (10.14) contiene una lección profunda: lo que decide el destino de un alelo no es s sino el producto $4Ns$. Si $|4Ns| \ll 1$, la ecuación (10.14) se reduce a $u \approx p$ y el alelo se comporta como neutral aunque no lo sea; si $4Ns \gg 1$, la selección domina. Incluso una ventaja apreciable se pierde casi siempre por azar cuando el mutante es nuevo: con $N = 10^4$ y $s = 0,01$, $u = 0,0198$, de modo que el 98 % de estas mutaciones ventajosas desaparece, aunque su probabilidad de fijarse sea 400 veces mayor que la de una neutral ($1/2N = 5 \times 10^{-5}$). Sobre esta observación construyó Kimura la teoría neutral de la evolución molecular.

Tiempos de fijación. La misma maquinaria da el tiempo medio hasta la absorción (fijación o pérdida) de un alelo neutral con frecuencia inicial p :

$$\bar{i}(p) = -4N[p \ln p + (1 - p) \ln(1 - p)]. \quad (10.15)$$

Para $p = 1/2$, $\bar{i} = 4N \ln 2 \approx 2,77N$ generaciones: con $N = 50$, 138,6 generaciones (la simulación da 135,9). Y una mutación neutral nueva que sí llega a fijarse tarda en promedio unas $4N$ generaciones en hacerlo. En humanos, eso son cientos de

miles de años: la mayor parte de la variación que observamos hoy es antigua y compartida entre poblaciones.

```
wright_fisher.py

import numpy as np

def wright_fisher(N, p0, T, reps, s=0.0, seed=1):
    """Frecuencias alélicas (T+1 x reps) bajo Wright-Fisher."""
    rng = np.random.default_rng(seed)
    X = np.empty((T + 1, reps)); X[0] = p0
    for t in range(T):
        p = X[t] * (1 + s) / (1 + s * X[t]) # selección
        X[t + 1] = rng.binomial(2 * N, p) / (2 * N) # deriva
    return X

X = wright_fisher(N=50, p0=0.5, T=100, reps=4000)
H = (2 * X * (1 - X)).mean(axis=1)
print(H[100], 0.5 * (1 - 1 / 100) ** 100) # ~0.18 frente a 0.183
```

El coalescente: mirar la deriva hacia atrás

La figura 10.2 sugiere otra manera de estudiar la deriva: en lugar de seguir frecuencias hacia adelante, seguir hacia atrás los linajes de una *muestra*. Kingman (1982) demostró que, cuando N es grande y el tiempo se mide en unidades de $2N$ generaciones, la genealogía de n copias es un proceso sencillo y universal: mientras quedan k linajes, cada uno de los $\binom{k}{2}$ pares coalesce a tasa 1, así que el tiempo hasta la siguiente fusión es exponencial con media

$$\mathbb{E}[T_k] = \frac{2N}{\binom{k}{2}} = \frac{4N}{k(k-1)} \text{ generaciones}, \quad \mathbb{E}[T_{\text{MRCA}}] = \sum_{k=2}^n \frac{4N}{k(k-1)} = 4N \left(1 - \frac{1}{n}\right).$$

Sorprendentemente, el ancestro común más reciente de una muestra de 100 copias ($3,96N$ generaciones) es apenas un 10 % más antiguo que el de dos copias ($2N$): más de la mitad de la historia de la muestra transcurre mientras solo quedan dos linajes. Si las mutaciones caen sobre las ramas a tasa μ por generación, dos copias difieren en promedio en $\theta = 4N\mu$ posiciones, la base de los estimadores de diversidad nucleotídica. El coalescente es hoy el motor de casi todos los simuladores de genomas de poblaciones y de la inferencia demográfica.

La deriva no tiene dirección, pero sí destino: cada generación reduce la heterocigosidad en una fracción $1/2N_e$, y todo alelo neutral termina perdido o fijado. La selección solo gana a la deriva cuando $|4N_e s| \gg 1$.

10.2 Desequilibrio de ligamiento y PCA



Colab · 10.2 · Desequilibrio de ligamiento y PCA



abrir notebook

Una baraja que nunca se mezcla del todo

Piense en una baraja nueva, ordenada por palos y números, que se mezcla con un único corte al azar por ronda. Tras la primera ronda, casi todas las cartas siguen junto a sus vecinas originales; solo el par que quedó a ambos lados del corte se separó. Tras diez rondas, las cartas lejanas ya se han separado muchas veces, pero el as y el dos de corazones, que eran contiguos, probablemente siguen juntos. Un cromosoma se hereda así: cada generación, la recombinación hace uno o pocos cortes, y los alelos que estaban cerca en el cromosoma de un antepasado siguen viajando juntos durante cientos de generaciones. Si conozco la carta que tiene usted en la mano, puedo adivinar con bastante acierto la que viene detrás. Ese “poder adivinar” es el desequilibrio de ligamiento, y sin él los estudios de asociación serían imposibles.

10.2.1 Haplotipos y el coeficiente D

Hasta ahora hemos considerado un locus aislado. Tomemos dos loci bialélicos en el mismo cromosoma, con alelos A/a y B/b . Un *haplotipo* es la combinación de alelos presente en un mismo cromosoma, y hay cuatro posibles: AB , Ab , aB y ab , con frecuencias p_{AB} , p_{Ab} , p_{aB} y p_{ab} que suman uno. Si los alelos de ambos loci se combinaran al azar, la frecuencia de cada haplotipo sería el producto de las frecuencias alélicas: $p_{AB} = p_A p_B$. El desequilibrio de ligamiento (LD, *linkage disequilibrium*) mide cuánto se aparta la realidad de esa independencia.

Definición 10.3. (Desequilibrio de ligamiento)

El coeficiente de desequilibrio entre los loci A y B es

$$D = p_{AB} - p_A p_B = p_{AB} p_{ab} - p_{Ab} p_{aB}. \quad (10.16)$$

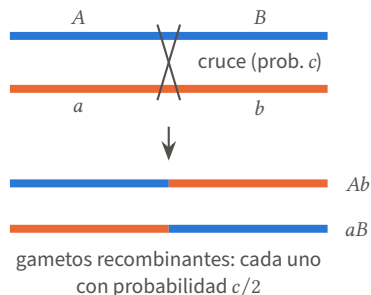
A partir de él se definen dos medidas normalizadas:

$$D' = \frac{D}{D_{\max}}, \quad D_{\max} = \begin{cases} \min\{p_A p_b, p_a p_B\} & D > 0 \\ \min\{p_A p_B, p_a p_b\} & D < 0 \end{cases} \quad r^2 = \frac{D^2}{p_A p_a p_B p_b}. \quad (10.17)$$

DONDE

$p_{AB}, \dots, p_{ab}$	Frecuencias de los cuatro haplotipos en la población.
p_A, p_B	Frecuencias alélicas marginales: $p_A = p_{AB} + p_{Ab}$, $p_B = p_{AB} + p_{aB}$.
D	Covarianza entre los indicadores “el cromosoma lleva A ” y “lleva B ”.
D'	D reescalado por su máximo posible dadas las frecuencias alélicas; $ D' = 1$ si falta alguno de los cuatro haplotipos.
r^2	Cuadrado del coeficiente de correlación de Pearson entre los dos indicadores alélicos.

La segunda forma de la ecuación (10.16) es la más intuitiva: D compara los haplotipos “en fase” (AB y ab) con los “en contrafase” (Ab y aB). Las dos medidas normalizadas responden a preguntas distintas. Lewontin (1964) introdujo D' para eliminar la dependencia de D respecto a las frecuencias alélicas: $|D'| = 1$ significa que no ha habido recombinación visible entre los loci desde que apareció la mutación más reciente, pues uno de los cuatro haplotipos no existe. Hill y Robertson (1968) usaron r^2 , que tiene una interpretación estadística directa: es la fracción de la varianza de un locus explicada por el otro. Por eso r^2 , y no D' , es la medida relevante para la asociación indirecta: si una variante causal tiene un efecto que produce, en una muestra de n individuos, un estadístico de prueba con no centralidad λ , una variante marcadora con correlación r^2 con ella produce aproximadamente $r^2 \lambda$. Hacen falta n/r^2 individuos para tener, con el marcador, la misma potencia que con la variante causal.

a. Recombinación en la meiosis**b. Tabla de haplotipos**

	B	b	
A	0,50	0,10	$p_A = 0,60$
a	0,05	0,35	$p_a = 0,40$
	$p_B = 0,55$	$p_b = 0,45$	

$$D = 0,50 \cdot 0,35 - 0,10 \cdot 0,05 = 0,17$$

$$D' = 0,77, \quad r^2 = 0,49$$

Figura 10.6. a: un individuo doblemente heterocigoto, con haplotipos AB (azul) y ab (naranja), produce gametos recombinantes Ab y aB solo si ocurre un cruce entre los dos loci, lo que sucede con probabilidad c (la fracción de recombinación). Cada generación, este mecanismo convierte una fracción c del desequilibrio en haplotipos “mezclados” (ecuación (10.18)). b: frecuencias haplotípicas del ejemplo 10.4. El exceso de haplotipos en fase (AB , ab) sobre los esperados por independencia es el desequilibrio D .

Ejemplo 10.4. (Tres medidas, tres respuestas)

En una población, los haplotipos tienen frecuencias $p_{AB} = 0,50$, $p_{Ab} = 0,10$, $p_{aB} = 0,05$ y $p_{ab} = 0,35$ (figura 10.6b). Las frecuencias alélicas son $p_A = 0,60$ y $p_B = 0,55$. Entonces:

- $D = 0,50 - 0,60 \cdot 0,55 = 0,17$; la forma cruzada da lo mismo: $0,50 \cdot 0,35 - 0,10 \cdot 0,05 = 0,17$.
- Como $D > 0$, $D_{\text{máx}} = \min\{0,60 \cdot 0,45, 0,40 \cdot 0,55\} = \min\{0,27, 0,22\} = 0,22$ y $D' = 0,773$.
- $r^2 = 0,17^2 / (0,60 \cdot 0,40 \cdot 0,55 \cdot 0,45) = 0,0289 / 0,0594 = 0,487$.

El locus B “explica” la mitad de la varianza del locus A . Si A fuera causal y solo genotipáramos B , necesitaríamos unas $1/0,487 \approx 2$ veces más individuos para alcanzar la misma potencia.

10.2.2 El decaimiento del desequilibrio

¿Por qué hay LD en primer lugar? Cada mutación nueva aparece en un único cromosoma, con un haplotipo concreto de alelos vecinos: nace en desequilibrio máximo ($|D'| = 1$) con todo lo que la rodea. La deriva en poblaciones pequeñas,

los cuellos de botella, la mezcla de poblaciones y la selección también generan LD. Lo único que lo destruye es la recombinación. Si la fracción de recombinación entre los loci es c , un haplotipo AB de la generación siguiente puede provenir de un cromosoma no recombinante que ya era AB (probabilidad $(1 - c) p_{AB}$) o de un recombinante que tomó A de un cromosoma y B de otro (probabilidad $c p_A p_B$, bajo apareamiento aleatorio). Restando $p_A p_B$ a ambos lados,

$$p'_{AB} = (1 - c) p_{AB} + c p_A p_B \implies D_t = (1 - c)^t D_0. \quad (10.18)$$

DONDE

c	Fracción de recombinación entre los dos loci por meiosis ($0 \leq c \leq \frac{1}{2}$); en humanos, del orden de 10^{-8} por par de bases, es decir, alrededor de 1 cM/Mb.
D_t	Desequilibrio en la generación t en una población infinita.

Para loci en cromosomas distintos ($c = 1/2$), el LD se reduce a la mitad cada generación y en cinco generaciones queda un 3 %. Para loci a 1 cM ($c = 0,01$), tras 50 generaciones subsiste el 60 %; y para loci a 100 kb ($c \approx 0,001$), tras 100 generaciones subsiste el 90 %. En una población finita, la deriva genera LD nuevo continuamente al mismo tiempo que la recombinación lo destruye, y se alcanza un equilibrio. Hill y Robertson (1968) estudiaron este balance y propusieron r^2 como medida; el análisis clásico de ese equilibrio conduce a la aproximación

$$\mathbb{E}[r^2] \approx \frac{1}{1 + 4N_e c}, \quad (10.19)$$

que depende, otra vez, de un único producto: $4N_e c$ mide cuántas oportunidades de recombinación ha tenido la genealogía de la muestra. Con $N_e = 10^4$ y $c = 10^{-8}$ por par de bases, $\mathbb{E}[r^2] \approx 0,20$ a 10 kb y 0,024 a 100 kb (figura 10.7b). Esta es la razón de que las poblaciones con historia de cuello de botella (menor N_e) tengan LD más extenso, y de que en las poblaciones africanas, con mayor N_e histórico, el LD decaiga en distancias más cortas.

10.2.3 Bloques de haplotipos, HapMap y 1000 Genomas

La ecuación (10.18) supone que la recombinación es uniforme a lo largo del cromosoma. No lo es. La mayor parte de los entrecruzamientos humanos se concentran en *puntos calientes* de uno o dos kilobases, separados por regiones donde la recombinación es muy escasa. El resultado es un mosaico: tramos de decenas

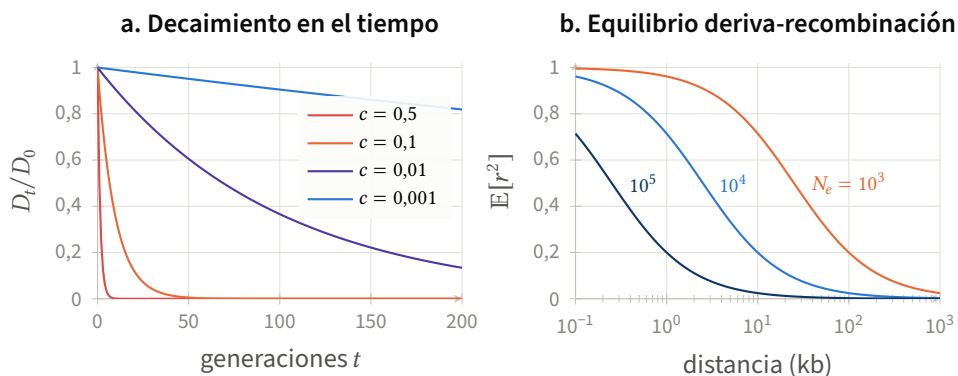


Figura 10.7. a: en una población infinita, el desequilibrio decae geométricamente, $D_t = (1 - c)^t D_0$ (ecuación (10.18)). Loci no ligados ($c = 0.5$) se equilibran en pocas generaciones; loci a unos 100 kb ($c \approx 0.001$) conservan el 82 % del LD tras 200 generaciones. b: en una población finita se alcanza un equilibrio entre la deriva, que crea LD, y la recombinación, que lo destruye (ecuación (10.19), con 1 cM/Mb). Cada factor 10 en N_e desplaza la curva una década a la izquierda: el LD útil para la asociación se extiende a decenas de kilobases en poblaciones de tamaño efectivo pequeño y se acorta en las de mayor tamaño efectivo.

de kilobases en los que el LD es alto y la diversidad haplotípica baja, separados por fronteras abruptas. Gabriel et al. (2002) analizaron 51 regiones autosómicas en muestras de cuatro poblaciones y definieron formalmente los *bloques de haplotipos* a partir de los intervalos de confianza de D' : un bloque es una región en la que casi todos los pares de marcadores muestran evidencia de LD fuerte, es decir, de recombinación histórica escasa. Observaron que dentro de cada bloque unos pocos haplotipos comunes explican la gran mayoría de los cromosomas, y que los bloques eran en promedio más cortos en las muestras africanas.

Estas observaciones tuvieron una consecuencia práctica enorme: si los alelos de un bloque se heredan en unas pocas combinaciones, no hace falta genotipar todos los SNP; basta con unos pocos *SNP etiqueta* (*tag SNPs*) que capturen, vía r^2 , a los demás. El Proyecto Internacional HapMap se propuso catalogar ese patrón de LD en todo el genoma: su primera fase genotipó más de un millón de SNP en 269 individuos de cuatro poblaciones (The International HapMap Consortium, 2005), y sirvió para diseñar los chips de genotipado que harían posibles los primeros GWAS. Una década después, el Proyecto 1000 Genomas secuenció a 2504 personas de 26 poblaciones y construyó un panel de referencia de más de 88 millones de variantes (The 1000 Genomes Project Consortium, 2015). Con paneles así se practica

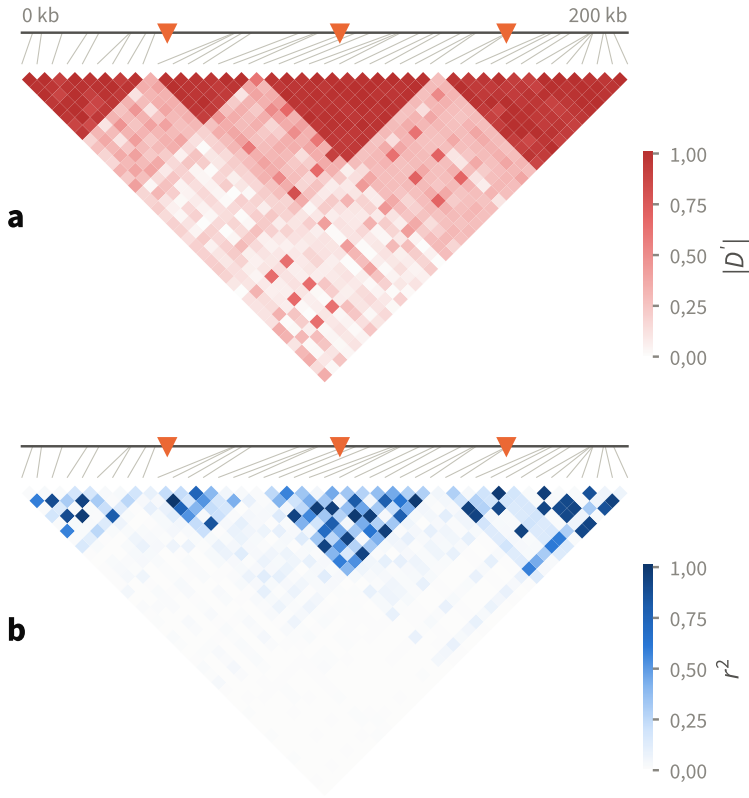


Figura 10.8. Mapa de LD al estilo de Haploview para 41 SNP en una región simulada de 200 kb (600 haplotipos), con tres puntos calientes de recombinación (triángulos naranjas). Cada rombo representa un par de SNP: el del vértice superior de cada triángulo compara vecinos inmediatos, y hacia abajo se comparan SNP cada vez más alejados. Las líneas grises conectan cada SNP con su posición física. **a:** $|D'|$; dentro de cada bloque casi todos los pares tienen $|D'| \approx 1$ (rojo oscuro), porque los haplotipos del bloque descenden de pocos ancestros sin recombinación entre ellos, y las fronteras coinciden con los puntos calientes. **b:** r^2 para los mismos pares; es mucho más irregular, porque dos SNP pueden estar en LD completo ($|D'| = 1$) y aun así tener r^2 bajo si sus frecuencias alélicas son muy distintas. D' revela la historia de recombinación; r^2 , la capacidad de un SNP para sustituir a otro en un estudio de asociación. Simulación: mosaico de siete haplotipos fundadores por bloque (figuras/cap10/generar.py).

la *imputación*: a partir de los SNP genotipados con un chip y de los haplotipos de referencia, se predicen probabilísticamente los genotipos de millones de variantes no genotipadas, aprovechando precisamente el LD.

```
ld.py

import numpy as np

def ld(h1, h2):
    """D, D' y r^2 entre dos columnas de haplotipos 0/1."""
    pA, pB = h1.mean(), h2.mean()
    D = (h1 & h2).mean() - pA * pB
    if D >= 0:
        Dmax = min(pA * (1 - pB), (1 - pA) * pB)
    else:
        Dmax = min(pA * pB, (1 - pA) * (1 - pB))
    r2 = D ** 2 / (pA * (1 - pA) * pB * (1 - pB))
    return D, D / Dmax, r2

# H: matriz (haplotipos x SNP) de 0/1, p. ej., de un VCF faseado
# R2 = np.corrcoef(H.T) ** 2 # todos los r^2 de una vez
```

⚠ Genotipos no son haplotipos

Un chip o una llamada de variantes estándar dan *genotipos*, no haplotipos: sabemos que un individuo es *Aa* y *Bb*, pero no si sus cromosomas son *AB/ab* o *Ab/aB*. Estimar la fase (*phasing*) es un problema estadístico en sí mismo. Calcular r^2 a partir de dosis genotípicas (la correlación entre columnas 0/1/2) es una aproximación útil y es lo que hacen herramientas como PLINK por defecto, pero D' requiere haplotipos o una estimación de ellos.

10.2.4 Estructura poblacional y F_{ST}

La sección 10.1 supuso una población panmíctica. La humanidad no lo es: la geografía, la historia y la cultura hacen que las personas se apareen preferentemente con otras de su misma región, y las frecuencias alélicas derivan de manera independiente en cada grupo. Wright (1931) cuantificó esta subdivisión con el *índice de fijación* F_{ST} : la reducción de heterocigosidad dentro de las subpoblaciones respecto a la que tendría la población total si fuera panmíctica,

$$F_{ST} = \frac{H_T - H_S}{H_T} = \frac{\text{Var}(p_k)}{\bar{p}(1 - \bar{p})}. \quad (10.20)$$

DONDE

H_T	Heterocigosidad esperada con las frecuencias de la población total, $2\bar{p}(1 - \bar{p})$.
H_S	Promedio de las heterocigosidades esperadas dentro de cada subpoblación, $2p_k(1 - p_k)$.
$p_k, \bar{p}$	Frecuencia alélica en la subpoblación k y su media ponderada.

La segunda forma muestra que F_{ST} es la varianza de las frecuencias entre subpoblaciones normalizada por su máximo posible. En la deriva de Wright-Fisher, subpoblaciones aisladas de tamaño N acumulan $F_{ST} \approx 1 - (1 - 1/2N)^t \approx t/2N$ al cabo de t generaciones de separación: es el mismo F_t de la ecuación (10.11). Estimar F_{ST} con muestras finitas requiere corregir el sesgo que introduce el propio muestreo; el estimador estándar es el de Weir y Cockerham (1984), que descompone la varianza de las frecuencias en componentes entre poblaciones, entre individuos y dentro de individuos. Entre continentes, F_{ST} humano es del orden de 0,1; dentro de Europa, del orden de 0,01 o menos.

10.2.5 Análisis de componentes principales de genotipos

Una medida resumen como F_{ST} exige saber de antemano a qué población pertenece cada individuo. A menudo no lo sabemos, o las poblaciones no son discretas. El análisis de componentes principales (PCA) invierte el problema: encuentra, sin etiquetas, los ejes de mayor variación de la matriz de genotipos. Patterson et al. (2006) le dieron un tratamiento estadístico riguroso.

Definición 10.4. (PCA de genotipos)

Sea G la matriz $n \times M$ de genotipos ($g_{ij} \in \{0, 1, 2\}$) de n individuos en M SNP, y $\hat{p}_j$ la frecuencia alélica del SNP j . Se estandariza cada columna,

$$Z_{ij} = \frac{g_{ij} - 2\hat{p}_j}{\sqrt{2\hat{p}_j(1 - \hat{p}_j)}}, \quad \Psi = \frac{1}{M} ZZ^\top = \sum_k \lambda_k \mathbf{u}_k \mathbf{u}_k^\top, \quad (10.21)$$

y las coordenadas del individuo i en el componente k son $\sqrt{\lambda_k} u_{ik}$.

DONDE

Z	Matriz de genotipos centrada y escalada: cada SNP con media 0 y varianza 1 bajo HWE.
Ψ	Matriz $n \times n$ de parentesco genómico (GRM); $\Psi_{ii'}$ mide cuántos alelos comparten i e i' por encima de lo esperado al azar.
$\lambda_k, \mathbf{u}_k$	Autovalores y autovectores de Ψ , ordenados de mayor a menor.

La estandarización de la ecuación (10.21) da el mismo peso a cada SNP en términos de su varianza esperada, de modo que los SNP raros (cuya varianza es pequeña) no quedan ahogados por los comunes. La descomposición de Ψ es, en la práctica, una descomposición en valores singulares de Z , lo que permite calcularla con miles de individuos y cientos de miles de SNP. Si no hubiera estructura, todos los autovalores serían parecidos; con K poblaciones, aparecen $K - 1$ autovalores destacados. Patterson et al. (2006) mostraron cómo decidir cuántos son significativos comparándolos con la distribución de Tracy-Widom, y descubrieron un fenómeno de umbral: la estructura solo es detectable si F_{ST} supera aproximadamente $1/\sqrt{nM}$. Con 1000 individuos y 100 000 SNP, eso es $F_{ST} \approx 10^{-4}$; el PCA puede separar poblaciones que ningún otro indicador distinguiría.

El resultado más célebre de esta técnica es el de Novembre et al. (2008), que aplicaron PCA a genotipos de más de mil europeos y encontraron que, al rotar ligeramente los dos primeros componentes, la nube de individuos reproducía el mapa de Europa: los genes reflejan la geografía. La interpretación es la del modelo de aislamiento por distancia: cuando las personas se aparean con sus vecinos, las frecuencias alélicas varían suavemente en el espacio y los primeros componentes principales recuperan las coordenadas geográficas.

pca_genotipos.py

```
import numpy as np

def pca_genotipos(G, k=10):
    """G: n x M con 0/1/2. Devuelve coordenadas y autovalores."""
    p = G.mean(axis=0) / 2
    ok = (p > 0.01) & (p < 0.99) # descartar monomórficos
    Z = (G[:, ok] - 2 * p[ok]) / np.sqrt(2 * p[ok] * (1 - p[ok]))
    U, S, _ = np.linalg.svd(Z, full_matrices=False)
    lam = S ** 2 / Z.shape[1] # autovalores de Psi
    return U[:, :k] * np.sqrt(lam[:k]), lam
```

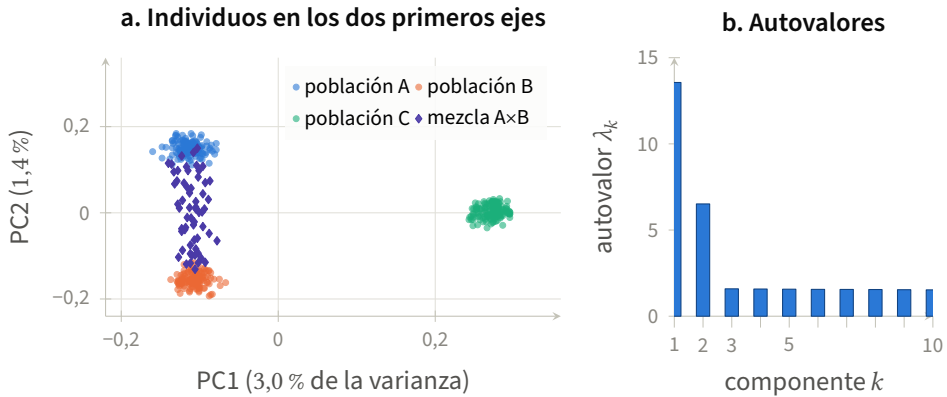


Figura 10.9. PCA de 450 individuos simulados en unos 6000 SNP. Tres poblaciones (130 individuos cada una) derivaron desde un ancestro común con el modelo de Balding-Nichols; C está más diferenciada (F_{ST} de Weir-Cockerham: A–B 0,020, A–C 0,040, B–C 0,040). Otros 60 individuos tienen ancestría mixta A×B en proporciones aleatorias. **a:** PC1 separa a C de las demás y PC2 separa a A de B; los individuos mezclados forman un puente entre A y B, y su posición en PC2 correlaciona en 0,98 con su fracción real de ancestría A. **b:** con tres poblaciones aparecen exactamente dos autovalores destacados; los demás forman el “mar” de ruido ($\lambda \approx 1,5$) que describe la teoría de matrices aleatorias.

⚠ El PCA también ve el LD

El PCA supone implícitamente que los SNP son independientes. Una región larga de LD fuerte (como el complejo HLA en el cromosoma 6, o una inversión cromosómica polimórfica) aporta cientos de SNP casi idénticos y puede “secuestrar” un componente entero, que entonces refleja el genotipo de esa región y no la ancestría. Por eso, antes del PCA, se *poda* el conjunto de SNP para eliminar pares con r^2 alto (en PLINK, `--indep-pairwise`) y se excluyen las regiones de LD extenso conocidas.

10.3 GWAS: Manhattan, QQ plots y corrección múltiple

La encuesta de los paraguas

Suponga que quiere saber si llevar paraguas protege contra los resfriados. Encuesta a mil personas en dos ciudades: una lluviosa, donde casi todos llevan paraguas, y otra seca. Si en la ciudad lluviosa, por su clima frío, también hay más resfriados, encontrará una asociación fuerte entre paraguas y resfriados, sin que el paraguas tenga nada que ver: la ciudad es una variable de confusión que se correlaciona con ambos. Un estudio de asociación genética es exactamente esta encuesta repetida un millón de veces, una por cada variante, y con el mismo peligro: si los casos y los controles provienen en proporciones distintas de poblaciones con frecuencias alélicas distintas, *todas* las variantes diferenciadas entre esas poblaciones parecerán asociadas con la enfermedad. La mitad de esta sección trata de cómo encontrar asociaciones; la otra mitad, de cómo no engañarse.

10.3.1 La idea y el diseño

Un estudio de asociación del genoma completo (GWAS, *genome-wide association study*) prueba, una a una, cientos de miles o millones de variantes para detectar las que difieren en frecuencia entre casos y controles, o que se correlacionan con un rasgo cuantitativo. No necesita familias ni una hipótesis previa sobre qué genes intervienen: se apoya en que, gracias al LD, una variante genotipada “etiqueta” a sus vecinas no genotipadas, incluida la causal. La premisa biológica es la hipótesis de *enfermedad común, variante común*: buena parte del riesgo genético de las enfermedades frecuentes se debería a variantes frecuentes, cada una con un efecto pequeño.

🔍 El estudio que abrió la era de los GWAS

En 2007, el Wellcome Trust Case Control Consortium publicó un estudio con unos 2000 casos de cada una de siete enfermedades comunes (trastorno bipolar, enfermedad coronaria, enfermedad de Crohn, hipertensión, artritis reumatoide y diabetes de tipos 1 y 2) y 3000 controles compartidos, genotipados en unos 500 000 SNP (The Wellcome Trust Case Control Consortium, 2007). Encontró 24 señales de asociación independientes con $p < 5 \times 10^{-7}$, muchas en genes nunca antes relacionados con esas enfermedades. Tan importante como los hallazgos fue el método: el artículo estableció buena parte del control de calidad, el análisis de estratificación y el uso de gráficos QQ que se volvieron estándar. Diez años después, los GWAS habían identificado miles de loci asociados con cientos de rasgos (Visscher et al., 2017).

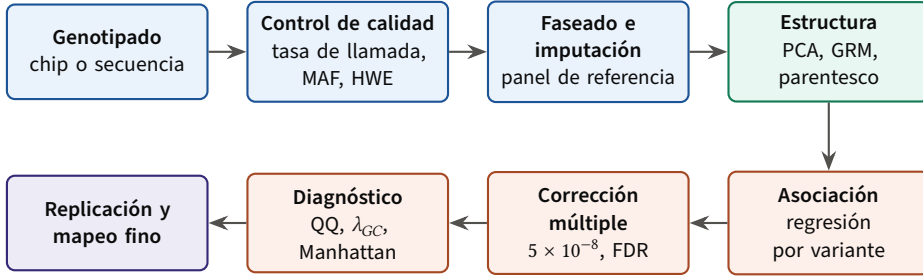


Figura 10.10. Etapas de un GWAS. En azul, la preparación de los datos: el control de calidad elimina individuos y variantes con muchos datos faltantes, frecuencias muy bajas o desviaciones extremas de Hardy-Weinberg (sección 10.1); la imputación aprovecha el LD con un panel de referencia (sección 10.2) para pasar de cientos de miles a decenas de millones de variantes. En verde, la estructura poblacional, que entra en el modelo como covariables. En naranja, el análisis estadístico propiamente dicho. En violeta, lo que ocurre después: toda asociación debe replicarse en una muestra independiente, y la variante más significativa rara vez es la causal.

La figura 10.10 resume las etapas de un GWAS típico. Todas son importantes, pero dos concentran la mayor parte de los errores: el control de la estructura poblacional y la corrección por comparaciones múltiples.

10.3.2 El modelo aditivo

Para cada variante j se ajusta un modelo de regresión en el que el genotipo entra como el número de copias del alelo de efecto, $g_{ij} \in \{0, 1, 2\}$ (o su dosis imputada, un número real entre 0 y 2). Para un rasgo cuantitativo,

$$y_i = \alpha + \beta_j g_{ij} + \gamma^\top \mathbf{c}_i + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \quad (10.22)$$

y para una enfermedad (caso/control), la regresión logística

$$\log \frac{\mathbb{P}(y_i = 1)}{1 - \mathbb{P}(y_i = 1)} = \alpha + \beta_j g_{ij} + \gamma^\top \mathbf{c}_i, \quad \text{OR}_j = e^{\beta_j}. \quad (10.23)$$

La hipótesis nula es $\beta_j = 0$, y se contrasta con el estadístico de Wald $z_j = \hat{\beta}_j / \widehat{\text{se}}(\hat{\beta}_j)$, que bajo la nula es aproximadamente normal estándar (o, equivalentemente, $z_j^2 \sim \chi_1^2$).

DONDE	
y_i	Fenotipo del individuo i : un valor continuo o un indicador de enfermedad (1 = caso).
g_{ij}	Número de copias del alelo de efecto en la variante j .
β_j	Efecto aditivo: cambio en el rasgo (o en el logaritmo de las probabilidades) por cada copia adicional.
c_i, γ	Covariables (sexo, edad, componentes principales) y sus coeficientes.
OR_j	Razón de probabilidades (<i>odds ratio</i>) por copia del alelo.

¿Por qué aditivo? Porque es el modelo con un solo grado de libertad que mejor potencia tiene frente a una amplia gama de modelos verdaderos: una variante dominante o recesiva sigue produciendo una tendencia monótona en la media de los tres genotipos, que la regresión lineal detecta. Además, la varianza del rasgo explicada por una variante aditiva tiene una forma sencilla:

$$q_j^2 = \frac{2p_j(1 - p_j) \beta_j^2}{\sigma_y^2}, \quad E[z_j^2] \approx 1 + \frac{n q_j^2}{1 - q_j^2}. \tag{10.24}$$

DONDE	
q_j^2	Fracción de la varianza fenotípica explicada por la variante j .
$2p_j(1 - p_j)$	Varianza del genotipo bajo HWE; las variantes raras explican poca varianza aunque su efecto sea grande.
$nq^2/(1 - q^2)$	Parámetro de no centralidad del χ_1^2 : determina la potencia.

La ecuación (10.24) explica por qué los GWAS modernos necesitan cientos de miles de participantes. Una variante con $p = 0,3$ que desplaza el rasgo 0,05 desviaciones estándar por copia explica $q^2 = 2 \cdot 0,3 \cdot 0,7 \cdot 0,05^2 = 0,00105$, apenas una milésima de la varianza. Con un umbral $\alpha = 5 \times 10^{-8}$ ($\chi_1^2 > 29,7$), para detectarla con 80 % de potencia hacen falta unos 40 000 individuos; si explicara la mitad de eso, unos 79 000 (figura 10.12b).

10.3.3 Estratificación poblacional

Volvamos a los paraguas. Suponga que la muestra mezcla dos poblaciones con frecuencias alélicas distintas en muchas variantes, y que el rasgo también difiere en promedio entre ellas, por razones genéticas o ambientales. Entonces toda variante

diferenciada entre las poblaciones mostrará una asociación con el rasgo, aunque no tenga ningún efecto. Este sesgo, la *estratificación poblacional*, no afecta a unas pocas variantes sino a todo el genoma, y deja una huella característica: infla la distribución completa de los estadísticos de prueba.

Control genómico. Devlin y Roeder (1999) propusieron medir esa inflación con la mediana. Bajo la nula, $z^2 \sim \chi_1^2$, cuya mediana es 0,4549. Si la estratificación multiplica todos los estadísticos por un factor aproximadamente constante,

$$\hat{\lambda}_{GC} = \frac{\text{mediana}(z_1^2, \dots, z_M^2)}{0,4549}, \quad \tilde{z}_j^2 = \frac{z_j^2}{\hat{\lambda}_{GC}}, \quad (10.25)$$

y dividir por $\hat{\lambda}_{GC}$ restaura la calibración. La mediana es robusta: las pocas variantes verdaderamente asociadas apenas la mueven. Un λ_{GC} cercano a 1 indica un estudio bien controlado. Hay una salvedad importante: en rasgos muy poligénicos con muestras enormes, miles de variantes causales de efecto diminuto inflan legítimamente la mediana, y un λ_{GC} de 1,1 o más no implica necesariamente confusión.

Componentes principales. La corrección por λ_{GC} es uniforme, pero la estratificación no lo es: afecta más a las variantes más diferenciadas entre poblaciones. Price et al. (2006) propusieron EIGENSTRAT: calcular los ejes principales de variación genética (ecuación (10.21)) y eliminar de genotipos y fenotipos la parte explicada por ellos, lo que equivale a incluir los primeros componentes como covariables c_i en la ecuación (10.22). Así, cada variante se corrige en proporción a su propia diferenciación.

Ejemplo 10.5. (Una asociación espuria y su corrección)

Simulamos 2000 individuos de dos poblaciones (1000 de cada una, $F_{ST} \approx 0,02$) y 10 000 SNP, con 10 variantes causales de efecto pequeño y un rasgo que, además, es 0,3 desviaciones estándar más alto en la segunda población. La regresión ingenua da $\hat{\lambda}_{GC} = 1,86$: 148 variantes superan $p < 10^{-3}$, cuando por azar se esperarían 10 más unas pocas causales. El primer componente principal correlaciona en 0,997 con la población de origen; al incluir dos componentes como covariables, $\hat{\lambda}_{GC} = 0,98$ y quedan 18 variantes con $p < 10^{-3}$ (figura 10.11b). En esta simulación modesta, las variantes que superan 5×10^{-8} son causales en ambos análisis: la inflación se concentra en la zona intermedia de valores p . Pero la inflación crece con el tamaño de muestra (la

no centralidad espuria es proporcional a n), y en un estudio diez veces mayor esas mismas diferencias de frecuencia producirían rascacielos falsos en el gráfico Manhattan.

Modelos lineales mixtos. Los componentes principales capturan bien la estructura a gran escala, pero no el parentesco críptico (primos lejanos en la muestra) ni la estructura fina. Los modelos mixtos modelan toda la similitud genética a la vez:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\gamma} + \mathbf{g}_j\beta_j + \mathbf{u} + \varepsilon, \quad \mathbf{u} \sim \mathcal{N}(\mathbf{0}, \sigma_g^2 \boldsymbol{\Psi}), \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma_e^2 \mathbf{I}). \quad (10.26)$$

DONDE

$\mathbf{u}$	Efecto genético poligénico aleatorio: la suma de los efectos de todas las demás variantes.
$\boldsymbol{\Psi}$	Matriz de parentesco genómico de la ecuación (10.21): individuos más emparentados tienen efectos $\mathbf{u}$ más correlacionados.
σ_g^2, σ_e^2	Varianzas genética y residual; $\sigma_g^2/(\sigma_g^2 + \sigma_e^2)$ es la heredabilidad atribuible a los SNP.

Yang et al. (2011) popularizaron, con el programa GCTA, el uso de la matriz $\boldsymbol{\Psi}$ calculada con SNP y la estimación por máxima verosimilitud restringida (REML) de la varianza que todos los SNP explican conjuntamente, la llamada heredabilidad SNP. Ajustar la ecuación (10.26) para cada una de millones de variantes era prohibitivo con los algoritmos clásicos, cuyo coste crece con el cubo del número de individuos; Loh et al. (2015) desarrollaron BOLT-LMM, que evita construir y factorizar $\boldsymbol{\Psi}$ mediante métodos iterativos y un modelo bayesiano de efectos, y mostraron que aumenta la potencia de asociación en cohortes grandes.

10.3.4 Comparaciones múltiples

Con un millón de pruebas independientes y el umbral clásico $\alpha = 0,05$, esperaríamos 50 000 falsos positivos. Hay dos filosofías para evitarlo.

Error por familia: Bonferroni y 5×10^{-8} . La tasa de error por familia (FWER) es la probabilidad de cometer *al menos un* falso positivo. Por la desigualdad de Boole, si cada una de M pruebas se realiza al nivel α/M ,

$$\text{FWER} = \mathbb{P}\left(\bigcup_{j=1}^M \{p_j \leq \alpha/M\}\right) \leq \sum_{j=1}^M \frac{\alpha}{M} = \alpha, \quad (10.27)$$

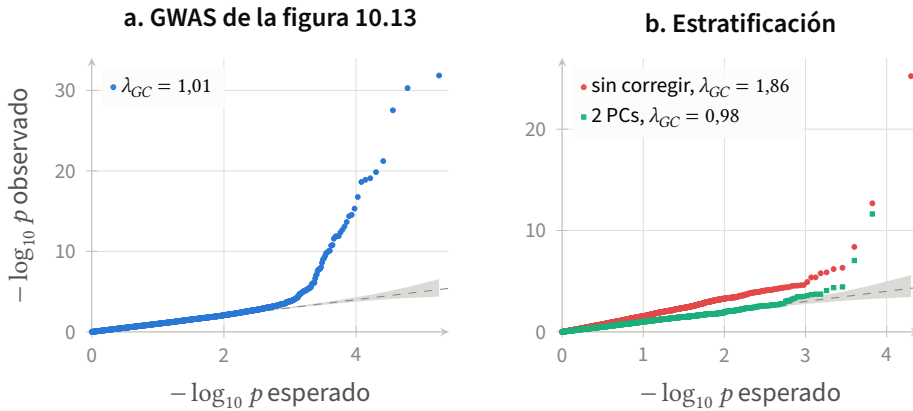


Figura 10.11. Gráficos cuantil-cuantil de los valores p : cada punto compara el k -ésimo valor p más pequeño observado con el esperado bajo la hipótesis nula global, $(k - 0,5)/M$. La banda gris es el intervalo del 95 % de cada estadístico de orden bajo la nula, y la diagonal es la identidad. **a:** estudio bien calibrado con señales reales; los puntos siguen la diagonal casi hasta el final y solo se despegan en la cola, donde están las verdaderas asociaciones. **b:** estudio con estratificación (ejemplo 10.5); sin corrección (rojo), los puntos se separan de la diagonal desde el principio, síntoma de inflación de *todo* el genoma; con dos componentes principales como covariables (verde), la nube vuelve a la banda. Se muestran todos los puntos de la cola y una submuestra del resto.

sin ningún supuesto sobre la dependencia entre pruebas. En un GWAS las pruebas no son independientes: el LD hace que muchas variantes vecinas sean casi redundantes, y Bonferroni sobre el número bruto de variantes sería demasiado conservador. Pe'er et al. (2008) estimaron el número *efectivo* de pruebas independientes al evaluar prácticamente todas las variantes comunes del genoma en poblaciones de ascendencia europea, que resultó del orden de un millón. De ahí el umbral convencional de significación genómica, $0,05/10^6 = 5 \times 10^{-8}$. En poblaciones africanas, con LD más corto, el número efectivo de pruebas es mayor y el umbral debería ser algo más estricto; y con variantes raras de secuenciación completa, también.

Tasa de falsos descubrimientos. En estudios exploratorios, controlar la probabilidad de un solo error puede ser excesivo. Benjamini y Hochberg (1995) propusieron controlar la *tasa de falsos descubrimientos* (FDR): la proporción esperada de falsos positivos entre las pruebas declaradas significativas.

Teorema 10.4. (Procedimiento de Benjamini-Hochberg)

Se ordenan los valores p de menor a mayor, $p_{(1)} \leq \dots \leq p_{(M)}$, y se busca

$$k^* = \max \left\{ k : p_{(k)} \leq \frac{k}{M} \alpha \right\}. \quad (10.28)$$

Se rechazan las hipótesis correspondientes a $p_{(1)}, \dots, p_{(k^*)}$. Si las pruebas son independientes, el procedimiento garantiza $\text{FDR} \leq \frac{M_0}{M} \alpha \leq \alpha$, donde M_0 es el número de hipótesis nulas verdaderas.

DONDE

$p_{(k)}$	k -ésimo valor p más pequeño.
k^*	Número de descubrimientos: el mayor rango cuyo valor p cae bajo la recta $k\alpha/M$.
M_0	Número (desconocido) de hipótesis nulas ciertas.

Ejemplo 10.6. (Bonferroni frente a Benjamini-Hochberg)

Diez pruebas dan, ordenados, los valores p : 0,0001, 0,0008, 0,0021, 0,0042, 0,0110, 0,0290, 0,0480, 0,0730, 0,21 y 0,62. Con $\alpha = 0,05$, Bonferroni exige $p \leq 0,005$ y rechaza 4. Los umbrales de Benjamini-Hochberg son 0,005, 0,010, 0,015, ..., 0,050; el sexto valor ($0,0290 \leq 0,030$) todavía cae bajo su umbral y el séptimo ($0,048 > 0,035$) ya no, de modo que $k^* = 6$ y se rechazan seis hipótesis. Observe que no es necesario que todos los valores anteriores cumplan su propio umbral: lo que cuenta es el *mayor* k que lo hace.

asociacion.py

```
import numpy as np
from scipy import stats

def gwas_lineal(y, G, C=None):
    """Regresión marginal por SNP; C = covariables (n x k)."""
    n = len(y)
    Q = np.ones((n, 1))
    if C is not None:
        Q = np.column_stack([Q, C])
```

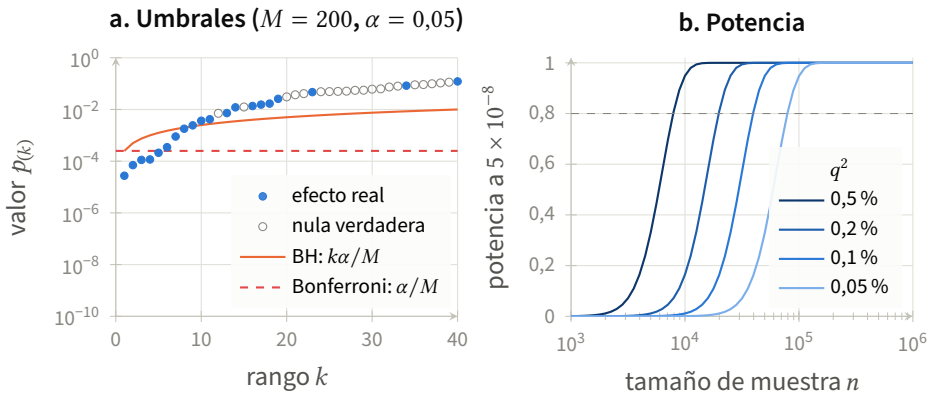


Figura 10.12. a: los 40 valores p más pequeños de 200 pruebas simuladas, de las cuales 20 tienen un efecto real (azul). Bonferroni (línea roja discontinua) declara 5 descubrimientos; Benjamini-Hochberg (recta naranja, que crece con el rango) declara 8, sin ningún falso positivo en este caso; y el umbral sin corregir $p < 0,05$ declararía 25, de los cuales 7 serían falsos. b: potencia de una prueba χ^2_1 al umbral genómico en función del tamaño de muestra, para variantes que explican distintas fracciones q^2 de la varianza (ecuación (10.24)). La transición de potencia casi nula a casi total ocurre en menos de un orden de magnitud de n : con $q^2 = 0,1 \%$, la potencia es del 1 % con 10 000 individuos y del 95 % con 50 000.

```

Q, _ = np.linalg.qr(Q)
yr = y - Q @ (Q.T @ y)                                     # residualizar (FWL)
Gr = G - Q @ (Q.T @ G)
sxx = (Gr ** 2).sum(axis=0)
b = Gr.T @ yr / sxx
df = n - Q.shape[1] - 1
se = np.sqrt(((yr ** 2).sum() - b ** 2 * sxx) / df / sxx)
return b, 2 * stats.t.sf(np.abs(b / se), df)

def lambda_gc(p):
    return np.median(stats.chi2.isf(p, 1)) / stats.chi2.ppf(0.5, 1)

def benjamini_hochberg(p, alpha=0.05):
    o = np.argsort(p); M = len(p)
    ok = p[o] <= alpha * np.arange(1, M + 1) / M
    k = ok.nonzero()[0].max() + 1 if ok.any() else 0
    rechazo = np.zeros(M, bool); rechazo[o[:k]] = True
    return rechazo

```

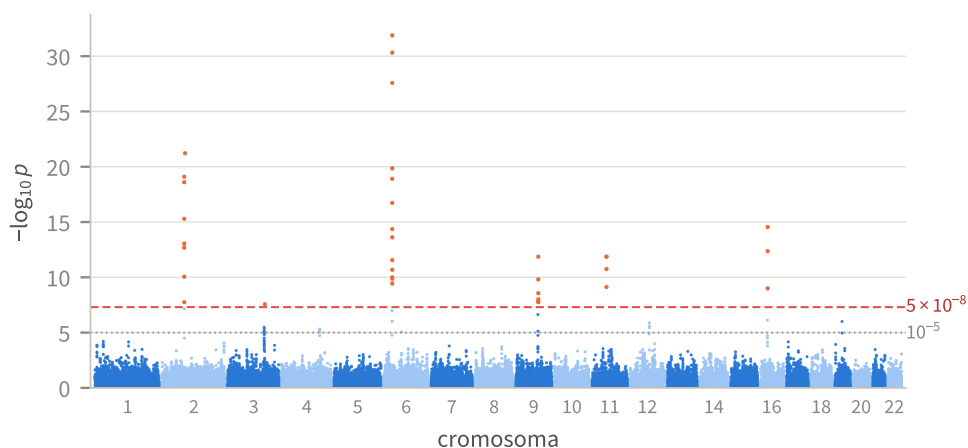


Figura 10.13. Gráfico Manhattan de un GWAS simulado con 89 999 variantes en los 22 autosomas, cada una representada por su $-\log_{10} p$. La línea roja marca la significación genómica (5×10^{-8}) y la punteada, el umbral “sugestivo” de 10^{-5} . Treinta y cinco variantes (naranja) superan el umbral genómico, agrupadas en seis loci de los cromosomas 2, 3, 6, 9, 11 y 16; el pico más alto ($p = 1,4 \times 10^{-32}$, cromosoma 6) imita la señal típica del complejo HLA en enfermedades autoinmunes. Cada pico es una torre y no una aguja aislada porque las variantes vecinas en LD con la causal heredan una parte de su señal, proporcional a su correlación. Las señales simuladas en los cromosomas 12 y 19 quedan en la zona sugestiva ($p \approx 10^{-6}$): con este tamaño de muestra, el estudio no tiene potencia para declararlas. El factor de inflación es $\lambda_{GC} = 1,01$.

La función `gwas_lineal` usa el teorema de Frisch-Waugh-Lovell: el coeficiente de un SNP en una regresión con covariables es igual al de la regresión simple entre los residuos del SNP y del fenotipo después de proyectar ambos fuera del espacio de las covariables. Así se prueban todas las variantes con dos productos de matrices, sin un bucle.

10.3.5 Leer un gráfico Manhattan

El resultado de un GWAS se resume en dos gráficos. El *gráfico Manhattan* (figura 10.13) sitúa cada variante según su posición en el genoma y su $-\log_{10} p$: las asociaciones aparecen como rascacielos sobre un horizonte de ruido. El *gráfico QQ* (figura 10.11) compara la distribución completa de los valores p con la esperada bajo la nula y sirve de diagnóstico: una desviación temprana indica inflación; una desviación solo en la cola, señales reales.

Al leer un Manhattan conviene tener presentes tres advertencias. Primera: la variante con el valor p más bajo de una torre (la variante “líder”) no es necesariamente la causal; es solo la que mejor etiqueta a la causal en esta muestra, y la identificación de la variante causal requiere *mapeo fino* estadístico y funcional. Segunda: el locus se nombra a menudo por el gen más cercano, pero la variante causal puede actuar sobre un gen a cientos de kilobases, a través de un elemento regulador. Tercera: un pico formado por una única variante aislada, sin vecinas que la acompañen, es sospechoso de ser un artefacto de genotipado.

GWAS con PLINK 2

```
# 1. Control de calidad de variantes e individuos
plink2 --bfile crudo --geno 0.02 --mind 0.02 --maf 0.01 \
  --hwe 1e-6 --make-bed --out qc

# 2. Podar por LD y calcular 10 componentes principales
plink2 --bfile qc --indep-pairwise 200 50 0.2 --out podado
plink2 --bfile qc --extract podado.prune.in --pca 10 --out pcs

# 3. Asociación con las PCs como covariables
plink2 --bfile qc --pheno fenotipo.txt --covar pcs.eigenvec \
  --glm hide-covar --out gwas
```

PLINK (Purcell et al., 2007) se convirtió en la navaja suiza del análisis de asociación: control de calidad, estadísticas de LD, PCA, pruebas de asociación y manejo de formatos, todo sobre un formato binario compacto (*.bed/.bim/.fam*). Su segunda generación (Chang et al., 2015) reescribió el núcleo con operaciones a nivel de bits y paralelismo, y lo aceleró en uno a varios órdenes de magnitud, lo que le permite trabajar con biobancos de cientos de miles de personas.

10.3.6 Lo que los GWAS encontraron, y lo que falta

Los resultados publicados se reúnen en el GWAS Catalog, mantenido por el NHGRI y el EBI, que recopila cientos de miles de asociaciones curadas a partir de miles de publicaciones y ofrece además un repositorio para depositar estadísticas resumidas completas (Sollis et al., 2023). Esa masa de resultados ha confirmado que la mayoría de los rasgos complejos son *altamente poligénicos*: miles de variantes, cada una con un efecto diminuto, casi siempre en regiones no codificantes (Visscher et al., 2017).

Manolio et al. (2009) plantearon el problema de la *heredabilidad faltante*: para la estatura, cuya heredabilidad estimada en estudios de familias ronda el 80 %, las decenas de loci conocidos entonces explicaban apenas un 5 % de la varianza. Las

explicaciones propuestas incluían variantes comunes de efecto demasiado pequeño para superar el umbral, variantes raras mal etiquetadas por los chips, variantes estructurales, interacciones y sobreestimaciones de la heredabilidad familiar. La primera explicación resultó ser la más importante: la heredabilidad SNP estimada con modelos como el de la ecuación (10.26) recupera una parte sustancial de la heredabilidad faltante, y cada aumento del tamaño de muestra ha traído nuevos loci, como predice la curva de potencia de la figura 10.12b.

⚠ Asociación no es causalidad (ni predicción universal)

Un GWAS encuentra correlaciones entre variantes y rasgos en una población concreta. La mayoría de los estudios se ha hecho en personas de ascendencia europea, y como el LD y las frecuencias alélicas difieren entre poblaciones (sección 10.2), una variante etiqueta que funciona bien en una población puede no hacerlo en otra. Las puntuaciones de riesgo poligénico construidas con esos resultados pierden buena parte de su capacidad predictiva cuando se aplican a otras ascendencias. Diversificar las cohortes es hoy una prioridad científica, no solo ética.

Un GWAS es un millón de regresiones con un solo enemigo sistemático, la estructura poblacional, y un solo enemigo aleatorio, la multiplicidad. El gráfico QQ vigila al primero; el umbral de 5×10^{-8} , al segundo.

☰ En resumen

- Bajo apareamiento aleatorio, las frecuencias genotípicas son $p^2 : 2pq : q^2$ tras una sola generación (Hardy-Weinberg). Las desviaciones se contrastan con χ^2 o, para variantes raras, con la prueba exacta; el coeficiente F mide el déficit de heterocigotos.
- El modelo de Wright-Fisher es una cadena de Márkov binomial: la frecuencia es una martingala, un alelo neutral se fija con probabilidad igual a su frecuencia, y la heterocigosidad decae como $(1 - 1/2N_e)^t$. El tamaño efectivo es una media armónica dominada por los cuellos de botella.

- La difusión de Kimura da distribuciones, tiempos de fijación ($4N \ln 2$ para $p = 1/2$) y la probabilidad de fijación con selección; lo que importa es $4N_e s$. El coalescente describe la misma deriva hacia atrás en el tiempo.
- El LD se mide con D , D' (historia de recombinación) y r^2 (poder de sustitución entre marcadores); decae como $(1 - c)^t$ y, en equilibrio con la deriva, $E[r^2] \approx 1/(1 + 4N_e c)$. Los puntos calientes de recombinación organizan el genoma en bloques de haplotipos.
- El PCA de la matriz de genotipos estandarizada revela estructura poblacional sin etiquetas previas; F_{ST} la cuantifica.
- Un GWAS ajusta una regresión aditiva por variante. La estratificación infla todo el genoma ($\lambda_{GC} > 1$) y se corrige con componentes principales o modelos mixtos. El umbral 5×10^{-8} controla el error por familia para un millón de pruebas efectivas; Benjamini-Hochberg controla la tasa de falsos descubrimientos.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 10.1** ●●● En una muestra de 500 personas hay 180 *AA*, 220 *Aa* y 100 *aa*. Calcule $\hat{p}$, los recuentos esperados bajo Hardy-Weinberg, el estadístico X^2 , su valor p y $\hat{F}$. ¿Qué explicaciones biológicas y técnicas consideraría?
- 10.2** ●●● Una población de tortugas pasa, durante cinco generaciones, por los tamaños 500, 40, 40, 500 y 500. Calcule su tamaño efectivo y la fracción de heterocigosidad que conserva al final.
- 10.3** ●●● Con las frecuencias haplotípicas $p_{AB} = 0,4$, $p_{Ab} = 0,1$, $p_{aB} = 0,1$, $p_{ab} = 0,4$, calcule D , D' y r^2 . ¿Cuántas generaciones tarda D en reducirse a la mitad si $c = 0,02$?
- 10.4** ●●● Use la función `wright_fisher` para estimar por simulación la probabilidad de fijación y el tiempo medio hasta la fijación de una mutación nueva ($p_0 = 1/2N$) con $N = 100$ y $s \in \{0; 0,005; 0,02\}$. Compare con la ecuación (10.14).
- 10.5** ●●● Demuestre que $D = p_{AB}p_{ab} - p_{Ab}p_{aB}$ a partir de la definición $D = p_{AB} - p_A p_B$, y que r^2 es el cuadrado de la correlación de Pearson entre los indicadores alélicos de ambos loci.

- 10.6** ●●● Descargue los genotipos de dos poblaciones del Proyecto 1000 Genomas para un cromosoma, pade por LD con PLINK y realice el PCA con `pca_genotipos`. Estime F_{ST} entre ambas y compare el primer autovalor con el umbral de detección de Patterson et al.
- 10.7** ●●● Simule un GWAS de 5000 individuos y 20 000 SNP independientes sin efectos genéticos. Verifique que $\lambda_{GC} \approx 1$ y que el número de variantes con $p < 10^{-4}$ es aproximadamente 2. Añada luego una estratificación como la del ejemplo 10.5 y estudie cómo crece λ_{GC} con la diferencia de medias entre poblaciones.
- 10.8** ●●● A partir de la ecuación (10.24), derive una fórmula aproximada para el tamaño de muestra necesario para detectar con potencia $1 - \beta$ una variante que explica q^2 de la varianza al nivel α . ¿Cómo cambia n si la variante no está genotipada y el mejor marcador tiene $r^2 = 0,6$ con ella?
- 10.9** ●●● Demuestre que el procedimiento de Benjamini-Hochberg controla la FDR al nivel $\alpha M_0/M$ cuando los valores p son independientes. (Pista: escriba la FDR como suma sobre las hipótesis nulas verdaderas y condicione en los demás valores p .)
- 10.10** ●●● Resuelva la ecuación de difusión hacia atrás para el caso neutral y demuestre la ecuación (10.15). Compruebe numéricamente con `wright_fisher` que, para $N = 50$ y $p = 0,1$, el tiempo medio hasta la absorción coincide con la predicción.

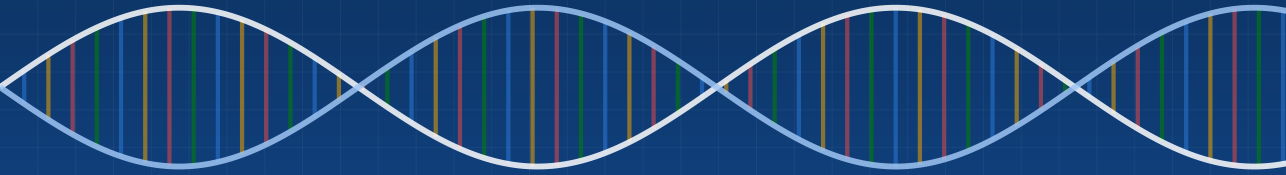
Referencias del capítulo

- Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1), 289-300. <https://doi.org/10.1111/j.2517-6161.1995.tb02031.x>
- Chang, C. C., Chow, C. C., Tellier, L. C. A. M., Vattikuti, S., Purcell, S. M., & Lee, J. J. (2015). Second-generation PLINK: Rising to the challenge of larger and richer datasets. *GigaScience*, 4(1), 7. <https://doi.org/10.1186/s13742-015-0047-8>
- Devlin, B., & Roeder, K. (1999). Genomic control for association studies. *Biometrics*, 55(4), 997-1004. <https://doi.org/10.1111/j.0006-341X.1999.00997.x>
- Fisher, R. A. (1930). *The genetical theory of natural selection*. Clarendon Press. <https://doi.org/10.5962/bhl.title.27468>

- Gabriel, S. B., Schaffner, S. F., Nguyen, H., Moore, J. M., Roy, J., Blumenstiel, B., Higgins, J., DeFelice, M., Lochner, A., Faggart, M., Liu-Cordero, S. N., Rotimi, C., Adeyemo, A., Cooper, R., Ward, R., Lander, E. S., Daly, M. J., & Altshuler, D. (2002). The structure of haplotype blocks in the human genome. *Science*, 296(5576), 2225-2229. <https://doi.org/10.1126/science.1069424>
- Hardy, G. H. (1908). Mendelian proportions in a mixed population. *Science*, 28(706), 49-50. <https://doi.org/10.1126/science.28.706.49>
- Hill, W. G., & Robertson, A. (1968). Linkage disequilibrium in finite populations. *Theoretical and Applied Genetics*, 38(6), 226-231. <https://doi.org/10.1007/BF01245622>
- Kimura, M. (1955). Solution of a process of random genetic drift with a continuous model. *Proceedings of the National Academy of Sciences*, 41(3), 144-150. <https://doi.org/10.1073/pnas.41.3.144>
- Kimura, M. (1962). On the probability of fixation of mutant genes in a population. *Genetics*, 47(6), 713-719. <https://doi.org/10.1093/genetics/47.6.713>
- Kingman, J. F. C. (1982). The coalescent. *Stochastic Processes and their Applications*, 13(3), 235-248. [https://doi.org/10.1016/0304-4149\(82\)90011-4](https://doi.org/10.1016/0304-4149(82)90011-4)
- Lewontin, R. C. (1964). The interaction of selection and linkage. I. General considerations; heterotic models. *Genetics*, 49(1), 49-67. <https://doi.org/10.1093/genetics/49.1.49>
- Loh, P.-R., Tucker, G., Bulik-Sullivan, B. K., Vilhjálmsson, B. J., Finucane, H. K., Salem, R. M., Chasman, D. I., Ridker, P. M., Neale, B. M., Berger, B., Patterson, N., & Price, A. L. (2015). Efficient Bayesian mixed-model analysis increases association power in large cohorts. *Nature Genetics*, 47(3), 284-290. <https://doi.org/10.1038/ng.3190>
- Manolio, T. A., Collins, F. S., Cox, N. J., Goldstein, D. B., Hindorff, L. A., Hunter, D. J., McCarthy, M. I., Ramos, E. M., Cardon, L. R., Chakravarti, A., Cho, J. H., Guttmacher, A. E., et al. (2009). Finding the missing heritability of complex diseases. *Nature*, 461(7265), 747-753. <https://doi.org/10.1038/nature08494>
- Novembre, J., Johnson, T., Bryc, K., Kutalik, Z., Boyko, A. R., Auton, A., Indap, A., King, K. S., Bergmann, S., Nelson, M. R., Stephens, M., & Bustamante, C. D. (2008). Genes mirror geography within Europe. *Nature*, 456(7218), 98-101. <https://doi.org/10.1038/nature07331>
- Patterson, N., Price, A. L., & Reich, D. (2006). Population structure and eigenanalysis. *PLoS Genetics*, 2(12), e190. <https://doi.org/10.1371/journal.pgen.0020190>
- Pe'er, I., Yelensky, R., Altshuler, D., & Daly, M. J. (2008). Estimation of the multiple testing burden for genomewide association studies of nearly all common variants. *Genetic Epidemiology*, 32(4), 381-385. <https://doi.org/10.1002/gepi.20303>
- Price, A. L., Patterson, N. J., Plenge, R. M., Weinblatt, M. E., Shadick, N. A., & Reich, D. (2006). Principal components analysis corrects for stratification in genome-wide association studies. *Nature Genetics*, 38(8), 904-909. <https://doi.org/10.1038/ng1847>
- Purcell, S., Neale, B., Todd-Brown, K., Thomas, L., Ferreira, M. A. R., Bender, D., Maller, J., Sklar, P., de Bakker, P. I. W., Daly, M. J., & Sham, P. C. (2007). PLINK: A tool set

- for whole-genome association and population-based linkage analyses. *The American Journal of Human Genetics*, 81(3), 559-575. <https://doi.org/10.1086/519795>
- Sollis, E., Mosaku, A., Abid, A., Buniello, A., Cerezo, M., Gil, L., Groza, T., Güneş, O., Hall, P., Hayhurst, J., Ibrahim, A., Ji, Y., et al. (2023). The NHGRI-EBI GWAS Catalog: Knowledgebase and deposition resource. *Nucleic Acids Research*, 51(D1), D977-D985. <https://doi.org/10.1093/nar/gkac1010>
- The 1000 Genomes Project Consortium. (2015). A global reference for human genetic variation. *Nature*, 526(7571), 68-74. <https://doi.org/10.1038/nature15393>
- The International HapMap Consortium. (2005). A haplotype map of the human genome. *Nature*, 437(7063), 1299-1320. <https://doi.org/10.1038/nature04226>
- The Wellcome Trust Case Control Consortium. (2007). Genome-wide association study of 14,000 cases of seven common diseases and 3,000 shared controls. *Nature*, 447(7145), 661-678. <https://doi.org/10.1038/nature05911>
- Visscher, P. M., Wray, N. R., Zhang, Q., Sklar, P., McCarthy, M. I., Brown, M. A., & Yang, J. (2017). 10 years of GWAS discovery: Biology, function, and translation. *The American Journal of Human Genetics*, 101(1), 5-22. <https://doi.org/10.1016/j.ajhg.2017.06.005>
- Weir, B. S., & Cockerham, C. C. (1984). Estimating F-statistics for the analysis of population structure. *Evolution*, 38(6), 1358-1370. <https://doi.org/10.1111/j.1558-5646.1984.tb05657.x>
- Wigginton, J. E., Cutler, D. J., & Abecasis, G. R. (2005). A note on exact tests of Hardy-Weinberg equilibrium. *The American Journal of Human Genetics*, 76(5), 887-893. <https://doi.org/10.1086/429864>
- Wright, S. (1931). Evolution in Mendelian populations. *Genetics*, 16(2), 97-159. <https://doi.org/10.1093/genetics/16.2.97>
- Yang, J., Lee, S. H., Goddard, M. E., & Visscher, P. M. (2011). GCTA: A tool for genome-wide complex trait analysis. *The American Journal of Human Genetics*, 88(1), 76-82. <https://doi.org/10.1016/j.ajhg.2010.11.011>

Transcriptómica (RNA-seq)



Los estudios que usan este método ya han alterado nuestra visión de la extensión y la complejidad de los transcriptomas eucariotas.

— Wang et al. (2009)

● Al terminar este capítulo usted podrá

- Diseñar un experimento de RNA-seq con réplicas biológicas suficientes y describir el camino que va del ARN extraído a una matriz de conteos, por alineamiento con empalmes o por pseudoalineamiento.
- Formular la cuantificación de isoformas como un problema de máxima verosimilitud con lecturas ambiguas, resolverlo con el algoritmo EM y distinguir con precisión RPKM, TPM y conteos estimados.
- Explicar por qué los conteos requieren normalización por composición (TMM, mediana de razones) y por qué su variabilidad biológica se modela con la binomial negativa y no con la distribución de Poisson.
- Ajustar e interpretar un modelo lineal generalizado binomial negativo con contracción empírica de dispersiones y de *log fold changes*, y leer gráficos MA y *volcano*.

- Controlar la tasa de falsos descubrimientos (Benjamini-Hochberg, q -valores, filtrado independiente) y diagnosticar un análisis con el histograma de valores p .
- Realizar e interpretar análisis de enriquecimiento funcional (sobrerrepresentación con GO y KEGG, corrección por sesgo de longitud y GSEA).

Salvo excepciones, las células de un organismo comparten el mismo genoma. Una neurona y un hepatocito se distinguen por los genes que expresan y por su nivel de expresión. El *transcriptoma* reúne las moléculas de ARN presentes en una muestra en un momento dado y permite estudiar ese estado funcional. Su medición ha pasado de los *northern blots* y los microarreglos al RNA-seq: convertimos el ARN en ADN complementario, lo secuenciamos y contamos fragmentos, en lugar de medir solo transcritos reconocidos por sondas predefinidas. Mortazavi et al. (2008) mostraron que, con decenas de millones de lecturas, el método cubre cinco órdenes de magnitud de expresión, distingue isoformas y descubre transcritos no anotados.

Pero contar bien es difícil. Una lectura puede ser compatible con varias isoformas de un gen; el número de lecturas de un gen depende de su longitud, de la profundidad de secuenciación y de lo que expresen *los demás* genes; y la variación entre réplicas biológicas es mucho mayor que la del muestreo. Encima, al comparar dos condiciones hacemos veinte mil pruebas estadísticas a la vez. Este capítulo recorre, con el rigor que el problema exige, las cuatro etapas del análisis estándar. Primero convertiremos lecturas en abundancias de transcritos con el algoritmo EM y el pseudoalineamiento (sección 11.1). Luego haremos comparables las muestras y modelaremos la variabilidad con la binomial negativa (sección 11.2). Después detectaremos genes diferencialmente expresados controlando la tasa de falsos descubrimientos (sección 11.3). Por último, traduciremos listas de genes en hipótesis biológicas mediante el enriquecimiento funcional (sección 11.4). La revisión de Conesa et al. (2016) es la mejor guía general del campo, y el libro de Holmes y Huber (2019) ofrece el fondo estadístico con un enfoque muy cercano al de este capítulo.

11.1 Cuantificación de transcritos

Tiras de papel de varias ediciones

Imagine que una trituradora ha convertido en tiras la biblioteca de una escuela y que le piden estimar cuántos ejemplares había de cada libro. Muchas tiras son inconfundibles: contienen una frase que solo aparece en un libro. Pero la biblioteca tenía tres ediciones de *Cien años de soledad*, que comparten casi todo el texto y difieren en el prólogo o en un capítulo añadido. Una tira del capítulo tercero pudo salir de cualquiera de las tres. ¿Qué hacer con ella? Descartarla desperdicia información; asignarla al azar introduce ruido. Lo sensato es repartirla *en proporción* a lo que ya sabemos: si las tiras exclusivas del prólogo de la segunda edición abundan, es probable que la tira ambigua también venga de ella. Y como ese reparto cambia nuestra estimación de cuántos ejemplares hay de cada edición, conviene repetir el razonamiento hasta que deje de cambiar. Ese ir y venir entre “repartir” y “volver a contar” es exactamente el algoritmo EM con el que se cuantifican las isoformas.

11.1.1 Del ARN a las lecturas: diseño y bibliotecas

Un experimento de RNA-seq empieza mucho antes del secuenciador, y las decisiones que se toman en el laboratorio limitan lo que la estadística podrá rescatar después. Conesa et al. (2016) enumeran las principales.

Réplicas biológicas. Una *réplica biológica* es una unidad experimental independiente (otro ratón, otro paciente, otro cultivo sembrado por separado); una *réplica técnica* es la misma muestra secuenciada dos veces. Las réplicas técnicas solo miden el ruido del muestreo de lecturas, que, como veremos en la sección 11.2, es pequeño comparado con la variación entre individuos. La inferencia sobre “el efecto del tratamiento” se refiere a la población de la que salen los individuos, y la única forma de estimar su variabilidad es tener varios. Tres réplicas por condición es el mínimo con el que los métodos de este capítulo funcionan; con más réplicas la potencia crece mucho más que con más profundidad, sobre todo para genes de expresión moderada.

Lotes y aleatorización. Si todas las muestras control se extraen un lunes y todas las tratadas un martes, el efecto del día (el *efecto de lote*) queda confundido con el del tratamiento y ningún método puede separarlos. Hay que repartir las condiciones entre lotes, carriles e índices de forma balanceada y registrar esas variables para incluirlas en el modelo (sección 11.2).

Selección del ARN. Más del 80 % del ARN total es ARN ribosómico. Para no gastar lecturas en él, se *seleccionan* los ARNm por su cola poli(A) con oligo-dT, o se *depleciona* el ARNr con sondas específicas. La primera opción da bibliotecas más limpias de ARN mensajero maduro; la segunda conserva ARN no poliadenilados (muchos ARN largos no codificantes, histonas) y pre-ARNm con intrones, útil con muestras degradadas.

Biblioteca. El ARN se fragmenta, se retrotranscribe a ADNc y se ligan adaptadores (capítulo 6). Los protocolos *con hebra (stranded)* marcan químicamente una de las hebras del ADNc y permiten saber si la lectura procede del transcrito sentido o antisentido, información valiosa cuando dos genes se solapan en hebras opuestas. Las lecturas pareadas mejoran la asignación a isoformas, porque el fragmento completo abarca más uniones exón-exón.

Profundidad. Entre 20 y 40 millones de fragmentos por muestra suelen bastar para expresión diferencial a nivel de gen en organismos con genomas del tamaño del humano; el estudio de isoformas o de transcritos poco abundantes exige más. La figura 11.1 resume el flujo completo que seguiremos.

11.1.2 Alinear lecturas que atraviesan intrones

En eucariotas, el ARNm maduro es una concatenación de exones; los intrones, a veces de decenas de kilobases, han sido eliminados. Una lectura de 100 bases tomada cerca de una unión exón-exón queda, en el genoma, partida en dos trozos separados por el intrón. Los alineadores de ADN del capítulo 7 no admiten ese salto: necesitamos alineadores *con empalmes (spliced aligners)*.

Dobin et al. (2013) diseñaron STAR alrededor de una idea sencilla: buscar, desde el inicio de la lectura, el *prefijo mapeable más largo (maximal mappable prefix)* que coincide exactamente con el genoma, usando un arreglo de sufijos sin comprimir. Si la lectura cruza una unión, el prefijo se detiene justo en el borde del exón; la búsqueda se reinicia con el resto de la lectura y encuentra el siguiente exón más adelante. Las semillas así obtenidas se agrupan y se “cosen” con un alineamiento local que permite un hueco largo (el intrón), favoreciendo los motivos canónicos GT...AG en sus bordes. El precio es memoria (unos 30 GB para el genoma humano); la recompensa, una velocidad que los autores cifraron en más de cincuenta veces la de los alineadores de su tiempo. HISAT2 (Kim et al., 2019) toma el camino opuesto: comprime el genoma, junto con variantes conocidas, en un índice FM sobre un *grafo*, dividido jerárquicamente en un índice global y miles de índices locales pequeños

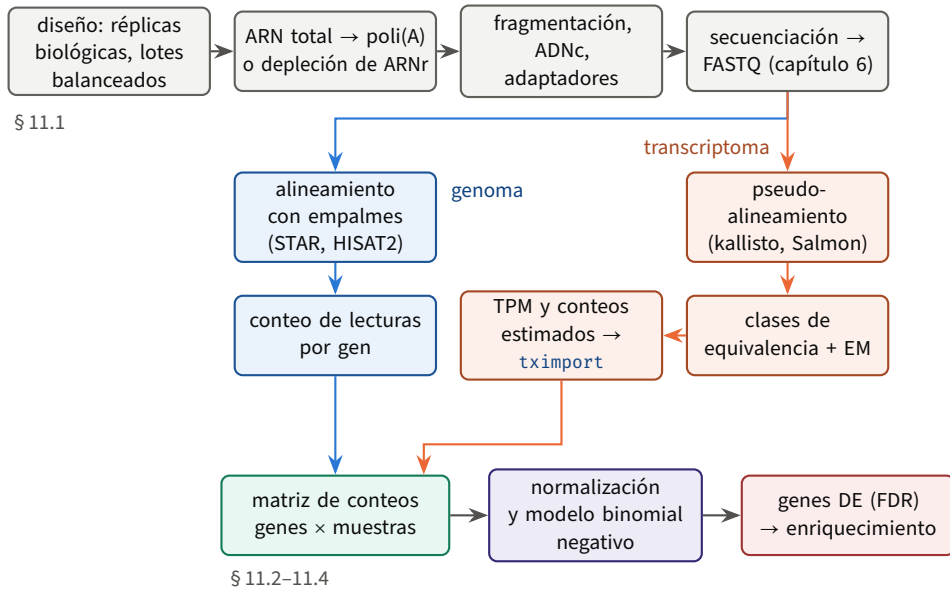


Figura 11.1. Flujo de un análisis de RNA-seq. Las decisiones del laboratorio (gris) condicionan todo lo demás. Las lecturas pueden seguir dos rutas: alinearse al *genoma* con un alineador que admite empalmes y contarse por gen (azul), o asignarse directamente al *transcriptoma* por pseudoalineamiento y cuantificarse por isoforma con el algoritmo EM (naranja); *tximport* agrega después las estimaciones por transcrito al nivel de gen. Ambas rutas desembocan en una matriz de conteos, que es la entrada de la normalización, el modelo estadístico y el análisis funcional.

que permiten extender con rapidez los fragmentos cortos que caen a ambos lados de un intrón, con requisitos de memoria del orden de un ordenador portátil.

Una vez alineadas, las lecturas se *cuentan* por gen, sumando las que caen en la unión de sus exones. Es la ruta azul de la figura 11.1, sencilla y robusta. Pero descarta, o resuelve de forma arbitraria, las lecturas que caen en exones compartidos por genes solapados, y renuncia a distinguir isoformas.

11.1.3 El problema de las isoformas

El empalme alternativo hace que un mismo gen produzca varios transcritos (*isoformas*) que comparten exones. La figura 11.2 muestra un gen de juguete con cuatro exones y tres isoformas. Una lectura que atraviesa la unión E1–E2 solo puede venir de T1; una que cae dentro de E4 es compatible con las tres. Si queremos saber

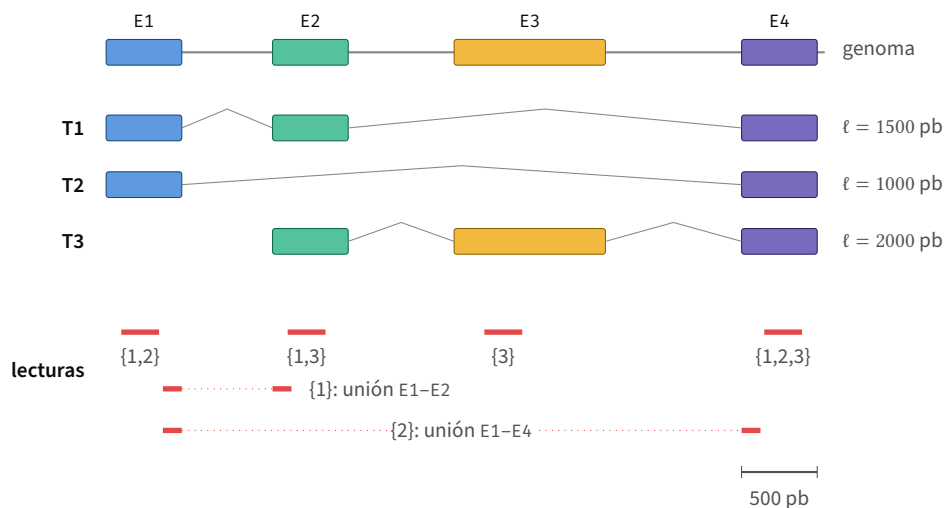


Figura 11.2. Un gen de juguete con cuatro exones y tres isoformas: T1 (E1-E2-E4), T2 (E1-E4, que omite el segundo exón) y T3 (E2-E3-E4, con un promotor alternativo). Cada lectura (rojo; las líneas punteadas indican el intrón que salta) se etiqueta con el conjunto de isoformas con las que es compatible. Solo las lecturas que tocan una unión o un exón exclusivo son informativas por sí solas; las demás, como las que caen en E4, forman *clases de equivalencia* ambiguas que el algoritmo EM debe repartir.

cuánto de cada isoforma hay en la muestra, las lecturas ambiguas, que suelen ser la mayoría, tienen que repartirse de algún modo.

11.1.4 Un modelo generativo de fragmentos

Para repartir con criterio necesitamos un modelo de cómo se generan los fragmentos. El modelo de RSEM (Li & Dewey, 2011), que heredan kallisto y Salmon, es el siguiente. Hay T transcritos; el transcrito t tiene longitud ℓ_t y está presente en la muestra en una fracción molar τ_t (fracción de *moléculas*, $\sum_t \tau_t = 1$). Un fragmento se genera eligiendo primero un transcrito y después una posición de inicio uniforme a lo largo de él. Como un transcrito largo ofrece más posiciones de inicio, la probabilidad de que un fragmento venga de t no es τ_t sino $\alpha_t \propto \tau_t \tilde{\ell}_t$.

Definición 11.1. (Longitud efectiva)

Si la longitud de los fragmentos sigue una distribución P_F , el número esperado de posiciones de inicio válidas en el transcrito t es

$$\tilde{\ell}_t = \sum_{l=1}^{\ell_t} P_F(l) (\ell_t - l + 1) \approx \ell_t - \mu_F + 1, \quad (11.1)$$

donde la aproximación vale cuando ℓ_t es mucho mayor que los fragmentos.

DONDE

ℓ_t	Longitud del transcrito t (en nucleótidos).
$\tilde{\ell}_t$	Longitud efectiva: número esperado de posiciones donde puede empezar un fragmento completo.
$P_F(l)$	Probabilidad de que un fragmento mida l nucleótidos, estimada de los pares de lecturas.
μ_F	Longitud media de los fragmentos.

La longitud efectiva importa sobre todo para transcritos cortos: uno de 300 nucleótidos con fragmentos de 250 tiene $\tilde{\ell} \approx 51$, seis veces menos posiciones de las que su longitud sugiere. Con ella, la relación entre las dos fracciones es

$$\alpha_t = \frac{\tau_t \tilde{\ell}_t}{\sum_u \tau_u \tilde{\ell}_u}, \quad \tau_t = \frac{\alpha_t / \tilde{\ell}_t}{\sum_u \alpha_u / \tilde{\ell}_u}. \quad (11.2)$$

DONDE

τ_t	Fracción de <i>moléculas</i> de ARN que son del transcrito t (la cantidad biológica de interés).
α_t	Fracción de <i>fragmentos</i> secuenciados que proceden de t (lo que las lecturas miden directamente).

Sea ahora $y_{jt} \in \{0, 1\}$ la indicadora de que el fragmento j es compatible con t . Ignorando por el momento sesgos y errores de secuenciación, la probabilidad de observar el fragmento j es la de elegir un transcrito compatible y una de sus $\tilde{\ell}_t$ posiciones:

$$\mathcal{L}(\alpha) = \prod_{j=1}^N \sum_{t=1}^T y_{jt} \frac{\alpha_t}{\tilde{\ell}_t} = \prod_C \left(\sum_{t \in C} \frac{\alpha_t}{\tilde{\ell}_t} \right)^{n_C}. \quad (11.3)$$

DONDE

N	Número total de fragmentos.
y_{jt}	1 si el fragmento j es compatible con el transcrito t ; o si no.
C	Clase de equivalencia: conjunto de transcritos con el que es compatible un grupo de fragmentos.
n_C	Número de fragmentos de la clase C .

La segunda igualdad es la observación que hace rápidos a los cuantificadores modernos: dos fragmentos compatibles con el mismo conjunto de transcritos aportan el mismo factor a la verosimilitud, así que basta guardar las *clases de equivalencia* y sus conteos. Treinta millones de lecturas se resumen en unas pocas centenas de miles de clases.

11.1.5 El algoritmo EM

Maximizar la ecuación (11.3) directamente es incómodo: el logaritmo de una suma no se separa. Pero si supiéramos de qué transcrito vino cada fragmento (una variable *latente* z_j), la estimación sería trivial: $\hat{\alpha}_t$ sería la fracción de fragmentos con $z_j = t$. El algoritmo *Expectation-Maximization* de Dempster et al. (1977) convierte esa intuición en un método: alterna entre adivinar, de forma probabilística, los z_j (paso E) y reestimar α como si esa adivinanza fuera cierta (paso M).

Teorema 11.1. (EM para cuantificación de transcritos)

Partiendo de $\alpha^{(0)}$ (por ejemplo, uniforme), repita hasta convergencia:

$$\text{Paso E: } \hat{n}_{Ct}^{(k)} = n_C \frac{\alpha_t^{(k)} / \tilde{\ell}_t}{\sum_{u \in C} \alpha_u^{(k)} / \tilde{\ell}_u} \quad (t \in C), \quad (11.4)$$

$$\text{Paso M: } \alpha_t^{(k+1)} = \frac{1}{N} \sum_{C \ni t} \hat{n}_{Ct}^{(k)}. \quad (11.5)$$

Cada iteración no disminuye la log-verosimilitud: $\log \mathcal{L}(\alpha^{(k+1)}) \geq \log \mathcal{L}(\alpha^{(k)})$.

DONDE

$\hat{n}_{Ct}^{(k)}$	Número esperado de fragmentos de la clase C que proceden de t , dada la estimación actual.
$\alpha^{(k)}$	Estimación de las fracciones de fragmentos en la iteración k .
$C \ni t$	Suma sobre todas las clases que contienen a t .

Observe la ecuación (11.4) con atención, porque es el lugar donde más se equivocan las implementaciones ingenuas: el reparto es proporcional a $\alpha_t/\tilde{\ell}_t$, no a α_t . La razón es que la probabilidad de un fragmento *concreto* dado el transcrito es $1/\tilde{\ell}_t$: entre dos transcritos igual de representados en fragmentos, uno corto concentra esos fragmentos en menos posiciones y, por tanto, explica mejor cada uno.

¿Por qué la verosimilitud no baja nunca? Llamemos *responsabilidades* a las probabilidades que calcula el paso E, $y_{jt} = \mathbb{P}(z_j = t \mid j, \alpha^{(k)})$. Para cualquier α , la desigualdad de Jensen aplicada al logaritmo (una función cóncava) da

$$\log \sum_t y_{jt} \frac{\alpha_t}{\tilde{\ell}_t} = \log \sum_t y_{jt} \frac{y_{jt} \alpha_t / \tilde{\ell}_t}{y_{jt}} \geq \sum_t y_{jt} \log \frac{y_{jt} \alpha_t / \tilde{\ell}_t}{y_{jt}}, \quad (11.6)$$

con igualdad cuando $\alpha = \alpha^{(k)}$. El lado derecho, sumado sobre j , es una cota inferior de $\log \mathcal{L}(\alpha)$ que toca a la curva en $\alpha^{(k)}$; el paso M la maximiza (es un problema de multinomial con solución cerrada, la ecuación (11.5)), y como la cota está por debajo de la curva, la verosimilitud en el nuevo punto es al menos tan alta como en el anterior. La ecuación (11.3) es, además, cóncava en α , así que EM converge al máximo global, aunque puede hacerlo con lentitud cuando dos isoformas son casi indistinguibles.

Ejemplo 11.1. (EM paso a paso con tres isoformas)

Tomemos el gen de la figura 11.2 con fragmentos de $\mu_F = 200$: las longitudes efectivas son $\tilde{\ell} = (1301, 801, 1801)$. Se observaron $N = 1200$ fragmentos repartidos así:

Clase C	$\{T_1\}$	$\{T_2\}$	$\{T_3\}$	$\{T_1, T_2\}$	$\{T_1, T_3\}$	$\{T_1, T_2, T_3\}$
n_C	120	30	200	250	150	450

Iteración 1. Con $\alpha^{(0)} = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ los pesos $\alpha_t/\tilde{\ell}_t$ son proporcionales a $1/\tilde{\ell}_t$, de modo que la clase $\{T_1, T_2, T_3\}$ se reparte en (134,5; 218,4; 97,1) fragmentos: T_2 , el más corto, recibe la mayor parte. La clase $\{T_1, T_2\}$ se reparte en (95,3; 154,7)

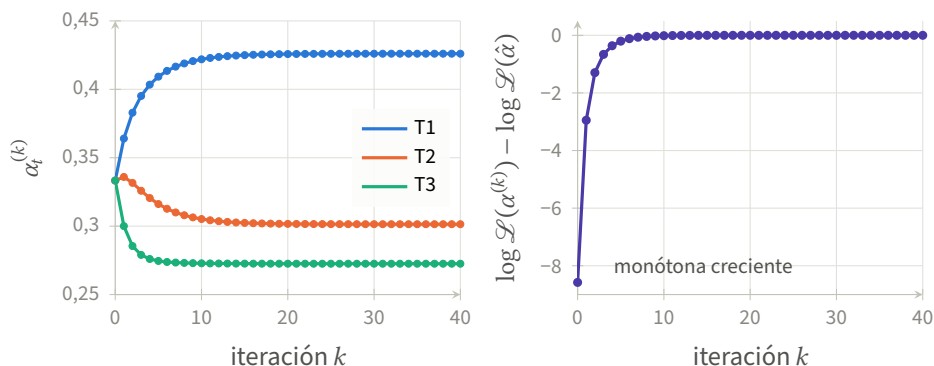


Figura 11.3. El algoritmo EM aplicado al ejemplo 11.1. **Izquierda:** trayectoria de las fracciones de fragmentos $\alpha_t^{(k)}$ desde el punto de partida uniforme. En la primera iteración T2 gana peso porque es corto y absorbe más de lo que le corresponde de las lecturas ambiguas; las lecturas exclusivas de T1 (clase {T1}, 120 fragmentos) y las de la unión E1–E4 (solo 30) terminan corrigiendo ese exceso. **Derecha:** la log-verosimilitud sube en cada paso, como garantiza el teorema 11.1; la mayor parte de la ganancia se obtiene en las primeras cinco iteraciones. Datos: `figuras/cap11/generar.py`.

y {T1,T3} en (87,1; 62,9). Sumando, T1 acumula $120 + 95,3 + 87,1 + 134,5 = 436,9$ fragmentos, y el paso M da $\alpha^{(1)} = (0,364; 0,336; 0,300)$.

Iteraciones siguientes. La segunda iteración da $\alpha^{(2)} = (0,383; 0,332; 0,286)$ y la tercera, $\alpha^{(3)} = (0,395; 0,326; 0,279)$. El cambio máximo cae por debajo de 10^{-4} en la iteración 20, y el punto fijo es $\hat{\alpha} = (0,426; 0,301; 0,273)$, es decir, 511,3, 361,7 y 327,0 fragmentos. La log-verosimilitud subió de $-9107,6$ a $-9099,0$.

Fracciones molares. Por la ecuación (11.2), $\hat{\tau} = (0,383; 0,440; 0,177)$. T2 recibe menos fragmentos que T1 pero tiene *más moléculas*, porque es más corto; T3 recibe un cuarto de los fragmentos pero solo un sexto de las moléculas, porque es el más largo. Confundir α con τ es confundir “cuántas lecturas” con “cuánto ARN”.

```
em_isoformas.py
```

```
import numpy as np
```



```
def em_isoformas(clases, leff, iters=500, tol=1e-8):
    """clases: lista de (tupla de índices, conteo); leff: array."""
    T = len(leff); N = sum(n for _, n in clases)
    alfa = np.full(T, 1.0 / T)
    for _ in range(iters):
        esperado = np.zeros(T)
        for C, n in clases:
            idx = list(C)
            w = alfa[idx] / leff[idx]
            esperado[idx] += n * w / w.sum()
        nuevo = esperado / N
        if np.abs(nuevo - alfa).max() < tol:
            break
        alfa = nuevo
    tau = (alfa / leff) / (alfa / leff).sum()
    return alfa, tau
```

De RPKM a RSEM

El primer gran estudio de RNA-seq en mamíferos (Mortazavi et al., 2008) ya tropezó con las lecturas que mapean en varios lugares: las asignó en proporción a las lecturas únicas de cada locus, una especie de “paso E” de una sola iteración. Li y Dewey (2011) formalizaron el problema como un modelo generativo completo, lo resolvieron con EM y añadieron intervalos de credibilidad por muestreo de Gibbs; su programa, RSEM, funciona incluso sin genoma de referencia, con un transcriptoma ensamblado *de novo*. Casi todos los cuantificadores posteriores son variaciones sobre ese modelo.

11.1.6 Pseudoalineamiento y mapeo selectivo

El modelo anterior no necesita saber *dónde* alinea un fragmento dentro de un transcrito, solo *con qué transcritos es compatible*. Bray et al. (2016) explotaron esa observación en kallisto. Construyen un grafo de De Bruijn del transcriptoma (con $k = 31$ por defecto), en el que cada k -mer lleva anotado el conjunto de transcritos que lo contienen. Para una lectura se buscan sus k -mers en el índice y se *intersecan* los conjuntos: el resultado es la clase de equivalencia de la lectura. Como los k -mers consecutivos de un mismo tramo del grafo tienen el mismo conjunto, basta consultar unos pocos y saltar el resto. No hay alineamiento base a base, y por eso el procedimiento se llama *pseudoalineamiento*. Los autores cuantificaron 30 millones de lecturas humanas en menos de tres minutos en un ordenador de escritorio, con una exactitud comparable a la de los mejores métodos basados en alineamiento.

Salmon (Patro et al., 2017) persigue el mismo objetivo con otra arquitectura. Su *cuasi-mapeo* usa un índice de sufijos del transcriptoma para localizar coincidencias exactas largas y obtener también la posición y la orientación aproximadas; la inferencia se hace en dos fases, una “en línea” mientras llegan las lecturas y otra “fuera de línea” que refina las abundancias sobre las clases de equivalencia con EM o con Bayes variacional. Su aportación central es incorporar al modelo los sesgos de las bibliotecas reales: la probabilidad de muestrear un fragmento depende de su contenido en GC, de la secuencia en torno a los extremos (los hexámeros aleatorios del cebado no son tan aleatorios) y de la posición a lo largo del transcrito (degradación en 5' o 3'). Salmon estima esos sesgos a partir de los propios datos y los usa para corregir las longitudes efectivas de cada transcrito, lo que mejora la reproducibilidad entre laboratorios. Versiones posteriores sustituyeron el cuasi-mapeo por un *alineamiento selectivo*, que verifica con un alineamiento rápido las coincidencias candidatas y puede usar el genoma como señuelo para evitar asignar al transcriptoma lecturas de intrones o de regiones no anotadas.

Cuantificación con Salmon

```
# 1. Índice del transcriptoma (con el genoma como señuelo)
grep '^>' genoma.fa | cut -d ' ' -f 1 | sed 's/> //' > decoys.txt
cat transcritos.fa genoma.fa > gentrome.fa
salmon index -t gentrome.fa -d decoys.txt -i idx_salmon -k 31

# 2. Cuantificación de una muestra pareada con corrección de sesgos
salmon quant -i idx_salmon -l A \
  -1 ctrl1_R1.fastq.gz -2 ctrl1_R2.fastq.gz \
  --validateMappings --gcBias --seqBias \
  -p 8 -o quant/ctrl1
# quant/ctrl1/quant.sf: Name Length EffectiveLength TPM NumReads
```

11.1.7 Unidades: RPKM, TPM y conteos

El archivo `quant.sf` ofrece dos columnas de abundancia que conviene no confundir. *NumReads* es el número *esperado* de fragmentos asignados al transcrito, $N\hat{\alpha}_i$; no es entero porque procede de repartos fraccionarios. *TPM* es una estimación de la fracción molar $\hat{\tau}_i$ multiplicada por un millón. Para comparar la expresión de genes de distinta longitud dentro de una muestra, Mortazavi et al. (2008) propusieron dividir el conteo por la longitud en kilobases y por el total de lecturas en millones, las

lecturas por kilobase por millón (RPKM, o FPKM si se cuentan fragmentos pareados). La otra unidad, TPM, invierte el orden de las operaciones:

$$\text{RPKM}_t = \frac{c_t}{(\tilde{\ell}_t/10^3)(N/10^6)}, \quad \text{TPM}_t = 10^6 \frac{c_t/\tilde{\ell}_t}{\sum_u c_u/\tilde{\ell}_u}. \quad (11.7)$$

DONDE

c_t	Fragmentos asignados al transcrito (o gen) t .
N	Fragmentos totales de la muestra, $\sum_u c_u$.
$\tilde{\ell}_t$	Longitud efectiva (en la práctica, a veces la longitud sin corregir).

La diferencia parece cosmética, pero no lo es. Los TPM de una muestra suman siempre 10^6 , y por la ecuación (11.2) estiman directamente $10^6 \tau_t$. Los RPKM suman $10^9/\bar{\ell}$, donde $\bar{\ell}$ es la longitud media, ponderada por fragmentos, de lo que se expresa en esa muestra, y esa media cambia de una muestra a otra. Wagner et al. (2012) mostraron que por eso los RPKM no son consistentes entre muestras: un mismo valor no significa la misma proporción de moléculas, y recomendaron usar TPM.

Ejemplo 11.2. (El mismo gen, dos unidades, dos historias)

Dos muestras expresan cuatro genes de longitudes 1, 2, 4 y 0,5 kb y se secuencian a 10 millones de fragmentos. En A, los cuatro tienen el mismo número de moléculas; en B, el gen 4 (el corto) tiene cuatro veces más moléculas que en A y los otros tres no cambian en números absolutos. En A, los fragmentos se reparten en proporción a moléculas $\times$ longitud: 1,33, 2,67, 5,33 y 0,67 millones; los cuatro RPKM valen 133 333 y los cuatro TPM, 250 000. En B, los fragmentos son 1,11, 2,22, 4,44 y 2,22 millones; los RPKM de los genes 1–3 bajan a 111 111 y el del gen 4 sube a 444 444, con una suma total de 777 778 frente a 533 333 en A. Los TPM son 142 857 para los genes 1–3 y 571 429 para el 4, con suma 10^6 . La proporción real de moléculas del gen 1 en B es $10/70 = 0,143$: exactamente lo que dice su TPM. Su RPKM, en cambio, sugiere una caída del 17 % cuando la fracción molar cayó un 43 %.

11.1.8 Del transcrito al gen: tximport

La expresión diferencial suele analizarse a nivel de gen, donde las estimaciones son más estables. Sumar los TPM de las isoformas de un gen da su fracción molar, pero los métodos de la sección 11.2 necesitan *conteos*, no TPM. Soneson et al. (2015)

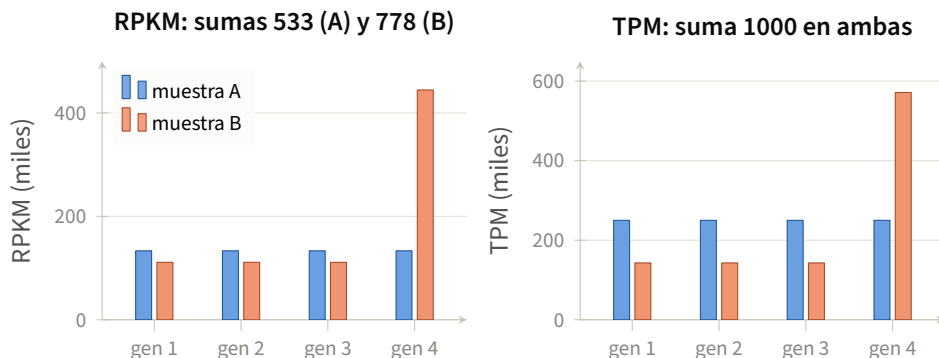


Figura 11.4. Los datos del ejemplo 11.2. Entre A y B, solo el gen 4 cambió su número absoluto de moléculas (se multiplicó por cuatro). Como el total de lecturas es fijo, los otros tres genes pierden lecturas en ambas unidades, pero los RPKM de cada muestra suman cantidades distintas y no pueden leerse como proporciones de moléculas. Los TPM suman siempre un millón y reflejan la fracción molar de cada transcrito. Ninguna de las dos unidades distingue “el gen 1 bajó” de “el gen 4 subió”: ese problema de *composición* se trata en la sección 11.2.

mostraron que la forma correcta de pasar de uno a otro no es contar lecturas por gen directamente sobre un alineamiento, sino sumar los conteos *estimados* por transcrito y, además, calcular para cada gen y muestra una *longitud efectiva media*, ponderada por la abundancia de sus isoformas. Si una muestra expresa sobre todo la isoforma larga y otra la corta, el gen producirá más fragmentos en la primera aunque tenga el mismo número de moléculas; la longitud media, usada como *desplazamiento* en el modelo, corrige ese efecto. Su paquete `tximport` hace esa agregación y encaja directamente con los métodos de la sección siguiente.

⚠ TPM no es una entrada para expresión diferencial

Los TPM sirven para describir y visualizar, pero no deben alimentar un modelo de conteos: al dividir por la longitud y reescalar se pierde la información sobre la precisión de cada medida (cien lecturas no son tan ruidosas como diez, aunque den el mismo TPM), y el reescalado a un millón hereda el problema de composición. Use los conteos estimados, con las longitudes medias como desplazamientos, tal como hace `tximport`.

11.2 Normalización y binomial negativa



Colab · 11.2 · Normalización y binomial negativa



repositorio del curso

El reparto de la pizza

Seis amigos piden una pizza de ocho porciones y cada uno come aproximadamente lo mismo. Al día siguiente vuelven con un séptimo invitado, un adolescente hambriento que se come la mitad de la pizza. Si solo le enseñan la *fracción* de pizza que comió cada uno, usted concluirá que los seis amigos perdieron el apetito: cada uno comió un doceavo en lugar de un sexto. Pero nadie cambió de hábitos salvo el invitado; lo que cambió es el denominador. El RNA-seq mide exactamente fracciones de una pizza de tamaño fijo, las lecturas de la muestra, y un puñado de genes muy expresados puede hacer que todos los demás parezcan disminuir.

11.2.1 Qué hay que corregir

El conteo K_{ij} del gen i en la muestra j depende de su expresión, pero también de factores técnicos: la *profundidad* de secuenciación, la *longitud* del gen, su contenido en GC y la *composición* del transcriptoma. Para comparar un *mismo gen* entre muestras, la longitud se cancela (salvo cambios de isoforma, que `tximport` corrige) y lo esencial es la profundidad y la composición.

La corrección más obvia divide por el total de lecturas: los *conteos por millón*, $\text{CPM}_{ij} = 10^6 K_{ij} / N_j$. Escribamos qué esperamos observar. Si μ_{ij} es el número de moléculas del gen i en la muestra j y L_i su longitud, la fracción de lecturas que produce es $\mu_{ij} L_i / S_j$, con

$$E[K_{ij}] = N_j \frac{\mu_{ij} L_i}{S_j}, \quad S_j = \sum_{g=1}^G \mu_{gj} L_g. \quad (11.8)$$

DONDE

K_{ij}	Conteo observado del gen i en la muestra j .
N_j	Total de lecturas asignadas en la muestra j (profundidad).
μ_{ij}, L_i	Número de moléculas del gen i en la muestra j y longitud del gen.
S_j	“Producción total de ARN” de la muestra: el tamaño de la pizza.

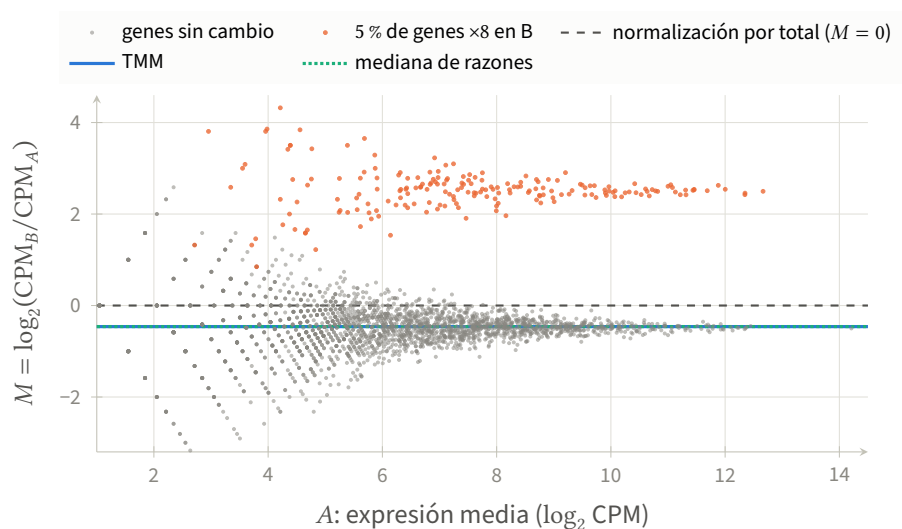


Figura 11.5. Efecto de composición en dos muestras simuladas con la misma profundidad. En B, un 5 % de los genes (naranja) multiplica por ocho su expresión y pasa a acaparar el 33 % de las lecturas (frente al 6 % en A). Los demás genes (gris) no cambiaron, pero con la normalización por total su M se centra en $-0,49$: parecen reprimidos. TMM ($\log_2 f = -0,461$, azul) y la mediana de razones ($-0,460$, puntos aqua) recuperan el centro de la nube de genes sin cambio, prácticamente sobre el valor teórico de $-0,486$; ambas líneas casi coinciden. Por debajo de $A \approx 4$ la nube se abre en abanico: con pocas lecturas, el cociente es muy ruidoso.

Dividir por N_j elimina la profundidad, pero deja el denominador S_j . Si un conjunto de genes se dispara en la muestra B, S_B crece y todos los demás genes ven reducida su fracción aunque su μ no haya cambiado. Robinson y Oshlack (2010) documentaron el efecto con datos reales de hígado y riñón: una parte pequeña de genes muy expresados en un tejido basta para sesgar el cociente de todos los demás, y la normalización por total convierte ese sesgo en miles de falsos positivos.

11.2.2 TMM: media recortada de cocientes

La solución de Robinson y Oshlack (2010) parte de un supuesto biológico: *la mayoría de los genes no cambia*. Si es así, sus cocientes de expresión entre muestras deben estar centrados en cero después de normalizar, y el desplazamiento de su

centro mide el sesgo de composición. Con una muestra de referencia r , se definen para cada gen el cociente logarítmico y la expresión media:

$$M_i = \log_2 \frac{K_{ij}/N_j}{K_{ir}/N_r}, \quad A_i = \frac{1}{2} \log_2 \left(\frac{K_{ij}}{N_j} \cdot \frac{K_{ir}}{N_r} \right). \quad (11.9)$$

Se descartan los genes con M extremo (por defecto, el 30 % de cada cola) y con A extremo (el 5 %), y se promedian los M restantes con pesos iguales al inverso de su varianza aproximada:

$$\log_2 f_j = \frac{\sum_{i \in \mathcal{R}} w_i M_i}{\sum_{i \in \mathcal{R}} w_i}, \quad w_i^{-1} = \frac{N_j - K_{ij}}{N_j K_{ij}} + \frac{N_r - K_{ir}}{N_r K_{ir}}. \quad (11.10)$$

DONDE

M_i, A_i	Cociente logarítmico y expresión media del gen i entre la muestra j y la referencia r .
$\mathcal{R}$	Genes que sobreviven al doble recorte.
w_i	Peso: inverso de la varianza aproximada (método delta) de M_i .
f_j	Factor TMM; el tamaño efectivo de la biblioteca es $N_j f_j$.

El recorte cumple el papel de la mediana: vuelve el estimador inmune a los genes que realmente cambian, siempre que sean minoría o que se repartan de forma aproximadamente simétrica entre subidas y bajadas. Los pesos dan más voz a los genes con muchas lecturas, cuyos cocientes son más precisos.

11.2.3 La mediana de razones

Anders y Huber (2010) propusieron, para DESeq, un estimador aún más directo. Se construye una *pseudomuestra de referencia* con la media geométrica de cada gen a través de todas las muestras, y el factor de tamaño de cada muestra es la mediana de sus cocientes con esa referencia:

$$\hat{s}_j = \underset{i: K_{iv} > 0 \ \forall v}{\text{mediana}} \frac{K_{ij}}{(\prod_{v=1}^m K_{iv})^{1/m}}. \quad (11.11)$$

DONDE

$\hat{s}_j$	Factor de tamaño de la muestra j ; los conteos normalizados son $K_{ij}/\hat{s}_j$.
m	Número de muestras.
$(\prod_v K_{iv})^{1/m}$	Media geométrica del gen i : la pseudomuestra de referencia.

La mediana se calcula solo sobre genes sin ceros, porque un cero anula la media geométrica. Como la media geométrica es simétrica respecto a todas las muestras, no hace falta elegir una referencia, y el producto de los factores es aproximadamente 1. En la figura 11.5 ambos métodos coinciden hasta la tercera cifra decimal.

Ejemplo 11.3. (Factores de tamaño a mano)

Cinco genes en tres muestras:

Gen	K_{i1}	K_{i2}	K_{i3}	media geom.
g1	500	1100	700	727,5
g2	80	190	105	116,9
g3	1200	2600	1550	1691,1
g4	30	64	42	43,2
g5	10	0	9	–

El gen g5 tiene un cero y queda fuera. Los cocientes con la media geométrica de la muestra 1 son 0,687, 0,685, 0,710 y 0,694; su mediana es $\hat{s}_1 = 0,691$. Del mismo modo, $\hat{s}_2 = 1,525$ y $\hat{s}_3 = 0,939$. La muestra 2 se secuenció, en efecto, a algo más del doble de profundidad que la 1. Observe que los cocientes de cada columna son muy parecidos entre sí: esa coherencia es la que justifica usar su mediana como factor común.

factores_tamano.py

```
import numpy as np
```

```
def factores_tamano(K):
```

```
    """K: matriz genes x muestras de conteos crudos."""
```

```
    K = np.asarray(K, dtype=float)
```

```
    ok = (K > 0).all(axis=1)           # genes sin ceros
```

```
    logK = np.log(K[ok])
```

```
    log_geo = logK.mean(axis=1, keepdims=True)
```

```
    return np.exp(np.median(logK - log_geo, axis=0))
```

```
K = [[500, 1100, 700], [80, 190, 105], [1200, 2600, 1550],
      [30, 64, 42], [10, 0, 9]]
```

```
print(factores_tamano(K).round(3))      # [0.691 1.525 0.939]
```


⚠ Cuando la mayoría sí cambia

TMM y la mediana de razones suponen que la mayoría de los genes no cambia, o que los cambios son equilibrados. Ese supuesto falla en situaciones como una amplificación transcripcional global (células con MYC hiperactivo que producen más ARN de todo tipo) o al comparar tejidos muy distintos. En esos casos ninguna normalización interna es fiable, y se recurre a controles externos añadidos en cantidad conocida a cada muestra (*spike-ins*) o a un número de células conocido.

11.2.4 De Poisson a la binomial negativa

Una vez normalizados, ¿cuánto varían los conteos? Si una biblioteca contiene un gen en proporción p y secuenciamos N fragmentos, el número de lecturas del gen sigue una binomial $\text{Bin}(N, p)$; con N enorme y p diminuta, eso es prácticamente una Poisson de media $\mu = Np$, cuya varianza es *igual* a la media. Las réplicas técnicas, secuenciar dos veces la misma biblioteca, se comportan en efecto así. Pero las réplicas biológicas añaden una segunda fuente de variación: la proporción p misma varía de un individuo a otro. El modelo natural es jerárquico.

Teorema 11.2. (Mezcla gamma-Poisson)

Si $\lambda \sim \text{Gamma}(\text{forma} = 1/\alpha, \text{escala} = \alpha\mu)$ y, condicionado a λ , $K \sim \text{Poisson}(\lambda)$, entonces K sigue una binomial negativa con

$$\mathbb{P}(K = k) = \frac{\Gamma(k + 1/\alpha)}{\Gamma(1/\alpha) k!} \left(\frac{1}{1 + \alpha\mu} \right)^{1/\alpha} \left(\frac{\alpha\mu}{1 + \alpha\mu} \right)^k, \quad (11.12)$$

y sus dos primeros momentos son

$$\mathbb{E}[K] = \mu, \quad \text{Var}[K] = \mu + \alpha\mu^2. \quad (11.13)$$

DONDE

λ	Expresión “verdadera” del gen en un individuo concreto (variable entre réplicas biológicas).
μ	Media poblacional del conteo.
α	Dispersión: $\sqrt{\alpha}$ es el coeficiente de variación biológico de λ .

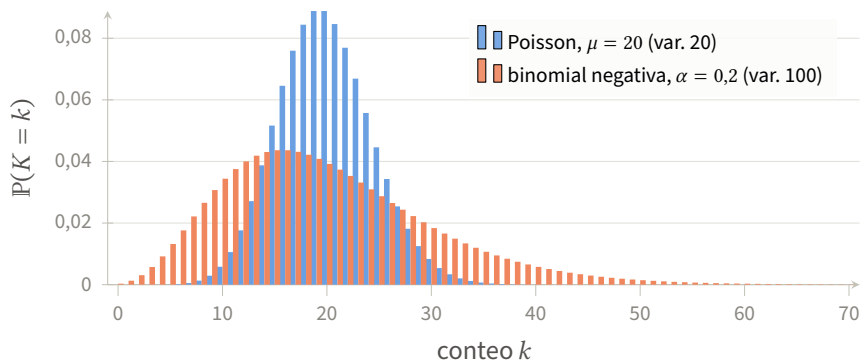


Figura 11.6. Dos distribuciones con la misma media. La Poisson (azul) concentra la masa entre 12 y 28 lecturas; la binomial negativa con dispersión $\alpha = 0,2$ (un coeficiente de variación biológico del 45 %, típico de muestras humanas) es cinco veces más variable. Un conteo de 40 o más tiene probabilidad 5×10^{-5} bajo la Poisson, pero 0,044 bajo la binomial negativa: lo que la Poisson declararía un cambio extraordinario es, con variación biológica, un suceso corriente.

La varianza de la ecuación (11.13) se obtiene con la ley de la varianza total, sin necesidad de la función de masa: $\text{Var}[K] = \text{E}[\text{Var}(K \mid \lambda)] + \text{Var}[\text{E}(K \mid \lambda)] = \text{E}[\lambda] + \text{Var}[\lambda] = \mu + \alpha\mu^2$. Los dos sumandos tienen una lectura transparente. El término μ es el ruido de Poisson del muestreo de lecturas, que domina para genes poco expresados. El término $\alpha\mu^2$ es la variación biológica, que domina para genes muy expresados: el coeficiente de variación al cuadrado es $\text{Var}[K]/\mu^2 = 1/\mu + \alpha$, y tiende a α por mucho que se secuencie. *Más profundidad reduce el primer término; solo más réplicas permiten estimar y promediar el segundo.* Es el argumento cuantitativo a favor de las réplicas que anticipamos en la sección 11.1.

11.2.5 El modelo lineal generalizado

Con la distribución elegida, el modelo completo de DESeq2 (Love et al., 2014), casi idéntico al de edgeR (Robinson et al., 2010), es un *modelo lineal generalizado* (GLM) con enlace logarítmico:

$$K_{ij} \sim \text{NB}(\mu_{ij}, \alpha_i), \quad \mu_{ij} = s_j q_{ij}, \quad \log_2 q_{ij} = \sum_r x_{jr} \beta_{ir}. \quad (11.14)$$

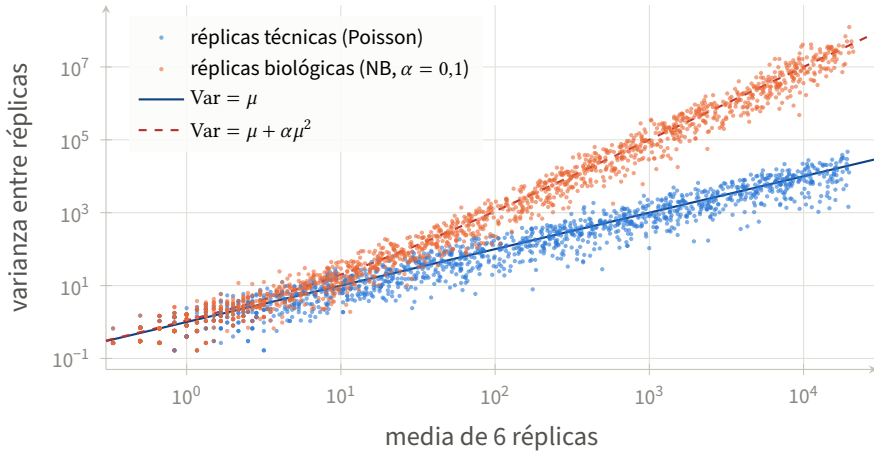


Figura 11.7. Relación media-varianza en 1500 genes simulados con seis réplicas, en escala doble logarítmica. Con variación puramente técnica (azul) la nube sigue la diagonal $\text{Var} = \mu$. Con variación biológica (naranja) coincide con ella para genes poco expresados, donde manda el ruido de muestreo, pero se separa con pendiente 2 a partir de $\mu \approx 1/\alpha = 10$. Para los genes con media superior a 1000, la varianza es unas 330 veces la media. La dispersión de los puntos alrededor de cada curva es el ruido de estimar una varianza con solo seis observaciones, y anticipa el problema de la sección 11.2: estimar α gen a gen es muy impreciso.

DONDE

s_j	Factor de tamaño de la muestra (ecuación (11.11)); entra como desplazamiento fijo, no se estima gen a gen.
q_{ij}	Expresión normalizada esperada del gen i en la muestra j .
x_{jr}	Matriz de diseño: condición, lote, sexo... de la muestra j .
β_{ir}	Coefficientes del gen i ; con un diseño “~ lote + condición”, el coeficiente de la condición es el $\log_2$ del cambio de expresión (LFC) corregido por lote.
α_i	Dispersión propia del gen i .

Esta formulación tiene tres virtudes. Los conteos se modelan *crudos*, con la normalización como desplazamiento, así que el modelo sabe que 10 lecturas son menos fiables que 10 000. El enlace logarítmico hace que los efectos sean multiplicativos, que es como se comporta la expresión. Y la matriz de diseño admite cualquier experimento: covariables, lotes, interacciones, datos pareados. Los β se estiman por

máxima verosimilitud con mínimos cuadrados iterativamente reponderados (IRLS), cuyos pesos para la NB son

$$W_{jj} = \frac{\mu_{ij}}{1 + \alpha_i \mu_{ij}}, \quad \widehat{\text{Cov}}(\hat{\beta}_i) = (X^\top W X)^{-1}. \quad (11.15)$$

DONDE

W	Matriz diagonal de pesos: la información de Fisher que aporta cada observación.
X	Matriz de diseño (m muestras $\times$ p coeficientes).

El peso tiende a $1/\alpha_i$ cuando μ crece: por muchos conteos que tenga una muestra, su información está acotada por la variación biológica. De aquí sale el error estándar de cada LFC, que usaremos en la sección 11.3.

11.2.6 Estimar la dispersión con pocas réplicas: contracción

Todo depende de α_i , y α_i es muy difícil de estimar con tres réplicas por grupo: la figura 11.7 muestra cuánto se dispersan las varianzas muestrales. Si subestimamos α_i , los errores estándar serán demasiado pequeños y aparecerán falsos positivos; si la sobreestimamos, perderemos potencia. La salida es compartir información entre genes. Aunque cada gen tiene su propia dispersión, los genes de expresión similar tienen dispersiones parecidas, y con veinte mil genes podemos aprender esa regularidad con gran precisión. El procedimiento de DESeq2, un ejemplo de *Bayes empírico*, tiene tres pasos:

1. **Estimación gen a gen**, $\hat{\alpha}_i^{\text{gw}}$, por máxima verosimilitud con el ajuste de Cox-Reid, que corrige el sesgo de estimar la dispersión después de haber estimado los β con las mismas observaciones.
2. **Tendencia**: se ajusta una curva $\alpha_{\text{tr}}(\bar{\mu})$ de las estimaciones gen a gen frente a la media normalizada.
3. **Contracción**: se combina la verosimilitud de cada gen con una distribución previa log-normal centrada en la tendencia y se toma el máximo *a posteriori*.

En fórmulas,

$$\alpha_{\text{tr}}(\bar{\mu}) = a_0 + \frac{a_1}{\bar{\mu}}, \quad \log \alpha_i \sim \mathcal{N}(\log \alpha_{\text{tr}}(\bar{\mu}_i), \sigma_d^2), \quad (11.16)$$

$$\hat{\alpha}_i^{\text{MAP}} = \arg \max_{\alpha} \left[\ell_i(\alpha) + \log p(\alpha) \right]. \quad (11.17)$$

DONDE

$\bar{\mu}_i$	Media de los conteos normalizados del gen i .
a_0, a_1	Parámetros de la tendencia: a_0 es la dispersión biológica asintótica; $a_1/\bar{\mu}$ recoge el exceso de variabilidad de los genes poco expresados.
σ_d^2	Varianza previa: cuánto se apartan de verdad las dispersiones de la tendencia, estimada restando a la varianza de los residuos la varianza de muestreo esperada.
$\ell_i(\alpha)$	Log-verosimilitud (ajustada por Cox-Reid) del gen i como función de la dispersión.

La cantidad de contracción se regula sola. Con pocas réplicas, la verosimilitud de cada gen es plana y la previa pesa mucho; con muchas, la verosimilitud es aguda y domina. Los genes cuya estimación gen a gen queda muy por encima de la tendencia (más de dos desviaciones previas) conservan su valor propio, para no ocultar una variabilidad real. edgeR sigue la misma filosofía con una verosimilitud ponderada entre la del gen y la común (Robinson et al., 2010). limma-voom toma otro camino: transforma los conteos a $\log_2$ CPM, estima la tendencia media-varianza de esos logaritmos y la convierte en un *peso de precisión* para cada observación, lo que permite usar toda la maquinaria de modelos lineales normales y la contracción de varianzas de limma (Law et al., 2014; Ritchie et al., 2015).

Ejemplo 11.4. (Un gen con media 20)

En la simulación de la figura 11.8, un gen con media normalizada $\bar{\mu} = 20,0$ tiene una dispersión verdadera de 0,162. Su estimación gen a gen es 0,351: más del doble, por el azar de tres réplicas. La tendencia en ese nivel de expresión vale 0,136, y la estimación contraída, 0,183, un compromiso entre ambas mucho más cercano a la verdad. Con 0,351 el gen habría perdido potencia; con la tendencia sola, se habría ignorado que es algo más variable que sus vecinos. La varianza previa usada fue $\hat{\sigma}_d^2 = 0,257$: la varianza robusta de los residuos logarítmicos alrededor de la tendencia ($0,950^2 = 0,903$) menos la que se espera solo por muestreo con cuatro grados de libertad residuales (0,645).

⚠ Conteos normalizados: para mirar, no para modelar

Los conteos divididos por $\hat{s}_j$ son útiles para gráficos y tablas, pero nunca deben entrar en DESeq2 o edgeR, que esperan conteos crudos y aplican

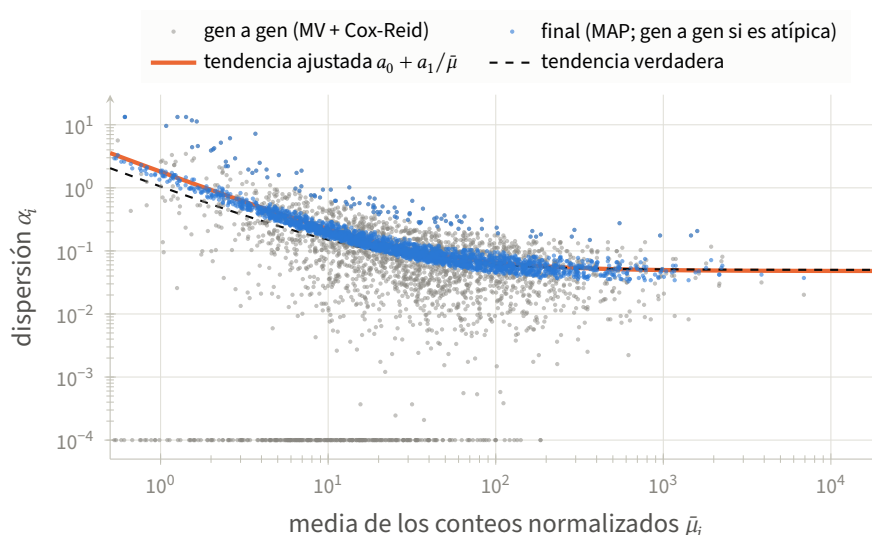


Figura 11.8. Estimación de dispersiones en un experimento simulado de 9992 genes con tres réplicas por grupo (se muestran 2500). Las estimaciones gen a gen (gris) son muy ruidosas; muchas caen al límite inferior de la búsqueda (10^{-4}) porque, con seis muestras, la varianza observada puede ser menor que la de Poisson por puro azar. La tendencia ajustada (naranja, $a_0 = 0,048$, $a_1 = 1,76$) sigue de cerca a la verdadera (0,05 y 1). Las estimaciones contraídas (azul) se agrupan alrededor de la tendencia sin colapsar sobre ella: conservan la variación genuina entre genes y reducen el error cuadrático medio en escala logarítmica de 6,6 a 0,17. Los puntos azules muy por encima de la curva son genes con dispersión atípica que conservan su estimación propia.

la normalización internamente. Para análisis exploratorios (agrupamiento, componentes principales) conviene además estabilizar la varianza, con la transformación logarítmica regularizada o la estabilizadora de varianza de DESeq2, porque en la escala logarítmica simple los genes poco expresados dominan la variabilidad.

11.3 Expresión diferencial y FDR

Diez mil detectores de metales

Una playa se divide en diez mil cuadrículas y en cada una se pasa un detector de metales que pita, por error, una vez de cada veinte cuando no hay nada enterrado. Si en la playa hay quinientas monedas, el detector pitará sobre muchas de ellas, pero también sobre unas quinientas cuadrículas vacías. Excavar todas las que pitaron significa que la mitad de las excavaciones serán inútiles. Si solo nos preocupara no excavar *nunca* en vano, podríamos exigir un pitido tan fuerte que casi ninguna cuadrícula vacía lo produjera, pero perderíamos la mayoría de las monedas. Hay un término medio razonable: aceptar que, de todas las excavaciones, una fracción pequeña y conocida (digamos un diez por ciento) sea en balde. Esa fracción es la tasa de falsos descubrimientos.

11.3.1 Contrastes sobre el modelo: Wald y razón de verosimilitudes

Con el GLM de la ecuación (11.14) ajustado, la pregunta “¿cambia la expresión del gen i entre condiciones?” se convierte en la hipótesis nula $H_0 : \beta_{i,\text{cond}} = 0$. DESeq2 ofrece dos contrastes (Love et al., 2014).

Definición 11.2. (Prueba de Wald)

El estadístico de Wald divide el coeficiente estimado por su error estándar, obtenido de la matriz de información de la ecuación (11.15):

$$z_i = \frac{\hat{\beta}_{i,\text{cond}}}{\widehat{\text{SE}}(\hat{\beta}_{i,\text{cond}})}, \quad p_i = 2 \Phi(-|z_i|). \quad (11.18)$$

DONDE

$\hat{\beta}_{i,\text{cond}}$	LFC estimado del gen i entre condiciones (en $\log_2$).
$\widehat{\text{SE}}$	Raíz del elemento diagonal correspondiente de $(X^T W X)^{-1}$, convertido a escala $\log_2$.
Φ	Función de distribución de la normal estándar.

Definición 11.3. (Prueba de razón de verosimilitudes (LRT))

Se ajustan un modelo completo y uno reducido sin los términos de interés, y se compara su log-verosimilitud:

$$D_i = 2 \left[\ell_i(\hat{\beta}^{\text{completo}}) - \ell_i(\hat{\beta}^{\text{reducido}}) \right] \stackrel{H_0}{\sim} \chi^2_{p_c - p_r}. \quad (11.19)$$

DONDE

$\ell_i(\cdot)$	Log-verosimilitud binomial negativa del gen i , con la dispersión fija en su estimación final.
p_c, p_r	Número de coeficientes de los modelos completo y reducido.

Para un solo coeficiente, ambas pruebas son asintóticamente equivalentes; en nuestra simulación, los $\log_{10} p$ de una y otra tienen una correlación de 0,990. La LRT es preferible cuando se quiere contrastar varios coeficientes a la vez: “¿cambia la expresión a lo largo de cualquiera de los cinco tiempos?” o “¿hay efecto de lote?”. La prueba de Wald, además de un valor p , entrega directamente el tamaño del efecto con su error estándar.

11.3.2 Tamaños de efecto: contracción del LFC

El LFC de máxima verosimilitud tiene un defecto grave para genes con pocas lecturas: es enormemente variable. Un gen con 0, 1 y 0 lecturas en el control y 4, 2 y 6 en el tratamiento tiene un LFC de más de 3, y no significa casi nada. Si ordenamos los genes por LFC para elegir candidatos, la cabeza de la lista se llenará de genes ruidosos poco expresados. Love et al. (2014) propusieron *contraer* los LFC hacia cero con una previa normal ajustada a los propios datos, $\beta_i \sim \mathcal{N}(0, \sigma_p^2)$. La estimación resultante es, aproximadamente, la media *a posteriori*:

$$\tilde{\beta}_i \approx \hat{\beta}_i \frac{\sigma_p^2}{\sigma_p^2 + \widehat{\text{SE}}_i^2}. \quad (11.20)$$

DONDE

$\tilde{\beta}_i$	LFC contraído del gen i .
σ_p^2	Varianza previa de los LFC verdaderos, estimada de la distribución de los LFC observados.
$\widehat{\text{SE}}_i$	Error estándar del LFC de máxima verosimilitud.

El factor de contracción depende del error estándar: los genes muy expresados, con SE pequeño, apenas cambian; los poco expresados se acercan mucho a cero. La previa normal tiene un inconveniente: también encoge los efectos grandes y *bien medidos*, porque la cola normal considera muy improbables los cambios de 5 o 6 unidades $\log_2$. Zhu et al. (2019) la reemplazaron por una previa de colas pesadas (Cauchy) en el método aplegm, que contrae con fuerza el ruido pero deja casi intactos los efectos grandes con evidencia sólida; es la opción que recomiendan hoy DESeq2 y PyDESeq2. Una idea importante: la contracción cambia la *estimación* del tamaño de efecto, pero en DESeq2 los valores p siguen saliendo de la prueba de Wald sobre el LFC sin contraer.

Ejemplo 11.5. (Un gen ruidoso)

En la simulación (9992 genes, 3 frente a 3 réplicas, 946 genes con cambio real), un gen *sin* cambio real y con media normalizada de 3,15 lecturas obtuvo un LFC de máxima verosimilitud de 4,20, con error estándar 1,76 y $p = 0,017$. Con la previa estimada, $\sigma_p = 0,650$, su LFC contraído es $4,20 \times 0,650^2 / (0,650^2 + 1,76^2) \approx 0,51$. En el conjunto de genes con media mayor que 1, el error cuadrático medio del LFC respecto al valor verdadero baja de 1,24 (máxima verosimilitud) a 0,117 (contraído).

11.3.3 Diez mil pruebas a la vez

En un experimento de RNA-seq contrastamos $m \approx 10\,000$ – $20\,000$ hipótesis. Si usamos el umbral clásico $p < 0,05$ para cada una, esperamos que un 5 % de los genes *sin* cambio resulten “significativos”: cientos de falsos positivos. Clasifiquemos los resultados:

	declarados no significativos	declarados significativos	total
H_0 verdadera	U	V	m_0
H_0 falsa	T	S	$m - m_0$
total	$m - R$	R	m

Solo m y R son observables. La corrección clásica controla la *tasa de error por familia*, $\text{FWER} = P(V \geq 1)$; el método de Bonferroni lo consigue declarando significativos los genes con $p_i \leq \alpha/m$. Es una exigencia apropiada cuando un único falso positivo es costoso, pero en genómica es excesiva: con $m = 10\,000$, un gen necesita $p < 10^{-5}$ para ser significativo al nivel 0,1.

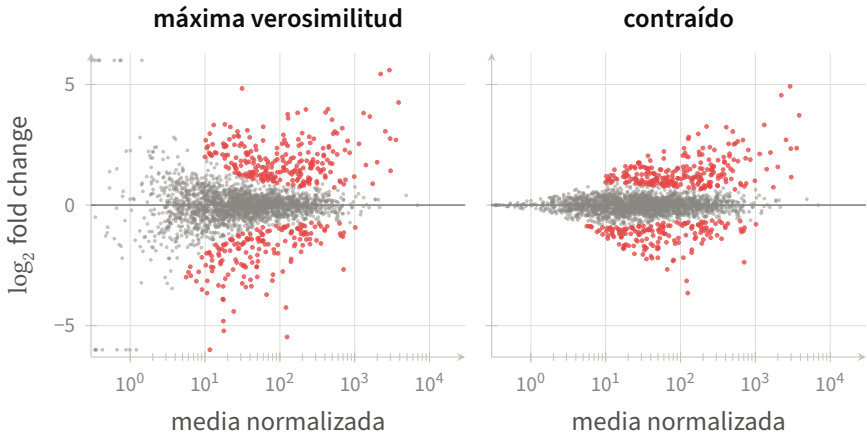


Figura 11.9. Gráficos MA del experimento simulado: cada punto es un gen, con su expresión media en el eje horizontal (escala logarítmica) y su LFC en el vertical; en rojo, los 404 genes significativos con $FDR < 0,1$. **Izquierda:** los LFC de máxima verosimilitud forman un “embudo” abierto hacia la izquierda, donde los genes con pocas lecturas alcanzan cambios aparentes de ± 6 (valores recortados a ese rango). **Derecha:** tras la contracción (ecuación (11.20)), el embudo se cierra; los genes poco expresados se acercan a cero y los significativos, que tienen errores estándar pequeños, apenas se mueven. Ordenar genes por LFC solo tiene sentido en el panel derecho.

Definición 11.4. (Tasa de falsos descubrimientos)

La FDR es la proporción esperada de falsos positivos entre los descubrimientos,

$$FDR = \mathbb{E} \left[\frac{V}{\max(R, 1)} \right]. \quad (11.21)$$

DONDE

V	Número de hipótesis nulas verdaderas rechazadas (falsos descubrimientos).
R	Número total de rechazos (descubrimientos).
m_0	Número de hipótesis nulas verdaderas, desconocido.

Benjamini y Hochberg (1995) propusieron controlar esta cantidad, mucho más adecuada para un experimento de cribado: lo que importa no es evitar todo error,

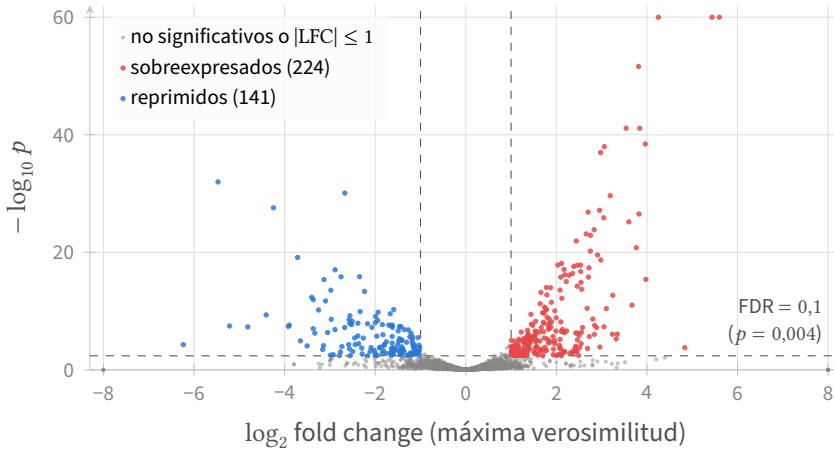


Figura 11.10. Gráfico *volcano*: tamaño del efecto frente a significación. La línea horizontal marca el valor p que corresponde a $\text{FDR} = 0,1$ según Benjamini-Hochberg y las verticales, un cambio de dos veces. Los genes de las esquinas superiores combinan un efecto grande y una evidencia fuerte. Los puntos grises con $|\text{LFC}| > 3$ y p modesto son genes poco expresados: efecto aparente grande, evidencia débil. Los valores de $-\log_{10} p$ se recortaron en 60 para la figura.

sino que la lista de candidatos que pasará a validación sea mayoritariamente correcta. Y dieron un procedimiento sorprendentemente sencillo.

Teorema 11.3. (*Procedimiento de Benjamini-Hochberg*)

Ordene los valores p , $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$, y sea

$$k^* = \max \left\{ i : p_{(i)} \leq \frac{i}{m} \alpha \right\}. \quad (11.22)$$

Si se rechazan las hipótesis $H_{(1)}, \dots, H_{(k^*)}$ y las pruebas son independientes, entonces $\text{FDR} \leq \frac{m_0}{m} \alpha \leq \alpha$.

DONDE

- $p_{(i)}$ El i -ésimo valor p más pequeño.
- α Nivel deseado de FDR (en RNA-seq es habitual 0,05 o 0,1).
- k^* Número de descubrimientos.

El umbral crece linealmente con el rango: el gen más significativo debe superar la exigencia de Bonferroni, α/m , pero el décimo solo necesita $10\alpha/m$. Si hay muchos genes con señal, el procedimiento se vuelve permisivo; si no hay ninguno, se comporta casi como Bonferroni. La demostración original supone independencia; resultados posteriores la extienden a formas de dependencia positiva que cubren razonablemente la correlación entre genes coexpresados. El *valor p ajustado* de BH (la columna padj de DESeq2) es el menor α al que el gen sería declarado significativo.

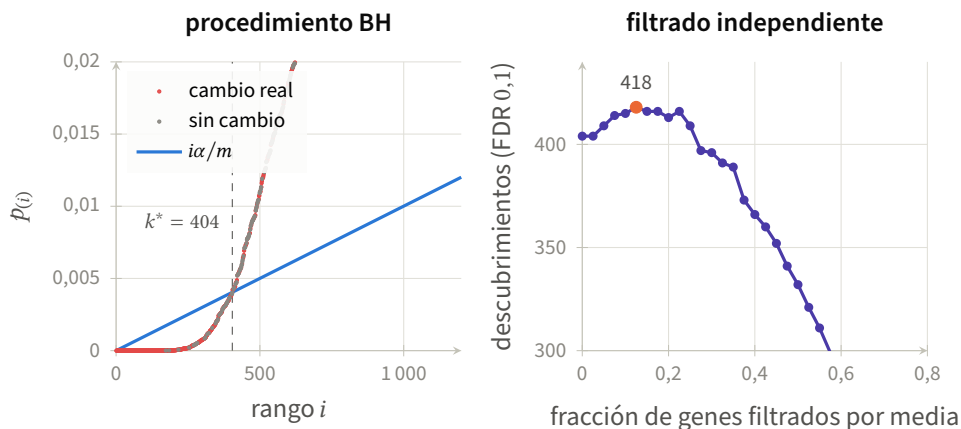


Figura 11.11. Izquierda: los 1200 valores p más pequeños del experimento simulado, ordenados, frente a la recta $i\alpha/m$ con $\alpha = 0,1$. El último punto bajo la recta está en $i = 404$ ($p_{(404)} = 0,0040$), y todos los genes hasta ese rango se declaran significativos, aunque alguno anterior quede por encima de la recta: por eso el procedimiento se llama “de subida”. Entre los 404 hay 33 genes sin cambio real (gris), una proporción de 0,082. Derecha: número de descubrimientos al eliminar antes de la corrección una fracción creciente de los genes menos expresados. Quitar el 12,5 % inferior (media normalizada $< 5,6$) aumenta los descubrimientos de 404 a 418 sin romper el control de la FDR; filtrar demasiado acaba desechando genes con señal.

Ejemplo 11.6. (Tres criterios sobre el mismo experimento)

En la simulación, con 9992 genes y 946 cambios reales:

- $p < 0,05$ sin corrección: 915 genes, de los que 373 (41 %) no tienen cambio real.

- Bonferroni con FWER 0,1 ($p < 10^{-5}$): 185 genes, solo 2 falsos, pero se detecta menos de una quinta parte de los cambios.
- Benjamini-Hochberg con FDR 0,1: 404 genes, 33 falsos. La proporción observada de falsos descubrimientos, 0,082, está por debajo del nivel nominal, y la potencia es del 39 %.

El umbral efectivo de BH resulta ser $p \leq 0,0040$, cuatrocientas veces más permisivo que Bonferroni y doce veces más exigente que el umbral ingenuo. La potencia modesta no es un fallo del método: con tres réplicas, muchos de los cambios reales son pequeños o afectan a genes con pocas lecturas.

benjamini_hochberg.py

```
import numpy as np

def bh(p):
    """Valores p ajustados de Benjamini-Hochberg."""
    p = np.asarray(p, float); m = len(p)
    orden = np.argsort(p)
    ajust = p[orden] * m / np.arange(1, m + 1)
    ajust = np.minimum.accumulate(ajust[::-1])[::-1] # monotonía
    salida = np.empty(m); salida[orden] = np.minimum(ajust, 1)
    return salida
```

11.3.4 El histograma de valores p y los q -valores

Bajo la hipótesis nula, un valor p continuo es uniforme en $[0, 1]$. El histograma de los m valores p de un experimento es por tanto una mezcla: una base plana de altura proporcional a m_0 más un pico cerca de cero producido por los genes con señal. Storey y Tibshirani (2003) usaron esa estructura para estimar la proporción de nulas verdaderas, $\pi_0 = m_0/m$: por encima de un umbral λ casi todos los valores p vienen de nulas, así que

$$\hat{\pi}_0(\lambda) = \frac{\#\{p_i > \lambda\}}{m(1 - \lambda)}. \quad (11.23)$$

DONDE

π_0	Proporción de genes sin cambio real.
λ	Umbral a partir del cual se supone que solo hay nulas (p. ej., 0,5).

Como la base del histograma contiene también algunos valores p de genes con cambios pequeños, $\hat{\pi}_0$ tiende a *sobreestimar* π_0 , un sesgo conservador. Con esa estimación se define el q -valor de un gen como la menor FDR a la que puede ser declarado significativo,

$$q(p_i) = \min_{t \geq p_i} \frac{\hat{\pi}_0 m t}{\#\{p_j \leq t\}}, \quad (11.24)$$

que es el valor p ajustado de BH multiplicado por $\hat{\pi}_0$. Cuando π_0 es claramente menor que 1, los q -valores ganan potencia; en RNA-seq, con π_0 cercano a 1, la diferencia suele ser pequeña.

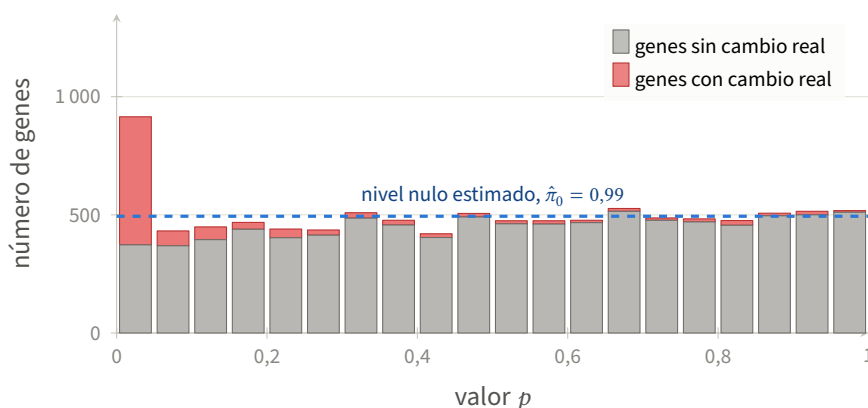


Figura 11.12. Histograma de los 9992 valores p de la simulación, con barras apiladas según la verdad conocida. El pico en la barra izquierda contiene la mayoría de los cambios reales; el resto del histograma es aproximadamente plano, como corresponde a nulas uniformes. La línea discontinua es la altura de la base según $\hat{\pi}_0(0,5) = 0,99$, mayor que el valor verdadero (0,905) porque algunos cambios pequeños tienen valores p grandes. La barra de la derecha sobresale: los genes con muy pocas lecturas producen valores p discretos y acumulados cerca de 1 (el 22 % de los genes con media < 2 tiene $p > 0,95$). Un histograma con forma de U, o creciente hacia la derecha, delata un modelo mal especificado (dispersión sobreestimada, lotes no incluidos).

11.3.5 Filtrado independiente

Los genes con muy pocas lecturas no tienen ninguna posibilidad de alcanzar significación, pero cuentan en m y encarecen la corrección para todos los demás. ¿Podemos eliminarlos antes de aplicar BH? Bourgon et al. (2010) mostraron que sí, con una condición precisa: el estadístico de filtrado debe ser *independiente del*

estadístico de prueba bajo la hipótesis nula. La media de los conteos normalizados de todas las muestras, sin mirar a qué grupo pertenecen, cumple esa condición: para un gen sin cambio, saber que su media global es alta no dice nada sobre su valor p . Filtrar por el valor p mismo, o por el LFC, no la cumple y destruye el control de la FDR. DESeq2 elige automáticamente el umbral de media que maximiza el número de descubrimientos, algo que, como muestra la figura 11.11 derecha, suele ganar un puñado de genes a costa de ninguno.

```

deseq2_python.py

import pandas as pd
from pydeseq2.dds import DeseqDataSet
from pydeseq2.ds import DeseqStats

cuentas = pd.read_csv("cuentas.csv", index_col=0).T # muestras x genes
meta = pd.read_csv("muestras.csv", index_col=0) # columna condicion
dds = DeseqDataSet(counts=cuentas, metadata=meta,
                    design="~condicion", refit_cooks=True)
dds.deseq2() # factores, dispersiones, GLM
print(dds.obs["size_factors"])
ds = DeseqStats(dds, contrast=["condicion", "trat", "ctrl"],
                alpha=0.1) # FDR objetivo
ds.summary() # Wald + filtrado + BH
ds.lfc_shrink(coeff="condicion[T.trat]") # previa de colas pesadas
res = ds.results_df.sort_values("padj")
print(res.head())

```

⚠ Tres errores clásicos de la expresión diferencial

1. *Comparar listas.* Que un gen sea significativo en el tratamiento A y no en el B no demuestra que respondan distinto: hay que contrastar la diferencia de LFC directamente, con un término de interacción.
2. *Confundir significación con relevancia.* Con muchas réplicas, un cambio del 5 % puede ser muy significativo. Si interesan solo cambios grandes, contraste $H_0 : |\beta| \leq \log_2 1.5$ en lugar de filtrar los resultados por LFC a posteriori, lo cual no controla la FDR.
3. *Ignorar el diseño.* Si hubo lotes, parejas o sexos, deben estar en la fórmula; un lote omitido infla la dispersión y se ve en el histograma de valores p .

11.4 Enriquecimiento funcional



Colab · 11.4 · Enriquecimiento (GO, KEGG, GSEA)



repositorio del curso

Demasiados pelirrojos en el autobús

En una ciudad donde uno de cada cincuenta habitantes es pelirrojo, sube a un autobús de cuarenta pasajeros y cuenta siete pelirrojos. Algo pasa: quizá el autobús viene de un congreso de pelirrojos. Pero antes de concluirlo conviene hacerse dos preguntas. ¿Cuántos pelirrojos cabría esperar por azar en un autobús de ese tamaño? (Menos de uno.) ¿Y el autobús recoge a la gente al azar, o solo pasa por un barrio donde los pelirrojos abundan? La primera pregunta es la prueba hipergeométrica; la segunda, el sesgo de selección que obliga a corregirla. Cambie “pelirrojos” por “genes de la fosforilación oxidativa” y “autobús” por “lista de genes diferencialmente expresados”.

11.4.1 Vocabularios de función: GO y KEGG

Una lista de 400 genes no es un resultado biológico; para interpretarla necesitamos saber qué hacen esos genes, y hacerlo de forma sistemática. La *Gene Ontology* (Ashburner et al., 2000) nació para dar a las bases de datos de organismos modelo un vocabulario común con el que describir los productos génicos. Se compone de tres ontologías independientes: *proceso biológico* (lo que el gen contribuye a conseguir, como “fosforilación oxidativa”), *función molecular* (la actividad bioquímica, como “actividad de citocromo c oxidasa”) y *componente celular* (dónde actúa, como “membrana mitocondrial interna”). Los términos se organizan en un *grafo acíclico dirigido*: un término puede tener varios padres, conectados por relaciones como *is_a* (es un tipo de) o *part_of* (es parte de). Hoy el recurso reúne decenas de miles de términos y millones de anotaciones de miles de especies (The Gene Ontology Consortium, 2023).

Definición 11.5. (Regla del camino verdadero)

Si un gen está anotado a un término, está implícitamente anotado a todos sus ancestros en el grafo. Por eso los términos generales (“proceso metabólico”) contienen miles de genes y los específicos, unas decenas.

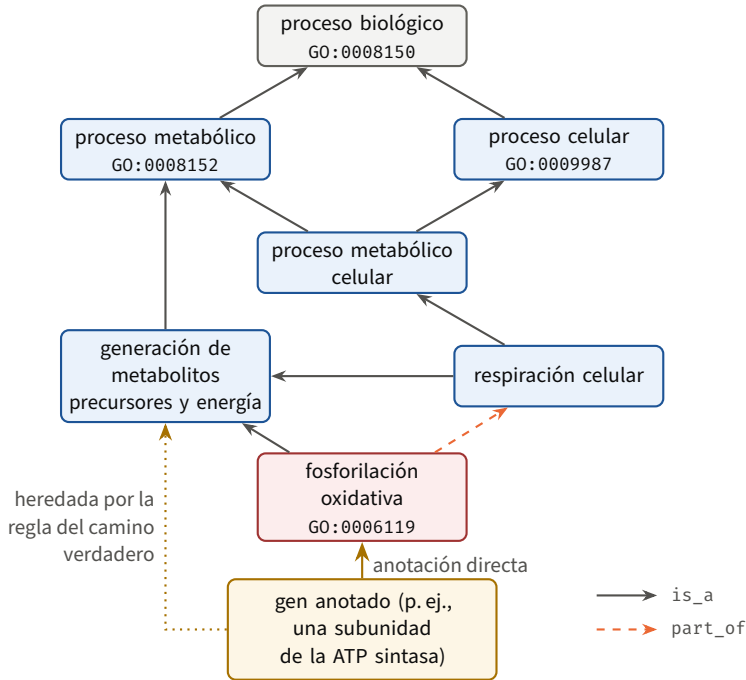


Figura 11.13. Un fragmento simplificado de la ontología de procesos biológicos, con las relaciones apuntando del término hijo al padre. Un término puede tener varios padres (“proceso metabólico celular” es a la vez metabólico y celular), por lo que la estructura es un grafo acíclico dirigido y no un árbol. Un gen anotado directamente a “fosforilación oxidativa” cuenta también, por la regla del camino verdadero, para todos sus ancestros. Esa herencia hace que los términos estén fuertemente anidados y que sus pruebas de enriquecimiento no sean independientes. Relaciones reales, completas y actualizadas, en los navegadores de la GO.

KEGG (Kanehisa & Goto, 2000) ofrece una organización complementaria: en lugar de un vocabulario jerárquico, un conjunto de *mapas de rutas* dibujados a mano (metabolismo, señalización, enfermedades) en los que cada gen ocupa un nodo concreto. Sus conjuntos son menos numerosos y más fáciles de interpretar, a cambio de cubrir menos genes. La colección MSigDB reúne además miles de conjuntos de genes de estudios publicados; su colección *hallmark*, construida por Liberzon et al. (2015), condensa la redundancia de miles de conjuntos en 50 “marcas” de procesos biológicos bien definidos, una buena elección para una primera exploración.

11.4.2 Análisis de sobrerrepresentación

La pregunta más simple es: entre los n genes de mi lista, ¿hay más genes del término T de los que esperaría por azar? Supongamos que el universo tiene N genes, de los cuales K pertenecen a T , y que la lista se hubiera formado eligiendo n genes al azar y sin reemplazo. El número X de genes de T en la lista seguiría una distribución hipergeométrica, y el valor p de haber observado k o más es

$$p = P(X \geq k) = \sum_{x=k}^{\min(n,K)} \frac{\binom{K}{x} \binom{N-K}{n-x}}{\binom{N}{n}}, \quad E[X] = \frac{nK}{N}. \quad (11.25)$$

DONDE

N	Tamaño del universo: genes <i>analizados</i> en el experimento.
K	Genes del universo anotados al término T .
n	Genes de la lista (p. ej., significativos con FDR < 0,1).
k	Genes de la lista que pertenecen a T .

Es exactamente la prueba exacta de Fisher unilateral sobre la tabla 2×2 que cruza “en la lista / fuera de ella” con “en el término / fuera de él”. Se repite para cada uno de los miles de términos, y los valores p resultantes se corrigen por BH (sección 11.3).

⚠ El universo no es el genoma

El universo debe ser el conjunto de genes que tuvieron alguna oportunidad de aparecer en la lista: los que se analizaron y pasaron el filtrado. Si se usa todo el genoma, los términos de genes que se expresan en el tejido estudiado aparecerán enriquecidos por el mero hecho de expresarse. Es un error frecuente en las herramientas web que piden solo la lista de genes.

11.4.3 El sesgo de longitud

El modelo hipergeométrico supone que todos los genes tenían la misma probabilidad de entrar en la lista. En RNA-seq no es así. Young et al. (2010) observaron que, para un mismo cambio de expresión, un gen largo produce más lecturas y, por tanto, tiene más potencia estadística: la probabilidad de ser declarado significativo crece con la longitud. Los términos GO que agrupan genes largos aparecen entonces enriquecidos sin que haya nada biológico detrás. Su método, goseq, estima una

función de probabilidad ponderada (PWF), la probabilidad de ser significativo en función de la longitud, ajustando una curva monótona a los datos, y sustituye la hipergeométrica por la distribución que resulta de muestrear genes con esos pesos: la hipergeométrica no central de Wallenius, o su aproximación por remuestreo.

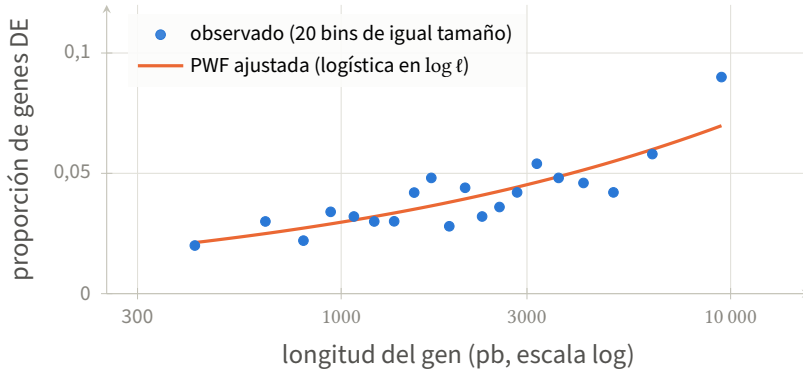


Figura 11.14. Probabilidad de ser declarado diferencialmente expresado en función de la longitud del gen, en la simulación. Los cambios verdaderos se asignaron al 10 % de los genes *con independencia de su longitud*; aun así, en el bin de genes más cortos se detecta un 2 % de genes y en el de los más largos, un 9 %, porque los genes largos acumulan más lecturas y sus pruebas tienen más potencia. Esta curva es la función de probabilidad ponderada de goseq.

Ejemplo 11.7. (Un enriquecimiento espurio)

En la simulación creamos un “término GO” de $K = 150$ genes elegidos entre el 20 % más largo, con 14 genes con cambio real, tantos como los que corresponden por azar ($150 \times 0,095 \approx 14,2$): no hay enriquecimiento biológico. La lista tiene $n = 404$ genes significativos entre $N = 9992$. Se esperaba $E[X] = 404 \times 150/9992 = 6,06$ genes del término en la lista y se observan $k = 12$. La tabla de Fisher es

	en el término	fuera
en la lista	12	392
fuera de la lista	138	9450

con una razón de momios de 2,10 y $p = 0,018$ (hipergeométrica y Fisher coinciden). El término parece enriquecido. Pero si la lista se genera muestreando genes con probabilidad proporcional a la PWF, el número esperado

de genes del término sube a 8,79 y el valor p por remuestreo es 0,168: el “enriquecimiento” era solo el reflejo de que los genes largos se detectan mejor.

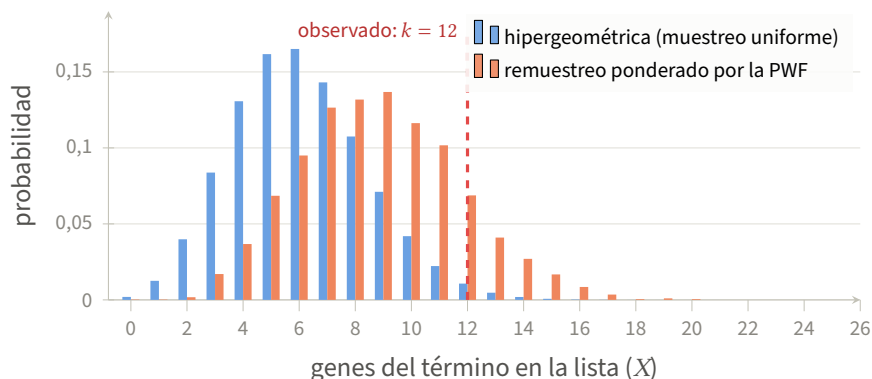


Figura 11.15. Distribución nula del número de genes de un término de 150 genes largos en una lista de 404 genes. La hipergeométrica (azul) supone que todos los genes tenían la misma probabilidad de entrar en la lista y está centrada en 6; el valor observado, 12, queda en su cola ($p = 0,018$). La distribución obtenida muestreando con los pesos de la figura 11.14 (naranja, 4000 remuestreos) se desplaza hacia la derecha, y el mismo valor observado deja de ser llamativo ($p = 0,17$).

11.4.4 GSEA: sin umbrales

El análisis de sobrerrepresentación tiene dos debilidades. Depende de un umbral arbitrario (¿FDR 0,05 o 0,1?) y es ciego a cambios *pequeños pero coordinados*: si los cien genes de una ruta bajan un 20 % cada uno, quizá ninguno sea significativo, pero el conjunto lo es de forma abrumadora. Así ocurrió en el estudio de Mootha et al. (2003) sobre músculo de pacientes con diabetes tipo 2: ningún gen individual destacaba tras corregir por pruebas múltiples, pero los genes de la fosforilación oxidativa aparecían consistentemente reprimidos. Para detectarlo idearon el *análisis de enriquecimiento de conjuntos de genes* (GSEA), que Subramanian et al. (2005) refinaron después en su forma actual.

La idea es trabajar con la lista *completa* de genes, ordenada por un estadístico de cambio (por ejemplo, el z de Wald), y preguntar si los genes del conjunto S tienden a concentrarse en uno de los extremos. Se recorre la lista de arriba abajo con un

paseo que sube cada vez que encuentra un gen de S y baja cuando encuentra uno que no está.

Definición 11.6. (Puntuación de enriquecimiento)

Sean $r_1 \geq r_2 \geq \dots \geq r_N$ los estadísticos ordenados, $N_H = |S|$ y $p \geq 0$ un exponente de ponderación. Para cada posición i ,

$$P_{\text{hit}}(i) = \sum_{\substack{j \leq i \\ g_j \in S}} \frac{|r_j|^p}{N_R}, \quad P_{\text{miss}}(i) = \sum_{\substack{j \leq i \\ g_j \notin S}} \frac{1}{N - N_H}, \quad N_R = \sum_{g_j \in S} |r_j|^p, \quad (11.26)$$

y la puntuación de enriquecimiento ES es la desviación máxima de $P_{\text{hit}}(i) - P_{\text{miss}}(i)$ respecto a cero, con su signo.

DONDE

r_j	Estadístico del gen en la posición j de la lista ordenada.
g_j	Gen en la posición j .
N_H	Número de genes del conjunto presentes en la lista.
p	Peso: con $p = 0$ se obtiene el estadístico de Kolmogórov-Smirnov del trabajo original; con $p = 1$ (por defecto), cada gen pesa según la magnitud de su cambio.

Las dos sumas acumuladas valen 1 al final de la lista, así que el paseo empieza y termina en cero. Si los genes de S se reparten al azar, el paseo oscila cerca de cero; si se concentran arriba, sube con fuerza al principio. Los genes de S que aparecen antes del máximo forman el *núcleo líder* (*leading edge*), el subconjunto que más contribuye a la señal y el primero que conviene examinar. La significación se evalúa por permutación: lo ideal es permutar las etiquetas de las muestras, que conserva la correlación entre genes; con tres réplicas por grupo solo hay diez permutaciones distintas, y en la práctica se permutan los genes, que ignora esa correlación y es por tanto algo anticonservador. Para comparar conjuntos de distinto tamaño, ES se divide por la media de las ES nulas del mismo signo y se obtiene la puntuación normalizada NES. Como se evalúan miles de conjuntos, al final se calcula una FDR. Calcular millones de permutaciones es costoso; Korotkevich et al. (2016) mostraron en fgsea que puede reutilizarse una misma muestra de permutaciones para todos los conjuntos de un tamaño dado, calculando las puntuaciones de forma acumulativa,

lo que acelera el análisis en órdenes de magnitud y permite estimar valores p muy pequeños con precisión.

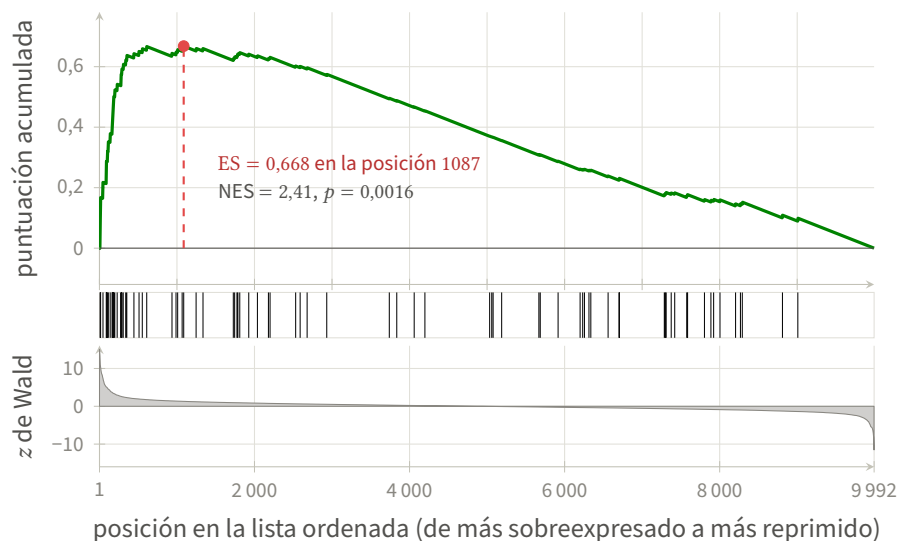


Figura 11.16. Anatomía de un GSEA sobre el experimento simulado, con un conjunto de 80 genes, 28 de ellos sobreexpresados de verdad y el resto sin cambio. **Arriba:** la puntuación acumulada $P_{\text{hit}} - P_{\text{miss}}$ (ecuación (11.26), $p = 1$) sube con fuerza en las primeras posiciones y alcanza su máximo, la ES, en la posición 1087; después desciende lentamente. **Centro:** cada raya es un gen del conjunto; se aprecia su concentración en la cabeza de la lista. **Abajo:** el estadístico usado para ordenar. Los 30 genes a la izquierda del máximo forman el núcleo líder.

Ejemplo 11.8. (Leer un resultado de GSEA)

En la figura 11.16, la ES es 0,668 y la NES, 2,41: el conjunto está mucho más concentrado en la cabeza de la lista que los conjuntos aleatorios del mismo tamaño. En 1000 permutaciones de genes, ninguna ES aleatoria positiva alcanzó el valor observado, así que $p \approx 1/(n_+ + 1) = 0,0016$, donde n_+ es el número de permutaciones con ES positiva: el valor p está limitado por el número de permutaciones, no por la evidencia. El núcleo líder tiene 30 genes, que incluyen casi todos los realmente sobreexpresados. Con la versión sin pesos ($p = 0$) la ES es solo 0,295: los 52 genes sin cambio diluyen la señal cuando todos los genes pesan lo mismo, mientras que la ponderación deja que los pocos genes con cambios fuertes dominen el paseo.

```

gsea_minimo.py

import numpy as np
from scipy import stats

def ora(k, N, K, n):
    """p de sobrerrepresentación:  $P(X \geq k)$ ,  $X$  hipergeométrica."""
    return stats.hypergeom.sf(k - 1, N, K, n)

def paseo_gsea(r, en_conjunto, p=1.0):
    """r: estadísticos; en_conjunto: booleanos. Devuelve paseo y ES."""
    o = np.argsort(-r)
    hit = en_conjunto[o]
    w = np.abs(r[o]) ** p
    phit = np.cumsum(np.where(hit, w, 0)) / w[hit].sum()
    pmiss = np.cumsum(~hit) / (~hit).sum()
    paseo = phit - pmiss
    return paseo, paseo[np.argmax(np.abs(paseo))]

print(ora(12, 9992, 150, 404))      # 0.0184

```

⚠ Lo que un enriquecimiento no dice

Un término enriquecido es una *hipótesis*, no una conclusión. Las anotaciones están sesgadas hacia los genes más estudiados; muchas se infieren electrónicamente sin validación experimental (código de evidencia IEA); y la estructura anidada de la GO hace que un mismo grupo de genes produzca decenas de términos redundantes que parecen resultados independientes. Informe siempre el universo, la versión de la anotación, el método de corrección y los genes que sostienen cada término, y prefiera pocos conjuntos bien definidos (*hallmarks*, rutas de KEGG) a miles de términos solapados cuando el objetivo sea interpretar.

☰ En resumen

- El diseño manda: réplicas biológicas (al menos tres por condición), lotes balanceados y la biblioteca adecuada (poli(A) o depleción, con hebra, pareada) limitan todo lo que la estadística puede hacer después.

- Las lecturas se alinean al genoma con alineadores con empalmes (STAR, HISAT2) o se asignan al transcriptoma por pseudoalineamiento (kallisto, Salmon). La ambigüedad entre isoformas se resuelve con máxima verosimilitud sobre clases de equivalencia mediante el algoritmo EM, cuyo paso E reparte las lecturas en proporción a $\alpha_i/\tilde{\ell}_i$.
- Los TPM estiman fracciones molares y suman un millón; los RPKM no son comparables entre muestras. Para expresión diferencial se usan conteos (estimados y agregados con tximport), nunca TPM.
- La normalización por total de lecturas no corrige la composición; TMM y la mediana de razones sí, bajo el supuesto de que la mayoría de los genes no cambia.
- La variación biológica hace que $\text{Var} = \mu + \alpha\mu^2$: los conteos siguen una binomial negativa. DESeq2 y edgeR ajustan un GLM con enlace logarítmico y contraen las dispersiones hacia una tendencia común; limma-voom usa pesos de precisión sobre log-CPM.
- La prueba de Wald y la LRT contrastan coeficientes; la contracción del LFC (previa normal o, mejor, de colas pesadas como en apeglm) estabiliza los tamaños de efecto de genes poco expresados.
- Con miles de pruebas se controla la FDR con Benjamini-Hochberg; los q -valores incorporan $\hat{\pi}_0$; el filtrado independiente por media aumenta la potencia sin romper el control. El histograma de valores p es el mejor diagnóstico del modelo.
- El enriquecimiento funcional traduce listas en hipótesis: la sobrerrepresentación (hipergeométrica, con el universo correcto y corrección del sesgo de longitud) o GSEA, que usa la lista completa y detecta cambios pequeños y coordinados.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 11.1** ●●● Un transcrito de 600 pb y otro de 6000 pb reciben el mismo número de fragmentos, con fragmentos de longitud media 250. Calcule sus longitudes efectivas aproximadas y la razón de sus fracciones molares τ . ¿Qué error se comete si se ignora la longitud efectiva?
- 11.2** ●●● Demuestre que los TPM de una muestra suman 10^6 y que la suma de sus RPKM es $10^9/\bar{\ell}$, donde $\bar{\ell}$ es la media de las longitudes ponderada por los conteos.

- 11.3** ●●● Modifique `em_isoformas` para que el paso E reparta en proporción a α_t en lugar de $\alpha_t/\tilde{l}_t$ y aplíquelo al ejemplo 11.1. Compare las estimaciones de τ con las correctas y explique la dirección del sesgo.
- 11.4** ●●● Con la ley de la varianza total, derive la ecuación (11.13). Luego simule 10 000 valores de una mezcla gamma-Poisson con $\mu = 50$ y $\alpha = 0,3$ y compruebe media y varianza empíricas.
- 11.5** ●●● Tome una matriz de conteos real (por ejemplo, la del notebook de la lección) y calcule factores de tamaño por mediana de razones y por total de lecturas. Represente el gráfico MA entre dos muestras y marque ambos factores. ¿Cuál centra mejor la nube?
- 11.6** ●●● Simule 10 000 valores p de los cuales 9000 son uniformes y 1000 proceden de una $\text{beta}(0,1; 1)$. Aplique BH con $\alpha = 0,1$ y calcule la proporción de falsos descubrimientos. Repita 200 veces y compare la media con $\pi_0\alpha$.
- 11.7** ●●● Demuestre que filtrar genes por su valor p antes de aplicar BH no controla la FDR, construyendo un contraejemplo con todas las hipótesis nulas verdaderas. ¿Por qué el filtrado por media global no sufre el mismo problema?
- 11.8** ●●● Implemente GSEA con permutaciones de las etiquetas de muestra en un conjunto de datos con al menos seis réplicas por grupo, y compare los valores p con los obtenidos permutando genes para un conjunto de genes muy correlacionados entre sí. Explique la diferencia.
- 11.9** ●●● Para el ejemplo 11.7, implemente la corrección por longitud con la distribución hipergeométrica no central de Wallenius (en SciPy, la clase `nchypergeom_wallenius` del módulo `scipy.stats`), usando como peso del término la razón entre la PWF media de sus genes y la del resto. Compare el resultado con el obtenido por remuestreo.

Referencias del capítulo

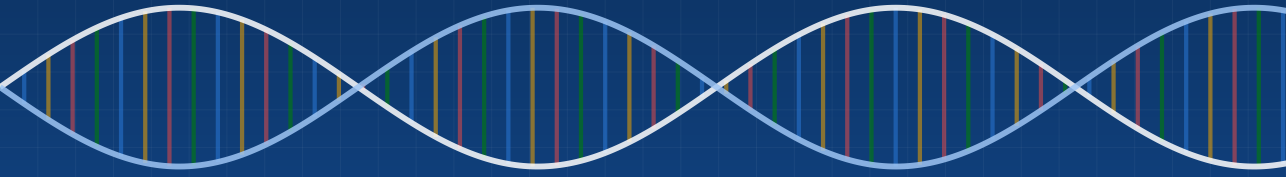
Anders, S., & Huber, W. (2010). Differential expression analysis for sequence count data. *Genome Biology*, 11(10), Artículo R106. <https://doi.org/10.1186/gb-2010-11-10-r106>

- Ashburner, M., Ball, C. A., Blake, J. A., Botstein, D., Butler, H., Cherry, J. M., Davis, A. P., Dolinski, K., Dwight, S. S., Eppig, J. T., Harris, M. A., Hill, D. P., Issel-Tarver, L., Kasarskis, A., Lewis, S., Matese, J. C., Richardson, J. E., Ringwald, M., Rubin, G. M., & Sherlock, G. (2000). Gene Ontology: tool for the unification of biology. *Nature Genetics*, 25(1), 25-29. <https://doi.org/10.1038/75556>
- Benjamini, Y., & Hochberg, Y. (1995). Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1), 289-300. <https://doi.org/10.1111/j.2517-6161.1995.tb02031.x>
- Bourgon, R., Gentleman, R., & Huber, W. (2010). Independent filtering increases detection power for high-throughput experiments. *Proceedings of the National Academy of Sciences*, 107(21), 9546-9551. <https://doi.org/10.1073/pnas.0914005107>
- Bray, N. L., Pimentel, H., Melsted, P., & Pachter, L. (2016). Near-optimal probabilistic RNA-seq quantification. *Nature Biotechnology*, 34(5), 525-527. <https://doi.org/10.1038/nbt.3519>
- Conesa, A., Madrigal, P., Tarazona, S., Gomez-Cabrero, D., Cervera, A., McPherson, A., Szczesniak, M. W., Gaffney, D. J., Elo, L. L., Zhang, X., & Mortazavi, A. (2016). A survey of best practices for RNA-seq data analysis. *Genome Biology*, 17(1), Artículo 13. <https://doi.org/10.1186/s13059-016-0881-8>
- Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1), 1-22. <https://doi.org/10.1111/j.2517-6161.1977.tb01600.x>
- Dobin, A., Davis, C. A., Schlesinger, F., Drenkow, J., Zaleski, C., Jha, S., Batut, P., Chaisson, M., & Gingeras, T. R. (2013). STAR: ultrafast universal RNA-seq aligner. *Bioinformatics*, 29(1), 15-21. <https://doi.org/10.1093/bioinformatics/bts635>
- Holmes, S., & Huber, W. (2019). *Modern Statistics for Modern Biology*. Cambridge University Press.
- Kanehisa, M., & Goto, S. (2000). KEGG: Kyoto Encyclopedia of Genes and Genomes. *Nucleic Acids Research*, 28(1), 27-30. <https://doi.org/10.1093/nar/28.1.27>
- Kim, D., Paggi, J. M., Park, C., Bennett, C., & Salzberg, S. L. (2019). Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype. *Nature Biotechnology*, 37(8), 907-915. <https://doi.org/10.1038/s41587-019-0201-4>
- Korotkevich, G., Sukhov, V., Budin, N., Shpak, B., Artyomov, M. N., & Sergushichev, A. (2016). Fast gene set enrichment analysis. *bioRxiv*. <https://doi.org/10.1101/060012>
- Law, C. W., Chen, Y., Shi, W., & Smyth, G. K. (2014). voom: precision weights unlock linear model analysis tools for RNA-seq read counts. *Genome Biology*, 15(2), Artículo R29. <https://doi.org/10.1186/gb-2014-15-2-r29>
- Li, B., & Dewey, C. N. (2011). RSEM: accurate transcript quantification from RNA-Seq data with or without a reference genome. *BMC Bioinformatics*, 12(1), Artículo 323. <https://doi.org/10.1186/1471-2105-12-323>

- Liberzon, A., Birger, C., Thorvaldsdóttir, H., Ghandi, M., Mesirov, J. P., & Tamayo, P. (2015). The Molecular Signatures Database hallmark gene set collection. *Cell Systems*, 1(6), 417-425. <https://doi.org/10.1016/j.cels.2015.12.004>
- Love, M. I., Huber, W., & Anders, S. (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology*, 15(12), Artículo 550. <https://doi.org/10.1186/s13059-014-0550-8>
- Mootha, V. K., Lindgren, C. M., Eriksson, K.-F., Subramanian, A., Sihag, S., Lehar, J., Puigserver, P., Carlsson, E., Ridderstråle, M., Laurila, E., Houstis, N., Daly, M. J., Patterson, N., Mesirov, J. P., Golub, T. R., Tamayo, P., Spiegelman, B., Lander, E. S., Hirschhorn, J. N., ... Groop, L. C. (2003). PGC-1 α -responsive genes involved in oxidative phosphorylation are coordinately downregulated in human diabetes. *Nature Genetics*, 34(3), 267-273. <https://doi.org/10.1038/ng1180>
- Mortazavi, A., Williams, B. A., McCue, K., Schaeffer, L., & Wold, B. (2008). Mapping and quantifying mammalian transcriptomes by RNA-Seq. *Nature Methods*, 5(7), 621-628. <https://doi.org/10.1038/nmeth.1226>
- Patro, R., Duggal, G., Love, M. I., Irizarry, R. A., & Kingsford, C. (2017). Salmon provides fast and bias-aware quantification of transcript expression. *Nature Methods*, 14(4), 417-419. <https://doi.org/10.1038/nmeth.4197>
- Ritchie, M. E., Phipson, B., Wu, D., Hu, Y., Law, C. W., Shi, W., & Smyth, G. K. (2015). limma powers differential expression analyses for RNA-sequencing and microarray studies. *Nucleic Acids Research*, 43(7), Artículo e47. <https://doi.org/10.1093/nar/gkv007>
- Robinson, M. D., McCarthy, D. J., & Smyth, G. K. (2010). edgeR: a Bioconductor package for differential expression analysis of digital gene expression data. *Bioinformatics*, 26(1), 139-140. <https://doi.org/10.1093/bioinformatics/btp616>
- Robinson, M. D., & Oshlack, A. (2010). A scaling normalization method for differential expression analysis of RNA-seq data. *Genome Biology*, 11(3), Artículo R25. <https://doi.org/10.1186/gb-2010-11-3-r25>
- Soneson, C., Love, M. I., & Robinson, M. D. (2015). Differential analyses for RNA-seq: transcript-level estimates improve gene-level inferences. *F1000Research*, 4, 1521. <https://doi.org/10.12688/f1000research.7563.1>
- Storey, J. D., & Tibshirani, R. (2003). Statistical significance for genomewide studies. *Proceedings of the National Academy of Sciences*, 100(16), 9440-9445. <https://doi.org/10.1073/pnas.1530509100>
- Subramanian, A., Tamayo, P., Mootha, V. K., Mukherjee, S., Ebert, B. L., Gillette, M. A., Paulovich, A., Pomeroy, S. L., Golub, T. R., Lander, E. S., & Mesirov, J. P. (2005). Gene set enrichment analysis: A knowledge-based approach for interpreting genome-wide expression profiles. *Proceedings of the National Academy of Sciences*, 102(43), 15545-15550. <https://doi.org/10.1073/pnas.0506580102>

- The Gene Ontology Consortium. (2023). The Gene Ontology knowledgebase in 2023. *Genetics*, 224(1), Artículo iyado31. <https://doi.org/10.1093/genetics/iyad031>
- Wagner, G. P., Kin, K., & Lynch, V. J. (2012). Measurement of mRNA abundance using RNA-seq data: RPKM measure is inconsistent among samples. *Theory in Biosciences*, 131(4), 281-285. <https://doi.org/10.1007/s12064-012-0162-3>
- Wang, Z., Gerstein, M., & Snyder, M. (2009). RNA-Seq: a revolutionary tool for transcriptomics. *Nature Reviews Genetics*, 10(1), 57-63. <https://doi.org/10.1038/nrg2484>
- Young, M. D., Wakefield, M. J., Smyth, G. K., & Oshlack, A. (2010). Gene ontology analysis for RNA-seq: accounting for selection bias. *Genome Biology*, 11(2), Artículo R14. <https://doi.org/10.1186/gb-2010-11-2-r14>
- Zhu, A., Ibrahim, J. G., & Love, M. I. (2019). Heavy-tailed prior distributions for sequence count data: removing the noise and preserving large differences. *Bioinformatics*, 35(12), 2084-2092. <https://doi.org/10.1093/bioinformatics/bty895>

Transcriptómica de célula única



Las células, unidades básicas de la estructura y la función biológicas, varían ampliamente en tipo y en estado.

— Macosko et al. (2015)

● Al terminar este capítulo usted podrá

- Explicar cómo las gotas, los códigos de barras celulares y los UMI convierten miles de células en una matriz dispersa de cuentas, y qué artefactos (gotas vacías, ARN ambiental, dobletes) introduce la tecnología.
- Aplicar un control de calidad razonado (genes detectados, UMI totales, porcentaje mitocondrial, puntuación de dobletes) y justificar la normalización por tamaño de librería, la transformación logarítmica y la selección de genes altamente variables.
- Derivar el PCA como problema de valores propios y formular t-SNE y UMAP como optimizaciones sobre probabilidades de vecindad; reconocer lo que un *embedding* bidimensional puede y no puede decir sobre distancias.
- Construir un grafo de vecinos más cercanos, agruparlo maximizando la modularidad (Louvain, Leiden), encontrar genes marcadores con la prueba de Wilcoxon y corregir efectos de lote.

- Inferir trayectorias y pseudotiempo con mapas de difusión, resumirlas con PAGA e interpretar la velocidad de ARN a partir del modelo cinético de transcripción, maduración y degradación.

El RNA-seq *bulk* del capítulo 11 mide la expresión promedio de una muestra formada por muchas células. Esa medida es útil, pero puede ocultar diferencias entre tipos celulares. Un tumor combina células malignas, fibroblastos, células vasculares y linfocitos; la sangre reúne numerosos tipos inmunitarios. Si cambia la proporción de esos grupos, también cambia la media. Solo con ella no podemos saber si aumentó la expresión de un gen dentro de las células o si ahora hay más células que ya lo expresaban.

La transcriptómica de célula única (scRNA-seq) resuelve esa ambigüedad midiendo cada célula por separado. El precio es alto: cada perfil se construye con apenas unos miles de moléculas de ARN capturadas de entre cientos de miles, de modo que los datos son dispersos, ruidosos y dominados por ceros. El capítulo recorre el flujo de análisis estándar en el orden en que se ejecuta. Primero veremos cómo la química de gotas produce una matriz de cuentas y cómo limpiarla y normalizarla (sección 12.1). Después reduciremos miles de genes a unas pocas dimensiones con PCA y las visualizaremos con t-SNE y UMAP, aprendiendo a desconfiar de lo que las imágenes sugieren (sección 12.2). Luego agruparemos las células en un grafo, buscaremos los genes que definen cada grupo e integraremos muestras de distintos lotes (sección 12.3). Por último, ordenaremos las células a lo largo de procesos continuos, como la diferenciación, mediante pseudotiempo y velocidad de ARN (sección 12.4).

Todos los ejemplos usan datos *simulados* con una estructura conocida: unas 3000 células mononucleares de sangre con siete tipos celulares y un proceso de diferenciación ramificado. Conocer la verdad nos permitirá medir cuánto se equivoca cada método, algo imposible con datos reales. El código de producción usa Scanpy (Wolf et al., 2018); los cálculos de las figuras se implementan desde cero en `figuras/cap12/generar.py` para que cada número del texto sea reproducible.

12.1 Control de calidad y normalización

El licuado y la ensalada de frutas

Imagine que alguien le entrega un vaso de licuado y le pregunta qué frutas lleva. Por el color y el sabor puede adivinar que hay fresa y plátano, quizá algo de mango; pero si el licuado de hoy sabe más a fresa que el de ayer, no sabrá si cada fresa era más dulce o si simplemente pusieron más fresas. Ahora imagine que, en lugar del licuado, le entregan la ensalada de frutas antes de licuarla: puede contar las piezas, separarlas por tipo y probar cada una. El RNA-seq *bulk* es el licuado; el de célula única es la ensalada. La ensalada, eso sí, llega con trozos magullados, piezas pegadas entre sí y jugo de todas las frutas en el fondo del plato: antes de analizarla hay que limpiarla.

12.1.1 De la media a la mezcla

Supongamos que un tejido contiene K tipos celulares en proporciones $\pi_1, \dots, \pi_K$ y que el tipo k expresa el gen g con una abundancia media μ_{gk} . Un experimento *bulk* mide, salvo constantes de normalización,

$$\bar{x}_g = \sum_{k=1}^K \pi_k \mu_{gk}. \quad (12.1)$$

DONDE

$\bar{x}_g$	Expresión observada del gen g en la muestra agregada.
π_k	Fracción de células del tipo k en la muestra ($\sum_k \pi_k = 1$).
μ_{gk}	Expresión media del gen g dentro de las células del tipo k .

La ecuación (12.1) tiene K incógnitas de composición y K de expresión por cada gen, pero una sola observación. Un aumento de $\bar{x}_g$ al doble puede deberse a que μ_{gk} se duplicó en un tipo celular o a que π_k se duplicó sin que ninguna célula cambiara. Con datos de célula única estimamos π_k contando células y μ_{gk} promediando dentro de cada grupo: la ecuación deja de ser un problema mal planteado.

Del tubo único a los cientos de miles de gotas

El primer transcriptoma completo de una sola célula se publicó en 2009: Tang et al. (2009) amplificaron el ARNm de un único blastómero de ratón y lo secuenciaron, con un protocolo manual que procesaba una célula cada vez. Seis años después, dos grupos de Harvard publicaron en el mismo número de *Cell* la idea que cambió la escala: encerrar cada célula en una gota de aceite de un nanolitro junto con una

microesfera que porta millones de oligonucleótidos con un código de barras propio. Drop-seq (Macosko et al., 2015) perfiló 44 808 células de retina de ratón; inDrop (Klein et al., 2015) siguió la diferenciación de células madre embrionarias. En 2017, la versión comercial de 10x Genomics perfiló unas 68 000 células mononucleares de sangre periférica en un solo experimento (Zheng et al., 2017). Hoy los atlas celulares superan los millones de células.

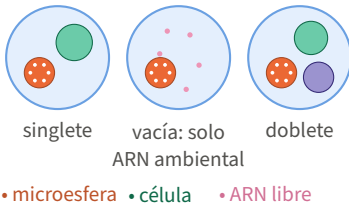
12.1.2 Gotas, códigos de barras y UMI

El principio de las plataformas de gotas se resume en la figura 12.1. Un chip microfluídico hace confluir tres corrientes: una suspensión de células, una suspensión de microesferas de gel y aceite. En la unión, el flujo se rompe en gotas de volumen casi constante, cada una de las cuales atrapa, con suerte, exactamente una célula y exactamente una microesfera. Dentro de la gota la célula se lisa, y su ARNm poliadenilado se hibrida con los oligonucleótidos de la microesfera. Cada oligonucleótido contiene:

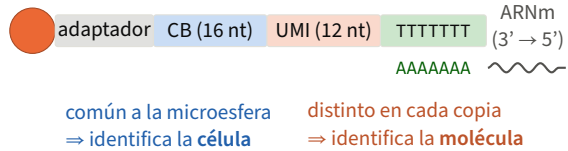
- un *código de barras celular* (*cell barcode*, CB), idéntico en todas las copias de una misma microesfera y distinto entre microesferas; identifica la *gota* y, por extensión, la célula;
- un *identificador molecular único* (UMI), una secuencia aleatoria distinta en cada copia; identifica la *molécula* de ARNm capturada;
- una cola de poli(dT), que se aparea con la cola poli(A) del ARNm y sirve de cebador para la transcripción inversa.

Tras la transcripción inversa se rompen las gotas, se juntan todos los ADNc (ya marcados) y se amplifican y secuencian juntos. Cada lectura trae, por un lado, el CB y el UMI (lectura 1) y, por el otro, un fragmento del transcrito (lectura 2), que se alinea al genoma para saber de qué gen procede. En Drop-seq el código de barras celular tiene 12 nucleótidos y el UMI 8 (Macosko et al., 2015); en la química actual de 10x, 16 y 12, respectivamente.

1. Encapsulación



2. Oligonucleótido de la microesfera



3. De lecturas a matriz de cuentas

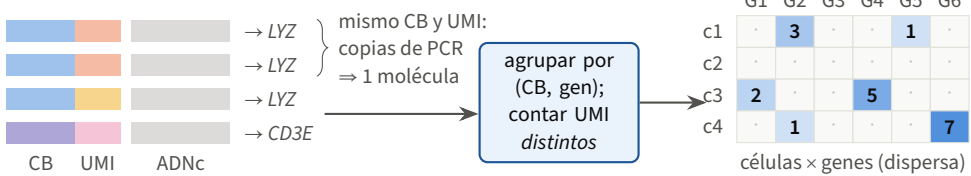


Figura 12.1. Flujo de una plataforma de gotas. (1) Cada gota de aceite atrapa idealmente una célula y una microesfera; también se forman gotas sin célula (que capturan ARN libre del medio, el ARN *ambiental*) y gotas con dos células (*dobletes*). (2) Cada oligonucleótido de la microesfera lleva un código de barras celular (CB) común a toda la microesfera, un UMI aleatorio propio y una cola poli(dT) que captura el ARNm por su cola poli(A). (3) Tras secuenciar, las lecturas con el mismo CB, el mismo UMI y el mismo gen son copias de PCR de una única molécula; contar UMI distintos por célula y gen produce la matriz de cuentas, en la que la mayoría de las entradas son cero. La matriz de ejemplo es la que se codifica en formato CSR en el ejemplo 12.1.

Cuántas células por gota. El número de células que cae en una gota es aleatorio. Si las células se diluyen de modo que haya en promedio λ células por gota, el número K de células en una gota dada sigue una distribución de Poisson:

$$P(K = k) = \frac{e^{-\lambda} \lambda^k}{k!}, \quad (12.2)$$

$$P(K \geq 2 \mid K \geq 1) = \frac{1 - e^{-\lambda} - \lambda e^{-\lambda}}{1 - e^{-\lambda}} \approx \frac{\lambda}{2}.$$

DONDE

λ	Número medio de células por gota (concentración de carga).
K	Número de células que contiene una gota concreta.
$P(K \geq 2 \mid K \geq 1)$	Fracción de dobletes entre las gotas que contienen al menos una célula.

La aproximación final sale de desarrollar $e^{-\lambda}$ en serie de Taylor hasta segundo orden: el numerador es $\lambda^2/2 + O(\lambda^3)$ y el denominador $\lambda + O(\lambda^2)$. La consecuencia práctica es un compromiso inevitable: con $\lambda = 0,05$ el 95,1 % de las gotas queda vacío y solo el 2,5 % de las ocupadas son dobletes; con $\lambda = 0,2$ el rendimiento se cuadruplica, pero los dobletes suben al 9,7 %, y con $\lambda = 0,5$, al 22,9 %. Cargar más células abarata el experimento y lo contamina. Los primeros artículos midieron esta tasa mezclando células humanas y de ratón: una gota cuyo transcriptoma es de ambas especies es, sin ambigüedad, un doblete (Macosko et al., 2015).

Por qué hacen falta los UMI. La PCR multiplica unas moléculas más que otras; contar lecturas mezcla la abundancia real con la eficiencia de amplificación. Kivioja et al. (2012) propusieron marcar cada molécula *antes* de amplificar con una secuencia aleatoria: todas las lecturas que comparten UMI (y gen, y célula) son copias de la misma molécula, y basta con contar UMI distintos. ¿Cuán largo debe ser el UMI? Si una célula tiene n moléculas de un gen y los UMI son de longitud L , el número esperado de pares que comparten UMI por azar es, por el argumento del problema del cumpleaños,

$$E[\text{colisiones}] \approx \binom{n}{2} \frac{1}{4^L} = \frac{n(n-1)}{2 \cdot 4^L}. \quad (12.3)$$

Con $L = 12$ ($4^{12} \approx 1,68 \times 10^7$) incluso un gen con 1000 moléculas en la célula produce $\approx 0,03$ colisiones; con $L = 10$ esa cifra sube a $\approx 0,48$. Por la misma cuenta, los $4^{16} \approx 4,3 \times 10^9$ códigos de barras posibles de 16 nt bastan para que dos microesferas compartan código muy rara vez.

12.1.3 La matriz de cuentas y su almacenamiento

El resultado del preprocesamiento (con herramientas como Cell Ranger, STARsolo o alevín) es una matriz $X \in \mathbb{N}^{n \times G}$ cuya entrada x_{cg} es el número de UMI del gen g en la célula c . Por convención de Scanpy las filas son células (observaciones) y las columnas genes (variables); el objeto AnnData guarda además, en tablas alineadas,

los metadatos de células (obs), de genes (var), representaciones de baja dimensión (obs_m) y grafos (obs_p) (Wolf et al., 2018).

Un conjunto típico de 10x detecta entre mil y tres mil genes por célula de unos 33 000 anotados: más del 90 % de las entradas son cero. Almacenar esos ceros es un despilfarro, y por eso la matriz se guarda en formato *disperso*.

Definición 12.1. (Formato CSR)

Una matriz en formato *compressed sparse row* (CSR) se representa con tres vectores: *data*, con los valores no nulos leídos por filas; *indices*, con la columna de cada valor; e *indptr*, de longitud $n + 1$, tal que los valores de la fila i ocupan las posiciones $\text{indptr}[i]$ a $\text{indptr}[i+1]-1$ de los dos primeros vectores.

Ejemplo 12.1. (Una matriz dispersa a mano)

La matriz de la figura 12.1 tiene 4 células, 6 genes y 6 entradas no nulas:

$$X = \begin{pmatrix} 0 & 3 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 5 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 7 \end{pmatrix} \Rightarrow \begin{aligned} \text{data} &= [3, 1, 2, 5, 1, 7] \\ \text{indices} &= [1, 4, 0, 3, 1, 5] \\ \text{indptr} &= [0, 2, 2, 4, 6] \end{aligned}$$

La fila 1 (la segunda, contando desde cero) ocupa el intervalo vacío $[2, 2)$: es una célula sin cuentas. Para un atlas de 10^6 células y 33 538 genes, la matriz densa en float32 ocupa $10^6 \times 33\,538 \times 4 \approx 134$ GB. Si cada célula tiene en promedio 2000 genes detectados, CSR necesita 2×10^9 valores y 2×10^9 índices de 4 bytes: unos 16 GB, ocho veces menos. En nuestra simulación de 2000 genes ya prefiltrados la densidad es del 47 % y el ahorro es mínimo; el formato disperso rinde cuando la matriz incluye todo el genoma.

12.1.4 Gotas vacías y ARN ambiental

La mayoría de los códigos de barras que aparecen en los datos no corresponden a células: vienen de gotas vacías que capturaron ARN libre, liberado por células rotas en la suspensión. La forma clásica de separarlas es ordenar los códigos por su número total de UMI y dibujar la curva en escala log-log (figura 12.2). Las células

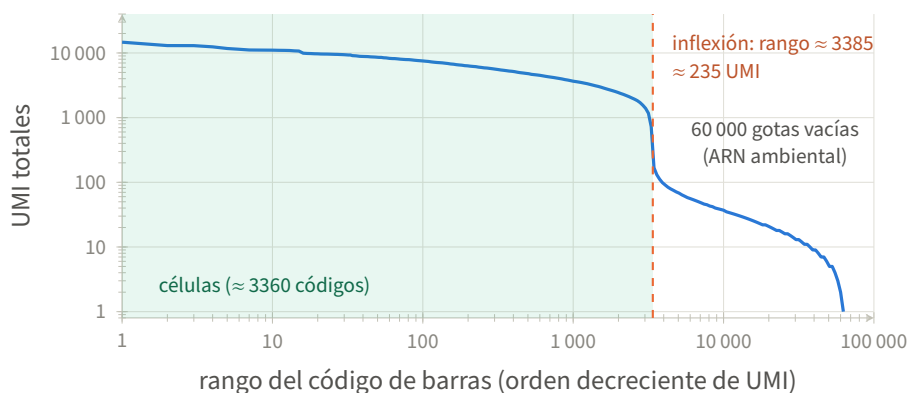


Figura 12.2. Curva de rodilla de un experimento simulado con 3360 códigos de barras que contienen células (incluidas células dañadas y dobles) y 60 000 gotas vacías. Cada punto es un código de barras; el eje horizontal es su posición al ordenarlos de mayor a menor número de UMI. La meseta de la izquierda son células; la cola de la derecha, gotas con unas decenas de moléculas de ARN ambiental. El punto de máxima pendiente (línea naranja), calculado sobre la curva suavizada, separa ambos regímenes. Las células pequeñas o dañadas caen cerca de la rodilla y son las más difíciles de clasificar. Generada con `figuras/cap12/generar.py`.

forman una meseta alta; las gotas vacías, una cola larga de pocas cuentas; entre ambas hay una caída abrupta, la «rodilla».

Las gotas vacías se descartan, pero su huella queda en las gotas con célula: todas contienen además una pequeña muestra del ARN ambiental. Si llamamos p_{cg} a la fracción de las moléculas endógenas de la célula c que proceden del gen g , a_g a la fracción del gen g en la «sopa» ambiental y ρ_c a la fracción contaminante, el valor esperado de la cuenta observada es

$$\mathbb{E}[x_{cg}] = N_c \left[(1 - \rho_c) p_{cg} + \rho_c a_g \right]. \quad (12.4)$$

DONDE

N_c	Número total de UMI de la célula c (tamaño de su librería).
p_{cg}	Perfil de expresión verdadero de la célula (fracción del gen g).
a_g	Perfil ambiental, estimable a partir de las gotas vacías.
ρ_c	Fracción de las moléculas de la célula que proviene del ambiente.

SoupX estima a_g con las gotas vacías y ρ_c con genes que se sabe que *no* se expresan en ciertos tipos celulares, y resta la contribución ambiental (Young &

Behjati, 2020). El efecto no es cosmético. En nuestra simulación, con un $\rho = 2\%$, el 15,2 % de las células B muestra al menos un UMI de *LYZ*, un gen de monocitos que las células B no expresan: 0,23 UMI de media, frente a 5,7 en los monocitos CD14. Un análisis ingenuo diría que «algunas células B expresan *LYZ*».

12.1.5 Métricas de control de calidad

Tras separar las células de las gotas vacías, el control de calidad (QC) busca células cuyo perfil no refleja una célula sana e individual. Las tres métricas estándar (Luecken & Theis, 2019) se calculan por célula:

$$N_c = \sum_g x_{cg}, \quad D_c = \sum_g \mathbb{1}[x_{cg} > 0], \quad m_c = 100 \frac{\sum_{g \in \mathcal{M}} x_{cg}}{N_c}. \quad (12.5)$$

DONDE

N_c	UMI totales (<code>total_counts</code> en Scanpy).
D_c	Genes detectados (<code>n_genes_by_counts</code>).
$\mathcal{M}$	Conjunto de genes mitocondriales (nombres que empiezan por MT-en humano).
m_c	Porcentaje de UMI mitocondriales (<code>pct_counts_mt</code>).

La interpretación biológica es lo que las hace útiles. Una célula cuya membrana se ha roto pierde ARNm citoplasmático, pero conserva el que está protegido dentro de las mitocondrias: su N_c y su D_c bajan y su m_c sube. Un doblete, al revés, tiende a tener más UMI y más genes que un singlete. Una gota vacía con mucho ambiente tiene pocas cuentas y un perfil mezclado.

En lugar de umbrales fijos, las guías actuales recomiendan umbrales adaptativos basados en la desviación absoluta mediana (MAD), robusta frente a los propios valores atípicos que se buscan (Heumos et al., 2023):

$$\text{descartar } c \iff |\log N_c - \text{med}(\log N)| > k \cdot \text{MAD}(\log N), \quad (12.6)$$

$$\text{MAD}(z) = 1,4826 \text{ med}|z - \text{med}(z)|,$$

con $k = 5$ para N_c y D_c (umbrales permisivos) y reglas análogas, más estrictas ($k = 3$), para m_c . El factor 1,4826 hace que el MAD estime la desviación estándar cuando los datos son normales.

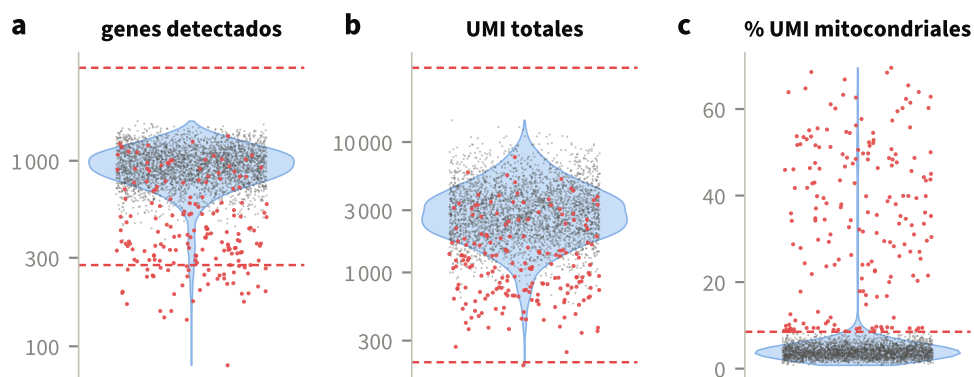


Figura 12.3. Métricas de QC de los 3360 códigos de barras con célula de la simulación: 3000 células sanas, 150 células dañadas y 210 dobles. Cada punto es una célula; las líneas rojas discontinuas son los umbrales adaptativos (ecuación (12.6)): 5 MAD sobre la escala logarítmica para genes (a) y UMI (b), y la mediana más 3 MAD, 8,52 %, para la fracción mitocondrial (c). En rojo, las 185 células descartadas: las 150 dañadas y 35 sanas del extremo inferior. Obsérvese que la cola de células dañadas se ve con claridad en c pero se confunde con las células sanas pequeñas en a y b, y que ningún doblete supera los umbrales superiores: el QC univariante no los detecta.

Ejemplo 12.2. (QC adaptativo sobre la simulación)

En los 3360 códigos con célula, la mediana es de 2763 UMI, 933 genes y 3,89 % mitocondrial, y el MAD de $\log N$ vale 0,519. Los umbrales resultan: UMI entre 205 y 37 117, genes entre 274 y 3171, y porcentaje mitocondrial menor que 8,52 %. Se descartan las 150 células dañadas (el 100 %) y 35 de las 3000 sanas (1,2 %), y quedan 3175. Pero los 210 dobletes pasan todos: una célula doble no es anómala en ninguna métrica por separado, porque la variabilidad natural del tamaño de las células (unas tres veces entre monocitos y linfocitos) cubre el aumento de UMI.

⚠ Los umbrales dependen del tejido

No existe un porcentaje mitocondrial «correcto». Los cardiomiocitos, los hepatocitos o las células renales tienen de forma natural una alta carga mitocondrial, y un umbral del 5 % copiado de un tutorial de sangre eliminaría precisamente las células de interés. Del mismo modo, una población rara de

células pequeñas (neutrófilos, plaquetas) puede caer por debajo del umbral inferior de genes. Mire siempre las distribuciones por muestra, descarte con umbrales permisivos y revise después si algún *cluster* está definido solo por métricas de calidad.

12.1.6 Dobletes

Los dobles *heterotípicos* (dos tipos celulares distintos) son peligrosos porque parecen un estado intermedio, o un tipo celular nuevo, que no existe. Los *homotípicos* (dos células del mismo tipo) son casi indistinguibles de una célula grande, pero también mucho menos dañinos. Scrublet (Wolock et al., 2019) ataca el problema con una idea sencilla: si no sabemos cómo es un doblete real, fabriquemos dobles artificiales sumando los perfiles de pares de células elegidas al azar, y preguntemos a cada célula observada cuántos de sus vecinos más cercanos son artificiales. Si en el espacio combinado hay r dobles simulados por cada célula observada y q_c es la fracción de vecinos simulados de la célula c , la puntuación ajustada por la tasa esperada ρ es

$$s_c = \frac{q_c \rho / r}{1 - \rho - q_c (1 - \rho - \rho / r)} \tag{12.7}$$

DONDE		
q_c		Fracción de los k vecinos más cercanos de c que son dobles simulados.
r		Razón entre el número de dobles simulados y el de células observadas.
ρ		Tasa de dobles esperada, fijada según la carga del experimento (ecuación (12.2)).
s_c		Probabilidad estimada de que c sea un doblete (puntuación de Scrublet).

La ecuación (12.7) es la regla de Bayes: la densidad de dobles simulados alrededor de c (corregida por r) frente a la de singletes observados, con ρ como probabilidad previa.

12.1.7 Normalización

Dos células idénticas pueden producir librerías de tamaño muy distinto: la eficiencia de captura y de transcripción inversa varía de una gota a otra. Antes de

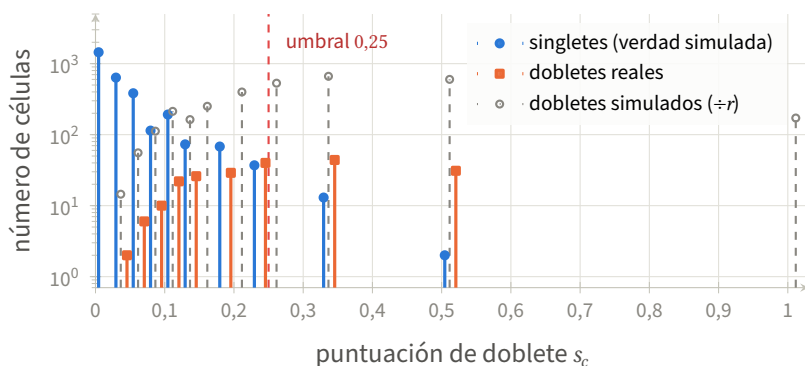


Figura 12.4. Distribución de la puntuación de doblete (ecuación (12.7)) en las 3175 células que superaron el QC, con $k = 28$ vecinos, $r = 2$ y $\rho = 0,07$. Los singletes se concentran cerca de cero (mediana 0,027); los dobletes simulados (gris) se reparten por valores mucho más altos y marcan dónde deben caer los dobletes reales. Como q_c solo puede tomar los valores $j/28$, la puntuación es discreta, y por eso cada distribución aparece como un conjunto de barras aisladas. Con un umbral de 0,25 se detecta el 40,9 % de los dobletes heterotípicos y solo el 12,8 % de los homotípicos, a costa de marcar el 0,51 % de los singletes. La detección es imperfecta, pero elimina justamente los dobletes que más engañan: los que unen tipos celulares distantes. Escala vertical logarítmica.

comparar células hay que eliminar esa variación técnica, sin borrar la biológica. El modelo de partida para las cuentas de UMI es la binomial negativa (BN), el mismo modelo gamma-Poisson del capítulo 11:

$$x_{cg} \sim \text{BN}(\mu_{cg}, \theta_g), \quad \mu_{cg} = s_c q_{g,t(c)},$$

$$\text{Var}[x_{cg}] = \mu_{cg} + \frac{\mu_{cg}^2}{\theta_g}, \quad \mathbb{P}(x_{cg} = 0) = \left(\frac{\theta_g}{\theta_g + \mu_{cg}} \right)^{\theta_g}. \quad (12.8)$$

DONDE

s_c	Factor de tamaño de la célula c : eficiencia técnica global de su librería.
$q_{g,t(c)}$	Expresión del gen g en el estado biológico $t(c)$ de la célula.
θ_g	Parámetro de dispersión inversa; $\theta \rightarrow \infty$ recupera la Poisson.

Un punto que costó años aclarar: los muchos ceros de los datos de gotas *no* requieren un modelo con inflación de ceros. Svensson (2020) comprobó, en experimentos de control con ARN sintético sin variación biológica, que la fracción de

ceros observada es la que predice la BN con la media observada. Los ceros son, sobre todo, genes poco expresados muestreados con poca profundidad.

Normalización por tamaño de librería y logaritmo. El procedimiento por defecto en Scanpy estima $s_c \propto N_c$, escala cada célula a un total común (típicamente 10^4 , «cuentas por diez mil» o CP10k) y aplica un logaritmo desplazado:

$$y_{cg} = \log\left(1 + \frac{x_{cg}}{N_c} \cdot 10^4\right). \quad (12.9)$$

¿Por qué el logaritmo? Porque la varianza de las cuentas crece con la media, y los métodos posteriores (PCA, distancias euclídeas) dan más peso a los genes de mayor varianza. Por el método delta, si X tiene media μ y varianza σ^2 ,

$$\text{Var}[\log(1 + X)] \approx \frac{\sigma^2}{(1 + \mu)^2} = \frac{\mu + \mu^2/\theta}{(1 + \mu)^2} \xrightarrow{\mu \rightarrow \infty} \frac{1}{\theta}. \quad (12.10)$$

Para genes muy expresados el logaritmo estabiliza la varianza en $1/\theta$, independiente de la media. Para genes poco expresados ($\mu \ll 1$) la varianza queda $\approx \mu$: la transformación no los estabiliza del todo, y el pseudoconteo 1 (arbitrario) influye en el resultado.

Ejemplo 12.3. (Normalizar una célula)

Un monocito CD14 de la simulación tiene $N_c = 6359$ UMI, 5 de ellos de *LYZ*. Su valor normalizado es $5/6359 \times 10^4 = 7,9$ CP10k, y tras la ecuación (12.9), $y = \ln(1 + 7,9) = 2,182$. Un linfocito con la mitad de UMI y la misma proporción de *LYZ* tendría exactamente el mismo y , que es lo que buscamos: la normalización compara proporciones, no cuentas.

Factores de tamaño por agrupamiento. Suponer $s_c \propto N_c$ equivale a suponer que todas las células tienen el mismo contenido total de ARN, lo cual es falso entre tipos celulares, y la estimación es muy ruidosa en células con pocas cuentas. El método de Lun et al. (2016), implementado en *scrn*, suma las cuentas de grupos (*pools*) de células parecidas, donde hay pocos ceros y los cocientes de medianas son estables, estima un factor para cada grupo y recupera los factores individuales resolviendo un sistema lineal:

$$\hat{S}_p = \text{med}_g \frac{\sum_{c \in P} x_{cg}}{\bar{x}_g} \approx \sum_{c \in P} s_c \quad \text{para muchos grupos } P, \quad (12.11)$$

con $\bar{x}_g$ la media del gen en una pseudocélula de referencia. Cada célula participa en varios grupos, de modo que hay más ecuaciones que incógnitas y el sistema se resuelve por mínimos cuadrados.

Residuos de Pearson (sctransform). Hafemeister y Satija (2019) propusieron abandonar la división por N_c y modelar la profundidad directamente. Para cada gen se ajusta una regresión BN con el logaritmo del tamaño de la librería como covariable, se regularizan los parámetros compartiendo información entre genes de expresión parecida, y se usan como datos normalizados los residuos de Pearson:

$$\log \mu_{cg} = \beta_{0g} + \beta_{1g} \log N_c, \quad r_{cg} = \frac{x_{cg} - \hat{\mu}_{cg}}{\sqrt{\hat{\mu}_{cg} + \hat{\mu}_{cg}^2 / \hat{\theta}_g}}. \quad (12.12)$$

DONDE

β_{0g}, β_{1g}	Intercepto y pendiente de la regresión del gen g sobre la profundidad.
$\hat{\mu}_{cg}$	Cuenta esperada si la célula solo difiriera de las demás en profundidad.
r_{cg}	Residuo de Pearson: desviación en unidades de desviación estándar del modelo.

Si el gen solo varía por razones técnicas, r_{cg} tiene media 0 y varianza 1 en todas las células, sea cual sea la profundidad; la variación biológica aparece como exceso de varianza. Seurat la implementa en la función `SCTransform`.

12.1.8 Genes altamente variables

De unos 20 000 genes, la mayoría no distingue tipos celulares: son genes constitutivos o genes apenas detectados cuya variación es ruido de muestreo. Restringir el análisis a los genes *altamente variables* (HVG) reduce el ruido y el coste. El criterio no puede ser la varianza bruta, porque la varianza crece con la media; hay que comparar cada gen con los genes de expresión media similar. En el sabor clásico, se calcula la dispersión $d_g = \log(\sigma_g^2 / \mu_g)$, se agrupan los genes en intervalos de media y dentro de cada intervalo se estandariza:

$$z_g = \frac{d_g - \text{media}\{d_h : h \in B(g)\}}{\text{de}\{d_h : h \in B(g)\}}, \quad (12.13)$$

con $B(g)$ el intervalo de medias al que pertenece g . Se retienen los genes de mayor z_g (de 2000 a 5000 en datos reales).

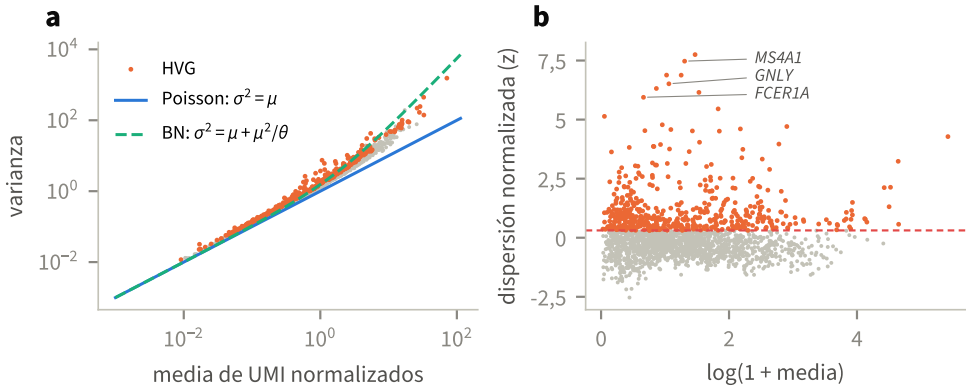


Figura 12.5. (a) Varianza frente a media de las cuentas normalizadas por factor de tamaño para los 2000 genes simulados (escala log-log). La línea azul es la varianza de Poisson; la verde, una BN con la dispersión estimada por momentos. Casi todos los genes siguen la curva de ruido; los que la superan con creces (naranja) son los que varían entre tipos celulares. (b) Dispersión normalizada dentro de intervalos de media (ecuación (12.13)) frente a la expresión media. Los 500 genes por encima de la línea discontinua son los HVG; entre ellos están 31 de los 32 marcadores simulados y ningún gen mitocondrial.

La figura 12.5 muestra el resultado en la simulación. Un detalle instructivo: la dispersión BN estimada por momentos con los datos normalizados de las células T CD4 vale $\hat{\theta} \approx 1,83$, muy por debajo del valor simulado (mediana 6,02). No es un error de cálculo: al dividir por un factor de tamaño aleatorio, las células pequeñas inflan la componente de Poisson de la varianza, y las cuentas normalizadas ya no son binomiales negativas. Es la razón de fondo por la que Hafemeister y Satija (2019) prefieren modelar la profundidad a dividir por ella.

```
qc_normalizacion.py
```

```
import scanpy as sc
```

```
adata = sc.read_10x_h5("filtered_feature_bc_matrix.h5")
adata.var_names_make_unique()
adata.var["mt"] = adata.var_names.str.startswith("MT-")
sc.pp.calculate_qc_metrics(adata, qc_vars=["mt"],
                           percent_top=None, log1p=False,
                           inplace=True)
sc.pl.violin(adata, ["n_genes_by_counts", "total_counts",
                    "pct_counts_mt"], jitter=0.4, multi_panel=True)
```

```

ok = ((adata.obs.n_genes_by_counts > 250)
      & (adata.obs.pct_counts_mt < 8))
adata = adata[ok].copy()
sc.pp.filter_genes(adata, min_cells=3)
sc.pp.scrublet(adata) # Scanpy >= 1.10
adata = adata[~adata.obs["predicted_doublet"]].copy()
adata.layers["counts"] = adata.X.copy() # cuentas crudas
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)
sc.pp.highly_variable_genes(adata, n_top_genes=2000,
                             flavor="seurat_v3", layer="counts")

```

⚠ El orden importa

La selección de HVG con `flavor="seurat_v3"` espera cuentas *crudas* (por eso se guardan en una capa); los sabores "seurat" y "cell_ranger" esperan datos log-normalizados. Scrublet debe ejecutarse sobre cuentas crudas y, si hay varias muestras, por separado en cada una, porque los dobletes solo pueden formarse dentro de un mismo canal.

12.2 PCA, t-SNE y UMAP

 Colab · 12.2 · PCA, t-SNE y UMAP

 repositorio del curso

Sombras chinescas

Un titiritero proyecta en la pared la sombra de sus manos. De una misma mano, según cómo la gire frente a la lámpara, puede salir un conejo, un perro o un borrón irreconocible. Toda proyección pierde información; la cuestión es qué ángulo conserva lo que nos importa. Reducir la dimensión de los datos de célula única es buscar ese ángulo: pasar de miles de genes a dos coordenadas que quepan en una página. El PCA elige el ángulo en el que la sombra es más *grande*; t-SNE y UMAP renuncian a la proyección rígida y moldean una sombra nueva en la que los vecinos siguen siendo vecinos, aunque las distancias largas dejen de significar algo.

Después de la normalización tenemos una matriz de $n \approx 3000$ células por 500 HVG. Cada célula es un punto en un espacio de 500 dimensiones. Pero los genes no varían de forma independiente: los programas transcripcionales mueven decenas de genes a la vez, así que la información está concentrada en un subespacio de dimensión mucho menor. La reducción de dimensión cumple dos papeles distintos que conviene no confundir: *compresión* para el análisis (PCA, unas 30–50 dimensiones, sobre las que se calculan vecinos, grupos y trayectorias) y *visualización* (t-SNE, UMAP, dos dimensiones, solo para mirar).

12.2.1 Análisis de componentes principales

Sea $Z \in \mathbb{R}^{n \times p}$ la matriz de expresión con cada gen centrado (y, habitualmente, escalado a varianza unitaria y truncado en ± 10 para que ningún gen domine). La matriz de covarianzas es $S = \frac{1}{n-1} Z^T Z$. Buscamos la dirección unitaria w en la que la proyección Zw tiene varianza máxima:

$$\max_{w \in \mathbb{R}^p} w^T S w \quad \text{sujeto a} \quad w^T w = 1. \quad (12.14)$$

Con el lagrangiano $\mathcal{L}(w, \lambda) = w^T S w - \lambda(w^T w - 1)$ e igualando el gradiente a cero,

$$\nabla_w \mathcal{L} = 2Sw - 2\lambda w = 0 \implies Sw = \lambda w, \quad w^T Sw = \lambda w^T w = \lambda. \quad (12.15)$$

DONDE

Z	Matriz células $\times$ genes centrada (y escalada).
S	Matriz de covarianzas entre genes, simétrica y semidefinida positiva.
w	Vector de cargas (<i>loadings</i>): peso de cada gen en la componente.
λ	Multiplicador de Lagrange; resulta ser la varianza capturada.

Los puntos críticos son los vectores propios de S , y la varianza capturada es el valor propio correspondiente: el máximo es el vector propio del mayor valor propio. Repitiendo el argumento con la restricción adicional de ortogonalidad a las direcciones ya encontradas se obtienen las siguientes componentes, en orden decreciente de λ . Como S es simétrica, sus vectores propios son ortogonales y el procedimiento está bien definido. En la práctica no se forma S : se usa la descomposición en valores singulares $Z = U\Sigma V^T$, de la que se lee todo a la vez.

Teorema 12.1. (PCA mediante la SVD)

Si $Z = U\Sigma V^T$ con $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$, entonces las columnas de V son las direcciones principales, las coordenadas de las células son $ZV = U\Sigma$, la varianza de la componente k es $\lambda_k = \sigma_k^2/(n-1)$ y la fracción de varianza explicada es $\lambda_k / \sum_j \lambda_j$. Además, $U_k \Sigma_k V_k^T$ (truncada a las k primeras) es la mejor aproximación de rango k de Z en norma de Frobenius (Hastie et al., 2009).

La última afirmación da la justificación estadística de usar las primeras componentes como entrada de todo lo que sigue: son la mejor compresión lineal posible, y las componentes de menor varianza, descartadas, contienen sobre todo ruido de muestreo. La derivación desde el punto de vista de la genética de poblaciones se discute en el capítulo 10.

¿Cuántas componentes?. La figura 12.6 muestra la varianza explicada en la simulación. Es poca en términos absolutos: PC1 explica el 2,45 % y las 30 primeras, el 15,2 %; el resto es ruido de Poisson, que se reparte en todas las direcciones. Para saber qué componentes contienen señal, comparamos con un modelo nulo en el que se permutan las células independientemente en cada gen, lo que destruye las correlaciones y conserva las distribuciones marginales. En los datos permutados ninguna componente supera el 0,39 %; en los reales, siete sí, tantas como la estructura simulada (siete tipos celulares definen seis contrastes, más un estado continuo dentro de las células T CD4). En la práctica se usan 30–50 componentes; tomar de más es barato, tomar de menos puede fundir poblaciones raras.

12.2.2 t-SNE: conservar vecindarios

El PCA es lineal y global: preserva bien las grandes distancias y comprime los grupos pequeños unos sobre otros (figura 12.7a). El *t-distributed stochastic neighbor embedding* (t-SNE), propuesto por van der Maaten y Hinton en 2008 y popularizado en célula única, invierte la prioridad: renuncia a las distancias largas para conservar los vecindarios (Kobak & Berens, 2019).

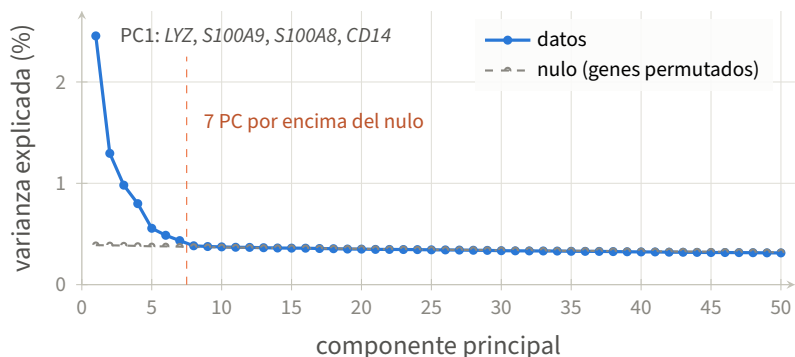


Figura 12.6. Gráfico de codo (*elbow plot*) de las 50 primeras componentes principales de los 500 HVG de la simulación, frente a un modelo nulo en el que cada gen se permuta de forma independiente. Las siete primeras componentes superan claramente al nulo; a partir de ahí la curva se aplana en una meseta de ruido. PC1 separa células mieloides de linfoides (sus mayores cargas son genes de monocitos), mientras que PC2 está dominada por los genes del estado continuo simulado dentro de las células T CD4.

Similitudes en alta dimensión. Para cada célula i , t-SNE define una distribución de probabilidad sobre las demás, gaussiana en la distancia:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}, \quad (12.16)$$

$$\text{Perp}(P_i) = 2^{H(P_i)}, \quad H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}.$$

DONDE

x_i	Coordenadas de la célula i en el espacio de entrada (p. ej., 30 PC).
σ_i	Ancho de banda propio de cada célula, elegido por bisección.
Perp	Perplejidad: número efectivo de vecinos que «ve» cada célula; es el único parámetro que fija el usuario.
$H(P_i)$	Entropía de Shannon (en bits) de la distribución de vecinos de i .

El ancho σ_i no es común: se ajusta en cada célula para que la perplejidad sea la deseada. Así, en regiones densas σ_i es pequeño y en regiones dispersas es grande, y cada célula considera aproximadamente el mismo número de vecinos. Las probabilidades se simetrizan, $p_{ij} = (p_{j|i} + p_{i|j})/2n$.

Ejemplo 12.4. (La perplejidad como número de vecinos)

Para una célula de la simulación, en el espacio de 30 PC, la bisección da $\sigma = 0,893$ con perplejidad 5, $\sigma = 1,722$ con 30 y $\sigma = 2,834$ con 300. Con perplejidad 5, su vecino más cercano recibe el 42,4 % de la probabilidad y solo 13 células superan la probabilidad uniforme $1/(n-1)$; con 300, el vecino más cercano recibe el 2,5 % y 331 células superan ese nivel. Una perplejidad de 2^H equivale a la incertidumbre de elegir uniformemente entre 2^H vecinos.

Similitudes en baja dimensión y la cola pesada. En el mapa bidimensional $y_1, \dots, y_n$ las similitudes se miden con una distribución t de Student con un grado de libertad (Cauchy):

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|y_k - y_i\|^2)^{-1}}. \quad (12.17)$$

¿Por qué no otra gaussiana? Por el *problema de hacinamiento*: en 30 dimensiones caben muchos más puntos a distancia moderada de una célula que en un plano. Si forzamos a todos esos vecinos moderados a caber alrededor de ella en dos dimensiones, se amontonan. La cola pesada de la t permite que los puntos moderadamente lejanos se coloquen mucho más lejos en el mapa sin penalización, lo que abre espacio y separa los grupos.

Teorema 12.2. (Función de costo y gradiente de t-SNE)

t-SNE minimiza la divergencia de Kullback-Leibler entre las similitudes de alta y baja dimensión,

$$C = \text{KL}(P \| Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}, \quad (12.18)$$

cuyo gradiente respecto de la posición de la célula i es

$$\frac{\partial C}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) (y_i - y_j) (1 + \|y_i - y_j\|^2)^{-1}. \quad (12.19)$$

La ecuación (12.19) se lee como un sistema de resortes: si $p_{ij} > q_{ij}$ (las células son más vecinas en el espacio original que en el mapa) el término atrae a y_i hacia y_j ; si $p_{ij} < q_{ij}$, las repele. La KL es asimétrica, y esa asimetría es la esencia del método: poner lejos a dos vecinos reales (grande p , pequeño q) cuesta mucho; poner cerca a

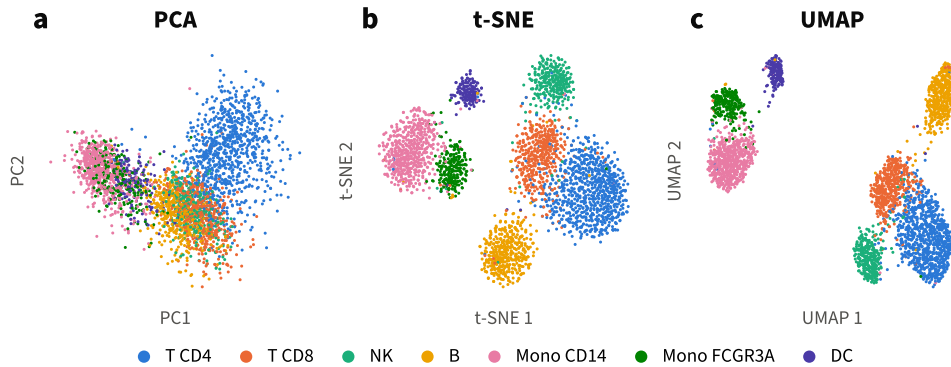


Figura 12.7. Las 3085 células que superaron el QC, representadas con tres métodos a partir de las mismas 30 componentes principales y coloreadas por el tipo celular *simulado* (que ningún método conoce). (a) Las dos primeras PC separan el eje mieloide-linfoide y el estado continuo de las células T CD4, pero superponen tipos que difieren en direcciones de menor varianza. (b) t-SNE (perplejidad 30, inicialización PCA) separa los siete tipos en islas compactas. (c) UMAP ($k = 15$, $\text{min_dist} = 0.3$) produce grupos similares, más compactos y con más espacio vacío entre ellos. Ni el tamaño de las islas ni la distancia entre ellas deben leerse al pie de la letra (figura 12.9).

dos células lejanas (pequeño p , grande q) cuesta poco. t-SNE conserva vecindarios y deja libres las distancias largas.

La perplejidad cambia la imagen. La figura 12.8 muestra el efecto del único parámetro de t-SNE. Con perplejidad baja, cada célula solo atiende a un puñado de vecinos y los grupos se fragmentan en subgrupos espurios; con perplejidad alta, el mapa conserva mejor la estructura global, pero los tipos cercanos (monocitos CD14 y FCGR3A; linfocitos T CD4 y CD8) se funden. Kobak y Berens (2019) recomiendan inicializar con PCA (para heredar la estructura global en lugar de partir de posiciones aleatorias), usar una perplejidad del orden de $n/100$ en conjuntos grandes o combinar varias escalas, y aumentar la «exageración» de las atracciones al inicio. Para millones de células, FIt-SNE acelera el cálculo interpolando las fuerzas repulsivas en una rejilla (Linderman et al., 2019).

12.2.3 UMAP: un grafo difuso

UMAP (*uniform manifold approximation and projection*) parte de la teoría de conjuntos difusos y la topología algebraica, pero su algoritmo se parece mucho a

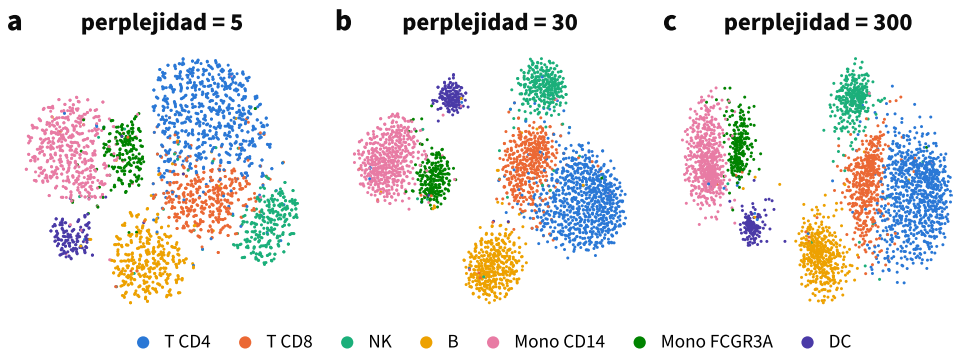


Figura 12.8. t-SNE de los mismos datos con perplejidades 5, 30 y 300 (misma semilla, inicialización PCA). **(a)** Con perplejidad 5 las islas se ven granuladas: cada célula solo «ve» a unos pocos vecinos y aparecen agrupamientos locales sin significado biológico. **(b)** El valor por defecto, 30. **(c)** Con perplejidad 300 (el 10 % de las células) la disposición global es más fiel a las relaciones entre linajes, pero los tipos emparentados se funden. No hay una perplejidad «correcta»: conviene mirar varias.

t-SNE (McInnes et al., 2018). Primero construye un grafo de k vecinos más cercanos con pesos

$$v_{ji} = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right), \quad \sum_{j=1}^k v_{ji} = \log_2 k, \quad (12.20)$$
$$v_{ij} = v_{ji} + v_{ilj} - v_{ji} v_{ilj},$$

DONDE	
ρ_i	Distancia de i a su vecino más cercano: garantiza que toda célula tenga al menos un vecino con peso 1.
σ_i	Escala local, elegida para que la suma de pesos sea $\log_2 k$ (análoga a la perplejidad).
v_{ij}	Peso simétrico de la arista: la probabilidad de que exista al menos una de las dos aristas dirigidas (unión difusa).
k	Número de vecinos (n_neighbors); controla el equilibrio entre estructura local y global.

y después busca coordenadas y_i cuyas similitudes $w_{ij} = (1 + a\|y_i - y_j\|^{2b})^{-1}$ se parezcan a v_{ij} , minimizando la entropía cruzada

$$CE = \sum_{i \neq j} \left[v_{ij} \log \frac{v_{ij}}{w_{ij}} + (1 - v_{ij}) \log \frac{1 - v_{ij}}{1 - w_{ij}} \right]. \quad (12.21)$$

El primer término es el atractivo de t-SNE; el segundo, ausente en la KL, penaliza explícitamente colocar cerca a células que no son vecinas. Los parámetros a y b se ajustan a partir de `min_dist`, que fija lo compactos que se permite que queden los grupos. Como las similitudes no se normalizan globalmente, la optimización se puede hacer por descenso de gradiente estocástico muestreando aristas, lo que la hace muy rápida. Becht et al. (2019) mostraron en datos de citometría y de célula única que UMAP es más rápido que las implementaciones de t-SNE de entonces y que organiza mejor los grupos y la continuidad entre poblaciones emparentadas. Conviene matizarlo: t-SNE con inicialización PCA y perplejidades grandes también conserva buena parte de la estructura global (Kobak & Berens, 2019).

12.2.4 Lo que un embedding no dice

Un mapa bidimensional es una herramienta de exploración, no una medida. La figura 12.9 cuantifica la advertencia con la simulación: para 4000 pares de células al azar comparamos su distancia en el espacio de 30 PC con su distancia en el mapa.

Chari y Pachter (2023) llevaron el argumento al extremo: mostraron que los *embeddings* de t-SNE y UMAP distorsionan severamente las distancias de conjuntos de datos reales, y que se puede forzar a que los datos adopten casi cualquier forma deseada (incluso la de un elefante) conservando niveles de preservación de vecindarios comparables. La lección práctica es una lista de prohibiciones:

- **No interprete tamaños.** t-SNE y UMAP adaptan la escala local a la densidad; un grupo grande en el mapa no es un grupo más heterogéneo.
- **No interprete distancias entre grupos.** En nuestra simulación, la correlación entre las distancias de los centroides de los siete tipos en el espacio de PC y en el mapa es de 0,835 para t-SNE y 0,801 para UMAP: el orden se conserva en gran parte, pero no la magnitud.
- **No cuantifique en 2D.** Agrupamientos, trayectorias y distancias se calculan en el espacio de PC (o en el grafo de vecinos), nunca en las coordenadas del mapa.

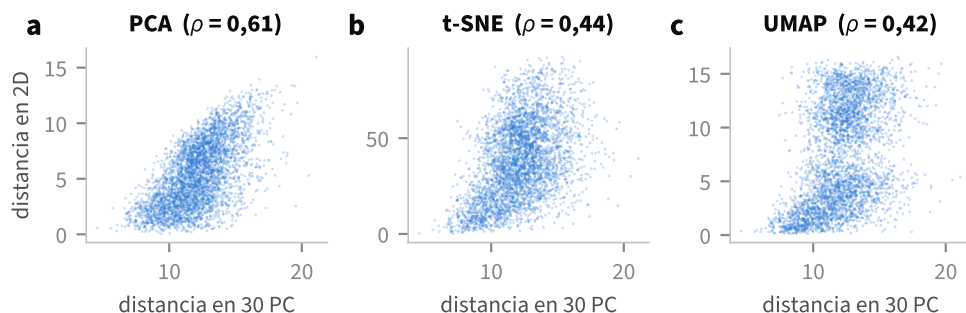


Figura 12.9. Distancia entre 4000 pares de células al azar en el espacio de 30 PC (horizontal) frente a su distancia en cada mapa bidimensional (vertical), con la correlación de Spearman ρ . El PCA en 2D conserva mejor el orden de las distancias ($\rho = 0,61$) pero es una mala representación local: solo el 4,3 % de los 15 vecinos de cada célula siguen siéndolo en el plano. t-SNE ($\rho = 0,44$) y UMAP ($\rho = 0,42$) conservan más vecinos (23,3 % y 11,9 %), pero la relación entre distancias reales y aparentes es no monótona: a igual distancia real, dos pares pueden quedar en el mismo grupo del mapa o en extremos opuestos.

PCA para calcular; t-SNE y UMAP para mirar. Un vecino en el mapa es probablemente un vecino real; una distancia en el mapa no es una distancia.

reduccion.py

```
sc.pp.scale(adata, max_value=10)           # z-score, truncado en 10
sc.tl.pca(adata, n_comps=50, mask_var="highly_variable")
sc.pl.pca_variance_ratio(adata, n_pcs=50, log=True)
sc.pp.neighbors(adata, n_neighbors=15, n_pcs=30) # grafo kNN
sc.tl.umap(adata, min_dist=0.3)
sc.tl.tsne(adata, n_pcs=30, perplexity=30)
sc.pl.umap(adata, color=["CD3E", "MS4A1", "LYZ", "pct_counts_mt"])
```

⚠ UMAP necesita el grafo

En Scanpy, `sc.tl.umap` no recalcula vecinos: usa el grafo de `sc.pp.neighbors`. Cambiar `n_neighbors` en el grafo cambia a la vez el mapa y los *clusters* de la sección siguiente. Colorear el mapa por `pct_counts_mt` o `total_counts` es

la forma más rápida de descubrir que un «tipo celular nuevo» es en realidad un grupo de células de mala calidad.

12.3 Clustering y genes marcadores

 Colab · 12.3 · Clustering y genes marcadores

 repositorio del curso

Grupos de amigos en una fiesta

En una fiesta grande nadie conoce a todos, pero cada invitado conoce bien a unas pocas personas. Si dibujamos a cada invitado como un punto y unimos a los que se conocen, sin mirar edades ni profesiones, aparecen solos los grupos: los compañeros de la oficina, la familia, los amigos del colegio. Dentro de cada grupo hay muchas líneas; entre grupos, pocas, casi siempre a través de alguien que pertenece a dos mundos. Agrupar células funciona igual: no definimos de antemano qué es un linfocito T; dejamos que cada célula «conozca» a sus vecinas más parecidas y buscamos comunidades densas en esa red.

12.3.1 El grafo de vecinos más cercanos

Los métodos de agrupamiento clásicos, como k -medias, suponen grupos esféricos de tamaño parecido y requieren fijar su número. Los tipos celulares no cumplen nada de eso: hay poblaciones de miles de células junto a otras de decenas, y formas alargadas por estados continuos. Por eso el enfoque dominante, adoptado por Seurat y Scanpy, trabaja sobre un *grafo*: cada célula se une a sus k vecinas más cercanas en el espacio de PC, y el grafo resultante se divide en comunidades (Hao et al., 2021; Wolf et al., 2018).

Definición 12.2. (Grafo kNN y grafo de vecinos compartidos)

Sea $\mathcal{N}_k(i)$ el conjunto de las k células más cercanas a i (distancia euclídea en las primeras PC). El grafo kNN tiene una arista entre i y j si $j \in \mathcal{N}_k(i)$ o $i \in \mathcal{N}_k(j)$. En el grafo de vecinos compartidos (SNN), cada arista recibe el peso de Jaccard

$$w_{ij} = \frac{|\mathcal{N}_k(i) \cap \mathcal{N}_k(j)|}{|\mathcal{N}_k(i) \cup \mathcal{N}_k(j)|}, \quad (12.22)$$

que es alto cuando dos células comparten vecindario y bajo cuando la arista es un «puente» accidental.

Scanpy usa, en lugar de Jaccard, los pesos difusos de UMAP (ecuación (12.20)), guardados en `adata.obsp["connectivities"]`; la idea es la misma. En la simulación, con $k = 15$ sobre 30 PC, el grafo tiene 38 688 aristas y un grado medio de 25,1 (mayor que 15 porque la simetrización añade las aristas recíprocas). La figura 12.10 lo dibuja sobre las coordenadas de UMAP.

12.3.2 Modularidad

¿Qué es una «buena» partición de un grafo? Newman y Girvan (2004) propusieron medirlo comparando el número de aristas internas de cada comunidad con el que tendría un grafo aleatorio con los mismos grados.

Definición 12.3. (Modularidad con resolución)

Para un grafo con matriz de adyacencia (o de pesos) A , grados $k_i = \sum_j A_{ij}$, $m = \frac{1}{2} \sum_{ij} A_{ij}$ aristas y una asignación de comunidades c_i ,

$$Q_\gamma = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \gamma \frac{k_i k_j}{2m} \right] \delta(c_i, c_j). \quad (12.23)$$

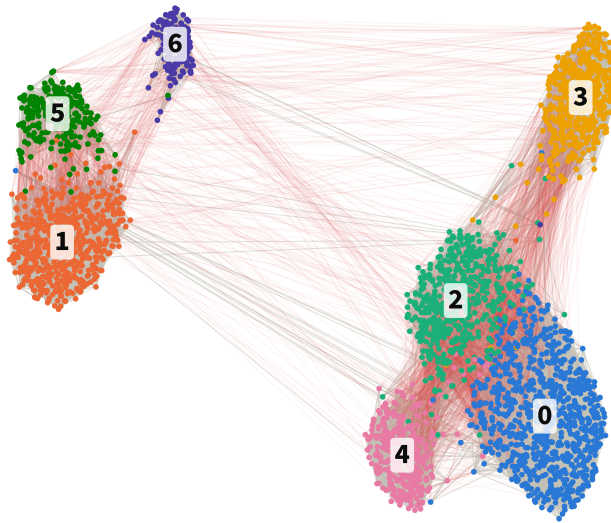


Figura 12.10. Grafo de 15 vecinos más cercanos de las 3085 células, dibujado sobre las coordenadas de UMAP y coloreado por la partición de Louvain con resolución $\gamma = 1$. Las aristas grises unen células del mismo *cluster*; las rojas (tenues) cruzan de un *cluster* a otro. Solo el 10,0 % de las aristas son de este segundo tipo, y se concentran entre poblaciones emparentadas: linfocitos T CD4 (0), T CD8 (2) y NK (4), por un lado; monocitos CD14 (1), FCGR3A (5) y células dendríticas (6), por otro. El algoritmo de agrupamiento solo ve estas aristas, no las coordenadas del dibujo.

DONDE

A_{ij}	Peso de la arista entre i y j (o si no existe).
$k_i k_j / 2m$	Peso esperado entre i y j en un grafo aleatorio que conserva los grados (modelo de configuración).
$\delta(c_i, c_j)$	Vale 1 si i y j están en la misma comunidad.
γ	Parámetro de resolución: multiplica el término nulo; $\gamma = 1$ es la modularidad original.

Q es la fracción de aristas dentro de comunidades menos la fracción esperada por azar. Una partición en una sola comunidad da $Q = 0$; valores por encima de 0,3 suelen indicar estructura clara. El parámetro γ cambia la escala: al aumentarlo, el término nulo pesa más y solo sobreviven comunidades muy densas, de modo que el óptimo tiene más grupos y más pequeños.

12.3.3 El algoritmo de Louvain y su corrección, Leiden

Maximizar Q exactamente es NP-difícil. Blondel et al. (2008) propusieron una heurística voraz de dos fases, llamada de Louvain por la universidad de sus autores, que escala a millones de nodos:

1. **Movimiento local.** Cada nodo empieza en su propia comunidad. Se recorren los nodos y cada uno se mueve a la comunidad vecina que más aumenta Q , si alguna lo aumenta. Se repite hasta que ningún movimiento mejora.
2. **Agregación.** Cada comunidad se contrae en un supernodo, con aristas cuyos pesos suman los de las originales, y se vuelve a la fase 1 sobre el grafo reducido.

La eficiencia proviene de que la ganancia de mover un nodo aislado i a una comunidad C se calcula con información local:

$$\Delta Q = \frac{k_{i,C}}{m} - \gamma \frac{\Sigma_C k_i}{2m^2}, \quad (12.24)$$

DONDE

$k_{i,C}$	Suma de los pesos de las aristas entre i y los nodos de C .
Σ_C	Suma de los grados de los nodos de C .

que se obtiene restando la modularidad de C y de $\{i\}$ por separado de la de $C \cup \{i\}$: el término de aristas crece en $2k_{i,C}/2m$ y el término nulo en $\gamma[(\Sigma_C + k_i)^2 - \Sigma_C^2 - k_i^2]/(2m)^2 = 2\gamma \Sigma_C k_i/(2m)^2$. Solo intervienen las aristas de i y un total por comunidad.

Traag et al. (2019) mostraron que Louvain tiene un defecto grave: al mover un nodo que servía de puente dentro de una comunidad, esta puede quedar internamente *desconectada*, y las fases posteriores nunca lo corrigen. El algoritmo de Leiden añade una fase de *refinamiento* entre el movimiento y la agregación: dentro de cada comunidad, los nodos se reagrupan en subcomunidades bien conectadas, y es sobre esas subcomunidades sobre las que se agrega. Con ello Leiden garantiza comunidades conectadas y, además, converge más rápido. Es el método recomendado en Scanpy y está disponible como opción en Seurat. En nuestra simulación, relativamente sencilla, ninguna de las particiones de Louvain produjo comunidades desconectadas, pero Traag et al. (2019) sí las encontraron al aplicar Louvain a redes reales.

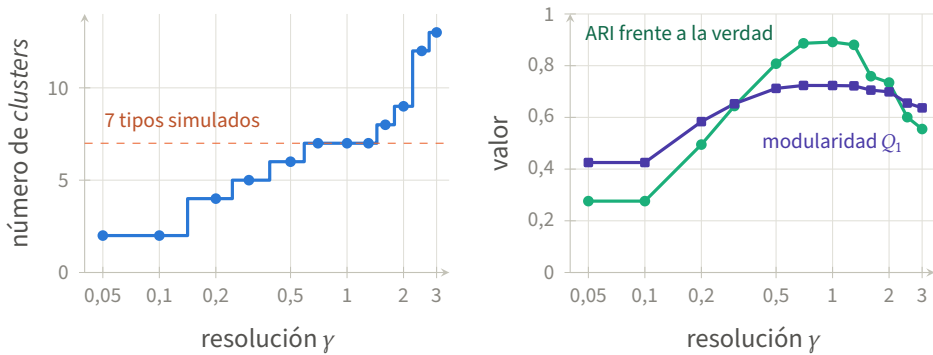


Figura 12.11. Barrido de la resolución de Louvain sobre el grafo kNN de la simulación. **Izquierda:** número de comunidades. Entre $\gamma = 0,7$ y $1,3$ el número se estabiliza en 7, el de tipos simulados: una meseta de este tipo es un indicio (no una prueba) de estructura robusta. **Derecha:** concordancia con los tipos simulados (índice de Rand ajustado, ARI; 1 = partición idéntica, 0 = azar) y modularidad evaluada con $\gamma = 1$. Ambas alcanzan su máximo en la meseta (ARI = 0,892 y $Q = 0,723$ con $\gamma = 1$). Con resoluciones altas los tipos se fragmentan en subgrupos, en particular el estado continuo de las células T CD4.

12.3.4 ¿Cuántos clusters? La resolución

No existe un número «correcto» de grupos. Los tipos celulares forman una jerarquía (linfocitos $\supset$ linfocitos T $\supset$ T CD4 $\supset$ T CD4 vírgenes), y el parámetro γ elige a qué nivel cortarla. La figura 12.11 lo muestra: al barrer γ de 0,05 a 3, el número de grupos pasa de 2 (mieloides y linfoides) a 13.

⚠ Un cluster no es un tipo celular

Un *cluster* es la salida de un algoritmo con un parámetro arbitrario; un tipo celular es una hipótesis biológica. Al subir la resolución, Louvain y Leiden parten con gusto un gradiente continuo (un estado de activación, un ciclo celular) en trozos discretos, que después se «anotan» como subtipos. Antes de bautizar un grupo nuevo, compruebe que tiene marcadores propios que no son simplemente los extremos de un gradiente, que aparece en varias muestras y que no está definido por métricas de calidad ni por genes de estrés de la disociación.

En la simulación, con $\gamma = 1$, las siete comunidades se corresponden una a una con los tipos simulados, con purezas entre el 88,3 % (T CD8, que recibe algunas células T CD4 y NK) y el 98,0 % (células B). Parte de los errores procede de los 135 dobletes que Scrublet no detectó y de células situadas en la frontera entre poblaciones emparentadas, como T CD8 y NK.

12.3.5 Genes marcadores

Para interpretar un grupo buscamos genes cuya expresión lo distinga del resto. La prueba más usada es la de suma de rangos de Wilcoxon (equivalente a la U de Mann-Whitney), no paramétrica, que compara las distribuciones de expresión del grupo contra todas las demás células.

Definición 12.4. (Estadístico U y AUC)

Sean n_1 células del grupo y n_2 del resto, y R_1 la suma de los rangos de las del grupo cuando se ordenan las $N = n_1 + n_2$ expresiones (los empates reciben el rango medio). Entonces

$$U = R_1 - \frac{n_1(n_1 + 1)}{2}, \quad \text{AUC} = \frac{U}{n_1 n_2} = \mathbb{P}(Y_1 > Y_2) + \frac{1}{2}\mathbb{P}(Y_1 = Y_2). \quad (12.25)$$

Bajo la hipótesis nula de distribuciones iguales, $\mathbb{E}[U] = n_1 n_2 / 2$ y

$$\text{Var}[U] = \frac{n_1 n_2}{12} \left[(N + 1) - \frac{\sum_t (t^3 - t)}{N(N - 1)} \right], \quad z = \frac{U - n_1 n_2 / 2}{\sqrt{\text{Var}[U]}}, \quad (12.26)$$

donde la suma recorre los grupos de valores empatados, de tamaño t .

DONDE	
Y_1, Y_2	Expresión de una célula elegida al azar dentro y fuera del grupo.
AUC	Área bajo la curva ROC del gen como clasificador del grupo: 0,5 = no informativo, 1 = separación perfecta.
t	Tamaño de cada grupo de empates; en célula única, el gran grupo de ceros domina la corrección.

La interpretación como AUC es la más útil: dice qué tan bien el gen, por sí solo, identifica a las células del grupo, con independencia de cuántas células haya. El

valor p , en cambio, depende brutalmente del tamaño: con cientos de células por grupo, casi cualquier diferencia es «significativa».

Ejemplo 12.5. (Wilcoxon a mano)

Cuatro células de un grupo tienen expresiones log-normalizadas 0; 1,9; 2,3 y 2,8, y cinco del resto, 0; 0; 0,7; 1,1 y 0. Al ordenar las nueve, los cuatro ceros empatan en los rangos 1 a 4 y reciben el rango medio 2,5; después van $0,7 \rightarrow 5$, $1,1 \rightarrow 6$, $1,9 \rightarrow 7$, $2,3 \rightarrow 8$ y $2,8 \rightarrow 9$. La suma de rangos del grupo es $R_1 = 2,5 + 7 + 8 + 9 = 26,5$, luego $U = 26,5 - 4 \cdot 5/2 = 16,5$ y $AUC = 16,5/20 = 0,825$: en el 82,5 % de los pares (célula del grupo, célula de fuera) la primera expresa más el gen. El resultado coincide con `scipy.stats.mannwhitneyu`.

Aplicada a cada *cluster* de la simulación frente al resto, la prueba recupera los marcadores simulados: para los monocitos CD14, *S100A8* ($AUC = 0,940$), *S100A9* y *LYZ*; para las células B, *MS4A1* ($AUC = 0,942$) y *CD79A*; para las NK, *GNLY* ($AUC = 0,949$); para las dendríticas, *CLEC10A* y *CD1C*. En el grupo de linfocitos T CD4, en cambio, los primeros genes de la lista no son *IL7R* ni *CCR7* sino genes anónimos del estado continuo simulado: la prueba señala lo que *más* distingue al grupo, no lo que un inmunólogo esperaría leer. Tras la corrección de Benjamini-Hochberg (capítulo 11), los valores q de los mejores marcadores son del orden de 10^{-100} a 10^{-300} , una ilustración de lo poco informativos que son en este contexto.

⚠ Doble uso de los datos

Los *clusters* se definieron con la expresión génica, y después se prueba si la expresión génica difiere entre ellos. Por construcción, sí difiere: incluso en datos sin estructura, un algoritmo de agrupamiento partirá las células en grupos, y la prueba de Wilcoxon encontrará «marcadores» muy significativos entre ellos. Los valores p de los marcadores sirven para *ordenar* genes, no como evidencia de que los grupos existen. Para comparar condiciones (tratado frente a control), sume las cuentas de cada tipo celular por muestra (*pseudobulk*) y use los modelos BN del capítulo 11 con las muestras, no las células, como réplicas (Heumos et al., 2023).

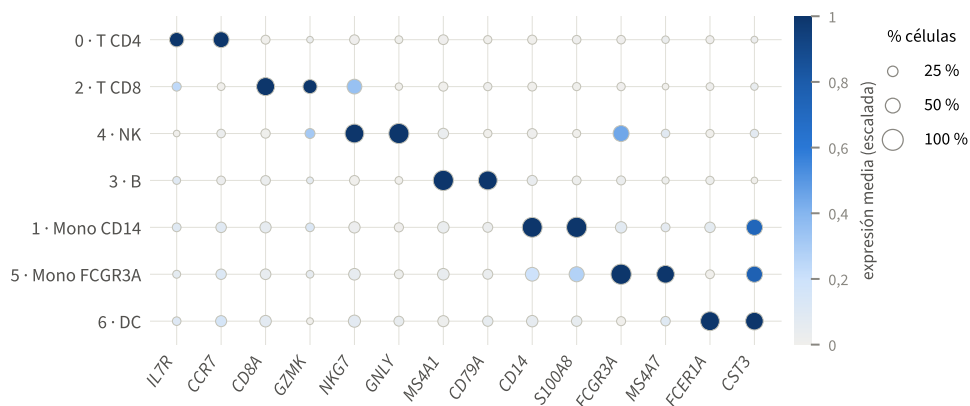


Figura 12.12. Dot plot de 14 marcadores canónicos de células mononucleares de sangre en los siete *clusters* de Louvain ($\gamma = 1$), anotados por el tipo celular mayoritario. El tamaño del punto es el porcentaje de células del grupo con al menos un UMI del gen; el color, la expresión media log-normalizada, escalada entre 0 y 1 para cada gen. La diagonal de bloques permite asignar cada grupo a un tipo: *IL7R/CCR7* (T CD4), *CD8A/GZMK* (T CD8), *NKG7/GNLY* (NK), *MS4A1/CD79A* (B), *CD14/S100A8* (monocitos clásicos), *FCGR3A/MS4A7* (monocitos no clásicos) y *FCER1A/CST3* (dendríticas). Los puntos pequeños y claros fuera de la diagonal son la huella del ARN ambiental y de los dobletes residuales.

12.3.6 Anotación de tipos celulares

Con los marcadores en mano, la anotación clásica es manual: se contrastan los genes de cada grupo con el conocimiento previo. El *dot plot* de la figura 12.12 es la visualización estándar para ello, porque codifica a la vez dos cantidades que en célula única no coinciden: *qué fracción* de las células del grupo detecta el gen (tamaño del punto) y *cuánto* lo expresan en promedio (color).

La anotación manual no escala a atlas de cientos de tipos ni es del todo reproducible. La alternativa es la anotación por referencia: proyectar las células nuevas sobre un atlas ya anotado y transferir las etiquetas de los vecinos. Seurat v4 construyó así un atlas multimodal de sangre, combinando ARN y proteínas de superficie medidas en las mismas células, y lo usa como referencia para anotar conjuntos nuevos (Hao et al., 2021). Cualquiera de los dos caminos requiere juicio: una referencia no puede asignar un tipo que no contiene, y un marcador «de libro» puede no expresarse en otra especie o en otro tejido.

12.3.7 Efectos de lote e integración

Cuando se combinan muestras procesadas en días, canales o laboratorios distintos, aparecen diferencias sistemáticas que no tienen que ver con la biología: son los efectos de lote. En el espacio de PC, las células de un mismo tipo pero de lotes distintos forman grupos separados, y el agrupamiento las divide por lote en lugar de por tipo (figura 12.13a). Los métodos de *integración* buscan una representación en la que el lote se mezcle y el tipo celular se conserve.

Harmony (Korsunsky et al., 2019) trabaja directamente sobre las coordenadas de PC, alternando dos pasos hasta converger:

1. **Agrupamiento difuso con penalización de diversidad.** Cada célula i recibe una pertenencia R_{ki} a cada uno de K grupos, que favorece los grupos cercanos y, a la vez, penaliza los grupos donde el lote de i está sobrerrepresentado:

$$R_{ki} \propto \exp\left(-\frac{\|\hat{z}_i - y_k\|^2}{\sigma}\right) \prod_b \left(\frac{E_{kb} + 1}{O_{kb} + 1}\right)^{\theta \phi_{ib}}. \quad (12.27)$$

2. **Corrección lineal por grupo.** En cada grupo k se ajusta, ponderando por R_{ki} , una regresión de las coordenadas sobre el lote, y se resta a cada célula la mezcla de los efectos de lote de sus grupos: $\hat{z}_i \leftarrow z_i - \sum_k R_{ki} \beta_k^\top \phi_i$.

DONDE

$\hat{z}_i, y_k$	Coordenadas (normalizadas) de la célula i y centroide del grupo k .
O_{kb}, E_{kb}	Número observado y esperado (según la proporción global del lote) de células del lote b en el grupo k .
ϕ_{ib}	Indicador de que la célula i pertenece al lote b .
θ, σ	Fuerza de la penalización de diversidad y suavidad del agrupamiento.
β_k	Efectos de lote estimados dentro del grupo k .

El truco está en que la corrección es *local*: un lote puede desplazar a los monocitos en una dirección y a los linfocitos en otra, y Harmony estima un desplazamiento por grupo. La penalización de diversidad evita la solución trivial en la que cada grupo contiene un solo lote y, por tanto, no hay nada que corregir.

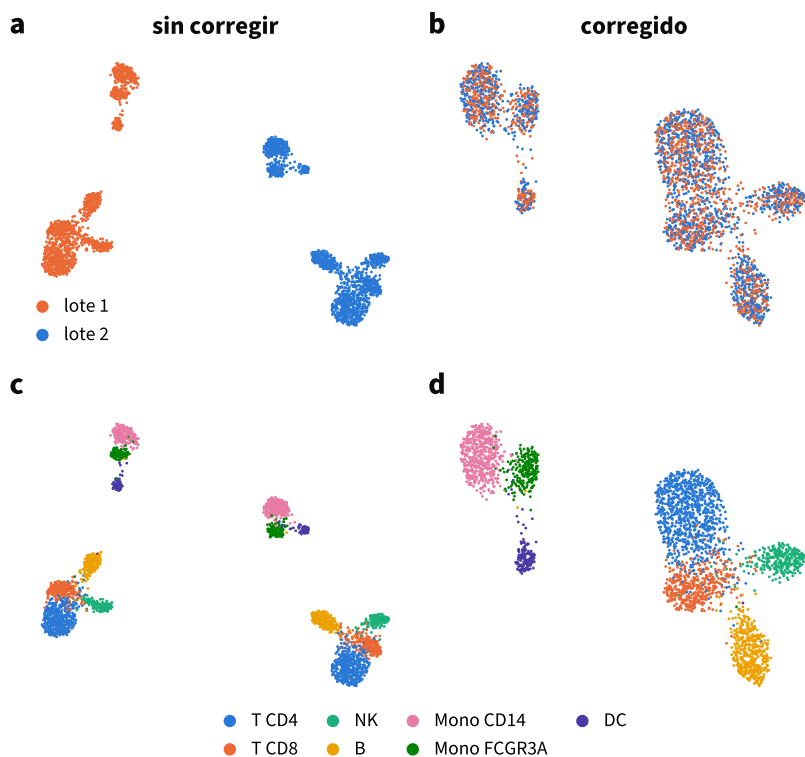


Figura 12.13. Dos lotes simulados de 1500 células cada uno, con los mismos tipos celulares pero con un efecto de lote multiplicativo aleatorio sobre cada gen (desviación estándar 0,45 en escala logarítmica). **(a, c)** UMAP calculado sobre las PC sin corregir, coloreado por lote y por tipo: cada tipo aparece duplicado. **(b, d)** Tras una versión didáctica de Harmony (ecuación (12.27), con $K = 20$, $\theta = 2$ y $\sigma = 0,1$), los lotes se mezclan y los tipos se reúnen. La fracción de vecinos de otro lote pasa de 0,001 a 0,502 (el ideal es 0,5), y el ARI de Louvain frente a los tipos simulados sube de 0,382 a 0,816, mientras que frente al lote cae de 0,360 a 0.

Un método de integración no sabe distinguir un efecto de lote de un efecto biológico que coincide con el lote. Si todas las muestras de pacientes se procesaron en un lote y todos los controles en otro, la integración borraría también la enfermedad. El diseño experimental (repartir condiciones entre lotes) es la única defensa real. Además, las coordenadas corregidas sirven para agrupar y visualizar, pero la expresión diferencial debe hacerse sobre las cuentas originales, incluyendo el lote como covariable.

clustering.py

```
sc.tl.leiden(adata, resolution=1.0, flavor="igraph",
             n_iterations=2, key_added="leiden")
sc.tl.rank_genes_groups(adata, "leiden", method="wilcoxon")
print(sc.get.rank_genes_groups_df(adata, group="0").head(10))
marcadores = ["IL7R", "CCR7", "CD8A", "NKG7", "GNLY", "MS4A1",
              "CD79A", "CD14", "S100A8", "FCGR3A", "FCER1A"]
sc.pl.dotplot(adata, marcadores, groupby="leiden",
              standard_scale="var")
# integración de lotes (requiere el paquete harmonypy)
sc.external.pp.harmony_integrate(adata, key="lote")
sc.pp.neighbors(adata, use_rep="X_pca_harmony")
sc.tl.umap(adata)
```

12.4 Trayectorias y pseudotiempo

 Colab · 12.4 · Trayectorias y pseudotiempo

 repositorio del curso

Una fotografía de un maratón

Suponga que un dron toma una única fotografía aérea de un maratón una hora después de la salida. No hay película: solo una imagen congelada. Aun así, puede reconstruir el recorrido. Los corredores forman una línea continua sobre las calles; los más adelantados marcan el camino hacia la meta; donde la carrera se divide en dos recorridos (el maratón y la media maratón) la línea se bifurca. Si además cada corredor tuviera en la camiseta un velocímetro, sabría hacia dónde se mueve cada uno. Un experimento de célula

única es esa fotografía: las células de un tejido en diferenciación están, en el mismo instante, en todas las etapas del proceso. El pseudotiempo ordena a los corredores por la distancia recorrida; la velocidad de ARN es el velocímetro.

12.4.1 Del agrupamiento a los procesos continuos

No toda la variación entre células es discreta. En la hematopoyesis, la espermatogénesis o el desarrollo neural, las células atraviesan un continuo de estados entre un progenitor y uno o varios destinos. Forzar ese continuo en *clusters* lo trocea arbitrariamente. La *inferencia de trayectorias* busca, en cambio, una estructura de dimensión uno (una línea, un árbol, a veces un ciclo) que recorra los datos, y un *pseudotiempo*: la posición de cada célula a lo largo de esa estructura.

Definición 12.5. (Pseudotiempo)

Dada una célula raíz r elegida con información biológica (p. ej., la que más expresa marcadores de progenitor), el pseudotiempo τ_c de la célula c es una medida de distancia de r a c a lo largo de la variedad de los datos, normalizada a $[0, 1]$. Es una coordenada de *progreso transcripcional*, no un tiempo físico: dos células con el mismo pseudotiempo han recorrido la misma distancia en expresión, no necesariamente el mismo número de horas.

El primer método de amplio uso fue Monocle: Trapnell et al. (2014) redujeron la dimensión con análisis de componentes independientes, construyeron un árbol de expansión mínima entre las células y ordenaron las células por su posición en el camino más largo del árbol, lo que les permitió seguir la diferenciación de mioblastos humanos y encontrar reguladores que cambiaban a lo largo del proceso. Desde entonces se han publicado decenas de métodos. La comparación sistemática de Saelens et al. (2019), con 45 métodos evaluados sobre conjuntos reales y sintéticos, concluyó que ningún método es el mejor en todos los casos: el rendimiento depende sobre todo de la topología de la trayectoria (lineal, bifurcada, en árbol, cíclica o desconectada), y recomendaron elegir el método según la topología esperada y comparar varios.

12.4.2 Mapas de difusión

Las distancias euclídeas en el espacio de PC funcionan bien a escala local, pero una trayectoria curva puede acercar en línea recta células que en el proceso están

lejos (piense en una herradura: los dos extremos están cerca por el aire y lejos por el camino). Los mapas de difusión sustituyen la distancia en línea recta por la distancia a lo largo de los datos: una partícula que salta al azar entre células vecinas tarda poco en recorrer una región densa y mucho en atravesar la herradura.

Para cada par de células vecinas se define un núcleo gaussiano con anchos locales, como en t-SNE, y a partir de él una matriz de transición de un paseo aleatorio (Haghverdi et al., 2016):

$$K_{ij} = \sqrt{\frac{2\sigma_i\sigma_j}{\sigma_i^2 + \sigma_j^2}} \exp\left(-\frac{\|x_i - x_j\|^2}{\sigma_i^2 + \sigma_j^2}\right), \quad (12.28)$$

$$T = D^{-1}K, \quad D = \text{diag}\left(\sum_j K_{ij}\right).$$

DONDE

σ_i	Ancho local de la célula i (en la implementación, la distancia a su vecino número $k/2$).
K_{ij}	Afinidad entre i y j ; el prefactor garantiza que K siga siendo un núcleo válido con anchos distintos.
T_{ij}	Probabilidad de que el paseo aleatorio salte de i a j en un paso; cada fila suma 1.

En la práctica, antes de normalizar por filas se divide K por las densidades de ambos extremos, para que las regiones muy pobladas no atraigan al paseo; así, los componentes reflejan la geometría de los datos y no su densidad. T tiene valores propios $1 = \lambda_0 > \lambda_1 \geq \lambda_2 \geq \dots$ con vectores propios por la derecha ψ_0 (constante), $\psi_1, \psi_2, \dots$. Las coordenadas $(\psi_1(c), \psi_2(c), \dots)$ son los *componentes de difusión* (DC) de la célula c : los modos de variación más lentos del paseo aleatorio, que en una trayectoria ramificada se alinean con las ramas.

Teorema 12.3. (*Distancia de pseudotiempo de difusión (DPT)*)

Sumando las transiciones de todas las longitudes y eliminando el estado estacionario, se define $M = \sum_{t=1}^{\infty} (T - \psi_0 \phi_0^{\top})^t$, donde ϕ_0 es la distribución estacionaria. Por la descomposición espectral de T ,

$$M = \sum_{l \geq 1} \frac{\lambda_l}{1 - \lambda_l} \psi_l \phi_l^{\top}, \quad \text{dpt}(x, y)^2 = \sum_{l \geq 1} \left(\frac{\lambda_l}{1 - \lambda_l} \right)^2 (\psi_l(x) - \psi_l(y))^2, \quad (12.29)$$

y el pseudotiempo de la célula c es $\tau_c = \text{dpt}(r, c)$ para la raíz r (Haghverdi et al., 2016).

La serie geométrica $\sum_{t \geq 1} \lambda^t = \lambda/(1 - \lambda)$ explica los pesos: los componentes con λ_l cercano a 1, que corresponden a los procesos lentos y de gran escala, reciben un peso enorme, mientras que los de λ_l pequeño (ruido local) se desvanecen. La DPT es, por tanto, una distancia euclídea en el espacio de difusión con cada eje reescalado según su persistencia. Tiene dos ventajas sobre los árboles de expansión mínima: al promediar sobre todos los caminos es robusta frente al ruido, y las ramas se detectan como puntos donde el orden de la DPT medida desde distintos extremos deja de ser coherente.

Ejemplo 12.6. (*Pseudotiempo en una bifurcación simulada*)

Simulamos 2000 células de una diferenciación con un tronco progenitor (tiempo simulado $t < 0,3$, genes como *CD34* y *GATA2* en descenso) que se bifurca en una rama eritroide (*GATA1*, *KLF1*, *HBB*) y otra mieloide (*CEBPA*, *MPO*, *LYZ*), con 600 genes y cuentas BN. Sobre 20 PC, con $k = 30$ vecinos, los primeros valores propios del mapa de difusión son 0,9943, 0,9928 y 0,9781, lo que da pesos $\lambda/(1 - \lambda)$ de 173,7, 138,3 y 44,7: los dos primeros componentes dominan, uno por rama. Tomando como raíz, entre las 20 células con mayor puntuación de genes progenitores, la más extrema en DC1, la DPT correlaciona con el tiempo simulado con $\rho = 0,978$ (Spearman); dentro de cada rama, 0,983 y 0,987, y en el tronco, donde el progreso es más sutil, 0,891.

12.4.3 PAGA: el mapa de metro de las células

Una trayectoria detallada célula a célula es difícil de leer cuando hay muchas ramas, y los métodos que ajustan un árbol imponen esa topología aunque los datos

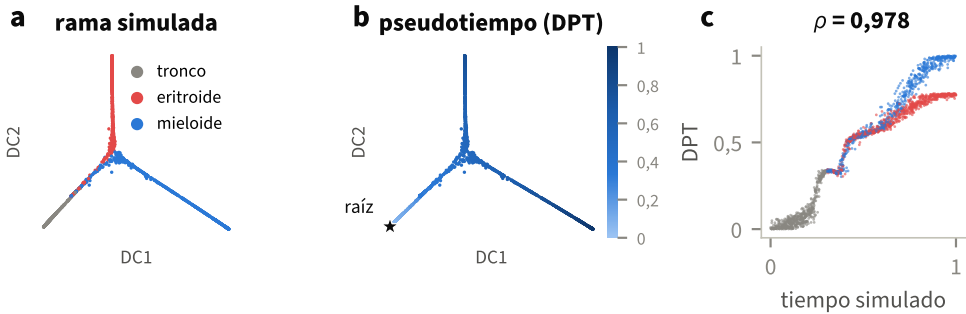


Figura 12.14. Diferenciación simulada con una bifurcación, representada en los dos primeros componentes de difusión. **(a)** Rama a la que pertenece cada célula en la simulación: el tronco progenitor (gris) se abre en una rama eritroide (rojo) y una mieloide (azul); el mapa de difusión muestra la topología en «Y» sin ningún conocimiento previo. **(b)** Pseudotiempo de difusión desde la raíz (estrella), elegida entre las células con más expresión de genes progenitores. **(c)** DPT frente al tiempo simulado ($\rho = 0,978$). Obsérvense las dos mesetas: la DPT avanza deprisa donde muchos genes cambian a la vez (la bifurcación) y despacio donde la expresión cambia poco, y alcanza valores máximos distintos en cada rama. El pseudotiempo mide cambio transcripcional, no reloj.

no la tengan. La abstracción de grafos basada en particiones (PAGA) de Wolf et al. (2019) toma un camino intermedio: agrupa las células (con Louvain o Leiden) y construye un grafo más grueso cuyos nodos son los grupos y cuyas aristas pesan según la conectividad entre ellos en el grafo kNN. La conectividad compara el número de aristas observadas entre dos grupos con el esperado si las aristas se repartieran al azar; en nuestra versión didáctica,

$$c_{ab} = \min \left\{ 1, \frac{e_{ab}}{k_a k_b / 2m} \right\}, \quad (12.30)$$

DONDE

e_{ab}	Número de aristas del grafo kNN entre células del grupo a y del grupo b .
k_a	Suma de los grados de las células del grupo a .
m	Número total de aristas del grafo.

Las aristas con conectividad baja se descartan, y el grafo resultante es un «mapa de metro» que muestra qué grupos se tocan. PAGA no presupone que la topología

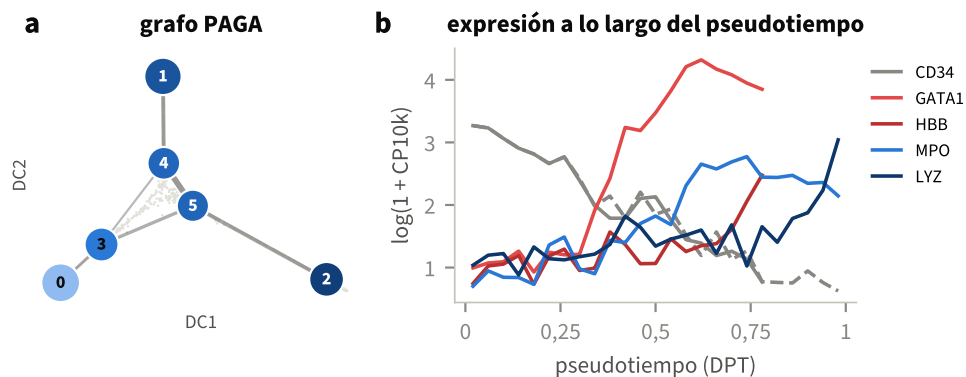


Figura 12.15. (a) Grafo PAGA de la simulación ramificada. Los seis nodos son los grupos de Louvain, situados en la mediana de sus células en el espacio de difusión y coloreados por su pseudotiempo mediano; el grosor de cada arista es proporcional a la conectividad (ecuación (12.30)); se muestran las mayores que 0,05). El grupo 0, que contiene la raíz, se une al 3, y éste a los grupos 4 y 5, que forman un triángulo en la zona de la bifurcación (la arista 4–5, con conectividad 0,40, es la más fuerte del grafo) y continúan hacia los extremos 1 y 2. (b) Expresión media de cinco genes en intervalos de pseudotiempo. *CD34* desciende a lo largo de ambas ramas (línea continua: tronco y rama eritroide; discontinua: tronco y rama mieloide); *GATA1* sube antes que *HBB* en la rama eritroide, y *MPO* antes que *LYZ* en la mieloide, reproduciendo el orden temporal simulado.

sea un árbol: si los datos contienen un ciclo o componentes desconectados, el grafo lo refleja. Además, las posiciones de los nodos de PAGA sirven para inicializar UMAP, lo que hace que el mapa respete la topología global.

Una vez ordenadas las células, el análisis más frecuente es buscar genes cuya expresión cambia con el pseudotiempo, ajustando curvas suaves (*splines* o modelos aditivos generalizados) a la expresión frente a τ dentro de cada rama. La figura 12.15b muestra el resultado esperable: los reguladores tempranos de cada linaje (*GATA1*, *MPO*) se activan antes que los genes efectores (*HBB*, *LYZ*). En datos reales, ese orden es una hipótesis sobre causalidad que hay que validar experimentalmente.

12.4.4 Velocidad de ARN

El pseudotiempo ordena, pero no orienta: la DPT no sabe si el proceso va del tronco a las ramas o al revés, y por eso necesita una raíz elegida por el usuario. La Manno et al. (2018) observaron que la dirección está escrita en los propios datos. Un protocolo de scRNA-seq con cebadores poli(dT) captura también una fracción de

transcritos *no maduros*, que aún conservan intrones. Si un gen acaba de activarse, sus transcritos no maduros abundan en relación con los maduros; si se está apagando, escasean. El cociente entre ambos es un velocímetro.

Teorema 12.4. (*Modelo cinético de la transcripción*)

Sean $u(t)$ y $s(t)$ las abundancias de ARN no maduro (*unspliced*) y maduro (*spliced*) de un gen. Con tasa de transcripción α , de maduración β y de degradación γ ,

$$\frac{du}{dt} = \alpha - \beta u, \quad \frac{ds}{dt} = \beta u - \gamma s. \quad (12.31)$$

Con α constante y condiciones iniciales u_0, s_0 , la solución es

$$\begin{aligned} u(t) &= u_0 e^{-\beta t} + \frac{\alpha}{\beta} (1 - e^{-\beta t}), \\ s(t) &= s_0 e^{-\gamma t} + \frac{\alpha}{\gamma} (1 - e^{-\gamma t}) + \frac{\alpha - \beta u_0}{\gamma - \beta} (e^{-\gamma t} - e^{-\beta t}), \end{aligned} \quad (12.32)$$

y el estado estacionario es $u^* = \alpha/\beta$, $s^* = \alpha/\gamma$, que satisface $u^* = (\gamma/\beta) s^*$.

DONDE

α	Tasa de transcripción: alta mientras el gen está activo, cero cuando se reprime.
β	Tasa de corte y empalme (<i>splicing</i>): convierte u en s .
γ	Tasa de degradación del ARNm maduro.
$v = ds/dt$	Velocidad de ARN del gen: positiva si el ARN maduro está aumentando.

La primera ecuación se resuelve directamente como lineal de primer orden; la segunda, sustituyendo $u(t)$ y usando el factor integrante $e^{\gamma t}$. La idea clave es geométrica (figura 12.16): en el plano (s, u) , las células en estado estacionario caen sobre la recta $u = \gamma' s$, con $\gamma' = \gamma/\beta$. Una célula por *encima* de la recta tiene más ARN no maduro del que corresponde a su ARN maduro: el gen se está induciendo y $v = \beta u - \gamma s > 0$. Por *debajo*, el gen se está reprimiendo y $v < 0$.

Estimación en el modelo estacionario. La Manno et al. (2018) supusieron que las células con valores extremos de u y s (los percentiles superiores e inferiores) están

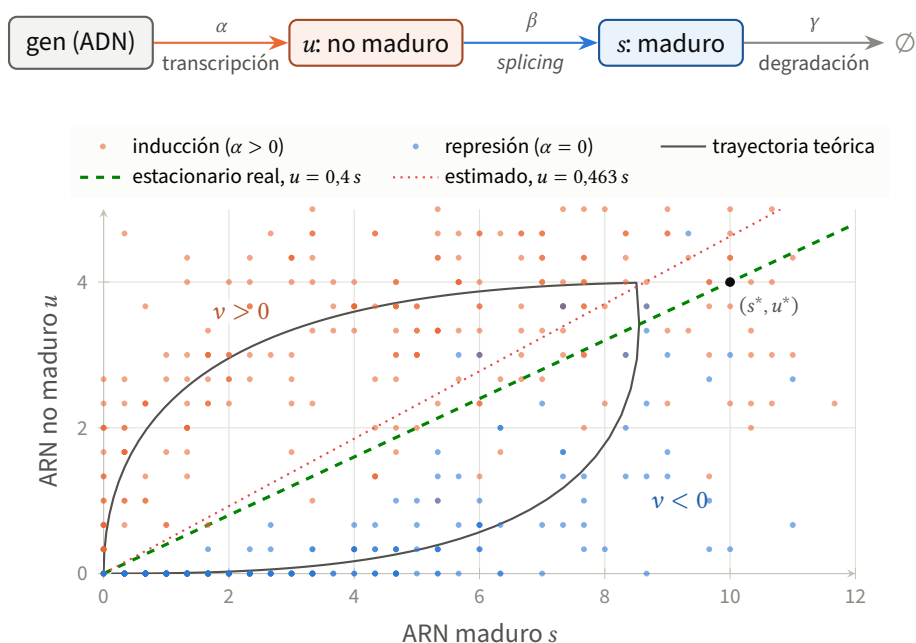


Figura 12.16. Arriba: el modelo cinético de la ecuación (12.31). Abajo: retrato de fase de un gen simulado con $\alpha = 4$, $\beta = 1$ y $\gamma = 0,4$. Durante la inducción (naranja) las células recorren la rama superior del ciclo hacia el estado estacionario activo $(s^*, u^*) = (10, 4)$; al reprimirse el gen (azul) vuelven al origen por la rama inferior. La recta discontinua verde es el estado estacionario real, $u = (\gamma/\beta)s$; la punteada roja, la estimación por percentiles extremos de la ecuación (12.33), $\hat{\gamma}' = 0,463$, algo sesgada porque no todas las células extremas están en equilibrio. Aun así, el signo de $u - \hat{\gamma}'s$ clasifica correctamente la fase del 86,3 % de las células.

cerca del estado estacionario, activo o inactivo, y estimaron γ' por regresión por el origen sobre ellas:

$$\hat{\gamma}' = \frac{\sum_{c \in \mathcal{E}} u_c s_c}{\sum_{c \in \mathcal{E}} s_c^2}, \quad \hat{\gamma}_c \propto u_c - \hat{\gamma}' s_c, \quad (12.33)$$

con $\mathcal{E}$ el conjunto de células extremas. La velocidad de todos los genes forma un vector por célula que, proyectado sobre el *embedding*, indica hacia dónde se dirige cada célula en las próximas horas.

El modelo dinámico. El supuesto de que existen células en equilibrio falla cuando el proceso es transitorio: si el experimento capta la inducción pero no la meseta, el

ajuste por extremos subestima o sobreestima γ' y la velocidad cambia de signo. Bergen et al. (2020), en scVelo, resolvieron la cinética completa: para cada gen estiman α , β , γ , el tiempo de cambio entre inducción y represión y un tiempo latente para cada célula, maximizando la verosimilitud con un algoritmo de esperanza-maximización sobre las soluciones de la ecuación (12.32). El tiempo latente compartido entre genes es además un pseudotiempo con dirección, que no requiere elegir raíz.

⚠ Cuándo desconfiar de las flechas

Las flechas de velocidad son atractivas y engañosas. El modelo supone tasas constantes por gen y un único programa de inducción-represión; los genes con varias fases de expresión, o con tasas que cambian entre linajes, lo violan. Los transcritos no maduros son pocos y ruidosos, de modo que la velocidad se suaviza promediando entre vecinos, y las flechas proyectadas sobre un UMAP heredan todas las distorsiones de la sección 12.2. Compruebe que los genes que dirigen la velocidad tienen retratos de fase con la forma de la figura 12.16 y que la dirección inferida concuerda con el conocimiento biológico antes de interpretarla.

trayectorias.py

```
import numpy as np
import scvelo as scv

sc.tl.diffmap(adata, n_comps=15)
cd34 = adata[:, "CD34"].X
cd34 = cd34.toarray().ravel() if hasattr(cd34, "toarray") else cd34
adata.uns["iroot"] = int(np.argmax(cd34)) # célula raíz
sc.tl.dpt(adata) # -> obs["dpt_pseudotime"]
sc.tl.paga(adata, groups="leiden")
sc.pl.paga(adata, threshold=0.05)
sc.tl.umap(adata, init_pos="paga")
# velocidad de ARN: requiere capas "spliced" y "unspliced"
scv.pp.filter_and_normalize(adata, min_shared_counts=20,
                           n_top_genes=2000)
scv.pp.moments(adata, n_pcs=30, n_neighbors=30)
scv.tl.recover_dynamics(adata)
scv.tl.velocity(adata, mode="dynamical")
scv.tl.velocity_graph(adata)
scv.pl.velocity_embedding_stream(adata, basis="umap")
```

Las capas `spliced` y `unspliced` no las produce el cuantificador estándar: hay que generarlas con herramientas que cuentan por separado las lecturas exónicas e intrónicas ([velocyto](#), STARsolo con la opción `Velocyto`, o [alevin-fry](#)). Las guías de buenas prácticas recogen las opciones de cada paso y cómo encadenarlas (Heumos et al., 2023).

Más allá del ARN

El flujo de este capítulo es el núcleo de una familia creciente de técnicas de célula única: accesibilidad de cromatina (scATAC-seq), proteínas de superficie con anticuerpos marcados con códigos de barras (CITE-seq), repertorios inmunitarios y transcriptómica espacial. Las ideas centrales (matrices dispersas de cuentas, normalización, grafos de vecinos, integración) se trasladan casi sin cambios; lo que cambia es el modelo de ruido de cada modalidad. Seurat v4 integra ARN y proteínas en un único grafo de vecinos ponderado por la fiabilidad de cada modalidad en cada célula (Hao et al., 2021).

En resumen

- Las plataformas de gotas marcan cada molécula con un código de barras celular y un UMI; la carga de células sigue una Poisson, de modo que la tasa de dobletes crece como $\lambda/2$. Contar UMI distintos elimina los duplicados de PCR y produce una matriz dispersa (CSR) de cuentas.
- El control de calidad combina genes detectados, UMI totales y porcentaje mitocondrial con umbrales adaptativos (MAD); los dobletes requieren métodos específicos como Scrublet, y el ARN ambiental, métodos como SoupX.
- Las cuentas de UMI siguen aproximadamente una binomial negativa sin inflación de ceros. La normalización por tamaño de librería más $\log(1+x)$ estabiliza la varianza de los genes muy expresados; los residuos de Pearson de `sctransform` y los factores por agrupamiento de `scran` corrigen sus limitaciones. Los genes altamente variables concentran la señal.
- El PCA es el problema de valores propios de la matriz de covarianzas y da la mejor compresión lineal; se usan 30–50 PC para todo el análisis. t-SNE (KL con cola de Student) y UMAP (entropía cruzada sobre un grafo difuso) conservan vecindarios, no distancias: sirven para mirar, no para medir.
- Los *clusters* se obtienen maximizando la modularidad de un grafo kNN; Leiden corrige las comunidades desconectadas de Louvain, y la resolución fija el nivel de la jerarquía. Los marcadores se ordenan por Wilcoxon/AUC, cuyos valores

p están inflados por el doble uso de los datos. Harmony integra lotes con correcciones lineales locales.

- El pseudotiempo ordena células a lo largo de procesos continuos; la DPT lo define como una distancia en el espacio de difusión con pesos $\lambda/(1-\lambda)$, y PAGA resume la topología entre grupos. La velocidad de ARN usa la proporción entre ARN no maduro y maduro para inferir la dirección del cambio.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 12.1** ●●● Usando la ecuación (12.2), calcule qué valor de λ produce un 5 % de dobles entre las gotas ocupadas. Si cada gota tiene 1 nL y se generan 80 000 gotas, ¿cuántas células se recuperan y cuántas se cargaron? Compare con la aproximación $\lambda/2$.
- 12.2** ●●● Escriba la representación CSR de la traspuesta de la matriz del ejemplo 12.1 (genes $\times$ células). ¿Por qué se llama CSC (*compressed sparse column*) a la misma estructura leída por columnas, y qué operaciones son más rápidas en cada formato?
- 12.3** ●●● Un investigador de tejido cardíaco aplica el umbral de 5 % mitocondrial de un tutorial de sangre y pierde el 60 % de las células. Proponga, con base en la ecuación (12.6), un procedimiento para fijar umbrales por muestra y explique cómo comprobaría que no está eliminando cardiomiocitos sanos.
- 12.4** ●●● Demuestre la ecuación (12.10) desarrollando $\log(1 + X)$ en serie de Taylor alrededor de μ . Para una BN con $\theta = 5$, calcule la varianza aproximada de $\log(1 + X)$ para $\mu = 0, 1, 10$ y 100 y compruébela por simulación con `numpy.random.negative_binomial`. ¿A partir de qué media se estabiliza?
- 12.5** ●●● Demuestre que la segunda componente principal es el vector propio del segundo mayor valor propio de S , añadiendo a la ecuación (12.14) la restricción $w^\top w_1 = 0$ y usando dos multiplicadores de Lagrange.
- 12.6** ●●● Implemente la bisección que encuentra σ_i en la ecuación (12.16) para una perplejidad dada. Verifique con los datos de `figuras/cap12/generar.py`

que la perplejidad resultante coincide con la pedida y explique por qué la entropía es monótona en σ_i .

- 12.7** ●●● Construya a mano un grafo de 6 nodos formado por dos triángulos unidos por una arista. Calcule la modularidad (ecuación (12.23), $\gamma = 1$) de la partición natural en dos triángulos y de la partición en una sola comunidad. ¿Para qué valores de γ deja de ser óptima la partición en dos?
- 12.8** ●●● Con `scipy.stats.mannwhitneyu`, compare un gen expresado en el 30 % de las células de un grupo de 50 frente al 25 % de un resto de 5000. Repita con grupos de 5000 frente a 50 000. ¿Cómo cambian el valor p y el AUC? ¿Qué le dice esto sobre la forma de ordenar marcadores?
- 12.9** ●●● Resuelva la ecuación (12.31) con $u_0 = s_0 = 0$ y α constante, y verifique que la solución es la ecuación (12.32). Demuestre que, en el plano (s, u) , la trayectoria de inducción queda siempre por encima de la recta $u = (\gamma/\beta)s$ cuando $\beta > \gamma$.
- 12.10** ●●● Genere datos sin estructura (cuentas Poisson con la misma media para todas las células y genes), aplique el flujo completo (normalización, PCA, grafo kNN, Leiden con $\gamma = 1$) y cuente cuántos *clusters* y cuántos «marcadores» con $q < 0,05$ obtiene. Relacione el resultado con la advertencia sobre el doble uso de los datos.

Referencias del capítulo

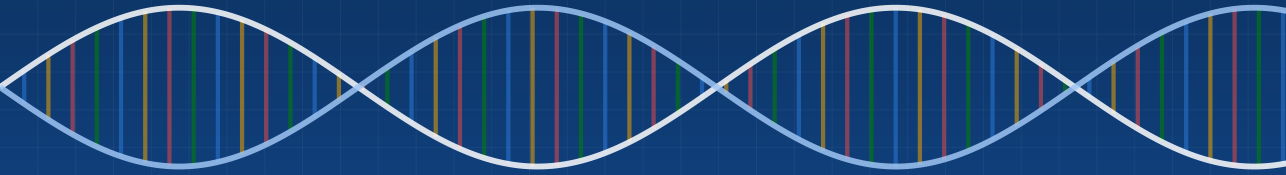
- Becht, E., McInnes, L., Healy, J., Dutertre, C.-A., Kwok, I. W. H., Ng, L. G., Ginhoux, F., & Newell, E. W. (2019). Dimensionality reduction for visualizing single-cell data using UMAP. *Nature Biotechnology*, 37(1), 38-44. <https://doi.org/10.1038/nbt.4314>
- Bergen, V., Lange, M., Peidli, S., Wolf, F. A., & Theis, F. J. (2020). Generalizing RNA velocity to transient cell states through dynamical modeling. *Nature Biotechnology*, 38(12), 1408-1414. <https://doi.org/10.1038/s41587-020-0591-3>
- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), P10008. <https://doi.org/10.1088/1742-5468/2008/10/P10008>
- Chari, T., & Pachter, L. (2023). The specious art of single-cell genomics. *PLOS Computational Biology*, 19(8), e1011288. <https://doi.org/10.1371/journal.pcbi.1011288>

- Hafemeister, C., & Satija, R. (2019). Normalization and variance stabilization of single-cell RNA-seq data using regularized negative binomial regression. *Genome Biology*, 20(1), 296. <https://doi.org/10.1186/s13059-019-1874-1>
- Haghverdi, L., Büttner, M., Wolf, F. A., Buettner, F., & Theis, F. J. (2016). Diffusion pseudotime robustly reconstructs lineage branching. *Nature Methods*, 13(10), 845-848. <https://doi.org/10.1038/nmeth.3971>
- Hao, Y., Hao, S., Andersen-Nissen, E., Mauck, W. M., Zheng, S., Butler, A., Lee, M. J., Wilk, A. J., Darby, C., Zager, M., Hoffman, P., Stoeckius, M., et al. (2021). Integrated analysis of multimodal single-cell data. *Cell*, 184(13), 3573-3587.e29. <https://doi.org/10.1016/j.cell.2021.04.048>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2.^a ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
- Heumos, L., Schaar, A. C., Lance, C., Litinetskaya, A., Drost, F., Zappia, L., Lücken, M. D., Strobl, D. C., Henao, J., Curion, F., et al. (2023). Best practices for single-cell analysis across modalities. *Nature Reviews Genetics*, 24(8), 550-572. <https://doi.org/10.1038/s41576-023-00586-w>
- Kivioja, T., Vähärautio, A., Karlsson, K., Bonke, M., Enge, M., Linnarsson, S., & Taipale, J. (2012). Counting absolute numbers of molecules using unique molecular identifiers. *Nature Methods*, 9(1), 72-74. <https://doi.org/10.1038/nmeth.1778>
- Klein, A. M., Mazutis, L., Akartuna, I., Tallapragada, N., Veres, A., Li, V., Peshkin, L., Weitz, D. A., & Kirschner, M. W. (2015). Droplet barcoding for single-cell transcriptomics applied to embryonic stem cells. *Cell*, 161(5), 1187-1201. <https://doi.org/10.1016/j.cell.2015.04.044>
- Kobak, D., & Berens, P. (2019). The art of using t-SNE for single-cell transcriptomics. *Nature Communications*, 10(1), 5416. <https://doi.org/10.1038/s41467-019-13056-x>
- Korsunsky, I., Millard, N., Fan, J., Slowikowski, K., Zhang, F., Wei, K., Baglaenko, Y., Brenner, M., Loh, P.-r., & Raychaudhuri, S. (2019). Fast, sensitive and accurate integration of single-cell data with Harmony. *Nature Methods*, 16(12), 1289-1296. <https://doi.org/10.1038/s41592-019-0619-0>
- La Manno, G., Soldatov, R., Zeisel, A., Braun, E., Hochgerner, H., Petukhov, V., Lidschreiber, K., Kastrioti, M. E., Lönnerberg, P., Furlan, A., Fan, J., Borm, L. E., et al. (2018). RNA velocity of single cells. *Nature*, 560(7719), 494-498. <https://doi.org/10.1038/s41586-018-0414-6>
- Linderman, G. C., Rachh, M., Hoskins, J. G., Steinerberger, S., & Kluger, Y. (2019). Fast interpolation-based t-SNE for improved visualization of single-cell RNA-seq data. *Nature Methods*, 16(3), 243-245. <https://doi.org/10.1038/s41592-018-0308-4>
- Luecken, M. D., & Theis, F. J. (2019). Current best practices in single-cell RNA-seq analysis: A tutorial. *Molecular Systems Biology*, 15(6), e8746. <https://doi.org/10.15252/msb.20188746>
- Lun, A. T. L., Bach, K., & Marionni, J. C. (2016). Pooling across cells to normalize single-cell RNA sequencing data with many zero counts. *Genome Biology*, 17(1), 75. <https://doi.org/10.1186/s13059-016-0947-7>

- Macosko, E. Z., Basu, A., Satija, R., Nemesh, J., Shekhar, K., Goldman, M., Tirosh, I., Bialas, A. R., Kamitaki, N., Martersteck, E. M., Trombetta, J. J., Weitz, D. A., Sanes, J. R., Shalek, A. K., Regev, A., & McCarroll, S. A. (2015). Highly parallel genome-wide expression profiling of individual cells using nanoliter droplets. *Cell*, 161(5), 1202-1214. <https://doi.org/10.1016/j.cell.2015.05.002>
- McInnes, L., Healy, J., & Melville, J. (2018). *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. arXiv: 1802.03426 [stat.ML]. <https://doi.org/10.48550/arXiv.1802.03426>
- Newman, M. E. J., & Girvan, M. (2004). Finding and evaluating community structure in networks. *Physical Review E*, 69(2), 026113. <https://doi.org/10.1103/PhysRevE.69.026113>
- Saelens, W., Cannoodt, R., Todorov, H., & Saeys, Y. (2019). A comparison of single-cell trajectory inference methods. *Nature Biotechnology*, 37(5), 547-554. <https://doi.org/10.1038/s41587-019-0071-9>
- Svensson, V. (2020). Droplet scRNA-seq is not zero-inflated. *Nature Biotechnology*, 38(2), 147-150. <https://doi.org/10.1038/s41587-019-0379-5>
- Tang, F., Barbacioru, C., Wang, Y., Nordman, E., Lee, C., Xu, N., Wang, X., Bodeau, J., Tuch, B. B., Siddiqui, A., Lao, K., & Surani, M. A. (2009). mRNA-Seq whole-transcriptome analysis of a single cell. *Nature Methods*, 6(5), 377-382. <https://doi.org/10.1038/nmeth.1315>
- Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports*, 9(1), 5233. <https://doi.org/10.1038/s41598-019-41695-z>
- Trapnell, C., Cacchiarelli, D., Grimsby, J., Pokharel, P., Li, S., Morse, M., Lennon, N. J., Livak, K. J., Mikkelsen, T. S., & Rinn, J. L. (2014). The dynamics and regulators of cell fate decisions are revealed by pseudotemporal ordering of single cells. *Nature Biotechnology*, 32(4), 381-386. <https://doi.org/10.1038/nbt.2859>
- Wolf, F. A., Angerer, P., & Theis, F. J. (2018). SCANPY: Large-scale single-cell gene expression data analysis. *Genome Biology*, 19(1), 15. <https://doi.org/10.1186/s13059-017-1382-0>
- Wolf, F. A., Hamey, F. K., Plass, M., Solana, J., Dahlin, J. S., Göttgens, B., Rajewsky, N., Simon, L., & Theis, F. J. (2019). PAGA: Graph abstraction reconciles clustering with trajectory inference through a topology preserving map of single cells. *Genome Biology*, 20(1), 59. <https://doi.org/10.1186/s13059-019-1663-x>
- Wolock, S. L., Lopez, R., & Klein, A. M. (2019). Scrublet: Computational identification of cell doublets in single-cell transcriptomic data. *Cell Systems*, 8(4), 281-291.e9. <https://doi.org/10.1016/j.cels.2018.11.005>
- Young, M. D., & Behjati, S. (2020). SoupX removes ambient RNA contamination from droplet-based single-cell RNA sequencing data. *GigaScience*, 9(12), g1aa151. <https://doi.org/10.1093/gigascience/g1aa151>

Zheng, G. X. Y., Terry, J. M., Belgrader, P., Ryvkin, P., Bent, Z. W., Wilson, R., Ziraldo, S. B., Wheeler, T. D., McDermott, G. P., Zhu, J., Gregory, M. T., Shuga, J., et al. (2017). Massively parallel digital transcriptional profiling of single cells. *Nature Communications*, 8(1), 14049. <https://doi.org/10.1038/ncomms14049>

Epigenómica y regulación



La naturaleza combinatoria de las modificaciones de los extremos amino de las histonas revela así un “código de histonas” que amplía considerablemente el potencial informativo del código genético.

— Jenuwein y Allis (2001)

● Al terminar este capítulo usted podrá

- Describir la organización de la cromatina (nucleosomas, marcas de histonas, estados de cromatina) y los grandes atlas epigenómicos (ENCODE, Roadmap Epigenomics).
- Explicar la geometría de las lecturas de ChIP-seq, estimar el tamaño de fragmento por correlación cruzada y llamar picos con el modelo de Poisson local de MACS; evaluar un experimento con FRiP, NSC/RSC, IDR y la lista negra.
- Interpretar la distribución de tamaños de fragmento y el enriquecimiento en TSS de un experimento de ATAC-seq.
- Deducir la química de la conversión por bisulfito, alinear lecturas convertidas y cuantificar la metilación con valores β y M ; detectar regiones diferencialmente metiladas con modelos beta-binomiales.

- Formular el descubrimiento de motivos como un problema de datos incompletos y resolverlo con el algoritmo EM (MEME) y con muestreo de Gibbs; evaluar motivos por enriquecimiento diferencial y centralidad.

Una neurona y un linfocito comparten, salvo excepciones, el mismo genoma, pero expresan conjuntos distintos de genes. La neurona produce canales iónicos y receptores de neurotransmisores; el linfocito, receptores de antígenos y citocinas. Para entender esa diferencia hay que observar qué regiones del ADN son accesibles, dónde se unen los factores de transcripción, qué modificaciones llevan las histonas y qué citosinas están metiladas. Estos estados pueden persistir a través de las divisiones celulares y también cambiar. Constituyen el *epigenoma*, que la *epigenómica* mide a escala del genoma completo.

La secuenciación masiva convirtió la epigenómica en una disciplina cuantitativa: casi todas sus técnicas terminan en millones de lecturas cortas que se mapean contra la referencia (capítulo 7) y cuyo patrón de acumulación hay que interpretar estadísticamente. En este capítulo recorreremos las tres preguntas centrales del área. Primero, ¿*dónde* se une una proteína o dónde está abierta la cromatina? Lo responden ChIP-seq y ATAC-seq, y la herramienta matemática será el modelo de Poisson para distinguir un pico de un fondo (sección 13.1). Segundo, ¿*qué* citosinas están metiladas y cuánto? Lo responde la secuenciación con bisulfito, y la herramienta será el modelo beta-binomial (sección 13.2). Tercero, ¿*por qué* se une un factor allí y no en otro lugar? Lo responde el descubrimiento de motivos, que retoma las matrices de peso posicional del capítulo 4 y las aprende de los datos con los algoritmos EM y de Gibbs (sección 13.3).

13.1 ChIP-seq y ATAC-seq

 Colab · 13.1 · ChIP-seq y ATAC-seq

 repositorio del curso

Huellas en la nieve

Imagine que quiere saber dónde se detienen los ciervos en un bosque nevado, pero nunca los ve: solo encuentra huellas al amanecer. Si caminara al azar por el bosque, vería huellas dispersas por todas partes; pero alrededor de los comederos las huellas se amontonan, llegan desde el norte por un lado y se

van hacia el sur por el otro. Para decidir si un amontonamiento es un comedero o una casualidad necesita saber cuántas huellas *esperaría* en esa zona si los ciervos pasearan sin rumbo (el fondo), y ese fondo no es igual en todo el bosque: junto al arroyo siempre hay más tránsito. ChIP-seq es exactamente esto. Las lecturas son huellas de fragmentos de ADN que estuvieron unidos a una proteína; se amontonan alrededor del sitio de unión, llegan por la hebra + desde un lado y por la hebra – desde el otro, y la estadística consiste en compararlas con un fondo local bien estimado.

13.1.1 La cromatina: un genoma empaquetado

El ADN de una célula humana mide unos dos metros y cabe en un núcleo de pocas micras. Lo consigue enrollándose alrededor de octámeros de histonas (dos copias de cada una de H2A, H2B, H3 y H4). Cada unidad, el *nucleosoma*, envuelve unos 147 pares de bases en una superhélice de casi dos vueltas; la estructura cristalográfica de Luger et al. (1997) mostró en detalle atómico cómo 146 pares de bases se organizan alrededor del octámero y cómo las colas amino-terminales de las histonas salen entre las vueltas del ADN para contactar con partículas vecinas. Entre un nucleosoma y el siguiente queda un segmento de ADN *conector* (*linker*) de longitud variable, de modo que la repetición nucleosomal típica es de unos 180 a 200 pb. Esta cifra reaparecerá, como una huella dactilar, en los datos de ATAC-seq.

Las colas de las histonas son el soporte de un rico repertorio de modificaciones postraduccionales: acetilación, metilación (mono-, di- o tri-), fosforilación, ubiquitinación. Se nombran con una notación compacta: H3K27ac es la acetilación de la lisina 27 de la histona H3; H3K4me3, la trimetilación de la lisina 4. Jenuwein y Allis (2001) propusieron que la combinación de marcas forma un *código de histonas* que, leído por proteínas con dominios específicos, determina transiciones entre cromatina activa y silenciada. La tabla 13.1 resume las marcas más estudiadas.

Tres grandes consorcios han cartografiado estas marcas junto con la unión de factores de transcripción y la accesibilidad de la cromatina. El proyecto ENCODE (*Encyclopedia of DNA Elements*) publicó en 2012 un mapa integrado de transcripción, unión de factores, estructura de la cromatina y modificación de histonas en líneas celulares humanas, y afirmó poder asignar alguna “función bioquímica” al 80 % del genoma (ENCODE Project Consortium, 2012). La cifra provocó un intenso debate, porque actividad bioquímica no equivale a función biológica; la lección metodológica es que *toda señal epigenómica es una medida, no una interpretación*. La tercera fase del proyecto amplió la enciclopedia a humano y ratón y organizó

Tabla 13.1. Marcas de histonas más usadas en los atlas epigenómicos, con su localización típica y la forma de su señal en ChIP-seq. Las marcas “anchas” cubren decenas de kilobases y requieren estrategias de llamada distintas de las de un factor de transcripción.

Marca	Dónde aparece	Asociación	Señal
H3K4me3	promotores, alrededor del TSS	activa	estrecha
H3K4me1	potenciadores (<i>enhancers</i>) activos o preparados	activa/latente	intermedia
H3K27ac	potenciadores y promotores activos	activa	intermedia
H3K36me3	cuerpos de genes transcritos	elongación	ancha
H3K27me3	genes reprimidos por Polycomb	represiva	ancha
H3K9me3	heterocromatina constitutiva, repeticiones	represiva	ancha

los datos en un registro de elementos candidatos *cis*-reguladores (ENCODE Project Consortium et al., 2020). En paralelo, el consorcio Roadmap Epigenomics generó 111 epigenomas de referencia de tejidos y tipos celulares primarios e integró cinco marcas centrales (H3K4me3, H3K4me1, H3K36me3, H3K27me3 y H3K9me3) en un modelo de *estados de cromatina* con el que anotar cada segmento del genoma como promotor activo, potenciador, región transcrita, región reprimida por Polycomb o heterocromatina (Roadmap Epigenomics Consortium et al., 2015).

13.1.2 El experimento de ChIP-seq

La inmunoprecipitación de cromatina seguida de secuenciación (ChIP-seq) mide dónde está una proteína en el genoma de una población de células. Johnson et al. (2007) la usaron para mapear la unión del represor NRSF (también llamado REST) a 1946 sitios del genoma humano, con una resolución de unas ± 50 pb, lo bastante fina para descubrir variantes no canónicas de su motivo. El protocolo tiene cinco pasos (figura 13.1):

- 1. Entrecruzamiento.** El formaldehído forma enlaces covalentes entre proteínas y ADN cercanos, “congelando” las interacciones del momento.
- 2. Fragmentación.** La cromatina se rompe por sonicación (o con nucleasa) en fragmentos de unos 100 a 300 pb.
- 3. Inmunoprecipitación.** Un anticuerpo específico contra la proteína (o contra una marca de histona) captura los fragmentos que la llevan; el resto se lava.



Figura 13.1. Flujo de un experimento de ChIP-seq. (1) El formaldehído fija el factor de transcripción (naranja) a su sitio; los nucleosomas (gris) también quedan fijados. (2) La cromatina se fragmenta. (3) Un anticuerpo acoplado a una esfera magnética captura los fragmentos con el factor; los demás se descartan. (4) Tras purificar el ADN se secuencian los extremos 5' de cada fragmento: si se lee la hebra superior, el extremo 5' está a la izquierda del sitio; si se lee la inferior, a la derecha. (5) Al mapear, las lecturas + (azul) se acumulan a la izquierda del sitio y las - (rojo) a la derecha: la firma geométrica de un sitio de unión verdadero.

- 4. Reversión y secuenciación.** Se revierte el entrecruzamiento, se purifica el ADN, se prepara una biblioteca y se secuencian, casi siempre solo por un extremo de cada fragmento o en pares cortos.
- 5. Mapeo.** Las lecturas se alinean a la referencia (capítulo 7) y se buscan regiones con más lecturas de las esperadas: los *picos*.

Siempre se acompaña de un *control*: normalmente el *input*, ADN fragmentado de la misma muestra pero sin inmunoprecipitar, que registra los sesgos que nada tienen que ver con la proteína (regiones más fáciles de sonicar, amplificaciones del número de copias, zonas de mapeo problemático). Park (2009) ofrece una revisión clásica de las ventajas de ChIP-seq frente a su predecesor en micromatrices (ChIP-chip) y de los retos de diseño y análisis.

13.1.3 La geometría de las lecturas: dos hebras y un desplazamiento

El detalle clave del paso 4 es que la secuenciación lee solo las primeras decenas de bases de cada fragmento, desde su extremo 5'. Si el fragmento tiene longitud ℓ y está centrado en el sitio de unión μ , la lectura de la hebra + empieza en $\mu - \ell/2$ y la de la hebra - en $\mu + \ell/2$. Con muchos fragmentos, las lecturas de cada hebra forman una “montaña” desplazada del sitio.

Teorema 13.1. (Perfiles esperados por hebra)

Sea c la posición del centro de un fragmento, con densidad $\varphi(c)$ centrada en el sitio μ , y sea ℓ su longitud, independiente de c , con densidad $g(\ell)$. Las densidades esperadas de extremos 5' en cada hebra son

$$f_+(x) = \int g(\ell) \varphi\left(x + \frac{\ell}{2}\right) d\ell, \quad f_-(x) = \int g(\ell) \varphi\left(x - \frac{\ell}{2}\right) d\ell. \quad (13.1)$$

Si φ es simétrica alrededor de μ , f_+ y f_- son imágenes especulares una de otra y sus centros de masa están en $\mu \mp \bar{\ell}/2$; la distancia entre ambos es la longitud media del fragmento, $d = \bar{\ell}$.

DONDE

μ	Posición del sitio de unión en el genoma.
$\varphi(c)$	Densidad de la posición del centro del fragmento; su anchura refleja la imprecisión de la fragmentación.
$g(\ell), \bar{\ell}$	Densidad y media de la longitud de los fragmentos de la biblioteca.
$f_{\pm}(x)$	Densidad esperada de extremos 5' de lecturas en la hebra + o – en la posición x .
d	Desplazamiento entre las dos montañas; se estima de los datos y se usa para “centrar” las lecturas.

La demostración es un cambio de variable: una lectura + cae en x si el fragmento tiene centro $c = x + \ell/2$, de modo que basta integrar sobre las longitudes posibles. El resultado tiene una consecuencia práctica inmediata. Si desplazamos cada lectura + una distancia $d/2$ hacia la derecha y cada lectura – una distancia $d/2$ hacia la izquierda, las dos montañas se superponen en el sitio verdadero y la resolución mejora. Alternativamente, se puede *extender* cada lectura hasta la longitud d en dirección 3', reconstruyendo el fragmento completo; la suma de fragmentos reconstruidos se llama *pileup*. MACS estima d tomando las regiones más enriquecidas, separando sus lecturas por hebra y midiendo la distancia entre las modas de ambas (Zhang et al., 2008). La figura 13.2 (a) muestra el resultado con fragmentos simulados de longitud media 200 pb: las modas están en -100 y $+99$, y la suma desplazada queda centrada en el sitio.

13.1.4 Correlación cruzada entre hebras

Una estimación de d que no requiere elegir regiones de antemano usa *todo* el genoma. Sean $n_+(x)$ y $n_-(x)$ los conteos de extremos 5' en cada posición. Si los sitios de unión dominan la señal, n_- se parece a n_+ desplazado d posiciones, y la correlación entre ambos perfiles será máxima en ese desplazamiento (Kharchenko et al., 2008).

Definición 13.1. (Correlación cruzada entre hebras)

Para cada desplazamiento $s \geq 0$,

$$r(s) = \text{corr}(n_+(x), n_-(x+s)) = \frac{\sum_x (n_+(x) - \bar{n}_+)(n_-(x+s) - \bar{n}_-)}{(G-s)\sigma_+\sigma_-}, \quad (13.2)$$

donde la suma recorre las $G-s$ posiciones válidas del genoma. El *tamaño de fragmento estimado* es $\hat{d} = \arg \max_s r(s)$, excluyendo desplazamientos cercanos a la longitud de lectura.

DONDE

$n_{\pm}(x)$	Número de lecturas cuyo extremo 5' cae en x en cada hebra.
$\bar{n}_{\pm}, \sigma_{\pm}$	Media y desviación estándar de esos conteos a lo largo del genoma.
G	Longitud del genoma (o del cromosoma analizado).
s	Desplazamiento aplicado a la hebra $-$.

La curva $r(s)$ tiene casi siempre *dos* máximos. El primero, en $s = \hat{d}$, es la señal biológica. El segundo, en $s \approx L$ (la longitud de lectura), es el *pico fantasma* (*phantom peak*), un artefacto de la mapeabilidad: en regiones repetidas, donde las lecturas no pueden mapearse de forma única, desaparecen a la vez las lecturas $+$ que empiezan en x y las lecturas $-$ cuyo extremo 5' está en $x + L - 1$, lo que correlaciona ambas hebras a esa distancia aunque no haya proteína alguna. Las guías de ENCODE y modENCODE (Landt et al., 2012) convirtieron la curva en dos índices de calidad:

$$\text{NSC} = \frac{r(\hat{d})}{\min_s r(s)}, \quad \text{RSC} = \frac{r(\hat{d}) - \min_s r(s)}{r(L) - \min_s r(s)}. \quad (13.3)$$

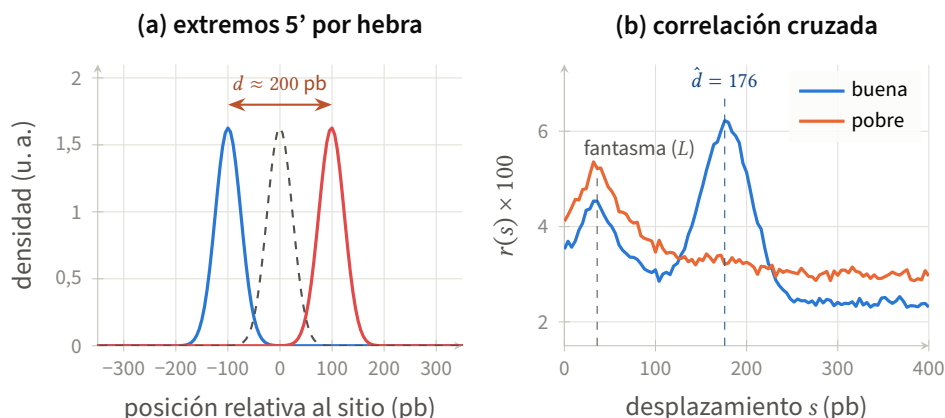


Figura 13.2. (a) Densidad de extremos 5' de 4000 fragmentos simulados alrededor de un sitio de unión (longitud media 200 pb). Las lecturas de la hebra + (azul) forman una montaña a la izquierda del sitio y las de la hebra - (rojo), su imagen especular a la derecha (ecuación (13.1)); al desplazarlas $d/2$ hacia el centro (línea discontinua) se superponen sobre el sitio. (b) Correlación cruzada entre hebras (ecuación (13.2)) en un cromosoma simulado de 3 Mb con regiones no mapeables. La muestra buena muestra un máximo claro en el tamaño de fragmento y un pico fantasma menor en la longitud de lectura ($L = 36$); en la muestra pobre el pico fantasma domina, señal de que el anticuerpo apenas enriqueció. Datos: figuras/cap13/generar.py.

DONDE

NSC	Coefficiente de hebras normalizado: cuánto sobresale el pico de fragmento sobre el fondo de la curva.
RSC	Coefficiente de hebras relativo: tamaño del pico de fragmento comparado con el del pico fantasma.
L	Longitud de las lecturas.

Un RSC mayor que 1 indica que la señal biológica domina sobre el artefacto de mapeo; según esas guías, valores de NSC por debajo de 1,05 y de RSC por debajo de 0,8 delatan un enriquecimiento pobre. En la simulación de la figura 13.2 (b), con lecturas de 36 pb, la muestra buena alcanza su máximo en $\hat{d} = 176$ pb con $NSC = 2,70$ y $RSC = 1,76$; en la muestra pobre, con siete veces menos lecturas procedentes de sitios reales, el pico de fragmento casi desaparece y el máximo fuera de la zona del fantasma cae en un desplazamiento de 104 pb sin significado biológico, con $RSC = 0,36$.

13.1.5 Llamar picos: el fondo de Poisson local

Una vez desplazadas las lecturas, la pregunta es estadística: ¿en qué ventanas del genoma hay *más* lecturas de las que produciría el fondo? El modelo más simple supone que, en ausencia de unión, las N lecturas se reparten uniformemente sobre las G posiciones mapeables.

Teorema 13.2. (*Conteo en una ventana bajo el fondo uniforme*)

Si cada una de las N lecturas cae en una ventana de ancho w con probabilidad $p = w/G$, de forma independiente, el número X de lecturas en la ventana sigue una distribución binomial $\text{Bin}(N, p)$. Cuando $N \rightarrow \infty$ y $p \rightarrow 0$ con $Np = \lambda$ fijo,

$$\mathbb{P}(X = k) \rightarrow \frac{e^{-\lambda} \lambda^k}{k!}, \quad \lambda = \frac{Nw}{G}, \quad (13.4)$$

y el valor p de observar al menos k lecturas es la cola superior

$$p(k) = \mathbb{P}(X \geq k) = 1 - \sum_{i=0}^{k-1} \frac{e^{-\lambda} \lambda^i}{i!}. \quad (13.5)$$

DONDE

N	Número total de lecturas (o fragmentos) mapeadas en la muestra tratada.
G	Tamaño efectivo del genoma: número de posiciones donde una lectura puede mapearse de forma única (unos $2,7 \times 10^9$ en humano).
w	Ancho de la ventana; en MACS, $w = 2d$.
λ	Número esperado de lecturas en la ventana bajo el fondo.
k	Número de lecturas observadas en la ventana candidata.

La demostración es el límite clásico de Poisson:

$$\binom{N}{k} p^k (1-p)^{N-k} = \frac{N(N-1) \cdots (N-k+1)}{N^k} \frac{\lambda^k}{k!} \left(1 - \frac{\lambda}{N}\right)^{N-k},$$

y el primer factor tiende a 1 mientras que el último tiende a $e^{-\lambda}$. Es la misma aproximación que, en el capítulo 6, nos dio la teoría de Lander-Waterman para la cobertura.

El fondo uniforme, sin embargo, es falso. La sonicación rompe mejor la cromatina abierta; las regiones con más copias en la línea celular producen más lecturas;

algunas zonas acumulan señal en cualquier experimento. Aplicar un λ global en esas regiones llena la lista de picos de falsos positivos. La solución de MACS (*Model-based Analysis of ChIP-Seq*), propuesta por Zhang et al. (2008), es un *Poisson dinámico*: para cada ventana candidata se toma como fondo el máximo de varias estimaciones, de modo que el modelo es conservador allí donde el control indica un sesgo.

Definición 13.2. (Lambda local de MACS)

Para una ventana candidata de ancho $2d$,

$$\lambda_{\text{local}} = \max(\lambda_{\text{BG}}, \lambda_{1k}, \lambda_{5k}, \lambda_{10k}), \quad \lambda_{w'} = c_{w'} \cdot \frac{2d}{w'} \cdot \frac{N_T}{N_C}, \quad (13.6)$$

donde $\lambda_{\text{BG}} = N_T 2d / G$ es el fondo global y $c_{w'}$ es el número de lecturas del control en una ventana de ancho $w' \in \{1, 5, 10\}$ kb centrada en el candidato.

DONDE

λ_{BG}	Fondo global: las lecturas tratadas repartidas uniformemente.
$c_{w'}$	Lecturas del control en la ventana ancha w' ; se reescalan al ancho $2d$ y a la profundidad del tratamiento.
N_T, N_C	Número de lecturas del tratamiento y del control.

Tomar el *máximo* tiene una lógica de protección: si cualquiera de las escalas indica que la región es ruidosa, el umbral sube. Las versiones posteriores, MACS2 y MACS3, conservan esta idea (por omisión con ventanas de d , 1 kb y 10 kb en el control) y añaden la corrección de Benjamini-Hochberg para obtener valores q , el modo de picos anchos para marcas como H3K27me3 y el uso directo de fragmentos pareados. El artículo original estimaba además la tasa de falsos descubrimientos de forma empírica, intercambiando los papeles de tratamiento y control: los “picos” que aparecen en el control frente al tratamiento dan una idea de cuántos falsos positivos produce el procedimiento con el mismo umbral (Zhang et al., 2008).

Ejemplo 13.1. (Un pico, dos fondos)

Un experimento tiene $N_T = 2 \times 10^7$ lecturas tratadas y $N_C = 2,5 \times 10^7$ de control; el tamaño efectivo del genoma es $G = 2,7 \times 10^9$ y $d = 200$, así que la ventana mide 400 pb. El fondo global es

$$\lambda_{BG} = \frac{2 \times 10^7 \times 400}{2,7 \times 10^9} = 2,963.$$

En el control hay 10 lecturas en la ventana de 1 kb, 45 en la de 5 kb y 95 en la de 10 kb. Reescalando con el factor de profundidad $N_T/N_C = 0,8$: $\lambda_{1k} = 10 \cdot 0,4 \cdot 0,8 = 3,200$, $\lambda_{5k} = 45 \cdot 0,08 \cdot 0,8 = 2,880$ y $\lambda_{10k} = 95 \cdot 0,04 \cdot 0,8 = 3,040$. Entonces $\lambda_{local} = 3,200$. Con $k = 17$ lecturas en la ventana,

$$p = \mathbb{P}(X \geq 17 \mid \lambda = 3,2) = 5,38 \times 10^{-8}, \quad -\log_{10} p = 7,27,$$

un pico claro. Con el umbral de MACS por omisión en su versión original ($p < 10^{-5}$), bastan 14 lecturas.

Ahora considere otra región, idéntica en el tratamiento, pero en una amplificación de la línea celular: el control tiene 400 lecturas en 10 kb, así que $\lambda_{10k} = 12,8$. Con $k = 14$ lecturas, el fondo global daría $p = 2,96 \times 10^{-6}$ (un “pico”), mientras que el fondo local da $p = 0,405$: nada notable. El control ha evitado un falso positivo que ninguna profundidad de secuenciación habría corregido.

Llamada de picos con MACS3

```
# ChIP de un factor de transcripción, pares de lecturas
macs3 callpeak -t chip_rep1.bam -c input.bam \
  -f BAMPE -g hs -n ctcf_rep1 -q 0.01 --outdir picos/
# salidas: *_peaks.narrowPeak (BED6+4) y *_summits.bed
```

13.1.6 Calidad y reproducibilidad

Una lista de picos no vale nada si no se sabe cuánto de ella es señal. Tres controles se han vuelto estándar.

FRiP. La fracción de lecturas en picos (*fraction of reads in peaks*) es el cociente entre las lecturas que caen dentro de los picos llamados y el total. Mide la relación señal/ruido de todo el experimento: si 3,1 millones de 20 millones de lecturas caen en

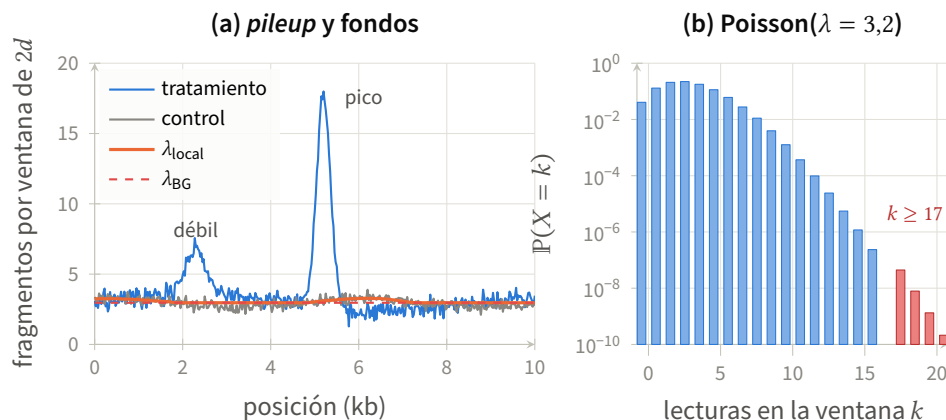


Figura 13.3. (a) Diez kilobases simuladas: el *pileup* del tratamiento (azul) muestra un pico fuerte cerca de 5,2 kb y un enriquecimiento débil cerca de 2,3 kb. La línea naranja es el fondo local estimado del control suavizado, que nunca baja del fondo global (roja discontinua), tal como impone el máximo de la ecuación (13.6). (b) Distribución de Poisson con $\lambda_{local} = 3,2$ en escala logarítmica. Diecisiete lecturas en una ventana de 400 pb caen en la cola roja, cuya probabilidad total es $5,4 \times 10^{-8}$ (ejemplo 13.1). Observe cuán rápido decae la cola: cada lectura adicional multiplica la evidencia.

picos, $FRiP = 0,155$. Las guías de ENCODE consideran aceptables, para factores de transcripción, valores del 1 % o más, aunque los buenos experimentos con factores abundantes como CTCF superan con holgura esa cifra (Landt et al., 2012).

Lista negra. Algunas regiones del genoma (repeticiones satélite cerca de centrómeros y telómeros, ADN ribosómico, secuencias con copias no representadas en la referencia) producen señal alta en *cualquier* experimento de secuenciación, independientemente del tipo celular o del anticuerpo. Amemiya et al. (2019) definieron la *lista negra de ENCODE* para humano, ratón, gusano y mosca, y recomiendan eliminar sus regiones como paso de calidad esencial antes de cualquier análisis.

IDR. Dos réplicas biológicas de un mismo experimento deberían coincidir en sus picos fuertes y discrepar en los débiles, que son ruido. Li et al. (2011) formalizaron esta intuición en la *tasa de irreproducibilidad* (*irreproducible discovery rate*, IDR). Su modelo supone que cada pico pertenece a uno de dos grupos: reproducible, cuyas puntuaciones en las dos réplicas están correlacionadas, o irreproducible, cuyas puntuaciones son independientes. Las puntuaciones se transforman en rangos (para

no depender de la escala de cada llamador de picos) y una mezcla de dos cópulas gaussianas se ajusta por EM, el mismo algoritmo que veremos en la sección 13.3. Para cada pico i se obtiene la probabilidad posterior de ser irreproducible, idr_i , y para una lista de los picos más fuertes S_γ ,

$$\text{IDR}(\gamma) = \frac{1}{|S_\gamma|} \sum_{i \in S_\gamma} \text{idr}_i, \quad (13.7)$$

DONDE

idr_i	Probabilidad posterior, según la mezcla, de que el pico i pertenezca al grupo irreproducible.
S_γ	Conjunto de picos seleccionados con el umbral γ (p. ej., los que tienen $\text{idr}_i \leq \gamma$).

que es el análogo exacto de la tasa de falsos descubrimientos: la fracción esperada de picos irreproducibles en la lista. El protocolo de ENCODE llama picos de forma permisiva en cada réplica y conserva los que superan el umbral de IDR elegido (típicamente 0,05) (Landt et al., 2012). En una simulación con 1500 picos reales y 2500 de ruido, la correlación de Spearman global entre réplicas es de solo 0,70, pero los 857 picos que están entre los mil primeros de *ambas* réplicas son todos reales: la estructura que el modelo aprende está en la cabeza de las listas.

⚠ Errores frecuentes en ChIP-seq

1. *Sin control o con un control poco profundo.* El control debe tener al menos tanta profundidad como el tratamiento; si no, λ_{local} se estima con mucho ruido.
2. *Duplicados de PCR.* Muchas lecturas idénticas apiladas en la misma posición inflan los conteos y violan la independencia del modelo de Poisson; se marcan y se eliminan (o se limita su número) antes de llamar picos.

13.1.7 ATAC-seq: medir la accesibilidad

ChIP-seq necesita un anticuerpo por proteína y millones de células. Una pregunta más general (¿qué regiones del genoma están *abiertas*, libres de nucleosomas y disponibles para cualquier factor?) se responde con una idea ingeniosa. La transposasa Tn5, en una versión hiperactiva cargada con adaptadores de secuenciación, corta el

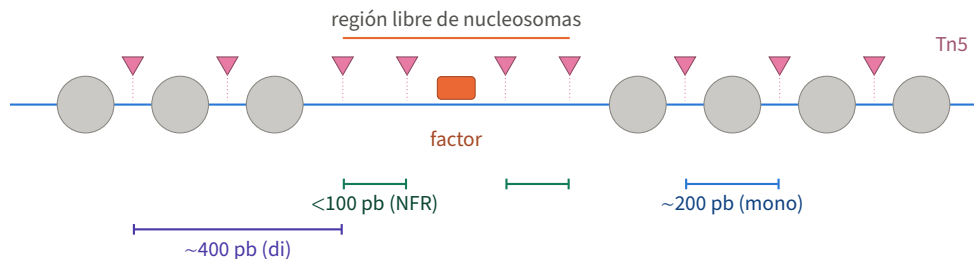


Figura 13.4. Principio de ATAC-seq. La transposasa Tn5 (triángulos magenta) inserta adaptadores solo donde el ADN no está protegido: en la región libre de nucleosomas (NFR) alrededor de un factor unido y en los conectores entre nucleosomas (círculos grises). Dos inserciones sucesivas definen un fragmento secuenciable; su longitud revela lo que había entre ellas: nada (fragmentos cortos de la NFR), un nucleosoma (unos 200 pb) o dos (unos 400 pb).

ADN e inserta los adaptadores en el mismo acto (“tagmentación”). En la cromatina nativa, Tn5 solo accede a los segmentos no protegidos: las regiones libres de nucleosomas y los conectores entre nucleosomas. Buenrostro et al. (2013) llamaron a la técnica ATAC-seq (*assay for transposase-accessible chromatin using sequencing*) y mostraron que un protocolo de dos pasos con 500 a 50 000 células revela a la vez la cromatina abierta, la unión de factores y la posición de nucleosomas individuales; el protocolo detallado está en Buenrostro et al. (2015). Como Tn5 corta como dímero con un escalonamiento de 9 pb, los análisis de alta resolución desplazan las lecturas $+4/-5$ pb para situar el centro exacto de cada inserción (Buenrostro et al., 2013).

Dos inserciones de Tn5 definen un fragmento, y su longitud es un registro directo de lo que había entre ellas (figura 13.4). Por eso la distribución de tamaños de fragmento de un buen experimento de ATAC-seq tiene una forma inconfundible: una gran población por debajo de 100 pb, procedente de regiones libres de nucleosomas, y una serie de “jorobas” en múltiplos de la repetición nucleosomal. Podemos escribirla como una mezcla:

$$h(\ell) = \pi_0 h_0(\ell) + \sum_{j=1}^J \pi_j \mathcal{N}(\ell; j \ell_{\text{nuc}}, \sigma_j^2), \quad \sum_{j=0}^J \pi_j = 1, \quad (13.8)$$

DONDE

$h(\ell)$	Densidad de la longitud ℓ de los fragmentos.
h_0	Componente libre de nucleosomas (fragmentos cortos, con moda por debajo de 100 pb).
ℓ_{nuc}	Repetición nucleosomal efectiva (unos 190–200 pb: 147 del núcleo más el conector).
π_j, σ_j	Proporción y dispersión de los fragmentos que abarcan j nucleosomas.

A esta forma gruesa se superpone una oscilación fina de unos 10,5 pb, el paso de la doble hélice: Tn5 corta preferentemente el lado de la hélice expuesto, de modo que las distancias entre cortes favorecen múltiplos de una vuelta. La figura 13.5 (a) muestra una distribución simulada con ese modelo, en la que el 38 % de los fragmentos mide menos de 100 pb y el 21 % cae en el intervalo mononucleosomal (180–247 pb).

Definición 13.3. (Enriquecimiento en TSS)

Sea $I(x)$ el número de inserciones de Tn5 acumuladas en la posición x relativa a todos los sitios de inicio de la transcripción (TSS) del genoma, orientados según la hebra del gen. El perfil normalizado es

$$E(x) = \frac{I(x)}{\frac{1}{2m} \left(\sum_{u=-2000}^{-2000+m-1} I(u) + \sum_{u=2000-m+1}^{2000} I(u) \right)}, \quad (13.9)$$

es decir, el conteo dividido por el promedio en los extremos de la ventana (m posiciones a cada lado, típicamente 100 pb). La *puntuación de enriquecimiento en TSS* es $\max_x E(x)$ cerca de $x = 0$.

DONDE

$I(x)$	Inserciones acumuladas a distancia x del TSS (en pb, con signo según la orientación del gen).
m	Ancho de los bordes usados como fondo.
$E(x)$	Enriquecimiento relativo al fondo lejano; vale ≈ 1 lejos del TSS.

Los promotores activos son las regiones abiertas más universales, así que un experimento que “ve” cromatina abierta debe mostrar un pico agudo en el TSS. Un enriquecimiento bajo (digamos, inferior a 5) indica demasiadas lecturas de fondo,

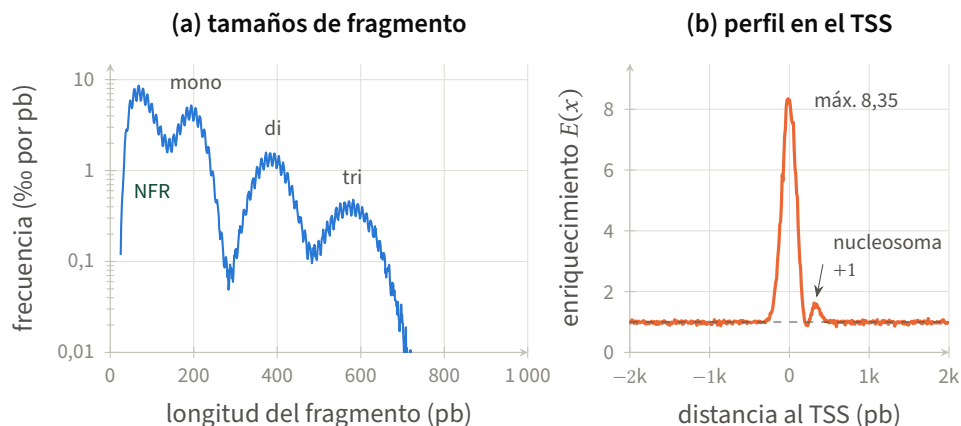


Figura 13.5. Controles de calidad de ATAC-seq sobre datos simulados con la mezcla de la ecuación (13.8). (a) Distribución de tamaños de fragmento en escala logarítmica: la población libre de nucleosomas (NFR) domina, y le siguen jorobas mono-, di- y trinucleosomales separadas por unos 190 pb; la ondulación fina de 10,5 pb refleja la periodicidad helicoidal del ADN. Una biblioteca sin jorobas sugiere digestión excesiva o ADN desnudo. (b) Enriquecimiento de inserciones alrededor del TSS (ecuación (13.9)), normalizado por los extremos de la ventana (línea discontinua en 1).

típicamente por células muertas o lisis excesiva de los núcleos. El perfil simulado de la figura 13.5 (b) alcanza 8,35, con un pequeño hombro río abajo que corresponde al nucleosoma +1, bien posicionado en los genes activos.

CUT&RUN. Una alternativa moderna a ChIP-seq invierte la lógica del experimento. En lugar de fragmentar toda la cromatina y luego pescar los trozos que interesan, CUT&RUN (*cleavage under targets and release using nuclease*) lleva una nucleasa hasta la proteína de interés: un anticuerpo se une al factor en núcleos intactos y una nucleasa micrococcal acoplada a proteína A corta el ADN a su alrededor, liberando solo los fragmentos unidos. Como no hay entrecruzamiento ni fragmentación global, el fondo es mínimo: Skene y Henikoff (2017) obtuvieron perfiles precisos con aproximadamente una décima parte de la profundidad de secuenciación de ChIP-seq.

Un pico es un exceso de lecturas sobre un fondo *local*. La geometría de hebras delata la señal verdadera, el control protege frente a sesgos del genoma y la reproducibilidad entre réplicas, no el valor p , decide qué picos se conservan.

13.2 Metilación del ADN



Colab · 13.2 · Metilación del ADN



repositorio del curso

Post-its en un libro de recetas

Piense en un libro de recetas compartido por toda una familia. Nadie reescribe las recetas, pero cada cocinero pega notas adhesivas sobre las que no quiere que se usen: “no preparar”, “solo en Navidad”. Las notas no cambian el texto, se copian cuando alguien fotocopia el libro para un hijo, y pueden despegarse. La metilación del ADN funciona de manera parecida: un grupo metilo sobre una citosina no altera la secuencia, se hereda en cada división celular gracias a una enzima que copia el patrón de la hebra vieja a la nueva, y puede borrarse. El problema técnico es que las fotocopias normales, es decir, la PCR y la secuenciación, no copian las notas. Hay que encontrar una tinta que revele dónde estaban antes de fotocopiar.

13.2.1 Citosinas metiladas e islas CpG

En los mamíferos, la metilación del ADN consiste casi exclusivamente en añadir un grupo metilo al carbono 5 de una citosina (5mC) seguida de una guanina, el dinucleótido CpG. Como el dinucleótido complementario de CpG es también CpG, la marca es simétrica en las dos hebras, y eso permite heredarla: tras la replicación, la hebra nueva se metila frente a los CpG metilados de la hebra vieja. La metilación de un promotor se asocia en general al silenciamiento estable del gen, y participa en la impronta genómica, la inactivación del cromosoma X y el control de elementos transponibles.

La metilación deja una huella en la propia secuencia del genoma. La 5mC tiende a desaminarse espontáneamente a timina, un error que la reparación corrige peor que la desaminación de la citosina normal a uracilo. A lo largo de la evolución,

los CpG metilados se han ido convirtiendo en TpG (y CpA en la otra hebra), de modo que el genoma humano tiene alrededor de una cuarta parte de los CpG que esperaríamos por su composición. La excepción son las *islas CpG*: tramos de cientos a miles de pares de bases con abundantes CpG no metilados, que Bird (1986) asoció con los extremos 5' de la mayoría de los genes de vertebrados y, en particular, de los genes de mantenimiento, cuya disponibilidad debe ser constante en el núcleo. Gardiner-Garden y Frommer (1987) dieron el criterio operativo que todavía se usa, basado en la razón entre CpG observados y esperados.

Definición 13.4. (Isla CpG (criterio de Gardiner-Garden y Frommer))

Una región de longitud $L \geq 200$ pb es una isla CpG si su contenido de GC supera el 50 % y

$$\frac{\text{CpG obs.}}{\text{CpG esp.}} = \frac{n_{CG}}{n_C n_G / L} = \frac{n_{CG} L}{n_C n_G} > 0,6. \quad (13.10)$$

DONDE

n_{CG}	Número de dinucleótidos CpG en la región (en una hebra).
n_C, n_G	Número de citosinas y guaninas.
L	Longitud de la región; $n_C n_G / L$ es el número de CpG esperado si C y G se sucedieran al azar.

Ejemplo 13.2. (Dos regiones de 100 pb)

Generamos dos secuencias sintéticas de 100 pb que imitan la evolución de la desaminación: una rica en GC en la que se conserva el 90 % de los CpG (“isla”) y otra con GC moderado en la que solo se conserva el 22 % (“mar”). En la primera hay $n_C = 37$, $n_G = 32$ y $n_{CG} = 12$: GC = 69 % y

$$\frac{\text{obs.}}{\text{esp.}} = \frac{12 \times 100}{37 \times 32} = 1,01 > 0,6.$$

En la segunda, $n_C = 22$, $n_G = 17$ y un único CpG: GC = 39 % y obs./esp. = $100/(22 \times 17) = 0,27$, el valor típico del “mar” genómico. (Para aplicar formalmente el criterio habría que usar ventanas de al menos 200 pb.)

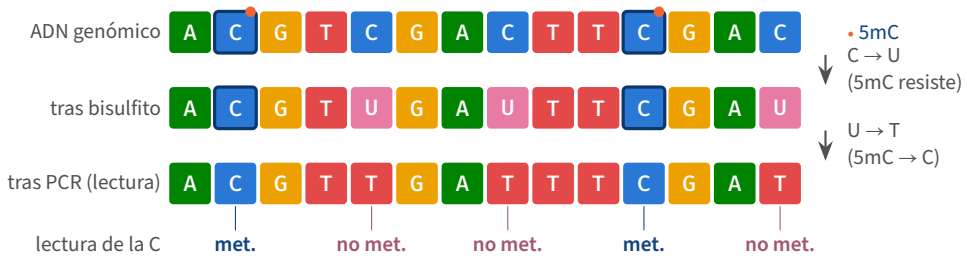


Figura 13.6. Química de la conversión por bisulfito en una hebra de 14 bases con dos CpG metilados (C con borde oscuro y punto naranja), un CpG no metilado (posición 5) y dos citosinas fuera de contexto CpG. El bisulfito desamina las citosinas no metiladas a uracilo (magenta); la PCR copia el uracilo como timina. Al comparar la lectura con la referencia, una C que sigue siendo C estaba metilada; una C que aparece como T no lo estaba. La hebra convertida ya no es complementaria de la original, y por eso cada hebra del genoma produce lecturas distintas.

13.2.2 La química del bisulfito

Ni la PCR ni los secuenciadores de síntesis distinguen 5mC de C: la polimerasa pone una G frente a ambas y el grupo metilo se pierde en la primera copia. La solución clásica es química. El bisulfito de sodio desamina la citosina a uracilo en condiciones en las que la 5-metilcitosina, protegida por el metilo, apenas reacciona. Frommer et al. (1992) convirtieron esta reacción en un método de secuenciación genómica que identifica positivamente cada 5mC: tras el tratamiento, se amplifica por PCR y en el producto todos los uracilos (y timinas) aparecen como T, mientras que solo las 5mC siguen leyéndose como C (figura 13.6).

La consecuencia para el análisis es doble. Primero, la medida es *digital* en cada molécula: cada lectura dice “C” o “T” en cada citosina de la referencia, y el nivel de metilación de un sitio es la fracción de lecturas que dicen “C”. Segundo, el alineamiento se complica, porque una lectura convertida difiere de la referencia en casi todas sus citosinas. Si la conversión fuera incompleta, algunas C no metiladas se leerían como metiladas; por eso se estima la tasa de no conversión con ADN control sin metilar añadido a la muestra, o con citosinas fuera de contexto CpG en tejidos donde apenas se metilan.

13.2.3 Secuenciación con bisulfito del genoma completo y alineamiento

La secuenciación con bisulfito del genoma completo (WGBS, *whole-genome bisulfite sequencing*) aplica la conversión a una biblioteca de todo el genoma. Lister

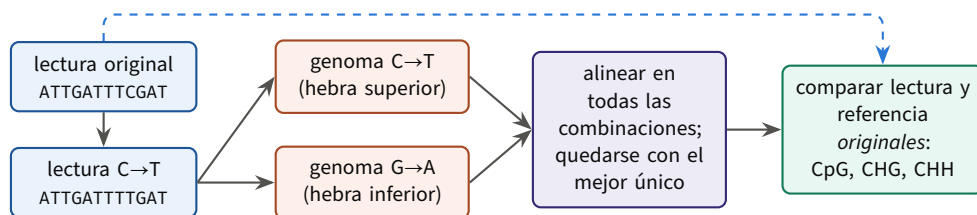


Figura 13.7. Estrategia de alineamiento de Bismark (Krueger & Andrews, 2011). La lectura y el genoma se convierten *in silico* al alfabeto reducido, de modo que las diferencias debidas al bisulfito desaparecen y solo quedan los verdaderos desajustes. Tras encontrar la posición única, la metilación se llama volviendo a la lectura original (flecha discontinua): la C del CpG de la lectura de ejemplo indica una citosina metilada; las T frente a C de la referencia, citosinas no metiladas.

et al. (2009) publicaron así los primeros metilomas humanos con resolución de una base, de células madre embrionarias y fibroblastos fetales, y descubrieron que casi una cuarta parte de la metilación de las células madre embrionarias estaba fuera del contexto CG, una forma que desaparecía al diferenciarse.

El problema del alineamiento. Un alineador convencional penalizaría cada T de la lectura frente a una C de la referencia. Pero no basta con permitir esos desajustes sin coste: tras la conversión, la lectura está escrita casi en un alfabeto de tres letras, y su complejidad baja tanto que muchas lecturas mapearían en varios lugares. La solución de Bismark (Krueger & Andrews, 2011) es convertir *todo* de forma computacional antes de alinear (figura 13.7): en las lecturas se reemplazan todas las C por T, y el genoma se prepara en dos versiones, una con todas las C convertidas en T (para la hebra superior) y otra con todas las G convertidas en A (para la inferior). Cada lectura se alinea con un alineador estándar contra las versiones convertidas en todas las combinaciones posibles de hebra; se conserva solo el alineamiento único con menos desajustes y, una vez ubicado, se compara la lectura *original* con la referencia *original* para llamar la metilación de cada citosina, distinguiendo los contextos CpG, CHG y CHH (con H = A, C o T).

Flujo básico de WGBS con Bismark

```

bismark_genome_preparation genoma/          # versiones C->T y G->A
bismark --genome genoma/ -1 m1_R1.fq.gz -2 m1_R2.fq.gz
deduplicate_bismark --bam m1_R1_bismark_bt2_pe.bam
  
```

```
bismark_methylation_extractor --bedGraph --cytosine_report \
  --genome_folder genoma/ m1_R1_bismark_bt2_pe.deduplicated.bam
# salida: por cada CpG, lecturas metiladas y no metiladas
```

13.2.4 Cuantificar: valores β y M

Para un CpG cubierto por n lecturas, de las cuales c dicen “C”, el modelo natural es binomial: $c \sim \text{Bin}(n, \beta)$, donde β es la fracción de moléculas metiladas en la muestra. El estimador de máxima verosimilitud es $\hat{\beta} = c/n$, con error estándar $\sqrt{\hat{\beta}(1 - \hat{\beta})/n}$.

Las micromatrices de Illumina (Infinium 450K y EPIC), aún muy usadas en estudios poblacionales, miden lo mismo de otra forma: para cada CpG hay dos intensidades de fluorescencia, M_s para el alelo metilado y U_s para el no metilado (tras la conversión, la sonda distingue C de T). Du et al. (2010) compararon las dos métricas en uso.

Definición 13.5. (Valores β y M)

$$\beta = \frac{\text{máx}(M_s, 0)}{\text{máx}(M_s, 0) + \text{máx}(U_s, 0) + \alpha}, \quad M = \log_2 \frac{\text{máx}(M_s, 0) + \alpha'}{\text{máx}(U_s, 0) + \alpha'}. \quad (13.11)$$

Ignorando las constantes, $M = \log_2 \frac{\beta}{1 - \beta}$: el valor M es la transformación *logit* (en base 2) del valor β .

DONDE

M_s, U_s	Intensidades de las sondas metilada y no metilada del CpG s (en WGBS, los conteos c y $n - c$).
α	Constante que estabiliza β cuando ambas intensidades son bajas (Illumina usa 100).
α'	Pequeña constante que evita el logaritmo de cero (p. ej., 1).
$\beta \in [0, 1]$	Fracción de metilación; interpretable como porcentaje de moléculas metiladas.
$M \in \mathbb{R}$	Valor M ; $M = 0$ corresponde a $\beta = 0.5$.

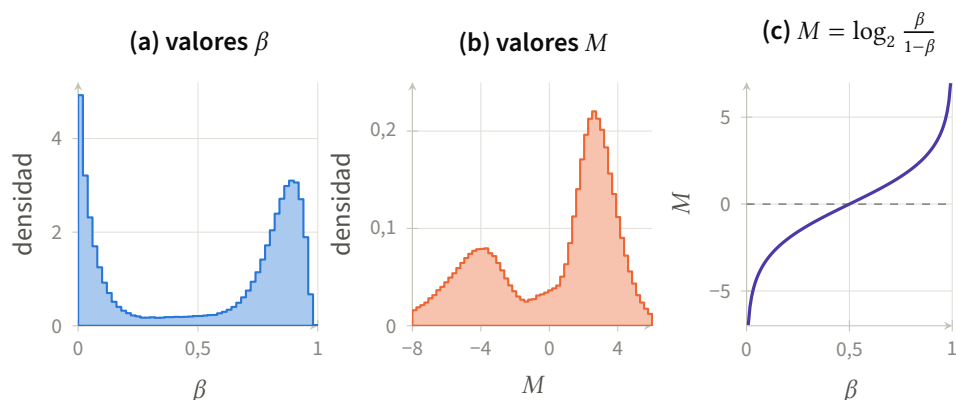


Figura 13.8. Distribución de 400 000 CpG simulados con intensidades de tipo Infinium. (a) La distribución de β es fuertemente bimodal: el 32,5 % de los sitios tiene $\beta < 0,2$ (sobre todo CpG de islas, no metilados) y el 43,1 % tiene $\beta > 0,8$ (el mar genómico, metilado). (b) Los mismos datos en escala M : los dos modos se separan y se ensanchan. (c) La transformación logit estira los extremos del intervalo $[0, 1]$, compensando la compresión de la varianza de β cerca de 0 y de 1 (ecuación (13.12)).

¿Por qué dos métricas? El valor β tiene una interpretación biológica directa, pero su varianza depende de su media: cerca de 0 y de 1 está comprimido por los límites del intervalo. El método delta muestra que la transformación logit compensa exactamente esa compresión. Si $\text{Var}(\hat{\beta}) \approx \beta(1 - \beta)/n$ (binomial), entonces

$$\text{Var}(\hat{M}) \approx \left(\frac{dM}{d\beta} \right)^2 \text{Var}(\hat{\beta}) = \frac{1}{(\beta(1 - \beta) \ln 2)^2} \cdot \frac{\beta(1 - \beta)}{n} = \frac{1}{n \beta(1 - \beta) (\ln 2)^2}, \quad (13.12)$$

DONDE

$dM/d\beta$	Derivada de la transformación logit en base 2, $1/(\beta(1 - \beta) \ln 2)$.
n	Número de moléculas (lecturas) que sustentan la medida.

y los contrastes en la escala M ya no tratan igual una diferencia de 0,05 en torno a $\beta = 0,5$ que en torno a $\beta = 0,97$, donde la segunda es mucho más significativa biológica y estadísticamente. Du et al. (2010) concluyeron que β es preferible para informar resultados y M para el análisis diferencial en micromatrices, cuya varianza técnica es más homogénea en esa escala. En WGBS, donde los datos son conteos, lo más limpio es modelar directamente la binomial, como veremos enseguida.

13.2.5 Metilación diferencial: el modelo beta-binomial

Al comparar dos grupos (tumor y tejido normal, tratados y controles) buscamos CpG individuales (*differentially methylated loci*, DML) o regiones (DMR) cuyo nivel de metilación difiera. El modelo binomial captura el muestreo de lecturas, pero no la variación *biológica* entre réplicas: dos individuos sanos no tienen exactamente la misma β en un CpG. Ignorar esa variabilidad subestima la varianza y produce avalanchas de falsos positivos, exactamente el mismo fenómeno de sobredispersión que en RNA-seq obliga a pasar de la Poisson a la binomial negativa (capítulo 11).

Teorema 13.3. (Varianza beta-binomial)

Si en la réplica r la fracción de metilación es $p_r \sim \text{Beta}(a, b)$ con media $\mu = a/(a + b)$ y dispersión $\phi = 1/(a + b + 1)$, y $c_r \mid p_r \sim \text{Bin}(n_r, p_r)$, entonces

$$\mathbb{E}[c_r] = n_r \mu, \quad \text{Var}(c_r) = n_r \mu(1 - \mu) [1 + (n_r - 1) \phi]. \quad (13.13)$$

DONDE

c_r, n_r	Lecturas metiladas y cobertura total del CpG en la réplica r .
p_r	Fracción de metilación verdadera de la réplica (varía entre individuos).
μ	Media de metilación del grupo en ese CpG.
$\phi \in [0, 1)$	Dispersión: correlación entre dos lecturas de la misma réplica; $\phi = 0$ recupera la binomial.

La demostración usa la ley de la varianza total: $\text{Var}(c) = \mathbb{E}[\text{Var}(c \mid p)] + \text{Var}(\mathbb{E}[c \mid p]) = n\mathbb{E}[p(1 - p)] + n^2 \text{Var}(p)$. Con $\mathbb{E}[p(1 - p)] = \mu(1 - \mu) - \text{Var}(p)$ y $\text{Var}(p) = \mu(1 - \mu)\phi$ para la beta, se obtiene $n\mu(1 - \mu)(1 - \phi) + n^2\mu(1 - \mu)\phi$, que es la ecuación (13.13). El factor $1 + (n - 1)\phi$ tiene una lectura importante: *aumentar la cobertura no elimina la variación biológica*. Con $n = 20$ y $\mu = 0,85$, la desviación estándar de c es 1,60 si $\phi = 0$, pero 2,23 con $\phi = 0,05$ y 2,72 con $\phi = 0,1$.

El paquete DSS de Feng et al. (2014) usa este modelo con una idea adicional: como con pocas réplicas la dispersión de cada CpG se estima muy mal, le asigna una distribución a priori lognormal común a todos los CpG, de modo que cada estimación se “encoge” hacia el valor típico del genoma (compartir información

entre sitios). Con la dispersión estimada, contrasta la diferencia de medias con una prueba de Wald:

$$z = \frac{\hat{\mu}_A - \hat{\mu}_B}{\sqrt{\widehat{\text{Var}}(\hat{\mu}_A) + \widehat{\text{Var}}(\hat{\mu}_B)}}, \quad \widehat{\text{Var}}(\hat{\mu}_g) \approx \frac{1}{R_g^2} \sum_{r=1}^{R_g} \frac{\hat{\mu}_g(1 - \hat{\mu}_g)[1 + (n_r - 1)\hat{\phi}]}{n_r}, \quad (13.14)$$

DONDE

$\hat{\mu}_g$	Metilación estimada del grupo g : $\sum_r c_r / \sum_r n_r$.
R_g	Número de réplicas del grupo g .
$\hat{\phi}$	Dispersión estimada (encogida) del CpG.
z	Estadístico de Wald; bajo H_0 sigue aproximadamente una normal estándar.

Ejemplo 13.3. (Un CpG, tres réplicas por grupo)

Primero, una sola muestra: 17 de 20 lecturas dicen “C”. Entonces $\hat{\beta} = 0,850$, con error estándar binomial 0,080. El intervalo de Wald al 95 %, (0,694; 1,006), se sale de $[0, 1]$: con proporciones cercanas a los extremos conviene un intervalo bayesiano de Jeffreys (a priori Beta($\frac{1}{2}, \frac{1}{2}$)), que da (0,651; 0,956). El valor M , con $\alpha' = 1$, es $\log_2(18/4) = 2,17$.

Ahora, el grupo A tiene conteos 17/20, 22/25 y 14/18, y el grupo B, 6/21, 9/24 y 5/19. Las medias son $\hat{\mu}_A = 53/63 = 0,841$ y $\hat{\mu}_B = 20/64 = 0,312$. Con $\hat{\phi} = 0,05$, la ecuación (13.14) da $\widehat{\text{Var}}(\hat{\mu}_A) = 0,00428$, $\widehat{\text{Var}}(\hat{\mu}_B) = 0,00680$ y

$$z = \frac{0,841 - 0,312}{\sqrt{0,00428 + 0,00680}} = 5,02, \quad p = 5,1 \times 10^{-7}.$$

Si ignoráramos la dispersión ($\phi = 0$) obtendríamos $z = 7,10$ y $p = 1,3 \times 10^{-12}$: cinco órdenes de magnitud de “evidencia” que no existen. Con $\phi = 0,1$, $z = 4,10$; con $\phi = 0,2$, $z = 3,18$ y $p = 0,0015$. La conclusión depende críticamente de cuánto varían los individuos, y por eso las réplicas biológicas son irrenunciables.

De sitios a regiones. Los CpG vecinos están fuertemente correlacionados: la metilación cambia por bloques de cientos de pares de bases, no sitio a sitio. Los métodos de DMR aprovechan esa estructura suavizando los niveles de metilación a lo largo del genoma (lo que además “presta” cobertura de los vecinos a los CpG poco cubiertos)

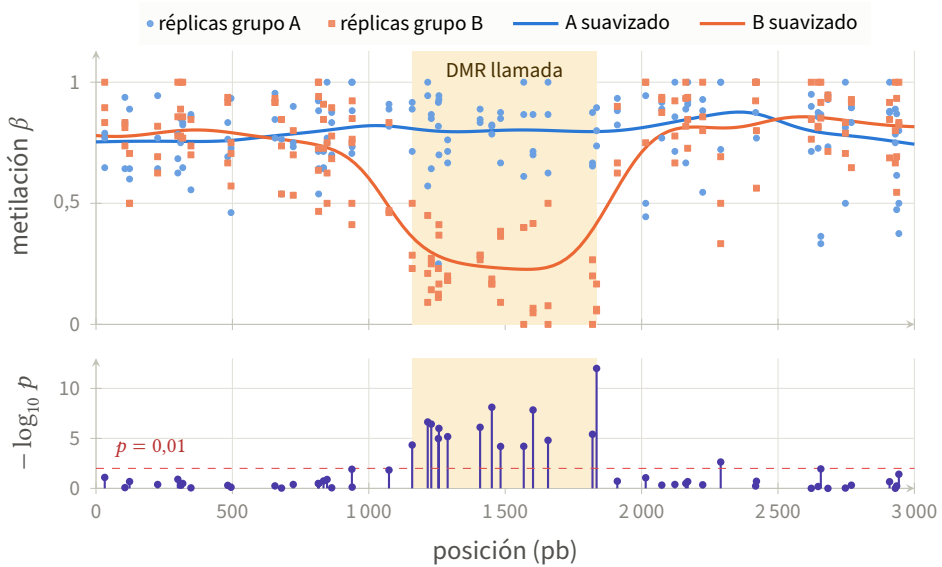


Figura 13.9. Región diferencialmente metilada simulada: 55 CpG en 3 kb, tres réplicas por grupo, cobertura media de 14 lecturas y datos beta-binomiales con $\phi = 0,05$. **Arriba:** niveles por réplica (puntos) y curvas suavizadas con un núcleo gaussiano de 150 pb. El grupo B pierde la metilación en un bloque central, como ocurre en la hipometilación de un potenciador activado. **Abajo:** significancia de la prueba de Wald beta-binomial por CpG (ecuación (13.14)). La banda amarilla es la DMR llamada como el tramo contiguo más largo de CpG con $p < 0,01$. Los puntos individuales son muy ruidosos; la región, en cambio, es inequívoca.

y agrupando CpG significativos contiguos en regiones, con requisitos mínimos de longitud, número de CpG y diferencia media. La figura 13.9 muestra una simulación de 55 CpG en 3 kb: la prueba de Wald con $\phi = 0,05$ declara significativos ($p < 0,01$) 15 CpG, de los que 14 están dentro de la DMR verdadera (1100 pb a 1900 pb); el tramo contiguo más largo de CpG significativos, de 1159 a 1835 pb, recupera la región casi exactamente.

13.2.6 Relojes epigenéticos y metilación sin bisulfito

La metilación cambia con la edad de manera tan regular que puede usarse como reloj. Horvath (2013) entrenó un predictor multitejido con unas 8000 muestras de micromatrices de 82 conjuntos de datos y 51 tejidos y tipos celulares sanos: una regresión penalizada (red elástica) seleccionó 353 CpG cuya combinación lineal, tras

una transformación de la edad logarítmica en la infancia y lineal en la adultez, estima la “edad epigenética”. Esta edad es cercana a cero en células madre embrionarias e inducidas, aumenta con el número de pases en cultivo y aparece acelerada en muchos tipos de cáncer. La diferencia entre la edad epigenética y la cronológica, la *aceleración*, se ha convertido en un biomarcador muy estudiado.

Nanoporos. La secuenciación por nanoporos (capítulo 6) lee la molécula nativa, sin amplificar, y la corriente iónica depende de las bases que ocupan el poro, incluidas las modificadas. Simpson et al. (2017) entrenaron un modelo oculto de Márkov con ADN metilado sintéticamente para distinguir 5mC de citosina en la señal eléctrica y obtuvieron el metiloma humano sin ningún paso especial de preparación. Sin bisulfito no hay degradación del ADN ni pérdida de complejidad, las lecturas largas permiten “fasear” la metilación con los alelos, y la misma corrida entrega secuencia y epigenoma.

Trampas de la metilación

1. *C→T que no es bisulfito.* Un SNP C/T en la muestra es indistinguible de una C no metilada en la hebra correspondiente; hay que filtrar posiciones polimórficas o mirar la hebra opuesta (donde el SNP aparece como G/A y el bisulfito no actúa).
2. *Composición celular.* Una diferencia de metilación entre tejidos enfermos y sanos puede reflejar solo una proporción distinta de tipos celulares. Deconvolucionar o ajustar por composición es obligatorio en muestras de sangre y tumores.

El bisulfito convierte una marca química invisible en una diferencia de secuencia (C frente a T). La metilación de un sitio es una proporción de lecturas, y su comparación entre grupos exige un modelo que separe el ruido de muestreo (binomial) de la variación biológica (dispersión beta).

13.3 Descubrimiento de motivos regulatorios

La contraseña escondida en treinta cartas

Suponga que intercepta treinta cartas escritas en un alfabeto de cuatro letras, casi todas con texto sin sentido. Le han dicho que en cada una hay escondida, en algún lugar, una misma contraseña de ocho letras, pero copiada con errores. Si supiera dónde está en cada carta, bastaría alinear los fragmentos y contar letras. Si conociera la contraseña, bastaría buscarla en cada carta. No sabe ninguna de las dos cosas. La salida es circular pero funciona: adivine una contraseña aproximada, busque en cada carta el fragmento que más se le parece, reconstruya la contraseña a partir de esos fragmentos, y repita. Cada vuelta mejora un poco las dos conjeturas a la vez, hasta que dejan de cambiar.

Un pico de ChIP-seq indica *dónde* se une un factor, pero con una resolución de decenas o cientos de pares de bases. La especificidad de la unión reside en un segmento corto de 6 a 20 pb, el *motivo*, que el capítulo 4 representó con una matriz de peso posicional (PWM) y un *sequence logo*. Allí la matriz se construía a partir de sitios conocidos. Aquí el problema es el inverso: dados cientos o miles de secuencias que *probablemente* contienen el motivo en alguna posición desconocida, aprender simultáneamente la matriz y las posiciones. Es un ejemplo de manual de problema con *datos incompletos*.

13.3.1 El modelo generativo

Sean $X_1, \dots, X_N$ secuencias de longitud L y W el ancho del motivo; cada secuencia tiene $m = L - W + 1$ posiciones de inicio posibles. El modelo más sencillo, llamado OOPS (*one occurrence per sequence*), supone que cada secuencia contiene exactamente un sitio, cuyas W bases se generan con la matriz θ (una distribución sobre $\{A, C, G, T\}$ por columna), mientras que el resto de las bases proviene de un fondo p_0 . Introducimos variables *ocultas* $Z_{ij} \in \{0, 1\}$ que valen 1 si el sitio de la secuencia i empieza en la posición j .

Definición 13.6. (Verosimilitud del modelo OOPS)

Con un inicio uniforme ($\mathbb{P}(Z_{ij} = 1) = 1/m$), la razón de verosimilitudes de la secuencia X_i frente al modelo de fondo puro es

$$\frac{\mathbb{P}(X_i | \theta)}{\mathbb{P}(X_i | p_0)} = \frac{1}{m} \sum_{j=1}^m r_{ij}(\theta), \quad r_{ij}(\theta) = \prod_{k=1}^W \frac{\theta_{k, x_{i,j+k-1}}}{p_0(x_{i,j+k-1})}, \quad (13.15)$$

y la log-verosimilitud del conjunto, en bits, es $\text{LLR}(\theta) = \sum_i \log_2 \left(\frac{1}{m} \sum_j r_{ij}(\theta) \right)$.

DONDE

$x_{i,j}$	Base en la posición j de la secuencia i .
$\theta_{k,b}$	Probabilidad de la base b en la columna k del motivo.
$p_0(b)$	Frecuencia de fondo de la base b .
r_{ij}	Razón de verosimilitudes de que el sitio esté en j frente a que sea fondo; $\log_2 r_{ij}$ es la puntuación PWM del capítulo 4.
m	Número de posiciones de inicio posibles, $L - W + 1$.

Las bases fuera del sitio se cancelan en el cociente, así que solo quedan las W columnas del candidato. Si conociéramos las Z_{ij} , estimar θ sería contar; si conociéramos θ , localizar los sitios sería escanear con la PWM. El algoritmo EM convierte esta circularidad en un procedimiento con garantías.

13.3.2 El algoritmo EM y MEME

Dempster et al. (1977) formularon el algoritmo de esperanza-maximización (EM) en general para problemas con datos faltantes. Aplicado a motivos alterna dos pasos:

Teorema 13.4. (EM para el modelo OOPS)

Dada una estimación $\theta^{(t)}$:

$$\text{Paso E: } Z_{ij}^{(t)} = \mathbb{P}(Z_{ij} = 1 \mid X_i, \theta^{(t)}) = \frac{r_{ij}(\theta^{(t)})}{\sum_{j'=1}^m r_{ij'}(\theta^{(t)})}, \quad (13.16)$$

$$\text{Paso M: } \theta_{k,b}^{(t+1)} = \frac{\sum_i \sum_j Z_{ij}^{(t)} \mathbb{1}[x_{i,j+k-1} = b] + \beta_b}{\sum_{b'} (\sum_i \sum_j Z_{ij}^{(t)} \mathbb{1}[x_{i,j+k-1} = b'] + \beta_{b'})}. \quad (13.17)$$

Con $\beta_b = 0$, cada iteración no disminuye la verosimilitud: $\text{LLR}(\theta^{(t+1)}) \geq \text{LLR}(\theta^{(t)})$.

DONDE

$Z_{ij}^{(t)}$	Probabilidad posterior de que el sitio de la secuencia i empiece en j ; es un “conteo fraccionario”.
β_b	Seudoconteos (a priori de Dirichlet) que evitan probabilidades nulas.
$\mathbb{1}[\cdot]$	Función indicadora.

El paso E reparte cada secuencia entre sus posiciones posibles en proporción a cuánto se parecen al motivo actual: en vez de decidir dónde está el sitio, reparte “votos”. El paso M construye una nueva PWM con esos votos fraccionarios. La garantía de monotonía viene de la desigualdad de Jensen: para cualquier distribución q sobre las variables ocultas, $\log \mathbb{P}(X \mid \theta) \geq \sum_Z q(Z) \log \frac{\mathbb{P}(X, Z \mid \theta)}{q(Z)}$, con igualdad cuando q es la posterior. El paso E hace la cota exacta en $\theta^{(t)}$; el paso M la maximiza en θ ; por tanto la verosimilitud en $\theta^{(t+1)}$ es al menos la cota, que es al menos la verosimilitud en $\theta^{(t)}$. Lo que EM *no* garantiza es el máximo global.

Ejemplo 13.4. (Un paso E)

En una colección simulada de 30 secuencias de 80 pb con un motivo de tipo AP-1 implantado (consenso **ATGASTCA**, $W = 8$; hay $m = 73$ posiciones por secuencia), la secuencia 18 contiene el sitio **ATGACTCA**, idéntico al consenso. Con la matriz verdadera y fondo uniforme, su puntuación es

$$\log_2 r = 1,26 + 1,77 + 1,77 + 1,82 + 0,91 + 1,77 + 1,77 + 1,82 \approx 12,87 \text{ bits,}$$

es decir, $r \approx 7500$: el sitio es unas siete mil quinientas veces más probable bajo el motivo que bajo el fondo. Al normalizar sobre las 73 posiciones, el paso E le asigna $Z = 0,9976$ y reparte el 0,24 % restante entre las otras 72. Pero no todos los sitios son tan claros: la mediana de las puntuaciones de los 30 sitios reales es de 8,65 bits, y en las secuencias con sitios degenerados el paso E reparte el voto entre varias posiciones. Cambiar una sola base (ATGACTTA) baja la puntuación a 8,78 bits.

```
em_oops.py

import numpy as np

def em_oops(seqs, theta, iters=30, beta=0.1):
    """seqs: matriz N x L de enteros 0-3; theta: W x 4 inicial."""
    N, L = seqs.shape
    W = theta.shape[0]
    win = np.stack([seqs[:, j:j + W] for j in range(L - W + 1)], 1)
    for _ in range(iters):
        lr = np.log2(theta[np.arange(W), win]).sum(-1) + 2 * W
        Z = np.exp2(lr - lr.max(1, keepdims=True))      # paso E
        Z /= Z.sum(1, keepdims=True)
        cnt = np.stack([(Z[..., None] * (win == b)).sum((0, 1))
                        for b in range(4)], 1)          # paso M
        theta = (cnt + beta) / (cnt + beta).sum(1, keepdims=True)
    return theta, Z
```

MEME. El programa MEME, presentado por Bailey y Elkan en 1994 y hoy núcleo de la MEME Suite (Bailey et al., 2009), es la implementación de referencia de esta idea. Añade tres ingredientes decisivos. Primero, modelos más realistas que OOPS: ZOOPS (*zero or one occurrence per sequence*), que admite secuencias sin sitio, y TCM (*two-component mixture*), que admite cualquier número. Segundo, una estrategia de *inicialización*: en lugar de partir de una matriz aleatoria, prueba como semillas subcadenas reales de los datos, corre una iteración corta desde cada una y lanza EM completo solo desde las más prometedoras. Tercero, un criterio estadístico (el valor E del motivo) para decidir cuántos motivos reportar y de qué ancho.

La figura 13.10 (a) muestra por qué la inicialización importa. Seis corridas de EM sobre el mismo conjunto, cinco con semillas aleatorias y una con una subcadena de la secuencia, terminan en tres destinos. Dos alcanzan la verosimilitud máxima encontrada, LLR = 96,4 bits, con el consenso ATGAGTCA (la S del motivo admite C o

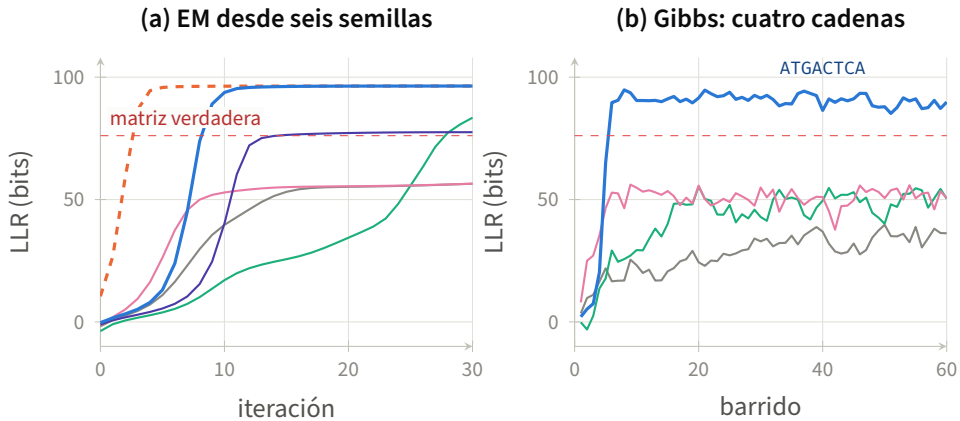


Figura 13.10. Descubrimiento de un motivo de 8 pb implantado en 30 secuencias de 80 pb. (a) Log-verosimilitud (ecuación (13.15)) a lo largo de las iteraciones de EM desde seis puntos de partida: cinco semillas aleatorias y una subcadena de los datos (naranja discontinua), como hace MEME. Todas las curvas son crecientes, como garantiza el teorema 13.4, pero terminan en óptimos distintos; la mejor corrida con semilla aleatoria (azul) muestra la típica curva en S. (b) Cuatro cadenas del muestreador de Gibbs: la trayectoria es ruidosa porque cada paso es aleatorio. Una cadena (azul) encuentra el motivo en seis barridos; dos quedan desfasadas dos posiciones y una no encuentra nada. La línea roja es la verosimilitud de la matriz verdadera.

G). Otras quedan atrapadas en versiones *desfasadas* del motivo (**T**G**A**G**T**C**A**T, **C**A**T**G**A**C**T**C), que capturan solo siete de las ocho columnas, o en un óptimo local pobre (LLR = 56,5). La curva de la corrida ganadora tiene forma de S: durante cinco iteraciones apenas mejora, hasta que un puñado de sitios reales “tira” de la matriz y el resto la sigue en cascada. En esa corrida, 26 de los 30 sitios implantados se recuperan en la posición exacta, y la matriz final tiene 8,75 bits de información, *más* que la verdadera (8,11): con solo 30 sitios, la máxima verosimilitud sobreajusta y hace el motivo más nítido de lo que es. Por la misma razón, la LLR de la solución (96,4 bits) supera a la de la matriz verdadera (76,1).

13.3.3 Muestreo de Gibbs

Lawrence et al. (1993) abordaron el mismo problema con una estrategia estocástica, el *muestreador de sitios* de Gibbs. En lugar de probabilidades fraccionarias, mantiene una asignación concreta $a = (a_1, \dots, a_N)$ de la posición del sitio en cada secuencia y la actualiza de una en una:

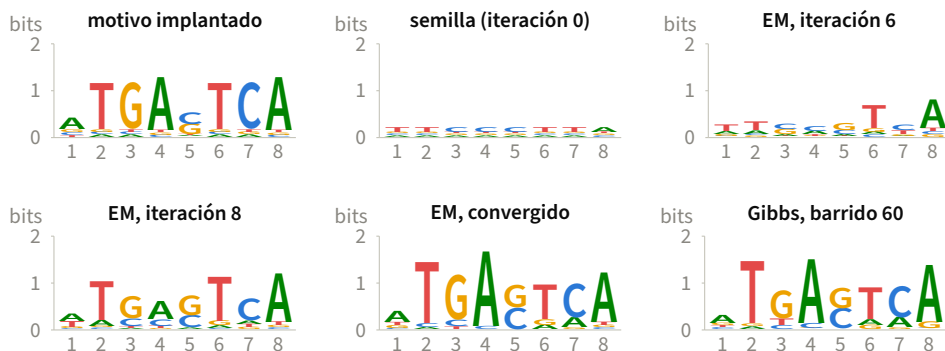


Figura 13.11. Un motivo emerge del ruido. **Arriba, izquierda:** logo de la matriz usada para implantar los sitios (8,11 bits en total; la columna 5 es una S = C/G). Los otros cinco logos siguen la corrida ganadora de EM desde una semilla aleatoria (TTCCCTTA): la semilla apenas tiene 1,66 bits; en la iteración 6 empiezan a asomar la G, la A y la T centrales (3,19 bits); en la iteración 8 el motivo ya es reconocible (6,39 bits), y al converger (8,75 bits) coincide con el implantado. **Abajo, derecha:** la mejor cadena de Gibbs tras 60 barridos llega al mismo motivo por un camino estocástico. Alturas en bits según la ecuación del logo del capítulo 4.

1. Elija una secuencia z y retire su sitio del alineamiento.
2. Construya la matriz con los $N - 1$ sitios restantes y pseudoconteos:

$$q_{k,b} = \frac{\sum_{i \neq z} \mathbb{1}[x_{i,a_i+k-1} = b] + \beta_b}{N - 1 + \sum_{b'} \beta_{b'}}. \quad (13.18)$$

3. Muestree una nueva posición para z de la distribución condicional

$$P(a_z = j \mid a_{-z}, X) \propto \prod_{k=1}^W \frac{q_{k, x_{z,j+k-1}}}{p_0(x_{z,j+k-1})}, \quad j = 1, \dots, m. \quad (13.19)$$

4. Repita para todas las secuencias (un *barrido*) hasta que la verosimilitud se estabilice.

DONDE

a_i	Posición asignada al sitio de la secuencia i .
a_{-z}	Posiciones de todas las secuencias excepto z .
$q_{k,b}$	Matriz estimada sin la secuencia z .

La diferencia con EM es de fondo. EM es determinista y sube siempre por la pendiente más cercana; Gibbs es una cadena de Márkov cuya distribución estacionaria es la posterior de las posiciones, así que puede *bajar* momentáneamente y saltar a otra colina. En la práctica, las dos estrategias tienen los mismos enemigos: los óptimos locales y, sobre todo, los *desfases* (*phase shifts*), en los que el alineamiento captura el motivo corrido una o dos posiciones. Los dos desfases de la figura 13.10 (b) (ACATGAGT y CCATGAGT) contienen seis columnas buenas de ocho y son muy estables, porque mover un único sitio no mejora nada; hace falta desplazar todos a la vez. Lawrence et al. (1993) propusieron por ello movimientos que desplazan todo el alineamiento, y la práctica habitual es lanzar muchas cadenas y quedarse con la mejor. En nuestra simulación, la cadena ganadora alcanzó el 95 % de su verosimilitud final en el sexto barrido y colocó 22 de los 30 sitios en la posición exacta.

13.3.4 De motivos nuevos a factores conocidos: bases de datos y enriquecimiento

Un motivo descubierto *de novo* es solo una matriz. Para ponerle nombre se compara con bases de datos de perfiles de unión. JASPAR, en su novena versión, es la colección abierta de referencia de perfiles de factores de transcripción curados y no redundantes (Castro-Mondragon et al., 2022); la comparación entre matrices (por ejemplo, con la herramienta Tomtom de la MEME Suite, que puntúa la similitud columna a columna considerando desplazamientos y la hebra complementaria) devuelve los factores conocidos más parecidos con un valor de significancia (Bailey et al., 2009).

Enriquecimiento diferencial. Una pregunta distinta, y a menudo más útil, es cuál de los miles de motivos conocidos aparece *más* en los picos que en secuencias de fondo comparables. HOMER (Heinz et al., 2010) popularizó este enfoque con un fondo de secuencias genómicas aleatorias emparejadas por contenido de GC con los picos, lo que evita que cualquier motivo rico en GC parezca enriquecido solo porque los promotores lo son. Con ese enfoque, Heinz et al. (2010) mostraron que el factor PU.1 colabora con pequeños conjuntos de factores determinantes de linaje para establecer los potenciadores propios de macrófagos y linfocitos B. La significancia se calcula con la distribución hipergeométrica:

$$p = \mathbb{P}(K \geq k_t) = \sum_{k \geq k_t} \frac{\binom{k_t + k_b}{k} \binom{n_t + n_b - k_t - k_b}{n_t - k}}{\binom{n_t + n_b}{n_t}}, \quad (13.20)$$

DONDE

n_t, k_t	Número de secuencias objetivo (picos) y cuántas contienen el motivo.
n_b, k_b	Número de secuencias de fondo y cuántas lo contienen.
K	Número de secuencias con motivo que caerían en el conjunto objetivo si la asignación fuera al azar.

es decir, la probabilidad de que, repartiendo al azar las $k_t + k_b$ secuencias con motivo entre objetivo y fondo, al menos k_t caigan en el objetivo.

Ejemplo 13.5. (Un motivo enriquecido)

De 2000 picos, 640 (32 %) contienen el motivo; de 20 000 secuencias de fondo emparejadas por GC, lo contienen 1200 (6 %). El enriquecimiento es $0,32/0,06 = 5,33$ veces, y la ecuación (13.20) da $p = 6,8 \times 10^{-234}$ ($\ln p = -536,9$, la escala que HOMER reporta). La aproximación binomial, que trata el fondo como una proporción conocida sin error, da un valor aún menor (2×10^{-276}). Valores tan extremos son habituales y no deben leerse literalmente: con miles de secuencias, cualquier sesgo sistemático de composición da p astronómicos. Lo que importa es el *orden* de los motivos y el tamaño del enriquecimiento.

13.3.5 Centralidad: ¿unión directa o indirecta?

Un motivo enriquecido en los picos no es necesariamente el del factor inmunoprecipitado: puede ser el de un cofactor que se une cerca, o reflejar un sesgo de las regiones abiertas. Bailey y Machanick (2012) observaron que, si el factor se une *directamente* al ADN, sus sitios deben concentrarse en el centro de los picos (la posición del *summit*), mientras que los de cofactores se distribuyen más anchos y los del fondo, uniformemente. Su herramienta CentriMo busca la ventana central donde el enriquecimiento es máximo y lo contrasta con una prueba binomial: si una ventana de ancho w_c ocupa una fracción $f = w_c/(L - W + 1)$ de las posiciones posibles, el número de mejores sitios que caen dentro sigue, bajo la hipótesis nula, una $\text{Bin}(n, f)$. Así mostraron, por ejemplo, que en un conjunto de datos de Smad1 no había evidencia de unión directa del factor inmunoprecipitado.

Análisis de motivos a partir de picos

100 pb alrededor de la cumbre de cada pico

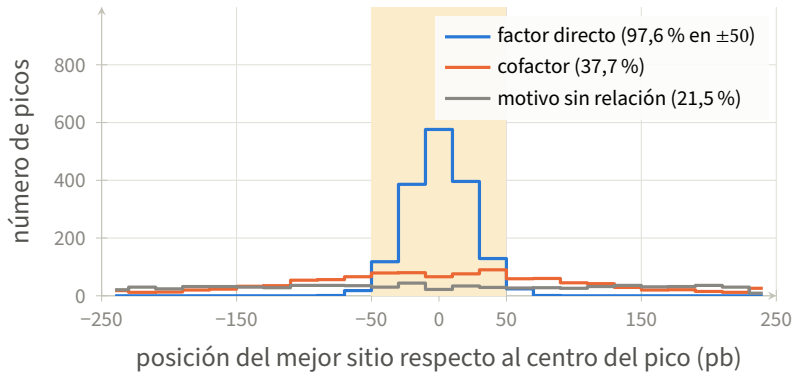


Figura 13.12. Análisis de centralidad al estilo de CentriMo sobre 3000 picos simulados de 500 pb. Los mejores sitios del motivo del factor inmunoprecipitado (azul) se concentran en los 100 pb centrales (banda amarilla), donde cae el 97,6 % de ellos frente al 21 % esperado; la prueba binomial da un valor p numéricamente nulo. El motivo de un cofactor (naranja) está enriquecido, pero de forma mucho más ancha ($p = 2,9 \times 10^{-35}$), y un motivo sin relación se distribuye de forma plana ($p = 0,39$). La centralidad distingue unión directa de indirecta, algo que el simple enriquecimiento no puede hacer.

```
awk -v OFS='\t' '{s=$2+$10-50; print $1,s,s+100}' \
ctcf_idr.narrowPeak > cumbres.bed
bedtools getfasta -fi hg38.fa -bed cumbres.bed -fo cumbres.fa
# MEME Suite: de novo + enriquecimiento + centralidad
meme-chip -oc memechip -meme-nmotifs 5 \
-db JASPAR2022_CORE_vertbrates.meme cumbres.fa
# HOMER: fondo emparejado por GC
findMotifsGenome.pl cumbres.bed hg38 homer_out -size given
```

⚠ Motivos que engañan

1. *Baja complejidad y repeticiones.* Tramos como CACACACA o poli-A se “descubren” con enorme significancia; enmascare repeticiones y use modelos de fondo de orden superior.
2. *Sesgo de GC.* Los promotores son ricos en GC; sin un fondo emparejado, cualquier motivo rico en GC parecerá enriquecido.

3. *Presencia no es ocupación.* Un genoma humano contiene cientos de miles de coincidencias de casi cualquier PWM; la inmensa mayoría no está ocupada, porque la accesibilidad de la cromatina (sección 13.1) decide.

Descubrir motivos es estimar una PWM con datos incompletos: EM reparte votos fraccionarios y sube por la verosimilitud; Gibbs muestrea posiciones y puede escapar de óptimos locales. Ninguno garantiza el óptimo global: múltiples arranques, fondos emparejados y el criterio de centralidad convierten una matriz en una hipótesis biológica.

☰ En resumen

- En ChIP-seq, las lecturas de las hebras + y – flanquean el sitio a una distancia igual al tamaño de fragmento, que se estima por correlación cruzada; NSC y RSC comparan el pico de fragmento con el pico fantasma de mapabilidad.
- MACS llama picos con una prueba de Poisson cuya media es el máximo entre el fondo global y varias estimaciones locales del control; FRiP, la lista negra de ENCODE e IDR evalúan la señal y la reproducibilidad.
- ATAC-seq mide la accesibilidad con Tn5; un buen experimento muestra fragmentos libres de nucleosomas, jorobas nucleosomales, periodicidad helicoidal y un fuerte enriquecimiento en el TSS. CUT&RUN reduce el fondo llevando la nucleasa al factor.
- El bisulfito convierte C no metiladas en T; Bismark alinea en un alfabeto reducido. La metilación es una proporción (β), su logit es el valor M , y la comparación entre grupos requiere un modelo beta-binomial que incluya la variación biológica.
- El descubrimiento de motivos se formula como un problema de datos incompletos: EM (MEME) y muestreo de Gibbs aprenden a la vez la PWM y las posiciones; el enriquecimiento diferencial (HOMER) y la centralidad (CentriMo) identifican los factores responsables.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 13.1** ●●● Una lectura de 50 pb de la hebra – tiene su extremo 5' en la coordenada 10 480. Si el tamaño de fragmento estimado es $d = 180$, ¿en qué coordenada colocaría el centro del fragmento? ¿Y si la lectura fuera de la hebra + con extremo 5' en 10 480?
- 13.2** ●●● Calcule λ_{BG} para un experimento de 3×10^7 lecturas en ratón ($G = 1,87 \times 10^9$) con $d = 150$. ¿Cuántas lecturas en una ventana hacen falta para $p < 10^{-5}$? Compruébelo con `scipy.stats.poisson.sf`.
- 13.3** ●●● Demuestre que, si $M = \log_2 \frac{\beta}{1-\beta}$, entonces $\beta = \frac{2^M}{1+2^M}$, y use el método delta para mostrar que $\text{Var}(\hat{M})$ es mínima en $\beta = 0,5$. ¿Qué valor mínimo toma con $n = 30$ lecturas?
- 13.4** ●●● Simule 10 000 CpG con $\mu = 0,7$, tres réplicas por grupo y cobertura 20, bajo la hipótesis nula (ambos grupos iguales) con $\phi = 0,1$. Aplique la prueba de Wald con $\phi = 0$ y con $\phi = 0,1$. ¿Qué fracción de valores p cae por debajo de 0,05 en cada caso? Relacione el resultado con la ecuación (13.13).
- 13.5** ●●● Modifique `em_oops` para el modelo ZOOPS añadiendo, para cada secuencia, un estado “sin sitio” con probabilidad a priori $1 - \gamma$. Escriba el paso E y el paso M para γ .
- 13.6** ●●● Implemente el muestreador de Gibbs de las ecuaciones (13.18) y (13.19) y añada un movimiento de desfase que proponga desplazar todas las posiciones ± 1 y lo acepte según la razón de verosimilitudes. Compare, con 20 cadenas, la fracción que alcanza el motivo correcto con y sin ese movimiento.
- 13.7** ●●● Demuestre la desigualdad que sustenta la monotonía de EM: para cualquier distribución $q(Z)$, $\log P(X | \theta) = \mathcal{L}(q, \theta) + \text{KL}(q \| P(Z | X, \theta))$, donde $\mathcal{L}$ es la cota de Jensen. Deduzca que el paso E anula la divergencia y que el paso M no puede disminuir la verosimilitud.
- 13.8** ●●● Descargue de ENCODE los picos IDR de CTCF en una línea celular, extraiga ± 50 pb alrededor de cada cumbre y ejecute `meme-chip`. ¿Coincide el motivo principal con el perfil MAO139 de JASPAR? ¿Qué fracción de los picos lo contiene y qué muestra el análisis de centralidad?

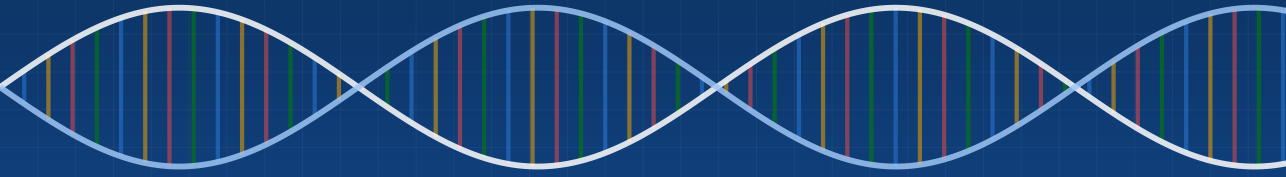
Referencias del capítulo

- Amemiya, H. M., Kundaje, A., & Boyle, A. P. (2019). The ENCODE blacklist: Identification of problematic regions of the genome. *Scientific Reports*, 9(1), 9354. <https://doi.org/10.1038/s41598-019-45839-z>
- Bailey, T. L., Boden, M., Buske, F. A., Frith, M., Grant, C. E., Clementi, L., et al. (2009). MEME SUITE: Tools for motif discovery and searching. *Nucleic Acids Research*, 37(Web Server), W202-W208. <https://doi.org/10.1093/nar/gkp335>
- Bailey, T. L., & Machanick, P. (2012). Inferring direct DNA binding from ChIP-seq. *Nucleic Acids Research*, 40(17), e128. <https://doi.org/10.1093/nar/gks433>
- Bird, A. P. (1986). CpG-rich islands and the function of DNA methylation. *Nature*, 321(6067), 209-213. <https://doi.org/10.1038/321209a0>
- Buenrostro, J. D., Giresi, P. G., Zaba, L. C., Chang, H. Y., & Greenleaf, W. J. (2013). Transposition of native chromatin for fast and sensitive epigenomic profiling of open chromatin, DNA-binding proteins and nucleosome position. *Nature Methods*, 10(12), 1213-1218. <https://doi.org/10.1038/nmeth.2688>
- Buenrostro, J. D., Wu, B., Chang, H. Y., & Greenleaf, W. J. (2015). ATAC-seq: A method for assaying chromatin accessibility genome-wide. *Current Protocols in Molecular Biology*, 109(1), 21.29.1-21.29.9. <https://doi.org/10.1002/0471142727.mb2129s109>
- Castro-Mondragon, J. A., Riudavets-Puig, R., Rauluseviciute, I., Berhanu Lemma, R., Turchi, L., Blanc-Mathieu, R., et al. (2022). JASPAR 2022: The 9th release of the open-access database of transcription factor binding profiles. *Nucleic Acids Research*, 50(D1), D165-D173. <https://doi.org/10.1093/nar/gkab1113>
- Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1), 1-22. <https://doi.org/10.1111/j.2517-6161.1977.tb01600.x>
- Du, P., Zhang, X., Huang, C.-C., Jafari, N., Kibbe, W. A., Hou, L., et al. (2010). Comparison of Beta-value and M-value methods for quantifying methylation levels by microarray analysis. *BMC Bioinformatics*, 11(1), 587. <https://doi.org/10.1186/1471-2105-11-587>
- ENCODE Project Consortium. (2012). An integrated encyclopedia of DNA elements in the human genome. *Nature*, 489(7414), 57-74. <https://doi.org/10.1038/nature11247>
- ENCODE Project Consortium, Abascal, F., Acosta, R., Addleman, N. J., Adrian, J., Afzal, V., et al. (2020). Expanded encyclopaedias of DNA elements in the human and mouse genomes. *Nature*, 583(7818), 699-710. <https://doi.org/10.1038/s41586-020-2493-4>
- Feng, H., Conneely, K. N., & Wu, H. (2014). A Bayesian hierarchical model to detect differentially methylated loci from single nucleotide resolution sequencing data. *Nucleic Acids Research*, 42(8), e69. <https://doi.org/10.1093/nar/gku154>
- Frommer, M., McDonald, L. E., Millar, D. S., Collis, C. M., Watt, F., Grigg, G. W., et al. (1992). A genomic sequencing protocol that yields a positive display of 5-methylcytosine

- residues in individual DNA strands. *Proceedings of the National Academy of Sciences*, 89(5), 1827-1831. <https://doi.org/10.1073/pnas.89.5.1827>
- Gardiner-Garden, M., & Frommer, M. (1987). CpG islands in vertebrate genomes. *Journal of Molecular Biology*, 196(2), 261-282. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9)
- Heinz, S., Benner, C., Spann, N., Bertolino, E., Lin, Y. C., Laslo, P., et al. (2010). Simple combinations of lineage-determining transcription factors prime cis-regulatory elements required for macrophage and B cell identities. *Molecular Cell*, 38(4), 576-589. <https://doi.org/10.1016/j.molcel.2010.05.004>
- Horvath, S. (2013). DNA methylation age of human tissues and cell types. *Genome Biology*, 14(10), R115. <https://doi.org/10.1186/gb-2013-14-10-r115>
- Jenuwein, T., & Allis, C. D. (2001). Translating the histone code. *Science*, 293(5532), 1074-1080. <https://doi.org/10.1126/science.1063127>
- Johnson, D. S., Mortazavi, A., Myers, R. M., & Wold, B. (2007). Genome-wide mapping of in vivo protein-DNA interactions. *Science*, 316(5830), 1497-1502. <https://doi.org/10.1126/science.1141319>
- Kharchenko, P. V., Tolstorukov, M. Y., & Park, P. J. (2008). Design and analysis of ChIP-seq experiments for DNA-binding proteins. *Nature Biotechnology*, 26(12), 1351-1359. <https://doi.org/10.1038/nbt.1508>
- Krueger, F., & Andrews, S. R. (2011). Bismark: A flexible aligner and methylation caller for Bisulfite-Seq applications. *Bioinformatics*, 27(11), 1571-1572. <https://doi.org/10.1093/bioinformatics/btr167>
- Landt, S. G., Marinov, G. K., Kundaje, A., Kheradpour, P., Pauli, F., Batzoglou, S., et al. (2012). ChIP-seq guidelines and practices of the ENCODE and modENCODE consortia. *Genome Research*, 22(9), 1813-1831. <https://doi.org/10.1101/gr.136184.111>
- Lawrence, C. E., Altschul, S. F., Boguski, M. S., Liu, J. S., Neuwald, A. F., & Wootton, J. C. (1993). Detecting subtle sequence signals: A Gibbs sampling strategy for multiple alignment. *Science*, 262(5131), 208-214. <https://doi.org/10.1126/science.8211139>
- Li, Q., Brown, J. B., Huang, H., & Bickel, P. J. (2011). Measuring reproducibility of high-throughput experiments. *The Annals of Applied Statistics*, 5(3), 1752-1779. <https://doi.org/10.1214/11-AOAS466>
- Lister, R., Pelizzola, M., Dowen, R. H., Hawkins, R. D., Hon, G., Tonti-Filippini, J., et al. (2009). Human DNA methylomes at base resolution show widespread epigenomic differences. *Nature*, 462(7271), 315-322. <https://doi.org/10.1038/nature08514>
- Luger, K., Mäder, A. W., Richmond, R. K., Sargent, D. F., & Richmond, T. J. (1997). Crystal structure of the nucleosome core particle at 2.8 Å resolution. *Nature*, 389(6648), 251-260. <https://doi.org/10.1038/38444>
- Park, P. J. (2009). ChIP-seq: Advantages and challenges of a maturing technology. *Nature Reviews Genetics*, 10(10), 669-680. <https://doi.org/10.1038/nrg2641>

- Roadmap Epigenomics Consortium, Kundaje, A., Meuleman, W., Ernst, J., Bilenky, M., Yen, A., et al. (2015). Integrative analysis of 111 reference human epigenomes. *Nature*, 518(7539), 317-330. <https://doi.org/10.1038/nature14248>
- Simpson, J. T., Workman, R. E., Zuzarte, P. C., David, M., Dursi, L. J., & Timp, W. (2017). Detecting DNA cytosine methylation using nanopore sequencing. *Nature Methods*, 14(4), 407-410. <https://doi.org/10.1038/nmeth.4184>
- Skene, P. J., & Henikoff, S. (2017). An efficient targeted nuclease strategy for high-resolution mapping of DNA binding sites. *eLife*, 6, e21856. <https://doi.org/10.7554/eLife.21856>
- Zhang, Y., Liu, T., Meyer, C. A., Eeckhoute, J., Johnson, D. S., Bernstein, B. E., et al. (2008). Model-based analysis of ChIP-Seq (MACS). *Genome Biology*, 9(9), R137. <https://doi.org/10.1186/gb-2008-9-9-r137>

Metagenómica y microbioma



Un análisis filogenético basado en la caracterización de secuencias de ARN ribosómico revela que los sistemas vivos representan una de tres líneas primordiales de descendencia.

— Woese y Fox (1977)

● Al terminar este capítulo usted podrá

- Explicar por qué el gen del ARNr 16S sirve como “código de barras” de bacterias y arqueas, y elegir una región hipervariable y un par de cebadores con criterio.
- Distinguir OTUs y ASVs, y derivar el modelo de error y la prueba de abundancia con que DADA2 separa variantes biológicas de errores de secuenciación.
- Calcular e interpretar medidas de diversidad alfa (riqueza, Shannon, Simpson, Chao1, números de Hill) y beta (Bray-Curtis, Jaccard, UniFrac), y contrastar diferencias entre grupos con PCoA y PERMANOVA.
- Reconocer que los datos de microbioma son composicionales y discutir con argumentos la controversia sobre la rarefacción.
- Describir la clasificación de lecturas por k -mers y ancestro común más bajo (Kraken 2, Bracken), el perfilado por marcadores (MetaPhlAn) y la reconstrucción de genomas a partir de metagenomas (ensamblaje, *binning*, CheckM, MIMAG).

En un gramo de heces humanas hay más células bacterianas que personas en el planeta, y la mayoría de sus especies no se ha cultivado en una placa de Petri. La secuenciación permite estudiarlas a través de su ADN, sin aislarlas previamente. La *metagenómica* analiza material genético recuperado directamente de una muestra de suelo, agua, intestino, piel u otro ambiente. El *microbioma* comprende la comunidad de microorganismos de ese ambiente, sus genes y sus funciones. Al estudiar 242 adultos sanos y hasta 18 lugares del cuerpo, Human Microbiome Project Consortium (2012) observaron una gran variación en la composición de las comunidades, junto con una mayor estabilidad de las funciones metabólicas que codifican.

Existen dos grandes estrategias para leer una comunidad microbiana. La *secuenciación de amplicones* amplifica por PCR un único gen marcador, casi siempre el 16S, y pregunta *quién está ahí*; su desafío estadístico es separar la variación biológica de los errores de la PCR y del secuenciador (sección 14.1). Con la tabla de abundancias resultante cuantificaremos cuán diversa es cada muestra y cuán distintas son entre sí, con herramientas de la ecología matemática y cuidado con la naturaleza relativa de los datos (sección 14.2). La *metagenómica shotgun* secuencia todo el ADN de la muestra: permite clasificar lecturas contra genomas de referencia, estimar abundancias de especies, inventariar genes de resistencia y reconstruir genomas de organismos nunca cultivados (sección 14.3). Usaremos ideas de la filogenia (capítulo 5), de la calidad y la cobertura (capítulo 6) y de los k -mers y grafos de de Bruijn (capítulo 8).

14.1 El gen 16S rRNA y las variantes de secuencia de amplicón

 Colab · 14.1 · 16S rRNA y ASVs

 repositorio del curso

El censo por el apellido

Imagine que debe hacer el censo de una ciudad enorme, pero nadie le abre la puerta. Lo único que puede hacer es recoger las cartas que llegan a los buzones y anotar los apellidos de los destinatarios. Con suerte, cada familia tiene un apellido distintivo; en la práctica, algunos apellidos se repiten entre familias no emparentadas, el cartero a veces escribe mal un nombre (“Gonzales” por “González”) y, de vez en cuando, dos sobres pegados producen un apellido

compuesto que no existe. Si se toma en serio cada grafía distinta, el censo inventará cientos de familias ficticias; si se agrupan los apellidos “parecidos”, se fundirán familias reales. El análisis de amplicones del 16S enfrenta exactamente este dilema: el gen es el apellido, los errores de secuenciación son las erratas del cartero y las quimeras de la PCR son los sobres pegados.

14.1.1 Un reloj molecular universal

Todo organismo celular necesita ribosomas, y todo ribosoma contiene ARN. En bacterias y arqueas, la subunidad pequeña del ribosoma lleva un ARN de unos 1500 nucleótidos, el ARNr 16S (la “S” es la unidad Svedberg de sedimentación). Tres propiedades lo convierten en un marcador casi ideal. Primero, es *universal*: está presente en todos los procariotas. Segundo, su función, ensamblar el ribosoma y reconocer el ARNm, es tan antigua y tan restrictiva que buena parte de su secuencia evoluciona muy lentamente, lo que permite alinear genes de organismos separados por miles de millones de años. Tercero, intercaladas entre esas regiones conservadas hay regiones *hipervariables* que evolucionan mucho más rápido y distinguen linajes cercanos.

Esta combinación fue la que explotaron Woese y Fox (1977). Sin secuenciadores automáticos, compararon catálogos de oligonucleótidos del ARNr 16S digerido con ribonucleasa T1 y encontraron que los metanógenos, clasificados hasta entonces como bacterias, eran tan distintos de las bacterias típicas como estas de los eucariotas. Propusieron una tercera línea primordial de la vida, las arqueobacterias (hoy *Archaea*), y con ella el árbol de tres dominios. La lección metodológica fue tan importante como la biológica: una sola molécula bien elegida basta para ordenar la diversidad microbiana en un árbol filogenético (capítulo 5).

De los catálogos de oligonucleótidos a los millones de lecturas

Woese y Fox trabajaron con fragmentos de ARN marcados radiactivamente y separados en geles bidimensionales, un proceso que tomaba semanas por organismo. Tres décadas después, Caporaso et al. (2011) secuenciaron la región V4 del 16S con Illumina a una profundidad de millones de lecturas por corrida, combinando cientos de muestras gracias a códigos de barras en los cebadores. El salto no fue solo de escala: con tantas lecturas, los errores del secuenciador dejaron de ser una curiosidad y pasaron a ser la principal fuente de diversidad *falsa*.

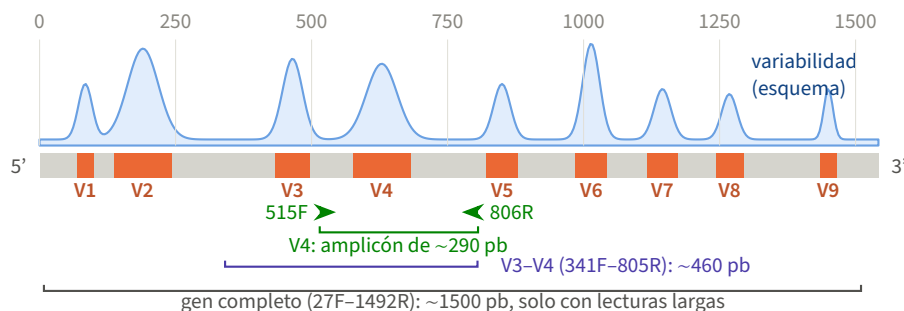


Figura 14.1. Anatomía del gen del ARNr 16S en coordenadas de *E. coli* (en nucleótidos, arriba). Las nueve regiones hipervariables (naranja) alternan con regiones conservadas (gris), donde se diseñan los cebadores. La curva azul es un perfil esquemático de variabilidad entre especies: los picos coinciden con las regiones V. Debajo, tres estrategias habituales: el amplicón V4 (515F/806R), el V3-V4, más largo e informativo pero más exigente con la calidad del extremo 3' de las lecturas, y el gen completo, accesible solo con lecturas largas (PacBio HiFi, Nanopore). Ninguna región discrimina igual de bien todos los grupos taxonómicos: la elección de cebadores es una decisión de diseño con consecuencias.

14.1.2 Regiones hipervariables y cebadores

El 16S de *Escherichia coli*, que se usa como sistema de coordenadas de referencia, mide 1542 nucleótidos y contiene nueve regiones hipervariables, V1 a V9 (figura 14.1). Las lecturas cortas de Illumina (150 a 300 bases por extremo) no alcanzan para cubrir el gen completo, de modo que en la práctica se amplifica una o dos regiones contiguas con cebadores que se unen a las regiones conservadas que las flanquean. El par 515F/806R, que amplifica la región V4, fue popularizado por Caporaso et al. (2011) y se usa en proyectos a gran escala; el par V3-V4 (aproximadamente 341F/805R) produce un amplicón más largo, que requiere lecturas de 300 bases por extremo.

Elegir la región es un compromiso. Una región más larga contiene más sitios informativos, pero exige lecturas más largas cuyo extremo 3' tiene peor calidad (capítulo 6); una región corta se lee con precisión, pero puede no distinguir géneros cercanos. Además, ningún par de cebadores es verdaderamente “universal”: los desajustes entre cebador y molde hacen que algunos linajes se amplifiquen peor que otros y queden subrepresentados. Por eso nunca deben compararse directamente abundancias obtenidas con cebadores distintos.

⚠ El número de copias del 16S

Un genoma bacteriano puede llevar entre una y más de una docena de copias del operón ribosómico. Si una especie tiene siete copias y otra solo una, la primera producirá, a igual número de células, unas siete veces más amplicones. Las abundancias relativas de una tabla 16S son, por lo tanto, abundancias de *genes*, no de células. Además, las copias de un mismo genoma pueden diferir ligeramente entre sí, de modo que un organismo puede aportar más de una variante de secuencia.

14.1.3 OTUs: agrupar al 97 %

Durante una década, el procedimiento estándar para convertir millones de lecturas en una tabla de “especies” fue agruparlas en *unidades taxonómicas operativas* (OTUs).

Definición 14.1. (Unidad taxonómica operativa (OTU))

Una OTU al nivel θ es un grupo de secuencias tal que cada miembro tiene una identidad de al menos θ con la secuencia representativa (centroide) del grupo. El umbral clásico es $\theta = 97\%$, que se tomó como aproximación a la “especie” bacteriana.

Para un amplicón V₄ de 253 nucleótidos (sin cebadores), el 97 % de identidad admite hasta $\lfloor 0,03 \times 253 \rfloor = 7$ diferencias. El agrupamiento resuelve un problema real: si cada lectura con un error de secuenciación se contara como un organismo distinto, la diversidad se inflaría de forma absurda. Pero el remedio tiene efectos secundarios serios (figura 14.2). *Pierde resolución*: dos cepas que difieren en tres nucleótidos de V₄, quizá una comensal y una patógena, quedan fundidas. *Depende del conjunto de datos*: los centroides *de novo* dependen del orden y la abundancia de las lecturas, así que al añadir muestras los límites cambian y una OTU de un estudio no es comparable con la “misma” OTU de otro. *Deja sobrevivir errores*: una lectura con ocho errores, o una quimera, forma una OTU espuria. Y el 3 % *carece de fundamento biológico*: es un compromiso histórico, no una ley.

14.1.4 De los errores a las variantes: la idea de DADA2

La alternativa moderna invierte la lógica. En lugar de preguntarse “¿qué lecturas se parecen lo suficiente?”, pregunta: “¿qué secuencias *reales* tuvieron que estar

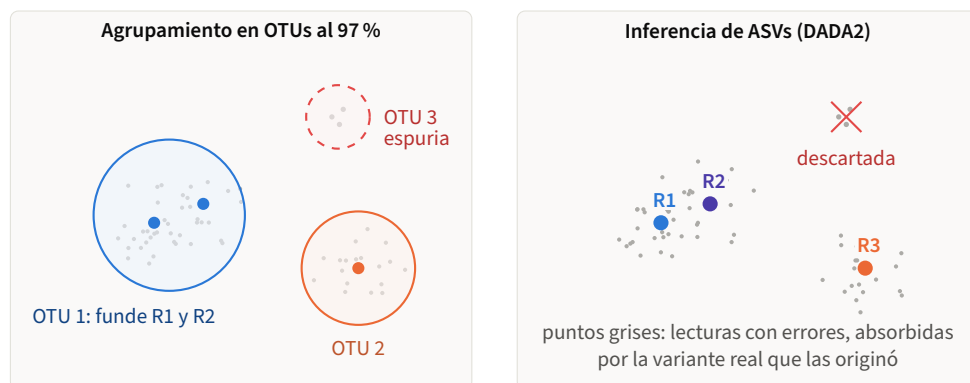


Figura 14.2. Dos maneras de resumir el mismo espacio de secuencias. Cada punto grande es una variante biológica real (R1 y R2 difieren en un solo nucleótido; R3 es otro linaje); los puntos grises son lecturas con errores de secuenciación, más numerosas alrededor de las variantes abundantes. **Izquierda:** el agrupamiento con un radio fijo del 3 % funde R1 y R2 en una sola OTU y convierte un pequeño grupo de lecturas aberrantes (quizá una quimera) en una OTU espuria. **Derecha:** la inferencia de ASVs no usa un radio fijo, sino un modelo de error: decide, lectura por lectura, si su abundancia es compatible con haber sido producida por errores de una variante más abundante.

en la muestra para producir las lecturas observadas, dado lo que sabemos de los errores del secuenciador?”. El resultado es un conjunto de *variantes de secuencia de amplicón* (ASVs), secuencias exactas inferidas con resolución de un nucleótido. Callahan et al. (2016) implementaron este enfoque en DADA2, y un año después Callahan et al. (2017) argumentaron que las ASVs deben reemplazar a las OTUs como unidad de análisis por tres razones: son *precisas* (distinguen variantes que difieren en una base), *reproducibles* (la misma secuencia biológica produce la misma ASV en cualquier estudio) y *reutilizables* (las tablas de distintos estudios pueden combinarse directamente, porque la etiqueta de cada unidad es su propia secuencia).

Definición 14.2. (Variante de secuencia de amplicón (ASV))

Una ASV es una secuencia exacta que un procedimiento de inferencia estadística considera presente en la muestra, porque su abundancia observada es demasiado alta para explicarse como producto de errores de amplificación y secuenciación de otras secuencias más abundantes.

El núcleo de DADA2 es un modelo de error. Supongamos que una secuencia real i produce lecturas que, por errores, se observan como la secuencia j . Si los errores en cada posición son independientes, la probabilidad de que una lectura de i se lea exactamente como j es el producto, posición a posición, de las probabilidades de transición:

$$\lambda_{ji} = \prod_{\ell=1}^L p(j(\ell) \mid i(\ell), q_j(\ell)). \quad (14.1)$$

DONDE

λ_{ji}	Probabilidad de que una lectura originada en la secuencia real i se observe como la secuencia j .
L	Longitud de las secuencias (tras el recorte).
$i(\ell), j(\ell)$	Nucleótidos de i y de j en la posición ℓ .
$q_j(\ell)$	Puntuación de calidad Phred de la posición ℓ en la lectura j (capítulo 6).
$p(b \mid a, q)$	Probabilidad de leer la base b cuando la verdadera es a , con calidad q ; hay 16 transiciones por cada valor de q (las 4 de la diagonal son lecturas correctas).

Las probabilidades $p(b \mid a, q)$ no se toman de la calidad nominal: DADA2 las *aprende* de los propios datos, alternando entre inferir las variantes con la matriz actual y reestimar la matriz a partir de las transiciones observadas entre lecturas y variantes, hasta converger. Esto importa porque la calidad que reporta el secuenciador es solo aproximadamente calibrada, y porque cada corrida tiene su propio perfil de errores.

Si una secuencia real i tiene abundancia n_i (número de lecturas en su partición), el número esperado de lecturas idénticas a j que i produce por error es $n_i \lambda_{ji}$. Bajo el supuesto de errores independientes entre lecturas, el número de copias de j generadas por error sigue aproximadamente una distribución de Poisson con esa media. Esto permite preguntar si la abundancia observada de j , a_j , es *sorprendente*.

Teorema 14.1. (Prueba de abundancia de DADA2)

Sea a_j el número de lecturas idénticas a la secuencia j , y sea i la variante de la partición a la que j está asignada. Condicionando en que j fue observada al menos una vez, el valor p de abundancia es

$$p_A(j | i) = \frac{1}{1 - \rho(0; n_i \lambda_{ji})} \sum_{a \geq a_j} \rho(a; n_i \lambda_{ji}), \quad \rho(a; \mu) = \frac{\mu^a e^{-\mu}}{a!}. \quad (14.2)$$

Si $p_A(j | i) < \Omega_A$, la secuencia j se declara una nueva variante (se crea una nueva partición con j como centro); DADA2 usa por defecto $\Omega_A = 10^{-40}$.

DONDE

a_j	Abundancia observada (número de lecturas idénticas) de la secuencia única j .
n_i	Número total de lecturas asignadas a la partición cuyo centro es i .
$\rho(a; \mu)$	Función de probabilidad de Poisson con media μ .
$1 - \rho(0; \mu)$	Probabilidad de observar j al menos una vez; condicionar en ella corrige el hecho de que solo evaluamos secuencias que sí aparecieron.
Ω_A	Umbral de significancia; muy pequeño porque se hacen miles de pruebas por muestra.

El algoritmo es divisivo. Empieza con todas las secuencias únicas en una sola partición cuyo centro es la más abundante. Calcula p_A para cada secuencia; si la menor está por debajo de Ω_A , esa secuencia se convierte en el centro de una nueva partición, las secuencias se reasignan al centro que mejor las explica (mayor $n_i \lambda_{ji}$) y el proceso se repite hasta que ninguna secuencia sea sorprendente. Los centros finales son las ASVs.

Ejemplo 14.1. (¿Error o variante a un nucleótido?)

En la simulación de la figura 14.3, la variante dominante produce $n_i = 5000$ moléculas y la tasa de error es $e = 0,002$ por base, repartida por igual entre

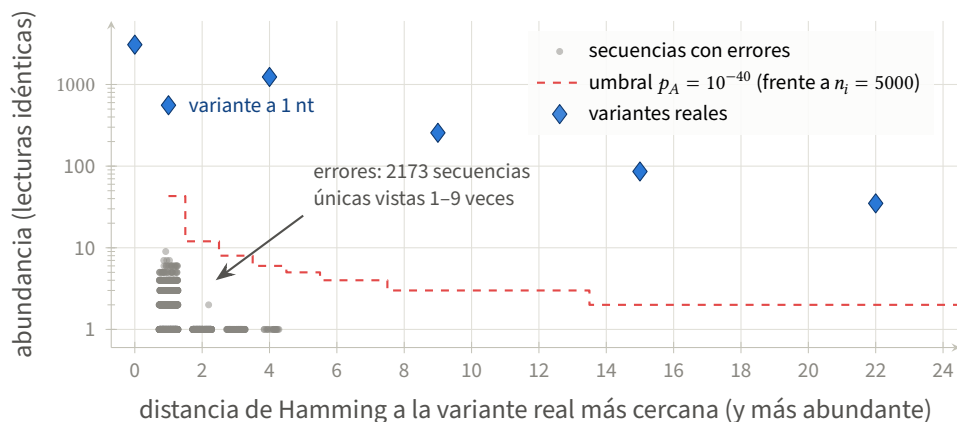


Figura 14.3. Simulación de un amplicón de 250 pb con seis variantes reales (8510 moléculas en total) y una tasa de sustitución de 0,002 por base. Cada punto gris es una secuencia única con al menos un error (hay 2179 secuencias únicas en total; el 38 % de las lecturas contiene errores), situada según su distancia al centro más cercano (con una pequeña dispersión horizontal para que se vean) y su abundancia. Los rombos azules son las variantes reales, con su abundancia de lecturas sin error. La línea roja es la abundancia mínima que resulta “sorprendente” frente a la variante más abundante: a un nucleótido de distancia hacen falta 43 copias idénticas, y a partir de cuatro nucleótidos bastan seis. Ningún error supera 9 copias y todas las variantes reales, incluida la que difiere en *un solo* nucleótido de la dominante, quedan muy por encima del umbral. Un agrupamiento al 97 % habría fundido las variantes a distancias 1 y 4 con la dominante. Datos: `figuras/cap14/generar.py`.

las tres bases incorrectas. La probabilidad de que una lectura de i se lea como una secuencia concreta j que difiere en una posición es

$$\lambda_{ji} = \frac{e}{3} (1 - e)^{249} = 6,67 \times 10^{-4} \times 0,607 = 4,05 \times 10^{-4},$$

de modo que el número esperado de copias de j generadas por error es $n_i \lambda_{ji} = 2,03$. Observar 3 o 4 copias no tiene nada de raro; observar 10 ya es improbable ($p_A \approx 5 \times 10^{-5}$), pero muy lejos de 10^{-40} . La abundancia umbral, calculada con la ecuación (14.2), es de 43 copias. La variante real a un nucleótido aparece 557 veces: su valor p es del orden de 10^{-119} , y DADA2 la reconoce sin ambigüedad. Observe que la prueba no depende de un umbral de identidad: a un nucleótido de una variante con cinco millones de lecturas se esperarían unas 2025 copias

por error, y 557 serían perfectamente compatibles con ruido, mientras que a diez nucleótidos de cualquier variante basta con unas pocas copias idénticas para que la secuencia sea real.

```
pvalor_abundancia.py

import numpy as np
from scipy.special import gammaln, logsumexp

def log10_pA(a_j, n_i, d, L=250, e=0.002):
    """log10 del valor p de abundancia de DADA2 (ec. 14.2)."""
    lam = (e / 3) ** d * (1 - e) ** (L - d)      # ec. 14.1
    mu = n_i * lam
    k = np.arange(a_j, a_j + 500)                # cola de Poisson
    lcola = logsumexp(k * np.log(mu) - mu - gammaln(k + 1))
    lden = np.log(-np.expm1(-mu))                 # P(A > 0)
    return (lcola - lden) / np.log(10)

print(log10_pA(10, 5000, 1))      # -4.3: raro, pero no significativo
print(log10_pA(557, 5000, 1))    # ~ -1119: variante real
```

14.1.5 Filtrado previo: errores esperados

Antes de la inferencia conviene descartar lecturas cuya calidad hace inútil cualquier modelo. DADA2 y QIIME 2 usan el número de *errores esperados*, que se deduce directamente de la escala Phred (capítulo 6):

$$EE = \sum_{\ell=1}^L 10^{-q(\ell)/10}. \quad (14.3)$$

DONDE

EE	Número esperado de errores en la lectura (suma de las probabilidades de error por posición).
$q(\ell)$	Calidad Phred de la base ℓ .

Por linealidad de la esperanza, la suma no requiere suponer independencia. Una lectura de 250 bases con $Q = 30$ en todas sus posiciones tiene $EE = 0,25$; con $Q = 20$, $EE = 2,5$. Un filtro habitual descarta lecturas con $EE > 2$, lo que es preferible a filtrar por calidad media, porque la media en escala Phred oculta las pocas bases malas que dominan la suma. El filtro se combina con un recorte de longitud elegido

a la vista del perfil de calidad, dejando solapamiento suficiente para unir los dos extremos de cada par.

14.1.6 Quimeras

Durante la PCR, una hebra cuya extensión quedó incompleta puede, en el ciclo siguiente, hibridarse con un molde *distinto* y completarse copiándolo. El producto es una *quimera*: su extremo 5' procede de un organismo y su extremo 3' de otro. Las quimeras no tienen errores de secuenciación, así que un modelo de error no las detecta; pueden ser muy abundantes y fáciles de confundir con organismos nuevos. En datos de amplicones pueden representar una fracción apreciable de las secuencias únicas.

La detección aprovecha una asimetría: una quimera se forma a partir de dos *padres* que, al estar en la misma PCR, suelen ser más abundantes que ella. UCHIME (Edgar et al., 2011) busca, para cada secuencia candidata, dos secuencias más abundantes cuya combinación (un segmento izquierdo de una y un segmento derecho de la otra) la explique mejor que cualquier secuencia individual. DADA2 aplica la versión exacta de esta idea sobre las ASVs: marca como “bimera” toda variante que pueda reconstruirse *sin errores* uniendo un prefijo de un padre más abundante con un sufijo de otro.

14.1.7 Asignación taxonómica: el clasificador bayesiano ingenuo

Una ASV es una secuencia sin nombre. Para darle uno, se compara con una base de datos de referencia curada. La más usada es SILVA, que mantiene alineamientos y una taxonomía revisada de ARN ribosómicos de la subunidad pequeña y grande de los tres dominios (Quast et al., 2013). El método de asignación clásico es el clasificador bayesiano ingenuo del Ribosomal Database Project (Wang et al., 2007), que QIIME 2 reimplementa con scikit-learn.

La idea es representar cada secuencia por el conjunto de sus palabras de longitud 8 (8-mers) y preguntar qué género hace más probable ese conjunto. Para un género G con M secuencias de referencia, la probabilidad de que una secuencia de G contenga la palabra w_i se estima como

$$P(w_i | G) = \frac{m(w_i) + P_i}{M + 1}, \quad P_i = \frac{n(w_i) + 0,5}{N + 1}, \quad (14.4)$$

y, suponiendo que las palabras son independientes (de ahí lo de “ingenuo”),

$$\log P(S | G) = \sum_{w_i \in S} \log P(w_i | G), \quad \hat{G} = \arg \max_G P(S | G). \quad (14.5)$$

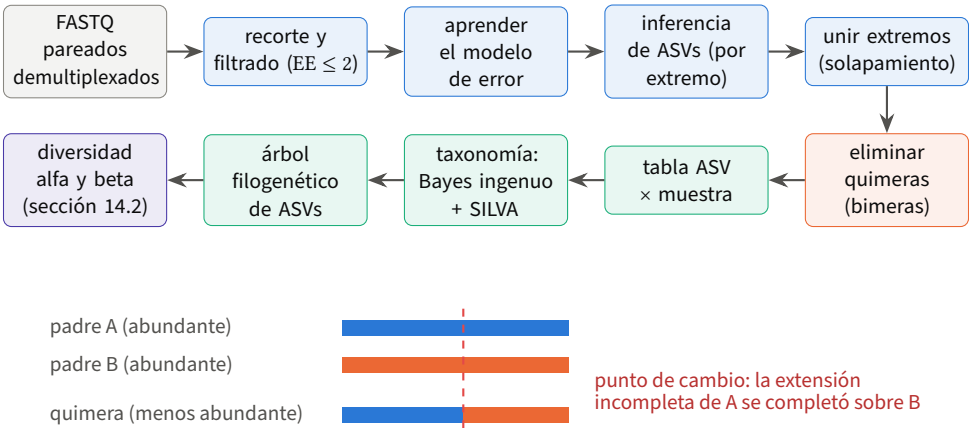


Figura 14.4. Flujo típico de análisis de amplicones del 16S con DADA2 dentro de QIIME 2. Azul: pasos de control de calidad y denoising, que se ejecutan por muestra; aqua: productos que se usan en todos los análisis posteriores. **Abajo:** cómo nace una quimera durante la PCR. Su prefijo coincide exactamente con el padre A y su sufijo con el padre B; esa estructura, junto con su menor abundancia, es lo que buscan UCHIME y la función de eliminación de bimeras de DADA2.

DONDE	
w_i	Una palabra (8-mer) presente en la secuencia consulta S .
$m(w_i)$	Número de secuencias de referencia del género G que contienen w_i .
M	Número de secuencias de referencia del género G .
$n(w_i), N$	Número de secuencias de <i>toda</i> la base que contienen w_i , y número total de secuencias.
P_i	Probabilidad previa de la palabra; actúa como pseudoconteo para que una palabra nunca vista en G no anule el producto.
$\hat{G}$	Género asignado.

Con una probabilidad previa uniforme sobre géneros, maximizar $P(S | G)$ equivale a maximizar la probabilidad posterior $P(G | S)$. La confianza no se obtiene de esa posterior, que suele ser engañosamente cercana a 1, sino por *remuestreo*: se repite la clasificación con 100 subconjuntos aleatorios de un octavo de las palabras y se cuenta en cuántas réplicas se obtiene el mismo género. Si la confianza (el porcentaje de réplicas coincidentes) cae por debajo de un umbral, habitualmente 80 %, la asignación se trunca al rango superior (familia, orden) en el que sí se alcanza.

Ejemplo 14.2. (Tres géneros y cuatro palabras)

Una base de juguete tiene $N = 125$ secuencias: 40 de *Bacteroides*, 25 de *Prevotella* y 60 de *Escherichia*. Una ASV contiene cuatro palabras $w_1, \dots, w_4$, presentes en $n(w) = 59, 37, 61, 61$ secuencias de la base, respectivamente. Los conteos por género son:

Género (M)	$m(w_1)$	$m(w_2)$	$m(w_3)$	$m(w_4)$	$\log P(S G)$
<i>Bacteroides</i> (40)	38	30	2	35	-3,31
<i>Prevotella</i> (25)	20	3	1	24	-5,22
<i>Escherichia</i> (60)	1	4	58	2	-9,62

Por ejemplo, para *Bacteroides* y w_1 : $P_1 = (59 + 0,5)/126 = 0,472$ y $P(w_1 | G) = (38 + 0,472)/41 = 0,938$. Sumando los logaritmos de las cuatro palabras se obtiene -3,31. Con probabilidad previa uniforme, las posteriores son 0,870, 0,129 y 0,002: *Bacteroides* es el género más probable. La palabra w_2 decide la competencia con *Prevotella*, en la que solo aparece 3 veces de 25. Con 250 palabras reales en lugar de cuatro, las diferencias de log-verosimilitud se vuelven enormes, y por eso la confianza se mide con remuestreo.

La resolución tiene un techo: una región de 250 nucleótidos contiene pocas docenas de sitios variables entre especies de un género, y muchas especies (por ejemplo, de *Escherichia/Shigella*) comparten secuencias V_4 idénticas. La ASV es exacta, pero un nombre de especie asignado a partir de ella rara vez lo es; para eso hacen falta el gen completo o la metagenómica shotgun (sección 14.3).

14.1.8 QIIME 2: reproducibilidad por diseño

QIIME 2 (Bolyen et al., 2019) no es un algoritmo sino una plataforma que envuelve decenas de métodos (DADA2, clasificadores, métricas de diversidad, pruebas estadísticas) bajo una interfaz común. Su rasgo más distintivo es el registro automático de procedencia: cada archivo .qza (artefacto) guarda, además de los datos, el historial completo de comandos, parámetros y versiones que lo produjeron, de modo que cualquier resultado puede rastrearse hasta los FASTQ originales. Los artefactos llevan *tipos semánticos* que impiden, por ejemplo, pasar una tabla sin rarefacción a un método que la espera rarificada.

De FASTQ a tabla de ASVs con QIIME 2

```
qiime tools import \
  --type 'SampleData[PairedEndSequencesWithQuality]' \
  --input-path manifiesto.tsv \
  --input-format PairedEndFastqManifestPhred33V2 \
  --output-path demux.qza
qiime dada2 denoise-paired \
  --i-demultiplexed-seqs demux.qza \
  --p-trunc-len-f 240 --p-trunc-len-r 160 \
  --p-max-ee-f 2 --p-max-ee-r 2 \
  --o-table tabla.qza \
  --o-representative-sequences asv.qza \
  --o-denoising-stats estadisticas.qza
qiime feature-classifier classify-sklearn \
  --i-classifier silva-515-806-nb-classifier.qza \
  --i-reads asv.qza --o-classification taxonomia.qza
```

Elija `--p-trunc-len` mirando el perfil de calidad de *sus* datos y revise las estadísticas de denoising: si la unión de extremos pierde más de la mitad de las lecturas, el recorte fue demasiado agresivo.

Una OTU es un promedio de secuencias parecidas; una ASV es una secuencia exacta que un modelo de error no puede explicar como ruido. La segunda es más precisa, reproducible entre estudios y reutilizable.

14.2 Diversidad alfa y beta

 Colab · 14.2 · Diversidad alfa y beta

 repositorio del curso

Dos bosques con veinte especies

Dos bosques tienen exactamente veinte especies de árboles. En el primero, cada especie aporta la misma cantidad de ejemplares: caminar por él es encontrarse a cada paso con algo distinto. En el segundo, un único pino representa el 95 % de los árboles y las otras diecinueve especies aparecen, con suerte,

una vez por hectárea. ¿Son igual de diversos? Contar especies dice que sí; la experiencia de cualquier caminante dice que no. Y aún hay otra pregunta: si el primer bosque se muestreara solo durante una hora, ¿cuántas de sus veinte especies llegaríamos a ver? La ecología lleva un siglo afinando respuestas a estas dos preguntas (cómo medir la diversidad y cómo corregir por el esfuerzo de muestreo), y la microbiología las heredó con el mismo entusiasmo con que heredó sus trampas.

14.2.1 La tabla de abundancias

El producto de la sección 14.1 (o de un perfilado shotgun, sección 14.3) es una tabla X de conteos, con una fila por muestra y una columna por taxón (ASV, género, especie). Llamaremos x_{ki} al número de lecturas del taxón i en la muestra k , $N_k = \sum_i x_{ki}$ a la profundidad de secuenciación de la muestra y $p_{ki} = x_{ki}/N_k$ a su abundancia relativa. En cada muestra, la mayoría de las columnas son ceros: una tabla de 16S típica tiene más del 90 % de celdas vacías.

Distinguiremos, siguiendo la tradición ecológica, dos escalas de diversidad. La *diversidad alfa* describe una sola muestra: cuántos taxones contiene y cuán equitativamente se reparten las abundancias. La *diversidad beta* describe la diferencia entre muestras: cuánto cambia la composición de una a otra.

14.2.2 Riqueza, Shannon y Simpson

Definición 14.3. (Índices clásicos de diversidad alfa)

Para una muestra con abundancias relativas $p_1, \dots, p_S$ (todas positivas),

$$S = \#\{i : p_i > 0\}, \quad (14.6)$$

$$H = - \sum_{i=1}^S p_i \ln p_i, \quad (14.7)$$

$$D = \sum_{i=1}^S p_i^2, \quad (14.8)$$

son, respectivamente, la riqueza observada, la entropía de Shannon y el índice de concentración de Simpson.

DONDE

S	Riqueza: número de taxones observados en la muestra.
p_i	Abundancia relativa del taxón i (fracción de las lecturas de la muestra).
H	Entropía de Shannon, en nats si se usa $\ln$; mide la incertidumbre sobre el taxón de una lectura elegida al azar.
D	Probabilidad de que dos lecturas elegidas al azar (con reposición) pertenezcan al mismo taxón.

Cada índice responde una pregunta distinta. S ignora las abundancias; da el mismo peso a un taxón con una lectura que a uno con un millón. H es la entropía de la teoría de la información: es máxima, $\ln S$, cuando todos los taxones son igual de abundantes, y vale cero cuando uno solo acapara la muestra. D está dominado por los taxones abundantes, porque eleva las proporciones al cuadrado; suele reportarse como $1 - D$ (índice de Gini-Simpson, la probabilidad de que dos lecturas sean de taxones distintos) o como $1/D$ (Simpson inverso). La *equidad* de Pielou, $J = H / \ln S$, normaliza la entropía por su máximo.

Ejemplo 14.3. (Una comunidad de nueve taxones)

Una muestra con $N = 100$ lecturas tiene los conteos (50, 20, 10, 10, 5, 2, 1, 1, 1). Entonces

$$H = -(0,5 \ln 0,5 + 0,2 \ln 0,2 + 2 \times 0,1 \ln 0,1 + 0,05 \ln 0,05 + 0,02 \ln 0,02 + 3 \times 0,01 \ln 0,01) = 1,495,$$

$D = 0,25 + 0,04 + 0,02 + 0,0025 + 0,0004 + 0,0003 = 0,3132$, de modo que $1 - D = 0,687$ y $1/D = 3,19$. La equidad es $J = 1,495 / \ln 9 = 0,680$. Observe que los tres índices viven en escalas incomparables: una entropía de 1,5 nats, una probabilidad de 0,69 y una “cantidad” de 3,2. En la sección 14.2.5 veremos cómo ponerlos en la misma escala.

14.2.3 Lo que no vimos: el estimador Chao1

La riqueza observada depende de la profundidad: con más lecturas aparecen más taxones raros. Lo que realmente interesa es la riqueza *de la comunidad*, $S_{\text{real}} = S_{\text{obs}} + f_0$, donde f_0 es el número de taxones presentes pero no observados. Parece imposible estimar lo que no se ha visto, pero los taxones vistos una o dos veces

contienen información sobre los que se han visto cero veces. Esta es la intuición del estimador de Chao (1987), desarrollado originalmente para estimar tamaños de poblaciones animales por captura y recaptura.

Llamemos f_k al número de taxones observados exactamente k veces (así, f_1 son los *singletons* y f_2 los *doubletons*). Supongamos que el taxón i aparece en la muestra según una distribución de Poisson de media λ_i , que depende de su abundancia. Entonces

$$E[f_k] = \sum_{i=1}^{S_{\text{real}}} \frac{\lambda_i^k e^{-\lambda_i}}{k!}, \quad k = 0, 1, 2, \dots \quad (14.9)$$

DONDE

f_k	Número de taxones con exactamente k lecturas en la muestra (f_0 : no observados).
λ_i	Número esperado de lecturas del taxón i con la profundidad dada.
S_{real}	Número total de taxones de la comunidad, observados o no.

Teorema 14.2. (Cota inferior de Chao)

Bajo el modelo de la ecuación (14.9), $E[f_0] \geq E[f_1]^2 / (2 E[f_2])$. En consecuencia, el estimador

$$\hat{S}_{\text{Chao1}} = S_{\text{obs}} + \frac{f_1^2}{2f_2} \quad (14.10)$$

estima una cota inferior de la riqueza real. Cuando $f_2 = 0$ se usa la forma corregida por sesgo $S_{\text{obs}} + f_1(f_1 - 1) / (2(f_2 + 1))$.

Demostración. Escribamos $a_i = e^{-\lambda_i/2}$ y $b_i = \lambda_i e^{-\lambda_i/2}$. Por la desigualdad de Cauchy-Schwarz,

$$E[f_1]^2 = \left(\sum_i \lambda_i e^{-\lambda_i} \right)^2 = \left(\sum_i a_i b_i \right)^2 \leq \left(\sum_i a_i^2 \right) \left(\sum_i b_i^2 \right) = E[f_0] \cdot 2 E[f_2],$$

y basta despejar. La igualdad se alcanza cuando todos los λ_i son iguales; cuanto más heterogéneas sean las abundancias de los taxones raros, más holgada es la cota. $\square$

La demostración explica también el uso correcto del estimador. Chao1 no “adivina” la riqueza: dice que, como mínimo, faltan unos $f_1^2 / (2f_2)$ taxones. Si muchos taxones aparecen una sola vez y pocos dos veces, la muestra está lejos de estar completa. Si casi no hay *singletons*, la muestra está saturada. En la muestra del ejemplo 14.3, con $f_1 = 3$ y $f_2 = 1$, obtenemos $\hat{S}_{\text{Chao1}} = 9 + 9/2 = 13,5$.

⚠ Los *singletons* de un amplicón no son los de un censo

Chao1 descansa por completo en f_1 y f_2 . Pero DADA2 elimina por defecto las secuencias vistas una sola vez, porque la mayoría son errores (sección 14.1). Una tabla de ASVs casi no tiene *singletons*, y en ella Chao1 coincide con la riqueza observada por construcción, no porque la comunidad esté completa. Con OTUs, el problema es el inverso: los errores de secuenciación inflan f_1 y con él la riqueza estimada. Los estimadores de riqueza basados en *singletons* deben usarse con mucha cautela en datos de amplicones.

14.2.4 Rarefacción

Si dos muestras tienen profundidades distintas, su riqueza observada no es comparable. La *curva de rarefacción* responde a la pregunta: ¿cuántos taxones distintos esperaríamos ver si hubiéramos secuenciado solo n de las N lecturas? Elegir n lecturas al azar sin reposición es un muestreo hipergeométrico, y la probabilidad de que el taxón i (con x_i lecturas) *no* aparezca en la submuestra es el número de maneras de elegir las n lecturas entre las $N - x_i$ que no son suyas, dividido por el total. Sumando sobre taxones (Chao et al., 2014):

$$E[S_n] = \sum_{i=1}^{S_{\text{obs}}} \left[1 - \frac{\binom{N-x_i}{n}}{\binom{N}{n}} \right]. \quad (14.11)$$

DONDE

$E[S_n]$	Riqueza esperada en una submuestra aleatoria de n lecturas.
N, x_i	Profundidad total de la muestra y lecturas del taxón i .
$\binom{a}{b}$	Coficiente binomial; se evalúa en escala logarítmica (con $\ln \Gamma$) para evitar desbordamientos.

La curva crece rápido al principio, cuando cada lectura nueva tiene alta probabilidad de pertenecer a un taxón no visto, y se aplanan cuando solo quedan por descubrir los raros (figura 14.5). Su pendiente final tiene una interpretación elegante: la probabilidad de que la *siguiente* lectura pertenezca a un taxón nuevo es aproximadamente f_1/N . Su complemento, $\hat{C} \approx 1 - f_1/N$, es la *cobertura de muestra* de Good y Turing: la fracción de la comunidad (ponderada por abundancia) que la muestra ya representa. Chao et al. (2014) proponen comparar muestras no a igual número de lecturas, sino a igual cobertura, y extender las curvas por extrapolación en lugar de recortarlas.

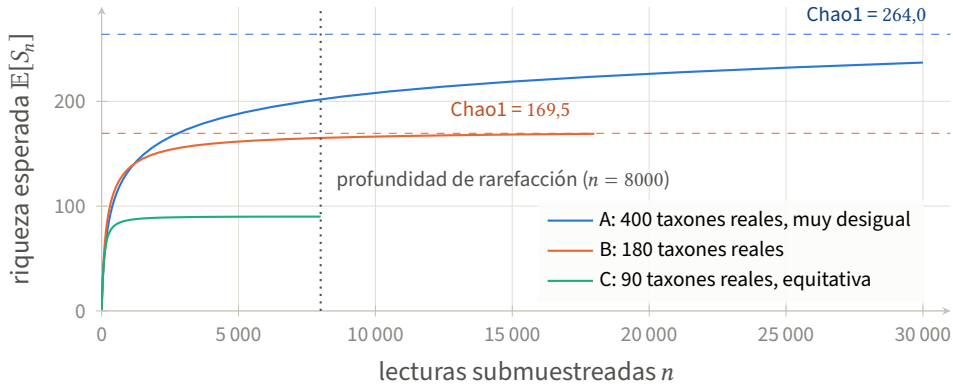


Figura 14.5. Curvas de rarefacción exactas (ecuación (14.11)) para tres comunidades simuladas secuenciadas a profundidades distintas (30 000, 18 000 y 8000 lecturas). La comunidad C, pequeña y equitativa, se satura enseguida: sus 90 taxones aparecen todos. La B casi se satura. La A, con 400 taxones y una larguísima cola de raros, sigue subiendo: se observan solo 237, y Chao1 (línea discontinua) estima 264, una cota inferior muy por debajo del valor real, como anticipa el teorema 14.2. Recortar las tres muestras a 8000 lecturas (línea punteada) hace comparables sus riquezas (202, 165 y 90), a costa de descartar el 73 % de las lecturas de A.

La controversia. En microbiología se popularizó otra práctica: *rareficar* la tabla, es decir, submuestrear cada muestra hasta la profundidad de la más pobre y tirar el resto. McMurdie y Holmes (2014) argumentaron que esto es “inadmisibile” para el análisis de abundancia diferencial: desechar datos válidos aumenta la varianza, reduce la potencia, añade incertidumbre artificial (dos rarefacciones dan resultados distintos) y obliga a eliminar muestras. Propusieron modelos que tratan explícitamente la profundidad, como los de mezcla binomial negativa de RNA-seq. Para la *diversidad alfa*, que depende intrínsecamente del esfuerzo de muestreo, alguna estandarización (por rarefacción, cobertura o extrapolación) sigue siendo necesaria: la curva de rarefacción como diagnóstico es siempre útil; la tabla rareficada para abundancia diferencial, no.

14.2.5 Números de Hill: una familia que unifica

La discusión anterior dejó una incomodidad: la riqueza, Shannon y Simpson miden cosas distintas en escalas distintas. Hill (1973) mostró que son, en realidad, miembros de una única familia.

Definición 14.4. (Números de Hill)

El número de Hill de orden $q \geq 0$ de una comunidad con abundancias relativas $p_1, \dots, p_S$ es

$${}^qD = \left(\sum_{i=1}^S p_i^q \right)^{1/(1-q)}, \quad q \neq 1, \quad (14.12)$$

y ${}^1D = \lim_{q \rightarrow 1} {}^qD$.

DONDE

qD	Diversidad de orden q , medida en “número efectivo de taxones”.
q	Orden: controla cuánto peso reciben los taxones abundantes frente a los raros.

Evaluemos los casos notables. Con $q = 0$, cada $p_i^0 = 1$ y ${}^0D = S$: la riqueza. Con $q = 2$, ${}^2D = 1 / \sum p_i^2 = 1/D$: el Simpson inverso. Con $q \rightarrow \infty$, domina el término mayor y ${}^\infty D = 1 / \max_i p_i$ (índice de Berger-Parker inverso). El caso $q = 1$ requiere un límite. Tomando logaritmos, $\ln {}^qD = \ln(\sum_i p_i^q) / (1 - q)$, que en $q = 1$ es de la forma $0/0$. Por la regla de L'Hôpital, derivando numerador y denominador respecto de q ,

$$\lim_{q \rightarrow 1} \ln {}^qD = \lim_{q \rightarrow 1} \frac{\sum_i p_i^q \ln p_i / \sum_i p_i^q}{-1} = - \sum_i p_i \ln p_i = H, \quad \text{es decir,} \quad {}^1D = e^H. \quad (14.13)$$

La exponencial de Shannon es, así, el número de Hill de orden 1.

¿Por qué molestarse en transformar índices que ya conocíamos? Jost (2006) dio el argumento decisivo: la entropía de Shannon y el índice de Gini-Simpson *no son diversidades*, sino funciones de ellas, y tratarlas como tales produce conclusiones absurdas.

Teorema 14.3. (Principio de replicación)

Si se combinan m comunidades igual de grandes, cada una con número de Hill qD , y sin ningún taxón en común, la comunidad combinada tiene número de Hill $m \cdot {}^qD$, para todo q .

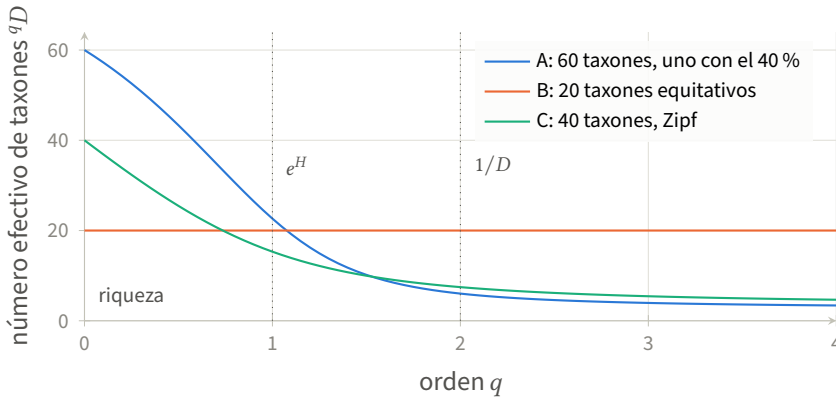


Figura 14.6. Perfiles de diversidad de Hill para tres comunidades simuladas. El perfil de una comunidad perfectamente equitativa (B) es plano: con 20 taxones igual de abundantes, cualquier orden da 20. Las comunidades desiguales decrecen con q , y su ordenamiento cambia: A es la más rica ($q = 0$: 60 taxones) y, gracias a sus 59 taxones raros, también la más diversa en $q = 1$ ($e^H = 22.6$), pero en $q = 2$ es la menos diversa ($1/D = 6.0$), porque un solo taxón domina. Las curvas que se cruzan son la razón por la que “¿cuál es más diversa?” no tiene respuesta sin especificar el orden.

La demostración es directa: en la comunidad combinada cada proporción se divide por m y hay m copias de cada término, de modo que $\sum p_i^q$ se multiplica por $m \cdot m^{-q} = m^{1-q}$ y, al elevar a $1/(1-q)$, qD se multiplica por m . En cambio, la entropía de Shannon solo *suma* $\ln m$: dos comunidades de ocho especies equiprobables tienen cada una $H = \ln 8 = 2.08$, y su unión, con 16 especies, tiene $H = 2.77$, apenas un 33 % más, aunque intuitivamente es el doble de diversa. Un tratamiento que reduce la entropía de Shannon “solo un 10 %” puede estar eliminando la mitad del número efectivo de taxones. Los números de Hill se miden en la unidad que la intuición espera: *número de taxones igualmente abundantes que produciría la misma diversidad*.

Si dos perfiles no se cruzan, una comunidad es inequívocamente más diversa que la otra; si se cruzan, la respuesta depende de si importan más los taxones raros (órdenes bajos) o los dominantes (órdenes altos). En datos de amplicones, los órdenes $q = 1$ y $q = 2$ son mucho más robustos que la riqueza.

14.2.6 Los datos de microbioma son composicionales

Hasta aquí hemos trabajado con proporciones p_i sin preguntar si representan algo absoluto. No lo hacen. El secuenciador produce un número de lecturas por

muestra que depende de la carga del equipo, no de la cantidad de bacterias en el intestino del paciente. Un aumento de x_{ki} puede significar que el taxón i creció, que otros taxones disminuyeron o simplemente que esa muestra recibió más lecturas. Gloor et al. (2017) insistieron en que estos datos son *composicionales*: solo contienen información sobre las razones entre componentes, y los métodos que ignoran esta restricción producen correlaciones espurias y falsos positivos.

Formalmente, un vector de abundancias absolutas $\mathbf{a}$ solo se observa a través de su *cierre* $\mathbf{a}/\sum_i a_i$, un punto del simplex; dos vectores proporcionales, $\mathbf{a}$ y $c\mathbf{a}$, tienen la misma composición.

La consecuencia más importante es que las proporciones no son independientes: como suman 1, si una aumenta, alguna otra debe disminuir. La figura 14.7 lo muestra con un ejemplo mínimo. La solución de la estadística composicional, iniciada por Aitchison, consiste en trabajar con *logaritmos de razones*, que son invariantes al cierre. La transformación más usada es la razón logarítmica centrada:

$$\text{clr}(\mathbf{p})_i = \ln \frac{p_i}{g(\mathbf{p})}, \quad g(\mathbf{p}) = \left(\prod_{j=1}^D p_j \right)^{1/D}, \quad (14.14)$$

DONDE

$\text{clr}(\mathbf{p})_i$	Coordenada del taxón i tras la transformación de razón logarítmica centrada.
$g(\mathbf{p})$	Media geométrica de las proporciones de la muestra.
D	Número de componentes (taxones) de la composición.

y la distancia euclidiana entre vectores clr , llamada *distancia de Aitchison*, es la distancia natural entre composiciones. Como $\ln(ca_i/g(c\mathbf{a})) = \ln(a_i/g(\mathbf{a}))$, la transformación da el mismo resultado con abundancias absolutas o relativas, y la profundidad de secuenciación desaparece. El precio es que el logaritmo de cero no existe: los ceros, omnipresentes en las tablas de microbioma, deben reemplazarse por un pequeño pseudoconteo o tratarse con modelos específicos, y esa decisión influye en los resultados.

La secuenciación mide proporciones, no cantidades. Cualquier conclusión del tipo “el taxón X aumentó” es, sin información externa (carga bacteriana total, estándares añadidos), una afirmación sobre razones: aumentó *respecto de los demás*.

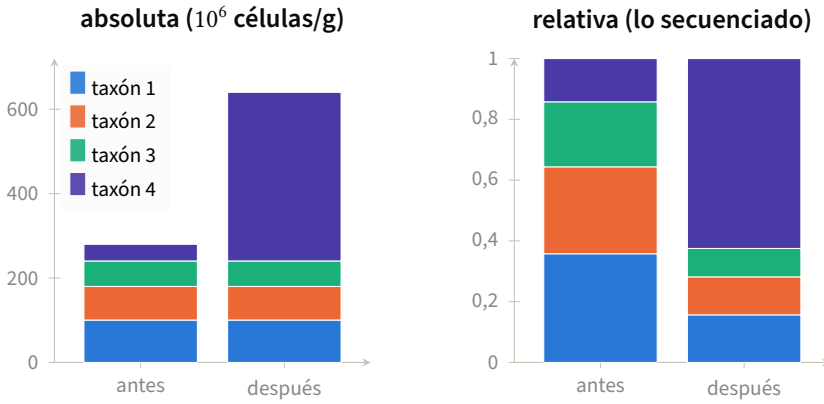


Figura 14.7. Un único taxón (violeta) se multiplica por diez entre dos momentos; los otros tres no cambian en absoluto (izquierda). Tras el cierre, que es lo que mide la secuenciación (derecha), los taxones 1, 2 y 3 parecen haber disminuido a menos de la mitad: el taxón 1 pasa de 0,357 a 0,156. Un análisis de abundancia diferencial ingenuo declararía que cuatro taxones cambiaron. En cambio, la razón logarítmica entre los taxones 1 y 2, $\ln(p_1/p_2) = 0,223$, es idéntica antes y después: las razones entre componentes sobreviven al cierre, las proporciones no.

14.2.7 Diversidad beta: distancias entre comunidades

Para comparar muestras necesitamos una disimilitud $d(u, v)$ entre dos vectores de abundancia. No existe una única elección correcta; cada índice responde una pregunta biológica distinta.

Definición 14.5. (Bray-Curtis y Jaccard)

Para dos muestras con abundancias u_i y v_i (conteos o proporciones),

$$d_{\text{BC}}(u, v) = \frac{\sum_i |u_i - v_i|}{\sum_i (u_i + v_i)} = 1 - \frac{2 \sum_i \min(u_i, v_i)}{\sum_i (u_i + v_i)}, \quad (14.15)$$

$$d_j(u, v) = 1 - \frac{|A \cap B|}{|A \cup B|}, \quad (14.16)$$

donde $A = \{i : u_i > 0\}$ y $B = \{i : v_i > 0\}$ son los conjuntos de taxones presentes.

DONDE

u_i, v_i	Abundancia del taxón i en cada muestra.
A, B	Conjuntos de taxones presentes en cada muestra.
d_{BC}, d_j	Disimilitudes entre 0 (idénticas) y 1 (sin nada en común).

Bray-Curtis pondera por abundancia: dos muestras dominadas por los mismos taxones son parecidas aunque difieran en muchos raros. Jaccard solo mira presencia y ausencia: es sensible a los raros y, por lo tanto, a la profundidad y a los errores. Con $u = (10, 0, 5, 3, 2)$ y $v = (4, 6, 5, 0, 5)$, $\sum |u_i - v_i| = 18$ y $\sum (u_i + v_i) = 40$, así que $d_{BC} = 0,45$; los taxones presentes son $A = \{1, 3, 4, 5\}$ y $B = \{1, 2, 3, 5\}$, con 3 en común de 5 en total, y $d_j = 1 - 3/5 = 0,40$. Bray-Curtis no es una métrica (no siempre cumple la desigualdad triangular), un detalle que tendrá consecuencias en la ordenación.

Ninguno de los dos índices sabe que dos ASVs pueden ser casi idénticas. Para una muestra dominada por *Lactobacillus crispatus* y otra dominada por *L. iners*, Jaccard y Bray-Curtis dan la máxima distancia, igual que si la segunda estuviera dominada por una arquea metanógena. Lozupone y Knight (2005) resolvieron esta ceguera incorporando la filogenia (capítulo 5). Dado un árbol con todas las secuencias de ambas muestras, la distancia UniFrac no ponderada es la fracción de la longitud de ramas del árbol que conduce a secuencias de *una sola* de las dos muestras:

$$d_U(u, v) = \frac{\sum_r b_r |1[r \in u] - 1[r \in v]|}{\sum_r b_r}, \quad (14.17)$$

DONDE

r	Una rama del árbol filogenético que contiene todas las secuencias de las dos muestras.
b_r	Longitud de la rama r .
$1[r \in u]$	Vale 1 si alguna secuencia de la muestra u desciende de la rama r .

Si las dos comunidades comparten linajes cercanos, las ramas exclusivas serán cortas y la distancia pequeña, aunque no compartan ni una ASV. La variante *ponderada*, propuesta después por los mismos autores, pondera cada rama por la diferencia de abundancias relativas en lugar de la presencia, y responde a cambios en los linajes dominantes.

14.2.8 Ordenación por coordenadas principales (PCoA)

Con n muestras obtenemos una matriz $n \times n$ de distancias. Para *verla*, buscamos puntos en un plano cuyas distancias euclidianas se parezcan lo más posible a las originales. El análisis de coordenadas principales (PCoA, o escalamiento multidimensional clásico) lo hace mediante una descomposición espectral.

Teorema 14.4. (Coordenadas principales)

Sea $\mathbf{D}$ la matriz de disimilitudes, $\mathbf{A} = -\frac{1}{2}[d_{kl}^2]$ y $\mathbf{J} = \mathbf{I} - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$ la matriz de centrado. Sea

$$\mathbf{B} = \mathbf{J}\mathbf{A}\mathbf{J} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top \quad (14.18)$$

su descomposición espectral, con autovalores $\lambda_1 \geq \lambda_2 \geq \dots$. Si $\mathbf{D}$ es euclidiana, las coordenadas $\mathbf{Y} = \mathbf{V}\mathbf{\Lambda}^{1/2}$ reproducen exactamente las distancias, y sus primeras m columnas dan la mejor representación en m dimensiones en el sentido de mínimos cuadrados sobre $\mathbf{B}$. La fracción de variación representada por el eje k es $\lambda_k / \sum_{j: \lambda_j > 0} \lambda_j$.

DONDE

d_{kl}	Disimilitud entre las muestras k y l .
$\mathbf{B}$	Matriz de productos escalares centrada (“doble centrado” de Gower).
$\mathbf{V}, \mathbf{\Lambda}$	Autovectores y matriz diagonal de autovalores de $\mathbf{B}$.
$\mathbf{Y}$	Coordenadas de las muestras en los ejes principales.

La idea detrás de la ecuación (14.18) es que, si las muestras fueran puntos $\mathbf{y}_k$ centrados en el origen, $d_{kl}^2 = \|\mathbf{y}_k\|^2 + \|\mathbf{y}_l\|^2 - 2\mathbf{y}_k^\top \mathbf{y}_l$, y el doble centrado elimina los términos de norma y deja solo los productos escalares $\mathbf{y}_k^\top \mathbf{y}_l$. De una matriz de productos escalares se recuperan las coordenadas por descomposición espectral, exactamente como en el análisis de componentes principales. Cuando la disimilitud no es euclidiana, como Bray-Curtis, aparecen autovalores negativos: no existe ninguna configuración de puntos que reproduzca las distancias. Si son pequeños, se ignoran; si son grandes, conviene aplicar una corrección o usar otra distancia.

14.2.9 PERMANOVA: ¿difieren los grupos?

Una ordenación sugiere, pero no prueba. Para contrastar si la composición difiere entre grupos (sanos frente a enfermos, dietas, tratamientos) necesitamos un análisis

de la varianza que funcione con cualquier disimilitud. Anderson (2001) lo formuló a partir de una identidad clásica: la suma de cuadrados de las distancias de los puntos a su centroide puede calcularse sin conocer el centroide, solo con las distancias entre pares.

Teorema 14.5. (Pseudo- F de PERMANOVA)

Sean N muestras repartidas en a grupos de tamaño n , y sea $\varepsilon_{kl} = 1$ si k y l pertenecen al mismo grupo (o en otro caso). Con

$$SS_T = \frac{1}{N} \sum_{k < l} d_{kl}^2, \quad SS_W = \frac{1}{n} \sum_{k < l} d_{kl}^2 \varepsilon_{kl}, \quad SS_A = SS_T - SS_W, \quad (14.19)$$

el estadístico

$$F = \frac{SS_A / (a - 1)}{SS_W / (N - a)} \quad (14.20)$$

se compara con su distribución bajo la hipótesis nula de “grupos intercambiables”, obtenida permutando las etiquetas de grupo. El valor p es $(\#\{F^* \geq F\} + 1) / (P + 1)$ para P permutaciones.

DONDE

SS_T, SS_W, SS_A	Sumas de cuadrados total, dentro de los grupos y entre grupos.
ε_{kl}	Indicador de que las muestras k y l pertenecen al mismo grupo.
F^*	Valor del estadístico en un conjunto de datos con etiquetas permutadas.
P	Número de permutaciones (típicamente 999).

Con la distancia euclidiana, este F coincide con el del ANOVA clásico; con cualquier otra disimilitud, conserva la lógica pero pierde la distribución F de Fisher, y por eso se recurre a permutaciones. La proporción $R^2 = SS_A / SS_T$ indica qué fracción de la variación total explican los grupos.

diversidad.py

```
import numpy as np
from scipy.spatial.distance import pdist, squareform
from skbio import DistanceMatrix
from skbio.stats.distance import permanova
from skbio.stats.ordination import pcoa
```

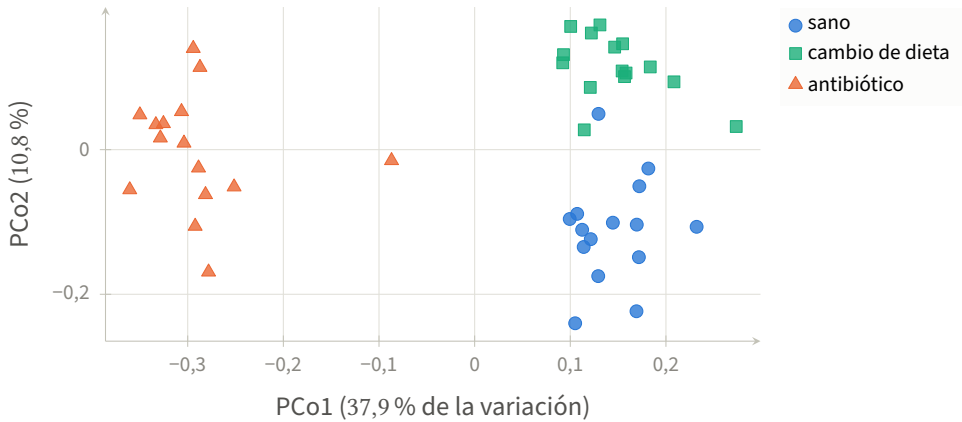



Figura 14.8. Ordenación PCoA de 45 muestras simuladas (60 taxones, 10 000 lecturas cada una) a partir de distancias de Bray-Curtis. El grupo “antibiótico” perdió sus ocho taxones más abundantes y se separa a lo largo del primer eje; el grupo “cambio de dieta” se solapa parcialmente con el grupo sano. PERMANOVA con 999 permutaciones: pseudo- $F = 17,9$, $R^2 = 0,46$, $p = 0,001$ (el mínimo posible con 999 permutaciones). Los dos ejes representan juntos menos de la mitad de la variación, y la matriz tiene 13 autovalores negativos (Bray-Curtis no es euclidiana): el plano es una sombra de la estructura, no la estructura completa.

```
def hill(p, q):
    p = p[p > 0] / p.sum()
    if np.isclose(q, 1):
        return np.exp(-(p * np.log(p)).sum())
    return (p ** q).sum() ** (1 / (1 - q))

# X: matriz muestras x taxones (proporciones); grupos: etiquetas
D = squareform(pdist(X, metric="braycurtis"))
dm = DistanceMatrix(D, ids=[f"m{k}" for k in range(len(X))])
ordenacion = pca(dm) # .samples, .proportion_explained
print(permanova(dm, grupos, permutations=999))
```

Una advertencia: PERMANOVA es sensible tanto a diferencias de posición (centroides) como de dispersión; si un grupo es mucho más variable que otro, puede dar significativo aunque los centroides coincidan, sobre todo con grupos de tamaño desigual. Acompáñelo de una prueba de homogeneidad de dispersiones (PERMDISP) y de la inspección visual de la ordenación.

El Proyecto del Microbioma Humano

El consorcio del *Human Microbiome Project* secuenció el 16S y los metagenomas de cientos de muestras de 242 adultos sanos, en sitios que iban de la boca a la vagina (Human Microbiome Project Consortium, 2012). Usó exactamente las herramientas de esta sección (diversidad alfa, UniFrac, ordenaciones) y encontró que el sitio del cuerpo es, con mucho, el principal determinante de la composición: la diversidad alfa era alta en la cavidad oral y baja en la vagina, y la diversidad beta entre personas era considerable en todos los sitios. El hallazgo más citado fue que, pese a esa variabilidad taxonómica, los perfiles de rutas metabólicas eran notablemente estables entre individuos: distintos elencos de microorganismos parecen desempeñar los mismos papeles.

14.3 Metagenómica shotgun

 Colab · 14.3 · Metagenómica shotgun (Kraken2)

 repositorio del curso

La biblioteca triturada

Suponga que alguien pasa por una trituradora de papel todos los libros de una biblioteca de barrio (novelas, recetarios, manuales de fontanería, algunos volúmenes únicos que no existen en ninguna otra parte) y le entrega a usted una bolsa con millones de tiras de papel. Hay dos cosas que puede hacer. Si tiene acceso al catálogo de una gran biblioteca nacional, puede tomar cada tira, buscar sus palabras en el catálogo y anotar de qué libro proviene; las tiras de los libros que el catálogo no conoce quedarán sin identificar. O bien puede ignorar el catálogo y empezar a pegar tiras que se solapan hasta reconstruir páginas, y luego agrupar las páginas por el tipo de letra y el color del papel hasta recomponer libros enteros, incluidos los que nadie había catalogado. La metagenómica shotgun hace ambas cosas: la primera es la *clasificación de lecturas*; la segunda, el *ensamblaje y binning*.

14.3.1 Del amplicón al metagenoma

En la metagenómica shotgun se extrae todo el ADN de la muestra, se fragmenta al azar y se secuencia sin ninguna amplificación dirigida. Las ventajas frente al 16S son considerables. Se eliminan los sesgos de los cebadores y del número de copias

del operón ribosómico. Se observan todos los dominios de la vida a la vez: bacterias, arqueas, hongos, protistas y virus, que no tienen 16S. La resolución llega a la especie y, a menudo, a la cepa. Y, sobre todo, se secuencian los *genes*: podemos preguntar no solo quién está, sino qué puede hacer la comunidad (qué rutas metabólicas codifica, qué genes de resistencia porta).

Los inconvenientes también lo son. El costo por muestra es mucho mayor, porque hace falta una profundidad de millones a decenas de millones de lecturas. En muestras de tejido, heces de lactantes o hisopos, la mayor parte del ADN puede ser del hospedador, que hay que retirar por mapeo contra su genoma antes de cualquier análisis. Y la teoría de cobertura del capítulo 6 impone un límite duro a los organismos raros. Si una muestra se secuencia con R lecturas de longitud ℓ y un genoma de tamaño G representa una fracción α del ADN, su cobertura esperada es

$$c = \frac{R \ell \alpha}{G}, \quad P(\text{base no cubierta}) \approx e^{-c}. \tag{14.21}$$

DONDE	
c	Cobertura media esperada del genoma del organismo.
R, ℓ	Número de lecturas y longitud de cada una.
α	Fracción del ADN secuenciado que proviene de ese organismo (abundancia relativa en ADN).
G	Tamaño del genoma, en pares de bases.

Con 2×10^7 pares de lecturas de 150 bases ($R = 4 \times 10^7$) y un genoma bacteriano de 4 Mb, un organismo con el 1 % del ADN alcanza $c = 15$ y prácticamente todo su genoma queda cubierto ($e^{-15} \approx 3 \times 10^{-7}$). Uno con el 0,1 % queda en $c = 1,5$: el 22 % de sus bases no recibe ninguna lectura, y su ensamblaje será imposible. Su presencia, en cambio, sí puede detectarse clasificando lecturas: para eso bastan unas pocas docenas.

14.3.2 Clasificar lecturas por k -mers exactos

La forma más directa de identificar una lectura es alinearla contra todos los genomas conocidos con BLAST (capítulo 3). Con decenas de millones de lecturas y cientos de gigabases de referencia, eso es inviable. Wood y Salzberg (2014) propusieron en Kraken sustituir el alineamiento por la búsqueda de k -mers exactos (capítulo 8) en una base de datos precalculada, lo que convirtió la clasificación en una sucesión de consultas a una tabla hash, órdenes de magnitud más rápida que los métodos basados en alineamiento.

La primera pregunta es qué longitud de k -mer usar. Si las bases fueran independientes y equiprobables, un k -mer concreto aparecería por azar en una posición dada con probabilidad 4^{-k} , y el número esperado de apariciones espurias en una base de referencia de B bases sería

$$E[\text{coincidencias espurias}] \approx B \cdot 4^{-k}. \quad (14.22)$$

Con $B = 10^{11}$ bases, un 15-mer aparecería por azar unas 93 veces, un 21-mer 0,02 veces y un 31-mer 2×10^{-8} veces. Un 31-mer compartido entre una lectura y un genoma es, casi con seguridad, evidencia de ascendencia común, no casualidad. Pero los k -mers largos tienen un precio: un solo error de secuenciación o una sola diferencia entre la cepa de la muestra y la de referencia destruye los k k -mers que la contienen. Kraken usó $k = 31$; Kraken 2 usa por defecto $k = 35$.

14.3.3 El ancestro común más bajo

Muchos k -mers no son exclusivos de un genoma: un 31-mer de un gen ribosómico puede estar en todos los miembros de una familia. Kraken resuelve la ambigüedad al construir la base, no al consultarla.

Definición 14.6. (Ancestro común más bajo (LCA))

Dado un conjunto de taxones T en un árbol taxonómico, su ancestro común más bajo, $LCA(T)$, es el nodo más profundo del árbol del que descienden todos los elementos de T . En la base de Kraken, cada k -mer κ se asocia a $\tau(\kappa) = LCA\{\text{taxones cuyos genomas contienen } \kappa\}$.

Un k -mer presente solo en *Escherichia coli* apunta a la especie; uno presente en *E. coli* y en *Shigella flexneri* apunta a la familia *Enterobacteriaceae*; uno presente en toda bacteria apunta al dominio. Así, cada k -mer lleva la etiqueta más específica que la base de datos puede justificar. Para clasificar una lectura, Kraken consulta todos sus k -mers y construye el *árbol de clasificación*: el subárbol formado por los taxones $\tau(\kappa)$ encontrados y sus ancestros, con un peso w_t igual al número de k -mers de la lectura asignados a cada nodo t .

Teorema 14.6. (Regla de clasificación de Kraken)

Para cada hoja h del árbol de clasificación, sea $\text{camino}(h)$ el conjunto de nodos desde la raíz hasta h , y

$$\text{RTL}(h) = \sum_{t \in \text{camino}(h)} w_t. \quad (14.23)$$

La lectura se asigna a la hoja de máxima puntuación RTL; si varias empatan, se asigna al LCA de las hojas empatadas. Si ningún k -mer se encuentra en la base, la lectura queda sin clasificar.

DONDE

w_t	Número de k -mers de la lectura cuyo LCA en la base es el taxón t .
$\text{RTL}(h)$	Puntuación del camino de la raíz a la hoja (<i>root-to-leaf</i>): todos los k -mers que son compatibles con que la lectura provenga de h .

La regla tiene una lógica clara: un k -mer asignado a la familia es compatible con cualquier especie de la familia, así que “vota” por todos los caminos que pasan por ella; un k -mer asignado a una especie solo vota por esa especie. El camino ganador es el que explica más k -mers de la lectura.

Ejemplo 14.4. (Leer el árbol de la figura 14.9)

Una lectura de 150 pb tiene $150 - 35 + 1 = 116$ k -mers. De ellos, 36 no aparecen en la base (quizá la cepa de la muestra difiere de las de referencia) y 80 sí. Las puntuaciones de los cuatro caminos son $\text{RTL}(E. coli) = 2 + 3 + 10 + 12 + 40 = 67$, $\text{RTL}(E. albertii) = 2 + 3 + 10 + 12 + 4 = 31$, $\text{RTL}(S. flexneri) = 2 + 3 + 10 + 1 + 6 = 22$ y $\text{RTL}(S. enterica) = 2 + 3 + 10 + 0 + 2 = 17$. La lectura se asigna a *E. coli*. Observe que los 6 k -mers exclusivos de *S. flexneri* no bastan para cambiar la decisión: en bacterias con mucha transferencia horizontal, que una lectura contenga algunos k -mers de otra especie es lo esperable.

14.3.4 Kraken 2: minimizadores y confianza

La base original de Kraken almacenaba todos los 31-mers con su LCA, lo que con las bases actuales exigía cientos de gigabytes de memoria. Wood et al. (2019) rediseñaron el sistema en Kraken 2 con dos ideas. La primera es almacenar solo

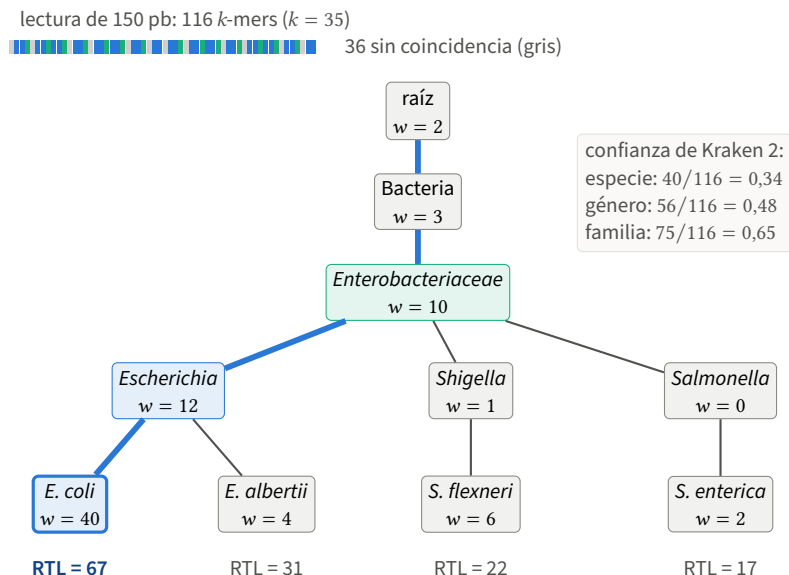


Figura 14.9. Clasificación de una lectura con Kraken. Cada k -mer de la lectura se busca en la base y devuelve el LCA de los genomas que lo contienen; w es el número de k -mers asignados a cada nodo (80 de 116 tienen coincidencia; la barra superior es solo ilustrativa). La puntuación de cada camino de la raíz a una hoja (RTL) suma los pesos de sus nodos: el camino hasta *E. coli* ($2 + 3 + 10 + 12 + 40 = 67$, en azul) gana con claridad y la lectura se asigna a esa especie. Con el umbral de confianza de Kraken 2 (recuadro), la asignación se mantiene en *E. coli* si el umbral es 0,1, pero sube hasta *Enterobacteriaceae* si es 0,5.

minimizadores: de cada k -mer se guarda su subcadena de longitud $\ell < k$ ($\ell = 31$ por defecto) de menor valor según un orden pseudoaleatorio; como k -mers consecutivos suelen compartir minimizador, el número de entradas cae drásticamente. La segunda es una tabla hash compacta y probabilística que guarda solo unos bits del hash junto con el LCA: a cambio de una pequeña tasa de falsas coincidencias, la memoria se reduce a una fracción y la velocidad aumenta. Kraken 2 admite además búsqueda traducida y bases de 16S como SILVA.

Kraken 2 añade un *umbral de confianza*. Para un nodo candidato t , la puntuación de confianza es

$$\text{conf}(t) = \frac{\#\{\kappa \text{ de la lectura} : \tau(\kappa) \in \text{subárbol}(t)\}}{\#\{\kappa \text{ de la lectura sin bases ambiguas}\}}. \quad (14.24)$$

Si la confianza del nodo asignado no alcanza el umbral elegido, la asignación sube por el árbol hasta el primer ancestro que sí lo alcanza. En el ejemplo, la especie

tiene $40/116 = 0,34$; el género, que acumula sus k -mers y los de sus especies, $(12 + 40 + 4)/116 = 0,48$; la familia, $75/116 = 0,65$. El umbral por defecto es 0, que maximiza la sensibilidad a costa de falsos positivos; en aplicaciones clínicas, donde un único falso positivo puede significar un diagnóstico erróneo, se usan umbrales más altos.

⚠ La base de datos es el experimento

Un clasificador de k -mers solo reconoce lo que su base contiene: las lecturas de una especie ausente se asignan al pariente más cercano o quedan sin clasificar, y una base contaminada produce detecciones fantasma. Informe la versión exacta de la base y el porcentaje de lecturas sin clasificar, y desconfíe de especies detectadas con pocas lecturas en muestras de baja biomasa, donde dominan los contaminantes de los reactivos.

14.3.5 De lecturas clasificadas a abundancias: Bracken

Kraken asigna cada lectura al nodo más específico que puede justificar, y eso tiene un efecto indeseado sobre las abundancias: las especies con parientes cercanos en la base (muchos k -mers compartidos) dejan buena parte de sus lecturas en el género o la familia, mientras que las especies “solitarias” retienen casi todas las suyas en la especie. Contar solo las lecturas asignadas a especie subestima de forma sistemática a las primeras. Lu et al. (2017) corrigieron este sesgo con Bracken, que redistribuye las lecturas asignadas a niveles superiores hacia las especies que probablemente las originaron.

Bracken empieza simulando lecturas de cada genoma de la base y clasificándolas con Kraken: así estima, para cada especie S_i , la probabilidad $P(G | S_i)$ de que una lectura suya quede asignada a un nodo superior G . Con la regla de Bayes, la probabilidad de que una lectura asignada a G provenga de S_i es

$$P(S_i | G) = \frac{P(G | S_i) P(S_i)}{\sum_k P(G | S_k) P(S_k)}, \quad \hat{n}_i = K_i + n_G P(S_i | G), \quad (14.25)$$

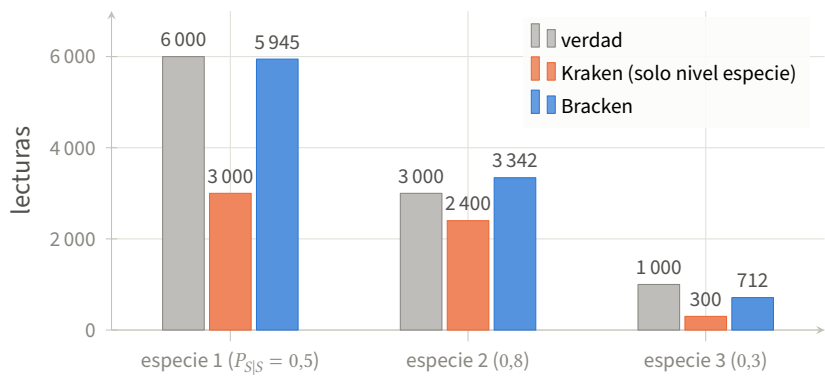


Figura 14.10. Tres especies de un mismo género con 6000, 3000 y 1000 lecturas verdaderas. Kraken asigna a nivel de especie solo la fracción P_{SjS} de las lecturas de cada una; el resto (4300 lecturas) se queda en el género. Contar solo el nivel de especie (naranja) invierte el orden aparente de las especies 1 y 2 y reduce la especie 3 a un 30 % de su valor. Bracken (azul) reparte las lecturas del género con la ecuación (14.25) y recupera el orden y, aproximadamente, las magnitudes (5945, 3342, 712). El error residual proviene de que la probabilidad previa $P(S_i)$ se estima con los conteos sesgados de Kraken.

DONDE	
$P(G \mid S_i)$	Fracción de las lecturas de la especie S_i que Kraken deja en el nodo superior G (estimada por simulación).
$P(S_i)$	Abundancia relativa previa de S_i dentro de G , estimada a partir de las lecturas asignadas a nivel de especie.
K_i	Lecturas asignadas directamente a la especie S_i .
n_G	Lecturas asignadas al nodo G (y no a sus descendientes).
$\hat{n}_i$	Estimación de Bracken del número de lecturas originadas en S_i .

Ejemplo 14.5. (Bracken en números)

En la figura 14.10, Kraken deja $K = (3000, 2400, 300)$ lecturas a nivel de especie y $n_G = 4300$ en el género. La probabilidad previa se estima con los conteos de especie, $P(S_i) = K_i / \sum K = (0,526; 0,421; 0,053)$, y $P(G \mid S_i) = 1 - P_{SjS} = (0,5; 0,2; 0,7)$. Los productos son $(0,263; 0,084; 0,037)$, que normalizados dan $P(S_i \mid G) = (0,685; 0,219; 0,096)$. Las estimaciones finales

son $\hat{n} = (3000 + 2945, 2400 + 942, 300 + 412) = (5945, 3342, 712)$, frente a los valores verdaderos (6000, 3000, 1000).

14.3.6 Perfilado con genes marcadores: MetaPhlAn

Una estrategia alternativa se concentra solo en las lecturas informativas. MetaPhlAn usa *marcadores específicos de clado*: genes presentes en todos los genomas de un clado y en ninguno fuera de él. La abundancia de cada clado se estima con la cobertura media robusta de sus marcadores, normalizada por longitud; como son de copia única, el resultado se aproxima a una abundancia de *células*, no de ADN. La cuarta versión (Blanco-Míguez et al., 2023) construyó sus marcadores a partir de alrededor de un millón de genomas de referencia y MAGs, agrupados en unos 27 000 *bins* genómicos a nivel de especie (SGBs), miles de ellos sin nombre: especies conocidas solo por su genoma.

Clasificación y perfilado de un metagenoma

```
kraken2 --db k2_standard --threads 8 --paired \
  --confidence 0.1 --report m1.k2report --output m1.kraken2 \
  m1_R1.fastq.gz m1_R2.fastq.gz
bracken -d k2_standard -i m1.k2report -o m1.bracken \
  -r 150 -l S -t 10
metaphlan m1_R1.fastq.gz,m1_R2.fastq.gz --input_type fastq \
  --nproc 8 -o m1_metaphlan.txt
```

14.3.7 Ensamblaje metagenómico

Clasificar lecturas contra una referencia solo encuentra lo ya conocido. Para descubrir genomas nuevos hay que ensamblarlos. El ensamblaje metagenómico hereda todas las ideas del ensamblaje de un genoma aislado (grafos de de Bruijn, resolución de repeticiones; capítulo 8), pero rompe dos de sus supuestos. Primero, la cobertura no es uniforme: varía en órdenes de magnitud entre organismos, de modo que no puede usarse un único umbral de cobertura para distinguir errores de secuencia real, ni la cobertura para detectar repeticiones. Segundo, conviven cepas muy parecidas de una misma especie, cuyas pequeñas diferencias aparecen en el grafo como “burbujas” que se confunden con errores de secuenciación.

metaSPAdes (Nurk et al., 2017) adapta el ensamblador SPAdes a este escenario: usa umbrales de cobertura locales en lugar de globales y, ante variantes de cepa, construye primero un *consenso* de la especie (colapsando las burbujas de variación

entre cepas) y luego usa ese esqueleto para resolver repeticiones. MEGAHIT (Li et al., 2015) prioriza la eficiencia: representa el grafo de de Bruijn en forma *sucinta* (una estructura de datos comprimida) y recorre iterativamente varios valores de k , de pequeños a grandes, lo que le permite ensamblar metagenomas de suelo con cientos de gigabases en un solo servidor. Ninguno resuelve bien las cepas muy cercanas: en la evaluación CAMI, los ensambladores reconstruyeron bien los genomas sin parientes cercanos en la muestra, pero su rendimiento cayó sustancialmente con cepas estrechamente emparentadas (Sczyrba et al., 2017).

14.3.8 Binning: agrupar contigs en genomas

El ensamblaje produce decenas de miles de *contigs* sin etiqueta de origen. El *binning* los agrupa en conjuntos (*bins*) que idealmente corresponden cada uno a un genoma. Se basa en dos señales que un contig hereda de su genoma. La *composición*: cada genoma tiene un contenido de GC y, más finamente, una frecuencia de tetranucleótidos (TNF; 136 una vez unidos cada uno y su reverso complementario) característicos y bastante estables a lo largo del genoma, por el sesgo de uso de codones y de la maquinaria de replicación y reparación. Y la *cobertura*: todos los contigs de un genoma tienen aproximadamente la misma profundidad y, si se secuencian varias muestras, su cobertura sube y baja de forma concertada; el perfil de coberturas en M muestras es una firma muy discriminante.

MetaBAT 2 (Kang et al., 2019) combina ambas señales en una distancia entre pares de contigs (una probabilidad de que dos contigs procedan de genomas distintos, calculada con la TNF y con la cobertura), construye un grafo con los contigs como nodos y las afinidades como aristas y lo particiona con un algoritmo de propagación de etiquetas. Su versión 2 adapta automáticamente los umbrales al conjunto de datos, lo que elimina la necesidad de ajustar parámetros a mano. El producto final de este proceso son los *genomas ensamblados a partir de metagenomas* (MAGs).

14.3.9 ¿Es bueno un MAG? CheckM y MIMAG

Un *bin* puede estar incompleto (le faltan contigs que quedaron en otro bin o no se ensamblaron) o contaminado (contiene contigs de otro organismo). Sin un genoma de referencia, ¿cómo saberlo? Parks et al. (2015) propusieron en CheckM usar genes marcadores de copia única: genes que, en los genomas completos de un linaje, aparecen exactamente una vez. Si un *bin* contiene el 90 % de esos marcadores, está completo en torno al 90 %; si algunos aparecen dos veces, probablemente hay

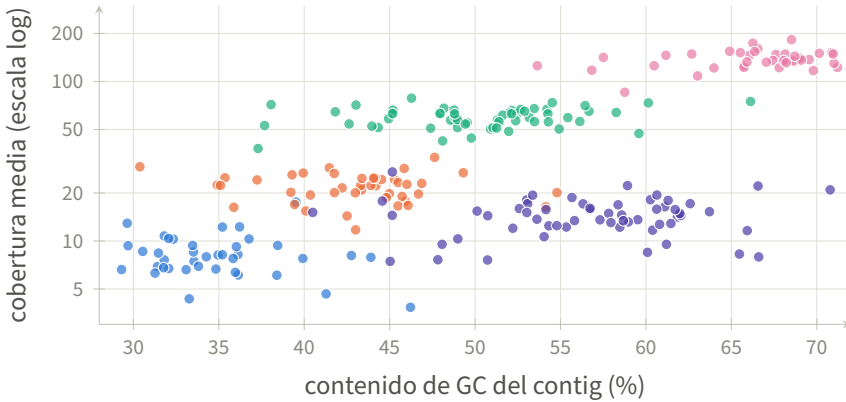


Figura 14.11. Principio del *binning*: 260 contigs simulados de cinco genomas, situados según su contenido de GC y su cobertura; el área de cada punto es proporcional a la longitud del contig. Los contigs de cada genoma forman una nube compacta. Los contigs cortos (puntos pequeños) se dispersan más, porque el GC de una secuencia corta es una estimación ruidosa del de su genoma; por eso los binners descartan contigs por debajo de unos pocos kilobases. Dos genomas con GC parecido (azul y violeta) solo se separan gracias a la cobertura. Los binners reales, como MetaBAT 2, usan las 136 frecuencias de tetranucleótidos en lugar del GC y el perfil de cobertura en varias muestras, lo que separa genomas que en este plano se solaparían.

contigs de otro genoma. En su forma más simple, con un conjunto $\mathcal{M}$ de marcadores y c_g copias del marcador g en el bin,

$$\text{completitud} = \frac{\#\{g \in \mathcal{M} : c_g \geq 1\}}{|\mathcal{M}|}, \quad \text{contaminación} = \frac{\sum_{g \in \mathcal{M}} \max(c_g - 1, 0)}{|\mathcal{M}|}. \quad (14.26)$$

DONDE

$\mathcal{M}$	Conjunto de genes marcadores de copia única del linaje al que pertenece el bin.
c_g	Número de copias del marcador g encontradas en el bin.

CheckM refina esta idea de dos maneras: primero ubica el bin en un árbol de referencia para elegir marcadores específicos de su linaje (más numerosos y más fiables que los universales), y agrupa los marcadores que suelen estar juntos en el cromosoma en *conjuntos colocalizados*, para no contar varias veces la ausencia de un mismo fragmento. Con estas medidas, Bowers et al. (2017) definieron el estándar

Tabla 14.1. Categorías de calidad de genomas ensamblados a partir de metagenomas según el estándar MIMAG (Bowers et al., 2017).

Borrador	Compleitud	Contaminación	Requisitos adicionales
Alta calidad	> 90 %	< 5 %	ARNr 23S, 16S y 5S, y ≥ 18 ARNt
Calidad media	≥ 50 %	< 10 %	—
Baja calidad	< 50 %	< 10 %	—

MIMAG, que establece la información mínima que debe acompañar a un MAG publicado y sus categorías de calidad (tabla 14.1).

Ejemplo 14.6. (Clasificar un MAG)

Un bin contiene 95 de los 104 marcadores de copia única de su linaje, y 6 de ellos aparecen dos veces. Su completitud es $95/104 = 91,3\%$ y su contaminación $6/104 = 5,8\%$. Aunque supera el 90 % de completitud, la contaminación excede el 5 %: es un borrador de *calidad media*. Además, los genes de ARNr 16S rara vez se ensamblan bien en metagenomas (las copias casi idénticas del operón colapsan en el grafo), de modo que muchos MAGs con completitud y contaminación excelentes no alcanzan la categoría alta por ese requisito.

De lecturas a MAGs

```
megahit -1 m1_R1.fq.gz -2 m1_R2.fq.gz -o asm_m1 -t 16
bowtie2-build asm_m1/final.contigs.fa idx_m1
bowtie2 -x idx_m1 -1 m1_R1.fq.gz -2 m1_R2.fq.gz -p 16 \
  | samtools sort -o m1.bam
jgi_summarize_bam_contig_depths --outputDepth prof.txt m1.bam
metabat2 -i asm_m1/final.contigs.fa -a prof.txt -o bins/bin
checkm lineage_wf -x fa -t 16 bins/ checkm_out/
```

14.3.10 El resistoma

Una de las aplicaciones más relevantes de la metagenómica shotgun es la vigilancia de la resistencia a antibióticos. Wright (2007) llamó *resistoma* al conjunto de todos los genes que contribuyen a la resistencia a antibióticos en una comunidad, incluidos los de bacterias no patógenas y los genes “precursores” que podrían evolu-

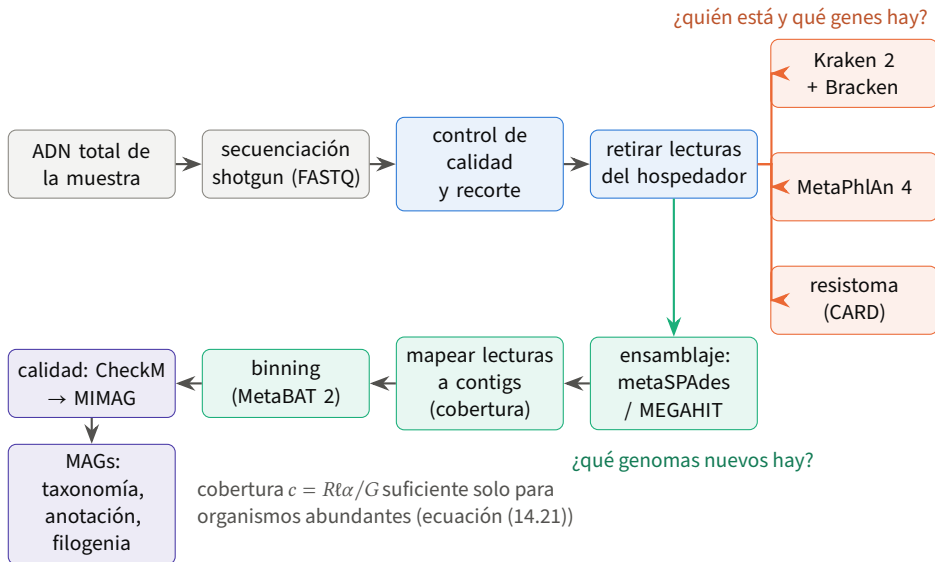


Figura 14.12. Del ADN total a los MAGs. Tras el control de calidad y la eliminación de lecturas del hospedador, el análisis se bifurca. La rama de perfilado (naranja) clasifica lecturas contra bases de referencia para estimar composición taxonómica y genes de resistencia; es rápida y sensible a organismos raros, pero ciega a lo desconocido. La rama de reconstrucción (verde) ensambla sin referencia, calcula la cobertura de cada contig mapeando las lecturas de vuelta, agrupa los contigs en bins y evalúa cada bin con marcadores de copia única; solo recupera organismos con cobertura suficiente, pero puede descubrir genomas de especies nunca cultivadas.

cionar hacia la resistencia. Las bacterias ambientales y comensales son un reservorio del que, por transferencia horizontal, los patógenos pueden adquirir resistencias. Para inventariarlo, las lecturas o los contigs se comparan con bases curadas como CARD, que asocia cada gen a su mecanismo, a la familia de antibióticos afectada y a modelos de detección con umbrales calibrados; su herramienta RGI permite además predecir resistomas a partir de metagenomas (Alcock et al., 2023).

CAMI: evaluar sin saber la verdad

¿Cómo saber qué herramienta funciona mejor si en una muestra real nunca conocemos la composición verdadera? La iniciativa CAMI (*Critical Assessment of Metagenome Interpretation*) construyó metagenomas sintéticos a partir de cientos de genomas recién secuenciados, que las herramientas no podían conocer, con abundancias y

cepas controladas, y pidió a los desarrolladores que los analizaran a ciegas (Sczyrba et al., 2017). Además de la dificultad de ensamblar cepas cercanas, el estudio mostró que el rendimiento de los clasificadores y perfiladores caía mucho en los rangos taxonómicos bajos para organismos sin parientes cercanos en las bases, y que la elección de parámetros cambiaba sustancialmente los resultados. La lección general es que las comparaciones de herramientas publicadas por sus propios autores deben complementarse con evaluaciones independientes y ciegas.

El perfilado por referencia responde rápido “quién está”, pero solo entre lo ya conocido; el ensamblaje y el binning descubren lo desconocido, pero solo entre lo abundante.

☰ En resumen

- El gen del ARNr 16S combina regiones conservadas, donde se diseñan los cebadores, con nueve regiones hipervariables que distinguen linajes. Su uso como marcador filogenético permitió a Woese y Fox descubrir las arqueas.
- Las OTUs al 97 % agrupan secuencias con un radio arbitrario; las ASVs se infieren con un modelo de error (λ_{μ}) y una prueba de abundancia de Poisson, tienen resolución de un nucleótido y son comparables entre estudios. Las quimeras se eliminan buscando dos padres más abundantes.
- La asignación taxonómica usa un clasificador bayesiano ingenuo de 8-mers con confianza por remuestreo; su resolución rara vez llega a especie con amplicones cortos.
- Riqueza, Shannon y Simpson son casos particulares de los números de Hill qD , que se miden en número efectivo de taxones y cumplen el principio de replicación. Chao1 es una cota inferior de la riqueza basada en *singletons* y *doubletons*.
- Rarificar para comparar diversidad es defendible; rarificar para abundancia diferencial desperdicia datos. Los datos de microbioma son composicionales: solo las razones entre taxones son interpretables sin información externa.
- Bray-Curtis, Jaccard y UniFrac responden preguntas distintas sobre la diferencia entre comunidades; PCoA las visualiza y PERMANOVA contrasta diferencias entre grupos mediante permutaciones.

- Kraken asigna cada k -mer al LCA de los genomas que lo contienen y clasifica cada lectura por el camino raíz-hoja de mayor peso; Kraken 2 usa minimizadores y un umbral de confianza, y Bracken corrige las abundancias redistribuyendo lecturas de niveles superiores.
- El ensamblaje metagenómico (metaSPAdes, MEGAHIT) y el binning por composición y cobertura (MetaBAT 2) reconstruyen MAGs, cuya calidad se evalúa con marcadores de copia única (CheckM) y se reporta según MIMAG.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 14.1** ●●● Calcule H , $1/D$, 0D , 1D y 2D para la comunidad con abundancias $(0,4; 0,3; 0,2; 0,1)$. Verifique numéricamente que qD es decreciente en q .
- 14.2** ●●● Usando la ecuación (14.2), calcule la abundancia mínima que DADA2 exigiría a una secuencia a dos nucleótidos de una variante con 20 000 lecturas, con $L = 250$ y $e = 0,005$. Repita con $\Omega_A = 10^{-5}$ y discuta cuántos falsos positivos esperarías con 50 000 secuencias únicas.
- 14.3** ●●● Demuestre el principio de replicación (teorema 14.3) para $q = 1$ directamente a partir de la entropía de Shannon, y muestre con un contraejemplo que el índice de Gini-Simpson no lo cumple.
- 14.4** ●●● Implemente la ecuación (14.11) y el estimador Chao1 en Python. Simule una comunidad con 500 taxones de abundancias log-normales, tome muestras de 1000, 10 000 y 100 000 lecturas, y grafique cómo cambian S_{obs} y $\hat{S}_{\text{Chao1}}$. ¿Converge Chao1 a 500?
- 14.5** ●●● Implemente en Python la regla RTL (teorema 14.6), construya el árbol de la figura 14.9 y compruebe las puntuaciones RTL. Añada la regla de confianza de Kraken 2 (ecuación (14.24)) y encuentre el umbral a partir del cual la lectura deja de asignarse al género.
- 14.6** ●●● Implemente PERMANOVA (ecuaciones (14.19) y (14.20)) desde cero y compárela con `skbio.stats.distance.permanova`. Luego simule dos grupos con el mismo centroide pero dispersiones distintas (una el triple de la otra) y tamaños 10 y 40: ¿con qué frecuencia rechaza la hipótesis nula al 5 %?

- 14.7** ●●● Un metagenoma de heces tiene 3×10^7 pares de lecturas de 150 pb, de las cuales el 40 % son humanas. ¿Qué abundancia relativa mínima debe tener una bacteria de 3 Mb para alcanzar una cobertura de 10? Si la contaminación esperada de un bin crece cuando dos cepas de la misma especie coexisten, proponga una forma de detectarlo con CheckM.

Referencias del capítulo

- Alcock, B. P., Huynh, W., Chalil, R., Smith, K. W., Raphenya, A. R., Wlodarski, M. A., Edalatmand, A., Petkau, A., et al. (2023). CARD 2023: Expanded curation, support for machine learning, and resistome prediction at the Comprehensive Antibiotic Resistance Database. *Nucleic Acids Research*, 51(D1), D690-D699. <https://doi.org/10.1093/nar/gkac920>
- Anderson, M. J. (2001). A new method for non-parametric multivariate analysis of variance. *Austral Ecology*, 26(1), 32-46. <https://doi.org/10.1111/j.1442-9993.2001.01070.pp.x>
- Blanco-Míguez, A., Beghini, F., Cumbo, F., McIver, L. J., Thompson, K. N., Zolfo, M., Manghi, P., Dubois, L., et al. (2023). Extending and improving metagenomic taxonomic profiling with uncharacterized species using MetaPhlAn 4. *Nature Biotechnology*, 41(11), 1633-1644. <https://doi.org/10.1038/s41587-023-01688-w>
- Bolyen, E., Rideout, J. R., Dillon, M. R., Bokulich, N. A., Abnet, C. C., Al-Ghalith, G. A., Alexander, H., Alm, E. J., et al. (2019). Reproducible, interactive, scalable and extensible microbiome data science using QIIME 2. *Nature Biotechnology*, 37(8), 852-857. <https://doi.org/10.1038/s41587-019-0209-9>
- Bowers, R. M., Kyrpides, N. C., Stepanauskas, R., Harmon-Smith, M., Doud, D., Reddy, T. B. K., Schulz, F., et al. (2017). Minimum information about a single amplified genome (MISAG) and a metagenome-assembled genome (MIMAG) of bacteria and archaea. *Nature Biotechnology*, 35(8), 725-731. <https://doi.org/10.1038/nbt.3893>
- Callahan, B. J., McMurdie, P. J., & Holmes, S. P. (2017). Exact sequence variants should replace operational taxonomic units in marker-gene data analysis. *The ISME Journal*, 11(12), 2639-2643. <https://doi.org/10.1038/ismej.2017.119>
- Callahan, B. J., McMurdie, P. J., Rosen, M. J., Han, A. W., Johnson, A. J. A., & Holmes, S. P. (2016). DADA2: High-resolution sample inference from Illumina amplicon data. *Nature Methods*, 13(7), 581-583. <https://doi.org/10.1038/nmeth.3869>
- Caporaso, J. G., Lauber, C. L., Walters, W. A., Berg-Lyons, D., Lozupone, C. A., Turnbaugh, P. J., Fierer, N., & Knight, R. (2011). Global patterns of 16S rRNA diversity at a depth of millions of sequences per sample. *Proceedings of the National Academy of Sciences*, 108(supplement_1), 4516-4522. <https://doi.org/10.1073/pnas.1000080107>
- Chao, A. (1987). Estimating the population size for capture-recapture data with unequal catchability. *Biometrics*, 43(4), 783-791. <https://doi.org/10.2307/2531532>

- Chao, A., Gotelli, N. J., Hsieh, T. C., Sander, E. L., Ma, K. H., Colwell, R. K., & Ellison, A. M. (2014). Rarefaction and extrapolation with Hill numbers: A framework for sampling and estimation in species diversity studies. *Ecological Monographs*, 84(1), 45-67. <https://doi.org/10.1890/13-0133.1>
- Edgar, R. C., Haas, B. J., Clemente, J. C., Quince, C., & Knight, R. (2011). UCHIME improves sensitivity and speed of chimera detection. *Bioinformatics*, 27(16), 2194-2200. <https://doi.org/10.1093/bioinformatics/btr381>
- Gloor, G. B., Macklaim, J. M., Pawlowsky-Glahn, V., & Egozcue, J. J. (2017). Microbiome datasets are compositional: And this is not optional. *Frontiers in Microbiology*, 8, 2224. <https://doi.org/10.3389/fmicb.2017.02224>
- Hill, M. O. (1973). Diversity and evenness: A unifying notation and its consequences. *Ecology*, 54(2), 427-432. <https://doi.org/10.2307/1934352>
- Human Microbiome Project Consortium. (2012). Structure, function and diversity of the healthy human microbiome. *Nature*, 486(7402), 207-214. <https://doi.org/10.1038/nature11234>
- Jost, L. (2006). Entropy and diversity. *Oikos*, 113(2), 363-375. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>
- Kang, D. D., Li, F., Kirton, E., Thomas, A., Egan, R., An, H., & Wang, Z. (2019). MetaBAT 2: An adaptive binning algorithm for robust and efficient genome reconstruction from metagenome assemblies. *PeerJ*, 7, e7359. <https://doi.org/10.7717/peerj.7359>
- Li, D., Liu, C.-M., Luo, R., Sadakane, K., & Lam, T.-W. (2015). MEGAHIT: An ultra-fast single-node solution for large and complex metagenomics assembly via succinct de Bruijn graph. *Bioinformatics*, 31(10), 1674-1676. <https://doi.org/10.1093/bioinformatics/btv033>
- Lozupone, C., & Knight, R. (2005). UniFrac: A new phylogenetic method for comparing microbial communities. *Applied and Environmental Microbiology*, 71(12), 8228-8235. <https://doi.org/10.1128/AEM.71.12.8228-8235.2005>
- Lu, J., Breitwieser, F. P., Thielen, P., & Salzberg, S. L. (2017). Bracken: Estimating species abundance in metagenomics data. *PeerJ Computer Science*, 3, e104. <https://doi.org/10.7717/peerj-cs.104>
- McMurdie, P. J., & Holmes, S. (2014). Waste not, want not: Why rarefying microbiome data is inadmissible. *PLoS Computational Biology*, 10(4), e1003531. <https://doi.org/10.1371/journal.pcbi.1003531>
- Nurk, S., Meleshko, D., Korobeynikov, A., & Pevzner, P. A. (2017). metaSPAdes: A new versatile metagenomic assembler. *Genome Research*, 27(5), 824-834. <https://doi.org/10.1101/gr.213959.116>
- Parks, D. H., Imelfort, M., Skennerton, C. T., Hugenholtz, P., & Tyson, G. W. (2015). CheckM: Assessing the quality of microbial genomes recovered from isolates, single cells, and metagenomes. *Genome Research*, 25(7), 1043-1055. <https://doi.org/10.1101/gr.186072.114>

- Quast, C., Pruesse, E., Yilmaz, P., Gerken, J., Schweer, T., Yarza, P., Peplies, J., & Glöckner, F. O. (2013). The SILVA ribosomal RNA gene database project: Improved data processing and web-based tools. *Nucleic Acids Research*, 41(D1), D590-D596. <https://doi.org/10.1093/nar/gks1219>
- Sczyrba, A., Hofmann, P., Belmann, P., Koslicki, D., Janssen, S., Dröge, J., Gregor, I., Majda, S., et al. (2017). Critical Assessment of Metagenome Interpretation—a benchmark of metagenomics software. *Nature Methods*, 14(11), 1063-1071. <https://doi.org/10.1038/nmeth.4458>
- Wang, Q., Garrity, G. M., Tiedje, J. M., & Cole, J. R. (2007). Naïve Bayesian classifier for rapid assignment of rRNA sequences into the new bacterial taxonomy. *Applied and Environmental Microbiology*, 73(16), 5261-5267. <https://doi.org/10.1128/AEM.00062-07>
- Woese, C. R., & Fox, G. E. (1977). Phylogenetic structure of the prokaryotic domain: The primary kingdoms. *Proceedings of the National Academy of Sciences*, 74(11), 5088-5090. <https://doi.org/10.1073/pnas.74.11.5088>
- Wood, D. E., Lu, J., & Langmead, B. (2019). Improved metagenomic analysis with Kraken 2. *Genome Biology*, 20(1), 257. <https://doi.org/10.1186/s13059-019-1891-0>
- Wood, D. E., & Salzberg, S. L. (2014). Kraken: Ultrafast metagenomic sequence classification using exact alignments. *Genome Biology*, 15(3), R46. <https://doi.org/10.1186/gb-2014-15-3-r46>
- Wright, G. D. (2007). The antibiotic resistome: The nexus of chemical and genetic diversity. *Nature Reviews Microbiology*, 5(3), 175-186. <https://doi.org/10.1038/nrmicro1614>

PARTE

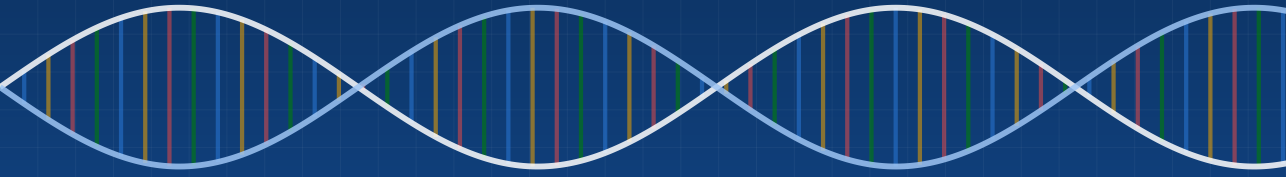


Estructura, sistemas e inteligencia

De la proteína a la red y al modelo

La última parte sube de escala: de la forma tridimensional de las proteínas a las redes que forman, de los modelos de aprendizaje automático a los flujos de trabajo que hacen todo esto reproducible.

Bioinformática estructural



La conformación nativa está determinada por la totalidad de las interacciones interatómicas y, por lo tanto, por la secuencia de aminoácidos, en un ambiente dado.

— Anfinsen (1973)

● Al terminar este capítulo usted podrá

- Describir los niveles de organización de una proteína, la geometría del enlace peptídico y los ángulos diedros ϕ y ψ , e interpretar un diagrama de Ramachandran construido con estructuras reales.
- Comparar la cristalografía de rayos X, la resonancia magnética nuclear y la criomicroscopía electrónica; leer en un archivo del PDB la resolución, el factor B y el método, y trabajar con el formato mmCIF.
- Derivar el algoritmo de Kabsch mediante la descomposición en valores singulares, calcular el RMSD y el TM-score entre dos estructuras y explicar por qué miden cosas distintas.
- Explicar cómo la coevolución, el aprendizaje profundo y los modelos de lenguaje permitieron predecir estructuras con exactitud experimental (AlphaFold2,

RoseTTAFold, ESMFold, AlphaFold3) e interpretar sus medidas de confianza pLDDT y PAE.

- Formular el acoplamiento molecular (*docking*) como un problema de búsqueda y puntuación, ejecutar AutoDock Vina, validar un protocolo por *redocking* y reconocer sus limitaciones.

Hasta ahora hemos tratado las proteínas sobre todo como secuencias: las alineamos, agrupamos en familias, anotamos y contamos. Para entender su actividad necesitamos también su estructura tridimensional, cuyo plegamiento se describe a escala de décimas de nanómetro. Sus cavidades pueden reconocer sustratos, sus superficies interactúan con otras proteínas y sus movimientos transmiten señales. La bioinformática estructural estudia cómo describir, almacenar y comparar esas formas, cómo predecirlas a partir de la secuencia y cómo usarlas para investigar funciones y diseñar fármacos.

El capítulo avanza en tres lecciones. En la sección 15.1 construiremos el vocabulario geométrico (el enlace peptídico, los ángulos ϕ/ψ , las hélices y láminas), veremos cómo se determinan experimentalmente las estructuras, qué contiene el Protein Data Bank y cómo se superponen y comparan dos estructuras con el algoritmo de Kabsch. En la sección 15.2 recorreremos medio siglo de intentos de predecir la estructura a partir de la secuencia, desde el modelado por homología hasta AlphaFold, y aprenderemos a leer las medidas de confianza que acompañan a cada modelo. En la sección 15.3 usaremos estructuras para predecir cómo se une una molécula pequeña a una proteína, el punto de partida del diseño racional de fármacos. El formato PDB y las bases de datos estructurales ya aparecieron en el capítulo 2, y los perfiles y alineamientos múltiples del capítulo 4 reaparecerán aquí como la materia prima de la predicción.

15.1 Estructura de proteínas y el Protein Data Bank

 Colab · 15.1 · Estructura de proteínas y PDB

 repositorio del curso

Un collar de cuentas con bisagras rígidas

Tome un collar hecho de pequeñas placas planas, unidas por sus esquinas con remaches que solo pueden girar. Cada placa es rígida; toda la flexibilidad

del collar está en los remaches. Si lo deja caer sobre la mesa adopta una forma cualquiera, pero si las placas llevaran imanes de distintas intensidades, algunas formas serían mucho más estables que otras, y el collar tendería a “encontrarlas” por sí solo. Una proteína es ese collar. Las placas son los enlaces peptídicos, planos por razones químicas; los remaches son los dos enlaces de cada carbono alfa, alrededor de los cuales la cadena puede girar; y los imanes son los puentes de hidrógeno, el efecto hidrofóbico y las interacciones electrostáticas. Describir una estructura, en el fondo, es dar la lista de los ángulos de giro de cada remache.

15.1.1 Cuatro niveles de organización

Desde los trabajos de Linderstrøm-Lang en los años cincuenta se describe la estructura de una proteína en cuatro niveles jerárquicos, que siguen siendo el lenguaje común de la disciplina (Branden & Tooze, 1999):

- **Estructura primaria:** la secuencia de aminoácidos, leída del extremo amino (N) al carboxilo (C). Es lo que codifica el gen y lo que hemos estudiado hasta ahora.
- **Estructura secundaria:** patrones locales y repetitivos del esqueleto, estabilizados por puentes de hidrógeno entre grupos N–H y C=O de la cadena principal. Las dos formas dominantes son la hélice α y la lámina β ; el resto se agrupa en giros y lazos.
- **Estructura terciaria:** la disposición tridimensional completa de una cadena, es decir, cómo se empaquetan hélices y láminas entre sí. Sus unidades de plegamiento independientes se llaman *dominios*, y suelen tener entre 50 y 250 residuos.
- **Estructura cuaternaria:** la asociación de varias cadenas en un complejo, como las cuatro subunidades de la hemoglobina.

Esta jerarquía no es solo un recurso didáctico. Refleja la física del plegamiento: la estructura secundaria se forma en microsegundos por interacciones locales, mientras que la terciaria requiere que partes distantes de la secuencia se encuentren. Y refleja la evolución: los dominios se recombinan como piezas de un mecano, y dos proteínas pueden compartir un dominio con el mismo plegamiento aunque sus secuencias hayan divergido hasta ser irreconocibles.

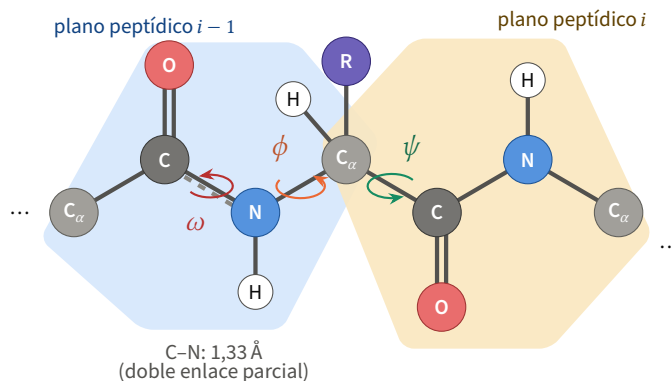


Figura 15.1. Dos planos peptídicos consecutivos (sombreados) articulados en el C_{α} del residuo i . El enlace C-N tiene carácter de doble enlace parcial (línea discontinua) y mantiene coplanares los seis átomos de cada plano; el ángulo ω en torno a él vale casi siempre 180° (configuración *trans*). Toda la libertad conformational del esqueleto se concentra en dos giros por residuo: ϕ , alrededor del enlace N- C_{α} , y ψ , alrededor del enlace C_{α} -C. La cadena lateral R sobresale del C_{α} y es la que choca estéricamente con el esqueleto cuando (ϕ, ψ) toman valores prohibidos.

15.1.2 El enlace peptídico y los ángulos diedros

Cada aminoácido aporta al esqueleto tres átomos pesados: el nitrógeno amida (N), el carbono alfa (C_{α}), que porta la cadena lateral R, y el carbono carbonílico (C). Los residuos se unen por un enlace amida entre el C de uno y el N del siguiente, el *enlace peptídico*. Por resonancia entre las formas $C=O / C-N$ y $C-O^{-} / C=N^{+}$, este enlace tiene un carácter de doble enlace parcial: mide unos 1,33 Å, más corto que un enlace C-N simple (1,47 Å), y no gira libremente. En consecuencia, los seis átomos C_{α} , C, O, N, H y el C_{α} siguiente quedan en un mismo plano, el *plano peptídico* (figura 15.1).

Si los planos son rígidos, la conformación del esqueleto queda determinada por los ángulos de giro en torno a los dos enlaces restantes de cada C_{α} . Esos giros se miden con ángulos *diedros*.

Definición 15.1. (Ángulo diedro)

Dados cuatro átomos consecutivos con posiciones $\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d} \in \mathbb{R}^3$, sean $\mathbf{b}_1 = \mathbf{b} - \mathbf{a}$, $\mathbf{b}_2 = \mathbf{c} - \mathbf{b}$, $\mathbf{b}_3 = \mathbf{d} - \mathbf{c}$, y $\mathbf{n}_1 = \mathbf{b}_1 \times \mathbf{b}_2$, $\mathbf{n}_2 = \mathbf{b}_2 \times \mathbf{b}_3$ las normales de los dos planos que forman. El ángulo diedro es

$$\theta = \text{atan2}\left((\mathbf{n}_1 \times \mathbf{n}_2) \cdot \frac{\mathbf{b}_2}{\|\mathbf{b}_2\|}, \mathbf{n}_1 \cdot \mathbf{n}_2\right) \in (-180^\circ, 180^\circ]. \quad (15.1)$$

Para el residuo i : $\phi_i = \theta(C_{i-1}, N_i, C_{\alpha,i}, C_i)$, $\psi_i = \theta(N_i, C_{\alpha,i}, C_i, N_{i+1})$ y $\omega_i = \theta(C_{\alpha,i}, C_i, N_{i+1}, C_{\alpha,i+1})$.

DONDE

$\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}$	Coordenadas cartesianas (en Å) de cuatro átomos enlazados en secuencia.
$\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3$	Vectores de enlace; $\mathbf{b}_2$ es el eje de giro.
$\mathbf{n}_1, \mathbf{n}_2$	Normales a los planos $(\mathbf{a}, \mathbf{b}, \mathbf{c})$ y $(\mathbf{b}, \mathbf{c}, \mathbf{d})$.
$\text{atan2}(y, x)$	Arco tangente de dos argumentos: devuelve el ángulo con el signo correcto en los cuatro cuadrantes.
ϕ_i, ψ_i, ω_i	Diedros del esqueleto del residuo i .

El signo importa: mirando a lo largo del eje $\mathbf{b}_2$, un diedro positivo corresponde a un giro en sentido horario del enlace lejano respecto al cercano. Con $\omega \approx 180^\circ$ fijo, una proteína de n residuos queda descrita, en primera aproximación, por $2n$ números: sus pares (ϕ_i, ψ_i) . Es una reducción enorme respecto a las $3 \times 8n$ coordenadas cartesianas de sus átomos pesados, y es la razón por la que tantas ideas de este capítulo, desde el diagrama de Ramachandran hasta los métodos de predicción, trabajan en el espacio de los ángulos.

15.1.3 El diagrama de Ramachandran

No todos los pares (ϕ, ψ) son posibles. Ramachandran et al. (1963) modelaron los átomos como esferas rígidas con distancias mínimas de contacto y calcularon, para un dipéptido, qué combinaciones de ángulos provocan choques entre los átomos del esqueleto y el C_β de la cadena lateral. El resultado, un mapa de regiones permitidas en el plano (ϕ, ψ) , se conoce desde entonces como *diagrama de Ramachandran*. Lo notable es que se obtuvo *antes* de que existieran suficientes estructuras para comprobarlo: es una predicción de pura stereoquímica.

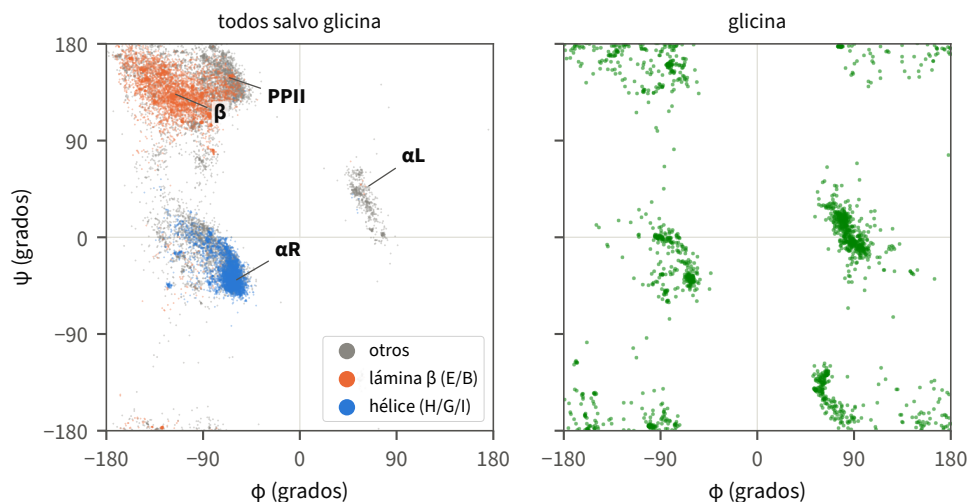


Figura 15.2. Diagrama de Ramachandran de 19 460 residuos de 79 estructuras cristalográficas del PDB con resolución igual o mejor que 1,0 Å, coloreados según la estructura secundaria asignada por DSSP. **Izquierda:** todos los residuos excepto glicina. Se distinguen la región de hélice derecha (α_R), la amplia región β en el cuadrante superior izquierdo, con la hélice de poliprolina II (PPII) en su borde derecho, y la pequeña isla de hélice izquierda (α_L). **Derecha:** la glicina, sin cadena lateral, ocupa regiones simétricas prohibidas para los demás residuos: el 55 % de sus residuos tiene $\phi > 0$, frente al 3,4 % del resto. Datos descargados del RCSB y procesados con `figuras/cap15/generar.py`.

La figura 15.2 muestra el diagrama construido con estructuras reales de muy alta resolución. Para cada residuo calculamos (ϕ, ψ) y su estructura secundaria con DSSP (que veremos enseguida). Tres regiones concentran casi todos los puntos:

- **Hélice derecha (α_R)**, en torno a $(\phi, \psi) \approx (-63^\circ, -42^\circ)$. En nuestros datos, los residuos asignados a hélice α tienen medias $\bar{\phi} = -64,6^\circ$ y $\bar{\psi} = -39,8^\circ$.
- **Región β** , amplia, en el cuadrante $\phi < 0, \psi > 0$. Los residuos en lámina tienen medias $\bar{\phi} = -110,5^\circ$, $\bar{\psi} = 121,0^\circ$. En su borde, cerca de $(-75^\circ, 145^\circ)$, se sitúa la hélice de poliprolina II, frecuente en lazos y regiones desordenadas.
- **Hélice izquierda (α_L)**, cerca de $(60^\circ, 45^\circ)$, poco poblada y ocupada sobre todo por glicina, asparagina y aspartato.

Dos residuos son especiales. La *glicina*, cuyo “cadena lateral” es un solo hidrógeno, no sufre el choque con el C_β y puede adoptar conformaciones con $\phi > 0$, lo que la convierte en la bisagra preferida de los giros cerrados. La *prolina*, cuya cadena

lateral forma un anillo que se cierra sobre el propio nitrógeno, tiene el ángulo ϕ casi bloqueado: en nuestros datos, $\bar{\phi}_{\text{Pro}} = -67,3^\circ$ con una desviación típica de solo $10,7^\circ$. Por eso la prolina rompe hélices y aparece en los extremos de las láminas.

⚠ Un residuo en zona prohibida no es necesariamente un error

El diagrama de Ramachandran es la primera herramienta de validación de una estructura: un modelo con muchos residuos fuera de las regiones permitidas probablemente está mal construido. Pero una fracción pequeña de residuos atípicos (*outliers*) aparece incluso en estructuras excelentes, casi siempre por una razón funcional: un residuo tensionado en un sitio activo, o una conformación forzada por la unión de un ligando. La pregunta correcta ante un *outlier* es si la densidad electrónica lo respalda.

15.1.4 Hélices y láminas: geometría de los puentes de hidrógeno

¿Por qué esas dos regiones y no otras? Porque son las que permiten que los grupos N–H y C=O del esqueleto, que de otro modo quedarían insatisfechos en el interior hidrofóbico de la proteína, formen puentes de hidrógeno entre sí de forma *regular*. En 1951, antes de que se resolviera ninguna estructura de proteína, Pauling et al. (1951) dedujeron la hélice α a partir de tres restricciones: enlace peptídico plano, puentes de hidrógeno lineales entre el C=O del residuo i y el N–H del residuo $i + 4$, y equivalencia de todos los residuos. La hélice resultante avanza unos $1,5 \text{ \AA}$ por residuo y tiene cerca de 3,6 residuos por vuelta, lo que da un paso de $5,4 \text{ \AA}$; todos los N–H apuntan hacia el extremo N y todos los C=O hacia el C, de modo que la hélice entera es un dipolo. En el mismo año, Pauling y Corey propusieron las láminas β , en las que los puentes se forman *entre* hebras extendidas vecinas, paralelas o antiparalelas.

🔄 El modelo antes que el dato

Pauling cuenta que concibió la hélice α en 1948, convaleciente de un resfriado en Oxford, doblando una hoja de papel en la que había dibujado una cadena polipeptídica con las distancias y ángulos correctos. Cuando Kendrew et al. (1958) publicaron el primer modelo tridimensional de una proteína, la mioglobina de cachalote a $6 \text{ \AA}$ de resolución, las varillas de densidad que recorrían la molécula eran, precisamente, hélices α . Kendrew describió la estructura como “más complicada de lo que había predicho cualquier teoría” y, sin embargo, sus piezas estaban previstas desde hacía una década.

15.1.5 Asignar estructura secundaria: DSSP

Mirar una estructura y decir “esto es una hélice” parece trivial, pero dos expertos pueden discrepar en los extremos. Kabsch y Sander (1983) sustituyeron el juicio visual por un algoritmo, DSSP (*Dictionary of Secondary Structure of Proteins*), que primero identifica los puentes de hidrógeno del esqueleto con un modelo electrostático sencillo y después reconoce patrones en ellos.

Definición 15.2. (Energía de puente de hidrógeno de DSSP)

Para un grupo C=O y un grupo N–H del esqueleto, se colocan cargas parciales $\pm q_1$ en C y O, y $\pm q_2$ en N y H. La energía electrostática de interacción es

$$E = f q_1 q_2 \left(\frac{1}{r_{\text{ON}}} + \frac{1}{r_{\text{CH}}} - \frac{1}{r_{\text{OH}}} - \frac{1}{r_{\text{CN}}} \right), \quad (15.2)$$

y se declara un puente de hidrógeno si $E < -0,5$ kcal/mol.

DONDE

q_1, q_2	Cargas parciales: $q_1 = 0,42e$ en el carbonilo y $q_2 = 0,20e$ en la amida.
f	Factor dimensional, $332 \text{ kcal} \cdot \text{\AA} / (\text{mol} \cdot e^2)$, que expresa la energía en kcal/mol.
r_{XY}	Distancia (en \AA) entre los átomos X e Y de los dos grupos.

A partir de la matriz de puentes, DSSP define “giros” n (un puente entre el C=O de i y el N–H de $i+n$) y “puentes” entre hebras. Dos giros de tipo 4 consecutivos forman una hélice α (código H); los de tipo 3 y 5 forman hélices 3_{10} (G) y π (I); una escalera de puentes entre hebras es una lámina (E), y un puente aislado, B. Los códigos T (giro), S (curvatura) y el espacio en blanco completan el alfabeto. Esta asignación es la “verdad” contra la que se evalúan todos los predictores de estructura secundaria y la que usamos para colorear la figura 15.2: en nuestras 79 estructuras, el 30,9 % de los residuos está en hélice, el 25,8 % en lámina y el 43,3 % restante en giros y lazos.

```
phi_psi.py
```

```
from Bio.PDB import MMCIFParser, PDBList, PPBuilder
import math

ruta = PDBList().retrieve_pdb_file("1UBQ", file_format="mmCif",
                                   pdir=".")
```

```

est = MMCIFParser(QUIET=True).get_structure("ubq", ruta)
for pp in PPBuilder().build_peptides(est[0]):
    for res, (phi, psi) in zip(pp, pp.get_phi_psi_list()):
        if phi is None or psi is None:           # extremos de cadena
            continue
        print(res.get_resname(), res.id[1],
              f"{math.degrees(phi):7.1f} {math.degrees(psi):7.1f}")

```

15.1.6 Cómo se determina una estructura

Ninguna técnica “fotografía” una proteína: todas miden una señal indirecta y construyen un modelo atómico compatible con ella. Conocer la señal es indispensable para saber cuánto confiar en cada parte del modelo.

Cristalografía de rayos X. Una proteína purificada se cristaliza, de modo que billones de copias quedan ordenadas en una red periódica. Al iluminar el cristal con rayos X, los electrones dispersan la radiación y las ondas interfieren constructivamente solo en direcciones discretas, que cumplen la ley de Bragg,

$$2d \sin \theta = \lambda, \quad (15.3)$$

donde d es el espaciado de una familia de planos del cristal, θ el ángulo de incidencia y λ la longitud de onda. Cada reflexión $\mathbf{h} = (h, k, l)$ tiene un *factor de estructura* $F_{\mathbf{h}} = |F_{\mathbf{h}}|e^{i\varphi_{\mathbf{h}}}$, y la densidad electrónica en el punto $\mathbf{x}$ de la celda unidad es su transformada de Fourier inversa:

$$\rho(\mathbf{x}) = \frac{1}{V} \sum_{\mathbf{h}} |F_{\mathbf{h}}| e^{i\varphi_{\mathbf{h}}} e^{-2\pi i \mathbf{h} \cdot \mathbf{x}}. \quad (15.4)$$

DONDE

$\rho(\mathbf{x})$	Densidad electrónica (electrones/Å ³) en la posición fraccionaria $\mathbf{x}$ de la celda.
V	Volumen de la celda unidad.
$ F_{\mathbf{h}} $	Amplitud de la reflexión $\mathbf{h}$; se obtiene de la intensidad medida, $I_{\mathbf{h}} \propto F_{\mathbf{h}} ^2$.
$\varphi_{\mathbf{h}}$	Fase de la reflexión; <i>no</i> se mide (el “problema de la fase”).

El detector registra intensidades, no fases, y sin fases la ecuación (15.4) no puede evaluarse. Resolver el problema de la fase (con átomos pesados, dispersión anómala

o, cada vez más, reemplazo molecular a partir de un modelo predicho) es el paso crítico. La *resolución* es el menor espaciado d para el que se midieron reflexiones útiles: a 3,5 Å se distingue el trazado de la cadena y las hélices; a 2 Å, la forma de las cadenas laterales; por debajo de 1,2 Å, átomos individuales e incluso algunos hidrógenos. La calidad del ajuste se mide con el factor R y, para evitar el sobreajuste, con el R_{free} , calculado sobre un 5 % de reflexiones que se excluyen del refinamiento, una forma temprana de validación cruzada.

Resonancia magnética nuclear (RMN). La proteína se estudia en disolución. Los espectros multidimensionales revelan qué núcleos (normalmente ^1H , marcados con ^{15}N y ^{13}C) están a menos de unos 5 Å entre sí (efecto Overhauser nuclear) y qué valores toman ciertos diedros. El resultado no es una estructura sino un conjunto de *restricciones* de distancia y ángulo, y el modelo se entrega como un *ensamble* de 10 a 40 conformaciones que las satisfacen. La RMN está limitada a proteínas de tamaño modesto (en la práctica, hasta unos 30–40 kDa), pero observa la dinámica en disolución, algo que el cristal oculta.

Criomicroscopía electrónica (crio-EM). Millones de copias de un complejo se congelan en una capa fina de hielo vítreo y se fotografían con un haz de electrones. Cada imagen es una proyección bidimensional ruidosa desde una orientación desconocida; el procesamiento las clasifica, estima sus orientaciones y reconstruye un mapa tridimensional. La resolución se estima comparando dos reconstrucciones independientes, con la curva de correlación de capas de Fourier (FSC), y se informa donde esta cae por debajo de 0,143. Con la llegada de los detectores directos de electrones y nuevos algoritmos, la técnica pasó en pocos años de producir “manchas” a mapas con detalle atómico, lo que Kühlbrandt (2014) bautizó como la “revolución de la resolución”. La crio-EM no necesita cristales y brilla con complejos grandes, flexibles o de membrana.

15.1.7 El Protein Data Bank

Desde 1971, las estructuras de macromoléculas se depositan en un único archivo público, el Protein Data Bank (PDB). Berman et al. (2000) describieron su reorganización bajo el consorcio RCSB, con procedimientos estandarizados de depósito, validación y distribución, y hoy lo gestiona el consorcio internacional wwPDB. La descripción reciente de Burley et al. (2023) muestra hasta qué punto el archivo ha cambiado: el portal RCSB.org ofrece, junto a las estructuras experimentales, cerca

Tabla 15.1. Comparación de los tres métodos experimentales principales. Las fracciones del PDB corresponden a las entradas liberadas en 2025 (consulta a la API del RCSB con `figuras/cap15/generar.py`).

	Rayos X	RMN	Crio-EM
Muestra	crystal	disolución	partículas en hielo vítreo
Señal	difracción (amplitudes)	acoplamiento y NOE entre núcleos	proyecciones 2D
Tamaño idóneo	cualquiera que cristalice	$\lesssim 40$ kDa	$\gtrsim 50$ kDa
Medida de calidad	resolución, R_{free} , factor B	restricciones por residuo, dispersión del ensamble	resolución FSC, resolución local
Dinámica	indirecta (factor B)	directa (ensamble, relajación)	heterogeneidad por clasificación
Entradas en 2025	10 083 (57,6 %)	225 (1,3 %)	7198 (41,1 %)

de un millón de modelos calculados por inteligencia artificial, claramente separados de los experimentales.

La figura 15.3 resume medio siglo de crecimiento. Cada entrada del PDB tiene un identificador de cuatro caracteres (como 1UBQ, la ubiquitina) y contiene mucho más que coordenadas: la secuencia de cada entidad polimérica, los ligandos, las condiciones experimentales, la resolución, las estadísticas de refinamiento, las referencias bibliográficas y un informe de validación.

El formato mmCIF. El formato histórico del PDB, de columnas fijas y 80 caracteres por línea, se describió en el capítulo 2. Sus límites (cinco dígitos para el número de átomo, un solo carácter para la cadena) lo hicieron insuficiente para los grandes complejos de la crio-EM, y desde 2014 el formato de archivo oficial es PDBx/mmCIF. Un archivo mmCIF es una colección de tablas con nombre, cada una precedida de `loop_` y de la lista de sus columnas:

Fragmento de 1UBQ.cif (tabla `_atom_site`, simplificada)

`loop_`

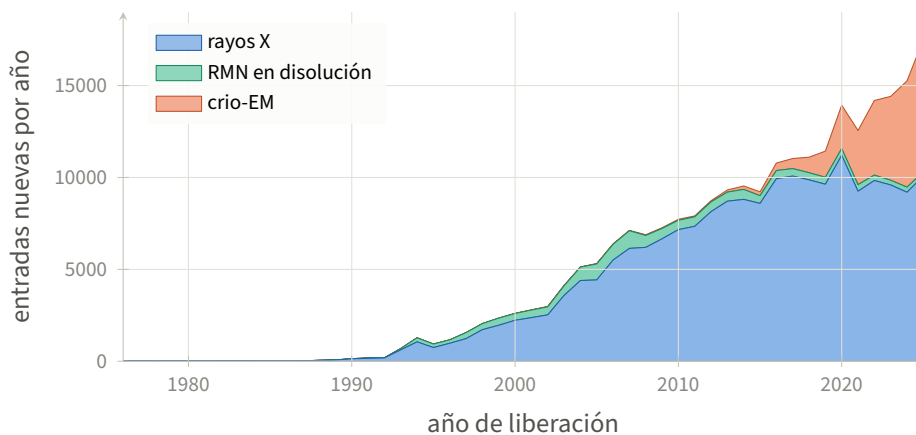


Figura 15.3. Entradas liberadas cada año en el PDB según el método experimental, 1976–2025 (áreas apiladas). La cristalografía domina durante cuatro décadas; la RMN alcanza su máximo en torno a 2005 y declina lentamente. A partir de 2014 la crio-EM crece de forma explosiva: pasa del 2,3 % de las entradas en 2015 al 16,9 % en 2020 y al 41,1 % en 2025, mientras que el número de estructuras cristalográficas se estanca. En total se liberaron 246 212 entradas en el periodo. Datos: API de búsqueda del RCSB, consulta hecha con `figuras/cap15/generar.py`.

```

_atom_site.group_PDB
_atom_site.id
_atom_site.type_symbol
_atom_site.label_atom_id
_atom_site.label_comp_id
_atom_site.label_asym_id
_atom_site.label_seq_id
_atom_site.Cartn_x
_atom_site.Cartn_y
_atom_site.Cartn_z
_atom_site.occupancy
_atom_site.B_iso_or_equiv
ATOM 1 N N MET A 1 27.340 24.430 2.614 1.00 9.67
ATOM 2 C CA MET A 1 26.266 25.413 2.842 1.00 10.38
ATOM 3 C C MET A 1 26.913 26.639 3.531 1.00 9.62

```

Al ser autodescritivo, el formato admite tablas nuevas sin romper los programas existentes. En la práctica casi nunca se lee a mano: **Biopython** (`MMCIFParser`), **gemmi** o **biotite** lo convierten en objetos jerárquicos de modelo, cadena, residuo y átomo.

15.1.8 El factor B : cuánto se mueve cada átomo

La penúltima columna del fragmento anterior, occupancy, indica qué fracción de las moléculas del cristal tiene el átomo en esa posición (menor que 1 si hay conformaciones alternativas). La última, B_iso_or_equiv, es el *factor de desplazamiento atómico* o factor B , que mide cuánto se “emborrona” la densidad de cada átomo por vibración térmica y por desorden estático entre las copias del cristal.

Teorema 15.1. (*Factor B y desplazamiento cuadrático medio*)

En el modelo isótropo, la contribución de un átomo a la difracción se atenúa por el factor $\exp(-B \sin^2 \theta / \lambda^2)$, con

$$B = 8\pi^2 \langle u_x^2 \rangle \quad \Rightarrow \quad \sqrt{\langle u^2 \rangle} = \sqrt{3\langle u_x^2 \rangle} = \sqrt{\frac{3B}{8\pi^2}}. \quad (15.5)$$

DONDE

B	Factor B del átomo, en $\text{\AA}^2$.
$\langle u_x^2 \rangle$	Desplazamiento cuadrático medio del átomo a lo largo de una dirección.
$\sqrt{\langle u^2 \rangle}$	Desplazamiento cuadrático medio total en tres dimensiones (isótropo).

Así, $B = 10 \text{ \AA}^2$ corresponde a un desplazamiento tridimensional típico de $0,62 \text{ \AA}$; $B = 20$, a $0,87 \text{ \AA}$; $B = 80$, a $1,74 \text{ \AA}$. Un átomo con B alto está mal definido en el modelo, y conviene no apoyar conclusiones finas (una distancia de puente de hidrógeno, la orientación de una cadena lateral) en él.

Ejemplo 15.1. (*Cristal frente a disolución: la ubiquitina*)

La ubiquitina humana, de 76 residuos, se ha resuelto por cristalografía a $1,8 \text{ \AA}$ (1UBQ) y por RMN con un ensamble de 10 modelos (1D3Z). Superponiendo los C_α con el algoritmo de Kabsch (que veremos a continuación), cada modelo de RMN difiere del cristal en un RMSD medio de $0,97 \text{ \AA}$ (entre $0,47$ y $1,55$). Si se excluye la cola C-terminal (residuos 71–76), la diferencia cae a $0,37 \text{ \AA}$: el núcleo es prácticamente idéntico en el cristal y en disolución.

La figura 15.4 explica por qué. El factor B medio de los C_α del cristal es de $8,9 \text{ \AA}^2$ en los residuos 1–70 y de $32,8 \text{ \AA}^2$ en los residuos 72–76, y la fluctuación

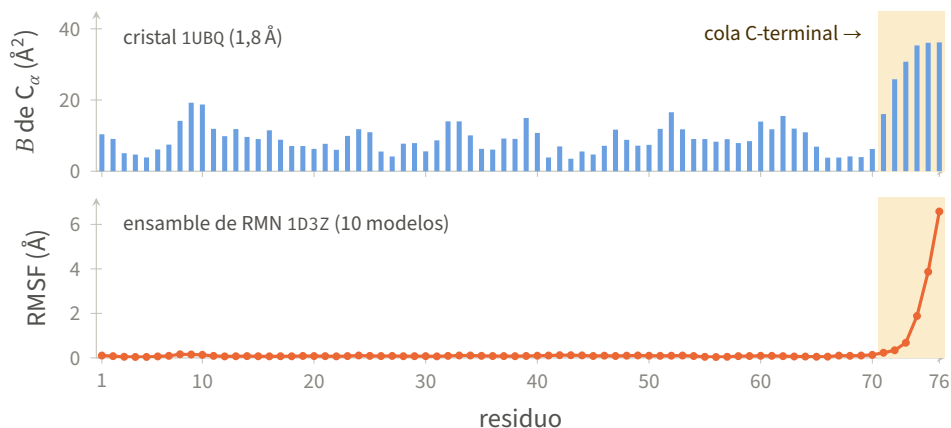


Figura 15.4. Dos medidas independientes de la flexibilidad de la ubiquitina. **Arriba:** factor B de los C_{α} en el cristal 1UBQ. **Abajo:** fluctuación cuadrática media (RMSF) de los C_{α} entre los 10 modelos del ensamble de RMN 1D3Z, superpuestos sobre el núcleo (residuos 1–70). Ambas señalan la cola C-terminal (sombreada) como la región más móvil, y ambas detectan una flexibilidad moderada en el lazo de los residuos 7–10. Lo que en cristalografía aparece como densidad borrosa, en RMN aparece como desacuerdo entre modelos.

(RMSF) de esos mismos residuos entre los modelos de RMN alcanza 6,6 Å en el último. Dos experimentos completamente distintos coinciden en que la cola es flexible: la correlación entre el factor B y la RMSF es de 0,71. Y no es un detalle menor: la glicina 76 de esa cola es la que se une covalentemente a las proteínas marcadas para su degradación, una función que exige flexibilidad.

15.1.9 Superponer dos estructuras: el algoritmo de Kabsch

Una pregunta aparece una y otra vez en bioinformática estructural: ¿cuánto se parecen dos estructuras? Dos modelos de la misma proteína, un modelo predicho y la estructura experimental, una proteína antes y después de unir un ligando. Si tenemos una correspondencia entre átomos (el C_{α} k de una con el C_{α} k de la otra), la medida natural es la raíz de la desviación cuadrática media (RMSD). Pero las coordenadas de dos archivos están en sistemas de referencia arbitrarios; antes de medir hay que *superponer*, es decir, encontrar la rotación y la traslación que mejor llevan una estructura sobre la otra.

Definición 15.3. (RMSD tras superposición óptima)

Sean $\{\mathbf{p}_k\}_{k=1}^N$ y $\{\mathbf{q}_k\}_{k=1}^N$ dos conjuntos de puntos en $\mathbb{R}^3$ con correspondencia conocida. El RMSD tras superposición óptima es

$$\text{RMSD} = \min_{\mathbf{R} \in \text{SO}(3), \mathbf{t} \in \mathbb{R}^3} \sqrt{\frac{1}{N} \sum_{k=1}^N \|\mathbf{R}\mathbf{p}_k + \mathbf{t} - \mathbf{q}_k\|^2}. \quad (15.6)$$

DONDE

$\mathbf{p}_k, \mathbf{q}_k$	Coordenadas del átomo k en la estructura móvil y en la de referencia.
N	Número de pares de átomos comparados (por ejemplo, los C_α alineados).
$\mathbf{R}$	Matriz de rotación: ortogonal ($\mathbf{R}^\top \mathbf{R} = \mathbf{I}$) y con $\det \mathbf{R} = +1$.
$\text{SO}(3)$	Grupo de las rotaciones propias en tres dimensiones (sin reflexiones).
$\mathbf{t}$	Vector de traslación.

Kabsch (1976) mostró que este problema de optimización, que parece requerir una búsqueda sobre todas las rotaciones posibles, tiene solución cerrada. Presentamos la derivación moderna, basada en la descomposición en valores singulares (SVD).

Paso 1: la traslación óptima une los centroides. Fijada $\mathbf{R}$, la función $E(\mathbf{t}) = \sum_k \|\mathbf{R}\mathbf{p}_k + \mathbf{t} - \mathbf{q}_k\|^2$ es cuadrática en $\mathbf{t}$. Igualando su gradiente a cero,

$$\nabla_{\mathbf{t}} E = 2 \sum_k (\mathbf{R}\mathbf{p}_k + \mathbf{t} - \mathbf{q}_k) = \mathbf{0} \implies \mathbf{t}^* = \bar{\mathbf{q}} - \mathbf{R}\bar{\mathbf{p}},$$

donde $\bar{\mathbf{p}}$ y $\bar{\mathbf{q}}$ son los centroides. Por lo tanto, basta centrar ambas estructuras en el origen, $\mathbf{p}_k \leftarrow \mathbf{p}_k - \bar{\mathbf{p}}$ y $\mathbf{q}_k \leftarrow \mathbf{q}_k - \bar{\mathbf{q}}$, y buscar solo la rotación.

Paso 2: minimizar el error equivale a maximizar una traza. Con los puntos centrados, y usando que una rotación conserva las normas ($\|\mathbf{R}\mathbf{p}_k\| = \|\mathbf{p}_k\|$),

$$E(\mathbf{R}) = \sum_k \|\mathbf{p}_k\|^2 + \sum_k \|\mathbf{q}_k\|^2 - 2 \sum_k \mathbf{q}_k^\top \mathbf{R} \mathbf{p}_k = \text{const} - 2 \text{tr}(\mathbf{R}\mathbf{H}), \quad \mathbf{H} = \sum_k \mathbf{p}_k \mathbf{q}_k^\top. \quad (15.7)$$

El paso intermedio usa la identidad $\mathbf{q}^\top \mathbf{R} \mathbf{p} = \text{tr}(\mathbf{R} \mathbf{p} \mathbf{q}^\top)$. Minimizar E equivale, pues, a maximizar $\text{tr}(\mathbf{R}\mathbf{H})$, donde $\mathbf{H}$ es la matriz 3×3 de covarianza cruzada entre las dos nubes de puntos.

Paso 3: la SVD resuelve la maximización. Escribamos $\mathbf{H} = \mathbf{U}\Sigma\mathbf{V}^T$, con $\mathbf{U}, \mathbf{V}$ ortogonales y $\Sigma = \text{diag}(\sigma_1, \sigma_2, \sigma_3)$, $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$. Por la propiedad cíclica de la traza,

$$\text{tr}(\mathbf{R}\mathbf{U}\Sigma\mathbf{V}^T) = \text{tr}(\underbrace{\mathbf{V}^T\mathbf{R}\mathbf{U}}_{\mathbf{M}}\Sigma) = \sum_{j=1}^3 M_{jj}\sigma_j.$$

Como $\mathbf{M}$ es ortogonal, sus elementos diagonales cumplen $M_{jj} \leq 1$, así que la suma es máxima cuando $\mathbf{M} = \mathbf{I}$, es decir, $\mathbf{R} = \mathbf{V}\mathbf{U}^T$. Queda un detalle: si $\det(\mathbf{V}\mathbf{U}^T) = -1$, esa matriz es una *reflexión*, que convertiría una proteína en su imagen especular. La mejor rotación propia se obtiene entonces sacrificando el término del valor singular más pequeño.

Teorema 15.2. (Algoritmo de Kabsch)

Con las estructuras centradas y $\mathbf{H} = \sum_k \mathbf{p}_k \mathbf{q}_k^T = \mathbf{U}\Sigma\mathbf{V}^T$, la rotación óptima y el error mínimo son

$$\mathbf{R}^* = \mathbf{V}\mathbf{D}\mathbf{U}^T, \quad \mathbf{D} = \text{diag}(1, 1, d), \quad d = \text{sign} \det(\mathbf{V}\mathbf{U}^T), \quad (15.8)$$

$$N \cdot \text{RMSD}^2 = \sum_k \|\mathbf{p}_k\|^2 + \sum_k \|\mathbf{q}_k\|^2 - 2(\sigma_1 + \sigma_2 + d\sigma_3).$$

DONDE

$\mathbf{H}$	Matriz de covarianza cruzada 3×3 entre las estructuras centradas.
$\mathbf{U}, \mathbf{V}$	Matrices ortogonales de vectores singulares izquierdos y derechos de $\mathbf{H}$.
σ_j	Valores singulares de $\mathbf{H}$, ordenados de mayor a menor.
d	Corrección de quiralidad: +1 si $\mathbf{V}\mathbf{U}^T$ ya es una rotación, -1 si es una reflexión.

El coste es el de construir $\mathbf{H}$, $O(N)$, más una SVD de 3×3 , que es constante: superponer dos proteínas de mil residuos toma microsegundos. La figura 15.5 muestra los tres pasos en dos dimensiones, donde las matrices son 2×2 y la geometría se ve de un vistazo.

Ejemplo 15.2. (Kabsch a mano, en el plano)

En la figura 15.5, la estructura móvil se construyó girando la de referencia 52° alrededor de su centroide, trasladándola y añadiendo ruido de

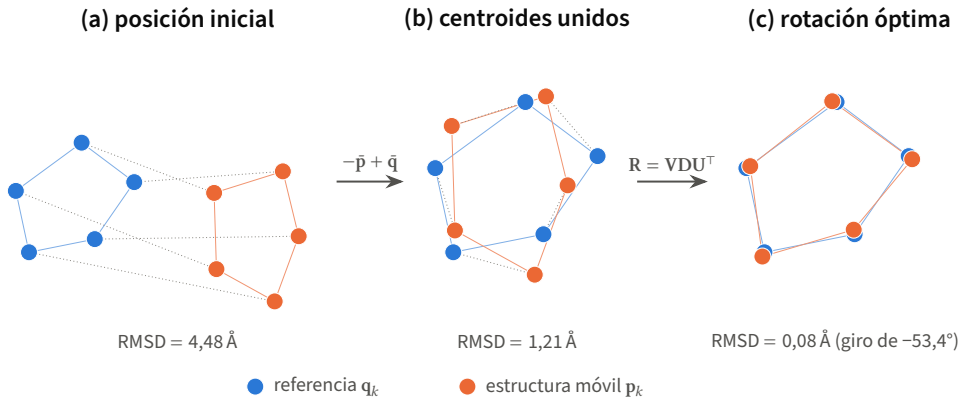


Figura 15.5. El algoritmo de Kabsch en dos dimensiones, con cinco pares de puntos. (a) Las estructuras están en posiciones y orientaciones arbitrarias; las líneas punteadas unen los puntos correspondientes. (b) Al trasladar el centroide de la estructura móvil (naranja) sobre el de la referencia (azul), el RMSD cae de 4,48 a 1,21 Å. (c) La rotación $R = VDU^T$ obtenida de la SVD de H elimina el resto, salvo el ruido añadido a propósito a cada punto: RMSD = 0,08 Å. Coordenadas generadas con `figuras/cap15/generar.py`.

unas centésimas. Tras centrar, las sumas de cuadrados son $\sum \|p_k\|^2 = 9,154$ y $\sum \|q_k\|^2 = 9,064$, y la matriz de covarianza cruzada vale

$$H = \begin{pmatrix} 1,906 & -2,558 \\ 4,741 & 3,519 \end{pmatrix}, \quad \sigma_1 = 5,905, \quad \sigma_2 = 3,190, \quad d = +1.$$

La ecuación (15.8) (en su versión bidimensional, con $D = \text{diag}(1, d)$) da $N \cdot \text{RMSD}^2 = 9,154 + 9,064 - 2(5,905 + 3,190) = 0,029$, de modo que $\text{RMSD} = \sqrt{0,029/5} = 0,076$ Å, sin haber calculado todavía la rotación. La matriz $R^* = VU^T$ resulta ser $\begin{pmatrix} 0,597 & 0,803 \\ -0,803 & 0,597 \end{pmatrix}$, un giro de $-53,4^\circ$: el algoritmo recupera el giro original de 52° con un error de algo más de un grado, causado por el ruido.

`kabsch.py`

```
import numpy as np
```

```
def kabsch_rmsd(P, Q):
```

```
    """P, Q: arrays (N, 3) con correspondencia fila a fila."""
```

```

P = P - P.mean(axis=0)                                # paso 1: centrar
Q = Q - Q.mean(axis=0)
H = P.T @ Q                                            # paso 2: covarianza 3x3
U, S, Vt = np.linalg.svd(H)                            # paso 3: SVD
d = np.sign(np.linalg.det(Vt.T @ U.T)) # evitar reflexiones
D = np.diag([1.0, 1.0, d])
R = Vt.T @ D @ U.T
diff = P @ R.T - Q
return np.sqrt((diff ** 2).sum(axis=1).mean()), R

```

15.1.10 Más allá del RMSD: TM-score

El RMSD tiene dos defectos serios como medida de similitud global. Primero, depende de la longitud: dos proteínas grandes no relacionadas tienen un RMSD mayor que dos pequeñas, así que un mismo valor no significa lo mismo para 50 que para 500 residuos. Segundo, está dominado por los peores pares: como promedia *cuadrados*, una cola desordenada o un dominio que gira como bisagra puede inflar el RMSD de una estructura que, en el resto, es idéntica. La ubiquitina lo ilustró: 0,97 Å con la cola, 0,37 Å sin ella.

Zhang y Skolnick (2004) propusieron una medida que resuelve ambos problemas.

Definición 15.4. (TM-score)

Para una estructura modelo y una de referencia de longitud L , con N_{al} pares de residuos alineados,

$$TM = \max_{\mathbf{R}, \mathbf{t}} \frac{1}{L} \sum_{k=1}^{N_{\text{al}}} \frac{1}{1 + (d_k/d_0(L))^2}, \quad d_0(L) = 1,24\sqrt[3]{L} - 1,8, \quad (15.9)$$

donde d_k es la distancia entre el par k tras la superposición $(\mathbf{R}, \mathbf{t})$.

DONDE

L	Longitud de la estructura de referencia (normaliza el puntaje).
N_{al}	Número de pares de residuos alineados.
d_k	Distancia (Å) entre los C_α del par k tras la superposición.
$d_0(L)$	Escala de distancia que crece con la longitud; para $L = 141$ vale 4,42 Å.

Cada par contribuye entre 0 y 1: cerca de 1 si $d_k \ll d_0$ y casi nada si $d_k \gg d_0$, de modo que los residuos muy desviados dejan de pesar en lugar de dominar. La escala $d_0(L)$ se eligió para que el puntaje medio de estructuras no relacionadas apenas dependa de la longitud. El resultado va de 0 a 1; en la práctica, un TM-score mayor que 0,5 indica, casi siempre, el mismo plegamiento. Observe también el “máx”: la superposición óptima para el TM-score *no* es la de Kabsch (que minimiza el RMSD), sino la que maximiza el número de residuos bien superpuestos, y se busca de forma heurística a partir de fragmentos semilla.

Ejemplo 15.3. (Mioglobina frente a hemoglobina)

La mioglobina de cachalote (1A6M, 151 residuos) y la cadena α de la hemoglobina humana (2DN2, 141 residuos) son un ejemplo clásico de homología estructural. Un alineamiento global con BLOSUM62 empareja 138 residuos con solo un 28,3 % de identidad, cerca de la “zona crepuscular” del capítulo 3. Sin embargo, la superposición de Kabsch de esos 138 C_α da un RMSD de 1,93 Å; si se descartan iterativamente los pares a más de 3 Å, quedan 129 con un RMSD de 1,38 Å. El TM-score, normalizado por la cadena α , es 0,87, inequívocamente el mismo plegamiento. Como control, si se baraja al azar la correspondencia entre residuos, el TM-score cae a 0,12. *La estructura se conserva mucho más que la secuencia*, una regla que justifica el modelado por homología de la próxima lección.

El RMSD responde “¿cuán lejos están los átomos, en promedio?” y lo dominan los peores; el TM-score responde “¿qué fracción de la estructura coincide?” y es comparable entre proteínas de distinta longitud.

15.1.11 Clasificar el universo de plegamientos: SCOP y CATH

Si la estructura se conserva más que la secuencia, entonces clasificar las proteínas por su estructura revela parentescos que la secuencia ya no muestra. Murzin et al. (1995) crearon SCOP (*Structural Classification of Proteins*), una clasificación jerárquica hecha en gran parte a mano por expertos, con cuatro niveles: *clase* (todo α , todo β , α/β con hebras y hélices alternadas, $\alpha + \beta$ con regiones separadas), *plegamiento* (misma disposición y topología de los elementos secundarios), *superfamilia* (proba-

ble origen evolutivo común, inferido por estructura y función aunque la identidad de secuencia sea baja) y *familia* (homología clara a nivel de secuencia). Casi al mismo tiempo, Orengo et al. (1997) presentaron CATH, más automatizada, cuyo nombre enumera sus niveles: *Clase*, *Arquitectura* (forma general, sin considerar la conectividad), *Topología* (conectividad, equivalente al plegamiento) y superfamilia *Homóloga*. Ambas clasifican *dominios*, no cadenas completas, y ambas mostraron que el número de plegamientos distintos en la naturaleza es sorprendentemente pequeño: unos pocos miles, reutilizados una y otra vez.

Alineamiento estructural sin correspondencia conocida

Todo lo anterior supone que sabemos qué residuo corresponde a cuál. Cuando comparamos dos proteínas distintas, esa correspondencia es precisamente lo que queremos averiguar, y el problema se vuelve mucho más difícil: hay que optimizar simultáneamente el alineamiento y la superposición. Programas como TM-align alternan ambos pasos (alinear con programación dinámica sobre una matriz de distancias tras superponer; superponer con los pares alineados), y herramientas recientes como Foldseek traducen la geometría local a un “alfabeto estructural” de 20 letras para usar la maquinaria rápida de búsqueda de secuencias del capítulo 3 sobre millones de estructuras.

15.2 Predicción de estructura: de la homología a AlphaFold y ESMFold

 Colab · 15.2 · Predicción de estructura: AlphaFold y ESMFold  repositorio del curso

Reconstruir un edificio a partir de sus reformas

Suponga que quiere saber la forma de un edificio antiguo que nunca ha visto, pero dispone de los registros de reformas de cientos de edificios hermanos contruidos con el mismo plano. Observa algo curioso: cada vez que alguien cambió una viga en la tercera planta, casi siempre cambió también un pilar de la séptima, en otro ala. La explicación más sencilla es que la viga y el pilar se *tocan*: modificar uno obliga a ajustar el otro. Con suficientes registros podría reconstruir qué partes están en contacto y, de ahí, la forma del edificio, sin

haberlo medido nunca. Así funciona la predicción de estructura moderna: las familias de proteínas son los edificios hermanos, las mutaciones son las reformas, y las mutaciones correlacionadas delatan qué residuos están juntos en el espacio.

15.2.1 El experimento de Anfinsen y la paradoja de Levinthal

La premisa de toda predicción es que la secuencia contiene la información necesaria para el plegamiento. Anfinsen (1973) lo demostró con la ribonucleasa A: al tratarla con urea y β -mercaptoetanol, la enzima se desplegaba y sus cuatro puentes disulfuro se rompían; al retirar lentamente los agentes desnaturalizantes, la proteína recuperaba espontáneamente su estructura y su actividad. De las 105 formas posibles de emparejar ocho cisteínas en cuatro puentes, la cadena elegía por sí sola la correcta. Anfinsen formuló así la *hipótesis termodinámica*: la estructura nativa es la de mínima energía libre en las condiciones fisiológicas, y está determinada por la secuencia. Le valió el premio Nobel de Química de 1972.

Pero si la estructura nativa es un mínimo de energía, ¿cómo lo encuentra la proteína? Cyrus Levinthal señaló en 1969 una dificultad aritmética. Supongamos, de forma muy conservadora, que cada residuo puede adoptar solo tres conformaciones. Una proteína de 100 residuos tiene entonces

$$\begin{aligned} 3^{100} &\approx 5,2 \times 10^{47} \text{ conformaciones,} \\ t_{\text{búsqueda}} &\approx 5,2 \times 10^{47} \times 10^{-13} \text{ s} \approx 5 \times 10^{34} \text{ s} \approx 1,6 \times 10^{27} \text{ años,} \end{aligned} \quad (15.10)$$

si explorara una conformación cada 0,1 ps, el tiempo de una vibración molecular. Es un tiempo 10^{17} veces mayor que la edad del universo, y sin embargo las proteínas pequeñas se pliegan en milisegundos. Esta es la *paradoja de Levinthal*.

DONDE

3^{100}	Número de conformaciones si cada uno de 100 residuos tiene 3 estados posibles.
10^{-13} s	Tiempo optimista para visitar una conformación (una vibración de enlace).
$t_{\text{búsqueda}}$	Tiempo de una búsqueda exhaustiva y aleatoria.

La resolución es que el plegamiento no es una búsqueda aleatoria. Zwanzig et al. (1992) mostraron con un modelo sencillo que basta un pequeño sesgo energético a favor de las conformaciones locales correctas para reducir el tiempo de búsqueda

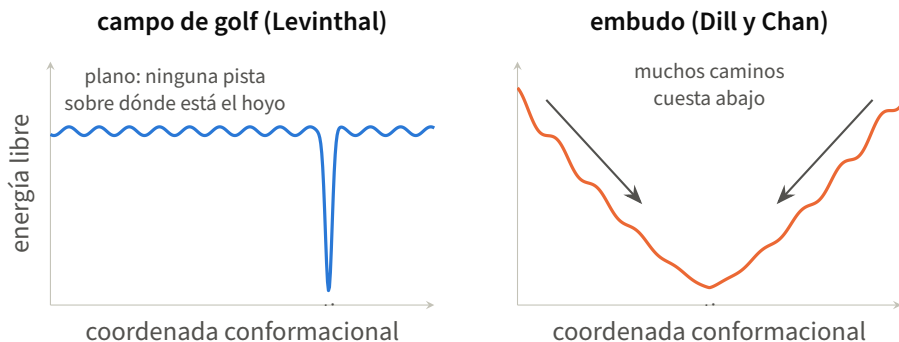


Figura 15.6. Dos paisajes de energía libre idealizados, en una sola coordenada. **Izquierda:** si el paisaje fuera plano salvo por un hoyo estrecho, la cadena tendría que encontrarlo por búsqueda aleatoria, y tardaría más que la edad del universo (ecuación (15.10)). **Derecha:** en un embudo, cada paso que reduce la energía acerca la cadena al estado nativo; la rugosidad produce mínimos locales transitorios, pero no impide llegar al fondo. La misma imagen de embudo reaparecerá en la sección 15.3 para la unión de ligandos.

de escalas cósmicas a segundos. Dill y Chan (1997) lo expresaron con una imagen que se ha vuelto canónica: el paisaje de energía de una proteína no es un campo de golf plano con un único hoyo, sino un *embudo* rugoso, en el que casi cualquier movimiento que reduzca la energía acerca a la cadena al estado nativo (figura 15.6).

Resolver la paradoja no resolvió el problema computacional. Simular físicamente el plegamiento desde la secuencia exige campos de fuerza exactos y tiempos de simulación inalcanzables para casi todas las proteínas. Durante cinco décadas, la predicción avanzó por otro camino: aprovechar la información evolutiva.

15.2.2 Modelado por homología

El ejemplo de las globinas mostró que proteínas con un 28 % de identidad comparten el mismo plegamiento. Si nuestra proteína tiene un homólogo de estructura conocida (una *plantilla*), podemos construir un modelo copiando su esqueleto. Es el *modelado comparativo* o por homología, cuyo representante más influyente es MODELLER (Šali & Blundell, 1993). Su idea no es copiar coordenadas sino *satisfacer restricciones espaciales*: del alineamiento con una o varias plantillas se derivan densidades de probabilidad para distancias y diedros del modelo; se combinan con términos estereoquímicos (longitudes y ángulos de enlace, contactos no permitidos) en una función objetivo, y se optimiza con gradientes conjugados y dinámica molecular con recocido simulado.

La calidad del modelo depende casi por completo del alineamiento y de la identidad con la plantilla. Por encima del 50 % los modelos suelen ser muy buenos; entre el 30 y el 50 % el núcleo es fiable pero lazos e inserciones no; por debajo del 30 % los errores de alineamiento se vuelven la principal fuente de error. Y cuando no hay plantilla, el método simplemente no puede empezar.

15.2.3 CASP: medir el progreso a ciegas

Para saber qué métodos funcionan de verdad hace falta evaluarlos sin que los autores conozcan la respuesta. Moult et al. (1995) organizaron en 1994 el primer CASP (*Critical Assessment of Structure Prediction*): los organizadores recogen secuencias de proteínas cuya estructura está a punto de resolverse experimentalmente, los grupos envían sus predicciones antes de que se publique, y evaluadores independientes las comparan con la estructura real. El experimento se repite cada dos años, y es el equivalente estructural del ensayo clínico doble ciego.

La medida principal de CASP es el GDT_TS (*Global Distance Test*):

$$\text{GDT_TS} = \frac{100}{4} (P_1 + P_2 + P_4 + P_8), \quad (15.11)$$

donde P_c es la mayor fracción de residuos del modelo que pueden superponerse a menos de c Å de sus posiciones reales. Como el TM-score, premia la fracción bien predicha e ignora los residuos muy desviados. Un GDT_TS por encima de 90 se considera competitivo con una estructura experimental.

DONDE

P_c	Fracción máxima de C_α a menos de c Å de la referencia, optimizando la superposición para cada umbral.
c	Umbral de distancia: 1, 2, 4 y 8 Å.

15.2.4 La señal de la coevolución

La intuición de la apertura se formaliza así. Si en un alineamiento múltiple (capítulo 4) las columnas i y j mutan de forma correlacionada, es probable que los residuos i y j estén en contacto, porque un cambio en uno exige un cambio compensatorio en el otro para mantener la estructura (figura 15.7). La primera medida que viene a la mente es la *información mutua* entre columnas,

$$\text{MI}_{ij} = \sum_{a,b} f_{ij}(a,b) \log \frac{f_{ij}(a,b)}{f_i(a) f_j(b)}, \quad (15.12)$$

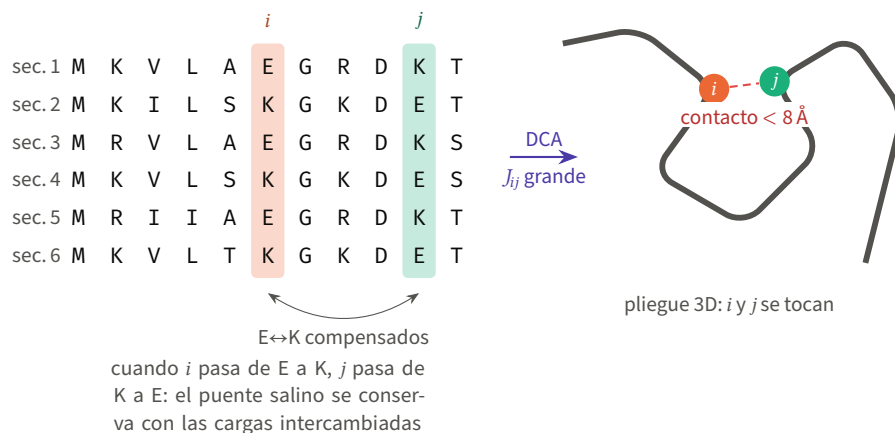


Figura 15.7. La señal de la coevolución. **Izquierda:** en un alineamiento de seis homólogos, las columnas i (naranja) y j (verde) cambian de forma coordinada: cuando i es glutamato (E), j es lisina (K), y viceversa, de modo que el puente salino entre ambos se conserva aunque los dos residuos cambien. **Derecha:** la explicación estructural más sencilla es que i y j están en contacto en el plegamiento. El análisis de acoplamiento directo (DCA) estima un acoplamiento J_{ij} por par, separando la covariación directa de la transmitida a través de terceros residuos.

donde $f_i(a)$ es la frecuencia del aminoácido a en la columna i y $f_{ij}(a, b)$ la frecuencia del par (a, b) en las columnas (i, j) . Pero la información mutua confunde correlaciones directas e indirectas: si i toca a k y k toca a j , las columnas i y j covarían aunque no se toquen, y además la filogenia (secuencias emparentadas que comparten mutaciones por herencia) añade correlaciones espurias.

DONDE

$f_i(a)$	Frecuencia (con pesos y pseudocuentas) del aminoácido a en la columna i del MSA.
$f_{ij}(a, b)$	Frecuencia conjunta del par (a, b) en las columnas i y j .
MI_{ij}	Información mutua entre las columnas i y j , en nats.

Morcos et al. (2011) resolvieron el problema con el *análisis de acoplamiento directo* (DCA). En lugar de medir correlaciones par a par, ajustan a todo el MSA un modelo probabilístico global, un modelo de Potts de la física estadística,

$$P(a_1, \dots, a_L) = \frac{1}{Z} \exp \left(\sum_{i < j} J_{ij}(a_i, a_j) + \sum_i h_i(a_i) \right), \quad (15.13)$$

cuyos parámetros de acoplamiento J_{ij} capturan solo las dependencias *directas*, porque las indirectas ya quedan explicadas por las cadenas de acoplamientos intermedios. Con una aproximación de campo medio, los J_{ij} se obtienen invirtiendo la matriz de covarianzas del alineamiento. Aplicado a más de un centenar de familias, los pares con mayor acoplamiento directo resultaron ser, en su gran mayoría, contactos reales, y con unas decenas de ellos como restricciones fue posible plegar proteínas sin plantilla.

DONDE

$P(a_1, \dots, a_L)$	Probabilidad de una secuencia de longitud L bajo el modelo de la familia.
$h_i(a)$	Campo local: preferencia de la columna i por el aminoácido a .
$J_{ij}(a, b)$	Acoplamiento directo entre el aminoácido a en i y el b en j .
Z	Constante de normalización (función de partición).

⚠ La coevolución necesita muchas secuencias

El número de parámetros de la ecuación (15.13) crece como $L^2 \times 20^2$: para una proteína de 200 residuos, unos ocho millones. Estimarlos requiere alineamientos con miles de secuencias *efectivas* (tras reducir el peso de las muy parecidas). Para familias pequeñas, proteínas huérfanas o anticuerpos, la señal es débil, y esa limitación reaparece en todos los métodos que dependen del MSA, incluido AlphaFold2.

15.2.5 AlphaFold2

En CASP14 (2020), un sistema de DeepMind obtuvo predicciones cuya exactitud, para la mayoría de las dianas, era comparable a la de las estructuras experimentales (Kryshtafovych et al., 2021). Jumper et al. (2021) describieron el método, AlphaFold2, e informaron una exactitud mediana del esqueleto de 0,96 Å (RMSD de los C_α sobre el 95 % de los residuos), frente a 2,8 Å del siguiente mejor método. La diferencia no fue un avance incremental: fue un cambio de régimen.

AlphaFold2 no abandona la coevolución: la aprende. En lugar de extraer contactos con un modelo estadístico fijo y luego plegar, entrena una red neuronal de extremo a extremo que lee el MSA y produce directamente coordenadas atómicas. Su arquitectura tiene tres partes (figura 15.8).

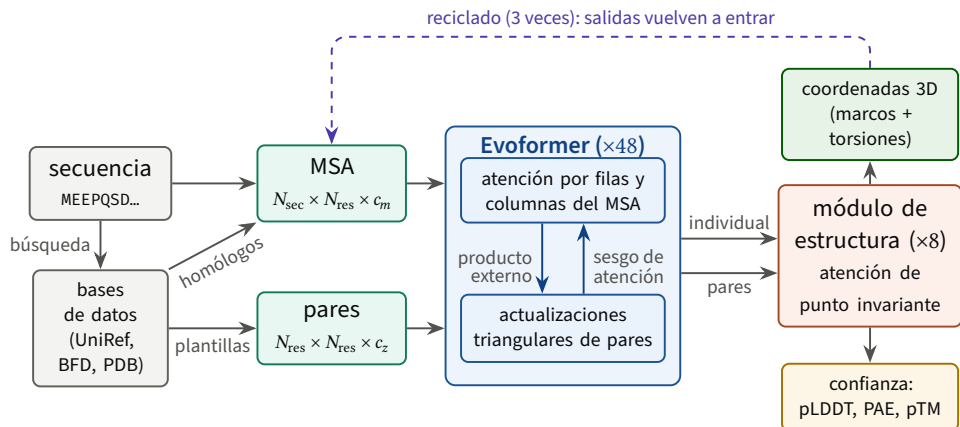


Figura 15.8. Arquitectura simplificada de AlphaFold2 (Jumper et al., 2021). La secuencia se usa para buscar homólogos (MSA) y plantillas. Dos representaciones se refinan juntas en 48 bloques Evoformer: la del MSA ($N_{\text{sec}} \times N_{\text{res}}$) y la de pares ($N_{\text{res}} \times N_{\text{res}}$); la información fluye del MSA a los pares mediante un producto externo, y de los pares al MSA como sesgo de la atención. El módulo de estructura convierte la representación individual y la de pares en un marco rígido (rotación y traslación) por residuo más sus ángulos de torsión, mediante la atención de punto invariante. Las salidas, incluida la estructura, se reciclan tres veces. Junto a las coordenadas, la red predice su propia confianza (pLDDT y PAE).

Entradas. La secuencia se busca en grandes bases de datos de secuencias con herramientas como las del capítulo 4 (*jackhmmer*, *HHblits*) para construir un MSA, y en el PDB para encontrar plantillas. El MSA se inicializa como un tensor $N_{\text{sec}} \times N_{\text{res}} \times c_m$ (una incrustación de dimensión c_m por cada letra del alineamiento) y la representación de pares como un tensor $N_{\text{res}} \times N_{\text{res}} \times c_z$, en el que la celda (i, j) acabará codificando todo lo que la red “cree” sobre la relación espacial entre los residuos i y j .

Evoformer y atención. El corazón es la *atención*, el mecanismo de los *transformers*. Para un conjunto de vectores de entrada $\mathbf{x}_1, \dots, \mathbf{x}_n$, cada uno se proyecta en una consulta $\mathbf{q}_i$, una clave $\mathbf{k}_j$ y un valor $\mathbf{v}_j$, y la salida en la posición i es un promedio ponderado de los valores:

$$\mathbf{o}_i = \sum_{j=1}^n \alpha_{ij} \mathbf{v}_j, \quad \alpha_{ij} = \frac{\exp(\mathbf{q}_i^T \mathbf{k}_j / \sqrt{c} + b_{ij})}{\sum_{j'} \exp(\mathbf{q}_i^T \mathbf{k}_{j'} / \sqrt{c} + b_{ij'})}. \quad (15.14)$$

DONDE

$\mathbf{q}_i, \mathbf{k}_j, \mathbf{v}_j$	Consulta, clave y valor: proyecciones lineales aprendidas de las entradas.
α_{ij}	Peso de atención de la posición i sobre la j ; los pesos de cada i suman 1 (<i>softmax</i>).
c	Dimensión de las consultas y claves; $\sqrt{c}$ estabiliza la escala del producto escalar.
b_{ij}	Sesgo aditivo; en AlphaFold2 procede de la representación de pares.

La atención permite que cada residuo “consulte” a todos los demás y decida a cuáles hacer caso. En el Evoformer se aplica en varias direcciones: por filas del MSA (cada secuencia mira a lo largo de sus residuos, con el sesgo b_{ij} tomado de la representación de pares, de modo que la geometría supuesta guía la lectura del alineamiento), por columnas (cada posición compara lo que ocurre en distintas secuencias, que es donde vive la coevolución) y sobre la propia representación de pares. Para esta última, Jumper et al. (2021) introdujeron las *actualizaciones triangulares*: la celda (i, j) se actualiza combinando las celdas (i, k) y (k, j) para todos los k . La motivación es geométrica: las distancias deben cumplir la desigualdad triangular, $d_{ij} \leq d_{ik} + d_{kj}$, y hacer que cada par “razone” sobre los triángulos que forma con terceros residuos impone consistencia tridimensional antes de que exista ninguna coordenada. Finalmente, un *producto externo* promediado sobre las secuencias del MSA transfiere la información de coevolución a la representación de pares.

Módulo de estructura. Cada residuo se representa como un cuerpo rígido, con un marco local (rotación y traslación, definidas por $\mathbf{N}$, $\mathbf{C}_\alpha$ y $\mathbf{C}$) que inicialmente está en el origen. Durante ocho bloques con pesos compartidos, la *atención de punto invariante* (IPA) actualiza esos marcos: es una atención en la que las consultas y claves incluyen puntos tridimensionales expresados en el marco de cada residuo, construida para que el resultado no cambie si se rota o traslada toda la estructura. Al final se predicen los ángulos de torsión de las cadenas laterales, y con ellos las coordenadas de todos los átomos pesados.

Entrenamiento y reciclado. La función de pérdida principal, FAPE (*frame aligned point error*), compara las posiciones atómicas predichas y reales expresadas en el marco local de cada residuo, lo que la hace sensible también a la quiralidad. La red se entrenó con estructuras del PDB y, en una segunda fase, con predicciones propias de alta confianza sobre secuencias sin estructura (autodestilación). La salida

completa se vuelve a introducir como entrada tres veces (*recycling*), lo que permite refinar progresivamente la predicción.

15.2.6 Confianza: pLDDT y PAE

Una predicción sin estimación de su error sería peligrosa. AlphaFold2 aprende a predecir, además de la estructura, cuánto se equivoca.

Definición 15.5. (IDDT y pLDDT)

El IDDT (Mariani et al., 2013) compara un modelo con la referencia *sin superponerlos*. Para cada átomo i , se consideran los pares (i, j) con distancia de referencia $d_{ij}^{\text{ref}} < R_0 = 15 \text{ \AA}$, y se cuenta la fracción de ellos cuya distancia en el modelo se conserva dentro de cada tolerancia:

$$\text{IDDT}_i = \frac{1}{4} \sum_{\tau \in \{0.5, 1, 2, 4\}} \frac{|\{j : d_{ij}^{\text{ref}} < R_0, |d_{ij}^{\text{mod}} - d_{ij}^{\text{ref}}| < \tau\}|}{|\{j : d_{ij}^{\text{ref}} < R_0\}|}. \quad (15.15)$$

El pLDDT es la predicción que hace la red de $100 \times \text{IDDT}_i$ (calculado sobre los C_α) para cada residuo, sin conocer la estructura real.

DONDE

$d_{ij}^{\text{ref}}, d_{ij}^{\text{mod}}$	Distancias entre los átomos i y j en la estructura de referencia y en el modelo.
R_0	Radio de inclusión: solo cuentan los vecinos a menos de $15 \text{ \AA}$ en la referencia.
τ	Tolerancias de 0,5, 1, 2 y $4 \text{ \AA}$, promediadas.

Por ser local y no requerir superposición, el IDDT evalúa bien proteínas con varios dominios que se mueven entre sí, y el pLDDT hereda esa propiedad: mide si el entorno local de cada residuo está bien predicho, no si los dominios están bien colocados unos respecto a otros. Por convención, se interpreta en cuatro bandas: muy alta (> 90), segura (70–90), baja (50–70) y muy baja (< 50). Las regiones con pLDDT muy bajo suelen corresponder a segmentos intrínsecamente desordenados, que no tienen una estructura única que predecir.

Para la disposición relativa de dominios, AlphaFold2 ofrece el *error alineado predicho* (PAE). El elemento PAE_{ij} es el error esperado, en $\text{\AA}$, en la posición del residuo j si se superponen la predicción y la estructura real sobre el marco local

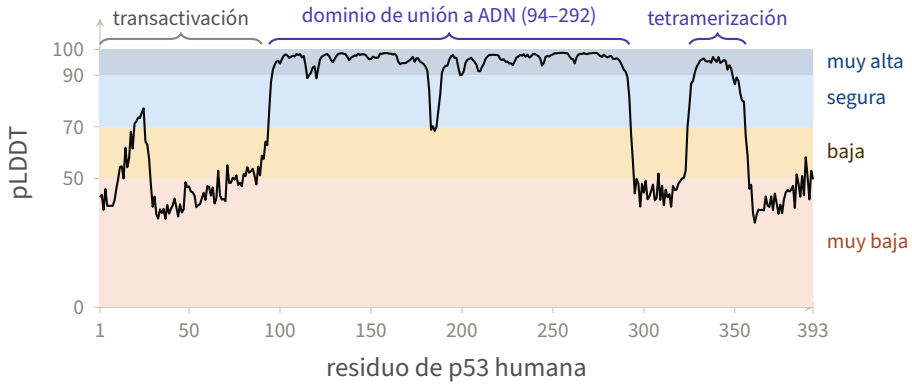


Figura 15.9. pLDDT por residuo del modelo de AlphaFold DB para la proteína supresora de tumores p53 humana (UniProt P04637, versión 6 del modelo), con las cuatro bandas de confianza sombreadas. El dominio de unión a ADN (pLDDT medio 95,3) y el de tetramerización (90,9) se predicen con confianza muy alta. La región N-terminal de transactivación (48,4), el enlace entre dominios (47,4) y el extremo C-terminal (43,4) caen en la banda muy baja: son regiones intrínsecamente desordenadas, que en la célula solo adoptan estructura al unirse a otras proteínas. Datos: AlphaFold DB, procesados con `figuras/cap15/generar.py`.

del residuo i . Una matriz de PAE con bloques de valores bajos en la diagonal indica dominios bien definidos; valores altos *fuera* de los bloques indican que la posición relativa de esos dominios es incierta.

Ejemplo 15.4. (Leer un modelo de AlphaFold DB: p53)

La p53 humana (393 residuos) es un caso de estudio ideal porque mezcla dominios compactos con largas regiones desordenadas. En su modelo de AlphaFold DB, el 52,7 % de los residuos tiene pLDDT muy alto, el 7,1 % seguro, el 10,4 % bajo y el 29,8 % muy bajo (figura 15.9).

La matriz de PAE (figura 15.10) añade la información que el pLDDT no da. Dentro del dominio de unión a ADN, el PAE medio es de 3,6 Å, y dentro del de tetramerización, de 2,9 Å; pero el PAE medio de las posiciones del dominio de tetramerización vistas desde el marco del dominio de unión a ADN es de 20,1 Å. Traducción: cada dominio está bien predicho, pero su orientación relativa en el modelo es arbitraria, y sería un error medir distancias entre ellos.

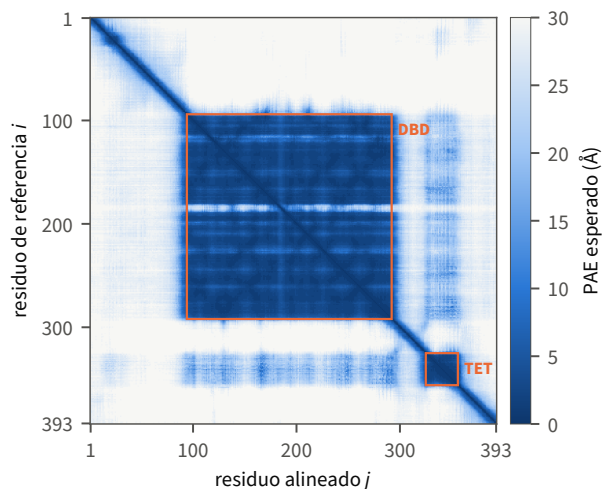


Figura 15.10. Matriz de error alineado predicho (PAE) del modelo de AlphaFold DB para la p53 humana. La celda (i, j) es el error esperado en la posición del residuo j tras superponer sobre el residuo i ; los tonos oscuros indican confianza. Los dos bloques oscuros de la diagonal (recuadros) son el dominio de unión a ADN (DBD) y el de tetramerización (TET): cada uno es rígido y está bien definido. Las regiones claras fuera de ellos indican que la posición relativa de los dominios, y la de las regiones desordenadas, es desconocida. Datos: AlphaFold DB (AF-P04637-F1, v6).

¿Es fiable el dominio de alta confianza? Comparando sus C_α (residuos 94–292) con la estructura cristalográfica 20CJ, 194 pares tienen un RMSD de 0,51 Å y un TM-score de 0,99. Es una concordancia casi perfecta, aunque conviene una advertencia honesta: esa estructura, depositada en 2007, pudo formar parte del conjunto de entrenamiento. La evaluación justa de un predictor exige estructuras publicadas después de su entrenamiento, que es exactamente lo que garantiza CASP.

```
alphafold_db.py
```

```
import requests
```

```
acc = "P04637" # p53 humana
api = f"https://alphafold.ebi.ac.uk/api/prediction/{acc}"
meta = requests.get(api, timeout=30).json()[0]
```

```

pdb = requests.get(meta["pdbUrl"], timeout=60).text
plddt = {}
for linea in pdb.splitlines():
    # pLDDT = columna B
    if linea.startswith("ATOM") and linea[12:16] == " CA ":
        plddt[int(linea[22:26])] = float(linea[60:66])
bajos = [r for r, v in plddt.items() if v < 50]
print(f"{len(plddt)} residuos; {len(bajos)} con pLDDT < 50")

```

⚠ pLDDT alto no significa estructura correcta en la célula

El pLDDT mide la confianza de la red en una conformación, no la verdad biológica. Un modelo puede tener pLDDT alto y representar solo uno de varios estados funcionales (activo o inactivo, con o sin ligando), porque AlphaFold2 tiende a predecir la conformación más representada en su entrenamiento. Tampoco modela ligandos, iones, modificaciones postraduccionales ni otras cadenas. Y un pLDDT bajo no es un fracaso: a menudo es la mejor predicción posible, la de que la región está desordenada.

15.2.7 AlphaFold DB, RoseTTAFold, ESMFold, AlphaFold3 y ColabFold

AlphaFold DB. Con el método publicado, DeepMind y el EMBL-EBI calcularon modelos para proteomas completos y los ofrecieron gratuitamente. Varadi et al. (2022) describieron la primera versión de la AlphaFold Protein Structure Database, con los proteomas de referencia de humanos y de decenas de organismos modelo, y la ampliación posterior a prácticamente todo UniProt, con más de 200 millones de modelos, cambió la escala del campo: por primera vez, la mayoría de las proteínas conocidas tiene un modelo estructural. Como vimos, el PDB integra hoy estos modelos calculados en su portal, separados de las estructuras experimentales (Burley et al., 2023).

RoseTTAFold. Casi simultáneamente, Baek et al. (2021) presentaron RoseTTAFold, una red de “tres pistas” en la que la información de la secuencia (1D), de las distancias entre residuos (2D) y de las coordenadas (3D) fluye y se actualiza en paralelo. Con una exactitud cercana a la de AlphaFold2 y un código abierto desde el principio, mostró además que la misma arquitectura podía predecir complejos de proteínas.

ESMFold. Tanto AlphaFold2 como RoseTTAFold dependen de un MSA, cuya construcción es lenta y que falla para proteínas con pocos homólogos. Lin et al. (2023)

sustituyeron el MSA por un *modelo de lenguaje de proteínas*, ESM-2, un *transformer* con hasta 15 000 millones de parámetros entrenado únicamente a predecir aminoácidos enmascarados en millones de secuencias. Para acertar qué aminoácido falta, el modelo tiene que aprender implícitamente qué residuos interactúan, es decir, la coevolución, y esa información queda en sus representaciones internas. ESMFold acopla ESM-2 a un módulo de plegamiento y predice estructuras a partir de una sola secuencia, hasta un orden de magnitud más rápido que AlphaFold2, a cambio de una exactitud algo menor en proteínas con pocos homólogos. Su velocidad permitió plegar más de 600 millones de proteínas metagenómicas.

AlphaFold3. Abramson et al. (2024) extendieron el enfoque a casi toda la química de la célula: proteínas, ácidos nucleicos, ligandos pequeños, iones y residuos modificados, en un mismo complejo. Los cambios de arquitectura son profundos: el Evoformer se reemplaza por un módulo más ligero (Pairformer) que procesa el MSA de forma reducida, y el módulo de estructura se reemplaza por un *módulo de difusión* que genera directamente coordenadas atómicas partiendo de ruido y eliminándolo paso a paso, sin marcos ni torsiones. En las pruebas de sus autores, AlphaFold3 superó a los métodos de *docking* clásicos en la predicción de complejos proteína-ligando, un resultado que conecta directamente con la sección 15.3.

ColabFold. Mirdita et al. (2022) hicieron AlphaFold2 accesible a cualquier investigador: reemplazaron la búsqueda de homólogos por MMseqs2 en un servidor, lo que acelera la construcción del MSA en uno o dos órdenes de magnitud, y lo empaquetaron en cuadernos de Google Colab y en una herramienta de línea de órdenes. Es la forma más sencilla de predecir la estructura de una proteína o de un complejo sin infraestructura propia.

Predicción con ColabFold en local

```
# instalación: https://github.com/sokrypton/ColabFold
colabfold_batch --num-recycle 3 --amber --use-gpu-relax \
  p53_dbd.fasta resultados/
ls resultados/
# p53_dbd_unrelaxed_rank_001..._model_3_seed_000.pdb
# p53_dbd_scores_rank_001..._model_3_seed_000.json (pLDDT, PAE)
# p53_dbd_coverage.png p53_dbd_pae.png p53_dbd_plddt.png
```

Tabla 15.2. Medidas de confianza que acompañan a los modelos de AlphaFold y afines, y la pregunta que responde cada una.

Medida	Alcance	Pregunta que responde
pLDDT	por residuo (0–100)	¿está bien predicho el entorno local de este residuo?
PAE	por par de residuos (Å)	¿está bien colocado <i>j</i> respecto a <i>i</i> ? Revela dominios y su orientación relativa.
pTM	global (0–1)	TM-score esperado del modelo completo.
ipTM	interfaz (0–1)	en complejos, ¿está bien predicha la interfaz entre cadenas?

 **El premio del problema de cincuenta años**

El problema del plegamiento se formuló, en su versión moderna, con el experimento de Anfinsen a principios de los sesenta. CASP se creó en 1994 precisamente porque los avances anunciados rara vez resistían una evaluación a ciegas. Veintiséis años después, en CASP14, los organizadores declararon que, para las proteínas individuales, el problema estaba esencialmente resuelto (Kryshtafovych et al., 2021). En 2024, Demis Hassabis y John Jumper compartieron el premio Nobel de Química por AlphaFold, junto con David Baker, del grupo de RoseTTAFold, por el diseño computacional de proteínas.

15.3 Acoplamiento molecular (docking)

 **Colab · 15.3 · Docking molecular básico**

 **repositorio del curso**

Una llave en la oscuridad

Imagine que debe encontrar, a oscuras, cuál de mil llaves abre una cerradura, y en qué posición entra. Puede palpar la forma de la cerradura, pero no ver. Para cada llave tiene que hacer dos cosas: *probar* posiciones (girlinga, inclinarla, empujarla, doblar las llaves articuladas) y *juzgar* cada posición (¿encaja sin forzar?, ¿hace contacto con los pernos?). Si prueba pocas posiciones, puede descartar la llave correcta porque nunca la metió bien; si su tacto es torpe, puede aceptar una llave que entra pero no gira. El acoplamiento molecular

tiene exactamente esos dos componentes, la búsqueda y la puntuación, y casi todos sus fracasos provienen de uno de ellos.

15.3.1 El problema

El *acoplamiento molecular* (*docking*) predice la posición, la orientación y la conformación (juntas, la *pose*) de una molécula pequeña, el *ligando*, unida a una macromolécula, el *receptor*, y estima la fuerza de esa unión. Es la herramienta central del diseño de fármacos basado en estructura: permite cribar virtualmente millones de compuestos contra una diana, proponer cómo se une un inhibidor conocido o sugerir modificaciones químicas que mejoren su afinidad.

Kuntz et al. (1982) formularon el primer método: representaron el bolsillo del receptor como un conjunto de esferas que rellenan su hueco y buscaron cómo superponer los átomos del ligando sobre los centros de esas esferas, evaluando la complementariedad de forma. Su programa, DOCK, estableció el principio que sigue vigente: el ligando debe encajar geométricamente (complementariedad de *forma*) y sus grupos deben enfrentarse a los grupos adecuados del receptor (complementariedad *química*: donadores frente a aceptores de puentes de hidrógeno, cargas opuestas, superficies hidrofóbicas juntas).

Formalmente, si $\mathbf{x} = (\mathbf{t}, \mathbf{r}, \tau)$ reúne la traslación $\mathbf{t} \in \mathbb{R}^3$, la orientación $\mathbf{r}$ (tres parámetros, por ejemplo un cuaternión unitario o un vector de rotación) y los N_{rot} ángulos de torsión τ del ligando, el *docking* busca

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathcal{B} \times SO(3) \times \mathbb{T}^{N_{\text{rot}}}} S(\mathbf{x}), \quad (15.16)$$

donde S es una función de puntuación que aproxima la energía libre de unión y $\mathcal{B}$ es la caja de búsqueda. El receptor suele tratarse como rígido.

DONDE

$\mathbf{x}$	Pose del ligando: posición, orientación y torsiones.
$\mathcal{B}$	Caja de búsqueda (región del receptor donde se permite colocar el centro del ligando).
$\mathbb{T}^{N_{\text{rot}}}$	Toro de dimensión N_{rot} : cada torsión es un ángulo periódico.
$S(\mathbf{x})$	Puntuación de la pose; más negativa significa unión más favorable.

Ambos factores son difíciles. El espacio de búsqueda de un ligando típico, con 5 a 10 enlaces rotables, tiene 11 a 16 dimensiones continuas y está lleno de mínimos locales. Y la energía libre de unión real incluye efectos que ninguna función

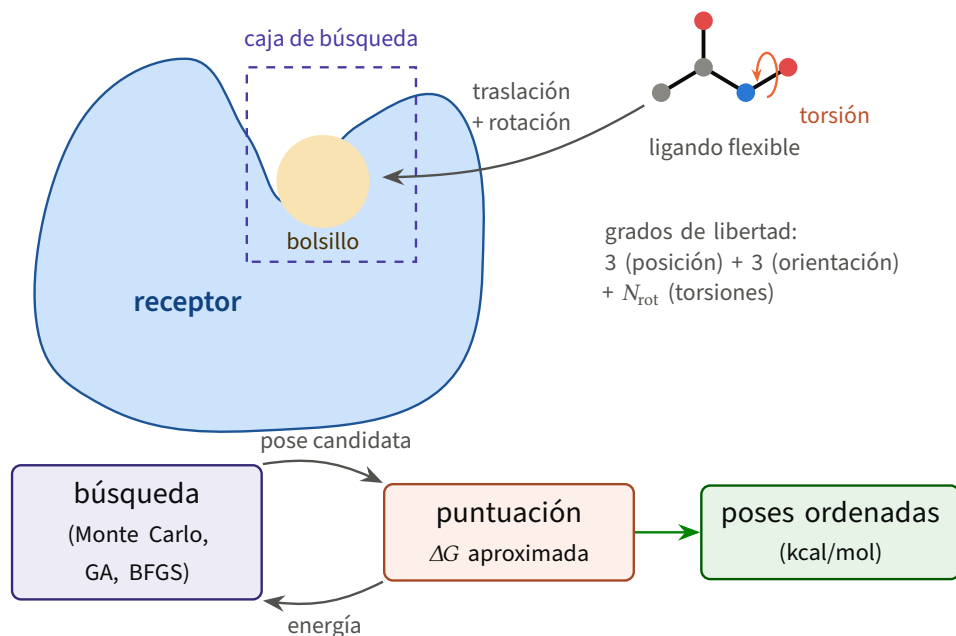


Figura 15.11. Los dos componentes del *docking*. **Arriba:** el ligando flexible debe colocarse dentro de una caja de búsqueda que rodea el bolsillo del receptor; su pose se describe con tres coordenadas de posición, tres de orientación y un ángulo por cada enlace rotable, $6 + N_{\text{rot}}$ grados de libertad en total. **Abajo:** el algoritmo de búsqueda propone poses y la función de puntuación las evalúa; el ciclo se repite hasta converger, y la salida es una lista de poses ordenadas por su energía estimada.

rápida captura bien: la desolvatación de ligando y bolsillo, la pérdida de entropía conformacional, el papel de moléculas de agua concretas, la polarización.

15.3.2 Encontrar el sitio de unión

Antes de acoplar hay que saber dónde. Si existe una estructura del receptor con un ligando cocrystalizado, el bolsillo es conocido y la caja se centra en él. Si no, los detectores de bolsillos buscan cavidades en la superficie con criterios geométricos (regiones cóncavas donde cabe una esfera de sonda pero que están rodeadas de proteína en muchas direcciones), a menudo combinados con la conservación evolutiva de los residuos que las tapizan: un bolsillo conservado en toda una familia probablemente tiene función. El docking “ciego”, con una caja que abarca toda la proteína, es posible pero multiplica el espacio de búsqueda y las poses falsas.

15.3.3 Funciones de puntuación

Hay tres grandes familias de funciones de puntuación: las basadas en *campos de fuerza* (términos de van der Waals y electrostáticos de la mecánica molecular), las *empíricas* (suma de términos físicamente motivados cuyos pesos se ajustan por regresión a afinidades experimentales) y las *basadas en conocimiento* (potenciales estadísticos derivados de las frecuencias de contactos observadas en complejos del PDB). La función de AutoDock Vina es un híbrido de las dos últimas y merece un estudio detallado, porque es la más usada en la práctica.

Trott y Olson (2010) definen la energía de interacción como una suma sobre pares de átomos (i, j) del ligando y del receptor, separados por más de tres enlaces y a menos de 8 Å, de una función de la *distancia de superficie*

$$d_{ij} = r_{ij} - R_t - R_j, \quad (15.17)$$

donde r_{ij} es la distancia entre centros y R_t el radio de van der Waals del tipo de átomo t (por ejemplo, 1,9 Å para el carbono, 1,8 para el nitrógeno, 1,7 para el oxígeno). Los términos son

$$\begin{aligned} \text{gauss}_1(d) &= e^{-(d/0,5)^2}, & \text{gauss}_2(d) &= e^{-((d-3)/2)^2}, \\ \text{rep}(d) &= \begin{cases} d^2 & d < 0 \\ 0 & d \geq 0 \end{cases}, & \text{hidrof}(d) &= \begin{cases} 1 & d < 0,5 \\ 1,5 - d & 0,5 \leq d \leq 1,5 \\ 0 & d > 1,5 \end{cases}, \\ \text{puenteH}(d) &= \begin{cases} 1 & d < -0,7 \\ -d/0,7 & -0,7 \leq d \leq 0 \\ 0 & d > 0 \end{cases} \end{aligned} \quad (15.18)$$

con pesos $w_{g1} = -0,0356$, $w_{g2} = -0,00516$, $w_{\text{rep}} = 0,840$, $w_{\text{hidrof}} = -0,0351$ y $w_H = -0,587$. El término hidrofóbico solo se aplica entre átomos hidrofóbicos, y el de puente de hidrógeno solo entre un donador y un aceptor. La energía intermolecular total se corrige por la flexibilidad del ligando:

$$S = \frac{\sum_{i < j} \sum_m w_m h_m(d_{ij})}{1 + w_{\text{rot}} N_{\text{rot}}}, \quad w_{\text{rot}} = 0,0585. \quad (15.19)$$

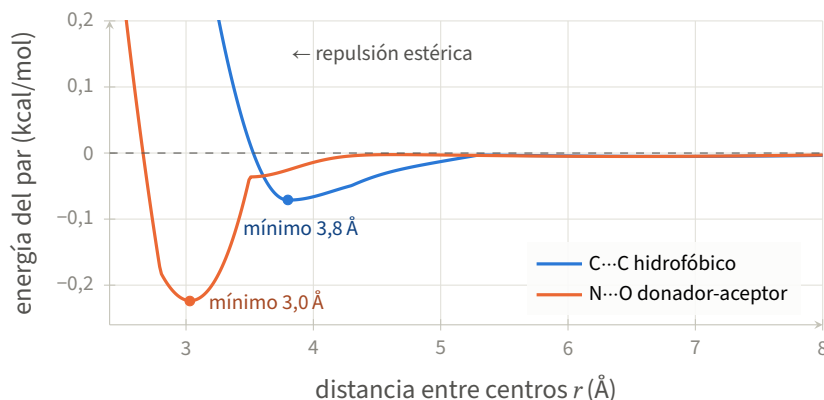


Figura 15.12. Energía de un par de átomos según la función de puntuación de AutoDock Vina (ecuación (15.18)), en función de la distancia entre centros. Para dos carbonos hidrofóbicos (azul), el mínimo, de $-0,071$ kcal/mol, está en $3,8$ Å, justo donde se tocan sus superficies de van der Waals. Un par donador-aceptor de puente de hidrógeno (naranja) es tres veces más favorable y su mínimo se desplaza a $3,0$ Å, la distancia típica N...O de un puente de hidrógeno. A distancias cortas domina la repulsión cuadrática. Un ligando con decenas de contactos suma estas pequeñas contribuciones hasta alcanzar varias kcal/mol. Curvas calculadas con `figuras/cap15/generar.py`.

DONDE

r_{ij}	Distancia entre los centros de los átomos i (ligando) y j (receptor).
R_t	Radio de van der Waals del tipo de átomo t .
d_{ij}	Distancia entre las superficies atómicas; negativa si las esferas se interpenetran.
h_m, w_m	Término m de la ecuación (15.18) y su peso ajustado.
N_{rot}	Número de enlaces rotables del ligando (entre átomos pesados, no terminales).
S	Puntuación final, que Vina expresa en kcal/mol como estimación de ΔG de unión.

La figura 15.12 muestra la forma de estos términos. Tienen una lógica clara: la atracción gaussiana estrecha premia el contacto íntimo; la ancha, la proximidad general; la repulsión castiga los choques; y los dos términos lineales recompensan los contactos hidrofóbicos y los puentes de hidrógeno. Llama la atención lo que *falta*: no hay término electrostático explícito ni desolvatación. Los pesos se ajustaron

para reproducir afinidades experimentales, así que esos efectos quedan absorbidos, de forma promedio, en los términos existentes. Es una función deliberadamente simple, y su virtud es la velocidad.

Ejemplo 15.5. (De la puntuación a la constante de disociación)

Suponga que la suma intermolecular de una pose es de $-10,5$ kcal/mol y que el ligando tiene $N_{\text{rot}} = 5$ enlaces rotables. La ecuación (15.19) divide por $1 + 0,0585 \times 5 = 1,29$, de modo que $S = -8,12$ kcal/mol: la flexibilidad le cuesta al ligando más de 2 kcal/mol, porque al unirse pierde la libertad de girar esos enlaces. Si interpretamos S como energía libre de unión, la relación termodinámica $\Delta G = RT \ln K_d$ a 298 K ($RT = 0,593$ kcal/mol) da

$$K_d = e^{\Delta G/RT} = e^{-8,12/0,593} \approx 1,1 \times 10^{-6} \text{ M} \approx 1,1 \text{ } \mu\text{M}.$$

Cada $RT \ln 10 = 1,36$ kcal/mol adicionales multiplica la afinidad por diez: -9 kcal/mol corresponde a unos $0,25 \text{ } \mu\text{M}$ y -12 kcal/mol a unos $1,6 \text{ nM}$. Pero no tome estas cifras al pie de la letra: el error típico de las funciones de puntuación al predecir afinidades absolutas es de unas 2 kcal/mol, es decir, más de un orden de magnitud en K_d . El *docking* ordena poses de un mismo ligando mucho mejor que afinidades entre ligandos distintos.

15.3.4 Búsqueda conformacional

Los algoritmos de búsqueda también son variados: búsqueda sistemática sobre una rejilla de ángulos, construcción incremental del ligando fragmento a fragmento dentro del bolsillo, algoritmos genéticos (el de AutoDock 4) y métodos de Monte Carlo. Vina usa una *búsqueda local iterada*: parte de una pose aleatoria, la perturba (un desplazamiento, un giro o el cambio de una torsión), minimiza localmente la energía con el método cuasi-Newton BFGS usando las derivadas analíticas de la función de puntuación, y acepta o rechaza la nueva pose con el criterio de Metropolis,

$$P(\text{aceptar}) = \min(1, e^{-(S_{\text{nueva}} - S_{\text{actual}})/T}), \quad (15.20)$$

que acepta siempre las mejoras y ocasionalmente los empeoramientos, para escapar de mínimos locales. Varias de estas cadenas corren en paralelo (el parámetro exhaustiveness controla cuántas), y para acelerar la evaluación, la contribución de cada tipo de átomo del ligando se precalcula en una rejilla tridimensional sobre la caja de búsqueda. Con estas decisiones, Trott y Olson (2010) informaron una

aceleración de unos dos órdenes de magnitud respecto a AutoDock 4 junto con una mayor exactitud en la predicción de poses.

DONDE

$S_{\text{nueva}}, S_{\text{actual}}$	Puntuaciones de la pose propuesta (ya minimizada) y de la actual.
T	“Temperatura” del algoritmo: controla cuánto se toleran los empeoramientos.

La versión 1.2 (Eberhardt et al., 2021) añadió enlaces de Python, la función de puntuación de AutoDock 4 como alternativa, el acoplamiento simultáneo de varios ligandos, el tratamiento de macrociclos flexibles y un protocolo con moléculas de agua explícitas. El flujo completo en Python es breve:

vina_docking.py

```
from vina import Vina                                # AutoDock Vina >= 1.2

v = Vina(sf_name="vina", seed=42)
v.set_receptor("receptor.pdbqt")                    # con hidrógenos polares
v.set_ligand_from_file("ligando.pdbqt")             # torsiones definidas
v.compute_vina_maps(center=[15.2, 53.9, 16.9],        # centro del bolsillo
                    box_size=[20, 20, 20])           # caja en angstroms
v.dock(exhaustiveness=16, n_poses=9)
print(v.energies(n_poses=5)[: , 0])                 # kcal/mol por pose
v.write_poses("poses.pdbqt", n_poses=5, overwrite=True)
```

La preparación de los archivos .pdbqt (añadir hidrógenos, asignar tipos de átomo y cargas, definir las torsiones rotables del ligando) se hace con herramientas como Meeko u Open Babel, y es una fuente frecuente de errores silenciosos: un estado de protonación equivocado de una histidina del bolsillo o de un grupo ácido del ligando puede cambiar por completo el resultado.

15.3.5 El embudo de unión y la validación por redocking

La imagen del embudo de la figura 15.6 vuelve a ser útil. Si la función de puntuación es buena, las poses se ordenan como un embudo: cuanto más se parecen a la pose real, mejor puntuación tienen, y la mejor puntuada está cerca de la real. Si la función tiene un mínimo falso, un “embudo trampa” en otra región del bolsillo, la búsqueda puede converger allí con total confianza.

Por eso, antes de usar un protocolo de *docking* con compuestos nuevos, se *valida* con un caso de respuesta conocida: el *redocking*. Se toma una estructura

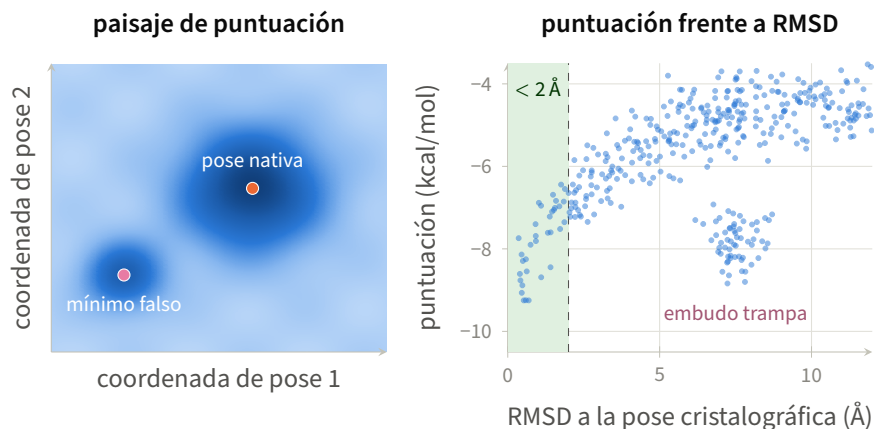


Figura 15.13. El embudo de unión en un ejemplo simulado. **Izquierda:** paisaje de puntuación sobre dos coordenadas de pose (tonos oscuros = puntuación más favorable), con el mínimo global en la pose nativa y un mínimo local profundo en otra región del bolsillo. **Derecha:** 400 poses generadas por la búsqueda, con su puntuación frente a su RMSD respecto a la pose real. La mayoría forma un embudo cuyo fondo está a menos de 2 Å (franja verde): en este caso la mejor pose tiene un RMSD de 0,53 Å y 7 de las 10 mejores son correctas. El grupo en torno a 7,5 Å es un embudo trampa: si la función de puntuación lo favoreciera un poco más, el *docking* fallaría con una puntuación excelente. Datos simulados con `figuras/cap15/generar.py`.

cristalográfica del receptor con su ligando, se extrae el ligando, se aleatoriza su conformación y se vuelve a acoplar con el protocolo que se quiere evaluar. Después se compara la pose obtenida con la cristalográfica mediante el RMSD de los átomos pesados, *sin superponer* (ambas están ya en el sistema de referencia del receptor) y teniendo en cuenta la simetría del ligando (un anillo de benceno girado 180° es la misma pose):

$$\text{RMSD}_{\text{sim}} = \min_{\pi \in \text{Aut}(G)} \sqrt{\frac{1}{N} \sum_{k=1}^N \|\mathbf{p}_k - \mathbf{q}_{\pi(k)}\|^2}. \quad (15.21)$$

DONDE

$\mathbf{p}_k, \mathbf{q}_k$	Posiciones del átomo pesado k del ligando en la pose predicha y en la cristalográfica.
$\text{Aut}(G)$	Automorfismos del grafo molecular: permutaciones de átomos que dejan la molécula invariante.
N	Número de átomos pesados del ligando.

El criterio convencional es que una pose es correcta si $\text{RMSD}_{\text{sim}} < 2 \text{ \AA}$. Un protocolo que no reproduce la pose cristalográfica del propio ligando en su propio receptor difícilmente acertará con ligandos nuevos. Se informan dos tasas: la de éxito *top-1* (la pose mejor puntuada es correcta) y la de éxito *top-N* (alguna de las N mejores lo es); la diferencia entre ambas separa los fallos de puntuación de los de búsqueda. Si la pose correcta nunca aparece entre las generadas, falló la búsqueda; si aparece pero no en primer lugar, falló la puntuación.

redocking_rmsd.py

```
from rdkit import Chem
from rdkit.Chem import rdMolAlign

ref = Chem.MolFromMolFile("ligando_cristal.sdf")
poses = Chem.SDMolSupplier("poses_vina.sdf")
for i, pose in enumerate(poses, start=1):
    # CalcRMS no superpone y considera la simetría del ligando
    r = rdMolAlign.CalcRMS(pose, ref)
    ok = "correcta" if r < 2.0 else "incorrecta"
    print(f"pose {i}: RMSD = {r:5.2f} A ({ok})")
```

15.3.6 Limitaciones, y el aprendizaje profundo

El *docking* clásico es útil pero tiene limitaciones que todo usuario debe conocer:

- **Receptor rígido.** Las proteínas se ajustan al ligando (*induced fit*); una estructura cristalizada con otro ligando, o sin ninguno, puede tener el bolsillo cerrado o con cadenas laterales en posiciones incompatibles. Acoplar contra varias conformaciones del receptor (*ensemble docking*) o permitir flexibilidad en algunas cadenas laterales mitiga el problema a un coste computacional mayor.
- **Agua y protonación.** Las moléculas de agua que median puentes entre ligando y proteína, y los estados de protonación, rara vez se modelan bien.

- **Puntuaciones que no son afinidades.** Como vimos en el ejemplo 15.5, el error en afinidad es del orden de 2 kcal/mol. En cribado virtual, las puntuaciones sirven para *enriquecer* la lista de candidatos, no para predecir cuáles son activos.
- **Modelos predichos.** Acoplar contra un modelo de AlphaFold es posible, pero las cadenas laterales del bolsillo, incluso con pLDDT alto, pueden no estar en la conformación de unión; la exactitud suele caer respecto a las estructuras experimentales cocrystalizadas.

El aprendizaje profundo también ha llegado al *docking*. Corso et al. (2022) replantearon el problema como uno de *modelado generativo*: en lugar de buscar el mínimo de una función de puntuación, DiffDock aprende a generar poses mediante un proceso de difusión sobre las traslaciones, rotaciones y torsiones del ligando, y un modelo de confianza ordena las poses generadas. En su evaluación sobre complejos de PDBBind, obtuvo una tasa de éxito *top-1* ($\text{RMSD} < 2 \text{ \AA}$) del 38 %, frente al 23 % de los mejores métodos de búsqueda tradicionales, sin necesidad de indicar el bolsillo. AlphaFold3, como vimos, va más allá y predice de una vez la estructura de la proteína con su ligando (Abramson et al., 2024). Estos métodos no han jubilado a Vina: son más lentos por compuesto, pueden producir geometrías químicamente imposibles y su rendimiento cae en proteínas muy diferentes de las de entrenamiento. En la práctica actual, ambos enfoques se combinan.

⚠ Cinco comprobaciones antes de creer un resultado de docking

1. ¿Se validó el protocolo por *redocking* en la misma diana, con $\text{RMSD} < 2 \text{ \AA}$?
2. ¿Es razonable la química de la pose (puentes de hidrógeno con geometría correcta, grupos polares no enterrados sin pareja)?
3. ¿Son robustas las mejores poses frente a cambios en la semilla aleatoria, la exhaustiveness y el tamaño de la caja?
4. ¿Se comparó la puntuación con la de ligandos conocidos y con la de señuelos (*decoys*) de propiedades similares?
5. ¿Hay evidencia experimental independiente (mutagénesis, relación estructura-actividad) que apoye la pose?

El *docking* es búsqueda más puntuación. Valide ambas con *re-docking* ($\text{RMSD} < 2 \text{ \AA}$) y use las puntuaciones para ordenar, no para medir afinidades.

☰ En resumen

- La conformación del esqueleto proteico se describe con dos diedros por residuo, ϕ y ψ , porque el enlace peptídico es plano. El diagrama de Ramachandran, predicho por estereoquímica en 1963, se confirma con estructuras reales: hélices en torno a $(-64^\circ, -40^\circ)$, láminas en torno a $(-110^\circ, 120^\circ)$ y la glicina como excepción.
- La cristalografía, la RMN y la crio-EM miden señales distintas y producen modelos con medidas de calidad distintas (resolución, R_{free} , factor B , ensambles); en 2025 la crio-EM ya aporta el 41 % de las nuevas entradas del PDB, cuyo formato oficial es mmCIF.
- El algoritmo de Kabsch obtiene la superposición óptima con una SVD de la matriz de covarianza cruzada, corrigiendo las reflexiones. El RMSD está dominado por los peores residuos y depende de la longitud; el TM-score mide la fracción de estructura que coincide y es comparable entre proteínas.
- La hipótesis de Anfinsen justifica predecir la estructura a partir de la secuencia; la paradoja de Levinthal se resuelve con paisajes de energía en embudo. La predicción pasó del modelado por homología a la coevolución (DCA) y al aprendizaje profundo.
- AlphaFold2 aprende de extremo a extremo con el Evoformer (atención sobre MSA y pares, actualizaciones triangulares) y un módulo de estructura equivariante, y estima su error con el pLDDT (local) y el PAE (relativo entre residuos). ESMFold sustituye el MSA por un modelo de lenguaje; AlphaFold3 extiende la predicción a complejos con ligandos y ácidos nucleicos.
- El *docking* combina una búsqueda en $6 + N_{\text{rot}}$ dimensiones con una función de puntuación aproximada, como la de Vina. Se valida por *redocking* con un RMSD simétrico menor de $2 \text{ \AA}$, y sus puntuaciones sirven para ordenar poses, no para medir afinidades con precisión.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 15.1** ●●● Descargue 1UBQ en formato mmCIF, calcule (ϕ, ψ) de todos sus residuos con el programa `phi_psi.py` y dibuje el diagrama de Ramachandran. ¿Qué residuos tienen $\phi > 0$? Compruebe cuántos de ellos son glicinas.
- 15.2** ●●● Usando la ecuación (15.5), calcule el desplazamiento tridimensional típico de un átomo con $B = 30 \text{ \AA}^2$ y con $B = 120 \text{ \AA}^2$. ¿Confiaría en la orientación de una cadena lateral de lisina cuyos átomos tienen $B \approx 120$?
- 15.3** ●●● Implemente la función `kabsch_rmsd` y compruebe con estructuras sintéticas que: (a) si $\mathbf{Q} = \mathbf{R}_0\mathbf{P} + \mathbf{t}_0$ para una rotación conocida, recupera $\mathbf{R}_0$ y $\text{RMSD} = 0$; (b) si $\mathbf{Q}$ es la imagen especular de $\mathbf{P}$, el RMSD no es cero cuando se aplica la corrección d , y sí lo es si se omite. ¿Qué significa esto biológicamente?
- 15.4** ●●● Demuestre que, para matrices ortogonales $\mathbf{M}$, se cumple $|M_{jj}| \leq 1$ y, por lo tanto, $\text{tr}(\mathbf{M}\Sigma) \leq \sum_j \sigma_j$. ¿Por qué, cuando $\det(\mathbf{V}\mathbf{U}^\top) = -1$, la mejor rotación propia sacrifica precisamente el menor valor singular?
- 15.5** ●●● Calcule el TM-score entre la ubiquitina cristalográfica y cada modelo del ensamble de RMN 1D3Z, con y sin la cola C-terminal. Compare la variación relativa del TM-score con la del RMSD y explique la diferencia a partir de la ecuación (15.9).
- 15.6** ●●● Obtenga de AlphaFold DB el modelo de una proteína de su interés que tenga estructura experimental publicada después de 2021. Superponga ambas en los residuos con pLDDT > 70 y calcule RMSD y TM-score. ¿Coinciden las regiones de pLDDT bajo con las que faltan o son flexibles en la estructura experimental?
- 15.7** ●●● Implemente la función de puntuación de Vina (ecuaciones (15.17) a (15.19)) para un complejo proteína-ligando de su elección, usando solo los átomos pesados y una tipificación simplificada (C hidrofóbico, N/O donador o aceptor). Compare su valor para la pose cristalográfica con el que devuelve `Vina.score()`.
- 15.8** ●●● Elija cinco complejos proteína-ligando del PDB y realice un experimento de *redocking* con Vina variando exhaustiveness (1, 8, 32). Calcule

las tasas de éxito *top-1* y *top-5* con RMSD simétrico $< 2 \text{ \AA}$, y decida en cada fallo si es un error de búsqueda o de puntuación.

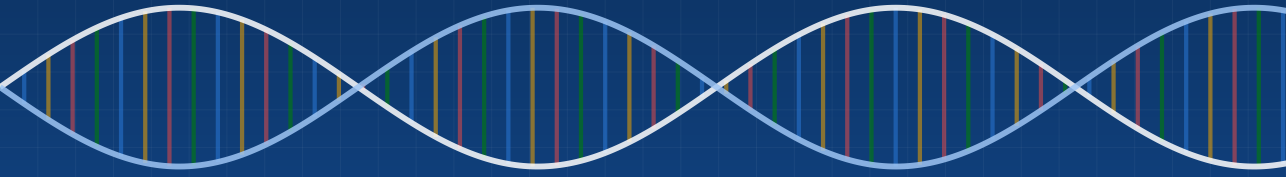
Referencias del capítulo

- Abramson, J., Adler, J., Dunger, J., Evans, R., Green, T., Pritzel, A., Ronneberger, O., et al. (2024). Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630(8016), 493-500. <https://doi.org/10.1038/s41586-024-07487-w>
- Anfinsen, C. B. (1973). Principles that govern the folding of protein chains. *Science*, 181(4096), 223-230. <https://doi.org/10.1126/science.181.4096.223>
- Baek, M., DiMaio, F., Anishchenko, I., Dauparas, J., Ovchinnikov, S., Lee, G. R., Wang, J., et al. (2021). Accurate prediction of protein structures and interactions using a three-track neural network. *Science*, 373(6557), 871-876. <https://doi.org/10.1126/science.abj8754>
- Berman, H. M., Westbrook, J., Feng, Z., Gilliland, G., Bhat, T. N., Weissig, H., Shindyalov, I. N., & Bourne, P. E. (2000). The Protein Data Bank. *Nucleic Acids Research*, 28(1), 235-242. <https://doi.org/10.1093/nar/28.1.235>
- Branden, C., & Tooze, J. (1999). *Introduction to Protein Structure* (2.^a ed.). Garland Science.
- Burley, S. K., Bhikadiya, C., Bi, C., Bittrich, S., Chao, H., Chen, L., Craig, P. A., et al. (2023). RCSB Protein Data Bank (RCSB.org): delivery of experimentally-determined PDB structures alongside one million computed structure models of proteins from artificial intelligence/machine learning. *Nucleic Acids Research*, 51(D1), D488-D508. <https://doi.org/10.1093/nar/gkac1077>
- Corso, G., Stärk, H., Jing, B., Barzilay, R., & Jaakkola, T. (2022). *DiffDock: Diffusion Steps, Twists, and Turns for Molecular Docking* [Publicado en la International Conference on Learning Representations (ICLR 2023)]. arXiv: 2210.01776. <https://doi.org/10.48550/arXiv.2210.01776>
- Dill, K. A., & Chan, H. S. (1997). From Levinthal to pathways to funnels. *Nature Structural Biology*, 4(1), 10-19. <https://doi.org/10.1038/nsb0197-10>
- Eberhardt, J., Santos-Martins, D., Tillack, A. F., & Forli, S. (2021). AutoDock Vina 1.2.0: New Docking Methods, Expanded Force Field, and Python Bindings. *Journal of Chemical Information and Modeling*, 61(8), 3891-3898. <https://doi.org/10.1021/acs.jcim.1c00203>
- Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583-589. <https://doi.org/10.1038/s41586-021-03819-2>
- Kabsch, W. (1976). A solution for the best rotation to relate two sets of vectors. *Acta Crystallographica Section A*, 32(5), 922-923. <https://doi.org/10.1107/S0567739476001873>

- Kabsch, W., & Sander, C. (1983). Dictionary of protein secondary structure: Pattern recognition of hydrogen-bonded and geometrical features. *Biopolymers*, 22(12), 2577-2637. <https://doi.org/10.1002/bip.360221211>
- Kendrew, J. C., Bodo, G., Dintzis, H. M., Parrish, R. G., Wyckoff, H., & Phillips, D. C. (1958). A Three-Dimensional Model of the Myoglobin Molecule Obtained by X-Ray Analysis. *Nature*, 181(4610), 662-666. <https://doi.org/10.1038/181662a0>
- Kryshtafovych, A., Schwede, T., Topf, M., Fidelis, K., & Moult, J. (2021). Critical assessment of methods of protein structure prediction (CASP)—Round XIV. *Proteins: Structure, Function, and Bioinformatics*, 89(12), 1607-1617. <https://doi.org/10.1002/prot.26237>
- Kühlbrandt, W. (2014). The Resolution Revolution. *Science*, 343(6178), 1443-1444. <https://doi.org/10.1126/science.1251652>
- Kuntz, I. D., Blaney, J. M., Oatley, S. J., Langridge, R., & Ferrin, T. E. (1982). A geometric approach to macromolecule-ligand interactions. *Journal of Molecular Biology*, 161(2), 269-288. [https://doi.org/10.1016/0022-2836\(82\)90153-X](https://doi.org/10.1016/0022-2836(82)90153-X)
- Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., Smetanin, N., et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130. <https://doi.org/10.1126/science.ade2574>
- Mariani, V., Biasini, M., Barbato, A., & Schwede, T. (2013). IDDT: a local superposition-free score for comparing protein structures and models using distance difference tests. *Bioinformatics*, 29(21), 2722-2728. <https://doi.org/10.1093/bioinformatics/btt473>
- Mirdita, M., Schütze, K., Moriwaki, Y., Heo, L., Ovchinnikov, S., & Steinegger, M. (2022). ColabFold: making protein folding accessible to all. *Nature Methods*, 19(6), 679-682. <https://doi.org/10.1038/s41592-022-01488-1>
- Morcos, F., Pagnani, A., Lunt, B., Bertolino, A., Marks, D. S., Sander, C., Zecchina, R., et al. (2011). Direct-coupling analysis of residue coevolution captures native contacts across many protein families. *Proceedings of the National Academy of Sciences*, 108(49), E1293-E1301. <https://doi.org/10.1073/pnas.1111471108>
- Moult, J., Pedersen, J. T., Judson, R., & Fidelis, K. (1995). A large-scale experiment to assess protein structure prediction methods. *Proteins: Structure, Function, and Bioinformatics*, 23(3), ii-v. <https://doi.org/10.1002/prot.340230303>
- Murzin, A. G., Brenner, S. E., Hubbard, T., & Chothia, C. (1995). SCOP: A structural classification of proteins database for the investigation of sequences and structures. *Journal of Molecular Biology*, 247(4), 536-540. [https://doi.org/10.1016/S0022-2836\(05\)80134-2](https://doi.org/10.1016/S0022-2836(05)80134-2)
- Orengo, C., Michie, A., Jones, S., Jones, D., Swindells, M., & Thornton, J. (1997). CATH – a hierarchic classification of protein domain structures. *Structure*, 5(8), 1093-1109. [https://doi.org/10.1016/S0969-2126\(97\)00260-8](https://doi.org/10.1016/S0969-2126(97)00260-8)
- Pauling, L., Corey, R. B., & Branson, H. R. (1951). The structure of proteins: Two hydrogen-bonded helical configurations of the polypeptide chain. *Proceedings of the National Academy of Sciences*, 37(4), 205-211. <https://doi.org/10.1073/pnas.37.4.205>

- Ramachandran, G., Ramakrishnan, C., & Sasisekharan, V. (1963). Stereochemistry of polypeptide chain configurations. *Journal of Molecular Biology*, 7(1), 95-99. [https://doi.org/10.1016/S0022-2836\(63\)80023-6](https://doi.org/10.1016/S0022-2836(63)80023-6)
- Šali, A., & Blundell, T. L. (1993). Comparative Protein Modelling by Satisfaction of Spatial Restraints. *Journal of Molecular Biology*, 234(3), 779-815. <https://doi.org/10.1006/jmbi.1993.1626>
- Trott, O., & Olson, A. J. (2010). AutoDock Vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of Computational Chemistry*, 31(2), 455-461. <https://doi.org/10.1002/jcc.21334>
- Varadi, M., Anyango, S., Deshpande, M., Nair, S., Natassia, C., Yordanova, G., Yuan, D., et al. (2022). AlphaFold Protein Structure Database: massively expanding the structural coverage of protein-sequence space with high-accuracy models. *Nucleic Acids Research*, 50(D1), D439-D444. <https://doi.org/10.1093/nar/gkab1061>
- Zhang, Y., & Skolnick, J. (2004). Scoring function for automated assessment of protein structure template quality. *Proteins: Structure, Function, and Bioinformatics*, 57(4), 702-710. <https://doi.org/10.1002/prot.20264>
- Zwanzig, R., Szabo, A., & Bagchi, B. (1992). Levinthal's paradox. *Proceedings of the National Academy of Sciences*, 89(1), 20-22. <https://doi.org/10.1073/pnas.89.1.20>

Biología de sistemas y redes



Este enfoque puede revelar los bloques de construcción básicos de la mayoría de las redes.

— Milo et al. (2002)

● Al terminar este capítulo usted podrá

- Representar un interactoma como un grafo y calcular, a mano y con NetworkX, grado, coeficiente de agrupamiento, caminos mínimos, intermediación y modularidad.
- Explicar cómo se obtienen las interacciones proteína-proteína (doble híbrido, purificación por afinidad con espectrometría de masas), qué errores introduce cada método y cómo los integran STRING y BioGRID.
- Derivar las propiedades de los modelos de Erdős-Rényi, Watts-Strogatz y Barabási-Albert, y evaluar con espíritu crítico la afirmación de que las redes biológicas son “libres de escala”.
- Construir modelos de ecuaciones diferenciales de la expresión génica con funciones de Hill, y deducir de ellos el tiempo de respuesta, el efecto de la autorregulación negativa y la función del *feed-forward loop*.

- Analizar la estabilidad de un circuito (nulclinas, jacobiano, autovalores) para explicar la biestabilidad del interruptor genético y las oscilaciones del *repressor*.
- Simular el ruido de la expresión con el algoritmo de Gillespie e inferir redes de regulación a partir de datos de expresión con GENIE3.

Hasta aquí hemos estudiado secuencias, estructuras y transcritos principalmente por separado; en el capítulo 11, por ejemplo, medimos cambios de abundancia entre condiciones. Sin embargo, la función biológica depende también de las interacciones. En la levadura, la quinasa FUS3 transmite la señal de una feromona gracias a una proteína andamio que la acerca a su activadora. Después fosforila un factor de transcripción que activa decenas de genes. Para representar estas *relaciones* entre moléculas utilizaremos grafos.

La biología de sistemas estudia esas relaciones con dos miradas complementarias, que corresponden a las dos lecciones de este capítulo. La primera es *estructural*: ¿cómo está cableada la célula?, ¿qué proteínas son centrales?, ¿se agrupan en módulos? La abordaremos con las redes de interacción proteína-proteína, sus métodos experimentales, sus bases de datos y la teoría de grafos que permite medirlas (sección 16.1). La segunda es *dinámica*: dado un pequeño circuito de genes que se regulan entre sí, ¿qué hace en el tiempo? Responderemos con ecuaciones diferenciales, análisis de estabilidad y simulación estocástica, y terminaremos preguntándonos cómo reconstruir el circuito a partir de datos de expresión (sección 16.2). Ambas miradas comparten una idea que Barabási y Oltvai (2004) resumieron bien: entender la organización funcional de la célula exige pasar de catálogos de componentes a mapas de sus interacciones.

16.1 Redes de interacción proteína-proteína

 Colab · 16.1 · Redes proteína-proteína

 repositorio del curso

El mapa del metro

Un mapa del metro no respeta distancias ni la forma de las calles: solo dice qué estaciones están conectadas. Y sin embargo, con esa información mínima podemos responder preguntas muy útiles. ¿Cuántos transbordos hay entre

dos estaciones? ¿Qué estación, si se cierra, desconecta media ciudad? ¿Qué líneas forman barrios bien comunicados entre sí pero mal comunicados con el resto? Un mapa de interacciones proteicas funciona igual: olvida la química, la estructura tridimensional y la cinética, y guarda solo quién toca a quién. A cambio, nos permite preguntar qué proteínas son “estaciones de transbordo”, cuán lejos está una proteína de otra o qué grupos de proteínas trabajan juntos.

16.1.1 El vocabulario de los grafos

Definición 16.1. (Grafo y matriz de adyacencia)

Un grafo $G = (V, E)$ es un conjunto de $n = |V|$ nodos (vértices) y $m = |E|$ aristas (enlaces), cada una un par de nodos. Si los pares no tienen orden, el grafo es *no dirigido* (interacciones físicas); si lo tienen, es *dirigido* (regulación: el factor de transcripción i actúa sobre el gen j). El grafo se codifica en la matriz de adyacencia $A \in \{0, 1\}^{n \times n}$:

$$A_{ij} = \begin{cases} 1 & \text{si hay una arista entre } i \text{ y } j, \\ 0 & \text{en otro caso.} \end{cases} \quad (16.1)$$

En un grafo no dirigido A es simétrica; en uno ponderado, A_{ij} puede ser un peso real (por ejemplo, una puntuación de confianza).

DONDE

V, E	Conjunto de nodos (proteínas) y de aristas (interacciones).
n, m	Número de nodos y de aristas.
A_{ij}	Entrada de la matriz de adyacencia: 1 si i y j interactúan.

La cantidad más sencilla que podemos asociar a un nodo es su *grado*, el número de vecinos:

$$k_i = \sum_{j=1}^n A_{ij}, \quad \sum_{i=1}^n k_i = 2m, \quad \langle k \rangle = \frac{2m}{n}. \quad (16.2)$$

DONDE

k_i	Grado del nodo i (número de interacciones de la proteína).
$\langle k \rangle$	Grado medio de la red.

La segunda igualdad (el “lema del apretón de manos”) se debe a que cada arista aporta un vecino a cada uno de sus dos extremos. La fracción de nodos con grado k , $P(k)$, es la *distribución de grado*, y será protagonista más adelante.

La matriz de adyacencia no es solo una tabla: sus potencias cuentan caminos. El elemento $(A^2)_{ij} = \sum_l A_{il}A_{lj}$ suma 1 por cada nodo l vecino a la vez de i y de j , es decir, cuenta los recorridos de longitud 2 entre ellos. Por inducción,

$$(A^\ell)_{ij} = \text{número de recorridos de longitud } \ell \text{ entre } i \text{ y } j. \quad (16.3)$$

En particular, $(A^3)_{ii}$ cuenta los recorridos que salen de i y vuelven a él en tres pasos; cada triángulo que contiene a i se puede recorrer en dos sentidos, así que el número de triángulos de i es $t_i = (A^3)_{ii}/2$. Esto nos lleva a una medida de lo “apiñado” que está el vecindario de un nodo.

Definición 16.2. (Coeficiente de agrupamiento)

El coeficiente de agrupamiento local de un nodo con $k_i \geq 2$ es la fracción de pares de sus vecinos que también son vecinos entre sí:

$$C_i = \frac{t_i}{\binom{k_i}{2}} = \frac{2t_i}{k_i(k_i - 1)}, \quad C = \frac{1}{n} \sum_i C_i. \quad (16.4)$$

DONDE

t_i	Número de triángulos que contienen al nodo i , igual a $(A^3)_{ii}/2$.
$\binom{k_i}{2}$	Número de pares posibles entre los k_i vecinos de i .
C	Coeficiente de agrupamiento medio de la red (con $C_i = 0$ si $k_i < 2$).

La *distancia* d_{ij} entre dos nodos es la longitud del camino más corto que los une; se calcula por búsqueda en anchura en tiempo $O(n + m)$ desde cada origen. Su promedio sobre todos los pares conectados, L , y su máximo, el *diámetro*, resumen cuán “compacta” es la red. Finalmente, la *intermediación* (*betweenness*) de un nodo mide cuántos caminos mínimos pasan por él:

$$b_v = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}, \quad (16.5)$$

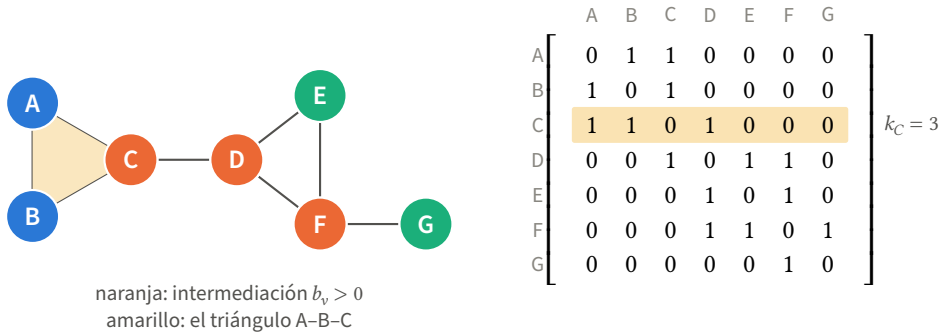


Figura 16.1. Un grafo de siete proteínas y su matriz de adyacencia (ecuación (16.1)). La fila resaltada suma el grado de C, $k_C = 3$. Los nodos A, B y E tienen $C_i = 1$ (sus dos vecinos están conectados entre sí), mientras que C, D y F tienen $C_i = 1/3$. Aunque C, D y F tienen el mismo grado, D tiene la mayor intermediación (9 caminos mínimos entre pares la atraviesan), porque es el único puente entre los dos grupos. Cortar la arista C-D da la mejor partición en comunidades (ejemplo 16.2).

DONDE

σ_{st}	Número de caminos mínimos entre s y t .
$\sigma_{st}(v)$	Cuántos de ellos pasan por v .
b_v	Intermediación: nodos “puente” por los que fluye mucha comunicación.

Un nodo puede tener grado bajo y aun así intermediación altísima, si es el único enlace entre dos regiones de la red. El grado mide popularidad local; la intermediación, importancia estratégica global.

Ejemplo 16.1. (Medir a mano un grafo pequeño)

En el grafo de la figura 16.1 hay $n = 7$ nodos y $m = 8$ aristas. Los grados son $(2, 2, 3, 3, 2, 3, 1)$ para A,...,G; suman $16 = 2m$, y $\langle k \rangle = 16/7 \approx 2,29$.

- **Caminos.** $(A^2)_{A,D} = 1$: hay un único recorrido de longitud 2 de A a D (por C). $(A^3)_{A,E} = 1$: el recorrido A-C-D-E.
- **Agrupamiento.** C tiene vecinos A, B y D; de los tres pares posibles solo A-B está unido, así que $C_C = 1/3$. Promediando, $C = (1 + 1 + \frac{1}{3} + \frac{1}{3} + 1 + \frac{1}{3} + 0)/7 = 4/7 \approx 0,571$.

- **Distancias.** La distancia media entre los 21 pares es $L \approx 2,05$ y el diámetro es 4 (de A o B a G).
- **Intermediación.** $b_C = 8$, $b_D = 9$, $b_F = 5$, y cero para los demás. D gana porque todo camino entre $\{A, B, C\}$ y $\{E, F, G\}$ la cruza.

Todas estas cifras se reproducen con `figuras/cap16/generar.py`.

16.1.2 Cómo se mide un interactoma

Los grafos de interacción no se observan directamente: se *construyen* a partir de experimentos, y cada experimento deja su huella en la red resultante. Dos familias de métodos dominaron la primera década de la interactómica.

Doble híbrido en levadura (Y2H). Un factor de transcripción como Gal4 tiene dos dominios separables: uno que se une al ADN (DBD) y otro que activa la transcripción (AD). Por separado no hacen nada; si se acercan, reconstituyen un activador funcional. El método fusiona una proteína “cebo” al DBD y otra “presa” al AD: si cebo y presa interactúan, el activador se reconstituye en el promotor y se expresa un gen reportero que permite a la levadura crecer en un medio selectivo. Uetz et al. (2000) aplicaron el método a escala de genoma con dos estrategias (una matriz ordenada de unos 6000 transformantes y un cribado de bibliotecas mezcladas) y detectaron 957 interacciones putativas entre 1004 proteínas de *Saccharomyces cerevisiae*. Casi al mismo tiempo, Ito et al. (2001) completaron un cribado exhaustivo que identificó 4549 interacciones entre 3278 proteínas y notaron algo inquietante: sus datos *no se solapaban en gran medida* con los de Uetz y colaboradores.

Purificación por afinidad y espectrometría de masas (AP-MS). En lugar de pares, AP-MS mide complejos. Se marca una proteína cebo con una etiqueta de afinidad, se purifica en condiciones suaves junto con todo lo que arrastra, y los acompañantes se identifican por espectrometría de masas. Gavin et al. (2002) usaron purificación en tándem (TAP) con 1739 genes de levadura, purificaron 589 ensamblajes proteicos y definieron 232 complejos multiproteicos distintos. El resultado no es una lista de pares sino de conjuntos, y convertirlos en aristas exige una decisión de modelado (figura 16.2): en el modelo *estrella* (*spoke*) se une el cebo a cada presa; en el modelo *matriz* (*matrix*) se unen todos con todos. Con un cebo y cinco presas, el primero produce 5 aristas y el segundo $\binom{6}{2} = 15$, la mayoría no verificadas como contactos directos.

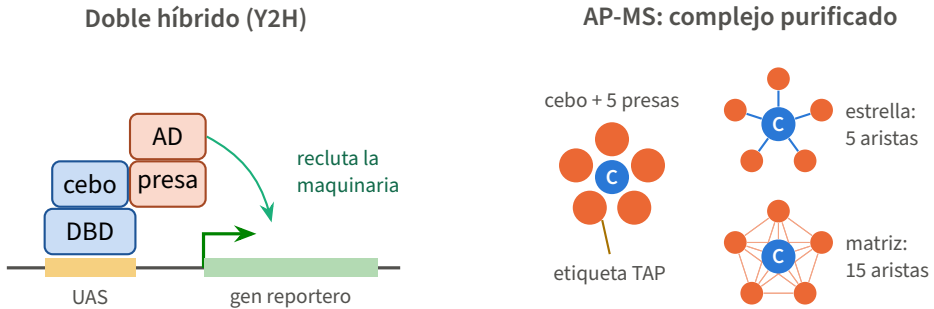


Figura 16.2. Los dos grandes métodos experimentales de la interactómica. **Izquierda:** en el doble híbrido, el cebo fusionado al dominio de unión al ADN (DBD) y la presa fusionada al dominio de activación (AD) reconstituyen un activador solo si interactúan; la lectura es un par binario. **Derecha:** AP-MS purifica un complejo completo a partir de un cebo etiquetado. Traducirlo a aristas exige elegir un modelo: *estrella* (subestima contactos entre presas) o *matriz* (sobreestima: la mayoría de los 15 pares no se tocan físicamente). Esa elección explica en parte por qué los complejos grandes aparecen en las redes como densas “bolas” de aristas.

16.1.3 Falsos positivos, falsos negativos e integración de evidencias

El escaso solapamiento entre los primeros cribados obligó a preguntarse cuánto había de señal y cuánto de ruido. von Mering et al. (2002) compararon sistemáticamente los grandes conjuntos de datos de levadura entre sí y con un conjunto de referencia de interacciones ya descritas, para medir su exactitud y sus sesgos. La conclusión general, que la interactómica ha confirmado desde entonces, es que cada método tiene su propio perfil de errores: el doble híbrido detecta contactos transitorios pero produce falsos positivos (proteínas “pegajosas”, autoactivadoras o que nunca coinciden en la célula) y pierde proteínas de membrana; AP-MS captura complejos estables pero confunde asociación indirecta con contacto directo y favorece a las proteínas abundantes. Las interacciones respaldadas por varios métodos independientes son mucho más fiables que las que aparecen en uno solo.

Esa observación sugiere cómo integrar. Si cada fuente de evidencia c asigna a un par una probabilidad s_c de que la asociación sea real, y las fuentes fallan de forma independiente, la probabilidad de que *todas* se equivoquen es el producto de sus errores:

$$S = 1 - \prod_c (1 - s_c). \quad (16.6)$$

DONDE

s_c	Confianza de la fuente o “canal” c (experimentos, coexpresión, texto, contexto genómico...).
S	Confianza combinada, bajo el supuesto de independencia entre canales.

Con tres canales de confianza moderada, $s = (0,6; 0,5; 0,7)$, la ecuación (16.6) da $S = 1 - 0,4 \times 0,5 \times 0,3 = 0,94$. Esta es la lógica de fondo de **STRING**, que integra interacciones físicas y asociaciones funcionales procedentes de experimentos, bases de datos curadas, coexpresión, contexto genómico conservado y minería automática de textos, las puntúa y las transfiere a organismos poco estudiados por ortología; su versión 12.0 permite incluso construir la red de un genoma nuevo enviando su proteoma (Szklarczyk et al., 2023). En el otro extremo del espectro, **BioGRID** no predice nada: cura manualmente interacciones proteicas y genéticas de la literatura primaria, alrededor de 1,93 millones en 2021, además de modificaciones postraduccionales, interacciones con fármacos y cribados CRISPR (Oughtred et al., 2021).

⚠ Una asociación no es un contacto

Una arista de **STRING** con puntuación alta puede significar que dos proteínas se tocan, que forman parte del mismo complejo sin tocarse, o simplemente que sus genes se coexpresan o se citan juntos en los artículos. Para preguntas de estructura física, filtre por canal (**STRING** ofrece una red “física”) o use datos curados como **BioGRID**. Y recuerde que la ausencia de arista casi nunca significa ausencia de interacción: los interactomas siguen incompletos y sesgados hacia las proteínas más estudiadas.

16.1.4 Tres modelos de red y lo que predicen

Para saber si una propiedad de una red real es notable, necesitamos compararla con lo que produciría el azar. Tres modelos marcan los hitos de esa comparación.

Erdős-Rényi (ER). Se unen n nodos colocando cada una de las $\binom{n}{2}$ aristas posibles con probabilidad p , de forma independiente. El grado de un nodo es la suma de $n - 1$

ensayos de Bernoulli, así que sigue una binomial que, para n grande y $\langle k \rangle = p(n-1)$ fijo, converge a una Poisson:

$$P(k) = \binom{n-1}{k} p^k (1-p)^{n-1-k} \xrightarrow{n \rightarrow \infty} e^{-\langle k \rangle} \frac{\langle k \rangle^k}{k!}, \quad C = p, \quad L \approx \frac{\ln n}{\ln \langle k \rangle}. \quad (16.7)$$

DONDE

p	Probabilidad de cada arista.
$P(k)$	Distribución de grado: concentrada alrededor de $\langle k \rangle$, con colas que decaen más rápido que exponencialmente.

El agrupamiento vale p porque dos vecinos de un nodo están unidos, como cualquier otro par, con probabilidad p . La fórmula de L se obtiene contando: desde un nodo se alcanzan unos $\langle k \rangle$ nodos a distancia 1, $\langle k \rangle^2$ a distancia 2 y $\langle k \rangle^\ell$ a distancia ℓ ; toda la red queda cubierta cuando $\langle k \rangle^L \approx n$. La distancia crece solo con el *logaritmo* del tamaño.

Watts-Strogatz (WS). Las redes reales combinan dos rasgos que ER no reproduce a la vez: distancias cortas y vecindarios muy agrupados. Watts y Strogatz (1998) mostraron que ambos coexisten en un modelo sencillo: se parte de un anillo en el que cada nodo se une a sus k vecinos más próximos (muy agrupado, pero con $L \approx n/2k$) y se reconecta cada arista al azar con probabilidad p . Unos pocos “atajos” bastan para desplomar L , mientras que C apenas cambia porque destruir triángulos requiere reconectar muchas aristas. Esa franja intermedia es el régimen de *mundo pequeño* (figura 16.3).

Barabási-Albert (BA). Ni ER ni WS producen nodos con muchísimas más conexiones que la media. Barabási y Albert (1999) observaron distribuciones de grado de cola larga en varias redes reales y propusieron dos ingredientes para explicarlas: *crecimiento* (la red gana nodos con el tiempo) y *enlace preferencial* (los nodos nuevos prefieren unirse a los ya muy conectados, “el rico se hace más rico”).

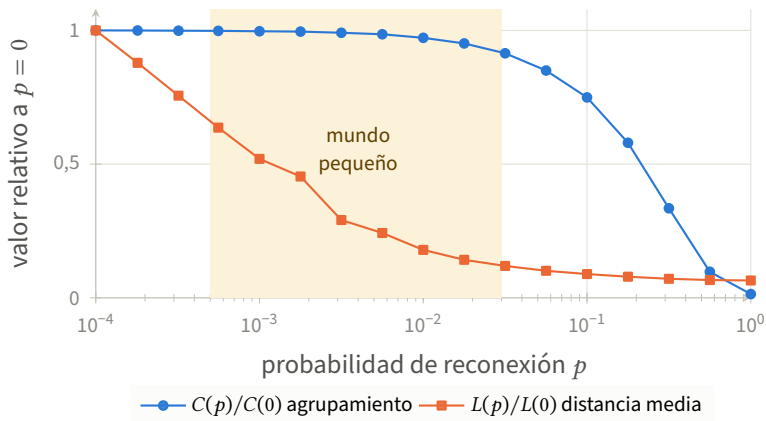


Figura 16.3. Modelo de Watts-Strogatz con $n = 1000$ nodos y $k = 10$ vecinos (promedio de cuatro redes por punto). Con $p = 0,01$ (uno de cada cien enlaces reconectado) la distancia media ya cayó al 18 % de su valor inicial, mientras que el agrupamiento conserva el 97 %. La banda amarilla marca el régimen de mundo pequeño: tan agrupado como una red regular y casi tan compacto como una aleatoria. Calculado con NetworkX en `figuras/cap16/generar.py`.

Teorema 16.1. (Distribución de grado del modelo BA)

Si en cada paso de tiempo se añade un nodo con m aristas y cada arista se une al nodo i con probabilidad $\Pi(k_i) = k_i / \sum_j k_j$, entonces, en la aproximación de continuo y para $t \rightarrow \infty$,

$$k_i(t) = m \left(\frac{t}{t_i} \right)^{1/2}, \quad P(k) = \frac{2m^2}{k^3} \quad (k \geq m). \tag{16.8}$$

Es decir, una ley de potencias $P(k) \propto k^{-\gamma}$ con $\gamma = 3$, independiente de m .

DONDE

- t_i Instante en que el nodo i se incorporó a la red.
- $\Pi(k_i)$ Probabilidad de que una arista nueva elija al nodo i .
- γ Exponente de la ley de potencias.

La derivación es breve y muy instructiva. En el instante t hay unos t nodos y mt aristas, así que $\sum_j k_j = 2mt$. Cada nodo nuevo reparte m aristas, y el nodo i recibe en promedio $m \Pi(k_i)$ de ellas:

$$\frac{dk_i}{dt} = m \frac{k_i}{2mt} = \frac{k_i}{2t} \implies \ln k_i = \frac{1}{2} \ln t + \text{cte} \implies k_i(t) = m \left(\frac{t}{t_i} \right)^{1/2},$$

usando la condición inicial $k_i(t_i) = m$. Los nodos más viejos son los más conectados. Como los tiempos de llegada t_i están repartidos uniformemente en $[0, t]$, la probabilidad de que un nodo tenga grado menor que k es

$$\mathbb{P}(k_i(t) < k) = \mathbb{P}\left(t_i > \frac{m^2 t}{k^2}\right) = 1 - \frac{m^2}{k^2},$$

y al derivar respecto a k obtenemos $P(k) = 2m^2/k^3$. Una ley de potencias no tiene una escala característica (de ahí el nombre de *red libre de escala*): su varianza diverge cuando $\gamma \leq 3$, y siempre hay una probabilidad no despreciable de encontrar nodos con grado cientos de veces mayor que la media, los llamados *hubs*.

16.1.5 La red de la levadura frente a los modelos

¿Qué aspecto tiene una red real? Tomemos la red física de levadura de STRING v12.0 con puntuación combinada de al menos 700 (“confianza alta”). Tiene $n = 3384$ proteínas y $m = 43\,030$ interacciones, con $\langle k \rangle = 25,4$ y una componente gigante de 2780 nodos. Su coeficiente de agrupamiento es $C = 0,58$, mientras que una red ER con los mismos n y m tendría $C = p = 0,0075$: casi ochenta veces menos. La distancia media en la componente gigante es $L \approx 5,2$ pasos, frente a 2,8 en la red ER equivalente; ambas son minúsculas comparadas con el número de nodos. La red de levadura es, pues, un mundo pequeño muy agrupado.

La figura 16.4 muestra su distribución de grado. A diferencia de la red ER, la levadura tiene una cola larga: proteínas con 200 o más interacciones. Pero la cola no es la recta limpia del modelo BA, y los nodos de mayor grado cuentan una historia menos romántica que “el rico se hace más rico”: 49 de las 50 proteínas más conectadas son proteínas ribosómicas (RPS13, RPS7A, RPS5...). El ribosoma es un complejo enorme purificado muchas veces, y el canal físico de STRING lo representa como un grupo casi completamente conectado. Una estimación por máxima verosimilitud del exponente en la cola ($k \geq 26$), $\hat{\gamma} = 1 + N \left[\sum_i \ln(k_i / (k_{\min} - \frac{1}{2})) \right]^{-1}$, da 1,99 para la levadura y 2,83 para la red BA (cuyo valor teórico es 3). Un exponente tan cercano a 2, el valor por debajo del cual ni siquiera el grado medio estaría definido en una

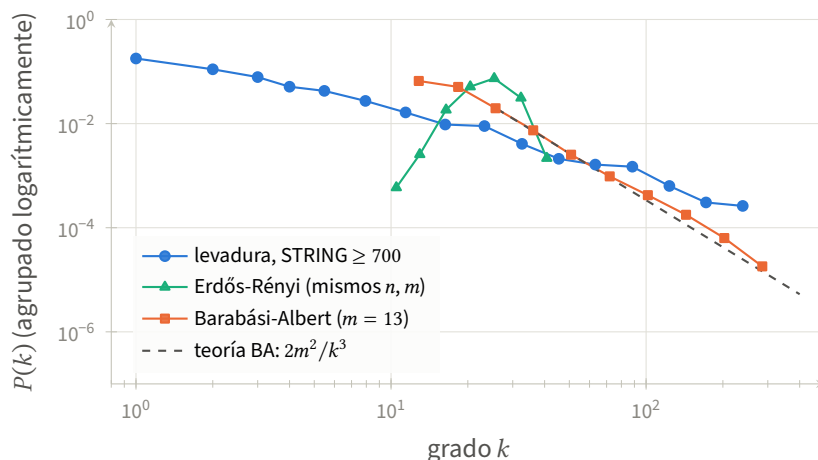


Figura 16.4. Distribución de grado de la red física de levadura (STRING v12.0, confianza ≥ 700) comparada con dos modelos del mismo tamaño. La red ER (verde) concentra todos sus nodos alrededor de $\langle k \rangle \approx 25$, sin cola. El modelo BA (naranja) sigue la recta de pendiente -3 predicha por la ecuación (16.8). La levadura muestra una cola ancha y extendida, pero no una recta limpia: una meseta hasta $k \approx 300$ producida por las proteínas ribosómicas, que forman una “bola” casi completa de aristas. Los puntos resultan de agrupar los grados en intervalos de anchura logarítmica creciente y dividir por la anchura de cada uno.

red infinita, delata que la cola está dominada por unos pocos complejos grandes y no por un proceso de crecimiento como el del modelo BA.

¿Son raras las redes libres de escala?

Durante una década, la “ley de potencias” fue casi un dogma de la ciencia de redes. Broido y Clauset (2019) la sometieron a una prueba severa: aplicaron herramientas estadísticas modernas a casi 1000 redes sociales, biológicas, tecnológicas, de transporte y de información, y encontraron que las redes *fuertemente* libres de escala son empíricamente raras, que para la mayoría una distribución log-normal se ajusta igual de bien o mejor que una ley de potencias, y que solo un puñado de redes tecnológicas y biológicas parecen fuertemente libres de escala. El debate que siguió fue intenso. La lección práctica es metodológica: una recta aproximada en un gráfico log-log *no* demuestra una ley de potencias; hay que ajustar por máxima verosimilitud, elegir $k_{\min}$ con criterio y comparar con distribuciones alternativas.

16.1.6 Centralidad, esencialidad y medicina de redes

Si la topología importa, las proteínas centrales deberían ser especiales. Jeong et al. (2001) analizaron la red de interacciones de levadura y encontraron que la probabilidad de que la eliminación de una proteína sea letal para la célula crece con su número de interacciones: las proteínas muy conectadas tienden a ser esenciales. Este principio de *centralidad-letalidad* ha sido matizado después (parte de la correlación se debe a que los complejos esenciales son grandes, justamente el efecto que vimos con el ribosoma), pero la idea de que la posición en la red informa sobre la función se mantiene.

La *medicina de redes* lleva esta idea a la enfermedad humana. Barabási et al. (2011) argumentan que, dada la interdependencia funcional de los componentes celulares, una enfermedad rara vez es consecuencia de la anomalía de un único gen: refleja la perturbación de una red que conecta tejidos y órganos. Los genes asociados a una misma enfermedad tienden a agruparse en un *módulo de enfermedad*, lo que permite priorizar genes candidatos de un estudio de asociación por su proximidad en la red a genes ya conocidos, o proponer dianas terapéuticas y nuevos usos para fármacos existentes.

Grado: cuántos vecinos. Agrupamiento: cuán unidos están esos vecinos. Intermediación: cuánto tráfico pasa por el nodo. Tres números distintos para tres preguntas distintas.

16.1.7 Módulos y comunidades

Una célula no es una sopa homogénea de interacciones: tiene complejos, vías y orgánulos. En el lenguaje de redes, esto se traduce en *comunidades*: grupos de nodos densamente conectados entre sí y escasamente conectados con el resto. Girvan y Newman (2002) propusieron detectarlas eliminando progresivamente las aristas de mayor intermediación, que son las que hacen de puente entre grupos; al retirarlas, la red se fragmenta en sus comunidades naturales. Faltaba un criterio para decidir cuándo detenerse, y ese criterio es la modularidad.

Definición 16.3. (Modularidad)

Dada una partición de los nodos en comunidades c_i , su modularidad es la fracción de aristas dentro de las comunidades menos la fracción esperada si las aristas se colocaran al azar respetando los grados:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) = \sum_c \left[\frac{l_c}{m} - \left(\frac{d_c}{2m} \right)^2 \right]. \quad (16.9)$$

DONDE

$k_i k_j / 2m$	Número esperado de aristas entre i y j en el modelo nulo de configuración (mismos grados, aristas al azar).
$\delta(c_i, c_j)$	Vale 1 si i y j están en la misma comunidad.
l_c	Número de aristas dentro de la comunidad c .
d_c	Suma de los grados de los nodos de c .

El término $k_i k_j / 2m$ merece una explicación. Si cortamos cada arista en dos “medias aristas” y las volvemos a emparejar al azar, cada una de las k_i medias aristas de i se empareja con alguna de j con probabilidad $k_j / (2m - 1) \approx k_j / 2m$. Para pasar de la primera forma a la segunda, basta agrupar la suma por comunidades: $\sum_{i,j \in c} A_{ij} = 2l_c$ y $\sum_{i,j \in c} k_i k_j = d_c^2$. Newman (2006) mostró que Q se puede escribir con los autovectores de la *matriz de modularidad* $B_{ij} = A_{ij} - k_i k_j / 2m$, lo que da un algoritmo espectral eficaz; hoy el método más usado es el de Louvain (Blondel et al., 2008), que mueve nodos entre comunidades mientras Q aumente y luego agrega cada comunidad en un supernodo, repitiendo el proceso hasta converger.

Ejemplo 16.2. (Modularidad del grafo de juguete)

En la figura 16.1, con $m = 8$, consideremos la partición $\{A, B, C\} | \{D, E, F, G\}$. La primera comunidad tiene $l = 3$ aristas y grado total $d = 2 + 2 + 3 = 7$; la segunda, $l = 4$ y $d = 3 + 2 + 3 + 1 = 9$. Por la ecuación (16.9),

$$Q = \left[\frac{3}{8} - \left(\frac{7}{16} \right)^2 \right] + \left[\frac{4}{8} - \left(\frac{9}{16} \right)^2 \right] = 0,1836 + 0,1836 = 0,367.$$

Si en cambio movemos D al primer grupo, $\{A, B, C, D\} | \{E, F, G\}$, la modularidad baja a 0,219: la partición natural corta la única arista puente, C–D.

La figura 16.5 aplica todo esto a una red real: las proteínas a distancia ≤ 2 de la quinasa FUS3 en la red de levadura. El algoritmo de Louvain encuentra cinco comunidades con $Q = 0,57$ que coinciden con módulos biológicos reconocibles: el núcleo de la cascada MAP quinasa de la feromona (FUS3, STE7, KSS1, el factor de transcripción STE12 y sus inhibidores DIG1 y DIG2), la parte proximal de la vía (la subunidad $\beta\gamma$ de la proteína G, STE4–STE18, el andamio STE5, la quinasa STE20, la GTPasa CDC42), la vía de alta osmolaridad (HOG1, PBS2, SSK2, SSK22), y el control del ciclo celular (CDC28, las ciclinas CLN2 y CLB5, y FAR1, que detiene el ciclo en respuesta a la feromona). La quinasa FUS3 tiene la mayor intermediación porque conecta varios de estos módulos. Para saber si $Q = 0,57$ es alto, repetimos el análisis en 20 versiones aleatorizadas de la misma red que conservan el grado de cada nodo: dan $Q = 0,36 \pm 0,02$. La estructura modular es real, no un artefacto del algoritmo.

⚠ Comunidades a medida del observador

La maximización de la modularidad tiene limitaciones conocidas: las particiones con Q casi máximo pueden ser muy distintas entre sí, el algoritmo de Louvain depende del orden aleatorio en que visita los nodos (fije siempre la semilla) y existe un límite de resolución por debajo del cual las comunidades pequeñas se funden con sus vecinas. Además, una ego-red como la de la figura 16.5 está sesgada por construcción hacia su semilla. Trate las comunidades como hipótesis que hay que contrastar con anotaciones funcionales (Gene Ontology, complejos conocidos), no como descubrimientos.

16.1.8 Herramientas: NetworkX y Cytoscape

Para el análisis programático, la biblioteca de referencia en Python es NetworkX (Hagberg et al., 2008), que ofrece estructuras de grafo flexibles y cientos de algoritmos (camino, centralidades, comunidades, modelos aleatorios). Para la exploración visual e interactiva, y para integrar la red con datos de expresión o anotaciones, el estándar es Cytoscape (Shannon et al., 2003), un entorno de código abierto ampliable con extensiones, entre ellas una que consulta STRING directamente. El siguiente fragmento reproduce los números de esta sección.

```
red_levadura.py
```

```
import networkx as nx
import pandas as pd
```

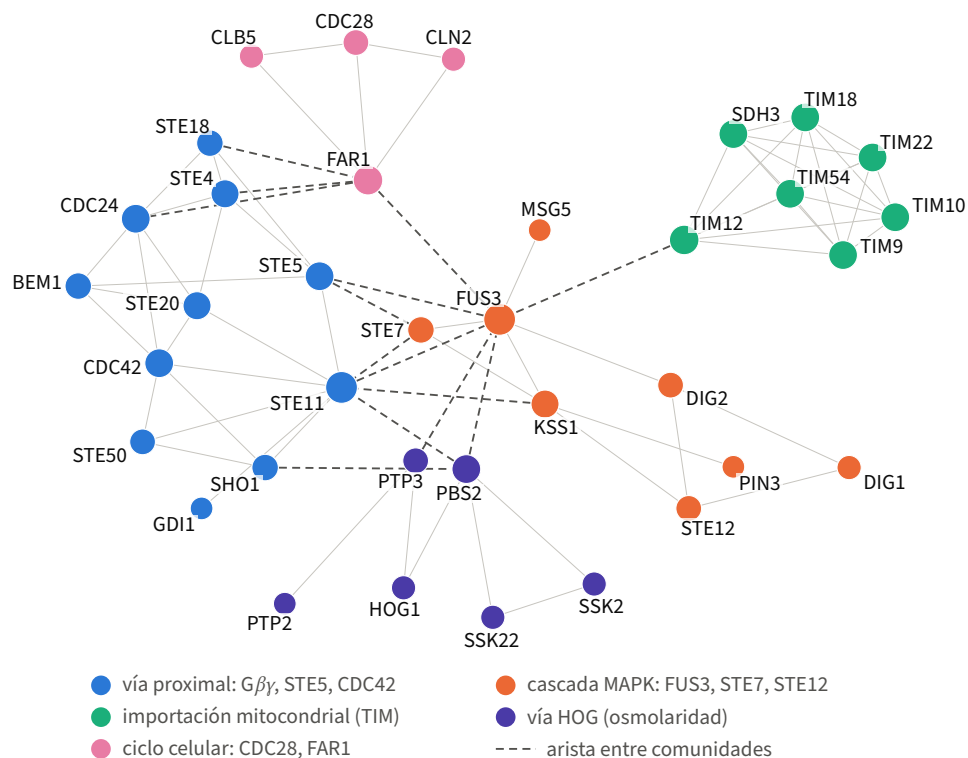


Figura 16.5. Subred de la vía de la feromona en levadura: todas las proteínas a distancia ≤ 2 de FUS3 en la red física de STRING v12.0 con confianza ≥ 700 (36 proteínas, 78 interacciones). Las coordenadas se calcularon con el algoritmo de Kamada-Kawai y los colores son las cinco comunidades de Louvain ($Q = 0,57$; en redes aleatorizadas con los mismos grados, $Q = 0,36 \pm 0,02$). El tamaño de cada nodo crece con su grado; las líneas discontinuas son aristas entre comunidades. FUS3 es el nodo de mayor intermediación. Observe el módulo verde de proteínas mitocondriales TIM: llega a la figura por una única arista, FUS3–TIM12, un recordatorio de que una red extraída por vecindad hereda cualquier interacción dudosa de su nodo semilla. Generado con `figuras/cap16/generar.py`.

```

aristas = pd.read_csv("levadura_string700.tsv.gz", sep="\t",
                      comment="#")
G = nx.from_pandas_edgelist(aristas, "prot1", "prot2", "score")
print(G.number_of_nodes(), G.number_of_edges()) # 3384 43030
print(nx.average_clustering(G))                 # 0.581

```

```
E = nx.ego_graph(G, "FUS3", radius=2)           # vecindad de FUS3
com = nx.community.louvain_communities(E, seed=1)
print(len(com), nx.community.modularity(E, com))
bc = nx.betweenness centrality(E)
print(max(bc, key=bc.get))                       # FUS3
```

16.2 Redes de regulación génica y modelos dinámicos



Colab · 16.2 · Regulación génica y modelos ODE



repositorio del curso

El termostato y la bañera

Piense en una bañera con el grifo abierto y el desagüe destapado. El nivel del agua sube hasta que lo que entra por el grifo iguala lo que sale por el desagüe, y ahí se estabiliza. Si queremos llenarla más rápido sin cambiar el nivel final, hay un truco: abrir el grifo a tope al principio y cerrarlo a medida que el agua se acerca a la marca, como hace un termostato. La concentración de una proteína en una célula se comporta exactamente como esa bañera: se sintetiza (el grifo), se degrada y se diluye al dividirse la célula (el desagüe), y un gen que reprime su propio promotor es un termostato molecular. Con dos o tres “bañeras” que se abren y cierran los grifos unas a otras, la célula construye memorias, relojes y filtros.

Las redes de la sección anterior eran mapas estáticos. Las redes de regulación génica, en cambio, son *dirigidas*: sus nodos son genes, y una arista $X \rightarrow Y$ significa que el producto del gen X , un factor de transcripción, modifica la tasa de transcripción de Y (activándola, $X \rightarrow Y$, o reprimiéndola, $X \dashv Y$). Para saber qué hace un circuito así no basta con su topología: necesitamos ecuaciones que describan cómo cambian las concentraciones en el tiempo. El libro de Alon (2019) desarrolla este programa con detalle y es la referencia natural para lo que sigue.

16.2.1 De la unión al promotor a la función de Hill

La cinética enzimática ofrece el primer modelo de respuesta saturable. En la reacción $E + S \rightleftharpoons ES \rightarrow E + P$, si el complejo ES alcanza rápidamente un estado cuasiestacionario, la velocidad de formación de producto es

$$v = \frac{V_{\text{máx}} [S]}{K_M + [S]}, \quad (16.10)$$

DONDE

v	Velocidad de la reacción.
$V_{\text{máx}}$	Velocidad máxima, cuando la enzima está saturada.
K_M	Constante de Michaelis: concentración de sustrato que da la mitad de $V_{\text{máx}}$.

la ley que Michaelis y Menten publicaron en 1913 y cuya traducción comentada al inglés publicaron Johnson y Goody (2011). La misma forma aparece cuando un factor de transcripción X se une a un sitio del promotor D : en equilibrio, $[D][X]/[DX] = K$, y la fracción de promotores ocupados es $[X]/(K + [X])$.

Muchos factores de transcripción actúan como dímeros o tetrámeros, o se unen a varios sitios de forma cooperativa. Si el promotor solo se activa cuando se unen n moléculas a la vez ($D + nX \rightleftharpoons DX_n$, con $K^n = [D][X]^n/[DX_n]$), la fracción ocupada es

$$\theta = \frac{[DX_n]}{[D] + [DX_n]} = \frac{[X]^n/K^n}{1 + [X]^n/K^n}.$$

Definición 16.4. (Funciones de Hill)

La tasa de producción de un gen regulado por X se modela como

$$f_{\text{act}}(X) = \beta \frac{X^n}{K^n + X^n}, \quad f_{\text{rep}}(X) = \beta \frac{1}{1 + (X/K)^n}. \quad (16.11)$$

DONDE

β	Tasa máxima de producción (fuerza del promotor).
K	Coefficiente de activación o represión: concentración de X que da la mitad del efecto.
n	Coefficiente de Hill: mide la cooperatividad y, con ella, lo abrupta que es la respuesta.

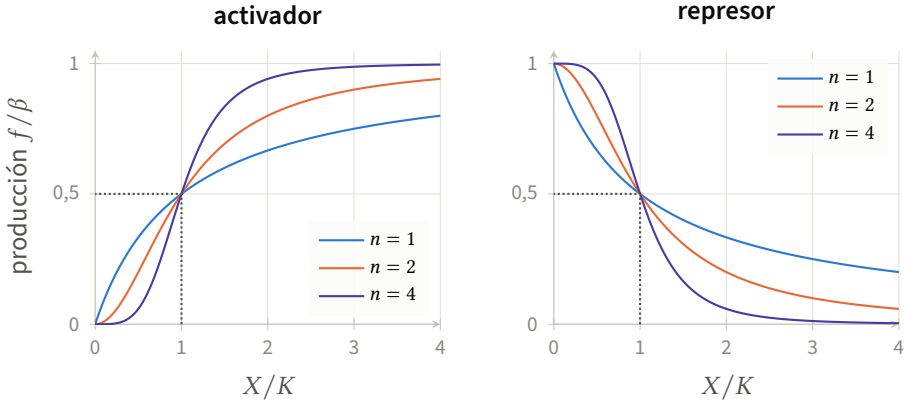


Figura 16.6. Funciones de Hill para un activador (izquierda) y un represor (derecha), ecuación (16.11). Todas pasan por la mitad del efecto en $X = K$ (líneas punteadas), pero la cooperatividad cambia la forma: con $n = 1$ la respuesta es gradual y hace falta multiplicar X por 81 para pasar del 10 % al 90 %; con $n = 4$ basta un factor 3 (ecuación (16.12)). La pendiente en $X = K$, en escala logarítmica, es $n/4$.

¿Cuán abrupta? Llamemos $x = X/K$. Para pasar del 10 % al 90 % de activación hay que resolver $x^n/(1 + x^n) = 0,1$ y $0,9$, es decir, $x_{10} = 9^{-1/n}$ y $x_{90} = 9^{1/n}$. Su cociente,

$$\frac{X_{90}}{X_{10}} = 81^{1/n}, \quad (16.12)$$

vale 81 para $n = 1$, 9 para $n = 2$ y solo 3 para $n = 4$. Con n grande la función de Hill se aproxima a un escalón, y el gen se comporta como un interruptor lógico: encendido por encima de K , apagado por debajo (figura 16.6). Esta es la base de las aproximaciones “lógicas” que usaremos para razonar sobre circuitos.

16.2.2 El modelo básico: producción y eliminación

Consideremos un gen Y cuyo activador X pasa de golpe, en $t = 0$, a un nivel que satura su promotor. A partir de ese momento Y se produce a tasa constante β y se elimina en proporción a su concentración:

$$\frac{dY}{dt} = \beta - \alpha Y, \quad \alpha = \alpha_{\text{deg}} + \alpha_{\text{dil}}. \quad (16.13)$$

DONDE

$Y(t)$	Concentración de la proteína Y .
α_{deg}	Tasa de degradación activa (proteólisis).
α_{dil}	Tasa de dilución por crecimiento: $\ln 2/\tau$ si la célula se divide cada τ .

Teorema 16.2. (Respuesta de la regulación simple)

La solución de la ecuación (16.13) con $Y(0) = 0$ es

$$Y(t) = Y_{\text{est}}(1 - e^{-\alpha t}), \quad Y_{\text{est}} = \frac{\beta}{\alpha}, \quad t_{1/2} = \frac{\ln 2}{\alpha}. \quad (16.14)$$

El tiempo de respuesta $t_{1/2}$, en el que Y alcanza la mitad de su valor final, depende solo de la tasa de eliminación, no de la de producción. Al apagar el gen, $Y(t) = Y_{\text{est}}e^{-\alpha t}$ decae con el mismo $t_{1/2}$.

La demostración es directa: con $u = Y_{\text{est}} - Y$, la ecuación queda $du/dt = -\alpha u$, cuya solución es $u(t) = u(0)e^{-\alpha t} = Y_{\text{est}}e^{-\alpha t}$. Imponer $Y(t_{1/2}) = Y_{\text{est}}/2$ da $e^{-\alpha t_{1/2}} = 1/2$.

La consecuencia es contraintuitiva y profunda. Una célula que quiere producir *más* proteína puede usar un promotor más fuerte; pero si quiere producirla *más rápido*, un promotor fuerte no sirve: $t_{1/2}$ no depende de β . Para las proteínas estables de bacterias, que solo se eliminan por dilución, $\alpha = \ln 2/\tau$ y el tiempo de respuesta es exactamente un tiempo de generación.

Ejemplo 16.3. (Cuánto tarda en responder un gen)

En *Escherichia coli* creciendo con $\tau = 30$ min, una proteína estable tiene $\alpha = \ln 2/30 = 0,0231 \text{ min}^{-1}$ y $t_{1/2} = 30$ min. Si la proteína además se degrada activamente con una vida media de 5 min, $\alpha = 0,0231 + \ln 2/5 = 0,162 \text{ min}^{-1}$ y $t_{1/2} = 4,3$ min, siete veces más rápido; con una vida media de 60 min, $t_{1/2} = 20$ min. La degradación acelera la respuesta, pero a un costo: para mantener el mismo nivel final hay que producir la proteína (y gastar energía) a una tasa proporcionalmente mayor.

16.2.3 Autorregulación negativa: acelerar sin desperdiciar

Existe una forma más económica de acelerar. Supongamos que X reprime su propio promotor:

$$\frac{dX}{dt} = \frac{\beta}{1 + (X/K)^n} - \alpha X. \quad (16.15)$$

Al principio, con $X \ll K$, el promotor funciona a pleno rendimiento y X crece a la tasa máxima β . Cuando X se acerca a K , la represión frena la producción, y el sistema se estabiliza. Si elegimos un promotor muy fuerte (β grande) y una represión que lo compense, obtenemos el mismo nivel final que con regulación simple pero con un arranque mucho más rápido: es el termostato de la bañera. Además, en el límite de represión fuerte ($X_{\text{est}} \gg K$) el estado estacionario satisface $\beta(K/X)^n \approx \alpha X$, es decir,

$$X_{\text{est}} \approx \left(\frac{\beta K^n}{\alpha} \right)^{1/(n+1)}, \quad (16.16)$$

DONDE

X_{est}	Nivel estacionario de la proteína autorreprimida.
n	Coefficiente de Hill de la autorrepresión.

de modo que el nivel depende de β solo a través de una raíz de orden $n+1$: si una fluctuación duplica la fuerza del promotor, con $n=2$ el nivel sube un $2^{1/3} - 1 = 26\%$, no un 100 %. La autorregulación negativa acelera la respuesta y amortigua el ruido en la producción.

Rosenfeld et al. (2002) midieron el efecto con circuitos sintéticos en *E. coli*: la unidad sin autorregulación tarda un ciclo celular en alcanzar la mitad de su nivel final, mientras que la autorregulada lo hace en aproximadamente una quinta parte de ciclo, en acuerdo con la solución analítica del modelo. También señalaron que este motivo aparece en más del 40 % de los factores de transcripción conocidos de *E. coli*.

16.2.4 Motivos de red

¿Hay patrones que se repiten en las redes de regulación? Milo et al. (2002) definieron los *motivos de red* como patrones de interconexión que aparecen significativamente más a menudo que en redes aleatorizadas, y los encontraron en redes de bioquímica, neurobiología, ecología e ingeniería. Para cada subgrafo de tres o cuatro nodos se cuenta su número de apariciones N_{real} y se compara con el de un

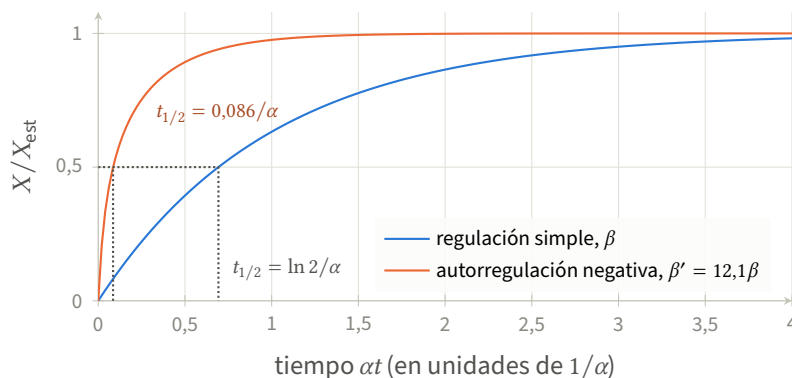


Figura 16.7. Respuesta a la activación en $t = 0$ de un gen con regulación simple (ecuación (16.14)) y de uno que reprime su propio promotor (ecuación (16.15)), con $K = 0,3X_{\text{est}}$ y $n = 2$). Ambos llegan al mismo nivel final, pero el autorregulado usa un promotor 12,1 veces más fuerte que queda frenado a medida que X se acerca a K . El resultado es un tiempo de respuesta 8,1 veces menor sin cambiar el estado estacionario ni la tasa de eliminación.

conjunto de redes aleatorizadas que conservan el grado de entrada y de salida de cada nodo:

$$Z = \frac{N_{\text{real}} - \langle N_{\text{azar}} \rangle}{\sigma_{\text{azar}}}. \quad (16.17)$$

DONDE

N_{real}	Apariciones del subgrafo en la red real.
$\langle N_{\text{azar}} \rangle, \sigma_{\text{azar}}$	Media y desviación estándar en las redes aleatorizadas.
Z	Puntuación de significación del motivo.

Shen-Orr et al. (2002) aplicaron la idea a la red de interacciones transcripcionales directas de *E. coli* y encontraron que buena parte de ella está compuesta por apariciones repetidas de tres motivos muy significativos: el *feed-forward loop* (FFL), el módulo de entrada única (*single-input module*, SIM) y los regulones densos superpuestos. Cada motivo tiene una función específica en la dinámica de la expresión. Alon (2007) revisa la teoría y los experimentos que las han puesto a prueba, desde bacterias hasta humanos (figura 16.8).

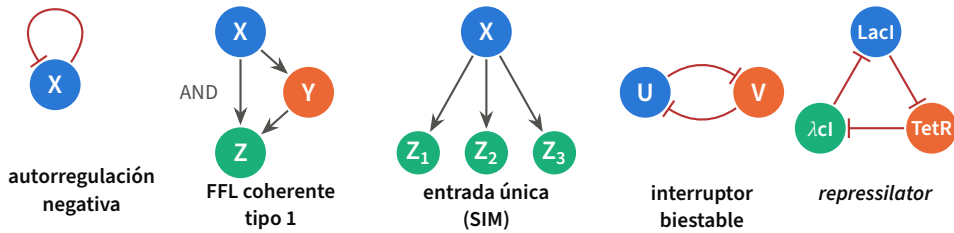


Figura 16.8. Los circuitos que estudiaremos. Las flechas indican activación y las barras rojas, represión. De izquierda a derecha: la autorregulación negativa acelera la respuesta (figura 16.7); el FFL coherente con compuerta AND filtra pulsos breves de la señal (figura 16.9); el SIM coordina la expresión de un grupo de genes; dos represores mutuos forman una memoria bistable (figura 16.10); tres represores en ciclo producen oscilaciones (figura 16.11). Los tres primeros son motivos naturales; los dos últimos, circuitos sintéticos construidos en *E. coli*.

16.2.5 El feed-forward loop coherente: un filtro de persistencia

En un FFL, un regulador X controla a otro regulador Y , y ambos controlan conjuntamente a un gen diana Z . Como cada una de las tres aristas puede ser activadora o represora, hay ocho tipos. Mangan y Alon (2003) analizaron teóricamente los ocho y mostraron que los cuatro *coherentes* (el signo del camino directo coincide con el del indirecto) actúan como *retardos sensibles al signo*, mientras que los cuatro *incoherentes* actúan como *aceleradores sensibles al signo*; también observaron que algunos tipos son mucho más frecuentes que otros en las bases de datos de regulación.

Veamos el más común, el coherente tipo 1 con compuerta AND. Con una señal S_x que activa a X (la tomamos como escalón $X^* \in \{0, 1\}$),

$$\frac{dY}{dt} = \beta_y f(X^*, K_{xy}) - \alpha_y Y, \quad \frac{dZ}{dt} = \beta_z f(X^*, K_{xz}) f(Y, K_{yz}) - \alpha_z Z, \quad (16.18)$$

DONDE

X^*	Forma activa de X (tras recibir su señal).
$f(\cdot, K)$	Función de Hill activadora normalizada, $f(u, K) = u^n / (K^n + u^n)$.
K_{xy}, K_{xz}, K_{yz}	Umbral de activación de cada arista.

El producto $f(X^*)f(Y)$ implementa la compuerta AND: Z solo se produce si X^* y Y están presentes. Cuando la señal aparece, X^* se activa de inmediato, pero Y

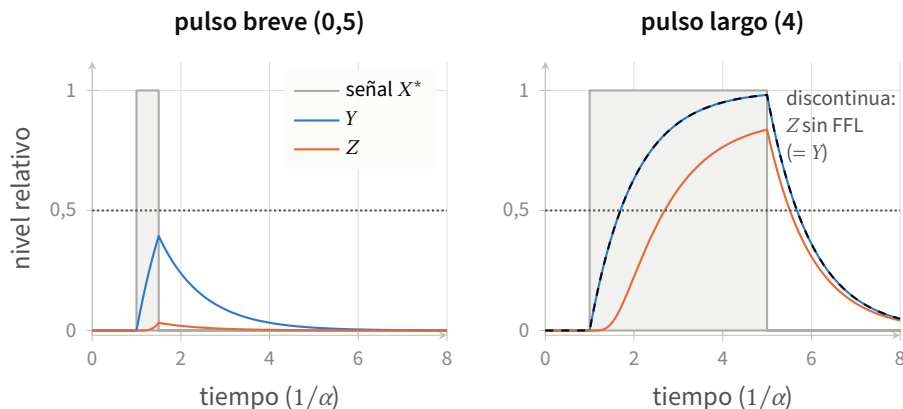


Figura 16.9. Simulación del FFL coherente tipo 1 con compuerta AND (ecuación (16.18), $n = 4$, $K_{yz} = 0,5$, $\alpha_y = \alpha_z = 1$). **Izquierda:** ante un pulso de duración 0,5, Y no llega al umbral K_{yz} (línea punteada) y Z apenas alcanza un 3 % de su máximo: el pulso se filtra. **Derecha:** ante un pulso de duración 4, Z se enciende con retraso (tarda 1,46 en llegar a la mitad de su máximo, frente a 0,69 con regulación simple, línea discontinua, que coincide con la curva de Y porque Y tiene justamente regulación simple) pero se apaga sin retraso (0,70) cuando la señal desaparece: un retardo sensible al signo.

tiene que acumularse hasta superar K_{yz} ; solo entonces empieza a producirse Z . Si la señal desaparece antes, Z nunca se enciende. Cuando la señal se apaga, en cambio, basta con que X^* desaparezca para que la compuerta AND se cierre: no hay retardo. El circuito ignora los pulsos breves y responde a los persistentes (figura 16.9), lo que es útil para no activar un programa costoso, como el de utilización de un azúcar, ante una fluctuación pasajera.

16.2.6 El interruptor biestable: nulclinas y estabilidad

Gardner et al. (2000) construyeron en *E. coli* un circuito sintético con dos promotores reprimibles dispuestos en inhibición mutua: el producto de cada gen reprime al otro. El resultado fue un *interruptor genético*: una red biestable que se puede conmutar entre sus dos estados con un pulso transitorio de un inductor químico o de temperatura, y que recuerda su estado cuando el pulso termina. Un modelo adimensional del circuito es

$$\frac{du}{dt} = \frac{\alpha_1}{1 + v^\beta} - u, \quad \frac{dv}{dt} = \frac{\alpha_2}{1 + u^\gamma} - v, \quad (16.19)$$

DONDE

u, v	Concentraciones de los dos represores, en unidades de sus constantes de represión.
α_1, α_2	Tasas de síntesis efectivas (fuerza de cada promotor).
β, γ	Cooperatividad de la represión de cada promotor.

con el tiempo medido en unidades de $1/\alpha$ (la tasa de eliminación, que suponemos igual para ambos).

Nulclinas y puntos fijos. Un sistema de dos variables se entiende mejor en su *plano de fases*, donde cada punto (u, v) es un estado y la ecuación (16.19) asigna a cada uno una velocidad. La *nulclina* de u es la curva donde $du/dt = 0$, es decir, $u = \alpha_1/(1 + v^\beta)$; la de v es $v = \alpha_2/(1 + u^\gamma)$. Donde se cortan, ambas derivadas se anulan: son los *puntos fijos*. Para decidir si un punto fijo es estable, linealizamos.

Teorema 16.3. (Estabilidad lineal en dos dimensiones)

Sea (u^*, v^*) un punto fijo de $\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x})$ y J su matriz jacobiana, $J_{ij} = \partial F_i / \partial x_j$ evaluada en él. Las pequeñas perturbaciones evolucionan como $\delta \mathbf{x}(t) = \sum_k c_k \mathbf{e}_k e^{\lambda_k t}$, donde λ_k y $\mathbf{e}_k$ son los autovalores y autovectores de J . El punto fijo es estable si todos los autovalores tienen parte real negativa; en dos dimensiones, esto equivale a

$$\text{tr } J < 0 \quad \text{y} \quad \det J > 0, \quad \lambda_{1,2} = \frac{\text{tr } J \pm \sqrt{(\text{tr } J)^2 - 4 \det J}}{2}. \quad (16.20)$$

Si $\det J < 0$, los autovalores son reales y de signo opuesto: el punto fijo es una silla.

DONDE

J	Jacobiano: cómo responde cada tasa de cambio a pequeñas variaciones de cada variable.
λ_k	Autovalores de J ; su parte real da la tasa de crecimiento o decaimiento de cada modo.
$\text{tr } J, \det J$	Traza ($\lambda_1 + \lambda_2$) y determinante ($\lambda_1 \lambda_2$) de J .

Para el interruptor, derivando la ecuación (16.19),

$$J = \begin{pmatrix} -1 & -\frac{\alpha_1 \beta v^{\beta-1}}{(1+v^\beta)^2} \\ -\frac{\alpha_2 \gamma u^{\gamma-1}}{(1+u^\gamma)^2} & -1 \end{pmatrix} \equiv \begin{pmatrix} -1 & -a \\ -b & -1 \end{pmatrix}, \quad \lambda = -1 \pm \sqrt{ab}.$$

La traza siempre vale -2 ; la estabilidad depende solo de si $ab < 1$ (estable) o $ab > 1$ (silla). En palabras: un punto fijo es inestable cuando la represión mutua es tan sensible que una pequeña ventaja de un represor se amplifica en el otro más de lo que la eliminación puede corregir.

Ejemplo 16.4. (Biestabilidad con $\alpha_1 = \alpha_2 = 10$ y $\beta = \gamma = 2$)

Sustituyendo $v = 10/(1+u^2)$ en la nulclina de u y resolviendo numéricamente, aparecen tres puntos fijos:

punto fijo (u^*, v^*)	a	b	$\lambda_{1,2}$	tipo
(0,101; 9,899)	0,020	1,980	-0,8; -1,2	estable (“V encendido”)
(2; 2)	1,6	1,6	+0,6; -2,6	silla
(9,899; 0,101)	1,980	0,020	-0,8; -1,2	estable (“U encendido”)

La silla está en la diagonal y su autovector estable, a lo largo de ella, forma la *separatriz*: toda trayectoria que empieza por encima termina en el estado “V encendido”, y toda la que empieza por debajo, en “U encendido”. En cambio, sin cooperatividad ($\beta = \gamma = 1$) las nulclinas se cortan en un único punto, $u^* = v^* = 2,70$, que es estable: no hay memoria.

¿Cuánta síntesis hace falta? En el caso simétrico ($\alpha_1 = \alpha_2 = \alpha$, $\beta = \gamma = 2$) el punto diagonal cumple $u(1+u^2) = \alpha$ y tiene $a = b = 2\alpha u/(1+u^2)^2 = 2u^2/(1+u^2)$. La condición de silla $a > 1$ equivale a $u > 1$, es decir, $\alpha > 2$. Con promotores más débiles que ese umbral, el interruptor no funciona.

interruptor.py

```
import numpy as np
from scipy.integrate import solve_ivp

a1 = a2 = 10.0
n = 2
```

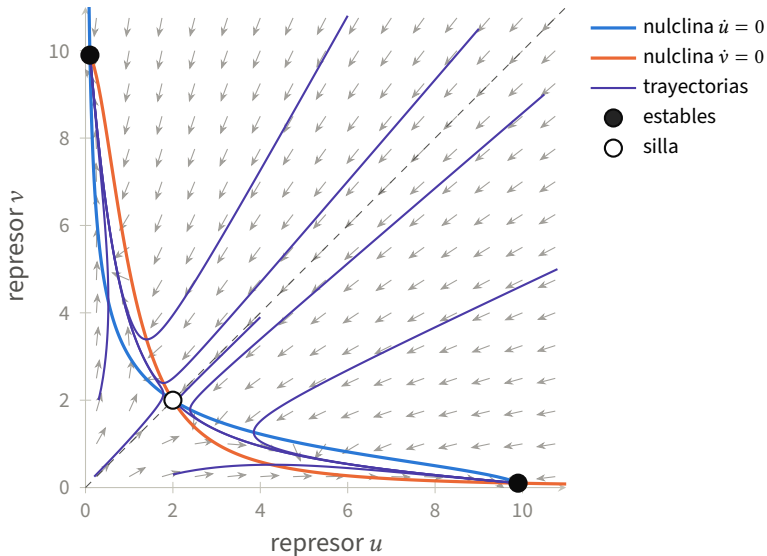


Figura 16.10. Plano de fases del interruptor genético (ecuación (16.19), $\alpha_1 = \alpha_2 = 10$, $\beta = \gamma = 2$). Las flechas grises indican la dirección del flujo; las curvas azul y naranja son las nulclinas, que se cortan en tres puntos fijos. Las trayectorias (violeta), integradas desde ocho estados iniciales, terminan en uno de los dos estados estables según el lado de la diagonal discontinua (la separatriz) en el que empiezan. Un pulso de inductor que empuje al sistema al otro lado de la separatriz conmuta el estado, y el sistema lo recuerda al retirarse el pulso.

```
def toggle(t, y):
    u, v = y
    return [a1 / (1 + v**n) - u, a2 / (1 + u**n) - v]

for y0 in [(0.3, 2.0), (2.0, 0.3), (4.0, 3.9)]:
    s = solve_ivp(toggle, (0, 30), y0, rtol=1e-8)
    print(y0, "->", s.y[:, -1].round(2))
# (0.3, 2.0) -> [0.1 9.9]   (2.0, 0.3) -> [9.9 0.1]
# (4.0, 3.9) -> [9.9 0.1]   (empieza bajo la diagonal)
```

16.2.7 El repressilator: oscilaciones a partir de retroalimentación negativa

En el mismo número de *Nature* en que apareció el interruptor, Elowitz y Leibler (2000) presentaron un oscilador sintético, el *repressilator*: tres represores (LacI, TetR y cI del fago λ) dispuestos en un ciclo en el que cada uno reprime al siguiente. El circuito inducía periódicamente la síntesis de una proteína fluorescente verde en células individuales, con periodos típicos de horas, más largos que el ciclo de división, de modo que el estado del oscilador se transmitía de una generación a la siguiente; las oscilaciones eran ruidosas, posiblemente por fluctuaciones estocásticas de sus componentes. El modelo adimensional usual distingue el ARNm m_i y la proteína p_i de cada gen:

$$\frac{dm_i}{dt} = -m_i + \frac{\alpha}{1 + p_j^n} + \alpha_0, \quad \frac{dp_i}{dt} = -\beta(p_i - m_i), \quad (i, j) \in \{(1, 3), (2, 1), (3, 2)\}. \quad (16.21)$$

DONDE

m_i, p_i	ARNm y proteína del gen i (adimensionales).
α, α_0	Transcripción máxima y basal (“fuga” del promotor reprimido).
β	Cociente entre la tasa de eliminación de la proteína y la del ARNm.
n	Coefficiente de Hill de la represión.

El tiempo se mide en unidades del tiempo de vida del ARNm.

¿Por qué oscila? Un ciclo de tres represiones es, en conjunto, una retroalimentación *negativa* (tres signos negativos dan uno negativo) con retraso, porque la señal tarda en recorrer las seis etapas (tres transcripciones, tres traducciones). Si la retroalimentación es lo bastante fuerte, el sistema se “pasa de largo” una y otra vez en lugar de asentarse. Hagamos el análisis de estabilidad. Por simetría existe un punto fijo con $m_i = p_i = p^*$, donde $p^* = \alpha/(1 + p^{*n}) + \alpha_0$. Llamemos $X = -f'(p^*) = \alpha n p^{*n-1}/(1 + p^{*n})^2 > 0$ a la sensibilidad de la represión. Linealizando la ecuación (16.21), $(\lambda + 1)\delta m_i = -X\delta p_j$ y $(\lambda + \beta)\delta p_i = \beta\delta m_i$; combinándolas y recorriendo el ciclo,

$$\left[\frac{(\lambda + 1)(\lambda + \beta)}{\beta} \right]^3 = -X^3 \implies \frac{(\lambda + 1)(\lambda + \beta)}{\beta} \in \{-X, Xe^{i\pi/3}, Xe^{-i\pi/3}\}.$$

La raíz real $-X$ solo da autovalores con parte real negativa. Las complejas pueden cruzar el eje imaginario: sustituyendo $\lambda = i\omega$ en $(\lambda + 1)(\lambda + \beta) = \beta Xe^{i\pi/3}$ y separando

parte real e imaginaria se obtienen $\beta - \omega^2 = \beta X/2$ y $\omega(1 + \beta) = \beta X\sqrt{3}/2$. Eliminando ω llegamos al criterio siguiente.

Teorema 16.4. (Condición de oscilación del repressilator)

El punto fijo simétrico de la ecuación (16.21) es inestable, y el sistema oscila, si y solo si

$$\frac{(\beta + 1)^2}{\beta} < \frac{3X^2}{4 - 2X} \quad \text{o bien} \quad X \geq 2. \quad (16.22)$$

DONDE

X	Pendiente (cambiada de signo) de la función de represión en el punto fijo.
$(\beta + 1)^2/\beta$	Término mínimo (igual a 4) cuando $\beta = 1$: oscilar es más fácil si el ARNm y la proteína viven lo mismo.

Es la misma condición que publicaron Elowitz y Leibler (2000), quienes definen X con el signo de la pendiente (negativo) y escriben por eso $4 + 2X$ en el denominador. Hemos comprobado la ecuación (16.22) contra los autovalores del jacobiano de 6×6 en 1600 combinaciones de X y β , sin ninguna discrepancia. Dos consecuencias saltan a la vista. Primera: sin cooperatividad ($n = 1$, $\alpha_0 = 0$) se tiene $p^*(1 + p^*) = \alpha$ y $X = p^*/(1 + p^*) < 1$, de modo que el lado derecho es menor que $3/2$ y nunca supera al izquierdo, que vale al menos 4: *un repressilator con $n = 1$ no puede oscilar*. Segunda: las oscilaciones exigen transcripción fuerte (con $n = 2$, α debe superar 4,24 incluso en el caso óptimo $\beta = 1$) y vidas medias de ARNm y proteína comparables.

16.2.8 Ruido: cuando las moléculas son pocas

Las ecuaciones diferenciales describen concentraciones continuas y deterministas. Pero en una bacteria muchos factores de transcripción están presentes en decenas de copias, y sus ARNm en unas pocas. A esas escalas, cada evento de síntesis o degradación es un suceso aleatorio, y dos células genéticamente idénticas en el mismo ambiente difieren. Elowitz et al. (2002) construyeron cepas de *E. coli* con dos reporteros fluorescentes idénticos controlados por el mismo promotor, lo que les permitió separar el ruido *intrínseco* (inherente a la expresión de cada gen, que hace diferir a los dos reporteros dentro de una célula) del *extrínseco* (fluctuaciones de otros componentes celulares, que los mueve juntos). Ambos contribuían sus-

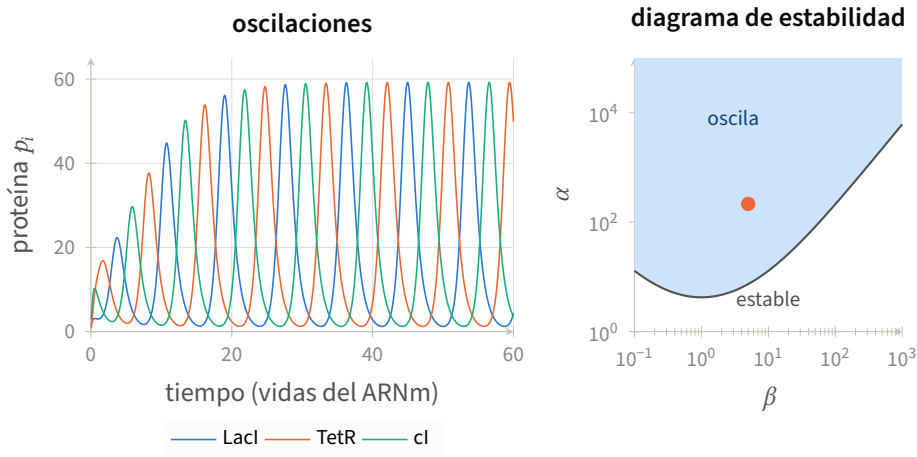


Figura 16.11. Izquierda: simulación de la ecuación (16.21) con parámetros ilustrativos $\alpha = 216$, $\alpha_0 = 0,216$, $\beta = 5$, $n = 2$. Tras un transitorio, las tres proteínas oscilan con un periodo de 8,7 unidades de tiempo, desfasadas un tercio de ciclo: cuando LacI cae, TetR se libera, lo que reprime a cl, lo que a su vez libera a LacI. Derecha: frontera de estabilidad en el plano (β, α) para $n = 2$ según la ecuación (16.22); en la región azul el punto fijo es inestable y el sistema oscila. El punto naranja marca los parámetros de la izquierda; allí el jacobiano tiene un par de autovalores $0,208 \pm 1,266i$ con parte real positiva.

tancialmente, y los bajos números de copia imponían un límite a la precisión de la regulación.

La ecuación maestra. El modelo más sencillo es un nacimiento y muerte: la proteína se produce a tasa k y cada molécula se degrada a tasa γ . La probabilidad $P_x(t)$ de tener x moléculas cumple

$$\frac{dP_x}{dt} = kP_{x-1} + \gamma(x+1)P_{x+1} - (k + \gamma x)P_x. \quad (16.23)$$

DONDE

$P_x(t)$	Probabilidad de que haya exactamente x moléculas en el tiempo t .
k	Tasa de producción (moléculas por unidad de tiempo).
γ	Tasa de degradación por molécula.

En el estado estacionario, el flujo de $x - 1$ a x equilibra el de x a $x + 1$: $kP_{x-1} = \gamma x P_x$, de donde $P_x = (\mu/x)P_{x-1}$ con $\mu = k/\gamma$, y por inducción $P_x = e^{-\mu} \mu^x / x!$. La

distribución es de Poisson: su media coincide con la solución de la ODE, y su coeficiente de variación es $CV = \sigma/\mu = 1/\sqrt{\mu}$. Con 10 moléculas de media, las fluctuaciones típicas son del 32 %; con 10 000, del 1 %.

El algoritmo de Gillespie. Para redes más complejas la ecuación maestra no se puede resolver, pero sí se pueden generar trayectorias exactas. Gillespie (1977) observó que, si el sistema está en un estado con *propensiones* $a_1, \dots, a_R$ (la probabilidad por unidad de tiempo de que ocurra cada reacción), el tiempo hasta la siguiente reacción es exponencial con tasa $a_0 = \sum_r a_r$, y la reacción que ocurre es la r -ésima con probabilidad a_r/a_0 :

$$\tau = \frac{1}{a_0} \ln \frac{1}{u_1}, \quad r = \min \left\{ r' : \sum_{s \leq r'} a_s > u_2 a_0 \right\}, \quad u_1, u_2 \sim \mathcal{U}(0, 1). \quad (16.24)$$

DONDE

a_r	Propensión de la reacción r en el estado actual (p. ej., k para la síntesis y γx para la degradación).
τ	Tiempo de espera hasta la próxima reacción.
u_1, u_2	Números aleatorios uniformes independientes.

El algoritmo es exacto en el sentido de que las trayectorias generadas tienen exactamente la distribución que describe la ecuación maestra.

gillespie.py

```
import numpy as np

def gillespie_nm(k, g, x0, T, rng):
    """Nacimiento (tasa k) y muerte (tasa g*x) de una proteína."""
    t, x = 0.0, x0
    ts, xs = [t], [x]
    while t < T:
        a1, a2 = k, g * x                    # propensiones
        a0 = a1 + a2
        t += rng.exponential(1 / a0)          # ¿cuándo?
        x += 1 if rng.random() * a0 < a1 else -1  # ¿cuál?
        ts.append(t)
        xs.append(x)
    return np.array(ts), np.array(xs)

ts, xs = gillespie_nm(1.0, 0.1, 10, 20000, np.random.default_rng(7))
```

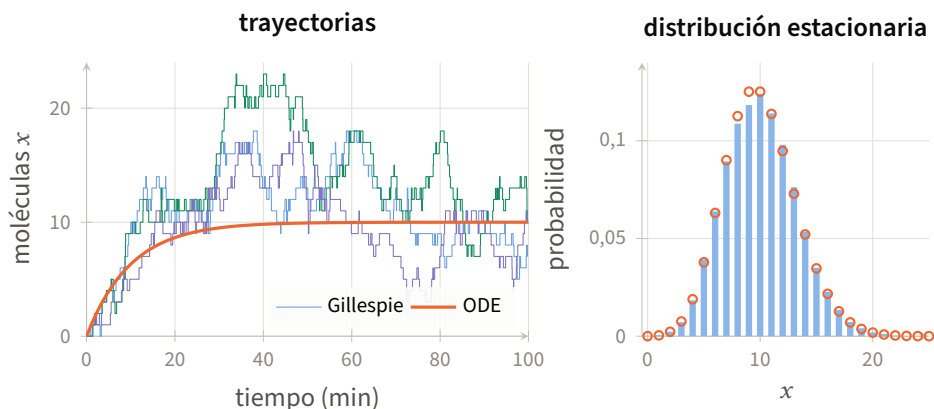


Figura 16.12. **Izquierda:** tres trayectorias exactas del algoritmo de Gillespie (ecuación (16.24)) para una proteína con síntesis $k = 1 \text{ min}^{-1}$ y degradación $\gamma = 0,1 \text{ min}^{-1}$, partiendo de cero, frente a la solución de la ODE, $x(t) = 10(1 - e^{-0,1t})$ (naranja). La ODE describe bien la media, pero no las fluctuaciones de célula a célula. **Derecha:** distribución del número de moléculas en una simulación larga (39 960 reacciones, barras) frente a la distribución de Poisson de media 10 que predice la ecuación (16.23) (círculos). La simulación da media 10,05, varianza 10,19 y $\text{CV} = 0,318$, frente al $1/\sqrt{10} = 0,316$ teórico.

Ruido que decide

El ruido no es solo una molestia. En un sistema biestable como el de la figura 16.10, las fluctuaciones pueden empujar a una célula a través de la separatriz y conmutar su estado sin ninguna señal externa. Así, una población clonal puede dividirse en dos subpoblaciones con fenotipos distintos, una estrategia de “apuesta diversificada” ante ambientes impredecibles. Simular estos fenómenos exige el algoritmo de Gillespie (o sus aproximaciones más rápidas, como el *tau-leaping*) aplicado al circuito completo, porque la ODE, que solo ve la media, predice que cada célula se queda para siempre en el estado donde empezó.

16.2.9 Inferir la red a partir de los datos

Hasta aquí supusimos conocido el circuito. En la práctica, a menudo tenemos lo contrario: una matriz de expresión (genes por muestras, como las de RNA-seq del capítulo 11) y la pregunta de qué gen regula a cuál. La idea más ingenua es unir los genes muy correlacionados, pero la correlación no distingue un efecto directo ($X \rightarrow Z$) de uno indirecto ($X \rightarrow Y \rightarrow Z$) ni de una causa común.

Huynh-Thu et al. (2010) propusieron GENIE3, que descompone el problema de inferir una red de p genes en p problemas de regresión. Para cada gen diana j , se predice su expresión a partir de la de todos los reguladores candidatos con un conjunto de árboles (*Random Forests* o *Extra-Trees*), y la importancia de cada regulador en esa predicción se toma como evidencia de una arista:

$$x_j = f_j(\mathbf{x}_{-j}) + \varepsilon_j, \quad w_{ij} = \text{Imp}_i(\hat{f}_j), \quad (16.25)$$

DONDE

x_j	Expresión del gen diana j en cada muestra.
$\mathbf{x}_{-j}$	Expresión de los reguladores candidatos (excluido j).
$\hat{f}_j$	Conjunto de árboles ajustado para predecir x_j .
w_{ij}	Importancia del regulador i (reducción total de varianza en los nodos de los árboles que lo usan).

Las importancias de todas las dianas se ordenan en una lista global de aristas candidatas. El método no supone ninguna forma particular de regulación, capta interacciones combinatorias y no lineales, produce redes dirigidas y escala bien; fue el mejor en el desafío DREAM4 *In Silico Multifactorial*.

```

genie3.py

import numpy as np
from sklearn.ensemble import RandomForestRegressor

def genie3(X, reguladores, n_arboles=300):
    """X: muestras x genes (estandarizada). Devuelve W[i, j]."""
    n_genes = X.shape[1]
    W = np.zeros((n_genes, n_genes))
    for j in range(n_genes):
        entradas = [i for i in reguladores if i != j]
        rf = RandomForestRegressor(n_estimators=n_arboles,
                                   max_features="sqrt")
        rf.fit(X[:, entradas], X[:, j])
        W[entradas, j] = rf.feature_importances_
    return W

```

¿Cómo se evalúa un método de inferencia? Con una red de referencia, se recorre la lista ordenada de aristas y se calcula, para cada umbral, la *precisión* (fracción de aristas predichas que son reales) y la *exhaustividad* (fracción de aristas reales recuperadas); el área bajo esa curva (AUPR) resume el desempeño. La figura 16.13

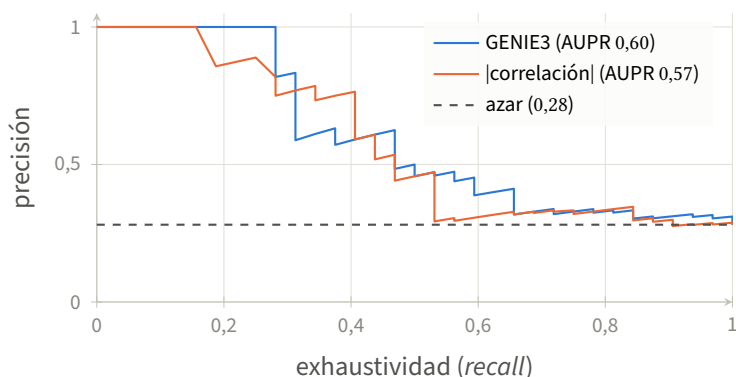


Figura 16.13. Evaluación de la inferencia en una red sintética de 20 genes (6 reguladores, 32 aristas verdaderas entre 114 candidatas) a partir de 300 perfiles de expresión simulados. Ambos métodos recuperan las primeras aristas con precisión alta, pero la precisión cae rápidamente: a mitad de exhaustividad, aproximadamente la mitad de las predicciones son falsas. La línea discontinua es la precisión de una lista ordenada al azar. Simulación en `figuras/cap16/generar.py`.

muestra un experimento controlado: generamos redes aleatorias de 20 genes con 6 reguladores, simulamos 300 estados estacionarios bajo perturbaciones aleatorias y comparamos GENIE3 con la correlación absoluta. En promedio sobre cinco redes, GENIE3 obtiene $AUPR = 0,54 \pm 0,06$ y la correlación $0,53 \pm 0,09$, ambos el doble de lo esperable al azar (0,27). La ventaja de GENIE3 es pequeña y no aparece en todas las redes.

Un resultado tan modesto no es excepcional en la inferencia de redes. En el proyecto DREAM5, Marbach et al. (2012) evaluaron a ciegas más de 30 métodos de inferencia sobre datos de *E. coli*, *Staphylococcus aureus*, *S. cerevisiae* y datos simulados, y observaron que ningún método era el mejor en todos los conjuntos de datos. En cambio, *integrar* las predicciones de muchos métodos (la “sabiduría de las multitudes”) daba un desempeño alto y robusto, y con ello construyeron redes de alta confianza para *E. coli* y *S. aureus*, de unas 1700 interacciones transcripcionales cada una, con una precisión de alrededor del 50 %.

⚠ Lo que los datos de expresión no pueden decir

Los datos observacionales de expresión, por sí solos, rara vez identifican la causalidad: dos genes coregulados por un tercero no medido parecen

regularse entre sí, y un factor de transcripción cuyo ARNm apenas cambia (porque se regula por fosforilación) es invisible. Las perturbaciones (*knockouts*, CRISPRi), las series temporales y los datos de unión al ADN (ChIP-seq, motivos del capítulo 4) restringen enormemente el espacio de redes compatibles. Una red inferida es un conjunto de hipótesis ordenadas por plausibilidad, no un mapa.

La topología dice qué *puede* hacer un circuito; los parámetros (cooperatividad, fuerzas de promotor, vidas medias) deciden qué *hace*. Un mismo ciclo de represores es estable u oscila según n , α y β .

≡ En resumen

- Un interactoma es un grafo; su matriz de adyacencia codifica grados ($k_i = \sum_j A_{ij}$), caminos $((A^t)_{ij})$ y triángulos $((A^3)_{ii}/2)$, de los que salen el coeficiente de agrupamiento, las distancias y la intermediación.
- El doble híbrido mide pares binarios y AP-MS mide complejos (modelos estrella o matriz); cada método tiene su perfil de falsos positivos y negativos. STRING integra canales de evidencia y BioGRID cura interacciones de la literatura.
- Las redes biológicas son mundos pequeños muy agrupados (Watts-Strogatz). El enlace preferencial de Barabási-Albert produce $P(k) \propto k^{-3}$, pero una cola ancha en un gráfico log-log no demuestra una ley de potencias; en la red de levadura, los grandes complejos dominan la cola.
- La modularidad Q compara las aristas internas de una partición con las esperadas por azar; Louvain la maximiza y las comunidades deben validarse con anotaciones y redes aleatorizadas.
- En el modelo $\dot{Y} = \beta - \alpha Y$, el tiempo de respuesta es $\ln 2/\alpha$, independiente de la producción. La autorregulación negativa lo acorta y amortigua el ruido; el FFL coherente con AND filtra pulsos breves.
- La represión mutua cooperativa crea biestabilidad (dos puntos estables y una silla, identificados por los autovalores del jacobiano); un ciclo de tres represores oscila si se cumple $(\beta + 1)^2/\beta < 3X^2/(4 - 2X)$.

- Con pocas moléculas la expresión es estocástica; el algoritmo de Gillespie simula trayectorias exactas y, en nacimiento-muerte, reproduce la distribución de Poisson con $CV = 1/\sqrt{\mu}$.
- Inferir redes a partir de la expresión es difícil: GENIE3 convierte el problema en regresiones con árboles, y la integración de muchos métodos es más robusta que cualquiera de ellos por separado.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 16.1** ●●● Para el grafo de la figura 16.1, calcule A^2 y verifique que su diagonal es el vector de grados. ¿Qué cuenta $(A^2)_{ij}$ cuando $i \neq j$?
- 16.2** ●●● Una purificación AP-MS recupera un cebo y 11 presas. ¿Cuántas aristas genera el modelo estrella y cuántas el modelo matriz? Si el complejo real tiene forma de anillo, ¿qué fracción de las aristas del modelo matriz son contactos físicos?
- 16.3** ●●● Una proteína de *E. coli* estable tiene $t_{1/2}$ igual a un tiempo de generación. ¿Cuál debería ser la vida media de degradación activa para que el tiempo de respuesta sea de un décimo de generación? ¿Cuánto debe aumentar la síntesis para conservar el nivel estacionario?
- 16.4** ●●● Con `networkx`, genere una red ER y otra BA con $n = 2000$ y $\langle k \rangle \approx 8$. Compare C , L y la distribución de grado con la de la figura 16.4. ¿Qué modelo se parece más a la red de levadura y en qué falla cada uno?
- 16.5** ●●● Descargue de STRING la red física de su organismo favorito, extraiga la ego-red de radio 2 de una proteína de interés, detecte comunidades con Louvain y compare Q con el de 20 redes aleatorizadas con `nx.double_edge_swap`. Pruebe varias semillas: ¿cambian las comunidades?
- 16.6** ●●● Demuestre la ecuación (16.16) y compruebe numéricamente, integrando la ecuación (16.15), cuánto cambia X_{est} al duplicar β con $n = 1, 2$ y 4.

- 16.7** ●●● Simule el FFL de la ecuación (16.18) con pulsos de duración creciente y grafique el máximo de Z en función de la duración. Estime la duración mínima de pulso que el circuito deja pasar.
- 16.8** ●●● Para el interruptor simétrico con $n = 3$, encuentre el valor crítico de α a partir del cual aparecen tres puntos fijos, generalizando el razonamiento del ejemplo 16.4.
- 16.9** ●●● Complete la derivación de la ecuación (16.22): sustituya $\lambda = i\omega$, elimine ω y muestre que para $X \geq 2$ siempre existe un autovalor con parte real positiva. Verifique el periodo de la figura 16.11 con la frecuencia ω del par de autovalores dominante y explique la discrepancia.
- 16.10** ●●● Implemente el algoritmo de Gillespie para el interruptor biestable (cuatro reacciones: síntesis y degradación de u y de v , con propensiones $\Omega\alpha/(1+(v/\Omega)^2)$ y u , etc.) y mida el tiempo medio de conmutación espontánea en función del tamaño del sistema Ω .

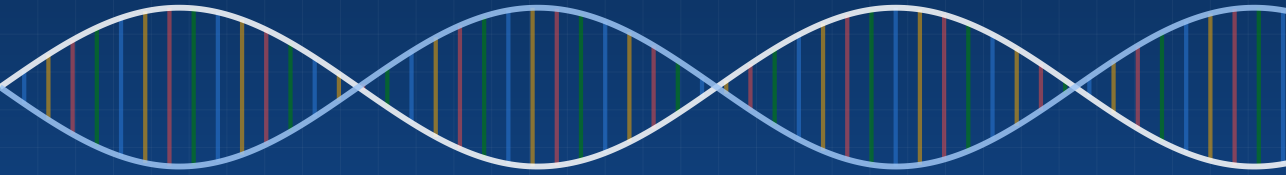
Referencias del capítulo

- Alon, U. (2007). Network motifs: Theory and experimental approaches. *Nature Reviews Genetics*, 8(6), 450-461. <https://doi.org/10.1038/nrg2102>
- Alon, U. (2019). *An introduction to systems biology: Design principles of biological circuits* (2.^a ed.). Chapman; Hall/CRC. <https://doi.org/10.1201/9780429283321>
- Barabási, A.-L., & Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286(5439), 509-512. <https://doi.org/10.1126/science.286.5439.509>
- Barabási, A.-L., Gulbahce, N., & Loscalzo, J. (2011). Network medicine: A network-based approach to human disease. *Nature Reviews Genetics*, 12(1), 56-68. <https://doi.org/10.1038/nrg2918>
- Barabási, A.-L., & Oltvai, Z. N. (2004). Network biology: Understanding the cell's functional organization. *Nature Reviews Genetics*, 5(2), 101-113. <https://doi.org/10.1038/nrg1272>
- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), P10008. <https://doi.org/10.1088/1742-5468/2008/10/P10008>
- Broido, A. D., & Clauset, A. (2019). Scale-free networks are rare. *Nature Communications*, 10(1), 1017. <https://doi.org/10.1038/s41467-019-08746-5>
- Elowitz, M. B., & Leibler, S. (2000). A synthetic oscillatory network of transcriptional regulators. *Nature*, 403(6767), 335-338. <https://doi.org/10.1038/35002125>

- Elowitz, M. B., Levine, A. J., Siggia, E. D., & Swain, P. S. (2002). Stochastic gene expression in a single cell. *Science*, 297(5584), 1183-1186. <https://doi.org/10.1126/science.1070919>
- Gardner, T. S., Cantor, C. R., & Collins, J. J. (2000). Construction of a genetic toggle switch in *Escherichia coli*. *Nature*, 403(6767), 339-342. <https://doi.org/10.1038/35002131>
- Gavin, A.-C., Bösch, M., Krause, R., Grandi, P., Marzioch, M., Bauer, A., Schultz, J., Rick, J. M., Michon, A.-M., Cruciat, C.-M., Remor, M., Höfert, C., Schelder, M., Brajenovic, M., Ruffner, H., Merino, A., Klein, K., Hudak, M., Dickson, D., ... Superti-Furga, G. (2002). Functional organization of the yeast proteome by systematic analysis of protein complexes. *Nature*, 415(6868), 141-147. <https://doi.org/10.1038/415141a>
- Gillespie, D. T. (1977). Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25), 2340-2361. <https://doi.org/10.1021/j100540a008>
- Girvan, M., & Newman, M. E. J. (2002). Community structure in social and biological networks. *Proceedings of the National Academy of Sciences*, 99(12), 7821-7826. <https://doi.org/10.1073/pnas.122653799>
- Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference*, 11-15. <https://doi.org/10.25080/TCWV9851>
- Huynh-Thu, V. A., Irrthum, A., Wehenkel, L., & Geurts, P. (2010). Inferring regulatory networks from expression data using tree-based methods. *PLoS ONE*, 5(9), e12776. <https://doi.org/10.1371/journal.pone.0012776>
- Ito, T., Chiba, T., Ozawa, R., Yoshida, M., Hattori, M., & Sakaki, Y. (2001). A comprehensive two-hybrid analysis to explore the yeast protein interactome. *Proceedings of the National Academy of Sciences*, 98(8), 4569-4574. <https://doi.org/10.1073/pnas.061034498>
- Jeong, H., Mason, S. P., Barabási, A.-L., & Oltvai, Z. N. (2001). Lethality and centrality in protein networks. *Nature*, 411(6833), 41-42. <https://doi.org/10.1038/35075138>
- Johnson, K. A., & Goody, R. S. (2011). The original Michaelis constant: Translation of the 1913 Michaelis-Menten paper. *Biochemistry*, 50(39), 8264-8269. <https://doi.org/10.1021/bi201284u>
- Mangan, S., & Alon, U. (2003). Structure and function of the feed-forward loop network motif. *Proceedings of the National Academy of Sciences*, 100(21), 11980-11985. <https://doi.org/10.1073/pnas.2133841100>
- Marbach, D., Costello, J. C., Küffner, R., Vega, N. M., Prill, R. J., Camacho, D. M., Allison, K. R., Kellis, M., Collins, J. J., Stolovitzky, G., & The DREAM5 Consortium. (2012). Wisdom of crowds for robust gene network inference. *Nature Methods*, 9(8), 796-804. <https://doi.org/10.1038/nmeth.2016>
- Milo, R., Shen-Orr, S., Itzkovitz, S., Kashtan, N., Chklovskii, D., & Alon, U. (2002). Network motifs: Simple building blocks of complex networks. *Science*, 298(5594), 824-827. <https://doi.org/10.1126/science.298.5594.824>

- Newman, M. E. J. (2006). Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, 103(23), 8577-8582. <https://doi.org/10.1073/pnas.0601602103>
- Oughtred, R., Rust, J., Chang, C., Breitkreutz, B.-J., Stark, C., Willems, A., Boucher, L., Leung, G., Kolas, N., Zhang, F., Dolma, S., Coulombe-Huntington, J., Chatr-aryamontri, A., Dolinski, K., & Tyers, M. (2021). The BioGRID database: A comprehensive biomedical resource of curated protein, genetic, and chemical interactions. *Protein Science*, 30(1), 187-200. <https://doi.org/10.1002/pro.3978>
- Rosenfeld, N., Elowitz, M. B., & Alon, U. (2002). Negative autoregulation speeds the response times of transcription networks. *Journal of Molecular Biology*, 323(5), 785-793. [https://doi.org/10.1016/S0022-2836\(02\)00994-4](https://doi.org/10.1016/S0022-2836(02)00994-4)
- Shannon, P., Markiel, A., Ozier, O., Baliga, N. S., Wang, J. T., Ramage, D., Amin, N., Schwikowski, B., & Ideker, T. (2003). Cytoscape: A software environment for integrated models of biomolecular interaction networks. *Genome Research*, 13(11), 2498-2504. <https://doi.org/10.1101/gr.1239303>
- Shen-Orr, S. S., Milo, R., Mangan, S., & Alon, U. (2002). Network motifs in the transcriptional regulation network of *Escherichia coli*. *Nature Genetics*, 31(1), 64-68. <https://doi.org/10.1038/ng881>
- Szklarczyk, D., Kirsch, R., Koutrouli, M., Nastou, K., Mehryary, F., Hachilif, R., Gable, A. L., Fang, T., Doncheva, N. T., Pyysalo, S., Bork, P., Jensen, L. J., & von Mering, C. (2023). The STRING database in 2023: Protein-protein association networks and functional enrichment analyses for any sequenced genome of interest. *Nucleic Acids Research*, 51(D1), D638-D646. <https://doi.org/10.1093/nar/gkac1000>
- Uetz, P., Giot, L., Cagney, G., Mansfield, T. A., Judson, R. S., Knight, J. R., Lockshon, D., Narayan, V., Srinivasan, M., Pochart, P., Qureshi-Emili, A., Li, Y., Godwin, B., Conover, D., Kalbfleisch, T., Vijayadamodar, G., Yang, M., Johnston, M., Fields, S., & Rothberg, J. M. (2000). A comprehensive analysis of protein-protein interactions in *Saccharomyces cerevisiae*. *Nature*, 403(6770), 623-627. <https://doi.org/10.1038/35001009>
- von Mering, C., Krause, R., Snel, B., Cornell, M., Oliver, S. G., Fields, S., & Bork, P. (2002). Comparative assessment of large-scale data sets of protein-protein interactions. *Nature*, 417(6887), 399-403. <https://doi.org/10.1038/nature750>
- Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of 'small-world' networks. *Nature*, 393(6684), 440-442. <https://doi.org/10.1038/30918>

Machine learning e inteligencia artificial en bioinformática



Puesto que todos los modelos son erróneos, el científico debe estar alerta a lo que es importantemente erróneo.

— Box (1976)

● Al terminar este capítulo usted podrá

- Formular un problema bioinformático como aprendizaje supervisado y convertir secuencias en vectores (codificación *one-hot*, espectro de *k*-mers, núcleo de espectro).
- Derivar el gradiente de la regresión logística y explicar el funcionamiento de las máquinas de vectores de soporte y de los *random forests*.
- Diseñar una validación honesta: descomposición sesgo-varianza, validación cruzada por grupos para evitar la fuga de información por homología y métricas adecuadas para clases desbalanceadas (curvas ROC y de precisión-exhaustividad).
- Explicar la retropropagación y por qué un filtro convolucional aplicado a ADN codificado en *one-hot* es una matriz de pesos posicionales; interpretar un modelo con mapas de saliencia.

- Describir el mecanismo de atención y el modelado de lenguaje enmascarado, y usar un modelo de lenguaje de proteínas (ESM-2) para obtener *embeddings*, contactos y predicciones de efecto de variantes sin supervisión.

Muchos métodos anteriores parten de una descripción explícita del proceso biológico: las matrices de sustitución representan la evolución de secuencias, los HMM de perfil describen familias de proteínas y los modelos de coalescencia, genealogías. El aprendizaje automático (*machine learning*, ML) sigue otra ruta: ajusta a partir de muchos ejemplos una función que relaciona entradas y salidas. Este enfoque es útil cuando abundan los datos y cuesta describir el mecanismo, por ejemplo, al predecir qué fragmentos de ADN actúan como potenciadores en el hígado o qué mutaciones desestabilizan una proteína (Greener et al., 2022; LeCun et al., 2015).

Pero la potencia tiene un precio. Un modelo con millones de parámetros puede memorizar sus datos de entrenamiento y aparentar una exactitud que se desploma ante secuencias nuevas; puede aprender atajos (el contenido de GC, un artefacto del protocolo) en lugar de biología; y puede producir números que parecen probabilidades sin serlo. Por eso este capítulo dedica tanto espacio a *evaluar* modelos como a construirlos, siguiendo la tradición estadística que resumen Hastie et al. (2009).

El recorrido tiene tres etapas. En la sección 17.1 convertiremos secuencias en vectores y estudiaremos los clasificadores clásicos (regresión logística, SVM, *random forests*) junto con las herramientas para validarlos sin engañarnos. En la sección 17.2 pasaremos a las redes neuronales profundas: la retropropagación, las redes convolucionales que leen ADN y la forma de abrir la “caja negra”. En la sección 17.3 llegaremos a los modelos de lenguaje de proteínas, que aprenden la gramática de la evolución a partir de cientos de millones de secuencias sin una sola etiqueta, y los usaremos, con un modelo real, sobre la ubiquitina humana.

17.1 Clasificación de secuencias con ML clásico

 Colab · 17.1 · ML clásico para secuencias

 repositorio del curso

El catador que nunca leyó un manual

Un catador experto distingue un Rioja de un Burdeos con un sorbo, pero sería incapaz de escribir las reglas que usa. Aprendió probando miles de vinos *etiquetados*, y su cerebro ajustó poco a poco una frontera entre “esto es Rioja” y “esto no”. Si solo hubiera probado vinos de tres bodegas, confundiría “Rioja” con “el estilo de esas tres bodegas” y fracasaría ante la cuarta. El aprendizaje supervisado hace lo mismo con secuencias, y comparte el mismo riesgo: aprender las bodegas en lugar de la región.

17.1.1 El problema del aprendizaje supervisado

Supongamos que tenemos n secuencias de ADN de 200 pb y, para cada una, un experimento (por ejemplo, ChIP-seq) que dice si un factor de transcripción se une a ella ($y = 1$) o no ($y = 0$). Queremos una función que, dada una secuencia *nueva*, prediga la etiqueta. Ese es el esquema general del aprendizaje supervisado.

Definición 17.1. (Aprendizaje supervisado y riesgo empírico)

Dado un conjunto de entrenamiento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ extraído de una distribución desconocida $P(\mathbf{x}, y)$, y una familia de funciones f_θ , el aprendizaje supervisado busca los parámetros que minimizan el *riesgo empírico* regularizado:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_\theta(\mathbf{x}_i)) + \lambda \Omega(\theta). \quad (17.1)$$

El objetivo real, sin embargo, es el *riesgo esperado* $R(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim P}[\ell(y, f_\theta(\mathbf{x}))]$ sobre datos que el modelo nunca vio.

DONDE

$\mathbf{x}_i, y_i$	Representación numérica de la i -ésima secuencia y su etiqueta (clase o valor).
f_θ	Modelo con parámetros θ (pesos de una regresión, de una red, umbrales de un árbol).
$\ell(y, \hat{y})$	Función de pérdida: cuánto cuesta predecir $\hat{y}$ cuando la verdad es y .
$\Omega(\theta), \lambda$	Penalización de complejidad (regularización) y su peso.
$P(\mathbf{x}, y)$	Distribución de la población de secuencias sobre la que se usará el modelo.

Toda la dificultad del aprendizaje automático está en la distancia entre las dos cantidades de la definición 17.1: minimizamos el error sobre los datos que tenemos, pero nos importa el error sobre los que no tenemos. Un modelo que minimiza el primero sin controlar el segundo *sobreajusta*. Las herramientas de esta sección (regularización, validación cruzada, particiones por homología) existen para mantener honesta esa distancia. Greener et al. (2022) ofrecen una guía introductoria excelente, escrita para biólogos, de todo este vocabulario.

17.1.2 Representar una secuencia como vector

Los algoritmos de ML trabajan con vectores de números, no con cadenas de letras. La elección de la representación, o *ingeniería de atributos*, decide en buena medida qué puede aprender el modelo.

Codificación one-hot. La representación más fiel asigna a cada posición un vector indicador de longitud 4:

$$X_{c,i} = \mathbb{1}[x_i = c], \quad c \in \{A, C, G, T\}, \quad i = 1, \dots, L. \quad (17.2)$$

DONDE

X	Matriz <i>one-hot</i> de tamaño $4 \times L$; cada columna tiene un único 1.
x_i	Base en la posición i de la secuencia.
L	Longitud de la secuencia.

No introduce ningún orden artificial entre las bases (codificar A=1, C=2, G=3, T=4 haría creer al modelo que T es “más” que A) y conserva la posición exacta de cada nucleótido. Su inconveniente es que un motivo desplazado una base produce un vector completamente distinto: un clasificador lineal sobre *one-hot* no puede reconocer un motivo que aparece en posiciones variables. Las redes convolucionales de la sección 17.2 resuelven exactamente ese problema.

Espectro de k -mers. La alternativa clásica renuncia a la posición y cuenta palabras. Para cada una de las 4^k palabras u de longitud k ,

$$\Phi_k(x) = (\phi_u(x))_{u \in \mathcal{A}^k}, \quad \phi_u(x) = \#\{i : x_i x_{i+1} \dots x_{i+k-1} = u\}. \quad (17.3)$$

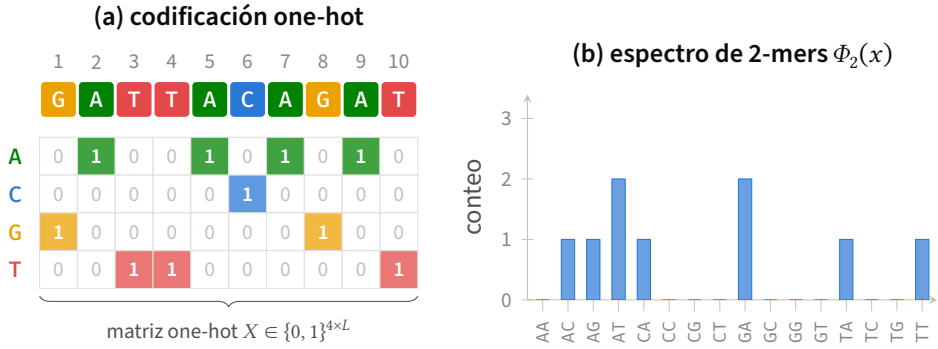


Figura 17.1. Dos maneras de convertir la secuencia **GATTACAGAT** en números. (a) La codificación *one-hot* (ecuación (17.2)) produce una matriz 4×10 con un único 1 por columna; conserva la posición de cada base, pero no generaliza a desplazamientos. (b) El espectro de 2-mers (ecuación (17.3)) cuenta las 9 palabras solapadas en un vector de 16 componentes; AT y GA aparecen dos veces. El espectro olvida *dónde* está cada palabra, y por eso dos secuencias que son permutaciones de bloques pueden tener espectros casi idénticos. Datos de `figuras/cap17/generar.py`.

DONDE

$\Phi_k(x)$ Vector de conteos de k -mers de la secuencia x (dimensión 4^k en ADN, 20^k en proteínas).

$\phi_u(x)$ Número de apariciones (solapadas) de la palabra u en x .

$\mathcal{A}^k$ Conjunto de todas las palabras de longitud k sobre el alfabeto $\mathcal{A}$.

El espectro es invariante a desplazamientos y permite comparar secuencias de longitudes distintas. A cambio pierde el orden de las palabras y crece exponencialmente con k : con $k = 6$ hay 4096 dimensiones para ADN, y con $k = 3$ ya hay 8000 para proteínas, casi todas con conteo cero en una secuencia concreta. La figura 17.1 contrasta las dos representaciones.

El núcleo de espectro. Muchos algoritmos solo necesitan saber cuánto se parecen dos secuencias, no sus coordenadas. Leslie et al. (2002) definieron el *núcleo de espectro* (*spectrum kernel*) como el producto interno de los espectros:

$$K_k(x, y) = \langle \Phi_k(x), \Phi_k(y) \rangle = \sum_{u \in \mathcal{A}^k} \phi_u(x) \phi_u(y), \quad \tilde{K}_k(x, y) = \frac{K_k(x, y)}{\sqrt{K_k(x, x) K_k(y, y)}}. \quad (17.4)$$

DONDE

$K_k(x, y)$	Similitud entre x e y : número de pares de k -mers idénticos que comparten.
$\tilde{K}_k$	Versión normalizada (coseno), entre 0 y 1; elimina el efecto de la longitud.

Su gracia computacional es que se calcula en tiempo lineal en la longitud de las secuencias recorriendo solo los k -mers presentes (con un árbol de sufijos o una tabla *hash*), sin construir nunca los vectores de dimensión $|\mathcal{A}|^k$. Combinado con una SVM, obtuvo en la detección de homología remota sobre SCOP resultados comparables a métodos mucho más costosos (Leslie et al., 2002).

Ejemplo 17.1. (Núcleo de espectro a mano)

Sean $x = \text{GATTACAGAT}$ e $y = \text{TACAGATTAC}$, con $k = 2$. Los espectros (solo componentes no nulas) son

$$\Phi_2(x) : \text{AC} : 1, \text{AG} : 1, \text{AT} : 2, \text{CA} : 1, \text{GA} : 2, \text{TA} : 1, \text{TT} : 1;$$

$$\Phi_2(y) : \text{AC} : 2, \text{AG} : 1, \text{AT} : 1, \text{CA} : 1, \text{GA} : 1, \text{TA} : 2, \text{TT} : 1.$$

El producto interno es $K_2(x, y) = 1 \cdot 2 + 1 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 2 \cdot 1 + 1 \cdot 2 + 1 \cdot 1 = 11$, y como $K_2(x, x) = K_2(y, y) = 13$, el núcleo normalizado vale $\tilde{K}_2 = 11/13 \approx 0,846$. Sin embargo, x e y alineadas sin huecos coinciden en solo 2 de 10 posiciones: el núcleo de espectro ve que ambas están hechas de los mismos “ladrillos” (TACAGAT aparece en las dos), aunque desplazados.

17.1.3 Regresión logística: el clasificador lineal probabilístico

El modelo más sencillo que produce probabilidades es la *regresión logística*. Combina linealmente los atributos y comprime el resultado al intervalo $(0, 1)$ con la función sigmoide:

$$p(\mathbf{x}) = \mathbb{P}(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (17.5)$$

DONDE

$\mathbf{w}, b$	Vector de pesos (uno por atributo) y término independiente (sesgo).
$z = \mathbf{w}^\top \mathbf{x} + b$	Puntuación lineal o <i>logit</i> ; su valor es el logaritmo de las probabilidades a favor, $\log \frac{p}{1-p}$.
$\sigma(z)$	Sigmoide logística: convierte cualquier real en una probabilidad.

La interpretación del *logit* conecta con algo conocido: z es un logaritmo de razón de probabilidades, igual que las puntuaciones de las matrices de sustitución del capítulo 3 o de las PWM del capítulo 4. Cada peso w_j dice cuánto aumenta el logaritmo de las probabilidades a favor de la clase 1 por cada unidad adicional del atributo j .

De la verosimilitud a la entropía cruzada. Si las etiquetas son independientes y cada una sigue una Bernoulli con parámetro $p_i = p(\mathbf{x}_i)$, la verosimilitud es $\prod_i p_i^{y_i} (1 - p_i)^{1-y_i}$. Maximizarla equivale a minimizar el negativo de su logaritmo promediado, la *entropía cruzada binaria*:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \quad (17.6)$$

DONDE

$\mathcal{L}$	Pérdida media (en nats); vale $\ln 2 \approx 0,693$ para un clasificador que siempre dice $p = 0,5$.
p_i	Probabilidad predicha para la secuencia i .

Derivación del gradiente. No existe una fórmula cerrada para el mínimo de la ecuación (17.6), pero su gradiente es sorprendentemente simple. Tomemos una sola observación, $\ell = -[y \log p + (1 - y) \log(1 - p)]$ con $p = \sigma(z)$ y $z = \mathbf{w}^\top \mathbf{x} + b$. Por la regla de la cadena,

$$\frac{\partial \ell}{\partial w_j} = \frac{\partial \ell}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial w_j}.$$

Los tres factores son: $\partial \ell / \partial p = -y/p + (1 - y)/(1 - p) = (p - y)/[p(1 - p)]$; la derivada de la sigmoide, que cumple la identidad $\sigma'(z) = \sigma(z)[1 - \sigma(z)] = p(1 - p)$; y $\partial z / \partial w_j = x_j$. Al multiplicarlos, el factor $p(1 - p)$ se cancela:

Teorema 17.1. (Gradiente de la regresión logística)

Para la pérdida de entropía cruzada de la ecuación (17.6),

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) \mathbf{x}_i, \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i). \quad (17.7)$$

Además $\mathcal{L}$ es convexa en $(\mathbf{w}, b)$, de modo que cualquier mínimo local es global.

DONDE

$p_i - y_i$ Error de predicción (residuo) de la observación i : positivo si el modelo sobreestima.

$\nabla_{\mathbf{w}} \mathcal{L}$ Vector de derivadas parciales de la pérdida respecto a cada peso.

El resultado es tan intuitivo como elegante: cada observación “empuja” los pesos en la dirección de sus atributos, con una fuerza proporcional a cuánto se equivocó. Las secuencias bien clasificadas ($p_i \approx y_i$) casi no contribuyen. El algoritmo de *descenso por gradiente* itera

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(t)}), \quad (17.8)$$

DONDE

η Tasa de aprendizaje (tamaño del paso).

t Número de iteración.

y, en la práctica, se añade una penalización $\Omega = \frac{1}{2} \|\mathbf{w}\|^2$ (regularización L_2) que evita que los pesos crezcan sin límite cuando los datos son separables y que, con miles de k -mers como atributos, reduce el sobreajuste. El gradiente simplemente gana un término $\lambda \mathbf{w}$.

Ejemplo 17.2. (Un paso de descenso por gradiente)

Cuatro secuencias se describen con dos atributos: x_1 , número de copias de un motivo, y x_2 , número de dinucleótidos CpG. Los datos son (2, 1), (1, 0), (0, 1) y (0, 0), con etiquetas 1, 1, 0 y 0. Partimos de $\mathbf{w} = \mathbf{o}$, $b = 0$, así que

$p_i = \sigma(0) = 0,5$ para todas y $\mathcal{L} = \ln 2 = 0,6931$. Los residuos son $p - y = (-0,5, -0,5, 0,5, 0,5)$ y, por la ecuación (17.7),

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w_1} &= \frac{1}{4}[(-0,5)(2) + (-0,5)(1) + 0 + 0] = -0,375, \\ \frac{\partial \mathcal{L}}{\partial w_2} &= \frac{1}{4}[(-0,5)(1) + 0 + (0,5)(1) + 0] = 0, \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{4} \sum_i (p_i - y_i) = 0.\end{aligned}$$

Con $\eta = 1$, los nuevos parámetros son $w_1 = 0,375$, $w_2 = 0$, $b = 0$. Las probabilidades pasan a $\sigma(0,75) = 0,679$, $\sigma(0,375) = 0,593$, $0,5$ y $0,5$, y la pérdida baja a $0,5741$. El modelo aprendió en un paso que el motivo discrimina y que los CpG, en este ejemplo, no aportan nada: aparecen igual en una secuencia positiva y en una negativa.

17.1.4 Máquinas de vectores de soporte

La regresión logística traza *una* frontera lineal, la que maximiza la verosimilitud. Cuando las clases son separables hay infinitas fronteras que separan perfectamente los datos de entrenamiento; ¿cuál elegir? Cortes y Vapnik (1995) propusieron la que deja el mayor *margen*, es decir, la franja vacía más ancha entre las dos clases. La intuición es que una frontera alejada de todos los ejemplos es más robusta ante secuencias nuevas ligeramente distintas.

Con etiquetas $y_i \in \{-1, +1\}$, la anchura del margen de la frontera $\mathbf{w}^\top \mathbf{x} + b = 0$ es $2/\|\mathbf{w}\|$. Permitiendo que algunos puntos violen el margen (datos no separables), el problema de la SVM de margen blando es

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{sujeto a} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \quad (17.9)$$

DONDE

$\ \mathbf{w}\ $	Norma de los pesos; minimizarla ensancha el margen.
ξ_i	Holgura: cuánto invade el punto i el margen o cruza la frontera.
C	Compromiso entre margen ancho (C pequeño) y pocos errores de entrenamiento (C grande).

Eliminando las holguras, la ecuación (17.9) equivale a minimizar $\sum_i \max\{0, 1 - y_i f(\mathbf{x}_i)\} + \frac{1}{2C} \|\mathbf{w}\|^2$: la pérdida *hinge* con regularización L_2 . Se diferencia de la logística en que la *hinge* es exactamente cero para los puntos bien clasificados fuera del

margen. Por eso, al resolver el problema dual, la solución depende solo de los puntos que tocan o invaden el margen, los *vectores de soporte*:

$$f(\mathbf{x}) = \sum_{i \in \mathcal{S}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b, \quad 0 < \alpha_i \leq C. \quad (17.10)$$

DONDE

$\mathcal{S}$	Conjunto de vectores de soporte (los α_i no nulos).
α_i	Multiplicadores de Lagrange obtenidos del problema dual.
$K(\mathbf{x}_i, \mathbf{x})$	Núcleo: producto interno, quizá en un espacio de atributos implícito.

La ecuación (17.10) contiene el famoso *truco del núcleo*: los datos solo aparecen a través de productos internos, de modo que podemos sustituir $\mathbf{x}_i^\top \mathbf{x}$ por cualquier núcleo válido, como el de espectro de la ecuación (17.4), y obtener una SVM que trabaja en el espacio de 4^k dimensiones sin construirlo nunca. La figura 17.2 compara ambas fronteras en un problema bidimensional.

17.1.5 Árboles de decisión y random forests

Un *árbol de decisión* clasifica mediante una cascada de preguntas binarias (“¿el conteo de TGA es mayor que 2?”). En cada nodo elige el atributo y el umbral que más reducen la *impureza* de los grupos resultantes, típicamente medida por el índice de Gini:

$$G(\text{nodo}) = \sum_c \hat{p}_c (1 - \hat{p}_c), \quad (17.11)$$

DONDE

$\hat{p}_c$	Fracción de las secuencias del nodo que pertenecen a la clase c .
G	Impureza de Gini: o si el nodo es puro, máxima si las clases están mezcladas a partes iguales.

Los árboles son interpretables y capturan interacciones no lineales, pero son inestables: pequeños cambios en los datos producen árboles muy distintos. Es decir, tienen *alta varianza*. Breiman (2001) propuso atacar precisamente esa debilidad con los *random forests*: entrenar B árboles profundos, cada uno sobre una muestra *bootstrap* de los datos y eligiendo en cada nodo el mejor atributo entre un subconjunto aleatorio, y promediar sus votos. ¿Por qué funciona? Si cada árbol tiene varianza

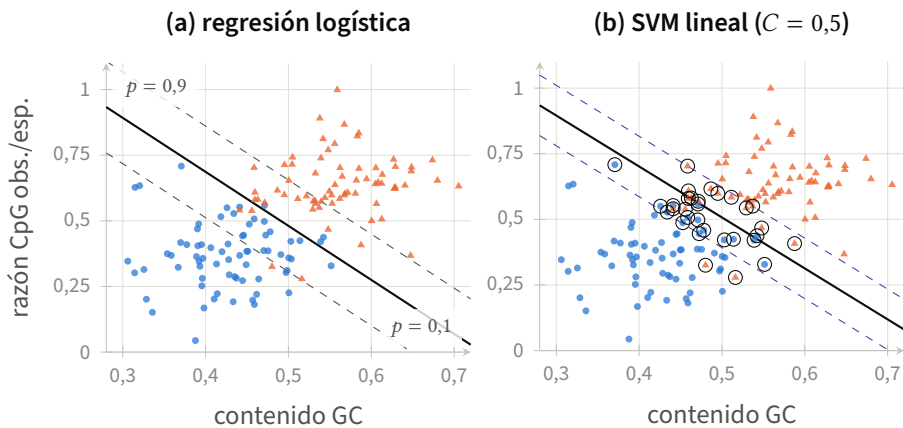


Figura 17.2. Dos clasificadores lineales sobre datos simulados: 70 regiones “de fondo” (círculos azules) y 70 promotores con isla CpG (triángulos naranjas), descritos por su contenido de GC y su razón CpG observada/esperada. **(a)** La regresión logística traza la frontera $p = 0,5$ (línea continua) y, en trazos, las curvas de nivel $p = 0,9$ y $p = 0,1$: la franja entre ellas es la zona de incertidumbre. **(b)** La SVM lineal maximiza el margen (trazos violetas, $f = \pm 1$); solo los 31 vectores de soporte rodeados con un círculo determinan la frontera, y mover cualquier otro punto sin cruzar el margen no la cambia. Las dos fronteras son casi paralelas y aciertan en el 95 % (logística) y el 95,7 % (SVM) de los datos de entrenamiento. Los atributos se estandarizaron antes de ajustar y las rectas se transformaron de vuelta a las unidades originales.

σ^2 y la correlación entre dos árboles es ρ , la varianza del promedio es (Hastie et al., 2009)

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x})\right) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2. \quad (17.12)$$

DONDE

$T_b(\mathbf{x})$	Predicción del árbol b para la entrada $\mathbf{x}$.
B	Número de árboles del bosque.
ρ	Correlación entre las predicciones de dos árboles distintos.
σ^2	Varianza de la predicción de un árbol individual.

El segundo término desaparece al aumentar B , pero el primero no: la única forma de reducirlo es *decorrelacionar* los árboles, que es exactamente lo que logra la selección aleatoria de atributos. Como regalo adicional, cada árbol deja fuera alrededor de un tercio de los datos (los que no salieron en su *bootstrap*), lo que permite estimar el error de generalización “fuera de la bolsa” (*out-of-bag*) sin un conjunto de prueba separado (Breiman, 2001).

17.1.6 Sesgo, varianza y complejidad

Para la pérdida cuadrática, el error esperado de un modelo en un punto $\mathbf{x}$, promediado sobre todos los conjuntos de entrenamiento posibles, se descompone exactamente en tres términos (Hastie et al., 2009):

$$\mathbb{E}\left[(y - \hat{f}(\mathbf{x}))^2\right] = \underbrace{\sigma_\varepsilon^2}_{\text{ruido}} + \underbrace{(\mathbb{E}[\hat{f}(\mathbf{x})] - f(\mathbf{x}))^2}_{\text{sesgo}^2} + \underbrace{\mathbb{E}\left[(\hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})])^2\right]}_{\text{varianza}}. \quad (17.13)$$

DONDE

$f(\mathbf{x})$	Función verdadera; $y = f(\mathbf{x}) + \varepsilon$.
$\hat{f}(\mathbf{x})$	Modelo ajustado; es aleatorio porque depende de la muestra de entrenamiento.
σ_ε^2	Varianza del ruido irreducible ε (errores experimentales, etiquetas ambiguas).

Un modelo demasiado simple (un árbol de profundidad 1) no puede representar la frontera verdadera: sesgo alto. Uno demasiado flexible (un árbol que crece hasta dejar una hoja por secuencia) reproduce el ruido de su muestra concreta: varianza alta. La figura 17.3 muestra el compromiso con árboles de profundidad creciente.

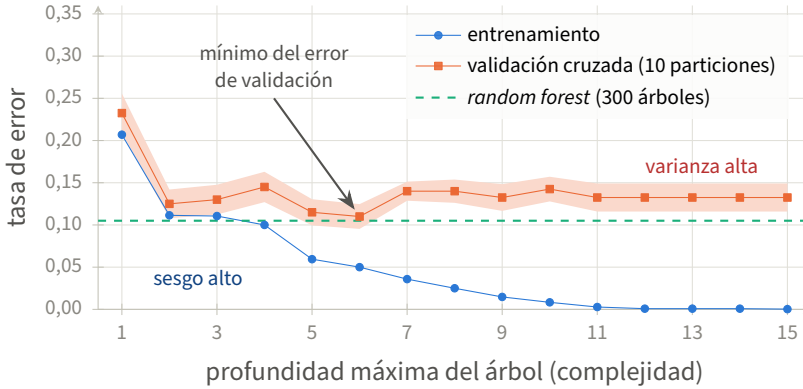


Figura 17.3. El compromiso sesgo-varianza con árboles de decisión sobre 400 puntos simulados con dos clases entrelazadas y ruido. El error de entrenamiento (azul) disminuye sin cesar con la profundidad, hasta llegar a cero; el de validación cruzada (naranja, con banda de ± 1 error estándar) primero baja, porque el modelo gana capacidad (menos sesgo), y luego sube, porque empieza a ajustar el ruido (más varianza). La distancia creciente entre ambas curvas es la firma del sobreajuste. Un *random forest* (línea verde) promedia árboles profundos decorrelacionados y consigue un error menor que el mejor árbol individual.

El error de entrenamiento baja monótonamente hasta cero, pero el de validación cruzada dibuja una “U” con mínimo en profundidad 6 (error 0,110); con profundidad 15 el árbol memoriza por completo el entrenamiento y su error de validación sube a 0,132. Un *random forest* de árboles profundos alcanza 0,105 sin necesidad de ajustar la profundidad: el promedio de la ecuación (17.12) se encarga de la varianza.

17.1.7 Validación cruzada y la fuga de información por homología

Para estimar el riesgo esperado de la definición 17.1 sin gastar datos, la *validación cruzada* de K particiones divide el conjunto en K bloques $F_1, \dots, F_K$, entrena K veces dejando fuera uno cada vez y promedia los errores:

$$\widehat{CV}_K = \frac{1}{K} \sum_{k=1}^K \frac{1}{|F_k|} \sum_{i \in F_k} \ell(y_i, \hat{f}^{(-k)}(\mathbf{x}_i)). \quad (17.14)$$

DONDE

F_k	Índices de las observaciones del bloque k (usadas como prueba en la ronda k).
$\hat{f}^{(-k)}$	Modelo entrenado con todos los bloques excepto F_k .
K	Número de particiones (habitualmente 5 o 10).

La ecuación (17.14) es un estimador insesgado del error de generalización *solo si* las observaciones de prueba son independientes de las de entrenamiento. En biología, esa suposición se rompe casi siempre, porque las secuencias no son independientes: están emparentadas por la evolución. Si una globina de ratón queda en el entrenamiento y la de rata en la prueba, el modelo no necesita haber aprendido nada sobre globinas; le basta con reconocer a la prima. Jones (2019) advirtió que la separación insuficiente entre los datos de entrenamiento y de prueba es un problema recurrente en los trabajos de ML aplicados a biología, y las recomendaciones DOME (Walsh et al., 2021), acrónimo de datos, optimización, modelo y evaluación, exigen que todo trabajo declare explícitamente cómo se garantizó la independencia entre los conjuntos de entrenamiento y de prueba.

La figura 17.4 cuantifica el problema con un experimento diseñado para no tener ninguna señal biológica. Simulamos 60 familias de 15 secuencias de 200 pb, cada una derivada de un ancestro aleatorio con un 15 % de sustituciones, y asignamos una etiqueta *al azar* a cada familia completa. Ningún clasificador puede superar el 50 % de exactitud con secuencias de familias nuevas. Sin embargo, con una partición aleatoria de las secuencias el vecino más cercano (1-NN) alcanza un 96,9 % y un *random forest* un 87,8 %. Cuando la partición mantiene cada familia completa en un mismo bloque, las cifras caen a 52,9 % y 47,0 %, alrededor del azar, que es la verdad.

⚠ Cómo evitar la fuga de información

1. *Agrupe antes de partir.* Agrupe las secuencias por similitud (por ejemplo, con MMseqs2 o CD-HIT al 30–40 % de identidad para proteínas) y use los grupos como unidad de partición (GroupKFold en scikit-learn).
2. *Preprocese dentro de cada partición.* Seleccionar los k -mers más discriminativos, estandarizar o imputar usando *todos* los datos antes de la validación cruzada filtra información de la prueba. Encadene los pasos en un Pipeline.

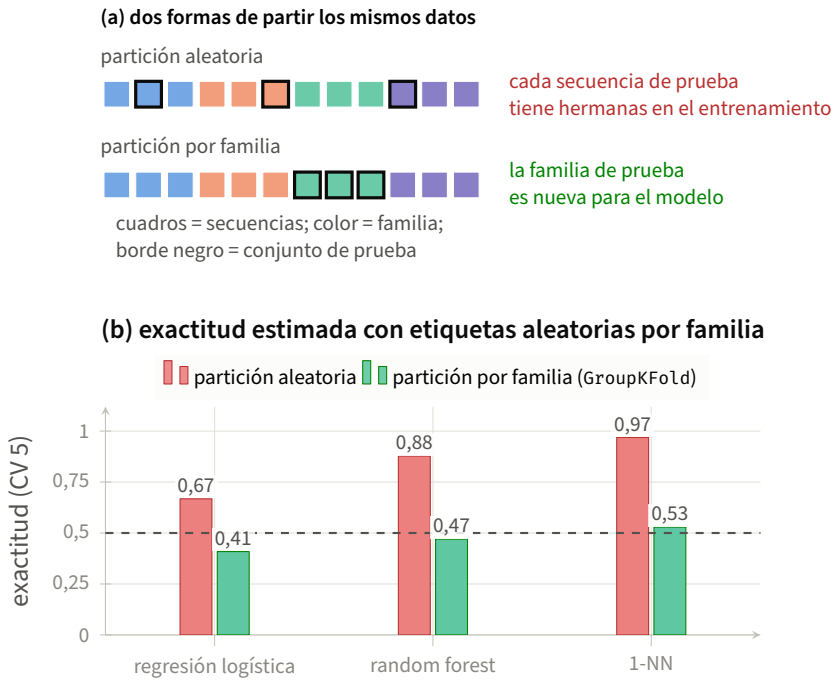


Figura 17.4. La fuga de información por homología infla la exactitud. **(a)** En una partición aleatoria, cada secuencia de prueba (borde negro) tiene parientes cercanos en el entrenamiento; en una partición por familia, la familia de prueba es completamente nueva. **(b)** Con 60 familias simuladas cuyas etiquetas se asignaron *al azar*, el valor correcto de la exactitud es 0,5 (línea de trazos). La partición aleatoria produce estimaciones espectaculares, sobre todo para los métodos que memorizan (1-NN, *random forest*); la partición por familia devuelve cifras compatibles con el azar. Los valores algo inferiores a 0,5 se deben a que, al retirar familias enteras, la proporción de clases del entrenamiento se desplaza en sentido contrario a la del bloque de prueba. Atributos: espectro de 3-mers normalizado.

- 3. *Reserve un conjunto de prueba final* que no se toque durante la selección de hiperparámetros; la validación cruzada anidada estima el error del procedimiento completo.
- 4. *Piense en el uso real.* Si el modelo se aplicará a especies no vistas, particione por especie; si se aplicará a datos futuros, particione por fecha.

17.1.8 Métricas para clases desbalanceadas

En genómica, los positivos suelen ser raros: menos del 1 % de las posiciones de un genoma son sitios de unión de un factor dado, y la mayoría de las variantes de sentido erróneo son benignas. Con un 1 % de positivos, un clasificador que siempre dice “negativo” tiene un 99 % de exactitud y no sirve para nada. Necesitamos métricas que miren por separado a cada clase.

Para un umbral de decisión τ , sean VP, FP, VN y FN los verdaderos y falsos positivos y negativos. La *curva ROC* representa la tasa de verdaderos positivos $TPR = VP/(VP + FN)$ (sensibilidad o exhaustividad) frente a la de falsos positivos $FPR = FP/(FP + VN)$ al variar τ . Ambas tasas se calculan *dentro* de cada clase, así que la curva ROC no depende de la proporción de positivos. La *curva de precisión-exhaustividad* (PR) representa, en cambio, la precisión $VP/(VP + FP)$, que mezcla las dos clases. Por el teorema de Bayes, con prevalencia π ,

$$\text{Precisión}(\tau) = \frac{TPR(\tau) \pi}{TPR(\tau) \pi + FPR(\tau) (1 - \pi)} \tag{17.15}$$

DONDE	
π	Prevalencia: fracción de positivos en la población donde se aplicará el modelo.
TPR, FPR	Tasas de verdaderos y falsos positivos al umbral τ .
τ	Umbral sobre la puntuación del clasificador a partir del cual se predice “positivo”.

Cuando π es pequeño, el término $FPR(1 - \pi)$ domina el denominador y la precisión se hunde, aunque la curva ROC no cambie. Saito y Rehmsmeier (2015) mostraron, con simulaciones y revisando la literatura, que en conjuntos desbalanceados la curva PR es más informativa que la ROC y que la línea base de la curva PR no es 0,5 sino π .

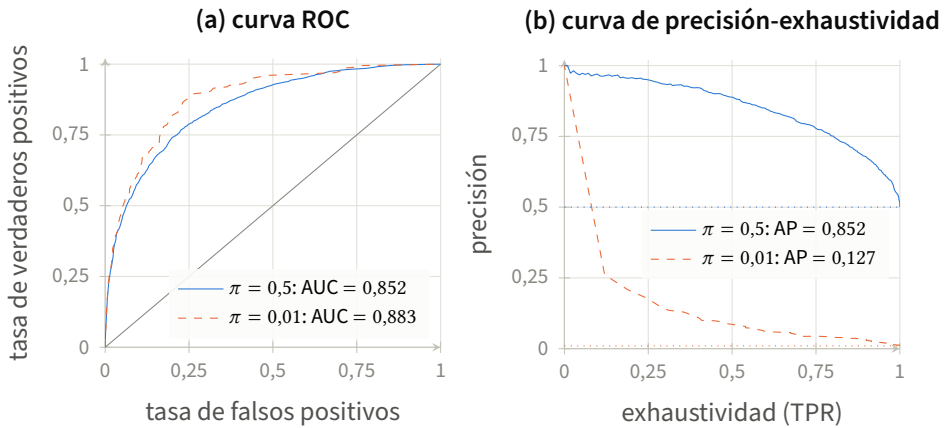


Figura 17.5. El mismo clasificador evaluado con dos prevalencias. Las puntuaciones de positivos y negativos siguen normales $\mathcal{N}(1,5, 1)$ y $\mathcal{N}(0, 1)$ (AUC teórica 0,856). En azul, 2000 positivos y 2000 negativos; en naranja, 200 positivos entre 20 000 secuencias. **(a)** Las curvas ROC son prácticamente iguales: la ROC es ciega a la prevalencia, y la pequeña diferencia de AUC es fluctuación muestral de los 200 positivos. **(b)** Las curvas PR son radicalmente distintas: con $\pi = 0,01$ la precisión media (AP) cae de 0,85 a 0,13. Las líneas punteadas marcan el rendimiento de un clasificador aleatorio en cada caso (precisión = π). Cuando los positivos son raros, informe siempre la curva PR.

Ejemplo 17.3. (Un clasificador excelente que casi siempre se equivoca)

Un predictor de sitios de unión tiene $\text{TPR} = 0,90$ y $\text{FPR} = 0,05$ a su umbral por defecto: parece muy bueno. Si lo aplicamos a regiones donde solo el 1 % son sitios reales ($\pi = 0,01$), la ecuación (17.15) da

$$\text{Precisión} = \frac{0,90 \times 0,01}{0,90 \times 0,01 + 0,05 \times 0,99} = \frac{0,009}{0,0585} \approx 0,154.$$

De cada 100 predicciones positivas, unas 85 son falsas. El mismo clasificador, con clases equilibradas ($\pi = 0,5$), tendría una precisión de 0,947.

17.1.9 Todo junto con scikit-learn

La biblioteca scikit-learn (Pedregosa et al., 2011) ofrece una interfaz uniforme (`fit`, `predict`, `predict_proba`) para todos los modelos de esta sección, y herramientas para encadenar el preprocesamiento con el modelo de modo que la validación

cruzada no filtre información. El siguiente fragmento construye un clasificador de secuencias con espectro de 3-mers y regresión logística, y lo evalúa con particiones por familia y precisión media:

```
clasificar_kmers.py

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GroupKFold, cross_val_score
from sklearn.pipeline import make_pipeline

# seqs: lista de str; y: etiquetas 0/1; fam: clúster de homología
modelo = make_pipeline(
    CountVectorizer(analyzer="char", ngram_range=(3, 3),
                   lowercase=False),          # espectro de 3-mers
    LogisticRegression(C=1.0, max_iter=5000)) # regularización L2
cv = GroupKFold(n_splits=5)                  # familias separadas
ap = cross_val_score(modelo, seqs, y, groups=fam, cv=cv,
                     scoring="average_precision")
print(f"AP = {ap.mean():.3f} +/- {ap.std():.3f}")
```

Como el vectorizador forma parte del *pipeline*, el vocabulario de *k*-mers se aprende de nuevo en cada partición usando solo los datos de entrenamiento. Para cambiar de modelo basta con sustituir la regresión logística por una SVM lineal (`SVC(kernel="linear")`) o por un *random forest* (`RandomForestClassifier()`); no hay que cambiar nada más.

En ML para biología, la pregunta decisiva no es “¿qué modelo uso?”, sino “¿a qué secuencias *nuevas* se parece mi conjunto de prueba?”.

17.2 Deep learning para ADN

El corrector de pruebas con una plantilla

Un corrector de imprenta busca una palabra prohibida en un libro. No lee cada página de principio a fin: desliza una pequeña plantilla transparente con la palabra impresa sobre cada renglón y se detiene donde las letras coinciden. No importa en qué renglón esté la palabra; la plantilla la encuentra igual. Si le damos varias plantillas, una por palabra, y anotamos para cada una “¿apareció en algún lugar de la página?”, habremos resumido la página en unas pocas respuestas. Una red convolucional hace eso con el ADN, con una diferencia decisiva: nadie le da las plantillas; las aprende.

Los métodos de la sección anterior requieren que *nosotros* elijamos los atributos: k -mers, contenido de GC, conteos de motivos. El aprendizaje profundo (*deep learning*) propone aprender también la representación, apilando capas de transformaciones simples que se ajustan todas a la vez a partir de los datos (LeCun et al., 2015). En genómica, esta idea transformó la predicción de la actividad regulatoria a partir de la secuencia (Eraslan et al., 2019).

17.2.1 Del perceptrón a las redes profundas

La unidad básica es la neurona artificial de Rosenblatt (1958), que es, esencialmente, la regresión logística de la ecuación (17.5) con otra función de activación. Una red *multicapa* compone muchas de ellas:

$$\mathbf{a}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \varphi(\mathbf{z}^{(l)}), \quad l = 1, \dots, D, \quad (17.16)$$

DONDE

$\mathbf{a}^{(l)}$	Activaciones (salidas) de la capa l ; $\mathbf{a}^{(0)}$ es la entrada.
$\mathbf{W}^{(l)}, \mathbf{b}^{(l)}$	Matriz de pesos y vector de sesgos de la capa l .
$\mathbf{z}^{(l)}$	Preactivaciones: combinaciones lineales antes de la no linealidad.
φ	Función de activación no lineal, aplicada elemento a elemento; hoy casi siempre $\text{ReLU}(z) = \max(0, z)$.
D	Profundidad: número de capas.

y en la última capa se usa una sigmoide (clasificación binaria) o una *softmax* (varias clases). La no linealidad φ es imprescindible: sin ella, la composición de capas lineales sería otra transformación lineal y la red entera no sería más expresiva que una regresión logística. Con ella, una red con una sola capa oculta suficientemente

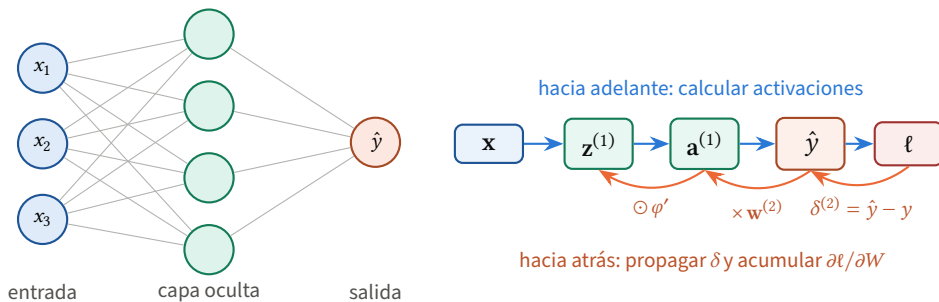


Figura 17.6. **Izquierda:** una red con tres entradas, una capa oculta de cuatro neuronas, $\mathbf{a}^{(1)} = \varphi(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$, y una salida $\hat{y} = \sigma(\mathbf{w}^{(2)\top}\mathbf{a}^{(1)} + b^{(2)})$ (ecuación (17.16)); cada línea es un peso ajustable. **Derecha:** el mismo cálculo como grafo computacional. La pasada hacia adelante (azul) evalúa las activaciones y la pérdida ℓ ; la pasada hacia atrás (naranja) propaga la “señal de error” δ desde la pérdida hasta la entrada, multiplicando en cada paso por la derivada local (teorema 17.2). El coste de la pasada hacia atrás es del mismo orden que el de la pasada hacia adelante.

ancha puede aproximar cualquier función continua; las redes *profundas* consiguen lo mismo con muchas menos neuronas, porque reutilizan en las capas altas los rasgos que detectan las capas bajas.

17.2.2 Retropropagación

Entrenar una red es, otra vez, minimizar una pérdida por descenso de gradiente (ecuación (17.8)). El problema es calcular el gradiente respecto de millones de pesos de forma eficiente. Rumelhart et al. (1986) popularizaron la solución, la *retropropagación* (*backpropagation*), que no es más que la regla de la cadena aplicada con un orden inteligente.

Derivémosla para la red de la figura 17.6, con salida sigmoide y entropía cruzada. De la teorema 17.1 ya sabemos que, en la última capa, $\partial\ell/\partial z^{(2)} = \hat{y} - y \equiv \delta^{(2)}$, y por lo tanto $\partial\ell/\partial \mathbf{w}^{(2)} = \delta^{(2)}\mathbf{a}^{(1)}$. Para bajar a la capa oculta aplicamos la regla de la cadena a través de $z^{(2)} = \mathbf{w}^{(2)\top}\mathbf{a}^{(1)} + b^{(2)}$ y de $\mathbf{a}^{(1)} = \varphi(\mathbf{z}^{(1)})$:

$$\frac{\partial\ell}{\partial z_j^{(1)}} = \frac{\partial\ell}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial a_j^{(1)}} \cdot \frac{\partial a_j^{(1)}}{\partial z_j^{(1)}} = \delta^{(2)} w_j^{(2)} \varphi'(z_j^{(1)}).$$

Llamando $\delta_j^{(1)}$ a esta cantidad, el gradiente de los pesos de la primera capa es $\partial\ell/\partial W_{jk}^{(1)} = \delta_j^{(1)} x_k$. El patrón se repite en cualquier número de capas:

Teorema 17.2. (Retropropagación)

Definiendo el error de la capa l como $\delta^{(l)} = \partial \ell / \partial \mathbf{z}^{(l)}$, se cumple la recurrencia hacia atrás

$$\delta^{(l)} = \left(W^{(l+1)\top} \delta^{(l+1)} \right) \odot \varphi'(\mathbf{z}^{(l)}), \quad \frac{\partial \ell}{\partial W^{(l)}} = \delta^{(l)} \mathbf{a}^{(l-1)\top}, \quad \frac{\partial \ell}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \quad (17.17)$$

Con una pasada hacia adelante (que guarda $\mathbf{z}^{(l)}$ y $\mathbf{a}^{(l)}$) y una hacia atrás se obtienen todas las derivadas con un coste del mismo orden que evaluar la red una vez.

DONDE

$\delta^{(l)}$	Sensibilidad de la pérdida a las preactivaciones de la capa l .
$\odot$	Producto elemento a elemento (de Hadamard).
φ'	Derivada de la activación; para ReLU vale 1 si $z > 0$ y 0 si $z < 0$.

La alternativa ingenua, perturbar cada peso y medir cómo cambia la pérdida, costaría una evaluación completa de la red *por cada peso*. La retropropagación reduce ese factor de millones a dos. Las bibliotecas modernas (PyTorch, JAX, TensorFlow) implementan la ecuación (17.17) automáticamente para cualquier composición de operaciones diferenciables (*diferenciación automática*); en la práctica se usan pequeños lotes (*minibatches*) de datos para estimar el gradiente en cada paso, lo que se conoce como descenso por gradiente estocástico.

17.2.3 Convolución sobre ADN: un filtro es una PWM

Una red densa sobre la matriz *one-hot* de 1000 pb necesitaría pesos distintos para “el motivo en la posición 17” y “el motivo en la posición 18”. Las *redes convolucionales* (CNN) imponen, en cambio, que el mismo detector se aplique en todas las posiciones. Una capa convolucional unidimensional con F filtros de ancho w calcula

$$h_{k,i} = \text{ReLU} \left(b_k + \sum_{j=1}^w \sum_{c \in \{A, C, G, T\}} W_{k,c,j} X_{c,i+j-1} \right), \quad (17.18)$$

con $k = 1, \dots, F$ e $i = 1, \dots, L - w + 1$,

DONDE

$W_{k,c,j}$	Peso del filtro k para la base c en la posición j de la ventana ($4 \times w$ números por filtro).
b_k	Sesgo del filtro k ; actúa como umbral de detección.
$h_{k,i}$	Activación del filtro k cuando la ventana empieza en la posición i de la secuencia.
F, w	Número de filtros y ancho de cada uno (en pb).

Como X es *one-hot*, en la suma interior solo sobrevive el término de la base observada, y la ecuación (17.18) se simplifica de un modo revelador:

Teorema 17.3. *(Un filtro convolucional es una matriz de pesos posicionales)*

Sobre una secuencia codificada en one-hot,

$$h_{k,i} = \text{ReLU} \left(b_k + \sum_{j=1}^w W_{k, x_{i+j-1}, j} \right). \quad (17.19)$$

Si $W_{k,c,j} = \log(p_{j,c}/q_c)$ es la matriz de log-probabilidades de una PWM y $b_k = -\tau$, la activación es exactamente la puntuación de la PWM en la ventana i , recortada por debajo del umbral τ .

DONDE

x_{i+j-1}	Base observada en la posición j de la ventana.
$p_{j,c}, q_c$	Frecuencia de la base c en la posición j del motivo y en el fondo (capítulo 4).
τ	Umbral de puntuación por debajo del cual el filtro “no ve” nada.

Este resultado explica por qué las CNN tuvieron tanto éxito con el ADN regulatorio: la primera capa es un banco de PWM que se aprenden solas, sin necesidad de alinear sitios conocidos. Tras la convolución, una operación de *max-pooling* resume cada filtro:

$$m_k = \max_i h_{k,i}, \quad (17.20)$$

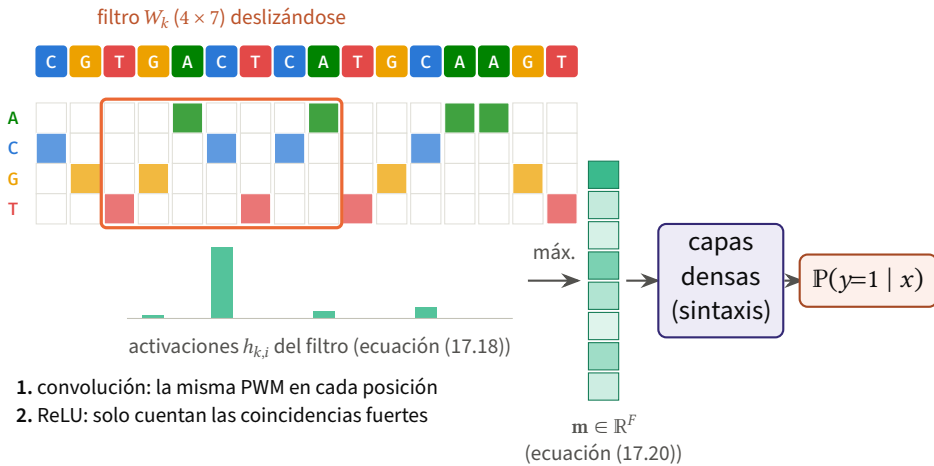


Figura 17.7. Anatomía de una CNN para ADN. La secuencia se codifica en *one-hot* (matriz $4 \times L$). Cada filtro, una matriz $4 \times w$ de pesos, se desliza por la secuencia y produce una pista de activaciones (verde), que se dispara donde la ventana se parece al motivo que el filtro representa (aquí **TGA**CTCA, recuadrado). El *max-pooling* resume cada pista en un número, de modo que la detección no depende de la posición; las capas densas combinan las detecciones de todos los filtros y producen la probabilidad final. Esquema ilustrativo; las alturas de las barras no provienen de un modelo entrenado.

DONDE

m_k

Respuesta máxima del filtro k en toda la secuencia: “¿aparece el motivo k en algún lugar?”.

lo que hace que la predicción sea invariante a la posición del motivo. Las capas siguientes combinan esas detecciones (“motivo 1 y motivo 3 a menos de 20 pb”) y aprenden la sintaxis regulatoria. La figura 17.7 resume la arquitectura.

Ejemplo 17.4. (Un filtro a mano)

Consideremos un filtro de ancho $w = 3$ diseñado para **TGA**: pesos $+2$ para la base de consenso en cada posición y -1 para las demás, con sesgo $b = -4$. Sobre la ventana **TGA**, la ecuación (17.19) da $\text{ReLU}(-4 + 2 + 2 + 2) = 2$. Sobre **TCA** (una discrepancia), $\text{ReLU}(-4 + 2 - 1 + 2) = \text{ReLU}(-1) = 0$. El sesgo actúa como umbral: este filtro solo “ve” coincidencias perfectas. Con $b = -2$ toleraría una discrepancia, pues **TCA** daría $\text{ReLU}(1) = 1$.

¿Cuántos parámetros tiene la red que entrenaremos a continuación? La convolución tiene $F = 8$ filtros de 4×11 pesos más un sesgo cada uno: $8 \times (44 + 1) = 360$. La capa de salida combina los 8 máximos con 8 pesos y un sesgo: 9. En total, 369 parámetros; DeepSEA o Basset tienen millones.

17.2.4 Un experimento: una CNN aprende un motivo

Para ver el teorema 17.3 en acción, simulamos 6000 secuencias aleatorias de 100 pb. En la mitad, elegidas al azar, insertamos en una posición aleatoria un sitio muestreado de una PWM de tipo AP-1 (consenso **TGASTCA**, donde S es C o G; probabilidad 0,91 para la base de consenso en las posiciones no degeneradas). Entrenamos la red del ejemplo 17.4 con 4000 secuencias, usamos 1000 para validación y reservamos 1000 para la prueba final.

```
cnn_motivo.py

import torch
import torch.nn as nn

class CNNMotivo(nn.Module):
    def __init__(self, n_filtros=8, ancho=11):
        super().__init__()
        self.conv = nn.Conv1d(4, n_filtros, ancho) # canales ACGT
        self.fc = nn.Linear(n_filtros, 1)

    def forward(self, x):
        h = torch.relu(self.conv(x)) # x: (lote, 4, L)
        m = h.max(dim=2).values      # (lote, F, L - w + 1)
        return self.fc(m).squeeze(1) # max-pooling global
                                     # logit

red = CNNMotivo()
opt = torch.optim.Adam(red.parameters(), lr=3e-3)
perdida = nn.BCEWithLogitsLoss() # entropía cruzada
for epoca in range(40):
    for xb, yb in cargador:      # lotes de 64 secuencias
        opt.zero_grad()
        perdida(red(xb), yb).backward() # retropropagación
        opt.step()
```

Tras 40 épocas, la red alcanza en el conjunto de prueba un AUROC de 0,896 y una exactitud de 0,822. Puede parecer poco para un problema “fácil”, pero el límite no está en la red, sino en los datos: muchas copias muestreadas del motivo

tienen una o dos discrepancias con el consenso, y en 100 pb aleatorios aparecen por azar palabras a una discrepancia del consenso con frecuencia apreciable. Ningún clasificador puede separar perfectamente esos casos. Las curvas de aprendizaje de la figura 17.8c muestran además el sobreajuste incipiente: la pérdida de entrenamiento (0,327) queda por debajo de la de validación (0,426).

Lo interesante es *qué* aprendió. Tomamos el filtro con mayor peso positivo en la capa de salida, localizamos en cada secuencia de validación y prueba la ventana de máxima activación y, si supera la mitad del máximo global, la añadimos a un alineamiento; con las 1115 ventanas resultantes construimos un logo, que es el procedimiento empleado por DeepBind y Basset (Alipanahi et al., 2015; Kelley et al., 2016). El resultado (figura 17.8b) es el motivo que plantamos: consenso **TGAGTCA**, con contenidos de información de 1,44 a 1,53 bits en las posiciones fijas (el valor teórico es 1,42) y 0,67 bits en la posición degenerada, exactamente lo que predice una mezcla al 50 % de C y G. Nadie le enseñó a la red qué es AP-1.

17.2.5 De DeepBind a Enformer

La historia reciente de la genómica regulatoria puede leerse como una carrera por ampliar el *contexto* que ve la red (figura 17.9).

DeepBind (Alipanahi et al., 2015) aplicó por primera vez CNN a gran escala para predecir la especificidad de proteínas de unión a ADN y ARN a partir de datos de microarreglos de unión (PBM), SELEX, ChIP-seq y CLIP-seq. Su arquitectura era esencialmente la de nuestro experimento: una capa convolucional, *pooling* y una pequeña red densa, un modelo por proteína. Introdujo además los “mapas de mutación”, que muestran cómo cambia la predicción al mutar cada base.

DeepSEA (Zhou & Troyanskaya, 2015) dio el salto a la predicción *multitarea*: una sola red que, a partir de 1000 pb, predice simultáneamente 919 perfiles de cromatina (sitios de unión de factores de transcripción, hipersensibilidad a DNasa, marcas de histonas) de ENCODE y Roadmap. Y, sobre todo, lo usó para puntuar variantes no codificantes por la diferencia entre las predicciones de los dos alelos:

$$\Delta_t(v) = f_t(x^{\text{alt}}) - f_t(x^{\text{ref}}), \quad (17.21)$$

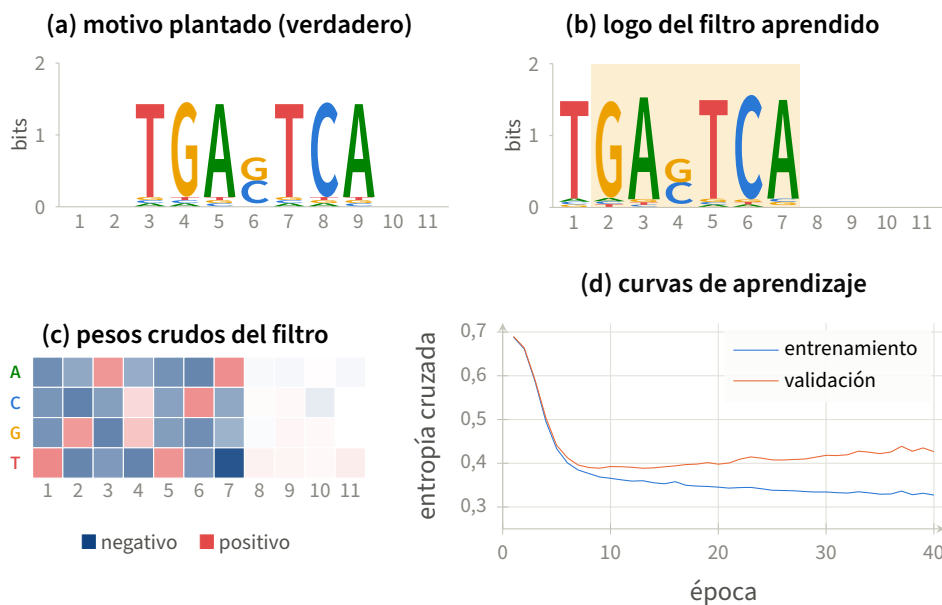


Figura 17.8. Lo que aprendió una CNN de 369 parámetros entrenada con secuencias simuladas. **(a)** Logo de la PWM de tipo AP-1 usada para plantar el motivo (flanqueado por dos columnas de fondo a cada lado para igualar el ancho del filtro). **(b)** Logo construido con las 1115 ventanas de 11 pb que más activan el filtro más relevante: las siete primeras columnas (sombreadas) reproducen el motivo, incluida la posición degenerada C/G, y las cuatro últimas quedan vacías porque el filtro, de 11 pb, es más ancho que el motivo. **(c)** Los pesos del filtro (4×11) son directamente una matriz de tipo PWM (teorema 17.3): rojo en la base de consenso, azul en las demás. **(d)** La pérdida de validación se estabiliza mientras la de entrenamiento sigue bajando: señal de que más épocas solo añadirían sobreajuste.

DONDE

$f_t(x)$	Predicción del modelo para la pista t (p. ej., unión de CTCF en células K562) con la secuencia x .
$x^{\text{ref}}, x^{\text{alt}}$	Secuencia de referencia y la misma con el alelo alternativo de la variante v .
$\Delta_t(v)$	Efecto predicho de la variante sobre la pista t .

lo que permite priorizar variantes de GWAS sin haber medido nunca su efecto. **Basset** (Kelley et al., 2016) entrenó una CNN profunda sobre sitios accesibles medidos por DNase-seq en 164 tipos celulares y mostró que sus predicciones de cambio de accesibilidad eran mucho mayores para las variantes de GWAS probablemente causales que para sus vecinas en desequilibrio de ligamiento.

El siguiente obstáculo era la distancia. Los potenciadores pueden actuar a decenas de kilobases del promotor que regulan, pero una pila de convoluciones con filtros de ancho w solo “ve” unos pocos cientos de pares de bases. **Basenji** (Kelley et al., 2018) usó *convoluciones dilatadas*, que saltan posiciones con un paso d creciente, para cubrir secuencias de 131 kb. Con D capas de ancho w y dilataciones d_l , el *campo receptivo* (el tramo de secuencia que influye en una salida) es

$$R = 1 + \sum_{l=1}^D (w-1) d_l \stackrel{d_l=2^{l-1}}{=} 1 + (w-1)(2^D - 1), \quad (17.22)$$

DONDE

R	Campo receptivo en pares de bases.
d_l	Dilatación de la capa l (cuántas posiciones salta el filtro entre elementos).
w, D	Ancho de los filtros y número de capas dilatadas.

de modo que duplicar la dilatación en cada capa hace crecer el campo receptivo de forma *exponencial* con la profundidad, en lugar de lineal: con $w = 3$ y $D = 11$ capas, $R = 1 + 2 \times 2047 = 4095$ posiciones. Finalmente, **Enformer** (Avsec et al., 2021) sustituyó las convoluciones dilatadas por capas de atención (el mecanismo de la sección 17.3) sobre secuencias de unos 200 kb, integrando información de elementos situados hasta a 100 kb de distancia, lo que mejoró de forma sustancial la predicción de la expresión génica y del efecto de variantes medido por ensayos de reporteros masivamente paralelos.

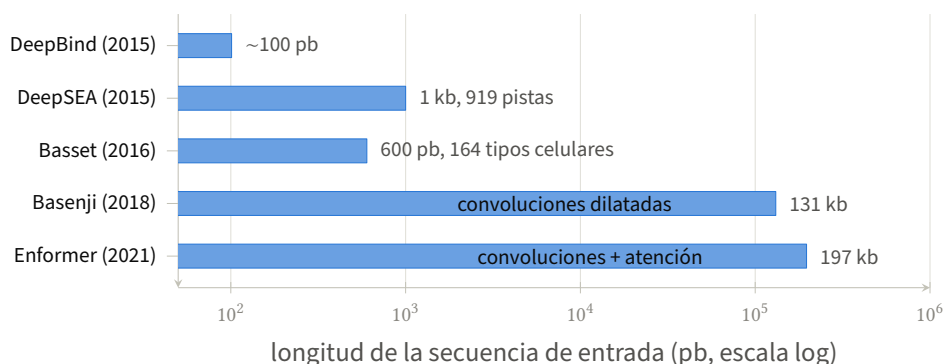


Figura 17.9. La carrera por el contexto. Longitud de la secuencia de entrada de cinco modelos emblemáticos, en escala logarítmica. DeepBind trabajaba con fragmentos del orden de 100 pb (la longitud variaba según el tipo de dato); DeepSEA y Basset, con ventanas de 1 kb y 600 pb centradas en picos o sitios accesibles; Basenji y Enformer procesan secuencias de más de cien kilobases, suficientes para incluir potenciadores distales. Cada salto de escala requirió una idea de arquitectura nueva: *pooling* y multitarea, dilatación, atención.

17.2.6 Abrir la caja negra: saliencia, DeepLIFT y TF-MoDISco

Un modelo con millones de parámetros que predice bien no explica *por qué* predice. Para convertir predicciones en hipótesis biológicas necesitamos saber qué bases de la entrada fueron responsables. El método más directo son los *mapas de saliencia* de Simonyan et al. (2013): la derivada de la salida respecto de la entrada, que la retropropagación calcula gratis. En secuencias *one-hot* se usa la variante “gradiente por entrada”:

$$s_i = \sum_c X_{c,i} \frac{\partial f(X)}{\partial X_{c,i}} = \left. \frac{\partial f}{\partial X_{c,i}} \right|_{c=x_i}, \quad (17.23)$$

DONDE

s_i	Contribución atribuida a la base observada en la posición i .
$f(X)$	Salida del modelo (<i>logit</i>) para la secuencia codificada X .

que, igual que en la ecuación (17.19), selecciona la derivada correspondiente a la base presente. La alternativa de fuerza bruta es la *mutagénesis in silico*: mutar cada posición a las tres bases alternativas y registrar Δ con la ecuación (17.21); es exacta pero cuesta $3L$ evaluaciones.

El gradiente es una aproximación *local*: mide el efecto de un cambio infinitesimal, y con activaciones ReLU saturadas puede valer cero aunque la base sea importante. DeepLIFT (Shrikumar et al., 2017) compara en su lugar la activación con la de una *secuencia de referencia* X^0 (por ejemplo, una versión barajada con la misma composición) y reparte la diferencia total entre las entradas, de modo que las contribuciones cumplen una propiedad de conservación:

$$\sum_i \sum_c C_{\Delta X_{c,i} \Delta f} = \Delta f = f(X) - f(X^0). \quad (17.24)$$

DONDE

X^0	Secuencia de referencia (“neutra”).
$C_{\Delta X_{c,i} \Delta f}$	Contribución de la diferencia en la entrada (c, i) a la diferencia en la salida.
Δf	Cambio total de la salida entre la secuencia y la referencia.

Los mapas de contribución de miles de secuencias contienen fragmentos de alta importancia (*seqlets*) que TF-MoDISco (Shrikumar et al., 2018) extrae, agrupa por similitud y consolida en motivos no redundantes. El resultado es un catálogo de motivos *aprendidos por el modelo*, que puede compararse con bases de datos como JASPAR y que a menudo incluye variantes o combinaciones de motivos que los métodos clásicos de descubrimiento no detectan.

saliencia.py

```
x = x0.clone().requires_grad_(True)      # x0: (1, 4, L) one-hot
red(x).sum().backward()                  # d logit / d x
s = (x.grad * x).sum(dim=1)[0]            # gradiente x entrada
top = s.abs().argsort(descending=True)[:10]
print("posiciones más influyentes:", (top + 1).tolist())
```

⚠ Atajos y confusores

Una red aprende *cualquier* regularidad que separe las clases, sea biológica o no. Si los positivos de ChIP-seq son más ricos en GC que los negativos elegidos al azar, la red puede aprender “GC alto” en lugar del motivo; si las secuencias positivas provienen de promotores y las negativas de regiones intergénicas, aprenderá a reconocer promotores. Elija negativos con la misma composición (por ejemplo, barajando dinucleótidos), compruebe con saliencia

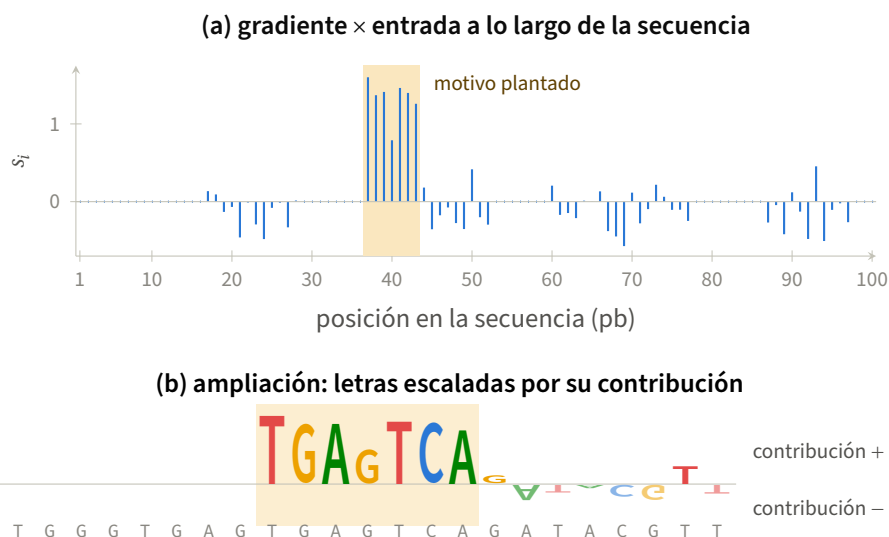


Figura 17.10. Interpretación de una predicción de la CNN de la figura 17.8 para una secuencia de prueba que contiene el sitio **TGAGTCA** en las posiciones 37–43 (sombreado). **(a)** Contribución de cada base según la ecuación (17.23). El 46 % de la contribución absoluta total se concentra en las 7 posiciones del motivo, que son el 7 % de la secuencia; el resto se reparte en las ventanas de máxima activación de los demás filtros, que también intervienen en la salida. **(b)** La misma información como “logo de contribución”: la altura de cada letra es proporcional a s_i (hacia abajo y en tono pálido si es negativa). El motivo emerge sin que el modelo lo haya visto nunca etiquetado.

que el modelo mira donde debe y, sobre todo, evalúe en cromosomas completos que no se usaron para entrenar.

🔗 El año en que la genómica descubrió las CNN

En 2012 una red convolucional profunda ganó por un margen enorme el concurso de clasificación de imágenes ImageNet, y en menos de tres años la idea saltó a la genómica. DeepBind y DeepSEA se publicaron el mismo verano de 2015 (Alipanahi et al., 2015; Zhou & Troyanskaya, 2015), y en 2016 Basset ya se distribuía como paquete de código abierto (Kelley et al., 2016). La revisión de Eraslan et al. (2019) resume la explosión posterior. Lo notable no es solo la precisión, sino que el primer filtro de estas redes, visto como un logo, redescubrió por sí solo buena parte del catálogo de motivos que la comunidad había reunido durante décadas.

17.3 Modelos de lenguaje de proteínas (ESM)



Colab · 17.3 · Modelos de lenguaje de proteínas



repositorio del curso

Completar la palabra que falta

En la frase “El gato persiguió al _____ por el jardín” casi cualquiera escribiría “ratón”. Para acertar no hace falta una regla gramatical explícita: basta haber leído mucho. Y quien acierta sistemáticamente este tipo de huecos ha aprendido, sin proponérselo, sintaxis, semántica y bastante sobre gatos. Ahora imagine el mismo juego con proteínas: ocultamos un aminoácido y pedimos adivinarlo a partir del resto. Para acertar en el núcleo hidrofóbico de una proteína globular hay que “saber” qué residuos están en contacto en la estructura tridimensional. Ese es el experimento que están haciendo, a escala evolutiva, los modelos de lenguaje de proteínas.

La evolución ha escrito cientos de millones de “textos” en el alfabeto de 20 letras de las proteínas, y la selección natural ha actuado como un editor implacable: solo sobreviven las secuencias que se pliegan y funcionan. Las restricciones estructurales y funcionales quedan grabadas en los patrones estadísticos de las secuencias. Los perfiles y los HMM del capítulo 4 explotan esos patrones dentro de *una* familia

alineada; un modelo de lenguaje intenta aprenderlos para *todas* las proteínas a la vez, sin alineamientos ni etiquetas.

17.3.1 Atención: cada residuo mira a todos los demás

Una CNN combina información local: cada filtro ve w posiciones contiguas. Pero en una proteína, los residuos 8 y 70 pueden estar en contacto directo aunque los separen 60 posiciones en la secuencia. El mecanismo de *atención* (Vaswani et al., 2017) permite que cada posición recoja información de cualquier otra en un solo paso, con pesos que dependen del contenido.

Cada posición i tiene un vector $\mathbf{h}_i \in \mathbb{R}^d$ (inicialmente, el *embedding* de su aminoácido). A partir de él se calculan tres proyecciones lineales aprendidas: una *consulta* $\mathbf{q}_i = W_Q \mathbf{h}_i$ (“qué busco”), una *clave* $\mathbf{k}_i = W_K \mathbf{h}_i$ (“qué ofrezco”) y un *valor* $\mathbf{v}_i = W_V \mathbf{h}_i$ (“qué información transmito”). Apiladas por filas en matrices Q , K y V de tamaño $L \times d_k$:

Definición 17.2. (Atención de producto escalar escalado)

$$\text{Atención}(Q, K, V) = \underbrace{\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)}_{A \in \mathbb{R}^{L \times L}} V, \quad A_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k})}{\sum_{j'=1}^L \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d_k})}.$$

(17.25)

La salida de la posición i es $\mathbf{z}_i = \sum_j A_{ij} \mathbf{v}_j$: un promedio de los valores de todas las posiciones, ponderado por la afinidad entre la consulta de i y la clave de cada j .

DONDE	
Q, K, V	Matrices de consultas, claves y valores ($L \times d_k$), una fila por posición.
W_Q, W_K, W_V	Matrices de proyección aprendidas.
d_k	Dimensión de consultas y claves.
A_{ij}	Peso de atención: cuánto atiende la posición i a la posición j ; cada fila de A suma 1.
L	Longitud de la secuencia (número de <i>tokens</i>).

¿Por qué dividir entre $\sqrt{d_k}$? Supongamos que las componentes de $\mathbf{q}$ y $\mathbf{k}$ son independientes, con media 0 y varianza 1. Entonces el producto escalar $\mathbf{q}^\top \mathbf{k} = \sum_{m=1}^{d_k} q_m k_m$ es una suma de d_k términos de media 0 y varianza $\mathbb{E}[q_m^2] \mathbb{E}[k_m^2] = 1$, y por lo tanto tiene varianza d_k . Con $d_k = 64$, los productos escalares tendrían desviación típica 8, la *softmax* se saturaría (casi todo el peso en una sola posición) y sus gradientes serían minúsculos. Dividir por $\sqrt{d_k}$ devuelve la varianza a 1 (Vaswani et al., 2017).

En la práctica se usan varias “cabezas” de atención en paralelo, cada una con sus propias proyecciones, que pueden especializarse en relaciones distintas (vecinos en la secuencia, residuos en contacto, pares de cisteínas). Un bloque *transformer* combina la atención de varias cabezas con una pequeña red densa aplicada a cada posición, conexiones residuales y normalización; un modelo apila decenas de bloques. Como la atención es invariante a permutaciones, la posición se inyecta explícitamente; ESM-2 usa codificaciones posicionales rotatorias. El coste es $O(L^2)$ en tiempo y memoria, lo que limita la longitud de las secuencias.

Ejemplo 17.5. (Atención con cuatro residuos)

Tomemos el péptido C A K C con $d_k = 2$ y, para simplificar, consultas, claves y valores ya dados:

$$Q = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0,5 & 1 \\ 2 & 0 \end{pmatrix}, \quad K = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 1 \\ 2 & 0 \end{pmatrix}, \quad V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Para la primera cisteína, $\mathbf{q}_1^\top K^\top / \sqrt{2} = (4, 0, 0, 4) / 1,414 = (2,83; 0; 0; 2,83)$. La *softmax* da $A_1 = (0,472; 0,028; 0,028; 0,472)$: la Cys1 reparte casi toda su atención entre sí misma y la Cys4, y su salida es $\mathbf{z}_1 = (0,944; 0,056)$, dominada por el “valor de cisteína”. La lisina, en cambio, tiene puntuaciones iguales con todos (0,707) y atiende uniformemente (0,25 a cada uno). La figura 17.11 muestra la matriz completa.

17.3.2 Modelado de lenguaje enmascarado

¿Cómo se entrena un modelo así sin etiquetas? Con la tarea de la intuición inicial. En el *modelado de lenguaje enmascarado* (MLM), popularizado por BERT en el procesamiento de lenguaje natural (Devlin et al., 2019), se elige al azar un 15 %

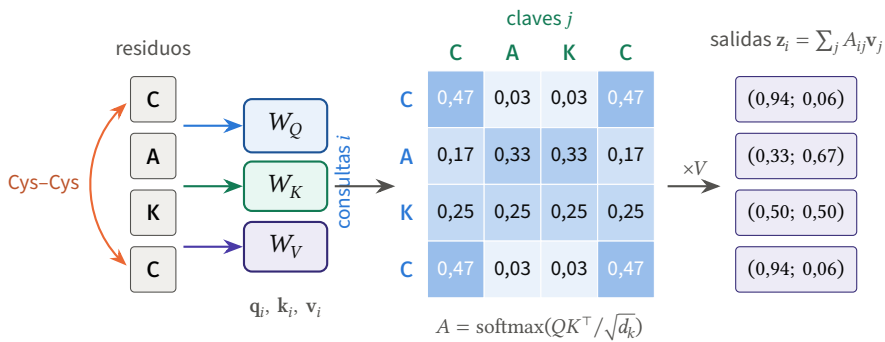


Figura 17.11. El mecanismo de atención sobre el péptido de juguete del ejemplo 17.5. Cada residuo se proyecta en una consulta, una clave y un valor. La matriz A (centro; más oscura cuanto mayor el peso) contiene, en la fila i , cómo reparte su atención la posición i entre todas las posiciones; cada fila suma 1. Las dos cisteínas se atienden mutuamente con peso 0,47 (el tipo de relación que un modelo real podría usar para representar un puente disulfuro), mientras que la lisina atiende a todas por igual. A diferencia de una convolución, la atención no tiene ventana: la distancia en la secuencia no limita qué pares pueden interactuar.

de las posiciones de cada secuencia; la mayoría se sustituye por un *token* especial $\langle \text{mask} \rangle$, algunas por un aminoácido aleatorio y otras se dejan intactas, y el modelo debe recuperar el aminoácido original a partir del contexto:

$$\mathcal{L}_{\text{MLM}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_M \left[\sum_{i \in M} \log p_{\theta}(x_i | x_{\setminus M}) \right]. \quad (17.26)$$

DONDE	
$x \sim \mathcal{D}$	Secuencia tomada de la base de datos de entrenamiento (p. ej., Uni-Ref).
M	Conjunto aleatorio de posiciones enmascaradas.
$x_{\setminus M}$	Secuencia con esas posiciones ocultas: el contexto visible.
$p_{\theta}(x_i x_{\setminus M})$	Distribución sobre los 20 aminoácidos que predice el modelo en la posición i .

La ecuación (17.26) es una entropía cruzada multiclase, la generalización directa de la ecuación (17.6), y se minimiza con retropropagación. Como cualquier secuencia proporciona sus propios “problemas” de entrenamiento, se puede aprovechar toda la diversidad de las bases de datos de secuencias: es aprendizaje *autosupervisado*. La

calidad del modelo se resume en la *perplejidad*, la exponencial de la pérdida media por posición:

$$\text{PPL} = \exp\left(-\frac{1}{L} \sum_{i=1}^L \log p_{\theta}(x_i | x_{\setminus i})\right), \quad (17.27)$$

DONDE

PPL	Perplejidad: número efectivo de aminoácidos entre los que el modelo “duda” en cada posición. Vale 20 para un modelo que no sabe nada y 1 para uno que acierta siempre con certeza.
$x_{\setminus i}$	La secuencia con solo la posición i enmascarada (por eso se habla de <i>pseudoperplejidad</i>).

La familia ESM. Rives et al. (2021) entrenaron transformers con MLM sobre 86 mil millones de aminoácidos de 250 millones de secuencias, y mostraron que las representaciones internas, aprendidas solo a partir de secuencias, organizan las proteínas en varias escalas: las propiedades bioquímicas de los aminoácidos, la homología remota entre proteínas, y la estructura secundaria y terciaria, recuperables con simples proyecciones lineales. Su modelo ESM-1b, de 650 millones de parámetros, se convirtió en una referencia. Lin et al. (2023) escalaron la familia hasta ESM-2 de 15 mil millones de parámetros y observaron que, al crecer el modelo, emerge en sus representaciones una imagen de la estructura a resolución atómica; sobre ESM-2 construyeron ESMFold, que predice estructuras a partir de una sola secuencia, sin alineamiento múltiple, un orden de magnitud más rápido que los métodos basados en MSA, y lo usaron para predecir más de 617 millones de estructuras de proteínas metagenómicas. En paralelo, ProtTrans (Elnaggar et al., 2022) entrenó varias arquitecturas (entre ellas BERT y T5) con hasta 393 mil millones de aminoácidos de UniRef y BFD, y mostró que los *embeddings* de ProtT5 permiten predecir la estructura secundaria mejor que los métodos anteriores sin usar información evolutiva explícita.

17.3.3 Qué aprende un modelo de lenguaje: contactos y embeddings

Contactos. Si dos residuos están en contacto, sus mutaciones están correlacionadas a lo largo de la evolución (coevolución), y un modelo que predice bien los huecos tiene que haber capturado esas correlaciones. Por eso los mapas de atención de ESM-2 contienen información sobre los contactos; una regresión logística sobre ellos basta para predecir el mapa de contactos (Lin et al., 2023). La figura 17.13 lo comprueba con la ubiquitina: de los 76 pares con mayor probabilidad predicha y

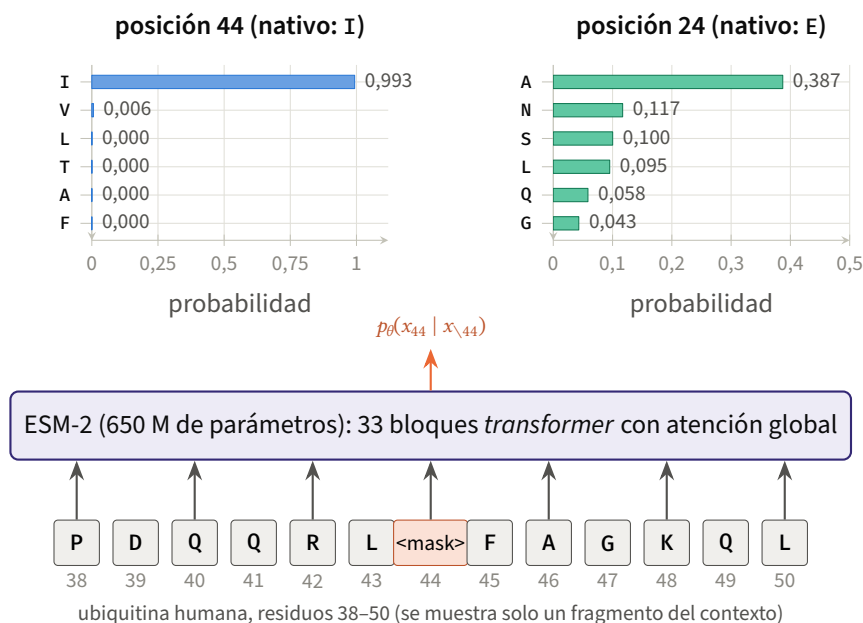


Figura 17.12. Modelado de lenguaje enmascarado con un modelo real (ESM-2 de 650 M de parámetros, `esm2_t33_650M_UR50D`) sobre la ubiquitina humana. **Abajo:** se oculta el residuo 44 y el modelo recibe el resto de la secuencia (aquí se muestran solo los residuos 38–50). **Arriba:** las seis probabilidades más altas que predice en dos posiciones. En la Ile44, que forma parte del parche hidrofóbico reconocido por muchas proteínas que se unen a la ubiquitina, el modelo asigna 0,993 al residuo nativo y casi todo lo demás a Val, otro hidrofóbico ramificado. En el Glu24, expuesto al solvente, la distribución es amplia (entropía 2,1 nats) y el nativo recibe solo 0,027: el modelo “sabe” que esa posición tolera muchos residuos. Cifras de `figuras/cap17/generar.py`.

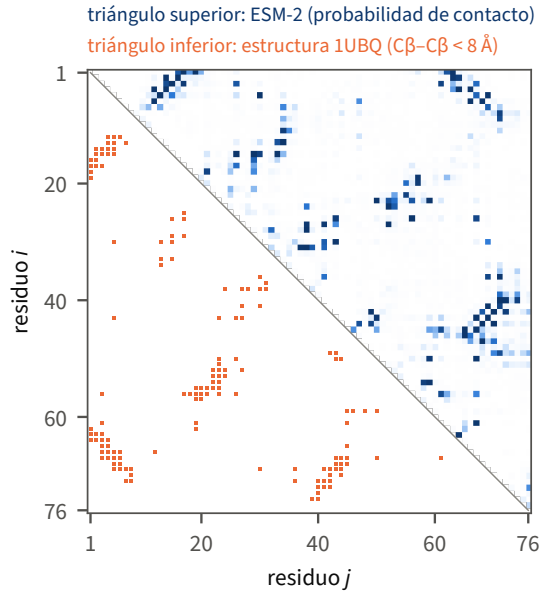


Figura 17.13. Mapa de contactos de la ubiquitina humana. Triángulo superior: probabilidad de contacto predicha por ESM-2 (650 M) a partir de sus mapas de atención, sin alineamiento ni estructura. Triángulo inferior: contactos observados en la estructura cristalográfica 1UBQ (distancia $C\beta-C\beta$ menor de 8 Å, $C\alpha$ para la glicina; separación $|i - j| \geq 6$). El patrón es casi especular: las franjas perpendiculares a la diagonal corresponden a la horquilla β inicial (residuos 1–17) y las manchas alejadas de la diagonal, al emparejamiento de las hebras de la lámina β mixta y al empaquetamiento de la hélice (23–34) contra ella. Precisión de los L pares mejor puntuados con $|i - j| \geq 6$: 0,87.

separación en la secuencia de al menos 6 residuos, el 86,8 % son contactos reales en la estructura cristalográfica 1UBQ; entre los 15 pares más probables separados por 24 o más residuos, todos lo son. Es la misma información que AlphaFold (Jumper et al., 2021) extrae de un alineamiento múltiple (capítulo 15), obtenida aquí de una sola secuencia. Una advertencia honesta: la ubiquitina es una de las proteínas más conservadas y estudiadas, y sus homólogos estaban en los datos de entrenamiento; el rendimiento en proteínas huérfanas es bastante menor.

Embeddings. La última capa de un modelo de lenguaje asigna a cada posición un vector $\mathbf{h}_i \in \mathbb{R}^d$ ($d = 1280$ en ESM-2 de 650 M). Para obtener una representación de

la proteína completa, de dimensión fija sea cual sea su longitud, lo más sencillo es promediar:

$$\mathbf{e}(x) = \frac{1}{L} \sum_{i=1}^L \mathbf{h}_i(x). \quad (17.28)$$

DONDE

$\mathbf{h}_i(x)$	Vector de la última capa para la posición i de la proteína x (<i>embedding</i> por residuo).
$\mathbf{e}(x)$	<i>Embedding</i> de la proteína completa.

Estos vectores son atributos excelentes para cualquiera de los clasificadores de la sección 17.1: en lugar de un espectro de k -mers, que solo ve palabras idénticas, usamos un vector que resume lo que el modelo aprendió de toda la evolución. Esta estrategia, llamada *aprendizaje por transferencia*, es hoy la forma más común de usar los modelos de lenguaje con conjuntos de datos etiquetados pequeños. La figura 17.14 compara ambas representaciones con 150 proteínas reales de cinco familias de UniProt, elegidas para que no haya pares muy parecidos. Proyectadas con UMAP (capítulo 12), las familias se separan con nitidez en el espacio de ESM-2 y se entremezclan en el de 3-mers; el clasificador del vecino más cercano acierta la familia en el 95,3 % de los casos con *embeddings* y en el 89,3 % con k -mers.

17.3.4 Predicción de efectos de variantes sin supervisión

La aplicación que más ha llamado la atención de la genética médica es la predicción *zero-shot* (sin ningún dato experimental de entrenamiento) del efecto de las mutaciones. La idea, formalizada por Meier et al. (2021), es que el modelo ya ha aprendido qué secuencias son “gramaticales” para la evolución: si una mutación convierte la proteína en algo improbable para el modelo, probablemente sea perjudicial. La puntuación de *marginal enmascarado* de una variante con posiciones mutadas M es

$$s(x^{\text{mut}}) = \sum_{i \in M} \left[\log p_{\theta}(x_i = x_i^{\text{mut}} \mid x_{\setminus M}) - \log p_{\theta}(x_i = x_i^{\text{wt}} \mid x_{\setminus M}) \right]. \quad (17.29)$$

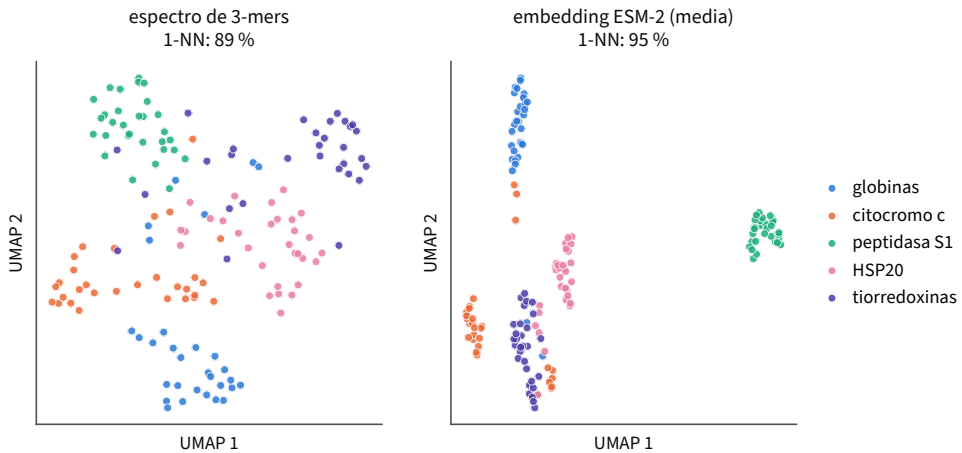


Figura 17.14. Proyección UMAP de 150 proteínas revisadas de UniProt, 30 por familia (globinas, citocromos c, peptidasas S1, proteínas de choque térmico pequeñas HSP20 y tioredoxinas), seleccionadas con un filtro de redundancia que descarta secuencias cuyo espectro de 3-mers tenga una similitud coseno mayor que 0,35 con otra ya elegida. **Izquierda:** espectro de 3-mers (8000 dimensiones). **Derecha:** *embedding* medio de ESM-2 (1280 dimensiones, ecuación (17.28)). En los títulos, la exactitud del clasificador del vecino más cercano (similitud coseno, dejando uno fuera) en el espacio original. El modelo de lenguaje agrupa homólogos remotos que apenas comparten palabras idénticas; los grupos del panel izquierdo reflejan sobre todo composición de aminoácidos.

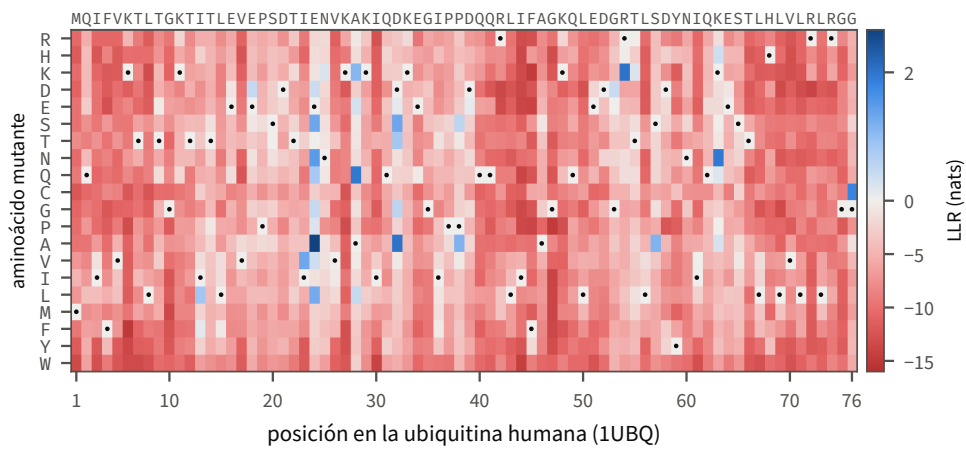


Figura 17.15. Paisaje completo de las 19×76 sustituciones puntuales de la ubiquitina humana, puntuadas con el marginal enmascarado de ESM-2 (650 M; ecuación (17.29)). Cada columna es una posición (secuencia silvestre arriba) y cada fila un aminoácido mutante; el punto negro marca el residuo silvestre ($\text{LLR} = 0$). Rojo: el modelo considera la mutación mucho menos probable que el silvestre (presuntamente perjudicial); azul: más probable. El 97,6 % de las sustituciones tienen LLR negativo. Destacan como intolerantes la Ile44 y la Val70 del parche hidrofóbico, la Gly47, la Phe45, la Gly10 y las Lys6 y Lys27; las columnas pálidas (E24, A28, D32, K63) son posiciones expuestas donde el modelo acepta casi cualquier residuo. Los valores se calcularon con `figuras/cap17/generar.py`.

DONDE	
$x_i^{\text{wt}}, x_i^{\text{mut}}$	Aminoácido silvestre (<i>wild type</i>) y mutante en la posición i .
$x_{\setminus M}$	Secuencia silvestre con las posiciones mutadas enmascaradas.
$s(x^{\text{mut}})$	Razón de log-verosimilitudes (LLR, en nats): negativa si el modelo prefiere el silvestre.

La ecuación (17.29) es un viejo conocido con ropa nueva: igual que la puntuación de una matriz de sustitución del capítulo 3 o de una PWM del capítulo 4, es el logaritmo de una razón de probabilidades, solo que ahora la probabilidad depende de *todo el contexto* de la proteína concreta y no de una tabla fija. Con una sola pasada del modelo por posición se obtiene el paisaje completo de las $19L$ mutaciones puntuales posibles (figura 17.15).

Ejemplo 17.6. (Leer el paisaje de la ubiquitina)

En la Ile44, la sustitución conservadora I44V tiene $LLR = -5,10$ nats, I44A baja a $-9,54$ y la introducción de una carga, I44D, a $-13,62$: el orden coincide con la intuición bioquímica de que un núcleo hidrofóbico tolera mejor otro residuo hidrofóbico que un hueco o una carga. En el Glu24, en cambio, E24A tiene $LLR = \ln(0,387/0,027) \approx +2,66$: el modelo considera la alanina *más* verosímil que el glutamato nativo (figura 17.12).

El paisaje también muestra los límites del método. Las dos glicinas C-terminales (Gly75 y Gly76) son imprescindibles para que la ubiquitina se conjugue con otras proteínas, pero el modelo penaliza G75A ($-7,18$) mucho más que G76A ($-1,30$). Un modelo de lenguaje captura sobre todo las restricciones que la evolución imprime en muchas familias de proteínas (plegamiento, empaquetamiento) y no necesariamente las funciones específicas de una proteína concreta; sus puntuaciones son hipótesis, no mediciones.

Meier et al. (2021) mostraron que esta estrategia sin supervisión compite con los modelos entrenados específicamente para cada familia, y Brandes et al. (2023) la escalaron al proteoma humano: con ESM1b predijeron el efecto de las ~ 450 millones de variantes de sentido erróneo posibles, superaron a los métodos existentes al clasificar $\sim 150\,000$ variantes de ClinVar/HGMD como patogénicas o benignas y al predecir las mediciones de 28 experimentos de mutagénesis profunda, y extendieron el enfoque a inserciones y deleciones sin cambio de marco y a codones de parada prematuros.

```
esm_llr.py

import torch
from transformers import AutoTokenizer, EsmForMaskedLM

nombre = "facebook/esm2_t33_650M_UR50D"
tok = AutoTokenizer.from_pretrained(nombre)
esm = EsmForMaskedLM.from_pretrained(nombre).eval()
ubq = ("MQIFVKLTGTGKITLEVPSDTIENVKAKIQDKEGIPPDQQLIFAG"
       "KQLEDGRTLSDYNIQKESTLHLVLRGG")

def llr(seq, pos, mut):
    """Marginal enmascarado de la mutación wt->mut (pos 1-based)."""
    ids = tok(seq, return_tensors="pt")["input_ids"]
    ids[0, pos] = tok.mask_token_id          # índice 0 = token <cls>
```

```

with torch.no_grad():
    logp = esm(input_ids=ids).logits[0, pos].log_softmax(-1)
    a, b = tok.convert_tokens_to_ids([mut, seq[pos - 1]])
    return (logp[a] - logp[b]).item()

print(f"I44A: {llr(ubq, 44, 'A'):.2f}")      # -9.54 nats
with torch.no_grad():                      # embedding medio
    h = esm.esm(**tok(ubq, return_tensors="pt")).last_hidden_state
    emb = h[0, 1:-1].mean(0)                # vector de 1280 dim.

```

Por qué funciona: una lectura estadística

En la ecuación (17.26), el modelo óptimo es aquel cuya $p_\theta(x_i | x_{\setminus i})$ coincide con la distribución condicional real de las secuencias naturales. Las secuencias naturales, a su vez, están moldeadas por la selección: si una sustitución destruye la función, los linajes que la portan desaparecen y la sustitución es rara en las bases de datos. La LLR de la ecuación (17.29) estima, por tanto, un cociente de aptitudes relativas integrado a lo largo de toda la historia evolutiva de la familia. Los modelos de familia única (perfiles, modelos de Potts) hacen la misma estimación con un alineamiento; los modelos de lenguaje comparten parámetros entre todas las familias, lo que les permite funcionar con familias pequeñas o sin alineamiento posible, a cambio de mezclar a veces señales de familias distintas.

17.3.5 Riesgos y limitaciones

Los modelos de esta sección son herramientas poderosas, y precisamente por eso conviene conocer sus puntos débiles:

- **Fuga de información a escala de base de datos.** Los modelos de lenguaje se entrenan con casi todas las proteínas conocidas. Evaluar un modelo derivado sobre un conjunto de prueba cuyas secuencias (u homólogos cercanos) estaban en el preentrenamiento reproduce, a gran escala, el problema de la figura 17.4. Las recomendaciones DOME (Walsh et al., 2021) se aplican igual.
- **Sesgo de representación.** Las bases de datos están dominadas por organismos modelo, patógenos y familias muy estudiadas. El modelo es más fiable allí donde menos lo necesitamos, y la baja perplejidad de la ubiquitina (PPL = 1,78 según nuestros cálculos) refleja en parte que ha visto miles de sus homólogos.

- **Probabilidades que no son probabilidades.** Una LLR de -9 no significa que la variante sea patogénica con una probabilidad determinada; hay que calibrarla con datos clínicos o experimentales antes de usarla en decisiones.
- **Interpretabilidad limitada.** Saliencia y atención indican *dónde* mira el modelo, no *por qué*; una cabeza de atención que coincide con los contactos no demuestra que el modelo “entienda” la física del plegamiento.
- **Costes y acceso.** Entrenar un modelo de miles de millones de parámetros exige recursos que pocos laboratorios tienen; usar modelos preentrenados es asequible, pero hace depender la investigación de las decisiones de quien los entrenó.

De la traducción automática a las proteínas

El *transformer* nació en 2017 para traducir del inglés al alemán y al francés (Vaswani et al., 2017). Dos años después, BERT mostró que un *transformer* preentrenado con palabras enmascaradas podía adaptarse a casi cualquier tarea de lenguaje (Devlin et al., 2019). La biología computacional adoptó la receta casi de inmediato: la primera versión del trabajo de Rives et al. (2021) circuló como preprint en 2019, y en solo cuatro años se pasó de la idea a un atlas de cientos de millones de estructuras predichas (Lin et al., 2023). El epígrafe de este capítulo, escrita por George Box en 1976, sigue siendo el mejor consejo para usarlos: ningún modelo es correcto; la pregunta útil es en qué se equivoca de forma importante.

Un modelo de lenguaje de proteínas es un perfil de familia generalizado a toda la evolución: sus probabilidades condicionales resumen qué ha tolerado la selección en cada contexto.

En resumen

- El aprendizaje supervisado minimiza el riesgo empírico (ecuación (17.1)), pero lo que importa es el error sobre datos nuevos. Las secuencias se convierten en vectores mediante codificación *one-hot* (conserva posiciones) o espectros de *k*-mers (invariantes a desplazamientos); el núcleo de espectro mide su similitud sin construir los vectores.

- La regresión logística tiene gradiente $\frac{1}{n} \sum (p_i - y_i) \mathbf{x}_i$ y pérdida convexa; la SVM maximiza el margen y depende solo de los vectores de soporte; los *random forests* reducen la varianza promediando árboles decorrelacionados.
- El error se descompone en ruido, sesgo² y varianza. La validación cruzada solo es honesta si entrenamiento y prueba son independientes: en biología hay que particionar por familias de homología (GroupKFold). Con clases desbalanceadas, la curva PR es más informativa que la ROC.
- Las redes neuronales se entrenan por retropropagación (regla de la cadena de atrás hacia adelante). En ADN *one-hot*, un filtro convolucional es una PWM que se aprende sola; el *max-pooling* da invariancia a la posición. DeepBind, DeepSEA, Basset, Basenji y Enformer ampliaron el contexto de ~100 pb a ~200 kb.
- La saliencia (gradiente por entrada), DeepLIFT y TF-MoDISco convierten un modelo entrenado en mapas de contribución y catálogos de motivos.
- La atención ($\text{softmax}(QK^\top / \sqrt{d_k})V$) permite que cada residuo integre información de toda la secuencia. Los modelos de lenguaje de proteínas (ESM, ProtTrans) se entrenan prediciendo aminoácidos enmascarados y aprenden contactos, estructura y homología remota.
- La razón de log-verosimilitudes de marginal enmascarado predice sin supervisión el efecto de las mutaciones; es informativa pero no calibrada, y hereda los sesgos de los datos de entrenamiento.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 17.1** ●●● Calcule a mano el espectro de 3-mers de **ACGTACGTAC** y el núcleo de espectro normalizado $\tilde{K}_3$ con **CGTACGTACG**. ¿Qué valor obtendría con la codificación *one-hot* y un producto interno posición a posición? Explique la diferencia.
- 17.2** ●●● Un clasificador de variantes patogénicas tiene sensibilidad 0,95 y especificidad 0,98. Use la ecuación (17.15) para calcular su precisión si el 50 %, el 10 % y el 1 % de las variantes evaluadas son patogénicas. ¿Cuál de esos escenarios corresponde a un panel clínico real?

- 17.3** ●●● Explique con sus palabras por qué un 1-NN obtiene casi un 97 % de exactitud en la figura 17.4 con etiquetas aleatorias. ¿Qué informaría usted en la sección de métodos de un artículo para convencer a un revisor de que su evaluación no sufre de fuga de información?
- 17.4** ●●● Demuestre la identidad $\sigma'(z) = \sigma(z)[1 - \sigma(z)]$ y úsela para comprobar que la matriz hessiana de la ecuación (17.6) es $\frac{1}{n} \sum_i p_i(1 - p_i) \tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_i^\top$, donde $\tilde{\mathbf{x}}_i = (1, \mathbf{x}_i)$. ¿Por qué esto prueba que la pérdida es convexa?
- 17.5** ●●● Continúe el ejemplo 17.2 durante 20 iteraciones con $\eta = 1$ (programe el bucle con numpy). ¿Converge w_1 a un valor finito? Añada una penalización L_2 con $\lambda = 0,1$ y compare. Relacione el resultado con la separabilidad de los datos.
- 17.6** ●●● Con `sklearn.datasets.make_moons`, reproduzca la figura 17.3 y añada la curva de un `RandomForestClassifier` en función del número de árboles B (de 1 a 500). ¿Se sobreajusta un *random forest* al aumentar B ? Relacione su respuesta con la ecuación (17.12).
- 17.7** ●●● Calcule el campo receptivo de una pila de 7 capas convolucionales con filtros de ancho 5 (a) sin dilatación y (b) con dilataciones 1, 2, 4, ..., 64. ¿Cuántas capas sin dilatación harían falta para alcanzar el mismo campo receptivo que (b)?
- 17.8** ●●● Modifique la CNN de la sección 17.2 para que reconozca el motivo en cualquiera de las dos hebras: plante el sitio en orientación aleatoria y compare (a) duplicar los datos con el reverso complementario y (b) aplicar cada filtro también al reverso complementario de la secuencia y quedarse con el máximo. ¿Qué logo obtiene en cada caso?
- 17.9** ●●● Con el código de `esm_llr.py`, calcule el paisaje de marginal enmascarado de una proteína de su interés con un modelo pequeño (`esm2_t12_35M_UR50D`) y con el de 650 M. Calcule la correlación de Spearman entre ambos paisajes. Si existe un experimento de mutagénesis profunda publicado para esa proteína, compare ambos con los datos y discuta qué posiciones predice peor el modelo y por qué.
- 17.10** ●●● Diseñe un experimento para decidir si la alta precisión de contactos de la figura 17.13 se debe a que ESM-2 “memorizó” la familia de la ubiquitina.

(Pista: considere proteínas diseñadas *de novo*, familias ausentes de UniRef en la fecha de entrenamiento, o secuencias ancestrales reconstruidas.)

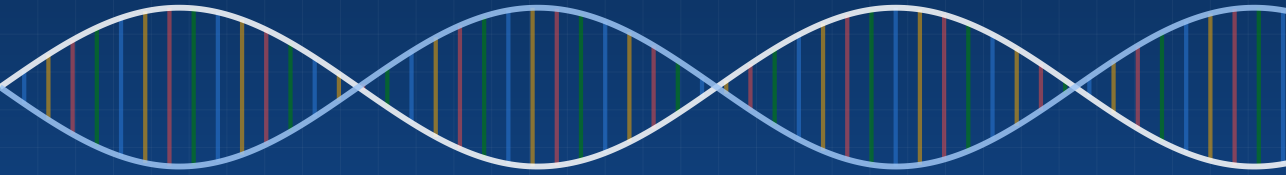
Referencias del capítulo

- Alipanahi, B., Delong, A., Weirauch, M. T., & Frey, B. J. (2015). Predicting the sequence specificities of DNA- and RNA-binding proteins by deep learning. *Nature Biotechnology*, 33(8), 831-838. <https://doi.org/10.1038/nbt.3300>
- Avsec, Ž., Agarwal, V., Visentin, D., Ledsam, J. R., Grabska-Barwinska, A., Taylor, K. R., Assael, Y., Jumper, J., Kohli, P., & Kelley, D. R. (2021). Effective gene expression prediction from sequence by integrating long-range interactions. *Nature Methods*, 18(10), 1196-1203. <https://doi.org/10.1038/s41592-021-01252-x>
- Box, G. E. P. (1976). Science and Statistics. *Journal of the American Statistical Association*, 71(356), 791-799. <https://doi.org/10.1080/01621459.1976.10480949>
- Brandes, N., Goldman, G., Wang, C. H., Ye, C. J., & Ntranos, V. (2023). Genome-wide prediction of disease variant effects with a deep protein language model. *Nature Genetics*, 55(9), 1512-1522. <https://doi.org/10.1038/s41588-023-01465-0>
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297. <https://doi.org/10.1007/BF00994018>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 4171-4186. <https://doi.org/10.18653/v1/N19-1423>
- Elnaggar, A., Heinzinger, M., Dallago, C., Rehawi, G., Wang, Y., Jones, L., Gibbs, T., Feher, T., Angerer, C., Steinegger, M., Bhowmik, D., & Rost, B. (2022). ProtTrans: Toward Understanding the Language of Life Through Self-Supervised Learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10), 7112-7127. <https://doi.org/10.1109/TPAMI.2021.3095381>
- Eraslan, G., Avsec, Ž., Gagneur, J., & Theis, F. J. (2019). Deep learning: New computational modelling techniques for genomics. *Nature Reviews Genetics*, 20(7), 389-403. <https://doi.org/10.1038/s41576-019-0122-6>
- Greener, J. G., Kandathil, S. M., Moffat, L., & Jones, D. T. (2022). A guide to machine learning for biologists. *Nature Reviews Molecular Cell Biology*, 23(1), 40-55. <https://doi.org/10.1038/s41580-021-00407-0>

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2.^a ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
- Jones, D. T. (2019). Setting the standards for machine learning in biology. *Nature Reviews Molecular Cell Biology*, 20(11), 659-660. <https://doi.org/10.1038/s41580-019-0176-5>
- Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583-589. <https://doi.org/10.1038/s41586-021-03819-2>
- Kelley, D. R., Reshef, Y. A., Bileschi, M., Belanger, D., McLean, C. Y., & Snoek, J. (2018). Sequential regulatory activity prediction across chromosomes with convolutional neural networks. *Genome Research*, 28(5), 739-750. <https://doi.org/10.1101/gr.227819.117>
- Kelley, D. R., Snoek, J., & Rinn, J. L. (2016). Basset: Learning the regulatory code of the accessible genome with deep convolutional neural networks. *Genome Research*, 26(7), 990-999. <https://doi.org/10.1101/gr.200535.115>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444. <https://doi.org/10.1038/nature14539>
- Leslie, C., Eskin, E., & Noble, W. S. (2002). The spectrum kernel: A string kernel for SVM protein classification. *Biocomputing 2002: Proceedings of the Pacific Symposium*, 564-575. https://doi.org/10.1142/9789812799623_0053
- Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., Smetanin, N., Verkuil, R., Kabeli, O., Shmueli, Y., dos Santos Costa, A., Fazel-Zarandi, M., Sercu, T., Candido, S., & Rives, A. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130. <https://doi.org/10.1126/science.ade2574>
- Meier, J., Rao, R., Verkuil, R., Liu, J., Sercu, T., & Rives, A. (2021). *Language models enable zero-shot prediction of the effects of mutations on protein function* [Publicado en Advances in Neural Information Processing Systems 34]. bioRxiv. <https://doi.org/10.1101/2021.07.09.450648>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Müller, A., Nothman, J., Louppe, G., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://doi.org/10.48550/arXiv.1201.0490>
- Rives, A., Meier, J., Sercu, T., Goyal, S., Lin, Z., Liu, J., Guo, D., Ott, M., Zitnick, C. L., Ma, J., & Fergus, R. (2021). Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15), e2016239118. <https://doi.org/10.1073/pnas.2016239118>
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408. <https://doi.org/10.1037/h0042519>

- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536. <https://doi.org/10.1038/323533a0>
- Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- Shrikumar, A., Greenside, P., & Kundaje, A. (2017). *Learning Important Features Through Propagating Activation Differences*. arXiv: 1704.02685. <https://doi.org/10.48550/arXiv.1704.02685>
- Shrikumar, A., Tian, K., Avsec, Ž., Shcherbina, A., Banerjee, A., Sharmin, M., Nair, S., & Kundaje, A. (2018). *Technical Note on Transcription Factor Motif Discovery from Importance Scores (TF-MoDISco) version 0.5.6.5*. arXiv: 1811.00416. <https://doi.org/10.48550/arXiv.1811.00416>
- Simonyan, K., Vedaldi, A., & Zisserman, A. (2013). *Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps*. arXiv: 1312.6034. <https://doi.org/10.48550/arXiv.1312.6034>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. arXiv: 1706.03762. <https://doi.org/10.48550/arXiv.1706.03762>
- Walsh, I., Fishman, D., Garcia-Gasulla, D., Titma, T., Pollastri, G., et al. (2021). DOME: Recommendations for supervised machine learning validation in biology. *Nature Methods*, 18(10), 1122-1127. <https://doi.org/10.1038/s41592-021-01205-4>
- Zhou, J., & Troyanskaya, O. G. (2015). Predicting effects of noncoding variants with deep learning-based sequence model. *Nature Methods*, 12(10), 931-934. <https://doi.org/10.1038/nmeth.3547>

Flujos reproducibles y proyecto final



Un artículo sobre ciencia computacional en una revista científica no es la investigación misma: es solo la publicidad de la investigación. La investigación de verdad es el entorno completo de desarrollo de software y el conjunto completo de instrucciones que generaron las figuras.

— Buckheit y Donoho (1995), a propósito de J. Claubout

● Al terminar este capítulo usted podrá

- Representar un análisis bioinformático como un grafo acíclico dirigido de trabajos, derivar el algoritmo de Kahn para ordenarlo y demostrar que detecta ciclos en tiempo $O(|V| + |E|)$.
- Escribir un flujo completo en Snakemake (reglas, comodines, inferencia hacia atrás) y su equivalente en Nextflow (procesos y canales), y explicar la diferencia entre un modelo *pull* y uno *push*.
- Fijar las cuatro capas de la reproducibilidad (código, entorno, datos y aleatoriedad) con Git, Bioconda, contenedores, sumas de verificación y semillas.

- Acotar el tiempo de ejecución de un flujo en paralelo con el trabajo y el camino crítico, y usar las leyes de Amdahl y Gustafson para decidir dónde vale la pena invertir núcleos.
- Diseñar, ejecutar, documentar y publicar un proyecto integrador de extremo a extremo que cumpla los principios FAIR y las obligaciones éticas y legales sobre datos humanos.

El capítulo o introdujo semillas, versiones y sumas de verificación. En los capítulos 6 a 9 pasamos de lecturas crudas a variantes anotadas; en el capítulo 5 reconstruimos árboles y en el capítulo 11 pasamos de lecturas de ARN a genes diferencialmente expresados y términos funcionales enriquecidos. Hasta ahora hemos ejecutado muchos bloques de código por separado. Un proyecto real debe enlazar decenas de pasos y aplicarlos a numerosas muestras. También debe poder reanudarse tras un fallo, incorporar datos nuevos y producir resultados reproducibles para otros investigadores o para nosotros mismos en el futuro.

Este capítulo final trata de esa cadena. En la primera lección (sección 18.1) formalizamos un análisis como un grafo acíclico dirigido, derivamos el algoritmo que lo ordena y estudiamos los dos gestores de flujos más usados en bioinformática, Snakemake y Nextflow, junto con las herramientas que congelan el entorno de software (Bioconda y los contenedores) y la teoría que dice cuánto se gana al paralelizar. En la segunda (sección 18.2) aplicamos todo a un *proyecto integrador*: de las lecturas FASTQ de aislados de un patógeno a variantes anotadas y una filogenia (o, alternativamente, de un experimento de RNA-seq a un análisis de enriquecimiento), con su estructura de directorios, control de versiones, pruebas, informe reproducible, depósito de datos, consideraciones éticas y rúbrica de evaluación. La revisión de Wratten et al. (2021) y las “reglas” de Sandve et al. (2013) y Wilson et al. (2017) son buenas compañeras de lectura para todo el capítulo.

18.1 Snakemake y Nextflow

 Colab · 18.1 · Snakemake y Nextflow

 repositorio del curso

La cocina de un restaurante a la hora punta

Un cocinero que prepara un plato en casa sigue la receta de arriba abajo: pica, sofríe, hierve, sirve. En la cocina de un restaurante, a la hora punta, esa estrategia sería un desastre. Allí nadie piensa en la receta como una lista, sino como una red de dependencias: la salsa necesita el fondo, el fondo necesita los huesos tostados, el emplatado necesita la salsa y la guarnición, que no dependen entre sí y pueden prepararse a la vez en fogones distintos. El jefe de cocina hace tres cosas que un cocinero doméstico no hace: reparte en paralelo lo que es independiente, no repite lo que ya está listo en la cámara (*mise en place*) y, cuando un plato vuelve a la cocina porque la salsa salió mal, rehace la salsa y todo lo que depende de ella, pero no vuelve a tostar los huesos. Un gestor de flujos de trabajo es ese jefe de cocina para nuestros análisis.

18.1.1 Por qué los *scripts* no escalan

La forma más natural de automatizar un análisis es escribir un *script* de `bash` que llame a las herramientas en orden. Para una muestra y una tarde de trabajo, funciona. Pensemos, sin embargo, en el flujo que construimos a lo largo de los capítulos 6 a 9, aplicado a 24 aislados bacterianos: control de calidad con `fastp`, mapeo con BWA-MEM, marcado de duplicados, llamado conjunto de variantes con `bcftools`, filtrado, anotación, secuencias consenso y un árbol filogenético. Un *script* lineal tiene cuatro defectos que crecen con el tamaño del problema:

1. **No sabe qué está hecho.** Si el mapeo de la muestra 17 falla por falta de memoria, volver a lanzar el *script* repite el control de calidad y el mapeo de las 16 anteriores. Añadir la muestra 25 obliga a reprocesar las otras 24, o a editar el *script* a mano.
2. **No sabe qué es independiente.** Los 24 mapeos no dependen unos de otros y podrían ejecutarse a la vez, pero un bucle `for` los ejecuta en serie. Paralelizar a mano (con `&` y `wait`) es frágil y no respeta las dependencias.
3. **Mezcla la lógica con el entorno.** Las rutas absolutas, el número de núcleos o la versión de `samtools` que “haya en el sistema” quedan escondidos en el código, y el mismo *script* produce resultados distintos en otra máquina.
4. **Falla en silencio.** Sin `set -euo pipefail`, un error en medio de una tubería deja un archivo truncado que el paso siguiente consume sin quejarse.

Los gestores de flujos de trabajo resuelven los cuatro problemas con una misma idea: en lugar de describir *en qué orden* ejecutar los pasos, describimos *qué produce*

cada paso a partir de qué, y dejamos que un programa deduzca el orden, el paralelismo y lo que falta por hacer (Wratten et al., 2021).

18.1.2 Un análisis es un grafo acíclico dirigido

Definición 18.1. (Grafo de trabajos de un flujo)

Un *trabajo* (*job*) es la ejecución de un paso sobre entradas concretas: por ejemplo, “mapear la muestra A”. El *grafo de trabajos* de un flujo es el grafo dirigido $G = (V, E)$ cuyos vértices V son los trabajos y en el que existe una arista $(u, v) \in E$ si algún archivo producido por u es una entrada de v . Un *orden topológico* de G es una biyección $\pi : V \rightarrow \{1, \dots, |V|\}$ tal que

$$(u, v) \in E \implies \pi(u) < \pi(v). \tag{18.1}$$

DONDE

V, E	Conjunto de trabajos y conjunto de dependencias (aristas dirigidas del productor al consumidor).
(u, v)	Arista: v no puede empezar hasta que u termine.
$\pi(v)$	Posición de v en una ejecución en serie que respete todas las dependencias.
$ V , E $	Número de trabajos y de dependencias.

La figura 18.1 muestra el grafo de trabajos del flujo que usaremos como hilo conductor de todo el capítulo, con tres muestras (A, B y C). Tiene 19 trabajos y 26 aristas, y cada color corresponde a una *regla*, es decir, a una plantilla que se instancia una vez por muestra (`fastp`, `mapear`, `dedup`, `consenso`) o una sola vez para todas (`bwa_index`, `llamar`, `filtrar`, `anotar`, `filogenia`, `multiqc`). El último vértice, `all`, no ejecuta nada: solo declara qué archivos finales queremos.

¿Cuándo existe un orden topológico? La respuesta es el primer resultado que justifica todo el edificio.

Teorema 18.1. (Existencia del orden topológico)

Un grafo dirigido finito admite un orden topológico si y solo si no contiene ciclos dirigidos.

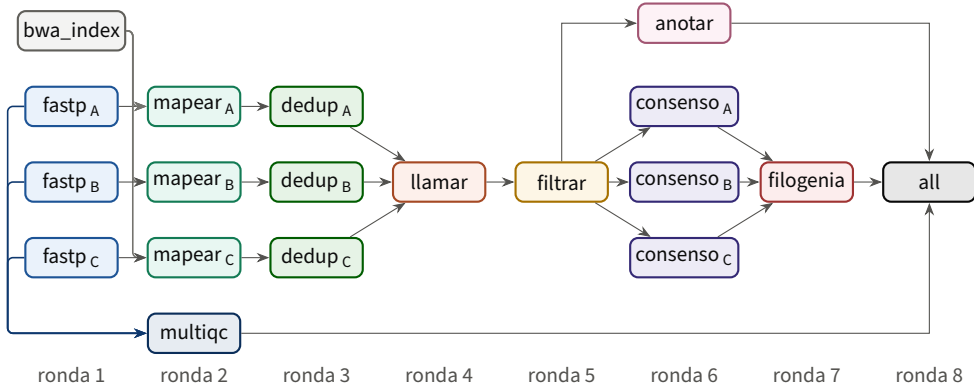


Figura 18.1. Grafo de trabajos del flujo de ejemplo con tres muestras (A, B, C): de lecturas crudas a variantes anotadas, secuencias consenso, un árbol filogenético y un informe de calidad. Cada color es una regla del Snakefile de la sección 18.1.5; los subíndices son los valores del comodín m . Las columnas son las *rondas* del algoritmo de Kahn: todos los trabajos de una ronda son independientes entre sí y pueden ejecutarse a la vez (anchos 4, 4, 3, 1, 1, 4, 1, 1). Observe los dos cuellos de botella de anchura 1, **llamar** y **filtrar**: el llamado conjunto necesita *todas* las muestras, y por ahí pasa todo el flujo. El índice de BWA (gris) se construye una sola vez y alimenta los tres mapeos; los tres informes de **fastp** confluyen en **multiqc** (azul oscuro, abajo). Grafo calculado con `figuras/cap18/generar.py`.

Demostración. ($\Rightarrow$) Si hubiera un ciclo $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$, la ecuación (18.1) exigiría $\pi(v_1) < \pi(v_2) < \dots < \pi(v_k) < \pi(v_1)$, lo cual es imposible. ($\Leftarrow$) Primero, un *lema*: todo grafo dirigido finito y acíclico no vacío tiene al menos una *fuentes*, un vértice sin aristas entrantes. En efecto, si todo vértice tuviera un predecesor, podríamos caminar hacia atrás indefinidamente, $v_0 \leftarrow v_1 \leftarrow v_2 \leftarrow \dots$; como solo hay $|V|$ vértices, tras $|V| + 1$ pasos repetiríamos alguno, y el tramo entre las dos visitas sería un ciclo. Ahora, por inducción sobre $|V|$: tomamos una fuente s , le asignamos $\pi(s) = 1$, la eliminamos junto con sus aristas (el grafo restante sigue siendo acíclico) y ordenamos el resto con las posiciones $2, \dots, |V|$. Ninguna arista entra en s , así que ninguna puede violar la ecuación (18.1). $\square$

La demostración es constructiva: *contiene* un algoritmo. Basta repetir “toma una fuente y elimínala”. El único problema es encontrar fuentes deprisa; recorrer todo el grafo en cada paso costaría $O(|V|^2)$. Kahn (1962), que buscaba ordenar las redes de actividades de los proyectos de ingeniería de su época (las redes PERT), resolvió el problema con una contabilidad sencilla.

18.1.3 El algoritmo de Kahn

Sea $d^-(v)$ el *grado de entrada* de v , el número de aristas que llegan a v . Eliminar una fuente s no cambia el grado de entrada de ningún vértice salvo el de sus sucesores, cada uno de los cuales pierde exactamente una unidad. Por lo tanto, las únicas fuentes *nuevas* que pueden aparecer al eliminar s son sucesores de s cuyo grado acaba de llegar a cero. Esta observación convierte la búsqueda global en una actualización local:

1. Calcular $d^-(v)$ para todo v recorriendo una vez todas las aristas. Poner en una cola Q todos los v con $d^-(v) = 0$.
2. Mientras Q no esté vacía: extraer v de Q , añadirlo al orden y, para cada sucesor u de v , hacer $d^-(u) \leftarrow d^-(u) - 1$; si $d^-(u)$ llega a 0, añadir u a Q .
3. Si al vaciarse Q el orden contiene menos de $|V|$ vértices, el grafo tiene un ciclo.

Teorema 18.2. (Corrección y coste del algoritmo de Kahn)

En todo momento, $d^-(v)$ es el número de predecesores de v que aún no están en el orden, y Q contiene exactamente los vértices no ordenados cuyos predecesores ya fueron ordenados. En consecuencia: (i) el orden producido satisface la ecuación (18.1); (ii) si G es acíclico, el algoritmo ordena los $|V|$ vértices; (iii) si termina con vértices sin ordenar, estos contienen un ciclo; y (iv) el tiempo total es

$$T_{\text{Kahn}} = O(|V| + |E|). \quad (18.2)$$

DONDE

$d^-(v)$	Grado de entrada “residual”: dependencias de v todavía pendientes.
Q	Cola de trabajos listos (todas sus dependencias satisfechas).
T_{Kahn}	Número de operaciones elementales del algoritmo.

Demostración. El invariante se cumple al inicio y cada decremento lo preserva, porque se hace justo cuando un predecesor entra en el orden. (i): un vértice entra en Q solo cuando todos sus predecesores ya fueron ordenados, luego queda detrás de todos ellos. (ii) y (iii): si Q se vacía con un conjunto $R \neq \emptyset$ de vértices sin ordenar, por el invariante ningún vértice de R tiene grado residual cero, es decir, el subgrafo inducido por R no tiene fuentes; por el lema de la demostración anterior, ese subgrafo contiene un ciclo. Si G es acíclico, esto no puede ocurrir. (iv): el cálculo

inicial de grados recorre cada arista una vez; cada vértice entra y sale de Q a lo sumo una vez, y cada arista se “consume” (se decrementa su destino) exactamente una vez, cuando sale su origen. $\square$

El punto (iii) es la razón por la que todos los gestores de flujos rechazan, antes de ejecutar nada, un flujo en el que una regla necesita (directa o indirectamente) su propia salida: la detección de ciclos sale gratis del ordenamiento.

Ejemplo 18.1. (Kahn sobre el flujo de tres muestras)

Los grados de entrada iniciales son $d^- = 0$ para `bwa_index` y los tres `fastp`; $d^- = 2$ para cada `mapear` (índice y lecturas limpias); $d^- = 1$ para cada `dedup`, para `filtrar`, `anotar` y cada `consenso`; y $d^- = 3$ para `llamar`, `multiqc`, `filogenia` y `all`. La suma es $4 \cdot 0 + 3 \cdot 2 + 3 \cdot 1 + 3 + 1 + 1 + 3 \cdot 1 + 3 + 3 + 3 = 26 = |E|$, como debe ser, porque cada arista aporta exactamente una unidad a un grado de entrada.

En el paso 1 sale `bwa_index`: sus tres sucesores bajan de 2 a 1 y ninguno se libera. En el paso 2 sale `fastpA`: `mapearA` baja a 0 y entra en la cola, mientras que `multiqc` baja de 3 a 2. Tras los pasos 3 y 4 se han liberado los tres mapeos y `multiqc`. Así continúa (figura 18.2) hasta el paso 19, en que sale `all` y la cola queda vacía con los 19 trabajos ordenados. Si en lugar de una cola procesamos rondas completas (todas las fuentes de una vez), obtenemos 8 rondas de anchos 4, 4, 3, 1, 1, 4, 1, 1: las columnas de la figura 18.1. El ancho máximo, 4, es el número de núcleos a partir del cual este flujo ya no puede aprovechar más paralelismo, un resultado que precisaremos en la sección 18.1.10.

kahn.py

```
from collections import deque

def orden_topologico(sucesores):
    """Kahn. sucesores = {nodo: [hijos]}; error si hay un ciclo."""
    grado = {v: 0 for v in sucesores}
    for hijos in sucesores.values():
        for u in hijos:
            grado[u] = grado.get(u, 0) + 1
    cola = deque(v for v, g in grado.items() if g == 0)
    orden = []
    while cola:
        v = cola.popleft()
```

paso	sale de <i>Q</i>	grado llega a 0	cola <i>Q</i> tras el paso
0			bwa_index fastp_A fastp_B fastp_C
1	bwa_index	—	fastp_A fastp_B fastp_C
2	fastp_A	mapear_A	fastp_B fastp_C mapear_A
3	fastp_B	mapear_B	fastp_C mapear_A mapear_B
4	fastp_C	mapear_C multiqc	mapear_A mapear_B mapear_C multiqc
5	mapear_A	dedup_A	mapear_B mapear_C multiqc dedup_A
6	mapear_B	dedup_B	mapear_C multiqc dedup_A dedup_B
7	mapear_C	dedup_C	multiqc dedup_A dedup_B dedup_C
8	multiqc	—	dedup_A dedup_B dedup_C
9	dedup_A	—	dedup_B dedup_C
10	dedup_B	—	dedup_C
11	dedup_C	llamar	llamar
12	llamar	filtrar	filtrar
13	filtrar	anotar consenso_A consenso_B consenso_C	anotar consenso_A consenso_B consenso_C
14	anotar	—	consenso_A consenso_B consenso_C
15	consenso_A	—	consenso_B consenso_C
16	consenso_B	—	consenso_C
17	consenso_C	filogenia	filogenia
18	filogenia	all	all
19	all	—	vacía

Figura 18.2. El algoritmo de Kahn aplicado al grafo de la figura 18.1, con una cola FIFO y desempate por el orden en que aparecen las reglas en el Snakefile. En el paso 0 la cola contiene las cuatro fuentes (grado de entrada 0). Cada fila extrae un trabajo, decrementa el grado de sus sucesores y encola los que llegan a cero. Observe tres detalles: mapear_A solo se libera en el paso 2, cuando ya salieron sus *dos* predecesores (el índice y fastp_A); multiqc espera a los tres fastp (paso 4); y llamar espera a los tres dedup (paso 11). La última columna es, literalmente, la lista de trabajos que un planificador podría lanzar en ese instante. Tabla generada con figuras/cap18/generar.py.

```

orden.append(v)
for u in sucesores.get(v, []):
    grado[u] -= 1
    if grado[u] == 0:
        cola.append(u)
if len(orden) < len(grado):                                # teorema, punto (iii)
    resto = [v for v, g in grado.items() if g > 0]
    raise ValueError(f"cíclo entre: {resto}")
return orden

```

18.1.4 Dependencias por archivos: la herencia de Make

La idea de describir un cálculo como un grafo de archivos es mucho más antigua que la bioinformática. **make**, escrito en los Laboratorios Bell en la segunda mitad de los setenta y descrito por Feldman (1979), nació para recompilar programas en C: un *makefile* declara, para cada *objetivo*, de qué *prerrequisitos* depende y qué orden lo fabrica. **make** reconstruye el grafo, lo ordena y ejecuta solo lo necesario. El criterio de “necesario” es la clave de toda la familia de herramientas que le siguió:

Definición 18.2. (Objetivo desactualizado)

Sea $\text{pre}(t)$ el conjunto de prerrequisitos del objetivo t y $\tau(f)$ la fecha de última modificación del archivo f . El objetivo t debe reconstruirse si y solo si

$$\text{rehacer}(t) = \neg \text{existe}(t) \vee \bigvee_{d \in \text{pre}(t)} [\text{rehacer}(d) \vee \tau(d) > \tau(t)]. \quad (18.3)$$

DONDE

$\text{pre}(t)$	Archivos de los que depende t (aristas entrantes en el grafo de archivos).
$\tau(f)$	Marca de tiempo de modificación (<i>mtime</i>) que guarda el sistema de archivos.
$\neg, \vee$	Negación y disyunción lógicas.

La definición es recursiva y se evalúa precisamente en orden topológico: cuando llegamos a t , ya sabemos si cada uno de sus prerrequisitos se rehace. Su consecuencia práctica es que un cambio se propaga a *todos los descendientes* del archivo modificado en el grafo, y a nadie más. En la figura 18.1, modificar las lecturas de la muestra A obliga a rehacer 11 de los 18 trabajos reales (su **fastp**, su mapeo y su **dedup**, el

llamado conjunto y todo lo que cuelga de él, y `multiqc`); cambiar el umbral de `filtrar`, solo 6; retocar `multiqc`, solo uno. Esta es la *mise en place* de la intuición inicial.

🔗 De los compiladores al genoma

La idea de `make` era sencilla: describir una vez las dependencias entre archivos y dejar que el programa, comparando fechas de modificación, decida qué recompilar (Feldman, 1979). Cuatro décadas después, el problema que resuelve sigue vivo con otros nombres: una tabla de conteos calculada con una versión vieja del GTF, una figura que no se regeneró tras corregir un filtro. Muchos bioinformáticos usaron `make` para sus flujos durante años, pero sus limitaciones (un único comodín % por regla, un lenguaje propio poco expresivo, ninguna noción de clúster ni de entornos de software) motivaron la generación de gestores que estudiamos a continuación.

18.1.5 Snakemake: reglas, comodines e inferencia hacia atrás

Snakemake (Köster & Rahmann, 2012) conserva la filosofía de `make` (reglas que producen archivos a partir de archivos) pero la escribe en un lenguaje que es una extensión de Python. Cada rule declara sus input, output y la orden que los conecta (shell, run con Python o script con un archivo externo), y puede añadir threads, log, params, resources o el entorno de software (conda, container). La revisión de Mölder et al. (2021) describe cómo el diseño persigue un análisis *sostenible*, que combine reproducibilidad, adaptabilidad y transparencia.

Comodines. La pieza que hace escalar una regla es el *comodín* (*wildcard*): un nombre entre llaves dentro de un patrón de archivo, como `mapeo/{m}.dedup.bam`. Formalmente, un patrón p con comodines $w_1, \dots, w_k$ define una expresión regular $\rho(p)$ en la que cada $\{w_i\}$ se sustituye por un grupo con nombre (por defecto, $\cdot +$) y el resto del texto se toma literalmente. Un archivo f coincide con p si $\rho(p)$ reconoce f completo, y la coincidencia asigna valores $\sigma = \{w_i \mapsto s_i\}$ que luego se sustituyen en los patrones de entrada:

$$f \in \mathcal{L}(\rho(p_{\text{sal}})) \implies \text{entradas}(f) = \{\sigma(p) : p \in P_{\text{ent}}\}. \quad (18.4)$$

DONDE

$p_{\text{sal}}, P_{\text{ent}}$	Patrón de salida de la regla y conjunto de sus patrones de entrada.
$\rho(p)$	Expresión regular asociada al patrón p .
$\mathcal{L}(\rho)$	Lenguaje reconocido por ρ : el conjunto de nombres de archivo que coinciden.
σ	Asignación de valores a los comodines obtenida de la coincidencia.

Inferencia hacia atrás. A diferencia de un *script*, Snakemake no empieza por el principio sino por el final. Parte de los archivos objetivo (las entradas de la primera regla, por convención llamada `all`) y, para cada uno, busca la regla cuya salida coincide; con la asignación σ calcula las entradas, y repite el proceso recursivamente con cada entrada hasta llegar a archivos que ya existen. El grafo de trabajos de la figura 18.1 *no está escrito en ninguna parte*: se deduce de los nombres de archivo. Es un modelo *bajo demanda* (*pull*): solo se construye lo que alguien pidió. El siguiente programa reproduce la idea en unas pocas líneas.

```
inferir_dag.py

import re

REGLAS = [ # (nombre, salidas, entradas)
    ("fastp", ["limpio/{m}_R1.fq.gz", "limpio/{m}_R2.fq.gz"],
             ["datos/{m}_R1.fastq.gz", "datos/{m}_R2.fastq.gz"]),
    ("mapear", ["mapeo/{m}.ordenado.bam"],
             ["limpio/{m}_R1.fq.gz", "limpio/{m}_R2.fq.gz"]),
    ("dedup", ["mapeo/{m}.dedup.bam"], ["mapeo/{m}.ordenado.bam"]),
]

def a_regex(patron):
    # 'mapeo/{m}.bam' -> 'mapeo/(?P<m>.+)\.bam$'
    partes = re.split(r"\{(\w+)\}", patron)
    rx = "".join(re.escape(p) if i % 2 == 0 else f"(?P<{p}>.+)"
                 for i, p in enumerate(partes))
    return re.compile(rx + "$")

def inferir(objetivo, dag):
    for nombre, salidas, entradas in REGLAS:
        for s in salidas:
            hallado = a_regex(s).match(objetivo)
            if hallado:
                comodines = hallado.groupdict()
```

```

        ins = [e.format(**comodines) for e in entradas]
        dag[objetivo] = (nombre, comodines, ins)
        for f in ins:
            if f not in dag:
                inferir(f, dag)
        return
    dag[objetivo] = ("(archivo fuente)", {}, [])

dag = {}
inferir("mapeo/A.dedup.bam", dag)
for f, (regla, w, ins) in dag.items():
    print(f"{f:24s} <- {regla:16s} {w}")

```

Salida de inferir_dag.py

```

mapeo/A.dedup.bam      <- dedup      {'m': 'A'}
mapeo/A.ordenado.bam   <- mapear     {'m': 'A'}
limpio/A_R1.fq.gz      <- fastp      {'m': 'A'}
datos/A_R1.fastq.gz    <- (archivo fuente) {}
datos/A_R2.fastq.gz    <- (archivo fuente) {}
limpio/A_R2.fq.gz      <- fastp      {'m': 'A'}

```

Nuestra versión de juguete no distingue que los dos archivos de lecturas limpias de la muestra A salen de *un mismo* trabajo; Snakemake sí, porque agrupa todas las salidas de una regla con la misma asignación σ . Tampoco resuelve ambigüedades: si dos reglas pueden producir el mismo archivo, Snakemake se detiene con un error y pide al autor que las ordene (`ruleorder`) o que restrinja los comodines con expresiones regulares (`wildcard_constraints`).

El flujo completo. El *Snakefile* siguiente implementa el grafo de la figura 18.1. Cada regla es un paso que ya conocemos: `fastp` (capítulo 6), BWA-MEM y `samtools` (capítulo 7), `bcftools` y SnpEff (capítulo 9) e IQ-TREE (capítulo 5). Las muestras, la referencia y los umbrales se leen de un archivo de configuración, de modo que el mismo código sirve para 3 o para 300 aislados. Hemos evitado a propósito las tuberías con “|” dentro de las órdenes, para que cada archivo intermedio sea visible; en producción se sustituirían por tuberías o por la función `pipe()` de Snakemake.

Snakefile

```
# De lecturas crudas a variantes anotadas y filogenia
```



```

configfile: "config/config.yaml"

MUESTRAS = config["muestras"]          # p. ej. ["A", "B", "C"]
REF = config["referencia"]              # p. ej. "ref/genoma.fa"

wildcard_constraints:
    m="[A-Za-z0-9-]+"

rule all:
    input:
        "resultados/anotado.vcf",
        "resultados/arboreal.treefile",
        "resultados/multiqc.html",

rule fastp:
    input:
        r1="datos/{m}_R1.fastq.gz",
        r2="datos/{m}_R2.fastq.gz",
    output:
        r1=temp("limpio/{m}_R1.fq.gz"),
        r2=temp("limpio/{m}_R2.fq.gz"),
        js="qc/{m}.fastp.json",
    log: "logs/fastp/{m}.log"
    threads: 4
    conda: "envs/qc.yaml"
    shell:
        "fastp -i {input.r1} -I {input.r2} -w {threads} "
        "-o {output.r1} -O {output.r2} -j {output.js} 2> {log}"

rule bwa_index:
    input: REF
    output: multiext(REF, ".amb", ".ann", ".bwt", ".pac", ".sa")
    conda: "envs/mapeo.yaml"
    shell: "bwa index {input}"

rule mapear:
    input:
        ref=REF,
        idx=rules.bwa_index.output,
        r1="limpio/{m}_R1.fq.gz",
        r2="limpio/{m}_R2.fq.gz",

```

```

output: temp("mapeo/{m}.ordenado.bam")
params: rg=lambda wc: f"@RG\\tID:{wc.m}\\tSM:{wc.m}"
log: "logs/mapear/{m}.log"
threads: 8
conda: "envs/mapeo.yaml"
shell:
    """
    bwa mem -t {threads} -R '{params.rg}' {input.ref} \
        {input.r1} {input.r2} > {output}.sam 2> {log}
    samtools fixmate -m {output}.sam {output}.fm.bam
    samtools sort -@ {threads} -o {output} {output}.fm.bam
    rm {output}.sam {output}.fm.bam
    """

```

rule dedup:

```

input: "mapeo/{m}.ordenado.bam"
output:
    bam="mapeo/{m}.dedup.bam",
    bai="mapeo/{m}.dedup.bam.bai",
conda: "envs/mapeo.yaml"
shell:
    "samtools markdup {input} {output.bam} && "
    "samtools index {output.bam}"

```

rule llamar:

```

input:
    ref=REF,
    bams=expand("mapeo/{m}.dedup.bam", m=MUESTRAS),
    bais=expand("mapeo/{m}.dedup.bam.bai", m=MUESTRAS),
output: "variantes/crudo.bcf"
conda: "envs/variantes.yaml"
shell:
    """
    bcftools mpileup -f {input.ref} -a AD,DP -Ob \
        -o {output}.pila {input.bams}
    bcftools call --ploidy 1 -mv -V indels -Ob \
        -o {output} {output}.pila
    rm {output}.pila
    """

```

rule filtrar:

```

input: "variantes/crudo.bcf"

```

```

output:
    vcf="variantes/filtrado.vcf.gz",
    tbi="variantes/filtrado.vcf.gz.tbi",
params: expr=config["filtro"]      # "QUAL>=30 && INFO/DP>=10"
conda: "envs/variantes.yaml"
shell:
    "bcftools filter -i '{params.expr}' -Oz -o {output.vcf} "
    "{input} && bcftools index -t {output.vcf}"

```

```

rule anotar:
    input: "variantes/filtrado.vcf.gz"
    output: "resultados/anotado.vcf"
    params: db=config["snpeff_db"]
    conda: "envs/anotacion.yaml"
    shell: "snpeff ann -noStats {params.db} {input} > {output}"

```

```

rule consenso:
    input:
        ref=REF,
        vcf="variantes/filtrado.vcf.gz",
        tbi="variantes/filtrado.vcf.gz.tbi",
    output: "consenso/{m}.fa"
    conda: "envs/variantes.yaml"
    shell:
        """
        bcftools consensus -f {input.ref} -s {wildcards.m} \
        -o {output}.tmp {input.vcf}
        sed 's/^>.*>{wildcards.m}/' {output}.tmp > {output}
        rm {output}.tmp
        """

```

```

rule filogenia:
    input:
        ref=REF,
        fas=expand("consenso/{m}.fa", m=MUESTRAS),
    output:
        arbol="resultados/arbol.treefile",
        aln="resultados/aln.fa",
    threads: 4
    conda: "envs/filogenia.yaml"
    shell:
        """

```

```
cat {input.ref} {input.fas} > {output.aln}
iqtree2 -s {output.aln} -m GTR+G -B 1000 \
  -T {threads} --prefix resultados/arbol
"""
```

```
rule multiqc:
    input: expand("qc/{m}.fastp.json", m=MUESTRAS)
    output: "resultados/multiqc.html"
    conda: "envs/qc.yaml"
    shell: "multiqc qc -n multiqc.html -o resultados"
```

Vale la pena detenerse en algunos detalles. `expand()` genera la lista de archivos de todas las muestras y es lo que crea las aristas “muchos a uno” hacia `llamar`, `filogenia` y `multiqc`. `temp()` marca archivos que se borran en cuanto ningún trabajo pendiente los necesita, lo que ahorra disco sin romper la reanudación. `--ploidy 1` y `-V indels` reflejan que trabajamos con un genoma bacteriano haploide y que, para la filogenia, solo usamos SNV: así las secuencias consenso tienen la misma longitud que la referencia y, concatenadas, forman directamente un alineamiento. Y el orden de las reglas en el archivo es irrelevante (salvo la primera, que define el objetivo por defecto): el orden de ejecución lo decide el grafo.

Explorar y ejecutar el flujo con Snakemake

```
snakemake -n                # ensayo en seco: qué trabajos y por qué
snakemake --dag > dag.dot    # grafo de trabajos en formato Graphviz
dot -Tpdf dag.dot -o dag.pdf
snakemake --cores 16 --sdm conda # entornos conda por regla
snakemake --cores 16 -R filtrar # rehace filtrar y descendientes
snakemake --report informe.html # informe HTML con procedencia
```

⚠ Versiones de Snakemake y opciones de la línea de órdenes

La interfaz cambió en la versión 8: la antigua `--use-conda` se escribe ahora `--software-deployment-method conda` (abreviada `--sdm conda`), y el envío a clústeres pasó a *complementos* de ejecución (`--executor slurm`, por ejemplo). Consulte siempre `snakemake --help` de la versión que tenga fijada en su entorno y anote esa versión en el README.

18.1.6 Nextflow: procesos, canales y flujo de datos

Nextflow (Di Tommaso et al., 2017) parte de otro modelo de computación: el *flujo de datos* (*dataflow*). En lugar de reglas que producen archivos con nombre, hay *procesos* aislados que se comunican a través de *canales*, colas asíncronas por las que circulan valores (rutas de archivos, cadenas, tuplas). Un proceso se *dispara* cada vez que encuentra un elemento disponible en cada uno de sus canales de entrada, y deposita sus resultados en canales de salida que alimentan a otros procesos. No hay que declarar nombres de archivos intermedios ni reconstruir el grafo hacia atrás: el grafo se desenvuelve hacia adelante, a medida que los datos llegan. Es un modelo *por empuje* (*push*).

Definición 18.3. (Canales de cola y de valor)

Un *canal de cola* es una secuencia finita de elementos que se consumen: cada elemento es leído por un único disparo del proceso. Un *canal de valor* contiene un solo elemento que puede leerse un número ilimitado de veces. Si un proceso tiene canales de entrada de cola $c_1, \dots, c_k$ (y quizá otros de valor), el número de tareas que ejecuta es

$$n_{\text{tareas}} = \min_{1 \leq i \leq k} |c_i|, \quad (18.5)$$

y vale 1 si todas sus entradas son canales de valor.

DONDE

c_i	i -ésimo canal de entrada <i>de cola</i> del proceso.
$ c_i $	Número de elementos que llegan por ese canal.
n_{tareas}	Número de veces que el proceso se ejecuta (tareas).

La ecuación (18.5) explica el error más frecuente de quien empieza con Nextflow. Si el índice de BWA saliera por un canal de *cola* (con un único elemento) y las lecturas por otro canal de cola con tres elementos, el proceso de mapeo se ejecutaría $\min(1, 3) = 1$ vez: solo se mapearía la primera muestra, sin ningún mensaje de error. La solución es que el índice viaje por un canal de *valor*, que se “relee” para cada muestra. En Nextflow, un proceso cuyas entradas son todas canales de valor produce canales de valor, y el operador `collect()` convierte una cola en un único valor (la lista completa), que es justo lo que necesita el llamado conjunto. La figura 18.3 dibuja este flujo de datos.

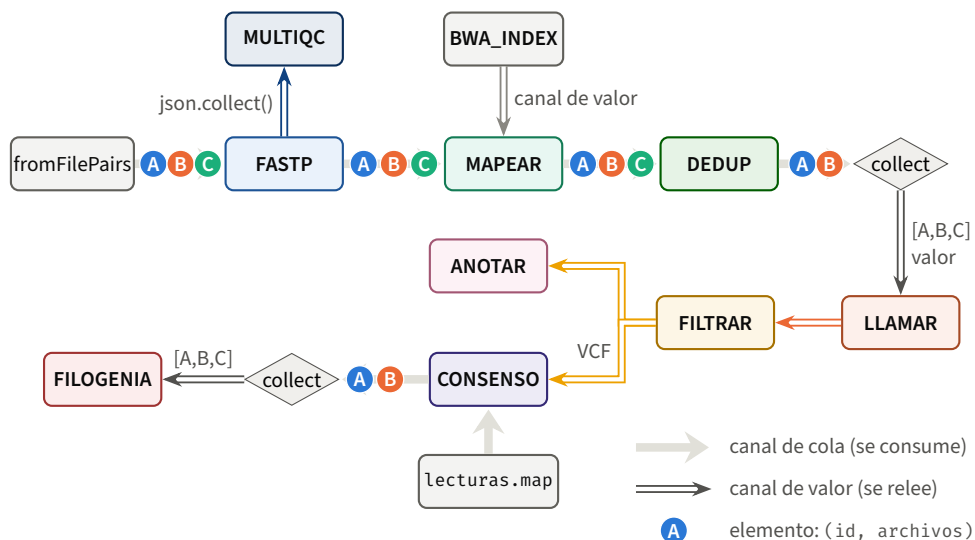


Figura 18.3. El mismo flujo de la figura 18.1 visto como flujo de datos en Nextflow. Los procesos (cajas) están aislados y solo se comunican por canales. Por los canales de cola (tubos grises) circulan elementos, uno por muestra, que cada proceso consume en cuanto llegan: la muestra A puede estar ya en DEDUP mientras C sigue en FASTP. El índice de BWA viaja por un canal de valor (doble línea) que MAPEAR relee para cada muestra, y el operador `collect` convierte la cola de BAM en un único valor, la lista completa, que dispara una sola vez el llamado conjunto. Lo mismo ocurre con el VCF filtrado, que CONSENSO relee para cada identificador de muestra, y con las secuencias consenso, que se reúnen antes de la filogenia.

El flujo completo en Nextflow. El archivo `main.nf` siguiente es el equivalente del Snakefile anterior, escrito en la sintaxis DSL2 (el lenguaje de Nextflow es una extensión de Groovy). Cada `process` declara sus entradas, sus salidas (con nombres `emit` para distinguirlas) y un bloque `script`; el bloque `workflow` final conecta los procesos con canales, como si fueran funciones.

```
main.nf

params.lecturas = "datos/*_R{1,2}.fastq.gz"
params.ref      = "ref/genoma.fa"
params.snpeff   = "Mycobacterium_tuberculosis_h37rv"
params.outdir   = "resultados"

process FASTP {
```

```

tag "${id}"
conda "envs/qc.yaml"
cpus 4
input:
tuple val(id), path(reads)
output:
tuple val(id), path("${id}_R*.fq.gz"), emit: reads
path "${id}.fastp.json", emit: json
script:
"""
fastp -i ${reads[0]} -I ${reads[1]} -w ${task.cpus} \
-o ${id}_R1.fq.gz -O ${id}_R2.fq.gz -j ${id}.fastp.json
"""
}

process BWA_INDEX {
conda "envs/mapeo.yaml"
input:
path fa
output:
tuple path(fa), path("${fa}.*")
script:
"""
bwa index ${fa}
"""
}

process MAPEAR {
tag "${id}"
conda "envs/mapeo.yaml"
cpus 8
input:
tuple val(id), path(reads)
tuple path(fa), path(idx)
output:
tuple val(id), path("${id}.ordenado.bam")
script:
"""
bwa mem -t ${task.cpus} -R '@RG\tID:${id}\tSM:${id}' \
${fa} ${reads} > ${id}.sam
samtools fixmate -m ${id}.sam ${id}.fm.bam
samtools sort -@ ${task.cpus} -o ${id}.ordenado.bam ${id}.fm.bam
"""
}

```

```

process DEDUP {
    tag "${id}"
    conda "envs/mapeo.yaml"
    input:
        tuple val(id), path(bam)
    output:
        tuple val(id), path("${id}.dedup.bam"), path("${id}.dedup.bam.bai")
    script:
        """
        samtools markdup ${bam} ${id}.dedup.bam
        samtools index ${id}.dedup.bam
        """
}

process LLAMAR {
    conda "envs/variantes.yaml"
    input:
        path bams
        path fa
    output:
        path "crudo.bcf"
    script:
        """
        bcftools mpileup -f ${fa} -a AD,DP -Ob -o pila.bcf *.dedup.bam
        bcftools call --ploidy 1 -mv -V indels -Ob -o crudo.bcf pila.bcf
        """
}

process FILTRAR {
    publishDir params.outdir, mode: 'copy'
    conda "envs/variantes.yaml"
    input:
        path bcf
    output:
        tuple path("filtrado.vcf.gz"), path("filtrado.vcf.gz.tbi")
    script:
        """
        bcftools filter -i 'QUAL>=30 && INFO/DP>=10' \\\
            -Oz -o filtrado.vcf.gz ${bcf}
        bcftools index -t filtrado.vcf.gz
        """
}

process ANOTAR {
    publishDir params.outdir, mode: 'copy'

```



```

conda "envs/anotacion.yaml"
input:
tuple path(vcf), path(tbi)
output:
path "anotado.vcf"
script:
"""
snpeff ann -noStats ${params.snpeff} ${vcf} > anotado.vcf
"""
}

process CONSENSO {
tag "${id}"
conda "envs/variantes.yaml"
input:
val id
tuple path(vcf), path(tbi)
path fa
output:
path "${id}.fa"
script:
"""
bcftools consensus -f ${fa} -s ${id} -o tmp.fa ${vcf}
sed 's/^>.*>${id}/' tmp.fa > ${id}.fa
"""
}

process FILOGENIA {
publishDir params.outdir, mode: 'copy'
conda "envs/filogenia.yaml"
cpus 4
input:
path fastas
path fa
output:
path "arbol.treefile"
script:
"""
cat ${fa} ${fastas} > aln.fa
iqtree2 -s aln.fa -m GTR+G -B 1000 -T ${task.cpus} --prefix arbol
"""
}

process MULTIQC {
publishDir params.outdir, mode: 'copy'

```

```

conda "envs/qc.yaml"
input:
path jsons
output:
path "multiqc.html"
script:
"""
multiqc . -n multiqc.html
"""
}

workflow {
  lecturas = Channel.fromFilePairs(params.lecturas) // cola
  ref = file(params.ref) // valor
  FASTP(lecturas)
  MULTIQC(FASTP.out.json.collect())
  BWA_INDEX(ref) // valor
  MAPEAR(FASTP.out.reads, BWA_INDEX.out)
  DEDUP(MAPEAR.out)
  bams = DEDUP.out.map { id, bam, bai -> [bam, bai] }.collect()
  LLAMAR(bams, ref)
  FILTRAR(LLAMAR.out)
  ANOTAR(FILTRAR.out)
  ids = lecturas.map { id, reads -> id }
  CONSENSO(ids, FILTRAR.out, ref)
  FILOGENIA(CONSENSO.out.collect(), ref)
}

```

Compare los dos archivos. En Snakemake, las dependencias *emergen* de los nombres de archivo; en Nextflow se *declaran* al conectar canales en el bloque workflow. Snakemake necesita conocer el grafo completo antes de empezar (lo construye hacia atrás desde el objetivo), mientras que Nextflow puede empezar a mapear la muestra A antes de saber cuántas muestras hay, lo que resulta natural cuando las muestras llegan por un flujo continuo. Cada tarea de Nextflow se ejecuta en su propio directorio de trabajo aislado, con enlaces a sus entradas, de modo que dos tareas nunca pisan los archivos de otra; publishDir copia a la carpeta de resultados solo lo que nos interesa conservar.

Ejecutar main.nf y un flujo de nf-core

```

nextflow run main.nf -with-conda -resume
nextflow run main.nf -profile docker -with-report -with-dag dag.html

```

```
# Flujo comunitario: fije siempre la version con -r
nextflow run nf-core/viralrecon -r <version> -profile test,docker \
--outdir resultados_prueba
```

nf-core y CWL. Sobre Nextflow creció nf-core (Ewels et al., 2020), una comunidad que mantiene flujos curados (RNA-seq, llamado de variantes, metagenómica, epigenómica...) con normas estrictas: cada flujo tiene un perfil `test` con datos diminutos que se ejecuta en integración continua, contenedores para cada herramienta, parámetros documentados con un esquema y versiones publicadas. Antes de escribir un flujo propio para un análisis estándar, vale la pena buscar si ya existe uno allí. En otra dirección, el *Common Workflow Language* (CWL) no es un gestor sino un *estándar* abierto para describir herramientas y flujos en YAML de forma declarativa, que varios motores distintos pueden ejecutar (Crusoe et al., 2022). Su lema es la portabilidad: el mismo documento debería producir el mismo resultado en un portátil, en un clúster o en la nube, y en cualquier motor que cumpla la especificación. La tabla 18.1 resume las diferencias.

18.1.7 Congelar el entorno: Bioconda y contenedores

Un flujo que funciona en mi portátil y no en el suyo no es reproducible. La causa casi nunca es el código del flujo, sino el *entorno*: la versión de `samtools`, de la biblioteca `htslib` contra la que se compiló, de Python, de la biblioteca matemática del sistema. Di Tommaso et al. (2017) mostraron que un mismo análisis ejecutado en sistemas operativos distintos podía producir resultados numéricamente diferentes, y que empaquetar las herramientas en contenedores eliminaba esa discrepancia. Grüning et al. (2018) proponen pensar la reproducibilidad como una pila de capas, de los paquetes al sistema operativo, y fijar cada una con la herramienta adecuada.

Gestores de paquetes. Conda instala binarios precompilados con todas sus dependencias en un entorno aislado, sin permisos de administrador. Bioconda (The Bioconda Team et al., 2018) es el canal comunitario de recetas para bioinformática: ya reúne miles de paquetes, y es la forma estándar de instalar casi cualquier herramienta de este libro. Cada regla del Snakefile o proceso de `main.nf` apunta a un pequeño archivo de entorno con versiones *exactas*:

Tabla 18.1. Comparación esquemática de cuatro formas de describir un flujo. Las cuatro construyen y ordenan un grafo acíclico dirigido; difieren en cómo se escribe, cómo se infiere y qué ecosistema las rodea. Para una comparación más amplia, véase Wratten et al. (2021).

	Make	Snakemake	Nextflow	CWL
Lenguaje	propio	Python + reglas	Groovy (DSL2)	YAML declarativo
Unidad	archivo	archivo con comodines	canal de valores	paso con entradas tipadas
Grafo	hacia atrás (<i>pull</i>)	hacia atrás (<i>pull</i>)	hacia adelante (<i>push</i>)	declarado explícitamente
Reanudación	marcas de tiempo	marcas de tiempo, parámetros, código	<i>hash</i> de cada tarea (-resume)	depende del motor
Entornos	ninguno	conda, contenedores	conda, contenedores	contenedores
Ejecución	local	local, clúster, nube	local, clúster, nube	según el motor
Comunidad	general	catálogo de flujos	nf-core	estándar multi-motor

```
envs/mapeo.yaml

channels:
  - conda-forge
  - bioconda
dependencies:
  - bwa=0.7.18
  - samtools=1.20
```

Un archivo así fija la versión de las herramientas que nombramos, pero no la de sus dependencias transitivas, que el resolutor elige en el momento de instalar. Para congelarlo todo se exporta un archivo de bloqueo con la lista explícita de paquetes (`conda list --explicit` o herramientas como `conda-lock`), o se da el paso siguiente: el contenedor.

Contenedores. Un contenedor empaqueta un sistema de archivos completo (binarios, bibliotecas, configuración) en una *imagen* que se ejecuta aislada sobre el núcleo del sistema anfitrión, sin el coste de una máquina virtual. Docker popularizó el formato, pero su demonio con privilegios de administrador lo hace inaceptable en la mayoría de los clústeres compartidos. Singularity (hoy también distribuido como Apptainer) se diseñó para ese entorno: el usuario dentro del contenedor es el mismo que fuera, no hace falta ningún demonio privilegiado y la imagen es un único archivo que se copia como cualquier otro (Kurtzer et al., 2017). El proyecto BioContainers (da Veiga Leprevost et al., 2017) construye automáticamente una imagen para cada paquete de Bioconda, de modo que la misma versión de una herramienta está disponible como paquete conda, como imagen Docker y como imagen Singularity.

⚠ La etiqueta `latest` no es una versión

Una imagen como `samtools:latest` cambia de contenido cada vez que alguien publica una versión nueva. Fije siempre una etiqueta de versión y, para máxima garantía, el *digest* de la imagen (`imagen@sha256:...`), que es la suma de verificación de su contenido y no puede cambiar. Lo mismo vale para los flujos de `nf-core` (`-r` con una versión) y para los entornos `conda` (versiones exactas, nunca solo el nombre del paquete).

18.1.8 Caché y reanudación

La ecuación (18.3) decide qué rehacer comparando marcas de tiempo. Es un criterio barato pero frágil: copiar un directorio, descomprimir un respaldo o tocar un archivo sin cambiar su contenido altera las fechas y desencadena recálculos innecesarios; y, al revés, cambiar un parámetro o la versión de una herramienta *no* altera ninguna fecha. Por eso las versiones modernas de Snakemake también vuelven a ejecutar un trabajo cuando cambian sus parámetros, su código o su entorno de software, y Nextflow abandona las fechas y usa un *hash* por tarea.

Definición 18.4. (Clave de caché de una tarea)

Sea H una función *hash* criptográfica. La clave de la tarea v es

$$k(v) = H(\text{orden}(v) \parallel \text{parámetros}(v) \parallel \text{entorno}(v) \parallel H(x_1) \parallel \dots \parallel H(x_r)), \quad (18.6)$$

donde $x_1, \dots, x_r$ son sus archivos de entrada. Si existe un resultado guardado con la clave $k(v)$, la tarea se da por hecha y se reutiliza.

DONDE

H	Función <i>hash</i> (p. ej., SHA-256): transforma cualquier cadena de bits en una huella de longitud fija.
$\parallel$	Concatenación de cadenas.
$\text{orden}(v)$	Texto de la orden o <i>script</i> de la tarea, ya con sus variables sustituidas.
$\text{entorno}(v)$	Identificador del contenedor o del entorno de software.
x_j	j -ésimo archivo de entrada de la tarea.

Como las entradas de una tarea son salidas de otras, un cambio en cualquier punto altera las claves de todos sus descendientes, y solo de ellos: la ecuación (18.6) reproduce la propagación de la ecuación (18.3) sin depender de relojes. En Nextflow, la opción `-resume` aplica esta idea con el directorio `work/`: por defecto, la huella de un archivo de entrada se calcula a partir de su ruta, tamaño y fecha (rápido), y con la directiva `cache 'deep'` a partir de su contenido (lento pero inmune a copias y restauraciones).

```
clave_tarea.py

import hashlib, json

def clave_tarea(orden, params, entorno, entradas):
    """Huella de una tarea al estilo de -resume (modo 'deep')."""
    h = hashlib.sha256()
    for parte in (orden, json.dumps(params, sort_keys=True), entorno):
        h.update(parte.encode())
    for ruta in sorted(entradas):
        with open(ruta, "rb") as fh:
            h.update(hashlib.sha256(fh.read()).digest())
    return h.hexdigest()[:12]
```

18.1.9 Ejecución en clúster y en la nube

El mismo grafo puede ejecutarse en un portátil, en un clúster con un gestor de colas (SLURM, PBS, SGE) o en la nube (AWS Batch, Google Cloud, Kubernetes). Ambos gestores separan la *lógica* del flujo de la *infraestructura*: en Snakemake mediante *perfiles* y complementos de ejecución; en Nextflow mediante el archivo `nextflow.config`, donde cada perfil elige el ejecutor, los contenedores y los recursos por proceso. El gestor se convierte en un planificador: mantiene la cola de trabajos listos del algoritmo de Kahn, envía cada uno al gestor de colas en cuanto se libera, vigila su terminación y reintenta los que fallan (por ejemplo, con más memoria).

```
nextflow.config

profiles {
  laptop {
    process.executor = 'local'
    conda.enabled = true
  }
  cluster {
    process.executor = 'slurm'
    process.queue = 'normal'
    singularity.enabled = true
    process {
      withName: 'MAPEAR' { cpus = 8; memory = '16 GB' }
      errorStrategy = 'retry'
      maxRetries = 2
    }
  }
}
```

18.1.10 Paralelismo: trabajo, camino crítico y la ley de Amdahl

¿Cuánto más rápido termina el flujo si pasamos de 4 a 64 núcleos? La pregunta tiene una respuesta precisa en términos del grafo. Asignemos a cada trabajo v una duración t_v y supongamos que cada trabajo ocupa un núcleo.

Definición 18.5. (Trabajo y camino crítico)

El *trabajo* de un flujo es la suma de las duraciones, $W = \sum_{v \in V} t_v$, que es el tiempo en un solo núcleo, $T_1 = W$. El *camino crítico* (*span*) es la duración del camino dirigido más largo,

$$S = \max_{\text{caminos } v_1 \rightarrow \dots \rightarrow v_\ell} \sum_{j=1}^{\ell} t_{v_j}, \quad (18.7)$$

que se calcula en tiempo $O(|V| + |E|)$ recorriendo el grafo en orden topológico con $F(v) = t_v + \max_{u \rightarrow v} F(u)$.

DONDE

t_v	Duración del trabajo v en un núcleo.
W, T_1	Trabajo total; tiempo de ejecución en serie.
S	Camino crítico: tiempo mínimo aun con infinitos núcleos.
$F(v)$	Instante más temprano en que puede terminar v .

Teorema 18.3. (Cotas del planificador voraz)

Sea T_P el tiempo que tarda un planificador voraz (que nunca deja un núcleo ocioso si hay un trabajo listo) con P núcleos. Entonces

$$\max\left(\frac{W}{P}, S\right) \leq T_P \leq \frac{W}{P} + S. \quad (18.8)$$

Demostración. La cota inferior es evidente: P núcleos no pueden hacer W unidades de trabajo en menos de W/P , y ningún planificador puede acortar la cadena de dependencias más larga. Para la superior, dividamos el tiempo en instantes *completos*, en que los P núcleos están ocupados, e *incompletos*. En los completos se realizan P unidades de trabajo por unidad de tiempo, así que suman como mucho W/P . En un instante incompleto, el planificador voraz está ejecutando *todos* los trabajos listos, entre ellos el primer trabajo pendiente de cualquier camino crítico del grafo restante; por lo tanto, la longitud de ese camino disminuye a la misma velocidad que el reloj, y los instantes incompletos suman como mucho S . $\square$

La ecuación (18.8) dice que la aceleración máxima alcanzable es W/S , el *parallelismo* del flujo, por muchos núcleos que compremos. Un caso particular famoso

se obtiene cuando una fracción s del trabajo es intrínsecamente serial y el resto es perfectamente divisible.

Teorema 18.4. (Ley de Amdahl)

Si una fracción s del tiempo en serie no puede paralelizarse y la fracción $1 - s$ se reparte sin coste entre P núcleos, la aceleración es

$$\psi(P) = \frac{T_1}{T_P} = \frac{T_1}{sT_1 + (1-s)T_1/P} = \frac{1}{s + \frac{1-s}{P}} \xrightarrow{P \rightarrow \infty} \frac{1}{s}. \quad (18.9)$$

DONDE

s	Fracción serial del trabajo ($0 < s \leq 1$).
P	Número de núcleos (o de trabajos simultáneos).
$\psi(P)$	Aceleración (<i>speedup</i>) con P núcleos respecto de uno.

El argumento es de Amdahl (1967), que lo usó para defender los procesadores rápidos frente a las máquinas masivamente paralelas: si el 10 % del trabajo es serial ($s = 0,1$), 8 núcleos dan una aceleración de 4,71, 64 núcleos apenas 8,77, y ninguna cantidad de núcleos supera 10. Gustafson (1988) replicó que en la práctica no se resuelve el mismo problema más deprisa, sino un problema *más grande* en el mismo tiempo: si con P núcleos la parte paralela crece en proporción a P y s es la fracción serial *medida en la máquina paralela*, la aceleración escalada es

$$\psi_G(P) = s + (1-s)P = P - s(P-1), \quad (18.10)$$

que con $s = 0,1$ y $P = 64$ vale 57,7. Ambas leyes son correctas; responden a preguntas distintas. En bioinformática se ven las dos: cuando añadimos muestras a un estudio crece sobre todo la parte paralela (Gustafson), pero cada paso conjunto que las reúne a todas (el llamado de variantes, la filogenia, la normalización de una matriz de conteos) añade un tramo serial que limita la aceleración de una ejecución concreta (Amdahl). Para diagnosticar un flujo real, Karp y Flatt (1990) propusieron estimar la fracción serial *efectiva* a partir de una aceleración medida:

$$e(P) = \frac{1/\psi(P) - 1/P}{1 - 1/P}. \quad (18.11)$$

Si e se mantiene constante al aumentar P , el límite es una parte serial genuina; si crece con P , el problema es la sobrecarga de coordinación.

Ejemplo 18.2. (¿Cuántos núcleos pedir para el proyecto?)

Asignemos al flujo de la figura 18.1 duraciones ilustrativas (en minutos) por trabajo: **fastp** 3, **mapear** 12, **dedup** 4 y **consenso** 1 por muestra; **bwa_index** 2, **filtrar** 1, **anotar** 2 y **multiqc** 1; y, como los pasos conjuntos crecen con el número n de muestras, **llamar** $6 + 3n$ y **filogenia** $4 + 2n$.

Tres muestras. El trabajo es $W = 91$ min y el camino crítico, **fastp** → **mapear** → **dedup** → **llamar** → **filtrar** → **consenso** → **filogenia**, suma $S = 3 + 12 + 4 + 15 + 1 + 1 + 10 = 46$ min. El paralelismo es $W/S = 1,98$: nunca terminaremos en menos de 46 minutos. Con $P = 2$ el planificador voraz tarda 61 min (dentro de las cotas $[45,5; 91,5]$) y con $P = 4$ alcanza ya los 46 min, el mínimo absoluto. Pedir 8 núcleos sería desperdiciar la mitad.

Veinticuatro muestras. Ahora $W = 616$ min y $S = 151$ min, así que $W/S = 4,08$. La simulación del planificador (figura 18.4) da aceleraciones de 3,19 con 8 núcleos, 3,71 con 16 y 4,08 a partir de 32. La fracción serial de Karp-Flatt con 16 núcleos es $e = 0,221$: más de una quinta parte del tiempo es serial. ¿Dónde? En **llamar** ($6 + 3 \cdot 24 = 78$ min) y **filogenia** (52 min), que suman 130 de los 151 min del camino crítico. Si dividimos el llamado en 24 regiones del genoma que se procesan en paralelo y luego se concatenan (*scatter-gather*, 1 min adicional), el camino crítico cae a 77,25 min, el paralelismo sube a 7,99 y la aceleración con 16 núcleos pasa de 3,71 a 6,46 ($e = 0,098$). El siguiente cuello de botella es la filogenia, que habría que paralelizar internamente (IQ-TREE admite varios hilos).

El tiempo de un flujo en paralelo está entre $\max(W/P, S)$ y $W/P + S$. Más núcleos solo ayudan mientras W/P domine; después, la única forma de ir más rápido es acortar el camino crítico.

Planificar no es trivial

Encontrar el orden que minimiza T_P para un grafo arbitrario con $P \geq 2$ es un problema NP-difícil, por lo que los gestores usan heurísticas. La más común, la que empleamos en la simulación de la figura 18.4, da prioridad al trabajo listo con el camino más largo

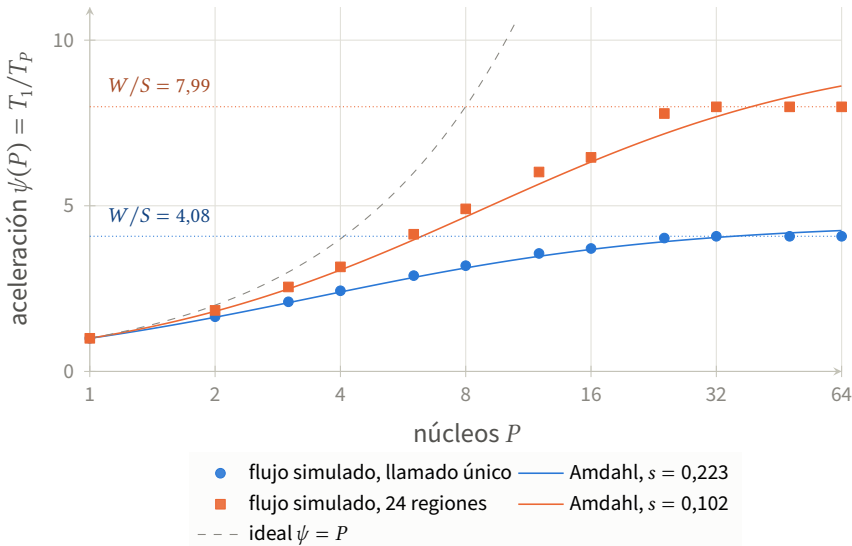


Figura 18.4. Aceleración del flujo de ejemplo con 24 muestras en función del número de núcleos, obtenida simulando un planificador voraz sobre el grafo de trabajos (puntos), junto a la curva de Amdahl (ecuación (18.9)) con la fracción serial que mejor se ajusta (líneas continuas). Con el llamado de variantes como un único trabajo (azul) la aceleración se estanca en el paralelismo $W/S = 4,08$ desde unos 32 núcleos. Dividir el llamado en 24 regiones (naranja) reduce la fracción serial a la mitad y casi duplica la aceleración alcanzable. La lección práctica: antes de pedir más núcleos, busque el camino crítico. Datos de `figuras/cap18/generar.py`.

hasta el final (*critical-path first*); la ecuación (18.8) garantiza que *cualquier* planificador voraz queda a menos de un factor 2 del óptimo, porque $W/P + S \leq 2 \max(W/P, S)$. En la práctica, la planificación real importa menos que otros costes que el modelo ignora: la espera en la cola del clúster, la lectura y escritura de archivos en un sistema de archivos compartido y el arranque de contenedores, que pueden dominar cuando los trabajos son muy cortos. De ahí la recomendación de agrupar trabajos diminutos (`group` en Snakemake, `array` o procesos por lotes en Nextflow).

18.1.11 Buenas prácticas y principios FAIR

Un gestor de flujos resuelve la parte mecánica de la reproducibilidad, pero no sustituye a los hábitos. Sandve et al. (2013) los condensaron en diez reglas que siguen vigentes: registrar cómo se produjo cada resultado; evitar los pasos manuales de manipulación de datos; archivar las versiones exactas de los programas externos;

poner bajo control de versiones todos los *scripts* propios; guardar los resultados intermedios, cuando sea posible, en formatos estándar; anotar las semillas de todo análisis con componentes aleatorios; guardar los datos crudos detrás de cada gráfica; generar salidas jerárquicas que permitan inspeccionar los detalles; conectar cada afirmación del texto con el resultado que la sustenta; y dar acceso público a *scripts*, ejecuciones y resultados. Wilson et al. (2017), pensando en quien no es informático de profesión, proponen un mínimo “suficientemente bueno”: conservar los datos crudos intactos, dar a cada archivo un nombre informativo, escribir código en funciones pequeñas con nombres claros, usar control de versiones y documentar el proyecto con un README que explique cómo reproducirlo.

Los datos y los resultados, por su parte, deben ser *FAIR* (Wilkinson et al., 2016): localizables (*Findable*: con identificadores persistentes y metadatos ricos, indexados en un buscador), accesibles (*Accessible*: recuperables por su identificador mediante un protocolo abierto, aunque el acceso esté controlado), interoperables (*Interoperable*: en formatos y vocabularios compartidos) y reutilizables (*Reusable*: con licencia clara, procedencia detallada y metadatos que sigan los estándares de la comunidad). Un aspecto que suele pasarse por alto es que los principios se dirigen tanto a las personas como a las *máquinas*: un conjunto de datos es FAIR cuando un programa puede encontrarlo, descargarlo y entenderlo sin intervención humana. Un flujo bien escrito es la contraparte computacional de esa idea: describe, de forma ejecutable por una máquina, cómo se pasa de los datos a los resultados.

18.2 Proyecto integrador

 Colab · 18.2 · Proyecto integrador

 repositorio del curso

Una receta que otro pueda cocinar

Un buen libro de cocina no dice “añada sal al gusto y hornee hasta que esté listo”. Dice cuántos gramos, a qué temperatura, durante cuántos minutos y en qué molde; y la prueba de que la receta está bien escrita es que alguien que nunca vio al autor la cocine y obtenga el mismo plato. El proyecto integrador es su primera receta de ese tipo. No se evalúa solo si su plato está rico (si sus resultados son biológicamente interesantes), sino si un compañero, con su repositorio y una sola orden, obtiene exactamente las mismas tablas y figuras.

Tabla 18.2. Las dos opciones del proyecto integrador, sus etapas y los capítulos del libro en los que se apoya cada una.

Etapas	A: patógeno (ADN)	B: RNA-seq
Datos	FASTQ pareados de aislados (SRA/ENA)	FASTQ de un diseño con réplicas
Calidad	<code>fastp</code> , MultiQC (capítulo 6)	<code>fastp</code> , MultiQC (capítulo 6)
Lecturas → señal	BWA-MEM, <code>samtools</code> (capítulo 7)	pseudoalineamiento o mapeo, algoritmo EM (capítulo 11)
Inferencia	llamado y filtrado de variantes (capítulo 9)	normalización, binomial negativa, FDR (capítulo 11)
Interpretación	anotación funcional, resistencia (capítulo 9)	enriquecimiento GO/KEGG (capítulo 11)
Síntesis	filogenia de consensos (capítulo 5)	mapa de calor, rutas enriquecidas

18.2.1 Elegir la pregunta y dimensionar el proyecto

Todo proyecto empieza por una pregunta biológica concreta, y el resto (datos, métodos, recursos) se deriva de ella. Proponemos dos opciones que recorren buena parte del libro; la tabla 18.2 las compara.

- **Opción A: genómica de un patógeno.** A partir de lecturas Illumina pareadas de 20 a 30 aislados clínicos de una bacteria (por ejemplo, *Mycobacterium tuberculosis*, cuyo genoma de referencia H37Rv tiene unos 4,41 Mb), identificar variantes respecto a la referencia, anotar las que caen en genes asociados a resistencia antimicrobiana y reconstruir la filogenia de los aislados para discutir si hay transmisión reciente. Es exactamente el flujo de la figura 18.1. Plataformas de vigilancia genómica como Nextstrain se construyen sobre este mismo esquema, con Snakemake como motor (Hadfield et al., 2018).
- **Opción B: transcriptómica.** A partir de un experimento de RNA-seq público con al menos tres réplicas biológicas por condición, cuantificar transcritos, detectar genes diferencialmente expresados con control de la tasa de falsos descubrimientos e interpretar la lista con un análisis de enriquecimiento funcional (capítulo 11).

Antes de escribir una sola regla conviene estimar cuánto disco y cuánto cómputo hará falta. Las fórmulas de cobertura del capítulo 6 bastan.

Ejemplo 18.3. (Presupuesto de la opción A)

Queremos 24 aislados de *M. tuberculosis* ($G = 4,41$ Mb) a una profundidad media de $c = 100\times$ con lecturas pareadas de 2×150 pb. El número de pares por muestra es $N = cG/(2L) = 100 \times 4,41 \times 10^6 / 300 \approx 1,47 \times 10^6$, es decir, $4,41 \times 10^8$ bases. En FASTQ, cada base ocupa aproximadamente dos bytes (la base y su calidad) más las cabeceras; con unas 60 letras de cabecera por lectura, el tamaño sin comprimir es de unos 1,06 GB por muestra, y comprimido con `gzip` (factor aproximado de 4) unos 0,27 GB. Los 24 aislados ocupan unos 6,4 GB de datos crudos: cabe en un portátil. Con las duraciones del ejemplo 18.2, el trabajo total es de $W = 616$ min $\approx 10,3$ horas-núcleo; con 16 núcleos, el planificador termina en unas 2,8 horas (1,6 horas si se divide el llamado por regiones). El proyecto es viable en un servidor modesto o en unas pocas horas de un clúster universitario. Con estas cifras en mano se puede decidir, antes de empezar, si los archivos intermedios deben borrarse (`temp()`) y cuántos núcleos pedir.

18.2.2 Estructura del proyecto

Un proyecto computacional bien organizado se entiende al abrir su carpeta principal. Noble (2009) propuso una organización que se ha vuelto casi un estándar: una carpeta para datos, otra para resultados, otra para código y otra para documentación, con el principio de que los datos crudos son de *solo lectura* y todo lo demás puede regenerarse desde ellos con una única orden. La figura 18.5 adapta esa idea a un flujo con Snakemake; con Nextflow basta cambiar `workflow/Snakefile` por `main.nf` y `nextflow.config`.

18.2.3 Las capas de la reproducibilidad

Conviene precisar el vocabulario. Llamaremos *reproducible* a un análisis que, con los mismos datos y el mismo código, produce los mismos resultados; *replicable*, a un hallazgo que se sostiene cuando se repite el experimento con datos nuevos. La reproducibilidad es el requisito mínimo: si ni siquiera el propio análisis puede repetirse, no hay forma de saber si un resultado sorprendente es biología o un error. Peng (2011) la describió como un espectro que va desde la publicación sin nada más hasta el estándar de oro de código y datos enlazados y ejecutables; cada escalón hace el trabajo más verificable.

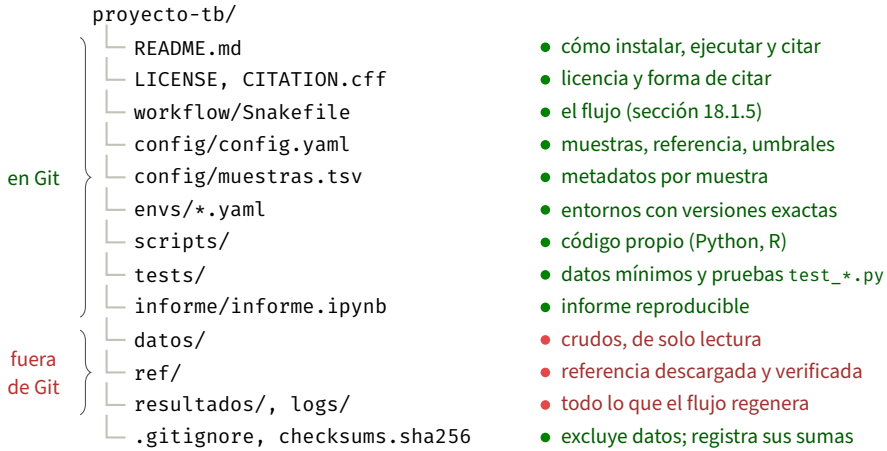


Figura 18.5. Estructura de directorios recomendada para el proyecto integrador, inspirada en Noble (2009). En verde, lo que se versiona con Git: código, configuración, entornos, pruebas y documentación, todo texto ligero. En rojo, lo que *no* se versiona: los datos crudos (que se descargan con un *script* a partir de sus accesiones y se verifican con sumas SHA-256), la referencia y todo lo que el flujo regenera. La regla de oro es que borrar `resultados/` nunca debe ser una catástrofe: una orden lo reconstruye.

Formalmente, el resultado R de un análisis es una función de todo lo que lo determina:

$$R = f\left(\underbrace{C}_{\text{código}}, \underbrace{\mathcal{E}}_{\text{entorno}}, \underbrace{D}_{\text{datos}}, \underbrace{\theta}_{\text{parámetros}}, \underbrace{\omega}_{\text{semilla}}\right). \quad (18.12)$$

DONDE

C	Código propio y del flujo; se fija con el identificador de un <i>commit</i> de Git.
$\mathcal{E}$	Entorno de software (paquetes, bibliotecas, sistema); se fija con entornos bloqueados o contenedores con <i>digest</i> .
D	Datos de entrada; se fijan con accesiones públicas y sumas de verificación.
θ	Parámetros y umbrales; se fijan en un archivo de configuración versionado.
ω	Estado inicial de los generadores de números aleatorios (capítulo o).

La ecuación (18.12) es trivial de escribir y exigente de cumplir: *cada* argumento que no se fije es una fuente de discrepancia. La figura 18.6 muestra las capas y

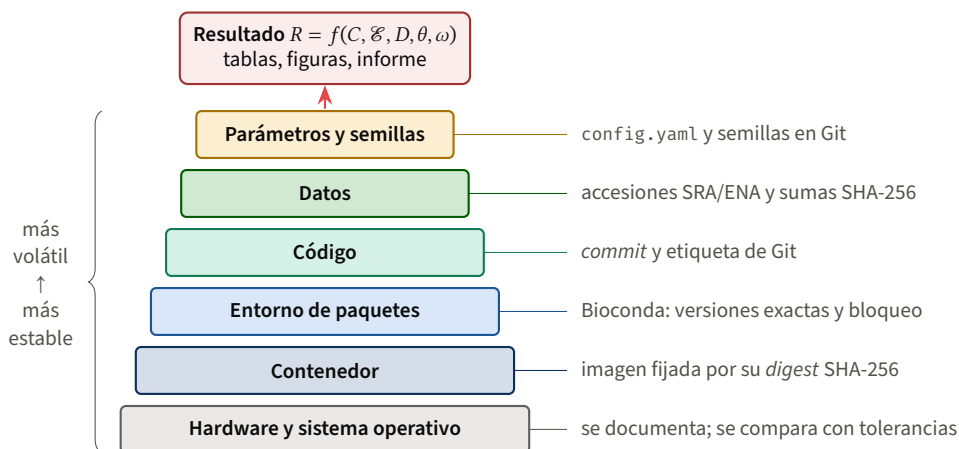


Figura 18.6. Las capas que determinan un resultado computacional (ecuación (18.12)) y la herramienta que fija cada una. Las capas inferiores cambian con menos frecuencia pero son las más difíciles de reconstruir años después; las superiores cambian en cada análisis y son las más fáciles de olvidar. Un contenedor fija de una vez las capas de entorno y parte del sistema, pero no los datos ni los parámetros; Git fija el código y la configuración, pero no el software. Solo la combinación de todas cierra la ecuación (18.12). Adaptado de la discusión de Grüning et al. (2018).

la herramienta que fija cada una. Hay, además, una fuente residual que ninguna herramienta elimina del todo: la aritmética de coma flotante no es asociativa. En Python, $(0.1 + 0.2) + 0.3$ da 0.6000000000000001 , mientras que $0.1 + (0.2 + 0.3)$ da 0.6 . Un programa que suma en paralelo en un orden que depende de qué hilo termina antes puede producir resultados que difieren en el último decimal entre ejecuciones; lo sensato es documentarlo y comparar resultados con tolerancias, no bit a bit.

Sumas de verificación. La capa de datos se fija con sumas de verificación: el flujo descarga las lecturas por su accesión y comprueba que su huella SHA-256 coincide con la registrada. ¿Qué tan seguras son? Si una función *hash* de b bits se comporta como una asignación aleatoria uniforme, la probabilidad de que entre n archivos distintos haya dos con la misma huella es, por el argumento del cumpleaños,

$$\mathbb{P}(\text{colisión}) = 1 - \prod_{i=1}^{n-1} \left(1 - \frac{i}{2^b}\right) \approx 1 - \exp\left(-\frac{n(n-1)}{2^{b+1}}\right). \quad (18.13)$$

DONDE

b	Longitud de la huella en bits (32 para CRC32, 128 para MD5, 256 para SHA-256).
n	Número de archivos distintos comparados.

La aproximación se obtiene tomando logaritmos, usando $\ln(1 - x) \approx -x$ para x pequeño y sumando $\sum_{i=1}^{n-1} i = n(n-1)/2$. Con una suma de 32 bits (como CRC32) y $n = 10^5$ archivos, la probabilidad de colisión ya es de 0,69: las sumas cortas sirven para detectar errores de transmisión, no para identificar contenidos. Con MD5 (128 bits) y mil millones de archivos, la probabilidad accidental es de $1,5 \times 10^{-21}$, y con SHA-256, de 4×10^{-60} . MD5 sigue siendo útil contra la corrupción accidental (el SRA y el ENA lo publican para cada archivo), pero hoy se pueden fabricar colisiones deliberadas, así que para garantías frente a manipulaciones se prefiere SHA-256.

Registrar y verificar sumas SHA-256

```
sha256sum datos/*.fastq.gz ref/genoma.fa > checksums.sha256
git add checksums.sha256           # la lista de sumas SI va en Git
sha256sum -c checksums.sha256      # en otra maquina: OK / FAILED
```

18.2.4 Control de versiones y pruebas

El código del proyecto vive en un repositorio Git desde el primer día (capítulo 0). Las reglas de Perez-Riverol et al. (2016) resumen los hábitos que más rinden: *commits* pequeños con mensajes que expliquen el porqué, ramas para cada cambio sustancial, etiquetas (v1.0) para las versiones que producen resultados reportados y un repositorio público enlazado desde la publicación. Los datos y resultados no van en Git (figura 18.5), pero sí la lista de sus accesiones y sumas.

Las *pruebas* son la parte más descuidada de los análisis académicos, y la que más ahorra. Distinguimos tres niveles:

1. **Pruebas unitarias** de las funciones propias (en `scripts/`), con casos pequeños cuyo resultado se conoce de antemano, ejecutadas con `pytest`.
2. **Pruebas de integración** del flujo completo sobre un conjunto de datos *diminuto* (unos miles de lecturas simuladas sobre un fragmento de la referencia), que debe correr en segundos; es la idea de los perfiles test de nf-core (Ewels et al., 2020).

3. **Comprobaciones de cordura** sobre los resultados reales: la proporción de lecturas mapeadas, la razón Ti/Tv de las variantes (capítulo 9), el número de genes detectados, la correlación entre réplicas.

```
tests/test_kahn.py

import pytest
from scripts.kahn import orden_topologico

def test_respeta_dependencias():
    g = {"fastp": ["mapear"], "indice": ["mapear"],
         "mapear": ["dedup"], "dedup": []}
    pos = {v: i for i, v in enumerate(orden_topologico(g))}
    for v, hijos in g.items():
        assert all(pos[v] < pos[u] for u in hijos)

def test_detecta_ciclos():
    with pytest.raises(ValueError):
        orden_topologico({"a": ["b"], "b": ["c"], "c": ["a"]})
```

Probar antes de ejecutar a escala

```
pytest -q tests/           # pruebas unitarias
snakemake --lint           # buenas practicas del flujo
snakemake -n --configfile tests/config_mini.yaml # ensayo en seco
snakemake --cores 2 --sdm conda --configfile tests/config_mini.yaml
```

Si esas cuatro órdenes se ejecutan automáticamente en cada *commit* (integración continua, por ejemplo con GitHub Actions), un error se detecta minutos después de introducirlo, y no semanas después, cuando un resultado no cuadra.

18.2.5 Documentación e informe reproducible

La documentación mínima de un proyecto es un README que responda cuatro preguntas: qué hace el proyecto, cómo se instala el entorno, cómo se ejecuta (idealmente una orden) y cómo se cita. A él se suman una licencia (sin licencia, legalmente nadie puede reutilizar el código), un archivo CITATION.cff y, para el código publicado, un DOI de archivo permanente.

El *informe* es el último eslabón del flujo, no un documento aparte. Un cuaderno Jupyter que lee las tablas de resultados/ y produce todas las figuras es una regla más, y debe poder ejecutarse de principio a fin sin intervención. Rule et al. (2019)

dan diez reglas para que un cuaderno sea legible y reutilizable, entre ellas contar una historia para una audiencia concreta, documentar el proceso y no solo los resultados, dividir el código en celdas con un propósito claro, registrar las dependencias y construir el cuaderno como parte de un flujo. MultiQC (Ewels et al., 2016) reúne en un solo informe HTML las métricas de calidad de todas las herramientas y muestras, y Snakemake puede generar un informe con la procedencia de cada resultado (`snakemake --report`). La regla 9 de Sandve et al. (2013), conectar cada afirmación del texto con el resultado que la sustenta, se cumple de forma natural si cada número del informe se lee de un archivo generado por el flujo en lugar de copiarse a mano.

18.2.6 Metadatos y publicación de datos

Las lecturas crudas de cualquier estudio publicado deben depositarse en un archivo público del consorcio INSDC: el *Sequence Read Archive* (SRA) del NCBI (Leinonen, Sugawara & Shumway, 2011) o el *European Nucleotide Archive* (ENA) del EMBL-EBI (Leinonen, Akhtar et al., 2011), que se replican entre sí. Ya los consultamos en el capítulo 2; ahora nos toca depositar. Ambos organizan los metadatos en una jerarquía (figura 18.7) que refleja la estructura de un experimento: un *proyecto* agrupa *muestras* biológicas; de cada muestra se preparan uno o varios *experimentos* (bibliotecas secuenciadas con una tecnología); cada experimento produce una o varias *corridas*, que contienen los archivos.

Los metadatos son la diferencia entre unos datos reutilizables y unos bytes huérfanos. Para la opción A, la fecha y el país de colecta, el hospedero y el tipo de muestra son imprescindibles para cualquier análisis epidemiológico posterior; para la opción B, la condición, la réplica, el tejido y el lote de preparación de bibliotecas. Los depósitos exigen *listas de verificación* (*checklists*) con campos obligatorios y vocabularios controlados precisamente para que los principios FAIR se cumplan en la práctica (Wilkinson et al., 2016). El archivo `config/muestras.tsv` del proyecto debería contener ya esas columnas, de modo que la misma tabla alimente el flujo y el depósito.

Las hojas de cálculo reescriben sus datos

Abrir una tabla de metadatos o de genes en una hoja de cálculo es arriesgado: los programas convierten silenciosamente nombres como *SEPT2* o *MARCH1* en fechas, e identificadores largos en notación científica. Ziemann et al. (2016) encontraron este tipo de error en aproximadamente una de cada

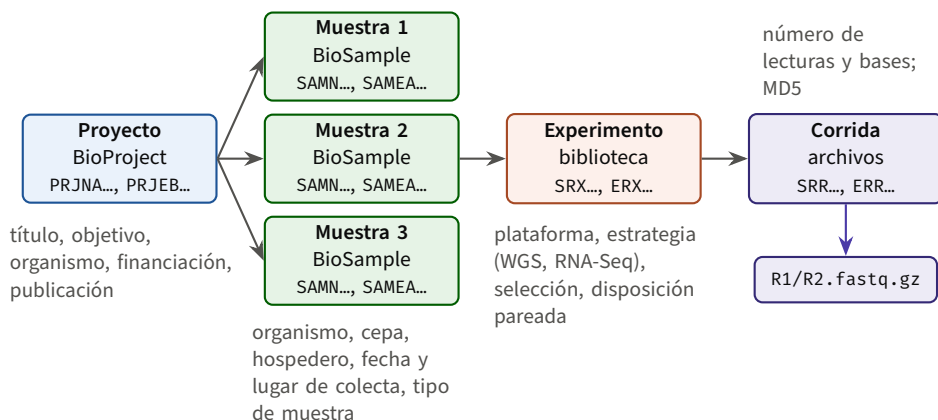


Figura 18.7. La jerarquía de objetos del SRA y el ENA, con los prefijos de sus accesiones (NCBI / EBI) y los metadatos que se describen en cada nivel. Un proyecto contiene muestras; cada muestra puede secuenciarse en varios experimentos (bibliotecas), y cada experimento en varias corridas. Las muestras llevan la información biológica que hará reutilizables los datos; los experimentos, la técnica. El artículo debe citar la accesión del proyecto, y el flujo debe descargar las lecturas por la accesión de cada corrida.

cinco publicaciones que incluían listas de genes en archivos de Excel como material suplementario. Edite los metadatos como texto plano (TSV), validelos con un *script* y nunca guarde una tabla “pasada” por una hoja de cálculo como fuente de verdad.

18.2.7 Ética y datos humanos

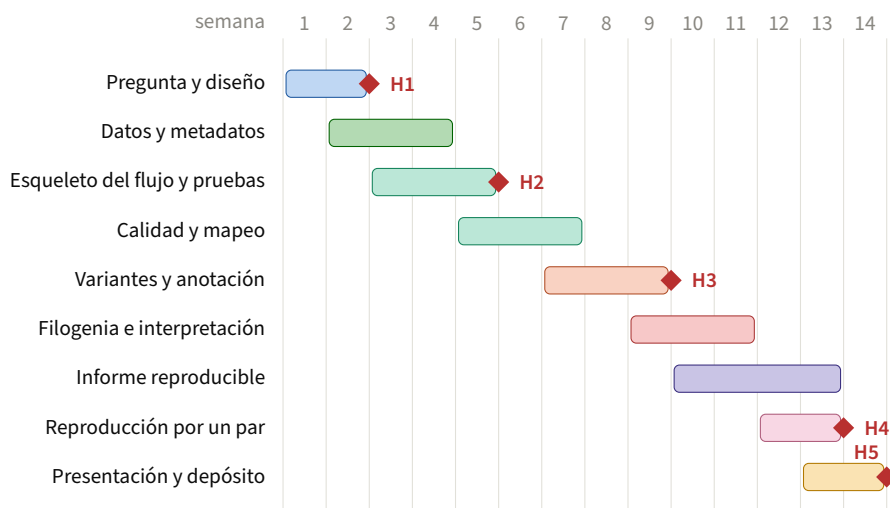
La opción A trabaja con genomas de bacterias, pero las muestras vienen de pacientes; la opción B puede usar ARN humano. En ambos casos entran en juego obligaciones éticas y legales que deben planificarse desde el diseño, no resolverse en el momento de publicar.

- **Consentimiento y aprobación ética.** El uso de datos humanos requiere la aprobación de un comité de ética y un consentimiento informado cuyo alcance cubra el uso previsto, incluido el depósito en repositorios públicos o de acceso controlado. Reutilizar datos públicos no exime de respetar las condiciones con que se recogieron.

- **La anonimización genómica es frágil.** Gymrek et al. (2013) demostraron que es posible inferir el apellido de participantes supuestamente anónimos a partir de los marcadores STR de su cromosoma Y, cruzándolos con bases de datos genealógicas recreativas, y, combinándolo con la edad y el lugar de residencia, reidentificarlos. Un genoma humano es, en la práctica, un identificador personal. Por eso los datos humanos individuales se depositan en archivos de *acceso controlado* (como dbGaP o el *European Genome-phenome Archive*), donde un comité evalúa cada solicitud.
- **Protección de datos.** En la Unión Europea, el Reglamento General de Protección de Datos (GDPR, aplicable desde mayo de 2018) clasifica los datos genéticos como una *categoría especial* de datos personales, cuyo tratamiento solo es lícito bajo condiciones específicas, como el consentimiento explícito o las excepciones para investigación con salvaguardas adecuadas, entre ellas la seudonimización; los datos seudonimizados siguen siendo datos personales (Shabani & Borry, 2018). Otros países tienen leyes propias (en Colombia, por ejemplo, la Ley 1581 de 2012 de protección de datos personales); compruebe cuál se aplica a sus datos y a su institución.
- **Lecturas del hospedero.** Las lecturas de un patógeno secuenciado a partir de una muestra clínica contienen a menudo lecturas humanas. Antes de depositarlas, elimínelas (mapeando contra la referencia humana y descartando lo que coincida) y documente ese paso en el flujo.
- **Metadatos identificadores.** Una fecha exacta de colecta, un hospital y una edad pueden bastar para identificar a un paciente en una comunidad pequeña. Reduzca la resolución (mes en lugar de día, región en lugar de dirección) cuando no afecte a la pregunta científica.

18.2.8 Cronograma

Un proyecto de este tamaño cabe en un semestre si se planifica con cuidado. Un cronograma es, de nuevo, un grafo acíclico dirigido de tareas con duraciones: las mismas redes PERT que Kahn (1962) quería ordenar. Su camino crítico indica qué retrasos retrasan la entrega final y cuáles tienen holgura. La figura 18.8 propone una distribución en 14 semanas con cinco hitos verificables.



Hitos: H1 propuesta aprobada (pregunta, datos, rúbrica); H2 el flujo corre de principio a fin sobre los datos mínimos de prueba; H3 resultados preliminares con todas las muestras; H4 un compañero reproduce el análisis desde el repositorio; H5 entrega: informe, repositorio etiquetado y presentación.

Figura 18.8. Cronograma orientativo del proyecto integrador en 14 semanas. Las tareas se solapan porque no todas dependen unas de otras: el esqueleto del flujo (con datos de prueba) se escribe mientras se descargan y curan los datos reales, y el informe se empieza a redactar en cuanto hay resultados preliminares. El camino crítico recorre diseño → esqueleto → mapeo → variantes → filogenia → informe: un retraso en cualquiera de ellos retrasa la entrega. El hito H4, en el que otra persona reproduce su análisis, es la prueba de fuego del capítulo; resérvele tiempo.

18.2.9 Rúbrica de evaluación

La rúbrica hace explícito lo que se espera y permite que la autoevaluación coincida con la evaluación. La tabla 18.3 propone seis criterios, cada uno puntuado en una escala de 1 (insuficiente) a 4 (excelente), con pesos w_i que suman 1. La nota final es la media ponderada

$$N = \sum_{i=1}^6 w_i x_i, \quad \sum_{i=1}^6 w_i = 1, \quad x_i \in \{1, 2, 3, 4\}, \quad (18.14)$$

que se reescala a la escala de la institución.

Tabla 18.3. Rúbrica del proyecto integrador. Cada criterio se puntúa de 1 a 4; se describen los extremos de la escala.

Criterio	w_i	Nivel 4 (excelente)	Nivel 1 (insuficiente)
Pregunta y diseño	0,20	pregunta precisa; datos adecuados y justificados; limitaciones reconocidas	pregunta vaga; datos elegidos por conveniencia
Corrección del análisis	0,20	métodos apropiados, parámetros justificados, controles de calidad interpretados	errores metodológicos; salidas sin inspeccionar
Reproducibilidad	0,15	un par reproduce todo con una orden; entornos y datos fijados	pasos manuales; versiones desconocidas
Calidad del código y pruebas	0,15	flujo legible, modular, con pruebas unitarias y de integración	<i>script</i> monolítico, sin pruebas
Datos, metadatos y ética	0,15	metadatos completos; depósito o plan de depósito; ética atendida	metadatos ausentes; ética ignorada
Informe y comunicación	0,15	figuras claras generadas por el flujo; conclusiones apoyadas en resultados	figuras copiadas a mano; conclusiones sin sustento

DONDE

w_i	Peso del criterio i en la nota final.
x_i	Nivel alcanzado en el criterio i .
N	Nota ponderada, entre 1 y 4.

Ejemplo 18.4. (Aplicar la rúbrica)

Un proyecto obtiene los niveles $x = (4, 3, 3, 4, 2, 3)$ en los seis criterios, en el orden de la tabla 18.3. Su nota es

$$\begin{aligned}
 N &= 0,20 \cdot 4 + 0,20 \cdot 3 + 0,15 \cdot 3 + 0,15 \cdot 4 + 0,15 \cdot 2 + 0,15 \cdot 3 \\
 &= 0,80 + 0,60 + 0,45 + 0,60 + 0,30 + 0,45 = 3,20,
 \end{aligned}$$

es decir, un 80 % de la nota máxima (4,0 sobre 5). El punto débil está en datos, metadatos y ética ($x_5 = 2$): subirlo a 4, completando la tabla de metadatos y el plan de depósito, añadiría $0,15 \times 2 = 0,30$ puntos y llevaría la nota a 3,50 (87,5 %). La rúbrica no solo califica: indica dónde rinde más el esfuerzo que queda.

El proyecto está terminado cuando otra persona, con su repositorio, las accesiones de los datos y una sola orden, obtiene las mismas tablas y figuras que usted.

En resumen

- Un análisis es un grafo acíclico dirigido de trabajos. Existe un orden de ejecución válido si y solo si no hay ciclos, y el algoritmo de Kahn lo encuentra, o detecta el ciclo, en tiempo $O(|V| + |E|)$ manteniendo los grados de entrada residuales.
- *make* introdujo las dependencias por archivos y el criterio de rehacer lo desactualizado. Snakemake lo lleva a Python con comodines e inferencia del grafo hacia atrás (*pull*); Nextflow usa procesos aislados comunicados por canales de cola y de valor en un modelo de flujo de datos (*push*); CWL es un estándar declarativo multimotor.
- La reproducibilidad exige fijar código (Git), entorno (Bioconda, contenedores con *digest*), datos (accesiones y SHA-256), parámetros (configuración versionada) y semillas. Las claves de caché por *hash* permiten reanudar un flujo rehaciendo solo los descendientes de lo que cambió.
- El tiempo en paralelo está acotado por $\max(W/P, S) \leq T_p \leq W/P + S$; la ley de Amdahl limita la aceleración a $1/s$ y la de Gustafson muestra que los problemas más grandes escalan mejor. Acortar el camino crítico (p. ej., dividiendo el llamado conjunto por regiones) rinde más que añadir núcleos.
- Un proyecto integrador reproducible tiene una estructura de directorios clara, control de versiones, pruebas a varios niveles, un informe generado por el propio flujo, datos depositados con metadatos FAIR y un tratamiento ético y legal de los datos humanos planificado desde el diseño.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 18.1** ●●● Dibuje el grafo de trabajos del Snakefile con dos muestras en lugar de tres. ¿Cuántos trabajos y aristas tiene? Compruebe que la suma de los grados de entrada es igual al número de aristas.
- 18.2** ●●● Ejecute a mano el algoritmo de Kahn sobre el grafo del ejercicio anterior con una *pila* (LIFO) en lugar de una cola. ¿Obtiene el mismo orden? ¿Es igualmente válido? ¿Por qué el teorema de corrección no depende de la estructura de datos elegida?
- 18.3** ●●● Con la ecuación (18.3), determine qué trabajos del flujo de la figura 18.1 se rehacen si se actualiza el archivo de la referencia. Compare con el caso en que solo cambia la base de datos de SnpEff.
- 18.4** ●●● Añada al Snakefile una regla `mosdepth` que calcule la cobertura de cada muestra (capítulo 7) y haga que `multiqc` la incluya. ¿Qué aristas nuevas aparecen en el grafo? ¿Cambia el camino crítico?
- 18.5** ●●● Escriba la versión Nextflow de la regla del ejercicio anterior. Explique, usando la ecuación (18.5), qué pasaría si conectara el proceso a la salida de DEDUP y a un canal de cola que contiene solo la referencia.
- 18.6** ●●● Implemente en Python el cálculo del camino crítico (ecuación (18.7)) sobre el orden que devuelve `orden_topologico`, y verifique que con las duraciones del ejemplo 18.2 obtiene $S = 46$ min para tres muestras.
- 18.7** ●●● Mida la aceleración de un flujo real (por ejemplo, el Snakefile con datos de prueba) con 1, 2, 4 y 8 núcleos, calcule la fracción serial de Karp-Flatt (ecuación (18.11)) y diga si el límite es una parte serial genuina o una sobrecarga.
- 18.8** ●●● Demuestre que el número de rondas de la versión por niveles del algoritmo de Kahn es igual al número de vértices del camino más largo del grafo, y que el ancho máximo de una ronda es una cota inferior del número de núcleos necesarios para alcanzar $T_P = S$ cuando todas las duraciones son iguales. ¿Es también una cota superior?

- 18.9** ●●● Deduzca la ecuación (18.13) y calcule cuántos archivos harían falta para que la probabilidad de una colisión accidental de SHA-256 alcanzara 10^{-6} . Compare con el número estimado de archivos del SRA.
- 18.10** ●●● Diseñe su propio proyecto integrador: escriba la pregunta, elija las accesiones de los datos, dibuje el grafo de trabajos, estime el presupuesto de disco y cómputo como en el ejemplo 18.3, redacte la sección de ética y autoevalúe el diseño con la rúbrica de la tabla 18.3.

Referencias del capítulo

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67)*, 483–485. <https://doi.org/10.1145/1465482.1465560>
- Buckheit, J. B., & Donoho, D. L. (1995). WaveLab and Reproducible Research. En A. Antoniadis & G. Oppenheim (Eds.), *Wavelets and Statistics* (pp. 55–81). Springer. https://doi.org/10.1007/978-1-4612-2544-7_5
- Crusoe, M. R., Abeln, S., Iosup, A., Amstutz, P., Chilton, J., Tijanić, N., Ménager, H., Soiland-Reyes, S., Gavrilović, B., Goble, C., & The CWL Community. (2022). Methods included: Standardizing computational reuse and portability with the Common Workflow Language. *Communications of the ACM*, 65(6), 54–63. <https://doi.org/10.1145/3486897>
- da Veiga Leprevost, F., Grüning, B. A., Alves Aflitos, S., Röst, H. L., Uszkoreit, J., Barsnes, H., Vaudel, M., Moreno, P., Gatto, L., Weber, J., Bai, M., Jimenez, R. C., Sachsenberg, T., Pfeuffer, J., Vera Alvarez, R., Griss, J., Nesvizhskii, A. I., & Perez-Riverol, Y. (2017). BioContainers: An open-source and community-driven framework for software standardization. *Bioinformatics*, 33(16), 2580–2582. <https://doi.org/10.1093/bioinformatics/btx192>
- Di Tommaso, P., Chatzou, M., Floden, E. W., Barja, P. P., Palumbo, E., & Notredame, C. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4), 316–319. <https://doi.org/10.1038/nbt.3820>
- Ewels, P., Magnusson, M., Lundin, S., & Käller, M. (2016). MultiQC: Summarize analysis results for multiple tools and samples in a single report. *Bioinformatics*, 32(19), 3047–3048. <https://doi.org/10.1093/bioinformatics/btw354>
- Ewels, P. A., Peltzer, A., Fillinger, S., Patel, H., Alneberg, J., Wilm, A., Garcia, M. U., Di Tommaso, P., & Nahnsen, S. (2020). The nf-core framework for community-curated bioinformatics pipelines. *Nature Biotechnology*, 38(3), 276–278. <https://doi.org/10.1038/s41587-020-0439-x>
- Feldman, S. I. (1979). Make—a program for maintaining computer programs. *Software: Practice and Experience*, 9(4), 255–265. <https://doi.org/10.1002/spe.4380090402>

- Grüning, B., Chilton, J., Köster, J., Dale, R., Soranzo, N., van den Beek, M., Goecks, J., Backofen, R., Nekrutenko, A., & Taylor, J. (2018). Practical computational reproducibility in the life sciences. *Cell Systems*, 6(6), 631-635. <https://doi.org/10.1016/j.cels.2018.03.014>
- Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM*, 31(5), 532-533. <https://doi.org/10.1145/42411.42415>
- Gymrek, M., McGuire, A. L., Golan, D., Halperin, E., & Erlich, Y. (2013). Identifying personal genomes by surname inference. *Science*, 339(6117), 321-324. <https://doi.org/10.1126/science.1229566>
- Hadfield, J., Megill, C., Bell, S. M., Huddleston, J., Potter, B., Callender, C., Sagulenko, P., Bedford, T., & Neher, R. A. (2018). Nextstrain: Real-time tracking of pathogen evolution. *Bioinformatics*, 34(23), 4121-4123. <https://doi.org/10.1093/bioinformatics/bty407>
- Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, 5(11), 558-562. <https://doi.org/10.1145/368996.369025>
- Karp, A. H., & Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5), 539-543. <https://doi.org/10.1145/78607.78614>
- Köster, J., & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19), 2520-2522. <https://doi.org/10.1093/bioinformatics/bts480>
- Kurtzer, G. M., Sochat, V., & Bauer, M. W. (2017). Singularity: Scientific containers for mobility of compute. *PLOS ONE*, 12(5), e0177459. <https://doi.org/10.1371/journal.pone.0177459>
- Leinonen, R., Akhtar, R., Birney, E., Bower, L., Cerdeno-Tarraga, A., Cheng, Y., Cleland, I., Faruque, N., Goodgame, N., Gibson, R., Hoad, G., Jang, M., Pakseresht, N., Plaister, S., Radhakrishnan, R., Reddy, K., Sobhany, S., Ten Hoopen, P., Vaughan, R., Zalunin, V., et al. (2011). The European Nucleotide Archive. *Nucleic Acids Research*, 39(Database), D28-D31. <https://doi.org/10.1093/nar/gkq967>
- Leinonen, R., Sugawara, H., & Shumway, M. (2011). The Sequence Read Archive. *Nucleic Acids Research*, 39(Database), D19-D21. <https://doi.org/10.1093/nar/gkq1019>
- Mölder, F., Jablonski, K. P., Letcher, B., Hall, M. B., Tomkins-Tinch, C. H., Sochat, V., Forster, J., Lee, S., Twardziok, S. O., Kanitz, A., Wilm, A., Holtgrewe, M., Rahmann, S., Nahnsen, S., & Köster, J. (2021). Sustainable data analysis with Snakemake. *F1000Research*, 10, 33. <https://doi.org/10.12688/f1000research.29032.2>
- Noble, W. S. (2009). A quick guide to organizing computational biology projects. *PLoS Computational Biology*, 5(7), e1000424. <https://doi.org/10.1371/journal.pcbi.1000424>
- Peng, R. D. (2011). Reproducible research in computational science. *Science*, 334(6060), 1226-1227. <https://doi.org/10.1126/science.1213847>
- Perez-Riverol, Y., Gatto, L., Wang, R., Sachsenberg, T., Uszkoreit, J., Leprevost, F. d. V., Fufezan, C., Ternent, T., Eglen, S. J., Katz, D. S., Pollard, T. J., Kononov, A., Flight, R. M., Blin, K., & Vizcaíno, J. A. (2016). Ten simple rules for taking advantage of Git and GitHub. *PLOS Computational Biology*, 12(7), e1004947. <https://doi.org/10.1371/journal.pcbi.1004947>

- Rule, A., Birmingham, A., Zuniga, C., Altintas, I., Huang, S.-C., Knight, R., Moshiri, N., Nguyen, M. H., Rosenthal, S. B., Pérez, F., & Rose, P. W. (2019). Ten simple rules for writing and sharing computational analyses in Jupyter Notebooks. *PLOS Computational Biology*, 15(7), e1007007. <https://doi.org/10.1371/journal.pcbi.1007007>
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10), e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>
- Shabani, M., & Borry, P. (2018). Rules for processing genetic data for research purposes in view of the new EU General Data Protection Regulation. *European Journal of Human Genetics*, 26(2), 149-156. <https://doi.org/10.1038/s41431-017-0045-7>
- The Bioconda Team, Grüning, B., Dale, R., Sjödin, A., Chapman, B. A., Rowe, J., Tomkins-Tinch, C. H., Valieris, R., & Köster, J. (2018). Bioconda: Sustainable and comprehensive software distribution for the life sciences. *Nature Methods*, 15(7), 475-476. <https://doi.org/10.1038/s41592-018-0046-7>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., Gonzalez-Beltran, A., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>
- Wratten, L., Wilm, A., & Göke, J. (2021). Reproducible, scalable, and shareable analysis pipelines with bioinformatics workflow managers. *Nature Methods*, 18(10), 1161-1168. <https://doi.org/10.1038/s41592-021-01254-9>
- Ziemann, M., Eren, Y., & El-Osta, A. (2016). Gene name errors are widespread in the scientific literature. *Genome Biology*, 17(1), 177. <https://doi.org/10.1186/s13059-016-1044-7>